

DIGITAL DESIGN

With An Introduction to the Verilog HDL

FIFTH EDITION

M. MORRIS MANO | MICHAEL D. CILETTI



Dijital Tasarım

Verilog HDL'ye Giriş ile

Bu sayfa kasıtlı olarak boş bırakılmıştır

Dijital Tasarım

Verilog HDL'ye Giriş ile

BEŞİNCİ BASKI

M. Morris Mano

*Bilgisayar Mühendisliği Emekli Profesörü
California Eyalet Üniversitesi, Los Angeles*

Michael D. Ciletti

*Elektrik ve Bilgisayar Mühendisliği Emekli Profesörü
Colorado Springs, Colorado Üniversitesi*

PEARSON

Üst Eyer Nehri Boston Columbus San Francisco New York
Indianapolis Londra Toronto Sydney Singapur Tokyo Montreal
Dubai Madrid Hong Kong Meksiko Şehri Münih Paris Amsterdam Cape Town

Başkan Yardımcısı ve Editöryal Direktör, ECS:

Marcia J. Horton

İcra Editörü: *Andrew Gilfillan*

Başkan Yardımcısı, Üretim: *Vince O'Brien*

İcra Pazarlama Müdürü: *Tim Galligan*

Pazarlama Asistanı: *Jon Bryant*

İzinler Proje Yöneticisi: *Karen Sanatar*

Kıdemli Yönetici Editör: *Scott Disanno*

Üretim Proje Yöneticisi/Editöryal Üretim

Yöneticisi: *Greg Dulles*

Kapak Tasarımcısı: *Jayne Conte*

Kapak Fotoğrafı: *Michael D. Ciletti*

Kompozisyon: *Jouve India Private Limited*

Tam Hizmet Proje Yönetimi: *Jouve India Private Limited*

Yayımcı/Bağlayıcı: *Edwards Brothers*

Yazı Tipi: Times Ten 10/12

Telif Hakkı © 2013, 2007, 2002, 1991, 1984 Pearson Education, Inc., Prentice Hall olarak yayın yapmaktadır, One Lake Street, Upper Saddle River, New Jersey 07458. Tüm hakları saklıdır. Amerika Birleşik Devletleri'nde üretilmiştir. Bu yayının Telif Hakkı ile korunmaktadır ve herhangi bir yasaklı çoğaltma, bir geri alma sisteminde depolama veya herhangi bir form da veya herhangi bir yöntemle, elektronik, mekanik, fotokopi, kayıt veya benzeri ile iletim öncesinde yayıncıdan izin alınmalıdır. Bu çalışmadan materyal kullanmak için izin almak üzere, lütfen yazılı bir talep gönderin Pearson Education, Inc., İzinler Departmanı, One Lake Street, Upper Saddle River, New Jersey 07458.

Üreticiler ve satıcılar tarafından ürünlerini ayırt etmek için yapılan birçok ad, ticari marka olarak talep edilmektedir. Bu kitapta bu adlar geçtiğinde ve yayıncı bir ticari marka talebinden haberdar olduğunda, adlar büyük harfle veya tamamen büyük harfle basılmıştır.

Tüm hakları saklıdır. Bu kitabın hiçbir bölümü, yayıncının yazılı izni olmadan, herhangi bir biçimde veya herhangi bir yöntemle çoğaltılamaz.

Verilogger Pro ve SynaptiCAD, SynaptiCAD, Inc.'in ticari markalarıdır, Blacksburg, VA 24062-0608.

Bu kitabın yazarı ve yayıncısı, bu kitabı hazırlamak için en iyi çabalarını göstermiştir. Bu çabalar, teorilerin ve programların etkinliğini belirlemek için geliştirme, araştırma ve test etme süreçlerini içermektedir. Yazar ve yayıncı, bu programlar veya bu kitapta yer alan belgelerle ilgili olarak herhangi bir türde, açık veya zımni, garanti vermemektedir. Yazar ve yayıncı, bu programların sağlanması, performansı veya kullanımı ile bağlantılı veya bunlardan kaynaklanan herhangi bir durumda, dolaylı veya sonuç olarak ortaya çıkan zararlardan sorumlu olmayacaktır.

Kapak hakkında: "Canyon de Chelly'deki Spider Rock," Chinle, Arizona, ABD, Ocak 2011. Fotoğraf mdc Images, LLC (www.mdcilettiphotography.com) izniyle kullanılmıştır.

Kongre Kütüphanesi Yayın Kataloğu Verileri

Mano, M. Morris, 1927-

Dijital tasarım : verilog hdl'ye giriş ile / M. Morris Mano, Michael D. Ciletti.—5. baskı.
s. cm.

İçindekiler dizini içerir.

ISBN-13: 978-0-13-277420-8

ISBN-10: 0-13-277420-8

1. Elektronik dijital bilgisayarlar—Devreler. 2. Mantık devreleri. 3. Mantık tasarımı. 4. Dijital entegre devreler. I. Ciletti, Michael D. II. Başlık.

TK7888.3.M343 2011

621.39'5—dc23

2011039094

PEARSON

10 9 8 7 6 5 4 3 2 1

ISBN-13: 978-0-13-277420-8

ISBN-10: 0-13-277420-8

İçindekiler

Önsöz

ix

1 Dijital Sistemler ve İkili Sayılar

1

1.1	Dijital Sistemler	1
1.2	İkili Sayılar	3
1.3	Sayı Tabanı Dönüşümleri	6
1.4	Sekizli ve Onaltılı Sayılar	8
1.5	Sayıların Tamamlayıcıları	10
1.6	İmzalı İkili Sayılar	14
1.7	İkili Kodlar	18
1.8	İkili Depolama ve Kayıtlar	27
1.9	İkili Mantık	30

2 Boolean Cebiri ve Mantık Kapıları

38

2.1	Giriş	38
2.2	Temel Tanımlar	38
2.3	Aksiyomatik Boolean Cebiri Tanımı	40
2.4	Boolean Cebirinin Temel Teoremleri ve Özellikleri	43
2.5	Boolean Fonksiyonları	46
2.6	Kanonik ve Standart Şekiller	51
2.7	Diğer Mantık İşlemleri	58
2.8	Dijital Mantık Kapıları	60
2.9	Entegre Devreler	66

v

3 Kapı Seviyesi Minimizasyonu

73

3.1 Giriş	73
3.2 Harita Yöntemi	73
3.3 Dört Değişkenli K-Map	80
3.4 Toplamların Çarpımı Basitleştirilmesi	84
3.5 Önemsiz Koşullar	88
3.6 NAND ve NOR Uygulaması	90
3.7 Diğer İki Düzeyli Uygulamalar	97
3.8 Özel-VE Fonksiyonu	103
3.9 Donanım Tanım Dili	108

4 Kombinasyonel Mantık

125

4.1 Giriş	125
4.2 Kombinasyonel Devreler	125
4.3 Analiz Prosedürü	126
4.4 Tasarım Prosedürü	129
4.5 İkili Toplayıcı-Çıkarıcı	133
4.6 Onluk Toplayıcı	144
4.7 İkili Çarpan	146
4.8 Büyüklük Karşılaştırıcı	148
4.9 Kodlayıcılar	150
4.10 Kod çözücüler	155
4.11 Çoklayıcılar	158
4.12 Birleşik Devrelerin HDL Modelleri	164

5 Senkron Sıralı Mantık

190

5.1 Giriş	190
5.2 Sıralı Devreler	190
5.3 Depolama Elemanları: Latch'ler	193
5.4 Depolama Elemanları: Flip-Flop'lar	196
5.5 Saatli Sıralı Devrelerin Analizi	204
5.6 Sıralı Devrelerin Sentetik HDL Modelleri	217
5.7 Durum Azaltma ve Atama	231
5.8 Tasarım Prosedürü	236

6 Kayıtlar ve Sayıcılar

255

6.1 Kayıtlar	255
6.2 Kaydırma Kayıtları	258
6.3 Dalgalı Sayıcılar	266
6.4 Senkron Sayıcılar	271
6.5 Diğer Sayıcılar	278
6.6 Kayıtlar ve Sayıcılar için HDL	283

7 Bellek ve Programlanabilir Mantık

299

7.1 Giriş	299
7.2 Rastgele Erişim Belleği	300
7.3 Bellek Kod Çözme	307
7.4 Hata Tespiti ve Düzeltme	312
7.5 Sadece Okunabilir Bellek	315
7.6 Programlanabilir Mantık Dizisi	321
7.7 Programlanabilir Dizi Mantiği	325
7.8 Sıralı Programlanabilir Cihazlar	329

Kaydedici Düzeyde Tasarım Transfer Düzeyi

351

8.1 Giriş	351
8.2 Kaydedici Transfer Düzeyi Notasyonu	351
8.3 HDL'de Kaydedici Transfer Düzeyi	354
8.4 Algoritmik Durum Makineleri (ASM'ler)	363
8.5 Tasarım Örneği (ASMD Grafiği)	371
8.6 Tasarım Örneğinin HDL Açıklaması	381
8.7 Sıralı İkili Çarpan	391
8.8 Kontrol Mantiği	396
8.9 İkili Çarpanın HDL Açıklaması	402
8.10 Multiplexer ile Tasarım	411
8.11 Yarışsız Tasarım (Yazılım Yarış Koşulları)	422
8.12 Latch'sız Tasarım (Neden Silikon İsrafi?)	425
8.13 Diğer Dil Özellikleri	426

Laboratuvar Deneyleri Standart IC'ler ve FPGA'lar ile

438

9.1 Deneylere Giriş	438
9.2 Deney 1: İkili ve Onluk Sayılar	443
9.3 Deney 2: Dijital Mantık Kapıları	446
9.4 Deney 3: Boolean Fonksiyonlarının Basitleştirilmesi	448
9.5 Deney 4: Kombinasyon Devreleri	450
9.6 Deney 5: Kod Dönüştürücüler	452
9.7 Deney 6: Çoklayıcılar ile Tasarım	453
9.8 Deney 7: Toplayıcılar ve Çıkarıcılar	455
9.9 Deney 8: Flip-Flop'lar	457
9.10 Deney 9: Sıralı Devreler	460
9.11 Deney 10: Sayıcılar	461
9.12 Deney 11: Kaydırma Register'ları	463
9.13 Deney 12: Seri Toplama	466
9.14 Deney 13: Bellek Birimi	467
9.15 Deney 14: Lamba Hentbolu	469

viii İçindekiler

9.16	Deney 15: Saat Darbe Üreteci	473
9.17	Deney 16: Paralel Toplayıcı ve Akümülatör	475
9.18	Deney 17: İkili Çarpan	478
9.19	Verilog HDL Simülasyon Deneyleri ve FPGA'larla Hızlı Prototipleme	480

10 Standart Grafik Sembolleri

488

10.1	Dikdörtgen Şekil Sembolleri	488
10.2	Nitelik Sembolleri	491
10.3	Bağımlılık Notasyonu	493
10.4	Birleşik Elemanlar için Semboller	495
10.5	Flip-Flop'lar için Semboller	497
10.6	Register'lar için Semboller	499
10.7	Sayıcılar için Semboller	502
10.8	RAM için Sembol	504

Ek

507

Seçilen Problemlere Cevaplar

521

İndeks

539

Önsöz

Dördüncü baskısından bu yana *Dijital Tasarım*, dijital teknolojiyi kullanarak bilgi almak, işlemek ve iletmek için cihazların ticari olarak mevcut olmasının patladığı görülmektedir. Cep telefonları ve çeşitli türdeki taşınabilir cihazlar neredeyse her gün yeni, rekabetçi özellikler sunmaktadır. Bu cihazların çekici grafik kullanıcı arayüzünün altında, verileri ikili formatta işleyen bir dijital sistem bulunmaktadır. Bu sistemlerin teorik temelleri pek değişmemiştir; aslında, temel teoremin istikrarının modern tasarım araçlarıyla birleştiği, üreticilerin piyasa fırsatlarına yanıt vermesini teşvik ettiğini savunmak mümkündür. Sonuç olarak, metnimizi geliştirme sürecimiz, mezunlarımızı dijital makineler hakkında sağlam bir anlayışla donatmak ve onları modern tasarım metodolojisi ile tanıştırma ihtiyacı tarafından yönlendirilmiştir.

Bu *Dijital Tasarım* baskısı, önceki dört baskının üzerine inşa edilmiştir ve sunumumuz için bir yön belirlemeye yardımcı olan inceleme ekibinin geri bildirimlerine dayanmaktadır. Metnin odak noktası, dijital tasarımda bir temel dersin içeriğini ve günümüz dijital sistemlerinin ana akım teknolojisi olan CMOS devrelerini daha yakından yansıtacak şekilde keskinleştirilmiştir. Hedef kitle genişler ve bilgisayar bilimi, bilgisayar mühendisliği ve elektrik mühendisliği öğrencilerini kapsamaktadır. Kitabın odaklandığı ana unsurlar (1) Boolean mantığı, (2) tasarımcılar tarafından kullanılan mantık kapıları, (3) senkron sonlu durum makineleri ve (4) veri yolu kontrolörü tasarımıdır—hepsi dijital sistemler tasarlamak için perspektifinden. Bu odak, elektronik dersi için daha uygun olan materyalin ortadan kaldırılmasına yol açmıştır. Bu nedenle okuyucu burada asenkron makineler veya bipolar transistörlerin tanımlarıyla ilgili içerik bulamayacaktır. Ayrıca, web tabanlı yardımcı materyalin yaygın olarak mevcut olması, tartışmamızı yalnızca bir üretici tarafından sunulan cihazların tanıtımıyla sınırlamaya yönlendirmiştir, iki üretici yerine. Günün tasarımcıları, donanım tanım dillerine büyük ölçüde güvenmektedirler.

(HDL'ler) ve bu kitabın bu baskısı, bunların kullanımına daha fazla dikkat vermekte ve Verilog HDL kullanarak bir tasarım metodolojisinin net bir gelişimini sunduğumuzu düşünüyoruz.

ÇOK MODLU ÖĞRENME

Dijital Tasarım çok modlu bir öğrenme yaklaşımını destekler. Öğrenme modlarının VARK karakterizasyonu, insanların öğrenme şekillerini tanımlayan dört ana modu belirler: (V) görsel, (A) işitsel, (R) okuma ve (K) kinestetik. Geriye dönüp baktığımızda, metnimizin nispeten yüksek düzeydeki illüstrasyonları ve grafik içeriğinin VARK'ın görsel (V) bileşene hitap ettiğini not ediyoruz; tartışmalar ve sayısız örnek, okuma (R) bileşenine hitap etmektedir. Ücretsiz simülatörlerin kullanılabilirliğinden yararlanan öğrenciler, çalışan bir mantık sistemi tasarılmasının olumlu geri bildirimleri ve keyfi ile birlikte kinestetik (K) bir öğrenme deneyimine yönlendirilmektedir. VARK'ın kalan unsuru, işitsel/auditory (A) deneyimi ise eğitime bırakılmıştır. Sınıf derslerini desteklemek için bol miktarda materyal ve örnek sağladık. Böylece, Dijital Tasarım kullanarak bir dijital tasarım dersi, zengin, dengeli bir öğrenme deneyimi sunabilir ve VARK tarafından tanımlanan tüm modları ele alabilir.

HDL'lerin (Yüksek Tanımlı Diller) sunumu ve kullanımını ilk dijital tasarım dersinde sorgulayanlar için, endüstrinin büyük ölçüde şematik tabanlı tasarım girişini terk ettiğini belirtelim. Bu tarz, entegre devre (IC) tasarımı için CAD araçlarının yeni gelişimi sırasında 1980'lerde ortaya çıkmıştır. Şematik giriş, şemanın düzeninde örtük olarak bulunan bir işlevselliğin temsiliyi oluşturur. Ne yazık ki, bir mantık devresinin şemasının temsil ettiği işlevselliği, onun inşasında rol oynamadan veya tasarım amacını ifade eden ek belgeler olmadan, makul bir süre içinde belirlemek zordur. Sonuç olarak, endüstri, bir tasarımın işlevselliğini tanımlamak ve tasarımın standart hücre tabanlı ASIC veya FPGA'daki donanım uygulamasını belgelemek, simüle etmek, test etmek ve sentezlemek için HDL'lere (örneğin, Verilog) geçmiştir. Bir şemanın faydası, dikkatlice oluşturulmuş bir tasarım modülleri hiyerarşisinin titiz ve detaylı belgelenmesine bağlıdır. Eski paradigmada, tasarımcılar işlevselliği uygulamak için bir devrenin şemasını oluşturmak üzere yılların deneyimine güveniyorlardı. Günümüz tasarım akışında, HDL kullanan tasarımcılar işlevselliği doğrudan ve açık bir şekilde, yıllarca birikmiş deneyim olmadan ifade edebilir ve şemayı otomatik olarak bir yan ürün olarak oluşturmak için sentez araçlarını kullanabilirler. Endüstri uygulamaları buraya ulaştı çünkü şematik giriş, büyük ve karmaşık IC'leri anlamada ve tasarlamada bizi verimsizliğe, hatta başarısızlığa mahkum ediyor.

Bu baskıda tekrar belirtmek isteriz ki, dijital devrelerin tasarımında birinci ders olarak HDL'lerin tanıtılması, bu tür devrelerin yapı taşlarının temel anlayışını değiştirmek veya manuel tasarım yöntemlerinin tartışmasını ortadan kaldırmak amacıyla değildir. Bir öğrencinin donanımın nasıl çalıştığını anlaması hala esastır. Bu nedenle, kombinasyonel ve sıralı mantık cihazlarının kapsamlı bir şekilde ele alınmasını sürdürüyoruz. Manuel tasarım uygulamaları sunulmakta ve bunların sonuçları HDL tabanlı bir paradigma ile karşılaştırılmaktadır. Ancak sunduğumuz şey, donanımın nasıl tasarlandığına vurgu yapmaktır; bu, öğrenciyi günümüz endüstrisinde, HDL tabanlı tasarım uygulamalarının baskın olduğu bir kariyere daha iyi hazırlamak içindir.

ESNEKLİK

Metindeki konu sıralaması, dijital tasarımın geleneksel, manuel tabanlı ele alınışına, bir HDL kullanarak tasarımı ele alan kurslara ve iki yaklaşım arasında geçiş yapan veya bunları harmanlayan kurslara uyum sağlayabilir. Modern sentez araçları otomatik olarak mantık minimizasyonu gerçekleştirdiğinden, Karnaugh haritaları ve optimizasyonla ilgili konular dijital tasarımın ele alınışının başında sunulabilir veya devreler ve bunların uygulamaları bir HDL ile incelendikten ve simüle edildikten sonra sunulabilir. Metin, hem manuel hem de HDL tabanlı tasarım örneklerini içermektedir. Bölüm sonu problemlerimiz, geleneksel bir manuel tasarım görevini ele alan problemler ile bu görevi başarmak için bir HDL kullanan bir eşlik eden problem arasında çapraz referans vererek bu esnekliği daha da kolaylaştırmaktadır. Ayrıca, metinde, metnin sonundaki seçilmiş problemlere verilen yanıtlarda ve çözüm kılavuzunda simülasyonların notlandırılmış sonuçlarını sunarak manuel ve HDL tabanlı yaklaşımları birbirine bağlıyoruz.

BU BASKIYA YENİ

Bu baskı, *Dijital Tasarım* IEEE Standard 1364'ün en son özelliklerini kullanmaktadır, ancak yalnızca pedagojik hedeflerimizi desteklediği ölçüde. Metindeki revizyonlar ve güncellemeler şunları içermektedir:

- Mantık devreleri ve dijital tasarımda genellikle birinci ders kapsamında ele alınmayan özel devre düzeyindeki içeriğin kaldırılması (örneğin, RTL, DTL ve emiter bağlı mantık devreleri)
- Her bölümün sonunda öğrencilere web'de mevcut ek konulara yönlendiren "Web Arama Konuları" eklenmesi
- Bölümlerin sonunda yer alan problemlerin yaklaşık üçte birinin revize edilmesi
- Tüm metin için basılı bir çözüm kılavuzu, tüm yeni problemleri de içermektedir
- Karnaugh haritalarının tartışmasının sadeleştirilmesi
- Temel CMOS teknolojisinin mantık kapıları ile birlikte ele alınması
- Yarı iletken teknolojisini tanıtan bir ekin dahil edilmesi

TASARIM YÖNTEMİ

Bu metin, bir dijital sistemin veri yolunu kontrol etmek için bir durum makinesi tasarlamak üzere sistematik bir metodoloji sunmaktadır. Ayrıca, bu materyalin sunulduğu çerçeve, veri yolundan gelen durum sinyallerinin kontrolör tarafından kullanıldığı gerçekçi durumu ele almaktadır, yani sistemin geri beslemesi vardır. Böylece, ele alışımız karmaşık ve etkileşimli dijital sistemlerin tasarımı için bir temel sağlamaktadır. HDL tabanlı tasarıma vurgu ile sunulmuş olmasına rağmen, metodoloji manuel tabanlı tasarım yaklaşımlarına da uygulanabilir.

YETERLİ HDL

Bu metnin düzeyine ve kapsamına uygun olan Verilog dilinin yalnızca o unsurlarını sunuyoruz. Ayrıca, doğru sözdizimi, bir modelin işlevsel bir spesifikasyona uyduğunu veya fiziksel donanımın sentezlenebileceğini garanti etmez. Bu nedenle, öğrencilere, davranışsal bir tanımın fiziksel donanıma sentezlenmesini sağlamak ve sentezlenen devrenin davranışının davranışsal tanımla eşleşmesini sağlamak için modeller yazarken endüstri tabanlı uygulamaların disiplinli bir şekilde kullanılmasını tanıtıyoruz. Bu disiplini takip etmemek, bu tür makinelerin HDL modellerinde yazılım yarış koşullarına, bunları doğrulamak için kullanılan test tezgahında yarış koşullarına ve bir davranışsal modelin simülasyonu ile onun sentezlenmiş fiziksel karşılığı arasındaki uyumsuzluğa yol açabilir. Benzer şekilde, endüstri uygulamalarına uymamak, doğru simüle edilen ancak tasarımcı tarafından kullanılan modelleme tarzının bir sonucu olarak tasarıma kazara eklenen donanım kilitleri olan tasarımlara yol açabilir. Sunmuş olduğumuz endüstri tabanlı metodoloji, yarış koşullarından ve kilitlerden arındırılmış tasarımlara yol açar. Öğrencilerin HDL modellerini kullanırken endüstri uygulamalarını öğrenmeleri ve takip etmeleri önemlidir; bu, öğrencinin müfredatının sentez araçlarına erişimi olup olmamasından bağımsızdır.

DOĞRULAMA

Sanayide, bir devrenin işlevselliğinin doğru olduğunu doğrulamak için önemli bir çaba harcanmaktadır. Ancak, dijital tasarım üzerine giriş metinlerinde doğrulamaya pek fazla dikkat edilmemekte, burada odak tasarımın kendisinde olup, test belki de ikincil bir faaliyet olarak görülmektedir. Deneyimlerimiz, bu görüşün erken “yüksek beşler” ve “devre harika çalışıyor” açıklamalarına yol açabileceğini göstermektedir. Benzer şekilde, sanayi, bir HDL modeline yapılan yatırımlardan tekrar tekrar kazanç elde etmektedir; bu, modelin okunabilir, taşınabilir ve yeniden kullanılabilir olmasını sağlamakla mümkündür.

Yeniden kullanılabilirliği ve taşınabilirliği kolaylaştırmak için adlandırma uygulamalarını ve parametrelerin kullanımını gösteriyoruz. Ayrıca, (1) devrenin işlevselliğini doğrulamak, (2) kapsamlı testi önemi vurgulamak ve (3) öğrencilere kendinden kontrol eden test tezgahları gibi önemli kavramları tanıtmak için tüm çözümler ve alıştırmalar için test tezgahları sağlıyoruz. Bir test tezgahının geliştirilmesine rehberlik edecek bir test planının geliştirilmesini savunarak ve göstererek, metinde basit bir şekilde test planlarını tanıtıyoruz ve bunları çözümler kılavuzunda ve metnin sonunda seçilen problemlere verilen yanıtlarda genişletiyoruz.

HDL İÇERİĞİ

Metin ve çözüm kılavuzundaki tüm örneklerin, dijital donanım modelleme için kabul edilen endüstri uygulamalarına uygun olmasını sağladık. Önceki baskıda olduğu gibi, HDL materyali ayrı bölümlere yerleştirilmiştir, böylece istenildiği gibi kapsanabilir veya atlanabilir, manuel tabanlı tasarımın ele alınmasını azaltmaz ve sunum sırasını belirlemez. Ele alma, dijital devreleri ve aynı anda bir HDL öğrenen başlangıç öğrencileri için uygun bir seviyededir. Metin,

ncileri önemli bağımsız tasarım projeleri üzerinde çalışmaya ve daha sonraki bir bilgisayar mimarisi ve ileri dijital tasarım dersinde başarılı olmaya hazırlar.

Öğretmen Kaynakları

Öğretmenler, metin için yayınevinin web sitesinden (www.pearsonhighered.com/mano) aşağıdaki sınıf hazır kaynakları indirebilirler:

- Testteki tüm Verilog HDL örnekleri için kaynak kodu ve test tezgahları
- Metindeki tüm şekiller ve tablolar
- Çözüm kılavuzundaki tüm HDL modelleri için kaynak kodu
- Sınıf sunumu için uygun grafiklerle indirilebilir bir çözüm kılavuzu

HDL Simülatörleri

Companion Web Sitesi, Synapti-CAD tarafından sağlanan iki simülatörün web URL'lerini tanımlar. İlk simülatör, *VeriLogger Pro*, kitapta yer alan HDL örneklerini simüle etmek ve HDL problemlerinin çözümlerini doğrulamak için kullanılabilen geleneksel bir Verilog simülatörüdür. Bu simülatör, IEEE-1995 standardının sözdizimini kabul eder ve eski modellere sahip olanlar için faydalı olacaktır. Etkileşimli bir simülatör olarak, *Verilogger Extreme* IEEE-2001 ve IEEE-1995 sözdizimini kabul eder, tasarımcının simülasyon modelinin veya şemasının tamamlanmadan önce tasarım fikirlerini simüle etmesine ve analiz etmesine olanak tanır. Bu teknoloji, öğrenciler için özellikle faydalıdır çünkü hızlı bir şekilde Boolean ve D flip-flop veya latch giriş denklemleri girerek eşdeğerliliği kontrol edebilir veya flip-flop ve latch tasarımlarıyla deney yapabilirler. Öğrenciler Companion Web Sitesi'ne [at www.pearsonhighered.com/mano](http://www.pearsonhighered.com/mano) erişebilir.

Bölüm Özeti

Aşağıda her bölümde ele alınan konuların kısa bir özeti bulunmaktadır.

Bölüm 1 dijital sistemlerde bilgiyi temsil etmek için uygun çeşitli ikili sistemleri sunar. İkili sayı sistemi açıklanır ve ikili kodlar gösterilir. İşaretsiz ikili sayılar ve ikili kodlu ondalık (BCD) formatındaki ondalık sayılar için toplama ve çıkarma örnekleri verilir.

Bölüm 2 Boolean cebirinin temel postulatlarını tanıtır ve Boolean ifadeleri ile bunların karşılık gelen mantık diyagramları arasındaki ilişkiyi gösterir. İki değişken için tüm olası mantık işlemleri incelenir ve dijital sistemlerin tasarımında kullanılan en yararlı mantık kapıları belirlenir. Bu bölüm ayrıca temel CMOS mantık kapılarını tanıtır.

Bölüm 3 Boolean ifadelerini basitleştirmek için harita yöntemini kapsar. Harita yöntemi, AND-OR, NAND veya NOR kapıları ile inşa edilen dijital devreleri basitleştirmek için de kullanılır. Diğer tüm olası iki seviyeli kapı devreleri dikkate alınır ve bunların uygulanma yöntemi açıklanır. Verilog HDL, kapı düzeyindeki modellerin basit örnekleri ile birlikte tanıtılır.

Bölüm 4 kombinasyonel devrelerin analizi ve tasarımı için resmi prosedürleri özetler. Dijital sistemlerin tasarımında kullanılan bazı temel bileşenler, örneğin toplayıcılar ve kod dönüştürücüler, tasarım örnekleri olarak tanıtılır. Paralel toplayıcılar ve çıkarıcılar, kod çözücüler, kodlayıcılar ve çoklayıcılar gibi sıkça kullanılan dijital mantık fonksiyonları açıklanır ve bunların kombinasyonel devrelerin tasarımındaki kullanımı gösterilir. HDL örnekleri, Verilog HDL'de kombinasyonel devreleri tanımlamak için mevcut alternatif yolları göstermek amacıyla kapı düzeyinde, veri akışı ve davranışsal modellerde verilmektedir. Bir HDL tasarımına uyarı sağlamak için basit bir test tezgahı yazma prosedürü sunulmaktadır.

Bölüm 5 saatli (senkron) ardışık devrelerin analiz ve tasarımı için resmi prosedürleri özetler. Birkaç tür flip-flop'un kapı yapısı, seviye ve kenar tetikleme arasındaki fark üzerine bir tartışma ile birlikte sunulmaktadır. Ardışık bir devreyi analiz ederken durum tablosunun ve durum diyagramının türetilmesini göstermek için belirli örnekler kullanılmaktadır. D-tipi flip-flop'lar kullanan ardışık devrelere vurgu yapılarak birçok tasarım örneği sunulmaktadır. Ardışık devreler için Verilog HDL'de davranışsal modelleme açıklanmaktadır. Mealy ve Moore ardışık devre modellerini göstermek için HDL örnekleri verilmektedir.

Bölüm 6 çeşitli sıralı devre bileşenleri ile ilgilenmektedir; bunlar arasında kayıtlar, kaydırma kayıtları ve sayıcılar bulunmaktadır. Bu dijital bileşenler, daha karmaşık dijital sistemlerin yapıldığı temel yapı taşlarıdır. Kaydırma kayıtları ve sayıcıların HDL tanımları sunulmaktadır.

Bölüm 7 rastgele erişim belleği (RAM) ve programlanabilir mantık cihazları ile ilgilenmektedir. Bellek kodlama ve hata düzeltme şemaları tartışılmaktadır. ROM'lar, PLA'lar, PAL'lar, CPLD'ler ve FPGA'lar gibi kombinasyonel ve sıralı programlanabilir cihazlar sunulmaktadır.

Bölüm 8 dijital sistemlerin kayıt transfer seviyesi (RTL) temsilini ele almaktadır. Algoritmik durum makinesi (ASM) şeması tanıtılmaktadır. ASM şeması, ASMD şeması, RTL temsili ve HDL taşımalarının dijital sistem tasarımındaki kullanımlarını gösteren birçok örnek bulunmaktadır. Bir veri yolunu kontrol etmek için bir sonlu durum makinesinin tasarımı detaylı bir şekilde sunulmaktadır; bu, veri yolundan gelen durum sinyallerinin onu kontrol eden durum makinesi tarafından kullanıldığı gerçekçi bir durumu içermektedir. Bu bölüm, öğrencilere daha ileri tasarım projelerine sistematik bir yaklaşım sunduğu için kitapta en önemli bölümdür.

Bölüm 9 laboratuvar ortamında ticari olarak kolayca temin edilebilen donanımlarla gerçekleştirilebilecek deneyleri özetlemektedir. Deneylerde kullanılan entegre devrelerin (IC) çalışması, önceki bölümlerde tanıtılan benzer bileşenlerin diyagramlarına atıfta bulunarak açıklanmaktadır. Her deney gayri resmi bir şekilde sunulmakta ve öğrenciden devreyi tasarlama ve laboratuvar ortamında çalışmasını kontrol etmek için bir prosedür oluşturma beklenmektedir. Laboratuvar deneyleri bağımsız bir şekilde kullanılabilir ve geleneksel bir yaklaşım ile, bir breadboard ve TTL devreleri kullanarak veya HDL/sentez yaklaşımı ile FPGA'lar kullanarak gerçekleştirilebilir. Günümüzde, bir HDL modelini sentezlemek ve bir devreyi FPGA ile uygulamak için yazılımlar, FPGA tedarikçilerinden ücretsiz olarak temin edilebilmektedir; bu da öğrencilerin prototipleme kartları ve laboratuvar kaynaklarını kullanmadan önce kişisel ortamlarında önemli miktarda çalışma yapmalarına olanak tanımaktadır.

FPGA'lar ile hızlı prototipleme devreleri için devre kartları nominal bir maliyetle mevcuttur ve genellikle butonlar, anahtarlar, yedi segmentli göstergeler, LCD'ler, tuş takımları ve diğer girdi/çıkıcı hazırlarını içermektedir. Bu kaynaklarla, öğrenciler belirlenen laboratuvar alıştırmalarını veya kendi projelerini gerçekleştirebilir ve sonuçları hemen alabilirler.

Bölüm 10 ANSI/IEEE standardı tarafından önerilen mantık fonksiyonları için standart grafik sembolleri sunmaktadır. Bu grafik semboller, kullanıcının her fonksiyonu atanmış beşerlik grafik sembolünden tanıyabilmesi için küçük ölçekli entegrasyon (SSI) ve orta ölçekli entegrasyon (MSI) bileşenleri için geliştirilmiştir. Bölüm, laboratuvar deneylerinde kullanılan entegre devrelerin standart grafik sembollerini göstermektedir.

TEŞEKKÜRLER

Digital Design, 5e. Bu baskının şekillenmesine yardımcı olan uzmanlıkları, dikkatli incelemeleri ve önerileri için değerlendireceğimize minnettarız.

Dmitri Donetski, Stony Brook Üniversitesi
 Ali Amini, California Eyalet Üniversitesi, Northridge
 Mihaela Radu, Rose Hulman Teknoloji Enstitüsü
 Stephen J. Kuyath, Kuzey Carolina Üniversitesi, Charlotte
 Peter Pachowicz, George Mason Üniversitesi
 David Jeff Jackson, Alabama Üniversitesi
 A. John Boye, Nebraska Üniversitesi, Lincoln
 William H. Robinson, Vanderbilt Üniversitesi
 Dinesh Bhatia, Texas Üniversitesi, Dallas

Ayrıca, metnimizin bu baskısını destekleyen Prentice Hall/Pearson Education editörlük ve yayın ekibine de minnettarlığımızı ifade etmek isteriz. Eşlerimiz Sandra ve Jerilyn'nin sürekli desteği ve teşvikleri için de müteşekkirimiz.

M. MORRIS MANO
Emekli Bilgisayar Mühendisliği Profesörü
California Eyalet Üniversitesi, Los Angeles

MICHAEL D. CILETTI
Emekli Elektrik ve Bilgisayar Mühendisliği Profesörü
Colorado Üniversitesi, Colorado Springs

Bu sayfa kasıtlı olarak boş bırakılmıştır

Bölüm 1

Sayısal Sistemler ve İkili Sayılar

1.1 SAYISAL SİSTEMLER

Sayısal sistemler, günlük yaşamda o kadar belirgin bir rol oynamaktadır ki, mevcut teknolojik dönemi *sayısal çağ* olarak adlandırıyoruz. Sayısal sistemler, iletişim, işletme işlemleri, trafik kontrolü, uzay aracı yönlendirme, tıbbi tedavi, hava durumu izleme, İnternet ve birçok diğer ticari, endüstriyel ve bilimsel işletmelerde kullanılmaktadır. Sayısal telefonlarımız, sayısal televizyonlarımız, sayısal çok amaçlı disklerimiz, sayısal kameralarımız, cep telefonlarımız ve elbette sayısal bilgisayarlarımız var. Taşınabilir medya oynatıcılarımıza (örneğin, iPod Touch™) ve yüksek çözünürlüklü ekranlara sahip diğer el cihazlarımıza indirilen müziğin tadını çıkarıyoruz. Bu cihazlar, kullanıcıya basit gibi görünen komutları yerine getirmelerini sağlayan grafik kullanıcı arayüzlerine (GUI) sahiptir, ancak aslında, karmaşık iç talimatların bir dizisinin hassas bir şekilde yürütülmesini içerir. Bu cihazların çoğu, hatta hepsi, içinde gömülü özel amaçlı bir sayısal bilgisayar barındırmaktadır. Sayısal bilgisayarın en çarpıcı özelliği genel oluşudur. Verilen veriler üzerinde çalışan bir program olarak adlandırılan bir talimat dizisini takip edebilir. Kullanıcı, belirli bir ihtiyaca göre programı veya verileri belirleyip değiştirebilir. Bu esneklik sayesinde, genel amaçlı sayısal bilgisayarlar, geniş bir uygulama yelpazesini kapsayan çeşitli bilgi işleme görevlerini yerine getirebilir.

Bir dijital sistemin bir özelliği, bilgilerin ayrıık unsurlarını temsil etme ve bunları manipüle etme yeteneğidir. Sınırlı sayıda unsura sahip herhangi bir küme, ayrık bilgi içerir. Ayrık kümelere örnekler, 10 ondalık basamak, 26 harfli alfabeyi, 52 oyun kartını ve 64 kareli bir satranç tahtasını içerir. Erken dijital bilgisayarlar sayısal hesaplamalar için kullanılıyordu. Bu durumda, ayrık unsurlar basamaklardı. Bu uygulamadan, dijital bilgisayar terimi ortaya çıktı. Bilgi ayrık unsurları, dijital bir sistemde fiziksel nicelikler olarak temsil edilir.

2 Bölüm 1 Dijital Sistemler ve İkili Sayılar

sinyal olarak adlandırılır. Voltajlar ve akımlar gibi elektrik sinyalleri en yaygın olanlardır. Bu sinyalleri uygulayan devrelerde, transistör adı verilen elektronik cihazlar baskındır. Günümüzdeki çoğu elektronik dijital sistemde sinyaller yalnızca iki ayrı değer kullanır ve bu nedenle ikili olarak adlandırılır. İkili basamak, bit olarak adlandırılır ve iki değere sahiptir: 0 ve 1. Bilgi ayrıık unsurları, bit gruplarıyla ikili kodlar olarak temsil edilir. Örneğin, ondalık basamaklar 0'dan 9'a kadar bir dijital sistemde dört bitlik bir kodla temsil edilir (örneğin, 7 sayısı 0111 ile temsil edilir). Bitlerin bir deseninin bir sayı olarak nasıl yorumlandığı, bulunduğu kod sistemine bağlıdır. Bu ayrımı yapmak için, $(0111)_2$ yazarak 0111 deseninin bir ikili sistemde yorumlanacağını, $(0111)_{10}$ yazarak ise referans sisteminin ondalık olduğunu belirtebiliriz. Sonra $0111_2 = 7_{10}$, bu 0111_{10} ile aynı değildir, yani yüz on bir. Bir desenin yorumlanması için tabanı gösteren alt simge yalnızca açıklama gerektiğinde kullanılacaktır. Çeşitli teknikler aracılığıyla, bit grupları, mutlaka sayılar olmayan ayrıık sembollerini temsil edecek şekilde oluşturulabilir ve bu semboller daha sonra dijital formatta sistemi geliştirmek için kullanılır. Böylece, dijital bir sistem, içsel olarak ikili biçimde temsil edilen bilgi elemanlarının ayrıık unsurlarını manipüle eden bir sistemdir. Günümüz teknolojisinde, ikili sistemler en pratik olanlardır çünkü, göreceğimiz gibi, elektronik bileşenlerle uygulanabilirler.

Ayrıık bilgi miktarları ya işlenmekte olan verinin doğasından ortaya çıkar ya da sürekli bir süreçten kuantize edilebilir. Bir yandan, bir bordro programı, çalışan isimleri, sosyal güvenlik numaraları, haftalık maaşlar, gelir vergileri vb. içeren doğası gereği ayrıık bir süreçtir. Bir çalışanın maaş çeki, alfabedeki harfler (isimler), rakamlar (maaş) ve özel semboller (örneğin \$) gibi ayrıık veri değerleri aracılığıyla işlenir. Öte yandan, bir araştırma bilimcisi sürekli bir süreci gözlemleyebilir, ancak yalnızca belirli miktarları tablo biçiminde kaydedebilir. Bilimci böylece sürekli verileri kuantize etmekte, tablosundaki her sayıyı ayrıık bir miktar haline getirmektedir. Bir çok durumda, bir sürecin kuantizasyonu, bir analogdan dijital çevirici tarafından otomatik olarak gerçekleştirilebilir; bu cihaz, bir analog (sürekli) miktarın dijital (ayrıık) temsilini oluşturur.

Genel amaçlı dijital bilgisayar, dijital sistemlerin en iyi bilinen örneğidir. Bir bilgisayarın ana parçaları bir bellek birimi, bir merkezi işlem birimi ve girdi-çıkıktı birimleridir. Bellek birimi, programları ve girdi, çıktı ve ara verileri depolar. Merkezi işlem birimi, program tarafından belirtilen aritmetik ve diğer veri işleme işlemlerini gerçekleştirir. Kullanıcı tarafından hazırlanan program ve veriler, bir klavye gibi bir girdi cihazı aracılığıyla belleğe aktarılır. Bir çıktı cihazı, örneğin bir yazıcı, hesaplamaların sonuçlarını alır ve basılı sonuçlar kullanıcıya sunulur. Bir dijital bilgisayar, birçok girdi ve çıktı cihazını barındırabilir. Çok yararlı bir cihaz, diğer kullanıcılarla İnternet üzerinden etkileşim sağlayan bir iletişim birimidir. Dijital bilgisayar, yalnızca aritmetik hesaplamalar yapmakla kalmayıp, aynı zamanda mantıksal işlemler de gerçekleştirebilen güçlü bir araçtır. Ayrıca, iç ve dış koşullara dayanarak kararlar vermek üzere programlanabilir.

Ticari ürünlerin dijital devrelerle üretilmesinin temel nedenleri vardır. Dijital bir bilgisayar gibi, çoğu dijital cihaz programlanabilir. Programlanabilir bir cihazdaki programı değiştirerek, aynı temel donanım birçok farklı uygulama için kullanılabilir, böylece geliştirme maliyetinin daha geniş bir müşteri tabanına yayılmasına olanak tanır. Dijital cihazlardaki dramatik maliyet düşüşleri bu şekilde gerçekleşmiştir.

gre devre teknolojisindeki ilerlemeler nedeniyle. Bir parça silikona yerleştirilebilecek transistor sayısı arttıkça karmaşık işlevler üretmek için, birim başına maliyet azalır ve dijital cihazlar giderek daha düşük fiyatlarla satın alınabilir. Dijital entegre devrelerle inşa edilen ekipman, saniyede yüz milyonlarca işlem hızında çalışabilir. Hata düzeltme kodları kullanarak dijital sistemler son derece güvenilir bir şekilde çalıştırılabilir. Bu stratejinin bir örneği, video, ses ve diğer verileri temsil eden dijital bilgilerin tek bir öge kaybı olmadan kaydedildiği dijital çok yönlü disk (DVD) dir. Bir DVD'deki dijital bilgi, her dijital örnekteki kodu geri oynatmadan önce inceleyerek, herhangi bir hatanın otomatik olarak tanımlanıp düzeltilbileceği şekilde kaydedilir.

Dijital bir sistem, dijital modüllerin birbiriyle bağlantısıdır. Her dijital modülün işleyişini anlamak için, dijital devreler ve bunların mantıksal işlevleri hakkında temel bir bilgiye sahip olmak gereklidir. Bu kitabın ilk yedi bölümü, mantık kapısı yapıları, kombinasyonel ve sıralı devreler ve programlanabilir mantık cihazları gibi dijital tasarımın temel araçlarını sunmaktadır. Bölüm 8, modern bir donanım tanım dili (HDL) kullanarak kayıt transfer seviyesi (RTL) düzeyinde dijital tasarımı tanıtmaktadır. Bölüm 9, metni dijital devreler kullanarak laboratuvar alıştırmaları ile tamamlamaktadır.

Dijital tasarım metodolojisindeki önemli bir trend, bir HDL (Donanım Tanım Dili) kullanarak dijital devrelerin işlevselliğini tanımlamak ve simüle etmektir. Bir HDL, bir programlama diline benzer ve dijital devreleri metin biçiminde tanımlamak için uygundur. Donanım inşa edilmeden önce bir dijital sistemin çalışmasını doğrulamak için simüle etmek amacıyla kullanılır. Ayrıca, tasarım sürecini otomatikleştirmek için mantık sentez araçlarıyla birlikte kullanılır. Çünkü öğrencilerin HDL tabanlı bir tasarım metodolojisine aşina olanlar için, dijital devrelerin HDL tanımları kitap boyunca sunulmaktadır. Bu örnekler, bir HDL'nin özelliklerini açıklamaya yardımcı olurken, aynı zamanda endüstrinin HDL'leri kullanma konusundaki en iyi uygulamalarını da göstermektedir. Bu uygulamaların çeşitliliği, sevimli ama değersiz HDL modellerine yol açacaktır; bu modeller bir fenomeni simüle edebilir, ancak tasarım araçları tarafından sentezlenemez veya silikon alanını israf eden ya da doğru çalışmayacak donanıma sentezlenen modellere yol açabilir.

Daha önce belirtildiği gibi, dijital sistemler, ikili biçimde temsil edilen ayrık bilgi miktarlarını işler. Hesaplamalarda kullanılan operandlar, ikili sayı sisteminde ifade edilebilir. Ondalık rakamlar ve alfabedeki karakterler gibi diğer ayrık unsurlar, ikili kodlarla temsil edilir. Mantık devreleri olarak da adlandırılan dijital devreler, ikili sinyalleri kullanarak ikili mantık elemanları (mantık kapıları) aracılığıyla verileri işler. Miktarlar, ikili (iki değerli) depolama elemanlarında (flip-flop'lar) saklanır. Bu bölümün amacı, sonraki bölümlerdeki daha ileri çalışmalar için bir referans çerçevesi olarak çeşitli ikili kavramları tanıtmaktır.

1.2 İKİLİ SAYILAR

7.392 gibi bir ondalık sayı, 7 bin, 3 yüz, 9 on ve 2 birimden oluşan bir miktarı temsil eder. Binler, yüzler vb. sayının içindeki katsayıların (sembollerin) konumuna atıfta bulunulan 10'un kuvvetleridir. Daha kesin olmak gerekirse, 7.392, şu şekilde yazılması gereken bir kısayol notasyonudur

$$7 \times 10^3 + 3 \times 10^2 + 9 \times 10^1 + 2 \times 10^0$$

4 Bölüm 1 Dijital Sistemler ve İkili Sayılar

Ancak, gelenek, yalnızca sayısal katsayıları yazmak ve bunların konumundan gerekli 10'un kuvvetlerini çıkarmaktır; kuvvetler sağdan sola doğru artar. Genel olarak, ondalık noktalı bir sayı, bir dizi katsayı ile temsil edilir:

$$a_5a_4a_3a_2a_1a_0 \cdot a_{-1}a_{-2}a_{-3}$$

Katsayılar a_j 10 rakamından herhangi biri (0, 1, 2, ..., 9) olup, alt simge değeri j yer değerini verir ve dolayısıyla katsayının çarpılması gereken 10'un kuvvetini belirtir. Böylece, önceki ondalık sayı şu şekilde ifade edilebilir

$$10^5a_5 + 10^4a_4 + 10^3a_3 + 10^2a_2 + 10^1a_1 + 10^0a_0 + 10^{-1}a_{-1} + 10^{-2}a_{-2} + 10^{-3}a_{-3}$$

$a_3 = 7, a_2 = 3, a_1 = 9, \text{ ve } a_0 = 2.$

Ondalık sayı sistemi, 10 rakamı kullandığı ve katsayılar 10'un kuvvetleri ile çarpıldığı için *taban*, veya *radix*, 10 olarak adlandırılır. *İkili* sistem ise farklı bir sayı sistemidir. İkili sayı sisteminin katsayıları yalnızca iki olası değere sahiptir: 0 ve 1. Her katsayı a_j bir tabanın kuvveti ile çarpılır, örneğin, 2^j , ve sonuçlar, sayının ondalık karşılığını elde etmek için toplanır. Taban noktası (örneğin, 10 taban olduğunda ondalık nokta), 10'un pozitif kuvvetlerini negatif kuvvetlerden ayırır. Örneğin, ikili sayı 11010.11'in ondalık karşılığı 26.75'tir; bu, katsayıların 2'nin kuvvetleri ile çarpılmasından gösterilmektedir:

$$1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} = 26.75$$

Farklı sayı sistemleri vardır. Genel olarak, bir taban- r

sisteminde ifade edilen bir sayı, r 'nin kuvvetleri ile çarpılan katsayılar içerir:

$$a_n \cdot r^n + a_{n-1} \cdot r^{n-1} + \dots + a_2 \cdot r^2 + a_1 \cdot r + a_0 + a_{-1} \cdot r^{-1} \\ + a_{-2} \cdot r^{-2} + \dots + a_{-m} \cdot r^{-m}$$

Katsayılar a_j 0 ile $r-1$ arasında değer alır. Farklı tabanlardaki sayıları ayırt etmek için, katsayıları parantez içine alır ve kullanılan tabana eşit bir alt simge y azarız (bazen ondalık sayılar için, içeriğin ondalık tabanın açıkça belirtmesi dışında). Bir taban-5 sayısının örneği

$$(4021.2)_5 = 4 \times 5^3 + 0 \times 5^2 + 2 \times 5^1 + 1 \times 5^0 + 2 \times 5^{-1} = (511.4)_{10}$$

Taban 5 için katsayı değerleri yalnızca 0, 1, 2, 3 ve 4 olabilir. Sekizli sayı sistemi, sekiz basamağa sahip bir taban-8 sistemidir: 0, 1, 2, 3, 4, 5, 6, 7. Sekizli bir sayının örneği 127.4'tür. Eşdeğer ondalık değerini belirlemek için, sayıyı 8 tabanında bir kuvvet serisi ile genişletiriz:

$$(127.4)_8 = 1 \times 8^2 + 2 \times 8^1 + 7 \times 8^0 + 4 \times 8^{-1} = (87.5)_{10}$$

8 ve 9 rakamlarının sekizli bir sayıda yer alamayacağını unutmayın.

Bir sayının tabanı 10'dan küçük olduğunda, katsayılar için gerekli r rakamları ondalık sistemden ödünç almak adettendir. Sayının tabanı 10'dan büyük olduğunda, 10 ondalık rakamı tamamlamak için alfabadeki harfler kullanılır. Örneğin, onaltılı (taban-16) sayı sisteminde, ilk 10 rakam ödünç alınır.

ondalık sistemden. A, B, C, D, E ve F harfleri sırasıyla 10, 11, 12, 13, 14 ve 15 rakamları için kullanılır. Bir onaltılık sayının örneği

$$(B65F)_{16} = 11 \times 16^3 + 6 \times 16^2 + 5 \times 16^1 + 15 \times 16^0 = (46,687)_{10}$$

Onaltılık sistem, tasarımcılar tarafından dijital sistemlerdeki adreslerde, talimatlarda ve veriler e uzun bit dizilerini temsil etmek için yaygın olarak kullanılır. Örneğin, B65F, 10110110010100 00'i temsil etmek için kullanılır.

Daha önce belirtildiği gibi, bir ikili sayının rakamlarına *bit* denir. Bir bit 0'a eşit olduğunda, dönüşüm sırasında toplamda katkıda bulunmaz. Bu nedenle, ikili sayı dan ondalık sayıya dönüşüm, yalnızca 1'e eşit olan bitlere karşılık gelen iki kuvvetli eri ile sayıları ekleyerek elde edilebilir. Örneğin,

$$(110101)_2 = 32 + 16 + 4 + 1 = (53)_{10}$$

İkili sayıda dört 1 vardır. Karşılık gelen ondalık sayı, dört iki kuvvetinin toplamıdır. Sıfır ve 2'nin kuvvetinden elde edilen ilk 24 sayı

n Tablo 1.1 'de listelenmiştir. Bilgisayar çalışmalarında, 2^{10} K (kilo) olarak, 2^{20} M (mega) olarak, 2^{30} G (giga) olarak ve 2^{40} T (tera) olarak adlandırılır. Böylece, $4K=2^{12}=4,096$ ve $16M=2^{24}=16,777,216$. Bilgisayar kapasitesi genellikle bayt cinsinden verilir. Bir *bayt* sekiz bite eşittir ve bir klavye karakterinin kodunu barındırabilir (yani temsil edebilir). Dört gigabayt depolama alanına sahip bir bilgisayar sabit disk, $4G=2^{32}$ bayt kapasitesine sahiptir (yaklaşık 4 milyar bayt). Bir terabayt 1024 gigabayttır, yaklaşık 1 trilyon bayt.

Taban *r* ile sayılarla yapılan aritmetik işlemler, ondalık sayılar için geçerli olan aynı kuralları izler. Tanıdık taban 10 dışında bir taban kullanıldığında, yalnızca *r*-izin verilen rakamları kullanmaya dikkat edilmelidir. İki ikili sayının toplama, çıkarma ve çarpma örnekleri aşağıdaki gibidir:

toplanan:	101101	çıkartılan:	101101	çarpın:	1011
eklenen: +	100111	çıkaran: -	100111	çarpın: ×	101
toplam:	1010100	fark:	000110		1011
				kısmi ürün:	0000
				ürün:	1011
					110111

Tablo 1.1
İki Gücü

<i>n</i>	2^n	<i>n</i>	2^n	<i>n</i>	2^n
0	1	8	256	16	65,536
1	2	9	512	17	131,072
2	4	10	1.024 (1K)	18	262,144
3	8	11	2,048	19	524,288
4	16	12	4,096 (4K)	20	1,048,576 (1M)
5	32	13	8,192	21	2,097,152
6	64	14	16,384	22	4,194,304
7	128	15	32,768	23	8,388,608

6 Bölüm 1 Dijital Sistemler ve İkili Sayılar

İki ikili sayının toplamı, ondalık sistemdeki kurallarla hesaplanır, tek fark toplamın herhangi bir anlamlı pozisyonundaki basamaklarının yalnızca 0 veya 1 olabileceğidir. Verilen bir anlamlı pozisyonunda elde edilen herhangi bir taşma, bir anlamlı pozisyonundaki basamak çiftine kullanılır. Çıkarma biraz daha karmaşıktır. Kurallar hala ondalık sistemdeki gibidir, tek fark, verilen bir anlamlı pozisyonundaki borcun bir çıkaran basamağına 2 eklemesidir. (Ondalık sistemde bir borç, bir çıkaran basamağına 10 ekler.) Çarpma basittir: Çarpan basamakları her zaman 1 veya 0'dır; bu nedenle, kısmi çarpımlar ya çarpan sayısının kaydırılmış (sola) bir kopyasına ya da 0'a eşittir.

1.3 SAYI-BASAMAK DÖNÜŞÜMLERİ

Farklı bir tabandaki bir sayının temsilleri, aynı ondalık temsile sahiplerse eşdeğer olarak kabul edilir. Örneğin, $(0011)_8$ ve $(1001)_2$ eşdeğerdir—her ikisi de ondalık değeri 9'dur. Bir sayının r tabanından ondalık sisteme dönüşümü, sayıyı bir kuvvet serisi olarak genişleterek ve daha önce gösterildiği gibi tüm terimleri toplayarak yapılır. Şimdi, bir ondalık sayıyı r tabanındaki bir sayıya dönüştürmenin ters işlemi için genel bir prosedür sunuyoruz. Sayı bir taban noktası içeriyorsa, sayıyı bir tam kısım ve bir kesirli kısma ayırmak gereklidir, çünkü her kısım farklı şekilde dönüştürülmelidir. Bir ondalık tam sayının r tabanındaki bir sayıya dönüşümü, sayıyı ve tüm ardışık bölümlmeleri r ile bölerek ve kalanları biriktirerek yapılır. Bu prosedür en iyi örnekle gösterilir.

ÖRNEK 1.1

Ondalık 41'i ikiliye çevirin. İlk olarak, 41, 2'ye bölünerek 20'lik bir tam sayı bölüm ve 12 kalanı verir. Ardından bölüm tekrar 2'ye bölünerek yeni bir bölüm ve kalan elde edilir. Bu işlem, tam sayı bölümü 0 olana kadar devam eder. İstenilen ikili sayının katsayıları, kalanlardan şu şekilde elde edilir:

	Tam Bölüm	Kalan		Katsayı
$41/2=$	20	+	12	$a_0=1$
$20/2=$	10	+	0	$a_1=0$
$10/2=$	5	+	0	$a_2=0$
$5/2=$	2	+	12	$a_3=1$
$2/2=$	1	+	0	$a_4=0$
$1/2=$	0	+	12	$a_5=1$

Bu nedenle, cevap şudur: $(41)_{10}=(a_5a_4a_3a_2a_1a_0)_2=(101001)_2$.

Aritmetik işlem aşağıdaki gibi daha uygun bir şekilde manipüle edilebilir:

Tamsayı	Kalan
41	
20	1
10	0
5	0
2	1
1	0
0	1 10100

Onluk tamsayıların herhangi bir taban- r sistemine dönüşümü bu örneğe benzer, tek fark 2 yerine r ile bölme yapılmasıdır.

ÖRNEK 1.2

Onluk 153'ü sekizliğe dönüştürün. Gerekli taban r 8'dir. Öncelikle, 153 8'e bölüne rek 19'luk bir tamsayı bölüm ve 1 kalan verir. Ardından 19, 8'e bölünerek 2'lik bir tamsayı bölüm ve 3 kalan verir. Son olarak, 2, 8'e bölünerek 0 bölüm ve 2 kalan verir. Bu işlem aşağıdaki gibi rahatlıkla manipüle edilebilir:

153	
19	1
2	3
0	$2=(231)_8$

Ondalık *kesirin* ikiliye dönüştürülmesi, tam sayılar için kullanılan bir yöntem le gerçekleştirilir. Ancak, bölme yerine çarpma kullanılır ve kalıntılar yerine tam sayılar birikir. Yine, yöntem en iyi örnekle açıklanır.

ÖRNEK 1.3

$(0.6875)_{10}$ sayısını ikiliye dönüştürün. İlk olarak, 0.6875 sayısı 2 ile çarpılır ve bir tam sayı ile bir kesir elde edilir. Daha sonra yeni kesir 2 ile çarpılır ve yeni bir tam sayı ile yeni bir kesir elde edilir. İşlem, kesir 0 olana kadar veya basamak sayısı yeterli doğruluğa ulaşana kadar devam eder. İkili sayının katsayıları, tam sayılardan şu şekilde elde edilir:

	Tamsayı	Kesir	Katsayı
$0.6875 \times 2 =$	1	+	0.3750 $a_{-1}=1$
$0.3750 \times 2 =$	0	+	0.7500 $a_{-2}=0$
$0.7500 \times 2 =$	1	+	0.5000 $a_{-3}=1$
$0.5000 \times 2 =$	1	+	0.0000 $a_{-4}=1$

8 Bölüm 1 Dijital Sistemler ve İkili Sayılar

Bu nedenle, cevap $(0.6875)_{10} = (0.a_{-1}a_{-2}a_{-3}a_{-4})_2 = (0.1011)_2$.

Bir ondalık kesiri, taban r cinsinden ifade edilen bir sayıya dönüştürmek için benzer bir işlem kullanılır. Ancak, çarpma 2 yerine r ile yapılır ve tam sayılardan bulunan katsayılar 0 ile $r-1$ arasında bir değere sahip olabilir.

ÖRNEK 1.4

$(0.513)_{10}$ sayısını sekizli (oktal) sayıya çevirin.

$$0.513 \times 8 = 4.104$$

$$0.104 \times 8 = 0.832$$

$$0.832 \times 8 = 6.656$$

$$0.656 \times 8 = 5.248$$

$$0.248 \times 8 = 1.984$$

$$0.984 \times 8 = 7.872$$

Cevap, yedi anlamlı rakamla, çarpımların tam kısımlarından elde edilir:

$$(0.513)_{10} = (0.406517 \dots)_8$$

Tam ve kesirli kısımlara sahip ondalık sayıların dönüşümü, tam ve kesirli kısımların ayrı ayrı dönüştürülmesi ve ardından iki sonucun birleştirilmesiyle yapılır. Örnek 1.1 ve 1.3'ün sonuçlarını kullanarak,

$$(41.6875)_{10} = (101001.1011)_2$$

Örnek 1.2 ve 1.4'ten,

$$(153.513)_{10} = (231.406517)_8$$

1.4 SEKİZLİ VE ONALTILI SAYILAR

İkili, sekizli ve onaltılı sistemler arasında dönüşüm, dijital bilgisayarlarda önemli bir rol oynar, çünkü onaltılı karakterlerin daha kısa desenleri, 1'lerin ve 0'ların uzun desenlerinden daha kolay tanınır. Çünkü $2^3=8$ ve $2^4=16$, her sekizli rakam üç ikili rakama ve her onaltılı rakam dört ikili rakama karşılık gelir. Ondalık, ikili, sekizli ve onaltılı sayı sistemlerindeki ilk 16 sayı, Tablo 1.2'de listelenmiştir.

İkili sayıdan sekizli sayıya dönüşüm, ikili sayıyı üçer rakam gruplarına ayırarak kolayca gerçekleştirilir; bu işlem ikili noktadan başlayarak sola ve sağa doğru ilerler. Her gruba karşılık gelen sekizli rakam atanır.

Aşağıdaki örnek prosedürü göstermektedir:

$$\begin{array}{cccccccccccc} (10 & 110 & 001 & 101 & 011 & \cdot & 111 & 100 & 000 & 110)_2 & = & (26153.7406)_8 \\ 2 & 6 & 1 & 5 & 3 & & 7 & 4 & 0 & 6 \end{array}$$

Tablo 1.2

Farklı Tabanlara Sahip Sayılar

Ondalık (basamak 10)	İkili (2. taban)	Sekizli (8. taban)	Onaltılı (16. taban)
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

İkilikten onaltılıya dönüşüm benzer bir şekilde yapılır, tek fark, ikilik sayının dört basamaklık gruplara ayrılmasıdır:

$$\begin{array}{ccccccc} (10 & 1100 & 0110 & 1011 & \cdot & 1111 & 0010)_2 = (2C6B.F2)_{16} \\ 2 & C & 6 & B & & F & 2 \end{array}$$

Her bir ikilik basamağı grubunun karşılık gelen onaltılı (veya sekizli) basamağı, Tablo 1.2'de listelenen değerlerden kolayca hatırlanabilir.

Sekizli veya onaltılıdan ikiliye dönüşüm, önceki prosedürün tersine çevrilmesiyle yapılır. Her sekizli basamak, üç basamaklı ikili karşılığına dönüştürülür. Benzer şekilde, her onaltılı basamak, dört basamaklı ikili karşılığına dönüştürülür. Prosedür aşağıdaki örneklerde gösterilmektedir:

$$\begin{array}{ccccccc} (673.124)_8 = (110 & 111 & 011 & \cdot & 001 & 010 & 100)_2 \\ & 6 & 7 & 3 & & 1 & 2 & 4 \end{array}$$

ve

$$\begin{array}{ccccccc} (306.D)_{16} = (0011 & 0000 & 0110 & \cdot & 1101)_2 \\ & 3 & 0 & 6 & & D \end{array}$$

İkili sayılarla çalışmak zordur çünkü onlara karşılık gelen ondalık sayılardan üç veya dört kat daha fazla basamak gerektirir. Örneğin, ikili sayı 111111111111, ondalık 4095'e eşittir. Ancak, dijital bilgisayarlar ikili sayıları kullanır ve bazen insan operatörün veya kullanıcının doğrudan iletişim kurması gerekebilir.

10 Bölüm 1 Dijital Sistemler ve İkili Sayılar

makineyi bu tür sayılarla çalıştırmak. İkili sistemin bilgisayarda korunmasını sağlayan bir yöntem, ancak insanın dikkate alması gereken basamak sayısını azaltır, ikili sayı sistemi ile sekizli veya onaltılı sistem arasındaki ilişkiyi kullanır. Bu yöntemle, insan sekizli veya onaltılı sayılarla düşünür ve makine ile doğrudan iletişim gerektiğinde gerekli dönüşümü gözle yapar. Böylece, ikili sayı 111111111111 12 basamağa sahiptir ve sekizli olarak 7777 (4 basamak) veya onaltılı olarak FFF (3 basamak) şeklinde ifade edilir. İnsanlar arasında (bilgisayardaki ikili sayılar hakkında) iletişim sırasında, sekizli veya onaltılı temsil daha arzu edilir çünkü bu, eşdeğer ikili sayının gerektirdiği basamak sayısının üçte biri veya dörtte biri kadar daha kompakt bir şekilde ifade edilebilir. Böylece, çoğu

bilgisayar kılavuzu ikili miktarları belirtmek için ya sekizli ya da onaltılı sayıları kullanır. Aralarındaki seçim keyfidir, ancak onaltılı genellikle kazanır, çünkü bir baytı i ki basamakla temsil edebilir.

1.5 SAYILARIN TAMAMLAMALARI

Tamamlamalar, dijital bilgisayarlarda çıkarma işlemini basitleştirmek ve mantıksal manipülasyon için kullanılır. İşlemleri basitleştirmek, işlemleri gerçekleştirmek için daha basit ve daha az maliyetli devreler ile sonuçlanır. Her taban- r sistemi için iki tür tamamlamalar vardır: taban tamamlayıcısı ve azaltılmış taban tamamlayıcısı. İlkine taban r 'nin tamamlayıcısı denir ve ikincisine $(r-1)$'s tamamlayıcısı denir. Taban r değerinin isimde yerini almasıyla, iki tür ikili sayılar için 2'nin tamamlayıcısı ve 10'luk sayılar için 10'un tamamlayıcısı ve 9'un tamamlayıcısı olarak adlandırılır.

Azaltılmış Taban Tamamlayıcısı

Bir N sayısı r tabanında n basamağa sahip olduğunda, N 'nin $(r-1)$'s tamamlayıcısı, yani azaltılmış taban tamamlayıcısı, N 'yi tanımlamak için $(r^n-1)-N$ olarak tanımlanır. Onluk sayılar için, $r = 10$ ve $r-1 = 9$, bu nedenle N 'nin 9'un tamamlayıcısı $(10^n-1)-N$ 'dir. Bu durumda, 10^n tek bir 1'in ardından n 0'dan oluşan bir sayıyı temsil eder. 10^n-1 ise n 9 ile temsil edilen bir sayıdır. Örneğin, eğer $n = 4$ ise, $10^4 = 10,000$ ve $10^4-1 = 9999$ 'dur. Böylece, bir onluk sayının 9'un tamamlayıcısı, her basamağı 9'dan çıkarmakla elde edilir.

İşte bazı sayısal örnekler:

The 9's complement of 546700 is $999999 - 546700 = 453299$.

012398'in 9's tamamlayıcısı $999999-012398=987601$ 'dir.

İkili sayılar için, $r = 2$ ve $r-1 = 1$, bu nedenle N 'nin 1'in tamamlayıcısı $(2^n-1)-N$ 'dir. Yine, 2^n bir 1'in ardından n tane 0'dan oluşan bir ikili sayı ile temsil edilir. 2^n-1 ise n tane 1 ile temsil edilen bir ikili sayıdır. Örneğin, eğer $n = 4$ ise, $2^4 = (10000)_2$ ve $2^4-1 = (1111)_2$. Böylece, bir ikili sayının 1'in tamamlayıcısı, her basamağı 1'den çıkarmakla elde edilir. Ancak, 1'den ikili basamakları çıkardığımızda,

$1-0=1$ ya da $1-1=0$ alabiliriz, bu da bitin 0'dan 1'e veya 1'den 0'a değişmesine neden olur. Bu nedenle, bir ikili sayının 1'in tamamlayıcı, 1'leri 0'a ve 0'ları 1'e çevirerek oluşur. Aşağıda bazı sayısal örnekler verilmiştir:

1011000'in 1'in tamamlayıcı 0100111'dir.

0101101'in 1'in tamamlayıcı 1010010'dır.

$(r-1)$'s sekizli veya onaltılı sayıların tamamlayıcı, her basamağı sırasıyla 7 veya F (ondalık 15) 'ten çıkarmakla elde edilir.

Taban Tamamlayıcı

r 'nin tamamlayıcı, n basamaklı bir N sayısı için r tabanında $r-N$ olarak tanımlanır. $r \neq 0$ ve 0 için $N=0$. $(r-1)$'s tamamlayıcı ile karşılaştırıldığında, r 'nin tamamlayıcısının $(r-1)$'s tamamlayıcısına 1 eklenerek elde edildiğini not ediyoruz, çünkü $r^n - N = [(r^n - 1) - N] + 1$. Bu nedenle, ondalık 2389'un 10'luk tamamlayıcısı $7610+1=7611$ ve 9'un tamamlayıcı değerine 1 eklenerek elde edilir. İkilik 101100'ün 2'lik tamamlayıcısı $010011+1=010100$ ve 1'in tamamlayıcı değerine 1 eklenerek elde edilir.

10, 1'in ardından n 0'lar ile temsil edilen bir sayıdır, $10-N$, N 'nin 10'luk tamamlayıcısı, en az anlamlı 0'ların hepsini değiştirmeden bırakılarak, 10'dan ilk sıfırdan farklı en az anlamlı basamağı çıkartarak ve diğer tüm daha yüksek anlamlı basamakları 9'dan çıkartarak da oluşturulabilir. Böylece,

the 10's complement of 012398 is 987602

ve

246700'ün 10'luk tamamlayıcısı 753300'dür.

İlk sayının 10'luk tamamlayıcısı, en az anlamlı konumda 10'dan 8 çıkartılarak ve diğer tüm basamaklar 9'dan çıkartılarak elde edilir. İkinci sayının 10'luk tamamlayıcısı, en az anlamlı iki 0'ı değiştirmeden bırakılarak, 10'dan 7 çıkartılarak ve diğer üç basamak 9'dan çıkartılarak elde edilir.

Benzer şekilde, 2'nin tamamlayıcı, en az anlamlı 0'ları ve ilk 1'i değiştirmeden bırakıp, diğer tüm daha yüksek anlamlı basamaklarda 1'leri 0'larla ve 0'ları 1'lerle değiştirilerek oluşturulabilir. Örneğin,

the 2's complement of 1101100 is 0010100

ve

0110111'in 2'nin tamamlayıcı 1001001'dir.

İlk sayının 2'nin tamamlayıcı, en az anlamlı iki 0'ı ve ilk 1'i değiştirmeden bırakıp, diğer dört en anlamlı basamakta 1'leri 0'larla ve 0'ları 1'lerle değiştirilerek elde edilir. İkinci sayının 2'nin tamamlayıcı, en az anlamlı 1'i değiştirmeden bırakıp, diğer tüm basamakları tamamlayarak elde edilir.

12 Bölüm 1 Dijital Sistemler ve İkili Sayılar

Önceki tanımlarda, sayıların bir ondalık noktası olmadığı varsayılmıştır. Orijinal sayı N bir ondalık noktası içeriyorsa, r 'nin veya $(r-1)$'s tamamlayanın oluşturulabilmesi için nokta geçici olarak kaldırılmalıdır. Ondalık noktası, tamamlanan sayıya aynı görelî konumda geri getirilir. Ayrıca, tamamlayanın tamamlayanın sayıyı orijinal değerine geri getirdiğini belirtmek de önemlidir. Bu ilişkiyi görmek için, N sayısının r 'nin tamamlayanı $r-N$ olduğuna dikkat edin, böylece tamamlayanın tamamlayanı $r''-(r''-N)=N$ olup orijinal sayıya eşittir.

Tamamlayıcılarla Çıkarma

İlkokullarda öğretilen doğrudan çıkarma yöntemi, borç alma kavramını kullanır. Bu yöntemde, çıkarılacak sayı (minuend) rakamı, çıkaran sayı (subtrahend) rakamından daha küçük olduğunda, daha yüksek anlamlı bir pozisyondan 1 borç alırız. Bu yöntem, insanların kağıt ve kalemle çıkarma işlemi yaparken iyi çalışır. Ancak, çıkarma dijital donanımlarla uygulandığında, bu yöntem tamamlayıcıları kullanan yöntemden daha az etkilidir.

İki n basamaklı işaretsiz sayı $M-N$, taban r cinsinden aşağıdaki gibi çıkarılabilir:

1. Çıkarılacak sayı M 'yi çıkaran sayı N 'nin r tamamlayıcısına ekleyin. Matematiksel olarak, $M + (r^n - N) = M - N + r^n$.
2. Eğer $M > N$ ise, toplam bir son taşı r üretecektir, bu atılabilir; geriye kalan, sonuç $M-N$ 'dir.
3. Eğer $M < N$ ise, toplam bir son taşı üretmez ve $r^n-(N-M)$ ile eşittir, bu da r 'nin $(N-M)$ tamamlayıcısıdır. Cevabı tanıdık bir biçimde elde etmek için, toplamın r 'nin tamamlayıcısını alın ve önüne bir eksi işareti koyun.

Aşağıdaki örnekler prosedürü göstermektedir:

ÖRNEK 1.5

10'luk tamamlayıcı kullanarak, 72532– 3250 çıkarın.

$$\begin{array}{r} M = \quad 72532 \\ 10\text{'s complement of } N = + \quad 96750 \\ \text{Sum} = \quad 169282 \\ \text{Discard end carry } 10^5 = - \quad 100000 \\ \text{Answer} = \quad 69282 \end{array}$$

M beş basamağa ve N yalnızca dört basamağa sahiptir. Her iki sayının da aynı basamak sayısına sahip olması gerektiğinden, N 03250 olarak yazılır. N için 10'luk tamamlayıcı alındığında, en anlamlı konumda bir 9 oluşur. Son taşıma, MÜ N olduğunu gösterir ve bu nedenle sonuç pozitifdir.

ÖRNEK 1.6

10'luk tamamlayıcı kullanarak, $3250 - 72532$ çıkarın.

$$\begin{array}{r} M = \quad 03250 \\ 10\text{'s complement of } N = + \underline{27468} \\ \text{Sum} = \quad 30718 \end{array}$$

Son taşıma yoktur. Bu nedenle, cevap $-(10\text{'s tamamlayıcısı } 30718) = -69282$ 'dir.

$3250 < 72532$ olduğundan, sonuç negatiftir. İşlem yaptığımız için işaretli sayılarla, bu durumda işaretli bir sonuç elde etmenin gerçekten bir yolu yoktur. Tamamlayıcılarla çıkarma yaparken, son taşımanın yokluğundan ve tamamlanmış sonuçtan negatif cevabı tanırız. Kağıt ve kalemle çalışırken, cevabı tanımlı bir negatif sayıya dönüştürebiliriz, böylece tanıdık bir forma sokabiliriz.

Tamamlayıcılarla çıkarma, daha önce belirtilen prosedürü kullanarak ikili sayılarla benzer bir şekilde yapılır.

ÖRNEK 1.7

İki ikili sayı verilmiştir $X = 1010100$ ve $Y = 1000011$, çıkarma işlemini gerçekleştirin (a) $X - Y$ ve (b) $Y - X$ 2'nin tamamlayıcılarını kullanarak.

$$\begin{array}{r} \text{(a)} \quad X = \quad 1010100 \\ 2\text{'s complement of } Y = + \underline{0111101} \\ \text{Sum} = \quad 10010001 \\ \text{Discard end carry } 2^7 = - \underline{10000000} \\ \text{Answer: } X - Y = \quad 0010001 \\ \text{(b)} \quad Y = \quad 1000011 \\ 2\text{'s complement of } X = + \underline{0101100} \\ \text{Sum} = \quad 1101111 \end{array}$$

Son taşıma yoktur. Bu nedenle, cevap $Y - X = -(2\text{'s tamamlayıcısı } 1101111) = -0010001$.

İşaretsiz sayıların çıkarılması da $(r-1)$'s tamamlayıcısı ile yapılabilir. Unutmayın ki $(r-1)$'s tamamlayıcısı, r 'nin tamamlayıcısından bir eksiktir. Bu nedenle, çıkarılan sayının tamamlayıcısına minuend eklenmesi, bir son taşıma gerçekleştiğinde doğru farktan bir eksik toplam üretir. Son taşımanın çıkarılması ve toplamın 1 eklenmesi,

son taşıma olarak adlandırılır.

ÖRNEK 1.8

Örnek 1.7'yi tekrarlayın, ancak bu sefer 1'in tamamlayıcısını kullanarak.

(a)

$$X - Y = 1010100 - 1000011$$

$$\begin{array}{r} X = 1010100 \\ 1\text{'s complement of } Y = + 0111100 \\ \text{Sum} = 10010000 \\ \text{End-around carry} = + 1 \\ \text{Answer: } X - Y = 0010001 \end{array}$$

$$(b) Y - X = 1000011 - 1010100$$

$$\begin{array}{r} Y = 1000011 \\ 1\text{'s complement of } X = + 0101011 \\ \text{Sum} = 1101110 \end{array}$$

Son taşıma yoktur. Bu nedenle, cevap $Y - X = -(1\text{'s tamamlayıcısı } 1101110) = -0010001$.

Negatif sonucun, kullanılan tamamlayıcı türü olduğu için toplamın 1'in tamamlayıcısı nı olarak elde edildiğini unutmayın. Son taşıma ile birlikte uygulama, 9'un tamamlayıcı cısı ile işaretli ondalık sayıların çıkarılması için de geçerlidir.

1.6 İMZALI İKİLİ SAYILAR

Pozitif tam sayılar (sıfır dahil) işaretli sayılar olarak temsil edilebilir. Ancak, negatif tam sayıları temsil etmek için negatif değerler için bir notasyona ihtiyacımız var. Sıradan aritmetikte, negatif bir sayı eksi işareti ile ve pozitif bir sayı artı işareti ile gösterilir. Donanım sınırlamaları nedeniyle, bilgisayarlar her şeyi ikili basamaklarla temsil etmek zorundadır. Sayının en soldaki konumuna yerleştirilen bir bit ile işareti temsil etmek gelenekseldir. Gelenek, işaret bitini pozitif için 0 ve negatif için 1 yapmaktır.

Hem işaretli hem de işaretli ikili sayıların bir bilgisayarda temsil edildiğinde bir dizi bit içerdiğini anlamak önemlidir. Kullanıcı, sayının işaretli mi yoksa işaretli mi olduğunu belirler. Eğer ikili sayı işaretliyse, en soldaki bit işareti temsil eder ve geri kalan bitler sayıyı temsil eder. Eğer ikili sayının işaretli olduğu varsayılıyorsa, en soldaki bit sayının en anlamlı bitidir. Örneğin, 01001 bit dizisi 9 (işaretsiz ikili) veya +9 (işaretli ikili) olarak düşünülebilir çünkü en soldaki bit 0'dır. 11001 bit dizisi, işaretli bir sayı olarak düşünüldüğünde 25'in ikili karşılığı ve -9 olarak düşünüldüğünde işaretli bir sayı olarak 9'un ikili karşılığını temsil eder. Bu, en soldaki pozisyonun 1'in negatif bir değeri belirlemesi ve diğer dört bitin 9'u temsil etmesinden dolayıdır. Genellikle, sayının temsil türü önceden biliniyorsa, bitlerin yorumlanmasında bir karışıklık olmaz.

Son örnekteki imzalı sayıların temsili, *imzalı-büyüklik* konvansiyonu olarak adlandırılır. Bu notasyonda, sayı bir büyüklük ve bir sembol (+ veya -) ya da işareti belirten bir bit (0 veya 1) içerir. Bu, imzalı sayıların sıradan aritmetikte kullanılan temsidir. Aritmetik işlemler bir bilgisayarda gerçekleştirildiğinde, negatif sayıları temsil etmek için *imzalı-tamamlayıcı* sistemi kullanmak daha uygundur. Bu sistemde, bir negatif sayı complementi ile belirtilir. İmzalı-büyüklik sistemi bir sayıyı işaretini değiştirerek negatif hale getirirken, imzalı-tamamlayıcı sistemi bir sayıyı tamamlayıcısını alarak negatif hale getirir. Pozitif sayılar her zaman en soldaki pozisyonda 0 (artı) ile başlar, bu nedenle tamamlayıcı her zaman 1 ile başlar ve bu da negatif bir sayıyı gösterir. İmzalı-tamamlayıcı sistemi ya 1'in ya da 2'nin tamamlayıcısını kullanabilir, ancak 2'nin tamamlayıcısı en yaygın olanıdır.

Örnek olarak, sekiz bit ile ikili olarak temsil edilen 9 sayısını düşünelim. +9 en soldaki pozisyonda 0 işaret bitine sahip olarak temsil edilir, ardından 9'un ikili eşdeğeri gelir, bu da 00001001 verir. Tüm sekiz bitin bir değere sahip olması gerektiğini unutmayın; bu nedenle, işaret bitinin ardından ilk 1'e kadar 0'lar eklenir. +9'u temsil etmenin yalnızca bir yolu olmasına rağmen, -9'u sekiz bit ile temsil etmenin üç farklı yolu vardır:

imzalı-büyüklik temsili:

imzalı-1'in tamamlayıcı temsili: 11110110

imzalı-2'nin tamamlayıcı temsili: 11110111

İmzalı-büyüklikte, -9, yalnızca en soldaki sıra bitinin 0'dan 1'e değiştirilmesiyle +9'dan elde edilir. İmzalı-1'in tamamlayıcısında, -9, +9'un tüm bitlerinin, imza biti de dahil olmak üzere, tamamlanmasıyla elde edilir. -9'un imzalı-2'nin tamamlayıcı temsili, pozitif sayının 2'nin tamamlayıcısını alarak, imza biti de dahil olmak üzere elde edilir.

Tablo 1.3, üç temsildeki tüm olası dört bitli imzalı ikili sayıları listelemektedir. Eşdeğer ondalık sayı da referans için gösterilmektedir. Üç temsildeki pozitif sayılar aynı olup, en soldaki pozisyonda 0 vardır. İmzalı-2'nin tamamlayıcı sistemi, 0 için yalnızca bir temsil sunar ve bu her zaman pozitifdir. Diğer iki sistemde ya pozitif 0 ya da negatif 0 vardır; bu, sıradan aritmetikte karşılaşılmayan bir durumdur. Tüm negatif sayıların en soldaki bit pozisyonunda 1 vardır; bu, onları pozitif sayılardan ayırt etme şeklimizdir.

Dört bit ile 16 ikili sayıyı temsil edebiliriz. İmzalı-büyüklik ve 1'in tamamlayıcı temsillerinde, iki sıfır dahil olmak üzere sekiz pozitif sayı ve sekiz negatif sayı vardır. 2'nin tamamlayıcı temsili, bir sıfır dahil olmak üzere sekiz pozitif sayı ve sekiz negatif sayı vardır.

İmzalı büyüklik sistemi, sıradan aritmetikte kullanılır, ancak işaret ve büyüklüğün ayrı ayrı işlenmesi nedeniyle bilgisayar aritmetiğinde kullanıldığında rahatsız edicidir. Bu nedenle, genellikle imzalı tamamlayıcı sistemi kullanılır. 1'in tamamlayıcısı bazı zorluklar getirir ve aritmetik işlemler için nadiren kullanılır. Ancak, 1'in 0'a veya 0'ın 1'e dönüşümü, bir mantıksal tamamlayıcı işlemi ile eşdeğerdir ve bu, bir sonraki bölümde gösterilecektir. Aşağıdaki imzalı ikili aritmetik tartışması, yalnızca imzalı 2'nin tamamlayıcı temsiliyle ele alınır.

Tablo 1.3
İşaretili İkili Sayılar

Onluk	İşaretili-2'nin Tamamlayıcısı	İşaretili-1'in Tamamlayıcısı	İşaretili Büyüklik
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

Negatif sayıların imzalı 1'in tamamlayıcı sistemi için aynı prosedürler uygulanabilir; bu, işaretsiz sayılarda olduğu gibi son çevrim taşıyıcısını içermektedir.

Aritmetik Toplama

İmzalı büyüklük sisteminde iki sayının toplanması, sıradan aritmetik kurallarını takip eder. Eğer işaretler aynıysa, iki büyüklüğü toplarız ve toplamın ortak işareti verilir. Eğer işaretler farklıysa, daha küçük büyüklüğü daha büyük olandan çıkarırız ve farkın işaretini daha büyük büyüklüğe veririz. Örneğin, $(+25)+(-37) = -(37-25) = -12$ daha büyük büyüklük olan 37'den daha küçük büyüklük olan 25'i çıkararak ve sonuca 37'nin işaretini ekleyerek yapılır.

Bu, işaretlerin ve büyüklüklerin karşılaştırılmasını gerektiren bir süreçtir ve ardından ya toplama ya da çıkarma yapılır. Aynı prosedür, işaretili-büyüklik temsiliinde ikili sayılar için de geçerlidir. Buna karşılık, işaretili-tamamlayıcı sistemde sayıları toplama kuralı, bir karşılaştırma veya çıkarma gerektirmez, sadece toplama gerektirir. Prosedür çok basittir ve ikili sayılar için şu şekilde ifade edilebilir:

İşaretili ikili sayılardan iki negatif sayının, işaret bitleri de dahil olmak üzere, iki sayının toplamından elde edilen işaretili-2'nin-tamamlayıcı formunda temsil edilmesiyle elde edilir. İşaret bitinin pozisyonundan çıkan bir taşma atılır.

Toplama için sayısal örnekler aşağıda verilmiştir:

$$\begin{array}{rcl}
 + 6 & 00000110 & - 6 \quad 11111010 \\
 +13 & 00001101 & +13 \quad 00001101 \\
 +19 & 00010011 & + 7 \quad 00000111 \\
 \\
 + 6 & 00000110 & - 6 \quad 11111010 \\
 -13 & 11110011 & -13 \quad 11110011 \\
 - 7 & 11111001 & -19 \quad 11101101
 \end{array}$$

Negatif sayıların başlangıçta 2'nin tamamlayıcısı formunda olması gerektiğini ve eğer toplama işleminden sonra elde edilen toplam negatifse, bunun 2'nin tamamlayıcısı formunda olduğunu unutmayın. Örneğin, -7 , $+7$ 'nin 2'nin tamamlayıcısı olan 11111001 olarak temsil edilir.

Dört durumda da gerçekleştirilen işlem, işaret bitinin dahil olduğu toplama işlemidir. İşaret bitinin konumundan çıkan herhangi bir taşma atılır ve negatif sonuçlar otomatik olarak 2'nin tamamlayıcısı formundadır.

Doğru bir sonuç elde etmek için, sonucun toplamı karşılamak için yeterli bit sayısına sahip olduğundan emin olmalıyız. İki n bit sayısı ile başlarsak ve toplam $n+1$ bit alıyorsa, taşma meydana geldiğini söyleriz. Kağıt ve kalemle toplama işlemi yapıldığında, taşma bir sorun değildir, çünkü sayfanın genişliği ile sınırlı değiliz. Pozitif bir sayıya bir 0 veya negatif bir sayıya en anlamlı konumda bir 1 ekleyerek sayıyı $n+1$ bit'e uzatır ve ardından toplama işlemini gerçekleştiririz. Taşma, bilgisayarlarda bir sorundur çünkü bir sayıyı tutan bit sayısı sınırlıdır ve sınırlı değeri 1 aşan bir sonuç barındırılmaz.

Negatif sayıların tamamlayıcı formu, işaretli büyüklük sistemine alışkın olanlar için tanıdık değildir. İşaretli-2'nin tamamlayıcısındaki bir negatif sayının değerini belirlemek için, sayıyı daha tanıdık bir forma yerleştirmek üzere pozitif bir sayıya dönüştürmek gereklidir. Örneğin, işaretli ikili sayı 11111001 negatif bir sayıdır çünkü en soldaki bit 1'dir. 2'nin tamamlayıcısı 00000111 'dir, bu da $+7$ 'dir. Bu nedenle, orijinal negatif sayının eşit olduğunu kabul ederiz -7 .

Aritmetik Çıkarma

Negatif sayılar 2'nin tamamlayıcı formda olduğunda, iki işaretli ikili sayının çıkarılması basittir ve şu şekilde ifade edilebilir:

Çıkarma işlemi için çıkarılacak sayının (işaret bitini de dahil ederek) 2'nin tamamlayıcısını alın ve bunu çıkarılan sayıya (işaret bitini de dahil ederek) ekleyin. İşaret bitinin pozisyonundan çıkan taşma atılır.

Bu prosedür, çıkarma işleminin, çıkarılacak sayının işareti değiştirildiğinde bir toplama işlemine dönüştürülebileceği için benimsenmiştir; bu, aşağıdaki ilişki ile gösterilmektedir:

$$\begin{aligned}
 (\pm A) - (+B) &= (\pm A) + (-B); \\
 (\pm A) - (-B) &= (\pm A) + (+B).
 \end{aligned}$$

Ancak, pozitif bir sayıyı negatif bir sayıya dönüştürmek, pozitif sayının 2'nin tamamlayıcısını alarak kolayca yapılır. Terside doğrudur, çünkü tamamlayıcı

18 Bölüm 1 Dijital Sistemler ve İkili Sayılar

Negatif bir sayının tamamlayıcı formundaki çıkarılması, eşdeğer pozitif sayıyı üretir. Bunu görmek için, çıkarma işlemini düşünelim $(-6)-(-13)=+7$. Sekiz bitlik ikili sistemde, bu işlem $(11111010-11110011)$ olarak yazılır. Çıkarma işlemi, çıkarılan sayının 2'nin tamamlayıcısını alarak toplama dönüştürülür (-13) , bu da $(+13)$ olarak ifade edilir. İkili sistemde bu $11111010 + 00001101 = 100000111$. Son taşıma kaldırıldığında, doğru

cevap elde edilir: 00000111 (+7).

İkili sayılar, işaretli tamamlayıcı sistemde, işaretli sayılarla aynı temel toplama ve çıkarma kurallarıyla toplanır ve çıkarılır.

Bu nedenle, bilgisayarların her iki tür aritmetiği işlemek için yalnızca bir ortak donanım devresine ihtiyacı vardır. Bu durum, işaretli tamamlayıcı sistemin bilgisayar sistemlerinin neredeyse tüm aritmetik birimlerinde kullanılmasına yol açmıştır. Kullanıcı veya programcı, bu tür toplama veya çıkarma işlemlerinin sonuçlarını, sayıların işaretli mi yoksa işaretli mi olduğu varsayıldığına bağlı olarak farklı yorumlamalıdır.

1.7 İKİLİ KODLAR

Dijital sistemler, iki farklı değere sahip sinyaller ve iki kararlı duruma sahip devre elemanları kullanır. İkili sinyaller, ikili devre elemanları ve ikili basamaklar arasında doğrudan bir benzerlik vardır. Örneğin, n basamaklı bir ikili sayı, her biri 0 veya 1'e eşdeğer bir çıkış sinyaline sahip n ikili devre elemanı ile temsil edilebilir. Dijital sistemler yalnızca ikili sayıları değil, aynı zamanda birçok discrete bilgi öğesini de temsil eder ve işler. Bir grup miktar arasında belirgin olan herhangi bir discrete bilgi ögesi, ikili bir kodla (yani, 0'ların ve 1'lerin bir deseni) temsil edilebilir. Kodlar ikili olmalıdır çünkü günümüz teknolojisinde, yalnızca 0'ların ve 1'lerin desenlerini temsil eden ve işleyen devreler, bilgisayarlarda kullanılmak üzere ekonomik olarak üretilebilir. Ancak, ikili kodların yalnızca sembolleri değiştirdiği, temsil ettikleri bilgi öğelerinin anlamını değiştirmedeği anlaşılmalıdır. Bir bilgisayarın bitlerini rastgele incelediğimizde, çoğu zaman bunların ikili sayılar yerine bazı türde kodlanmış bilgileri temsil ettiğini göreceğiz.

Bir n -bit ikili kod, 1'lerin ve 0'ların 2'ye kadar farklı kombinasyonunu varsayan n bit in bir grubudur; her kombinasyon, kodlanan kümenin bir öğesini temsil eder. Dört öğeden oluşan bir küme, iki bit ile kodlanabilir; her öğeye aşağıdaki bit kombinasyonlarından biri atanır: 00, 01, 10, 11. Sekiz öğeden oluşan bir küme, üç bitlik bir koda ihtiyaç duyar ve 16 öğeden oluşan bir küme, dört bitlik bir koda ihtiyaç duyar. Bir n -bit kodunun bit kombinasyonu, 0'dan $2^n - 1$ 'e kadar ikili olarak sayılarla belirlenir. Her öğeye benzersiz bir ikili bit kombinasyonu atanmalı ve iki öğe aynı değere sahip olmamalıdır; aksi takdirde, kod ataması belirsiz olacaktır.

Her ne kadar 2 farklı miktarı kodlamak için gereken minimum bit sayısı n ise de, binary kod için kullanılabilecek maksimum bit sayısı yoktur. Örneğin, 10 ondalık basamak 10 bit ile kodlanabilir ve her ondalık basamağa dokuz 0 ve bir 1 içeren bir bit kombinasyonu atanabilir. Bu özel binary kodda, 6 rakamı 0001000000 bit kombinasyonuna atanmıştır.

İkili Kodlu Ondalık Kod

İkili sayı sistemi, günümüz elektronik teknolojisinde kolayca temsil edilebildiği için bir bilgisayar için en doğal sistem olmasına rağmen, çoğu insan ondalık sisteme daha alışkındır. Bu farkı çözenin bir yolu, ondalık sayıları ikiliye dönüştürmek, tüm aritmetik hesaplamaları ikili olarak yapmak ve ardından ikili sonuçları tekrar ondalığa dönüştürmektir.

Bu yöntem, ondalık sayıların bilgisayarda saklanmasını gerektirir, böylece ikiliye dönüşürülebilirler. Bilgisayar yalnızca ikili değerleri kabul edebildiğinden, ondalık basamakları 1'ler ve 0'lar içeren bir kod aracılığıyla temsil etmemiz gerekir. Ayrıca, ondalık sayılar kodlu formda bilgisayarda saklandığında aritmetik işlemleri doğrudan ondalık sayılar üzerinde gerçekleştirmek de mümkündür.

Bir ikili kod, kümedeki eleman sayısı 2'nin bir katı değilse, bazı atanmamış bit kombinasyonlarına sahip olacaktır. 10 ondalık basamak böyle bir küme oluşturur. 10 elemanı ayırt eden bir ikili kod en az dört bit içermelidir, ancak 16 olası kombinasyondan 6'sı atanmış durumda değildir. Dört biti 10 farklı kombinasyona düzenleyerek farklı ikili kodlar elde edilebilir. Ondalık basamaklar için en yaygın olarak kullanılan kod, Tablo 1.4'te listelenen düz ikili atamadır. Bu şemaya ikili kodlu ondalık (binary-coded decimal) denir ve genellikle BCD olarak anılır. Diğer ondalık kodlar da mümkündür ve bunlardan bazıları bu bölümde daha sonra sunulacaktır.

Tablo 1.4, bir ondalık basamak için dört bit kodunu vermektedir. k ondalık basamağa sahip bir sayı, BCD'de $4k$ bit gerektirecektir. Ondalık 396, BCD'de 12 bit olarak 0011 1001 0110 ile temsil edilir; her 4 bit grubu bir ondalık basamağı temsil eder. BCD'deki bir ondalık sayı, yalnızca sayı 0 ile 9 arasında olduğunda, eşdeğer ikili sayısıyla aynıdır. 10'dan büyük bir BCD sayısı, eşdeğer ikili sayısından farklı görünür, her ne kadar her ikisi de 1'ler ve 0'lar içerse de. Ayrıca, 1010 ile 1111 arasındaki ikili kombinasyonlar kullanılmaz ve BCD'de bir anlamı yoktur. Ondalık 185'i ve BCD ile ikili arasında ki karşılık gelen değerini düşünün:

$$(185)_{10} = (0001\ 1000\ 0101)_{\text{BCD}} = (10111001)_2$$

Tablo 1.4
İkili Kodlu Ondalık (BCD)

Ondalık Sembol	BCD Rakam
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

20 Bölüm 1 Dijital Sistemler ve İkili Sayılar

BCD değeri, ondalık değer karakterlerini kodlamak için 12 bit kullanır, ancak eşdeğer ikili sayı yalnızca 8 bite ihtiyaç duyar. Bir BCD sayısının temsiline, eşdeğer ikili değerinden daha fazla bite ihtiyaç duyduğu açıktır. Ancak, ondalık sayıların kullanılmasında bir avantaj vardır, çünkü bilgisayar girdi ve çıktı verileri, ondalık sistemi kullanan insanlar tarafından üretilir.

BCD sayıların ondalık sayılar olduğunu ve ikili sayılar olmadığını anlamak önemlidir, ancak temsilde bit kullanırlar. Bir ondalık sayı ile BCD arasındaki tek fark, ondalıkların 0, 1, 2, . . . , 9 sembolleri ile yazılmasıdır ve BCD sayıları 0000, 0001, 0010, . . . , 1001 ikili kodunu kullanır. Ondalık değer tam olarak aynıdır. Ondalık 10, BCD'de sekiz bit olarak 0001 0000 ile temsil edilir ve ondalık 15 ise 0001 0101 olarak temsil edilir. Eşdeğer ikili değerler 1010 ve 1111'dir ve yalnızca dört bite sahiptir.

BCD Toplama

İki ondalık rakamın BCD'de toplanmasını düşünün, önceki daha az anlamlı rakam çiftinden olası bir taşınma ile birlikte. Her rakam 9'u geçmediğinden, toplam $9+9+1=19$ 'dan büyük olamaz, burada 1 önceki bir taşınmadır. BCD rakamlarını, sanki ikili sayılar gibi topladığımızı varsayalım. O zaman ikili toplam, 0 ile 19 arasında bir sonuç üretecektir. İkili olarak, bu aralık 0000 ile 10011 arasında olacaktır, ancak BCD'de 0000 ile 1 1001 arasındadır; ilk (yani, en soldaki) 1 bir taşınma ve sonraki dört bit BCD toplamıdır. İkili toplam 1001'e eşit veya daha az olduğunda (taşınma olmadıkça), karşılık gelen BCD rakamı doğrudur. Ancak, ikili toplam 1010'a eşit veya daha büyük olduğunda, sonuç geçersiz bir BCD rakamıdır. $6=(0110)_2$ ikili toplamına eklendiğinde, doğru rakama dönüştürür ve gerektiği gibi bir taşınma da üretir. Bunun nedeni, ikili toplamın en anlamlı bit konumundaki bir taşınma ile ondalık bir taşınmanın $16-10=6$ ile farklı olmasıdır. Aşağıdaki üç BCD toplamasını düşünün:

4	0100	4	0100	8	1000
+5	+0101	+8	+1000	+9	+1001
9	1001	12	1100	17	10001
			+0110		+0110
			10010		10111

Her durumda, iki BCD rakamı, iki ikili sayıyı gibi toplanır. Eğer ikili toplam 1010'a eşit veya ondan büyükse, doğru BCD toplamını ve bir taşıma elde etmek için 0110 ekleriz. İlk örnekte, toplam 9'a eşittir ve bu doğru BCD toplamıdır. İkinci örnekte, ikili toplam geçersiz bir BCD rakamı üretir. 0110 eklenmesi, doğru BCD toplamını, 0010 (yani, 2 numarasını) ve bir taşıma üretir. Üçüncü örnekte, ikili toplam bir taşıma üretir. Bu durum, toplam 16'dan büyük veya ona eşit olduğunda meydana gelir. Diğer dört bit 1001'den küçük olmasına rağmen, ikili toplam taşıma nedeniyle bir düzeltme gerektirir. 0110 ekleyerek, gerekli BCD toplamını 0111 (yani, 7 numarasını) ve bir BCD taşımasını elde ederiz.

İki n basamaklı işaretli BCD sayısının toplanması aynı prosedürü izler. BCD'de $184+576=760$ eklenmesini düşünün:

BCD	1	1		
	0001	1000	0100	184
	+0101	0111	0110	+576
İkili toplam	0111	10000	1010	
6 ekle		0110	0110	
BCD toplamı	0111	0110	0000	----

İlk, en az anlamlı BCD rakam çiftinin BCD rakam toplamı 0000 üretir ve bir sonraki rakam çiftine taşıma sağlar. İkinci BCD rakam çifti ve önceki bir taşıma rakam toplamı 0110 üretir ve bir sonraki rakam çifti için taşıma sağlar. Üçüncü rakam çifti bir taşıma ile birlikte, 0111'lik bir ikili toplam üretir ve düzeltme gerektirmez.

Onluk Aritmetik

BCD'deki işaretli ondalık sayıların temsili, ikili sistemdeki işaretli sayıların temsiline benzer. Tanıdık işaretli büyüklük sistemini veya işaretli tamamlayıcı sistemini kullanabiliriz. Bir ondalık sayının işareti genellikle ondalık rakamların dört bitlik koduna uygun olarak dört bit ile temsil edilir. Dört 0 ile artı ve 9'un BCD karşılığı olan 1001 ile eksi belirlemek gelenekseldir.

İşaretli büyüklük sistemi bilgisayarlarda nadiren kullanılır. İşaretli tamamlayıcı sistemi ya 9'un ya da 10'un tamamlayıcısı olabilir, ancak en sık kullanılan 10'un tamamlayıcısıdır. Bir BCD sayısının 10'un tamamlayıcısını elde etmek için önce 9'un tamamlayıcısını alırsınız ve ardından en az anlamlı rakama 1 eklersiniz. 9'un tamamlayıcısı, her rakamın 9'dan çıkarılmasıyla hesaplanır.

İmza-2'nin tamamlayıcı sistemi için geliştirilen prosedürler, ondalık sayılar için imza-a-10'un tamamlayıcı sistemi için de geçerlidir. Toplama, işaret rakamı da dahil olmak üzere tüm rakamların toplanması ve son taşımanın atılmasıyla yapılır. Bu işlem, tüm negatif sayıların 10'un tamamlayıcı formunda olduğunu varsayar. İmza-tamamlayıcı sistemde yapılan toplama işlemini düşünün: $(+375)+(-240)=+135$,

$$\begin{array}{r} 0 \ 375 \\ +9 \ 760 \\ \hline 0 \ 135 \end{array}$$

İkinci sayının en soldaki 9, bir eksiği temsil eder ve 9760, 0240'ın 10'un tamamlayıcısıdır. İki sayı toplanır ve son taşıma atılarak $+135$ elde edilir. Elbette, bilgi sayıdaki ondalık sayılar, işaret rakamları da dahil olmak üzere, BCD formatında olmalıdır. Toplama, daha önce açıklandığı gibi BCD rakamlarıyla yapılır.

Ondalık sayıların çıkarılması, ister işaretli ister imza-10'un tamamlayıcı sisteminde olsun, ikili duruma aynıdır: Çıkarılacak sayının 10'un tamamlayıcısını alın ve bunu çıkaran sayıya ekleyin. Birçok bilgisayar, BCD formatında ondalık sayılarla doğrudan aritmetik hesaplamalar yapmak için özel donanıma sahiptir.

22 Bölüm 1 Dijital Sistemler ve İkili Sayılar

Bilgisayar kullanıcısı, ondalık sayılarla aritmetik işlemi doğrudan gerçekleştirmek için programlanmış talimatları belirtebilir; bu, sayıları ikili formata dönüştürmek zorunda kalmadan yapılabilir.

Diğer Ondalık Kodlar

Ondalık rakamlar için ikili kodlar, her rakam için en az dört bit gerektirir. Dört biti 10 farklı kombinasyonla düzenleyerek birçok farklı kod oluşturulabilir. BCD ve üç diğer temsilci kod, Tablo 1.5'te gösterilmiştir. Her kod, dört bit ile düzenlenebilecek 16 olası bit kombinasyonundan yalnızca 10'unu kullanır. Diğer altı kullanılmayan kombinasyonun bir anlamı yoktur ve bunlardan kaçınılmalıdır.

BCD ve 2421 kodu, ağırlıklı kod örnekleridir. Ağırlıklı bir kodda, her bit pozisyonuna bir ağırlık faktörü atanır, böylece her rakam, kodlanmış kombinasyondaki tüm 1'lerin ağırlıklarını toplayarak değerlendirilebilir. BCD kodunun 8, 4, 2 ve 1 ağırlıkları vardır; bu, her bitin iki üssü değerlerine karşılık gelir. Örneğin, bit ataması 0110 , ağırlıklar tarafından ondalık 6'yı temsil edecek şekilde yorumlanır çünkü $8 \times 0 + 4 \times 1 + 2 \times 1 + 1 \times 0 = 6$. Ağırlıklı olarak

ilgili rakamlar 2421 ile 1101 bit kombinasyonu, ondalık karşılığı $2 \times 1 + 4 \times 1 + 2 \times 0 + 1 \times 1 = 7$ 'dir.

2421 kodunda bazı rakamların iki olası şekilde kodlanabileceğini unutmayın. Örneğin, ondalık 4, 0100 veya 1010 bit kombinasyonuna atanabilir, çünkü her iki kombinasyon toplamda 4 ağırlığına sahiptir.

Tablo 1.5
Onluk Basamaklar için Dört Farklı İkili Kod

Onluk Basamak	BCD			8, 4, 2, 1
0	0000	0000	0011	0000
1	0001	0001	0100	0001
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111
1010		0101	0000	0001
Kullanılmayan bit	1011	0110	0001	0010
	1100	0111	0010	0011
				1100
nations	1110	1001	1110	1101
	1111	1010	1111	1110

BCD toplayıcıları, sayıları ikiliye dönüştürmeden, BCD değerlerini doğrudan, basamak basamak toplar. Ancak, sonuç 9'dan büyükse, sonuca 6 eklemek gereklidir. BCD toplayıcıları, önemli ölçüde daha fazla donanım gerektirir ve artık geleneksel ikili toplayıcılara göre hız avantajına sahip değildir [5].

2421 ve fazla-3 kodları, kendiliğinden tamamlayıcı kodların örnekleridir. Bu tür kodlar, bir ondalık sayının 9'lu tamamlayıcısının, 1'leri 0'lara ve 0'ları 1'lere değiştirerek (yani, desenin her bitini tamamlayarak) doğrudan elde edildiği özelliğine sahiptir. Örneğin, ondalık 395, fazla-3 kodunda 0110 1100 1000 olarak temsil edilir. 604'ün 9'lu tamamlayıcısı, kodun her bitini tamamlayarak (ikili sayıların 1'li tamamlayıcısında olduğu gibi) elde edilen 1001 0011 0111 olarak temsil edilir.

Fazla-3 kodu, kendiliğinden tamamlayıcı özelliği nedeniyle bazı eski bilgisayarlar da kullanılmıştır. Fazla-3, her kodlanmış kombinasyonun karşılık gelen ikili değere 3 eklenerek elde edildiği ağırlıksız bir koddur. BCD kodunun kendiliğinden tamamlayıcı olmadığını unutmayın.

8, 4, -2, -1 kodu, hem pozitif hem de negatif ağırlıkların bir ondalık koda atanmasına bir örnektir. Bu durumda, bit kombinasyonu 0110 ondalık 2 olarak yorumlanır ve $8 \times 0 + 4 \times 1 + (-2) \times 1 + (-1) \times 0 = 2$ olarak hesaplanır.

Gri Kod

Birçok fiziksel sistemin çıkış verileri sürekli miktarlardır. Bu veriler, dijital bir sisteme uygulanmadan önce dijital forma dönüştürülmelidir.

Sürekli veya analog bilgiler, bir analogdan dijitale dönüştürücü aracılığıyla dijital forma dönüştürülür. Bazen, analog verilerden dönüştürülen dijital verileri temsil etmek için Tablo 1.6'da gösterilen Gri kodunu kullanmak uygun olabilir. Gri kodunun düz ikili sayı dizisine göre avantajı, bir sayıdan diğerine geçerken kod grubundaki yalnızca bir bitin değişmesidir. Örneğin, 7'den 8'e geçerken, Gri kodu 0100'den 1100'e değişir. Sadece ilk bit değişir, 0'dan 1'e; diğer üç bit aynı kalır. Buna karşılık, ikili sayılarda 7'den 8'e geçiş 0111'den 1000'e olacak ve bu da dört bitin değerinin değişmesine neden olacaktır.

Gri kod, donanım tarafından üretilen ikili sayıların normal dizisinin bir sayıdan diğerine geçiş sırasında hata veya belirsizlik üretebileceği uygulamalarda kullanılır. İkili sayılar kullanıldığında, örneğin, 0111'den 1000'e bir değişiklik, en sağdaki bitin değerinin diğer üç bitin değerlerinden daha uzun sürede değişmesi durumunda ara bir hatalı sayı olan 1001'i üretebilir. Bu, bilgiyi kullanan makine için ciddi sonuçlar doğurabilir. Gri kod, iki sayı arasındaki herhangi bir geçiş sırasında yalnızca bir bitin değerini değiştirdiğinden bu sorunu ortadan kaldırır.

Gray kodunun tipik bir uygulaması, bir şaftın açısız pozisyonundaki sürekli değişimle analog verilerin temsildir. Şaft segmentlere ayrılmıştır ve her segmentte bir numara atanmıştır. Eğer bitişik segmentler Gray kodu dizisi ile eşleştirilirse, şaftın açısı ile sensör tarafından kodlanan değer arasında belirsizlik ortadan kaldırılır.

Tablo 1.6
Gri Kod

Gray Kod	Ondalık Eşdeğer
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

ASCII Karakter Kodu

Dijital bilgisayarların birçok uygulaması yalnızca sayıların değil, aynı zamanda alfabe-deki harfler gibi diğer karakterlerin veya sembollerin de işlenmesini gerektirir. Örneğin, binlerce çalışanı olan yüksek teknolojiye sahip bir şirketi düşünün. İsimleri ve diğer ilgili bilgileri temsil etmek için, alfabedeki harfler için bir ikili kod oluşturmak gereklidir. Ayrıca, aynı ikili kodun sayıları ve özel karakterleri (örneğin \$) temsil etmesi gerekir. Alfano-merik bir karakter seti, 10 ondalık rakam, 26 alfabedeki harfi ve bir dizi özel karakteri içeren bir elemanlar kümesidir. Böyle bir küme yalnızca büyük harfler dahil edilirse 36 ile 64 eleman arasında, hem büyük hem de küçük harfler dahil edilirse 64 ile 128 eleman arasında içerir. İlk durumda, altı bitlik bir ikili koda, ikinci durumda ise yedi bitlik bir ikili koda ihtiyacımız vardır.

Alfano-merik karakterler için standart ikili kod, 128 karakteri kodlamak için yedi bit kullanan Amerikan Standart Bilgi Değişimi Kodu (ASCII) olup, bu durum Tablo 1.7'de gösterilmiştir. Kodun yedi biti, en anlamlı bit olan b_7 'den b_1 'e kadar tanımlanmıştır. Örneğin, *A* harfi ASCII'de şu şekilde temsil edilir: 1000001 (sütun 100, satır 0001). ASCII kodu ayrıca 94 grafik karakter içerir yazdırılabilen ve çeşitli kontrol işlevleri için kullanılan 34 yazdırılmayan karakter. Grafik karakterler, 26 büyük harf (A'dan Z'ye), 26 küçük harf (a'dan z'ye), 10 rakam (0'dan 9'a) ve %, *, ve \$ gibi 32 özel yazdırılabilir karakterden oluşur.

Tablo 1.7
Amerikan Standart Bilgi Değişim Kodu (ASCII)

$b_4b_3b_2b_1$	$b_7b_6b_5$							111
	-	-	-	-	-	-	-	
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	“	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	‘	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	—	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	EVET	ABD	/	?	VE	—	o	DEL

Kontrol Karakterleri

NUL	Null	DLE	Veri bağlantısı kaçıışı
SOH	Başlık başlangıcı	DC1	Cihaz kontrolü 1
STX	Metin başlangıcı	DC2	Cihaz kontrolü 2
ETX	Metin sonu	DC3	Aygıt kontrolü 3
EOT	İletim sonu	DC4	Aygıt kontrolü 4
ENQ	Sorgulama	NAK	Olumsuz onay
ACK	Onayla	SYN	Eşzamanlı boşta
BEL	Çan	ETB	İletim sonu bloğu
BS	Geri alma	CAN	İptal
HT	Yatay sekme	EM	Ortam sonu
LF	Satır beslemesi	SUB	Yerine geçme
VT	Dikey sekme	ESC	Kaçış
FF	Form beslemesi	FS	Dosya ayırıcı
CR	Taşıma dönüşü	GS	Grup ayırıcı
SO	Shift out	RS	Kayıt ayırıcı
SI	İçeri aktarma	ABD	Birim ayırıcı
SP	Boşluk	DEL	Sil

34 kontrol karakteri, ASCII tablosunda kısaltılmış adlarla belirtilmiştir. Bunlar, işlevsel adlarıyla birlikte tablonun altında tekrar listelenmiştir. Kontrol karakterleri, verileri yönlendirmek ve basılı metni belirli bir formatta düzenlemek için kullanılır. Üç tür kontrol karakteri vardır: format etkileyicileri, bilgi ayırıcıları ve iletişim kontrol karakterleri.

26 Bölüm 1 Dijital Sistemler ve İkili Sayılar

Format etkileyicileri, yazdırma düzenini kontrol eden karakterlerdir. Bunlar, geri boşluk (BS), yatay tabülasyon (HT) ve satır sonu (CR) gibi tanınmış kelime işlemci ve daktilo kontrollerini içerir. Bilgi ayırıcıları, verileri paragraflar ve sayfalar gibi bölümlere ayırmak için kullanılır. Kayıt ayırıcı (RS) ve dosya ayırıcı (FS) gibi karakterleri içerirler. İletişim kontrol karakterleri, metinlerin uzak cihazlar arasında iletimi sırasında faydalıdır, böylece aynı iletişim kanalı üzerinden önce ve sonra gelen diğer mesajlardan ayırt edilebilir. İletişim kontrol karakterlerine örnekler, bir iletişim kanalı üzerinden iletilen bir metin mesajını çerçevelemek için kullanılan STX (metin başlangıcı) ve ETX (metin sonu) gibi karakterlerdir.

ASCII, yedi bitlik bir koddur, ancak çoğu bilgisayar, bir bayt olarak adlandırılan tek bir birim olarak sekiz bitlik bir miktarı işler. Bu nedenle, ASCII karakterleri genellikle bir bayt başına bir tane saklanır. Ekstra bit, uygulamaya bağlı olarak bazen başka amaçlar için kullanılır. Örneğin, bazı yazıcılar, en anlamlı bitin 0 olarak ayarlandığı sekiz bitlik ASCII karakterlerini tanıır. En anlamlı bitin 1 olarak ayarlandığı ek 128 sekiz bitlik karakter, Yunan alfabesi veya italik yazı tipi gibi diğer semboller için kullanılır.

Hata Tespit Kodu

Veri iletişimi ve işleme hatalarını tespit etmek için, ASCII karakterine paritesini belirtmek için bazen sekizinci bir bit eklenir. Bir parite biti, toplam 1'lerin sayısını ya çift ya da tek yapmak için bir mesajla birlikte eklenen ekstra bir bittir. Aşağıdaki iki karakteri ve onların çift ve tek paritelerini düşünün:

Çift parite ile		Tek parite ile	
A	ASCII	=1000001 01	000001 11000001
T	ASCII	=1010100 11010100	01010100

Her durumda, çift parite için karakterde çift sayıda 1 üretmek veya tek parite için karakterde tek sayıda 1 üretmek amacıyla kodun en soldaki konumuna ekstra bir bit ekliyoruz. Genel olarak, ya çift ya da tek parite benimsenir, çift parite daha yaygındır.

Parite biti, bilgilerin bir yerden diğerine iletimi sırasında hataların tespit edilmesine yardımcı olur. Bu işlev, her karakter için gönderim ucunda bir çift parite biti üretmek gerçekleştirilir. Parite bitlerini içeren sekiz bitlik karakterler, hedeflerine iletilir. Her karakterin paritesi, alım ucunda kontrol edilir. Alınan karakterin paritesi çift değilse, iletim sırasında en az bir bitin değeri değişmiştir. Bu yöntem, iletilen her karakterde bir, üç veya herhangi bir tek hata kombinasyonunu tespit eder. Ancak, çift hata kombinasyonları tespit edilmez ve bu olasılığı ele almak için ek hata tespit kodları gerekebilir.

Bir hata tespit edildikten sonra ne yapılacağı, belirli uygulamaya bağlıdır. Bir olasılık, hatanın rastgele olduğu ve tekrar meydana gelmeyeceği varsayımıyla mesajın yeniden iletimini talep etmektir. Böylece, alıcı bir parite hatası tespit ederse, çift pariteye sahip sekiz bitlik 10010101 ASCII NAK (negatif onay) kontrol karakterini geri gönderir.

ata tespit edilmezse, alıcı 00000110 olan ACK (onay) kontrol karakterini geri gönderir. Gönderim ucu, doğru parite alınana kadar mesajı yeniden ileterek NA K'ye yanıt verecektir. Birkaç denemeden sonra iletim hala hatalıysa, operatöre iletim yolunda arızaları kontrol etmesi için bir mesaj gönderilebilir.

1.8 İKİLİ DEPOLAMA VE KAYITLAR

Dijital bir bilgisayardaki ikili bilgi, bireysel bitleri depolamak için bir ortamda fiziksel bir varlığa sahip olmalıdır. Bir *ikili hücre*, iki kararlı duruma sahip olan ve bir bit (0 veya 1) bilgi depolayabilen bir cihazdır. Hücreye giren sinyaller, onu iki durumdan birine ayarlamak için uyarı sinyalleri alır. Hücrenin çıkışı, iki durum arasında ayırım yapan fiziksel bir miktardır. Bir hücrede depolanan bilgi, hücre bir kararlı durumda olduğunda 1 ve hücre diğer kararlı durumda olduğunda 0'dır.

Kayıtlar

Bir *kayıt*, ikili hücrelerin bir grubudur. n hücresine sahip bir kayıt, n bit içeren herhangi bir ayrık bilgi miktarını depolayabilir. Bir kaydın durumu, 1'lerin ve 0'ların n -demetidir; her bit, kayıttaki bir hücrenin durumunu belirtir. Bir kaydın içeriği, içinde depolanan bilgilere verilen yorumun bir fonksiyonudur. Örneğin, aşağıdaki ikili içeriğe sahip 16 bitlik bir kaydı düşünelim:

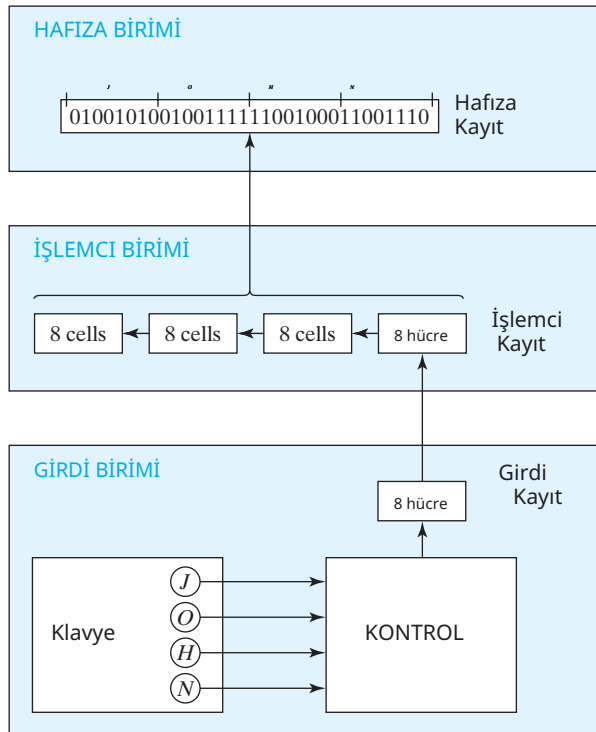
1100001111001001

16 hücreye sahip bir kayıt, 2^{16} olası durumdan birinde olabilir. Kayıt içeriğinin bir ikili tam sayıyı temsil ettiğini varsayarsak, kayıt 0 ile $2^{16} - 1$ arasında herhangi bir ikili sayıyı depolayabilir. Gösterilen özel örnekte, kaydın içeriği ondalık sayı 50,121'in ikili karşılığıdır. Bunun yerine kaydın, sekiz bitlik bir kodun alfasayısal karakterlerini depoladığını varsayarsak, kaydın içeriği iki anlamlı karakterdir. Sekiz en anlamlı bit pozisyonuna yerleştirilen çift pariteye sahip ASCII kodu için, kayıt iki karakter içerir: C (en soldaki sekiz bit) ve I (en sağdaki sekiz bit). Ancak, kaydın içeriği dört bitlik bir kodla temsil edilen dört ondalık basamak olarak yorumlanırsa, kaydın içeriği dört basamaklı bir ondalık sayıdır. Excess-3 kodunda, kayıt 9,096 ondalık sayısını tutar. Kayıt içeriği BCD'de anlamsızdır, çünkü 1100 bit kombinasyonu herhangi bir ondalık basamağa atanmaz. Bu örnekten, bir kaydın ayrık bilgi unsurlarını depolayabileceği ve aynı bit yapılandırmasının, uygulamaya bağlı olarak farklı veri türleri için farklı yorumlanabileceği açıktır.

Kayıt Transferi

Bir dijital sistem, kayıtları ve veri işleme gerçekleştiren bileşenleri ile karakterize edilir. Dijital sistemlerde, bir *kayıt transferi* işlemi, bir grup kayıttan diğer bir grup kayda ikili bilginin transferini içeren temel bir işlemdir. Transfer doğrudan, bir kayıttan diğerine olabilir veya bir işlem gerçekleştirmek için veri işleme devrelerinden geçebilir. Şekil 1.1, kayıtlar arasındaki bilgi transferini gösterir ve bir klavyeden bellek birimindeki bir kayda ikili bilginin transferini resimsel olarak gösterir. Giriş biriminin bir klavye, bir kontrol devresi ve bir giriş kaydı olduğu varsayılmaktadır. Her seferinde bir tuşa basıldığında, kontrol devresi, giriş kaydına eşdeğer sekiz bitlik alfanümerik karakter kodunu girer.

Kullanılan kodun, tek parite bitine sahip ASCII kodu olduğunu varsayacağız. Giriş kaydından alınan bilgi, bir işlemci kaydının sekiz en az anlamlı hücresine aktarılır. Her transferden sonra, kontrolün klavye tekrar basıldığında yeni bir sekiz bitlik kodu eklemesine olanak tanımak için giriş kaydı temizlenir. İşlemci kaydına aktarılan her sekiz bitlik karakter, önceki karakterin solundaki bir sonraki sekiz hücreye kaydırılmasıyla başlar. Dört karakterin transferi tamamlandığında, işlemci kaydı doludur ve içeriği bir bellek kaydına aktarılır. Depolanan içerik

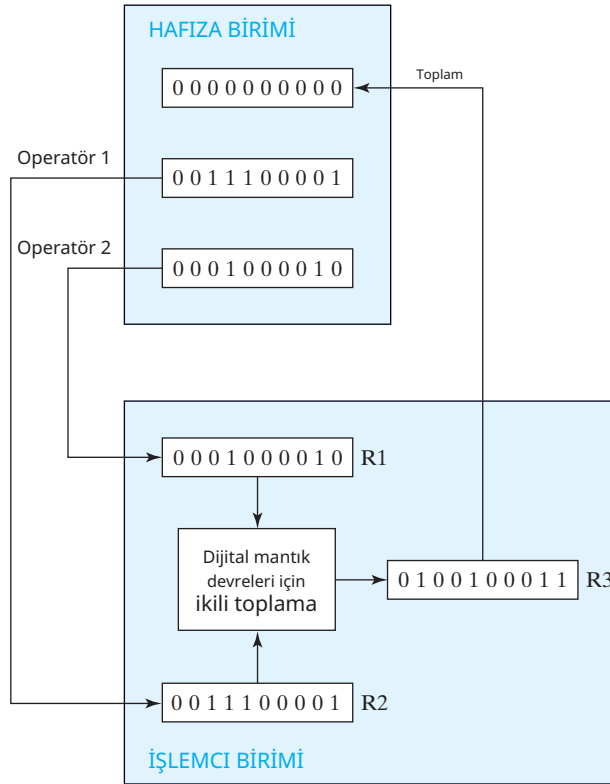


ŞEKİL 1.1
Kayıtlar arasında bilgi transferi

1'de gösterilen bellek kaydı, dört uygun tuşa basıldıktan sonra "J," "O," "H," ve "N" karakterlerinin transferinden gelmektedir.

İkili biçimde ayrıık bilgi miktarlarını işlemek için bir bilgisayar, işlenecek verileri tutan cihazlar ve bireysel bilgi bitlerini manipüle eden devre elemanları ile donatılmalıdır. Verileri tutmak için en yaygın olarak kullanılan cihaz bir kayıttır. İkili değişkenler dijital mantık devreleri aracılığıyla işlenir.

Şekil 1.2, iki 10 bitlik ikili sayının toplama sürecini göstermektedir. Normalde milyonlarca kayıttan oluşan bellek birimi, yalnızca üç kaydı ile gösterilmektedir. Gösterilen işlemci biriminin kısmı, R1, R2 ve R3 olmak üzere üç kayıttan oluşmaktadır—ve R1 ve R2 bitlerini manipüle eden dijital mantık devreleri ile birlikte, R3'e aritmetik toplamalarına eşit bir ikili sayı transfer edilmektedir. Bellek kayıtları bilgi depolar ve iki operatörü işleme yeteneğine sahip değildir. Ancak, bellekte depolanan bilgi işlemci kayıtlarına transfer edilebilir ve işlemci kayıtlarında elde edilen sonuçlar, tekrar ihtiyaç duyulana kadar depolama için bir bellek kaydına geri transfer edilebilir. Şemada, iki bellek kaydından aktarılan iki operanın içeriği gösterilmektedir



ŞEKİL 1.2
İkili bilgi işleme örneği

R1 ve R2. Dijital mantık devreleri toplamı üretir, bu da kayıt R3. R3 içeriği artık bellek kayıtlarından birine geri aktarılabilir.

Son iki örnek, bir dijital sistemin bilgi akış yeteneklerini basit bir şekilde göstermiştir. Sistemin kayıtları, ikili bilgiyi depolamak ve tutmak için temel unsurlardır. Dijital mantık devreleri, kayıtlar içinde depolanan ikili bilgiyi işler. Dijital mantık devreleri ve kayıtlar, Bölüm 2 ile 6 arasında ele alınmaktadır. Bellek birimi, Bölüm 7 'de açıklanmaktadır. Kayıt transfer düzeyindeki kayıt işlemleri ve dijital sistemlerin tasarımı, Bölüm 8 'de ele alınmaktadır.

1.9 İKİLİ MANTIK

İkili mantık, iki ayrı değeri alabilen değişkenlerle ve mantıksal anlam taşıyan işlemlerle ilgilenir. Değişkenlerin aldığı iki değer farklı adlarla anılabilir (*doğru* ve *yanlış*, *evet* ve *hayır*, vb.), ancak amacımız için, bitler açısından düşünmek ve 1 ve 0 değerlerini atamak uygundur. Bu bölümde tanıtılan ikili mantık, Boolean cebiri adı verilen bir cebire eşdeğerdir. Boolean cebirinin resmi sunumu, Bölüm 2 'de daha ayrıntılı olarak ele alınmaktadır. Bu bölümün amacı, Boolean cebirini sezgisel bir şekilde tanıtmak ve bunu dijital mantık devreleri ve ikili sinyallerle ilişkilendirmektir.

İkili Mantığın Tanımı

İkili mantık, ikili değişkenlerden ve bir dizi mantıksal işlemlerden oluşur. Değişkenler, A, B, C, x, y, z gibi alfabedeki harflerle belirtilir ve her bir değişkenin yalnızca iki farklı değeri vardır: 1 ve 0. Üç temel mantıksal işlem vardır: VE, VEYA ve DEĞİL. Her işlem, z ile gösterilen ikili bir sonuç üretir.

VE: Bu işlem bir nokta veya bir operatörün yokluğu ile temsil edilir. Örneğin, $x \# y = z$ veya $xy = z$ "x VE y eşittir z" şeklinde okunur. Mantıksal işlem VE, $z = 1$ anlamına gelir yalnızca $x = 1$ ve $y = 1$ olduğunda; aksi takdirde $z = 0$ 'dır. (x, y ve z'nin ikili değişkenler olduğunu ve yalnızca 1 veya 0'a eşit olabileceğini unutmayın, başka bir şey olamaz.) $x \# y$ işleminin sonucu z'dir.

VEYA: Bu işlem bir artı işareti ile temsil edilir. Örneğin, $x + y = z$ "x VEYA y eşittir z" şeklinde okunur, bu da $z = 1$ anlamına gelir eğer $x = 1$ veya $y = 1$ ise ya da her ikisi de $x = 1$ ve $y = 1$ ise. Eğer hem $x = 0$ hem de $y = 0$ ise, o zaman $z = 0$ 'dır.

NOT: Bu işlem bir prim ile temsil edilir (bazen bir üst çizgi ile). Örneğin, $x = z$ (veya $x = \bar{z}$) "not xz'ye eşittir" şeklinde okunur, bu da z'nin x'nin ne olmadığını ifade eder. Başka bir deyişle, eğer $x = 1$ ise, o zaman $z = 0$ 'dır, ancak eğer $x = 0$ ise, o zaman $z = 1$ 'dir. NOT işlemi aynı zamanda tamamlayıcı işlem olarak da adlandırılır, çünkü 1'i 0'a ve 0'ı 1'e çevirir, yani 1'in tamamlanmasının sonucu 0'dır ve tersine.

İkili mantık, ikili aritmetiğe benzer ve AND ile OR işlemleri sırasıyla çarpma ve toplama ile benzerlikler taşır. Aslında,

Tablo 1.8

Mantıksal İşlemlerin Doğruluk Tabloları

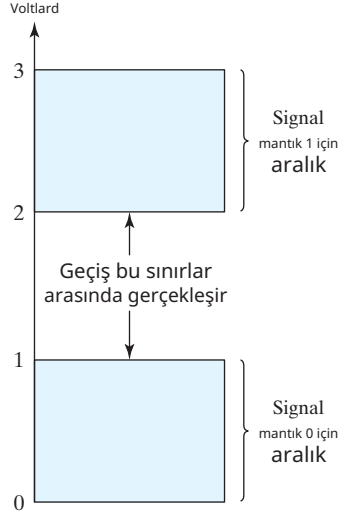
VE		VEYA		DEĞİL	
x	y	x	y	x	$\bar{x}$
0	0	0	0	0	1
0	1	0	1	1	0
1	0	1	0	1	0
1	1	1	1	1	0

AND ve OR için kullanılan semboller, çarpma ve toplama için kullanılanlarla aynıdır. Ancak, ikili mantık ikili aritmetik ile karıştırılmamalıdır. Bir aritmetik değişken, birçok basamaktan oluşabilen bir sayıyı belirtirken, bir mantık değişkeni her zaman ya 1 ya da 0'dır. Örneğin, ikili aritmetikte, $1+1=10$ ("bir artı bir ikiye eşittir" şeklinde okunur) iken, ikili mantıkta, $1+1=1$ ("bir veya bir bir'e eşittir" şeklinde okunur).

Her bir x ve y değer kombinasyonu için, mantıksal işlemin tanımı tarafından belirlenen bir z değeri vardır. Mantıksal işlemlerin tanımları, doğru tabloları olarak adlandırılan kompakt bir biçimde listelenebilir. Doğru tablosu, değişkenlerin alabileceği tüm olası kombinasyonların bir tablosudur ve değişkenlerin alabileceği değerler ile işlemin sonucu arasındaki ilişkiyi gösterir. AND ve OR işlemleri için x ve y değişkenleri ile doğru tablolar, değişkenlerin çiftler halinde birleştirildiğinde alabileceği tüm olası değerler listelenerek elde edilir. Her kombinasyon için, işlemin sonucu ayrı bir satıra listelenir. AND, OR ve NOT işlemleri için doğru tablolar, Tablo 1.8'de verilmiştir. Bu tablolar, işlemlerin tanımını açıkça göstermektedir.

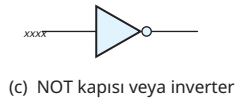
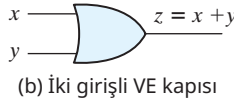
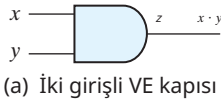
Mantık Kapıları

Mantık kapıları, bir veya daha fazla giriş sinyali üzerinde çalışan elektronik devrelerdir ve bir çıkış sinyali üretirler. Voltajlar veya akımlar gibi elektrik sinyalleri, belirli bir sürekli aralıkta, örneğin 0 ile 3 V arasında değerler alabilen analog sinyaller olarak mevcuttur, ancak dijital bir sistemde bu voltajlar, 0 veya 1 gibi iki tanınabilir değerden biri olarak yorumlanır. Voltaja dayalı mantık devreleri, mantık 1 veya mantık 0'a eşit olan bir ikili değişkeni temsil eden iki ayrı voltaj seviyesine yanıt verir. Örneğin, belirli bir dijital sistem, mantık 0'ı 0 V'a eşit bir sinyal olarak ve mantık 1'i 3 V'a eşit bir sinyal olarak tanımlayabilir. Pratikte, her voltaj seviyesinin kabul edilebilir bir aralığı vardır, bu Fig. 1.3'te gösterilmiştir. Dijital devrelerin giriş terminalleri, izin verilen aralık içindeki ikili sinyalleri kabul eder ve çıkış terminallerinde belirtilen aralık içinde kalan ikili sinyallerle yanıt verir. İzin verilen bölgeler arasındaki ara bölge, yalnızca bir durum geçişi sırasında geçerli. Hesaplama veya kontrol için istenen herhangi bir bilgi, ikili sinyalleri çeşitli mantık kapıları kombinasyonlarından geçirerek işlenebilir; her sinyal belirli bir ikili değişkeni temsil eder. Fiziksel sinyal belirli bir aralıkta olduğunda, ya 0 ya da 1 olarak yorumlanır.



ŞEKİL 1.3

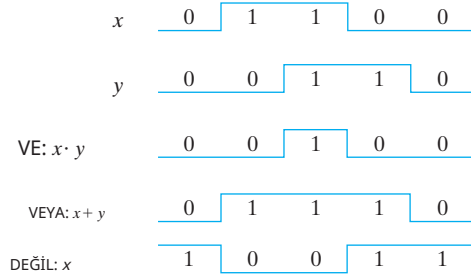
İkili mantık değerleri için sinyal seviyeleri



ŞEKİL 1.4

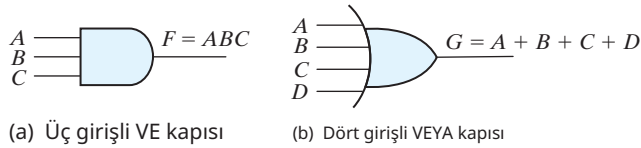
Sayısal mantık devreleri için semboller

Üç tür kapıyı belirtmek için kullanılan grafik semboller Şekil 1.4'te gösterilmiştir. Kapılar, giriş mantık gereksinimleri karşılandığında mantık-1 veya mantık-0 çıkış sinyalleri üreten donanım bloklarıdır. AND ve OR kapılarındaki giriş sinyalleri x ve y dört olası durumdan birinde bulunabilir: 00, 10, 11 veya 01. Bu giriş sinyalleri, her kapı için karşılık gelen çıkış sinyali ile birlikte Şekil 1.5'te gösterilmektedir. Zaman diyagramları, her kapının dört giriş sinyali kombinasyonuna idealize edilmiş yanıtını göstermektedir. Zaman diyagramının yatay eksenı zamanı, dikey eksenı ise iki olası voltaj seviyesi arasında değişen sinyali temsil eder. Gerçekte, mantık değerleri arasındaki geçişler hızlı bir şekilde gerçekleşir, ancak anında değildir. Düşük seviye mantık 0'ı, yüksek seviye mantık 1'i temsil eder. AND kapısı, her iki giriş sinyali mantık 1 olduğunda mantık 1 çıkış sinyali ile yanıt verir. OR kapısı, herhangi bir giriş sinyali mantık 1 ise mantık 1 çıkış sinyali ile yanıt verir. NOT kapısı genellikle bir inverter olarak adlandırılır. Bu ismin nedeni, zaman diyagramındaki sinyal yanıtından açıktır; bu, çıkış sinyalinin giriş sinyalinin mantık anlamını tersine çevirdiğini gösterir.



ŞEKİL 1.5

Kapılar için giriş-çıkış sinyalleri



ŞEKİL 1.6

Birden fazla girişi olan kapılar

VE ve VEYA kapıları iki girişten fazla olabilir. Üç girişli bir VE kapısı ve dört girişli bir VEYA kapısı Şekil 1.6'da gösterilmiştir. Üç girişli VE kapısı, tüm üç girişi mantık 1 olduğunda mantık 1 çıkışı verir. Herhangi bir giriş mantık 0 olduğunda çıktı mantık 0 üretir. Dört girişli VEYA kapısı, herhangi bir giriş mantık 1 olduğunda mantık 1 ile yanıt verir; çıktısı yalnızca tüm girişler mantık 0 olduğunda mantık 0 olur.

PROBLEMLER

(Soruların * ile işaretlenmiş cevapları metnin sonunda görünmektedir.)

- 16 ile 32 arasındaki sekizli ve onaltılı sayıları listeleyin. Son iki basamak için A ve B'yi kullanarak, 8 ile 28 arasındaki sayıları 12 tabanında listeleyin.
- 1.2** * 32K bayt (a), 64M bayt (b) ve 6.4G bayt (c) içeren bir sistemde toplam kaç bayt vardır?
- Aşağıdaki sayıları belirtilen tabanlarla ondalık sisteme dönüştürün:
- (a)* $(4310)_5$ (b)* $(198)_{12}$
(c) $(435)_8$ (d) $(345)_6$
- 1.4** 16 bit ile ifade edilebilecek en büyük ikili sayı nedir? Eşdeğer ondalık ve onaltılı sayılar nelerdir?
- 1.5** * Aşağıdaki işlemlerin doğru olması için her durumda sayıların tabanını belirleyin:
(a) $14/2 = 5$ (b) $54/4 = 13$ (c) $24 + 17 = 40$.
- 1.6** * İkinci dereceden denklemin $x^2 - 11x + 22 = 0$ çözümü $x = 3$ ve $x = 6$ 'dır. Sayıların basamağı nedir?

34 Bölüm 1 Dijital Sistemler ve İkili Sayılar

1.7 * Onaltılık sayı 64CD'yi ikiliye çevirin, ardından ikiliden sekizliğe çevirin.

Onluk sayı 431'i iki şekilde ikiliye çevirin: (a) doğrudan ikiliye çevirin;
(b) önce onaltılıya çevirin ve ardından onaltılıdan ikiliye çevirin. Hangi yöntem daha hızlı?

Aşağıdaki sayıları onluk olarak ifade edin:

- (a)* $(10110.0101)_2$ (b)* $(16.5)_{16}$
(c)* $(26.24)_8$ (d) $(DADA.B)_{16}$
(e) $(1010.1101)_2$

1.10 Aşağıdaki ikili sayıları onaltılı ve ondalık sayılara dönüştürün: (a) 1.10010,
(b) 110.010. (b) kısmındaki ondalık sonucun (a) kısmındakinin 4 katı olmasının nedenini açıklayın.

1.11 Aşağıdaki bölmeyi ikili olarak gerçekleştirin: $111011 \div 101$.

1.12 * Aşağıdaki sayıları ondalığa dönüştürmeden toplayın ve çarpın.

- (a) İkili sayılar 1011 ve 101.
(b) Onaltılı sayılar 2E ve 34.

1.13 Aşağıdaki dönüşüm problemlerini yapın:

- (a) Ondalık 27.315'i ikiliye dönüştürün.
(b) $2/3$ 'ün sekiz basamağa kadar ikili karşılığını hesaplayın. Ardından ikiliden ondalığa dönüştürün. Sonuç $2/3$ 'e ne kadar yakın?
(c) İçindeki ikili sonucu onaltılıya çevirin. Ardından sonucu ondalık sayıya çevirin. Cevap aynı mı?

1.14 Aşağıdaki ikili sayıların 1'ler ve 2'ler tamamlayıcılarını elde edin:

- (a) 00010000 (b) 00000000
(c) 11011010 (d) 10101010
(e) 10000101 (f) 11111111.

1.15 Aşağıdaki ondalık sayıların 9'luk ve 10'luk tamamlayıcılarını bulun:

- (a) 25.478.036 (b) 63.325.600
(c) 25.000.000 (d) 00.000.000.

1.16 (a) C3DF'nin 16'lık tamamlayıcısını bulun.

(b) C3DF'yi ikiliye çevirin.

(c) (b) sonucunun 2'lik tamamlayıcısını bulun.

(d) (c) içindeki cevabı onaltılıya çevirin ve (a) ile karşılaştırın.

1.17 Verilen işaretli sayılar üzerinde, çıkarılan sayının 10'luk tamamlayıcısını kullanarak çıkarma işlemi yapın.

Sonuç negatif olduğunda, 10'luk tamamlayıcısını bulun ve bir eksi

işareti ekleyin. Cevaplarınızı doğrulayın.

- (a) $4,637 - 2,579$ (b) $125 - 1,800$
(c) $2,043 - 4,361$ (d) $1,631 - 745$

1.18 Verilen işaretli sayılarda çıkarma işlemi yapın, çıkarılacak sayının 2'nin tamamlayıcısını kullanarak. Sonuç negatifse, 2'nin tamamlayıcısını bulun ve bir eksi işareti ekleyin.

- (a) $10011 - 10010$ (b) $100010 - 100110$
(c) $1001 - 110101$ (d) $101000 - 10101$

1.19 * Aşağıdaki ondalık sayılar işaret-büyüklik formunda gösterilmiştir: +9,286 ve +801.

Onları işaretli-10'un tamamlayıcı formuna dönüştürün ve aşağıdaki işlemleri gerçekleştirin (toplamın +10,627 olduğunu ve beş basamak ve bir işaret gerektirdiğini unutmayın).

- (a) $(+9,286) + (+801)$ (b) $(+9,286) + (-801)$
(c) $(-9,286) + (+801)$ (d) $(-9,286) + (-801)$

Ondalık +49 ve +29'u, işaretli-2'nin-tamamlama temsili kullanarak ikiliye çevirin ve sayıları karşılamak için yeterli basamak kullanın. Ardından,

$(+29) + (-49)$, $(-29) + (+49)$ ve $(-29) + (-49)$ işlemlerinin ikili karşılıklarını gerçekleştirin. Cevapları tekrar ondalık sisteme çevirin ve doğru olduklarını doğrulayın.

Eğer sayılar $(+9,742)_{10}$ ve $(+641)_{10}$ işaretli büyüklük formatındaysa, toplamaları $(+10,383)_{10}$ ve beş basamak ile bir işaret gerektirir. Sayıları işaretli-10'luk tamamlayıcı forma dönüştürün ve aşağıdaki toplamaları bulun:

- (a) $(+9,742) + (+641)$ (b) $(+9,742) + (-641)$
 (c) $(-9,742) + (+641)$ (d) $(-9,742) + (-641)$

1.22

Ondalık 6,514'ü hem BCD hem de ASCII kodlarına dönüştürün. ASCII için, solda bir çift parite e biti eklenmelidir.

791 ve 658 sayılarının işaretlessiz ondalık sayılarını BCD formatında temsil edin ve ardından toplamalarını oluşturma k için gerekli adımları gösterin.

Aşağıdaki ağırlıkları kullanarak ondalık basamaklar için ağırlıklı ikili kod formüle edin:

- (a) * 6, 3, 1, 1
 (b) 6, 4, 2, 1

6,248 ondalık sayısını (a) BCD, (b) eksik-3 kodu, (c) 2421 kodu ve

(d) 6311 kodu olarak temsil edin.

6,248 ondalık sayısının 9'lu tamamlayıcısını bulun ve bunu 2421 kodu ile ifade edin. Sonucun, CR_PROBLEM 1.25'teki (c) yanıtının 1'li tamamlayıcısı olduğunu gösterin. Bu, 2421 kodunun kendine tamamlayıcı olduğunu göstermektedir.

52 oyun kartına belirli bir düzen içinde ikili kod atayın. Minimum

bit sayısını kullanın.

"G. Boole" ifadesini ASCII'de, sekiz bitlik kod kullanarak yazın. Noktayı ve boşluğu dahil edin. Her karakterin en soldaki bitini parite biti olarak kabul edin. Her sekiz bitlik kodun tek pariteye sahip olması gerekir. (George Boole, 19. yüzyıl matematikçisidir. Boolean cebiri, bir sonraki bölümde t anıtılacak, onun adıyla anılmaktadır.)

1.29

* Aşağıdaki ASCII kodunu çözün:

1010011 1110100 1100101 1110110 0000010 10011010 110 1111 11 00010 1110011.

Aşağıda, bit desenleri sıkıştırma amacıyla onaltılıya dönüştürülmüş bir ASCII karakter dizisi bulunmaktadır: 73 F4 E5 76 E5 4A EF 62 73. Her iki basamaklı rakam çiftindeki se kiz bitten, en soldaki bir parite bitidir. Kalan bitler ASCII kodudur.

- (a) Diziyi bit formuna dönüştürün ve ASCII'yi çözün.
 (b) Kullanılan pariteyi belirleyin: tek mi yoksa çift mi?

1.31

* ASCII'de kaç tane yazdırılabilir karakter vardır? Bunlardan kaç t özel karakterdir (harf veya rakam olmayan)?

1.32

* Bir ASCII harfini büyükten küçüğe ve tersi için hangi bitin tamamlanması gerekir?

1.33

* 12 bit'lik bir kaydın durumu 100010010111'dir. Eğer bu,

- (a) BCD'de üç ondalık basamağı temsil ediyorsa, içeriği nedir?
 (b) Üç ondalık basamak, fazla-3 kodunda mı?
 (c) Üç ondalık basamak, 84-2-1 kodunda mı?
 (d) Bir ikili sayı mı?

36 Bölüm 1 Dijital Sistemler ve İkili Sayılar

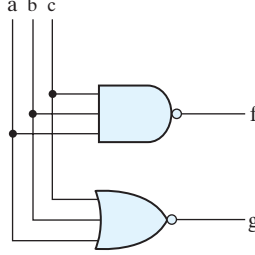
1.34

En soldaki

pozisyonda çift parite bitine sahip 10 ondalık basamağın ASCII kodunu listeleyin.

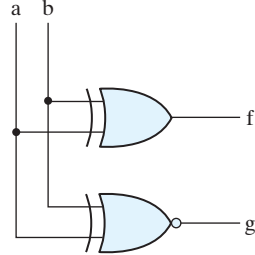
1.35

Şekil 1.5'e benzer bir zamanlama diyagramı aracılığıyla, Şekil P1.35'teki f ve g çıkışlarının sinyallerini a, b ve c üç girdi fonksiyonu olarak gösterin. a, b ve c'nin sekiz olası kombinasyonu nu kullanın.



ŞEKİL P1.35

- 1.36** Şekil 1.5'e benzer bir zamanlama diyagramı kullanarak, Fig. P1.36'daki f ve g çıkışlarının sinyallerini a ve b'nin iki girişi olarak gösterin. a ve b'nin dört olası kombinasyonunu kullanın.



ŞEKİL P1.36

REFERANSLAR

1. CAVANAGH, J. J. 1984. *Dijital Bilgisayar Aritmetiği*. New York: McGraw-Hill.
2. MANO, M. M. 1988. *Bilgisayar Mühendisliği: Donanım Tasarımı*. Englewood Cliffs, NJ: Prentice-Hall.
3. NELSON, V. P., H. T. NAGLE, J. D. IRWIN ve B. D. CARROLL. 1997. *Dijital Mantık Devresi Analizi ve Tasarımı*. Upper Saddle River, NJ: Prentice Hall.
4. SCHMID, H. 1974. *Ondalık Hesaplama*. New York: John Wiley.
5. KATZ, R. H. ve BORRIELLO, G. 2004. *Çağdaş Mantık Tasarımı*, 2. baskı. Upper Saddle River, NJ: Prentice-Hall.

AĞ ARAMA KONULARI

BCD kodu
ASCII
Depolama kaydı
İkili mantık
BCD toplama
İkili kodlar
İkili sayılar
Excess-3 kodu

Bölüm 2

Boolean Cebiri ve Mantık Kapıları

2.1 GİRİŞ

İkili mantık, günümüz dijital bilgisayarları ve cihazlarının hepsinde kullanıldığı için, bunu uygulayan devrelerin maliyeti, bilgisayar mühendisleri, elektrik mühendisleri veya bilgisayar bilimcileri olsun, tasarımcılar tarafından ele alınan önemli bir faktördür. Bir devrenin daha basit ve daha ucuz, ancak eşdeğer, gerçekleştirimlerini bulmak, tasarımın toplam maliyetini azaltmada büyük kazançlar sağlayabilir. Devreleri basitleştiren matematiksel yöntemler esasen Boolean cebirine dayanır. Bu nedenle, bu bölüm, basit devreleri optimize etmenizi ve milyonlarca mantık kapısını içeren karmaşık devreleri optimize etmek için yazılım araçları tarafından kullanılan algoritmaların amacını anlamanızı sağlayacak temel bir kelime dağarcığı ve kısa bir Boolean cebiri temeli sunmaktadır.

2.2 TEMEL TANIMLAR

Boolean cebiri, diğer herhangi bir çıkarımsal matematiksel sistem gibi, bir elemanlar kümesi, bir işlemciler kümesi ve bir dizi kanıtlanmamış aksiyom veya postülat ile tanımlanabilir. Elemanlar kümesi, genellikle ortak bir özelliğe sahip nesnelerin herhangi bir koleksiyonudur. Eğer S bir küme ise, ve x ve y belirli nesneler ise, o zaman notasyon $x \in S$ x 'in S kümesinin bir üyesi olduğunu ve $y \notin S$ y 'nin S 'nin bir elemanı olmadığını ifade eder. Sayılabilir sayıda eleman içeren bir küme, süslü parantezler ile belirtilir: $A = \{1, 2, 3, 4\}$ kümesinin elemanlarının 1, 2, 3 ve 4 sayıları olduğunu gösterir. S kümesindeki elemanlar üzerinde tanımlı bir ikili işlemci, S 'den her eleman çiftine, S 'den benzersiz bir eleman atayan bir kuraldır. Örneğin, $a * b = c$ ilişkisini düşünelim. $*$ bir ikili işlemci olarak adlandırılır, eğer c 'yi (a, b) çiftinden bulmak için bir kural belirliyorsa ve ayrıca $a, b, c \in S$ ise. Ancak, $*$ bir ikili işlemci değildir $a, b \in S$ ise, ve eğer $c \notin S$ ise.

Bir matematik sisteminin postülatları, sistemin kurallarını, teoremlerini ve özel liklerini türetebileceğimiz temel varsayımları oluşturur. Çeşitli cebirsel yapıların formüle edilmesinde en yaygın kullanılan postülatlar şunlardır:

1. *Kapatma.* Bir küme S bir ikili işlem açısından kapalıdır eğer, S kümesinin her bir eleman çifti için, ikili işlem, S kümesinden benzersiz bir eleman elde etmek için bir kural belirliyorsa. Örneğin, doğal sayılar kümesi $N = \{1, 2, 3, 4, \dots\}$ aritmetik toplama kurallarıyla ikili işlem $+$ açısından kapalıdır, çünkü, herhangi bir $a, b \in N$, öyle bir $c \in N$ vardır ki $a+b=c$. Doğal sayılar kümesi *kapalı değildir* ikili işlem $-$ açısından aritmetik çıkarma kurallarıyla, çünkü $2-3=-1$ ve $2, 3 \in N$, ama $-1 \notin N$.

2. *Birleşme yasası.* Bir küme S üzerindeki bir ikili işlem $*$ her zaman birleşmeli olarak adlandırılır.

$$(x * y) * z = x * (y * z) \text{ for all } x, y, z, \in S$$

3. *Değişme yasası.* Bir küme S üzerindeki bir ikili işlem $*$

$$x * y = y * x \text{ for all } x, y \in S$$

4. *Kimlik elemanı.* Bir küme S bir ikili

işlem $*$ ile S üzerinde bir kimlik elemanına sahip olduğu söylenir, eğer $e \in S$ olan bir eleman varsa ve bu elemanın özelliği şudur:

$$e * x = x * e = x \text{ for every } x \in S$$

Örnek: Eleman 0, ikili operatör üzerinde kimlik elemanıdır.

+ Tam sayılar kümesi $I = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$ olduğundan,

$$x + 0 = 0 + x = x \text{ for any } x \in I$$

Doğal sayılar kümesi, N , kimlik elemanına sahip değildir, çünkü 0 kümeden hariçtir.

5. *Ters.* Bir küme S ikili operatör $*$ ile ilgili kimlik elemanı e varsa, her $x \in S$ için, $y \in S$ olan bir eleman varsa, bu kümeye t

ters bir eleman denir.

$$x * y = e$$

Örnek: Tam sayılar kümesinde, I , ve operatör $+$, $e = 0$ olduğunda, bir eleman a için ters eleman $(-a)$ dir, çünkü $a + (-a) = 0$.

6. *Dağıtım yasası.* Eğer $*$ ve $\cdot$ bir küme S üzerinde i

ki ikili operatör ise, $*$ $\cdot$ üzerinde dağıtıcıdır, eğer

$$x * (y \cdot z) = (x * y) \cdot (x * z)$$

Bir alan, cebirsel bir yapının bir örneğidir. Bir alan, elemanlardan oluşan bir küme ile birlikte, her biri 1'den 5'e kadar olan özelliklere sahip iki ikili operatör içerir ve her iki operatör de 6. özelliği sağlamak için bir araya gelir. Gerçek sayılar kümesi, ikili operatörler $+$ ve $\cdot$ ile birlikte,

40 Bölüm 2 Boolean Cebiri ve Mantık Kapıları

gerçek sayılar alanını oluşturur. Gerçek sayılar alanı, aritmetiğin ve sıradan cebirin temelidir. Operatörlerin ve postülatların aşağıdaki anlamları vardır:

İkili operatör $+$ toplama tanımlar.

Toplama kimliği 0'dır.

Toplama tersini tanımlar.

İkili operatör $\cdot$ çarpma tanımlar.

Çarpma kimliği 1'dir.

$a \neq 0, a=1/a$ çarpma tersini tanımlar (yani, $a \cdot 1/a=1$).

Geçerli olan tek dağıtım yasası $\cdot$ üzerinde $+$:

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

2.3 AXIOMATİK TANIM BOOLEAN CEBİRİ

1854 yılında, George Boole artık *Boolean cebiri* olarak adlandırılan bir cebirsel sistem geliştirdi. 1938'de, Claude E. Shannon, bistable elektrikli anahtarlama devrelerinin özelliklerini temsil eden *anahtarlama cebiri* olarak adlandırılan iki değerli bir Boolean cebiri tanıttı. Boolean cebirinin resmi tanımı için, 1904 yılında E. V. Huntington tarafından formüle edilen postülatları kullanacağız.

Boolean cebiri, bir dizi eleman, B ve iki ikili işlemci, $+$ ve $\cdot$ ile tanımlanan bir cebirsel yapıdır; aşağıdaki (Huntington) postülatlarının sağlanması koşuluyla:

- (a) Yapı, operatör $+$ açısından kapalıdır.
(b) Yapı, operatör $\cdot$ açısından kapalıdır.
- (a) Eleman 0, $+$ açısından bir kimlik elemanıdır; yani, $x+0=x$.
(b) Eleman 1, $\cdot$ açısından bir kimlik elemanıdır; yani, $x \cdot 1=x$.
- (a) Yapı, $+$ açısından komutatiftir; yani, $x+y=y+x$.
(b) Yapı, $\cdot$ açısından komutatiftir; yani, $x \cdot y=y \cdot x$.
- (a) Operatör $\cdot$, $+$ üzerinde dağıtıcıdır; yani, $x \cdot (y+z)=(x \cdot y)+(x \cdot z)$.
(b) Operatör $+$, $\cdot$ üzerinde dağıtıcıdır; yani, $x+(y \cdot z)=(x+y) \cdot (x+z)$.
- Her eleman için $x \in B$, bir eleman vardır $x \in B$ (bu eleman x 'nin *tamamlayanı* olarak adlandırılır) şöyle ki (a) $x+x=1$ ve (b) $x \cdot x=0$.
- En az iki eleman vardır $x, y \in B$ öyle ki $x \neq y$.

Boolean cebirini aritmetik ve sıradan cebir (gerçek

nsayılarla) ile karşılaştırdığımızda, aşağıdaki farklılıkları not ediyoruz:

- Huntington postülatları birleşme yasasını içermez. Ancak, bu yasa Boolean cebiri için geçerlidir ve diğer postülatlardan (her iki operatör için) türetilir.
- $+$ üzerindeki dağıtım yasası (yani, $x+(y \cdot z)=(x+y) \cdot (x+z)$) Boolean cebiri için geçerlidir, ancak sıradan cebir için geçerli değildir.

Boolean cebirinin toplama veya çarpma tersleri yoktur; bu nedenle, çıkarma veya bölme işlemleri yoktur.

Postülat 5, sıradan cebirde mevcut olmayan bir *tamamlayıcı* olarak adlandırılan bir operatörü tanımlar.

Sıradan cebir, sonsuz bir eleman kümesini oluşturan gerçek sayılarla ilgilenir. Boolean cebiri ise henüz tanımlanmamış eleman kümesi B ile ilgilenir, ancak sonraki tanımlanan iki değerli Boolean cebirinde (ve bu cebiri sonraki kullanımlarımızda ilgi çekici kılan), B yalnızca 0 ve 1'den oluşan iki elemanlı bir küme olarak tanımlanır.

Boolean cebiri bazı açılardan sıradan cebire benzer. Sembollerin $+$ ve $\cdot$ seçimi kasıtlıdır, böylece sıradan cebirle zaten tanışık olan kişilerin Boolean cebirsel işlemleri kolaylaştırılır. Sıradan cebirden bazı bilgileri Boolean cebiri ile başa çıkma için kullanmak mümkün olsa da, acemi, uygulanmadığı yerlerde sıradan cebir kurallarını değiştirmemeye dikkat etmelidir.

Bir cebirsel yapının kümesinin elemanları ile bir cebirsel sistemin değişkenleri arasında ayırım yapmak önemlidir. Örneğin, reel sayıların alanının elemanları sayılardır, oysa sıradan cebirde kullanılan a, b, c gibi değişkenler, reel sayıları temsil eden sembollerdir. Benzer şekilde, Boolean cebirinde, kümenin elemanları B olarak tanımlanır ve x, y ve z gibi değişkenler yalnızca elemanları temsil eden sembollerdir. Bu noktada, bir Boolean cebiri oluşturmak için,

- kümenin elemanları B ,
- iki ikili operatör için işlem kuralları ve
- elemanlar kümesi, B , iki operatörle birlikte, altı Huntington postülatını sağlamalıdır.

Birçok Boole cebiri formüle edilebilir, bu da B ve işletim kuralları. Sonraki çalışmalarımızda, yalnızca iki değerli Boolean cebiriyle (yani, yalnızca iki eleman içeren bir Boolean cebiri) ilgileniyoruz. İki değerli Boolean cebiri, küme teorisinde (sınıfların cebiri) ve önermeli mantıkta uygulamalara sahiptir. Buradaki ilginç, dijital cihazlar ve bilgisayarlarda yaygın olarak kullanılan kapı tipi devreler Boolean cebirinin uygulanmasıdır.

İki Değerli Boolean Cebiri

İki değerli bir Boole cebiri, iki elemanlı bir küme üzerinde tanımlanır, $B = \{0, 1\}$, iki ikili operatör için kurallarla $+$ ve $\cdot$ aşağıdaki operatör tablolarında gösterildiği gibi (tamamlayıcı operatör için kural, postülat 5'in doğrulanması içindir):

x	y	xy
0	0	0
0	1	0
1	0	0
1	1	1

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

x	x'
0	1
1	0

42 Bölüm 2 Boolean Cebiri ve Mantık Kapıları

Bu kurallar, sırasıyla, Tablo 1.8'de tanımlanan AND, OR ve NOT işlemleriyle tamam en aynıdır. Şimdi Huntington postülatlarının, küme $B = \{0, 1\}$ ve iki ikili operatör $+$ ve $\cdot$ için geçerli olduğunu göstermeliyiz.

Yapının iki operatör açısından kapalı olduğu, tablolardan açıktır, çünkü her işlemin sonucu ya 1 ya da 0 ve $1, 0 \in B$. Tabloya göre,

$$\begin{aligned} (a) 0+0 &= 0 & 0+1 &= 1+0=1; \\ (b) 1+1 &= 1 & 1+0 &= 0+1=0. \end{aligned}$$

Bu, iki *kimlik elemanını*, 0 için $+$ ve 1 için $\cdot$, postülat 2 tarafından tanımlandığı gibi kurur.

Değişmeli yasalar, ikili işlem tablolarının simetrisinden açıktır.

(a) Dağıtıcı yasa $x \cdot (y+z) = (x \cdot y) + (x \cdot z)$ işlemci tablolarından, tüm olası x, y ve z değerlerinin bir doğruluk tablosunu oluşturarak geçerli olduğu gösterilebilir. Her kombinasyon için, $x \cdot (y+z)$ türetilir ve değerini, $(x \cdot y) + (x \cdot z)$ değerine eşit olduğu gösterilir:

		z		y	z	$x \cdot (y+z)$		$x \cdot y$	$x \cdot z$	$(x \cdot y) + (x \cdot z)$
0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	1	0	0	0	0	0	0
0	1	1	0	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0
1	0	1	0	1	1	1	0	1	1	1
1	1	0	0	1	1	1	1	0	1	1
1	1	1	0	1	1	1	1	1	1	1

(b) $+$ üzerindeki # dağıtım yasaının geçerli olduğunu bir doğruluk tablosu ile gösterilebilir a (a) kısmındaki gibi.

5. Tamamlayıcı tablodan, kolayca gösterilebilir ki

$$x+x=1, \text{ çünkü } 0+0=0+1=1 \text{ ve } 1+1=1+0=1.$$

$$(b) \quad x \cdot x=0, \text{ çünkü } 0 \cdot 0=0 \cdot 1=0 \text{ ve } 1 \cdot 1=1 \cdot 0=0.$$

Böylece, postülat 1 doğrulanmıştır.

6. Postülat 6, iki değerli Boole cebirinin iki elemanı,

$$1 \text{ ve } 0, \text{ ile sağlanmaktadır. } \neq 0.$$

Az önce, iki elemanlı bir iki değerli Boole cebiri kurduk, 1 ve 0, AND ve OR işlemlerine eşdeğer kurallara sahip iki ikili operatör ve NOT operatörüne eşdeğer bir tamamlayıcı operatör. Böylece, Boole cebiri resmi matematiksel bir şekilde tanımlanmış ve Bölüm 1.9'da sezgisel olarak sunulan binary mantık ile eşdeğer olduğu gösterilmiştir. Sezgisel sunum, Boole cebirinin kapı tipi devrelere uygulanmasını anlamada yardımcıdır. Resmi

birsel sistemin teoremlerini ve özelliklerini geliştirmek için gereklidir. Bu bölümde tanımlanan iki değerli Boole cebiri mühendisler tarafından “anahtar cebiri” olarak da adlandırılmaktadır. İki değerli Boole cebiri ile diğer ikili sistemler arasındaki benzerlikleri vurgulamak için, o cebir Bölüm 1.9’da “ikili mantık” olarak adlandırılmıştır. Buradan itibaren, sonraki tartışmalarda Boole cebirinden “iki değerli” sıfatını çıkaracağız.

2.4 BOOLE CEBİRİNİN TEMEL TEOREMLERİ VE ÖZELLİKLERİ

İkilik

Bölüm 2.3’te, Huntington postülatları çiftler halinde listelenmiş ve bölüm (a) ve bölüm (b) olarak adlandırılmıştır. Bir bölüm, ikili operatörler ve kimlik elemanları yer değiştirilirse diğerinden elde edilebilir. Bu önemli özellik, Boole cebirinin *ikilik prensibi* olarak adlandırılır ve Boole cebirinin postülatlarından türetilen her cebirsel ifadenin, operatörler ve kimlik elemanları yer değiştirildiğinde geçerli kalacağını belirtir. İki değerli bir Boole cebirinde, kimlik elemanları ve kümenin elemanları B aynıdır: 1 ve 0. İkilik prensibinin birçok uygulaması vardır. Eğer bir cebirsel ifadenin *ikili* isteniyorsa, OR ve AND operatörlerini yer değiştirir ve 1’leri 0’larla, 0’ları da 1’lerle değiştiririz.

Temel Teoremler

Tablo 2.1 Boole cebirinin altı teoremini ve dört postülatını listeler. Notasyon, karışıklığa neden olmadıkça ikili operatörün atlanmasıyla basitleştirilmiştir. Listelenen teoremler ve postülatlar, Boole

Tablo 2.1
Boole Cebirinin Postülatları ve Teoremleri

Postülat 2	(a)	$x + 0 = x$	(b)	$x \cdot 1 = x$
Postulat 5	(a)	$x + x = 1$	(b)	$x \cdot x' = 0$
Teorem 1	(a)	$x + x = x$	(b)	$x \cdot x = x$
Teorem 2	(a)	$x + 1 = 1$	(b)	$x \cdot 0 = 0$
Teorem 3, involüsyon		$(x')' = x$		
Postulat 3, değişmeli	(a)	$x + y = y + x$	(b)	$xy = yx$
Teorem 4, birleşme	Theorem 4, associative	(a) $x+(y+z)=(x+y)+z$	(b)	$x(yz) = (xy)z$
Postülat 4, dağıtıcı	(a)	$x(y + z) = xy + xz$	(b)	$x + yz=(x+y)(x+z)$
Teorem 5, DeMorgan	(a)	$(x + y)' = x'y'$	(b)	$(xy)'=x+y$
Teorem 6, emilim	(a)	$x + xy=x$	(b)	$x(x + y) = x$

44 Bölüm 2 Boolean Cebiri ve Mantık Kapıları

cebir. Teoremler, postülatlar gibi, çiftler halinde listelenmiştir; her ilişki, eşleştirildiği ilişkinin ikizidir. Postülatlar cebirsel yapının temel aksiyomlarıdır ve kanıta ihtiyaç duymazlar. Teoremler, postülatlardan kanıtlanmalıdır. Bir değişkenle ilgili teoremlerin kanıtları aşağıda sunulmuştur. Sağda, o kanıt adımını haklı çıkaran postülatın numarası listelenmiştir.

TEOREM 1(a): $x+x=x$.

Açıklama	Gerekçe
$x + x = (x + x) \cdot 1$	postülat 2(b)
$= (x + x)(x + x')$	5(a)
$= x + xx'$	4(b)
$= x + 0$	5(b)
$= x$	-

TEOREM 1(b): $xx=x$.

Açıklama	Gerekçe
$xx=xx+0$	postülat 2(a)
$= xx + xx'$	5(b)
$= x(x + x')$	4(a)
$= x \cdot 1$	5(a)
$= x$	2(b)

Teorem 1(b)'in teorem 1(a)'nın ikili olduğunu ve kanıtın her adımının bölüm (b)'de bölüm (a)'daki karşıtının ikilisi olduğunu unutmayın. Herhangi bir ikili teorem, kendi teoreminin kanıtından benzer şekilde türetilebilir.

TEOREM 2(a): $x+1=1$.

Açıklama	Gerekçe
$x + 1 = 1 \cdot (x + 1)$	postülat 2(b)
$= (x+x)(x+1)$	5(a)
$= x + x' \cdot 1$	4(b)
$= x + x'$	2(b)
$= 1$	5(a)

TEOREM 2(b): $x0=0$ ikiliği ile.

TEOREM 3: $(x)=x$. Postülat 5'ten, $x+x=1$ ve $xx=0$, bu tamamı x 'nin tamamlayıcısını tanımlar. x 'nin tamamlayıcısı x' 'dir ve ayrıca (x) .

Bu nedenle, tamamlayıcı benzersiz olduğundan, $(x)' = x'$. İki veya üç değişken içeren teoremler, postülatlardan ve daha önce kanıtlanmış teoremlerden cebirsel olarak kanıtlanabilir. Örneğin, emme teoremini ele alalım:

TEOREM 6(a): $x + xy = x$.

Açıklama	Gerekçe
$x + xy = x \cdot 1 + xy$	postülat 2(b)
$= x(1 + y)$	4(a)
$= x(y + 1)$	3(a)
$= x \cdot 1$	2(a)
$= x$	2(b)

TEOREM 6(b): $x(x+y) = x$ ikiliğe göre.

Boolean cebirinin teoremleri, doğruluk tabloları aracılığıyla kanıtlanabilir. Doğruluk tablolarında, ilişkinin her iki tarafı, ilgili değişkenlerin tüm olası kombinasyonları için aynı sonuçları verip vermediğini kontrol etmek için incelenir. Aşağıdaki doğruluk tablosu, ilk emme teoremini doğrular:

x	y	xy	$x + xy$
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

Birleşme yasası ve DeMorgan teoreminin cebirsel kanıtları uzundur ve burada gösterilmeyecektir. Ancak, geçerlilikleri doğruluk tablolarıyla kolayca gösterilebilir. Örneğin, ilk DeMorgan teoremi için doğruluk tablosu, $(x+y)' = x'y$ aşağıdaki gibidir:

x	y	$x + y$	$(x + y)'$	x'	y'	$x'y$
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Operatör Önceliği

Boolean ifadelerini değerlendirirken operatör önceliği (1) parantezler, (2) DEĞİL, (3) VE, ve (4) VEYA'dır. Diğer bir deyişle, parantez içindeki ifadeler, tüm diğer işlemlerden önce değerlendirilmelidir. Önceliği olan bir sonraki işlem tamamlayıcıdır, ardından VE ve nihayetinde VEYA gelir. Örnek olarak, DeMorgan teoremlerinden birinin doğruluk tablosunu düşünelim. İfadenin sol tarafı $(x+y)'$. Bu nedenle, parantez içindeki ifade önce değerlendirilir ve

sonuç daha sonra tamamlanır. İfadenin sağ tarafı $x \vee y$, bu nedenle x ve y 'nin tamamlayıcıları önce değerlendirilir ve sonuç daha sonra VE ile birleştirilir. Normal aritmetikte, çarpma ve toplama sırasıyla VE ve VEYA ile değiştirildiğin de aynı öncelik geçerlidir (tamamlayıcı hariç).

2.5 BOOLEAN FONKSİYONLARI

Boolean cebiri, ikili değişkenler ve mantık işlemleri ile ilgilenen bir cebirdir. Bir Boolean fonksiyonu, ikili değişkenler, 0 ve 1 sabitleri ve mantık işlem sembollerinde oluşan cebirsel bir ifade ile tanımlanır. İkili değişkenlerin belirli bir değeri için, fonksiyon ya 1 ya da 0'a eşit olabilir. Örnek olarak, Boolean fonksiyonunu düşünelim

$$F_1 = x + y'z$$

Fonksiyon F_1 1'e eşittir eğer x 1'e eşitse veya her ikisi de y ve z 1'e eşitse. F_1 0'a eşittir aksi takdirde. Tamamlayıcı işlem, $y=1$ olduğunda, $y=0$ olduğunu belirtir. Bu nedenle,

$F_1=1$ eğer $x=1$ veya eğer $y=0$ ve $z=1$. Bir Boolean fonksiyonu, ikili değişkenler arasındaki mantıksal ilişkiyi ifade eder ve değişkenlerin tüm olası değerleri için ifadenin ikili değerini belirleyerek değerlendirilir.

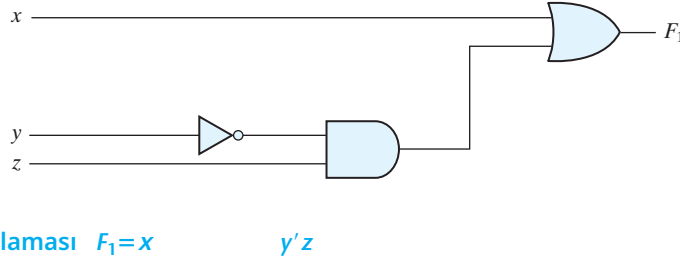
Bir Boolean fonksiyonu bir doğruluk tablosunda temsil edilebilir. Doğruluk tablosundaki satır sayısı 2^n , burada n fonksiyondaki değişken sayısını temsil eder. Doğruluk tablosu için ikili kombinasyonlar, 0'dan 2^n-1 'e kadar sayarak ikili sayılardan elde edilir. Tablo 2.2 fonksiyonu için doğruluk tablosunu göstermektedir F_1 . Üç değişken olan x, y ve z için n bit atamak üzere sekiz olası ikili kombinasyon vardır. F_1 ile etiketlenmiş sütun, bu kombinasyonların her biri için ya 0 ya da 1 içerir. Tablo, fonksiyonun $x=1$ veya $yz=01$ olduğunda 1'e eşit olduğunu ve aksi takdirde 0'a eşit olduğunu göstermektedir.

Bir Boolean fonksiyonu, cebirsel bir ifadeden belirli bir yapıda bağlı mantık kapılarından oluşan bir devre şemasına dönüştürülebilir. F_1 için mantık devre şeması (şematik olarak da adlandırılır) Şekil 2.1 'de gösterilmektedir. Giriş için bir inverter bulunmaktadır y tamamlayıcısını üretmek için. $y \cdot z$ terimi için bir AND kapısı ve x ile $y \cdot z$ 'yi birleştiren bir OR kapısı bulunmaktadır.

Tablo 2.2

F_1 ve F_2 için Doğruluk Tabloları

			F_1	F_2
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	0
1	1	1	1	0



mantık devre şemalarında, fonksiyonun değişkenleri devrenin girişleri olarak alınır ve ikili değişken F_1 devrenin çıkışı olarak alınır. Şematik, devrenin çıkışı ile girişleri arasındaki ilişkiyi ifade eder. Her bir giriş ve çıkış kombinasyonunu listelemek yerine, girişlerin mantık değerlerinden her bir çıkışın mantık değerini nasıl hesaplayacağınızı gösterir.

Bir Boolean fonksiyonunun yalnızca bir şekilde doğruluk tablosunda temsil edilebileceği vardır. Ancak, fonksiyon cebirsel formda olduğunda, çeşitli şekillerde ifade edilebilir; bunların hepsi eşdeğer mantığa sahiptir. Fonksiyonu temsil etmek için kullanılan belirli ifade, mantık devre diyagramındaki kapların bağlantısını belirleyecektir. Tersine, kapların bağlantısı mantık ifadesini belirleyecektir. Boolean cebirini kullanmamızı motive eden önemli bir gerçek şudur: Boolean ifadesini Boolean cebirinin kurallarına göre manipüle ederek, bazen aynı fonksiyon için daha basit bir ifade elde etmek ve böylece devredeki kap sayısını ve kap için giriş sayısını azaltmak mümkündür. Tasarımcılar, karmaşıklığı ve kap sayısını azaltmaya motive olurlar çünkü çabaları devrenin maliyetini önemli ölçüde azaltabilir. Örneğin, aşağıdaki Boolean fonksiyonunu düşünün:

$$F_2 = x'y'z + x'yz + xy'$$

Bu fonksiyonun mantık kapıları ile bir uygulamasının şeması Şekil 2.2 (a)'de gösterilmiştir. Giriş değişkenleri x ve y , inverterler ile tamamlanarak x' ve y' elde edilir. İfadedeki üç terim üç AND kapısıyla gerçekleştirilmiştir.

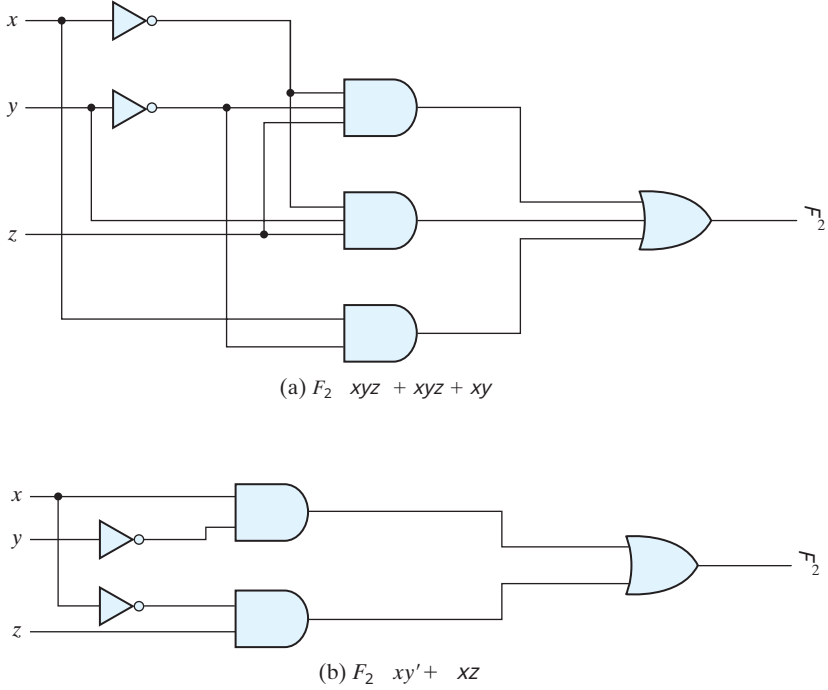
OR kapısı, üç terimin mantıksal OR'unu oluşturur. F_2 için doğruluk tablosu Tablo 2.2 'de listelenmiştir. Fonksiyon, $xyz=001$ veya 011 olduğunda veya $xy=10$ (z 'nin değerine bakılmaksızın) 1'e eşittir ve aksi takdirde 0'a eşittir. Bu koşullar seti, four 1'ler ve dört 0'ler üretir. F_2 .

Şimdi, bazı Boolean cebiri kimliklerini uygulayarak fonksiyonun olası basitleştirilmesini düşünelim:

$$F_2 = x'y'z + x'yz + xy' = x'z(y' + y) + xy' = x'z + xy'$$

Fonksiyon yalnızca iki terime indirgenir ve Şekil 2.2 (b)'de gösterildiği gibi kapılarla uygulanabilir. (b)'deki devrenin (a)'daki devreden daha basit olduğu açıktır, ancak her ikisi de aynı fonksiyonu gerçekleştirir. Bir doğruluk tablosu aracılığıyla, iki ifadenin eşdeğer olduğunu doğrulamak mümkündür. Basitleştirilmiş ifade, $xz=01$ veya $xy=10$ olduğunda 1'e eşittir. Bu, doğruluk tablosunda aynı dört 1'i üretir. Her iki ifade de

48 Bölüm 2 Boolean Cebiri ve Mantık Kapıları



ŞEKİL 2.2
Boolean fonksiyonu F_2 kapılarla uygulanması

aynı doğruluk tablosunu ürettiğinden, eşdeğerdirler. Bu nedenle, iki devre, üç değişkenin tüm olası ikili giriş kombinasyonları için aynı çıktıya sahiptir. Her devre, aynı kimyasal fonksiyonu gerçekleştirir, ancak daha az kapı ve daha az girişe sahip olan tercih edilir çünkü daha az tel ve bileşen gerektirir. Genel olarak, bir mantık fonksiyonunun birçok eşdeğer temsili vardır. Mantığın en ekonomik temsili bulmak önemli bir tasarım görevidir.

Algebraik Manipülasyon

Bir Boolean ifadesi mantık kapıları ile uygulandığında, her terim bir kapı gerektirir ve terim içindeki her değişken kapağı bir girişi belirtir. Bir *literal* terim içindeki tek bir değişkeni, tamamlanmış veya tamamlanmamış formda tanımlıyoruz. Şekil 2.2 (a)'daki fonksiyon üç terim ve sekiz literal içerirken, Şekil 2.2 (b)'deki iki terim ve dört literal içerir. Bir Boolean ifadesinde terim sayısını, literal sayısını veya her ikisini azaltarak, genellikle daha basit bir devre elde etmek mümkündür. Boolean cebirinin manipülasyonu, daha basit bir devre elde etmek amacıyla bir ifadeyi azaltmaktan oluşur. Beş değişkene kadar olan fonksiyonlar, bir sonraki bölümde açıklanan harita yöntemi ile basitleştirilebilir. Karmaşık Boolean fonksiyonları ve birçok farklı çıkış içeren, dijital devre tasarımcıları milyonlarca mantık kapısı ile optimal devreler üretebilen bilgisayar minimizasyon programları kullanır. Bu bölümde tanıttığımız kavramlar, bu araçlar için çerçeveyi sağlar. Mevcut olan tek manuel yöntem, temel ilişkileri ve kullanım ile tanıdık hale gelen diğer manipülasyon tekniklerini içeren bir deneme-yanılma prosedürüdür, ancak yine de insan hatasına tabidir. Aşağıdaki örnekler, okuyucuyu bu önemli tasarım görevi ile tanıştırmak için Boolean cebirinin cebirsel manipülasyonunu göstermektedir.

dijital devre tasarımcıları, milyonlarca mantık kapısı ile optimal devreler üretebilen bilgisayar minimizasyon programları kullanır. Bu bölümde tanıtılan kavramlar, bu araçlar için çerçeveyi sağlar. Mevcut olan tek manuel yöntem, temel ilişkileri ve kullanım ile tanıdık hale gelen diğer manipülasyon tekniklerini içeren bir deneme-yanılma prosedürüdür, ancak yine de insan hatasına tabidir. Aşağıdaki örnekler, okuyucuyu bu önemli tasarım görevi ile tanıştırmak için Boolean cebirinin cebirsel manipülasyonunu göstermektedir.

ÖRNEK 2.1

Aşağıdaki Boolean fonksiyonlarını minimum sayıda literal ile sadeleştirin.

1. $x(x+y)=xx+xy=0+xy=xy.$
2. $x + x'y = (x + x')(x + y) = 1(x + y) = x + y.$
3. $(x + y)(x + y') = x + xy + xy' + yy' = x(1 + y + y') = x.$
4. $xy+x \nabla yz=xy+x \nabla yz(x+x)$
 $=xy+x \nabla xyz+x yz$
 $=xy(1+z)+x \nabla (1+y)$
 $=xy+x z$
5. $(x+y)(x+z)(y+z)=(x+y)(x+z),$ dualite ile fonksiyon 4'ten.

Fonksiyon 1 ve 2 birbirinin ikizidir ve karşılık gelen adımlarda ikiz ifadeler kullanır. Fonksiyon 3'ü basitleştirmenin daha kolay bir yolu, Tablo 2.1'den 4(b) postülü kullanmaktır: $(x+y)(x+y)=x+yy=x$. Dördüncü fonksiyon, bazen literallerin sırayındaki bir artışın daha basit bir son ifadeye yol açtığını göstermektedir. Fonksiyon 5 doğrudan minimize edilmez, ancak fonksiyon 4'ü türetmek için kullanılan adımların ikizinden türetilir. Fonksiyon 4 ve 5 birlikte *konsensüs teoremi* olarak bilinir.

Bir Fonksiyonun Tamamlayanı

Bir fonksiyonun F tamamlayanı F' olup, F değerinde 0'ların 1'lerle ve 1'lerin 0'larla değiştirilmesiyle elde edilir. Bir fonksiyonun tamamlayanı, iki değişken için Tablo 2.1'de listelenen DeMorgan teoremleri aracılığıyla cebirsel olarak türetilir. DeMorgan teoremleri üç veya daha fazla değişken için genişletilebilir. İlk DeMorgan teoreminin üç değişkenli formu, Tablo 2.1'de listelenen postülatlar ve teoremlerden aşağıdaki gibi türetilir:

$$\begin{aligned}
 (A + B + C)' &= (A + x)' && \text{let } B + C = x \\
 &= A'x' && \text{by theorem 5(a) (DeMorgan)} \\
 &= A'(B + C)' && \text{substitute } B + C = x \\
 &= A'(B'C') && \text{by theorem 5(a) (DeMorgan)} \\
 &= A'B'C' && \text{by theorem 4(b) (associative)}
 \end{aligned}$$

DeMorgan teoremleri, herhangi bir sayıda değişken için form olarak iki değişkenli d uruma benzer ve önceki türetimde kullanılan yöntemle benzer ardışık yer değiştirmel erle türetilir. Bu teoremler aşağıdaki gibi genelleştirilebilir:

$$(A + B + C + D + \cdots + F)' = A'B'C'D' \dots F'$$

$$(ABCD \dots F)' = A' + B' + C' + D' + \dots + F'$$

DeMorgan teoremlerinin genelleştirilmiş formu, bir fonksiyonun tamamlayanın AND ve O R operatörlerinin yer değiştirilmesi ve her bir literalın tamamlanmasıyla elde edildiğini belirtir.

ÖRNEK 2.2

Fonksiyonların tamamlayıcıları $F_1 = x \vee x \vee y$ ve $F_2 = x(y \vee yz)$ bulun. DeMorgan teoremlerini gerekli olduğu kadar uygulayarak, tamamlayanlar aşağıdaki gibi elde edilir:

$$F_1' = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)' = (x + y' + z)(x + y + z')$$

$$\begin{aligned} F_2' &= [x(y'z' + yz)]' = x' + (y'z' + yz)' = x' + (y'z')'(yz)' \\ &= x' + (y + z)(y' + z') \\ &= x' + yz' + y'z \end{aligned}$$

Bir fonksiyonun tamamlayıcısını elde etmenin daha basit bir prosedürü, fonksiyonun ikilisini almak ve her bir literalı tamamlamaktır. Bu yöntem, DeMorgan teoremlerinin genelleştirilmiş formlarından gelmektedir. Bir fonksiyonun ikilisinin, VE (AND) ve VEYA (OR) operatörlerinin ve 1'lerin ve 0'ların yer değiştirilmesiyle elde edildiğini unutmayın.

ÖRNEK 2.3

Örnek 2.2'deki fonksiyonların F_1 ve F_2 ikililerini alarak ve her bir literalı tamamlayarak tamamlayıcısını bulun.

- $F_1 = x'yz' + x'y'z$.
F1 'in ikilisi $(x+y+z)(x+y+z)$.
Her bir literalı tamamlayın: $(x+y+z)(x+y+z)=F_1$.
- $F_2=x(y+z)$.
F2 'nin ikilisi $x+(y+z)(y+z)$.
Her bir literalı tamamlayın: $x+(y+z)(y+z)=F_2$.

2.6 KANONİK VE STANDART ŞEKİLLER

Mintermler ve Maxtermler

İki değerli bir değişken ya normal formunda (x) ya da tamamlayıcı formunda (x') görünebilir. Şimdi iki ikili değişken x ve y AND işlemi ile birleştirildiğinde düşünelim. Her bir değişken her iki formda da görünebileceğinden, dört olası kombinasyon vardır: xy , $x'y$, xy' ve $x'y'$. Bu dört AND teriminin her birine *minterm* veya *standart çarpan* denir. Benzer bir şekilde, n değişken birleştirilerek 2^n minterm oluşturulabilir. 2 farklı minterm, üç değişken için Tablo 2.3 'te gösterilen yöntemle benzer bir yöntemle belirlenebilir. O'dan $2^n - 1$ 'e kadar olan ikili sayılar n değişkenlerinin altında listelenmiştir. Her bir minterm, n değişkenlerinin bir AND teriminden elde edilir; her değişken, ikili sayının karşılık gelen biti 0 ise primed, 1 ise unprimed olur. Her minterm için bir sembol de tabloda gösterilmiştir ve şu formdadır: m_j , burada alt simge j ikili sayının ondalık karşılığını belirtir.

Benzer şekilde, n değişkenleri bir OR terimi oluşturur, her bir değişkenin primli or primsiz olması durumunda, 2^n olası kombinasyon sağlar, bunlara *maxterms* veya *standart toplamlar* denir. Üç değişken için sekiz maxterm, sembolik tanımlarıyla birlikte, Tablo 2.3 'te listelenmiştir. Herhangi bir 2^n maxterm, n değişkenleri için benzer şekilde belirlenebilir. Önemli olan, (1) her maxterm'in, n değişkenlerinin bir OR teriminden elde edildiğini, her bir değişkenin karşılık gelen bit 0 ise primsiz, 1 ise primli olduğunu ve (2) her maxterm'in karşılık gelen minterm'in tamamlayıcısı olduğunu ve tersinin de geçerli olduğunu not etmektir.

Bir Boolean fonksiyonu, verilen bir doğruluk tablosundan, fonksiyonda 1 üreten değişkenlerin her kombinasyonu için bir minterm oluşturarak ve ardından bu terimleri OR'unu alarak cebirsel olarak ifade edilebilir. Örneğin, Tablo 2.4 'teki fonksiyon f_1 kombinasyonları 001, 100 ve 111'i $x'y, xy'z$ ve xyz olarak ifade ederek belirtir. Bu mintermlerin her biri $f_1 = 1$ sonucunu verdiğinden,

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

Tablo 2.3

Üç İkili Değişken için Minterm ve Maxterm'ler

x	y	z	Minterm'ler		Maxterm'ler	
			Tanım		Terim Tanımı	
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Tablo 2.4
Üç Değişkenin Fonksiyonları

x	y	z	Fonksiyon f_1	Fonksiyon f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Benzer şekilde, kolayca doğrulanabilir ki

$$f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

Bu örnekler, Boolean cebirinin önemli bir özelliğini göstermektedir: Herhangi bir Boolean fonksiyonu, mintermlerin toplamı olarak ifade edilebilir (toplam, terimlerin OR'lanması anlamına gelir).

Şimdi bir Boolean fonksiyonunun tamamlayıcısını düşünelim. Bu, fonksiyonda 0 üreten her kombinasyon için bir minterm oluşturarak ve ardından bu terimleri OR'layarak doğruluk tablosundan okunabilir. f_1 'nin tamamlayıcısı olarak okunur

$$f_1' = x'y'z' + x'yz' + x'yz + xy'z + xyz'$$

Eğer f_1 'nin tamamlayıcısını alırsak, f_1 fonksiyonunu elde ederiz:

$$\begin{aligned} f_1 &= (x + y + z)(x + y' + z)(x' + y + z')(x' + y' + z) \\ &= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6 \end{aligned}$$

Benzer şekilde, f_2 ifadesini tablodan okumak mümkündür:

$$\begin{aligned} f_2 &= (x + y + z)(x + y + z')(x + y' + z)(x' + y + z) \\ &= M_0 M_1 M_2 M_4 \end{aligned}$$

Bu örnekler, Boolean cebirinin ikinci bir özelliğini göstermektedir: Herhangi bir Boolean fonksiyonu, maxtermlerin çarpımı (çarpım, terimlerin AND'lenmesi anlamına gelir) olarak ifade edilebilir. Doğruluk tablosundan maxtermlerin çarpımını elde etme prosedürü şu şekildedir: Fonksiyonda 0 üreten değişkenlerin her kombinasyonu için bir maxterm oluşturun ve ardından tüm bu maxtermlerin AND'ini oluşturun. Boolean fonksiyonları **minterm'lerin toplamı veya maxterm'lerin çarpımı olarak ifade edilenler kanonik form** da bulunmaktadır.

Minterm'lerin Toplamı

Önceden, n ikili değişkenler için, 2 farklı mintermi elde edebileceğimizi ve herhangi bir Boolean fonksiyonunun mintermlerin toplamı olarak ifade edilebileceğini belirttik. Boolean fonksiyonunu tanımlayan toplam, fonksiyonun 1'lerini veren mintermlerdir.

gerçeklik tablosu. Fonksiyon her minterm için 1 veya 0 olabilir ve çünkü orada 2^n minterms, bir n değişken ile oluşturulabilecek tüm fonksiyonları hesaplayabilirsiniz ve 2^n Bazen bir Boolean fonksiyonunu mintermlerin toplamı biçiminde ifade etmek kullanışlıdır. Eğer fonksiyon bu biçimde değilse, önce ifadeyi AND terimlerinin toplamına genişleterek bu hale getirilebilir. Her terim, tüm değişkenleri içerip içermediğini görmek için incelenir. Eğer bir veya daha fazla değişken eksikse, şu gibi bir ifade ile AND'lanır. $x + x'$, burada x kaybolan değişkenlerden biridir. Bir sonraki örnek bu prosedürü netleştirir.

ÖRNEK 2.4

Boolean fonksiyonu $F=A+B$ Cyi mintermlerin toplamı olarak ifade edin. Fonksiyonun üç değişkeni vardır: A, B ve C . İlk terim A iki değişkeni eksiktir; bu nedenle,

$$A = A(B + B') = AB + AB'$$

Bu fonksiyon hala bir değişkeni eksik, bu yüzden

$$\begin{aligned} A &= AB(C + C') + AB'(C + C') \\ &= ABC + ABC' + AB'C + AB'C' \end{aligned}$$

İkinci terim $B + C$ bir değişkeni eksik; bu nedenle,

$$B + C = B'C(A + A') = AB'C + A'B'C$$

Tüm terimleri birleştirerek,

$$\begin{aligned} F &= A + B + C \\ &= ABC + ABC' + AB'C + AB'C' + A'B'C \end{aligned}$$

Ancak $AB + C$ iki kez görünmektedir ve teorem 1'e ($x+x=x$) göre, bu tekrarların birini kaldırmak mümkündür. Mintermleri artan sırada yeniden düzenleyerek, sonunda

$$\begin{aligned} F &= A'B'C + AB'C + AB'C' + ABC' + ABC \\ &= m_1 + m_4 + m_5 + m_6 + m_7 \end{aligned}$$

Bir Boolean fonksiyonu toplam-minterm formunda olduğunda, bazen fonksiyonu aşağıdaki kısa notasyonla ifade etmek uygun olabilir:

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$$

Toplama sembolü Σ terimlerin OR işlemini temsil eder; ardından gelen sayılar, fonksiyonun mintermlerinin indeksleridir. Parantez içindeki harfler, minterm AND terimine dönüştürüldüğünde alınan değişkenlerin sırasını oluşturur.

Bir Boolean fonksiyonunun mintermlerini elde etmenin alternatif bir yöntemi, fonksiyonun doğruluk tablosunu doğrudan cebirsel ifadeden elde etmek ve ardından doğruluk tablosundan mintermleri okumaktır. Örnek 2.4'te verilen Boolean fonksiyonunu düşünün:

$$F = A + B'C$$

Tablo 2.5'te gösterilen doğruluk tablosu, cebirsel ifadeden doğrudan elde edilebilir; A, B ve C değişkenleri altında sekiz ikili kombinasyonu listeleterek ve ekleyerek

Tablo 2.5

F için Doğruluk Tablosu= A B'C

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

1'ler altında F bu kombinasyonlar için $A=1$ ve $BC=01$. Doğruluk tablosundan, fonksiyonun beş mintermi 1, 4, 5, 6 ve 7 olarak okunabilir.

Maxtermlerin Çarpımı

Her biri 2^{2^n} ikili değişkenlerin n fonksiyonları, maxtermlerin çarpımı olarak da ifa de edilebilir. Bir Boolean fonksiyonunu maxtermlerin çarpımı olarak ifade etmek için, önce OR terimleri biçimine getirilmelidir. Bu, dağıtım yasası kullanılarak yapılabilir, $x + yz = (x + y)(x + z)$. Daha sonra her OR teriminde eksik olan değişken x , x ile OR edilir. İşlem aşağıdaki örnekte açıklanmaktadır.

ÖRNEK 2.5

Boolean fonksiyonunu $F = xy + x'z$ maxtermlerin çarpımı olarak ifade edin. Öncelikle, fonksiyonu dağıtım yasasını kullanarak OR terimlerine dönüştürün:

$$\begin{aligned}
 F &= xy + x'z = (xy + x')(xy + z) \\
 &= (x + x')(y + x')(x + z)(y + z) \\
 &= (x' + y)(x + z)(y + z)
 \end{aligned}$$

Fonksiyonun üç değişkeni vardır: x, y ve z . Her OR teriminde bir değişken eksiktir; bu nedenle,

$$\begin{aligned}
 x' + y &= x' + y + zz' = (x' + y + z)(x' + y + z') \\
 x + z &= x + z + yy' = (x + y + z)(x + y' + z) \\
 y + z &= y + z + xx' = (x + y + z)(x' + y + z)
 \end{aligned}$$

Tüm terimleri birleştirip birden fazla görünenleri çıkardığımızda, nihayetinde

şunu elde ederiz

$$\begin{aligned}
 F &= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') \\
 &= M_0 M_2 M_4 M_5
 \end{aligned}$$

Bu fonksiyonu ifade etmenin pratik bir yolu aşağıdaki gibidir:

$$F(x, y, z) = \Pi(0, 2, 4, 5)$$

Ürün sembolü, Π , maxterm'lerin AND'ini belirtir; sayılar, fonksiyonun maxterm indeksleridir.



Kanonik Formlar Arasındaki Dönüşüm

Minterm'lerin toplamı olarak ifade edilen bir fonksiyonun tamamlayıcı, orijinal fonksiyondan eksik olan minterm'lerin toplamına eşittir. Bunun nedeni, orijinal fonksiyonun, fonksiyonu 1 yapan minterm'ler tarafından ifade edilmesidir; oysa tamamlayıcı, fonksiyonun 0 olduğu minterm'ler için 1'dir. Örnek olarak, aşağıdaki fonksiyonu düşünelim.

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$$

Bu fonksiyon, aşağıdaki gibi ifade edilebilen bir tamamlayıcıya sahiptir.

$$F'(A, B, C) = \Sigma(0, 2, 3) = m_0 + m_2 + m_3$$

Şimdi, DeMorgan teoremi ile F fonksiyonunun tamamlayıcısını alırsak, F farklı bir biçimde elde ederiz:

$$F = (m_0 + m_2 + m_3)' = m_0' \cdot m_2' \cdot m_3' = M_0 M_2 M_3 = \Pi(0, 2, 3)$$

Son dönüşüm, Tablo 2.3'te gösterildiği gibi minterm'ler ve maxterm'lerin tanımından kaynaklanmaktadır. Tablo'dan, aşağıdaki ilişkinin geçerli olduğu açıktır:

$$m_j' = M_j$$

Yani, j alt indeksi ile maxterm, aynı

alt indeksi j olan minterm'in tamamlayıcısıdır ve tersine.

Son örnek, minterm toplamı biçiminde ifade edilen bir fonksiyon ile maxterm çarpımı biçimindeki eşdeğeri arasındaki dönüşümü göstermektedir. Benzer bir argüman, maxterm çarpımı ile minterm toplamı arasındaki dönüşümün de benzer olduğunu gösterecektir. Şimdi genel bir dönüşüm prosedürü belirtelim: Bir kanonik formdan diğerine geçmek için, sembollerini ve değiştirin ve orijinal formdan eksik olan sayıları listeleyin. Eksik terimleri bulmak için, minterm veya maxterm sayısının 2 olduğunu, burada n fonksiyondaki ikili değişken sayısıdır, anlamak gerekir.

Bir Boolean fonksiyonu, bir cebirsel ifadeden bir doğruluk tablosu ve kanonik dönüşüm prosedürü aracılığıyla maksimum terimlerin çarpımına dönüştürülebilir. Örneğin, Boolean ifadesini düşünelim.

$$F = xy + x'z$$

Öncelikle, fonksiyonun doğruluk tablosunu çıkarıyoruz, bu Tablo 2.6'da gösterilmiştir. Tablo üzerindeki F altında yer alan 1'ler, değişkenlerin kombinasyonlarından belirlenir; burada $xy = 1$ 1 veya $xz = 01$. Fonksiyonun minimum terimleri doğruluk tablosundan 1, 3, 6 ve 7 olarak okunur. Fonksiyon, minimum terimlerin toplamı olarak ifade edilir.

$$F(x, y, z) = \Sigma(1, 3, 6, 7)$$

Tablo 2.6

 $F = xy$ için Doğruluk Tablosu $x'z$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Minterm'ler

Maxterimler

Üç değişkenli bir fonksiyonda toplamda sekiz minimum veya maksimum terim bulunduğundan, eksi terimleri 0, 2, 4 ve 5 olarak belirliyoruz. Fonksiyon, maksimum terimlerin çarpımı olarak ifade edilir.

$$F(x, y, z) = \Pi(0, 2, 4, 5)$$

Örnek 2.5'te elde edilen aynı cevaptır.

Standart Şekiller

Boolean cebirinin iki kanonik şekli, doğruluk tablosundan verilen bir fonksiyonu okuyarak elde edilen temel şekillerdir. Bu şekiller, genellikle en az sayıda literale sahip olanlar değildir, çünkü her minimum veya maksimum terim, tanım gereği, tüm değişkenleri, ya tamamlanmış ya da tamamlanmamış olarak içermelidir.

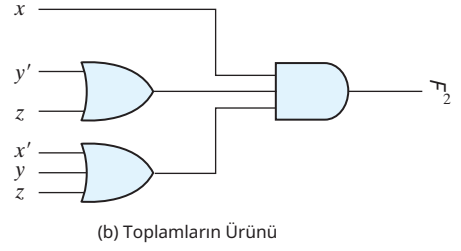
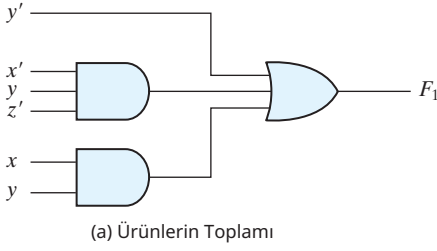
Boolean fonksiyonlarını ifade etmenin bir başka yolu, *standart* formda olmaktır. Bu yapılandırma, fonksiyonu oluşturan terimler bir, iki veya herhangi bir sayıda literal içerebilir. İki tür standart form vardır: ürünlerin toplamı ve toplamaların çarpımı.

Ürünlerin toplamı, her biri bir veya daha fazla literal içeren AND terimlerini içeren bir Boolean ifadesidir ve ürün terimleri olarak adlandırılır. Toplam, bu terimlerin OR işlemini ifade eder. Ürünlerin toplamı olarak ifade edilen bir fonksiyonun örneği şudur:

$$F_1 = y' + xy + x'yz'$$

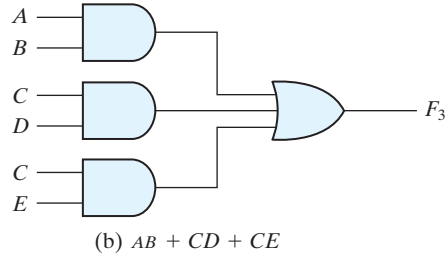
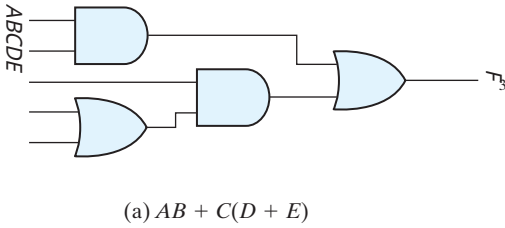
İfade, bir, iki ve üç literal olan üç ürün terimine sahiptir. Toplamları, aslında bir OR işlemi olarak kabul edilir.

Bir ürünlerin toplamı ifadesinin mantık diyagramı, bir grup AND kapısından sonra tek bir OR kapısından oluşur. Bu yapılandırma deseni Şekil 2.3 (a)'da gösterilmiştir. Her ürün terimi, tek bir literal olan bir terim hariç, bir AND kapısı gerektirir. Mantık toplamı, AND kapılarının çıkışları ve tek literalin girişleri ile oluşturulan bir OR kapısı ile oluşturulur. Giriş değişkenlerinin doğrudan tamamlayıcılarında mevcut olduğu varsayılmaktadır, bu nedenle diyagramda inverterler yer almamaktadır. Bu devre yapılandırmasına iki seviyeli uygulama denir.



ŞEKİL 2.3

İki seviyeli uygulama



ŞEKİL 2.4

Üç ve iki seviyeli uygulama

Toplamların ürünü, OR terimlerini içeren bir Boolean ifadesidir ve toplam terimleri olarak adlandırılır. Her terim herhangi bir sayıda literale sahip olabilir. *Çarpım* bu terimlerin VE işlemini ifade eder. Toplamların çarpımı olarak ifade edilen bir fonksiyonun örneği

$$F_2 = x(y' + z)(x' + y + z')$$

Bu ifade, bir, iki ve üç literali olan üç toplam terimine sahiptir. Ürün, bir AND işlemi ile ilgilidir. *ürün* ve *toplam* kelimelerinin kullanımı, AND işleminin aritmetik çarpıma (çarpma) ve OR işleminin aritmetik toplam (toplama) ile benzerliğinden kaynaklanmaktadır. Toplam terimleri için bir grup OR kapısından (tek bir literal hariç) oluşan ürün-toplam ifadesinin kapı yapısı, ardından bir AND kapısı ile devam eder; bu, Şekil 2.3 (b)'de gösterilmiştir. Bu standart türdeki ifade **iki seviyeli bir kapı yapısı ile sonuçlanır**.

Bir Boolean fonksiyonu, standart olmayan bir biçimde ifade edilebilir. Örneğin, bu fonksiyon

$$F_3 = AB + C(D + E)$$

ne toplam ürünler ne de ürün toplamları biçimindedir. Bu ifadenin uygulanması Şekil 2.4 (a)'da gösterilmiştir ve iki AND kapısı ile iki OR kapısı gerektirir. Bu devrede üç kapı seviyesi vardır. Parantezleri kaldırmak için dağıtım yasasını kullanarak standart bir forma dönüştürülebilir:

$$F_3 = AB + C(D + E) = AB + CD + CE$$

Toplam ürün ifadesi Şekil 2.4 (b)'de uygulanmıştır. Genel olarak, iki seviyeli uygulama tercih edilir çünkü sinyalin girişlerden çıkışa yayılması sırasında kapılardan geçen en az gecikmeyi üretir. Ancak, belirli bir kapı için giriş sayısı pratik olmayaabilir.

2.7 DİĞER LOJİK İŞLEMLER

İkili operatörler AND ve OR iki değişken arasında yerleştirildiğinde, x ve y , iki Boolean fonksiyonu oluştururlar, $x \cdot y$ ve $x + y$, sırasıyla. Daha önce, n ikili değişkenler için toplamda 2^{2^n} fonksiyon olduğunu belirtmiştik. Dolayısıyla, iki değişken için, $n = 2$ ve mümkün olan Boolean fonksiyonlarının sayısı 16'dır. Bu nedenle, AND ve OR fonksiyonları, iki ikili değişkenle oluşturulan toplam 16 olası fonksiyondan yalnızca 2'sidir. Diğer 14 fonksiyonu bulmak ve özelliklerini araştırmak öğretici olacaktır.

İki ikili değişkenle oluşturulan 16 fonksiyonun doğruluk tabloları Tablo 2.7'de listelenmiştir. 16 sütunun her biri, F_0 ile F_{15} arasında, iki değişken için bir olası fonksiyonun doğruluk tablosunu temsil eder, x ve y . Fonksiyonların, F 'ye atanabilecek 16 ikili kombinasyondan belirlendiğini unutmayın. 16 fonksiyon, Tablo 2.8'in ilk sütununda gösterildiği gibi, Boolean fonksiyonları aracılığıyla cebirsel olarak ifade edilebilir. Listelenen Boolean ifadeleri, minimum sayıda literale sadeleştirilmiştir.

Her bir fonksiyon, AND, OR ve NOT Boolean operatörleri cinsinden ifade edilebilse de, diğer fonksiyonları ifade etmek için özel operatör sembolleri atanmanın bir sakıncası yoktur. Bu tür operatör sembolleri, Tablo 2.8'in ikinci sütununda listelenmiştir. Ancak, gösterilen tüm yeni semboller arasında, yalnızca özel-VE sembolü, dijital tasarımcılar tarafından yaygın olarak kullanılmaktadır.

Tablo 2.8'deki her bir fonksiyon, eşlik eden bir isim ve fonksiyonu bir şekilde açıklayan bir yorum ile listelenmiştir.¹ Listelenen 16 fonksiyon üç kategoriye ayrılabilir:

1. Sürekli 0 veya 1 üreten iki fonksiyon.
2. Tamamlayıcı ve transfer olan dört tekil işlemli fonksiyon.
3. Sekiz farklı işlemi tanımlayan iki değişkenli on fonksiyon: AND, OR, NAND, NOR, özel-VE, eşdeğerlik, engelleme ve çıkarım.

Tablo 2.7
İki İkili Değişkenin 16 Fonksiyonu için Doğruluk Tabloları

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

¹ ^ sembolü, ayrıca özel veya operatörünü belirtmek için de kullanılır, örneğin, x^y . VE işlevinin sembolü ise bazen iki değişkenin çarpımından atlanır, örneğin, xy .

Tablo 2.8

İki Değişkenin 16 Fonksiyonu için Boolean İfadeleri

Boolean Fonksiyonları	Operatör Sembolü	İsim	Yorumlar
$F_0=0$		Null	İkili sabit 0
$F_1=xy$	xy	VE	x ve y
$F_2=xy'$	x/y	İnhibitör	x , ama y
$F_3=x$		Transfer	x
$F_4=x'y$	y/x	İnhibitör	y , ama x
$F_5=y$		Transfer	y
$F_6=xy'+x'y$	$x \oplus y$	Özel-VE	x veya y , ama ikisi birden değil
$F_7 = x + y$	$x + y$	VE	x veya y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Değil-VE
$F_9 = xy + x'y'$	$(x \oplus y)'$	Eşdeğerlik	x eşittir y
$F_{10} = y'$	y'	Tamamlayıcı	Değil y
$F_{11} = x + y'$	$x \subset y$	İçerme	Eğer y , o zaman x
$F_{12} = x'$	x'	Tamamlayıcı	Değil x
$F_{13} = x' + y$	$x \supset y$	İçerme	If x , then y
$F_{14}=(xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Kimlik	İkili sabit 1

İkili fonksiyonlar için sabitler yalnızca 1 veya 0'a eşit olabilir. Tamamlayıcı fonksiyon, her bir ikili değişkenin tamamlayıcısını üretir. Bir giriş değişkenine eşit olan bir fonksiyona *transfer* adı verilmiştir, çünkü değişken x veya y , değerini değiştirmeden fonksiyonu oluşturan kapıdan geçirilir. Sekiz ikili operatörden ikisi (engellemenin ve çıkarımın) mantıkçılar tarafından kullanılır, ancak bilgisayar mantığında nadiren kullanılır. VE ve VEYA operatörleri, Boolean cebiri ile birlikte anılmıştır. Diğer dört fonksiyon, dijital sistemlerin tasarımında yaygın olarak kullanılmaktadır.

NOR fonksiyonu, VEYA fonksiyonunun tamamlayıcısıdır ve adı *not-VEYA* kısaltmasıdır. Benzer şekilde, NAND VE'nin tamamlayıcısıdır ve *not-VE* kısaltmasıdır. Özel VEYA, kısaca XOR, VEYA'ya benzer, ancak *hem* x hem de y 'nin 1'e eşit olma kombinasyonunu hariç tutar; yalnızca x ve y değerleri farklı olduğunda geçerlidir. (Bazen ikili fark operatörü olarak adlandırılır.) Eşdeğerlik, iki ikili değişkenin eşit olduğunda (yani, her ikisi de 0 veya her ikisi de 1 olduğunda) 1 olan bir fonksiyondur. Özel VEYA ve eşdeğerlik fonksiyonları birbirinin tamamlayıcısıdır. Bu, Tablo 2.7'yi inceleyerek kolayca doğrulanabilir: Özel VEYA için doğruluk tablosu F_6 ve eşdeğerlik için F_9 'dur ve bu iki fonksiyon birbirinin tamamlayıcısıdır. Bu nedenle, eşdeğerlik fonksiyonuna özel NOR denir, kısaca XNOR.

Boolean cebiri, Bölüm 2.2'de tanımlandığı gibi, AND ve OR olarak adlandırdığı mız iki ikili operatöre ve NOT (tamamlayıcı) olarak adlandırılan bir tekil operatöre sahiptir. Tanımlardan, bu operatörlerin bir dizi özelliğini çıkardık ve şimdi bunları kullanarak diğer ikili operatörleri tanımladık. Bu prosedürde benzersiz bir şey yoktur. Örneğin, NOR ($\downarrow$) operatörü ile de başlayabilirdik ve daha sonra AND, OR ve NOT'u onun cinsinden tanımlayabilirdik. Yine de, Boolean cebirini bu şekilde tanıtmamızın iyi nedenleri vardır. "ve", "veya" ve "değil" kavramları tanıdık olup, insanlar tarafından günlük mantıksal fikirleri ifade etmek için kullanılmaktadır. Ayrıca, Huntington postülatları cebirin ikili doğasını yansıtarak, $+$ ve $\cdot$ arasındaki simetrisinin vurgulanmasını sağlar.

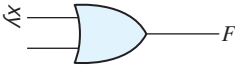
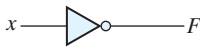
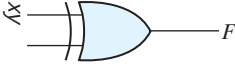
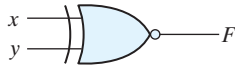
2.8 DİJİTAL MANTIK KAPILARI

Boolean fonksiyonları AND, OR ve NOT işlemleri cinsinden ifade edildiğinden, bu tür kapılarla bir Boolean fonksiyonunu uygulamak daha kolaydır. Yine de, diğer mantık işlemleri için kapılar inşa etme olasılığı pratik bir ilgi alanıdır. Diğer tür mantık kapılarının inşasını değerlendirirken göz önünde bulundurulması gereken faktörler (1) fiziksel bileşenlerle kapıyı üretmenin uygulanabilirliği ve ekonomisi, (2) kapıyı iki girişten daha fazlasına genişletme olasılığı, (3) ikili operatörün temel özellikleri, örneğin değişme ve birleştirme, ve (4) kapının Boolean fonksiyonlarını tek başına veya diğer kapılarla birlikte uygulama yeteneğidir.

Tablo 2.8'de tanımlanan 16 işlevden ikisi bir sabite eşittir ve dördü tekrarlıdır. Mantık kapıları için aday olarak değerlendirilecek yalnızca 10 işlev kalmıştır. İki - engelleme ve çıkarım - değişmeli veya birleşimli değildir ve bu nedenle standart mantık kapıları olarak kullanılması pratik değildir. Diğer sekiz - tamamlayıcı, transfer, VE, VEYA, NAND, NOR, özel VEYA ve eşdeğerlik - dijital tasarımda standart kapılar olarak kullanılır.

Sekiz kapının grafik sembolleri ve doğruluk tabloları Şekil 2.5'te gösterilmiştir. Her kapının, x ve y ile gösterilen bir veya iki ikili giriş değişkeni ve F ile gösterilen bir ikili çıkış değişkeni vardır. VE, VEYA ve tersleyici devreleri Şekil 1.6'da tanımlanmıştır. Tersleyici devresi, bir ikili değişkenin mantıksal anlamını tersine çevirerek NOT veya tamamlayıcı işlevini üretir. Tersleyicinin grafik sembolündeki çıkıştaki küçük daire (bir *baloncuk* olarak adlandırılır) mantıksal tamamlayıcıyı belirtir. Üçgen sembolü tek başına bir tampon devresini belirtir. Bir tampon, *transfer* işlevini üretir, ancak mantıksal bir işlem üretmez, çünkü çıkışın ikili değeri girişin ikili değerine eşittir. Bu devre, sinyalin güç amplifikasyonu için kullanılır ve ardışık bağlı iki tersleyiciye eşdeğerdir.

NAND fonksiyonu, AND fonksiyonunun tamamlayıcısıdır; bu, bir AND grafik sembolünün ardından gelen küçük bir daire ile gösterilir. NOR fonksiyonu, OR fonksiyonunun tamamlayıcısıdır ve bir OR grafik sembolünün ardından gelen küçük bir daire kullanır. NAND ve NOR kapıları, standart mantık kapıları olarak yaygın bir şekilde kullanılmakta olup, aslında AND ve OR kapılarından çok daha popülerdir. Bunun nedeni, NAND ve NOR kapılarının transistör devreleri ile kolayca inşa edilebilmesi ve dijital devrelerin bunlarla kolayca uygulanabilmesidir.

İsim	Grafik sembol	Cebirsel fonksiyon	Doğruluk tablosu															
VE		$F = x \cdot y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
VE		$F = x + y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	1
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Inverter		$F = x'$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	x	F	0	1	1	0									
x	F																	
0	1																	
1	0																	
Buffer		$F = x$	<table><tr><th>x</th><th>F</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	x	F	0	0	1	1									
x	F																	
0	0																	
1	1																	
NAND		$F = (xy)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = (xy)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	0
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Özel-VE (XOR)		$F = xy' + x'y$ $= x \oplus y$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	x	y	F	0	0	0	0	1	1	1	0	1	1	1	0
x	y	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
Özel-NOR veya eşdeğerlik		$F = xy + x'y'$ $= (x \oplus y)'$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	1	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

ŞEKİL 2.5

Dijital mantık kapıları

62 Bölüm 2 Boolean Cebiri ve Mantık Kapıları

Özel-VE kapısının, giriş tarafında ek bir kıvrım çizgisi dışında, VE kapısının grafik sembolüne benzer bir grafik sembolü vardır. Eşdeğerlik veya özel-VE DEĞİL kapısı, grafik sembolünün çıkış tarafındaki küçük daire ile gösterildiği gibi, özel-VE kapısının tamamlayıcısıdır.

Birden Fazla Girişe Uzatma

Şekil 2.5'te gösterilen kapılar - tersleyici ve tampon hariç - iki girişten fazla olacak şekilde genişletilebilir. Bir kapı, temsil ettiği ikili işlem değişmeli ve birleşmeli ise birden fazla girişi olacak şekilde genişletilebilir. Boolean cebirinde tanımlanan VE ve VEYA işlemleri, bu iki özelliğe sahiptir. VEYA fonksiyonu için, şunları elde ederiz:

$$x + y = y + x \quad (\text{commutative})$$

$$(x + y) + z = x + (y + z) = x + y + z \quad (\text{associative})$$

kapı girişlerinin değiştirilebileceğini ve OR fonksiyonunun üç veya daha fazla değışkene genişletilebileceğini gösterir.

NAND ve NOR fonksiyonları değişmeli olup, kapıları iki veya daha fazla giriş olacak şekilde genişletilebilir, sağlanan işlem tanımı biraz değiştirilirse. Zorluk, NAND ve NOR operatörlerinin birleşmeli olmamasıdır (yani, $(x \downarrow y) \downarrow z$

$\neq x \downarrow (y \downarrow z)$), Şekil 2.6 ve aşağıdaki denklemlerde gösterildiği gibi:

$$(x \downarrow y) \downarrow z = [(x + y)' + z]' = (x + y)z' = xz' + yz'$$

$$x \downarrow (y \downarrow z) = [x + (y + z)']' = x'(y + z) = x'y + x'z$$

Bu zorluğun üstesinden gelmek için, çoklu NOR (veya NAND) kapısını

tamamlanmış OR (veya AND) kapısı olarak tanımlıyoruz. Böylece, tanım gereği,

$$x \downarrow y \downarrow z = (x + y + z)'$$

$$x \uparrow y \uparrow z = (xyz)'$$

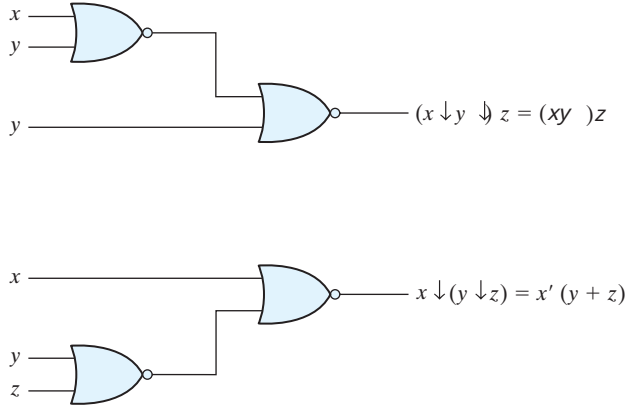
Üç girişli kapılar için grafik sembolleri Şekil 2.7'de gösterilmiştir. Zincirleme NOR ve NAND işlemlerini yazarken, kapıların doğru sırasını belirtmek için doğru parantezleri kullanmak gerekir. Bu ilkeyi göstermek için, Şekil 2.7 (c) devresini ele alalım.

Devre için Boolean fonksiyonu şu şekilde yazılmalıdır

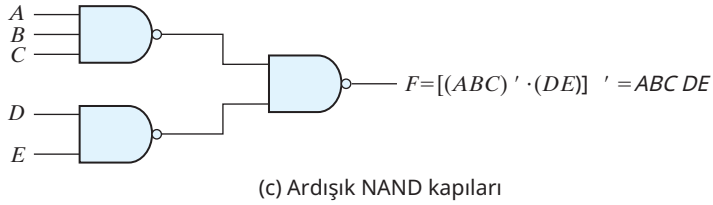
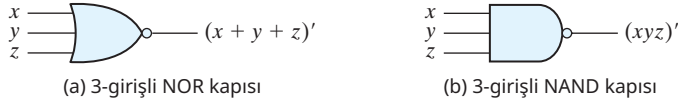
$$F = [(ABC)'(DE)']' = ABC + DE$$

İkinci ifade DeMorgan'ın teoremlerinden biri ile elde edilir. Ayrıca, bir toplam-çarpanlar formundaki ifadenin NAND kapıları ile uygulanabileceğini gösterir. (NAND ve NOR kapıları, Bölüm 3.7'de daha ayrıntılı olarak ele alınmaktadır.)

Özel-OR ve eşdeğer kapılar hem değişmeli hem de birleşmeli olup daha fazla girişe genişletilebilir. Ancak, çoklu girişli özel-OR kapıları hardware açısından yaygın değildir. Aslında, iki girişli bir fonksiyon genellikle daha başka türde kapılarla inşa edilir. Ayrıca, fonksiyonun tanımı daha fazla değışkene genişletildiğinde değiştirilmelidir. Özel-OR, tek bir fonksiyondur (yani, giriş değişkenleri tek sayıda 1'e sahipse 1'e eşittir). Yapı



ŞEKİL 2.6

NOR operatörünün birleşimsizliğini gösterme: $(x \downarrow y) \downarrow z$ $\neq x \downarrow (y \downarrow z)$ 

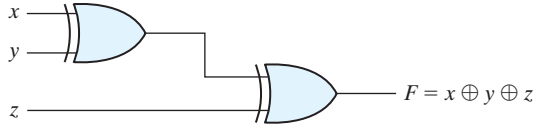
ŞEKİL 2.7

Çoklu girişli ve ardışık NOR ve NAND kapıları

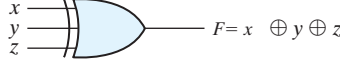
Üç girişli özel-VE işlevinin şekli Şekil 2.8'de gösterilmiştir. Bu işlev genellikle iki girişli kapıları ardışık olarak kullanarak uygulanır, (a) de gösterildiği gibi. Grafikselsel olarak, tek bir üç girişli kapı ile temsil edilebilir, (b) de gösterildiği gibi. (c) deki doğru tablo, çıkışın F yalnızca bir giriş 1'e eşit olduğunda veya tüm üç giriş 1'e eşit olduğunda (yani, giriş değişkenlerindeki 1'lerin toplam sayısının *tek* olduğu durumlarda) 1'e eşit olduğunu açıkça göstermektedir. (Özel-VE kapıları, Bölüm 3.9'da daha ayrıntılı olarak ele alınmaktadır.)

Pozitif ve Negatif Mantık

Herhangi bir kapının girişlerinde ve çıkışlarında ikili sinyal, geçiş sırasında hariç olmak üzere iki değerden birine sahiptir. Bir sinyal değeri mantık 1'i ve diğeri mantık 0'ı temsil eder. İki sinyal değerinin iki mantık değerine atanması nedeniyle, iki farklı atama vardır.



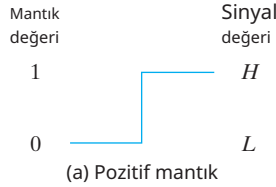
(a) 2 girişli kapılar kullanarak



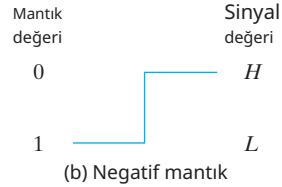
(b) 3 girişli kapı

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(c) Doğruluk tablosu

ŞEKİL 2.8**Üç girişli özel VEYA kapısı**

(a) Pozitif mantık



(b) Negatif mantık

ŞEKİL 2.9**Sinyal ataması ve mantık polaritesi**

sinyal seviyesini mantık değerine dönüştürmek için, Şekil 2.9'da gösterildiği gibi. Daha yüksek sinyal seviyesi H ve daha düşük sinyal seviyesi L ile belirtilir. Yüksek seviye H mantık 1'i temsil etmek için seçildiğinde, pozitif bir mantık sistemi tanımlar. Düşük seviye L mantık 1'i temsil etmek için seçildiğinde, negatif bir mantık sistemi tanımlar. Pozitif ve negatif terimleri biraz yanıltıcıdır, çünkü her iki sinyal de pozitif veya her ikisi de negatif olabilir. Sinyallerin gerçek değerleri, mantık türünü belirlemez; bunun yerine, mantık değerlerinin iki sinyal seviyesinin göreceli amplitüdlere atanması belirler.

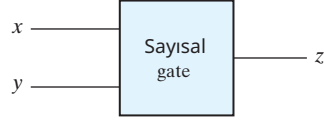
Donanım dijital kapıları, sinyal değerleri olan H ve L cinsinden tanımlanır. Pozitif veya negatif mantık polaritesine karar vermek kullanıcıya bağlıdır. Örneğin, Şekil 2.10 (b)'de gösterilen elektronik kapıyı düşünün. Bu kapının doğruluk tablosu Şekil 2.10 (a)'da listelenmiştir. Kapının fiziksel davranışını $H = 3\text{ V}$ ve $L = 0\text{ V}$ olduğunda belirtir. Şekil 2.10 (c)'deki doğruluk tablosu, $H = 1$ ve $L = 0$ olarak pozitif bir mantık atamasını varsayar. Bu doğruluk tablosu, VE işlemi için olanla aynıdır. Pozitif mantık VE kapısının grafik sembolü Şekil 2.10 (d)'de gösterilmiştir.

Şimdi aynı fiziksel kapı için negatif mantık atamasını düşünün $L = 1$ ve $H = 0$.

Sonuç, Şekil 2.10 (e)'nin doğruluk tablosudur. Bu tablo, girişler ters olsa bile, VE YA operasyonunu temsil eder. Negatif mantık VEYA kapısının grafik sembolü Şekil 2.10 (f)'de gösterilmiştir. Girişlerde ve çıkışta küçük üçgenler bulunmaktadır.

xyz		
L	L	L
L	H	L
H	L	H
H	H	H

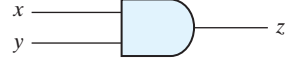
(a) H ve L ile doğruluk tablosu



(b) Kapı blok diyagramı

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

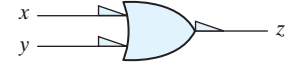
(c) Pozitif mantık için doğruluk tablosu



(d) Pozitif mantık VE kapısı

x	y	z
1	1	1
0	0	0
0	1	1
1	0	1

(e) Negatif mantık için doğruluk tablosu



(f) Negatif mantık VEYA kapısı

ŞEKİL 2.10

Pozitif ve negatif mantığın gösterimi

bir *polarite göstergesi* olarak belirlenir, bunun bir terminal boyunca varlığı, sinyal için negatif mantığın varsayıldığını gösterir. Böylece, aynı fiziksel kapı ya pozitif mantık VE kapısı ya da negatif mantık VEYA kapısı olarak çalışabilir.

Pozitif mantıktan negatif mantığa ve tersine dönüşüm, esasen bir kapta hem girişlerde hem de çıkışta 1'leri 0'lara ve 0'ları 1'lere değiştiren bir işlemdir. Bu işlem bir fonksiyonun ikizini ürettiğinden, tüm terminallerin bir polariteden diğerine değiştirilmesi, fonksiyonun ikizini almakla sonuçlanır. Sonuç olarak, tüm VE (AND) işlemleri VEYA (OR) işlemlerine (veya grafik sembollerine) dönüştürülür ve tersine. Ayrıca, negatif mantık varsayıldığında grafik sembollerinde polarite göstergesi üçgenini dahil etmeyi unutmamak gerekir. Bu kitapta, negatif mantık kapıları kullanılmayacağı ve tüm kapıların pozitif mantık ataması ile çalıştığını varsayacağız.

2.9 ENTEGRE DEVRELER

Bir entegre devre (IC), dijital kapıları inşa etmek için gerekli elektronik bileşenleri içeren, bir silikon yarı iletken kristalinin bir yongası (chip) üzerinde üretilir. Yarı iletken devre oluşturmak için kullanılan karmaşık kimyasal ve fiziksel süreçler bu kitabın konusu değildir. Çeşitli kapılar, gerekli devreyi oluşturmak için yonga içinde birbirine bağlanır. Yonga, seramik veya plastik bir kap içinde monte edilir ve bağlanırlar entegre devreyi oluşturmak için dış pinlere kaynaklanır. Pin sayısı, küçük bir IC paketinde 14'ten, daha büyük bir pakette birkaç bine kadar değişebilir. Her IC'nin tanımlama için paket yüzeyine basılmış bir sayısal tanımı vardır. Satıcılar, ürettikleri IC'ler hakkında açıklamalar ve bilgiler içeren veri kitapları, kataloglar ve internet siteleri sağlar.

Entegrasyon Seviyeleri

Dijital entegre devreler genellikle devrelerinin karmaşıklığına göre sınıflandırılır; bu, tek bir paketteki mantık kapılarının sayısı ile ölçülür. İçinde birkaç kapı bulunan çipler ile yüz binlerce kapı içerenler arasındaki ayrım, genellikle bir paketin küçük, orta, büyük veya çok büyük ölçekli entegrasyon cihazı olarak adlandırılmasıyla yapılır.

Küçük ölçekli entegrasyon (SSI) cihazları, tek bir pakette birkaç bağımsız kapı içerir. Kapıların giriş ve çıkışları doğrudan paketteki pinlere bağlanır. Kapı sayısı genellikle 10'dan azdır ve entegre devrede mevcut pin sayısı ile sınırlıdır.

Orta ölçekli entegrasyon (MSI) cihazları, tek bir pakette yaklaşık 10 ile 1.000 kapı arasında bir karmaşıklığa sahiptir. Genellikle belirli temel dijital işlemleri gerçekleştirirler. MSI dijital fonksiyonları, Bölüm 4'te kod çözücüler, toplayıcılar ve çoklayıcılar olarak tanımlanmakta ve Bölüm 6'da kayıtlar ve sayaçlar olarak sunulmaktadır.

Büyük ölçekli entegrasyon (LSI) cihazları, tek bir pakette binlerce kapı içerir. Bu cihazlar, işlemciler, bellek çipleri ve programlanabilir mantık cihazları gibi dijital sistemler içerir. Bazı LSI bileşenleri Bölüm 7'de sunulmaktadır.

Çok büyük ölçekli entegrasyon (VLSI) cihazları artık tek bir pakette milyonlarca kapı içermektedir. Örnekler arasında büyük bellek dizileri ve karmaşık mikro bilgisayar yongaları bulunmaktadır. VLSI cihazlarının küçük boyutları ve düşük maliyetleri nedeniyle, bilgisayar sistem tasarımı teknolojisini de aynı niteliğinde değiştirmiştir ve tasarımcılara daha önce ekonomik olarak inşa edilmesi mümkün olmayan yapılar oluşturma yeteneği kazandırmıştır.

Dijital Mantık Aileleri

Dijital entegre devreler, yalnızca karmaşıklıkları veya mantıksal işlemleri ile değil, aynı zamanda ait oldukları belirli devre teknolojisi ile de sınıflandırılmaktadır. Devre teknolojisi, dijital mantık ailesi olarak adlandırılmaktadır. Her mantık ailesinin, daha karmaşık dijital devreler ve bileşenlerin geliştirildiği kendi temel elektronik devresi vardır. Her teknolojiye temel devre bir NAND, NOR veya inverter kapısıdır. Elektronik

temel devrenin inşasında kullanılan bileşenler genellikle teknolojiyi adlandırarak için kullanılır. Ticari olarak birçok farklı dijital entegre devre mantık ailesi tanıtılmıştır. Aşağıdakiler en popüler olanlardır:

TTL	transistör-transistör mantığı;
ECL	emiter bağlı mantık;
MOS	metal-oksit yarı iletken;
CMOS	tamamlayıcı metal-oksit yarı iletken.

TTL, 50 yıldır kullanılan ve standart olarak kabul edilen bir mantık ailesidir. ECL, yüksek hızlı çalışma gerektiren sistemlerde avantaj sağlar. MOS, yüksek bileşen yoğunluğu gerektiren devreler için uygundur ve CMOS, dijital kameralar, kişisel medya oynatıcılar ve diğer taşınabilir cihazlar gibi düşük güç tüketimi gerektiren sistemlerde tercih edilir. Düşük güç tüketimi, VLSI tasarımı için esastır; bu nedenle, CMOS baskın mantık ailesi haline gelmiştir, TTL ve ECL ise kullanımda azalmaya devam etmektedir. Mantık ailelerini ayıran en önemli parametreler aşağıda listelenmiştir; CMOS entegre devreleri ek bölümde kısaca tartışılmaktadır.

Fan-out tipik bir kapının çıkışının normal çalışmasını bozmadan sürebileceği standart yüklerin sayısını belirtir. Standart bir yük, genellikle aynı ailedeki başka bir benzer kapının girişi için gereken akım miktarı olarak tanımlanır.

Fan-in bir kapıda mevcut olan giriş sayısını ifade eder.

Güç dağılımı kapının tükettiği ve güç kaynağından sağlanması gereken güçtür.

İletim gecikmesi bir sinyalin girişten çıkışa yayılma süresi için ortalama geçiş gecikme zamanıdır. Örneğin, bir inverterin girişi 0'dan 1'e geçerse, çıkışı 1'den 0'a geçecektir, ancak bu, cihazın iletim gecikmesi tarafından belirlenen bir süre sonra olacaktır. Çalışma hızı, iletim gecikmesi ile ters orantılıdır.

Gürültü marjı, istenmeyen bir değişikliğe neden olmayan maksimum dış gürültü voltajıdır giriş sinyaline eklenir.

VLSI Devrelerinin Bilgisayar Destekli Tasarımı

Submikron geometrik özelliklere sahip entegre devreler, silikon levhalara ışık desenleri optik olarak projeksiyon yapılarak üretilir. Maruz kalmadan önce, levhalar ışığa maruz kaldıklarında sertleşen veya yumuşayan bir foto dirençli malzeme ile kaplanır. Fazla foto direnç kaldırıldığında, maruz kalan silikon desenleri kalır. Maruz kalan bölgeler daha sonra, transistörlerin elektriksel özelliklerine ve kapıların mantıksal özelliklerine sahip bir yarı iletken malzeme oluşturmak için dopant atomları ile implant edilir. Tasarım süreci, devrenin işlevsel bir spesifikasyonunu veya tanımını (yani, ne yapması gerektiğini) fiziksel bir spesifikasyona veya tanıma (silikon üzerinde nasıl uygulanması gerektiği) dönüştürür.

Dijital sistemlerin, milyonlarca transistör ve kapı içeren VLSI devreleri ile tasarımı, büyük ve zorlu bir görevdir. Bu karmaşıklıkta sistemler, genellikle bilgisayar destekli tasarım (CAD) yardımı olmadan geliştirilmesi ve doğrulanması imkansızdır.

Yazılım programlarından oluşan araçlar, bilgisayar tabanlı devre temsillerini destekler ve tasarım sürecini otomatikleştirerek dijital donanımın geliştirilmesine yardımcı olur. Elektronik tasarım otomasyonu (EDA), entegre devrelerin tasarımının tüm aşamalarını kapsar. VLSI devreleri oluşturmak için tipik bir tasarım akışı, tasarım girişi (örneğin, bir şematik girmek) ile başlayıp, IC'yi üretmek için kullanılan fotomaskeyi içeren veritabanının oluşturulmasıyla sona eren bir dizi adımdan oluşur. Silikon içinde bir dijital devrenin fiziksel gerçekleştirilmesi için çeşitli seçenekler mevcuttur. Tasarımcı, uygulamaya özel entegre devre (ASIC), alan programlanabilir kapı dizisi (FPGA), programlanabilir mantık cihazı (PLD) ve tam özel IC arasında seçim yapabilir. Bu cihazların her biri, birimin donanım üretimini kolaylaştırmak için gerekli yazılımı sağlayan bir CAD araçları seti ile birlikte gelir. Bu teknolojilerin her birinin, bir tasarımı uygulamak için gereken cihazların piyasa büyüklüğü ve birim maliyeti tarafından belirlenen bir pazar nişi vardır.

Bazı CAD sistemleri, bilgisayar ekranında şematik diyagramlar oluşturmak ve değiştirmek için bir düzenleme programı içerir. Bu süreç, şematik yakalama veya şematik girişi olarak adlandırılır. Menüler, klavye komutları ve bir fare yardımıyla, bir şematik editör, bilgisayar ekranında dijital devrelerin devre diyagramlarını çizebilir. Bileşenler, dahili bir kütüphanedeki bir listeden ekran üzerine yerleştirilebilir ve daha sonra telleri temsil eden hatlarla bağlanabilir. Şematik giriş yazılımı, şematik ile üretilen bilgileri içeren bir veritabanı oluşturur ve yönetir. Temel kapılar ve fonksiyonel bloklar, devrenin işlevselliğini (yani, mantıksal davranışını) ve zamanlamasını doğrulamaya olanak tanıyan ilişkili modellerle sahiptir. Doğrulama, devreye girişler uygulayarak ve bir mantık simülatörü kullanarak çıkışları metin veya dalga biçimi formatında belirleyip görüntüleyerek gerçekleştirilir.

Dijital sistemlerin tasarımındaki önemli bir gelişme, bir donanım description dili (HDL) kullanımıdır. Böyle bir dil, bir bilgisayar programlama dilini andırır, ancak özellikle dijital donanımı tanımlamaya yöneliktir. Devrelerin işlevselliğini ve yapısını tanımlamak için mantık diyagramlarını ve diğer dijital bilgileri metin biçiminde temsil eder. Ayrıca, bir devrenin işlevselliğinin HDL tanımı, belirli bir donanıma atıfta bulunmadan soyut olabilir, bu da bir tasarımcının daha yüksek düzeyde işlevsel ayrıntılara (örneğin, belirli koşullar altında devrenin bir veri akışında belirli bir 1'ler ve 0'lar desenini tespit etmesi gerektiği) dikkat etmesine olanak tanır, transistör düzeyindeki ayrıntılardan ziyade. HDL tabanlı devre veya sistem modelleri, üretime sunulmadan önce işlevselliğini kontrol etmek ve doğrulamak için simüle edilir, böylece doğru çalışmayan bir devre üretme riskini ve israfını azaltır. HDL tabanlı tasarım dillerinin ortaya çıkmasıyla birlikte, bir devrenin HDL modeli tarafından tanımlanan mantığı otomatik ve optimal bir şekilde sentezlemek için araçlar geliştirilmiştir. Bu iki teknolojik ilerleme, sanayinin neredeyse tamamen bağımlı hale gelmesine yol açmıştır-

HDL tabanlı sentez araçları ve karmaşık dijital sistemlerin devre tasarımı için metodolojileri deneyin. İki HDL - Verilog ve VHDL - Elektrik ve Elektronik Mühendisleri Enstitüsü (IEEE) tarafından standart olarak onaylanmış ve dünya genelindeki tasarımcılar tarafından kullanılmaktadır. Verilog HDL, Bölüm 3.10'da tanıtılmaktadır ve önemi nedeniyle, kitap boyunca Verilog'a dayalı birkaç alıştırma ve tasarım problemi eklenmiştir.

PROBLEMLER

(Soruların * ile işaretlenmiş cevapları metnin sonunda görünmektedir.)

Aşağıdaki kimliklerin geçerliliğini doğruluk tabloları aracılığıyla gösterin:

Üç değişken için DeMorgan teoremi: $(x + y + z) = x' y' z'$ ve $(xyz) = x' + y' + z'$

(b) Dağıtım yasası: $x + yz = (x + y)(x + z)$

(c) Dağıtım yasası: $x(y + z) = xy + xz$

(d) Birleşme yasası: $x + (y + z) = (x + y) + z$

(e) Birleşme yasası ve $x(yz) = (xy)z$

Aşağıdaki Boolean ifadelerini minimum sayıda literale sadeleştirin:

- (a) $xy + x'y$ (b)* $(x + y)(x + y')$
 (c)* $xyz + x'y + xyz'$ (d)* $(A + B)'(A' + B)'$
 (e) $(a + b + c')(a' b' + c)$ (f) $a'bc + abc' + abc + a'bc'$

2.3 Aşağıdaki Boolean ifadelerini minimum sayıda literale sadeleştirin:

- (a)* $ABC + A'B + ABC'$ (b)* $x'yz + xz$
 (c)* $(x + y)'(x' + y')$ (d)* $xy + x(wz + wz')$
 (e)* $(BC' + A'D)(AB' + CD')$ (f) $(a' + c')(a + b' + c')$

2.4 Aşağıdaki Boolean ifadelerini belirtilen sayıda literale indirgeyin:

- (a)* $ABC + AC'$ üç literale
 (b)* $(x'y' + z)' + z + xy + wz$ üç literale
 (c)* $AB + C + D + B(A + A' + C)$ bir literale
 (d)* $(A + C)(A + C)(A + B + C)$ dört literale
 (e)* $BC'D + A'BD + ABCD$ iki literal için

Devam eden ve basitleştirilmiş ifadeleri uygulayan devrelerin mantık diyagramlarını çizin Problem 2.2.

2.6 Devam eden ve basitleştirilmiş ifadeleri uygulayan devrelerin mantık diyagramlarını çizin Problem 2.3.

2.7 Devam eden ve basitleştirilmiş ifadeleri uygulayan devrelerin mantık diyagramlarını çizin Problem 2.4.

2.8 $F = wx + yz$ 'nin tamamlayıcısını bulun; ardından $FF = 0$ ve $F + F = 1$ olduğunu gösterin.

2.9 Aşağıdaki ifadelerin tamamlayıcısını bulun:

- (a)* $xy + x'y$ (b) $(a + c)(a + b')(a' + b + c')$
 (c)* $(x + y)(x' + y')$

2.10

Boolean fonksiyonları F_1 ve F_2 gösterin ki

- (a) Boolean fonksiyonu $E = F_1 + F_2$ F_1 ve F_2 'nin mintermlerinin toplamını içerir.
 (b) Boolean fonksiyonu $G = F_1 F_2$ yalnızca F_1

ve F_2 'ye ait ortak mintermleri içerir.

2.11 Fonksiyonun doğruluk tablosunu listeleyin:

- (a)* $F = xy + xy' + y'z$ (b) $F = bc + a + c$

2.12

İkili bit dizileri üzerinde mantıksal işlemler gerçekleştirebiliriz, her bir karşılık gelen bit çiftini ayrı a yri ele alarak (bit düzeyinde işlem olarak adlandırılır). İki sekiz bitlik dizi verildiğinde $A = 10110001$ ve $B = 10101100$, aşağıdaki mantıksal işlemlerden sonra sekiz bitlik sonucu değerlendirin:

- (a) * VE (b) VEYA (c) XOR (d) * DEĞİL A (e) DEĞİL B

70 Bölüm 2 Boolean Cebiri ve Mantık Kapıları

2.13

Aşağıdaki Boolean ifadelerini uygulamak için mantık diyagramları çizin:

$$y = [(u + x)(y + z)]$$

$$(y) = (x \oplus y)' + x$$

$$(c) \ y = (u' + x')(y + z')$$

$$(d) \ y = u(x \oplus z) + y$$

$$(e) \ y = u + yz + uxy$$

$$y = u + x + x'(u + y')$$

2.14 Boolean fonksiyonunu uygulayın

$$F = xy + x'y' + y'z$$

(a) VE, VEYA ve tersleyici kapılar ile

(b) OR ve inverter kapıları ile

(c) VE ve inverter kapıları ile

(d) NAND ve inverter kapıları ile

(e) NOR ve inverter kapıları ile

2.15

* Aşağıdaki Boolean fonksiyonlarını T_1 ve T_2 en az sayıda literal ile basitleştirin:

	B	C	T_1	T_2
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	0	1

2.16

Boolean fonksiyonunun tüm mintermlerinin mantıksal toplamı n değişkeni için 1'dir.

(a) Önceki ifadeyi $n = 3$ için kanıtlayın.

(b) Genel bir kanıt için bir prosedür önerin.

2.17

Aşağıdaki fonksiyonların doğruluk tablosunu elde edin ve her fonksiyonu mintermlerin toplamı ve maksimum terimlerin çarpımı biçiminde ifade edin:

$$(a) \ * (b + cd)(c + bd)$$

$$(b) \ (cd + b'c + bd')(b + d)$$

$$= (c + d)(b + c)$$

$$(d)$$

$$)$$

$$(d) + acd + ab + c + a + c$$

2.18

Boolean fonksiyonu için

$$F = xy'z + x'y'z + w'xy + wx'y + wxy$$

(a) F 'nin doğruluk tablosunu elde edin.

(b) Orijinal Boolean ifadesini kullanarak mantık diyagramını çizin.

(c) Fonksiyonu minimum sayıda literale sadeleştirmek için Boolean cebirini kullanın.

(d) Sadeleştirilmiş ifadeden fonksiyonun doğruluk tablosunu elde edin ve bunun (a) bölümünde ki ile aynı olduğunu gösterin.

(e) Sadeleştirilmiş ifadeden mantık diyagramını çizin ve toplam kapı sayısını (b) bölümündeki diyagram ile karşılaştırın.

- 2.19** * Aşağıdaki fonksiyonu minterm toplamı ve maxterm çarpımı olarak ifade edin:

$$F(A, B, C, D) = B'D + A'D + BD$$

- 2.20** Aşağıdaki fonksiyonların tamamlayıcısını toplam-minterm formunda ifade edin:

(a) $F(A, B, C, D) = \Sigma(2, 4, 7, 10, 12, 14)$

(b) $F(x, y, z) = \Pi(3, 5, 7)$

- 2.21** Aşağıdakilerin her birini diğer kanonik forma dönüştürün:

(a) $F(x, y, z) = \Sigma(1, 3, 5)$

(b) $F(A, B, C, D) = \Pi(3, 5, 8, 11)$

- 2.22** * Aşağıdaki ifadelerin her birini çarpanların toplamı ve toplamaların çarpımı haline dönüştürün:

(a) $(u + xw)(x + u'v)$

(b) $x' + x(x + y')(y + z')$

- 2.23** Aşağıdaki Boole ifadelerine karşılık gelen mantık diyagramını basitleştirmeden çiz in:

(a) $BC' + AB + ACD$

(b) $(A + B)(C + D)(A + B + D)$

(c) $(AB + A'B')(CD' + C'D)$

(d) $A + CD + (A + D')(C' + D)$

- 2.24** Özel-VE'nin ikilisinin, tamamlayıcısına eşit olduğunu gösterin.

Boolean ifadelerin, Tablo 2.8'de tanımlandığı gibi ikili işlemlerin eşdeğerini yerine koyarak aşağıdakileri gösterin:

(a) Engelleme işlemi ne komütatif ne de birleşimlidir.

Özel-VE işlemi komütatif ve birleşimlidir.

- 2.26** Pozitif mantık NAND kapısının negatif mantık NOR kapısı olduğunu ve tersinin de geçerli olduğunu gösterin.

- 2.27** Aşağıdaki doğruluk tablosuyla tanımlanan devrenin Boolean denklemlerini yazın ve mantık diyagramını çizin:

Tablo P2.27

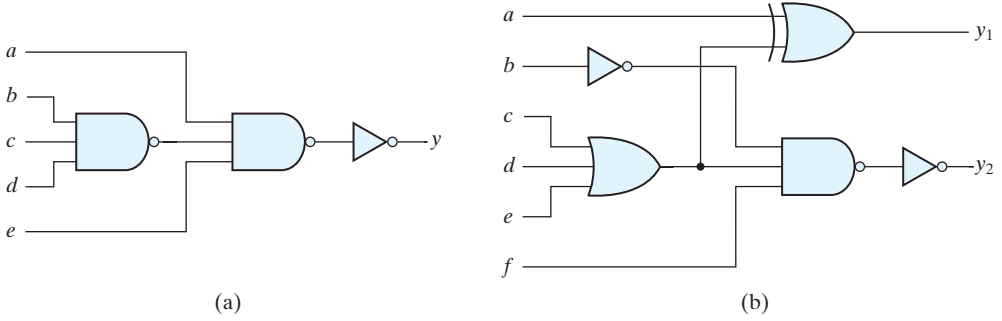
f_1	f_2	a	b	c
1	1	0	0	0
0	1	0	0	1
1	0	0	1	0
1	1	0	1	1
1	0	1	0	0
0	1	1	0	1
1	0	1	1	1

- 2.28** Boolean ifadeleri yazın ve Şekil P2.28'deki mantık diyagramları tarafından tanımlanan devrelerin çıkışını açıklayan doğruluk tablolarını oluşturun.

- 2.29** Aşağıdaki Boolean denkleminin doğru mu yanlış mı olduğunu belirleyin.

$$x'y' + x'z + x'z' = x'z' + y'z' + x'z$$

72 Bölüm 2 Boolean Cebiri ve Mantık Kapıları



ŞEKİL P2.28

Aşağıdaki Boolean ifadelerini çarpanların toplamı biçiminde yazın:

$$(b + d)(a' + b' + c)$$

2.31 Boolean ifadesini toplamaların çarpımı biçiminde yazın:

$$a'b + a'c' + abc$$

REFERANSLAR

1. BOOLE, G. 1854. *Düşünce Yasalarının Araştırması*. New York: Dover.
2. DIETMEYER, D. L. 1988. *Dijital Sistemlerin Mantık Tasarımı*, 3. baskı. Boston: Allyn ve Bacon.
3. Huntington, E. V. Mantığın cebiri için bağımsız postülatlar kümesi. *Trans. Am. Math. Soc.*, 5 (1904): 288–309.
4. *IEEE Standart Donanım Tanım Dili Verilog Donanım Tanım Dili'ne Dayanmaktadır*, Dil Referans Kılavuzu (LRM), IEEE Std.1364-1995, 1996, 2001, 2005, Elektrik ve Elektronik Mühendisleri Enstitüsü, Piscataway, NJ.
5. *IEEE Standard VHDL Language Reference Manual (LRM)*, IEEE Std. 1076-1987, 1988, The Institute of Electrical and Electronics Engineers, Piscataway, NJ.
6. Mano, M. M. ve C. R. Kime. 2000. *Mantık ve Bilgisayar Tasarım Temelleri*, 2. baskı. Upper Saddle River, NJ: Prentice Hall.
7. Shannon, C. E. Röle ve anahtarlama devrelerinin sembolik analizi. *Trans. AIEE* 57 (1938): 713–723.

WEB ARAMA KONULARI

Cebirsel alan
Boolean mantığı
Boolean kapıları
Bipolar transistör
Alan etkili transistör
Emiter bağlı mantık
TTL mantığı
CMOS mantığı
CMOS süreci

Bölüm 3

Kapı Seviyesi Minimizasyonu

3.1 GİRİŞ

Kapı düzeyinde minimizasyon dijital bir devreyi tanımlayan Boole fonksiyonlarının optimal kapı düzeyinde uygulanmasını bulma tasarımıdır. Bu görev iyi anlaşılmaktadır, ancak mantık birkaç girişten fazla olduğunda manuel yöntemlerle gerçekleştirilmesi zordur. Şans eseri, bilgisayar tabanlı mantık sentez araçları büyük bir Boolean eşitler kümesini verimli ve hızlı bir şekilde minimize edebilir. Bununla birlikte, bir tasarımcının problemin temel matematiksel tanımını ve çözümünü anlaması önemlidir. Bu bölüm, bu önemli konuyu anlamanız için bir temel oluşturur ve basit devrelerin manuel tasarımını yapmanıza olanak tanır, modern tasarım araçlarının yetkin bir şekilde kullanılmasına hazırlamanızı sağlar. Bölüm ayrıca modern tasarım araçları tarafından kullanılan bir donanım tanım dilini tanıttacaktır.

3.2 HARİTA YÖNTEMİ

Boolean fonksiyonunu uygulayan dijital mantık kapılarının karmaşıklığı, fonksiyonun uyguladığı cebirsel ifadenin karmaşıklığı ile doğrudan ilişkilidir. Bir fonksiyonun doğruluk tablosu temsili benzersizdir, ancak cebirsel olarak ifade edildiğinde birçok farklı, ancak eşdeğer biçimde görünebilir. Boolean ifadeleri, Bölüm 2.4'te tartışıldığı gibi cebirsel yöntemlerle basitleştirilebilir. Ancak, bu minimizasyon prosedürü, manipülatif süreçteki her bir sonraki adımı tahmin etmek için belirli kuralların eksik olması nedeniyle zorlayıcıdır. Burada sunulan harita yöntemi, Boolean fonksiyonlarını minimize etmek için basit ve doğrudan bir prosedür sağlar. Bu yöntem, bir doğruluk tablosunun görsel bir biçimi olarak kabul edilebilir. Harita yöntemi ayrıca *Karnaugh haritası* veya *K-haritası* olarak da bilinir.

Bir K-haritası, her bir kareyi minimize edilecek fonksiyonun bir mintermi olarak temsil eden karelerden oluşan bir diyagramdır. Herhangi bir Boolean fonksiyonu, mintermelere toplamı olarak ifade edilebileceğinden, bir Boolean fonksiyonu, fonksiyona dahil olan mintermelerin bulunduğu karelerin çevrelediği alandan grafiksel olarak tanınır. Aslında, harita, bir fonksiyonun standart formda ifade edilebileceği tüm olası yolların görsele bir diyagramını sunar. Kullanıcı, çeşitli desenleri tanıyarak aynı fonksiyon için alternatif cebirsel ifadeler türetebilir ve bunlar arasından en basit olanı seçebilir.

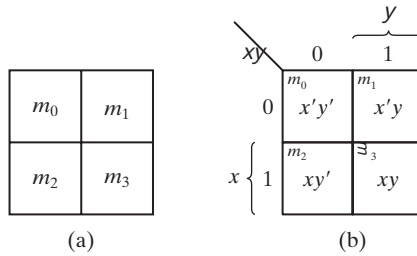
Harita tarafından üretilen basitleştirilmiş ifadeler her zaman iki standart formdan birindedir: ürünlerin toplamı veya toplamaların çarpımı. En basit cebirsel ifadenin, minimum terim sayısına ve her terimde mümkün olan en az sayıda literal içeren bir cebirsel ifade olduğu varsayılacaktır. Bu ifade, minimum kapı sayısı ve her kapiya minimum giriş sayısı ile bir devre diyagramı üretir. Daha sonra en basit ifadenin biricik olmadığını göreceğiz: Bazen minimizasyon kriterlerini karşılayan iki veya daha fazla ifade bulmak mümkündür. Bu durumda, her iki çözüm de tatmin edicidir.

İki Değişkenli K-Harita

İki değişkenli harita Şekil 3.1 (a)'da gösterilmiştir. İki değişken için dört minterm vardır; bu nedenle, harita her minterm için bir tane olmak üzere dört kareden oluşur. Harita, kareler ile iki değişken x ve y arasındaki ilişkiyi göstermek için (b)'de yeniden çizilmiştir. Her satır ve sütunda işaretlenen 0 ve 1, değişkenlerin değerlerini belirtir. Değişken x satır 0'da primli, satır 1'de primsiz görünmektedir. Benzer şekilde, y sütun 0'da primli, sütun 1'de primsiz görünmektedir.

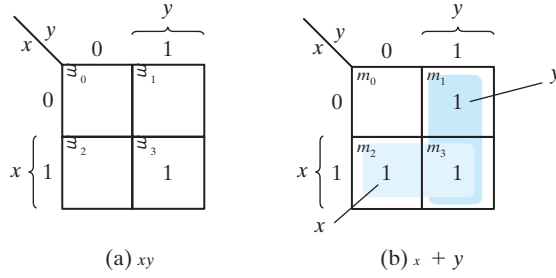
Eğer belirli bir fonksiyona ait minterm'leri içeren kareleri işaretlersek, iki değişkenli harita, iki değişkenin 16 Boolean fonksiyonundan herhangi birini temsil etmenin başka bir yararlı yolu haline gelir. Örnek olarak, xy fonksiyonu Şekil 3.2 (a)'da gösterilmiştir. xy , m_3 , a 1, ait olan kare içine yerleştirilmiştir m_3 . Benzer şekilde, fonksiyon $x+y$ Şekil 3.2 (b) haritasında 1'ler ile işaretlenmiş üç kare ile temsil edilmektedir. Bu kareler fonksiyonun mintermlerinden elde edilmiştir:

$$m_1 + m_2 + m_3 = x'y + xy' + xy = x + y$$



ŞEKİL 3.1

İki değişkenli K-haritası

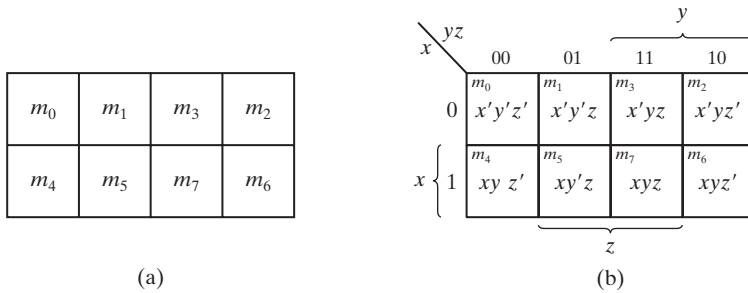


ŞEKİL 3.2
Haritadaki fonksiyonların temsili

De üç kare, değişkenin kesişiminden de belirlenebilirdi. x ikinci satırda ve değişken y ikinci sütunda, alanı kapsar x veya y ait. Her örnekte, fonksiyonun doğrulandığı mintermler 1 ile işaretlenmiştir.

Üç Değişkenli K-Haritası

Şekil 3.3'te üç değişkenli bir K haritası gösterilmektedir. Üç ikili değişken için sekiz mintermdir; bu nedenle, harita sekiz kareden oluşmaktadır. Mintermlerin, ikili bir sırada değil, Gray koduna benzer bir sırada düzenlendiğini unutmayın (Tablo 1.6). Bu dizinin özelliği, bir sütundan diğerine geçerken yalnızca bir bitin değerinin değişmesidir. (b) kısmında çizilen harita, kareler ile üç değişken arasındaki ilişkiyi göstermek için her satırda ve her sütunda sayılarla işaretlenmiştir. Örneğin, atanan kare m_5 1. satıra ve 01. sütuna karşılık gelir. Bu iki numaranın birleştirildiğinde, ondalık karşılığı 5 olan 101 ikili sayısını verir. Haritanın her hücre i benzersiz bir minterme karşılık gelir, bu nedenle square $m_5 = xyz$ ifadesine, x ile işaretlenmiş satırda ve $y z$ (sütun 01) ait sütunda bakmak başka bir yoldur. Her bir değişkenin 1'e eşit olduğu dört kare ve her birinin 0'a eşit olduğu dört kare olduğunu unutmayın. Değişken, ilk dört karede primlenmemiş olarak görünmektedir.



ŞEKİL 3.3
Üç değişkenli K-haritası

sadece iki ürün terimi. Bu iki ürün teriminin mantıksal toplamı, basitleştirilmiş ifade verir

$$F = x'y + xy'$$

Belirli durumlarda, haritadaki iki kare, birbirine dokunmasalar bile komşu olarak kabul edilir. Şekil 3.3 (b)'de, m_0 komşudur m_2 ve m_4 komşudur m_6 çünkü mintermleri bir değişkenle farklıdır. Bu fark, cebirsel olarak kolayca doğrulanabilir:

$$m_0 + m_2 = x'y'z' + x'yz' = x'z'(y' + y) = x'z'$$

$$m_4 + m_6 = xy'z' + xyz' = xz'(y' + y) = xz'$$

Sonuç olarak, komşu karelerin tanımını bu ve benzeri diğer durumları da kapsayacak şekilde değiştirmeliyiz. Bunu, haritayı, sağ ve sol kenarların birbirine dokunarak komşu kareler oluşturduğu bir yüzeyde çizilmiş olarak düşünerek yapıyoruz.

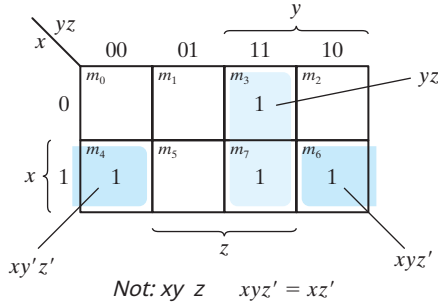
ÖRNEK 3.2

Boolean fonksiyonunu sadeleştir

$$F(x, y, z) = \Sigma(3, 4, 6, 7)$$

Bu fonksiyonun haritası Şekil 3.5'te gösterilmiştir. 1 ile işaretlenmiş dört kare vardır, her biri fonksiyonun bir mintermi için. Üçüncü sütunda iki komşu kare birleştirilerek iki-literal terim yz elde edilir. 1 ile işaretlenmiş kalan iki kare de yeni tanıma göre komşudur. Bu iki kare birleştirildiğinde, iki-literal terim verir xz . Basitleştirilmiş fonksiyon şu hale gelir

$$F = yz + xz'$$



ŞEKİL 3.5

Örnek 3.2 için Harita, $F(x, y, z) = \Sigma(3, 4, 6, 7) = yz + xz$

Şimdi üç değişkenli haritadaki dört bitişik kare kombinasyonunu düşünün. Herhangi bir böyle kombinasyon, dört mintermi mantıksal toplamını temsil eder ve yalnızca bir literalle sonuçlanan bir ifade oluşturur. Örneğin, bitişik dört minterm 0, 2, 4 ve 6'nın mantıksal toplamı, tek bir literalle z :

$$\begin{aligned} m_0 + m_2 + m_4 + m_6 &= x'y'z' + x'yz' + xy'z' + xyz' \\ &= x'z'(y' + y) + xz'(y' + y) \\ &= x'z' + xz' = z'(x' + x) = z' \end{aligned}$$

Bitişik karelerin kombinasyonunun sayısı her zaman 1, 2, 4 ve 8 gibi iki sayının kuvvetini temsil etmelidir. Daha fazla bitişik kare birleştirildiğinde, daha az literalli bir çarpan terimi elde ederiz.

Bir kare bir mintermi temsil eder ve üç literalli bir terim oluşturur.

İki bitişik kare, iki literalli bir terimi temsil eder.

Dört bitişik kare, bir literalli bir terimi temsil eder.

Sekiz bitişik kare, tüm haritayı kapsar ve her zaman 1'e eşit olan bir fonksiyon üretir.

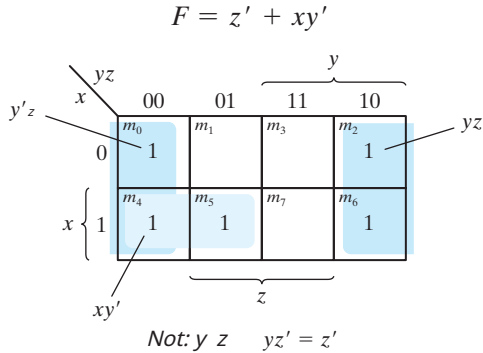
ÖRNEK 3.3

Boolean fonksiyonunu sadeleştir

$$F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$$

F için harita Şekil 3.6'da gösterilmiştir. Öncelikle, ilk ve son sütunlardaki dört bitişik kareyi birleştirerek tek literalli terim z elde ediyoruz. Kalan tek kare, minterm 5'i temsil eder ve daha önce bir kez kullanılmış olan bitişik bir kare ile birleştirilir. Bu sadece izin verilen bir durum değil, aynı zamanda arzu edilen bir durumdur, çünkü iki bitişik kare iki literalli terimi xy verir ve tek kare üç literalli mintermi temsil eder.

$xy z$ Basitleştirilmiş fonksiyon



ŞEKİL 3.6

Örnek 3.3 için Harita, $F(x, y, z) = \Sigma(0, 2, 4, 5, 6) = z' + xy'$

Bir fonksiyon toplam-minterm formunda ifade edilmemişse, haritayı kullanarak fonksiyonun minterm'lerini elde etmek ve ardından fonksiyonu minimum terim sayısına sahip bir ifadeye sadeleştirmek mümkündür. Ancak, cebirsel ifadenin toplam-çarpınlar formunda olduğundan emin olmak gereklidir. Her çarpın terimi haritada bir, iki veya daha fazla karede çizilebilir. Fonksiyonun minterm'leri doğrudan haritadan okunur.

ÖRNEK 3.4

Boolean fonksiyonu için

$$F = A'C + A'B + AB'C + BC$$

- (a) Bu fonksiyonu minterm'lerin toplamı olarak ifade edin.
- (b) Minimal toplam-çarpınlar ifadesini bulun.

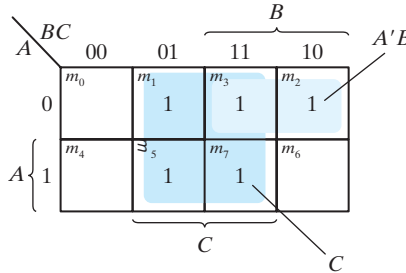
Dikkat edin ki F bir çarpınlar toplamıdır. İfadede üç çarpın terimi iki literale sahiptir ve üç değişkenli haritada her biri için iki kare ile temsil edilir. İlk terime karşılık gelen iki kare, A ve C (ilk satır ve C (iki orta sütun) çakışmasından Fig. 3.7' de 001 ve 011 karelerini verir. Karelerde 1'leri işaretlerken, önceden yerleştirilmiş bir 1 bulmanın mümkün olduğunu unutmayın.

Bu, 1'lerin 011 ve 010 karelerinde bulunduğu ikinci terim, A ve B ile gerçekleşir. Ancak, 011 kare ile A ve C ile ortak olduğundan, yalnızca bir 1 işaretlenmiştir. Bu şekilde devam ederek, AB çarpının 101 kareye ait olduğunu, bu karenin minterm 5'e karşılık geldiğini ve BC teriminin 011 ve 111 karelerinde iki 1'e sahip olduğunu belirliyoruz. Fonksiyonun toplamda beş minterm'i vardır; bu, Şekil 3.7'deki haritada görülen beş 1 ile gösterilmektedir. Mintermler haritadan doğrudan okunarak 1, 2, 3, 5 ve 7 olarak belirlenir. Fonksiyon, minterm toplamı biçiminde ifade edilebilir.

$$F(A, B, C) = \Sigma(1, 2, 3, 5, 7)$$

Başlangıçta verilen ürünlerin toplamı ifadesi, çok fazla terime sahiptir. Haritada gösterildiği gibi, yalnızca iki terim içeren bir ifadeye sadeleştirilebilir:

$$F = C + A'B$$



ŞEKİL 3.7

Örnek 3.4'ün Haritası, AC $A'B + ABC$ $BC = C + A'B$

3.3 DÖRT DEĞİŞKEN K-HARİTASI

Dört ikili değişkenin (w, x, y, z) Boolean fonksiyonları için harita Şekil 3.8'de gösterilmiştir. Şekil 3.8(a)'de 16 mintermi ve her birine atanan kareler listelenmiştir. Şekil 3.8(b)'de, kareler ile dört değişken arasındaki ilişkiyi göstermek için harita yeniden çizilmiştir. Satırlar ve sütunlar, yalnızca iki komşu satır veya sütun arasında bir basamağın değeri değişerek, bir Gray kodu dizisiyle numaralandırılmıştır. Her kareye karşılık gelen minterm, satır numarasının sütun numarasıyla birleştirilmesiyle elde edilebilir. Örneğin, üçüncü satırın (11) ve ikinci sütunun (01) numaraları birleştirildiğinde, 1101 ikili sayısını verir; bu da ondalık 13'ün ikili karşılığıdır.

Bu nedenle, üçüncü satır ve ikinci sütundaki kare, mintermi temsil eder. m_{13} .

Dört değişkenli Boole fonksiyonlarının harita minimizasyonu, üç değişkenli fonksiyonları minimize etmek için kullanılan yöntemle benzerdir. Komşu kareler, yan yana olan kareler olarak tanımlanır. Ayrıca, haritanın üst ve alt kenarlarının yanı sıra sağ ve sol kenarlarının birbirine dokunarak komşu kareler oluşturduğu bir yüzeyde yer aldığı kabul edilir. Örneğin, m_0 ve m_2 bitişik kareler oluşturur, tıpkı m_3 ve m_{11} . Yan yana olan karelerin kombinasyonu, basitleştirme sürecinde faydalıdır ve bu, dört değişkenli haritanın incelenmesiyle kolayca belirlenir:

Bir kare, dört literali olan bir terim veren bir minterm'i temsil eder.

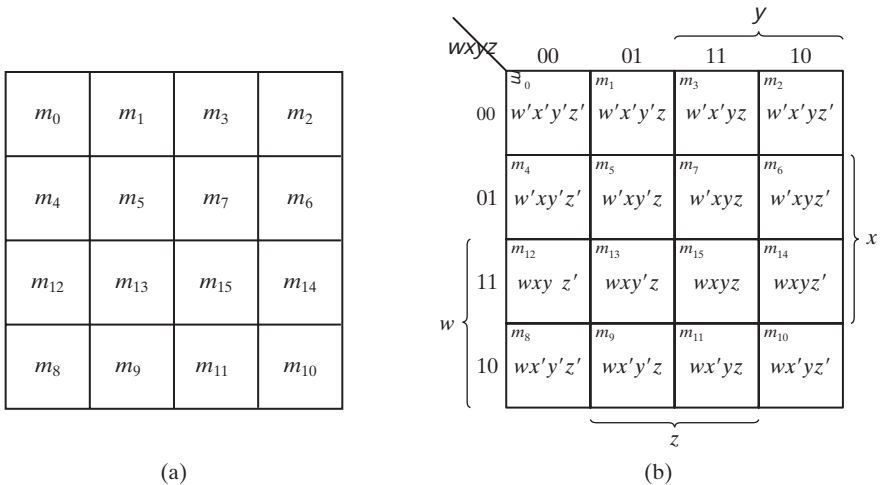
İki bitişik kare, üç literalli bir terimi temsil eder.

Dört bitişik kare, iki literalli bir terimi temsil eder.

Sekiz bitişik kare, bir literalli bir terimi temsil eder.

On altı bitişik kare, her zaman 1'e eşit olan bir fonksiyon üretir.

Diğer kare kombinasyonları fonksiyonu sadeleştiremez. Aşağıdaki iki örnek, dört değişkenli Boolean fonksiyonlarını sadeleştirmek için kullanılan prosedürü göstermektedir.



ŞEKİL 3.8

Dört değişkenli harita

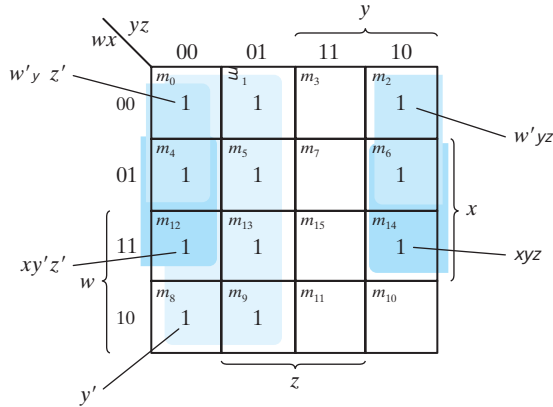
ÖRNEK 3.5

Boolean fonksiyonunu sadeleştir

$$F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$

Fonksiyon dört değişken içerdiğinden, dört değişkenli bir harita kullanılmalıdır. Toplamda listelenen mintermler, Şekil 3.9'daki haritada 1'ler ile işaretlenmiştir. 1'ler ile işaretlenmiş sekiz bitişik kare, bir literal terim olan y oluşturmak için birleştirilebilir. Sağdaki kalan üç 1, basitleştirilmiş bir terim vermek için birleştirilemez; bunlar iki veya dört bitişik kare olarak birleştirilmelidir. Birleştirilen karelerin sayısı arttıkça, terimde ki literal sayısı azalır. Bu örnekte, sağdaki üstteki iki 1, soldaki üstteki iki 1 ile birleştirilerek w terimini verir. Aynı kareyi birden fazla kez kullanmanın kabul edilebilir olduğunu unutmayın. Şimdi, üçüncü satır ve dördüncü sütunda (kare 1110) 1 ile işaretlenmiş bir kare kaldı. Bu kareyi tek başına almak yerine (bu dört literal içeren bir terim verecektir), daha önce kullanılan karelerle birleştirilerek dört bitişik karelik bir alan oluşturuyoruz. Bu kareler, iki orta satırı ve iki uç sütunu oluşturur ve xz terimini verir. Basitleştirilmiş fonksiyon şudur:

$$F = y' + w'z' + xz'$$



$$\text{Not: } w y z \quad w' y z' = w' z' \\ x y' z' + x y z' = x z'$$

ŞEKİL 3.9

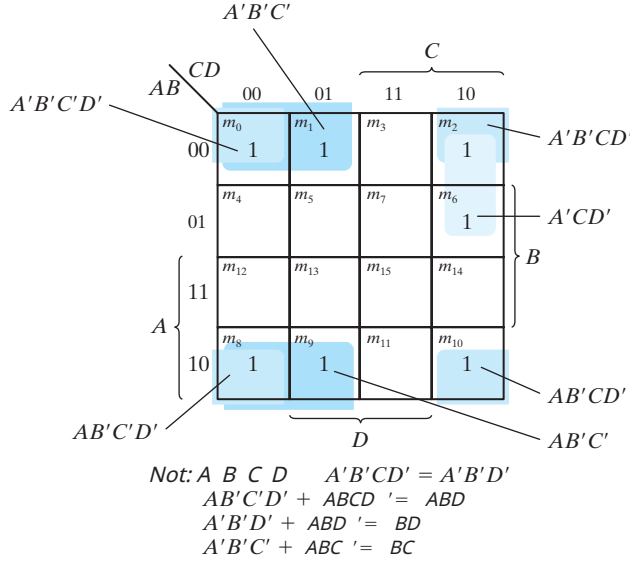
Örnek 3.5 için Harita, $F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14) = y' + w'z' + xz'$

ÖRNEK 3.6

Boolean fonksiyonunu sadeleştir

$$F = A'B'C' + B'CD' + A'BCD' + AB'C'$$

Bu fonksiyonun haritadaki kapsadığı alan, Şekil 3.10'daki 1 ile işaretlenmiş karelerden oluşmaktadır. Fonksiyonun dört değişkeni vardır ve ifade edildiği gibi, üç terimden oluşmaktadır.



ŞEKİL 3.10

Örnek 3.6 için Harita, ABC

$$B'CD' + A'BCD' + AB'C' = B'D' + B'C' + ACD$$

üç literalı ve dört literalı olan bir terim. Üç literalı olan her terim, haritada iki kare ile temsil edilir. Örneğin, $A B C$ karelerde 0000 ve 0001 ile temsil edilir. Fonksiyon, haritada dört köşedeki 1'leri alarak $B D$ terimini vermek suretiyle basitleştirilebilir. Bu, harita, üst ve alt kenarların yanı sıra sol ve sağ kenarların birbiriyle temas ettiği bir yüzeyde çizildiğinde bu dört karenin bitişik olmasından dolayı mümkündür. Üst sıradaki iki soldaki 1, alt sıradaki iki 1 ile birleştirilerek $B C$ terimini verir. Kalan 1, iki karelik bir alanda birleştirilerek $A C D$ terimini verebilir. Basitleştirilmiş fonksiyon

$$F = B'D' + B'C' + A'CD'$$

Önemli İkna Ediciler

Bir haritadaki bitişik kareleri seçerken, (1) kareleri birleştirdiğimizde fonksiyonun tüm mintermlerinin kapsandığından, (2) ifadedeki terim sayısının en aza indirildiğinde ve (3) gereksiz terimlerin (yani, diğer terimler tarafından zaten kapsanan mintermler) olmadığından emin olmalıyız. Bazen basitleştirme kriterlerini karşılayan iki veya daha fazla ifade olabilir. Haritadaki kareleri birleştirme prosedürü, iki özel terim türünün anlamını anladığımızda daha sistematik hale getirilebilir. **A prime Implicant**, haritadaki maksimum sayıda

bitişik kareyi birleştirerek elde edilen bir çarpan terimidir. Eğer bir karedeki bir mintermi yalnızca bir asal implicant kaplıyorsa, o asal implicant *esas olarak adlandırılır*.

Bir fonksiyonun temel etkileyenleri, haritadaki tüm mümkün maksimum kare sayılarının birleştirilmesiyle elde edilebilir. Bu, haritadaki tek bir 1'in, başka bir 1 ile bitişik değilse, bir temel etkileyen olduğunu gösterir. İki bitişik 1, dört bitişik kare grubunun içinde olmadıkları sürece bir temel etkileyen oluşturur. Dört bitişik 1, sekiz bitişik kare grubunun içinde olmadıkları sürece bir temel etkileyen oluşturur ve bu şekilde devam eder. Temel etkileyenler, 1 ile işaretlenmiş her kareye bakarak ve onu kapsayan temel etkileyenlerin sayısını kontrol ederek bulunur. Temel etkileyen, mintermi kapsayan tek temel etkileyen ise esastır.

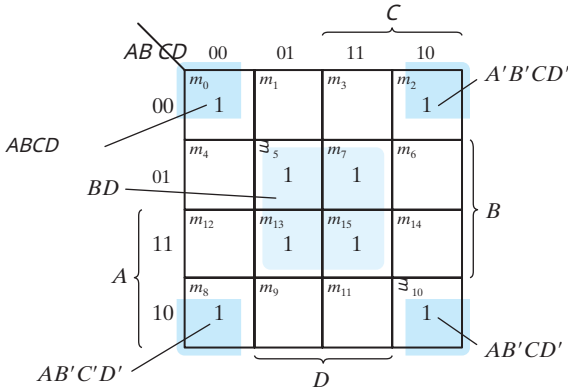
Aşağıdaki dört değişkenli Boole fonksiyonunu düşünün:

$$F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$$

Fonksiyonun mintermleri, Şekil 3.11'deki haritalarda 1'ler ile işaretlenmiştir. Kısmi harita (Şekil 3.11(a)), her biri yalnızca iki literalı olan bir terime dört hücreyi birleştirerek oluşturulmuş iki temel etkileyeni göstermektedir. Bir terim esastır çünkü dört bitişik kare içinde minterm m_0 dahil etmenin yalnızca bir yolu vardır. Bu dört kare, terimi tanımlar $B'D$. Benzer şekilde, minterm m_5 dört bitişik kare ile birleştirilebilir ve bu ikinci terimi verir BD . İki temel asal etkileyen sekiz mintermi kapsar. Kısmi haritadan çıkarılan üç minterm

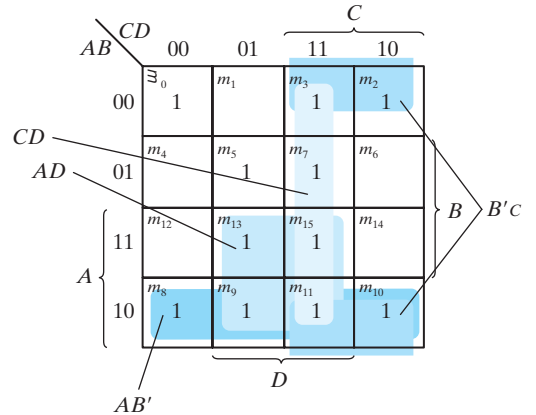
(m_3, m_9 , ve m_{11}) bir sonraki aşamada dikkate alınmalıdır.

Şekil 3.11 (b), üç minterm'in asal etkileyenlerle nasıl kapsanabileceğine dair tüm olası yolları göstermektedir. Minterm m_3 ya asal etkileyen CD ya da asal etkileyen $B'C$. Minterm m_9 ya AD ya da AB ile kaplanabilir. Minterm m_{11} herhangi bir dört temel etkileyiciden biriyle kapatılır. Basitleştirilmiş ifade, iki temel etkileyicinin mantıksal toplamından ve herhangi iki temel etkileyiciden elde edilir.



Not: $A B C D \quad A' B' C D' = A' B' D'$
 $AB' C' D' + AB' C D' = AB' D'$
 $AB' D' = B' D'$

(a) Temel asal implantlar
 BD ve $B'D'$



(b) Temel etkileyiciler $CD, B'C$,
 AD ve AB'

ŞEKİL 3.11

Asal örtücü kullanarak basitleştirme

84 Bölüm 3 Kapı Seviyesi Minimizasyonu

mintermleri kapsayan m_3, m_9 , ve m_{11} . Fonksiyonun her biri iki literalli dört çarpan terimi ile ifade edilebileceği dört olası yol vardır:

$$\begin{aligned} F &= BD + B'D' + CD + AD \\ &= BD + B'D' + CD + AB' \\ &= BD + B'D' + B'C + AD \\ &= BD + B'D' + B'C + AB' \end{aligned}$$

Önceki örnek, haritadaki asal etkileyenlerin tanımlanmasının, basitleştirilmiş bir ifadenin elde edilmesi için mevcut alternatiflerin belirlenmesine yardımcı olduğunu göstermiştir.

Harita üzerinden basitleştirilmiş ifadeyi bulma prosedürü, öncelikle tüm temel asal etkileyenleri belirlememizi gerektirir. Basitleştirilmiş ifade, tüm temel asal etkileyenlerin mantıksal toplamından ve temel asal etkileyenler tarafından kapsanmayan kalan mintermlerin karşılanması için gerekli olabilecek diğer asal etkileyenlerin toplamından elde edilir. Bazen kareleri birleştirmenin birden fazla yolu olabilir ve her kombinasyon eşit derecede basitleştirilmiş bir ifade üretebilir.

Beş Değişkenli Harita

Dört veya daha az değişken için haritalar kadar basit olmayan beş veya daha fazla değişkenli haritalar kullanmak zordur. Beş değişkenli bir harita 32 kareye, altı değişkenli bir harita ise 64 kareye ihtiyaç duyar. Değişken sayısı büyük olduğunda, kare sayısı aşırı hale gelir ve bitişik karelerin birleştirme geometrisi daha karmaşık hale gelir.

Dört veya daha fazla değişkenli haritalar kullanımı zordur ve burada dikkate alınmayacaktır.

3.4 TOPLAMLARIN ÇARPIMI İLE BASİTLEŞTİRME

Önceki tüm örneklerde haritadan türetilen minimize edilmiş Boolean fonksiyonları, çarpım-toplam formunda ifade edilmiştir. Küçük bir değişiklik, toplamların çarpımı formu elde edilebilir.

Toplamların çarpımı formunda minimize edilmiş bir fonksiyon elde etme prosedürü, Boolean fonksiyonlarının temel özelliklerinden kaynaklanır. Haritanın karelerinde yer alan 1'ler, fonksiyonun mintermlerini temsil eder. Standart çarpım-toplam formuna dahil edilmeyen mintermler, fonksiyonun tamamlayıcısını belirtir. Bu gözlemden, bir fonksiyonun tamamlayıcısının haritada 1'ler ile işaretlenmemiş kareler tarafından temsil edildiğini görüyoruz. Boş kareleri 0'lar ile işaretlersek ve bunları geçerli bitişik kareler haline getirirsek, fonksiyonun tamamlayıcısının (yani, F) basitleştirilmiş çarpım-toplam ifadesini elde ederiz. F 'nin tamamlayıcısı, bize çarpım-toplam formunda F fonksiyonunu geri verir (DeMorgan teoreminin bir sonucu). Genel DeMorgan teoremi nedeniyle, elde edilen fonksiyon otomatik olarak çarpım-toplam formundadır. Bu nu göstermek için en iyi yol bir örnek vermektir.

ÖRNEK 3.7

Aşağıdaki Boolean fonksiyonunu (a) çarpanların toplamı formuna ve

(b) toplamların çarpımı formuna basitleştirin:

$$F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10)$$

Şekil 3.12'deki haritada işaretlenen 1'ler, fonksiyonun tüm mintermlerini temsil eder. 0 ile işaretlenen kareler, F 'ye dahil olmayan mintermleri temsil eder ve bu nedenle F 'nin tamamlayıcısını gösterir. 1'ler ile işaretlenen kareleri birleştirerek,

sıralı çarpanların toplamı formunda basitleştirilmiş fonksiyonu elde ederiz:

$$(a) F = B'D' + B'C' + A'C'D$$

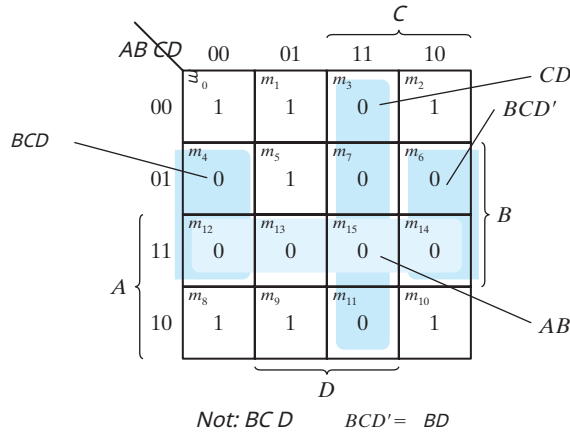
0 ile işaretlenen kareler birleştirildiğinde, diyagramda gösterildiği gibi, basitleştirilmiş tamamlayıcı fonksiyonu elde ederiz:

$$F = AB + CD + BD$$

DeMorgan teoremini uygulayarak (her bir literal'in ikili ve tamamlayıcı hale getirilmesi ile, Bölüm 2.4'te açıklandığı gibi), toplam-çarpan formunda basitleştirilmiş fonksiyonu elde ediyoruz:

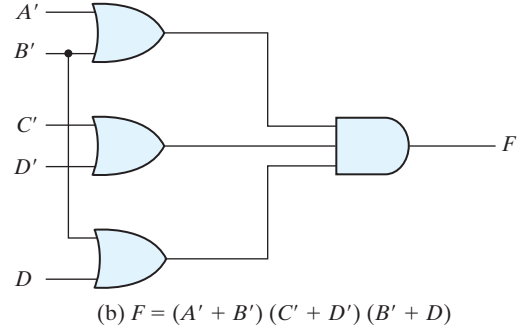
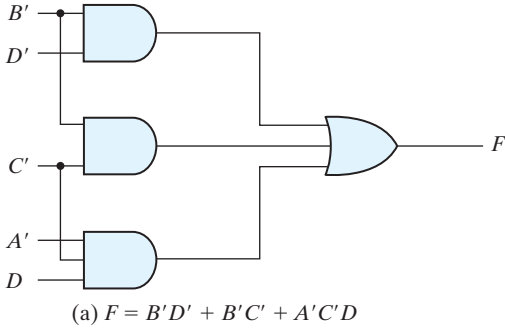
$$(b) F = (A' + B')(C' + D')(B' + D)$$

Örnek 3.7'de elde edilen basitleştirilmiş ifadelerin kapı düzeyindeki uygulaması, Şekil 3.13'te gösterilmiştir. Toplam-çarpan ifadesi (a) kısmında, her bir AND terimi için bir grup AND kapısı ile uygulanmıştır. AND kapılarının çıkışları, tek bir OR kapısının girişlerine bağlanmıştır. Aynı fonksiyon (b) kısmında toplam-çarpan formunda uygulanmıştır.



ŞEKİL 3.12

Örnek 3.7 için Harita, $F(A, B, C, D) = \Sigma(0, 1, 2, 5, 8, 9, 10) = B'D' + B'C' + A'C'D = (A' + B')(C' + D')(B' + D)$



ŞEKİL 3.13

Örnek 3.7'nin fonksiyonunun kapı uygulamaları

Tablo 3.1

Fonksiyon F'nin Gerçeklik Tablosu

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

OR kapılarından oluşan bir form, her OR terimi için bir tane. OR kapılarının çıkışları tek bir AND kapısının girişlerine bağlanır. Her durumda, giriş değişkenlerinin doğrudan ta mamlayıcılarında mevcut olduğu varsayılmaktadır, bu nedenle inverterlere ihtiyaç yoktur. Şekil 3.13'te belirlenen konfigürasyon deseni, herhangi bir Boolean fonksiyonunun standart formlardan birinde ifade edildiğinde uygulandığı genel formdur. AND kapıları, toplam ürünler formunda tek bir OR kapısına; OR kapıları, toplamaların çarpımı formunda tek bir AND kapısına bağlanır. Her iki konfigürasyon da iki seviye kapı oluşturur. Bu nedenle, bir fonksiyonun standart bir biçimde uygulanması iki seviyeli bir uygulama olarak adlandırılır. İki seviyeli uygulama, kapılara gelen giriş sayısına bağlı olarak pratik olmayabilir.

Örnek 3.7, fonksiyonun başlangıçta mintermlerin toplam biçiminde ifade edildiğinde toplamaların çarpımı basitleştirme prosedürünü göstermiştir.

Bu prosedür, fonksiyonun başlangıçta çarpan-maxtermelerin kanonik formunda ifade edildiği durumlarda da geçerlidir. Örneğin, Tablo 3.1'de tanımlanan fonksiyonu F için doğruluk tablosunu düşünelim. Minterm toplamı formunda, bu fonksiyon şu şekilde ifade edilir:

$$F(x, y, z) = \Sigma(1, 3, 4, 6)$$

ve ardından F' 'nin mintermleri temsil eden karelerde 0'ları işaretleyerek. Kalan saraylar 1'lerle işaretlenir.

3.5 ÖNEMSİZ DURUMLAR

Bir Boolean fonksiyonu ile ilişkili mintermlerin mantıksal toplamı, fonksiyonun 1'e eşit olduğu koşulları belirtir. Fonksiyon, diğer mintermler için 0'a eşittir. Bu koşul çifti, fonksiyonun değişkenlerinin değerlerinin tüm kombinasyonlarının geçerli olduğunu varsayar. Pratikte, bazı uygulamalarda fonksiyon belirli değişken kombinasyonları için belirtilmemiştir. Örneğin, ondalık rakamlar için dört bitlik ikili kodun kullanılmayan altı kombinasyonu vardır ve bu nedenle belirtilmemiş olarak kabul edilir. Bazı giriş kombinasyonları için belirtilmemiş çıktıları sahip olan fonksiyonlara *eksik tanımlı fonksiyonlar* denir. Çoğu uygulamada, belirtilmemiş mintermler için fonksiyonun hangi değeri aldığı umursanmaz. Bu nedenle, bir fonksiyonun belirtilmemiş mintermlerine atıfta bulunmak için genellikle alışılmıştır.

önemsememe koşulları. Bu önemsememe koşulları, Boolean ifadesinin daha fazla basitleştirilmesi için bir haritada kullanılabilir.

Bir önemsememe mintermi, mantıksal değeri belirtilmemiş değişkenlerin bir kombinasyonudur. Böyle bir minterm, haritada 1 ile işaretlenemez, çünkü bu, fonksiyonun bu kombinasyon için her zaman 1 olması gerektiği anlamına gelir. Benzer şekilde, kareye 0 koymak, fonksiyonun 0 olması gerektiğini gerektirir. Önemsememe koşulunu 1'lerden ve 0'lardan ayırt etmek için bir X kullanılır. Böylece, haritadaki bir kare içinde bir X, belirli bir minterm için F 'ye 0 veya 1 değerinin atanıp atanmadığına önemsemediğimizi gösterir.

Fonksiyonu bir haritada basitleştirmek için bitişik kareleri seçerken, önemsememe mintermleri ya 0 ya da 1 olarak varsayılabilir. Fonksiyonu basitleştirirken, her bir önemsememe mintermine 1'ler veya 0'lar ile dahil etmeyi seçebiliriz; bu, hangi kombinasyonun en basit ifadesi verdiğine bağlıdır.

ÖRNEK 3.8

Boolean fonksiyonunu basitleştir

$$F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$$

hangi önemsememe koşullarına sahiptir

$$d(w, x, y, z) = \Sigma(0, 2, 5)$$

F 'nin mintermleri, fonksiyonu 1 yapan değişken kombinasyonlarıdır. d 'nin mintermleri, ya 0 ya da 1 atanabilecek önemsememe mintermleridir. Harita basitleştirmesi Şekil 3.15'te gösterilmiştir. F 'nin mintermleri 1'ler ile, d 'nin mintermleri X'ler ile işaretlenmiştir ve kalan kareler 0'lar ile doldurulmuştur. Ürünlerin toplamı biçiminde basitleştirilmiş ifadeyi elde etmek için, haritadaki tüm beş 1'i dahil etmemiz gerekir, ancak X'lerden herhangi birini dahil edip etmeyeceğimiz, fonksiyonun nasıl basitleştirildiğine bağlıdır. Terim yz , üçüncü sütundaki dört mintermi kapsar. Kalan minterm, m_1 , birleştirilebilir

		y			
		00	01	11	10
wx	yz				
	00	m_0	m_1	m_3	m_2
$w'x'$	00	X	1	1	X
	01	m_4	m_5	m_7	m_6
01	01	0	X	1	0
	11	m_{12}	m_{13}	m_{15}	m_{14}
11	11	0	0	1	0
	10	m_8	m_9	m_{11}	m_{10}
10	10	0	0	1	0
		z			
		yz			

(a) $F = yz + w'x'$

		y			
		00	01	11	10
wx	yz				
	00	m_0	m_1	m_3	m_2
$w'z$	00	X	1	1	X
	01	m_4	m_5	m_7	m_6
01	01	0	X	1	0
	11	m_{12}	m_{13}	m_{15}	m_{14}
11	11	0	0	1	0
	10	m_8	m_9	m_{11}	m_{10}
10	10	0	0	1	0
		z			
		yz			

(b) $F = yz + w'z$

ŞEKİL 3.15

Örnek, dikkate alınmayan koşullarla

minterm ile m_3 üç-literal terimini vermek için wxz . Ancak, bir veya tane bitişik X ekleyerek dört bitişik kareyi birleştirip iki-literal terim elde edebiliriz. Şekil 3.15(a)'de, önemsenmeyen minterm 0 ve 2, 1'lerle birlikte dahil edilmiştir ve bu, basitleştirilmiş fonksiyonu son uçlandırır.

$$F = yz + w'x'$$

Şekil 3.15(b)'de, önemsenmeyen minterm 5, 1'lerle birlikte dahil edilmiştir ve basitleştirilmiş fonksiyon artık

$$F = yz + w'z$$

Önceki iki ifadenin herhangi biri, bu örnek için belirtilen koşulları karşılar.

Önceki örnek, haritadaki önemsenmeyen minterm'lerin başlangıçta X ile işaretlendiğini ve ya 0 ya da 1 olarak kabul edildiğini göstermiştir. 0 ve 1 arasındaki seçim, eksik tanımlı fonksiyonun nasıl basitleştirileceğine bağlı olarak yapılır. Seçim yapıldıktan sonra, elde edilen basitleştirilmiş fonksiyon, başlangıçta belirtilmemiş olan ve 1'lerle birlikte dahil edilmesi seçilen minterm'leri içeren bir t oplamdan oluşacaktır. Örnek 3.8'de elde edilen iki basitleştirilmiş ifadeyi düşünün:

$$F(w, x, y, z) = yz + w'x' = \Sigma(0, 1, 2, 3, 7, 11, 15)$$

$$F(w, x, y, z) = yz + w'z = \Sigma(1, 3, 5, 7, 11, 15)$$

Her iki ifade de fonksiyonu F 1'e eşit yapan mintermları 1, 3, 7, 11 ve 15'i içerir. İlgisiz mintermler 0, 2 ve 5 her ifadede farklı şekilde ele alınır.

İlk ifade, 1'lerle minterm 0 ve 2'yi içerir ve 0'larla minterm 5'i bırakır. İkinci ifade, 1'lerle minterm 5'i içerir ve 0'larla minterm 0 ve 2'yi bırakır. İki ifade, cebirsel olarak eşit olmayan iki fonksiyonu temsil eder. Her ikisi de fonksiyonun belirtilen minterm'lerini kapsar, ancak her biri farklı dikkate alınmayan minterm'leri kapsar. Tam olarak belirtilmemiş fonksiyon açısından, her iki ifade de kabul edilebilir çünkü tek fark, dikkate alınmayan minterm'ler için F değerindedir.

Şekil 3.15 için fonksiyonun basitleştirilmiş toplamaların çarpımı ifadesini elde etmek de mümkündür. Bu durumda, 0'ları birleştirmenin tek yolu, 0'larla dikkate alınmayan minterm 0 ve 2'yi dahil etmektir ve bu, basitleştirilmiş tamamlayıcı bir fonksiyon verir:

$$F = z + wy$$

F 'in tamamlayıcısını almak, toplamaların çarpımı biçiminde basitleştirilmiş ifadeyi verir:

$$F(w, x, y, z) = (z + wy) = \Sigma(1, 3, 5, 7, 11, 15)$$

Bu durumda, 0'lar ile minterm 0 ve 2'yi ve 1'ler ile minterm 5'i dahil ediyoruz.

3.6 NAND VE NOR UYGULAMASI

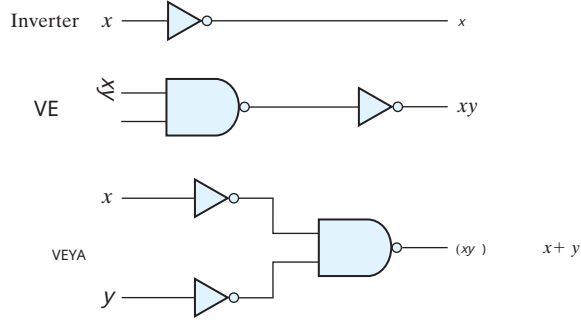
Dijital devreler genellikle AND ve OR kapıları yerine NAND veya NOR kapıları ile inşa edilir. NAND ve NOR kapıları, elektronik bileşenlerle üretimi daha kolaydır ve tüm IC dijital mantık ailelerinde kullanılan temel kapılardır. Dijital devrelerin tasarımında NAND ve NOR kapılarının önemi nedeniyle, AND, OR ve NOT terimleriyle verilen Boolean fonksiyonlarının eşdeğer NAND ve NOR mantık diyagramlarına dönüştürülmesi için kurallar ve prosedürler geliştirilmiştir.

NAND Devreleri

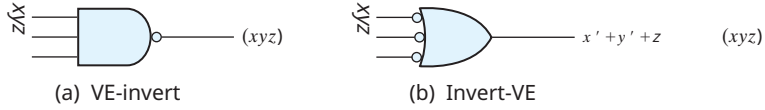
NAND kapısının evrensel bir kapı olduğu söylenir çünkü herhangi bir mantık devresi bununla uygulanabilir. Herhangi bir Boolean fonksiyonunun NAND kapıları ile uygulanabileceğini göstermek için, yalnızca AND, OR ve tamamlayıcı mantıksal işlemlerinin yalnızca NAND kapıları ile elde edilebileceğini göstermemiz gerekir. Bu gerçekten Şekil 3.16'da gösterilmektedir. Tamamlayıcı işlem, tam olarak bir inverter gibi davranan bir girişli NAND kapısından elde edilir. AND işlemi iki NAND kapısı gerektirir. İlk kapı NAND işlemini üretir ve ikinci kapı sinyalin mantıksal anlamını tersine çevirir. OR işlemi, her girişte ek inverterler ile bir NAND kapısı aracılığıyla gerçekleştirilir.

Bir Boolean fonksiyonunu NAND kapıları ile uygulamanın pratik bir yolu, Boolean operatörleri cinsinden basitleştirilmiş Boolean fonksiyonunu elde etmek ve ardından fonksiyonu NAND mantığına dönüştürmektir. AND, OR ve tamamlayıcıdan NAND'a bir cebirsel ifadenin dönüşümü, AND-OR diyagramlarını NAND diyagramlarına dönüştüren basit devre manipülasyon teknikleri ile yapılabilir.

NAND mantığına dönüşümü kolaylaştırmak için, kapı için alternatif bir grafik sembolü tanımlamak uygundur. NAND kapısı için iki eşdeğer grafik sembolü Şekil 3.17'de gösterilmiştir. AND-invert sembolü daha önce tanımlanmış olup, bir küçük daire negasyon göstergesi (kabarık) ile takip edilen bir AND grafik sembolünden oluşur. Alternatif olarak, her girişte bir kabarcık ile önceden gelen bir OR grafik sembolü ile bir NAND kapısını temsil etmek mümkündür. NAND kapısı için invert-OR



ŞEKİL 3.16
NAND kapıları ile mantık işlemleri



ŞEKİL 3.17
Üç girişli NAND kapısı için iki grafik sembol

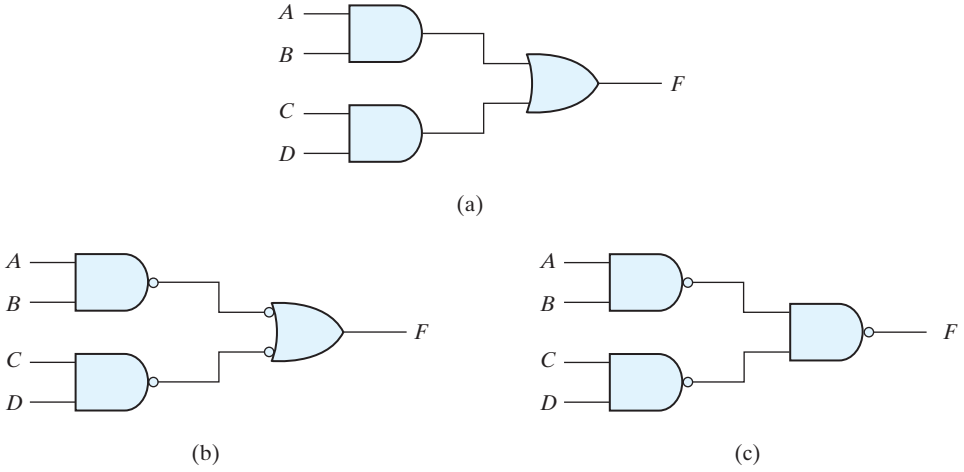
AND grafik sembolü ile bir kabarcık negasyon göstergesi (kabarcık) ile takip edilen bir AND grafik sembolünden oluşur. Alternatif olarak, her girişte bir kabarcık ile önceden gelen bir OR grafik sembolü ile bir NAND kapısını temsil etmek mümkündür. NAND kapısı için invert-OR sembolü DeMorgan teoremini ve negasyon göstergesinin (kabarcık) tamamlamayı belirttiği kuralını takip eder. İki grafik sembolünün temsilleri, NAND devrelerinin analizi ve tasarımında faydalıdır. Her iki sembol aynı diyagramda karşılaştırıldığında, devre karışık notasyonda olduğu söylenir.

İki Düzeyli Uygulama

Boolean fonksiyonlarının NAND kapıları ile uygulanması, fonksiyonların çarpanların toplamı biçiminde olmasını gerektirir. Bir çarpanların toplamı ifadesi ile eşdeğer NAND uygulaması arasındaki ilişkiyi görmek için, Şekil 3.18'de çizilen mantık diyagramlarını göz önünde bulundurun. Üç diyagram da eşdeğerdir ve fonksiyonu uygular.

$$F = AB + CD$$

Fonksiyon, Şekil 3.18(a)'de VE ve VEYA kapıları ile uygulanmıştır. Şekil 3.18(b)'de, VE kapıları NAND kapıları ile değiştirilmiş ve VEYA kapısı, bir VEYA-invert grafik sembolü ile bir NAND kapısı ile değiştirilmiştir. Unutmayın ki, bir baloncuk tamamlamayı belirtir ve aynı hat boyunca iki baloncuk çift tamamlamayı temsil eder, bu nedenle her ikisi de kaldırılabilir. (b) üzerindeki kapılardaki baloncukları kaldırmak, (a) devresini üretir. Bu nedenle, iki diyagram aynı fonksiyonu uygular ve eşdeğerdir.

**ŞEKİL 3.18**Üç yol ile uygulanabilir $F = AB + CD$

Şekil 3.18 (c)'de, çıkış NAND kapısı AND-invert grafik sembolü ile yeniden çizilmiştir. NAND mantık diyagramları çizerken, Şekil 3.18(b) veya (c)'de gösterilen devre kabul edilebilir. Şekil 3.18(b)'deki devre karışık notasyondadır ve uyguladığı Boolean ifadesi $F = (AB)' + (CD)'$ ile daha doğrudan bir ilişkiyi temsil eder. Şekil 3.18 (c)'deki NAND uygulaması cebirsel olarak doğrulanabilir. Uyguladığı fonksiyon, DeMorgan teoremi ile kolayca toplam-çarpınlar formuna dönüştürülebilir:

$$F = ((AB)' (CD)') = AB + CD$$

ÖRNEK 3.9

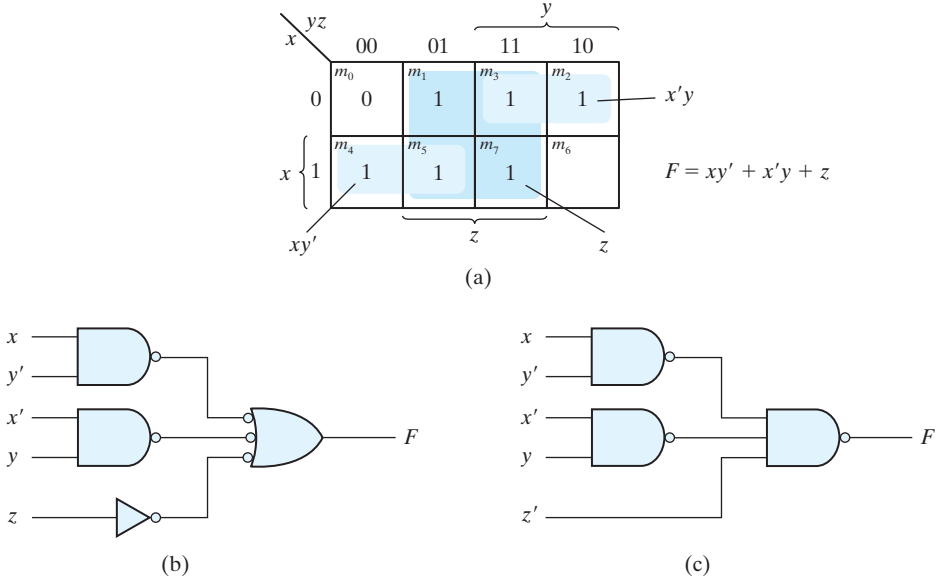
Aşağıdaki Boolean fonksiyonunu NAND kapıları ile uygulayın:

$$F(x, y, z) = (1, 2, 3, 4, 5, 7)$$

İlk adım, fonksiyonu toplam-çarpınlar formuna basitleştirmektir. Bu, Şekil 3.19 (a)'daki harita aracılığıyla yapılır ve basitleştirilmiş fonksiyon elde edilir:

$$F = xy' + x'y + z$$

İki seviyeli NAND uygulaması, Şekil 3.19 (b)'de karışık notasyonda gösterilmiştir. Giriş z 'nin ikinci seviye kapıdaki baloncuk için telafi etmek amacıyla bir girişli NAND kapısına (bir inverter) sahip olması gerektiğine dikkat edin. Mantık diyagramını çizmenin alternatif bir yolu, Şekil 3.19 (c)'de verilmiştir. Burada, tüm NAND kapıları aynı grafik sembolü ile çizilmiştir. Giriş z olan inverter kaldırılmıştır, ancak giriş değişkeni tamamlanmış ve z ile gösterilmiştir.



ŞEKİL 3.19
Örnek 3.9'un Çözümü

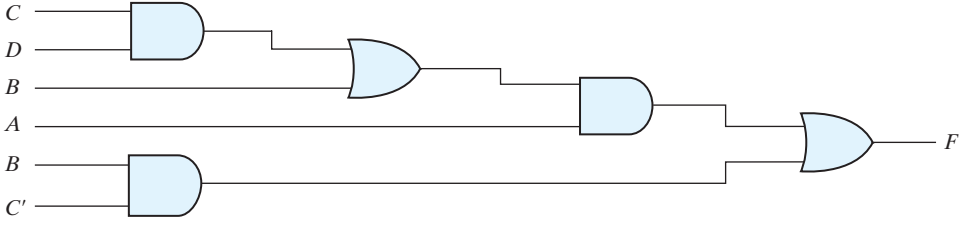
Önceki örnekte açıklanan prosedür, bir Boolean fonksiyonunun iki seviyeli NAND kapıları ile uygulanabileceğini göstermektedir. Bir Boolean fonksiyonundan mantık diyagramı elde etme prosedürü şu şekildedir:

- Fonksiyonu basitleştirip, toplam-çarpanlar formunda ifade et.
- İfadede en az iki literalı olan her çarpan terimi için bir NAND kapısı çiz. Her NAND kapısının girişi, terimin literalleridir. Bu prosedür, birinci seviye kapılardan oluşan bir grup üretir.
- AND-invert veya invert-OR grafik sembolünü kullanarak ikinci seviyede tek bir kapı çizin, girişler birinci seviye kapıların çıkışlarından gelmektedir.
- Tek bir literalı olan bir terim, birinci seviyede bir inverter gerektirir. Ancak, eğer tek literalı tamamlanmışsa, doğrudan ikinci seviye NAND kapısının bir girişine bağlanabilir.

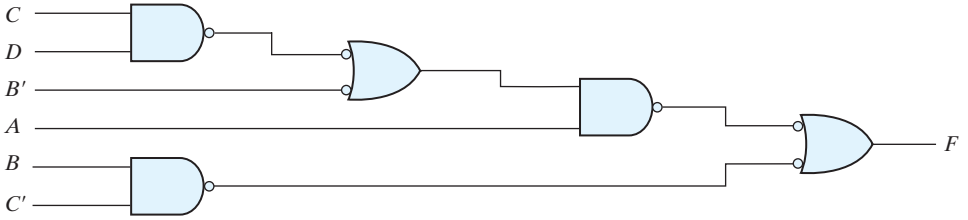
Çok seviyeli NAND Devreleri

Boolean fonksiyonlarını ifade etmenin standart şekli, iki seviyeli bir uygulama ile sonuçlanır. Ancak, dijital sistemlerin tasarımının üç veya daha fazla seviyeye sahip kaplama yapıları ile sonuçlandığı durumlar vardır. Çok seviyeli devrelerin tasarımındaki en yaygın prosedür, Boolean fonksiyonunu AND, OR ve tamamlayıcı işlemler cinsinden ifade etmektir. Fonksiyon daha sonra AND ve OR kapıları ile uygulanabilir. Daha sonra, gerekirse, tamamen NAND devresine dönüştürülebilir. Örneğin, Boolean fonksiyonunu düşünün

$$F = A(CD + B) + BC$$



(a) AND-OR kapıları



(b) NAND kapıları

ŞEKİL 3.20**Uygulama** $F=A(CD+B)+BC$

Parantezleri kaldırmak ve ifadeyi standart bir sum-of-products formuna indirmek mümkün olsa da, bunu bir çok seviyeli devre olarak uygulamayı seçiyoruz. AND-OR uygulaması Şekil 3.20 (a)'da gösterilmiştir. Devrede dört seviye kapı bulunmaktadır. İlk seviyede iki AND kapısı vardır. İkinci seviyede bir OR kapısı, üçüncü seviyede bir AND kapısı ve dördüncü seviyede bir OR kapısı bulunmaktadır. AND ve OR kapılarının alternatif seviyelerinin bir deseni e sahip bir mantık diyagramı, karışık notasyon kullanılarak kolayca bir NAND devresine dönüştürülebilir, bu Şekil 3.20 (b)'de gösterilmiştir. Prosedür, her AND kapısını bir AND-invert grafik sembolüne ve her OR kapısını bir invert-OR grafik sembolüne dönüştürmektir.

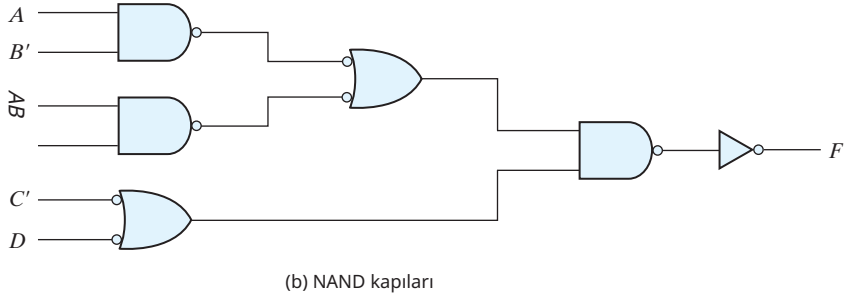
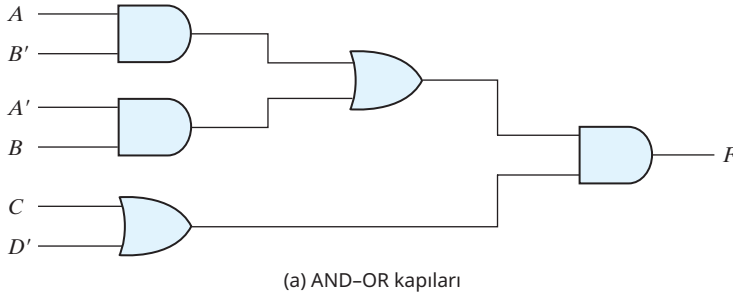
NAND devresi, aynı hat boyunca iki baloncuk olduğu sürece AND-OR diyagramıyla aynı mantığı gerçekleştirir. Girişle ilişkili baloncuk B , ek bir tamamlamaya neden olur; bu, giriş literalinin B olarak değiştirilmesiyle telafi edilmelidir.

Bir çok seviyeli AND-OR diyagramını karışık notasyon kullanarak tamamen NAND diyagramına dönüştürme genel prosedürü aşağıdaki gibidir:

1. Tüm AND kapılarını AND-invert grafik sembolleri ile NAND kapılarına dönüştürün.
2. Tüm OR kapılarını invert-OR grafik sembolleri ile NAND kapılarına dönüştürün.
3. Diyagramdaki tüm baloncukları kontrol edin. Aynı çizgi boyunca başka bir küçük daire ile telafi edilmeyen her baloncuk için, bir inverter (bir girişli NAND kapısı) ekleyin veya giriş literalini tamamlayın.

Başka bir örnek olarak, çok seviyeli Boolean fonksiyonunu düşünün

$$F = (AB' + A'B)(C + D')$$



ŞEKİL 3.21

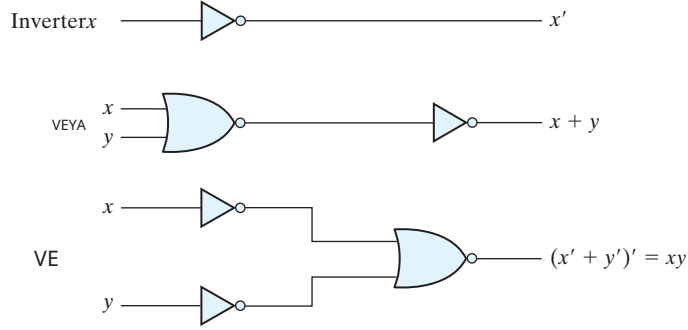
Uygulama $F=(AB+A'B)(C+D)$

Bu fonksiyonun AND-OR uygulaması, üç seviyeli gate ile Şekil 3.21 (a)'de gösterilmiştir. Karışık notasyonla NAND'a dönüşüm, diyagramın Şekil 3.21(b)'de sunulmuştur. Girişlerle ilişkili iki ek baloncuk C ve D bu iki literalin C ve D olarak tamamlar. Çıkış NAND kapısındaki baloncuk, çıkış değerini tamamlar, bu nedenle sinyali tekrar tamamlamak ve orijinal değeri geri almak için çıkışta bir inverter kapısı eklememiz gerekir.

NOR Uygulaması

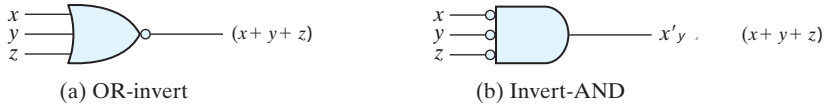
NOR işlemi, NAND işleminin ikizidir. Bu nedenle, NOR mantığı için tüm prosedürler ve kuralar, NAND mantığı için geliştirilen karşılık gelen prosedürlerin ve kuralların ikizleridir. NOR kapısı, herhangi bir Boolean fonksiyonunu uygulamak için kullanılabilir ve ek başka bir evrensel kapıdır. NOR kapıları ile tamamlayıcı, VEYA ve VE işlemlerinin uygulanması Şekil 3.22'de gösterilmiştir. Tamamlayıcı işlemi, tam olarak bir inverter gibi davranan bir girişli NOR kapısından elde edilir. VEYA işlemi iki NOR kapısı gerektirir ve VE işlemi, her girişte inverter bulunan bir NOR kapısı ile elde edilir.

Karışık notasyon için iki grafik sembolü Şekil 3.23'te gösterilmiştir. OR-invert sembolü, NOR işlemini bir OR işlemi ve ardından bir tamamlayıcı olarak tanımlar. Invert-AND sembolü, her girişi tamamlar ve ardından bir AND işlemi gerçekleştirir. İki sembol, aynı NOR işlemini belirtir ve DeMorgan teoremi nedeniyle mantıksal olarak birbirine eşittir.



ŞEKİL 3.22

NOR kapıları ile mantık işlemleri



ŞEKİL 3.23

NOR kapısı için iki grafik sembol

NOR kapıları ile iki seviyeli bir uygulama, fonksiyonun toplamaların çarpımı formunu a basitleştirilmesini gerektirir. Basitleştirilmiş toplamaların çarpımı ifadesinin, 0'ları birleştirerek ve tamamlayarak haritadan elde edildiğini unutmayın. Toplamaların çarpımı ifadesi, toplam terimlerini üreten birinci seviyede OR kapıları ile uygulanır ve ardından ürünü üretmek için ikinci seviyede bir AND kapısı gelir. OR-AND diyagramının NOR diyagramına dönüşümü, OR kapılarını OR-invert grafik sembolleri ile NOR kapılarına ve AND kapısını invert-AND grafik sembolü ile bir NOR kapısına değiştirmekle gerçekleştirilir. İkinci seviyedeki kapağı giden tek bir terim tamamlanmalıdır. Şekil 3.24, toplamaların çarpımı olarak ifade edilen bir fonksiyonun NOR uygulamasını göstermektedir:

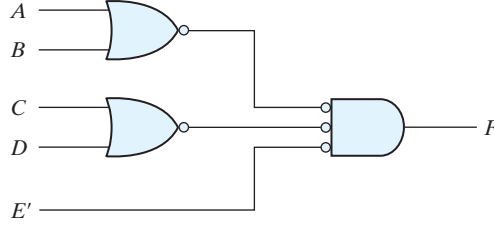
$$F = (A+B)(C+D)E$$

OR-AND deseni, aynı hat boyunca baloncukların kaldırılmasıyla kolayca tespit edilebilir. Değişken E , ikinci seviye kapının girişindeki üçüncü baloncuk için telafi edilmiştir.

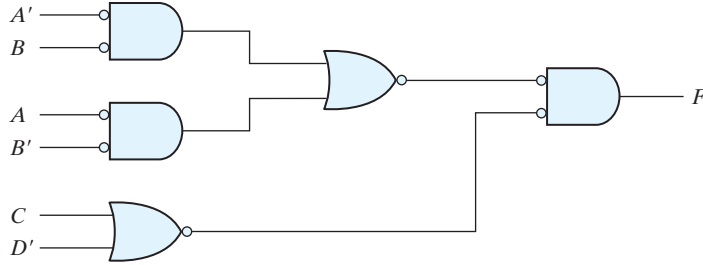
Çok seviyeli AND-OR diyagramını tamamen NOR diyagramına dönüştürme prosedürü, NAND kapıları için sunulanla benzerdir. NOR durumu için, her OR kapısını bir OR-invert sembolüne ve her AND kapısını bir invert-AND sembolüne dönüştürmemiz gerekir. Aynı hat boyunca başka bir baloncukla telafi edilmeyen her baloncuk bir inverter veya giriş literalinin tamamlanmasını gerektirir.

Şekil 3.21 (a) AND-OR diyagramının bir NOR diyagramına dönüşümü Şekil 3.25'te gösterilmiştir. Bu devre için Boolean fonksiyonu

$$F = (AB + A\bar{B})(C + D)$$



ŞEKİL 3.24

Uygulama $F=(A+B)(C+D)E$ 

ŞEKİL 3.25

NOR kapıları ile $F=(AB+A'B)(C+D)$ uygulaması

Eşdeğer AND-OR diyagramı, NOR diyagramından tüm baloncuklar kaldırılarak tanınabilir. Dört girişteki baloncukları telafi etmek için, ilgili giriş literallerinin tamamlanması gerekmektedir.

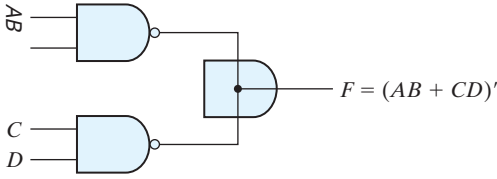
3.7 DİĞER İKİ DÜZEY UYGULAMALARI

Entegre devrelerde en sık bulunan kapı türleri NAND ve NOR kapılarıdır. Bu nedenle, NAND ve NOR mantık uygulamaları pratik açıdan en önemli olanlardır. Bazı (ama hepsi değil) NAND veya NOR kapıları, belirli bir mantık işlevi sağlamak için iki kapının çıkışları arasında bir tel bağlantısı olasılığına izin verir. Bu tür mantığa *tel mantık* denir. Örneğin, açık kollektör TTL NAND kapıları bir araya getirildiğinde, tel-AND mantığı gerçekleştirir. İki NAND kapısıyla gerçekleştirilen tel-AND mantığı Şekil 3.26 (a)'de gösterilmiştir. AND kapısı, geleneksel bir kapıdan ayırt etmek için kapının ortasından geçen çizgilerle çizilmiştir. Tel-AND kapısı fiziksel bir kapı değildir, yalnızca belirtilen tel bağlantısından elde edilen işlevi belirtmek için bir semboldür. Şekil 3.26 (a) devresi tarafından gerçekleştirilen mantık işlevi

$$F=(AB) \cdots (CD)=(AB+CD)=(A+B)(C+D)$$

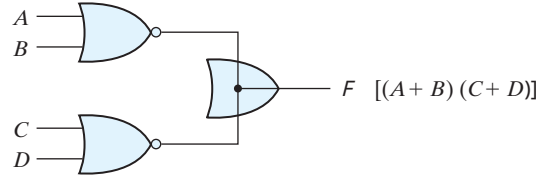
ve AND-OR-INVERT işlevi olarak adlandırılır.

98 Bölüm 3 Kapı Seviyesi Minimizasyonu



(a) Açık kolektör TTL NAND kapılarında Wired-AND.

(AND–OR–INVERT)



(b) ECL kapılarında Wired-OR

(OR–AND–INVERT)

ŞEKİL 3.26

Wired mantık

(a) İki NAND kapısıyla Wired-AND mantığı

(b) Emitter-coupled mantık (ECL) kapılarında Wired-OR

Aynı şekilde, ECL kapılarının NOR çıkışları birleştirilerek bir kablolu-VE işlevi gerçekleştirilebilir. Şekil 3.26 (b) devresinin uyguladığı mantık işlevi

$$F = (A+B) + (C+D) = [(A+B)(C+D)]$$

ve OR–AND–INVERT işlevi olarak adlandırılır.

Kablolu bir mantık kapısı fiziksel bir ikinci seviye kapı üretmez, çünkü sadece bir tel bağlantısıdır. Yine de, tartışma amaçları için, Şekil 3.26'nın devrelerini iki seviyeli uygulamalar olarak değerlendireceğiz. İlk seviye NAND (veya NOR) kapılarından oluşur ve ikinci seviye tek bir VE (veya VEYA) kapısına sahiptir. Grafik sembolündeki kablolu bağlantı sonraki tartışmalarda atlanacaktır.

Degeneres Olmayan Şekiller

Teorik açıdan, kaç tane iki seviyeli kapı kombinasyonunun mümkün olduğunu bulmak öğretici olacaktır. Dört tür kapıyı dikkate alıyoruz: VE, VEYA, NAND ve NOR. İlk seviye için bir tür kapı ve ikinci seviye için bir tür kapı atadığımızda, iki seviyeli formların 16 olası kombinasyonu olduğunu buluyoruz. (Aynı tür kapı, NAND–NAND uygulamasında olduğu gibi, birinci ve ikinci seviyelerde bulunabilir.) Bu kombinasyonların sekizi, tek bir işleme dönüşmeleri nedeniyle degeneres formlar olarak adlandırılmaktadır. Bu, birinci seviyede VE kapıları ve ikinci seviyede bir VE kapısı olan bir devreden görülebilir. Devrenin çıkışı, yalnızca tüm giriş değişkenlerinin VE işlevidir.

Kalan sekiz *degenerate olmayan* form, toplam ürün

formu veya toplamların çarpımı formunda bir uygulama üretir. Sekiz *degenerate olmayan* form şunlardır:

AND–OR	OR–AND
NAND–NAND	NOR–NOR
NOR–OR	NAND–AND
OR–NAND	AND–NOR

Her formda listelenen ilk kapı, uygulamanın birinci seviyesini oluşturur.

İkinci listedeki kapı, ikinci seviyede yer alan tek bir kapıdır. Aynı satırda listelenen herhangi iki formun birbirinin ikizi olduğunu unutmayın.

AND-OR ve OR-AND formları, Bölüm 3.4'te tartışılan temel iki seviyeli formlardır. NAND-NAND ve NOR-NOR formları Bölüm 3.5'te sunulmuştur.

Kalan dört form bu bölümde incelenmektedir.

AND-OR-İNVERT Uygulaması

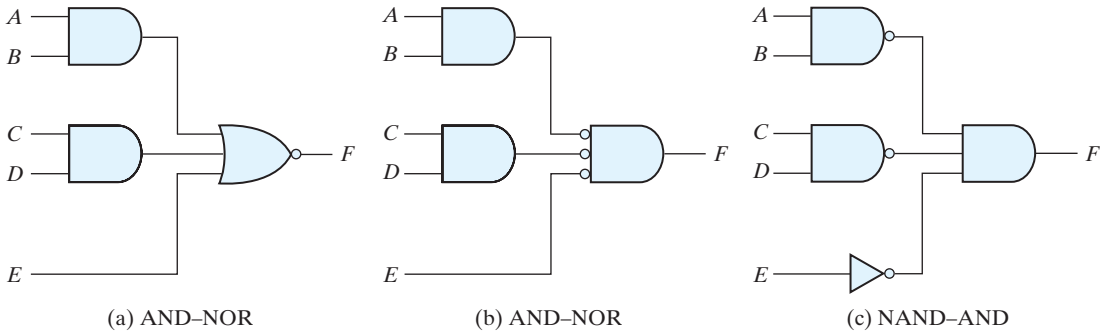
NAND-AND ve AND-NOR formları eşdeğerdir ve birlikte ele alınabilir. Her ikisi de AND-OR-İNVERT işlevini yerine getirir, bu Fig. 3.27'de gösterilmiştir. AND-NOR formu, AND-OR formuna benzer, ancak NOR kapısının çıkışındaki baloncuk tarafından bir tersine çevirme yapılmıştır. Bu işlevi uygular

$$F=(AB+CD+E)$$

NOR kapısı için alternatif grafik sembolü kullanarak, Fig. 3.27 (b) diyagramını elde ederiz. Tek değişken E tamamlanmamıştır, çünkü yapılan tek değişiklik NOR kapısının grafik sembolündedir. Şimdi baloncuklu ikinci seviye kapının girişi terminalinden birinci seviye kapıların çıkış terminallerine taşıyoruz. Baloncuk için tek değişkenin telafisi için bir inverter gereklidir.

Alternatif olarak, inverter kaldırılabilir, sağlanmak kaydıyla giriş E tamamlanmıştır. Fig. 3.27 (c) devresi bir NAND-AND formudur ve AND-OR-İNVERT işlevini uygulamak için Fig. 3.26'da gösterilmiştir.

Bir AND-OR uygulaması, ürünlerin toplamı biçiminde bir ifade gerektirir. AND-OR-İNVERT uygulaması benzer olup, tek farkı terslemedir. Bu nedenle, eğer *fonksiyonun tamamlayıcı* (haritadaki 0'ları birleştirerek) ürünlerin toplamı biçimi ne sadeleştirilirse, F fonksiyonun AND-OR kısmı ile uygulanabilir hale gelecektir. F her zaman mevcut olan çıkış terslemesinden (INVERT kısmı) geçtiğinde,



ŞEKİL 3.27

AND-OR-İNVERT devreleri, $F=(AB+CD+E)$

fonksiyonun çıkışını F üretir. AND-OR-INVERT uygulaması için bir örnek daha sonra gösterilecektir.

OR-AND-INVERT Uygulaması

OR-NAND ve NOR-OR formları, Şekil 3.28'de gösterildiği gibi, OR-AND-INVERT işlevi ni yerine getirir. OR-NAND formu, NAND kapısındaki baloncuk tarafından yapılan tersine çevirmeden başka, OR-AND formuna benzer. Bu işlevi uygular

$$F = [(A+B)(C+D)E]$$

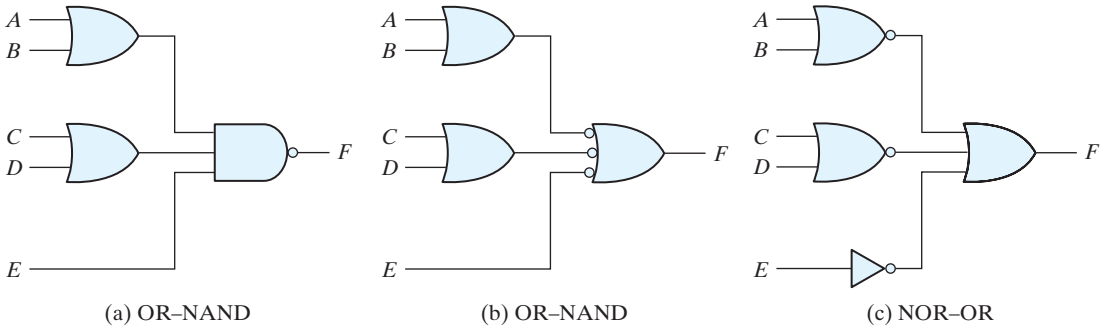
NAND kapısı için alternatif grafik sembolünü kullanarak, Şekil 3.28 (b) diyagramını elde ederiz. Şekil 3.28(c) 'deki devre, ikinci seviye kapının girişlerinden küçük dairelerin birinci seviye kapıların çıkışlarına taşınmasıyla elde edilmiştir. Şekil'in devresi 3.28 (c) NOR-OR formudur ve OR-AND-

INVERT fonksiyonunu uygulamak için Şekil 3.26'da gösterilmiştir.

OR-AND-INVERT uygulaması, toplamaların çarpımı formunda bir ifade gerektirir. Fonksiyonun tamamlayıcısı bu forma basitleştirildiğinde, F fonksiyonun OR-AND kısmı ile uygulanabilir. F INVERT kısmından geçtiğinde, çıktıda F 'nin tamamlayıcısını veya F 'yi elde ederiz.

Tablolarla Özet ve Örnek

Tablo 3.2, bir Boolean fonksiyonunu dört 2 seviyeli formdan herhangi birinde uygulama prosedürlerini özetlemektedir. Her durumda INVERT kısmı nedeniyle, fonksiyonun tamamlayıcısı olan F 'nin basitleştirilmesi kullanışlıdır. F bu formlardan birinde uygulandığında, AND-OR veya OR-AND formunda fonksiyonun tamamlayıcısını elde ederiz. Dört 2 seviyeli form bu fonksiyonu tersine çevirir ve çıktıda F 'nin tamamlayıcısını verir. Bu, normal çıktı F 'dir.



ŞEKİL 3.28

OR-AND-INVERT devreleri, $F = [(A+B)(C+D)E]$

Tablo 3.2
Diğer İki Düzeyli Biçimlerle Uygulama

Eşdeğer Degeneres Olmayan Form		Uygular the Fonksiyon	Sadeleştir F için	Bir Çık tı Almak iç in
(a)	(b)*			
AND-NOR	NAND-AND	AND-OR-INVERT	0'ları haritada birleştirerek ürünlerin toplamı formu.	F
OR-NAND	NOR-OR	OR-AND-INVERT	1'leri haritada birleştirerek ve ardından tamamlayarak ürünlerin çarpımı formu.	F

* Form (b), tek bir terim için bir inverter gerektirir.

ÖRNEK 3.10

Şekil 3.29 (a) fonksiyonunu, Tablo 3.2'de listelenen dört 2 seviyeli form ile uygulayın. Fonksiyonun tamamlayıcı, haritadaki 0'ları birleştirerek ürünlerin toplamı formuna sadeleştirilmiştir:

$$F' = x'y + xy' + z$$

Bu fonksiyonun normal çıktısı şu şekilde ifade edilebilir:

$$F = (x + xy + z)$$

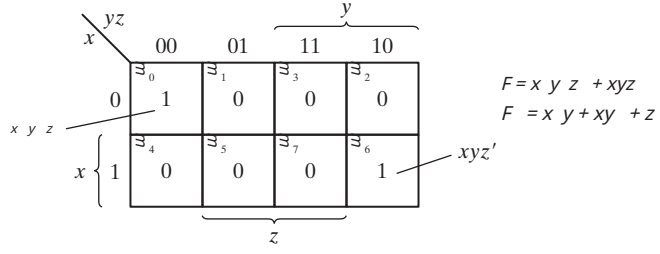
AND-VE-İNVERT formundadır. AND-NOR ve NAND-AND uygulamaları Şekil 3.29 (b)'de gösterilmiştir. NAND-AND uygulamasında bir girişli NAND veya inverter kapısına ihtiyaç vardır, ancak AND-NOR durumunda yoktur. Giriş değişkeni z yerine z uygulandığında inverter kaldırılabilir.

OR-AND-İNVERT formları, fonksiyonun toplamların çarpımı formundaki tamamlayıcısını basitleştirilmiş bir ifadesini gerektirir. Bu ifadeyi elde etmek için önce haritadaki 1'leri birleştiririz:

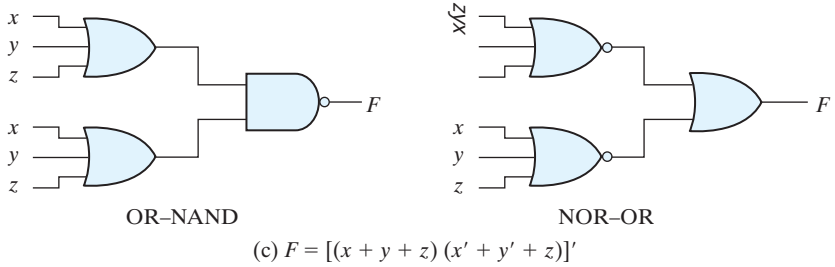
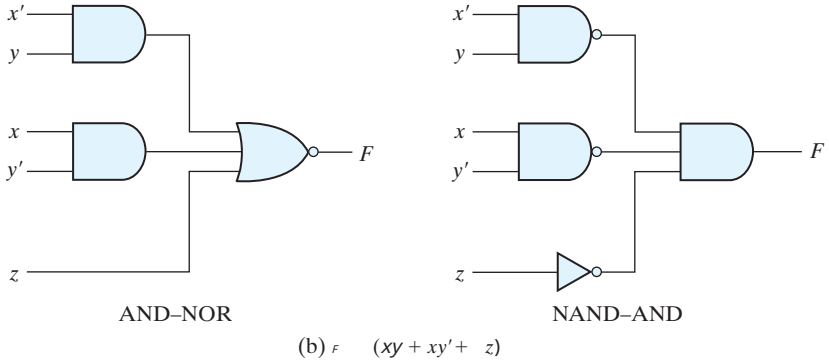
$$F = x'y'z' + xyz'$$

Daha sonra fonksiyonun tamamlayıcısını alırız:

$$F' = (x + y + z)(x' + y' + z)$$



(a) Ürünlerin toplamında harita basitleştirmesi



ŞEKİL 3.29
 Diğer iki seviyeli uygulamalar

Normal çıkış F artık şu formda ifade edilebilir:

$$F = [(x + y + z)(x' + y' + z)]$$

bu OR-AND-İNVERT formudur. Bu ifadeden, fonksiyonu OR-NAND ve NOR-OR formlarında uygulayabiliriz, Şekil 3.29 (c)'de gösterildiği gibi.

3.8 ÖZEL-VE FONKSİYONU

Özel-VE (XOR), sembolü ile gösterilen $\oplus$, aşağıdaki Boole işlemini gerçekleştir en bir mantıksal işlemdir:

$$x \oplus y = xy + x'y$$

Özel-VE, yalnızca x 1'e eşit olduğunda veya yalnızca y 1'e eşit olduğunda (yani, x ve y değer olarak farklı olduğunda) 1'e eşittir, ancak her ikisi de 1'e eşit olduğunda veya her ikisi de 0'a eşit olduğunda değildir. Özel-VE DEĞİL, eşdeğerlik olarak da bilinir, aşağıdaki Boole işlemini gerçekleştir:

$$(x \oplus y)' = xy + x'y'$$

Özel-VE DEĞİL, her iki x ve y 1'e eşit olduğunda veya her ikisi de 0'a eşit olduğunda 1'e eşittir. Özel-VE DEĞİL'in, bir doğruluk tablosu veya cebirsel manipülasyon yoluyla özel-VE'nin tamamlayıcısı olduğu gösterilebilir:

$$(x \oplus y)' = (xy + x'y)' = (x+y)(x+y)' = xy + x'y$$

Aşağıdaki kimlikler özel-VE işlemi için geçerlidir:

$$x \oplus 0 = x$$

$$x \oplus 1 = x'$$

$$x \oplus x = 0$$

$$x \oplus x' = 1$$

$$x \oplus y' = x' \oplus y = (x \oplus y)'$$

Bu kimliklerin herhangi biri, bir doğruluk tablosu ile veya $\oplus$ işlemi eşdeğer Boole an ifadesi ile değiştirilerek kanıtlanabilir. Ayrıca, özel-VE işleminin hem değişmeli hem de birleşmeli olduğu gösterilebilir; yani,

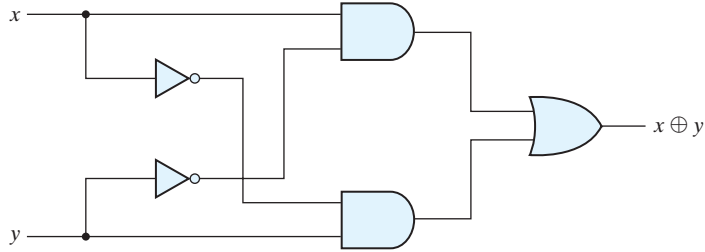
$$A \oplus B = B \oplus A$$

and

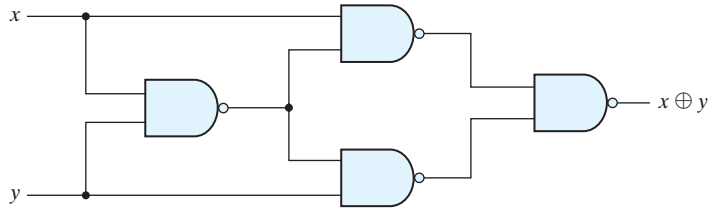
$$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$$

Bu, bir özel-VE kapısına iki girişin, işlemi etkilemeden yer değiştirebileceği anlamına gelir. Ayrıca, üç değişkenli bir özel-VE işlemini herhangi bir sırayla değerlendirebileceğimiz anlamına gelir ve bu nedenle, üç veya daha fazla değişken parantezsiz ifade edilebilir. Bu, üç veya daha fazla girişe sahip özel-VE kapılarının kullanılabilirliğini ima eder. Ancak, çoklu girişli özel-VE kapıları donanım ile üretmek zordur. Aslında, iki girişli bir fonksiyon genellikle diğer kapı türleri ile inşa edilir. İki girişli bir özel-VE fonksiyonu, iki inverter, iki VE kapısı ve bir VE kapısı kullanılarak geleneksel kapılarla inşa edilir; bu, Şekil 3.30 (a)'da gösterilmiştir. Şekil 3.30 (b), dört NAND kapısı ile özel-VE'nin uygulanmasını göstermektedir. İlk NAND kapısı, işlemi gerçekleştirir $(xy)' = (x+y)$. Diğer iki seviyeli NAND devresi, girişlerinin çarpımlarının toplamını üretir:

$$(x' + y')x + (x' + y')y = xy' + x'y = x \oplus y$$



(a) VE-VEYA-DEĞİL kapıları ile özel-VE



(b) NAND kapıları ile Özel-VE

ŞEKİL 3.30
Özel-VE uygulamaları

Sadece sınırlı sayıda Boole fonksiyonu, özel-VE işlemleri cinsinden ifade edilebilir. Bununla birlikte, bu fonksiyon dijital sistemlerin tasarımında oldukça sık ortaya çıkar. Özellikle aritmetik işlemler ve hata tespiti ile düzeltme devrelerinde oldukça faydalıdır.

Tekil Fonksiyon

Üç veya daha fazla değişkenle yapılan özel-VE işlemi, $\oplus$ sembolü, eşdeğer Boole ifadesi ile değiştirilerek sıradan bir Boole fonksiyonuna dönüştürülebilir. Özellikle, üç değişkenli durum, aşağıdaki gibi bir Boole ifadesine dönüştürülebilir:

$$\begin{aligned} A \oplus B \oplus C &= (AB + A\bar{B}C + (AB + A\bar{B})\bar{C}) \\ &= AB \oplus A\bar{B}C \oplus ABC + A\bar{B}\bar{C} \\ &= (1, 2, 4, 7) \end{aligned}$$

Boole ifadesi, üç değişkenli özel-VE fonksiyonunun yalnızca bir değişken 1'e eşit olduğunda 1'e eşit olduğunda 1'e eşit olduğunu açıkça göstermektedir. İki değişkenli durumda yalnızca bir değişkenin 1'e eşit olması gerektiğinin aksine, üç veya daha fazla değişken durumunda, 1'e eşit olan değişkenlerin tek sayıda olması gerekmektedir. Sonuç olarak, çok değişkenli özel-VE işlemi bir *tekil fonksiyon olarak tanımlanır*.

Üç değişkenli özel-VE işlemi ile türetilen Boole fonksiyonu, dört mintermi mantıksal toplamı olarak ifade edilir; bu mintermlerin ikili sayısal değerleri 001, 010, 100 ve 111'dir. Bu ikili sayıların her birinde tek sayıda 1 vardır. Fonksiyona dahil edilmeyen diğer dört minterm ise 000, 011, 101 ve 110'dur ve bunların ikili sayısal değerlerinde çift sayıda 1 vardır. Genel olarak, n -değişkenli özel-VE fonksiyonu, ikili sayısal değerlerinde tek sayıda 1 bulunan $2^n/2$ mintermlerin mantıksal toplamı olarak tanımlanan bir tek fonksiyondur.

Tek fonksiyonun tanımı, bir haritada çizilerek netleştirilebilir. Şekil 3.31 (a) üç değişkenli özel-VE fonksiyonu için haritayı göstermektedir. Fonksiyonun dört mintermi birbirinden bir birim mesafededir. Tek fonksiyon, ikili değerlerinde tek sayıda 1 bulunan dört mintermdan tanımlanır. Bir tek fonksiyonun tamamlayıcı, bir çift fonksiyondur. Şekil 3.31 (b)'de gösterildiği gibi, üç değişkenli çift fonksiyon, değişkenlerinin çift sayıda 1 olduğu durumlarda (hiçbir değişkenin 1 olmadığı durumu da dahil) 1'e eşittir.

Üç girişli tek fonksiyon, Şekil 3.32 (a)'da gösterildiği gibi iki girişli özel-VE kapıları aracılığıyla uygulanmaktadır. Tek bir fonksiyonun tamamlayıcı, çıkış kapısını özel-VE DEĞİL kapısı ile değiştirmek suretiyle elde edilir; bu, Şekil 3.32 (b)'de gösterilmiştir.

Şimdi dört değişkenli özel-VE işlemi ele alalım. Cebirsel manipülasyon ile bu fonksiyonun mintermlerin toplamını elde edebiliriz:

$$\begin{aligned} A \oplus B \oplus C \oplus D &= (AB + A\bar{B} + \bar{A}B + \bar{A}\bar{B}) \oplus (CD + C\bar{D} + \bar{C}D + \bar{C}\bar{D}) \\ &= (AB + A\bar{B} + \bar{A}B + \bar{A}\bar{B})(CD + C\bar{D} + \bar{C}D + \bar{C}\bar{D}) + (\bar{A}B + \bar{A}\bar{B})(CD + C\bar{D} + \bar{C}D + \bar{C}\bar{D}) \\ &= (1, 2, 4, 7, 8, 11, 13, 14) \end{aligned}$$

BC		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
			1		1
		1		1	

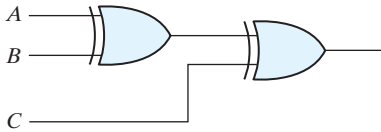
(a) Tek fonksiyon $F = A \oplus B \oplus C$

BC		B			
		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6
		1			
			1		
				1	
					1

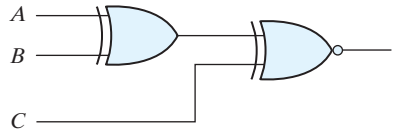
(b) Çift fonksiyon $F = (A \oplus B \oplus C)'$

ŞEKİL 3.31

Üç değişkenli özel-VE fonksiyonu için harita



(a) 3-girişli tek fonksiyon



(b) 3-girişli çift fonksiyon

ŞEKİL 3.32

Tek ve çift fonksiyonların mantık diyagramı

AB \ CD		C			
		00	01	11	10
A	00	m_0	Σ_1 1	Σ_3	Σ_2 1
	01	m_4 1	Σ_5	Σ_7 1	Σ_6
	11	Σ_{12}	Σ_{13} 1	Σ_{15}	Σ_{14} 1
	10	Σ_8 1	Σ_9	Σ_{11} 1	Σ_{10}

(a) Tek fonksiyon $F = A \oplus B \oplus C \oplus D$

AB \ CD		C			
		00	01	11	10
A	00	Σ_0 1	Σ_1	Σ_3 1	Σ_2
	01	Σ_4	Σ_5 1	Σ_7	Σ_6 1
	11	Σ_{12} 1	Σ_{13}	Σ_{15} 1	Σ_{14}
	10	Σ_8	Σ_9 1	Σ_{11}	Σ_{10} 1

(b) Çift fonksiyon $F = (A \oplus B \oplus C \oplus D)'$ **ŞEKİL 3.33**

Dört değişkenli özel-VEYA fonksiyonu için harita

Dört değişkenli bir Boole fonksiyonu için 16 mintermdir. Mintermlerin yarısı, binari sayısal değerleri tek sayıda 1'e sahiptir; diğer yarısı ise binari sayısal değerleri çift sayıda 1'e sahiptir. Fonksiyonu harita üzerinde çizerken, bir minterm için binari sayısal değer, minterm'i temsil eden karenin satır ve sütun numaralarından belirlenir. Şekil 3.33 (a) dört değişkenli özel-VE fonksiyonunun bir grafiğidir. Bu, tüm mintermlerin binari değerlerinin tek sayıda 1'e sahip olması nedeniyle bir tek fonksiyondur. Tek bir fonksiyonun tamamlayıcı, bir çift fonksiyondur. Şekil 3.33 (b)'de gösterildiği gibi, dört değişkenli çift fonksiyon, değişkenlerinin çift sayıda 1'e eşit olduğu durumlarda 1'e eşittir.

Parity Generation and Checking

Özel-VE işlevleri, hata tespiti ve düzeltme kodları gerektiren sistemlerde çok faydalıdır. Bölüm 1.6'da tartışıldığı gibi, bir parite biti, ikili bilginin iletimi sırasında hataları tespit etmek amacıyla kullanılır. Parite biti, 1'lerin sayısını ya tek ya da çift yapmak için ikili bir mesajla birlikte eklenen bir ekstra bittir. Parite bitini de içeren mesaj iletilir ve ardından alıcı tarafta hatalar için kontrol edilir. Kontrol edilen parite, iletilen parite ile uyuşmuyorsa bir hata tespit edilir. Verici tarafında parite bitini üreten devreye parite üretici denir. Alıcı tarafında pariteyi kontrol eden devreye ise parite kontrol cihazı denir.

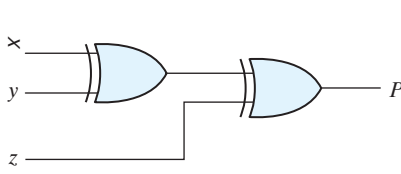
kontrolcü.

Örnek olarak, bir üç bitlik mesajın bir even-parity bit ile birlikte iletilmesini düşünelim. Tablo 3.3, parite üretici için doğruluk tablosunu göstermektedir. Üç bit x, y, z — mesajı oluşturur ve devreye girişlerdir. Parite bit P çıktıdır. Çift parite için, bit P toplam 1'lerin (P dahil) sayısını çift yapmak için üretilmelidir. Doğruluk tablosundan, P bir

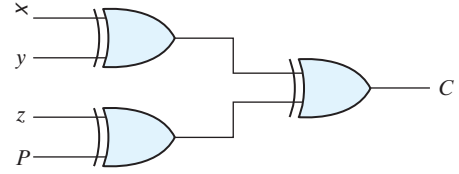
Tablo 3.3

Çift Parite Üreteci Doğruluk Tablosu

Üç Bitlik Mesaj			Parite Bit
x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



(a) 3 bit çift parite üreteci



(b) 4 bit çift parite kontrolcüsü

ŞEKİL 3.34

Bir parite üreteci ve kontrolcüsünün mantık diyagramı

tek fonksiyon çünkü sayısal değerleri tek sayıda 1'e sahip olan mintermler için 1'e eşittir. Bu nedenle, P üç değişkenli özel-VEYA fonksiyonu olarak ifade edilebilir:

$$P = x \oplus y \oplus z$$

Parity üreteci için mantık diyagramı Şekil 3.34 (a)'da gösterilmiştir.

Mesajdaki üç bit, parite biti ile birlikte, hedeflerine iletilir; burada, iletimde olası hataları kontrol etmek için bir parite kontrol devresine uygulanır. Bilgiler çift parite ile iletilendiğinden, alınan dört bitin de çift sayıda 1 içermesi gerekir. Alınan dört bitin tek sayıda 1 içermesi durumunda bir hata meydana gelir; bu, iletim sırasında bir bitin değiştiğini gösterir. Parite kontrol cihazının çıktısı, bir hata meydana gelirse, yani alınan dört bitin tek sayıda 1 içermesi durumunda, C ile gösterilecektir.

Tablo 3.4, çift parite kontrol cihazı için doğruluk tablosudur. Buradan, C fonksiyonunun tek sayıda 1 içeren sekiz mintermden oluştuğunu görmekteyiz. Tablo, Şekil 3.33 (a) haritasına karşılık gelmektedir.

Tablo 3.4
Çift Parite Kontrol Tablosu

Dört Bit Alındı				Parite Hatası Kontrolü
x	y	z	w	c
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

tek bir fonksiyonu temsil eder. Parite kontrolörü, özel-

VE kapıları ile uygulanabilir:

$$C = x \oplus y \oplus z \oplus P$$

Parite kontrolörünün mantık diyagramı Şekil 3.34 (b)'de gösterilmiştir.

Giriş P mantık 0'a bağlandığında ve çıkış P ile işaretlendiğinde, parite üretici Şekil 3.34 (b) devresi ile uygulanabilir. Bunun nedeni, $z \oplus 0 = z$ olmasıdır, bu da z değerinin kapıdan değişmeden geçmesini sağlar. Bu stratejinin avantajı, aynı d evrenin hem parite üretimi hem de kontrolü için kullanılabilmesidir.

Önceki örnekten açıkça görülmektedir ki, parite üretimi ve kontrol devreleri her zaman, sayısal değerleri tek veya çift sayıda 1 olan mintermlerin yarısını içeren bir çıkış fonksiyonuna sahiptir. Sonuç olarak, özel-VE kapıları ile uygulanabilirler. Çift sayıda 1'e sahip bir fonksiyon, tek bir fonksiyonun tamamlayıcısıdır. Bu, çıkışla ilişkili kapının gerekli tamamlamayı sağlamak için özel-NOR olması dışında, özel-VE kapıları ile uygulanır.

3.9 DONANIM TANIM DİLİ

Mantık devrelerini tasarlamak için manuel yöntemler yalnızca devre küçük olduğunda uygulanabilir. Başka bir şey için (yani, pratik bir devre), tasarımcılar bilgisayar tabanlı tasarım araçları kullanır. Doğru yapılandırma metodolojisi ile birleştirildiğinde, bilgisayar tabanlı tasarım araçları

raticılığını ve çabasını kullanarak hatalı bir tasarım üretme riskini azaltır. Prototip entegre devreler çok pahalı ve zaman alıcıdır, bu nedenle tüm modern tasarım araçları, bir devreyi üretmeden önce yazılımda tanımlamak, tasarlamak ve test etmek için bir donanım tanım diline (HDL) dayanır.

Bir *donanım tanım dili* (HDL), dijital sistemlerin donanımını metin biçiminde tanımlayan bilgisayar tabanlı bir dildir. C gibi sıradan bir bilgisayar programlama diline benzer, ancak özellikle donanım yapıları ve mantık devrelerinin davranışını tanımlamak için yönlendirilmiştir. Mantık diyagramlarını, doğruluk tablolarını, Boolean ifadelerini ve bir dijital sistemin davranışının karmaşık soyutlamalarını temsil etmek için kullanılabilir. Bir HDL'yi gözlemlemenin bir yolu, devreye giriş sinyalleri ile devreden çıkış sinyalleri arasındaki ilişkiyi tanımladığını gözlemlemektir. Örneğin, bir AND kapısının HDL tanımı, kapının çıkışının mantık değerinin, girişlerini n mantık değerleri tarafından nasıl belirlendiğini açıklar.

Bir *belgelendirme* dili olarak, HDL, dijital sistemleri hem insanlar hem de bilgisayarlar tarafından okunabilir bir biçimde temsil etmek ve belgelemek için kullanılır ve tasarımcılar arasında bir değişim dili olarak uygundur. Dil içeriği kolayca depolanabilir, alınabilir, düzenlenebilir ve iletilebilir ve bilgisayar yazılımları tarafından verimli bir şekilde işlenebilir.

HDL'ler, entegre devre tasarım akışında birkaç ana adımda kullanılır:

tasarım girişi, fonksiyonel simülasyon veya doğrulama, mantık sentezi, zamanlama doğrulaması ve hata simülasyonu.

Tasarım girişi donanımda uygulanacak işlevselliğin HDL tabanlı bir tanımını oluşturur. HDL'ye bağlı olarak, tanım çeşitli biçimlerde olabilir: Boolean mantık denklemleri, doğruluk tabloları, birbirine bağlı kapıların bir ağ listesi veya soyut bir davranış modeli. HDL modeli ayrıca daha büyük bir devrenin daha küçük, birbirine bağlı ve etkil eşimde bulunan işlevsel birimlere bölünmesini temsil edebilir.

Mantık simülasyonu bir dijital sistemin davranışını bir bilgisayar kullanarak gösterir.

Bir simülatör, HDL tanımını yorumlar ve ya okunabilir bir çıktı üretir, örneğin giriş ve çıkış sinyal değerlerinin zaman sırasına göre sıralanmış bir dizisi, ya da sinyallerin dalga formlarını gösterir. Bir devrenin simülasyonu, donanımın gerçekten üretilmeden önce nasıl davranacağını tahmin eder. Simülasyon, bir tasarımda fiziksel olarak devreyi oluşturup çalıştırmadan işlevsel hataları tespit eder. Simülasyon sırasında tespit edilen hatalar, uygun HDL ifadelerini değiştirerek düzeltiler. Tasarımın işlevselliğini test eden uyandırıcı (yani, bir devreye girişlerin mantık değerleri) bir *test tezgahı* olarak adlandırılır. Böylece, bir dijital sistemi simüle etmek için, tasarım önce bir HDL'de tanımlanır ve ardından tasarım simüle edilerek ve HDL'de yazılmış bir test tezgahı ile kontrol edilerek doğrulanır. Alternatif ve daha karmaşık bir yaklaşım, bir devrenin işlevsel olarak doğru olduğunu kanıtlamak için resmi matematiksel yöntemlere dayanır. Biz yalnızca simülasyona odaklanacağız.

Mantık sentezi fiziksel bileşenler ve bunların bağlantılarını (bir *netlist* olarak adlandırılır) bir dijital sistemin modelinden türetme sürecidir. Netlist, bir entegre devre üretmek veya listedeki kapıların donanım karşılıkları ile bir baskılı devre kartı düzenlemek için kullanılabilir. Mantık sentezi, geleneksel bir yüksek seviye dilde bir program derlemeye benzer. Fark,

bir nesne kodu üretmek yerine, mantık sentezinin bir devrenin elemanlarını ve yapısını tanımlayan bir veritabanı üretmesidir. Veritabanı, bir entegre devreyi fiziksel olarak üretmek için nasıl bir yol izleneceğini belirtir; bu devre, bir HDL'de yapılan ifadelerle tanımlanan işlevselliği silik on üzerinde gerçekleştirir. Mantık sentezi, dijital devreleri uygulayan ve bilgisayar yazılımı ile otomatikleştirilebilecek dijital tasarımın o kısmını ele alan resmi kesin prosedürlere dayanır. Bunun büyük, karmaşık devrelerinin tasarımı, mantık sentezi yazılımı sayesinde mümkün olmaktadır.

Zamanlama doğrulaması üretilen entegre devrenin belirli bir hızda çalışacağını doğrular. Bir devredeki her mantık kapısının bir yayılma gecikmesi olduğundan, bir devrenin girişindeki bir sinyal geçişi, devrenin çıkışındaki mantık değerinde hemen bir değişiklik yaratamaz. Yayılma gecikmeleri, nihayetinde bir devrenin çalışabileceği hızı sınırlar. Zamanlama doğrulaması, her sinyal yolunu kontrol ederek yayılma gecikmesi tarafından tehlikeye atılmadığını doğrular. Bu adım, mantık sentezi devreyi oluşturacak gerçek cihazları belirledikten sonra ve devre üretime verilmeden önce yapılır.

VLSI devre tasarımında, *arıza simülasyonu* ideal bir devrenin davranışını, bir süreç kaynaklı hataya sahip bir devrenin davranışıyla karşılaştırır. Temiz odanın atmosferindeki toz ve diğer parçacıklar, bir devrenin hatalı olarak üretilmesine neden olabilir. Hatalı bir devre, hatasız bir devreyle aynı işlevselliği göstermez. Arıza simülasyonu, hatalı devre ile hatasız devre arasındaki farkı ortaya çıkarmak için kullanılacak giriş uyarılarını tanımlamak için kullanılır. Bu test desenleri, yalnızca iyi cihazların müşteriye gönderildiğinden emin olmak için üretilen cihazları test etmek için kullanılacaktır. Test üretimi ve arıza simülasyonu, tasarım sürecinin farklı adımlarında gerçekleşebilir, ancak her zaman üretimden önce yapılır, böylece iç mantığı test edilemeyen bir devre üretme felaketinden kaçınılır.

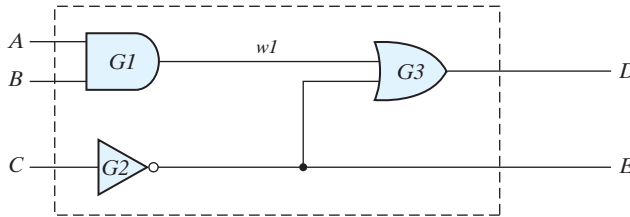
Entegre devre tasarlayan şirketler, özel ve kamuya açık HDL'ler kullanır. Kamu alanında, IEEE tarafından desteklenen iki standart HDL vardır: VHDL ve Verilog. VHDL, Savunma Bakanlığı tarafından zorunlu kılınan bir dildir. (VHDL'deki *V*, çok yüksek hızlı entegre devre anlamına gelen VHSIC kısaltmasındaki ilk harfi temsil eder.) Verilog, Cadence Design Systems'in özel bir HDL'si olarak başladı, ancak Cadence, Verilog'un kontrolünü Open Verilog International (OVI) olarak bilinen bir şirketler ve üniversiteler konsorsiyumuna devretti; bu, IEEE standardı olarak benimsenmesine giden bir adım olarak kabul edilir. VHDL, Verilog'dan daha zor öğrenilmektedir. Çünkü Verilog, VHDL'den daha kolay bir dil olup, tanımlamak, öğrenmek ve kullanmak için bu kitabı yazarken onu seçtik. Ancak, kitap boyunca listelenen Verilog HDL tanımları sadece Verilog ile ilgili değil, aynı zamanda tipik bir donanım tanım dili aracılığıyla dijital sistemlerin bilgisayar destekli modelleme kavramına dayanan bir tasarım metodolojisini tanıtmak için de hizmet etmektedir. Vurgumuz, belirli bir davranışa sahip devrelerin Verilog modellerinin modelleme, doğrulama ve sentezine (hem manuel hem de otomatik) olacaktır. Verilog HDL, başlangıçta 1995 yılında standart bir HDL olarak onaylanmıştır; dilin gözden geçirilmiş ve geliştirilmiş sürümleri 2001 ve 2005 yıllarında onaylanmıştır. Entegre devreler için HDL tabanlı tasarım metodolojisi tartışmamızı destekleyen, en son standart dahil olmak üzere, yalnızca Verilog'un bu özelliklerini ele alacağız.

Modül Bildirimi

Verilog HDL için dil referans kılavuzu, dilde kullanılabilecek yapıları kesin bir şekilde tanımlayan bir sözdizimi sunmaktadır. Özellikle, bir Verilog modeli, yaklaşık 100 adet anahtar kelime içeren metinlerden oluşur. Anahtar kelimeler, dil yapılarını tanımlayan önceden tanımlanmış küçük harfli tanımlayıcılardır. Anahtar kelimelere örnekler: **module**, **end module**, **input**, **output**, **wire**, **and**, **or**, ve **not**. Açıklık için, anahtar kelimeler, kod örneklerinde ve kullanımına dikkat çekmek için uygun olan her yerde metin içinde kalın yazı ile gösterilecektir. İki ileri eğik çizgi (//) ve satır sonu arasındaki herhangi bir metin yorum olarak yorumlanır ve model kullanılarak yapılan bir simülasyonda hiçbir etkisi olmayacaktır. Çok satırlı yorumlar / * ile başlar ve * / ile sona erer. Boşluklar göz ardı edilir, ancak bir anahtar kelimenin, kullanıcı tarafından belirtilen bir tanımlayıcının, bir operatörün veya bir sayının temsilinin metni içinde yer alamazlar. Verilog büyük/küçük harf duyarlıdır, bu da büyük ve küçük harflerin ayırt edilebilir olduğu anlamına gelir (örneğin, **not** NOT ile aynı değildir). **module** terimi, **module**... **endmodule** anahtar kelime çifti ile çevrelenmiş metni ifade eder. Bir modül, Verilog dilindeki temel tanımlayıcı birimdir. **module** anahtar kelimesi ile bildirilir ve her zaman **endmodule** anahtar kelimesi ile sonlandırılmalıdır.

Kombinasyonel mantık, kapıların şematik bağlantısı, bir dizi Boolean denklemi veya bir doğruluk tablosu ile tanımlanabilir. Her tanım türü Verilog'da geliştirilebilir. Dili anlamak için bazı yönleri göstermek amacıyla, basit bir Verilog kapı düzeyinde tanım örneği ile başlayarak her stili göstereceğiz.

Şekil 3.35'in devresinin HDL tanımı HDL Örneği 3.1'de gösterilmiştir. Metnin ilk satırı, okuyucuya faydalı bilgiler sağlayan bir yorumdur (isteğe bağlı). İkinci satır, **module** anahtar kelimesi ile başlar ve modülün tanımını (açıklamasını) başlatır; son satır, **endmodule** anahtar kelimesi ile tanımlı tamamlar. **module** anahtar kelimesini bir isim ve bir port listesi takip eder. İsim (bu örnekte *Simple_Circuit*) bir tanımlayıcıdır. Tanımlayıcılar, modüllere, değişkenlere (örneğin, bir sinyal) ve dilin diğer unsurlarına atıfta bulunulabilmesi için verilen isimlerdir. Genel olarak, modüller için anlamlı isimler seçeriz. Tanımlayıcılar, alfanümerik karakterler ve alt çizgi (_) ile oluşur ve büyük/küçük harf duyarlıdır. Tanımlayıcılar, bir alfabetik karakter veya bir alt çizgi ile başlamalıdır, ancak bir sayı ile başlayamazlar.



ŞEKİL 3.35
HDL'yi göstermek için devre

HDL Örneği 3.1 (Temel Bileşenlerle Modellenmiş Kombinasyonel Mantık)

// Şekil 3.35'in devresinin Verilog modeli. IEEE 1364–1995 Söz Dizimi

```

modül Simple_Circuit (A, B, C, D, E);
    çıkış      E; D;
    giriş      A, B, C;
    tel        w1;

    ve          G1 (w1, A, B); // Opsiyonel kapı örneği adı
    değil       G2 (E, C);
    veya        G3 (D, w1, E);
endmodule

```

Modülün *port listesi* modül ile çevresi arasındaki arayüzdür.

Bu örnekte, portlar devrenin girişleri ve çıkışlarıdır. Bir devreye giden girişlerin mantık değerleri çevre tarafından belirlenir; çıkışların mantık değerleri ise devre içinde belirlenir ve girişlerin devre üzerindeki etkisinden kaynaklanır.

Port listesi parantez içinde yer alır ve listedeki öğeleri ayırmak için virgüller kullanılır. İfade de bir noktalı virgül (;) ile sonlandırılır. Örneklerimizde, tüm anahtar kelimeler (küçük harfle yazılmalıdır) netlik için kalın yazılmıştır, ancak bu dilin bir gerekliliği değildir. Sonraki anahtar kelimeler **input** ve **output** hangi portların giriş ve hangi portların çıkış olduğunu belirtir. Dahili bağlantılar tel olarak tanımlanır. Bu örnekteki devrede bir dahili bağlantı vardır, *w1* terminalinde ve *wire* anahtar kelimesi ile tanımlanmıştır. Devrenin yapısı, her biri tanımlayıcı bir anahtar kelime (**and**, **not**, **or**) ile tanımlanan (önceden tanımlanmış) *primitive* kapılar listesinin bir listesi ile belirtilir. Listenin öğeleri, her biri *gate instance* olarak adlandırılan bir kapının *instantiations* olarak adlandırılır.

Her kapı örneği isteğe bağlı bir isim (örneğin *G1*, *G2*, vb.) ile başlar ve ardından kapı çıkışı ve virgülle ayrılmış girişler parantez içinde yer alır. Bir ilkel kapının çıkışı her zaman ilk olarak listelenir, ardından girişler gelir. Örneğin, şematikteki VE kapısı **or** ilkel ile temsil edilir, *G3* olarak adlandırılır ve çıkışı *D* ile girişleri *w1* ve *E* vardır. (Not: Bir ilkelin çıkışı her zaman ilk olarak listelenmelidir, ancak bir modülün girişleri ve çıkışları herhangi bir sırayla listelenebilir.) Modül tanımı **endmodule** anahtar kelimesi ile sona erer. Her ifade bir noktalı virgül ile sonlandırılmalıdır, ancak **endmodule** ifadesinden sonra noktalı virgül yoktur.

declaration ve **instantiation** terimleri arasındaki farkı anlamak önemlidir. Bir Verilog modülü tanımlanır. Tanımı, temsil ettiği donanımın giriş-çıkış davranışını belirtir.

Önceden tanımlanmış ilkel bileşenler tanımlanmaz, çünkü tanımları dil tarafından belirlenmiştir ve kullanıcı tarafından değiştirilemez. İlkel bileşenler, tıpkı kapıların bir baskılı devre kartını doldurmak için kullanılması gibi kullanılır (yani, örneklendirilir).

Bir modül tanımlandıktan sonra, bir dizayn içinde kullanılabileceğini göreceğiz. Not edin ki *Simple_Circuit* sıradan bir programlama dilinde geliştirilenler gibi bir hesaplama modeli değildir. Modeldeki kapıları örnekleyen ifadelerin ardışık sıralaması anlam taşımaz ve bir hesaplama sırası belirtmez. Bir Verilog modeli *tanımlayıcı* bir modeldir. *Simple_Circuit* bir devreyi oluşturan temel bileşenlerin neler olduğunu ve bunların nasıl bağlandığını tanımlar. Devrenin giriş-çıkış davranışı

Tablo 3.5

Gecikmeden Sonra Kapıların Çıkışı

Zaman Birimleri (ns)		Giriş	Çıkış
		ABC	E w1 D
Başlangıç	—	0 0 0	1 0 1
Değiştir	—	1 1 1	1 0 1
	10	1 1 1	0 0 1
	20	1 1 1	0 0 1
	30	1 1 1	0 1 0
	40	1 1 1	0 1 0
	50	1 1 1	0 1 1

Tanım tarafından dolaylı olarak belirtilmiştir çünkü her mantık kapısının davranışı tanımlanmıştır. Bu nedenle, temsil ettiği devreyi simüle etmek için HDL tabanlı bir model kullanılabilir.

Kapı Gecikmeleri

Tüm fiziksel devreler, bir girişin geçişi ile bir çıkışın sonuçlanan geçişi arasında bir yayılma gecikmesi sergiler. Bir devrenin HDL modeli simüle edildiğinde, bazen kapılarının girişinden çıkışına kadar olan gecikme miktarını belirtmek gereklidir. Verilog'da, bir kapının yayılma gecikmesi *zaman birimleri* cinsinden ve simge # ile belirtilir. Verilog'daki zaman gecikmeleri ile ilişkili sayılar boyutsuzdur. Bir zaman biriminin fiziksel zamanla ilişkisi, **timescale** derleyici direktifi ile yapılır. (Derleyici direktifleri, () ters tırnak veya grave accent simgesi ile başlar.) Böyle bir direktif, bir modülün tanımından önce belirtilir ve takip eden kodda tüm zaman sayısal değerlerine uygulanır. Bir timescale direktifinin örneği ti

```
'mescale 1ns/100ps
```

İlk sayı, zaman gecikmeleri için ölçüm birimini belirtir. İkinci sayı, gecikmelerin yuvarlandığı hassasiyeti belirtir; bu durumda 0.1 ns'ye yuvarlanır. Eğer bir timescale belirtilmezse, bir simülatör boyutsuz değerler gösterebilir veya genellikle 1 ns ($=10^{-9}$ s) olan belirli bir zaman birimine varsayılan olarak geçebilir. Örneklerimiz yalnızca varsayılan zaman birimini kullanacaktır.

HDL Örneği 3.2, Örnek 3.1'deki basit devrenin tanımını tekrarlar, ancak her kapı için belirtilen yayılma gecikmeleri ile. ve, veya, ve değil kapılarının zaman g gecikmeleri sırasıyla 30, 20 ve 10 ns'dir. Devre simüle edildiğinde ve girişler A, B, C=0 durumundan A, B, C=1 durumuna geçtiğinde, çıkışlar Tablo 3.5'te gösterildiği gibi değişir (elle hesaplanmış veya bir simülatör tarafından üretilmiştir). E noktasındaki inverterin çıkışı 1'den 0'a 10 ns'lik bir gecikmeden sonra değişir. w1 noktasındaki AND kapısının çıkışı 30 ns'lik bir gecikmeden sonra 0'dan 1'e değişir. D noktasındaki OR kapısının çıkışı, t=30 ns de 1'den 0'a değişir ve ardından t=50 ns'de 1'e geri döner. Her iki durumda da, OR kapısının çıkışındaki değişim, girişlerindeki değişimden 20 ns önce gerçekleşir. Bu sonuçtan, çıkışın değiştiği açıktır.

D nihayet giriş değiştikten sonra 1'lik bir son değere döner, kapı gecikmeleri son değere ulaşmadan önce 20 ns süren negatif bir darbe üretir.

HDL Örneği 3.2 (Geçiş Gecikmeleri ile Kapı Düzey Modeli)

```
// Geçiş gecikmesi olan basit devrenin Verilog modeli
```

```
module Basit_Devre_gecikme (A, B, C, D, E);
    output D, E;
    input  A, B, C;
    wire   w1;

    ve                                     #(30) G1 (w1, A, B);
    deęil                                  #(10) G2 (E, C);
    veya                                  #(20) G3 (D, w1, E);
endmodule
```

Bir devreyi HDL ile simüle etmek için, simülatörün bir çıkış tepkisi üretmesi için devreye girişler uygulamak gereklidir. Bir tasarıma uyarı sağlayan HDL tanımına *test tezgahı* denir. Test tezgahlarının yazımı, Bölüm 4.12'nin sonunda daha ayrıntılı olarak açıklanmaktadır. Burada, çok fazla detaya girmeden basit bir örnekle prosedürü gösteriyoruz. HDL Örneği 3.3 gecikmeli devreyi simüle etmek için bir test tezgahı göstermektedir. (Ayrımcı isim *Simple_Circuit_prop_gecikme*.) En basit haliyle, bir test tezgahı, bir sinyal üretici ve doğrulanacak modelin bir örneğini içeren bir modüldür. Test tezgahının (*t_Simple_Circuit_prop_gecikme*) giriş veya çıkış portu yoktur, çünkü çevresiyle etkileşimde bulunmaz. Genel olarak, test tezgahını, test tezgahı tarafından test edilecek modülün adıyla birleştirilmiş *t_* ön eki ile adlandırmayı tercih ederiz, ancak bu seçim tasarımcıya bırakılmıştır. Test tezgahı içinde, devreye girişler *reg* anahtar kelimesi ile, çıkışlar ise *wire* anahtar kelimesi ile tanımlanır. *Simple_Circuit_prop_gecikme* M1 örnek adıyla örneklendirilmiştir. Bir modülün her örneği, benzersiz bir örnek adı içermelidir. Test tezgahı kullanmanın, bir devrenin girişlerine sinyal üreticileri bağlayarak gerçek donanımı test etmeye benzer olduğunu unutmayın.

HDL Örneği 3.3 (Test Tezgahı)

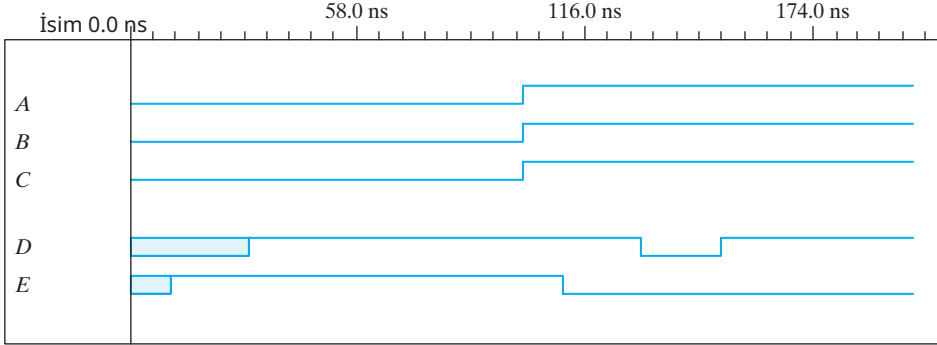
```
// Basit_Devre_gecikme için test tezgahı
```

```
module t_Basit_Devre_gecikme;
    wire   B;
    reg     A, B, C;
```

```
Simple_Circuit_prop_delay M1 (A, B, C, D, E); // Örnek adı gereklidir
```

```
initial
    begin
        A = 1'b0; B = 1'b0; C = 1'b0;
        #100 A = 1'b1; B = 1'b1; C = 1'b1;
    end
```

```
initial #200 $finish;
endmodule
```



ŞEKİL 3.36

HDL Simülasyon çıktısı Örnek 3.3

devreye çıkışlara (kablolar) probelar. (Uyarıcı modülün sinyal üreteçleri ile örneklenmiş devre modülü arasındaki etkileşim Şekil 4.36'da gösterilmektedir.) Donanım sinyali üreteçleri bir HDL modelini doğrulamak için kullanılmaz: Tüm simülasyon çalışması, bir dijital bilgisayar üzerinde bir HDL simülatörü yönetiminde çalışan yazılım modelleri ile gerçekleştirilir. Giriş sinyallerinin dalga biçimleri, dalga biçimi değerlerini ve geçişlerini belirten Verilog ifadeleri ile soyut olarak modellenir (üretilir). **initial** anahtar kelimesi, simülasyon başlatıldığında yürütülmeye başlayan bir dizi ifade ile birlikte kullanılır; **initial** ile ilişkili sinyal aktivitesi, son ifadenin yürütülmesi tamamlandığında yürütmeyi sonlandırır. **initial** ifadeleri genellikle bir test tezgahında dalga biçimlerini tanımlamak için kullanılır. Yürütülecek ifadeler kümesine *block statement* denir ve **begin** ve **end** anahtar kelimeleri ile çevrelenmiş birkaç ifadeden oluşur. İfadelerle belirtilen eylem, simülasyon başlatıldığında başlar ve ifadeler, simülatör tarafından devreye girdi sağlamak için üstten alta, soldan sağa sırayla yürütülür. Başlangıçta, $A, B, C = 0$. (A, B ve C her biri 1 b0 olarak ayarlanır, bu da 0 değerine sahip bir ikili basamağı ifade eder.) 100 ns sonra, girişler değişir. $A, B, C = 1$. 100 ns sonra, simülasyon 200 ns'de sona erer. İkinci başlangıç ifadesi, simülasyonun sona ermesini belirtmek için **\$finish** sistem görevini kullanır. Eğer bir ifade bir gecikme değeriyle (örneğin, #100) önceden geliyorsa, simülatör ifadenin yürütülmesini belirtilen zaman gecikmesi geçene kadar erteler. Simülasyondan kaynaklanan dalga formlarının zaman diyagramı Şekil 3.36'da gösterilmiştir. Toplam simülasyon, 200 ns'lik bir aralıkta dalga formları üretir. Girişler A, B ve C 100 ns sonra 0'dan 1'e değişir. Çıkış E ilk 10 ns için bilinmemektedir (gölgeleme ile gösterilmiştir) ve çıkış D ilk 30 ns için bilinmemektedir. Çıkış E 110 ns'de 1'den 0'a geçer. Çıkış D 130 ns'de 1'den 0'a geçer ve 150 ns'de tekrar 1'e döner, bu da Tablo 3.5'te öngördüğümüz gibidir.

Boolean İfadeleri

Boolean denklemleri, kombinasyonel mantığı tanımlamak için Verilog'da, anahtar kelime **assign** ile başlayan ve bir Boolean ifadesi içeren sürekli atama ifadesi ile belirtilir. Aritmetik operatörleri mantıksal operatörlerden ayırmak için, Verilog AND, OR ve NOT (tamamlayıcı) için sırasıyla (&), (/) ve (~) sembollerini kullanır. Böylece,

Şekil 3.35'teki basit devreyi bir Boolean ifadesi ile tanımlamak için, ifadeyi kullanıyoruz

$$\text{assign } D = (A \& \& B) \text{ } (!C);$$

HDL Örneği 3.4, aşağıdaki iki Boolean

ifadesi ile tanımlanan bir devreyi açıklar:

$$E = A + BC + B'D$$

$$F = B'C + BC D$$

Denklemler, mantık değerlerinin E ve F değerlerinin nasıl belirlendiğini tanımlar. $A, B, C,$ ve D .

HDL Örneği 3.4 (Boolean Denklemleri ile Modellenmiş Kombinyonel Mantık)

// Verilog modeli: Boolean ifadeleri ile devre

```
Circuit_Boolean_CA (E, F, A, B, C, D);
output      E, F;
giriş      A, B, C, D;

assign E = A || (B && C) || (!B) && D;
assign F = (!B) && C || (B && (!C) && (!D));
```

Devrede iki çıkış E ve F ile dört giriş $A, B, C,$ ve D bulunmaktadır. İki assign ifade si Boolean denklemlerini tanımlar. Simülasyon sırasında E ve F değerleri A, B, C ve D değerleri tarafından dinamik olarak belirlenir. Simülatör, test tezgahının bir veya daha fazla girişin değerini değiştirdiğini algılar. Bu olduğunda, simülatör E ve F değerlerini günceller. Sürekli atama mekanizması, atanan değer ile değişkenler arasında ki ilişkinin kalıcı olması nedeniyle bu şekilde adlandırılır. Mekanizma, kombinyonel mantık gibi çalışır, kapı seviyesinde eşdeğer bir devreye sahiptir ve *implicit combinational logic* olarak adlandırılır.

Dijital bir devrenin HDL ifadeleri ile tanımlanabileceğini, tıpkı bir devre şemasında çizilebileceği veya bir Boolean ifadesi ile belirtilebileceği gibi göstermiş olduk. Üçüncü bir alternatif, kombinyonel mantığı bir doğruluk tablosu ile tanımlamaktır.

Kullanıcı Tanımlı Primitifler

Verilog tanımlarında kullanılan mantık kapıları, anahtar kelimeler **and**, **or**, vb. ile tanımlanır ve sistem tarafından belirlenir, bu nedenle *sistem primitifleri olarak adlandırılır. (Dikkat: Diğer diller bu kelimeleri farklı kullanabilir.) Kullanıcı, bunları*

tablolar halinde tanımlayarak ek primitifler oluşturabilir. Bu tür devreler *kullanıcı tanımlı primitifler* (UDP'ler) olarak adlandırılır.

Bir dijital devreyi tablo halinde belirtmenin bir yolu, bir doğruluk tablosu kullanmaktır. UDP tanımları, anahtar kelime çifti **module... endmodule**. kullanmaz. Bunun yerine, anahtar kelime çifti **primitive... endprimitive**. ile tanımlanır. Bir UDP tanımını göstermek için en iyi yol, bir örnek kullanmaktır.

HDL Örneği 3.5, bir doğruluk tablosu ile bir UDP tanımlar. Aşağıdaki genel kurallara göre ilerler:

- Anahtar kelime **primitive** ile tanımlanır, ardından bir isim ve port listesi gelir.
- Sadece bir çıkış olabilir ve bu, port listesinde ilk olarak yer almalı ve anahtar kelime **output** ile tanımlanmalıdır.
- Herhangi bir sayıda giriş olabilir. Giriş tanımında listelenme sırası, ta-
kip eden tabloda değerlerin verildiği sıraya uymalıdır.
- Gerçeklik tablosu, **tablo** ve **son tablo** anahtar kelimeleri arasında yer almaktadır.
- Girişlerin değerleri sıralı olarak listelenir ve bir iki nokta üst üste (:) ile sona erer. Çıkış her zaman bir satırdaki son giriştir ve bir noktalı virgül (;) ile takip edilir.
- Bir UDP'nin bildirimi, **son primitive** anahtar kelimesi ile sona erer.

HDL Örneği 3.5 (Kullanıcı Tanımlı Primitive)

// Verilog modeli: Kullanıcı tanımlı Primitive

primitive UDP_02467 (D, A, B, C);

output D;

input A, B, C;

//D için gerçeklik tablosu 5 f (A, B, C) 5 Σ(0, 2, 4, 6, 7);

tablo

// A B C : D // Sütun başlığı yorumu

0 0 0 0 00:0

0 0 1 0 01:0

0 1 0 0 10:0

0 1 1 0 11:0

1 0 0 0 100:0

1 0 1 0 101:0

1 1 0 0 110:0

1 1 1 1 111:1

endtable

endprimitive

// Primitive'i örnekle

// Verilog modeli: Circuit_UDP_02467'nin devre örnekleme

module Circuit_with_UDP_02467 (e, f, a, b, c, d);

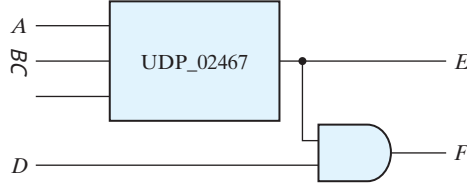
output e, a,

giriş a, b, c, d

UDP_02467 (e, a, b, c);

and (f, e, d); // Seçenek kapısı örneği adı atlandı

endmodule



ŞEKİL 3.37
Şematik için Devre ile UDP_02467

Tablonun üst kısmında listelenen değişkenlerin bir yorumun parçası olduğunu ve yalnızca açıklık için gösterildiğini unutmayın. Sistem, değişkenleri giriş bildiriminde li stelendiği sıraya göre tanır. Kullanıcı tanımlı bir ilkel, diğer modüllerin (sayısal devrel er) inşasında kullanılabilir, tıpkı sistemin önceden tanımlanmış ilkerlerinin kullanıldığı ı gibi. Örneğin,

Circuit_with_UDP_02467 (E, F, A, B, C, D);

şu şekilde bir devre üretecektir: Şekil 3.37'de gösterilen donanımı uygular.

Verilog HDL, bu tür bir tanımlı yalnızca UDP'ler için kullanmasına rağmen, diğer HDL'ler ve bilgi sayar destekli tasarım (CAD) sistemleri, sayısal devreleri tablo biçiminde belirtmek için farklı prose dürler kullanır. Tablolar, tasarımın verimli bir kapı yapısını elde etmek için CAD yazılımı tarafından işlenebilir. Verilog'un önceden tanımlanmış ilkerinin hiçbirisi ardışık mantığı tanımlamaz. Ardışık bi r UDP modelinin çıktısının bir **reg** veri türü olarak tanımlanması ve bir sütunun doğru tablosuna bi r sonraki durumu tanımlamak için eklenmesi gerekir. Bu nedenle sütunlar, girişler : durum : bir s onraki durum olarak düzenlenmiştir.

Bu bölümde, Verilog HDL'yi tanıttık ve kombinasyonel mantığı modellemek için alter natifleri göstermek amacıyla basit örnekler sunduk. Verilog HDL hakkında daha ayrıntılı bir sunum bir sonraki bölümde bulunabilir. Kombinasyonel devreler ile tanışık olan ok uyucu, bu konuyu devam ettirmek için doğrudan Bölüm 4.12'ye geçebilir.

PROBLEMLER

(* ile işaretlenmiş problemlerin cevapları metnin sonunda yer almaktadır.)

3.1*

Aşağıdaki Boolean fonksiyonlarını üç değişkenli haritalar kullanarak basitleştirin:

(a) $F(x, y, z) = \Sigma(0, 2, 4, 5)$

(b) $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$

(c) $F(x, y, z) = \Sigma(0, 1, 2, 3, 5)$

(d) $F(x, y, z) = \Sigma(1, 2, 3, 7)$

3.2

Aşağıdaki Boolean fonksiyonlarını üç değişkenli haritalar kullanarak basitleştirin:

(a) $F(x, y, z) = \Sigma(0, 1, 5, 7)$

(b) $F(x, y, z) = \Sigma(1, 2, 3, 6, 7)$

(c) $F(x, y, z) = \Sigma(2, 3, 4, 5)$

(d) $F(x, y, z) = \Sigma(1, 2, 3, 5, 6, 7)$

(e) $F(x, y, z) = \Sigma(0, 2, 4, 6)$

(f) $F(x, y, z) = \Sigma(3, 4, 5, 6, 7)$

3.3*

Aşağıdaki Boolean ifadelerini üç değişkenli haritalar kullanarak basitleştirin:

(a) $xy + xyz$

(e) $F(x, y, z) = xy + yz$

(b) $xy + yz + xyz$

(d) $F(x, y, z) = xyz + xyz$

+ xyz

Aşağıdaki Boolean fonksiyonlarını, *Karnaugh* haritalarını kullanarak basitleştirin:

$$(a) F(x, y, z) = \Sigma(2, 3, 6, 7)$$

$$(b) F(A, B, C, D) = \Sigma(4, 6, 7, 15)$$

$$(c) F(A, B, C, D) = \Sigma(3, 7, 11, 13, 14, 15)$$

$$(d) F(w, x, y, z) = \Sigma(2, 3, 12, 13, 14, 15)$$

$$F(w, x, y, z) = \Sigma(11, 12, 13, 14, 15)$$

$$(f) F(w, x, y, z) = \Sigma(8, 10, 12, 13, 14)$$

Aşağıdaki Boolean fonksiyonlarını dört değişkenli haritalar kullanarak basitleştirin:

$$(a) F(w, x, y, z) = \Sigma(1, 4, 5, 6, 12, 14, 15)$$

$$F(A, B, C, D) = \Sigma(2, 3, 6, 7, 12, 13, 14)$$

$$F(w, x, y, z) = \Sigma(1, 3, 4, 5, 6, 7, 9, 11, 13, 15)$$

$$(d) F(A, B, C, D) = \Sigma(0, 2, 4, 5, 6, 7, 8, 10, 13, 15)$$

Aşağıdaki Boolean ifadelerini dört değişkenli haritalar kullanarak basitleştirin:

$$(a) ABCD + ACD + BCD + ABCD + BCD$$

$$(b) xz + wxy + w(xy + xy)$$

$$ABCD + ABD + ABC + ABCD + ABC$$

$$ABCD + BCD + ACD + ABCD + ACD$$

Aşağıdaki Boolean ifadelerini dört değişkenli haritalar kullanarak basitleştirin:

$$(a) WZ + xz + xy + WXZ$$

$$AD + BCD + BCD + BCD$$

$$(e) ABC + BCD + BCD + ACD + ABC + ABCD$$

$$wxy + xz + WXZ + wx$$

Aşağıdaki Boolean ifadelerinin mintermlerini bulmak için önce her fonksiyonu bir haritada çiziniz:

$$(a) xy + yz + xyz$$

$$wyz + WX + WXZ$$

$$(b) \overline{B}D + ABC + ABD + ABD$$

$$(d) A'B + A'CD + B'CD + BC'D'$$

Aşağıdaki Boolean fonksiyonları için tüm asal implicantları bulun ve hangilerinin gerekli olduğunu belirleyin:

$$(a) F(w, x, y, z) = \Sigma(0, 2, 4, 5, 6, 7, 8, 10, 13, 15)$$

$$(b) F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 10, 11, 14, 15)$$

$$F(A, B, C, D) = \Sigma(2, 3, 4, 5, 6, 7, 9, 11, 12, 13)$$

$$F(w, x, y, z) = \Sigma(1, 3, 6, 7, 8, 9, 12, 13, 14, 15)$$

$$F(A, B, C, D) = \Sigma(0, 1, 2, 5, 7, 8, 9, 10, 13, 15)$$

$$F(w, x, y, z) = \Sigma(0, 1, 2, 5, 7, 8, 10, 15)$$

Aşağıdaki Boolean fonksiyonlarını, önce gerekli temel etkileyenleri bularak basitleştirin:

$$F(w, x, y, z) = \Sigma(0, 2, 5, 7, 8, 10, 12, 13, 14, 15)$$

$$F(A, B, C, D) = \Sigma(0, 2, 3, 5, 7, 8, 10, 11, 14, 15)$$

$$(e) F(A, B, C, D) = \Sigma(1, 3, 4, 5, 10, 11, 12, 13, 14, 15)$$

$$F(w, x, y, z) = \Sigma(0, 1, 4, 5, 6, 7, 9, 11, 14, 15)$$

$$F(A, B, C, D) = \Sigma(0, 1, 3, 7, 8, 9, 10, 13, 15)$$

$$F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 7, 10, 15)$$

Aşağıdaki Boolean fonksiyonunu toplam-çarpan formundan basitleştirilmiş çarpan-toplam formuna dönüştürün.

$$F(x, y, z) = \Sigma(0, 1, 2, 5, 8, 10, 13)$$

120 Bölüm 3 Kapı Seviyesi Minimizasyonu

3.12 Aşağıdaki Boolean fonksiyonlarını basitleştirin:

$$(a) F(A, B, C, D) = \Pi(1, 3, 5, 7, 13, 15)$$

$$- F(A, B, C, D) = \Pi(1, 3, 6, 9, 11, 12, 14)$$

3.13 Aşağıdaki ifadeleri (1) toplam-çarpın ve (2) çarpın-toplam olarak basitleştirin:

$$\begin{aligned} & (xyz + yz + yz + xy) \\ & - ACD + CD + AB + ABCD \\ & - (A + B + D)(A + B + C)(A + B + C)(B + C + D) \\ & - BCD + ABC + ACD \end{aligned}$$

3.14 Aşağıdaki Boolean fonksiyonunu sekiz veya daha az literal ile ifade etmenin üç olası yolunu verin:

$$F = ABCD + ABCD + ABC + ACD$$

3.15 Aşağıdaki Boolean fonksiyonunu F , don't-care

sonra basitleştirilmiş fonksiyonu mintermlerin toplamı biçiminde ifade edin:

$$\begin{aligned} & - F(x, y, z) = \Sigma(0, 1, 4, 5, 6) \quad * (F)(A, B, C, D) = \Sigma(0, 6, 8, 13, 14) \\ & d(x, y, z) = \Sigma(2, 3, 7) \quad d(A, B, C, D) = \Sigma(2, 4, 10) \\ & - F(A, B, C, D) = \Sigma(5, 6, 7, 12, 14, 15) \quad (F)(A, B, C, D) = \Sigma(4, 12, 7, 2, 10) \\ & d(A, B, C, D) = \Sigma(3, 9, 11, 15) \quad d(A, B, C, D) = \Sigma(0, 6, 8) \end{aligned}$$

3.16 Aşağıdaki fonksiyonları basitleştiriniz ve bunları iki seviyeli NAND kapı devreleri ile uygulayınız:

$$\begin{aligned} & - F(A, B, C, D) = ACD + AC + ABC + ABC + ACD \\ & - F(A, B, C, D) = ABCD + CD + ACD \\ & - F(A, B, C) = (A + C + D)(A + C)(C + D) \\ & - F(A, B, C, D) = A + B + D + BC \end{aligned}$$

3.17* Aşağıdaki fonksiyonun tamamlayıcısını uygulayan bir NAND mantık diyagramı çizin:

$$F(A, B, C, D) = \Sigma(0, 1, 2, 3, 6, 10, 11, 14)$$

3.18 Aşağıdaki fonksiyonu uygulamak için yalnızca iki girişli NOR kapıları kullanarak bir mantık diyagramı çizin:

$$F(A, B, C, D) = (A \oplus B)'(C \oplus D)$$

3.19 Aşağıdaki fonksiyonları basitleştirin ve bunları iki seviyeli NOR kapı devreleri ile uygulayın:

$$\begin{aligned} & (a) F = xyz + yz + wyz \\ & - F(w, x, y, z) = \Sigma(0, 3, 12, 15) \\ & - F(x, y, z) = [(x+y)(x=z)]' \end{aligned}$$

3.20 Aşağıdaki ifade için çok seviyeli NOR devresini çizin:

$$CD(B+C)A + (BC + DE)$$

3.21 Aşağıdaki ifade için çok seviyeli NAND devresini çizin:

$$w(x+y+z) + xyz$$

3.22 Şekil 4.4'te gösterilen devrenin mantık diyagramını çok seviyeli NAND devresine dönüştürün.

Aşağıdaki Boolean fonksiyonunu F ve önemsenmeyen koşulları d , en fazla iki NO R kapısı kullanarak uygulayın:

$$\begin{aligned} & F(A, B, C, D) = \Sigma(2, 4, 10, 12, 14,) \\ & d(A, B, C, D) = \Sigma(0, 1, 5, 8) \end{aligned}$$

Normal ve tamamlayıcı girişlerin mevcut olduğunu varsayın.

3.24

Aşağıdaki Boolean fonksiyonunu F , iki seviyeli mantık formlarını kullanarak uygulayın (a) NAND-AND, (b) AND-NOR, (c) OR-NAND ve (d) NOR-OR:

$$F(A,B,C,D)=\Sigma(0, 4, 8, 9, 10, 11, 12, 14)$$

3.25

Sekiz degenerate iki seviyeli formu listeleyin ve bunların tek bir işlemi nasıl sadeleştirdiğini gösterin. Degenerate iki seviyeli formların bir kapıya giriş sayısını nasıl artırmak için kullanılabileceğini açıklayın.

3.26

Haritaların kullanımıyla, fonksiyonun en basit toplam-çarpınlar formunu bulun $F = fg$, burada

$$f = abc + cd + acd + bcz$$

ve

$$g = (a+b+c + d)(b + c + d)(a + c+d)$$

3.27

Özel-VE'nin ikilinin de tamamlayıcısı olduğunu

3.28

Bir üç bitlik parite üretici ve dört bitlik parite kontrolörü devrelerini tek parite biti kullanarak türetin.

3.29

Aşağıdaki dört Boolean ifadesini üç yarım toplayıcı ile uygulayın:

$$\begin{aligned} D &= A \oplus B \oplus C \\ E &= A'BC + AB'C \\ F &= ABC' + (A + B)C \\ G &= ABC \end{aligned}$$

3.30*

Aşağıdaki Boolean ifadesini özel-VE ve VE kapıları ile uygulayın:

$$F = ABCD + ABCD + ABCD + ABCD + ABCD$$

3.31 Şekil 3.20 (a)'da gösterilen devrenin Verilog kapı düzeyinde tanımını yazın.

(d) Şekil 3.21 (b)

(b) Şekil 3.20 (b)

(c) Şekil 3.21 (a)

(e) Şekil 3.24

(f) Şekil 3.25

3.32

Sürekli atama ifadeleri kullanarak, aşağıda gösterilen devrenin Verilog tanımını yazın

(a) Şekil 3.20 (a)

(b) Şekil 3.20 (b)

(c) Şekil 3.21 (a)

(d) Şekil 3.21 (b)

(e) Şekil 3.24

(f) Şekil 3.25

3.33

Şekil 3.30 (a) 'daki özel-VE devresi, bir inverter için 3 ns, bir VE kapısı için 6 ns ve bir VEYA kapısı için 8 ns gecikme süresine sahip kapılara sahiptir. Devrenin girişi, $xy = 00$ to $xy = 01$.

(a) Her kapının çıkışındaki sinyalleri $t = 0$ 'dan $t = 50$ ns'ye kadar belirleyin.

(b) Devreyi, gecikmeleri de içerecek şekilde, Verilog kapı düzeyinde tanımlayın.

(c) Bir uyarı modülü yazın (yani, HDL Örneği 3.3'e benzer bir test tezgahı) ve devreyi simüle ederek (a) bölümündeki yanıtı doğrulayın.

3.34

Sürekli atamalar kullanarak, aşağıdaki Boolean fonksiyonlarıyla belirtilen devrenin Verilog tanımını yazın:

$$\begin{aligned} Out_1 &= (A+B')C'(C+D) \\ Out_2 &= (C'D+BCD+CD')(A' + B) \\ Out_3 &= (AB+C)D+B'C \end{aligned}$$

Devre davranışını simüle etmek için bir test tezgahı yazın.

3.35*

Aşağıdaki bildirimlerdeki sözdizimi hataları

isteğe bağlı olduğunu unutmayın):

```

modül   Exmpl-3(A, B, C, D, F) // Satır 1
    girişler A, B, C, Çıkış D, F, // Satır 2
    çıkış      B // Satır 3
    ve          g1(A, B, D); // Satır 4
    not         (D, A, C), // Satır 5
    OR          (F, B; C); // Satır 6
endmodule; // Satır 7

```

Aşağıdaki Verilog tanımıyla belirtilen dijital devrenin mantık diyagramını çizin:

```

(a) module Circuit_A (A, B, C, D, F);
    input A, B, C, D;
    output F;
    wire w, x, y, z, a, d;
    veya (x, B, C, d);
    and (y, a, C);
    ve (w, z, B);
    and (z, y, A);
    veya (F, x, w);
    A0 (a,
    B0 (d,
endmodule

```

```

module Circuit_B (F1, F2, F3, A0, A1, B0, B1);
    output F1, F2, F3;
    giriş A0, A1, B0, B1;
    veya (F1, F2, F3);
    veya (F2, w1, w2, w3);
    ve (F3, w4, w5);
    ve (w1, w6, B1);
    veya (w2, w6, w7, B0);
    ve (w3, w7, B0, B1);
    not (w6, A1);
    not (w7, A0);
    xor (w4, A1, B1);
    xnor (w5, A0, B0);
endmodule

```

```

(c) module Circuit_C (y1, y2, y3, a, b);
    output y1, y2, y3;
    input a, b;

    assign y1 = a || b;
    and (y2, a, b);
    assign y3 = a && b;
endmodule

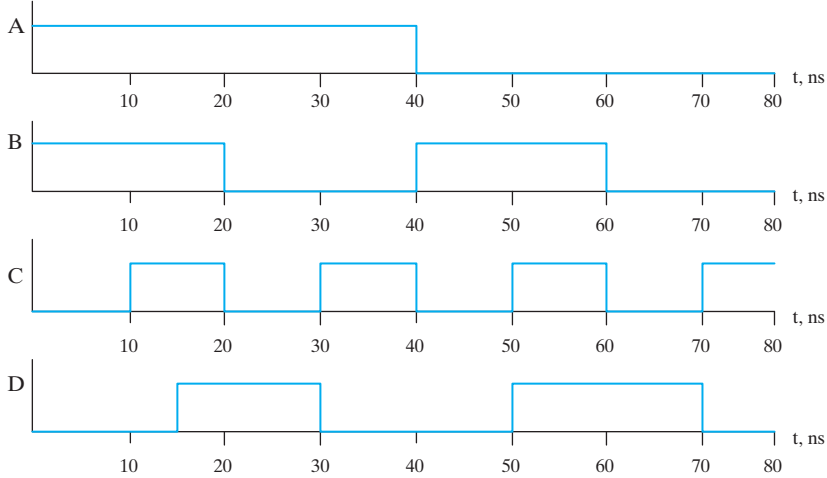
```

Bir çoğunluk mantık fonksiyonu, değişkenlerin çoğunluğu 1'e eşit olduğunda 1'e, aksi takdirde 0'a eşit olan bir Boole fonksiyonudur.

(a) Dört bitlik bir çoğunluk fonksiyonu için bir doğruluk tablosu yazın.

(b) Dört bitlik bir çoğunluk fonksiyonu için bir Verilog kullanıcı tanımlı ilkel yazın.

Circuit_with_UDP_02467 'nin davranışını simüle edin, Fig. P3.38'de gösterilen uyarı dalga formlarını kullanarak.



ŞEKİL P3.38

Problem 3.38 için uyarı dalga formları

İlkel kapıları kullanarak, iki çıkış üretecek bir devrenin Verilog modelini yazın, s ve c , iki ikili giriş biti a ve b ekleyerek üretilen toplam ve taşıma ile eşit olacak şekilde (örneğin, $s = 1$ ve $c = 0$ ise $a = 0$ ve $b = 1$). (İpucu: s ve c için bir doğruluk tablosu geliştirerek başlayın.)

REFERANSLAR

1. B HASKER , J. 1997. *Verilog HDL Primer*. Allentown, PA: Star Galaxy Press.
2. C İLETTİ , M. D. 1999. *Modeling, Synthesis and Rapid Prototyping with the Verilog HDL* . Upper Saddle River, NJ: Prentice Hall.
3. H ill , F. J., ve G. R. P erson . 1981. *Anahtar Teorisi ve Mantıksal Tasarıma Giriş*, 3. baskı. New York: John Wiley.
4. *IEEE Standard Donanım Tanım Dili Verilog Donanım Tanım Diline Dayalı* (IEEE Std . 1364-1995). 1995. New York: Elektrik ve Elektronik Mühendisleri Enstitüsü.
5. K arnauth , M. A Kombinasyonel Mantık Devrelerinin Sentezi için Bir Harita Yöntemi. *AİEE Dergisi, İletişim ve Elektronik*. 72, bölüm I (Kasım 1953): 593-99.
6. K OHAVI , Z. 1974. *Anahtar ve Otomata Teorisi*, 2. baskı. New York: McGraw-Hill.

124 Bölüm 3 Kapı Seviyesi Minimizasyonu

7. M ano , M. M. ve C. R. K ime . 2004. *Mantık ve Bilgisayar Tasarım Temelleri*, 3. baskı. Upper Saddle River, NJ: Prentice Hall.
8. M c C LUSKEY , E. J. 1994. *Mantık Tasarım İlkeleri*. Englewood Cliffs, NJ: Prentice-Hall.
9. P ALNITKAR , S. 1999. *Verilog HDL: Dijital Tasarım ve Sentez için Bir Rehber*. Mountain View, CA: SunSoft Press (bir Prentice Hall başlığı).

AĞ ARAMA KONULARI

Boolean minimizasyonu

Karnaugh haritası

Wired logic

Emitter-coupled logic

Open-collector logic

Quine McCluskey method

Expresso software

Consensus theorem

Don't-care conditions

Bölüm 4

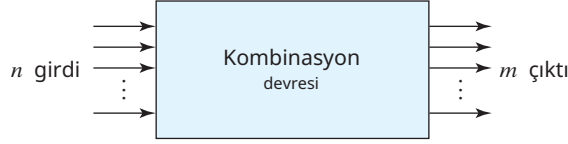
Kombinasyonel Mantık

4.1 GİRİŞ

Dijital sistemler için mantık devreleri kombinasyonel veya sıralı olabilir. Bir kombinasyonel devre, herhangi bir zamanda çıkışlarının yalnızca mevcut giriş kombinasyonuna bağlı olduğu mantık kapaklarından oluşur. Bir kombinasyonel devre, bir dizi Boolean fonksiyonu ile mantıksal olarak tanımlanabilen bir işlemi gerçekleştirir. Buna karşılık, sıralı devreler mantık kapaklarına ek olarak depolama elemanları kullanır. Çıkışları, girişlerin ve depolama elemanlarının durumunun bir fonksiyonudur. Depolama elemanlarının durumu önceki girişlerin bir fonksiyonu olduğundan, bir sıralı devrenin çıkışları yalnızca mevcut giriş değerlerine değil, aynı zamanda geçmiş girişlere de bağlıdır ve devre davranışı, girişlerin ve iç durumların zaman sırasına göre belirtilmelidir. Sıralı devreler dijital sistemlerin yapı taşlarıdır ve Bölüm 5 ve 8'de tartışılmaktadır.

4.2 KOMBİNASYONEL DEVRELER

Bir kombinasyonel devre, mantık kapaklarının birbiriyle bağlantısından oluşur. Kombinasyonel mantık kapakları, girişlerindeki sinyallerin değerlerine tepki verir ve çıkış sinyalinin değerini üretir, verilen giriş verilerinden gerekli çıkış verilerine ikili bilgiyi dönüştürür. Bir kombinasyonel devrenin blok diyagramı Şekil 4.1'de gösterilmiştir. n giriş binary değişkenleri dış bir kaynaktan gelir; m çıkış değişkenleri iç kombinasyonel mantık devresi tarafından üretilir ve dış bir hedefe gider. Her giriş ve çıkış değişkeni, değerleri mantık 1 ve mantık 0'ı temsil eden bir ikili sinyal olarak yorumlanan fiziksel bir analog sinyal olarak mevcuttur. (*Not* : Mantık simülatörleri yalnızca 0 ve 1 gösterir, gerçek analog sinyalleri değil.) Birçok uygulamada, kaynak ve

**ŞEKİL 4.1**

Kombinyasyonel devrelerin blok diyagramı

Hedef depolama kayıtlarıdır. Eğer kayıtlar kombinyasyonel

gate'lerle birlikte dahil edilirse, o zaman toplam devre sıralı bir devre olarak kabul edilmelidir.

n giriş değişkenleri için, ikili girişlerin 2^n olası kombinyasyonu vardır. Her olası k giriş kombinyasyonu için, her çıkış değişkeni için bir olası değer vardır. Böylece, bir kombinyasyonel devre, giriş değişkenlerinin her kombinyasyonu için çıkış değerlerini listeleleyen bir doğruluk tablosu ile tanımlanabilir. Bir kombinyasyonel devre ayrıca m Boole fonksiyonları ile tanımlanabilir, her çıkış değişkeni için bir tane. Her çıkış fonksiyonu, n giriş değişkenleri cinsinden ifade edilir.

Bölüm 1'de, ayrıık bilgi miktarlarını temsil eden ikili sayılar ve ikili kodlar hakkında bilgi edindik. İkili değişkenler fiziksel olarak elektrik voltajları veya başka bir tür sinyallerle temsil edilir. Sinyaller, gerekli işlevleri yerine getirmek için dijital mantık kapılarında işlenebilir. Bölüm 2'de, mantık fonksiyonlarını cebirsel olarak ifade etmenin bir yolu olarak Boole cebirini tanıttık. Bölüm 3'te, Boole fonksiyonlarını daha ekonomik (daha basit) kapı uygulamaları elde etmek için nasıl sadeleştirceğimizi öğrendik. Mevcut bölümün amacı, önceki bölümlerde edinilen bilgileri kullanarak kombinyasyonel devreler için sistematik analiz ve tasarım prosedürleri formüle etmektir. Bazı tipik örneklerin çözümü, dijital sistemlerin anlaşılması için önemli olan temel fonksiyonların yararlı bir kataloğunu sağlayacaktır. Üç görevi ele alacağız: (1) Verilen bir mantık devresinin davranışını analiz et, (2) belirli bir davranışa sahip bir devre sentezle, ve (3) bazı yaygın devreler için donanım tanım dili (HDL) modelleri yaz.

Sayısal sistemlerin tasarımında yaygın olarak kullanılan birkaç kombinyasyonel devresi vardır. Bu devreler entegre devrelerde mevcuttur ve standart bileşenler olarak sınıflandırılır. Sayısal sistemlerin tasarımında yaygın olarak ihtiyaç duyulan belirli sayısal işlevleri yerine getirirler. Bu bölümde, toplama, çıkarma, karşılaştırma, kod çözücü, kodlayıcı ve çoklayıcı gibi en önemli standart kombinyasyonel devrelerini tanıtıyoruz. Bu bileşenler, orta ölçekli entegrasyon (MSI) devreleri olarak entegre devrelerde mevcuttur. Ayrıca, uygulamaya özel entegre devreler (ASIC'ler) gibi karmaşık çok büyük ölçekli entegre (VLSI) devrelerinde standart hücreler olarak da kullanılırlar. Standart hücre işlevleri, VLSI devresinde, çoklu IC MSI tasarımında kullanıldığı şekilde birbirine bağlanır.

4.3 ANALİZ PROSEDÜRÜ

Kombinyasyon devresinin analizi, devrenin uyguladığı işlevi belirlememizi gerektirir. Bu görev, verilen bir mantık diyagramı ile başlar ve bir dizi Boole işlevi, bir doğruluk tablosu veya muhtemelen devre çalışmasının bir açıklaması ile sonuçlanır.

Analiz edilecek mantık diyagramı bir fonksiyon adı veya neyi başarması gerektiğine dair bir açıklama ile birlikteyse, o zaman analiz problemi belirtilen fonksiyonun doğrulanması ile a zalır. Analiz, Boolean fonksiyonlarını veya doğruluk tablosunu bulmakla veya bir bilgisayar simülasyon programı kullanarak manuel olarak gerçekleştirilebilir.

Analizdeki ilk adım, verilen devrenin kombinasyonel olduğundan ve ardışık olmadığına n emin olmaktır. Kombinasyonel bir devrenin diyagramı, geri besleme yolları veya bellek el emanları olmayan mantık kapıları içerir. Geri besleme yolu, bir kapının çıkışından ikinci bir kapının girişine olan bir bağlantıdır; bu ikinci kapının çıkışı, ilk kapının girişinin bir parçasını oluşturur. Dijital bir devredeki geri besleme yolları, ardışık bir devreyi tanımlar ve özel yön temlerle analiz edilmelidir; burada ele alınmayacaktır.

Bir mantık diyagramının kombinasyonel devre olduğunu doğruladıktan sonra, çıkış Boolean fonksiyonlarını veya doğruluk tablosunu elde etmeye devam edilebilir. Devrenin fonksiyonu inceleniyorsa, devrenin işleyişini türetilen Boolean fonksiyonların dan veya doğruluk tablosundan yorumlamak gereklidir. Böyle bir araştırmanın başa rısı, geniş bir dijital devre yelpazesi ile önceki deneyim ve aşinalık varsa artar.

Bir mantık diyagramından çıkış Boolean fonksiyonlarını elde etmek için şu şekilde ilerliyoruz:

1. Tüm kapı çıkışlarını, girdi değişkenlerinin bir fonksiyonu olarak, anlamlı isimlerle birlikte rastgele sembollerle etiketleyin. Her kapı çıkışı için Boolean fonksiyonlarını belirleyin.

Girdi değişkenlerinin ve daha önce etiketlenmiş kapıların bir fonksiyonu olan kapıları başka rastgele sembollerle etiketleyin. Bu kapılar için Boolean fonksiyonlarını bulun.

Devrenin çıkışları elde edilene kadar 2. adımda belirtilen süreci tekrarlayın.

Daha önce tanımlanmış fonksiyonların tekrar eden yerleştirilmesiyle, çıkış Boolean fonk siyonlarını girdi değişkenleri cinsinden elde edin.

Şekil 4.2'deki kombinasyonel devrenin analizi önerilen prosedürü göstermek tedir. Devrenin üç ikili girişi— A , B ve C —ve iki ikili çıkışı— F_1 ve F_2 vardır.

Çeşitli kapıların çıkışları ara sembollerle etiketlenmiştir. Yalnızca girdi değişkenlerinin bir fonksiyonu olan kapıların çıkışları T_1 ve T_2 . Çıkış F_2 girdi değişkenlerinden kolayca türetilir. Bu üç çıkış için Boolean fonksiyonları ş unlardır:

$$F_2 = AB + AC + BC$$

$$T_1 = A + B + C$$

$$T_2 = ABC$$

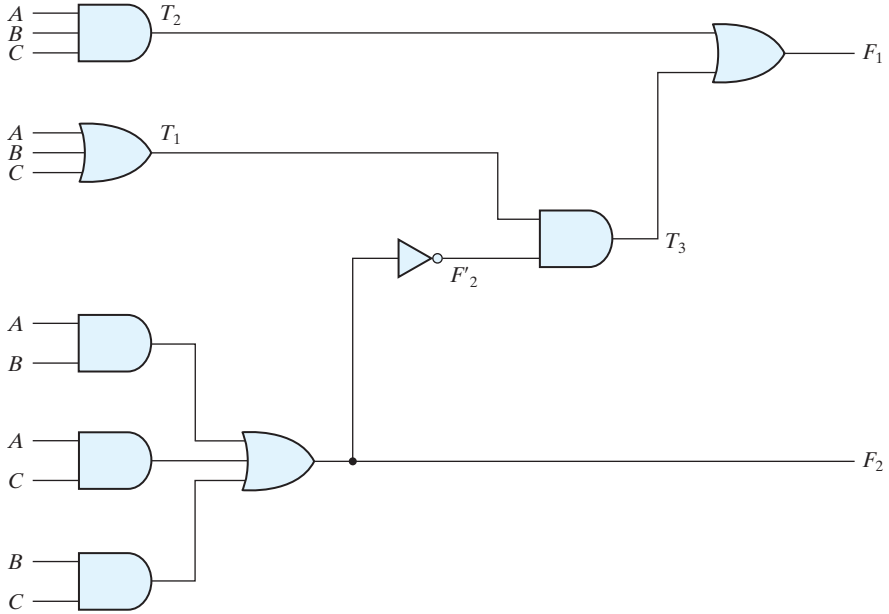
Sonra, zaten tanımlanmış sembollerin bir fonksiyonu olan kapıların çıktısını dikkate alıyoruz:

$$T_3 = F_2 T_1$$

$$F_1 = T_3 + T_2$$

A , B , ve C cinsinden F_1 elde etmek için, aşağıdaki gibi bir dizi yer değiştirme oluşturuyoruz:

$$\begin{aligned} F_1 &= T_3 + T_2 = F_2 T_1 + ABC = (AB + AC + BC)'(A + B + C) + ABC \\ &= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC \\ &= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC \\ &= A'BC' + A'B'C + AB'C' + ABC \end{aligned}$$

**ŞEKİL 4.2**

Analiz örneği için mantık diyagramı

Eğer bu devrenin gerçekleştirdiği bilgi dönüşüm görevini araştırmaya devam etmek istiyorsak, türetilen Boolean ifadelerinden devreyi çizebilir ve tanıdık bir işlemi tanımlamaya çalışabiliriz. F_1 ve F_2 için Boolean fonksiyonları, Bölüm 4.5'te tartışılan bir devreyi uygulamaktadır. Bir devrenin Boolean temsili bulmak, onun davranışı hakkında içgörü sağlamaz, ancak bu örnekte F_1 ve F_2 için Boolean denklemleri ve doğruluk tablosunun, tam toplayıcı olarak adlandırdığımız işlevselliği tanımlayanlarla eşleştirdiğini gözlemleyeceğiz.

Bir devre için doğru tabloyun türetilmesi, çıktı Boolean fonksiyonları bilindiğinde basit bir süreçtir. Boolean fonksiyonlarının türevlerinden geçmeden mantık diyagramından doğru tabloyu doğrudan elde etmek için şu şekilde ilerleriz:

1. Devredeki giriş değişkenlerinin sayısını belirleyin. n girişleri için, 2^n mümkün giriş kombinasyonları oluşturun ve 0'dan $(2^n - 1)$ kadar olan ikili sayıları bir tabloda listeleyn.
2. Seçilen kapların çıkışlarını rastgele sembollerle etiketleyin.
3. Yalnızca giriş değişkenlerinin bir fonksiyonu olan bu kapların çıkışları için doğru tabloyu elde edin.
4. Daha önce tanımlanmış değerlere bağlı olan bu kapların çıkışları için doğru tabloyu elde etmeye devam edin, ta ki tüm çıkışlar için sütunlar belirlenene kadar.

Tablo 4.1

Şekil 4.2 için Mantık Diyagramının Gerçeklik Tablosu

A	B	C	F_2	F_2'	r_1	r_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

Bu süreç, Şekil 4.2'deki devre ile gösterilmektedir. Tablo 4.1'de, üç giriş değişkeni için sekiz olası kombinasyon oluşturuyoruz. F_2 'nin doğruluk tablosu, A , B ve C değerlerinden doğrudan belirlenir; F_2 , iki veya üç girişin 1'e eşit olduğu herhangi bir kombinasyon için 1'e eşittir. F_2 'nin doğruluk tablosu, F_2 'nin tamamlayıcısıdır. T_1 ve T_2 'nin doğruluk tabloları, sırasıyla giriş değişkenlerinin VEYA ve V E fonksiyonlarıdır. T_3 'ün değerleri, T_1 ve F_2 'den türetilir: T_3 , hem T_1 hem de F_2 1'e eşit olduğunda 1'e eşittir ve aksi takdirde T_3 0'a eşittir. Son olarak, F_1 , T_2 ve T_3 veya her ikisinin 1'e eşit olduğu kombinasyonlar için 1'e eşittir. A , B , C , F_1 , ve F_2 için doğruluk tablosu kombinasyonlarının incelenmesi, sırasıyla x , y , z , S , ve C için Bölüm 4.5'te verilen tam toplayıcının doğruluk tablosuyla özdeş olduğunu göstermektedir.

Kombinasyonel bir devreyi analiz etmenin bir diğer yolu mantık simülasyonudur. Ancak bu pratik değildir, çünkü anlamlı çıktılar üretmek için gerekli olabilecek giriş desenlerinin sayısı çok büyük olabilir. Ancak simülasyon, bir devrenin işlevselliğinin gerçekte n spesifikasyonu ile eşleşip eşleşmediğini doğrulamada çok pratik bir uygulamaya sahiptir. Bölüm 4.12'de, Verilog HDL kullanarak Şekil 4.2'deki devrenin mantık simülasyonu ve doğrulamasını gösteriyoruz.

4.4 TASARIM PROSEDÜRÜ

Kombinasyonel devrelerin tasarımı, tasarım amacının spesifikasyonundan başlar ve bir mantık devre diyagramı veya mantık diyagramının elde edilebileceği bir dizi Boolean fonksiyonu ile sonuçlanır. Prosedür aşağıdaki adımları içerir:

1. Devrenin spesifikasyonlarından, gerekli giriş ve çıkış sayısını belirleyin ve her birine bir sembol atayın.
2. Girişler ve çıkışlar arasındaki gerekli ilişkiyi tanımlayan doğruluk tablosunu çıkarın.

Her çıkış için giriş

değişkenlerinin bir fonksiyonu olarak basitleştirilmiş Boolean fonksiyonlarını elde edin.

Mantık diyagramını çizin ve tasarımın doğruluğunu kontrol edin (elle veya simülasyonla).

Kombinasyonel bir devre için bir doğruluk tablosu, giriş sütunları ve çıkış sütunlarından oluşur. Giriş sütunları, n giriş değişkenleri için 2 ikili sayıdan elde edilir. Çıkışlar için ikili değerler, belirtilen özelliklerden belirlenir. Doğruluk tablosunda belirtilen çıkış fonksiyonları, kombinasyonel devrenin kesin tanımını verir. Sözlü özelliklerin doğruluk tablosunda doğru bir şekilde yorumlanması önemlidir, çünkü genellikle eksik olurlar ve herhangi bir yanlış yorum, yanlış bir doğruluk tablosuna yol açabilir.

Doğruluk tablosunda listelenen çıkış ikili fonksiyonları, cebirsel manipülasyon n , harita yöntemi veya bir bilgisayar tabanlı sim- plifikasyon programı gibi mevcut herhangi bir yöntemle basitleştirilir. Sıklıkla, seçim yapılacak çeşitli basitleştirilmiş ifadeler vardır. Belirli bir uygulamada, belirli kriterler, bir uygulama seçme sürecinde rehberlik edecektir. Pratik bir tasarım, kapı sayısı, bir kapıya giriş sayısı, sinyalin kapılar üzerinden yayılma süresi, bağlantı sayısı, her kapının sürme kapasitesinin sınırlamaları (yani, devrenin çıkışının bağlanabileceği kapı sayısı) ve entegre devreler tasarlarlarken dikkate alınması gereken çeşitli diğer kriterler gibi kısıtlamaları göz önünde bulundurmalıdır. Her kısıtlamanın önemi, belirli uygulama tarafından belirlenir, bu nedenle kabul edilebilir bir uygulamanın neyi oluşturduğuna dair genel bir ifade yapmak zordur. Çoğu durumda, basitleştirme, standart bir biçimde basitleştirilmiş Boolean fonksiyonlarını üretmek gibi temel bir hedefi karşılayarak başlar. Ardından, basitleştirme, diğer performans kriterlerini karşılamak için daha fazla adımla devam eder.

Kod Dönüşüm Örneği

Farklı dijital sistemler tarafından aynı ayrıık bilgi öğeleri için çok çeşitli kodların mevcut olması, farklı kodların kullanılmasına neden olur. Bazen bir sistemin çıktısını başka bir sistemin girişi olarak kullanmak gerekir. Eğer her biri aynı bilgi için farklı kodlar kullanıyorsa, iki sistem arasında bir dönüştürme devresi yerleştirilmelidir. Bu nedenle, bir kod dönüştürücü, her biri farklı bir ikili kod kullansa da iki sistemi uyumlu hale getiren bir devredir.

İkili kod A'dan ikili kod B'ye dönüştürmek için, giriş hatları kod A tarafından belirtilen elemanların bit kombinasyonunu sağlamalı ve çıkış hatları kod B'nin karşılık gelen bit kombinasyonunu üretmelidir. Bir kombinasyon devresi bu dönüşümü mantık kapıları aracılığıyla gerçekleştirir. Tasarım prosedürü, ondalık rakamlar için ikili kodlu ondalık (BCD) kodunu fazladan-3 koduna dönüştüren bir örnekle gösterilecektir.

BCD ve fazla-3 kodlarına atanan bit kombinasyonları Tablo 1.5'te

(Bölüm 1.7) listelenmiştir. Her kodun bir ondalık basamağı temsil etmek için dört bit kullanması gerektiğinden, bu nedenle

Tablo 4.2

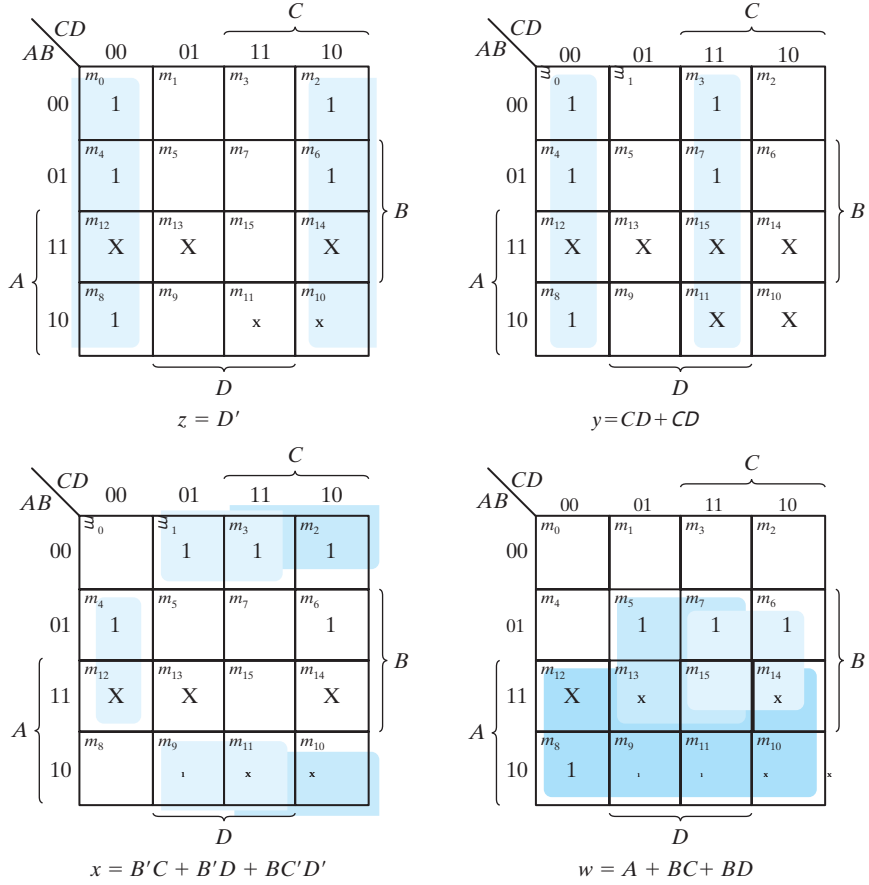
Kod Dönüşüm Örneği için Gerçeklik Tablosu

Girdi BCD				Çıktı Aşırı-3 Kodu			
				z			
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

dört giriş değişkeni ve dört çıkış değişkeni vardır. Dört giriş ikili değişkenini sembollerle $A, B, C,$ ve $D,$ ve dört çıkış değişkenini $w, x, y,$ ve z ile adlandırıyoruz. Giriş ve çıkış değişkenleri arasındaki ilişkiyi gösteren doğru tablo Tablo 4.2 'de verilmiştir. Girişler için bit kombinasyonları ve bunların karşılık gelen çıkışları doğrudan Bölüm 1.7'den elde edilmiştir. Dört ikili değişkenin 16 bit kombinasyonu olabileceğini, ancak doğru tabloda yalnızca 10'un listelendiğini unutmayın. Giriş değişkenleri için listelenmeyen altı bit kombinasyonu, dikkate alınmayan kombinasyonlardır. Bu değerlerin BCD'de hiçbir anlamı yoktur ve bunların devrenin gerçek çalışmasında asla meydana gelmeyeceğini varsayıyoruz. Bu nedenle, çıkış değişkenlerine ya 1 ya da 0 atama özgürlüğüne sahibiz; hangisi daha basit bir devre veriyorsa onu kullanabiliriz.

Şekil 4.3'teki haritalar, çıkışlar için basitleştirilmiş Boolean fonksiyonlarını elde etmek amacıyla çizilmiştir. Dört haritanın her biri, devrenin dört giriş değişkeninin bir fonksiyonu olarak dört çıkıştan birini temsil etmektedir. Karelerin içinde işaretlenen 1'ler, çıkışı 1 yapan mintermlerden elde edilmiştir. 1'ler, çıkış sütunlarını birer birer geçerek doğruluk tablosundan elde edilir. Örneğin, çıkış z altında bulunan sütun beş 1'e sahiptir; bu nedenle, z için harita beş 1 içerir ve her biri z 'yi 1 yapan minterm ile ilişkili bir karede yer alır. 10'dan 15'e kadar olan altı "önemsiz" minterm X ile işaretlenmiştir. Fonksiyonları çarpanların toplamı biçimine basitleştirmenin bir olası yolu, her değişkenin haritasının altında listelenmiştir. (Bkz. Bölüm 3.)

Her çıkış için iki seviyeli bir mantık diyagramı, haritalardan türetilen Boolean ifadelerinden doğrudan elde edilebilir. Bu devreyi uygulayan mantık diyagramı için çeşitli diğer olasılıklar vardır. Şekil 4.3'te elde edilen ifadeler, iki veya daha fazla çıkış için ortak kapılar kullanma amacıyla cebirsel olarak işlenebilir. Aşağıda gösterilen bu işlem, çoklu çıkış sistemlerinde elde edilen esnekliği göstermektedir.



ŞEKİL 4.3

BCD'den fazla-3 kod dönüştürücü için haritalar

üç veya daha fazla seviye kapılarla uygulanmıştır:

$$z = D'$$

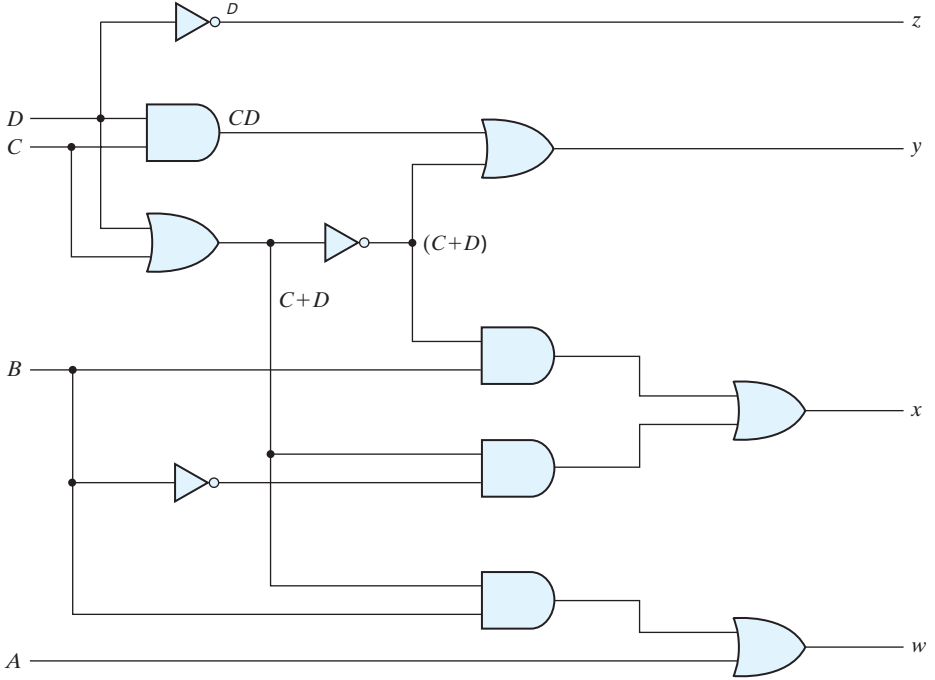
$$y = CD + C'D' = CD + (C + D)'$$

$$\begin{aligned} x &= B'C + B'D + BC'D' = B'(C + D) + BC'D' \\ &= B'(C + D) + B(C + D)' \end{aligned}$$

$$w = A + BC + BD = A + B(C + D)$$

Bu ifadeleri uygulayan mantık diyagramı Şekil 4.4'te gösterilmiştir. Çıkışı $C+D$ olan VEYA kapısının, üç çıkışın her birini kısmen uygulamak için kullanıldığını unutmayın.

Giriş inverterleri sayılmadığında, ürünlerin toplamı biçimindeki uygulama ya da VE kapısı ve üç VEYA kapısı gerektirir. Şekil 4.4'teki uygulama dört VE kapısı, dört VEYA kapısı ve bir inverter gerektirir. Sadece normal girişler mevcutsa, ilk



ŞEKİL 4.4

BCD'den fazladan-3 kod dönüştürücü için mantık diyagramı

uygulama, değişkenler için inverterler gerektirecektir B , C , ve D , ve ikinci uygulama, değişkenler için inverterler gerektirecektir B ve D . Böylece, üç seviyeli mantık circuit, hepsi de en fazla iki girdi gerektiren daha az kapı gerektirir.

4.5 İKİLİ TOPLAYICI-ÇIKARICI

Dijital bilgisayarlar çeşitli bilgi işleme görevlerini yerine getirir. Karşılaşılan işlevler arasında çeşitli aritmetik işlemler bulunmaktadır. En temel aritmetik işlem, iki ikili basamağın toplanmasıdır. Bu basit toplama, dört olası temel işlemi içerir: $0+0=0$, $0+1=1$, $1+0=1$, ve $1+1=10$. İlk üç işlem bir basamaklı bir toplam üretirken, hem artıran hem de eklenen bitler 1'e eşit olduğunda, ikili toplam iki basamaktan oluşur. Bu sonucun daha yüksek anlamlı biti *taşma* olarak adlandırılır. Artıran ve eklenen sayılar daha anlamlı basamaklar içerdiğinde, iki bitin toplanmasından elde edilen taşma, bir sonraki daha yüksek sıradaki anlamlı bit çiftine eklenir. İki bitin toplanmasını gerçekleştiren bir kombinasyon devresi *yarım toplayıcı* olarak adlandırılır. Üç bitin (iki anlamlı bit ve bir önceki taşma) toplanmasını gerçekleştiren bir devre ise *tam toplayıcı* olarak adlandırılır. Devrelerin isimleri, iki yarım toplayıcının bir tam toplayıcıyı uygulamak için kullanılabileceği gerçeğinden kaynaklanmaktadır.

İkili toplayıcı-çıkarcı, ikili sayılarla toplama ve çıkarma aritmetik işlemlerini gerçekleştiren bir kombinasyonel devredir. Bu devreyi hiyerarşik bir tasarım aracılığıyla geliştireceğiz. Öncelikle yarım toplayıcı tasarımı gerçekleştirilir, ardından tam toplayıcı geliştirilir. n tam toplayıcıyı ardışık olarak bağlamak, iki n bitlik sayı için bir ikili toplayıcı üretir. Çıkarma devresi, bir tamamlayıcı devrede yer alır.

Yarım Toplayıcı

Yarım toplayıcının sözlü açıklamasından, bu devrenin iki ikili girdi ve iki ikili çıkışa ihtiyaç duyduğunu buluyoruz. Girdi değişkenleri, toplama ve ekleme bitlerini belirtir; çıkış değişkenleri toplam ve taşıma üretir. İki girdi için x ve y sembollerini, çıkışlar için S (toplam için) ve C (taşıma için) atıyoruz. Yarım toplayıcının doğruluk tablosunu Tablo 4.3'te listelenmiştir. C çıkışı yalnızca her iki girdi 1 olduğunda 1'dir. S çıkışı, toplamın en az anlamlı bitini temsil eder.

İki çıkış için basitleştirilmiş Boolean fonksiyonları doğrudan doğruluk tablosundan elde edilebilir. Basitleştirilmiş çarpanların toplamı ifadeleri şunlardır:

$$S = x \oplus xy$$

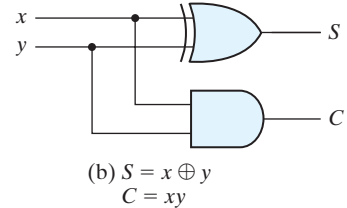
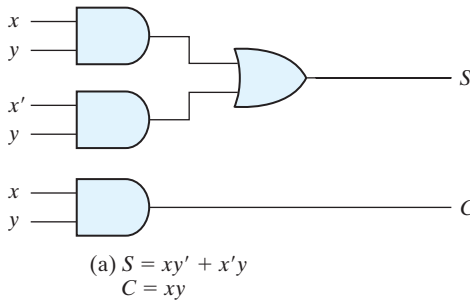
$$C = xy$$

Yarım toplayıcının ürünlerin toplamı şeklinde uygulanan mantık diyagramı Şekil 4.5(a)'de gösterilmiştir. Ayrıca, Şekil 4.5(b)'de gösterildiği gibi bir XOR ve bir AND kapısıyla da uygulanabilir. Bu form, iki yarım toplayıcının bir tam toplayıcı inşa etmek için kullanılabileceğini göstermektedir.

Tablo 4.3

Yarım Toplayıcı

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



ŞEKİL 4.5

Yarım toplayıcının uygulanması

Tam Toplayıcı

n -bit ikili sayıların toplanması, bir tam toplayıcının kullanılmasını gerektirir ve toplama işlemi, en az anlamlı bittten başlayarak, soldan sağa, bit bit ilerler. En az anlamlı bittten sonra, her pozisyondaki toplama, yalnızca kelimelerin ilgili bitlerini eklemekle kalmaz, aynı zamanda önceki pozisyondaki toplama işlemiyle oluşabilecek bir taşıma bitini de dikkate almalıdır.

Tam toplayıcı, üç bitin aritmetik toplamını oluşturan bir kombinasyon devresidir. Üç girişi ve iki çıkışı vardır. Giriş değişkenlerinden ikisi, x ve y , toplanacak iki anlamlı biti temsil eder. Üçüncü giriş, z , önceki daha düşük anlamlı pozisyonlardan gelen taşıma bitini temsil eder. Üç çıkış gereklidir çünkü üç ikili basamağın aritmetik toplamı 0 ile 3 arasında bir değere sahiptir ve 2 veya 3'ün ikili temsili iki bite ihtiyaç duyar. İki çıkış, toplam için S ve taşıma için C sembolleriyle belirtilir. İkili değişken S toplamın en az anlamlı bitinin değerini verir. İkili değişken C giriş taşıması ve kelimelerin bitlerini ekleyerek oluşan çıkış taşımasını verir. Tam toplayıcının doğruluk tablosu Tablo 4.4 'te listelenmiştir. Giriş değişkenlerinin altındaki sekiz satır, üç değişkenin tüm olası kombinasyonlarını belirtir. Çıkış değişkenleri, giriş bitlerinin aritmetik toplamından belirlenir. Tüm giriş bitleri 0 olduğunda, çıkış 0'dır. S çıkışı, yalnızca bir giriş 1 olduğunda veya tüm giriş 1 olduğunda 1'e eşittir. C çıkışı, iki veya üç giriş 1 olduğunda 1 taşımasına sahiptir.

Kombinasyonel devrenin giriş ve çıkış bitleri, problemin çeşitli aşamalarında farklı yorumlara sahiptir. Bir yandan, fiziksel olarak, girişlerin ikili sinyalleri, çıkışta iki basamaklı bir toplam oluşturmak için aritmetik olarak eklenmesi gereken ikili rakamlar olarak kabul edilir. Öte yandan, aynı ikili değerler, doğruluk tablosunda ifade edildiğinde veya devre mantık kapıları ile uygulandığında Boolean fonksiyonlarının değişkenleri olarak kabul edilir. Tam toplayıcının çıkışları için haritalar Şekil 4.6 'da gösterilmiştir. Basitleştirilmiş ifadeler

$$S = x \oplus y \oplus x \cdot y \oplus x \cdot y \cdot z$$

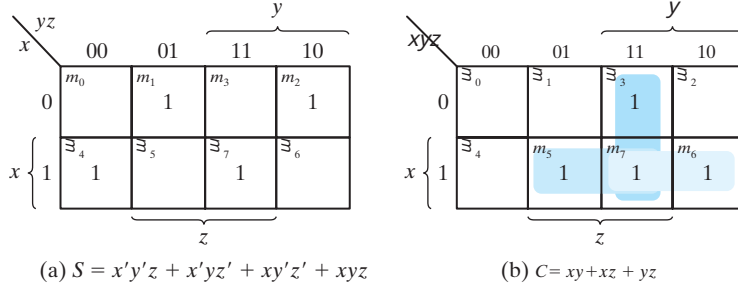
$$C = xy + xz + yz$$

Toplam ürün formunda uygulanan tam toplayıcının mantık diyagramı Şekil 4.7'de gösterilmiştir. Ayrıca, iki yarı toplayıcı ve bir OR kapısıyla da uygulanabilir, bu da Şekil 4.8'de gösterilmiştir.

Tablo 4.4

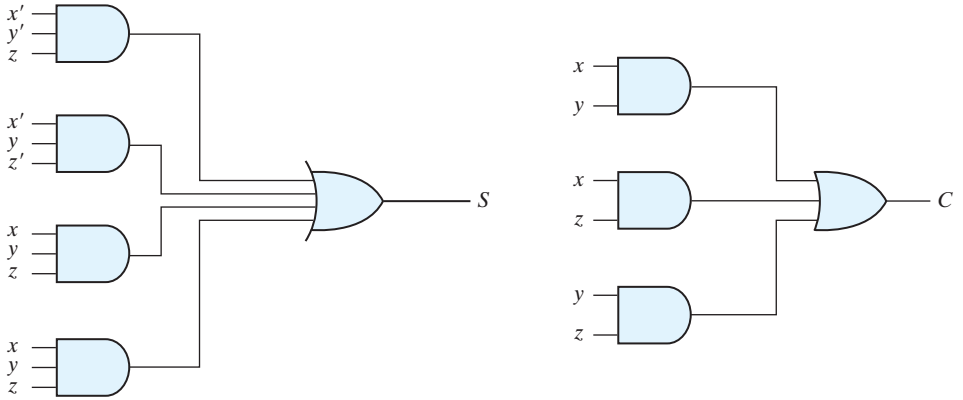
Tam Toplayıcı

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



ŞEKİL 4.6

K-topları tam toplayıcı için



ŞEKİL 4.7

Ürünlerin toplamı biçiminde tam toplayıcının uygulanması

İkinci yarı toplayıcının S çıkışı, z'nin ve ilk yarı toplayıcının çıkışının özel-VE'sidir, bu da

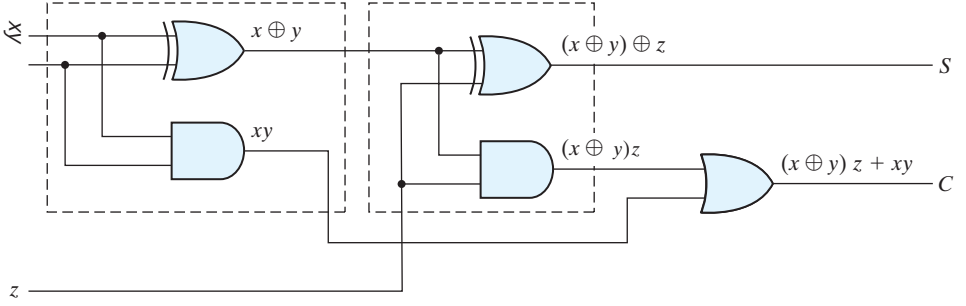
$$\begin{aligned}
 S &= z \oplus (x \oplus y) \\
 &= z(xy + x'y) + z(xy' + x'y') \\
 &= z(xy + x'y) + z(xy' + x'y') \\
 &= xy + x'y + xy' + x'y'
 \end{aligned}$$

Taşıma çıkışı

$$C = z(xy' + x'y) + xy = xy'z + x'yz + xyz$$

İkili Toplayıcı

İkili toplayıcı, iki ikili sayının aritmetik toplamını üreten dijital bir devredir. Tam toplayıcıların ardışık olarak bağlanmasıyla inşa edilebilir; her tam toplayıcının çıkışı taşı, zincirdeki bir sonraki tam toplayıcının giriş taşı ile bağlanır.



ŞEKİL 4.8

İki yarım toplayıcı ve bir VEYA kapısıyla tam toplayıcının uygulanması

n -bit sayıların toplanması, n tam toplayıcıdan oluşan bir zincir veya bir yarım toplayıcı ve $n-1$ tam toplayıcıdan oluşan bir zincir gerektirir. İlk durumda, en az anlamlı konuma gelen giriş taşı 0 olarak sabitlenmiştir. Şekil 4.9, dört bitlik ikili dalgalı taşıyıcı toplayıcı sağlamak için dört tam toplayıcı (FA) devresinin bağlantısını göstermektedir. A'nın toplama bitleri ve B'nin toplama bitleri, sağdan sola doğru alt simge numaraları ile belirtilmiştir; alt simge 0, en az anlamlı biti ifade eder. Taşımalar, tam toplayıcılar aracılığıyla bir zincir halinde bağlanmıştır. Toplayıcıya gelen giriş taşı C_0 'dur ve bu, tam toplayıcılardan çıkış taşı olarak dalgalanır.

C4. Çıktılar, gerekli toplam bitlerini üretir. Bir n bit toplayıcı, n tam toplayıcıya ihtiyaç duyar, her bir çıkış taşı, bir sonraki daha yüksek dereceli tam toplayıcının giriş taşı ile bağlantılıdır.

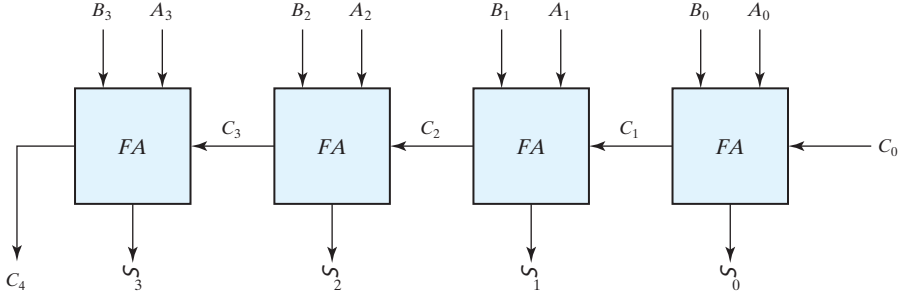
Özel bir örnekle göstermek gerekirse, iki ikili sayıyı ele alalım: $A=1011$

ve $B=0011$. Toplamları $S=1110$ dört bitli toplayıcı ile aşağıdaki gibi oluşturulur:

Alt simge n					
Giriş taşı	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Toplayıcı	0	0	1	1	B_i
Toplam	1	1	1	0	S_i
Çıkış taşı	0	0	1	1	C_{i+1}

Bitler, en az anlamlı konumdan (alt simge 0) başlayarak tam toplayıcılarla toplanır ve toplam bit ile taşıma bitini oluşturur. En az anlamlı konumdaki giriş taşı $C_0 = 0$ olmalıdır. Verilen anlamlı konumdaki C_{i+1} değeri, tam toplayıcının çıkış taşıdır. Bu değer, bitleri bir üst anlamlı konuma ekleyen tam toplayıcının giriş taşına aktarılır. Toplam bitler, en sağdaki konumdan başlayarak üretilir ve ilgilili önceki taşıma biti oluşturulduğu anda kullanılabilir hale gelir. Doğru toplam bitlerin çıkışlarda görünmesi için tüm taşların üretilmesi gerekmektedir.

Dört bitli toplayıcı, standart bir bileşenin tipik bir örneğidir. Aritmetik işlemlerle ilgili birçok uygulamada kullanılabilir. Bu devrenin tasarımına dikkat edin.



ŞEKİL 4.9

Dört bitli toplayıcı

Klasik yöntemle, devreye dokuz giriş olduğu için $2^9 = 512$ girişe sahip bir doğruluk tablosu gerekecektir. Standart bir fonksiyonun kaskadlama yöntemi kullanılarak, basit ve doğrudan bir uygulama elde etmek mümkündür.

Taşıma Yayılımı

İki ikili sayının paralel olarak toplanması, toplama ve ekleme işlemlerinin tüm bitlerinin aynı anda hesaplama için mevcut olduğu anlamına gelir. Herhangi bir kombinasyon devresinde olduğu gibi, sinyal doğru çıkış toplamı çıkış terminallerinde mevcut olmadan önce kapılardan geçmelidir. Toplam yayılma süresi, tipik bir kapının yayılma gecikmesi ile devredeki kapı seviyelerinin sayısının çarpımına eşittir. Bir toplayıcıdaki en uzun yayılma gecikme süresi, taşımanın tam toplayıcılardan geçmesi için gereken zamandır. Toplam çıkışın her bir biti, giriş taşımasının değerine bağlı olduğundan, toplayıcının herhangi bir aşamasındaki S'nin değeri, o aşamaya giriş taşıması yayılmadan yalnızca kararlı son değerinde olacaktır. Bu bağlamda, Şekil 4.9'daki çıkış S3 değerini düşünün. Giriş A3 ve B3, giriş sinyalleri toplayıcıya uygulandığı anda mevcut olacaktır.

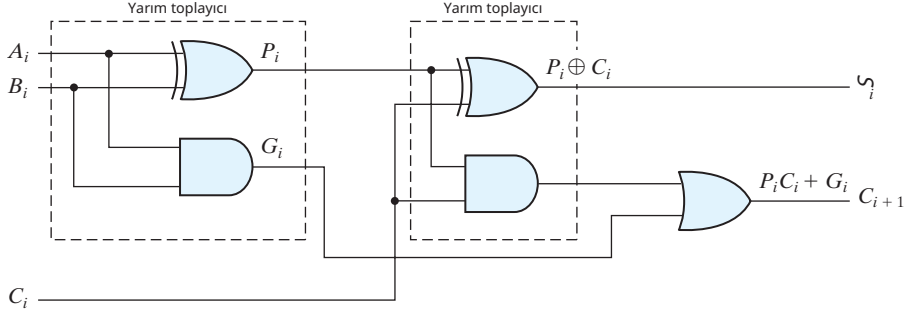
Ancak, giriş taşıması C3 nihai değerine, önceki aşamadan C2 mevcut olana kadar yerleşmez. Benzer şekilde, C2 C1 için beklemek zorundadır ve bu şekilde C0'a kadar devam eder.

Böylece, taşıma tüm aşamalardan geçip dalgalandığında, son çıkış S3 ve taşıma C4 nihai doğru değerine yerleşecektir.

Taşıma yayılma için kapı seviyelerinin sayısı, tam toplayıcının devresinden bulunabilir. Devre, kolaylık sağlamak amacıyla Şekil 4.10'da farklı etiketlerle yeniden çizilmiştir.

Girdi ve çıktı değişkenleri, toplayıcının tipik bir aşamasını belirtmek için i alt simgesini kullanır. Giriş sinyalleri P_i ve G_i ilgili kapılardan geçtikten sonra kararlı durum değerlerine yerleşir. Bu iki sinyal, tüm yarım toplayıcılara ortaktır ve yalnızca giriş toplama ve ekleme bitlerine bağlıdır. Giriş taşıma sinyali C_i çıkış taşıma sinyali C_{i+1} üzerinden bir AND kapısı ve bir OR kapısından geçerek iki kapı seviyesini oluşturur. Eğer toplayıcıda dört tam toplayıcı varsa, çıkış taşıma sinyali C_4 olacaktır.

$2 \times 4 = 8$ kapı seviyeleri C_0 ile C_4 arasında. Bir n bit toplayıcı için, 2^n kapı seviyesi vardır. girişten çıkışa taşımak için yaymak.

**ŞEKİL 4.10**Görüntülenen P ve G ile tam toplayıcı

Taşıma yayılma süresi, iki sayının toplandığı hızla sınırlı olduğu için toplayıcının önemli bir özelliğidir. Toplayıcı—ya da herhangi bir kombinasyon devresi—her zaman çıkış terminallerinde bir değer bulursa da, sinyallerin girişlerden çıkışlara bağlı kapılarda yayılması için yeterli zaman verilmediği sürece çıkışlar doğru olmaya caktır. Diğer tüm aritmetik işlemler ardışık toplama ile gerçekleştirildiğinden, toplama sürecinde harcanan zaman kritik öneme sahiptir. Taşıma yayılma gecikme süresini azaltmanın bariz bir çözümü, gecikmeleri azaltılmış daha hızlı kapılar kullanmaktır. Ancak, fiziksel devrelerin bir kapasite sınırı vardır. Diğer bir çözüm, taşıma gecikme süresinin azaltılacak şekilde ekipmanın karmaşıklığını artırmaktır. Paralel toplayıcılarda taşıma yayılma süresini azaltmak için birkaç teknik bulunmaktadır. En yaygın kullanılan teknik, taşıma önceden bakış mantığı ilkesini kullanmaktadır.

Şekil 4.10'da gösterilen tam toplayıcı devresini düşünün. İki yeni ikili değişken tanımlarsak

$$P_i = A_i \oplus B_i$$

$$G_i = A_i B_i$$

Çıktı toplamı ve taşıma sırasıyla şu şekilde ifade edilebilir:

$$S_i = P_i \oplus C_i$$

$$C_{i+1} = G_i + P_i C_i$$

G_i , *taşıma üreten* olarak adlandırılır ve hem A_i hem de B_i 1 olduğunda 1 taşıma üretir, input taşıma C_i 'ye bakılmaksızın. P_i , *taşıma yayılma* olarak adlandırılır, çünkü bir taşımanın i aşamasına girmesinin $i+1$ aşamasına yayılıp yayılmayacağını belirler (yani, C_i 'ye bir beyanın C_{i+1} 'de bir beyan yayılıp yayılmayacağı).

Şimdi her aşamanın taşıma çıkışları için Boolean fonksiyonlarını yazıyoruz ve önceki denklemlerden her C_i değerini yerine koyuyoruz:

$$C_0 = \text{giriş taşıması}$$

$$C_1 = G_0 + P_0 C_0$$

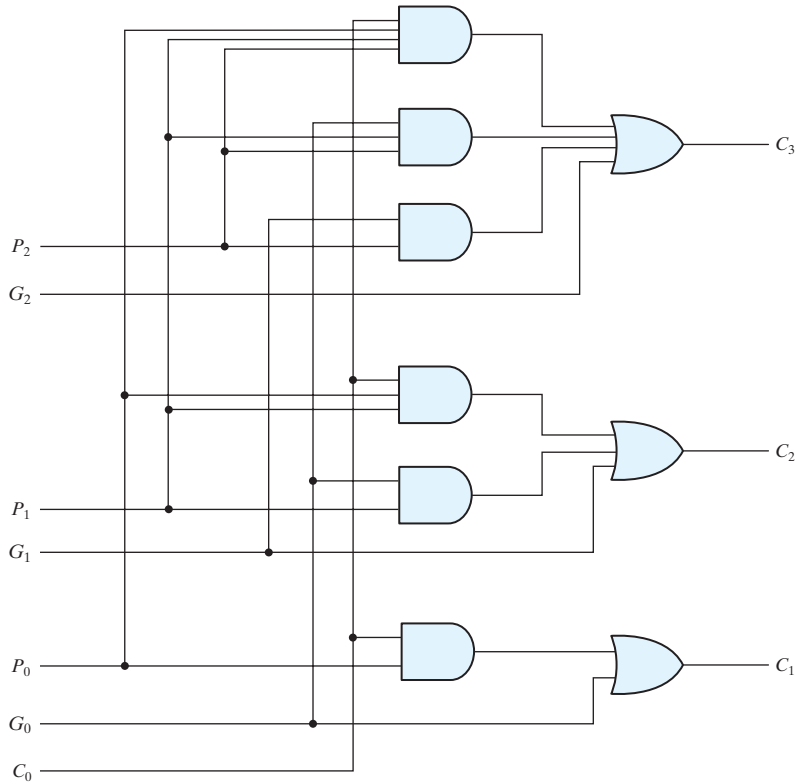
$$C_2 = G_1 + P_1 C_1 = G_1 + P_1 (G_0 + P_0 C_0) = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 = P_2 P_1 P_0 C_0$$

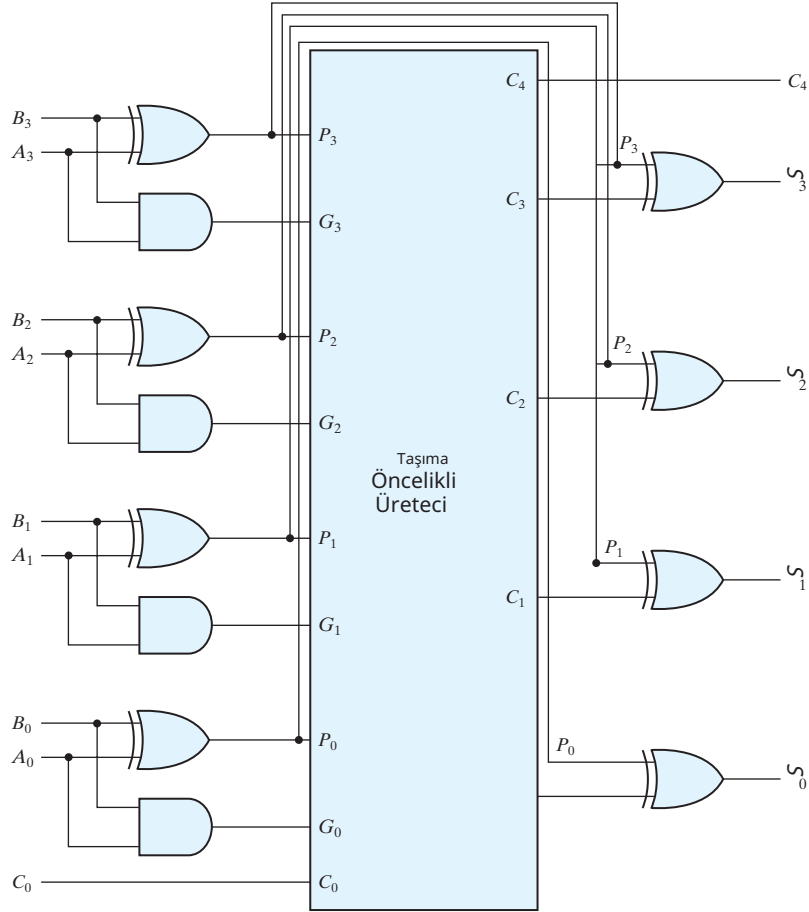
Her bir çıkış taşıma için Boolean fonksiyonu, ürünlerin toplamı biçiminde ifade edildiğinden, each function can be implemented with one level of AND gates followed by an OR gate (or by a two-level NAND). C_1 , C_2 ve C_3 için üç Boolean fonksiyonu, Şekil 4.11'de gösterilen taşıma önceden tahmin edici içinde uygulanmıştır. Bu devrenin daha az zama

nda toplama yapabileceğini unutmayın çünkü C_3 , C_2 ve C_1 'in yayılmasını beklemek zorunda değildir; aslında, C_3 , C_1 ve C_2 ile aynı anda yayılır. Bu işlem hızındaki kazanç, ek karmaşıklık (donanım) pahasına elde edilir.

Taşıma öncelikli düzenek ile dört bitlik bir toplayıcının yapısı Şekil 4.12'de gösterilmiştir. Her toplam çıkışı iki özel-VE kapısı gerektirir. İlk özel-VE kapısının çıkışı P_i değişkenini üretir ve VE kapısı G_i değişkenini üretir. Taşıma değerleri, taşıma öncelikli üretici aracılığıyla (Şekil 4.11'deki gibi) iletilir ve ikinci özel-VE kapısına giriş olarak uygulanır. Tüm çıkış taşıma değerleri,



ŞEKİL 4.11
Taşıma öncelikli üretcin mantık diyagramı



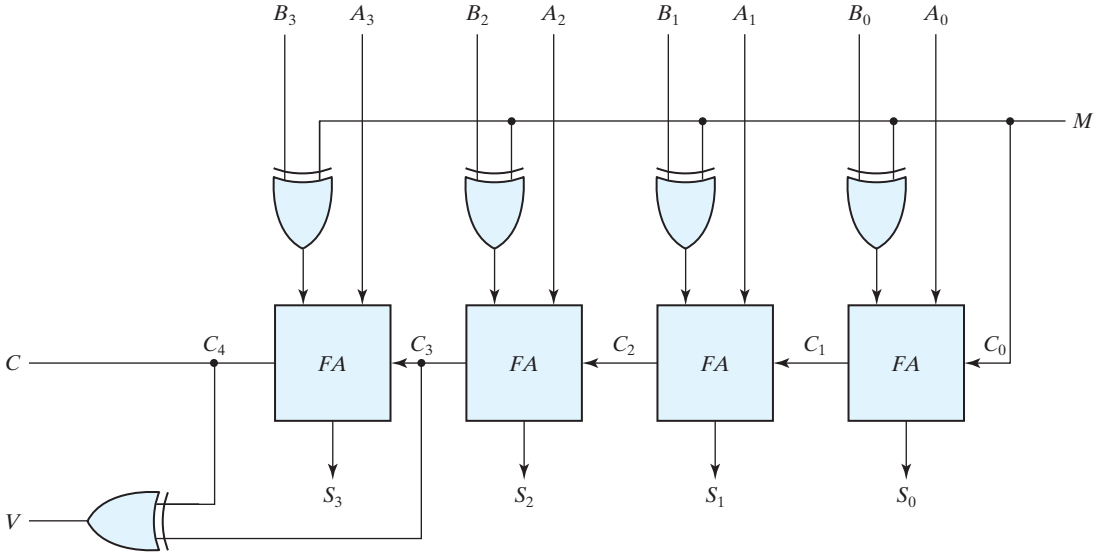
ŞEKİL 4.12

Dört bitli toplayıcı ile taşıma önceliği

iki seviye kapıdan geçen bir gecikme. Böylece, çıkışlar S_1 ile S_3 eşit yayılma gecikme sürelerine sahiptir. Çıkış taşıması için iki seviyeli devre C_4 gösterilmemiştir. Bu devre, d enklem-değiştirme yöntemiyle kolayca türetilir.

İkili Çıkarıcı

İşaretsiz ikili sayıların çıkarılması, Bölüm 1.5'te tartışıldığı gibi, en uygun şekilde tamamlamalar aracılığıyla yapılabilir. Unutmayın ki çıkarma $A-B$ işlemi, B sayısının 2'nin tamamlanmasını alarak ve bunu A ile toplayarak yapılabilir. 2'nin tamamlanması, 1'in tamamlanmasını alarak ve en az anlamlı bit çiftine 1 ekleyerek elde edilebilir. 1'in tamamlanması, inverterler ile uygulanabilir ve toplamaya giriş taşı aracılığıyla 1 eklenebilir.



ŞEKİL 4.13

Dört bitli toplayıcı-çıkarcı (taşma tespiti ile)

Çıkarma devresi $A-B$, her veri girişi B ile tam toplayıcının karşılık gelen girişi A arasında yerleştirilen inverterlerden oluşur. Giriş taşıması C_0 , çıkarma işlemi yapıldığında 1'e eşit olmalıdır. Böylece gerçekleştirilen işlem A , B 'nin 1'in tamamlayıcısı artı 1 olur. Bu, A ile B 'nin 2'nin tamamlayıcısının toplamına eşittir. İşaretsiz sayılar için, bu $A-B$ verir eğer $A \geq B$ veya $(B-A)$ 'nin 2'nin tamamlayıcısı eğer $A < B$. İşaretsiz sayılar için, sonuç $A-B$, taşma olmadığı sürece geçerlidir. (Bkz. Bölüm 1.6.)

Toplama ve çıkarma işlemleri, her tam toplayıcı ile bir özel-VE kapısı eklenerek tek bir devrede birleştirilebilir. Dört bitli toplayıcı-çıkarcı devresi Şekil 4.13'te gösterilmiştir. Mod girişi M işlemi kontrol eder.

When $M=0$, the circuit is an adder, and when $M=1$, the circuit becomes a subtractor. Her bir özel-VE kapısı, M girişini ve B 'nin girişlerinden birini alır. $M=0$ olduğunda, $B \oplus 0 = B$. Tam toplayıcılar, B değerini alır, giriş taşıyıcısı 0'dır ve circuit, A ile B 'yi toplar. $M=1$ olduğunda, $B \oplus 1 = \bar{B}$ ve $C_0 = 1$. B girişleri hepsi tamamlanmış ve giriş taşıyıcısı aracılığıyla 1 eklenmiştir. Devre, A ile B 'nin 2'nin tamamlayıcısını toplar. (Çıktısı V olan özel-VE, taşmayı tespit etmek içindir.)

Önemli bir nokta, işaretli-tamamlayıcı sistemdeki ikili sayıların, işaretsiz sayılarla aynı temel toplama ve çıkarma kurallarıyla toplandığı ve çıkarıldığıdır. Bu nedenle, bilgisayarların her iki tür aritmetiği işlemek için yalnızca bir ortak donanım devresine ihtiyacı vardır. Kullanıcı veya programcı, bu tür toplama veya çıkarma sonuçlarını, sayıların işaretli mi yoksa işaretsiz mi olduğu varsayıldığına bağlı olarak farklı yorumlamalıdır.

Taşma

İki n basamaklı sayılar toplandığında ve toplam, bir sayı oluşturduğunda $n + 1$ basamak, taşma meydana geldiğini söyleriz. Bu, ikili veya ondalık sayılar için, işaretli veya işaretsiz olanlar için doğrudur. Toplama kağıt ve kalemle yapıldığında, taşma bir sorun değildir, çünkü toplamı yazmak için sayfanın genişliğiyle sınırlı bir durum yoktur. Taşma, dijital bilgisayarlarda bir sorundur çünkü sayıyı tutan bit sayısı sınırlıdır ve $n + 1$ bit içeren bir sonuç, n bitlik bir kelime tarafından barındırılmaz. Bu nedenle, birçok bilgisayar taşmanın meydana geldiğini tespit eder ve meydana geldiğinde, kullanıcı tarafından kontrol edilebilecek bir flip-flop ayarlanır.

İki ikili sayının toplamından sonra taşmanın tespiti, sayıların işaretli mi yoksa işaretsiz mi olarak kabul edildiğine bağlıdır. İki işaretsiz sayı toplandığında, taşma en anlamlı pozisyonundan çıkan son taşma ile tespit edilir. İşaretli sayılar durumunda, iki detay önemlidir: en soldaki bit her zaman işareti temsil eder ve negatif sayılar 2'nin tamamlayıcı formundadır. İki işaretli sayı toplandığında, işaret biti sayının bir parçası olarak değerlendirilir ve son taşma bir taşma göstermez.

Bir toplama işleminden sonra bir taşma (overflow) meydana gelemez, eğer bir sayı pozitif ve diğeri negatifse, çünkü pozitif bir sayıyı negatif bir sayıya eklemek, iki orijinal sayıdan daha büyük olanın mutlak değerinden daha küçük bir sonuç üretir. İki eklenen sayı da pozitif veya her ikisi de negatifse bir taşma meydana gelebilir. Bunun nasıl olabileceğini görmek için, aşağıdaki örneği düşünün: İki işaretli ikili sayı, $+70$ ve $+80$, iki sekiz bitlik kaydedicide saklanmaktadır. Her kaydedicinin barındırabileceği sayı aralığı, ikili $+127$ ile ikili -128 arasındadır. İki sayının toplamı $+150$ olduğundan, bu sekiz bitlik kaydedicinin kapasitesini aşmaktadır. Bu, -70 ve -80 için de doğrudur. İkili olarak yapılan iki toplama işlemi aşağıda gösterilmiştir, son iki taşma ile birlikte:

taşmalar:	0 1	taşmalar:	1 0
$+70$	0 1000110	-70	1 0111010
$+80$	0 1010000	-80	1 0110000
$+150$	1 0010110	-150	0 1101010

Not edin ki, pozitif olması gereken sekiz bitlik sonuçta negatif bir işaret biti (yani, sekizinci bit) vardır ve negatif olması gereken sekiz bitlik sonuçta pozitif bir işaret biti bulunmaktadır. Ancak, işaret biti pozisyonundaki taşmanın sonuç işaret biti olarak alındığında, elde edilen dokuz bitlik cevap doğru olacaktır. Fakat cevap sekiz bit içinde yer almadığı için, bir taşma meydana geldiğini söyleriz.

Taşma durumu, işaret bit pozisyonuna taşıma ve işaret bit pozisyonundan taşıma gözlemlenerek tespit edilebilir. Bu iki taşıma eşit değilse, bir taşma meydana gelmiştir. Bu, iki taşmanın açıkça gösterildiği örneklerde belirtilmiştir. İki taşıma bir XOR kapısına uygulandığında, kapının çıkışı 1'e eşit olduğunda bir taşma tespit edilir. Bu yöntemin doğru çalışması için, negatif bir sayının 2'nin tamamlayıcısı, 1'in tamamlayıcısını alıp 1 eklenerek hesaplanmalıdır.

Bu, maksimum negatif sayının tamamlandığı durumu ele alır.

Çıktıları C ve V olan ikili toplayıcı-çıkarcı devresi Şekil 4.13'te gösterilmiştir. Eğer ikili sayı işaretli olarak kabul edilirse, o zaman C biti toplama sonrası bir taşıma veya çıkarma sonrası bir borç tespit eder. Sayılar işaretli olarak kabul edilirse, o zaman V biti bir taşıma tespit eder. Eğer toplama veya çıkarma sonrası $V=0$ ise, o zaman taşma meydana gelmemiştir ve n bit sonucu doğrudur. Eğer $V=1$ ise, o zaman işlemin sonucu $n+1$ bit içerir, ancak sayının yalnızca en sağdaki n bitleri mevcut alana sığar, bu nedenle bir taşma meydana gelmiştir. $(n+1)$ th bit gerçek işarettir ve yerinden kaydırılmıştır.

4.6 ONDALIK TOPLAYICI

Bilgisayarlar veya hesap makineleri, aritmetik işlemleri doğrudan ondalık sayı sisteminde gerçekleştiren, ondalık sayıları ikili kodlu biçimde temsil eder. Böyle bir bilgisayar için bir toplayıcı, kodlanmış ondalık sayıları kabul eden ve sonuçları aynı koda sunan aritmetik devreler kullanmalıdır. İkili toplama için, önemli bit çiftini ve önceki bir taşıyıcıyı birlikte düşünmek yeterlidir. Bir ondalık toplayıcı, her ondalık basamağı kodlamak için dört bit gerektiğinden ve devrenin bir giriş ve çıkış taşıyıcısına sahip olması gerektiğinden, en az dokuz giriş ve beş çıkış gerektirir. Ondalık basamakları temsil etmek için kullanılan koda bağlı olarak, çok çeşitli olası ondalık toplayıcı devreleri vardır. Burada BCD kodu için bir ondalık toplayıcıyı inceliyoruz. (Bkz. Bölüm 1.7.)

BCD Toplayıcı

İki ondalık basamağın BCD'de aritmetik toplanmasını, önceki aşamadan gelen bir giriş taşıyıcısıyla birlikte düşünelim. Her giriş basamağı 9'u geçmediğinden, çıkış toplamı $9+9+1=19$ 'dan büyük olamaz; toplamda 1, bir giriş taşıyıcısıdır. İki BCD basamağını dört bitlik bir ikili toplayıcıya uyguladığımızı varsayalım. Toplayıcı, toplamı *ikili* olarak oluşturacak ve 0 ile 19 arasında bir sonuç üretecektir. Bu ikili sayılar Tablo 4.5'te listelenmiştir ve K , Z_8 , Z_4 , Z_2 , ve Z_1 sembolleriyle etiketlenmiştir. K taşıyıcıdır ve Z harfinin altındaki alt simgeler, BCD kodundaki dört bite atanabilecek 8, 4, 2 ve 1 ağırlıklarını temsil eder. İkili toplamın altında ki sütunlar, dört bitlik ikili toplayıcının çıkışlarında görünen ikili değeri listeler. İki ondalık basamağın çıkış toplamı BCD'de temsil edilmeli ve "BCD Toplamı" altındaki sütunlarda listelenen biçimde görünmelidir. Problem, ikili toplamın BCD toplamındaki sayının doğru BCD basamak temsilini bulmak için bir kural bulmaktır.

Tablonun içeriğini incelerken, ikili toplamın 1001'e eşit veya daha az olduğunda, karşılık gelen BCD sayısının aynı olduğu ve bu nedenle dönüşüm gerektirmediği açıktır. İkili toplam 1001'den büyük olduğunda, geçersiz bir BCD temsili elde ederiz. İkili 6'nın (0110) ikili toplamına eklenmesi, onu doğru BCD temsiliinde dönüştürür ve ayrıca gerekli olan bir çıkış taşı üretir.

Tablo 4.5

BCD Toplayıcısının Türetilmesi

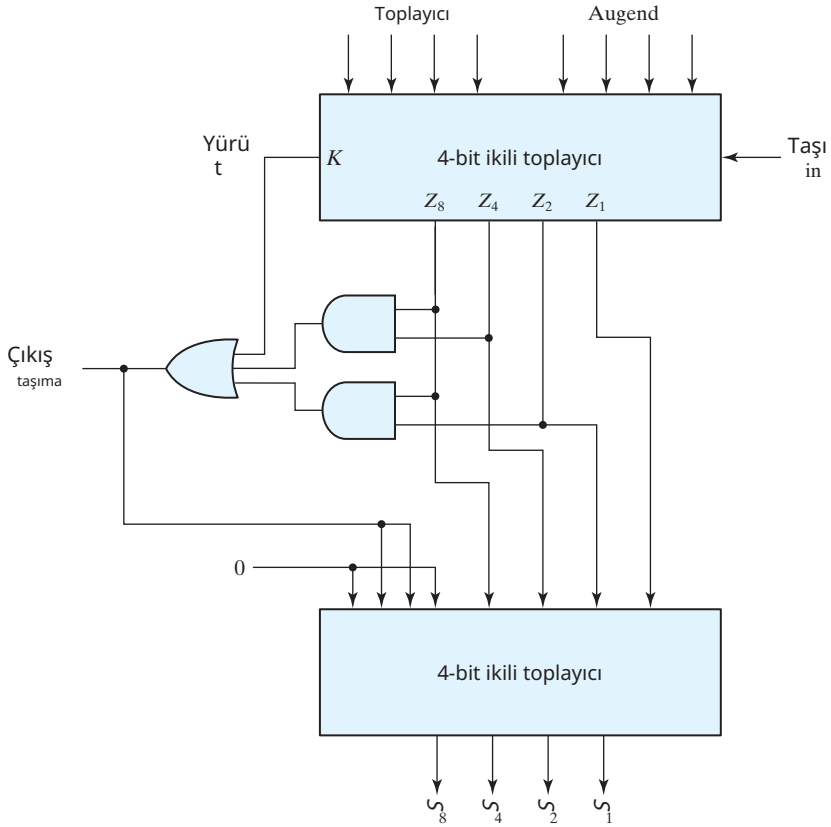
	İkili Toplam				BCD Toplam				Ondalık
	Z_8	Z_4	Z_2	Z_1	S_8	S_4	S_2	S_1	
0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	1	1
0	0	0	1	0	0	0	1	0	2
0	0	0	1	1	0	0	1	1	3
0	0	1	0	0	0	0	1	0	4
0	0	1	0	1	0	0	1	0	5
0	0	1	1	0	0	0	1	1	6
0	0	1	1	1	0	0	1	1	7
0	1	0	0	0	0	1	0	0	8
0	1	0	0	1	0	1	0	1	9
0	1	0	1	0	1	0	0	0	10
0	1	0	1	1	1	0	0	1	11
0	1	1	0	0	1	0	0	1	12
0	1	1	0	1	1	0	0	1	13
0	1	1	1	0	1	0	1	0	14
0	1	1	1	1	1	0	1	1	15
1	0	0	0	0	1	0	1	1	16
1	0	0	0	1	1	0	1	1	17
1	0	0	1	0	1	1	0	0	18
1	0	0	1	1	1	1	0	1	19

Gerekli düzeltmeyi tespit eden mantık devresi, tablodaki girdilerden türetilir. İkili toplamda bir çıkış taşıma olduğunda $K=1$, düzeltmeye ihtiyaç olduğu açıktır. Düzeltme gerektiren 1010'dan 1111'e kadar olan diğer altı kombinasyonu, pozisyonda Z_8 bir 1'e sahiptir. Ayrıca, pozisyonda Z_8 bir 1'e sahip olan iki 1000 ve 1001'den ayırt etmek için, ya Z_4 ya da Z_2 pozisyonunda bir 1 olmalıdır. Düzeltme ve çıkış taşıma koşulu Bool-een fonksiyonu ile ifade edilebilir.

$$C=K+Z_8Z_4+Z_8Z_2$$

$C=1$ olduğunda, ikili toplamın üzerine 0110 eklemek ve bir sonraki aşama için bir çıkış taşı sağlamak gereklidir.

İki BCD rakamını toplayan ve BCD'de bir toplam rakam üreten bir BCD toplayıcı, Şekil 4.14'te gösterilmiştir. İki ondalık rakam, giriş taşı ile birlikte önce üstteki dört bitlik toplayıcıda toplanarak ikili toplamı üretir. Çıkış taşı 0'a eşit olduğunda, ikili toplamına hiçbir şey eklenmez. 1'e eşit olduğunda, ikili toplamına alt dört bitlik toplayıcı aracılığıyla ikili 0110 eklenir. Alt toplayıcıdan üretilen çıkış taşı



ŞEKİL 4.14
BCD toplayıcısının blok diyagramı

toplayıcı göz ardı edilebilir, çünkü zaten çıkış taşıma terminalinde mevcut olan bilgiyi sağlar. n ondalık basamağı ekleyen bir ondalık paralel toplayıcıya n BCD toplayıcı aşamaları gerekir. Bir aşamadan çıkan taşıma, bir sonraki daha yüksek

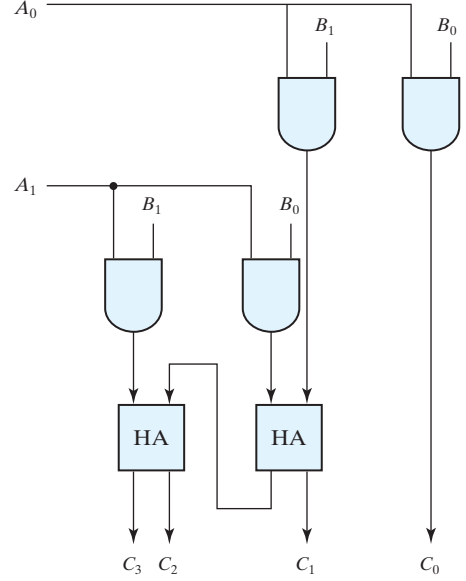
mertebe aşamasının giriş taşımasına bağlanmalıdır.

4.7 İKİLİ ÇARPAN

İkili sayıların çarpımı, ondalık sayıların çarpımıyla aynı şekilde gerçekleştirilir. Çarpın, çarpan her bir bit ile çarpılır, en az anlamlı bittten başlayarak. Her böyle çarpım bir kısmi ürün oluşturur. Başarıyla elde edilen kısmi ürünler bir pozisyon sola kaydırılır. Nihai ürün, kısmi ürünlerin toplamından elde edilir.

İkili bir çarpanın nasıl bir kombinasyonel devre ile uygulanabileceğini görmek için, iki 2-bit sayının çarpımını düşünelim, Şekil 4.15'te gösterildiği gibi. Çarpan

	B_1	B_0	
	A_1	A_0	
	A_0B_1	A_0B_0	
	A_1B_1	A_1B_0	
C_3	C_2	C_1	C_0

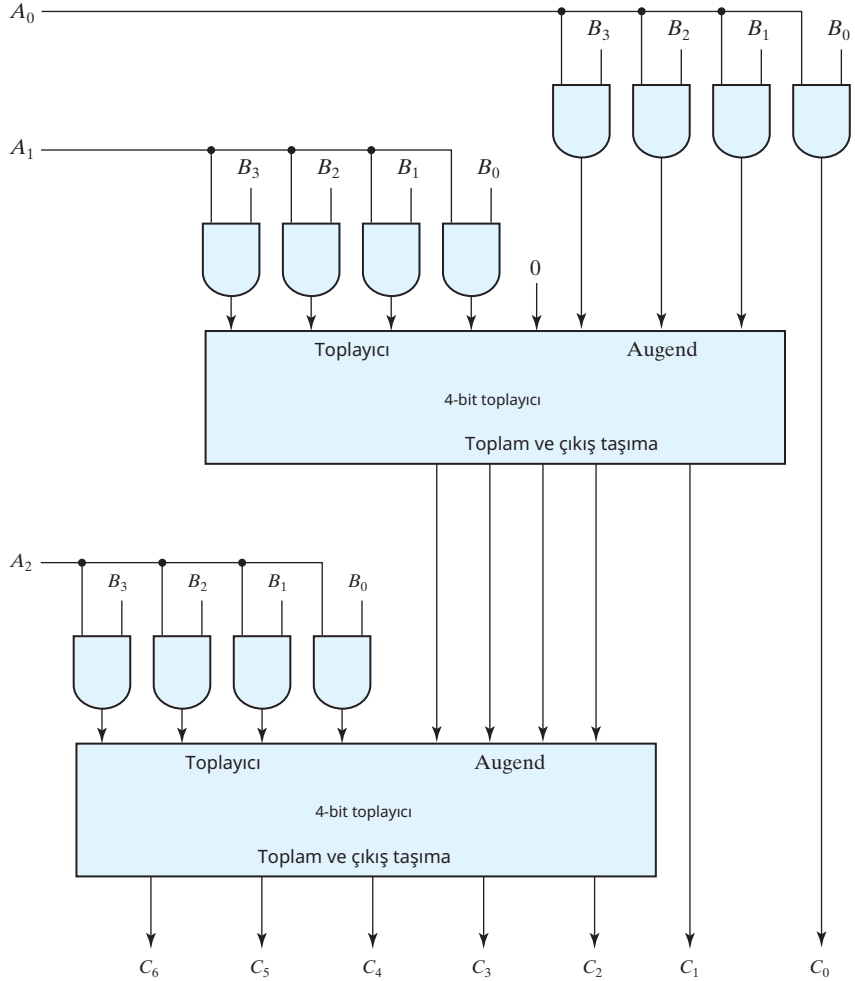


ŞEKİL 4.15
İki bit ile iki bit ikili çarpın

bitler B_1 ve B_0 , çarpın bitleri A_1 ve A_0 , ve ürün $C_3C_2C_1C_0$. İlk kısmi ürün, B_1B_0 ile A_0 çarpılarak oluşturulur. İki bitin çarpımı, A_0 ve B_0 her iki bit de 1 ise 1 üretir; aksi takdirde, 0 üretir. Bu, bir AND işlemi ile aynıdır. Bu nedenle, kısmi ürün, şemada gösterildiği gibi AND kapıları ile uygulanabilir. İkinci kısmi ürün, B_1B_0 ile A_1 çarpılarak ve bir pozisyon sola kaydırılarak oluşturulur. İki kısmi ürün, iki yarım toplayıcı (HA) devresi ile toplanır. Genellikle, kısmi ürünlerde daha fazla bit bulunur ve kısmi ürünlerin toplamını üretmek için tam toplayıcılar kullanmak gereklidir. Ürünün en az anlamlı biti, ilk AND kapısının çıktısı ile oluşturulduğundan, bir toplayıcıdan geçmek zorunda değildir.

Daha fazla bit içeren bir kombinasyonel devre ikili çarpın benzer bir şekilde inşa edilebilir. Çarpın bitinin her bir bit ile AND işlemi, çarpın bitlerinin sayısı k kadar seviyede gerçekleştirilir. Her AND kapısının seviyesindeki ikili çıkış, önceki seviyenin kısmi ürünü ile toplanarak yeni bir kısmi ürün oluşturur. Son seviye ürünü üretir. J çarpın bitleri ve K çarpın bitleri için, $(J \times K)$ AND kapıları ve $(J-1)K$ bit toplayıcıları, $(J+K)$ bitlik bir ürün üretmek için gereklidir.

İkinci bir örnek olarak, dört bit ile temsil edilen bir ikili sayıyı üç bit ile temsil edilen bir sayı ile çarpın bir çarpın devresini düşünelim. Çarpın, $B_3B_2B_1B_0$ ile temsil edilsin ve çarpın ise $A_2A_1A_0$ ile temsil edilsin. $K=4$ ve $J=3$ olduğundan, yedi bitlik bir çarpın üretmek için 12 AND kapısına ve iki adet 4-bit toplayıcıya ihtiyacımız var. Çarpın devresinin mantık diyagramı Şekil 4.16'da gösterilmiştir.



ŞEKİL 4.16
Dört bitlik üç bitlik ikili çarpan

4.8 BÜYÜKLÜK KARŞILAŞTIRICI

İki sayının karşılaştırılması, bir sayının diğerinden büyük, küçük veya eşit olup olmadığını belirleyen bir işlemdir. Bir *büyüklik karşılaştırıcı* iki sayıyı A ve B karşılaştıran ve bunların göreceli büyüklüklerini belirleyen bir kombinyonel devredir. Karşılaştırmının sonucu, $A > B$, $A = B$, veya $A < B$ olup olmadığını gösteren üç ikili değişkenle belirtilir.

Öte yandan, iki n bitlik sayıyı karşılaştıran devre, doğruluk tablosunda 2^{2n} girişi vardır ve $n = 3$ olduğunda bile çok karmaşık hale gelir. Diğer yandan, bir

nebilir, bir karşılaştırmacı devresi belirli bir düzenliliğe sahiptir. Doğal olarak iyi tanımlanmış bir düzenliliğe sahip dijital fonksiyonlar genellikle bir algoritma aracılığıyla tasarlanabilir—bu, izlenirse bir probleme çözüm veren sonlu bir adım setini belirten bir prosedürdür. Bu yöntemi burada dört bitlik bir büyüklük karşılaştırmacısının tasarımı için bir algoritma türeterek gösteriyoruz.

Algoritma, bir kişinin iki sayının göreceli büyüklüklerini karşılaştırmak için kullandığı prosedürün doğrudan bir uygulamasıdır. Dört basamağa sahip iki sayıyı düşünelim, A ve B . Sayıların katsayılarını önem sırasına göre azalan düzende yazın:

$$A = A_3 A_2 A_1 A_0$$

$$B = B_3 B_2 B_1 B_0$$

Her alt simgeli harf, sayının basamaklarından birini temsil eder. İki sayı, tüm önemli basamak çiftleri eşitse eşittir: $A_3=B_3$, $A_2=B_2$, $A_1=B_1$, ve $A_0=B_0$. Sayılar ikili olduğunda, basamaklar ya 1 ya da 0'dır ve her bit çiftinin eşitliği mantıksal olarak bir özel-NOR fonksiyonu ile ifade edilebilir.

$$x_i = A_i B_i + A_i' B_i' \quad i=0, 1, 2, 3$$

burada $x_i = 1$ yalnızca i konumundaki bit çiftleri eşit olduğunda (yani, her ikisi de 1 veya her ikisi de 0 ise) geçerlidir.

İki sayının eşitliği A ve B , bir çıkış ikili değişkeni ile bir kombinasyon devresinde gösterilir ve bu değişkeni $(A=B)$ sembolü ile adlandırırız. Bu ikili değişken, giriş sayıları A ve B eşit olduğunda 1'e eşittir ve aksi takdirde 0'a eşittir.

Eşitliğin var olması için, tüm x_i değişkenlerinin 1'e eşit olması gerekir; bu, tüm değişkenlerin AND işlemini gerektiren bir koşuldur:

$$(A = B) = x_3 x_2 x_1 x_0$$

$(A=B)$ ikili değişkeni yalnızca iki sayının tüm basamak çiftleri eşit olduğunda 1'e eşittir.

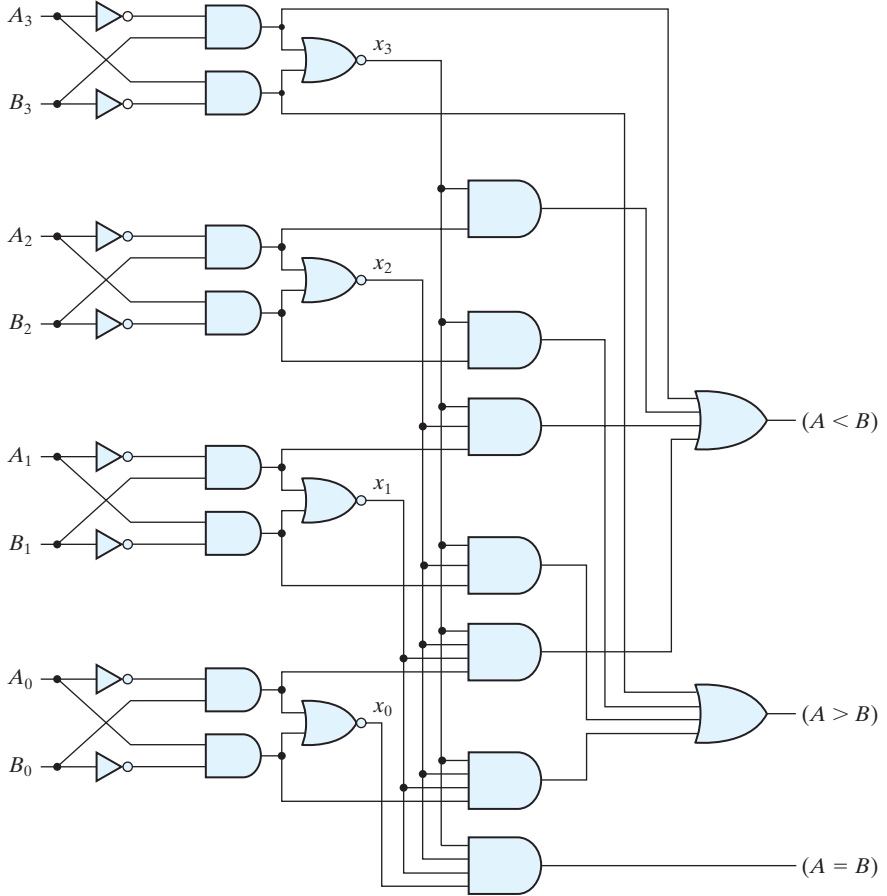
A 'nın B 'den büyük mü yoksa küçük mü olduğunu belirlemek için, en anlamlı konumdan başlayarak önemli basamak çiftlerinin göreceli büyüklüklerini inceleriz. Eğer bir çiftin iki basamağı eşitse, bir sonraki daha düşük anlamlı basamak çiftini karşılaştırırız. Karşılaştırma, eşit olmayan bir basamak çiftine ulaşıncaya kadar devam eder. Eğer A 'nın karşılık gelen basamağı 1 ve B 'nin karşılık gelen basamağı 0 ise, $A > B$ sonucuna varırız. Eğer A 'nın karşılık gelen basamağı 0 ve B 'nin karşılık gelen basamağı 1 ise, $A < B$ sonucuna varırız. Ardışık karşılaştırma, iki Boole fonksiyonu ile mantıksal olarak ifade edilebilir.

$$(A > B) = A_3 B_3' + x_3 A_2 B_2' + x_3 x_2 A_1 B_1' + x_3 x_2 x_1 A_0 B_0'$$

$$(A < B) = A_3' B_3 + x_3 A_2' B_2 + x_3 x_2 A_1' B_1 + x_3 x_2 x_1 A_0' B_0$$

Semboller $1A7B2$ ve $1A6B2$ ikili çıkış değişkenleridir ve 1'e eşittir $A7B$ ve $A6B$, sırasıyla.

Üç çıkış değişkeninin kapı uygulaması görüldüğünden daha basittir çünkü belirli bir tekrar içerir. Eşit olmayan çıkışlar, eşit çıkışı üretmek için gereken aynı kapıları kullanabilir. Dört bitlik büyüklük karşılaştırmacısının mantık diyagramı Şekil 4.17'de gösterilmiştir. Dört x çıkışı üretilir



ŞEKİL 4.17

Dört bitlik büyüklük karşılaştırıcı

özel-NOR devreleri ile üretilir ve çıkış ikili

değişkeni $1A = B2$. Diğer iki çıkış, daha önce listelenen Boole

fonksiyonlarını üretmek için x değişkenlerini kullanır. Bu çok seviyeli bir uygulamadır ve düzenli bir desene sahiptir.

İkili sayılar için büyüklük karşılaştırıcı devreleri elde etme prosedürü bu örnekte n açıktır.

4.9 DECODERS

Dijital sistemlerde ayrıık bilgi miktarları ikili kodlarla temsil edilir.

Bir n bitlik ikili kod, kodlanmış

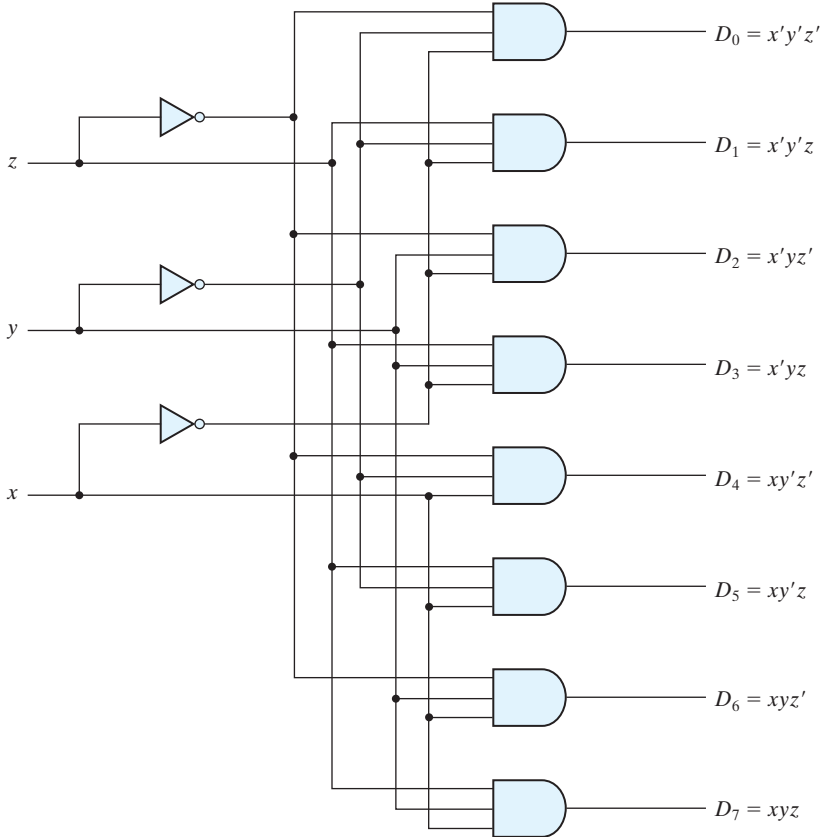
bilgilerin en fazla 2^n farklı öğesini temsil etme kapasitesine sahiptir. Bir *decoder* ikili bilgiyi

n giriş hatlarını en fazla 2^n benzersiz çıkış hattına dönüştüren bir kombinyonel devredir. Eğer n bitlik kodlanmış bilgi kullanılmayan kombinasyonlara sahipse, decoder'ın çıkış sayısı 2^n 'den az olabilir.

Burada sunulan decodera n -to- m -hat decodera denir, burada $m \leq 2^n$. Amaçları, n giriş değişkenlerinin 2^n (veya daha az) mintermlerini üretmektir. Girişlerin her kombinyonu benzersiz bir çıkışı belirleyecektir. *decoder* adı ayrıca BCD-yedi-segment decoder gibi diğer kod dönüştürücülerle birlikte de kullanılır.

Örnek olarak, Şekil 4.18'deki üçe sekiz hat çözümleyici devresini ele alalım. Üç giriş, üç giriş değişkeninin her birinin mintermlerini temsil eden sekiz çıkışa kodlanır. Üç inverter, girişlerin tamamlayıcısını sağlar ve sekiz AND kapısından her biri bir minterm üretir. Bu çözümleyicinin belirli bir uygulaması, ikili-sekizli dönüşümdür. Giriş değişkenleri bir ikili sayıyı temsil eder ve çıkışlar, sekizli sayı sistemindeki bir sayının sekiz basamağını temsil eder.

Ancak, üç ila sekiz satırlık bir kod çözücü, herhangi bir üç bitlik kodu çözmek için kullanılabilir ve her bir kod elemanı için bir tane olmak üzere sekiz çıkış sağlar.



ŞEKİL 4.18

Üçten sekize satıra kadar kodlayıcı

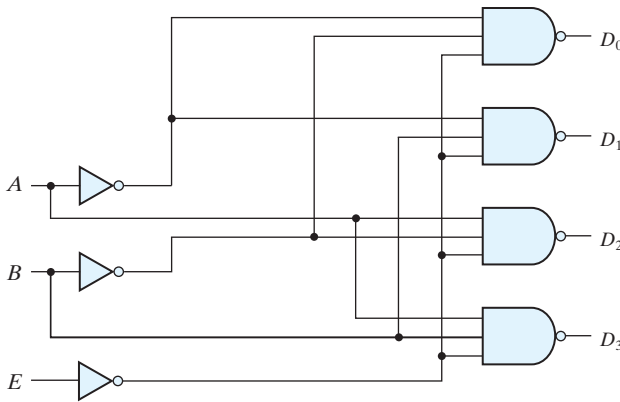
Tablo 4.6

Üçten Sekize Hattı Kodlayıcı Gerçeklik Tablosu

Girişler			Çıkışlar							
X			D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Kodlayıcının çalışması, Tablo 4.6'da listelenen doğruluk tablosu ile netleştirilebilir. Her olası giriş kombinasyonu için, 0'a eşit yedi çıkış ve yalnızca 1'e eşit bir çıkış vardır. Değeri 1'e eşit olan çıkış, giriş hatlarında mevcut olan ikili sayının minter mi eşdeğerini temsil eder.

Bazı kodlayıcılar NAND kapıları ile inşa edilir. Bir NAND kapısı, ters bir çıkışla AND işlemini ürettiğinden, kodlayıcı mintermlerini tamamlanmış formda üretmek daha ekonomik hale gelir. Ayrıca, kodlayıcılar devre çalışmasını kontrol etmek için bir veya daha fazla etkinleştirme girişi içerir. NAND kapıları ile inşa edilmiş bir etkinleştirme girişi olan iki ila dört satırlık bir kodlayıcı Şekil 4.19'da gösterilmiştir. Devre, tamamlanmış çıkışlar ve bir tamamlayıcı etkinleştirme girişi ile çalışır. Kodlayıcı, E 0'a eşit olduğunda etkinleştirilir (yani, aktif düşük etkinleştirme). Doğruluk tablosunda belirtildiği gibi, yalnızca bir



(a) Mantık diyagramı

E	A	B	D			
			${}_0D_1D_2D_3$			
1	x	x	1	1	1	1
0	0	0	0	1	1	1
		1	1	0	1	1
	1	0	1	1	0	1
0	1	1	1	1	1	1

(b) Doğruluk tablosu

ŞEKİL 4.19

İki-dört hat kodlayıcı, etkinleştirme girişi ile

ş herhangi bir anda 0'a eşit olabilir; diğer tüm çıkışlar 1'e eşittir. 0'a eşit olan çıkış, girişler A ve B tarafından seçilen mintermi temsil eder. Devre, diğer iki girişin değerlerinden bağımsız olarak $E = 1$ 'e eşit olduğunda devre dışı kalır. Devre devre dışı olduğunda, çıkışların hiçbiri 0'a eşit değildir ve hiçbiri minterm seçilmez. Genel olarak, bir kodlayıcı, tamamlanmış veya tamamlanmamış çıkışlarla çalışabilir. Etkinleştirme girişi, 0 veya 1 sinyali ile etkinleştirilebilir. Bazı kodlayıcılar, devreyi etkinleştirmek için belirli bir mantık koşulunu sağlaması gereken iki veya daha fazla etkinleştirme girişine sahiptir.

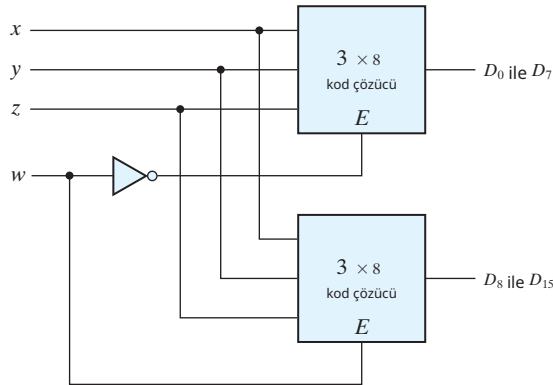
Etkinleştirme girişi olan bir kodlayıcı, *demultiplexer*— tek bir hattın bilgilerini alıp 2 olası çıkış hattından birine yönlendiren bir devre olarak işlev görebilir. Belirli bir çıkışın seçimi, n seçim hatlarının bit kombinasyonu ile kontrol edilir. Şekil 4.19'daki kodlayıcı, E veri girişi hattı olarak alındığında ve A ile B seçim girişi olarak alındığında birden dörde hat demultiplexer olarak işlev görebilir. Tek giriş değişkeni E dört çıkışın hepsine bir yol sahiptir, ancak giriş bilgisi yalnızca iki seçim hattı A ve B tarafından belirtilen ikili kombinasyona göre yalnızca bir çıkış hattına yönlendirilir. Bu özellik, devrenin doğruluk tablosundan doğrulanabilir. Örneğin, seçim hatları $AB = 10$ olduğunda, çıkış D_2 giriş değerine eşit olacaktır. E , diğer tüm çıktılar 1'de tutulurken. Çünkü kod çözücü ve demultiplexer işlemleri aynı devreden elde edilir, enable girişi olan bir kod çözücü

kod çözücü -demultiplexer olarak adlandırılır.

Enable girişlerine sahip kod çözümler bir araya getirilerek daha büyük bir kod çözümü devresi oluşturulabilir. Şekil 4.20, enable girişlerine sahip iki 3×8 hat kod çözümlerinin bir araya getirilerek 4×16 hat kod çözümlerini oluşturduğunu göstermektedir. $w = 0$ olduğunda, üst kod çözümü etkinleştirilir ve diğeri devre dışı bırakılır.

Alt kod çözümlerin tüm çıktıları 0'dır ve üstteki sekiz çıkış minterm

0000 ile 0111 arasında değerler üretir. $w = 1$ olduğunda, enable koşulları tersine döner: Alt kod çözümü çıktıları 1000 ile 1111 arasında minterm değerleri üretirken, üst kod çözümlerin çıktıları tamamen 0'dır. Bu örnek, kod çözümlerde ve diğlerinde enable girişlerinin faydasını göstermektedir.



ŞEKİL 4.20

4×16 kod çözümü, iki 3×8 kod çözümü ile inşa edilmiştir

kombinasyonel mantık bileşenleri. Genel olarak, etkinleştirme girişleri, iki veya daha fazla standart bileşeni birleştirerek daha fazla giriş ve çıkışa sahip benzer bir işlev oluşturmak amacıyla birbirine bağlamak için kullanışlı bir özelliktir.

Birleşik Mantık Uygulaması

Bir dekoder, n giriş değişkeninin 2^n mintermi sağlar. Dekoderin her bir etkin çıkışı, benzersiz bir giriş bitleri desenine ilişkilendirilmiştir. Herhangi bir Boole fonksiyonu, minterm toplamı biçiminde ifade edilebileceğinden, fonksiyonun mintermlerini üreten bir dekoder ile bunların mantıksal toplamını oluşturan harici bir VEYA kapısı, fonksiyonun donanım uygulamasını sağlar. Bu şekilde, herhangi bir kombinasyonel devre ile n girişler ve m çıkışlar n -dan- 2^n -hat çözücü ile ve m VE kapılar.

Bir kombinasyon devresinin bir dekoder ve VE kapıları aracılığıyla uygulanması prosedürü, devrenin Boolean fonksiyonunun mintermlerin toplamı olarak ifade edilmesini gerektirir. Daha sonra, giriş değişkenlerinin tüm mintermlerini üreten bir dekoder seçilir. Her VE kapısının girişleri, her fonksiyonun minterm listesindeki dekoder çıkışlarından seçilir. Bu prosedür, bir tam toplayıcı devresini uygulayan bir örnek gösterilecektir.

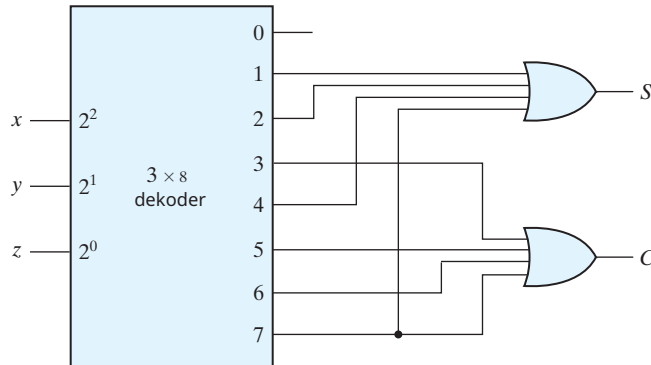
Tam toplayıcının doğruluk tablosundan (bkz. Tablo 4.4),

toplam-mintermlerin formunda kombinasyonel devre için fonksiyonları elde ederiz:

$$S(x, y, z) = \Sigma(1, 2, 4, 7)$$

$$C(x, y, z) = \Sigma(3, 5, 6, 7)$$

Üç giriş ve toplamda sekiz mintermi olduğu için, üçe sekiz hatlı bir dekoder gerekmektedir. Uygulama Şekil 4.21'de gösterilmiştir. Dekoder, x, y ve z için sekiz mintermi üretir. Çıkış S için OR kapısı, 1, 2, 4 ve 7 numaralı mintermlerin mantıksal toplamını oluşturur. Çıkış C için OR kapısı, 3, 5, 6 ve 7 numaralı mintermlerin mantıksal toplamını oluşturur.



ŞEKİL 4.21

Dekoder ile tam toplayıcının uygulanması

Uzun bir minterm listesine sahip bir fonksiyon, çok sayıda girişi olan bir OR kapısı gerektirir. k minterm listesine sahip bir fonksiyon, tamamlanmış formunda ifade edilebilir F ile $2^n - k$ minterm. Fonksiyondaki minterm sayısı $2^n/2$ 'den büyükse, o zaman F daha az minterm ile ifade edilebilir. Böyle bir durumda, F mintermlerini toplamak için bir NOR kapısı kullanmak avantajlıdır. NOR kapısının çıkışı bu toplamı tamamlar ve normal çıkış F üretir. Dekoder için NAND kapıları kullanılıyorsa, Şekil 4.19'da olduğu gibi, dış kapılar OR kapıları yerine NAND kapıları olmalıdır. Bunun nedeni, iki seviyeli bir AND kapı devresinin bir minterm toplamı fonksiyonunu uygulaması ve iki seviyeli bir AND-OR devresine eşdeğer olmasıdır.

4.10 KODLAYICILAR

Kodlayıcı, bir çözücünün ters işlemini gerçekleştiren dijital bir devredir. Bir kodlayıcının 2 (veya daha az) giriş hattı ve n çıkış hattı vardır. Çıkış hatları, bir bütün olarak, giriş değerine karşılık gelen ikili kodu üretir. Bir kodlayıcı örneği, doğruluk tablosu Tablo 4.7'de verilen sekizli-ikili kodlayıcıdır. Sekiz girişi vardır (her biri sekizli rakam için bir tane) ve karşılık gelen ikili sayıyı üreten üç çıkışı vardır. Herhangi bir anda yalnızca bir girişin 1 değeri alması varsayılmaktadır.

Kodlayıcı, girişleri doğrudan doğruluk tablosundan belirlenen VE kapıları ile uygulanabilir. Çıkış z , giriş sekizli rakamı 1, 3, 5 veya 7 olduğunda 1'e eşittir. Çıkış y , sekizli rakamlar 2, 3, 6 veya 7 için 1'dir ve çıkış x , 4, 5, 6 veya 7 rakamları için 1'dir. Bu koşullar aşağıdaki Boolean çıkış fonksiyonları ile ifade edilebilir:

$$\begin{aligned} z &= D_1 + D_3 + D_5 + D_7 \\ y &= D_2 + D_3 + D_6 + D_7 \\ x &= D_4 + D_5 + D_6 + D_7 \end{aligned}$$

Kodlayıcı, üç VE kapısı ile uygulanabilir.

Tablo 4.7
Bir Sekizli-İkili Kodlayıcının Doğruluk Tablosu

Girişler								Çıkışlar		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Tablo 4.7'de tanımlanan kodlayıcının, belirli bir anda yalnızca bir girişin aktif olabileceği sınırlaması vardır. Eğer iki giriş aynı anda aktif olursa, çıkış tanımsız bir kombinasyon üretir. Örneğin, eğer D_3 ve D_6 aynı anda 1 ise, kodlayıcının çıkışı 111 olacaktır çünkü üç çıkış da 1'e eşittir. Çıkış 111, ne ikili 3'ü ne de ikili 6'yı temsil eder. Bu belirsizliği çözmek için, kodlayıcı devreleri yalnızca bir girişin kodlandığından emin olmak için bir giriş önceliği belirlemelidir. Eğer daha yüksek alt simge numaralarına sahip girişler için daha yüksek bir öncelik belirlersek ve eğer D_3 ve D_6 aynı anda 1 ise, çıkış 110 olacaktır çünkü D_6 , D_3 'ten daha yüksek bir önceliğe sahiptir.

Sekizli-ikili kodlayıcısındaki bir diğer belirsizlik, tüm girişler 0 olduğunda tüm 0'ların ürettiği bir çıkışın oluşmasıdır; ancak bu çıkış, D_0 1'e eşit olduğunda üretilen çıkışla aynıdır. Bu tutarsızlık, en az bir girişin 1'e eşit olup olmadığını belirtmek için bir çıkış da ha sağlayarak çözülebilir.

Öncelik Kodlayıcı

Öncelik kodlayıcı, öncelik işlevini içeren bir kodlayıcı devresidir. Öncelik kodlayıcının işleyişi, iki veya daha fazla giriş aynı anda 1 olduğunda, en yüksek önceliğe sahip girişin öncelik alması şeklindedir. Dört girişli bir öncelik kodlayıcının doğru tablosu Tablo 4.8'de verilmiştir. İki çıkışın yanı sıra x ve y , devrede V ile belirtilen üçüncü bir çıkış bulunmaktadır; bu, bir veya daha fazla giriş 1 olduğunda 1'e ayarlanan bir *geçerli* bit görevi görür. Tüm girişler 0 olduğunda, geçerli bir giriş yoktur ve V 0'a eşittir.

Diğer iki çıkış, V 0'a eşit olduğunda incelenmez ve dikkate alınmayan koşullar olarak belirtilir. Çıkış sütunlarındaki X'lerin dikkate alınmayan koşulları temsil ettiğini, giriş sütunlarındaki X'lerin ise yoğunlaştırılmış bir biçimde doğru tablosunu temsil etmek için yararlı olduğunu unutmayın.

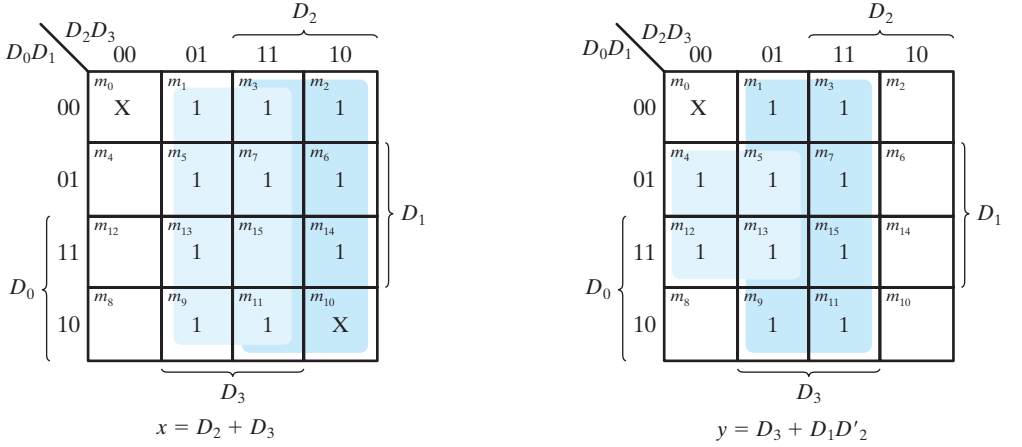
Dört değişkenin tüm 16 mintermi listelemek yerine, doğru tablosu 1 veya 0'ı temsil etmek için bir X kullanır. Örneğin, X 100, iki mintermi 0100 ve 1100'ü temsil eder.

Tablo 4.8'e göre, alt simge numarası ne kadar yüksekse, girişin önceliği o kadar yüksektir. Giriş D_3 en yüksek önceliğe sahiptir, bu nedenle diğer girişlerin değerlerine bakılmaksızın, bu giriş 1 olduğunda xy çıkışı 11 (ikilik 3) olur. D_2 bir sonraki öncelik seviyesine sahiptir. Çıkış, $D_2=1$ olduğunda 10'dur, $D_3=0$ olduğu sürece, diğer iki daha düşük öncelikli girişin değerlerine bakılmaksızın. D_1 için çıkış, yalnızca daha yüksek öncelikli girişler 0 olduğunda üretilir ve öncelik seviyeleri aşağı doğru devam eder.

Tablo 4.8

Bir Öncelik Kodlayıcısının Gerçeklik Tablosu

Girişler				Çıkışlar		V
D_0	D_1	D_2	D_3	x	y	
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

**FIGURE 4.22**

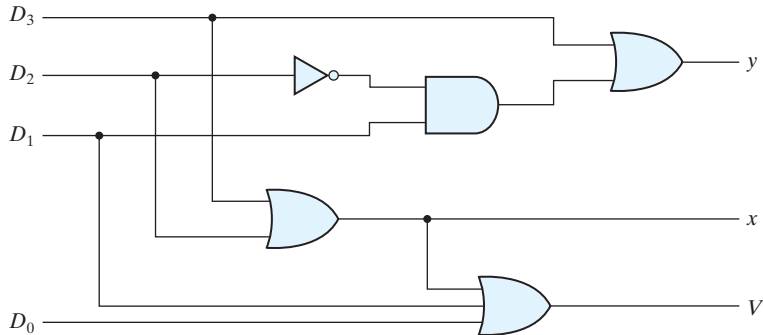
Öncelik kodlayıcı için haritalar

Çıktıları basitleştirmek için haritalar x ve y Şekil 4.22'de gösterilmiştir. İki fonksiyonun mintermleri Tablo 4.8'den türetilmiştir. Tablo yalnızca beş satır içeriyor olsa da, her satırdaki X'ler önce 0 ile, sonra 1 ile değiştirildiğinde, 16 olası giriş kombinasyonunu elde ederiz. Örneğin, tablodaki dördüncü satır, XX10 girişleri ile, dört minterm olan 0010, 0110, 1010 ve 1110'u temsil eder. Öncelik kodlayıcısının basitleştirilmiş Boole ifadeleri haritalardan elde edilir. Çıktı V için koşul, tüm giriş değişkenlerinin bir VEYA (OR) fonksiyonudur. Öncelik kodlayıcısı, aşağıdaki Boole fonksiyonlarına göre Şekil 4.23'te uygulanmıştır:

$$x = D_2 + D_3$$

$$y = D_3 + D_1 D'_2$$

$$V = D_0 + D_1 + D_2 + D_3$$

**ŞEKİL 4.23**

Dört girişli öncelik kodlayıcı

4.11 ÇOĞULAYICILAR

Çoğulayıcı, birçok giriş hattından ikili bilgiyi seçen ve bunu tek bir çıkış hattına yönlendiren bir kombinasyonel devredir. Belirli bir giriş hattının seçimi, bir dizi seçim hattı tarafından kontrol edilir. Normalde, 2 giriş hattı ve n seçim hattı vardır; bu seçim hatlarının bit kombinasyonları hangi girişin seçileceğini belirler.

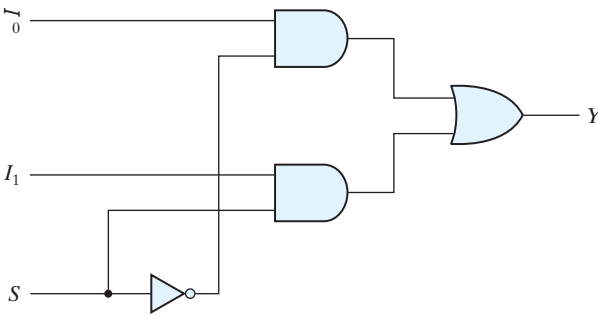
İki girişli bir çoğulayıcı, iki 1-bit kaynağından birini ortak bir hedefe bağlar; bu, Şekil 4.24'te gösterilmiştir. Devre, iki veri giriş hattı, bir çıkış hattı ve bir seçim hattı S içerir. $S=0$ olduğunda, üst AND kapısı etkinleştirilir ve I_0 çıkışa bir yol bulur. $S=1$ olduğunda, alt AND kapısı etkinleştirilir ve I_1 çıkışa bir yol bulur.

Çoğulayıcı, iki kaynaktan birini seçen bir elektronik anahtar gibi davranır. Bir çoğulayıcının blok diyagramı bazen, Şekil 4.24(b)'te gösterildiği gibi, bir kama şeklindeki sembol ile tasvir edilir.

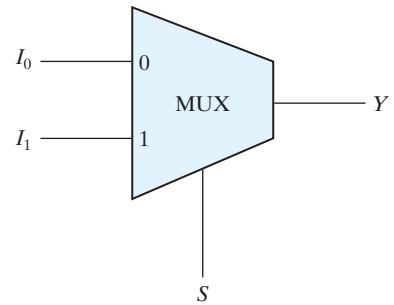
Bu, birden fazla veri kaynağından seçilen birinin tek bir hedefe nasıl yönlendirildiğini görsel olarak önerir. Çoğulayıcı, blok diyagramlarında genellikle "MUX" olarak etiketlenir.

Şekil 4.25'te gösterilen dört'e bir hat çoklayıcısıdır. Dört girişten her biri, I_0 ile I_3 arasında, bir AND kapısının bir girişine uygulanır. Seçim hatları S_1 ve S_0 , belirli bir AND kapısını seçmek için kodlanır. AND kapılarının çıkışları, tek bir OR kapısına uygulanır ve bu kapı bir hat çıkışı sağlar. Fonksiyon tablosu, ikili seçim değerlerinin her kombinasyonu için çıkışa geçirilen girişi listeler. Devrenin çalışmasını göstermek için, $S_1S_0=10$ durumunu düşünelim. Giriş I_2 ile ilişkili AND kapısının iki girişi 1'e eşittir ve üçüncü girişi I_2 'ye bağlıdır. Diğer üç AND kapısının en az bir girişi 0'a eşittir, bu da çıkışlarının 0'a eşit olmasına neden olur. OR kapısının çıkışı artık I_2 'nin değerine eşittir ve seçilen girişten çıkışa bir yol sağlar. Çoklayıcıya ayrıca *veri seçici* denir, çünkü birçok girişten birini seçer ve ikili bilgiyi çıkış hattına yönlendirir.

Çoklayıcıdaki AND kapıları ve inverterler, bir kod çözücü devresine benzer ve gerçekten de seçim giriş hatlarını kod çözerler. Genel olarak, bir $2^n - 1$ hat çoklayıcısı, bir $n - 2^n$ kod çözücüsünden, her AND kapısına bir tane olmak üzere 2 giriş hattı eklenerek inşa edilir. AND kapılarının çıkışları, tek bir OR kapısına uygulanır. Bir çoklayıcının boyutu,



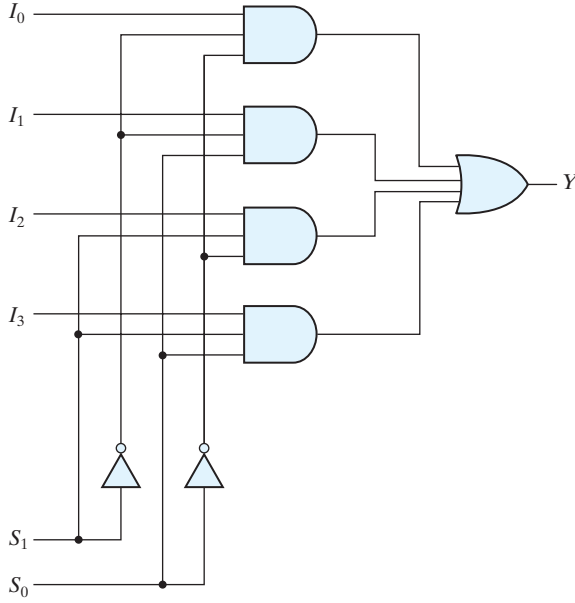
(a) Mantık diyagramı



(b) Blok diyagramı

ŞEKİL 4.24

İkiye bir hat çoklayıcı



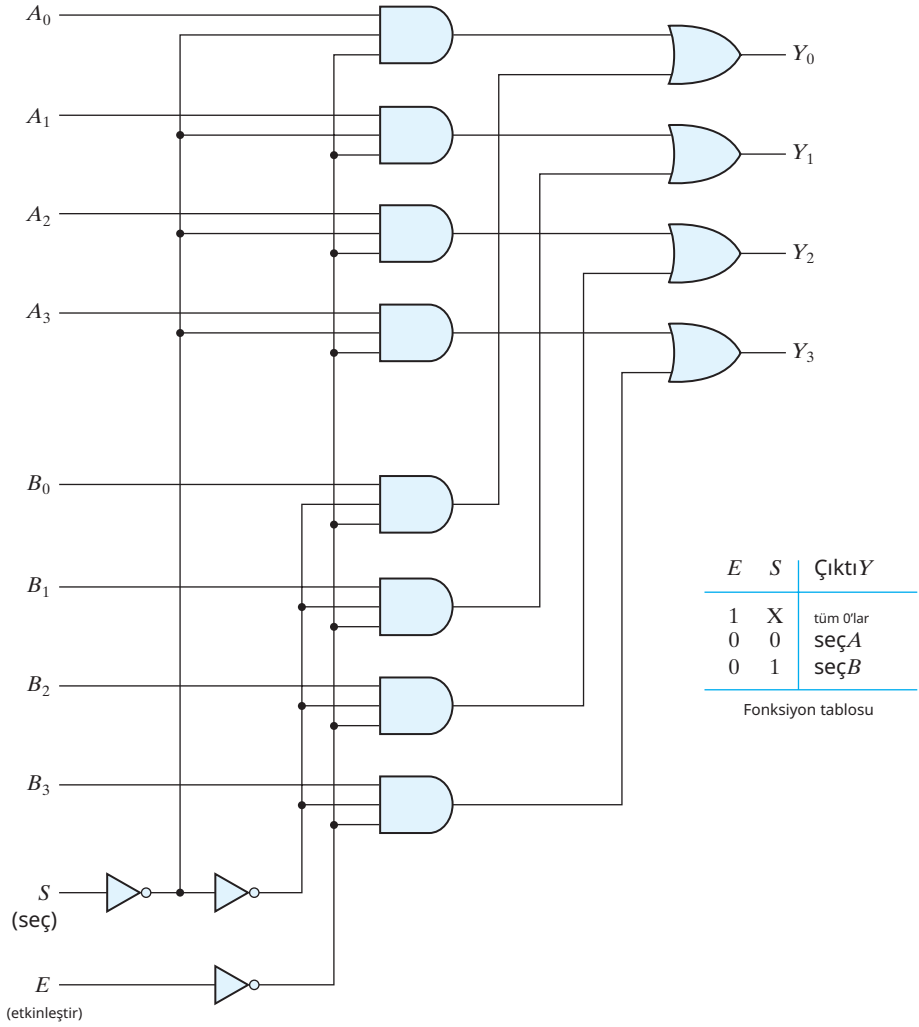
ŞEKİL 4.25
Dört'e bir hat çoklayıcı

veri giriş hatlarının sayısı 2 ve tek çıkış hattı. Seçim hatları n 2 veri hattından türetilmiştir. Kodlayıcılarda olduğu gibi, çoklayıcıların bir etkinleştirme girişi olabilir biri mi kontrol etmek için. Etkinleştirme girişi pasif durumda olduğunda, çıkışlar disab le olur ve aktif durumda olduğunda, devre normal bir çoklayıcı olarak çalışır.

Çoklayıcı devreleri, çoklu bit seçim mantığı sağlamak için ortak seçim girişleri ile birleştirilebilir. Örnek olarak, dört katlı 2'ye 1 hat çoklayıcısı Şekil 4.26'da gösterilmiştir. Devre, her biri iki giriş hattından birini seçme yeteneğine sahip dört çoklayıcı içerir. Çıkış Y_0 , ya giriş A_0 ya da giriş B_0 'dan gelmek üzere seçilebilir. Benzer şekilde, çıkış Y_1, A_1 veya B_1 değerine sahip olabilir ve devam eder. Giriş seçim hattı S , dört çoklayıcının her birinde bir hattı seçer. Etkinleştirme girişi E , normal çalışma için aktif olmalıdır (yani, belirtilmelidir). Devre dört 2'ye 1 hat çoklayıcı içeriyor olsa da, onu iki 4 bitlik veri hattı setinden birini seçen bir devre olarak görmek daha olasıdır. Fonksiyon tablosunda gösterildiği gibi, birim $E=0$ olduğunda etkinleştirilir. O zaman, eğer $S=0$ ise, dört A girişi dört çıkışa bir yol sağlar. Aksine, eğer $S=1$ ise, dört B girişi çıkışlara uygulanır. Çıkışlar, $E=1$ olduğunda, S değerine bakılmaksızın tüm 0'lara sahiptir.

Boolean Fonksiyonu Uygulaması

Bölüm 4.9'da, bir kod çözücünün dış OR kapıları kullanarak Boole fonksiyonlarını uygulamak için kullanılabileceği gösterilmiştir. Bir çoklayıcının mantık diyagramının incelenmesi, onun aslında birim içinde OR kapısını içeren bir kod çözücü olduğunu ortaya koymaktadır.



ŞEKİL 4.26
Dört giriшли ikiye bir hat çoklayıcı

Bir fonksiyonun mintermleri, seçim girişleriyle ilişkili devre tarafından bir çoklayıcıda üretilir. Bireysel mintermler, veri girişleri tarafından seçilebilir, böylece n değişkenli bir Boole fonksiyonunu, her minterm için bir tane olmak üzere n seçim girişı ve 2 ve ri girişı olan bir çoklayıcı ile uygulamanın bir yöntemi sağlanır.

Şimdi bir Boolean fonksiyonunu uygulamak için daha verimli bir yöntemi göstereceğiz. n çoklayıcı ile birlikte $n-1$ seçim girişı olan değişkenler. İlk $n-1$ değişkenler fonksiyonun seçim girişlerine bağlanır. Fonksiyonun kalan bir değişkeni veri girişleri için kullanılır. Eğer bu bir değişken belirtilirse

n z , çoklayıcının her veri girişi z , z , 1 veya 0 olacaktır. Bu işlemi göstermek için, Boolean fonksiyonunu dikkate alalım

$$F(x, y, z) = \Sigma(1, 2, 6, 7)$$

Bu üç değişkenli fonksiyon, Şekil 4.27'de gösterildiği gibi dört girişli bir çoklayıcı ile uygulanabilir. İki değişken x ve y seçim hatlarına bu sırayla uygulanır; x S_1 girişine, y ise S_0 girişine bağlanır. Veri girişi hatları için değerler, fonksiyonun doğruluk tablosundan belirlenir. $xy=00$ olduğunda, çıkış F z ile eşittir çünkü $F=0$ olduğunda $z=0$ ve $F=1$ olduğunda $z=1$. Bu, değişken z veri girişi 0'a uygulanması gerektiği anlamına gelir. Çoklayıcının işleyişi, $xy=00$ olduğunda, veri girişi 0'ın çıkışa bir yolu olduğu ve bu durumun F değerini z ile eşit kıldığı anlamına gelir. Benzer şekilde, F değerine göre veri hatları 1, 2 ve 3 için gerekli girişi belirleyebiliriz; $xy=01, 10$ ve 11 olduğunda sırasıyla. Bu özel örnek, veri girişleri için elde edilebilecek dört olasılığı göstermektedir.

Herhangi bir Boolean fonksiyonunun n değişkenleri ile bir multiplexer ile $n-1$ seçim girişi ve 2^{n-1} veri girişi ile uygulanması için genel prosedür, önceki örnekten gelmektedir. Öncelikle, Boolean fonksiyonu bir doğruluk tablosunda listelenir. Ardından, tablodaki ilk $n-1$ değişkenler multiplexer'ın seçim girişlerine uygulanır. Seçim değişkenlerinin her kombinasyonu için, çıktığı son değişkenin bir fonksiyonu olarak değerlendiririz. Bu fonksiyon 0, 1, değişken veya değişkenin tamamlayıcı olabilir. Bu değerler daha sonra uygun sırayla veri girişlerine uygulanır.

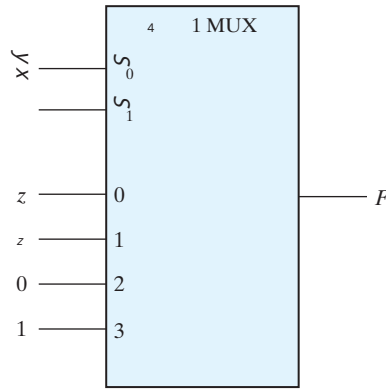
İkinci bir örnek olarak, Boolean fonksiyonunun uygulanmasını düşünelim

$$F(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$$

Bu fonksiyon, Şekil 4.28'de gösterildiği gibi üç seçim girişi olan bir multiplexer ile uygulanmaktadır. İlk değişkenin A seçim girişi S_2 ile bağlanması gerektiğini unutmayın, böylece A, B ve C sırasıyla seçim girişlerine S_2, S_1 ve S_0 karşılık gelir. Değerler için

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	1	0	0
1	1	1	1

(a) Doğruluk tablosu

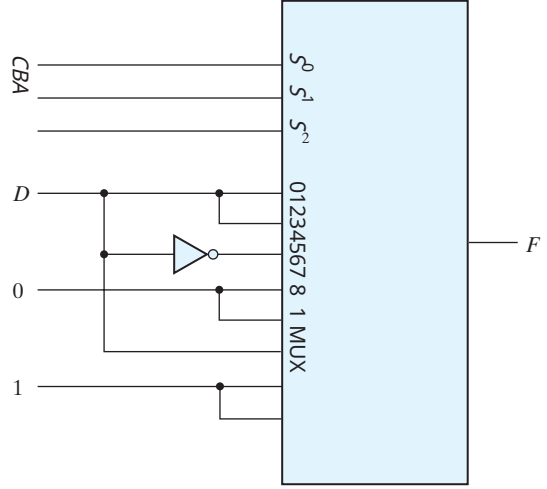


(b) Multiplexer uygulaması

ŞEKİL 4.27

Bir çoklayıcı ile bir Boolean fonksiyonu uygulamak

A	B	C	D	F
00	00	01	01	$F=D$
00	00	11	01	$F=D$
00	11	00	01	$F=D$
00	11	11	01	$F=0$
11	00	00	01	$F=0$
11	00	11	01	$F=D$
11	11	00	01	$F=1$
11	11	11	01	$F=1$

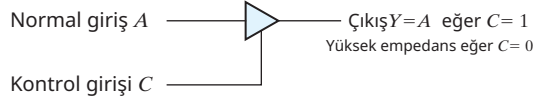
**ŞEKİL 4.28****Bir çoklayıcı ile dört girişli bir fonksiyon uygulamak**

Veri girişleri, şekil içinde listelenen doğruluk tablosundan belirlenir. İlgili data hat numarası, ABC 'nin ikili kombinasyonundan belirlenir. Örneğin, tablo, $ABC=101$ olduğunda, $F=D$ olduğunu gösterir, bu nedenle giriş değişkeni D veri girişi 5'e uygulanır. İkili sabitler 0 ve 1, iki sabit sinyal değerine karşılık gelir. Entegre devreler kullanıldığında, mantık 0 sinyal toprağına, mantık 1 ise güç sinyaline karşılık gelir, bu teknolojiye bağlıdır (örneğin, 3 V).

Üç Durumlu Kapılar

Bir çoklayıcı, üç durumlu kapılarla inşa edilebilir—üç durum sergileyen dijital devreler. İki durum, geleneksel bir kapıda olduğu gibi mantık 1 ve mantık 0'a eşdeğer sinyallerdir. Üçüncü durum, (1) mantığın açık devre gibi davrandığı, yani çıkışın bağlantısının kesilmiş gibi görüldüğü, (2) devrenin mantıksal bir önemi olmadığı ve (3) üç durumlu kapının çıkışına bağlı devrenin kapıya gelen girişlerden etkilenmediği yüksek empedans durumudur. Üç durumlu kapılar, AND veya NAND gibi herhangi bir geleneksel mantığı gerçekleştirebilir. Ancak, en yaygın olarak kullanılanı tampon kapıdır.

Üç durumlu tampon kapısının grafik sembolü Şekil 4.29'da gösterilmiştir. Normal bir tampon ile, sembolün altına giren bir giriş kontrol hattı ile ayırt edilir. Tamponun normal bir girişi, bir çıkışı ve çıkışın durumunu belirleyen bir kontrol girişi vardır. Kontrol girişi 1'e eşit olduğunda, çıkış etkinleştirilir ve kapı, çıkışın normal girişi eşit olacak şekilde geleneksel bir tampon gibi davranır. Kontrol



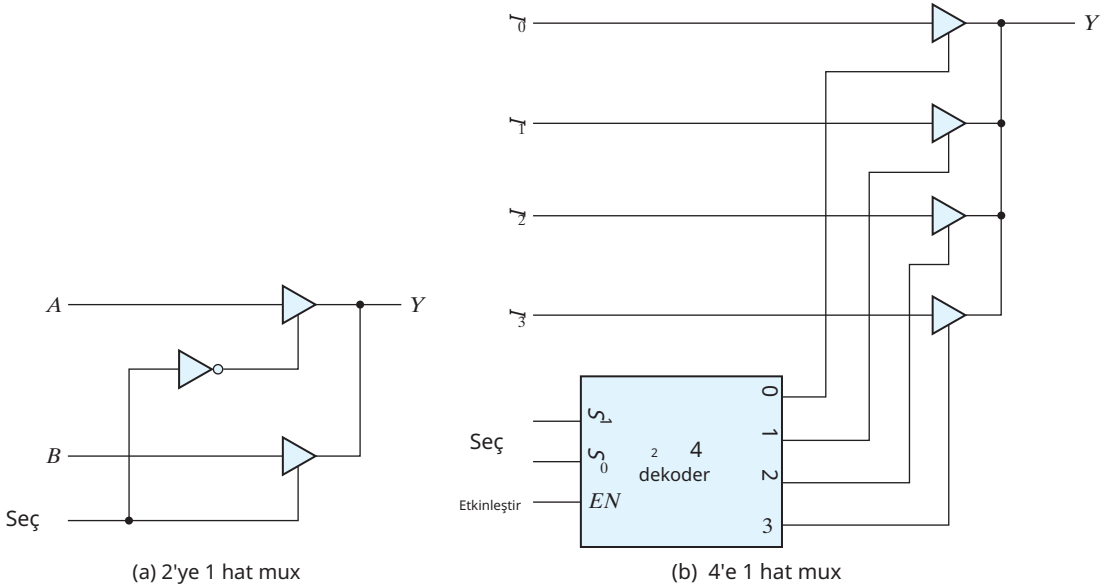
ŞEKİL 4.29

Üç durumlu tampon için grafik sembolü

giriş 0 ise, çıkış devre dışı bırakılır ve kapı yüksek empedans durumuna geçer, normal girişteki değerden bağımsız olarak. Üç durumlu bir kapının yüksek empedans durumu, diğer kapılarda bulunmayan özel bir özellik sağlar. Bu özellik sayesinde, çok sayıda üç durumlu kapı çıkışı, yükleme etkilerini tehlikeye atmadan ortak bir hat oluşturmak için tellerle bağlanabilir.

Üç durumlu tamponlarla çoklayıcıların yapımı Şekil 4.30'da gösterilmektedir. Şekil 4.30(a), 2 üç durumlu tampon ve bir inverter ile ikiye bir hat çoklayıcısının yapısını göstermektedir. İki çıkış, tek bir çıkış hattı oluşturmak için birleştirilmiştir. (Bu tür bir bağlantının, üç durumlu çıkışları olmayan kapılarla yapılamayacağını unutmayın.) Seçim girişi 0 olduğunda, üst tampon kontrol girişi tarafından etkinleştirilir ve alt tampon devre dışı bırakılır. Çıkış Y , o zaman giriş A 'ya eşittir. Seçim girişi 1 olduğunda, alt tampon etkinleştirilir ve Y , B 'ye eşit olur.

Bir dört'e bir hat çoklayıcısının yapısı Şekil 4.30(b)'de gösterilmiştir. 4 adet üç durumlu tamponun çıkışları bir araya getirilerek tek bir çıkış hattı oluşturulmuştur. Tamponlara giden kontrol girişleri, dört normal girişten hangisinin seçileceğini belirler. I_0 üzerinden



ŞEKİL 4.30

Üç durumlu kapılarla çoklayıcılar

$\frac{1}{3}$ çıkış hattına bağlanacaktır. Herhangi bir anda aktif durumda en fazla bir tampon olabilir. Bağlı tamponlar, yalnızca 1 üç durumlu tamponun çıkışa erişimini sağlamak için kontrol edilmelidir, diğer tüm tamponlar yüksek empedans durumunda tutulmalıdır. Herhangi bir anda birden fazla kontrol girişinin aktif olmadığından emin olmanın bir yolu, diyagramda gösterildiği gibi bir dekoder kullanmaktır. Dekoderin etkinleştirme girişi 0 olduğunda, dört çıkışının tamamı 0'dır ve veri yolu yüksek empedans durumundadır çünkü dört tampon da devre dışıdır. Etkinleştirme girişi aktif olduğunda, dekoderin seçim girişlerindeki ikili değere bağlı olarak üç durumlu tamponlardan biri aktif olacaktır. Dikkatli bir inceleme, bu devrenin dört girişli bir çoklayıcı inşa etmenin başka bir yolu olduğunu ortaya koymaktadır.

4.12 KOMBİNASYONEL

DEVRELERİN HDL MODELLERİ

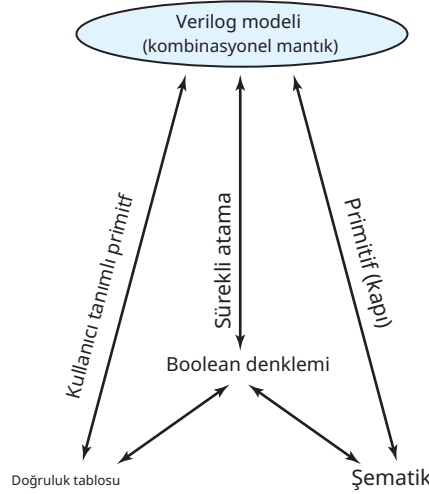
Verilog HDL, Bölüm 3.10'da tanıtılmıştır. Mevcut bölümde, Verilog'un ek özelliklerini tanıtıyor, daha ayrıntılı örnekler sunuyor ve Verilog'daki kombinasyonel devrelerin alternatif tanımlarını karşılaştırıyoruz. Ardışık devreler Bölüm 5'te sunulmaktadır. Daha önce belirtildiği gibi, modül, Verilog HDL ile donanım modellemenin temel yapı taşıdır. Bir modülün mantığı, aşağıdaki modelleme stillerinden herhangi birinde (veya bir kombinasyonunda) tanımlanabilir:

- Önceden tanımlanmış ve kullanıcı tanımlı ilkel kapıların örneklerini kullanarak kapı düzeyinde modelleme.
- Sürekli atama ifadeleri kullanarak veri akışı modelleme anahtar kelimesi ile **atama**.
- Davranışsal modelleme, anahtar kelime ile prosedürel atama ifadeleri kullanarak **her zaman**.

Kapı düzeyi (yapısal) modelleme, bir devreyi kapılarını belirterek ve bunların birbirleriyle nasıl bağlantılı olduğunu tanımlayarak açıklar. Veri akışı modelleme, genellikle kombinasyonel mantığın Boolean denklemlerini tanımlamak için kullanılır. Ayrıca burada, kombinasyonel ve sıralı devreleri daha yüksek bir soyutlama düzeyinde tanımlamak için kullanılan davranışsal modellemeyi de ele alacağız. Kombinasyonel mantık, doğru tablolar, Boolean denklemleri ve şemalar ile tasarlanabilir; Verilog, bu "klasik" tasarım yaklaşımlarının her birine karşılık gelen bir yapı sunar: kullanıcı tanımlı ilkelere, sürekli atamalara ve ilkelere, Şekil 4.31'de gösterildiği gibi. Bir diğer modelleme tarzı ise anahtar düzeyi modellemedir. Bu, bazen MOS transistör devre modellerinin simülasyonunda kullanılır, ancak mantık sentezinde kullanılmaz. Anahtar düzeyi modellemeyi dikkate almayacağız.

Kapı Düzeyinde Modelleme

Kapı düzeyinde modelleme, Bölüm 3.10'da basit bir örnekle tanıtılmıştır. Bu tür bir temsili biçiminde, bir devre mantık kapıları ve bunların bağlantıları ile tanımlanır. Kapı düzeyinde modelleme, bir şematik diyagramın metinsel tanımını sağlar. Verilog HDL



ŞEKİL 4.31

Verilog yapılarının doğruluk tabloları, Boolean denklemleri ve şemalar ile ilişkisi

12 temel kapıyı önceden tanımlanmış ilkelere dahil eder. Bu ilkel kapılardan dördü üç durumlu türdendir. Diğer sekiz kapı ise Bölüm 2.8'de listelenenlerle aynıdır. Hepsi küçük harfle tanımlanan anahtar kelimelerle **and**, **nand**, **or**, **nor**, **xor**, **xnor**, **not**, ve **buf** ile bildirilmiştir. **and** gibi ilkel kapılar n-giriş ilkel kapılardır. Herhangi bir sayıda skalar girişi olabilir (örneğin, üç girişli **and** ilkel kapısı). **buf** ve **not** ilkel kapıları ise n-çıkış ilkel kapılardır. Tek bir giriş, tanımlayıcılarıyla ayırt edilen birden fazla çıkış hattını sürükleyebilir.

Verilog dili, her tür kapının işlevsel tanımını da içerir. Her kapının mantığı dört değerli bir sisteme dayanır. Kapılar simüle edildiğinde, simülatör her bir kapının çıkışına her an için bir değer atar. 0 ve 1'in yanı sıra, iki başka değer daha vardır: *bilinmeyen* ve *yüksek empedans*.

Bilinmeyen bir değer **x** ile gösterilir ve yüksek empedans **z** ile gösterilir. Bir bilinmeyen değer, bir sinyalin mantıksal değeri belirsiz olduğunda simülasyon sırasında atanır—örneğin, değeri 0 mı yoksa 1 mi olduğu belirlenemediğinde (örneğin, sıfırlama koşulu olmayan bir flip-flop). Yüksek empedans durumu, etkinleştirilmemiş üç durumlu kapıların çıkışında veya bir telin yanlışlıkla bağlantısız bırakılması durumunda meydana gelir. **ve**, **veya**, **xor** ve **not** ilkelere ait dört değerli mantık doğruluk tabloları Tablo 4.9'da gösterilmiştir. Diğer dört kapının doğruluk tablosu aynıdır, tek fark çıkışların tamamlanmış olmasıdır. **ve** kapısı için, çıkış yalnızca her iki giriş 1 olduğunda 1'dir ve herhangi bir giriş 0 ise çıkış 0'dır. Aksi takdirde, bir giriş **x** veya **z** ise, çıkış **x** olur. **veya** kapısının çıkışı, her iki giriş 0 ise 0'dır, herhangi bir giriş 1 ise 1'dir ve aksi takdirde **x** olur.

Bir ilkel kapı bir modülde listelendiğinde, modülde *örneklendi* denir.

Genel olarak, bileşen örneklemeleri, tasarımdaki daha düşük seviyeli bileşenlere atıfta bulunan ifadeler olup, esasen bu bileşenlerin daha yüksek seviyeli modülde benzersiz kopyalarını (veya *örneklerini*) oluşturur. Bu nedenle, tanımında bir kapı kullanan bir modül,

Tablo 4.9

Önceden Tanımlanmış Temel Kapılar için Doğruluk Tablosu

and	0	1	x	z	veya	0	1	x	z
0	0	0	0	0	0	0	1	x	x
1	0	1	x	x	1	1	1	1	1
x	0	x	x	x	x	x	1	x	x
z	0	x	x	x	z	x	1	x	x

xor	0	1	x	z	not	giriş	çıkış
0	0	1	x	x		0	1
1	1	0	x	x		1	0
x	x	x	x	x		x	x
z	x	x	x	x		z	x

kapıyı örnekle. Örnekleme işlemini, bir devre kartında parçaları yerleştirme ve bağlama işleminin HDL karşılığı olarak düşünün.

Şimdi kapı düzeyinde modellemenin iki örneğini sunuyoruz. Her iki örnek de birden fazla bit genişliği ne sahip tanımlayıcılar kullanmaktadır, bunlara *vektörler* denir. Bir vektörü belirten sözdizimi, köşeli parantezler içinde iki sayı ile birlikte bir iki nokta üst üste ile ayrılmıştır. Aşağıdaki Verilog ifadeleri iki vektörü belirtmektedir:

çıkış [0: 3] D;

tel [7: 0] TOPLAM;

İlk ifade, 0'dan 3'e kadar dört bitli bir çıkış vektörü *D* tanımlar. İkincisi, 7'den 0'a kadar numaralandırılmış sekiz bitli bir tel vektörü *TOPLAM* tanımlar. (Not : Listelenen ilk (en soldaki) sayı (dizi indeksi) her zaman vektörün en anlamlı bitidir.) Bireysel bitler köşeli parantezler içinde belirtilir, bu nedenle *D[2]* *D*'nin 2. bitini belirtir. Vektörlerin parçalarını (bit gruplarını) adreslemek de mümkündür. Örneğin, *TOPLAM[2: 0]* vektörün *TOPLAM* en az anlamlı üç bitini belirtir.

HDL Örneği 4.1, iki girişli dört çıkışlı bir kodlayıcının kapı düzeyindeki tanımını göstermektedir. (Bkz. Şekil 4.19.) Bu kodlayıcının iki veri girişi *A* ve *B* ile bir etkinleştirme girişi *E* bulunmaktadır. Dört çıkış, *D* vektörü ile belirtilmiştir. **wire** bildirimi, iç bağlantılar içindir. Üç **not** kapısı, girişlerin tamamlayıcısını üretir ve dört **nand** kapısı, *D* için çıkışları sağlar. Unutmayın ki, çıkış her zaman bir primitive port listesinde ilk olarak listelenir, ardından girişler gelir. Bu örnek, Şekil 4.19'daki kodlayıcıyı tanımlar ve Bölüm 3.10'da belirlenen prosedürleri takip eder. Anahtar kelimelerin **not** ve **nand** yalnızca bir kez yazıldığını ve her kapı için tekrarlanmasına gerek olmadığını, ancak serideki her kapının sonuna virgül eklenmesi gerektiğini unutmayın; son ifadeyi ise noktalı virgül ile sonlandırmalısınız.

HDL Örneği 4.1 (İki'den Dörde Hat Çözücü)

```
// İki'den dörde hat çözücünün kapı düzeyindeki tanımı
// Açıklık için sembol E yerine enable kullanılarak Şekil 4.19'a bakınız.

modül decoder_2x4_gates (D, A, B, enable);
    çıkış[0]: D;
    girdi    B; A,
    girdi    enable;
    tel      A_not, B_not, enable_not;

    değil
        G1 (A_not, A),
        G2 (B_not, B),
        G3 (enable_not, enable);
    nand
        G4 (D[0], A_not, B_not, enable_not),
        G5 (D[1], A_not, B, enable_not),
        G6 (D[2], A, B_not, enable_not),
        G7 (D[3], A, B, enable_not);
endmodule
```

İki veya daha fazla modül, bir tasarımın hiyerarşik tanımını oluşturmak için birleştirilebilir. İki temel tasarım metodolojisi türü vardır: yukarıdan aşağıya ve aşağıdan yukarıya. Bir yukarıdan aşağıya tasarımda, üst düzey blok tanımlanır ve ardından üst düzey bloğu birleştirmek için gerekli alt bloklar belirlenir. Bir *aşağıdan yukarıya* tasarımda, yapı taşları önce belirlenir ve ardından üst düzey bloğu oluşturmak için birleştirilir. Örneğin, binary adder (ikili toplayıcı) Şekil 4.9'da gösterildiği gibi, dört full-adder (tam toplayıcı) bloğuyla inşa edilmiş bir üst blok bileşeni olarak düşünülebilir. Her tam toplayıcı ise iki yarım toplayıcı bloğuyla inşa edilmiştir. Yukarıdan aşağıya bir tasarımda, dört bitlik toplayıcı önce tanımlanır ve ardından iki toplayıcı tanımlanır. Aşağıdan yukarıya bir tasarımda, yarım toplayıcı tanımlanır, ardından her tam toplayıcı inşa edilir ve ardından dört bitlik toplayıcı tam toplayıcılardan oluşturulur.

Aşağıdan yukarıya hiyerarşik bir dört bitli toplayıcının HDL'deki tanımı gösterilmektedir. Örnek 4.2. Yarım toplayıcı, ilkel kapıları örnekleyerek tanımlanır. Sonraki modül, iki yarım toplayıcıyı örnekleyerek ve bağlayarak tam toplayıcıyı tanımlar. Üçüncü modül, dört tam toplayıcıyı örnekleyerek ve bağlayarak dört bitli toplayıcıyı tanımlar. Bir tanımlayıcının ilk karakterinin bir sayı olamayacağını, ancak bir alt çizgiyle başlayabileceğini unutmayın, bu nedenle modül adı *_4bitadder* geçerlidir. Anlamı olan, ancak önceden bir alt çizgi gerektirmeyen alternatif bir ad *adder_4_bit*'dir. Örnekleme, örneklenen modülün adı ile birlikte yeni (veya aynı) bir port adı seti kullanılarak yapılır. Örneğin, tam toplayıcı modülü içindeki yarım toplayıcı *HAI* portları ile örneklenir *S1*, *C1*, *x* ve *y*. Bu, *S1* ve

C1 ve girişler *x* ve *y* olan bir yarım toplayıcı üretir.

HDL Örneği 4.2 (Ripple-Carry Toplayıcı)

```

// Dört bitli ripple carry toplayıcının kapı düzeyindeki tanımı
// Yarım toplayıcının tanımı ( Şekil 4.5b )

// modül half_adder (S, C, x, y); // Verilog 1995 sözdizimi // Verilog 1995 sözdizimi
// çıkış S;
// girdi x, y;

modül yarım_toplayıcı ( çıkış S, C, girdi x, y); // Verilog 2001, 2005 sözdizimi // Verilog 2001, 2005 sözdizimi
// Temel kapıları örnekle
xor (S, x, y);
ve (C, x, y);
bitir

// Tam toplayıcının tanımı ( Şekil 4.8 ) // Verilog 1995 sözdizimi
// modül full_adder (S, C, x, y, z);
// çıkış S, C;
// girdi x, y, z;

modül full_adder ( çıkış S, C, girdi x, y, z); // Verilog 2001, 2005 sözdizimi // Verilog 2001, 2005 sözdizimi
tel S1, C1, C2;

// Yarım toplayıcıları örnekle
yarım_toplayıcı HA1 (S1, C1, x, y);
yarım_toplayıcı HA2 (S, C2, S1, z);
ve G1 (C, C2, C1);
endmodule

// Dört bitli toplayıcının tanımı ( Şekil 4.9 ) // Verilog 1995 sözdizimi
// modül ripple_carry_4_bit_adder (Sum, C4, A, B, C0);
// çıkış [3: 0] Sum; Sum;
// çıktı C4;
// girdi [3: 0] A, B;
// girdi C0;
// Alternatif Verilog 2001, 2005 sözdizimi:

modül ripple_carry_4_bit_adder ( çıkış [3: 0] Toplam, çıkış C4,
girdi [3: 0] A, B, girdi C0);
tel C1, C2, C3; // Ara taşıma

// Tam toplayıcılar zincirini oluştur
tam_toplayıcı FA0 (Toplam[0], C1, A[0], B[0], C0),
FA1 (Toplam[1], C2, A[1], B[1], C1),
FA2 (Toplam[2], C3, A[2], B[2], C2),
FA3 (Toplam[3], C4, A[3], B[3], C3);

endmodule

```

HDL Örneği 4.2, modun (örneğin, **output**), türün (**reg**) ve bir portun vektör aralığının (örneğin, [3: 0]) bildirimini tanımlayan tanımlayıcıların fazladan yazımını ortadan kaldıran Verilog 2001, 2005 sözdizimini göstermektedir. Standardın ilk versiyonu (1995) bu bildirimler için ayrı ifadeler kullanmaktadır.

Modüllerin diğer modüller içinde örneklendirilebileceğini (iç içe geçebileceğini) unutmayın, ancak modül bildirimleri iç içe geçemez; yani, bir modül tanımı (bildirimi) başka bir modül bildirimine yerleştirilemez. Diğer bir deyişle, bir modül tanımı, başka bir modülün **module** ve **endmodule** anahtar kelimeleri arasındaki metne eklenemez.

Bir modül tanımının başka bir modüle dahil edilmesinin tek yolu, onu örneklemeektir. Diğer modüller içinde modüllerin örneklenmesi, bir tasarımın hiyerarşik ayrıştırmasını oluşturur. Bir modülün tanımının, diğer modüllerin örneklemelelerinden oluşması durumunda *yapısal* bir tanım olduğu söylenir. Ayrıca, tanımlı modüller örneklenirken örnek isimlerinin belirtilmesi gerektiğini unutmayın (örneğin, üçüncü modüldeki ilk tam toplayıcı için *FA0* gibi), ancak ilkel kapıları örneklerken bir isim kullanmak isteğe bağlıdır. Modül *ripple_carry_4_bit_adder* örneklenmiş ve birbirine bağlı tam toplayıcılardan oluşur; her biri de yarım toplayıcılardan ve bazı *yapıştırıcı mantık* elemanlarından oluşur. Tasarım hiyerarşisinin en üst seviyesi veya ana modülü, *ripple_carry_4_bit_adder* modülüdür.

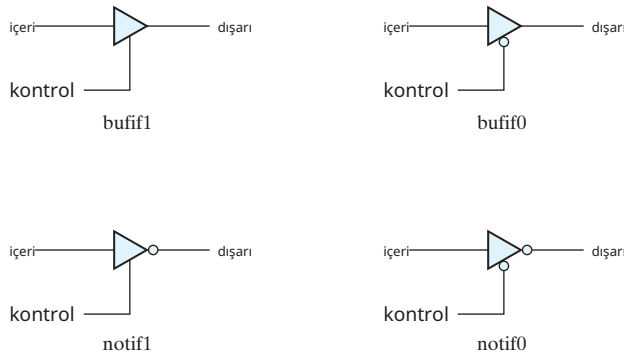
Dört kopya *full_adder* onun çocuk modülleridir, vb. *C0* zincirin en az anlamlı bitini oluşturan hücrenin bir girişi ve *C4* en anlamlı biti oluşturan hücrenin çıkışıdır.

Üç Durumlu Kapılar

Bölüm 4.11'de belirtildiği gibi, üç durumlu bir kapının, kapıyı yüksek empedans durumuna getirebilen bir kontrol girişi vardır. Yüksek empedans durumu Verilog'da **z** ile sembolize edilir.

Dört tür üç durumlu kapı vardır, Şekil 4.32'de gösterildiği gibi. **bufif1** kapısı, *control* = 1 olduğunda normal bir tampon gibi davranır. Çıkış, *control* = 0 olduğunda yüksek empedans durumuna *z* geçer. **bufif0** kapısı benzer bir şekilde davranır, ancak yüksek empedans durumu *control* = 1 olduğunda gerçekleşir. İki **notif** kapısı benzer bir şekilde çalışır, ancak kapı yüksek empedans durumunda değilken çıkış, girişin tamamlayıcısidir. Kapılar, aşağıdaki ifade ile oluşturulur.

kap adı(*çıkış, giriş, kontrol*);



ŞEKİL 4.32

Üç durumlu kapılar

Kapı adı, 4 üç durumlu kapıdan herhangi birinin adı olabilir. Simülasyonda, çıkış 0, 1, x veya z olarak sonuçlanabilir. Kapı örneklerinin iki tanesi şunlardır:

bufif1 (OUT, A, control);
notif0 (Y, B, enable);

İlk örnekte, giriş A $control = 1$ olduğunda OUT 'a aktarılır. OUT $control = 0$ olduğunda z 'ye gider. İkinci örnekte, çıkış $Y = z$ $enable = 1$ olduğunda ve çıkış $Y = B$ $enable = 0$ olduğunda gerçekleşir.

Üç durumlu kapıların çıktıları, ortak bir çıkış hat oluşturmak için bir araya getirilebilir. Böyle bir bağlantıyı tanımlamak için, Verilog HDL, çıkışın birden fazla sürücüye sahip olduğunu belirtmek için **tri** anahtar kelimesini kullanır. Örneğin, Şekil 4.33'te gösterilen üç durumlu kapılarla ikiye bir hat çoklayıcıyı düşünelim.

HDL tanımı, çıkış için **tri** veri türünü kullanmalıdır:

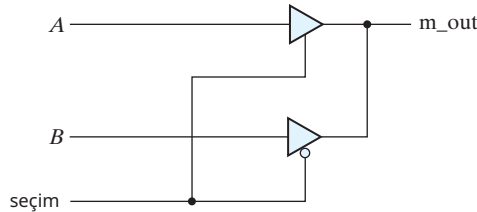
```
// Üç durumlu çıkışa sahip Mux
modül mux_tri (m_out, A, B, seçim);
    çıkış m_out;
    giriş A, B, seçim;
    tri    m_out;

    bufif1 (m_out, A, select);
    bufif0 (m_out, B, select);
endmodule
```

İki üç durumlu tampon aynı çıkışa sahiptir. Ortak bir bağlantıya sahip olduklarını göstermek için, m_out anahtar kelimesi ile bildirilmelidir.

Anahtar kelimeler **wire** ve **tri nets** olarak adlandırılan bir veri türleri kümesine örneklerdir ve bu, donanım elemanları arasındaki bağlantıları temsil eder. Simülasyonda, değerleri sürekli bir atama ifadesi veya temsil ettikleri cihazın çıkışı tarafından belirlenir.

net bir anahtar kelime değildir, ancak **wire** ve **wor**, **wand**, **tri**, **supply1**, ve **supply0** gibi veri türleri sınıfını temsil eder. **wire** bildirimi en sık kullanılanıdır. Aslında, bir tanımlayıcı kullanılırsa, ancak bildirilmezse, dil bunun (varsayılan olarak) **wire** olarak yorumlanacağını belirtir. Net **wor** kablolu-VE yapılandırmasının donanım uygulamasını modellemektedir (emiter bağlı mantık). **wand** kablolu-VE yapılandırmasını (açık toplayıcı teknolojisi; bkz. Şekil 3.26) modellemektedir. **supply1** ve **supply0** sırasıyla güç kaynağını ve toprağı temsil eder. Bir cihazın girişini ya 1 ya da 0'a sabitlemek için kullanılır.



ŞEKİL 4.33

Üç durumlu tamponlara sahip ikiye bir hat çoklayıcı

Veri Akışı Modelleme

Kombinasyon mantığının veri akışı modellemesi, ikili

operandlar üzerinde işlem yapan bir dizi operatör kullanır ve ikili bir sonuç üretir. Verilog HDL, yaklaşık 30 farklı operatör sağlar.

Tablo 4.10 bu operatörlerden bazılarını, sembollerini ve gerçekleştirdikleri işlemleri listeler. (Verilog 2001, 2005 tarafından desteklenen operatörlerin tam listesi, Tablo 8.1

Bölüm 8.2'de bulunabilir.) Aritmetik ve mantık işlemleri arasında ayırım yapmak gereklidir, bu nedenle her biri için farklı semboller kullanılır. Artı sembolü (+) toplama aritmetik işlemini gösterir; bit düzeyinde mantıksal VE işlemi (kesişim) & sembolünü kullanır. Bit düzeyinde mantıksal VEYA (birleşim), DEĞİL ve XOR için özel semboller vardır. Eşitlik sembolü, atama ifadesi ile kullanılan eşitlik işaretinden ayırt etmek için iki eşit işaret i (aralarında boşluk olmadan) kullanır. Bit düzeyindeki operatörler, bir çift vektör operandı üzerinde bit bit işlem yaparak bir vektör sonucu üretir. Birleştirme operatörü, birden fazla operandı eklemek için bir mekanizma sağlar. Örneğin, her biri iki bit olan iki operand, dört bitlik bir operand oluşturmak için birleştirilebilir.

Koşullu operatör, bir çoklayıcı gibi davranır ve daha sonra HDL Örneği 4.6 ile bir likte açıklanacaktır.

Bit düzeyinde bir operatör (örneğin, ~) ve onun karşılık gelen mantıksal operatörü (örneğin, !) operandlarına bağlı olarak farklı sonuçlar üretebilir. Eğer operandlar skalar ise sonuçlar aynı olacaktır; eğer operandlar vektör ise sonuç mutlaka eşleşmeyecektir. Örneğin, ~ (1010) (0101) ve !(1010) 0'dır. Bir ikili değer, 0 değilse mantıksal olarak doğru kabul edilir. Genel olarak, aritmetik işlemleri tanımlamak için bit düzeyindeki operatörleri ve mantıksal işlemleri tanımlamak için mantıksal operatörleri kullanır.

Veri akışı modelleme, sürekli atamaları ve assign anahtar kelimesini kullanır. Sürekli bir atama, bir değeri bir ağa atayan bir ifadedir. Veri türü ailesi net, devre elemanları arasındaki fiziksel bağlantıyı temsil etmek için Verilog HDL'de kullanılır. Bir ağ

Tablo 4.10

Bazı Verilog HDL Operatörleri

Sembol	İşlem Sembolü	İşlem
+	ikili toplama	
-	ikili çıkarma	
&	bit düzeyinde VE	
	bit düzeyinde VEYA	
^	bit düzeyinde XOR	
~	bit düzeyinde DEĞİL	! mantıksal DEĞİL
	eşitlik	
	büyüktür	
<	{ } 'den daha	
az	birleştirme	
?:	koşullu	

bir ağ anahtar kelimesi (örneğin, *tel*) ile açıkça tanımlanır veya bir tanımlayıcının giriş portu olarak tanımlanmasıyla. Bir ağ ile ilişkili mantık değeri, ağın bağlı olduğu yere göre belirlenir. Eğer ağ bir kapının çıkışına bağlıysa, ağın *sürüldüğü* söylenir ve ağın mantık değeri, kapının girişlerinin mantık değerleri ve kapının doğruluk tablosu tarafından belirlenir. Eğer bir ağın tanımlayıcısı sürekli atama ifadesinin sol tarafında veya bir prosedürel atama ifadesinde ise, ağa atanan değer, operatörler ve operatörler kullanan bir Boolean ifadesi ile belirtilir. Örneğin, değişkenler in tanımlandığını varsayarsak, skalar veri girişlerine sahip bir ikiye bir hat çoklayıcı *A* ve *B*, seçim girişi *S* ve çıkış *Y* ile sürekli atama ile tanımlanır.

atan *Y* (*A* && *S*) || (*B* && *S*)

Y, *A*, *B* ve *S* arasındaki ilişki, atama anahtar kelimesi ile tanımlanır, ardından her def çıkış *Y* ve bir eşittir işareti gelir. Eşittir işaretinin ardından bir Boolean ifadesi gelir. Donanım terimleriyle, bu atama, OR kapısının çıkışını *Y* teline bağlamakla eşdeğerdir.

İki önceki kapı seviyesinin veri akışı modellerini gösteren sonraki iki örnek. Aktif düşük çıkış etkinleştirme ve ters çıkışa sahip iki-dört hat kodlayıcısının veri akışı tanımları HDL Örneği 4.3'te gösterilmiştir. Devre, her bir çıkış için bir tane olmak üzere, Boolean ifadeleri kullanarak dört sürekli atama ifadesi ile tanımlanmıştır. Dört bitli toplayıcının veri akışı tanımları HDL Örneği 4.4'te gösterilmiştir. Toplama mantığı, toplama ve birleştirme operatörlerini kullanan tek bir ifade ile tanımlanmıştır. Artı sembolü (+) dört bitlik *A*'nın, dört bitlik *B*'nin ve bir bitlik *C_{in}* ile ikili toplamını belirtir. Hedef çıkış, çıkış taşıma *C_{out}* ve dört bitlik *Sum*'un birleştirilmesidir. Operatörlerin birleştirilmesi, süslü parantezler içinde ve operatörleri ayıran bir virgül ile ifade edilir. Böylece, {*C_{out}*, *Sum*} toplama işleminin beş bitlik sonucunu temsil eder.

HDL Örneği 4.3 (Veri Akışı: İki-Dört Hat Kodlayıcı)

// İki-dört hat kodlayıcısının veri akışı tanımları

// Bakınız Fig. 4.19. Not: Şekil E sembolünü kullanıyor, ancak

// Verilog modeli işlevselliği açıkça belirtmek için enable kullanıyor.

```
modül decoder_2x4_df ( // Verilog 2001, 2005 sözdizimi
    çıkış [0:3] D,
    giriş enable
);
    atanan D[0] !((!A) && (!B) && (!enable)),
           D[1] !(A && B && (!enable)),
           D[2] !(A && B && (!enable))
           D[3] !(A && B && (!enable))
bitir
```

HDL Örneği 4.4 (Veri Akışı: Dört Bitli Toplayıcı)

```
// Dört bitli toplayıcının veri akışı tanımı
// Verilog 2001, 2005 modül port sözdizimi

modül binary_adder (
    çıkış [3: 0] Toplam,          Toplam,
    giriş [3: 0] A, B,          C_out,
    giriş [3: 0] C_in;
);

    atan {C_out, Toplam} A + B + C_in;
modül sonu
```

Veri akışı HDL modelleri, kombinasyon devrelerini *işlevleri* ile tanımlar, kapı y apıları ile değil. Veri akışı tanımlarının dijital tasarımı nasıl kolaylaştırdığını göstermek için, HDL Örneği 4.5'te tanımlanan 4 bitlik büyüklük karşılaştırıcısını düşünün. Modül, iki 4 bitlik girişi A ve B ve üç çıkışı belirtir. Bir çıkış (A_lt_B) 1'dir eğer AB 'den küçüktür, ikinci çıkış (A_gt_B) 1'dir eğer AB 'den büyüktür, ve üçüncü çıkış (A_eq_B) 1'dir eğer $A B$ 'ye eşittir. Eşitlik (kimlik) iki eşit işareti () ile sembolize edilir atanma işlemi ile ayırt etmek için. Bir Verilog HDL sentez derleyicisi bu modül tanımını girdi olarak kabul edebilir, sentez algoritmalarını çalıştırabilir ve bir çıkış ağ listesi ve bir şematik sağlayabilir Fig. 4.17 'deki devreye eşdeğer bir devre, tüm bunları manuel müdahale olmadan! Tasarımcı şemayı çizmek zorunda değildir.

HDL Örneği 4.5 (Veri Akışı: Dört Bitli Karşılaştırıcı)

```
// Dört bitli karşılaştırıcının veri akışı tanımı //V2001, 2005 sözdizimi

modül mag_compare
( çıkış A_lt_B, A_eq_B, A_gt_B,
  giriş [3: 0] A, B );
);
    atan A_lt_B = (A < B);
    atan A_gt_B = (A > B);
    atan A_eq_B = (A == B);
endmodule
```

Bir sonraki örnek, koşullu operatörü (? :) kullanır. Bu operatör üç operand alır:

koşul ? doğru-ifade : yanlış-ifade;

Koşul değerlendirilir. Eğer sonuç mantıksal 1 ise, doğru ifade değerlendirilir ve değer atama ifadesinin sol tarafına bir değer atamak için kullanılır. Eğer sonuç

mantık 0, yanlış ifade değerlendirilir. İki koşul birlikte bir if-else koşuluna eşdeğerdir. HDL Örneği 4.6, koşullu operatörü kullanarak iki girişli bir çoklayıcıyı tanımlar. Sürekli atama

assign OUT = select ? A : B;

OUT'un koşulunu belirtir $OUT = A$ eğer $select = 1$, aksi takdirde $OUT = B$ eğer $select = 0$.

HDL Örneği 4.6 (Veri Akışı: İki Girişli Çoklayıcı)

```
// İki girişli bir çoklayıcının veri akışı tanımı
modül mux_2x1_df(m_out, A, B, select);
    çıkış      m_out;
    giriş      A,
    giriş      B,
    giriş      select;

    atan m_out = (select)? A : B;
endmodule
```

Davranışsal Modelleme

Davranışsal modelleme, dijital devreleri işlevsel ve algoritmik bir düzeyde temsil eder. Genellikle ardışık devreleri tanımlamak için kullanılır, ancak kombinasyonel devreleri tanımlamak için de kullanılabilir. Burada, konuyu tanıtmak için iki basit kombinasyonel devre örneği veriyoruz. Davranışsal modelleme, ardışık devrelerin incelenmesinden sonra Bölüm 5.6'da daha ayrıntılı olarak sunulmaktadır.

Davranışsal tanımlar, her zaman anahtar kelimesini kullanır, ardından isteğe bağlı bir olay kontrol ifadesi ve bir dizi prosedürel atama ifadesi gelir. Olay kontrol ifadesi, ifadelerin ne zaman yürütüleceğini belirtir. Bir prosedürel atama ifadesinin hedef çıktısı, **reg** veri türünde olmalıdır. Atamanın hedef çıktısının sürekli olarak güncellenebildiği **wire** veri türünün aksine, **reg** veri türü, yeni bir değer atanana kadar değerini korur.

HDL Örneği 4.7, ikiye bir hat çoklayıcısının davranışsal tanımını göstermektedir. (Bunu HDL Örneği 4.6 ile karşılaştırın.) Değişken m_out bir hedef çıkış olduğundan, **reg** veri olarak tanımlanmalıdır (**output** tanımına ek olarak). **always** bloğu içindeki prosedürel atama ifadeleri, @ sembolünden sonra listelenen değişkenlerden herhangi birinde bir değişim olduğunda her seferinde yürütülür. (**always** ifadesinin sonunda noktalı virgül (;) yoktur.) Bu durumda, bu değişkenler giriş değişkenleri A , B ve $select$ dir. İfadeler, A , B veya $select$ değeri değiştirirse yürütülür. Anahtar kelimenin kullanıldığını unutmayın veya , bit düzeyinde mantıksal VEYA operatörü " $|$ " yerine değişkenler arasında kullanılır. Koşullu ifade **if-else** $select$ girişinin değerine dayalı bir karar sağlar. **if** ifadesi eşitlik sembolü olmadan yazılabilir:

if (select) OUT = A;

Bu ifade, $select$ mantıksal 1 için kontrol edilir anlamına gelir.

HDL Örneği 4.7 (Davranışsal: İki'ye Bir Hat Çoklayıcı)

```
// İki'ye bir hat çoklayıcısının davranışsal tanımı
```

```
modül mux_2x1_beh (m_out, A, B, select);
```

```
çıktı          m_out;
```

```
giriş          A, B, select;
```

```
reg           m_out;
```

```
her zaman      @(A veya B veya select)
```

```
Eğer (select == 1) m_out = A;
```

```
else m_out = B;
```

```
endmodule
```

HDL Örneği 4.8, dörtte bir hat çoklayıcısının işlevini tanımlar. `select` girişi iki bitlik bir vektör olarak tanımlanır ve çıkış `y` **reg** türünde tanımlanır. Bu örnekte, `always` ifadesi, **case** ve **endcase** anahtar kelimeleri arasında yer alan ardışık bir blok a sahiptir. Bloğun yürütülmesi, @ sembolünden sonra listelenen girişlerden herhangi biri değeri değiştirdiğinde gerçekleşir. **case** ifadesi çok yönlü bir koşullu dal yapısıdır. `in_0`, `in_1`, `in_2`, `in_3` veya `select` değiştiğinde, `case` ifadesi (`select`) değerle indirilir ve değeri, ardından gelen ifadeler listesindeki değerlerle, yani **case** öğeleriyle yukarıdan aşağıya karşılaştırılır. **case** ifadesiyle eşleşen ilk **case** öğesiyle ilişkili ifade yürütülür. Eşleşme yoksa, hiçbir ifade yürütülmez. `select` iki bitlik bir sayı olduğundan, 00, 01, 10 veya 11'e eşit olabilir. `case` öğeleri, liste yukarıdan aşağıya değerlendirildiği için örtük bir önceliğe sahiptir.

Liste, *duyarlılık listesi* (Verilog 2001, 2005) olarak adlandırılır ve sinyalleri "OR" ile birleştirilerek oluşturulan *olay kontrol ifadesi* (Verilog 1995) ile eşdeğerdir. Kombinasyonel mantık, reaktif—bir girdi değiştiğinde bir çıktı değişebilir.

HDL Örneği 4.8 (Davranışsal: Dört'e Bir Hat Çoklayıcı)

```
// Dört'e bir hat çoklayıcısının davranışsal tanımı
```

```
// Verilog 2001, 2005 port sözdizimi
```

```
module mux_4x1_beh
```

```
( output reg m_out,
```

```
  input      in_0, in_1, in_2, in_3,
```

```
  input [1: 0] select
```

```
);
```

```
always @ (in_0, in_1, in_2, in_3, select) // Verilog 2001, 2005 sözdizimi
```

```
// Verilog 2001, 2005 sözdizimi
```

```
  case (select)
```

```
    2'b00:      m_out = in_0;
```

```
                = in_1;
```

```
    2'b10:      m_out = in_2;
```

```
    2'b11:      m_out = in_3;
```

```
  endcase
```

```
endmodule
```

Verilog'daki ikili sayılar, bir prim ile önceden belirtilen **b** harfi ile tanımlanır ve yorumlanır. Sayının boyutu önce yazılır ve ardından değeri gelir. Böylece, $2'b01$ iki bitlik bir ikili sayıyı belirtir ve değeri 01'dir. Sayılar, bellekte bir bit deseni olarak saklanır, ancak onlara ondalık, sekizli veya onaltılı formatlarda **d'**, **o'**, ve **h'**, harfleri ile a tıfta bulunulabilir. Örneğin, $4'HA$ $4'd10$ $4'b1010$ ve bir sim ülatörde aynı iç temsil ile sahiptir. Sayının tabanı belirtilmemişse, yorumlaması varsayılan olarak ondalık olur. Sayının boyutu belirtilmemişse, sistem sayının boyutunun en az 32 bit olduğunu varsayar; eğer bir ana simülatörün boyutu daha büyükse—örneğin, 64 bit—dil, boyutsuz sayıları saklamak için bu değeri kullanır. Tam sayı veri türü (anahtar kelime **integer**) 32 bitlik bir temsil ile saklanır. Alt çizgi () bir sayıya eklenerek kodun okunabilirliğini artırmak için kullanılabilir (örneğin, $16'b0101_1110_0101_0011$). Bunun başka bir etkisi yoktur.

durum yapısının iki önemli varyasyonu vardır: **casex** ve **casez**. İlk olarak, **case** ifadesinin veya **case** ögesinin **x** veya **z** mantıksal değerine sahip olan herhangi bir bitini don't-care (önemsemeyen) olarak ele alacaktır. **casez** yapısı ise, **case** ifadesi ile bir **case** ögesi arasında bir eşleşme tespit etmek amacıyla yalnızca **z** mantıksal değerini don't-care olarak ele alır.

case ögelerinin listesi tam olmak zorunda değildir. Eğer **case** ögeleri listesi, **case** ifadesinin tüm olası bit desenlerini içermiyorsa, hiçbir eşleşme tespit edilemez. Listede yer almayan **case** ögeleri, yani açıkça kodlanmamış bit desenleri, **default** anahtar kelimesi kullanılarak **case** ögeleri listesinin son ögesi olarak ele alınabilir. İlgili ifade, başka bir eşleşme bulunmadığında çalıştırılacaktır. Bu özellik, örneğin, bir ardışık makinede kullanılanlardan daha fazla olası durum kodu olduğunda faydalıdır. Bir **default case** ögesine sahip olmak, tasarımcının kullanı lmayan tüm durumları istenen bir sonraki duruma eşleştirmesine olanak tanır; her bir durumu ayrıntılı olarak açıklamak zorunda kalmadan, sentez aracının bir sonraki durumu keyfi olarak atamasını engeller.

Burada gösterilen kombinasyonel devrelerin davranış tanımları basit örneklerdir. Davranışsal modelleme ve prosedürel atama ifadeleri, sıralı devreler hakkında bilgi gerektirir ve Bölüm 5.6'da daha ayrıntılı olarak ele alınmaktadır.

Basit Bir Test Tezgahı Yazma

Bir test tezgahı, bir devrenin HDL modeline bir uyarı tanımlamak ve uygulamak için kullanılan bir HDL programıdır; bu, devreyi test etmek ve simülasyon sırasında yanıtını gözlemlemek amacıyla yapılır. Test tezgahları oldukça karmaşık ve uzun olabilir ve test edilen tasarımdan daha uzun sürede geliştirilebilir. Bir testin sonuçları, bir devreyi test etmek için kullanılan test tezgahı kadar iyidir. Bir devreyi kapsamlı bir şekilde test edecek uyarılar yazmak için dikkat edilmelidir; belirtilen tüm çalışma özelliklerini kullanmalıdır. Ancak burada ele alınan test tezgahları görece basittir, çünkü test etmek istediğimiz devreler yalnızca kombinasyonel mantık uygulamaktadır. Örnekler, HDL uyarı modüllerinin bazı temel özelliklerini göstermek için sunulmuştur. Bölüm 8, test tezgahlarını daha derinlemesine ele almaktadır.

Test tezgahları, test edilen devreye bir uyarı sağlamak için **always** ifadesini kullanmanın yanı sıra **initial** ifadesini de kullanır. "**always** ifadesi" terimini gevşek bir şekilde kullanıyoruz. Aslında, **always**, ilişkili ifadenin nasıl yürütüleceğini belirten bir Verilog dil yapısıdır (olay kontrol ifadesine tabi olarak). **always** ifadesi

öngüde tekrar tekrar çalıştırılır. **initial** ifadesi yalnızca bir kez, simülasyon zamanı 0'dan başlayarak çalıştırılır ve # sembolü ile belirtilen belirli bir zaman birimi sayısı kadar gecikmeli herhangi bir işlemi sürdürebilir. Örneğin, initial blokunu düşünelim

```
initial
begin
A 0; B 0;
#10 A 1;
#20 A 0; B 1;
end
```

Blok, **begin** ve **end** anahtar kelimeleri arasında yer alır. Zaman 0'da, A ve B 0'a ayarlanır. On zaman birimi sonra, A 1'e değiştirilir. Yirmi zaman birimi sonra ($t = 30$), A 0'a ve B 1'e değiştirilir. Üç bitlik bir doğru tablosu ile belirtilen girişler, the **initial** bloğu ile oluşturulabilir:

```
initial
begin
D 3'b000;
repeat (7)
#10  $D = D + 3'b001$ ;
end
```

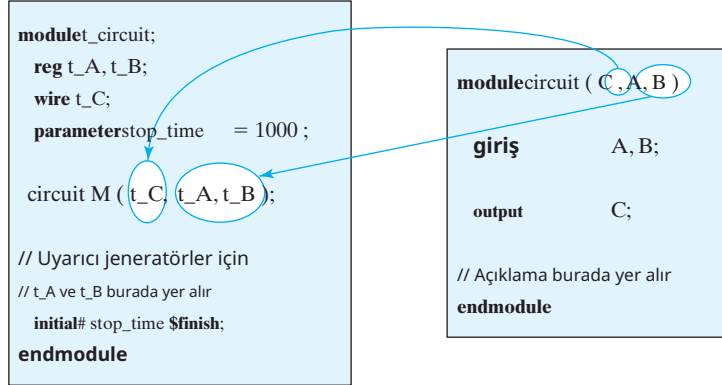
Simülatör çalıştığında, üç bitlik vektör D zaman 0'da 000 olarak başlatılır. Anahtar kelime **repeat** bir döngü ifadesini belirtir: D her 10 zaman biriminde bir kez, toplamda 7 kez 1 artırılır. Sonuç, 000'dan 111'e kadar bir dizi ikili sayı olur.

Bir uyarı modülü aşağıdaki biçime sahiptir:

```
modül test_modül_adı;
// Yerel reg ve tel tanımlayıcılarını bildir.
// Test modülünü başlatın.
// Simülasyonu sonlandırmak için $finish kullanarak bir kronometre belirtin.
// initial ve always ifadelerini kullanarak uyarıcı oluşturun.
// Çıktı yanıtını (metin veya grafik (veya her ikisi)) görüntüleyin.
endmodule
```

Bir test modülü, diğer modüller gibi yazılır, ancak genellikle girişi veya çıkışı yoktur. Simülasyon için tasarım modülüne giriş olarak uygulanan sinyaller, uyarıcı modülünde yerel **reg** veri türü olarak tanımlanır. Test için görüntülenen tasarım modülünün çıkışları, uyarıcı modülünde yerel **wire** veri türü olarak tanımlanır. Test edilen modül, port listesinde yerel tanımlayıcılar kullanılarak başlatılır.

Şekil 4.34 bu ilişkiyi netleştirir. Uyarıcı modül, yerel tanımlayıcıları t_A ve t_B **reg** türü olarak tanımlayarak tasarım modülüne girişler üretir ve tasarım biriminin çıkışını **wire** tanımlayıcısı t_C ile kontrol eder. Yerel tanımlayıcılar, test edilen tasarım modülünü başlatmak için kullanılır. Simülatör, test tezgahındaki (gerçek) yerel tanımlayıcıları, t_A , t_B ve t_C ile resmi tanımlayıcılarla ilişkilendirir.



ŞEKİL 4.34

Uyarıcı ve tasarım modülleri arasındaki etkileşim

modül (A, B, C). Burada gösterilen ilişki, *port listesindeki* konuma dayanmaktadır ve bu, ele alacağımız örnekler için yeterlidir. Ancak okuyucunun dikkat etmesi gereken, Verilog'un daha büyük devrelerde portları bağlamak için daha esnek bir *isim ilişkisi* mekanizması sağladığıdır.

Başlangıç ve her zaman blokları tarafından üretilen uyarana verilen yanıt, metin formatında standart çıktı olarak ve grafiksel çıktı yeteneğine sahip simülatörlerde dalga formları (zaman diyagramları) olarak görünecektir. Sayısal çıktılar, Verilog sistem görevleri kullanılarak görüntülenir. Bunlar, \$ sembolü ile başlayan anahtar kelimeler tarafından tanınan yerleşik sistem işlevleridir. Görüntüleme için yararlı olan bazı sistem görevleri şunlardır:

- \$display** —değişkenlerin veya dizelerin bir kez görüntülenmesi için satır sonu dönüşü ile,
- \$write** — \$display ile aynı, ancak bir sonraki satıra geçmeden,
- \$monitor** —simülasyon çalışması sırasında bir değer değiştiğinde değişkenleri görüntüle,
- \$time** —simülasyon zamanını görüntüle,
- \$finish** —simülasyonu sonlandır.

\$display, \$write, ve \$monitor için sözdizimi şu şekildedir

: Görev adı (format spesifikasyonu, argüman

an listesi); Format spesifikasyonu, görüntülenenen sayıların tabanını belirtmek için % sembolünü kullanır ve tırnak içinde bir dize içerebilir (""). Taban, sırasıyla %b, %d, %h ve %o sembolleri ile tanımlanan ikili, ondalık, onaltılık veya sekizli olabilir (%B, %D, %H ve %O da geçerlidir). Örneğin, ifade:

\$display ("%d %b %b", C, A, B);

C ondalık olarak ve A ile B ikili olarak görüntülenmesini belirtir. Format spesifikasyonunda virgül yoktur, format spesifikasyonu ve argüman listesi

virgülle ayrıldığını ve argüman listesinin

değişkenler arasında virgüller içerdiğini belirtir. Alıntı içinde yer alan bir dizeyi belirten bir örnek

şu ifadeye benzer

\$display ("zaman %0d A %b", \$time, A, B);

ve şu çıktıyı üretecektir

zaman 3 A 10 B

burada (*time*), (*A*), ve (*B*) görüntülenecek dize parçasıdır. Format specifier'ları %0d , %b ve %b sırasıyla **\$time** , *A* , ve *B* için tabanı belirtir. Zaman değerlerini görüntülerken, %d yerine %0d formatını kullanmak daha iyidir. Bu, %d'nin dahil edeceği önde gelen boşluklar olmadan anlamlı rakamların görüntülenmesini sağlar. (%d, zamanın 32-bit bir sayı olarak hesaplandığı için yaklaşık 10 önde gelen boşluk gösterecektir.)

Bir uyarı modülünün örneği HDL Örneği 4.9'da gösterilmektedir. Test edilecek devre, Örnek 4.6'da tanımlanan ikiye bir hat çoklayıcısıdır.

t_mux_2x1_df portları yoktur. Mux'un girişleri **reg** anahtar kelimesi ile tanımlanmış ve çıkışları **wire** anahtar kelimesi ile tanımlanmıştır. Mux, yerel değişkenlerle örneklenmiştir. Başlangıç bloğu, simülasyon sırasında uygulanacak ikili değerlerin bir dizisini belirtir. Çıkış tepkisi **\$monitor** sistem görevi ile kontrol edilir. Argümanındaki bir değişkenin değeri her değiştiğinde, simülatör girişleri, çıkışı ve zamanı gösterir. Simülasyonun sonucu, örnekte simülasyon günlüğü altında listelenmiştir. Mux'un çalışmasını doğrulamak için, *m_out* *A* seçildiğinde *select* 1 ve *m_out* *B* seçildiğinde *select* 0 olduğu gösterilmektedir.

HDL Örneği 4.9 (Test Tezgahı)

```
// mux_2x1_df için uyarı ile test tezgahı
```

```
module t_mux_2x1_df;
```

```
  wire          t_mux_out;
```

```
  reg t_A,
```

```
  reg          t_select;
```

```
  parameter stop_time = 50;
```

```
mux_2x1_df M1 (t_mux_out, t_A, t_B, t_select);
```

```
// Test edilecek devrenin örneklenmesi
```

```
initial # stop_time $finish;
```

```
  initial begin
```

```
// Uyarıcı üretici
```

```
    t_select 1; t_A 0; t_B 1;
```

```
    #10 t_A 1; t_B 0;
```

```
    #10 t_select 0;
```

```
    #10 t_A 0; t_B 1;
```

```
  end
```

```
  initial begin
```

```
// Yanıt izleyici
```

```
    // $display (" time Seç A B m_out ");
```

```
    // $monitor ( $time , %b %b %b %b          , t_select, t_A, t_B, t_m_out);
```

```

$monitor (" time =", $time ,," select = %b A = %b B = %b OUT =" ,%b
t_select, t_A, t_B, t_mux_out);
end
endmodule

// İki girişli bir çoklayıcının veri akışı tanımı

// Örnek 4.6'dan
modül mux_2x1_df (m_out, A, B, select);
    çıkış          m_out;
    giriş A,
    giriş          select;

    atama m_out= (select)? A : B;
modül sonu

Simülasyon kaydı:
seçim 1 A 0 B 1 ÇIKIŞ 0 zaman      0
seçim 1 A 1 B 0 ÇIKIŞ 1 zaman      10
seçim 0 A 1 B 0 ÇIKIŞ 0 zaman      20
seçim 0 A 0 B 1 ÇIKIŞ 1 zaman      30

```

Mantık simülasyonu, bir kombinasyonel devrenin modelinin doğru olduğunu doğrulamanın hızlı ve doğru bir yöntemidir. İki tür doğrulama vardır: işlevsel ve zamanlama. *işlevsel* doğrulamada, devrenin mantıksal işlemini zamanlama dikkate alınmadan inceliyoruz. Bu, kombinasyonel devrenin doğruluk tablosunu türeterek yapılabilir. *zamanlama* doğrulamasında, devrenin çalışmasını kapılar arasındaki gecikmelerin etkisini dahil ederek inceliyoruz. Bu, belirli bir girdiye yanıt verdiklerinde kapıların çıkışlarındaki dalga formlarını gözlemleyerek yapılabilir. Kapı gecikmeleri olan bir devre örneği, Bölüm 3.10'da HDL Örneği 3.3'te sunulmuştur. Sonraki adımda, bir kombinasyonel devrenin doğruluk tablosunu üreten bir HDL örneği gösteriyoruz. **\$monitor** sistem görevi, verilen uyarandan kaynaklanan çıktıyı görüntüler. Yorumlanmış alternatif bir ifade, **\$display** görevi ile birlikte kullanılacak bir başlık oluşturur ve bu, her çıkış satırında isimlerin tekrarlanmasını ortadan kaldırır.

Kombinasyon devrelerinin analizi Bölüm 4.3'te ele alınmıştır. Bir tam toplayıcının çok seviyeli devresi analiz edilmiş ve doğruluk tablosu gözlem yoluyla türetilmiştir. Bu devrenin kapı düzeyindeki tanımı HDL Örneği 4.10'da gösterilmiştir.

Devre üç girdi, iki çıkış ve dokuz kapı içermektedir. Devrenin tanımı, Şekil 4.2'deki şematik diyagramına göre kapılar arasındaki bağlantıları takip etmektedir. Devre için uyarıcı ikinci modülde listelenmiştir. Devreyi simüle etmek için girişler üç bitlik **reg** vektörü *D* ile belirtilmiştir. *D*[2] girişi *A* ile eşdeğerdir,

D[1] girişi *B* ile, ve *D*[0] girişi *C* ile eşdeğerdir. Devrenin çıkışları *F*1 ve *F*2 olarak wire olarak tanımlanmıştır. *F*2'nin tamamlayıcı *F*2_b olarak adlandırılmıştır; bu, bir sinyalin tamamlayıcısını belirtmek için yaygın bir endüstri uygulamasını göstermektedir (yerine **_not** eklemek yerine). Bu prosedür

Şekil 4.34'te belirtilen adımları izler. tekrar döngüsü, 000'dan sonraki yedi ikili sayıyı sağlar. Simülasyonun sonucu, örnekle birlikte gösterilen çıkış gerçeklik tablosunu oluşturur. Listelenen gerçeklik tablosu, devrenin

bir tam toplayıcı olduğunu göstermektedir.

HDL Örneği 4.10 (Kapı Düzeyinde Devre)

// Şekil 4.2'nin devresinin kapı düzeyindeki tanımı

```
modül Şekil_4_2_Devresi (A, B, C, F1, F2);
  input A, B, C;
  output F1, F2;
  kablo T1, T2, T3, F2_b, E1, E2, E3;
  veya g1 (T1, A, B, C);
  ve g2 (T2, A, B, C);
  ve g3 (E1, A, B);
  ve g4 (E2, A, C);
  ve g5 (E3, B, C);
  veya g6 (F2, E1, E2, E3);
  değil g7 (F2_b, F2);
  ve g8 (T3, T1, F2_b);
  veya g9 (F1, T2, T3);
endmodule
```

// Devreyi analiz etmek için uyarı

```
modül test_devresi;
  reg [2: 0] D;
  tel F1, F2;
  Şekil_4_2_D[2], D[1], D[0], F1, F2;
  başlangıç
  başla
    D = 3'b000;
    tekrar (7) #10 D = D 1 1'b1;
  son
  initial
  $monitor ("ABC = %b F1 = %b F2 = %b", D, F1, F2);
```

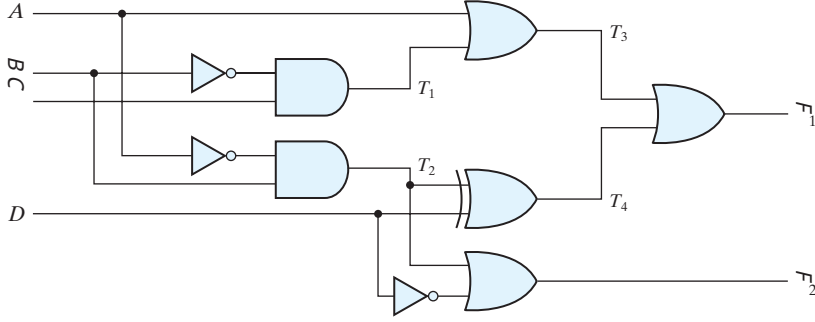
modül sonu

```
Simülasyon kaydı: ABC 000 F1 0 F2 0
ABC = 001 F1 = 1 F2 = 0 ABC = 010 F1 = 1 F2 = 0
ABC = 011 F1 = 0 F2 = 1 ABC = 100 F1 = 1 F2 = 0
ABC = 101 F1 = 0 F2 = 1 ABC = 110 F1 = 0 F2 = 1
ABC = 111 F1 = 1 F2 = 1
```

PROBLEMLER

(Soruların * ile işaretlenmiş cevapları metnin sonunda yer almaktadır. Uygun olduğunda, bir mantık dizaynı ve ilgili HDL modelleme problemi çapraz referans verilmiştir.)

Şekil P4.1'de gösterilen kombinyasyonel devreyi dikkate alın. (HDL—bkz. Problem 4.49.)

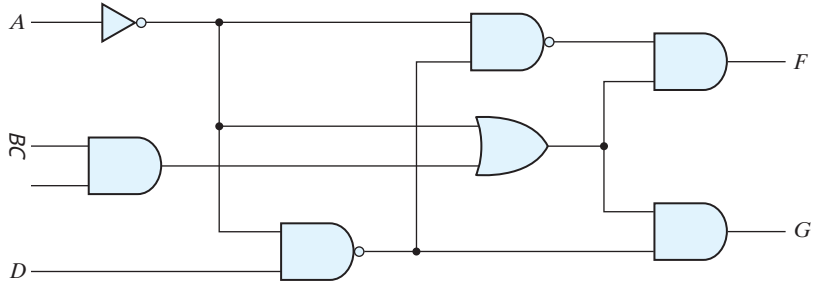


ŞEKİL P4.1

- (a) * T_1 ile T_4 için Boolean ifadelerini türetin. Çıktıları F_1 ve F_2 dört giriş bir fonksiyonu olarak değerlendirin.
- (b) Dört giriş değişkeninin 16 ikili kombinyasyonunu içeren doğruluk tablosunu listeleyin. Ardından, T_1 ile T_4 arasındaki ikili değerleri ve çıktıları F_1 ve F_2 tabloda listeleyin.
- (c) Çıktı Boolean fonksiyonlarını (b) kısmında elde edilen haritalarda çiziniz ve sadeleştirilmiş Boolean ifadelerinin (a) kısmında elde edilenlerle eşdeğer olduğunu gösterin.

4.2

* Şekil P4.2'deki devredeki giriş değişkenleri cinsinden çıktı F ve G için sadeleştirilmiş Boolean ifadelerini elde edin.



ŞEKİL P4.2

4.3

Şekil 4.26'da gösterilen devre için (Bölüm 4.11),

- (a) Giriş değişkenleri cinsinden dört çıkış için Boolean fonksiyonlarını yazın.

(*) Eğer devre bir doğruluk tablosunda tanımlanmışsa, ta

bloda kaç satır ve sütun olur?

Üç girişi ve bir çıkışı olan bir kombinyasyonel devre tasarlayın.

- (a) * Çıktı, girişlerin ikili değeri 3'ten küçük olduğunda 1'dir. Aksi takdirde çıktı 0'dır.
- (b) Çıktı, girişlerin ikili değeri çift bir sayı olduğunda 1'dir.

Üç girişi, x, y ve z olan ve üç çıkışı, A, B ve C olan bir kombinasyonel devre tasarlayın. İkili girdi 0, 1, 2 veya 3 olduğunda, ikili çıkış girdi değerinden bir fazladır. İkili girdi 4, 5, 6 veya 7 olduğunda, ikili çıkış girdi değerinden iki eksiktir.

Bir çoğunluk devresi, giriş değişkenlerinin 0'lardan daha fazla 1'e sahip olması durumunda çıkışı 1 olan bir kombinasyonel devredir. Aksi takdirde çıkış 0'dır.

(a) * Devrenin doğruluk tablosunu, Boolean denklemini ve bir mantık diyagramını bulup 3

girişli bir çoğunluk devresi tasarlayın.

(b) Devrenin bir Verilog kapı düzeyinde modelini yazın ve doğrulayın.

Dört bitlik bir Gray kodunu (Tablo 1.6) dört bitlik ikili sayıya dönüştüren bir kombinasyonel devre tasarlayın.

(a) * XOR kapıları ile devreyi uygulayın.

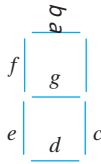
(b) Bir durum ifadesi kullanarak, devrenin Verilog modelini yazın ve doğrulayın.

Bir kod dönüştürücü tasarlayın, bu dönüştürücü bir ondalık rakamı

(a) * 8, 4, -2, -1 kodunu BCD'ye dönüştürür (bkz. Tablo 1.5). (HDL—bkz. Problem 4.50.)

(b) 8, 4, -2, -1 kodunu Gri koda dönüştürür.

Bir ABCD-yedi-segment kod çözücü, BCD'deki bir ondalık rakamı, bir göstergede segmentlerin seçimi için uygun bir koda dönüştüren kombinasyonel bir devredir. Kod çözücünün yedi çıkışı (a, b, c, d, e, f, g) ekrandaki ilgili segmentleri seçer, bu şekilde gösterilir Şekil. P4.9(a). Ondalık rakamı temsil etmek için seçilen sayısal ekrana Şekil. P4.9(b). Bir doğruluk tablosu ve Karnaugh haritaları kullanarak, BC D-yedi-segment kod çözücüyü minimum sayıda gate kullanarak tasarlayın. Altı geçersiz kombinasyon boş bir ekran göstermelidir. (HDL—bkz. Problem 4.51.)



(a) Segment tanımı



(b) Görüntüleme için sayısal tanım

ŞEKİL P4.9

4.10

* Dört bitlik bir kombinasyon devresi 2'nin tamamlayıcısını tasarlayın. (Çıkış, giriş ikili sayısının 2'nin tamamlayıcısını üretir.) Devrenin özel-VE kapıları ile inşa edilebileceğini gösterin. Beş bitlik bir 2'nin tamamlayıcısı için çıkış fonksiyonlarının ne olacağını tahmin edebilir misiniz?

Dört yarım-toplayıcı kullanarak (HDL—bkz. Problem

4.52), (a) Bir tam-çıkarcı devresi artırıcı tasarlayın. (Dört bitlik bir ikili sayıya bir ekleyen bir devre.)

(b) Dört bitlik bir kombinasyon azaltıcı tasarlayın (dört bit

ilk bir ikili sayıdan 1 çıkaran bir devre).

Girişleri x ve y olan ve çıkışları $Diff$ ve B_{out} olan bir yarım-çıkarcı devresi tasarlayın. Devre, $x - y$ bitlerini çıkarır ve farkı D ve borcu B_{out} içine yerleştirir.

(a) Üç girişi x, y, B_{in} ve iki çıkışı $Diff$ ve B_{out} olan bir tam-çıkarcı devresi tasarlayın. Devre, $x - y - B_{in}$ çıkarır; burada B_{in} giriş borcu, B_{out} çıkış borcu ve $Diff$ farktır.

4.13

* Şekil 4.13'teki toplayıcı-çıkarıcı devresinin mod girişi M ve veri girişleri A ve B için aşağıdaki değerleri vardır.

	M	A	B
(a)	0	0111	0110
(b)	0	1000	1001
(c)	1	1100	1000
(d)	1	0101	1010
(e)	1	0000	0001

Her durumda, dört SUM çıkışının, taşımanın C ve taşıma V değerlerini belirleyin. (HDL—bkz. Problemler 4.37 ve 4.40.)

4.14

* Özel-VE kapısının 10 ns'lik bir yayılma gecikmesine ve VE veya VEYA kapılarının 5 ns'lik bir yayılma gecikmesine sahip olduğunu varsayın. Şekil 4.12'deki dört bitli toplayıcının toplam yayılma gecikme süresi nedir?

Şekil 4.12'deki önceden taşıma üretici için gösterilen çıkış taşıması C_4 için iki seviyeli Boolean ifadesini türetin.

Taşıma yayılma ve taşıma üretimini tanımlayın.

$$= A + B_i$$

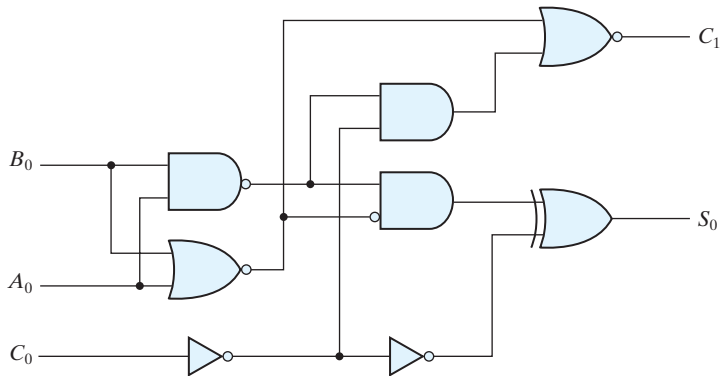
$$G_i = A_i B_i$$

Bir tam toplayıcının çıkış taşıması ve çıkış toplamının nasıl olduğunu gösterin.

$$C_{i+1} = (C_i' G_i' + P_i)'$$

$$S_i = (P_i G_i') \oplus C_i$$

IC tipi 74283'te uygulanan dört bitli paralel toplayıcının birinci aşamasının mantık diyagramı Şekil P4.16'da gösterilmiştir. P_i ve G_i terminallerini tanımlayın ve devrenin bir tam toplayıcı devresi olduğunu gösterin.



ŞEKİL P4.16

Paralel toplayıcının ilk aşaması

Tam toplayıcı devresindeki çıkış taşıma değerinin AND-OR-

INVERT biçiminde ifade edilebileceğini gösterin

$$C_{i+1} = G_i + P_i C_i = (G_i' P_i' + G_i' C_i')'$$

IC tipi 74182, AND-OR-INVERT kapıları ile taşımaları üreten bir ileri taşıma üretici devre esidir (bkz. Bölüm 3.8). Devre, giriş terminallerinin G 'lerin, P 'lerin ve C_1 'in tamamlayıcılarını içerdiğini varsayar. Bu IC'deki ileri taşımalar için Boolean fonksiyonlarını türetin C_2, C_3 ve C_4 . (İpucu: Taşımaları C_i' cinsinden türetmek için denklem-değiştirme yöntemini kullanın)

4.18 Bir kombinasyonel devre tasarlayın, bu devre bir

(a) * BCD basamağının 9'luk tamamlayıcısını üretir. (HDL—bkz. Problem 4.54(a).)

(b) Gray-kod basamağı. (HDL—bkz. Problem 4.54(b).)

Bir BCD toplama-çıkarma devresi oluşturun. BCD toplayıcısını Fig. 4.14'ten ve 9'luk tamamlayıcıyı problem 4.18'den kullanın. Bileşenler için blok diyagramları kullanın. (HDL—bkz. Problem 4.55.)

4.20 İki işaretsiz dört bitlik sayıyı çarpan bir ikili çarpan için,

(a) AND kapıları ve ikili toplayıcılar kullanarak (bkz. Fig. 4.16), devreyi tasarlayın.

(b) Devrenin bir Verilog veri akışı modelini yazın ve doğrulayın.

4.21 İki 4 bitlik sayıyı karşılaştırarak eşit olup olmadıklarını kontrol eden bir kombinasyonel devre tasarlayın. Döngü çıkışı, iki sayı eşitse 1, aksi takdirde 0'dır.

4.22 * Kullanılmayan kod kombinasyonlarını don't-care koşulları olarak kullanarak bir fazla-3'ten ikiliye çevirici tasarlayın. (HDL—bkz. Problem 4.42.)

4.23 (a) Sadece NOR kapıları ve (b) Sadece NAND kapıları kullanarak 2'den 4'e hat çeviricinin mantık diyagramını çizin. Bir etkinleştirme girişi ekleyin. (HDL—bkz. Problemler 4.36, 4.45.)

4.24 Kullanılmayan BCD kod kombinasyonlarını don't-care koşulları olarak kullanarak bir BCD'den ondalık çevirici tasarlayın.

4.25 5'ten 32'ye hat çözücü oluşturun, dört 3'ten 8'e hat çözücü ve bir 2'den 4'e hat çözücü ile birlikte. Bileşenler için blok diyagramları kullanın. (HDL—bkz. Problem 4.63.)

4.26 4'ten 16'ya hat çözücü oluşturun, beş 2'den 4'e hat çözücü ile birlikte. (HDL—bkz. Problem 4.64.)

Bir kombinasyon devresi aşağıdaki üç Boole fonksiyonu ile tanımlanmıştır:

$$F_1(A, B, C) = \Sigma(1, 4, 6)$$

$$F_2(A, B, C) = \Sigma(3, 5)$$

$$F_3(A, B, C) = \Sigma(2, 4, 6, 7)$$

Devreyi NAND kapıları ile inşa edilmiş bir çözücü ile uygulayın (Şekil 4.19'a benzer) ve çözücü çıkışlarına bağlı NAND veya AND kapıları kullanın. Çözücü için bir blok diyagramı kullanın. Dış kapılardaki giriş sayısını en aza indirin.

4.28 Bir çözücü ve dış kapılar kullanarak, aşağıdaki üç Boole fonksiyonu ile tanımlanan kombinasyon devresini tasarlayın:

$$(a) F_1 = x'y'z' + xz$$

$$F_2 = xy'z' + x'y$$

$$F_3 = x'y'z' + xy$$

$$(b) F_1 = (y' + x)z$$

$$F_2 = y'z' + x'y + yz'$$

$$F_3 = (x + y)z$$

4.29

* Dört girişli bir öncelik kodlayıcı tasarlayın, girişler Tablo 4.8'deki gibi olsun, ancak giriş D_0 en yüksek önceliğe ve giriş D_3 en düşük önceliğe sahip olsun.

Bir sekizli-binary öncelik kodlayıcısının doğruluk tablosunu belirtin. En az bir girişin mevcut olduğunu göstermek için bir çıkış V sağlayın. En yüksek alt indeks numarasına sahip giriş en yüksek önceliğe sahiptir. Girişler D_2 ve D_6 aynı anda 1 olduğunda dört çıkışın değeri ne olacaktır? (HDL—bkz. Problem 4.65.)

16×1 çoklayıcısını iki 8×1 ve bir 2×1 çoklayıcı ile oluşturun. Blok diyagramları kullanın. (HDL—bkz. Problem 4.67.)

Aşağıdaki Boolean fonksiyonunu bir çoklayıcı ile uygulayın (HDL—bkz. Problem 4.46):

$$F(A, B, C, D) = (0, 2, 5, 8, 10, 14)$$

$$F(A, B, C, D) = \Pi(2, 6, 11)$$

4.33

İki 4×1 çoklayıcı ile bir tam toplayıcı uygulayın.

8×1 çoklayıcının girişleri A, B , ve C seçim girişlerine S_2, S_1 , ve S_0 bağlıdır. Veri girişleri I_0 ile I_7 aşağıdaki gibidir:

$$\textcircled{a} \quad I_1 = I_2 = I_7 = 0; I_3 = I_5 = 1; I_0 = I_4 = D; \text{ ve } I_6 = D'.$$

$$I_1 = I_2 = 0; I_3 = I_7 = 1; I_4 = I_5 = D; \text{ ve } I_0 = I_6 = D'.$$

Çoklayıcının uyguladığı Boolean fonksiyonunu belirleyin.

Aşağıdaki Boolean fonksiyonunu 4×1 çoklayıcı ve harici kapılarla uygulayın.

$$\textcircled{a} \quad F_1(A, B, C, D) = (1, 3, 4, 11, 12, 13, 14, 15)$$

$$F_2(A, B, C, D) = (1, 2, 5, 7, 8, 10, 11, 13, 15)$$

Girişleri A ve B seçim hatlarına bağlayın. Dört veri hatları için giriş gereksinimleri, C ve D değişkenlerinin bir fonksiyonu olacaktır. Bu değerler, F 'yi C ve D 'nin bir fonksiyonu olarak ifa de ederek elde edilir. $AB = 00, 01, 10$ ve 11 durumları için dört durumda. Bu fonksiyonlar, harici kapılarla uygulanmak zorunda kalabilir. (HDL—bkz. Problem 4.47.)

HDL kapı düzeyinde öncelik kodlayıcı devresinin tanımını yazın. Fig. 4.23 . (HDL—bkz. Problem 4.45.)

İşaretsiz ikili sayılar için dört bitlik toplama-çıkarma işleminin HDL kapı düzeyinde hiyerarşik tanımını yazın. Devre, Fig. 4.13 ile benzer ancak çıkış V yoktur. HDL Örneği 4.2'de tanımlanan dört bitlik tam toplayıcıyı kullanabilirsiniz. (HDL—bkz. Problemler 4.13 ve 4.40.)

HDL veri akışı tanımını, bir etkinleştirme ile dört katmanlı 2'ye 1'lik çoklayıcı için yazın (bkz. Şekil 4.26).

4.39

* Altı bitlik bir çıkışa sahip dört bitlik bir karşılaştırıcının HDL davranışsal tanımını yazın $Y[5:0]$ Bit 5 of Y "eşittir" için, bit 4 "eşit değildir" için, bit 3 "büyüktür" için, bit 2 "küçüktür" için, bit 1 "büyük veya eşit" için ve bit 0 "küçük veya eşit" için kullanılır.

Koşullu operatörü (?) kullanarak, işaretsiz sayılar için dört bitlik bir toplama-çıkarma HDL veri akışı tanımını yazın. (Bkz. Problemler 4.13 ve 4.37.)

Her zaman ifadesi kullanarak problem 4.40'ı tekrarlayın.

- (a) Şekil 4.4'te gösterilen BCD'den fazla-3 dönüştürücü devresinin HDL kapı düzeyinde tanımını yazın (bkz. Problem 4.22).
- (b) Şekil 4.3'te listelenen Boolean ifadelerini kullanarak BCD'den fazla-3 dönüştürücünün veri akışı tanımını yazın.
- (c) BCD'den fazla-3 dönüştürücüsünün HDL davranışsal tanımını yazın.
- (d) BCD'den fazla-3 dönüştürücü devresini simüle etmek ve test etmek için bir test tezgahı yazın gerçek tabloyu doğrulamak amacıyla. Üç devreyi de kontrol edin.

Aşağıdaki HDL tanımıyla belirtilen devrenin işlevini açıklayın:

```
module Prob4_43 (A, B, S, E, Q);
  input [1:0] A, B;
  input SE;
  output [1:0] Q;
  assign Q = E ? (S ? A : B) : 'bz;
endmodule
```

Bir case ifadesi kullanarak, sekiz bitlik bir aritmetik-

lojik birimin (ALU) HDL davranışsal tanımını yazın. Devre, üç bitlik bir seçim veri yolu (Sel), on altı bitlik giriş veri yolları (A[15:0] ve B[15:0]), sekiz bitlik bir çıkış veri yolu (y[15:0]) içerir ve aşağıda listelenen aritmetik ve mantık işlemlerini gerçekleştirir.

Sel Operasyon Açıklaması

000 _{sel}	= 8b0	
001 _{sel}	= A & B Bitwise VE	Bitwise VE
010 _{sel}	= A B Bitwise VEYA	Bitwise VEYA
011 _{sel}	= A ^ B Bitwise ekslüzif VE	Bitwise ekslüzif VE
100 _{sel}	= ~A Bitwise complement	Bitwise complement
101 _{sel}	= A - B Çıkar	Çıkar
110 _{sel}	= A + B Topla	Topla (A ve B'nin işaretersiz olduğunu varsayalım)
111 _{sel}	= 8hFF	

Bir dört girişli öncelik kodlayıcısının HDL davranışsal tanımını yazın. D girişleri için dört bitlik bir ve ktör kullanın ve if-else ifadeleri ile bir always bloğu oluşturun. Giriş D[3]'ün en yüksek önceliğe sahip olduğunu varsayın (bkz. Problem 4.36).

Boolean fonksiyonu ile tanımlanan mantık devresinin Verilog veri akışı tanımını yazın, Problem 4.32'deki gibi.

Boolean fonksiyonu ile tanımlanan mantık devresinin Verilog veri akışı tanımını yazın, Problem 4.35'teki gibi.

Problem 4.44'te belirtilen sekiz bitlik ALU'yu geliştirin ve değiştirin, böylece bir enable girişi ile kontrol edilen üç durumlu bir çıkışa sahip olsun. Bir test tezgahı yazın ve devreyi simüle edin.

Şekil P4.1'de gösterilen devre için, (a)

) Devrenin kapı düzeyinde bir HDL modelini yazın ve doğrulayın.

(b) Sonuçlarınızı Problem 4.1 için elde edilenlerle karşılaştırın.

Bir durum ifadesi kullanarak,

(a) * Problem 4.8(a)'de tanımlanan 8, 4, -2, -1'den BCD kod dönüştürücüsünün davranışsal modelini geliştirin ve simüle edin.

(b) Problem 4.8(b)'de tanımlanan 8, 4, -2, -1'den Gray kod dönüştürücüsünün davranışsal modelini geliştirin ve simüle edin.

Problem 4.9'da tanımlanan ABCD'den yedi segmentli dekodere ait davranışsal modelini geliştirin ve simüle edin.

Sürekli bir atama kullanarak,

(a) Problem 4.11(a)'da tanımlanan dört bitlik artırıcıyı geliştirin ve simüle edin.

(b) Problem 4.11(b)'de tanımlanan dört bitlik azaltıcıyı geliştirin ve simüle edin.

Şekil 4.14'te gösterilen ondalık toplayıcının yapısal modelini geliştirin ve simüle edin.

9'un tamamlayıcısını üreten bir devrenin davranışsal modelini geliştirin ve simüle edin.

(a) bir BCD rakamı (bkz. Problem 4.18(a)).

(b) bir Gray kodu rakamı (bkz. Problem 4.18(b).)

Problem 4.19'da tanımlanan BCD toplayıcı-çıkartıcısının hiyerarşik modelini oluşturun. BCD toplayıcı ve 9'un tamamlayıcısı, ayrı modüllerde davranışsal modeller olarak tanımlanacak ve üst düzey bir modülde örnekleneyecektir.

4.56 * İki 4 bitlik sayıyı karşılaştıran ve bit desenlerinin eşleşip eşleşmediğini kontrol eden sürekli bir atama ifadesi yazın. Atamanın yapıldığı değişken, sayılar eşleşirse 1, aksi takdirde 0 olmalıdır.

4.57 * Problem 4.29'da tanımlanan dört bitlik öncelik kodlayıcısının davranışsal modelini geliştirin ve doğrulayın.

32 bitlik çıkışı, 32 bitlik girişini üç pozisyon sağa kaydırarak ve boş pozisyonları kaydırma gerçekleşmeden önce MSN'de bulunan bit ile doldurarak oluşan bir devrenin Verilog modelini yazın (aritmetik sağ kaydırma). 32 bitlik çıkışı, 32 bitlik girişini üç pozisyon sola kaydırarak ve boş pozisyonları 0 ile doldurarak oluşan bir devrenin Verilog modelini yazın (mantıksal sola kaydırma).

BCD kodunun kullanılmayan kombinasyonlarını don't-care koşulları olarak kullanarak bir BCD'den ondalık kod çözücü Verilog modeli yazın. (bkz. Problem 4.24).

IEEE 1364-2001 standardının port sözdizimini kullanarak, Şekil 3.34'te gösterilen dört bitlik çift parite kontrol cihazının kapı düzeyinde bir modelini yazın ve doğrulayın.

Sürekli atama ifadeleri ve IEEE 1364-2001 standardının port sözdizimini kullanarak, Şekil 3.34'te gösterilen dört bitlik çift parite kontrol cihazının kapı düzeyinde bir modelini yazın ve doğrulayın.

Problem 4.25'te tanımlanan devrenin kapı düzeyinde hiyerarşik bir modelini yazın ve doğrulayın.

Problem 4.26'da tanımlanan devrenin kapı düzeyinde hiyerarşik bir modelini yazın ve doğrulayın.

Problem 4.30'da tanımlanan sekizli-binary devresinin Verilog modelini yazın ve doğrulayın.

Problem 4.31'de tanımlanan çoklayıcının hiyerarşik kapı düzeyinde modelini yazın.

REFERANSLAR

1. B HASKER , J. 1997. *Verilog HDL Primer*. Allentown, PA: Star Galaxy Press.
2. B HASKER , J. 1998. *Verilog HDL Synthesis*. Allentown, PA: Star Galaxy Press.
3. C ILETTI , M. D. 1999. *Modeling, Synthesis, and Rapid Prototyping with Verilog HDL*. Upper Saddle River, NJ: Prentice Hall.
4. D IETMEYER , D. L. 1988. *Dijital Sistemlerin Mantık Tasarımı*, 3. baskı. Boston: Allyn Bacon.

5. G AJSKI , D. D. 1997. *Dijital Tasarımın İlkeleri*. Upper Saddle River, NJ: Prentice Hall.
6. H AYES , J. P. 1999. *Dijital Mantık Tasarımına Giriş*. Reading, MA: Addison-Wesley.
7. K ATZ , R. H. 2005. *Şağdaş Mantık Tasarımı*. Upper Saddle River, NJ: Pearson Prentice Hall.
8. M ano , M. M. ve C. R. K ime . 2007. *Mantık ve Bilgisayar Tasarım Temelleri*, 4. baskı. Upper Saddle River, NJ: Prentice Hall.
9. N elson , V. P., H. T. N aple , J. D. I rwin , ve B. D. C arroll . 1995. *Dijital Mantık Devresi Analizi ve Tasarımı*. Englewood Cliffs, NJ: Prentice Hall.
10. P ALNITKAR , S. 1996. *Verilog HDL: Dijital Tasarım ve Sentez için Bir Rehber*. Mountain View, CA: SunSoft Press (bir Prentice Hall başlığı).
11. R OTH , C. H. 2002. *Mantık Tasarımının Temelleri*, 6. baskı. St. Paul, MN: West.
12. T homas , D. E. ve P. R. M oorby . 2002. *Verilog Donanım Tanım Dili*, 5. baskı. Boston: Kluwer Akademik Yayıncılar.
13. W AKERLY , J. F. 2000. *Dijital Tasarım: İlkeler ve Uygulamalar*, 4. baskı. Upper Saddle River, NJ: Prentice Hall.

WEB ARAMA KONULARI

Boolean denklemi

Kombinasyonel mantık

Doğruluk tablosu

Özel-VEYA

Karşılaştırmacı

Çoklayıcı

Kodlayıcı

Öncelik kodlayıcı

Üç durumlu inverter

Üç durumlu tampon

Bölüm 5

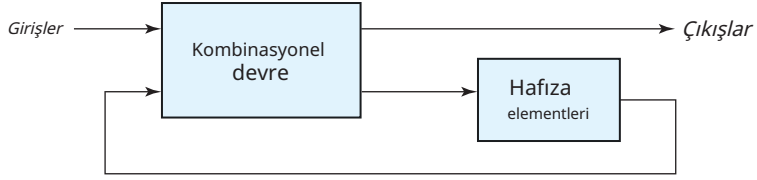
Senkranize Sıralı Mantık

5.1 GİRİŞ

Taşıyabilir cihazlar, cep telefonları, navigasyon alıcıları, kişisel bilgisayarlar, dijital kameralar, kişisel medya oynatıcıları ve neredeyse tüm elektronik tüketici ürünleri, ikili formatta temsil edilen bilgileri gönderebilme, alabilme, depolayabilme, geri alabilme ve işleyebilme yeteneğine sahiptir. Bu cihazları \ destekleyen ve mümkün kılan teknoloji, bilgiyi depolayabilen elektronik bileşenlere, yani belleğe kritik şekilde bağımlıdır. Bu bölüm, bu cihazların çalışma ve kontrolünü incelemekte ve onlarla etkileşimde bulunduğunuzda bu cihazlarda neler olduğunu daha iyi anlamanızı sağlamaktadır. Şu ana kadar ele alınan dijital circuitler kombinasyonel olmuştur—çıkışları yalnızca ve hemen girişlerine bağlıdır—bellekleri yoktur, yani girişlerinin geçmiş değerlerine bağımlılıkları yoktur. Ancak, sıralı devreler, depolama elemanları olarak işlev görür ve belleğe sahiptir. Bilgiyi depolayabilir, koruyabilir ve ihtiyaç duyulduğunda daha sonra geri alabilirler. İşleyişimiz, sıralı mantığı kombinasyonel mantıktan ayıracaktır.

5.2 SIRALI DEVRELER

Sıralı bir devrenin blok diyagramı Şekil 5.1'de gösterilmiştir. Bu, geri besleme yolu oluşturmak için depolama elemanlarının bağlandığı bir kombinasyon devresinden oluşur. Depolama elemanları, ikili bilgiyi depolayabilen cihazlardır. Bu elemanlarda herhangi bir anda depolanan ikili bilgi, o andaki sıralı devrenin *durumu*'nu tanımlar. Sıralı devre, depolama elemanlarının mevcut durumu ile birlikte dış girişlerden ikili bilgi alır ve bu, çıkışların ikili değerini belirler. Bu dış girişler ayrıca depolama elemanlarındaki durumu değiştirme koşulunu da belirler.

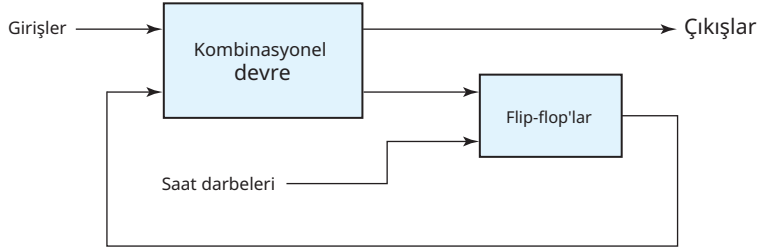


ŞEKİL 5.1
Sıralı devrelerin blok diyagramı

Depolama elemanlarındaki durum değişikliği için koşulları belirler. Blok diyagramı, sıralı bir devredeki çıkışların yalnızca girişlerin değil, aynı zamanda depolama elemanlarının mevcut durumunun bir fonksiyonu olduğunu göstermektedir. Depolama elemanlarının bir sonraki durumu da dış girişlerin ve mevcut durumun bir fonksiyonudur. Böylece, bir sıralı devre, girişlerin, çıkışların ve iç durumların zaman sırasıyla tanımlanır. Buna karşılık, kombinasyon mantığının çıkışları yalnızca girişlerin mevcut değerlerine bağlıdır.

İki ana tür ardışık devre vardır ve bunların sınıflandırılması, sinyallerinin zamanlamasına bağlıdır. Bir senkron ardışık devre, davranışı, sinyallerinin belirli anlarda bilgilere dayanarak tanımlanabilen bir sistemdir. Asenkron ardışık devrenin davranışı ise, herhangi bir anda giriş sinyallerine ve girişlerin değişim sırasına bağlıdır. Asenkron ardışık devrelerde yaygın olarak kullanılan depolama elemanları zaman gecikme cihazlarıdır. Bir zaman gecikme cihazının depolama kapasitesi, sinyalin cihazdan geçmesi için gereken zamana bağlı olarak değişir. Pratikte, mantık kapılarının içsel yayılma gecikmesi, gerekli gecikmeyi sağlamak için yeterli süreye sahiptir, bu nedenle gerçek gecikme birimleri gerekli olmayabilir. Kapı tipi asenkron sistemlerde, depolama elemanları, yayılma gecikmesi gereken depolamayı sağlayan mantık kapılarından oluşur. Böylece, bir asenkron ardışık devre, geri beslemeli bir kombinasyon devresi olarak kabul edilebilir. Mantık kapıları arasındaki geri besleme nedeniyle, bir asenkron ardışık devre zaman zaman kararsız hale gelebilir. Kararsızlık sorunu tasarımcı üzerinde birçok zorluk yaratır. Bu devreler bu metinde ele alınmayacaktır.

Senkron sekansal devre, depolama elemanlarını yalnızca belirli zaman anlarında etkileyen sinyalleri kullanır. Senkronizasyon, bir saat sinyali sağlayan bir zamanlama cihazı olan *saat üretici*, *periyodik bir saat sinyali şeklinde bir saat pulsu dizisi sağlar*. *Saat sinyali genellikle saat ve clk tanımlayıcılarıyla gösterilir. Saat pulsuları, depolama elemanlarının yalnızca her pulsun gelmesiyle etkilenecek şekilde sistem genelinde dağıtılır. Pratikte, saat pulsuları, devre içinde hesaplama faaliyetinin ne zaman gerçekleşeceğini belirler ve diğer sinyaller (dış girişler ve diğerleri) ne değişikliklerin depolama elemanlarını ve çıkışları etkileyeceğini belirler. Örneğin, iki ikili sayıyı toplamak ve depolamak için bir devre, sayıların değerlerinden toplamalarını hesaplayacak ve bir saat pulsunun meydana gelmesiyle toplamı depolayacaktır. Depolama elemanlarını kontrol etmek için saat pulsularını kullanan senkron sekansal devreler, saatli sekansal devreler olarak adlandırılır ve pratikte en sık karşılaşılan türdür. Bu devrelere senkron devreler denir çünkü devre içindeki faaliyet ve depolanan değerlerin güncellenmesi,*



(a) Blok diyagramı



(b) Saat darbelerinin zaman diyagramı

ŞEKİL 5.2**Senkranize saatli ardışık devre**

Saat darbeleri. Senkron devrelerin tasarımı mümkündür çünkü nadiren stabilite sorunları gösterirler ve zamanlamaları bağımsız aralık

adımlara kolayca bölünebilir, her biri ayrı olarak değerlendirilebilir.

Saatli sıralı devrelerde kullanılan depolama elemanlarına *flip-flop* denir. Bir flip-flop, bir bit bilgiyi depolayabilen ikili bir depolama cihazıdır. Kararlı bir durumda, bir flip-flop'un çıkışı ya 0 ya da 1'dir. Bir sıralı devre, gerekli olduğu kadar bit depolamak için birçok flip-flop kullanabilir. Senkron saatli sıralı devrenin blok diyagramı Şekil 5.2'de gösterilmiştir. *Çıkışlar*, devreye girişlerin veya flip-flop'larda depolanan değerlerin (veya her ikisinin) kombinasyonel mantık fonksiyonu ile oluşturulur. Saat darbesi gerçekleştiğinde flip-flop'ta depolanan değer, devreye girişlerin veya flip-flop'ta şu anda depolanan değerlerin (veya her ikisinin) etkisiyle belirlenir. Yeni değer, saat sinyalinin bir darbesi gerçekleştiğinde depolanır (yani, flip-flop güncellenir).

Saat darbesinin meydana gelmesinden önce, flip-flop'un bir sonraki değerini oluşturan kombinasyonel mantığın kararlı bir değere ulaşmış olması gerekir. Bu nedenle, kombinasyonel mantık devrelerinin çalışma hızı kritik öneme sahiptir. Saat (senkronizasyon) darbeleri, Şekil 5.2'de gösterildiği gibi düzenli bir aralıkta geldiğinde, kombinasyonel mantık, flip-flop'un durumundaki bir değişikliğe, bir sonraki darbe gelmeden önce güncellenmek için zamanında yanıt vermelidir. Yayılma gecikmeleri, devrenin doğru çalışmasını sağlayacak saat darbeleri arasındaki minimum aralığı belirlemede önemli bir rol oynar. Flip-flop'ların durumundaki bir değişiklik yalnızca bir saat darbesi geçişi ile başlatılır—örneğin, saat sinyallerinin değeri 0'dan 1'e değiştiğinde. Bir saat darbesi aktif olmadığına, flip-flop'ta saklanan değer ile flip-flop'un girişine oluşturulan değer arasındaki geri besleme döngüsü etkili bir şekilde kesilir çünkü flip-flop çıkışları, girişlerini yönlendiren kombinasyonel devrenin çıkışları değer değişirse bile değişemez. Bu nedenle, bir durumdan diğerine geçiş yalnızca saat darbeleri tarafından belirlenen önceden belirlenmiş aralıklarda gerçekleşir.

5.3 DEPOLAMA ELEMANLARI: LATCH'LER

Dijital bir devrede bir depolama elemanı, bir giriş sinyali tarafından durum değiştirmesi yönlenirile kadar (devreye güç verildiği sürece) sonsuza kadar ikili bir durumu sürdürebilir. Farklı depolama elemanları türleri arasındaki temel farklar, sahip oldukları giriş sayısı ve girişlerin ikili durumu nasıl etkilediği ile ilgilidir. Sinyal seviyeleri ile çalışan depolama elemanlarına latch'ler denir; saat geçişi ile kontrol edilenler ise flip-flop'lardır. Latch'ler seviye duyarlı cihazlar olarak tanımlanır; flip-flop'lar ise kenar duyarlı cihazlardır. İki tür depolama elemanı birbiriyle ilişkilidir çünkü latch'ler, tüm flip-flop'ların inşa edildiği temel devrelerdir. Latch'ler ikili bilgiyi depolamak ve asenkron ardışık devrelerin tasarımı için yararlı olsa da, senkron ardışık devrelerde depolama elemanları olarak kullanmak pratik değildir. Ancak, flip-flop'ların yapı taşları oldukları için, bir sonraki bölümde flip-flop'ları ele almadan önce latch'lerde kullanılan temel depolama mekanizmasını inceleyeceğiz.

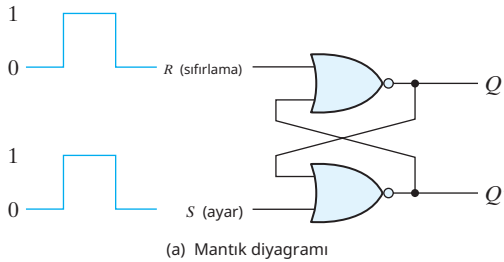
SR Kilit

SR kilidi, iki çapraz bağlı NOR kapısı veya iki çapraz bağlı NAND

gapısı ile oluşturulmuş bir devredir ve set için S ve reset için R olarak etiketlenmiş iki girişi vardır. İki cross-coupled NOR kapısı ile inşa edilen SR kilidi Şekil 5.3'te gösterilmiştir. Kilidin iki yararlı durumu vardır. Çıktı $Q=1$ ve $Q=0$ olduğunda, kilidin *set durumu*'nda olduğu söylenir. $Q=0$ ve $Q=1$ olduğunda, *reset durumu*'ndadır. Çıktılar Q ve Q' genellikle birbirinin tamamlayıcısıdır. Ancak, her iki giriş aynı anda 1 olduğunda, her iki çıkışın da 0'a eşit olduğu (birbirini tamamlamayıktan ziyade) bir durum ortaya çıkar. Eğer her iki giriş de aynı anda 0'a geçerse, cihaz öngörülemez veya tanımsız bir duruma veya meta-stabil bir duruma girecektir. Sonuç olarak, pratik uygulamalarda her iki girişi 1 yapmak yasaktır.

Normal koşullar altında, kilidin her iki girişi 0'da kalır, durum değiştirilmesi gerekmedikçe. S girişine anlık bir 1 uygulanması, kilidin set durumuna geçmesine neden olur. Diğer değişimler gerçekleşmeden önce S girişinin 0'a geri dönmesi gerekir, böylece yasaklı giriş durumundan kaynaklanan tanımsız bir sonraki durumun ortaya çıkması önlenir.

Şekil 5.3 (b)'deki fonksiyon tablosunda gösterildiği gibi, iki giriş koşulu devrenin

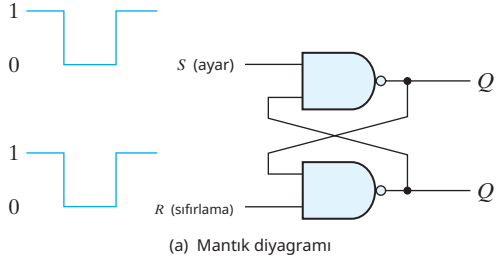


S	R	Q	Q'
1	0	1	0
0	0	0	1
0	1	0	1
1	1	0	0

(b) Fonksiyon tablosu

ŞEKİL 5.3

NOR kapıları ile SR latch



ŞEKİL 5.4

NAND kapıları ile SR kilidi

set durumunu. İlk koşul ($S=1, R=0$) girişin alması gereken eylemdir S devreyi set durumuna getirmek için. Aktif girişi S kaldırmak devreyi aynı durumda bırakır. Her iki giriş 0'a döndüğünde,

R girişine 1 uygulayarak sıfırlama durumuna geçmek mümkündür. 1 daha sonra R 'den kaldırılabilir, bu durumda circuit sıfırlama durumunda kalır. Böylece, her iki giriş S ve R 0'a eşit olduğunda, latch ya set ya da sıfırlama durumunda olabilir, hangi girişin en son 1 olduğu ile belirlenir.

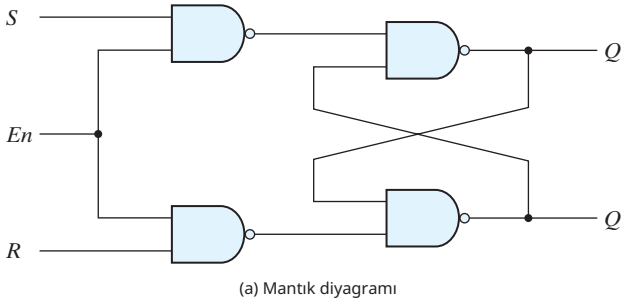
Eğer latch'in hem S hem de R girişlerine 1 uygulanırsa, her iki çıkış 0'a gider. Bu eylem, giriş geçişlerinin hangi sırayla 0'a döndüğüne bağlı olarak tanımsız bir sonraki durumu üretir. Ayrıca, çıkışların birbirinin tamamlayıcısı olma gerekliliğini ihlal eder. Normal çalışmada, bu durum, her iki girişe aynı anda 1 uygulanmadığından emin olunarak önlenir.

İki çapraz bağlı NAND kapısından oluşan SR latch'i Şekil 5.4'te gösterilmiştir. Normalde her iki girişi 1'de çalışır, ancak latch'in durumu değiştirilmesi gerektiğinde bu durum değişir. S girişine 0 uygulanması, Q çıkışının 1'e gitmesine neden olur ve latch'i set durumuna getirir. S girişi tekrar 1'e döndüğünde, devre set durumunda kalır. Her iki giriş tekrar 1'e döndüğünde, R girişine 0 yerleştirerek latch'in durumunu değiştirmemize izin verilir. Bu işlem, devrenin reset durumuna geçmesine ve her iki giriş 1'e döndükten sonra bile orada kalma sına neden olur. NAND latch için yasak olan durum, her iki girişin aynı anda 0 olmasıdır; bu giriş kombinasyonundan kaçınılmalıdır.

NAND latch ile NOR latch'i karşılaştırırken, NAND için giriş sinyallerinin NOR latch için kullanılan değerlerin tamamlayıcısı olduğunu unutmayın. NAND latch'in durumu değiştirmek için 0 sinyali gerektirdiğinden, bazen S R latch olarak adlandırılır. Prensipler (veya bazen harflerin üzerindeki çubuklar), girişlerin devreyi etkinleştirmek için tamamlayıcı formda olması gerektiğini belirtir.

Temel SR latch'in çalışması, latch'in durumunun değiştirilebileceği zamanı belirleyen (kontrol eden) ek bir giriş sinyali sağlanarak değiştirilebilir. S ve R (veya S ve R) girişlerinin devreyi etkileyip etkilemeyeceğini belirleyerek latch'in durumunun değiştirilebileceği zaman kontrol edilir. Kontrol girişi olan bir SR latch, Şekil 5.5'te gösterilmiştir. Temel SR latch ve iki ek NAND kapısından oluşur. Kontrol girişi En , diğer iki giriş için bir etkinleştirme sinyali olarak işlev görür. NAND kapılarının çıkışları, etkinleştirme sinyali 0'da kaldığı sürece mantık-1 seviyesinde kalır.

Bu, SR latch için durgun durumdur. Etkinleştirme girişi 1'e gittiğinde, S veya R girişinden gelen bilginin latch'i etkilemesine izin verilir. Set durumu, $S = 1, R = 0$ ve $En = 1$ ile ulaşılır.



En	S	R	Q'nun bir sonraki durumu
0	X	X	Değişiklik yok
1	0	0	Değişiklik yok
1	0	1	$Q = 0$; durumu sıfırla
1	1	0	$Q = 1$; durumu ayarla
1	1	1	Belirsiz

(b) Fonksiyon tablosu

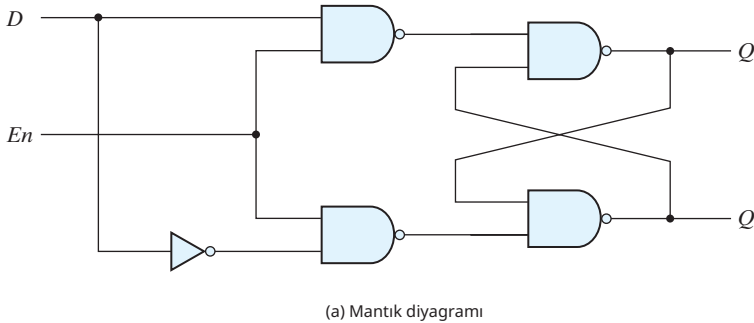
ŞEKİL 5.5**SR kilitli kontrol girişi ile**

(aktif-yüksek etkin). Sıfırlama durumuna geçmek için, girişler $S=0, R=1$, ve $En=1$. Her iki durumda da, En 0'a döndüğünde, devre mevcut durumunda kalır. Kontrol girişi, devreye 0 uygulayarak En 'yi devre dışı bırakır, böylece çıkışın durumu S ve R değerlerine bakılmaksızın değişmez. Ayrıca, $En=1$ ve hem S and R girişi 0'a eşit olduğunda, devrenin durumu değişmez. Bu koşullar, diyagramla birlikte gelen fonksiyon tablosunda listelenmiştir.

Tüm üç giriş 1'e eşit olduğunda belirsiz bir durum ortaya çıkar. Bu durum, temel SR kilitinin her iki girişine 0'lar yerleştirir ve onu tanımsız duruma getirir. Etkinleştirme girişi 0'a döndüğünde, bir sonraki durumu kesin olarak belirlemek mümkün değildir, çünkü bu, S veya R girişinin önce 0'a inip inmeyeceğine bağlıdır. Bu belirsiz durum, bu devreyi yönetmeyi zorlaştırır ve pratikte nadiren kullanılır. Yine de, SR kiliti, diğer yararlı kilitler ve flip-flop'ların ondan inşa edilmesi nedeniyle önemli bir devredir.

D Kilit (Şeffaf Kilit)

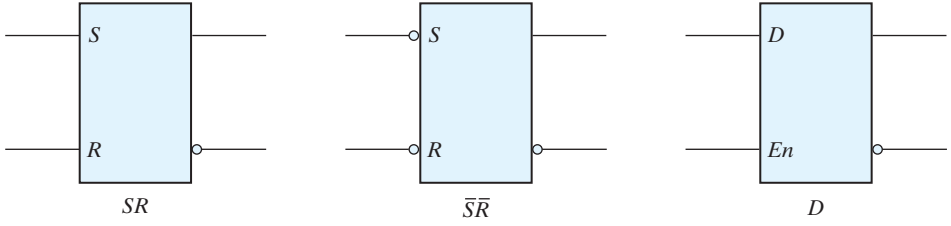
SR latch'teki belirsiz durumun istenmeyen koşulunu ortadan kaldırmanın bir yolu, S ve R girişlerinin aynı anda 1'e eşit olmamasını sağlamaktır. Bu, Şekil 5.6'da gösterilen D kilitinde yapılır. Bu kilitin yalnızca iki girişi vardır: D (veri) ve



En	D	Q'nun bir sonraki durumu
0	X	Değişiklik yok
1	0	$Q = 0$; durumu sıfırla
1	1	$Q = 1$; durumu ayarla

(b) Fonksiyon tablosu

ŞEKİL 5.6**D kilit**



ŞEKİL 5.7

Kilitler için grafik semboller

En (enable). D girişi doğrudan S girişine gider ve tamamlayıcı R girişine uygulanır. Enable girişi 0'da olduğu sürece, çapraz bağlı SR latch'inin her iki girişi de 1 seviyesindedir ve devre, D değerine bakılmaksızın durum değiştiremez. Giriş D , $En=1$ olduğunda örneklenir. Eğer $D=1$ ise, Q çıkışı 1'e gider ve devre set durumuna geçer. Eğer $D=0$ ise, çıkış Q 0'a gider ve devre sıfırlama durumuna geçer.

D kilidi, verileri iç depolama alanında tutma yeteneğinden dolayı bu ismi alır. Birim ile çevresi arasında ikili bilgilerin geçici depolama alanı olarak kullanılma için uygundur. D kilidinin veri girişinde bulunan ikili bilgi, etkinleştirme girişi onaylandığında Q çıkışına aktarılır. Çıkış, etkinleştirme girişi onaylandığı sürece veri girişindeki değişiklikleri takip eder. Bu durum, giriş D ile çıkış arasında bir yol sağlar ve bu nedenle devre genellikle bir *transparent* kilidi olarak adlandırılır. Etkinleştirme girişi sinyali devre dışı bırakıldığında, geçişin gerçekleştiği anda veri girişinde bulunan ikili bilgi, etkinleştirme girişi tekrar onaylanana kadar Q çıkışında saklanır (yani, depolanır). Dikkat edin ki, etkinleştirme girişine bir inverter yerleştirilebilir. O zaman, fiziksel devreye bağlı olarak, dış etkinleştirme sinyali 0 (aktif düşük) veya 1 (aktif yüksek) değeri olacaktır.

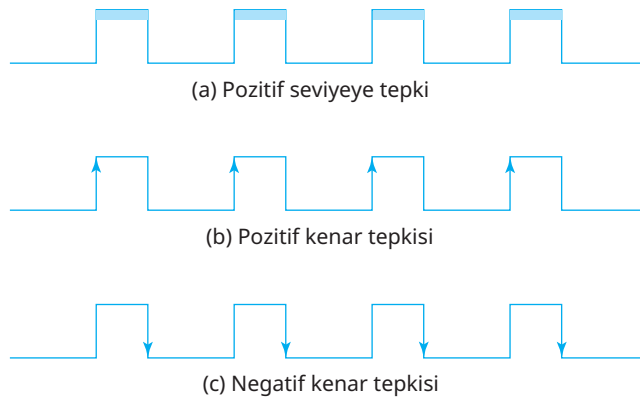
Farklı kilitlerin grafik sembolleri Şekil 5.7'de gösterilmiştir. Bir kilit, sol tarafta girişler ve sağ tarafta çıkışlar olan dikdörtgen bir blok ile tanımlanır. Bir çıkış normal çıkışı, diğeri (baloncuk tanımı ile) tamamlayıcı çıkışı belirtir. SR kilidi için grafik sembolü, blok içinde S ve R girişlerini gösterir. $NAND$ kapı kilidi durumunda, girişlere baloncuklar eklenir, bu da ayarlama ve sıfırlamanın bir mantık-0 sinyali ile gerçekleştiğini belirtir. D kilidi için grafik sembolü, blok içinde D ve En girişlerini gösterir.

5.4 DEPOLAMA ELEMANLARI: FLIP-FLOP'lar

Bir kilidin veya flip-flop'un durumu, kontrol girişindeki bir değişimle değiştirilir. Bu anlık değişime tetikleyici (trigger) denir ve neden olduğu geçiş, flip-flop'u tetiklemek olarak adlandırılır. Kontrol girişinde darbelerle D kilidi, aslında her seferinde darbe mantık-1 seviyesine çıktığında tetiklenen bir flip-flop'tur. Darbe girişi bu seviyede kaldığı sürece, veri girişindeki herhangi bir değişiklik çıkışı ve kilidin durumunu değiştirecektir.

Şekil 5.2'deki blok diyagramından görüldüğü gibi, ardışık bir devre, flip-flop'ların çıkışlarından kombinasyonel devrenin girişine bir geri besleme yoluna sahiptir. Sonuç olarak, flip-flop'ların girişleri kısmen aynı ve diğer flip-flop'ların çıkışlarından türetilir. Depolama elemanları için latch'ler kullanıldığında, ciddi bir zorluk ortaya çıkar. Latch'lerin durum geçişleri, saat darbesi mantık-1 seviyesine geçtiği anda başlar. Bir latch'in yeni durumu, darbe hala aktifken çıkışta görünür. Bu çıkış, kombinasyonel devre aracılığıyla latch'lerin girişlerine bağlanır. Eğer latch'lere uygulanan girişler, saat darbesi hala mantık-1 seviyesindeyken değişirse, latch'ler yeni değerlere yanıt verecek ve yeni bir çıkış durumu meydana gelebilir. Sonuç, öngörülemez bir durumdur, çünkü latch'lerin durumu, saat darbesi aktif seviyede kaldığı sürece sürekli değişebilir. Bu güvenilir çalışma nedeniyle, bir latch'in çıkışı, tüm latch'ler ortak bir saat kaynağı tarafından tetiklendiğinde, doğrudan veya kombinasyonel mantık aracılığıyla aynı veya başka bir latch'in girişine uygulanamaz.

Flip-flop devreleri, ortak bir saat kullanan ardışık devrelerin bir parçası oldukları arında düzgün çalışacak şekilde inşa edilmiştir. Latch ile ilgili sorun, bir saat darbesinin seviyesindeki değişikliğe yanıt vermesidir. Şekil 5.8 (a)'da gösterildiği gibi, enable girişindeki pozitif seviye yanıtı, saat darbesi mantık 1'de kalırken D girişiindeki değişiklikler sonucunda çıkışta değişikliklere izin verir. Bir flip-flop'un düzgün çalışmasının anahtarı, yalnızca bir sinyal geçişi sırasında tetiklenmesidir. Bu, latch'lerin kullanıldığı ardışık devrenin çalışmasında mevcut olan geri besleme yolunu ortadan kaldırarak gerçekleştirilebilir. Bir saat darbesi iki geçişten geçer: 0'dan 1'e ve 1'den 0'a dönüş. Şekil 5.8'de gösterildiği gibi, pozitif geçiş pozitif kenar olarak tanımlanırken, negatif geçiş negatif kenar olarak tanımlanır. Bir latch'in flip-flop oluşturmak için iki şekilde değiştirilmesi mümkündür. Bir yol, flip-flop'un çıkışını izole eden ve flip-flop'un girişi değişirken etkilenmesini önleyen özel bir yapılandırmada iki latch kullanmaktır. Başka bir yol, bir flip-flop üretmektir



ŞEKİL 5.8
Latch ve flip-flop'taki saat tepkisi

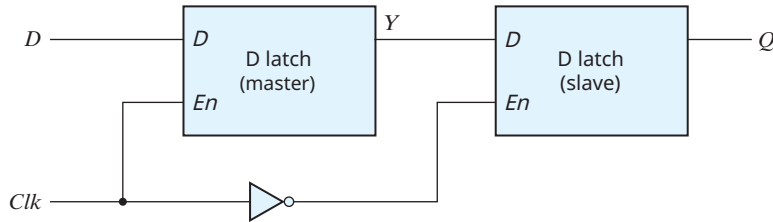
Yalnızca senkronizasyon sinyalinin (saat) bir sinyal geçişi (0'dan 1'e veya 1'den 0'a) sırasında tetiklenir ve saat darbesinin geri kalanında devre dışı bırakılır. Şimdi her iki tür flip-flop'un uygulanmasını göstereceğiz.

Kenar Tetikli D Flip-Flop

İki D latch ve bir inverter ile bir D flip-flop'un yapısı Şekil 5.9'da gösterilmiştir. İlk latch ana (master) olarak adlandırılır ve ikincisi köle (slave) olarak adlandırılır. Devre, D girişini örnekler ve yalnızca senkronizasyon veya kontrol saatinin negatif kenarında (Clk olarak belirtilmiştir) çıkışını Q değiştirir. Saat 0 olduğunda, inverterin çıkışı 1'dir. Köle latch etkinleştirilir ve çıkışı Q ana çıkış Y ile eşittir. Ana latch devre dışı bırakılır çünkü $Clk=0$. Giriş darbesi mantık-1 seviyesine geçtiğinde, dış D girişinden veriler ana latch'e aktarılır. Ancak, köle, saat 1 seviyesinde kaldığı sürece devre dışı bırakılır, çünkü etkinleştirme girişi 0'a eşittir. Girişteki herhangi bir değişiklik, ana çıkışı Y değiştirir, ancak köle çıkışını etkileyemez. Saat darbesi 0'a döndüğünde, ana cihaz devre dışı bırakılır ve D girişten izole edilir. Aynı zamanda, köle etkinleştirilir ve Y değeri flip-flop'un Q çıkışına aktarılır. Böylece, flip-flop'un çıkışındaki bir değişiklik yalnızca saat sinyalinin 1'den 0'a geçişi sırasında tetiklenebilir.

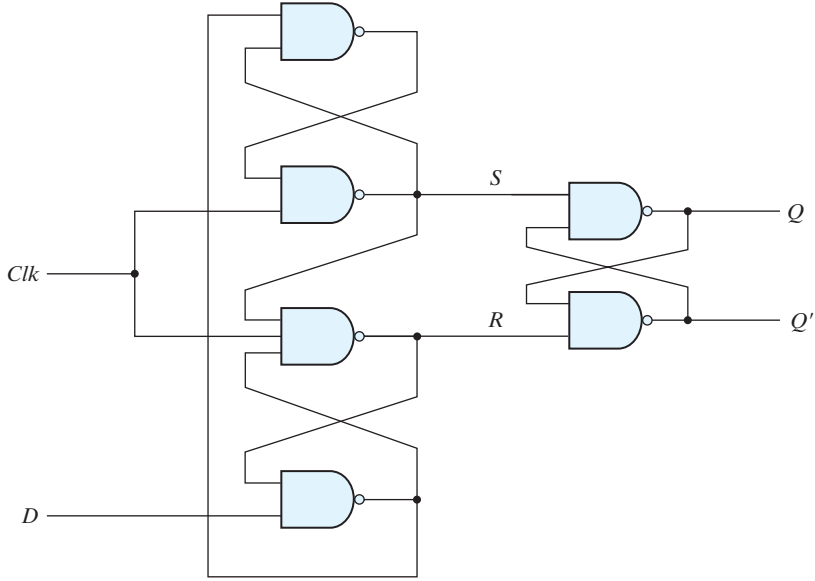
Tanımlanan ana-köle flip-flop'un davranışı, (1) çıkışın yalnızca bir kez değişebileceğini, (2) çıkıştaki bir değişikliğin saat sinyalinin negatif kenarı tarafından tetiklendiğini ve (3) değişikliğin yalnızca saat sinyalinin negatif seviyesinde gerçekleşebileceğini belirtir. Flip-flop'un çıkışında üretilen değer, negatif kenar gerçekleşmeden hemen önce ana cihazda depolanan değerdir. *Devreyi, flip-flop çıkışının saat sinyalinin pozitif kenarında değişecek şekilde tasarlamak da mümkündür. Bu, Clk terminali ile diğer inverter ve ana latch'in En girişi arasındaki ek bir invertere sahip bir flip-flop'ta gerçekleşir. Böyle bir flip-flop, negatif bir darbe ile tetiklenir, böylece saat sinyalinin negatif kenarı ana cihazı etkiler ve pozitif kenar köleyi ve çıkış terminalini etkiler.*

Bir kenar tetiklemeli D flip-flop'un başka bir yapısı, Şekil 5.10'da gösterildiği gibi üç SR latch kullanır. İki latch, dış D (veri) ve Clk (saat) girişlerine yanıt verir. Üçüncü latch $Clk=0$ olduğunda mantık-1 seviyesinde tutulur. Bu, çıkışın mevcut durumunda kalmasına neden olur. Giriş $D=0$ veya 1'e eşit olabilir. Eğer $D=0$ olduğunda $Clk=1$ olduğunda, $R=0$ olur. Bu, flip-flop'un sıfırlama durumuna geçmesine neden olur ve $Q=0$ olur. Eğer $Clk=1$ iken D girişinde bir değişiklik olursa, terminal $R=0$ 'da kalır çünkü $Q=0$ 'dır. Böylece, flip-flop'un sıfırlama durumuna geçmesine neden olur ve $Q=0$ olur. Eğer $Clk=1$ iken D girişinde bir değişiklik olursa, terminal $R=0$ 'da kalır çünkü $Q=0$ 'dır. Böylece, flip-flop'un sıfırlama durumuna geçmesine neden olur ve $Q=0$ olur.



ŞEKİL 5.9

Master-slave D flip-flop



ŞEKİL 5.10

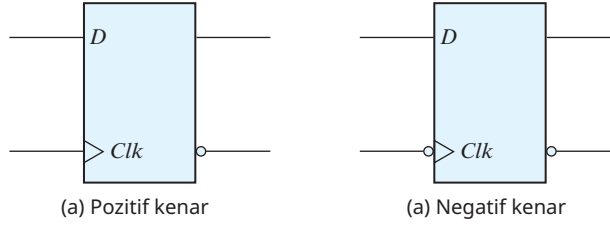
D -tipi pozitif kenar tetiklemeli flip-flop

Çıkış latch'inin S ve R girişleri, $Clk=0$ olduğunda mantık-1 seviyesinde tutulur. Bu, çıkışın mevcut durumunda kalmasına neden olur. Giriş D 0 veya 1'e eşit olabilir. Eğer $D=0$ olduğunda Clk 1 olduğunda, R 0'a değişir. Bu, flip-flop'un sıfırlama durumuna geçmesine neden olur ve $Q=0$ olur. Eğer $Clk=1$ iken D girişinde bir değişiklik olursa, terminal R 0'da kalır çünkü Q 0'dır. Böylece, flip-flop kilitlenir ve girişteki daha fazla değişikliklere yanıt vermez. Saat 0'a döndüğünde, R 1'e geçer ve çıkış latch'ini çıkış değiştirilmeden durgun duruma bırakır. Benzer şekilde, eğer $D=1$ olduğunda Clk 0'dan 1'e geçerse, S 0'a değişir.

Bu, devrenin ayarlanmış duruma geçmesine neden olur, bu da $Q=1$. D 'deki herhangi bir değişiklik, $Clk=1$ çıktısı etkilemez.

Özetle, pozitif kenar tetiklemeli flip-flop'ta giriş saati pozitif geçiş yaptığında, D 'nin değeri Q 'ya aktarılır. Saatin negatif geçişi (yani, 1'den 0'a) çıktısı etkilemez, ayrıca Clk sürekli mantık-1 seviyesinde veya mantık-0 seviyesinde olduğunda D 'deki değişiklikler de çıktısı etkilemez. Bu nedenle, bu tür bir flip-flop yalnızca 0'dan 1'e geçişe yanıt verir ve başka hiçbir şeye.

Bir flip-flop'un giriş verilerine ve saate yanıt verme zamanlaması, kenar tetiklemeli flip-flop'lar kullanılırken dikkate alınmalıdır. Saat geçişinin gerçekleşmesinden önce D 'nin sabit bir değerde tutulması gereken minimum bir süre vardır ki buna *setup time* denir. Benzer şekilde, pozitif geçişin uygulanmasından sonra D 'nin değişmemesi gereken minimum bir süre vardır ki buna *hold time* denir. Flip-flop'un yayılma gecikme süresi, tetikleme kenarı ile çıktının yeni bir duruma stabil hale gelmesi arasındaki aralık olarak tanımlanır. Bu ve diğer parametreler, belirli mantık aileleri için üreticilerin veri kitaplarında belirtilmiştir.

**ŞEKİL 5.11**

Kenara tetiklenen D flip-flop için grafik sembolü

Kenar tetiklemeli D flip-flop'un grafik sembolü Şekil 5.11'de gösterilmiştir. Bu, D latch için kullanılan sembole benzer, tek farkı, Clk harfinin önündeki ok başı benzeri semboldür; bu, *dinamik* bir girişi belirtir. *Dinamik gösterge* (>) flip-flop'un saat sinyalinin kenar geçişine yanıt verdiğini belirtir. Dinamik göstergeye bitişik blok dışındaki bir baloncuk, devreyi tetiklemek için negatif bir kenarı belirtir. Bir baloncuk olmaması, pozitif kenar yanıtını belirtir.

Diğer Flip-Flop'lar

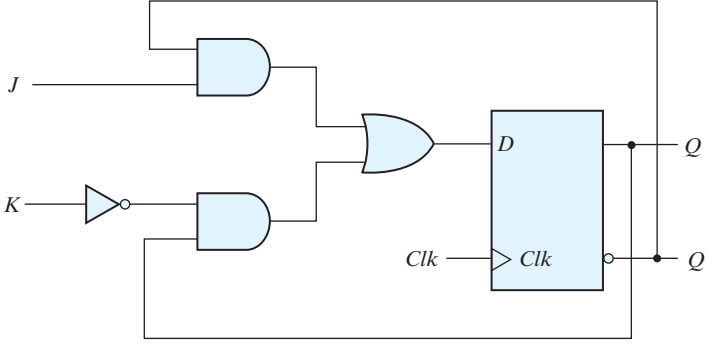
Çok büyük ölçekli entegrasyon devreleri, bir paket içinde birkaç bin kapı içerir. Devreler, çeşitli kapıları birleştirerek dijital bir sistem sağlamak için inşa edilir. Her flip-flop, kapıların birleştirilmesiyle inşa edilir. Bu şekilde inşa edilen en ekonomik ve verimli flip-flop, en az sayıda kapı gerektirdiği için kenar tetiklemeli D flip-flop'tur. D flip-flop ve dış mantık kullanılarak diğer tür flip-flop'lar inşa edilebilir. Dijital sistemlerin tasarımında daha az yaygın olarak kullanılan iki flip-flop, JK ve T flip-flop'lardır.

Bir flip-flop ile gerçekleştirilebilecek üç işlem vardır: 1'e ayarlamak, 0'a sıfırlamak veya çıkışı tamamlamak. Tek bir giriş ile, D flip-flop çıkışı ayarlayabilir veya sıfırlayabilir, bu da saat geçişinden hemen önceki D girişinin değerine bağlıdır. Saat sinyali ile senkronize olan JK flip-flop'u iki girişi vardır ve üç işlemi de gerçekleştirir. D flip-flop'u ve kapılarla inşa edilmiş bir JK flip-flopunun devre diyagramı Şekil 5.12 (a)'de gösterilmiştir. J girişi flip-flop'u 1'e ayarlar, K girişi 0'a sıfırlar ve her iki giriş etkinleştirildiğinde, çıkış tamamlanır. Bu, D girişine uygulanan devreyi inceleyerek doğrulanabilir:

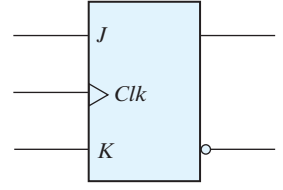
$$D = JQ' + K'Q$$

$J = 1$ ve $K = 0$ olduğunda, $D = Q + Q = 1$, bu nedenle bir sonraki saat kenarı çıkışı 1'e ayarlar. $J = 0$ ve $K = 1$ olduğunda, $D = 0$, bu nedenle bir sonraki saat kenarı çıkışı 0'a sıfırlar. Hem $J = K = 1$ ve $D = Q$ olduğunda, bir sonraki saat kenarı çıkışı tamamlar. Hem $J = K = 0$ ve $D = Q$ olduğunda, saat kenarı çıkışı değiştirmeden bırakır. JK flip-flop'u için grafik sembolü Şekil 5.12 (b)'de gösterilmiştir. D flip-flopunun grafik sembolüne benzer, tek farkla, girişler artık J ve K olarak işaretlenmiştir.

T (toggle) flip-flop, tamamlayıcı bir flip-flop'tur ve girişler J ve K birleştirildiğinde bir JK flip-flop'tan elde edilebilir. Bu, Şekil 5.13 (a)'de gösterilmiştir.

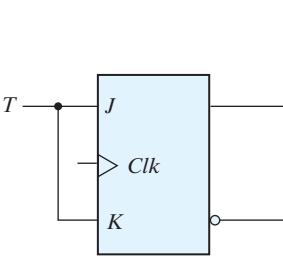


(a) Devre şeması

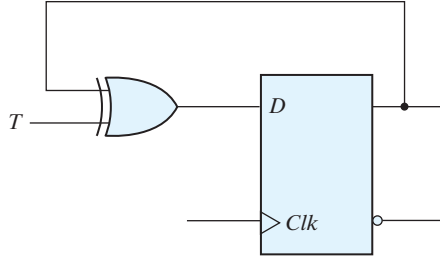


(b) Grafik sembolü

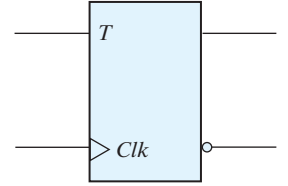
ŞEKİL 5.12
JK flip-flop



(a) JK flip-flop'tan



(b) D flip-flop'tan



(c) Grafik sembolü

ŞEKİL 5.13
T flip-flop

$T=0$ ($J=K=0$), bir saat kenarı çıkışı değiştirmez. $T=1$ ($J=K=1$) olduğunda, bir saat kenarı çıkışı tamamlar. Tamamlayıcı flip-flop, ikili sayaçların tasarımında faydalıdır.

T flip-flop, Şekil 5.13 (b)'de gösterildiği gibi bir D flip-flop ve bir özel-VE kapısı ile inşa edilebilir. D girişi için ifade

$$D = T \oplus Q = TQ' + T'Q$$

$T=0$ olduğunda, $D=Q$ ve çıkışta değişiklik yoktur. $T=1$ olduğunda, $D=Q'$ ve çıkış tamamlanır. Bu flip-flop için grafik sembolü, grişte bir T sembolü içerir.

Karakteristik Tablolar

Bir karakteristik tablo, flip-flop'un mantıksal özelliklerini, tabular biçimdeki işlemlerini tanımlayarak belirler. Üç tür flip-flop'un karakteristik tabloları Tablo 5.1'de sunulmuştur. Bu tablolar, bir saat geçişinden (yani, bir saat geçişinin sonucunda oluşan durum) sonraki durumu tanımlar.

Tablo 5.1
Flip-Flop Karakteristik Tabloları

JK Flip-Flop			
J	K	Q(t + 1)	
0	0	Q(t)	Değişiklik yok
0	1	0	Resetle
1	0	1	Ayarla
1	1	Q'(t)	Tamamlayıcı

D Flip-Flop		
D	Q(t + 1)	
0	0	Resetle
1	1	Ayarla

T Flip-Flop		
T	Q(t + 1)	
0	Q(t)	Değişiklik yok
1	Q'(t)	Tamamlayıcı

girişlerin ve mevcut durumun bir fonksiyonu olarak. $Q(t)$ mevcut durumu ifade eder (yani, bir saat kenarının uygulanmasından önceki mevcut durum). $Q(t+1)$ bir saat periyodu sonra gelecek durumu ifade eder. Saat kenarı girişi karakteristik tabloda yer almaz, ancak t ve $t+1$ zamanları arasında gerçekleştiği varsayılır. Bu nedenle, $Q(t)$ saat kenarından hemen önce flip-flop'un durumunu belirtir ve $Q(t+1)$ saat geçişinden kaynaklanan durumu belirtir.

JK flip-flop'un karakteristik tablosu, girişler J ve K her ikisi de 0 olduğunda gelecek durumun mevcut duruma eşit olduğunu gösterir. Bu durum, $Q(t+1)=Q(t)$ olarak ifade edilebilir ve saat durum değişikliği üretmediğini belirtir. $K=1$ ve $J=0$ olduğunda, saat flip-flop'u sıfırlar ve $Q(t+1)=0$ olur. $J=1$ ve $K=0$ olduğunda, flip-flop ayarlanır ve $Q(t+1)=1$ olur. Her iki J ve K 1'e eşit olduğunda, gelecek durum mevcut durumun tamamlayıcısına değişir, bu geçiş şu şekilde ifade edilebilir: $Q(t+1)=Q'(t)$.

Bir D flip-flop'un bir sonraki durumu yalnızca D girişine bağlıdır ve mevcut durumdan bağımsızdır. Bu, $Q(t+1)=D$ olarak ifade edilebilir. Bu, bir sonraki durum değerinin D'nin değerine eşit olduğu anlamına gelir. D flip-flop'un "değişiklik yok" durumu olmadığını unutmayın. Böyle bir durum, saati devre dışı bırakmak veya flip-flop'un çıkışını D girişine bağlayarak saati çalıştırmak suretiyle sağlanabilir. Her iki yöntem de flip-flop'un durumu değişmeden kalması gerektiğinde flip-flop'un çıkışını etkili bir şekilde dolaştırır.

T flip-flop'un karakteristik tablosunun yalnızca iki durumu vardır: $T=0$ olduğunda, saat kenarı durumu değiştirmez; $T=1$ olduğunda, saat kenarı flip-flop'un durumunu tamamlar.

Karakteristik Denklemler

Flip-flop'un mantıksal özellikleri, karakteristik tabloda açıklandığı gibi, bir karakteristik denklemle cebirsel olarak ifade edilebilir. D flip-flop'u için, karakteristik denklem

$$Q(t + 1) = D$$

çıkışın bir sonraki durumunun, mevcut durumdaki D girişinin değerine eşit olacağını belirtir. JK flip-flop'u için karakteristik denklem, karakteristik tablodan veya Şekil 5.12'den elde edilebilir.

$$Q(t + 1) = JQ' + K'Q$$

Burada Q flip-flop çıkışının, bir saat kenarının uygulanmasından önceki değeridir. T flip-flop'u için karakteristik denklem, Şekil 5.13'ten elde edilir:

$$Q(t + 1) = T \oplus Q = TQ' + T'Q$$

Doğrudan Girişler

Bazı flip-flop'lar, flip-flop'u saatten bağımsız olarak belirli bir duruma zorlamak için kullanılan asenkron girişlere sahiptir. Flip-flop'u 1'e ayarlayan giriş

preset veya *doğrudan ayar* olarak adlandırılır. Flip-flop'u 0'a sıfırlayan giriş ise *clear* veya *doğrudan sıfırlama* olarak adlandırılır.

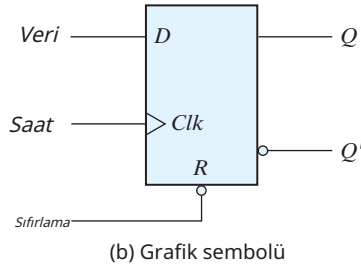
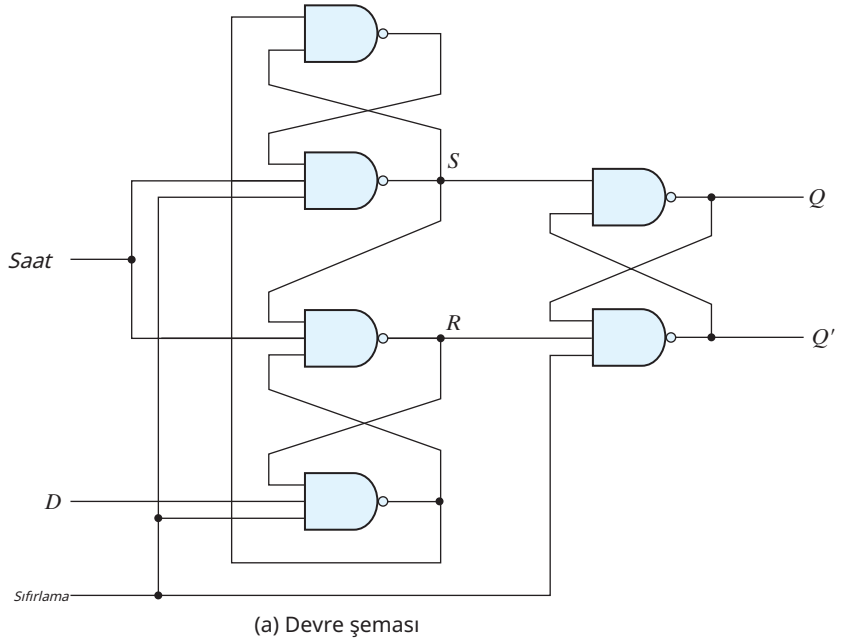
Bir dijital sistemde güç açıldığında, flip-flop'ların durumu bilinmemektedir. Doğrudan girişler, saatli işlemden önce sistemdeki tüm flip-flop'ları bilinen bir başlangıç durumuna getirmek için faydalıdır.

Pozitif kenar tetiklemeli D flip-flop, aktif düşük asenkron sıfırlama ile birlikte Şekil 5.14'te gösterilmiştir. Devre diyagramı, Şekil 5.10'daki ile aynıdır, tek farkı üç N AND kapısına eklenen sıfırlama girişi bağlantılarıdır. Sıfırlama girişi 0 olduğunda, çıkış Q 'yu 1'de tutar, bu da çıkış Q 'yu 0'a sıfırlar ve böylece flip-flop'u sıfırlar. Sıfırlama girişinden iki diğer bağlantı, üçüncü SR latch'in S girişinin sıfırlama girişi 0'da ayken mantık 1'de kalmasını sağlar, değerlerden bağımsız olarak.

D ve Clk .

Doğrudan sıfırlama ile D flip-flop'un grafik sembolü, R ile işaretlenmiş ek bir girişi vardır. Giriş boyunca bulunan baloncuk, sıfırlamanın mantık-0 seviyesinde aktif olduğunu gösterir. Doğrudan ayarlama yapan flip-flop'lar, asenkron ayar girişi için S sembolünü kullanır.

Fonksiyon tablosu devrenin çalışmasını belirtir. $R=0$ olduğunda, çıkış 0'a sıfırlanır. Bu durum, D veya Clk değerlerinden bağımsızdır. Normal saat işlemi, sıfırlama girişinin mantıksal 1'e gitmesinden sonra devam edebilir. Clk üzerindeki saat, flip-flop'un saat sinyalinin pozitif kenarında tetiklendiğini göstermek için yukarı doğru ile gösterilmiştir. D içindeki değer, her pozitif kenar saat sinyali ile Q 'ya aktarılır, şartıyla ki $R=1$.



R	Clk	D	Q	Q'
0	X	X	0	1
0	$\uparrow$	0	0	1
0	$\uparrow$	1	1	0

(b) Fonksiyon tablosu

ŞEKİL 5.14**D** asenkron sıfırlama ile flip-flop

5.5 ZAMANLANMIŞ SIRA LI DEVRELERİN ANALİZİ

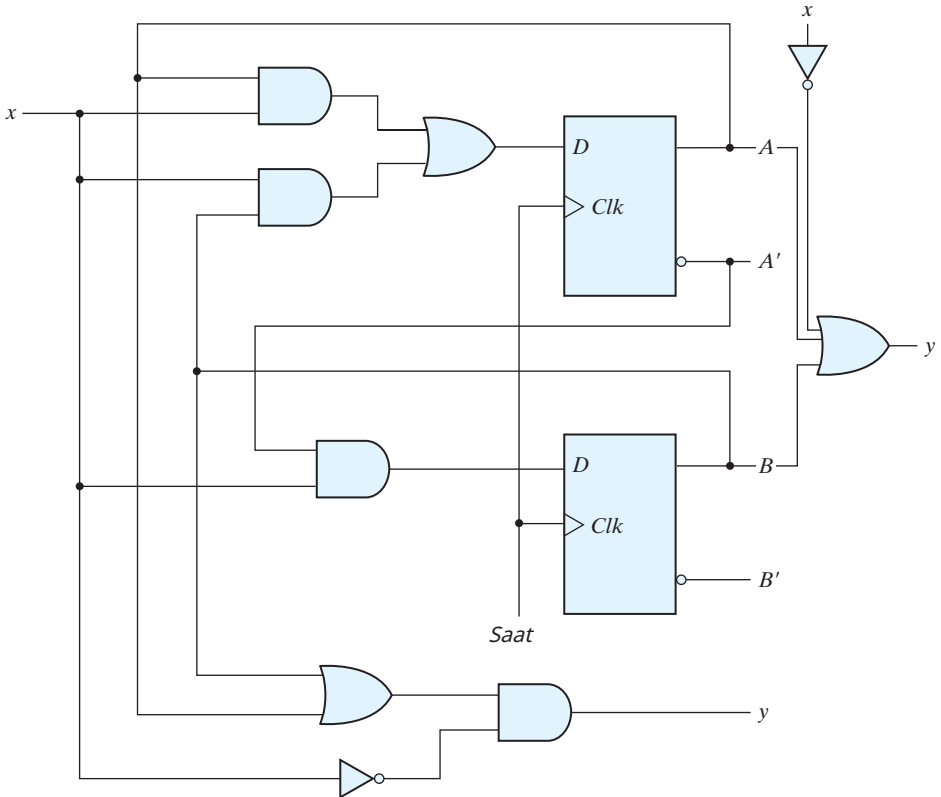
Analiz, belirli çalışma koşulları altında bir devrenin ne yapacağını tanımlar. Zamanlanmış sıralı bir devrenin davranışı, girişler, çıkışlar ve flip-flop'larının durumu ile belirlenir. Çıkışlar ve bir sonraki durum, hem girişlerin hem de mevcut durumun bir fonksiyonudur. Sıralı bir devrenin analizi, girişlerin, çıkışların ve iç durumların zaman sırasını gösteren bir tablo veya diyagram elde etmekten oluşur. Ayrıca mümkündür

sıralı devrenin davranışını tanımlayan Boolean ifadelerini yazmak için. Bu ifadeler, gerekli zaman sırasını doğrudan veya dolaylı olarak içermelidir.

Bir mantık diyagramı, saat girişlerine sahip flip-flop'lar içeriyorsa saatli sıralı devre olarak tanınır. Flip-flop'lar herhangi bir türde olabilir ve mantık diyagramı kombinasyonel mantık kapılarını içerebilir veya içermeyebilir. Bu bölümde, mevcut durum ve girişler açısından bir sonraki durum koşulunu belirtmek için cebirsel bir temsil tanıtıyoruz. Bir durum tablosu ve durum diyagramı, ardışık devrenin davranışını tanımlamak için sunulmaktadır. Ardışık devrelerin mantık diyagramını belirtmek için başka bir cebirsel temsil tanıtılmaktadır. Çeşitli prosedürleri açıklamak için örnekler kullanılmaktadır.

Durum Denklemleri

Bir saatli ardışık devrenin davranışı, durum denklemleri aracılığıyla cebirsel olarak tanımlanabilir. Bir durum denklemi (aynı zamanda geçiş denklemi olarak da adlandırılır), bir sonraki durumu mevcut durum ve girişlerin bir fonksiyonu olarak belirtir. Şekil 5.15'te gösterilen ardışık devreyi düşünelim. Daha sonra, çıkışını bir



ŞEKİL 5.15
Ardışık devre örneği

0, 1'ler akışında tespit edilmiştir. İki D flip-flop A ve B , bir giriş x ve bir çıkış y 'den oluşmaktadır. Bir flip-flop'un D girişi, bir sonraki durumun değerini belirlediğinden (yani, saat geçişinden sonra ulaşılan durum), devre için bir dizi durum denklemi yazmak mümkündür:

$$\begin{aligned}A(t + 1) &= A(t)x(t) + B(t)x(t) \\B(t + 1) &= A'(t)x(t)\end{aligned}$$

Bir durum denklemi, bir flip-flop durum geçişi için koşulu belirten cebirsel bir ifadedir. Denklem sol tarafı, $(t + 1)$ ile, flip-flop'un bir saat kenarı sonra ulaşacağı durumu belirtir. Denklem sağ tarafı, bir Boolean ifadesidir ve bir sonraki durumun 1'e eşit olmasını sağlayan mevcut durum ve giriş koşullarını belirtir. Boolean ifadelerindeki tüm değişkenler mevcut durumun bir fonksiyonu olduğundan, her değişkenin ardından (t) ifadesini kolaylık sağlamak için atlayabiliriz ve durum denklemlerini daha kompakt bir biçimde ifade edebiliriz.

$$\begin{aligned}A(t + 1) &= Ax + Bx \\B(t + 1) &= A'x\end{aligned}$$

Boolean ifadeleri, ardışık devrenin kombinasyonel devre kısmını oluşturan kapılardan doğrudan türetilir, çünkü kombinasyonel devrenin D değerleri bir sonraki durumu belirler. Benzer şekilde, çıkışın mevcut durum değeri cebirsel olarak şöyle ifade edilebilir:

$$y(t) = [A(t) + B(t)]x'(t)$$

Mevcut durum için sembolü (t) kaldırarak, çıkış Boolean denklemini elde ederiz:

$$y = (A + B)x'$$

Durum Tablosu

Girişlerin, çıkışların ve flip-flop durumlarının zaman sırası bir *durum tablosu* (bazen *geçiş tablosu* olarak adlandırılır) şeklinde sıralanabilir. Şekil 5.15 için devrenin durum tablosu Tablo 5.2'de gösterilmiştir. Tablo, *mevcut durum*, *giriş*, *sonraki durum* ve *çıkış* etiketli dört bölümden oluşmaktadır. Mevcut durum bölümü, flip-flop'ların A ve B durumlarını herhangi bir zamanda t gösterir. Giriş bölümü, her olası mevcut durum için x değeri verir. Sonraki durum bölümü, flip-flop'ların bir saat döngüsü sonra, $t + 1$ zamanında durumlarını gösterir. Çıkış bölümü, her mevcut durum ve giriş koşulu için t zamanında y değerini verir.

Bir durum tablosunun türetilmesi, mevcut durumların ve girişlerin tüm olası ikili kombinasyonlarını listelemeyi gerektirir. Bu durumda, 000'dan 111'e kadar sekiz ikili kombinasyonumuz vardır. Sonraki durum değerleri, mantık diyagramından veya durum denklemlerinden belirlenir. Flip-flop A 'nın sonraki durumu, durum denklemini sağlamalıdır.

$$A(t + 1) = Ax + Bx$$

Tablo 5.2

Şekil 5.15 için Devre Durum Tablosu

Mevcut Durum		Giriş	Sonraki Durum		Çıktı
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

Durum tablosundaki bir sonraki durum bölümü, A sütununda, mevcut durum A ve girdi x her ikisi de 1'e eşit olduğunda veya mevcut durum B ve girdi x her ikisi de 1'e eşit olduğunda üç 1 içerir. Benzer şekilde, flip-flop B için bir sonraki durum, durum denklemi

$$B(t+1) = A'x$$

ile elde edilir ve mevcut durum A 0 olduğunda ve girdi x 1'e eşit olduğunda 1'e eşittir. Çıktı sütunu, çıktı denklemi

$$y = Ax' + Bx'$$

ile elde edilir. D -tipi flip-flop'lara sahip bir sıralı devrenin durum tablosu, önceki örnekte belirtilen aynı prosedürle elde edilir. Genel olarak, m flip-flop ve n giriş i olan bir sıralı devre, durum tablosunda 2^{m+n} satıra ihtiyaç duyar. 0'dan

$2^{m+n}-1$ arasındaki ikili sayılar, mevcut durum ve girdi sütunlarının altında listelenir. Bir sonraki durum bölümü, m sütununa sahiptir, her flip-flop için bir tane. Bir sonraki durumun ikili değerleri, durum denklemlerinden doğrudan elde edilir. Çıktı bölümü, çıktı değişkenlerinin sayısı kadar sütuna sahiptir. İkili değeri, devreden veya bir Boolean fonksiyonundan, bir doğruluk tablosundaki gibi elde edilir.

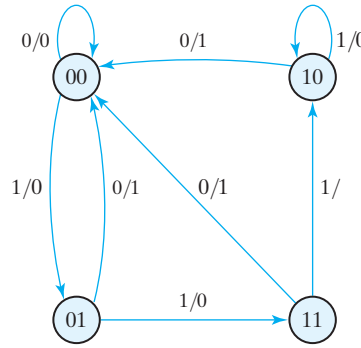
Durum tablosunu, yalnızca üç bölümden oluşan biraz farklı bir biçimde ifade etmek bazen uygundur: mevcut durum, sonraki durum ve çıktı. Giriş koşulları, sonraki durum ve çıktı bölümlerinin altında sıralanmıştır. Tablo 5.2'nin durum tablosu, bu ikinci biçimde Tablo 5.3'te tekrarlanmıştır. Her mevcut durum için, girişin değerine bağlı olarak iki olası sonraki durum ve çıktı vardır. Uygulamaya bağlı olarak bir biçim diğerine tercih edilebilir.

Durum Diyagramı

Bir durum tablosunda mevcut olan bilgiler, bir durum diyagramı biçiminde grafiksel olarak temsil edilebilir. Bu tür bir diyagramda, bir durum bir daire ile temsil edilir ve durumlar arasındaki (saat tetiklemeli) geçişler, yönlendirilmiş çizgilerle gösterilir.

Tablo 5.3
Durum Tablosunun İkinci Formu

Mevcut Durum		Sonraki Durum				Çıktı	
		$x = 0$		$x = 1$		$x = 0$	$x = 1$
A	B	A	B	A	B	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0



ŞEKİL 5.16
Şekil 5.15'in devre durum diyagramı

daireler. Şekil 5.15'teki sıralı devrenin durum diyagramı Şekil 5.16'da gösterilmiştir. Durum diyagramı, durum tablosuyla aynı bilgiyi sağlar ve doğrudan Tablo 5.2 veya Tablo 5.3'ten elde edilir. Her dairenin içindeki ikili sayı, flip-flop'ların durumunu tanımlar. Yönlendirilmiş hatlar, bir eğik çizgi ile ayrılmış iki ikili sayı ile etiketlenmiştir. Mevcut durumdaki giriş değeri önce etiketlenir ve eğik çizgi den sonraki sayı, belirtilen girişle mevcut durumdaki çıkışı verir. (Yönlendirilmiş hat boyunca listelenen çıkış için bit değeri, mevcut durumda ve belirtilen girişle gerçekleşir ve bir sonraki duruma geçişle hiçbir ilgisi yoktur.) Örneğin, durum 00'dan 01'e giden yönlendirilmiş hat 1/0 ile etiketlenmiştir, bu da sıralı devre mevcut durumda 00 iken ve giriş 1 olduğunda çıkışın 0 olduğu anlamına gelir. Bir sonraki saat döngüsünden sonra devre bir sonraki duruma, 01'e geçer. Eğer giriş 0'a değişirse, çıkış 1 olur, ancak giriş 1'de kalırsa, çıkış 0'da kalır. Bu bilgi, durum 01 olan daireden çıkan iki yönlendirilmiş hat boyunca durum diyagramından elde edilir. Kendisiyle bağlantılı bir yönlendirilmiş hat, durum değişikliği olmadığını gösterir.

Bu örnekte sunulan adımlar aşağıda özetlenmiştir:

Circuit diagram → Equations – State table → State diagram

Bu adım dizisi, devrenin yapısal bir temsil ile başlar ve davranışının soyut bir temsil ile devam eder. Bir HDL modeli, bir kapı düzeyi tanımı veya bir davranış tanımı şeklinde olabilir. Kapı düzeyi yaklaşımının, tasarımcının belirli bir davranışa sahip bir devre oluşturmak için kapıları ve flip-flop'ları nasıl seçeceğini ve bağlayacağını anlamasını gerektirdiğini belirtmek önemlidir. Bu anlayış deneyimle gelir. Öte yandan, davranışsal modelleme temelli bir yaklaşım, tasarımcının bir şematik icat etmesini bilmesini gerektirmez—tasarımcının yalnızca HDL'nin yapıları kullanarak davranış tanımlamayı bilmesi yeterlidir, çünkü devre otomatik olarak bir sentez aracı tarafından üretilir. Bu nedenle, dijital devrelerin üretken bir tasarımcısı olmak için yıllarca deneyim biriktirmeye gerek yoktur; ayrıca elektrik mühendisliği konusunda kapsamlı bir arka plana sahip olmak da gerekmez.

Bir durum tablosu ile bir durum diyagramı arasında, temsil biçiminden başka bir fark yoktur. Durum tablosu, verilen bir mantık diyagramından ve durum denklemi ile daha kolay türetilir. Durum diyagramı doğrudan durum tablosundan gelir. Durum diyagramı, durum geçişlerinin görsel bir görünümünü sunar ve devrenin çalışmasının insan tarafından yorumlanması için daha uygun bir formdur. Örneğin, Şekil 5.16'daki durum diyagramı, 00 durumundan başlayarak, giriş 1'de kaldığı sürece çıkışın 0 olduğunu açıkça gösterir. Bir dizinin 1'den sonra gelen ilk 0 girişi, çıkışı 1 yapar ve devreyi başlangıç durumu olan 00'a geri döndürür. Bu durum diyagramı ile temsil edilen makine, veri bit akışında bir sıfırı tespit etmek için çalışır. Bu, Şekil 5.15'teki devrenin davranışına karşılık gelir. Veri akışında bir sıfırı tespit eden diğer devreler daha basit bir devre diyagramına ve durum diyagramına sahip olabilir.

Flip-Flop Giriş Denklemleri

Sıralı bir devrenin mantık diyagramı, flip-flop'lar ve kapılardan oluşur. Kapılar arasındaki bağlantılar, bir kombinasyonel devre oluşturur ve cebirsel olarak Boolean ifadeleri ile tanımlanabilir. Flip-flop türü bilgisi ve kombinasyonel devrenin Boolean ifadeleri listesi, sıralı devrenin mantık diyagramını çizmek için gereken bilgiyi sağlar. Dışsal çıkışları üreten kombinasyonel devre kısmı, Boolean fonksiyonları seti ile cebirsel olarak tanımlanır.

çıkış denklemleri. Flip-flop'lara girişleri üreten devre kısmı, flip-flop *giriş denklemleri* (veya bazen, *uyandırma denklemleri*) olarak adlandırılan bir dizi Boolean fonksiyonu ile cebirsel olarak tanımlanır. Giriş denklemi değişkenini belirtmek için flip-flop giriş sembolünü ve flip-flop çıkışının adını belirtmek için bir alt simge kullanma kuralını benimseyeceğiz. Örneğin, aşağıdaki giriş denklemi, D girişine bağlı x ve y girişlerine sahip bir VE kapısını belirtir ve çıkışı Q sembolü ile etiketlenmiştir:

$$D_Q = x + y$$

Şekil 5.15'teki ardışık devre, iki D flip-flop A ve B , bir giriş x ve bir çıkış y içerir. Devrenin mantık diyagramı, iki flip-flop giriş denklemi ve bir çıkış denklemi ile cebirsel olarak ifade edilebilir:

$$D_A = Ax + Bx$$

$$D_B = A'x$$

$$y = (A + B)x'$$

Üç denklem, ardışık devrenin mantık diyagramını çizmek için gerekli bilgileri sağlar. Sembol D_A D flip-flop'un A olarak etiketlendiğini belirtir. D_B ikinci bir D flip-flop'un B olarak etiketlendiğini belirtir. Bu iki değişkenle ilişkili Boolean ifadele r ve çıkış y için ifade, ardışık devrenin kombinasyonel devre kısmını belirtir.

Flip-flop giriş denklemleri, ardışık bir devrenin mantık diyagramını belirtmek için uygun bir cebirsel form oluşturur. Bunlar, flip-flop'un türünü harf sembolünden çıkarır ve flip-flop'ları yönlendiren kombinasyonel devreyi tamamen tanımlar. D flip-flop için giriş denkleminin ifadesinin, ilgili durum denkleminin ifadesiyle aynı olduğunu unutmayın. Bu, bir sonraki durumu D girişinin değeriyle eşitleyen karakteristik denklemin bir sonucudur: $Q(t + 1) = DQ$.

D Flip-Flop'lar ile Analiz

D flip-flop'lar ile saatli bir ardışık devrenin analiz prosedürünü basit bir örnek ile özetleyeceğiz. Analiz etmek istediğimiz devre, giriş denklemi ile tanımlanmıştır.

$$D_A = A \oplus x \oplus y$$

D A sembolü, A çıkışına sahip bir D flip-flop'u ifade eder. x ve y değişkenleri devreye girişlerdir. Hiçbir çıkış denklemi verilmemiştir, bu da çıkışın flip-flop'un çıkışından geldiğini ima eder. Mantık diyagramı giriş denklemi ile elde edilir ve Şekil 5.17 (a)'da çizilmiştir.

Durum tablosu, flip-flop A 'nın mevcut durumu için bir sütun, iki giriş için iki sütun ve A 'nın bir sonraki durumu için bir sütun içerir. Axy altında yer alan ikili sayılar, Şekil 5.17 (b)'de gösterildiği gibi 000'dan 111'e kadar sıralanmıştır. Bir sonraki durum değerleri, durum denklemleri ile elde edilir.

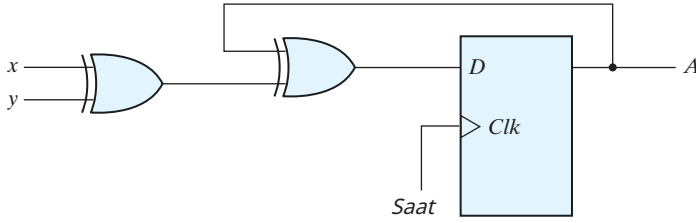
$$A(t + 1) = A \oplus x \oplus y$$

İfade, tek bir değişkenin 1 olduğu veya üç değişkenin de 1 olduğu durumlarda 1'e eşit olan bir tek işlevi belirtir. Bu, A için bir sonraki durum sütununda gösterilmektedir.

Devreye bir flip-flop ve iki duruma sahiptir. Durum diyagramı, her durum için birer tane olmak üzere iki daireden oluşmaktadır; bu, Şekil 5.17 (c)'de gösterilmiştir. Mevcut durum ve çıkış ya 0 ya da 1 olabilir; bu, dairelerin içindeki sayılarla belirtilmiştir. Yönlendirilmiş hatlar da bir kesik işareti gerekli değildir, çünkü kombinasyonel bir devreden çıkış yoktur. İki giriş, her durum için dört olası kombinasyona sahip olabilir. Her durum geçişindeki iki giriş kombinasyonu, notasyonu basitleştirmek için bir virgülle ayrılmıştır.

JK Flip-Flop'larla Analiz

Bir durum tablosu dört bölümden oluşur: mevcut durum, girişler, bir sonraki durum ve çıkışlar. İlk iki bölüm, tüm ikili kombinasyonları listeleterek elde edilir. Çıkış bölümü, çıkış denklemlerinden belirlenir. Bir sonraki durum değerleri, durum denklemlerinden değerlendirilir. D tipi flip-flop için durum denklemi, giriş denklemi ile aynıdır. D tipi dışında bir flip-flop kullanıldığında, örneğin JK veya T tipi, başvurmak gereklidir.



(a) Devre şeması

Mevcut durum	Girdi durumu		Sonraki durum
A	x	y	A
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(b) Durum tablosu



(c) Durum diyagramı

ŞEKİL 5.17 D flip-flop ile ardışık devre

İlgili karakteristik tabloya veya karakteristik denkleme giderek bir sonraki- durum değerleri elde edin. Öncelikle karakteristik tabloyu kullanarak, ardından da karakteristik denklemi kullanarak prosedürü göstereceğiz.

JK - veya *T* -tip flip-flop'lar kullanan bir sıralı devrenin bir sonraki durum değerleri aşağıda ki gibi türetilir:

Mevcut durum ve giriş

değişkenleri cinsinden flip-flop giriş denklemlerini belirleyin.

Her giriş denkleminin ikili değerlerini listeleyin.

Durum tablosundaki bir sonraki durum değerlerini belirlemek için ilgili flip-flop karakteristik tablosunu kullanın.

Örnek olarak, iki *JK* flip-flop *A* ve *B* ile bir giriş *x* olan sıralı devreyi düşünün, Şekil 5.18'de gösterildiği gibi. Devrenin çıkışı yoktur; bu nedenle, durum tablosunun bir çıkış sütununa ihtiyacı yoktur. (Bu durumda flip-flop'ların çıkışları çıkışlar olarak kabul edilebilir.) Devre, flip-flop giriş denklemleri ile tanımlanabilir.

$$J_A = B \quad K_A = Bx'$$

$$J_B = x' \quad K_B = A'x + Ax' = A \oplus x$$

Sıralı devrenin durum tablosu Tablo 5.4'te gösterilmiştir. Mevcut durum ve girdi sütunları sekiz ikili kombinasyonu listelemektedir. Altında listelenen ikili değerler

$K=0$, sonraki durum 1'dir. $J=0$ ve $K=1$ olduğunda, sonraki durum 0'dır. $J=K=0$ olduğunda, durum değişmez ve sonraki durum değeri mevcut durumun değeriyle aynıdır. $J=K=1$ olduğunda, sonraki durum biti mevcut durum bitinin tamamlayıcısıdır.

Son iki durumun örnekleri, mevcut durum AB 10 olduğunda ve

girdi x 0 olduğunda tabloda görülmektedir. JA ve KA her ikisi de 0'a eşittir ve A mevcut durumu 1'dir. Bu nedenle, A için sonraki durum aynı kalır ve 1'e eşittir. Tabloyun aynı satırında, JB ve KB her ikisi de 1'e eşittir. Mevcut durum B 0 olduğundan, B için sonraki durum tamamlanır ve 1'e dönüşür.

Sonraki durum değerleri, karakteristik denklemlerden durum denklemlerini değerlendirek elde edilebilir. Bu, aşağıdaki prosedür kullanılarak yapılır:

1. Mevcut durum ve giriş

değişkenleri cinsinden flip-flop giriş denklemlerini belirleyin.

2. Girdi denklemlerini flip-flop karakteristik denklemlerine yerleştirerek

durum denklemlerini elde edin.

3. Eyalet tablosundaki bir sonraki durum değerlerini belirlemek için ilgili durum denklemlerini kullanın.

Şekil 5.18'deki iki JK flip-flop için giriş denklemleri birkaç paragraf önce listelenmiştir. Flip-floplar için karakteristik denklemler, A veya B flip-flop'un adı için, Q yerine:

$$A(t+1) = JA' + K'A$$

$$B(t+1) = JB' + K'B$$

Girdi denklemlerinden J_A ve K_A değerlerini yerine koyarak, A için durum denklemini elde ediyoruz:

$$A(t+1) = BA' + (Bx)'A = A'B + AB' + Ax$$

Durum denklemi, durum tablosundaki "Sonraki Durum" başlıklı sütun için A için bit değerleri sağlar. Benzer şekilde, flip-flop B için durum denklemi, J_B ve K_B değerlerini yerine koyarak karakteristik denkleminin türetilmesiyle elde edilebilir:

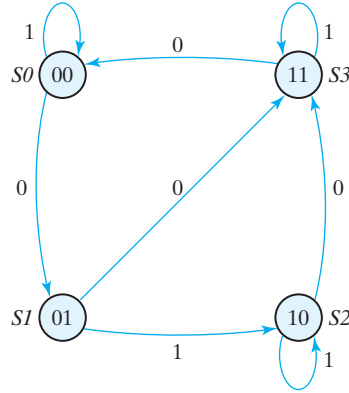
$$B(t+1) = x'B' + (A \oplus x)'B = B'x' + ABx + A'Bx'$$

Durum denklemi, durum tablosundaki "Sonraki Durum" başlıklı sütun için B bit değerleri sağlar. Durum denklemleri kullanıldığında, Tablo 5.4'teki "Flip-Flop Girişleri" başlıklı sütunların gerekli olmadığını unutmayın.

Sıralı devrenin durum diyagramı Şekil 5.19'da gösterilmiştir. Devrenin çıkışı olmadığı için, dairelerin dışındaki yönlü çizgiler yalnızca bir ikili sayı ile işaretlenmiştir; bu, girdi değerini x belirtmektedir.

Analiz ile T Flip-Flop'lar

Bir ardışık devrenin analizi, T flip-flop'ları için belirtilen aynı prosedürü izler. Durum tablosundaki bir sonraki durum değerleri, ya



ŞEKİL 5.19

Şekil 5.18'in devre durum diyagramı

Tablo 5.1'de listelenen karakteristik tabloyu ya da karakteristik denklemi kullanarak elde edilebilir.

$$Q(t + 1) = T \oplus Q = T'Q + TQ'$$

Şimdi Şekil 5.20'de gösterilen ardışık devreyi düşünelim. İki flip-flop A ve B , bir giriş x , ve bir çıkış y vardır ve iki giriş denklemi ve

bir çıkış denklemi ile cebirsel olarak tanımlanabilir:

$$T_A = Bx$$

$$T_B = x$$

$$y = AB$$

Devre için durum tablosu Tablo 5.5'te listelenmiştir. y için değerler çıkış denklemi nden elde edilir. Bir sonraki durum için değerler, karakteristik denklemlerde T_A ve T_B yerine koyarak durum denklemlerinden türetilebilir, bu da

$$A(t + 1) = (Bx)'A + (Bx)A' = AB' + Ax' + A'Bx$$

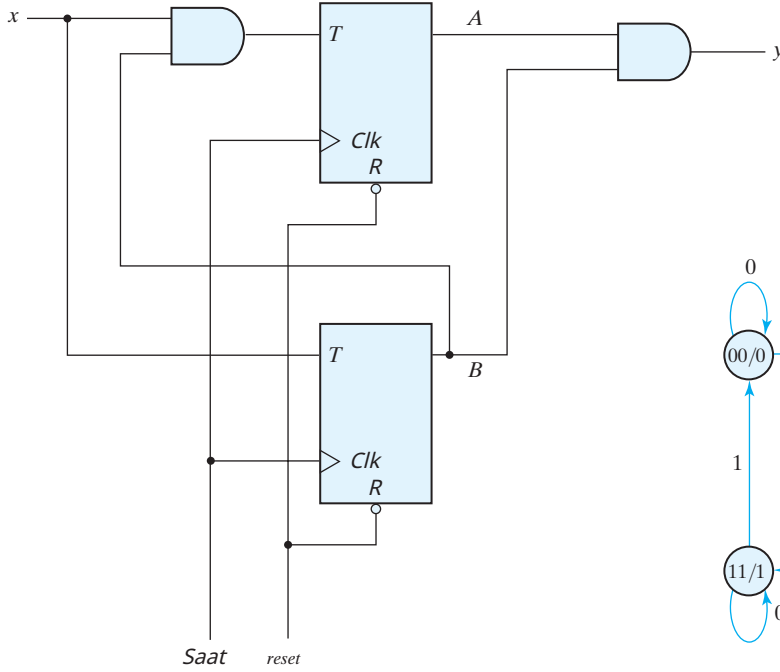
$$B(t + 1) = x \oplus B$$

Durum tablosundaki A ve B için bir sonraki durum değerleri, iki durum denkleminin ifadelerinden elde edilir.

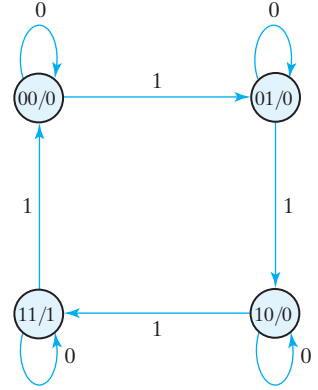
Devrenin durum diyagramı Şekil 5.20 (b)'de gösterilmektedir. Giriş x 1'e eşit olduğu sür ece, devre 00, 01, 10, 11 durumlarıyla bir ikili sayaç gibi davranır ve tekrar 00'a döner. 0 olduğunda, devre aynı durumda kalır. Mevcut durumda 11 olduğunda çıkış y 1'e eşittir. Burada, çıkış yalnızca mevcut duruma bağlıdır ve girdi ile bağımsızdır. Her bir dairenin içindeki ve bir eğik çizgi ile ayrılmış iki değer, mevcut durum ve çıkış içindir.

Sonlu Durum Makinelerinin Mealy ve Moore Modelleri

Bir ardışık devrenin en genel modeli, girişler, çıkışlar ve iç durumlara sahiptir. Ardışık devreleri n iki modeli arasında ayırmak alışılmıştır: Mealy modeli ve Moore modeli. Her ikisi de Şe kil 5.21'de gösterilmiştir. Çıkışın hesaplanma şekli bakımından yalnızca farklılık gösterirler.



(a) Devre şeması

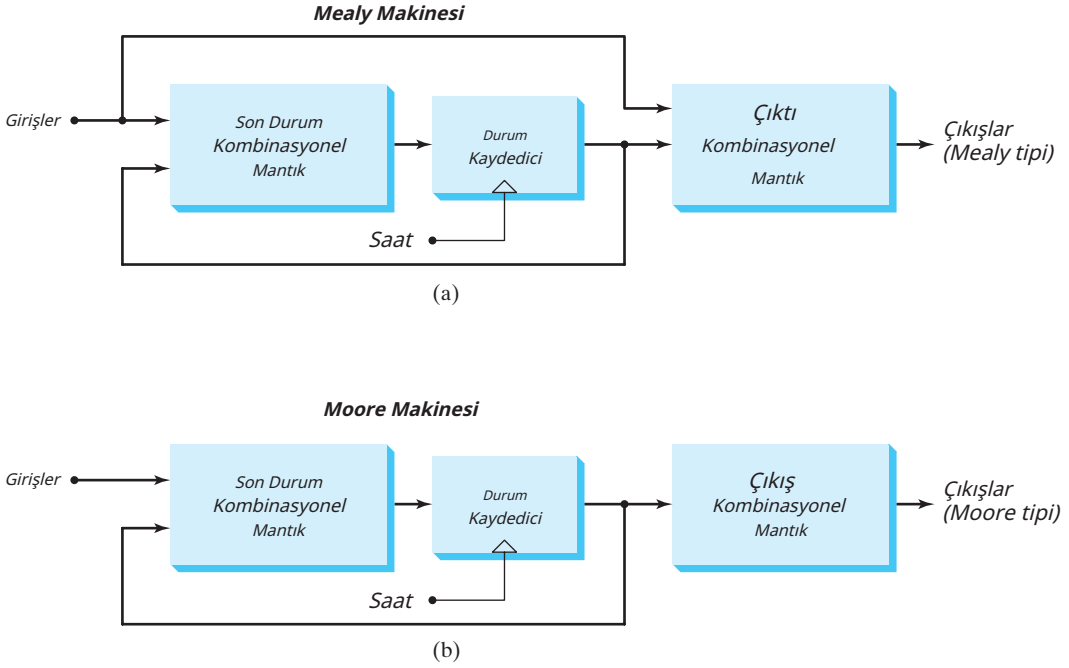


(b) Durum diyagramı

ŞEKİL 5.20
Artımlı devre *T* flip-flop'lar (İkili Sayıcı)

Tablo 5.5
T Flip-Flop'lu Ardışık Devre için Durum Tablosu

Mevcut Durum		Giriş <i>x</i>	Sonraki Durum		Çıktı <i>y</i>
<i>A</i>	<i>B</i>		<i>A</i>	<i>B</i>	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

**ŞEKİL 5.21**

Mealy ve Moore durum makinelerinin blok diyagramları

oluşturulur. Mealy modelinde, çıkış hem mevcut duruma hem de girdiye bağlıdır. Moore modelinde, çıkış yalnızca mevcut duruma bağlıdır. Bir devre birlikte her iki tür çıkışa sahip olabilir. Artımlı devrenin iki modeli genellikle 'sonlu durum makinesi' (finite state machine), kısaca FSM olarak adlandırılır. Artımlı devrenin Mealy modeli, Mealy FSM veya Mealy makinesi olarak adlandırılır. Moore modeli ise Moore FSM veya Moore makinesi olarak adlandırılır.

Daha önce sunulan devre, Şekil 5.15'te bir Mealy makinesi örneğidir. Çıkış y hem girdi x hem de mevcut durum A ve B 'ye bağlıdır. İlgili durum diyagramı Şekil 5.16'da, durumlar arasındaki yönlendirilmiş çizgiler boyunca bir eğik çizgi ile ayrılmış girdi ve çıkış değerlerini göstermektedir.

Bir Moore modelinin örneği Şekil 5.18'de verilmiştir. Burada, çıkış yalnızca mevcut duruma bağlıdır. İlgili durum diyagramı Şekil 5.19'da yalnızca yönlendirilmiş çizgiler boyunca işaretlenmiş girdiler bulunmaktadır. Çıkışlar, dairelerin içinde işaretlenmiş flip-flop durumlarıdır. Moore modelinin bir diğer örneği, Şekil 5.20'deki ardışık devredir. Çıkış, yalnızca flip-flop değişkenlerine bağlıdır ve bu da onu yalnızca mevcut duruma bağlı bir fonksiyon haline getirir. Durum diyagramındaki giriş değeri, yönlendirilmiş çizgi boyunca etiketlenmiştir, ancak çıkış değeri mevcut durumla birlikte dairenin içinde belirtilmiştir.

Bir Moore modelinde, ardışık devrenin çıkışları saatle senkronizedir, çünkü yalnızca saatle senkronize olan flip-flop çıkışlarına bağlıdır. Bir Mealy modelinde, çıkışlar, saat sırasında girişler değişirse değişebilir.

rica, çıkışlar, girişlerin değiştiği zamandan flip-flop çıkışlarının değiştiği zamana kadar geçen gecikme nedeniyle anlık yanlış değerlere sahip olabilir. Mealy tipi bir devreyi senkronize etmek için, sıralı devrenin girişleri saatle senkronize edilmeli ve çıkışlar saat kenarından hemen önce örneklenmelidir. Girişler, flip-flop'lara aktif saat kenarı gerçekleşmeden önce stabilize olmalarını sağlamak için saatinin inaktif kenarında değiştirilir. Böylece, Mealy makinesinin çıkışı, saatinin aktif kenarından hemen önce mevcut olan değerdir.

5.6 SIRALI DEVRELERİN SENTEZLENEBİLİR HDL MODELLERİ

Verilog HDL, Bölüm 3.9'da tanıtılmıştır. Kombinasyonel devreler, Bölüm 4.12'de tanımlanmış ve Verilog ile davranışsal modelleme de o bölümde tanıtılmıştır. Davranışsal modeller, dijital donanımın işlevselliğinin soyut temsilleridir. Yani, bir devrenin nasıl davrandığını tanımlar, ancak devrenin iç detaylarını belirtmez. Tarihsel olarak, soyutlama doğruluk tabloları, durum tabloları ve durum diyagramları ile tanımlanmıştır. Bir HDL, işlevselliği, bir makinedeki kayıtların işlemlerin temsil eden dil yapılarıyla farklı bir şekilde tanımlar.

Bu temsilin “katma değeri” vardır; yani, nasıl kullanılacağını bilmek önemlidir, çünkü makinenin davranışını gösteren dalga formlarını üretmek için simüle edilebilir.

Davranışsal Modelleme

Verilog HDL'de iki tür soyut davranış vardır. **initial** anahtar kelimesiyle tanımlanan davranış, tek geçişli davranış olarak adlandırılır ve tek bir ifade veya bir blok ifadesini (yani, ya bir **begin... end** ya da bir **fork... join** anahtar kelime çifti ile kapsanan ifadeler listesi) belirtir. Tek geçişli davranış, ilişkili ifadenin yürütülmesinden sonra sona erer. Pratikte, tasarımcılar tek geçişli davranış, bir test tezgahında uyarı sinyallerini tanımlamak için kullanır—bir devrenin davranışını modellemek için asla kullanmazlar—çünkü sentez araçları **initial** ifadesini kullanan tanımlamaları kabul etmez. **always** anahtar kelimesi, döngüsel davranış tanımlar. Her iki tür davranış, simülatör zaman $t = 0$ 'da başlatıldığı anda yürütülmeye başlar. **initial** davranış, ifadesi yürütüldükten sonra sona erer; **always** davranış ise simülasyon durdurulana kadar sonsuz bir şekilde yürütülür ve yeniden yürütülür. Bir modül, keyfi sayıda **initial** veya **always** davranışsal ifadeleri içerebilir. Bir birleriyle zaman 0'da eşzamanlı olarak yürütülür ve ortak değişkenler aracılığıyla etkileşimde bulunabilirler. İşte basit bir D flip-flop modelinin **always** ifadesinin nasıl çalıştığına dair bir kelime açıklaması: Saatin yükselen kenarı meydana geldiğinde, eğer sıfırlama girişi etkinleştirilmişse, çıkış q 0 alır; aksi takdirde çıkış Q giriş D değerini alır. Saat t arafından tetiklenen ifadelerin yürütülmesi, simülasyon sona erene kadar tekrarlanır. Bu açıklamayı Verilog'da nasıl yazacağımızı kısa süre içinde göreceğiz.

Bir başlangıç davranış ifadesi yalnızca bir kez çalıştırılır. Simülasyonun başlangıcında çalışmaya başlar ve tüm ifadeleri tamamlandıktan sonra sona erer. Bölüm 4.12'nin sonunda belirtildiği gibi, başlangıç ifadesi, bir tasarımı simüle etmek için giriş sinyalleri üretmekte faydalıdır. Sıralı bir devreyi simüle ederken, flip-flop'ları tetikleme için bir saat kaynağı üretmek gereklidir. Aşağıda, belirli bir döngü sayısı için çalışan serbest koşan bir saat sağlamanın iki olası yolu verilmiştir:

başlangıç	başlangıç
begin	
saat = 1'b0;	saat = 1'b0;
repeat (30)	bitiş
#10 saat = ~saat;	
bitiş	ilk 300 \$ bitir ;
	h sempre #10 saat = ~saat;

İlk versiyonda, ilk blok, **begin** and **end** anahtar kelimeleri içinde yer alan iki ifadeyi içerir. İlk ifade, *saat* değerini 0 olarak ayarlar zaman=0. İkinci ifade, 30 kez tekrar eden bir döngü tanımlar, 10 zaman birimi bekler ve ardından *saat* değerini tamamlar. Bu, her biri 20 zaman birimi döngü süresine sahip 15 saat döngüsü üretir. İkinci versiyonda, ilk ilk davranış, *saat* değerini 0 olarak ayarlayan tek bir ifadeye sahiptir zaman=0, ve ardından sona erer (daha fazla simülasyon etkinliği oluşturmaz). İkinci tek geçişli davranış, simülasyon için bir kronometre tanımlar. Sistem görevi bitir simülasyonun, 300 zaman birimi geçtikten sonra koşulsuz olarak sona ermesini sağlar. Bu davranışın yalnızca bir ifadeye sahip olması nedeniyle, **begin** yazmaya gerek yoktur. . . **end** anahtar kelime çifti. 10 zaman biriminden sonra, her zaman ifadesi tekrar tekrar *saat*, zaman döngüsü 20 zaman birimi olan bir saat üretici sağlar. İkinci örnekteki üç davranışsal ifade herhangi bir sırayla yazılabilir.

İşte serbest çalışan bir saati tanımlamanın başka bir yolu:

```
ilk başla saat = 0; sonsuza dek #10 saat = ~saat; son
```

Bu versiyon, bir blok ifadesinde iki ifade ile saati başlatır ve ardından 10 zaman adımımdan sonra saatin tamamlandığı bir belirsiz döngü (sonsuz) yürütür. Tek geçişli davranışın asla tamamlanmadığını ve bu nedenle sona ermediğini unutmayın. Simülasyonu sonlandırmak için başka bir davranış olmalıdır.

Her iki tür davranışsal ifadeye bağlı etkinlik, belirli bir süre bekleyen bir gecikme operatörü veya belirli koşulların doğru hale gelmesini veya belirli olayların (sinyallerdeki değişiklikler) gerçekleşmesini bekleyen bir olay kontrol operatörü ile kontrol edilebilir. # *gecikme kontrol operatörü* ile belirtilen zaman gecikmeleri genellikle tek geçişli davranışlarda kullanılır. Gecikme kontrol operatörü, belirtilen bir süre geçene kadar ifadelerin yürütülmesini askıya alır. Test tezgahında sinyalleri belirtmek için kullanımına dair örnekler zaten görüldü. Diğer bir operatör @, *olay kontrol operatörü* olarak adlandırılır ve kullanılır.

etkinliği bir olay gerçekleşene kadar askıya al. Bir olay, bir

sinyal değerinde koşulsuz bir değişiklik (örneğin, @ A) veya bir sinyal değerinin belirli bir geçişi (örneğin, @ (posedge clock)) olabilir. Bu tür bir ifadenin genel biçimi şudur:

```
her zaman @ (olay kontrol ifadesi) başla
// Koşul karşılandığında yürütülen prosedürel atama ifadeleri
bitir
```

Olay kontrol ifadesi, prosedürel atama ifadelerinin yürütülmesini başlatmak için gerçekleşmesi gereken koşulu belirtir. Prosedürel ifadelerin sol tarafındaki değişkenler **reg** veri türünde olmalı ve bu şekilde tanımlanmalıdır. Sağ taraf, Verilog tanımlı operatörleri kullanarak bir değer üreten herhangi bir ifade olabilir.

Olay kontrol ifadesi (duyarlılık listesi olarak da adlandırılır), her zaman blok ile ilişkili prosedürel ifadelerin yürütülmesini başlatmak için gerçekleşmesi gereken olayları belirtir. Blok içindeki ifadeler, yukarıdan aşağıya doğru sıralı olarak yürütülür. Son ifadeden sonra, davranış olay kontrol ifadesinin tatmin edilmesini bekler. Ardından ifadeler tekrar yürütülür. Duyarlılık listesi, seviye-duyarlı olayları, kenar-duyarlı olayları veya her ikisinin bir kombinasyonunu belirtebilir. Pratikte, tasarımcılar üçüncü seçeneği kullanmazlar, çünkü bu üçüncü biçim, sentez araçlarının fiziksel donanıma çeviremediği bir biçimdir. Seviye-duyarlı olaylar, kombinasyonel devrelerde ve latch'lerde meydana gelir. Örneğin, ifade

```
her zaman @ (A veya B veya C)
```

ilgili her zaman bloğundaki prosedürel ifadelerin yürütülmesini başlatacaktır. A, B, veya C 'de bir değişiklik meydana geldiğinde. Senkron sıralı devrelerde, flip-flop'lardaki değişiklikler sadece bir saat darbesinin geçişine yanıt olarak meydana gelir. Geçiş ya bir pozitif kenar ya da saat sinyalinin negatif kenarı olabilir, ancak her ikisi birden olamaz. Verilog HDL, bu koşulları sağlamak için iki anahtar kelime sunar: **posedge** ve **negedge**. Örneğin, ifade

```
her zaman @(posedge saat veya negedge sıfırlama) // Verilog 1995
```

```
// Verilog 1995
```

yalnızca saat pozitif bir geçişten geçerse veya *sıfırlama* negatif bir geçişten geçerse, ilgili prosedürel ifadelerin yürütülmesini başlatacaktır. Verilog dilinin 2001 ve 2005 revizyonları, olay kontrol ifadesi (veya duyarlılık listesi) için virgülle ayrılmış bir listeye izin verir:

```
her zaman @(posedge saat, negedge sıfırlama) // Verilog 2001, 2005
```

```
// Verilog 2001, 2005
```

Bir prosedürel atama, bir

ilk veya her zaman ifadesi içinde bir değişkene mantıksal bir değer atamasıdır. Bu, Bölüm 4.12'de veri akışı modellemesi ile tartışılan sürekli atama ile zıttır. Sürekli bir atama, atama ifadesinin sağ tarafındaki tüm değişkenlerden oluşan örtülü bir seviye-duyarlı duyarlılık listesine sahiptir. Sürekli bir atamanın güncellenmesi, ifadesinin sağ tarafında yer alan bir değişkende bir olay meydana geldiğinde tetiklenir. Buna karşılık, bir prosedürel atama yalnızca bir atama ifadesi yürütüldüğünde yapılır ve bir davranışsal ifade içinde ona değer atar. Örneğin, önceki örnekte saat sinyali yalnızca *saat* = ~ saat ifadesi yürütüldüğünde tamamlandı; ifade, sıfırlama başladığında 10 zaman birimi geçene kadar yürütülmedi.

Bir değişkenin **reg** türüne sahip olduğunu hatırlamak önemlidir; bu değişken, yeni bir değer vermek için bir prosedürel atama yapılana kadar değişmez.

İki tür prosedürel atama vardır: *blocking* ve *nonblocking*. İkisi, kullandıkları sembollerle ayırt edilir. Blocking atamaları, atama operatörü olarak (=) sembolünü kullanırken, nonblocking atamaları (<=) operatörünü kullanır.

Blocking atama ifadeleri, bir ifade bloğunda listelendiği sırayla ardışık olarak yürütülür. Nonblocking atamalar, ifadelerin sağ tarafındaki ifadelerin değerlendirilmesiyle eşzamanlı olarak yürütülür; tüm ifadeler değerlendirilene kadar sol taraflarına atama yapmazlar. İki tür atama, bir örnekle daha iyi anlaşılabilir. Bu iki prosedürel blocking atamasını düşünün:

$$\begin{aligned} B &= A \\ C &= B + 1 \end{aligned}$$

İlk ifade, A değerini B değişkenine aktarır. İkinci ifade, B değerini artırır ve yeni değeri C değişkenine aktarır. Atamaların tamamlanmasının ardından, C $A+1$ değerini içerir.

Şimdi iki ifadeyi engellemeyen atamalar olarak düşünün:

$$\begin{aligned} B &\leq A \\ C &\leq B + 1 \end{aligned}$$

İfadeler çalıştırıldığında, sağdaki ifadeler değerlendirilir ve geçici bir yerde saklanır. A değeri bir depolama alanında tutulur ve $tB+1$ değeri başka bir depolama alanında saklanır. Bloktaki tüm ifadeler değerlendirildikten ve saklandıktan sonra, soldaki hedeflere atama yapılır. Bu durumda, $C \leq B$ artı 1'in orijinal değerini içerecektir. Genel bir kural, ardışık sıralamanın zorunlu olduğu durumlarda ve seviyeye duyarlı döngüsel davranışta (yani, kombinasyonel mantıkta) engelleyici atamaları kullanmaktır. Eşzamanlı yürütmeyi modellemek için engellemeyen atamaları kullanın (örneğin, eşzamanlı, eşzamanlı kayıt transferleri gibi kenar duyarlı davranışlar) ve kilitlenmiş davranışı modellemek için kullanın. Engellemeyen atamalar, kayıt transfer seviyesi tasarımıyla başa çıkmada zorunludur, bu da Bölüm 8'de gösterilmiştir. Fiziksel donanımın ortak bir saatle senkronize edilmiş eşzamanlı işlemlerini modellemektedir. Günümüz tasarımcılarının, bir HDL'nin pratikte hangi özelliklerinin yararlı olduğunu ve hangi özelliklerden kaçınmaları gerektiğini bilmeleri beklenmektedir. Atama operatörlerini kullanma kurallarına uymak, sentez araçlarını yanıtlan koşulları önleyecek ve bir modelin davranışı ile bir sentez aracı tarafından üretilen fiziksel donanımın davranışı arasında uyumsuzluklar yaratacaktır.

Flip-Flop ve Latch'lerin HDL Modelleri

HDL Örnekleri 5.1'den 5.4'e kadar çeşitli flip-flop'ların ve bir D latch'in Verilog tanımlarını göstermektedir. D latch'in *şeffaf* olduğu söylenir çünkü enable girişi aktif olduğu sürece veri girişindeki bir değişikliğe, çıkışta bir değişiklik ile yanıt verir—çıkışı görmek, girişi görmekle aynıdır. D latch'in tanımı HDL Örneği 5.1'de gösterilmektedir. İki girişi vardır, D ve $enable$, ve bir çıkışı, Q . Q bir davranışta değer atandığı için, türü *reg* olarak beyan edilmelidir. Donanım latch'leri giriş sinyaline yanıt verir.

Seviyeleri, bu nedenle iki girişi, @ sembolünün ardından duyarlılık listesinde kenar nitelikleri olmadan listelenmiştir. Bu modelde yalnızca bir engelleyici prosedürel atama ifadesi vardır ve bu, giriş D'nin çıkışı Q'ya aktarımını belirtir eğer enable

doğruysa (mantık 1).¹ Bu ifadenin, D

'de bir değişiklik olduğunda her seferinde çalıştırıldığını unutmayın eğer *enable* 1 ise.

Bir D -tipi flip-flop, sıralı bir makinenin en basit örneğidir. HDL Örneği 5.2, iki modülde iki pozitif kenar D flip-flop'u tanımlar. İlk saat sinyaline yalnızca yanıt verir; ikincisi ise asenkron bir sıfırlama girişini içerir. Çıkış Q , bir çıkış olarak listelenmenin yanı sıra **reg** veri türü olarak beyan edilmelidir. Bunun nedeni, prosedürel atama ifadesinin hedef çıkışı olmasıdır. **posedge** anahtar kelimesi, giriş D 'nin Q 'ya aktarımının Clk 'nin pozitif kenar geçişiyle senkronize olmasını sağlar.

Başka bir zamanda D 'deki bir değişiklik Q 'yu değiştirmez.

HDL Örneği 5.1 (D -Latch)

```
// D latch'in tanımı (Bkz. Şekil 5.6)
modül D_latch (Q, D, enable);
    çıkış Q;
    girdi Denable;
    reg Q;
    her zaman @ (enable veya D)
        eğer (enable) Q <= D; // Aynı: // Aynı: eğer (enable == 1)
endmodule

// Alternatif sözdizimi (Verilog 2001, 2005)
module D_latch (output reg Q, input enable, D);
    always @(enable, D)
        if (enable) Q <= D; // Eğer etkin değilse hiçbir işlem yok // Eğer etkin değilse enable belirtilmemiş
endmodule
```

HDL Örneği 5.2 (D -Tipi Flip-Flop)

```
// D flip-flop sıfırlama olmadan
module D_FF (Q, D, Clk);
    output Q;
    input DClk;
    reg Q;
    always @posedge Clk
        Q <= D;
endmodule

// D flip-flop asenkron sıfırlama ile (V2001, V2005)
module DFF (output reg Q, input D, Clk, rst);
    always @posedge Clk, negedge rst
        if (!rst) Q <= 1'b0; // Aynı: eğer (rst == 0)
        else Q <= D;
endmodule
```

¹if (Boolean ifadesi) ile ilişkili olan ifade (tekil veya blok) Boolean ifadesi doğru olduğunda çalışır. doğrudur.

İkinci modül, senkronize saatin yanı sıra asenkron bir sıfırlama girişi içerir. Böyle bir flip-flop'u tanımlamak için belirli bir **if** ifadesi biçimi kullanılır, böylece model bir yazılım aracı tarafından sentezlenebilir. **always** ifadesindeki @ sembolünden sonraki olay ifadesi, herhangi bir sayıda kenar olayına sahip olabilir, ya **posedge** ya da **negedge**. Donanım modellemesi için, olaylardan biri bir saat olayı olmalıdır. Kalan olaylar, asenkron mantığın hangi koşullar altında çalıştırılacağını belirtir. Tasarımcı hangi sinyalin saat olduğunu bilir, ancak *clock* yazılım araçlarının otomatik olarak bir devrenin senkronize sinyali olarak tanıdığı bir tanımlayıcı değildir. Araç, hangi sinyalin saat olduğunu çıkarabilmelidir, bu nedenle *açıklamayı, aracın saati doğru bir şekilde çıkarmasını sağlayacak şekilde yazmalısınız*. Kurallar takip edilmesi kolaydır: (1) Her **if** veya **else if** p-rosedürel atama ifadelerindeki beyan, bir asenkron olaya karşılık gelmelidir, (2) son **else** beyanı saat olayına karşılık gelir ve (3) asenkron olaylar önce test edilir. HDL Örneği 5.2'nin ikinci modülünde iki kenar olayı vardır. **negedge** *rst* (sıfırlama) olayı asenkron olup, **if** (!*rst*) beyanına uyar. *rst* 0 olduğu sürece, *Q* 0'a temizlenir. Eğer *Clk* pozitif bir geçiş yaparsa, etkisi engellenir. Yalnızca *rst* = 1 olduğunda **posedge** saat olayı *D* değerini *Q* değerine senkronize bir şekilde aktarabilir.

Donanım her zaman bir sıfırlama sinyaline sahiptir. Tüm kenar duyarlı davranış modellerinin bir sıfırlama (veya ön ayar) giriş sinyali içermesi şiddetle önerilir; aksi takdirde, ardışık devrenin flip-flop'larının başlangıç durumu belirlenemez. Bir ardışık devre, bir giriş sinyali ile başlangıç durumu atanmadıkça HDL simülasyonu ile test edilemez.

HDL Örneği 5.3, bir D flip-flop ve kapılar kullanarak bir T veya JK flip-flop'un inşasını tanımlar. Devre, flip-flop'ların karakteristik denklemleri ile tanımlanmıştır:

$$\begin{aligned} Q(t+1) &= Q \oplus T && \text{for a } T \text{ flip-flop} \\ Q(t+1) &= JQ' + K'Q && \text{for a } JK \text{ flip-flop} \end{aligned}$$

İlk modül, TFF, bir T flip-flop'u DFF'yi örnekleyerek tanımlar. (Örnekleme, Bölüm 4.12'de açıklanmıştır.) Beyan edilen wire, DT, Q ve T'nin özel-VE'sine atanır; bu, bir D flip-flop ile bir T flip-flop inşa etmek için gereklidir. DT değerinin DFF modülündeki D'yi değiştirmesiyle yapılan örnekleme, gereken T flip-flop'u üretir. JK flip-flop'u, örneklenen DFF'deki D'nin yerine geçecek şekilde karakteristik denklemi kullanılarak benzer bir şekilde tanımlanır.

HDL Örneği 5.3 (Alternatif Flip-Flop Modelleri)

```
// T flip-flop D flip-flop ve kapılardan
modül TFF (Q, T, Clk, rst);
    çıkış Q;
    giriş T, Clk, rst;
    tel DT;

    atan DT = Q ^ T; // Sürekli atama // Sürekli atama
// D flip-flop'u örnekle
    DFF TF1 (Q, DT, Clk, rst);
endmodule
```



```
// JK fl ip-fl op D fl ip-fl op ve kapılardan (V2001, 2005)
modül JKFF ( çıkış reg Q, giriş J, K, Clk, rst);
    tel JK;
    atama JK = (J & ~Q) (~K & Q);
// D fl ip-fl op'u örnekle
DFF JK1 (Q, JK, Clk, rst);
endmodule

// D flip-flop (V2001, V2005)
module DFF (output reg Q, input D, Clk, rst);
    always @ (posedge Clk, negedge rst)
        if (!rst) Q <= 1'b0;
        else Q <= D;
endmodule
```

HDL Örneği 5.4, bir *JK* flip-flop'u tanımlamanın başka bir yolunu göstermektedir. Burada, flip-flop'u karakteristik denklemi yerine karakteristik tabloyu kullanarak tanımlıyoruz. **case** çoklu dal koşulu, *J* ve *K* bitlerinin birleştirilmesiyle elde edilen iki bitlik *s* ayıyı kontrol eder. **case** ifadesi (*J, K*) değerlendirilir ve ardından gelen ifadeler listesindeki değerlerle karşılaştırılır. Doğru koşulu karşılayan ilk değer yürütülür. *J* ve *K* birleştirildiğinde iki bitlik bir sayı üretir, bu nedenle 00, 01, 10 veya 11'e eşit olabilir. İlk bit *J* değerini, ikinci bit ise *K*. Dört olası durum, pozitif kenar saatinin uygulanmasından sonra *Q* durumunun sonraki değerini belirtir.

HDL Örneği 5.4 (*JK* Flip-Flop)

```
// JK flip-flop'un işlevsel tanımı (V2001, 2005)
modül JK_FF (input J, K, Clk, output reg Q, output Q_b);
    atama Q_b = ~ Q;
    her zaman @ ( posedge Clk)
        durum ({J,K})
            2'b00: Q <= Q;
            2'b01: Q <= 1'b0;
            2'b10: Q <= 1'b1;
            2'b11: Q <= !Q;
    endcase
endmodule
```

Durum diyagramına dayalı HDL Modelleri

Bir sıralı devrenin işleyişine dair bir HDL modeli, devrenin durum diyagramının formatına dayanabilir. Mealy HDL modeli, Şekil 5.15'te tanımlanan sıralı devre için HDL Örneği 5.5'te sunulmuştur ve durum diyagramı Şekil 5.16'da gösterilmiştir. Giriş, çıkış, saat ve sıfırlama, alışıldık şekilde tanımlanmıştır.

Flip-flop'ların durumu, *state* ve *next_state* tanımlayıcıları ile belirtilir. Bu değişkenler, mevcut durumun ve sıralı devrenin sonraki değerinin değerlerini tutar. Durumun ikili ataması, **parameter** ifadesi ile yapılır. (Verilog, sabitlerin bir modülde **parameter** anahtar kelimesi ile tanımlanmasına izin verir.) Dört durum *S0* ile *S3* arasında ikili 00 ile 11 olarak atanır. Notasyon $S2 = 2'b10$ alternatifine göre daha tercih edilir $S2 = 2$. İlk durumda sabiti saklamak için yalnızca iki bit kullanılırken, ikincisi, boyutsuz bir sayının bir tam sayı olarak yorumlanıp boyutlandırılması nedeniyle 32 (veya 64) bitlik bir ikili sayıya yol açar.

HDL Örneği 5.5 (Mealy Makinesi: Sıfır Dedektörü)

```
// Mealy FSM sıfır dedektörü (Bkz. Şekil 5.15 ve Şekil 5.16) Verilog 2001, 2005 sözdizimi
modül Mealy_Sıfır_Dedektörü (
    çıkış reg y_out,
    giriş x_in, saat, sıfır
);
    reg [1:0] durum, sonraki_durum;
    parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
    her zaman @ ( pozitif kenar saat, negatif kenar sıfır) Verilog 2001, 2005 sözdizimi Verilog 2001, 2005 sözdizimi
        if (reset == 0) state <= S0;
        else state <= next_state;

    always @ (state, x_in) // Sonraki durumu oluştur // Sonraki durumu oluştur
        case (state)
            S0: if (x_in) next_state = S1; else next_state = S0;
            S1: if (x_in) next_state = S3; else next_state = S0;
            S2: if (~x_in) next_state = S0; else next_state = S2;
            S3: if (x_in) next_state = S2; else next_state = S0;
        endcase

    always @ (state@x_in) // Mealy çıktısını oluştur // Mealy çıktısını oluştur
        case (durum)
            S0: y_out = 0;
            S1, S2, S3: y_out = ~x_in;
        endcase
endmodule

modül t_Mealy_Sıfır_Algılayıcı;
    tel t_y_out;
    reg t_x_in, t_clock, t_reset;

    Mealy_Sıfır_Algılayıcı M0 (t_y_out, t_x_in, t_clock, t_reset);
    ilk #200 $finish;
    ilk başla t_clock = 0; sonsuza dek #5 t_clock = ~t_clock; son

    ilk fork
        t_reset = 0;
        #2 t_reset = 1;
        #87 t_reset = 0;
        #89 t_reset = 1;
```

```

#10 t_x_in = 1;
#30 t_x_in = 0;
#40 t_x_in = 1;
#50 t_x_in = 0;
#52 t_x_in = 1;
#54 t_x_in = 0;
#70 t_x_in = 1;
#80 t_x_in = 1;
#70 t_x_in = 0;
#90 t_x_in = 1;
#100 t_x_in = 0;
#120 t_x_in = 1;
#160 t_x_in = 0;
#170 t_x_in = 1;
katil
endmodule

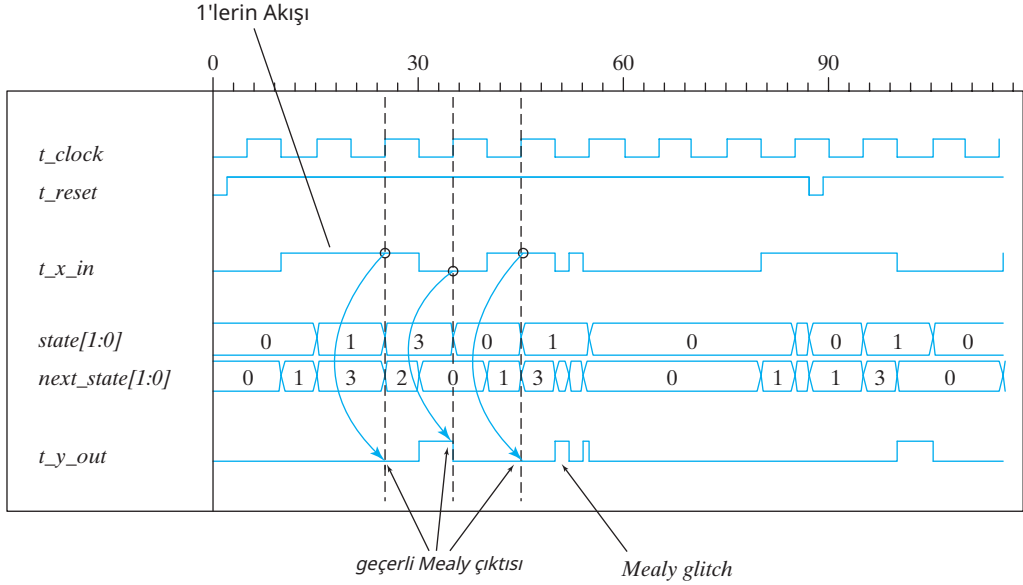
```

Devre I HDL Örneği 5.5, bir seri bit akışında 1'lerin ardından gelen 0'ı tespit eder. Verilog modeli, eşzamanlı olarak çalışan ve ortak değişkenler aracılığıyla etkileşime giren üç her zaman bloğu kullanır. İlk her zaman ifadesi devreyi başlangıç durumuna sıfırlar durumu $S0=00$ ve senkronize saatli işlemi belirtir. İfade $durum <=$

$next_state$ saatin pozitif kenar geçişine senkronize edilmiştir. Bu, ikinci *always* bloğunda ki $next_state$ değerindeki herhangi bir değişikliğin $state$ değerini yalnızca clock için bir *posedge* olayı sonucunda etkileyebileceği anlamına gelir. İkinci *always* bloğu, mevcut durum ve girdi fonksiyonu olarak bir sonraki durum geçişinin değerini belirler. $state$ için nonblocking atama ile atanan değer, $clock$ yükselen kenarından hemen önceki $next_state$ değeridir. Çok yönlü dal koşulunun, Şekil 5.16'daki durum diyagramında belirtilen durum geçişlerini nasıl uyguladığını gözlemleyin.

Üçüncü her zaman blok, çıktıyı mevcut duruma ve girdiye bağlı olarak tanımlar. Bu blok, açıklık için ayrı bir davranış olarak listelenmiş olsa da, ikinci blokla birleştirilebilir. Dikkat edin ki, çıkışın y_out değeri, devre herhangi bir durumda iken girdi x_in değeri değişirse değişebilir.

Şimdi modelin makinenin davranışını nasıl tanımladığını özetleyelim: Her *clock* yükselen kenarında, *reset* belirtilmemişse, makinenin durumu birinci her zaman bloğu tarafından güncellenir; birinci her zaman bloğu tarafından durum güncellendiğinde, durumdaki değişiklik ikinci her zaman bloğunun hassasiyet listesi mekanizması tarafından algılanır; ardından ikinci her zaman bloğu $next_state$ değerini günceller (bu, birinci her zaman bloğu tarafından saatin bir sonraki tikiinde kullanılacaktır); üçüncü her zaman bloğu da durumdaki değişikliği algılar ve çıkışın değerini günceller. Ayrıca, ikinci ve üçüncü her zaman blokları x_in değişikliklerini algılar ve $next_state$ ve y_out değerlerini buna göre günceller. Mealy_Zero_Detector ile sağlanan test tezgahı, modeli uyarmak için bazı dalga formları sağlar ve Şekil 5.22'de gösterilen sonuçları üretir. t_y_out değerinin hem durum hem de girdi değişikliklerine nasıl tepki verdiğini ve bir hata (geçici mantık değeri) içerdiğini gözlemleyin. $state [1:0]$ ve $next_state [1:0]$ değerlerini göstererek, t_x_in değişikliklerinin $next_state$ ve t_y_out değerini nasıl etkilediğini gösteriyoruz. t_y_out içindeki Mealy hatası, t_x_in 'in (kasıtlı) dinamik davranışından kaynaklanmaktadır. Girdi, t_x_in , yerleşir



ŞEKİL 5.22
Simülasyon çıktısı *Mealy_Zero_Detector*

Saatten hemen önce 0 değerine ve saatte, durum 0'dan 1'e geçiş yapar, bu da Şekil 5.16 ile tutarlıdır. Çıkış, saatten hemen önce S 1 durumunda 1'dir ve durumu S 0'a girdiğinde 0'a değişir.

Test tezgahındaki dalga formlarının tanımı **fork ... join** yapısını kullanır. **fork ... join** bloğundaki durum ifadeleri paralel olarak çalışır, bu nedenle zaman gecikmeleri $t = 0$ olan ortak bir referansa göredir; bu, bloğun çalışmaya başladığı zamandır.² Dalga formlarını tanımlarken **fork ... join** bloğunu kullanmak genellikle daha uygundur. Sıfırlama dalga formunun "anında" tetiklendiğini ve makinenin herhangi bir durumda beklenmedik (asenkron) sıfırlama koşulundan kurtulduğunu göstermek için tetiklendiğini unutmayın.

Verilog modelimiz *Mealy_Zero_Detector* donanıma nasıl karşılık geliyor? İlk **always** bloğu, Şekil 5.21'deki durum kaydının D flip-flop uygulamasına karşılık gelir; ikinci **always** bloğu, bir sonraki durumu tanımlayan kombinasyonel mantık bloğudur; üçüncü **always** bloğu, sıfır tespit eden Mealy makinesinin çıkış kombinasyonel mantığını tanımlar. Durum geçişinin kayıt işlemi, sıralı bir makinenin (kenar hassas) flip-flop'larının ortak bir saat tarafından eşzamanlı olarak güncellendiği için, nonblocking atama operatörünü ($<=$) kullanır. İkinci ve üçüncü **always** blokları, seviye hassas olan kombinasyonel mantığı tanımlar, bu nedenle blocking ($=$) atama operatörünü kullanırlar.

²A **fork ... join** bloğu, içindeki son yürütme ifadesi tamamlandığında yürütmeyi tamamlar. Yürütme.

Hassasiyet listeleri, mantıklarının her ikisine veya birine değişikliklere yanıt vermesi gerektiğinden, hem durumu hem de girişi içerir.

Not: *Mealy_Zero_Detector* tarafından gösterilen modelleme tarzı, tanımlanan makinenin durum diyagramı ile yakın bir ilişkiye sahip olduğu için tasarımcılar tarafından yaygın olarak kullanılmaktadır. Sıfırlama sinyalinin, durum geçişlerini senkronize eden **always** bloğu ile ilişkili olduğunu unutmayın. Bu örnekte, aktif-düşük sıfırlama olarak modellenmiştir. Sıfırlama koşulu, durum geçişlerinin tanımında yer aldığından, bir sonraki durumu ve çıktığı belirten kombinasyonel mantıkta dahil edilmesine gerek yoktur ve sonuçta elde edilen tanım daha az ayrıntılı, daha basit ve daha okunabilir hale gelir.

HDL Örneği 5.6, Şekil 5.18'de gösterilen Moore FSM'nin Verilog davranış modelini sunmaktadır ve durum diyagramı Şekil 5.19'da verilmiştir. Model, makinenin durum geçişlerinin tek bir saatli (yani, kenar duyarlı) döngüsel davranışla, yani bir her zaman bloğuyla tanımlandığı alternatif bir tarzı göstermektedir. Devrenin mevcut durumu, state değişkeni ile tanımlanır ve geçişleri, **case** ifadesinde listelenen koşullara göre saat sinyalinin yükselen kenarı tarafından tetiklenir. Bir sonraki durumu belirleyen kombinasyonel mantık, state'e yapılan engellenmeyen atama içinde yer almaktadır. Bu örnekte, devrelerin çıktısı girdi ile bağımsızdır ve doğrudan flip-flop'ların çıkışlarından alınmaktadır. İki bitlik çıkış *y_out* sürekli bir atama ifadesi ile belirtilmiştir ve mevcut durum vektörünün değerine eşittir. Şekil 5.23, *Moore_Model_Fig_5_19* için bazı simülasyon sonuçlarını göstermektedir. İşte bazı önemli gözlemler: (1) çıkış yalnızca duruma bağlıdır, (2) "anında" sıfırlama, makinenin durumunu S0 (00) durumuna geri döndürür ve (3) durum geçişleri Şekil 5.19 ile tutarlıdır.

HDL Örneği 5.6 (Moore Makinesi: Sıfır Dedektörü)

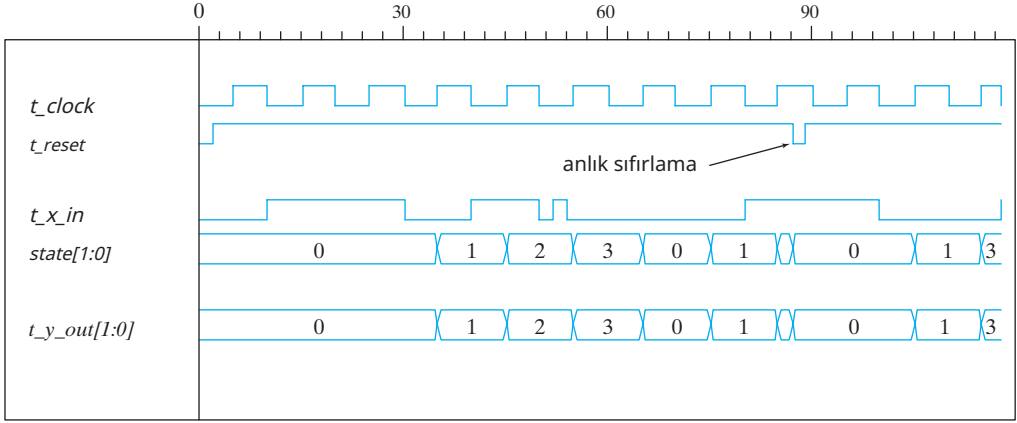
```
// Moore modeli FSM (bkz. Şekil 5.19 )                               Verilog 2001, 2005 sözdizimi
modül Moore_Model_Fig_5_19 (
    y_output [1: 0]      y_out,      y_out,
    input                x_in, saat, sıfırla
);
    reg [1: 0] durum;          durum;
    parametre                S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;

    her zaman @( ( yükselen kenar saat, düşen kenar sıfırla)
        eğer (sıfırla == 0) durum <= S0; // Durum S0'a başlat           // Başlangıç durumu S0 olarak ayarla
        aksi takdirde durum (durum)

    S0:      eğer (~x_in) durum <= S1; aksi takdirde durum <= S0;
    S1:      eğer (x_in) durum <= S2; aksi takdirde durum <= S3;
    S2:      eğer (~x_in) durum <= S3; aksi takdirde durum <= S2;
    S3:      eğer (x_in) durum <= S0; aksi takdirde durum <= S3;
    endcase

    atama y_out = durum; // flip-flop'ların çıktısı           // flip-flop'ların çıktısı

endmodule
```



ŞEKİL 5.23
HDL Örneği 5.6'nın simülasyon çıktısı

Saatli Sıralı Devrelerin Yapısal Tanımı

Kombinasyonel mantık devreleri, kapıların (primitifler ve UDP'ler) bağlantısı, veri akışı ifadeleri (sürekli atamalar) veya seviye duyarlı döngüsel davranışlar (always blokları) ile Verilog'da tanımlanabilir. Ardışık devreler, kombinasyonel mantık ve flip-flop'lardan oluşur ve HDL modelleri, flip-flop'ların çalışmasını tanımlamak için ardışık devreler ve davranışsal ifadeler (kenar duyarlı döngüsel davranışlar) kullanır. Bir ardışık devreyi tanımlamanın bir yolu, veri akışı ve davranışsal ifadelerin bir kombinasyonunu kullanmaktır. Flip-flop'lar, bir always ifadesi ile tanımlanır. Kombinasyonel kısım, assign ifadeleri ve Boolean denklemleri ile tanımlanabilir. Ayrıca modüller, bir modül içinde örnekleme yoluyla bir yapısal model oluşturmak için birleştirilebilir.

Bir Moore tipi sıfır dedektörü ardışık devresinin yapısal tanımı HDL Örneği 5.7'de gösterilmiştir. Okuyucuyu bir devreyi modellemenin alternatif yollarını düşünmeye teşvik etmek istiyoruz, bu nedenle karşılaştırma noktası olarak önce Moore_Model_Fig_5_20'yi sunuyoruz, bu, daha önce Fig. 5.20 (b)'de incelenen durum diyagramına sahip bir ikili sayacın Verilog davranışsal tanımıdır. Bu modelleme tarzı doğrudan durum diyagramından gelmektedir. Alternatif bir tarz, Moore_Model_STR_Fig_5_20'de kullanılmıştır ve Fig. 5.20 (a)'de gösterilen yapıyı temsil etmektedir. Bu tarz iki modül kullanır. İlki, Fig. 5.20 (a)'deki devreyi tanımlar. İkincisi, devre tarafından kullanılacak T flip-flop'u tanımlar. Ayrıca T flip-flop'u modellemenin iki yolunu da gösteriyoruz. İlk model, her saat tikinde, flip-flop'un çıkış değerinin, toggle girişi etkinleştirildiğinde değeri değiştiğini belirtir. İkinci model, toggle flip-flop'un davranışını karakteristik denklemi açısından tanımlar. İlk tarz, okuyucunun karakteristik denklemi hatırlamasını gerektirmediği için caziptir. Yine de, modeller birbirinin yerine geçebilir ve aynı donanım devresine sentezlenir. Bir test tezgahı modülü, devrenin işlevselliğini doğrulamak için bir uyarı sağlar. Ardışık devre, giriş x_{in} tarafından kontrol edilen iki bitlik bir ikili sayacıdır. Çıkış, y_{out} , sayım ikili 11'e ulaştığında etkinleştirilir. Flip-flop'lar

A ve B işlevlerini kontrol etmek için çıkışlar olarak dahil edilmiştir. Flip-flop giriş eşitlikleri ve çıkış eşitliği, ilgili Boolean ifadelerine sahip sürekli atama (**assign**) bildirimleri ile değerlendirilir. Oluşturulan T flip-flopları, giriş eşitlikleri tarafından tanımlanan TA ve TB kullanır.

İkinci modül, T flip-flopunu tanımlar. *reset* girişi, flip-flop 0'a sıfırlar aktif düşük sinyal ile. Flip-flopun işlevi, karakteristik eşitliği ile belirtilir, $Q(t+1)=Q \oplus T$.

Test tezgahı, makinenin her iki modelini de içerir. Uyarı modülü, devrelere ortak girişler sağlar çıkış tepkilerini aynı anda göstermek için. İlk **initial** bloğu, 10 ns periyoduna sahip sekiz saat döngüsü sağlar. İkinci **initial** bloğu, saat sinyalinin negatif kenar geçişinde meydana gelen giriş x_{in} değişimini belirtir. Simülasyonun sonucu Şekil 5.24'te gösterilmektedir. (A, B) çifti binary dizisi 00, 01, 10, 11 üzerinden geçer ve tekrar 00'a döner. Sayımdaki değişim, saat sinyalinin pozitif kenarı ile tetiklenir, eğer $x_{in}=1$. Eğer $x_{in}=0$, sayım değişmez. y_{out} hem A hem de B 1'e eşit olduğunda 1'dir. Bu, devrenin ana işlevselliğini doğrular, ancak beklenmeyen bir sıfırlama olayından kurtulmayı doğrulamaz.

HDL Örneği 5.7 (İkili Sayıcı_Moore Modeli)

```
// Durum diyagramına dayalı model (V2001, 2005)
modül Moore_Model_Fig_5_20 (
    çıkış y_out,
    girdi x_in, saat, sıfırla
);
    reg [1:0] durum;          durum;
    parametre                S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;

    her zaman @ (pozitif kenar saat, negatif kenar sıfırla)
        eğer (sıfırla == 0) durum <= S0; // Durum S0'a başlat // Başlangıç durumu S0 olarak ayarla
        aksi takdirde durum (durum)
            S0: eğer (x_in) durum <= S1; aksi takdirde durum <= S0;
            S1: eğer (x_in) durum <= S2; aksi takdirde durum <= S1;
            S2: if (x_in) state <= S3; else state <= S2;
            S3: if (x_in) state <= S0; else state <= S3;
        endcase

    assign y_out = (state == S3); // Flip-flop çıkışı // flip-flopların çıktısı
endmodule

// Yapısal model
module Moore_Model_STR_Fig_5_20 (
    output y_out, A, B,
    input x_in, saat, sıfırla
);
    wire TA,
    wire TB,

    // Flip-flop giriş denklemleri
    assign TA = x_in & B;
```

```

    assign TB = x_in;
// Çıktı denklemi
    assign y_out = A & B;

// Toggle flip-flop'ları başlat
    Toggle_flip_flop_3 M_A (A, TA, clock, reset);
    Toggle_flip_flop_3 M_B (B, TB, clock, reset);
endmodule

module Toggle_flip_flop (Q, T, CLK, RST_b);
    output      Q;
    input       T, CLK, RST_b;
    reg         Q;

    always @(posedge CLK, negedge RST_b)
        if (RST_b == 0) Q <= 1'b0;
        else if(T) Q <= ~Q;
endmodule

// Karakteristik denklemi kullanarak alternatif model
// module Toggle_flip_flop (Q, T, CLK, RST_b);
// output Q;
// girdi T, CLK, RST_b;
// reg Q;

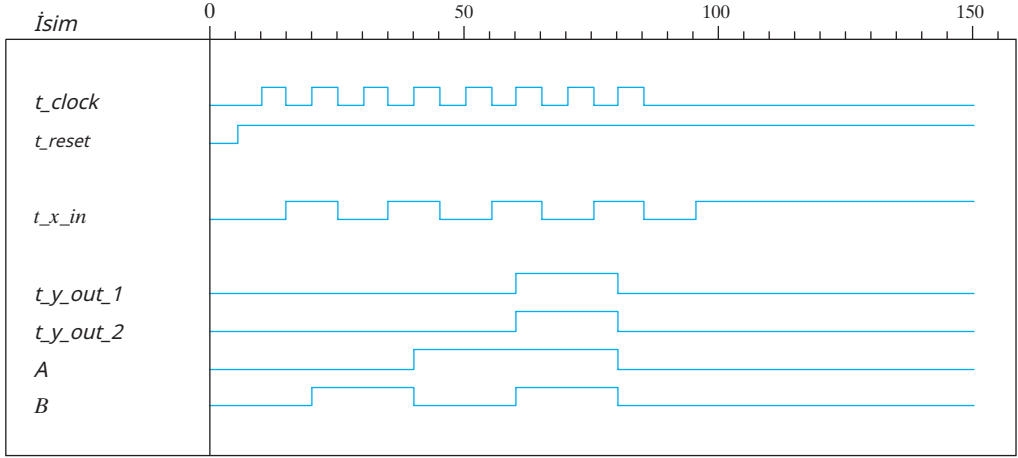
// her zaman @ (posedge CLK, negedge RST)
// eğer (RST_b == 0) Q <= 1'b0;
// aksi takdirde Q <= Q ^ T;
// endmodule

modül t_Moore_Fig_5_20;
    tel t_y_out_2, t_y_out_1;
    reg t_x_in, t_clock, t_reset;

    Moore_Model_Fig_5_20 M1(t_y_out_1, t_x_in, t_clock, t_reset);
    Moore_Model_STR_Fig_5_20 M2(t_y_out_2, A, B, t_x_in, t_clock, t_reset);

    initial #200 $finish;
    initial begin
        t_reset = 0;
        t_clock = 0;
        #5 t_reset = 1;
        repeat (16)
            #5 t_clock = ~t_clock;
        end
    initial begin
        t_x_in = 0;
        #15 t_x_in = 1;
        repeat (8)
            #10 t_x_in = ~t_x_in;
        end
    endmodule

```



ŞEKİL 5.24
HDL Örneği 5.7'nin simülasyon çıktısı

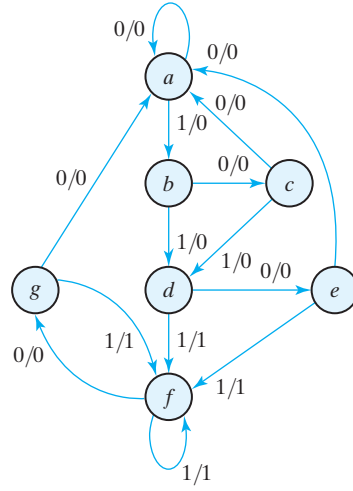
5.7 DURUM AZALTMA VE ATAMA

Sequential devrelerin analizi, bir devre şemasından başlar ve bir durum tablosu veya diyagramı ile sona erer. Sequential bir devrenin tasarımı (sentezi), bir dizi spesifikasyondan başlar ve bir mantık diyagramı ile sona erer. Tasarım prosedürleri Bölüm 5.8'de sunulmuştur. İki sequential devre aynı giriş-çıkış davranışını sergileyebilir, ancak durum diyagramlarında farklı sayıda iç duruma sahip olabilirler. Mevcut bölüm, bir tasarımı basit eştirebilecek bazı sequential devre özelliklerini tartışmaktadır; bu, kullanılan kapı ve flip-flop sayısını azaltarak mümkündür. Genel olarak, flip-flop sayısını azaltmak, bir devrenin maliyetini düşürür.

Durum Azaltma

Bir ardışık devredeki terlik sayısındaki azalma, *durum azaltma* problemi. Durum azaltma algoritmaları, dış girdi-çıkış gereksinimlerini değiştirmeden, bir durum tablosundaki durum sayısını azaltma prosedürleriyle ilgilendirir. m flip-flop'lar 2 durum üretir, durum sayısındaki bir azalma, flip-flop sayısında bir azalmaya (veya olmayabilir) yol açabilir. Flip-flop sayısını azaltmanın öngörülebilir bir etkisi, bazen daha az flip-flop'a sahip eşdeğer devrenin, bir sonraki durumu ve çıkış mantığını gerçekleştirmek için daha fazla kombinasyonel kapı gerektirebilmesidir.

Bir örnekle durum azaltma prosedürünü açıklayacağız. Bir ardışık devre ile başlıyoruz, spesifikasyonu Şekil 5.25'teki durum diyagramında verilmektedir. Örneğimizde, yalnızca giriş-çıkış dizileri önemlidir; iç durumlar yalnızca gerekli dizileri sağlamak için kullanılır. Bu nedenle, dairelerin içindeki durumlar, ikili değerleri yerine harf sembolleri ile gösterilmektedir.



ŞEKİL 5.25

Durum diyagramı

Bu, durumların kendilerinin ikili değer dizisinin çıktıları olarak alındığı bir ikili sayıcı ile tezat oluşturmaktadır.

Devreye uygulanabilecek sonsuz sayıda giriş dizisi vardır; her biri benzersiz bir çıkış dizisi ile sonuçlanır. Örnek olarak, başlangıç durumu a 'dan başlayan 010101 10100 giriş dizisini ele alalım. 0 veya 1 girişinin her biri, 0 veya 1 çıkışı üretir ve devrenin bir sonraki duruma geçmesine neden olur. Durum diyagramından, verilen giriş dizisi için çıkış ve durum dizisini şu şekilde elde ederiz: Devre başlangıç durumu a 'da iken, 0 girişi 0 çıkışı üretir ve devre a durumunda kalır.

Mevcut durum a 'da ve 1 girişi ile çıkış 0'dır ve bir sonraki durum b 'dir. Mevcut durum b 'da ve 0 girişi ile çıkış 0'dır ve bir sonraki durum c 'dir. Bu süreci devam ettirerek, tam dizinin şu şekilde olduğunu buluyoruz:

durum	a	a	b	c	d	e	f	f	g	f	g	a
input	0	1	0	1	0	1	1	0	1	0	0	
çıkış	0	0	0	0	0	1	1	0	1	0	0	

Her sütunda mevcut durum, giriş değeri ve çıkış değeri bulunmaktadır. Bir sonraki durum bir sonraki sütunun üstüne yazılmıştır. Bu devrede durumların kendisinin ikincil öneme sahip olduğunu anlamak önemlidir, çünkü yalnızca giriş dizileri tarafından neden olunan çıkış dizileriyle ilgileniyoruz.

Şimdi, yedi durumdan daha az duruma sahip bir ardışık devre bulduğumuz varsayalım ve bu devreyi Şekil 5.25 ile verilen devre ile karşılaştırmak istediğimizi düşünelim. İki devreye aynı giriş dizileri uygulanırsa ve tüm giriş dizileri için aynı çıkışlar oluşursa, o zaman iki devre eşdeğer olarak kabul edilir (giriş-çıkış açısından) ve biri diğeriyle değiştirilebilir. Durum azaltma problemi, giriş-çıkış ilişkilerini değiştirmeden bir ardışık devredeki durum sayısını azaltmanın yol arını bulmaktır.

Tablo 5.7

Durum Tablosunun Azaltılması

Mevcut Durum	Sonraki Durum		Çıktı	
	$x=0$	$x=1$	$x=0$	$x=1$
a	b	d	0	0
b	d	d	0	0
c	d	d	0	0
d	f	f	0	1
e	f	f	0	1
f	f	f	0	1

Tablo 5.8

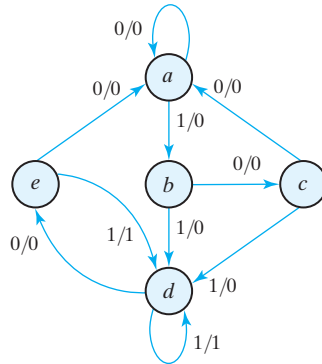
Azaltılmış Durum Tablosu

Mevcut Durum	Sonraki Durum		Çıktı	
	$x=0$	$x=1$	$x=0$	$x=1$
a	b	d	0	0
b	d	d	0	0
c	d	d	0	0
d	d	d	0	1
e	d	d	0	1

Aslında, bu dizilim, g yerine koyarsak, Şekil 5.25 için elde edilenle tam olarak aynıdır.

e ve f tarafından d .

Her bir durum çiftinin eşdeğerliğini kontrol etmek, her bir şüpheli eşdeğer durum çifti için bir kare içeren bir çıkarım tablosu kullanarak sistematik bir şekilde yapılabilir. Tabloyu dikkatli bir şekilde kullanarak, bir durum tablosundaki tüm eşdeğer durum çiftlerini belirlemek mümkündür.



ŞEKİL 5.26

Azaltılmış durum diyagramı

Bölüm 5.7 Durum Azaltma ve Atama 235

Bu örneğin ardışık devresi yedi durumdan beş duruma indirgenmiştir. Genel olarak, bir durum tablosundaki durum sayısını azaltmak, daha az ekipmana sahip bir devre ile sonuçlanabilir. Ancak, bir durum tablosunun daha az duruma indirgenmiş olması, flip-flop sayısının da veya kapı sayısında bir tasarruf sağlanacağı anlamına gelmez. Gerçek uygulamada tasarımcılar, hedef cihazların kaynak açısından zengin olması nedeniyle bu adımı atlayabilirler.

Durum Ataması

Fiziksel bileşenlerle bir ardışık devre tasarlamak için, durumlara benzersiz kodlanmış ikili değerler atamak gereklidir. m durumuna sahip bir devre için, kodlar n bit içermelidir, burada $2 \geq m$. Örneğin, üç bit ile sekiz duruma kod atamak mümkündür; bu durumlar ikili sayılar 000'dan 111'e kadar gösterilir. Eğer Tablo 5.6'nın durum tablosu kullanılıyorsa, yedi duruma ikili değerler atamalıyız; kalan durum kullanılmamıştır. Eğer Tablo 5.8'in durum tablosu kullanılıyorsa, yalnızca beş duruma ikili atama yapılması gerekmekte ve üç durum kullanılmamış kalmaktadır. Kullanılmayan durumlar tasarım sırasında dikkate alınmayan koşullar olarak ele alınır. Dikkate alınmayan koşullar genellikle daha basit bir devre elde etmeye yardımcı olduğundan, beş duruma sahip devrenin yedi duruma sahip olandan daha az kombinasyonel kapı gerektirmesi daha olasıdır, ancak bu kesin değildir.

Beş durumu kodlamanın en basit yolu, ikili sayma sırasındaki ilk beş tam sayıyı kullanmaktır, bu durum Tablo 5.9'un ilk atamasında gösterilmiştir. Benzer bir atama, atama 2'de gösterilen Gray kodudur. Burada, bir sayıdan diğerine geçerken kod grubundaki yalnızca bir bit değişir. Bu kod, Boolean fonksiyonlarının basitleştirme için harita üzerinde yerleştirilmesini kolaylaştırır. Veri yolu birimlerini kontrol etmek için durum makinelerinin tasarımında sıkça kullanılan bir diğer olası atama ise one-hot atamasıdır. Bu yapılandırma, devredeki durum sayısı kadar bit kullanır. Belirli bir anda, yalnızca bir bit 1'e eşitken, diğer tüm bitler 0'da tutulur. Bu tür bir atama, her durum için bir flip-flop kullanır; bu, kayıt açısından zengin alan programlanabilir kapı dizileri için bir sorun değildir. (Bkz. Bölüm 7.) *One-hot kodlama genellikle bir sonraki durum ve çıkış için daha basit bir çözümleme mantığına yol açar.* One-hot makineleri, ardışık ikili kodlama ile çalışan makinelerden daha hızlı olabilir ve ek flip-flop'lar için gereken silikon alanı, alanla dengelenebilir.

Tablo 5.9
Üç Olası İkili Durum Ataması

Durum	Atama 1, İkili	Atama 2, Gray Kodu	Atama 3, One-Hot
<i>a</i>	000	000	00001
<i>b</i>	001	001	00010
<i>c</i>	010	011	00100
<i>d</i>	011	010	01000
<i>e</i>	100	110	10000

Tablo 5.10
İkili Atama 1 ile Azaltılmış Durum Tablosu

Mevcut Durum	Sonraki Durum		Çıktı	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
000	000	001	0	0
001	010	011	0	0
010	000	011	0	0
011	100	011	0	1
100	000	011	0	1

daha basit kod çözme mantığı kullanarak kurtarıldı. Bu denge garanti edilmez, bu nedenle belirli bir tasarım için değerlendirilmelidir.

Tablo 5.10 durumların harf sembolleri için 1 ikili atama ile değiştirilmiş azaltılmış durum tablosudur. Farklı bir atama, durumlar için farklı ikili değerlerle bir durum tablosu ile sonuçlanacaktır. Durum tablosunun ikili biçimi, sıralı devrenin bir sonraki durumunu ve çıkış oluşturan kombinasyonel mantık kısmını türetmek için kullanılır. Kombinasyonel devrenin karmaşıklığı, seçilen ikili durum atamasına bağlıdır.

Bazen, *geçiş tablosu* ikili atama ile bir durum tablosu için kullanılır. Bu gelenek, onu durumlar için sembolik isimler taşıyan bir durum tablosundan ayırır. Bu kitapta, her iki tür durum tablosu için aynı ismi kullanıyoruz.

5.8 TASARIM PROSEDÜRÜ

Tasarım prosedürleri veya metodolojileri, istenen bir davranışı uygulayacak donanımı belirtir. Küçük devreler için tasarım çabası manuel olabilir, ancak sanayi, büyük entegre devrelerin tasarımı için otomatik sentez araçlarına güvenmektedir. Sentez araçları tarafından kullanılan ardışık yapı taşı, D flip-flop'tur. Ek mantık ile birlikte, JK ve T flip-flop'ların davranışını uygulayabilir. Aslında, tasarımcılar genellikle flip-flop türü ile ilgilenmez; daha ziyade, sentez aracı tarafından uygulanacak ardışık işlevsellik doğru bir şekilde tanımlamaya odaklanırlar. Burada, D , JK , ve T flip-flop'ları kullanarak manuel yöntemleri göstereceğiz.

Bir saatli ardışık devrenin tasarımı, bir dizi spesifikasyondan başlar ve mantık diyagramı veya mantık diyagramının elde edilebileceği bir Boolean fonksiyonları listesi ile sona erer. Tamamen bir doğruluk tablosu ile tanımlanan kombinasyonel devrenin aksine, bir ardışık devre, spesifikasyonu için bir durum tablosuna ihtiyaç duyar. Ardışık devrenin tasarımındaki ilk adım, bir durum tablosu veya bir durum diyagramı gibi eşdeğer bir temsil elde etmektir.³

A synchronous sequential circuit, flip-flop'lar ve kombinasyonel kapılardan oluşur. döngü tasarımı, flip-flop'ları seçmek ve ardından bir kombinasyonel

³ Daha sonra bir makinenin davranışının başka önemli bir temsiliyi inceleyeceğiz—algoritmik durum makineleri (ASM) şeması.

ngü yapısı, flip-flop'larla birlikte belirtilen yapılandırmaları yerine getiren bir devre üreten bir kombinasyonel kapıdır. Flip-flop sayısı, devrede gerekli olan durum sayısından ve durum atama kodlarının seçiminden belirlenir. Kombinasyonel devre, flip-flop giriş denklemlerini ve çıkış denklemlerini değerlendirerek durum tablosundan türetilir. Aslında, flip-flop'ların türü ve sayısı belirlendikten sonra, tasarım süreci, bir sıralı devre probleminden bir kombinasyonel devre problemine dönüşümünü içerir. Bu şekilde, kombinasyonel devre tasarım teknikleri uygulanabilir.

Senkron sıralı devrelerin tasarım prosedürü, önerilen adımların bir listesiyle özetlenebilir:

- İstenilen işlemin kelime tanımından ve spesifikasyonlarından, devre için bir durum diyagramı çıkarın.
- Gerekirse durum sayısını azaltın.
- Durumlara ikili değerler atayın.
- İkili kodlu durum tablosunu elde edin.
- Kullanılacak flip-flop türünü seçin.
- Sadeleştirilmiş flip-flop giriş denklemlerini ve çıkış denklemlerini türetin.
7. Mantık diyagramını çizin.

Devre davranışının kelime spesifikasyonu genellikle okuyucunun dijital mantık terminolojisine aşina olduğunu varsayar. Tasarımcının devre spesifikasyonlarının doğru yorumuna ulaşmak için sezgi ve deneyim kullanması gereklidir, çünkü kelime tanımları eksik ve kesin olmayabilir. Böyle bir spesifikasyon belirlendikten ve durum diyagramı elde edildikten sonra, tasarımı tamamlamak için bilinen sentez prosedürleri kullanılabilir. Durum azaltma ve atama için resmi prosedürler (adım 2 ve 3) olmasına rağmen, deneyimli tasarımcılar tarafından nadiren kullanılır. Tasarımın 4 ile 7 arasındaki adımlar kesin algoritmalarla uygulanabilir ve bu nedenle otomatikleştirilebilir. Tasarıma iyi tanımlanmış bir prosedürü takip eden kısma sentez denir. Mantık sentez araçları (yazılım) kullanan tasarımcılar, bir durum diyagramından doğrudan bir HDL tanımı geliştiren basitleştirilmiş bir süreci takip edebilirler ve sentez aracı tanımı uygulayan devre elemanlarını ve yapısını belirler.

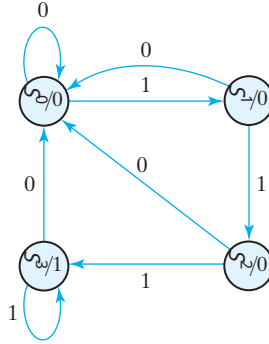
İlk adım, sürecin kritik bir parçasıdır, çünkü sonraki adımlar buna bağlıdır. Bir durum diyagramının nasıl elde edildiğini göstermek için basit bir örnek vereceğiz.

Bir giriş hattından (yani, giriş bir *seri bit akışı*

akışı) gelen bit dizisinde üç veya daha fazla ardışık 1'i tespit eden bir devre tasarlamak istediğimizi varsayalım. Bu tür bir devre için durum diyagramı Şekil 5.27'de gösterilmiştir. Durum S_0 , sıfırlama durumu ile başlanarak türetilmiştir. Giriş 0 ise, devre S_0 durumunda kalır, ancak giriş 1 ise, 1'in tespit edildiğini göstermek için S_1 durumuna geçer. Bir sonraki giriş 1 ise, iki ardışık 1'in geldiğini göstermek için durum S_2 'ye geçilir, ancak giriş 0 ise, durum S_0 'a geri döner. Üçüncü ardışık 1, devreyi S_3 durumuna gönderir. Daha fazla 1 tespit edilirse, devre S_3 durumunda kalır. Herhangi bir 0 girişi devreyi S_0 'a geri gönderir. Bu şekilde, devre üç veya daha fazla ardışık 1 alındığı sürece S_3 durumunda kalır.

Bu, devre S durumundayken çıkışın 1 olduğu bir Moore modeli ardışık devresidir.

S_3 ve aksi takdirde 0'dır.



ŞEKİL 5.27
İkili algılayıcı için durum diyagramı

Sentez Kullanarak D Flip-Flop'lar

Durum diyagramı çıkarıldıktan sonra, tasarımın geri kalanı doğrudan bir sentez prosedürünü takip eder. Aslında, devreyi durum diyagramının HDL açıklamasını ve uygun HDL sentez araçlarını kullanarak bir sentezlenmiş netlist elde etmek için tasarlatabiliriz. (Durum diyagramının HDL açıklaması, Bölüm 5.6'daki HDL Örneği 5.6'ya benzer olacaktır.) Devreyi elle tasarlamak için, durumlara ikili kodlar atamamız ve durum tablosunu listelememiz gerekir. Bu, Tablo 5.11'de yapılmıştır. Tablo, Şekil 5.27'deki durum diyagramından sıralı ikili atama ile türetilmiştir.

İki D flip-flop seçiyoruz ve dört durumu temsil etmek için çıkışlarını etiketliyoruz.

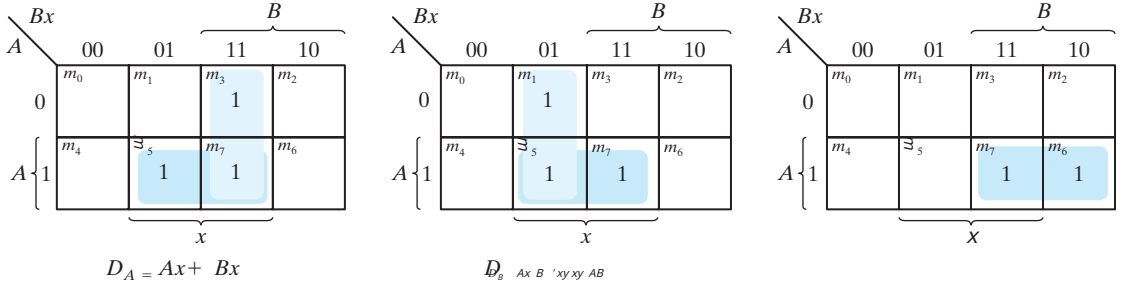
A ve B . Bir girdi x ve bir çıktı y vardır. Özellik denklemi

D flip-flop, $Q(t+1)=D_Q$, bu, durum

tablodaki bir sonraki durum değerlerinin flip-flop için D giriş koşulunu belirlediği anlamına gelir. Flip-flop giriş denklemleri

Tablo 5.11
İkili Algılayıcı için Durum Tablosu

Mevcut Durum		Giriş	Sonraki Durum		Çıktı
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	0
0	1	1	1	0	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	1	1	1	1



ŞEKİL 5.28

Dizi dedektörü için K-Haritaları

doğrudan A ve B 'nin bir sonraki durum sütunlarından elde edilebilir ve

sum-of-minterms biçiminde ifade edilebilir

$$A(t + 1) = D_A(A, B, x) = \Sigma(3, 5, 7)$$

$$B(t + 1) = D_B(A, B, x) = \Sigma(1, 5, 7)$$

$$y(A, B, x) = \Sigma(6, 7)$$

burada A ve B flip-flop'ların mevcut durum değerleridir, x giriştir ve D_A ve D_B giriş denklemleridir. Çıkış için minterms y durum tablosundaki çıkış sütunundan elde edilir.

Boolean denklemleri, Şekil 5.28'de çizilen haritalar aracılığıyla basitleştirilmiştir.

basitleştirilmiş denklemler şunlardır

$$D_A = Ax + Bx$$

$$D_B = Ax + B'x$$

$$y = AB$$

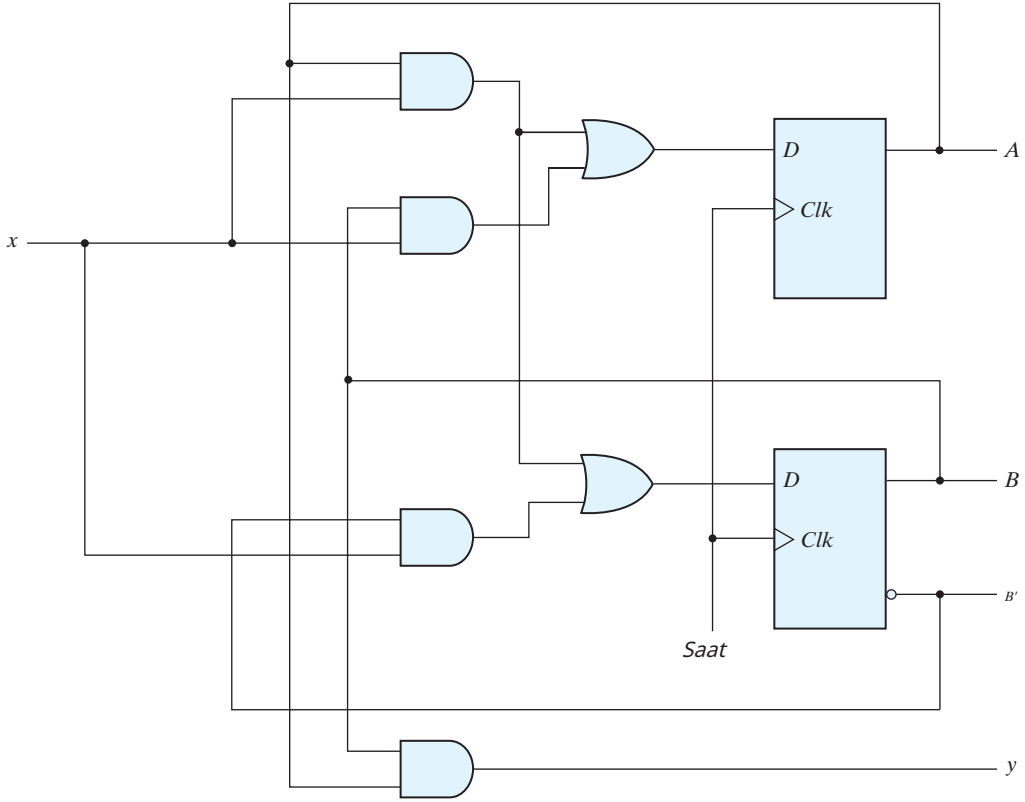
D flip-flop'ları ile tasarım yapmanın avantajı, flip-flop'lara girişi tanımlayan Boolean denklemlerinin doğrudan durum tablosundan elde edilebilmesidir. Yazılım araçları, düzgün yazılmış bir HDL modelinden otomatik olarak D -tip flip-flop'u çıkarır ve seçer.

Sequential devrenin şeması Şekil 5.29'da çizilmiştir.

Uyarı Tabloları

D tipi flip-flop'lar dışındaki flip-flop'larla bir sequential devre tasarımı, devrenin giriş denklemlerinin durum tablosundan dolayı olarak türetilmesi gerektiği gerçeğiyle karmaşıklaşmaktadır. D -tip flip-flop'lar kullanıldığında, giriş denklemleri doğrudan bir sonraki durumdan elde edilir. Bu, JK ve T tipi flip-flop'lar için geçerli değildir. Bu flip-flop'lar için giriş denklemlerini belirlemek amacıyla, durum tablosu ile giriş denklemleri arasında işlevsel bir ilişki türetmek gereklidir.

Tablo 5.1'de sunulan flip-flop karakteristik tabloları, girişler ve mevcut durum bilindiğinde bir sonraki durumun değerini sağlar. Bu tablolar faydalıdır



ŞEKİL 5.29
Moore tipi dizi dedektörünün mantık diyagramı

sıralı devreleri analiz etmek ve flip-flop'ların işleyişini tanımlamak için. Tasarım süreci sırasında, genellikle mevcut durumdan bir sonraki duruma geçişi biliyoruz ve gerekli geçişi sağlayacak flip-flop giriş koşullarını bulmak istiyoruz. Bu nedenle, belirli bir durum değişikliği için gerekli girişleri listeleyen bir tabloya ihtiyacımız var. Böyle bir tabloya *uyarı tablosu* denir.

Tablo 5.12 iki flip-flop için uyarı tablolarını göstermektedir (JK ve T). Her tablo hazır durum için bir sütun $Q(t)$, bir sonraki durum için bir sütun $Q(t+1)$, ve gerekli geçişin nasıl sağlandığını göstermek için her giriş için bir sütun içerir. Mevcut durumdan bir sonraki duruma dört olası geçiş vardır. Dört geçişin her biri için gerekli giriş koşulları, karakteristik tablodaki mevcut bilgilerden türetilmiştir. Tablolardaki X sembolü, girişin 1 veya 0 olup olmadığının önemli olmadığı bir durumu temsil eder.

JK flip-flop için uyarım tablosu (a) kısmında gösterilmiştir. Mevcut durum ve sonraki durum 0 olduğunda, J girişi 0'da kalmalı ve K girişi ya 0 ya da 1 olabilir. Aynı şekilde, hem mevcut durum hem de sonraki durum 1 olduğunda, K girişi 0'da kalmalıdır,

Tablo 5.12
Flip-Flop Uyarı Tabloları

$Q(t=1)$		J	K	$Q(t=1)$		T
0	0	0	X	0	0	0
0	1	1	X	0	1	1
1	0	X	1	1	0	1
1	1	X	0	1	1	0

(a) JK Flip-Flop

(b) T Flip-Flop

J girişi 0 veya 1 olabilir. Flip-flop'un 0 durumundan 1 durumuna geçiş yapması gerekiyorsa, J 1'e eşit olmalıdır, çünkü J girişi flip-flop'u ayarlar. Ancak, girdi K 0 veya 1 olabilir. Eğer $K=0$ ise, $J=1$ koşulu flip-flop'u gerektiği gibi ayarlar; eğer $K=1$ ve $J=1$ olduğunda, flip-flop tamamlanır ve 0 durumundan 1 durumuna geçer. Bu nedenle, K girişi 0'dan 1'e geçiş için bir önemsiz durumla işaretlenmiştir. 1 durumundan 0 durumuna geçiş için, $K=1$ olmalıdır, çünkü K girişi flip-flop'u sıfırlar. Ancak, J girişi ya 0 ya da 1 olabilir, çünkü $J=0$ etkisi yoktur ve $J=1$ ile birlikte $K=1$ flip-flop'u tamamlar ve 1 durumundan 0 durumuna geçiş ile sonuçlanır.

T flip-flop için uyarılma tablosu (b) kısmında gösterilmiştir. Karakteristik tablodan, giriş $T=1$ olduğunda, flip-flop'un durumu tamamlanır ve $T=0$ olduğunda, flip-flop'un durumu değişmez. Bu nedenle, flip-flop'un durumu aynı kalması gerektiğinde, gereklilik $T=0$ 'dır. Flip-flop'un durumu tamamlanması gerektiğinde, T 1'e eşit olmalıdır.

Sentez JK Flip-Flop'lar Kullanarak

JK flip-flop'lar ile ardışık devreler için manuel sentez prosedürü, D flip-flop'lar ile aynı olup, tek farkı giriş denklemlerinin mevcut durumdan sonraki duruma geçişin uyarılma tablosundan değerlendirilmesidir. Prosedürü göstermek için, Tablo 5.13 tarafından belirtilen ardışık devreyi sentezleyeceğiz. Geleneksel bir durum tablosunda olduğu gibi, mevcut durum, giriş ve sonraki durum için sütunlar olmasının yanı sıra, tablo, giriş denklemlerinin türetildiği flip-flop giriş koşullarını göstermektedir.

Flip-flop girişleri, JK flip-flop için durum tablosu ile uyarılma tablosu bir arada kullanılarak türetilir. Örneğin, Tablo 5.13'ün ilk satırında, flip-flop A'nın mevcut durumundaki 0'dan sonraki durumundaki 0'a geçişi vardır. Tablo 5.12'de, JK flip-flop için, mevcut durum 0'dan sonraki durum 0'a geçişin, giriş J'nin 0 ve giriş K'nin belirsiz (don't-care) olmasını gerektirdiğini buluyoruz. Bu nedenle, ilk satıra 0 ve X girilir.

J_A ve K_A , sırasıyla. İlk satır, flip-flop B için mevcut durumdan sonraki duruma 0'dan 0'a geçişi de gösterdiğinden, 0 ve X, J_B ve K_B , sırasıyla, ilk satırın altına eklenir. Tabloyun ikinci satırı, flip-flop B için mevcut durumdan sonraki duruma 0'dan 1'e geçişi gösterir. Uyarı tablosundan, 0'dan 1'e geçişin J 1 ve K bir önemsiz değer olmasını gerektirdiğini buluyoruz, bu nedenle 1 ve X,

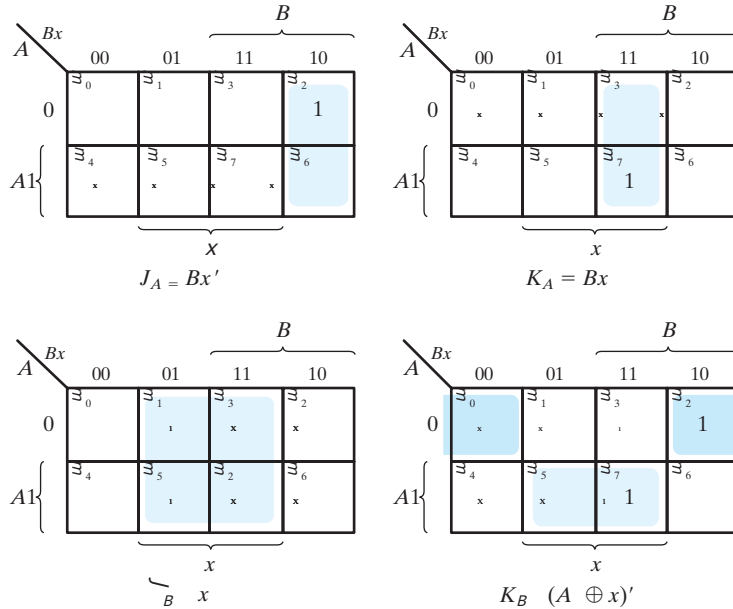
Tablo 5.13

Durum Tablosu ve JK Flip-Flop Girişleri

Mevcut Durum			Giriş		Sonraki Durum		Flip-Flop Girişleri			
A	B	x	A	B	J _A	K _A	J _B	K _B		
0	0	0	0	0	0	X	0	X		
0	0	1	0	1	0	X	1	X		
0	1	0	1	0	1	X	X	1		
0	1	1	0	1	0	X	X	0		
1	0	0	1	0	X	0	0	X		
1	0	1	1	1	X	0	1	X		
1	1	0	1	1	X	0	X	0		
1	1	1	0	0	X	1	X	1		

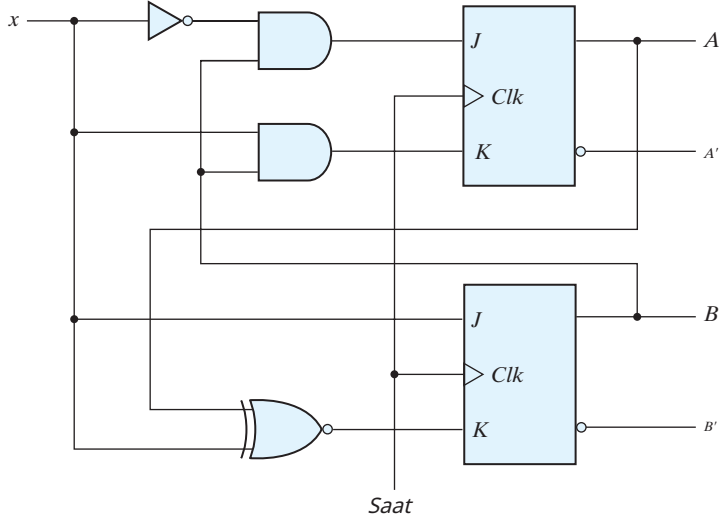
tablonun ikinci satırı altında J_B ve K_B , sırasıyla. Süreç, her satır için tabloda ve her flip-flop için, uyarım tablosundan gelen giriş koşulları, ilgili flip-flop'un uygun satırına kopyalanarak devam eder.

Tablo 5.13'teki flip-flop girişleri, mevcut durum A , mevcut durum B ve giriş x fonksiyonu olarak giriş denklemleri için doğru tablosunu belirtir. Giriş denklemleri, Şekil 5.30'daki haritalarda basitleştirilmiştir. Son durum değerleri, basitleştirme sırasında kullanılmaz,



ŞEKİL 5.30

Giriş denklemleri için J ve K haritaları



ŞEKİL 5.31

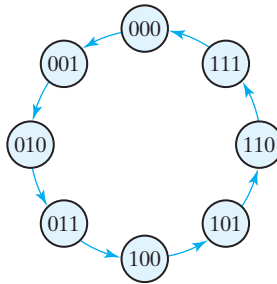
JK flip-flop'lar ile ardışık devre için mantık diyagramı

Giriş denklemleri mevcut duruma ve yalnızca girişi bir fonksiyonudur. Ardışık devreler tasarlandığında manuel olarak JK tipi flip-flop'ların kullanılmasının avantajını not edin. Bu kadar çok önemsiz girişin bulunması, giriş denklemleri için kombinasyonel devrenin muhtemelen daha basit olacağını gösterir, çünkü önemsiz mintermler genellikle daha basit ifadelerin elde edilmesine yardımcı olur. Durum tablosunda kullanılmayan durumlar varsa, haritada ek önemsiz koşullar olacaktır. Yine de, D -tipi flip-flop'lar otomatik bir tasarım akışına daha uygundur.

JK flip-flop'lar için dört giriş denklemi, Şekil 5.30'un altında listelenmiştir. Sıralı devrenin mantık diyagramı (şeması) Şekil 5.31'de çizilmiştir.

T Flip-Flop'lar Kullanarak Sentez

T flip-flop'lar kullanarak devrelerin sentezlenmesi prosedürü, bir ikili sayıcı tasarlaarak gösterilecektir. Bir n -bit ikili sayıcı, 0'dan 2^n-1 'e kadar ikili sayabilen n flip-flop'tan oluşur. Üç bitli bir sayıcının durum diyagramı Şekil 5.32'de gösterilmiştir.



ŞEKİL 5.32

Üç bitli ikili sayıcının durum diyagramı

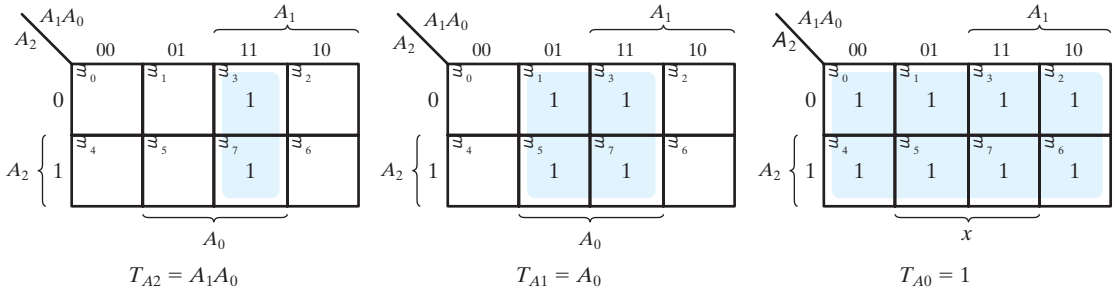
Daireler içinde belirtilen ikili durumlardan görüldüğü gibi, flip-flop çıkışları 111'den sonra 000'a geri dönerek ikili sayım dizisini tekrarlar. Daireler arasındaki yönlendirilmiş çizgiler, diğer durum diyagramlarındaki gibi giriş ve çıkış değerleri ile işaretlenmemiştir. Unutmayın ki, saatli sıralı devrelerde durum geçişleri bir saat kenarı ile başlatılır; saat uygulanmadığı sürece flip-flop'lar mevcut durumlarında kalır. Bu nedenle, saat durum diyagramında veya durum tablosunda açıkça bir giriş değişkeni olarak görünmez. Bu açıdan bakıldığında, bir sayıcının durum diyagramı yönlendirilmiş çizgiler boyunca giriş ve çıkış değerlerini göstermek zorunda değildir. Devrenin tek girişi saattir ve çıkışlar flip-flop'ların mevcut durumu tarafından belirlenir. Bir sayıcının bir sonraki durumu tamamen mevcut durumuna bağlıdır ve durum geçişi her seferinde saat bir geçişten geçtiğinde gerçekleşir.

Tablo 5.14, üç bitli ikili sayıcı için durum tablosudur. Üç flip-flop, A_2 , A_1 ve A_0 ile sembolize edilir. İkili sayıcılar, tamamlayıcı özelliği nedeniyle en verimli şekilde T flip-flop'lar ile inşa edilir. T girişleri için flip-flop uyarımı, T flip-flop'un uyarım tablosundan ve mevcut durumdan sonraki duruma geçişin incelenmesiyle elde edilir. Örnek olarak, 001 satırındaki flip-flop girişlerini düşünelim. Buradaki mevcut durum 001 ve sonraki durum 010'dur, bu da dizideki bir sonraki sayıdır. Bu iki sayıyı karşılaştırdığımızda, A_2 'nin 0'dan 0'a geçtiğini görüyoruz, bu nedenle TA_2 0 ile işaretlenmiştir çünkü flip-flop A_2 , bir saat sinyali geldiğinde değişmemelidir. Ayrıca, A_1 0'dan 1'e geçtiği için TA_1 1 ile işaretlenmiştir çünkü bu flip-flop, bir sonraki saat kenarında tamamlanmalıdır. Benzer şekilde, A_0 1'den 0'a geçiyor, bu da tamamlanması gerektiğini gösteriyor, bu nedenle TA_0 1 ile işaretlenmiştir. Son satır, mevcut durumu 111 olan, ilk sayım 000 ile karşılaştırılır, bu da onun sonraki durumudur. Tüm 1'lerden tüm 0'lara geçmek, üç flip-flop'un da tamamlanmasını gerektirir.

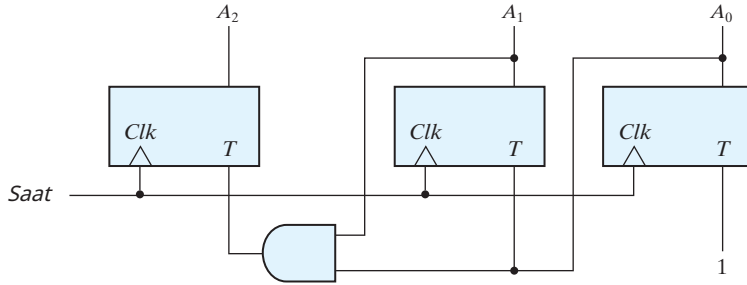
Flip-flop giriş denklemleri, Şekil 5.33'teki haritalarda basitleştirilmiştir. Not edin ki TA_0 tüm sekiz mintermi 1'lere sahiptir çünkü sayacın en az anlamlı biti her sayımda tamamlanmaktadır. Tüm mintermleri içeren bir Boolean fonksiyonu, 1'in sabit bir değerini tanımlar. Her haritanın altında listelenen giriş denklemleri, sayacın kombinasyonel kısmını belirtir. Bu fonksiyonları üç flip-flop ile birleştirerek elde ederiz.

Tablo 5.14
Üç Bitli Sayaç için Durum Tablosu

Mevcut Durum			Sonraki Durum			Flip-Flop Girişleri		
A_2	A_1	A_0	A_2	A_1	A_0	TA_2	TA_1	TA_0
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	1	1
1	1	1	0	0	0	1	1	1



ŞEKİL 5.33
Üç bitli ikili sayaç için haritalar



ŞEKİL 5.34
Üç bitli ikili sayaçın mantık diyagramı

Sayaçın mantık diyagramı, Şekil 5.34'te gösterildiği gibidir. Basitlik açısından, sıfırlama sinyali gösterilmemiştir, ancak her tasarımın bir sıfırlama sinyali içermesi gerektiğini unutmayın.

PROBLEMLER

(Yıldız ile işaretlenmiş problemlerin cevapları kitabın sonunda yer almaktadır. Uygun olduğunda, bir mantık tasarımı ve ilgili HDL modelleme problemi çapraz referans verilmiştir.)

Not: Bir HDL modelinin yazılması ve doğrulanmasını gerektiren her problem için, simülasyon sırasında hangi işlevsel özelliklerin test edileceğini ve nasıl test edileceğini belirlemek için bir test planı yazılmalıdır. Örneğin, simüle edilen makine sıfırlama durumunun dışında bir durumda iken sıfırlama sinyalinin etkinleştirilmesiyle anlık sıfırlama test edilebilir. Test planı, planı uygulayacak bir test tezgahının geliştirilmesine rehberlik etmelidir. Modeli test tezgahını kullanarak simüle edin ve davranışın doğru olduğunu doğrulayın. Sentez araçları ve bir ASIC hücre kütüphanesi mevcutsa, Problemeler 5.34–5.42 için geliştirilen Verilog tanım lamaları sentez egzersizleri olarak atanabilir. Sentez araçları tarafından üretilen kapı düzeyindeki devre simüle edilmeli ve ön sentez modelinin simülasyon sonuçlarıyla karşılaştırılmalıdır. Bir FPGA araç seti mevcutsa, aynı egzersizler atanabilir.

Şekil 5.6'daki D kilidi dört NAND kapısı ve bir inverter ile inşa edilmiştir. Aşağıdaki üç farklı yöntemi göz önünde bulundurun D kilidini elde etmek için. Her durumda, mantık diyagramını çizin ve devre çalışmasını doğrulayın.

- SR kilidi kısmı için NOR kapıları ve diğer iki için AND kapıları kullanın. Bir inverter gerekli olabilir.
- Tüm dört kapı için NOR kapıları kullanın. İnverterler gerekli olabilir.
- Sadece dört NAND kapısı kullanın (bir inverter olmadan). Bu, Şekil 5.6'daki üst kapının çıkışını alt kapının girişine (inverter çıkışı yerine) bağlayarak yapılabilir.

Bir D flip-flop, ikiye bir hat çoklayıcı ve bir inverter kullanarak bir JK flip-flop oluşturun. (HDL—Problem 5.34'e bakınız.)

JK flip-flop'un tamamlayıcı çıkışı için karakteristik denklemin gösterilmesi

$$Q'(t+1) = J'Q' + KQ$$

5.4 A PN flip-flop'un dört işlemi vardır: 0'a temizleme, değişiklik olmaması, tamamlayıcı ve 1'e ayarlama. a, girişler P ve N sırasıyla 00, 01, 10 ve 11 olduğunda.

(a) Karakteristik tabloyu tablo halinde gösterin.

(b) * Karakteristik denklemleri türetin.

(c) Uyarı tablosunu tablo halinde gösterin.

(d) PN flip-flop'un nasıl bir D flip-flop'a dönüşürülebileceğini gösterin.

5.5 Bir doğru tablosu, bir durum tablosu, bir özellik tablosu ve bir uyarı tablosu arasındaki farkları açıklayın. Ayrıca, bir Boolean denklemi, bir durum denklemi, bir özellik denklemi ve bir flip-flop giriş denklemi arasındaki farkı açıklayın.

5.6 İki D flip-flop'u A ve B , iki girişi x ve y ; ve bir çıkışı z olan bir ardışık devre, aşağıdaki sonrakı durum ve çıkış denklemleri ile tanımlanmıştır (HDL—bkz. Problem 5.35):

$$A(t+1) = xy' + xB$$

$$B(t+1) = xA + xB'$$

$$z = A$$

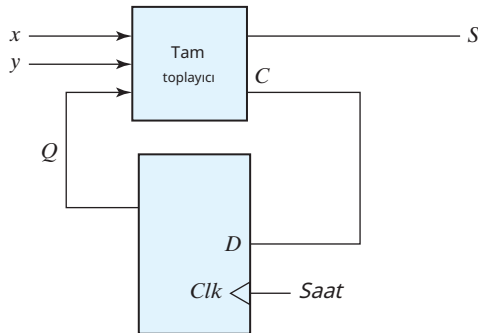
(a) Devrenin mantık diyagramını çizin.

(b) Ardışık devre için durum tablosunu listeleyn.

(c) İlgili durum diyagramını çizin.

5.7

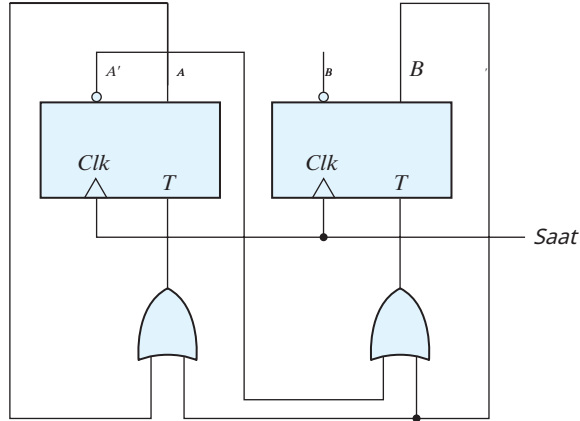
* Bir ardışık devre bir flip-flop Q , iki girişi x ve y , ve bir çıkışı S vardır. Tam toplayıcı devresinden oluşur ve bir D flip-flop'a bağlıdır, Şekil P5.7'de gösterildiği gibi. Ardışık devrenin durum tablosunu ve durum diyagramını türetin.



ŞEKİL P5.7

5.8

* Verilen Fig. P5.8'deki ardışık devrenin durum tablosunu ve durum diyagramını türetin. Devrenin gerçekleştirdiği işlevi açıklayın. (HDL—bkz. Problem 5.36.)



ŞEKİL P5.8

Bir ardışık devre, iki JK flip-flop A ve B ile bir giriş x içerir. Devre, aşağıdaki flip-flop giri ş denklemleriyle tanımlanır:

$$J_A = x \quad K_A = B$$

$$J_B = x \quad K_B = A'$$

(a) Durum denklemlerini türetin $A(t+1)$ ve $B(t+1)$ giri ş denklemlerini

J ve K de ğ i ş kenleri için yerine koyarak.

(b) Devrenin durum diyagramını ç izin.

Bir ardışık devre iki JK flip-flop A ve B, iki giri ş x ve y, ve bir ç ık ı ş z vardır.

Flip-flop giri ş denklemleri ve devre ç ık ı ş denklemleri ş unlardır

$$J_A = Bx + B'y' \quad K_A = B'xy'$$

$$J_B = A'x \quad K_B = A + xy'$$

$$z = Ax'y' + Bx'y'$$

(a) Devrenin mantık diyagramını ç izin.

(b) Durum tablosunu tablo halinde dü zenleyin.

(c) Durum denklemlerini türetin A ve B için.

5.11

Ş ekil 5.16'daki durum diyagramı ile tanımlanan devre için,

(a)* Devreye 01011011101110 giri ş dizisi uygulandı ğ ında ve ba ş langı ç ta durum 00' da oldu ğ unda olu ş turulacak durum ge ç i ş lerini ve ç ık ı ş dizisini belirleyin.

(b) Ş ekil 5.16'daki tüm e ş de ğ er durumları bulun ve daha basit, ancak e ş de ğ er bir durum diagramı ç izin.

(c) D flip-flop'ları kullanarak, (b) bölümündeki durum diagramı tarafından tanımlanan e ş de ğ er makin eyi (mantık diyagramı dahil) tasarlayın.

Aşağıdaki durum tablosu için

Mevcut Durum	Sonraki Durum		Çıktı	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
		b	0	0
		c	0	0
		e	0	0
		a	1	0
		c	0	0
		b	1	1
		h	0	1
		a	1	0

(a) İlgili durum diyagramını çizin.

(b) Azaltılmış durum tablosunu tablo haline getirin.

(c) Azaltılmış durum tablosuna karşılık gelen durum diyagramını çizin.

5.13 Durumdan başlayarak a , ve giriş dizisi 01110010011, çıkış dizisini belirleyin

(a) Önceki problemin durum tablosu.

(b) Önceki problemin azaltılmış durum tablosu. Her ikisi için de aynı çıkış sequencesinin elde edildiğini gösterin.

5.14 Tablo 5.9'dan bir-hot atamasını Tablo 5.8'deki durumlara yerleştirin ve binary durum tablosunu elde edin.

5.15 JK flip-flop için bir durum tablosu listeleyin, Q mevcut ve sonraki durum olarak ve J ve K girişler olarak kullanın. Durum tablosu tarafından belirtilen ardışık devreyi tasarlayın ve bunun Fig. 5.12 (a) ile eşdeğer olduğunu gösterin.

5.16 İki D flip-flop A ve B ile bir ardışık devre tasarlayın ve bir giriş x_{in} .

(a)* $x_{in} = 0$ olduğunda, devrenin durumu aynı kalır. $x_{in} = 1$ olduğunda, devre 00'dan 01'e, 11'e, 10'a, tekrar 00'a geçiş yapar ve tekrarlar.

(b) $x_{in} = 0$ olduğunda, devrenin durumu aynı kalır. $x_{in} = 1$ olduğunda, devre 00'dan 11'e, 01'e, 10'a, tekrar 00'a geçiş yapar ve tekrarlar.

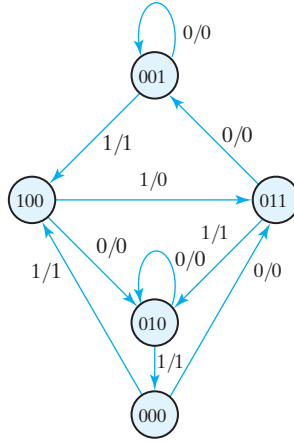
(HDL—Problem 5.38'e bakın.)

5.17 Bir girdi, bir çıktı seri 2'nin tamamlayıcısını tasarlayın. Devre, girişten bir bit dizisi alır ve çıkışta 2'nin tamamlayıcısını üretir. Devre, işlemi başlatmak ve sonlandırmak için asenkron olarak sıfırlanabilir. (HDL—bkz. Problem 5.39.)

5.18 * İki JK flip-flop A ve B ile iki girdi E ve F olan bir ardışık devre tasarlayın. Eğer $E = 0$ ise, devre F değerine bakılmaksızın aynı durumda kalır. $E = 1$ ve $F = 1$ olduğunda, devre 00'dan 01'e, 10'a, 11'e, tekrar 00'a geçiş yapar ve tekrar eder. $E = 1$ ve $F = 0$ olduğunda, devre 00'dan 11'e, 10'a, 01'e, tekrar 00'a geçiş yapar ve tekrar eder. (HDL—Problem 5.40'a bakın.)

5.19 Bir ardışık devrede üç flip-flop A, B, C; bir girdi x_{in} ; ve bir çıkış y_{out} vardır. Durum diyagramı Şekil P5.19'da gösterilmiştir. Devre, kullanılmayan durumları dikkate almadan tasarlanacaktır. Tasarımdan elde edilen devreyi analiz edin ve kullanılmayan durumların etkisini belirleyin. (HDL—Problem 5.41'e bakın.)

- (a)* Tasarımda D flip-flop'ları kullanın.
 (b) Tasarımda JK flip-flop'ları kullanın.



ŞEKİL P5.19

5.20

Şekil 5.19'da belirtilen durum diyagramına göre, T flip-flop'ları kullanarak ardışık devreyi tasarlayın. Verilog HDL'de **initial** ifadesi ile **always** ifadesi arasındaki ana fark nedir?

5.21

Aşağıdaki ifadelerle üretilen dalga formunu çizin:

(a) **başlangıçta**

$w = 0$; #10 $w = 1$; # 40 $w = 0$; # 20 $w = 1$; #15 $w = 0$;

bitiş

(b) **başlangıçta fork**

$w = 0$; #10 $w = 1$; # 40 $w = 0$; # 20 $w = 1$; #15 $w = 0$;

katıl

5.22

5.23

* Aşağıdaki ifadeleri düşünün, RegA'nın başlangıçta 50 değerini içerdiğini varsayarak.

(a) RegA = 125; (b)

RegB = RegA;

RegB <= RegA;

RegA ve RegB'nin yürütmeden sonraki değerleri nedir?

5.24

Asenkron ön ayar ve temizleme ile pozitif kenar duyarlı D flip-flop'un HDL davranışsal tanımını yazın ve doğrulayın.

5.25

Özel bir pozitif kenar tetiklemeli flip-flop devre bileşeni dört girdi D 1, D 2, D 3, ve D 4'e ve bunlar arasında seçim yapan iki bitlik bir kontrol girişine sahiptir. Bu bileşenin HDL davranışsal tanımını yazın ve doğrulayın.

5.26

Mevcut durumun değerine dayalı olarak bir if-else ifadesi kullanarak JK flip-flop'un HDL davranışsal tanımını yazın ve doğrulayın.

(a)* $Q = 0$ veya $Q = 1$ olduğunda karakteristik denklemi elde edin.

(b) Her saat darbesinde J ve K girişlerinin flip-flop'un çıkışını nasıl etkilediğini belirtin.

5.27

HDL Örneği 5.5'in tanımını durum geçişlerini ve çıktığı bir her zaman bloğunda birleştiren k yeniden yazın ve doğrulayın.

5.28

Şekil 5.17'de gösterilen ardışık devreyi simüle edin.

- (a) Durum diyagramının HDL tanımını yazın (yani, davranışsal model).
- (b) Mantık (devre) diyagramının HDL tanımını yazın (yani, yapısal model).
- (c) HDL ile 00, 01, 11, 10 girişlerinin bir dizisi ile bir uyarı yazın.

Her iki tanım için yanıtın aynı olduğunu doğrulayın.

Şekil P5.19'da gösterilen durum diyagramı tarafından tanımlanan durum makinesinin davranışsal tanımını yazın. Bir test tezgahı yazın ve tanımın işlevselliğini doğrulayın.

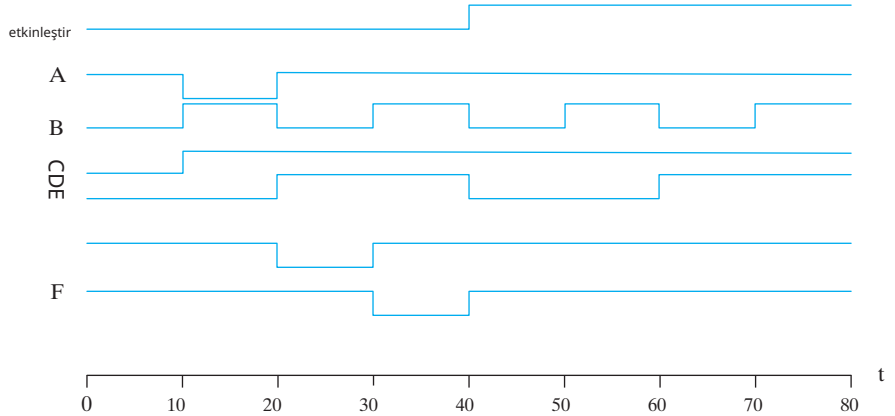
Aşağıdaki HDL modülü tarafından tanımlanan ardışık devre için mantık diyagramını çizin:

```
module Seq_Ckt ( input A, B, C, E output regQ, input CLK, );
    reg E;

    always @ (posedge CLK)
    begin
        E <= A || B;
        Q <= E && C;
    end
endmodule
```

Problem 5.30'daki açıklama, atamalar = ile yapıldığında aynı davranışı gösterecek şekilde nasıl yazılmalıdır?

Bir başlangıç ifadesi kullanarak bir **begin... end** bloğunda, Şekil P5.32'de gösterilen dalga formlarının Verilog tanımını yazın. Bir **fork... join** bloğu kullanarak tekrar edin.



ŞEKİL P5.32

Problem 5.32 için Dalga Şekilleri

Test tezgahındaki uyarı sinyallerinin, test edilecek ardışık devrenin saatinin inaktif kenarına senkronize olmasının neden önemli olduğunu açıklayın.

Şekil 5.5'te gösterilen devre şemasına (şematik) sahip makinenin HDL yapısal tanımını yazın ve doğrulayın.

Problem 5.6'da tanımlanan ardışık devrenin HDL modelini yazın ve doğrulayın.

Makinenin devre şemasını (şematik) gösteren Şekil P5.8'deki HDL yapısal tanımını yazın ve doğrulayın.

Figs. 5.25'te gösterilen durum makinelerinin HDL davranış tanımlarını yazın ve doğrulayın.

ve 5.26. İki makinenin durum dizilerini ve giriş-çıkış davranışlarını karşılaştırmak için bir test tezgahı yazın.

Problem 5.16'da tanımlanan makinenin HDL davranış tanımını yazın ve doğrulayın.

Problem 5.17'de belirtilen makinenin davranış tanımını yazın ve doğrulayın.

Problem 5.18'de belirtilen makinenin davranış tanımını yazın ve doğrulayın.

Problem 5.19'da belirtilen makinenin davranışsal tanımını yazın ve doğrulayın.

(*İpucu*: HDL Örneği 4.8'den önceki varsayılan durum ögesinin tartışmasına bakın Bölüm 4.)

Şekil 5.29'da gösterilen devrenin HDL yapısal tanımını yazın ve doğrulayın.

Şekil 5.34'teki üç bitlik ikili sayacın HDL davranışsal tanımını yazın ve doğrulayın.

Asenkron sıfırlama özelliğine sahip bir *D* flip-flop'un Verilog modelini yazın ve doğrulayın.

5.27 Şeklinde tanımlanan dizilim dedektörünün HDL davranışsal tanımını yazın ve doğrulayın.

Bir senkron sonlu durum makinesi, bir girdi x_{in} ve bir çıktı y_{out} ile donatılmıştır. x_{in} 0'dan 1'e değiştiğinde, çıktı y_{out} üç döngü boyunca aktif olmalıdır, x_{in} değerine bakılmaksızın, ardından makine başka bir

x_{in} aktif etmesine yanıt vermeden önce iki döngü boyunca pasif olmalıdır. Makinenin aktif-düşük senkron sıfırlama özelliği olmalıdır.

(a) Makinenin durum diyagramını çizin.

(b) Makinenin bir Verilog modelini yazın ve doğrulayın.

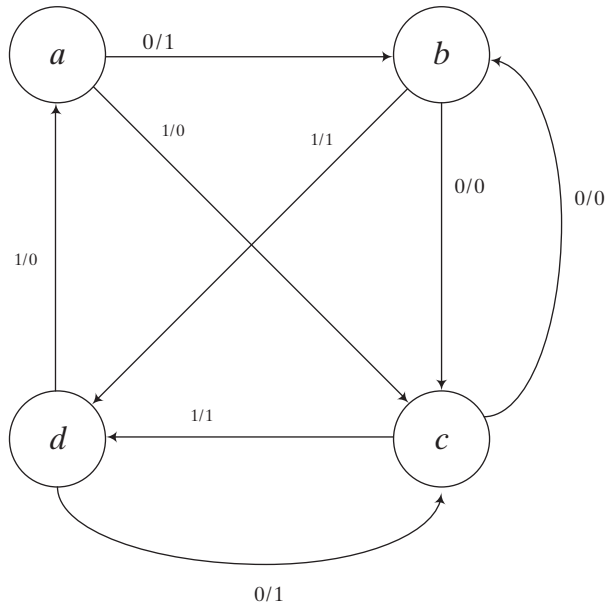
5.47

Bir senkron sonlu durum makinesi nin Verilog modelini yazın, çıktısı ise 0, 2, 4, 6, 8, 10, 12, 14, 0 şeklindedir. Makine, *Run* adlı tek bir girdi ile kontrol edilir, böylece sayma *Run* aktifken gerçek

leşir, *Run* pasifken durur ve *Run* yeniden aktif hale geldiğinde sayma devam eder. Yaptığınız varsayımları açıkça belirtin.

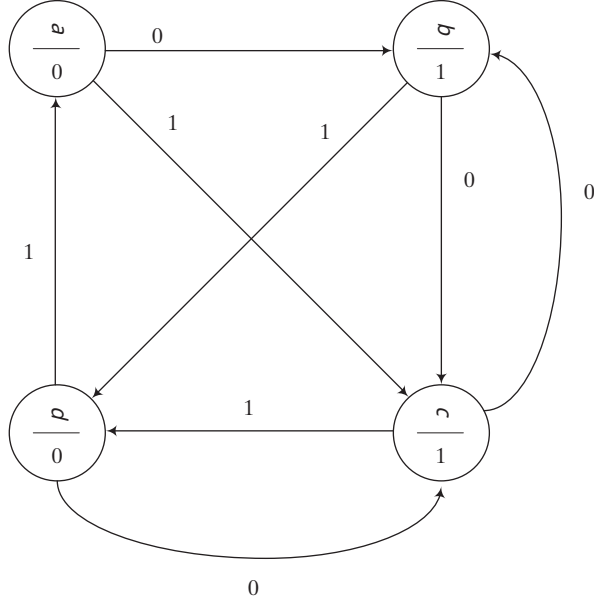
Durum diyagramında tanımlanan Mealy FSM'nin Verilog modelini yazın.

Bir test tezgahı geliştirin ve makine durum geçişlerinin ve çıktısının durum diyagramına uygun olduğunu gösterin.



ŞEKİL P5.48

Moore FSM'nin durum diyagramında tanımlandığı gibi bir Verilog modeli yazın. Bir test tezgahı geliştirin ve makinenin durum geçişlerinin ve çıktısının durum diyagramıyla örtüşüğünü gösterin.



ŞEKİL P5.49

Senkron bir Moore FSM'nin tek bir girişi, x_{in} ve tek bir çıkışı y_{out} vardır. Makine, girişi izlemek için tasarlanmıştır ve x_{in} 'un ikinci örneği 1 olarak algılanana kadar başlangıç durumunda kalacaktır. x_{in} 'un ikinci doğrulaması algılandığında, y_{out} doğrulanacak ve dördüncü doğrulama algılanana kadar doğrulanmış kalacaktır. x_{in} 'un dördüncü doğrulaması algılandığında, makine başlangıç durumuna dönecek ve x_{in} 'u izlemeye devam edecektir.

- Makinenin durum diyagramını çizin.
- Makinenin bir Verilog modelini yazın ve doğrulayın.

5.51 Aşağıda verilen Verilog modeli tarafından tanımlanan makinenin durum diyagramını çizin.

```

module Prob_5_51 ( output reg y_out, input x_in, clk, reset);
  parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10, s3 = 2'b11;
  reg [1:0] state, next_state;
  her zaman @(posedge clk, negedge reset) begin
    if (reset == 1'b0) state <= s0;
    else state <= next_state;
  her zaman @(state, x_in) begin
    y_out = 0;
    next_state=s0;
  case (state)
    s0: if x_in = 1 begin y_out = 0; if (x_in) next_state = s1; else next_state = s0; end
    s1: if x_in = 1 begin y_out = 0; if (x_in) next_state = s2; else next_state = s1; end
  endcase
end

```

```

s2:  if x_in = 1 begin y_out = 1; if (x_in) next_state = s3; else next_state = s2; end
s3:  if x_in = 1 begin y_out = 1; if (x_in) next_state = s0; else next_state = s3; end
default : next_state = s0;
endcase
end
endmodule

```

Aşağıda verilen Verilog modeli tarafından tanımlanan makinenin durum diyagramını çizin.

```

module Prob_5_52 ( output reg y_out, input x_in, clk, reset);
    parameter s0 = 2'b00, s1 = 2'b01, s2 = 2'b10, s3 = 2'b11;
    reg [1:0] state, next_state;
    always @ ( posedge clk, negedge reset) begin
        if (reset == 1'b0) state <= s0;
        else state <= next_state;
    always @(state, x_in) begin
        y_out = 0;
        next_state=s0;
        case (state)
s0:  eğer x_in = 1 başla y_out = 0; eğer (x_in) next_state = s1; aksi takdirde next_state = s0; bitir
s1:  eğer x_in = 1 başla y_out = 0; eğer (x_in) next_state = s2; aksi takdirde next_state = s1; bitir
s2:  eğer x_in = 1 eğer (x_in) başla next_state = s3; y_out = 0;
            else begin next_state = s2; y_out = 1; end
s3:  if x_in = 1 begin y_out = 1; if (x_in) next_state = s0; else next_state = s3; end
        default : next_state = s0;
        endcase
    end
endmodule

```

Bir Mealy senkron durum makinesinin durum diyagramını çizin ve tek bir girdi, x_{in} , ve tek bir çıktı y_{out} , olacak şekilde bir Verilog modeli yazın; y_{out} toplam alınan 1'lerin sayısı 3'ün katı olduğunda etkinleştirilecektir.

Senkron bir Moore makinesi iki girdi, x_1 ve x_2 , ve çıktı y_{out} vardır. Her iki girdi aynı değere sahipse, çıktı bir döngü boyunca etkinleştirilir; aksi takdirde çıktı 0'dır. Bir durum diyagramı geliştirin ve makinenin Verilog davranışsal modelini yazın. Makinenin doğru çalıştığını gösterin.

Bir giriş hattından gelen bit dizisinde üç veya daha fazla ardışık 1'i tespit eden bir Mealy durum makinesi için durum diyagramını geliştirin.

Manuel yöntemler kullanarak, 0, 2, 4, 6, 0, sırasını sayan üç bitlik bir sayacın mantık diyagramını elde edin.

Bir üç bitlik sayacın Verilog davranışsal modelini yazın ve doğrulayın, bu sayaç 0, 2, 4, 6, 0, sırasını sayar.

Problem 5.55'te tasarlanan sayacın Verilog davranışsal modelini yazın ve doğrulayın.

Problem 5.56'da tanımlanan sayacın Verilog yapısal modelini yazın ve doğrulayın.

0, 1, . . . , 9, 0, 1, 2, sırasını sayan dört bitlik bir sayacın Verilog davranışsal modelini yazın ve doğrulayın.

REFERANSLAR

1. B HASKER, J. 1997. *Verilog HDL: Primer*. Allentown, PA: Star Galaxy Press.
2. B HASKER, J. 1998. *Verilog HDL: Synthesis*. Allentown, PA: Star Galaxy Press.
3. C ILETTI, M. D. 1999. *Modeling, Synthesis, and Rapid Prototyping with Verilog HDL*. Upper Saddle River, NJ: Prentice Hall.
4. D IETMEYER, D. L. 1988. *Logic Design of Digital Systems*, 3rd ed. Boston: Allyn Bacon.
5. G AJSKI, D. D. 1997. *Principles of Digital Design*. Upper Saddle River, NJ: Prentice Hall.
6. H AYES, J. P. 1999. *Introduction to Digital Logic Design*. Reading, MA: Addison-Wesley.
7. K ATZ, R. H. 2005. *Contemporary Logic Design*. Upper Saddle River, NJ: Prentice Hall.
8. MANO, M. M. and C. R. KIME. 2007. *Logic and Computer Design Fundamentals & Xilinx 6.3 Student Edition*, 4th ed. Upper Saddle River, NJ: Prentice Hall.
9. NELSON, V. P., H. T. NAGLE, J. D. IRWIN, ve B. D. CARROLL. 1995. *Dijital Mantık Devresi Analizi ve Tasarımı*. Englewood Cliffs, NJ: Prentice Hall.
10. PALNITKAR, S. 1996. *Verilog HDL: Dijital Tasarım ve Sentez için Bir Rehber*. Mountain View, CA: SunSoft Press (bir Prentice Hall başlığı).
11. ROTH, C. H. 2002. *Mantık Tasarımının Temelleri*, 6. baskı. St. Paul, MN: Brooks/Cole.
12. THOMAS, D. E. ve P. R. MORTBY, 2002. *Verilog Donanım Tanım Dili*, 6. baskı. Boston: Kluwer Akademik Yayıncılar.
13. WAKERLY, J. F. 2007. *Dijital Tasarım: İlkeler ve Uygulamalar*, 4. baskı. Upper Saddle River, NJ: Prentice Hall.

WEB ARAMA KONULARI

Sonlu Durum Makinesi
 Senkron durum makinesi
 Asenkron durum makinesi
 D-tipi flip-flop
 Değiştirme flip-flop
 J-K tipi flip-flop
 İkili sayaç
 Durum diyagramı
 Mealy durum makinesi
 Moore durum makinesi
 One-hot/soguk kodlar

Bölüm 6

Kayıtlar ve Sayıcılar

6.1 KAYITLAR

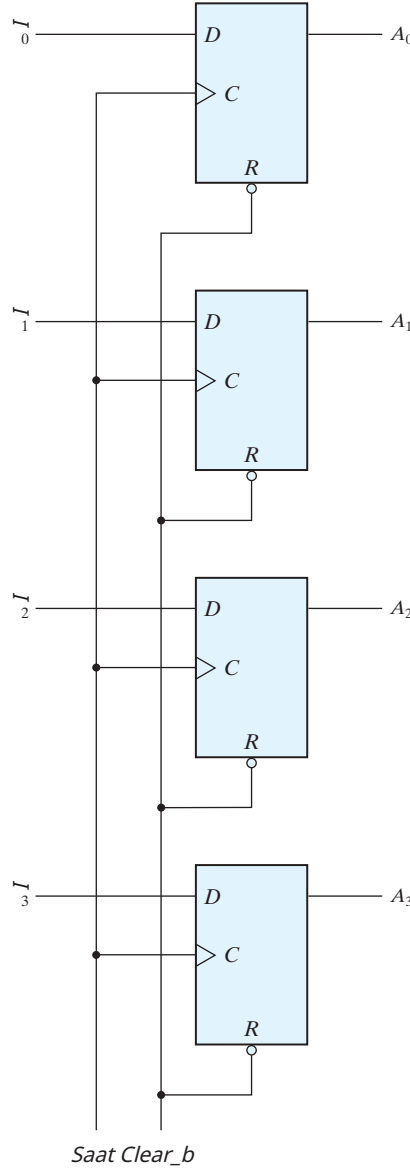
Bir saatli ardışık devre, bir grup flip-flop ve kombinasyonel kapıdan oluşur. Flip-flop'lar, yokluklarında devrenin tamamen kombinasyonel bir devreye dönüşmesi nedeniyle önemlidir (kapılar arasında geri besleme olmadığında). Flip-flop'lar içeren bir devre, kombinasyonel kapıların yokluğunda bile ardışık bir devre olarak kabul edilir. Flip-flop'ları içeren devreler genellikle, ardışık devrenin adı yerine, gerçekleştirdikleri işlemlerle sınıflandırılır. Bu tür iki devre kayıtlar ve sayıcılardır.

Bir *register*, her biri ortak bir saat paylaşan ve bir bit bilgi depolama kapasitesine sahip flip-flop'ların bir grubudur. Bir n -bit register, n bit ikili bilgiyi depolama kapasitesine sahip n flip-flop'tan oluşur. Flip-flop'ların yanı sıra, bir register belirli veri işleme görevlerini yerine getiren kombinasyonel kapılara sahip olabilir. En geniş tanımında, bir register, flip-flop'ların bir grubunu ve bunların işleyişini etkileyen kapıları içerir. Flip-flop'lar ikili bilgiyi tutar ve kapılar, bilginin register'a nasıl aktarılacağını belirler.

Bir *counter*, esasen önceden belirlenmiş bir ikili durum dizisinden geçen bir register'dır. Sayacın içindeki kapılar, belirlenen durum dizisini üretmek için birbirine bağlıdır. Sayıcılar özel bir register türü olmasına rağmen, genellikle onlara farklı bir isim vererek ayırt etmek yaygındır.

Piyasada çeşitli türde register'lar mevcuttur. En basit register, herhangi bir kapı olmadan yalnızca flip-flop'lardan oluşan bir register'dır. Şekil 6.1, dört D -tip flip-flop kullanılarak oluşturulmuş dört bitlik veri depolama register'ını göstermektedir.

Ortak saat girişi, her darbenin pozitif kenarında tüm flip-flop'ları tetikler ve dört girişteki mevcut ikili veriler register'a aktarılır. Saat kenarından hemen önceki (I_3, I_2, I_1, I_0) değeri, saat kenarından sonra (A_3, A_2, A_1, A_0) değerini belirler. Dört



ŞEKİL 6.1
Dört bitli kayıt

Çıktılar, kayıta saklanan ikili bilgiyi elde etmek için herhangi bir zamanda örneklenebilir.

Giriş *Clear_b* tüm dört flip-flop'un aktif-düşük *R* (sıfırlama) girişine gider. Bu giriş 0'a gittiğinde, tüm flip-flop'lar asenkron olarak sıfırlanır. *Clear_b* girişi, saatli işlemlerden önce kaydı tüm 0'lara sıfırlamak için faydalıdır. *R* girişleri normal saatli işlem sırasında mantık 1'de (yani, devre dışı bırakılmış) tutulmalıdır. Not : Flip-flop'a bağlı olarak, *Clear*, *Clear_b*, sıfırlama veya sıfırlama_b kaydın tüm 0'lar durumuna geçişini belirtmek için kullanılabilir.

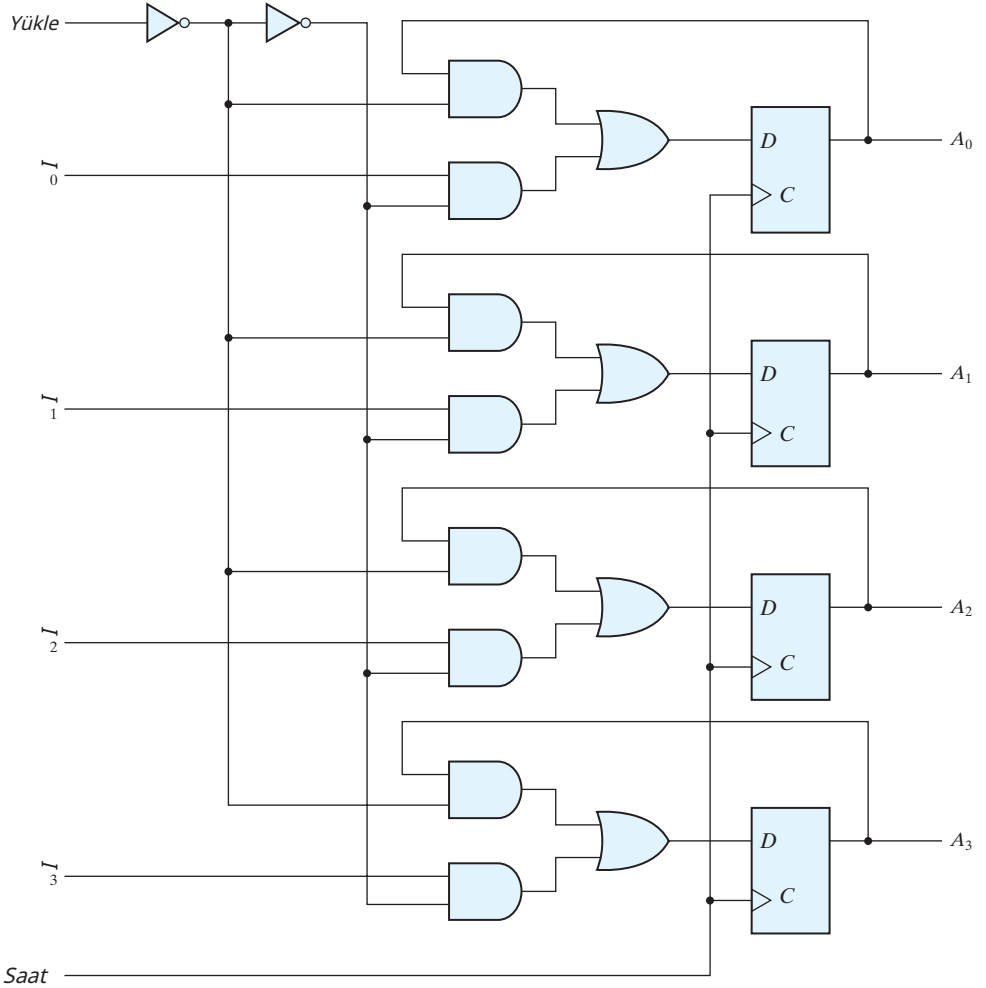
atli işlem sırasında mantık 1'de (yani, devre dışı bırakılmış) tutulmalıdır. Not: Flip-flop'a bağlı olarak, *Clear*, *Clear_b*, *sıfırlama* veya *sıfırlama_b* kaydın tüm 0'lar durumuna geçişini belirtmek için kullanılabilir.

Paralel Yüklemeli Kayıt

Paralel yüklemeli kayıtlar, dijital sistemlerin temel yapı taşlarıdır. Bu yapıların davranışını tam olarak anlamamız önemlidir. Senkron dijital sistemler, sürekli bir saat darbesi dizisi sağlayan bir ana saat üreticine sahiptir. Darbeler, sistemdeki tüm flip-flop'lara ve kayıtlara uygulanır. Ana saat, sistemin tüm parçalarına sürekli bir ritim sağlayan bir davul gibi işlev görür. Her saat darbesinde hangi kayıt işleminin gerçekleştirileceğini belirlemek için ayrı bir kontrol sinyali kullanılmalıdır. Yeni bilgilerin bir kayda aktarılması, yükleme veya güncelleme olarak adlandırılır.

kaydı yükleme. Eğer kaydın tüm bitleri ortak bir clock darbesi ile aynı anda yüklenirse, yüklemenin *paralel* yapıldığını söyleriz. Şekil 6.1'deki kaydın *C* girişlerine uygulanan bir saat kenarı, tüm dört girişi paralel olarak yükleyecektir. Bu yapılandırımda, eğer kaydın içeriği değiştirilmemesi gerekiyorsa, girişler sabit tutulmalı ve ya saat devreden çıkarılmalıdır. İlk durumda, kaydı yönlendiren veri yolu diğer trafik için kullanılamaz. İkinci durumda, saat kaynağının kayda ulaşmasını kontrol eden bir etkinleştirme gate'i ile saat sinyalini devreden çıkarabiliriz. Ancak, saat yoluna kapılar eklemek önerilmez çünkü bu, lojik işlemlerin saat darbeleri ile yapılması anlamına gelir. Lojik kapıların eklenmesi, ana saat ile flip-flopların girişleri arasında düzensiz yayılma gecikmeleri üretir. Sistemi tamamen senkronize etmek için, tüm saat darbelerinin sistemin her yerinde aynı anda ulaşmasını sağlamalıyız, böylece tüm flip-floplar aynı anda tetiklenir. Saat darbeleri ile lojik işlemler değişken gecikmeler ekler ve sistemin senkronizasyondan çıkmasına neden olabilir. Bununla birlikte, kaydın çalışmasını *D* girişleri ile kontrol etmek, flip-flopların *C* girişlerinde ki saati kontrol etmekten daha uygundur. Bu, bir gate'li saat etkisi yaratır, ancak devrenin saat yolunu etkilemeden.

Dört bitlik bir veri depolama kaydedici, yük kontrol girişi ile birlikte, kapılar aracılığıyla flip-flopların *D* girişlerine yönlendirilmiştir ve Şekil 6.2'de gösterilmiştir. Ek kapılar, çıkışının kaydediciye veri yolu veya kaydedicinin çıkışı ile yönlendirilmesini sağlayan iki kanallı bir mux'u uygulamaktadır. Kaydediciye yük girişi, her saat darbesi ile alınacak eylemi belirler. Yük girişi 1 olduğunda, dört dış girişteki veriler, saatın bir sonraki pozitif kenarı ile kaydediciye aktarılır. Yük girişi 0 olduğunda, flip-flopların çıkışları kendi girişlerine bağlanır. Çıkıştan girişe geri besleme bağlantısı gereklidir çünkü bir *D* flip-flop "değişiklik yok" durumuna sahip değildir. Her saat kenarında, *D* girişi kaydedicinin bir sonraki durumunu belirler. Çıkışı değiştirmeden bırakmak için, *D* girişinin çıkışın mevcut değeri ile eşit olması gerekir (yani, çıkış her saat darbesinde girişi döngüsel olarak besler). Saat darbeleri, *C* girişlerine kesintisiz olarak uygulanır. Yük girişi, bir sonraki darbenin yeni bilgileri kabul edip etmeyeceğini veya bilgilerin kaydedicide olduğu gibi kalıp kalmayacağını belirler. Bilgilerin veri girişlerinden veya kaydedicinin çıkışlarından aktarımı, bir saat kenarı na yanıt olarak dört bitin tamamı ile eşzamanlı olarak gerçekleştirilir.

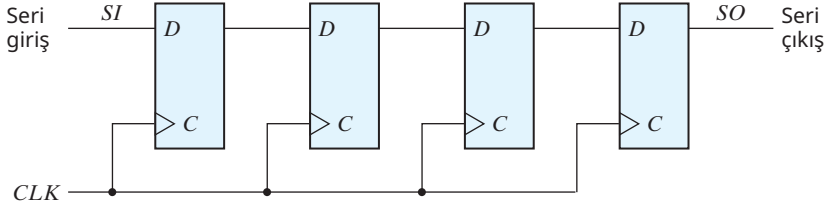


ŞEKİL 6.2
Dört bitlik paralel yükleme kaydı

6.2 SHIFT KAYITLARI

Her hücrede tutulan ikili bilgiyi, seçilen bir yönde komşu hücreye kaydırabilen bir kayıt, kaydırma kaydı olarak adlandırılır. Kaydırma kaydının mantıksal yapısı, bir flip-flop zincirinden oluşur; bir flip-flop'un çıkışı, bir sonraki flip-flop'un girişine bağlıdır. Tüm flip-flop'lar, verilerin bir aşamadan diğerine kaydırılmasını etkinleştiren ortak saat darbeleri alır.

En basit kaydırma kaydı, yalnızca flip-flop'lar kullanan kaydırma kaydırıcıdır; bu, Şekil 6.3'te gösterilmiştir. Verilen bir flip-flop'un çıkışı, sağındaki flip-flop'un D girişine bağlıdır. Bu kaydırma kaydı tek yönlüdür (soldan sağa). Her saat darbesi, kayıt içeriğini bir bit pozisyonu sağa kaydırır.



ŞEKİL 6.3

Dört bitli kaydırma kaydı

kayıt bir bit pozisyonunu sağa kaydırır. Yapı, sola kaydırmayı desteklemez. seri girişi en soldaki flip-flop'a ne gireceğini belirler. seri çıkış en sağdaki flip-flop'un çıkışından alınır. Bazen, kaydırmanın yalnızca belirli darbelerle gerçekleşmesini sağlamak gerekir, diğerleriyle değil. Önceki bölümde tartışılan veri kaydı gibi, saat sinyali, kaydın kaydırılmasını önlemek için saat sinyalini kapatarak bastırılabilir. Yüksek hızlı devrelerde tercih edilen bir alternatif, saat sinyalini kapatmak yerine, saat yolunu değiştirmeden bırakıp, her kayıt hücresinin çıkışını i ki kanallı bir mux üzerinden geri döndürmektir; bu mux'un çıkışı hücrenin girişi ne bağlıdır. Saat eylemi bastırılmadığında, mux'un diğer kanalı hücreye bir veri yolu sağlar.

Sonraki bölümde, kaydırma işleminin flip-flop'ların D girişleri aracılığıyla kontrol edilebileceği gösterilecektir, saat girişi aracılığıyla değil. Ancak, Şekil 6.3'teki kaydırma kaydedicisi kullanılıyor a, saat sinyalini bir AND kapısı aracılığıyla bağlayarak bir girişle kaydırma kontrol edilebilir. Bu ter cih edilen bir uygulama değildir. Basitleştirilmiş şemaların sıfırlama sinyalini göstermediğini unut mayın, ancak pratik tasarımlarda böyle bir sinyal gereklidir.

Seri Transfer

Bir dijital sistemin veri yolu, bilginin bir seferde bir bit olarak aktarıldığı ve işlen diği zaman seri modda çalışıyor denir. Bilgi, kaynak kaydından bitlerin kaydırıla rak hedef kayda aktarılmasıyla bir seferde bir bit olarak aktarılır.

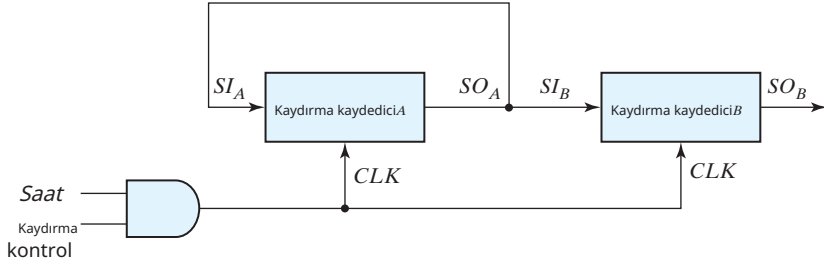
Bu tür bir transfer, tüm register bitlerinin aynı anda aktarıldığı paralel transferle e zıttır.

Bilgilerin kayıt A ile kayıt B arasında seri transferi, Şekil 6.4 (a)'da gösteril diği gibi kaydırma kayıtları ile yapılır. Kayıtın seri çıkışı (SO)

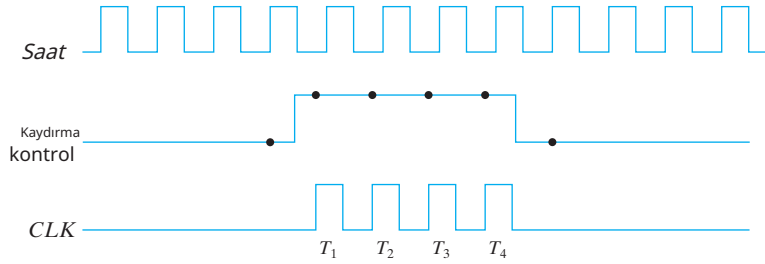
A kaydı B'nin seri girişi (SI) ile bağlantılıdır. Kaynak kaydında saklanan bilgileri n kaybını önlemek için, kayıt A'daki bilgiler, seri çıkışın seri girişi ile bağlanmasıyl a döngüye sokulur. Kayıt B'nin başlangıç içeriği, seri çıkışı aracılığıyla dışarıya kay dırılır ve üçüncü bir kaydırma kaydına aktarılmadığı sürece kaybolur. Kaydırma ko ntrol girişi, kayıtların ne zaman ve kaç kez kaydırılacağını belirler. Burada bir AND kapısı ile saat darbelerinin geçmesine izin verilerek bu işlem gerçekleştirilir.

CLK terminaleri yalnızca kaydırma kontrolü aktif olduğunda. (Bu uygulama, daha önce t artışıldığı gibi, devrenin saat yolunu tehlikeye atabileceğinden sorunlu olabilir.)

Şekil 6.4'teki kaydırma kayıtlarının her birinin dört biti olduğunu varsayalım. O zaman veril erin aktarımını denetleyen kontrol birimi, kaydırmayı etkinleştirecek şekilde tasarlanmalıdır.



(a) Blok diyagramı



(b) Zamanlama diyagramı

ŞEKİL 6.4**Kayıttan A 'dan kayı B 'ye seri transfer**

kaydediciler, bir kelimeyi geçirebilmek için dört saat darbesi boyunca sabit bir süre için kaydırma kontrol sinyali aracılığıyla kaydedilir. Bu tasarım, Şekil 6.4 (b) zamanlama diyagramı nda gösterilmiştir. Kaydırma kontrol sinyali saatle senkronizedir ve saatinin negatif kenarı ndan hemen sonra değer değiştirir. Sonraki dört saat darbesi, kaydırma kontrol sinyalini aktif durumda bulur, bu nedenle AND kapısının CLK girişlerine bağlı çıkışı dört darbe üretir: T_1 , T_2 , T_3 , ve T_4 . Darbenin her yükselen kenarı, her iki kaydedicide de bir kaydırma meydana getirir. Dördüncü darbe, kaydırma kontrolünü 0'a değiştirir ve kaydırma kayıtları devre dışı bırakılır.

Kaydırmadan önce A 'nın ikili içeriğinin 1011 ve B 'nin 0010 olduğunu varsayalım. Seri transfer, A 'dan B 'ye dört adımda gerçekleşir, bu da Tablo 6.1'de gösterilmiştir. İlk puls ile, T_1 , A 'nın en sağdaki biti B 'nin en soldaki bitine kaydırılır ve aynı zamanda A 'nın en soldaki konumuna da döndürülür. Aynı zamanda, A ve B 'nin tüm bitleri bir sağa kaydırılır. B 'den en sağdaki önceki seri çıkış kaybolur ve değeri 0'dan 1'e değişir. Sonraki üç puls, A 'nın bitlerini B 'ye birer birer kaydırarak aynı işlemleri gerçekleştirir. Dördüncü kaydırmadan sonra, kaydırma kontrolü 0'a geçer ve A ve B 'nin her ikisi de 1011 değerine sahip olur. Böylece, A 'nın içeriği B 'ye kopyalanır, böylece A 'nın içeriği değişmeden kalır, yani A 'nın içeriği orijinal değeri ne geri döner.

Bu örnekten seri ve paralel çalışma modları arasındaki fark belirgin olmalıdır. Paralel modda, bir kaydın tüm bitlerinden bilgi mevcut olup, tüm bitler bir saat darbesi sırasında aynı anda aktarılabilir. Seri

Tablo 6.1
Seri Aktarım Örneği

Zamanlama Darbesi	Kaydırma Kaydı A				Kaydırma Kaydı B						
Başlangıç değeri		1	0	1	1		0	0	1	0	
Sonra T_1		1	1	0	1	1	0	0	1	1	0
Sonra T_2		1	1	1	0	1	1	0	1	0	1
Sonra T_3		0	1	1	1	0	1	1	0	1	0
Sonra T_4		1	0	1	1	1	0	1	1	1	1

Modda, kayıtların tek bir seri girişi ve tek bir seri çıkışı vardır. Bilgi, kayıtlar aynı yönde kaydırılırken bir seferde bir bit olarak aktarılır.

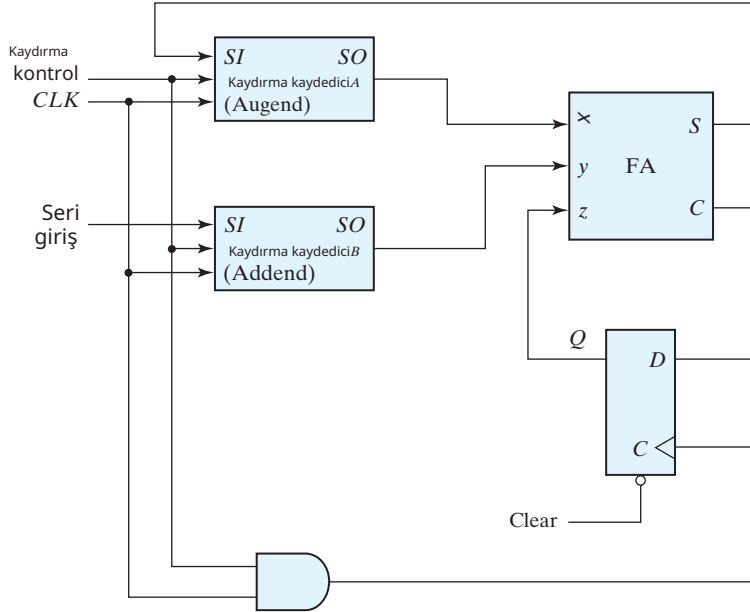
Seri Toplama

Dijital bilgisayarlardaki işlemler genellikle paralel olarak yapılır çünkü bu daha hızlı bir çalışma modudur. Seri işlemler daha yavaştır çünkü bir veri yolu işlemi birkaç saat döngüsü alır, ancak seri işlemlerin daha az donanım bileşeni gerektirme avantajı vardır. VLSI devrelerinde, bir çipte daha az silikon alanı gerektirir. Seri çalışma modunu göstermek için, bir seri toplayıcının tasarımını sunuyoruz. Paralel karşılığı Bölüm 4.4'te sunulmuştur.

Toplanacak iki ikili sayı iki kaydırma kaydında saklanır. En az anlamlı bit çiftinden başlayarak, devre bir seferde bir çift ekler ve bu işlem bir tam toplayıcı (FA) devresi aracılığıyla gerçekleştirilir, bu şekilde gösterilmiştir. Tam toplayıcının taşıma çıkışı bir D flip-flop'a aktarılır, bunun çıkışı daha sonra bir sonraki anlamlı bit çifti için taşıma girişi olarak kullanılır. Tam toplayıcının S çıkışından gelen toplam biti üçüncü bir kaydırma kaydına aktarılabilir. A kaydının bitleri dışarı kaydırılırken toplamı A kaydına kaydırarak, hem augend hem de toplam bitlerini saklamak için tek bir kayıt kullanmak mümkündür. B kaydının seri girişi, toplama sırasında ekleme bitleri dışarı kaydırılırken yeni bir ikili sayıyı aktarmak için kullanılabilir.

Seri toplayıcının çalışması şu şekildedir: İlk olarak, kayıt A toplama sayısını tutar, kayıt B ekleme sayısını tutar ve taşıma flip-flop'u 0'a sıfırlanır. A ve B çıkışları (SO) x ve y için tam toplayıcıya bir çift anlamlı bit sağlar. Flip-flop'un çıkışı Q giriş taşımasını z sağlar. Kaydırma kontrolü, her iki kayıt ve taşıma flip-flop'unu etkinleştirir, bu nedenle bir sonraki saat darbesinde, her iki kayıt bir kez sağa kaydırılır, S bitinin toplamı A içindeki en soldaki flip-flop'a girer ve çıkış taşıması flip-flop Q içine aktarılır. Kaydırma kontrolü, kayıtların bit sayısına eşit saat darbeleri sayısı kadar etkinleştirilir. Her bir sonraki saat darbesinde, yeni bir toplam biti A içine aktarılır, yeni bir taşıma Q içine aktarılır ve her iki kayıt bir kez sağa kaydırılır. Bu işlem, kaydırma kontrolü devre dışı bırakılana kadar devam eder. Böylece, toplama, her bit çiftinin önceki taşıma ile birlikte tek bir tam toplayıcı devresinden geçirilmesi ve toplamın, bir bit bir seferde, kayıt A içine aktarılmasıyla gerçekleştirilir.

Başlangıçta, kayıt A ve taşıma flip-flop'u 0'a sıfırlanır, ardından ilk sayı B'den eklenir. B tam toplayıcıdan geçirilirken, ikinci bir sayı aktarılır.



ŞEKİL 6.5
Seri toplayıcı

seri girişi aracılığıyla ona. İkinci sayı, kayıt içeriğine eklenir A , üçüncü bir sayı ise seri olarak kayıt B 'ye aktarılır. Bu, iki, üç veya daha fazla dört bitlik sayının toplamını almak ve toplamalarını kayıt A 'da biriktirmek için tekrarlanabilir.

Seri toplayıcı ile Bölüm 4.4'te tanımlanan paralel toplayıcıyı karşılaştırdığımızda, birçok fark olduğunu not ediyoruz. Paralel toplayıcı, paralel yüklemeye sahip kayıtlar kullanırken, seri toplayıcı kaydırma kayıtları kullanır. Paralel toplayıcıdaki tam toplayıcı devrelerinin sayısı, ikili sayıların bit sayısına eşittir, oysa seri toplayıcı sadece bir tam toplayıcı devresi ve bir taşıma flip-flop'u gerektirir. Kayıtlar hariç, paralel toplayıcı bir kombinasyon devresidir, oysa seri toplayıcı, çıkış taşımasını depolayan bir flip-flop ve bir tam toplayıcıdan oluşan bir ardışık devredir. Bu tasarım, seri işlemlerde tipiktir çünkü bir bit-zaman işleminin sonucu yalnızca mevcut girişlere değil, aynı zamanda flip-flop'larda depolanması gereken önceki girişlere de bağlı olabilir.

Seri işlemlerin ardışık devre prosedürü ile tasarlanabileceğini göstermek için, seri toplayıcıyı bir durum tablosu kullanarak yeniden tasarlayacağız. Öncelikle, seri olarak toplanacak ikili sayıları depolamak için iki kaydırma kaydının mevcut olduğunu varsayıyoruz. Kayıtlardan gelen seri çıkışlar x ve y ile gösterilmektedir. Tasarlanacak ardışık devre, kaydırma kayıtlarını içermeyecek, ancak tamamlayıcı devreyi göstermek için daha sonra eklenecektir. Ardışık devre, x ve y olmak üzere iki girişi, toplam bitini üreten bir çıkış S ve taşıma bitini depolamak için flip-flop Q içerir. Durum tablosu, ardışık devreyi belirten bilgilerle birlikte Tablo 6.2'de listelenmiştir. Q mevcut durumu, taşımanın mevcut değeridir. Mevcut taşıma,

Tablo 6.2

Seri Toplayıcı için Durum Tablosu

Mevcut Durum	Girişler		Sonraki Durum	Çıktı	Flip-Flop Girişleri	
Q	x	y	Q		J_Q	K_Q
0	0	0	0	0	0	X
0	0	1	0	1	0	X
0	1	0	0	1	0	X
0	1	1	1	0	1	X
1	0	0	0	1	X	1
1	0	1	1	0	X	0
1	1	0	1	0	X	0
1	1	1	1	1	X	0

Q girişler ile birlikte eklenir x ve y çıkışta toplam bitini üretmek için S . Q 'nun bir sonraki durumu çıkış taşıma ile eşittir. Durum tablosu girişlerinin, tam toplayıcı doğruluk tablosundaki girişlerle aynı olduğunu unutmayın, tek fark, giriş taşımanın artık Q 'nun mevcut durumu olması ve çıkış taşımanın artık Q 'nun bir sonraki durumu olmasıdır.

Eğer bir D flip-flop Q için kullanılıyorsa, devre Şekil 6.5'te gösterilen hale gelir. Eğer bir JK flip-flop Q için kullanılıyorsa, girişlerin J ve K değerlerini belirlemek için uyarı tablosuna (Tablo 5.12) başvurmak gereklidir. Bu, Tablo 6.2'nin son iki sütununda yapılır. İki flip-flop giriş denklemi ve çıkış denklemi haritalar aracılığıyla basitleştirilebilir.

$$J_Q = xy$$

$$K_Q = x'y' = (x + y)'$$

$$S = x \oplus y \oplus Q$$

Devre şeması Şekil 6.6'da gösterilmektedir. Devre üç kapı ve bir JK flip-flop'tan oluşmak tadır. İki kaydırıcı kayıt, tam seri toplama işlemini göstermek için diyagrama dahil edilmiştir. Çıkış S 'nin yalnızca x ve y 'ye değil, aynı zamanda Q 'nun mevcut durumuna da bağlı olduğunu unutmayın. Q 'nun bir sonraki durumu, Q 'nun mevcut durumu ve kaydırıcı kayıtların seri çıkışlarından gelen x ve y değerlerine bağlıdır.

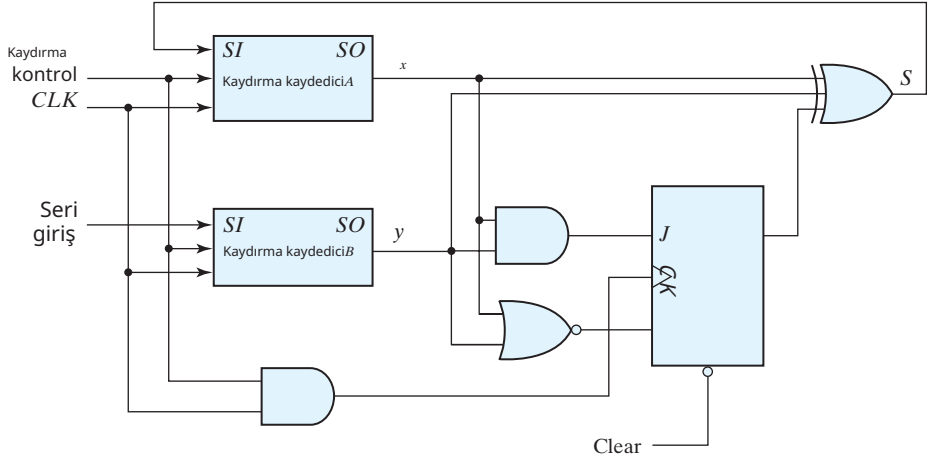
Evrensel Kaydırıcı Kayıt

Eğer bir kaydırıcı kaydın flip-flop çıkışlarına erişim varsa, o zaman seri olarak girilen bilgi, flip-flop'ların çıkışlarından paralel olarak alınabilir. Eğer bir kaydırıcı kayda paralel yükleme yeteneği eklenirse, o zaman paralel olarak girilen veriler, kayıta saklanan verilerin kaydırılmasıyla seri bir şekilde alınabilir.

Bazı kaydırma kayıtları, paralel

aktarım için gerekli giriş ve çıkış terminallerini sağlar. Ayrıca hem sağa hem de sola kaydırma yeteneklerine sahip olabilirler. En genel kaydırma kaydı aşağıdaki yeteneklere sahiptir:

- A temizleme kaydı 0'a temizlemek için kontrol.
- A saat işlemleri senkronize etmek için giriş.



ŞEKİL 6.6
İkinci seri toplayıcı şekli

A *sağa kaydırma* işlemini etkinleştirmek için bir kontrol ve *seri giriş* ve *çıkış* hatlarıyla ilgili sağa kaydırma.

A *sola kaydırma* işlemini etkinleştirmek için bir kontrol ve *seri giriş* ve *çıkış* hatlarıyla ilgili sola kaydırma.

A *paralel yükleme* işlemini etkinleştirmek için bir kontrol ve n giriş hatlarıyla ilgili paralel transfer.

6. n paralel çıkış hatları.

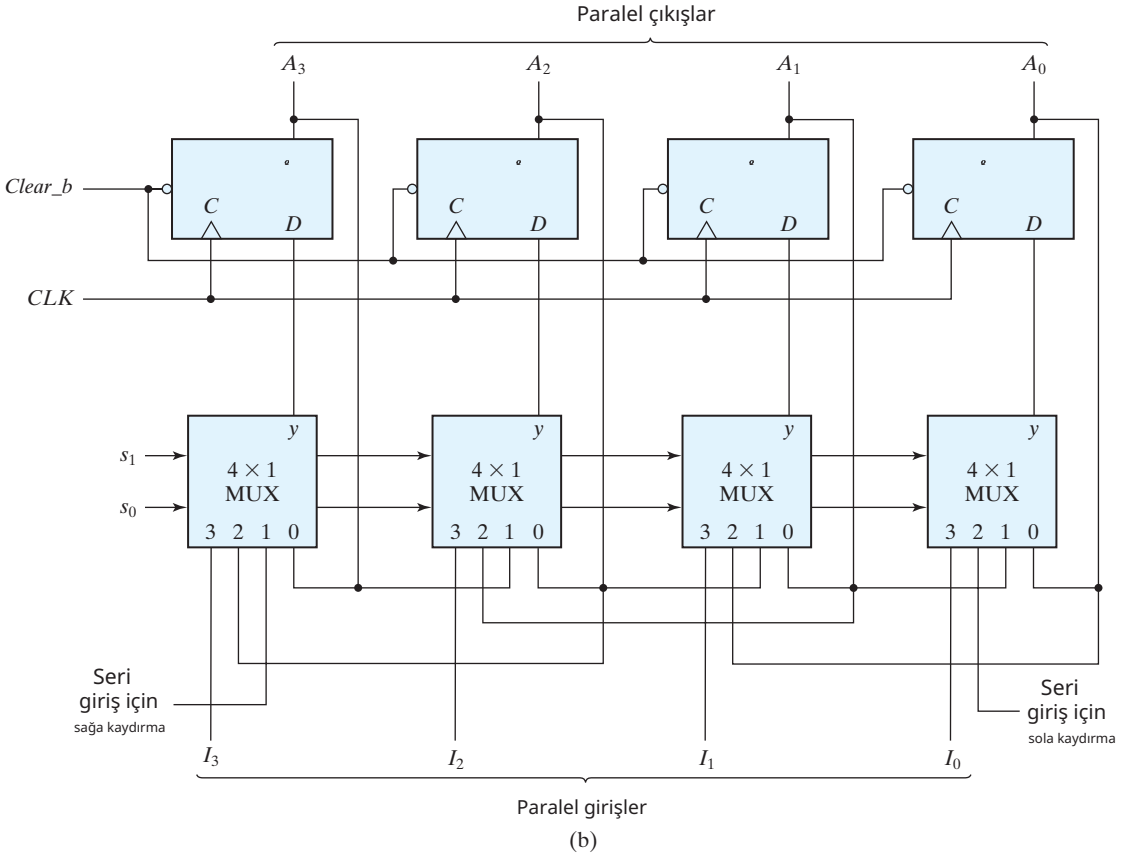
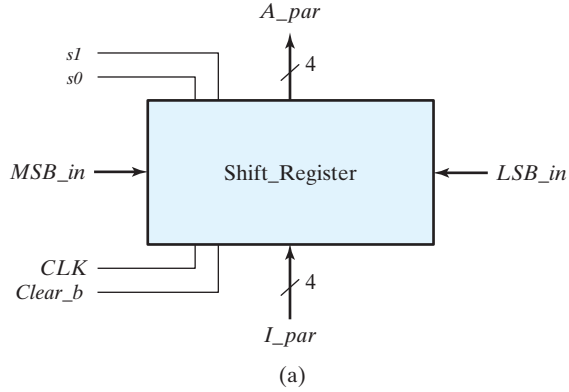
7. Saat sinyaline yanıt olarak kayıttaki bilgileri değiştirmeyen bir kontrol durumu. Diğer kaydırma kayıtları, en az bir kaydırma işlemi ile birlikte, yalnızca bazı önceki işlemlere sahip olabilir.

Yalnızca bir yönde kaydırma yapabilen bir kayıt, *tek yönlü* kaydırma kayıdır. İki yönde kaydırılabilen bir kaydırma kaydı, *iki yönlü* kaydırma kayıdır. Eğer kayıt hem kaydırmayı hem de paralel yükleme yeteneklerine sahipse, evrensel kaydırma kaydı olarak adlandırılır.

Tüm belirtilen yeteneklere sahip dört bitlik evrensel kaydırma kaydının blok diyagramı sembolü ve devre diyagramı Şekil 6.7'de gösterilmiştir. Devre, dört D flip-flop ve dört çoklayıcıdan oluşmaktadır. Dört çoklayıcının iki ortak seçim girişi vardır.

s_1 ve s_0 . Her çoklayıcıda giriş 0, $s_1s_0 = 00$ olduğunda seçilir, giriş 1 ise $s_1s_0 = 01$ olduğunda seçilir ve diğer iki giriş için de benzer şekilde devam eder.

$s_1s_0 = 01$, ve diğer iki giriş için de benzer şekilde devam eder. Seçim girişleri, kaydın çalışma modunu Tablo 6.3'teki fonksiyon girişlerine göre kontrol eder. $s_1s_0 = 00$ olduğunda, kaydın mevcut değeri flip-flopun D girişlerine uygulanır. Bu durumda, her flip-flopun çıkışından aynı flip-flopun girişine bir yol oluşturur, böylece çıkış bu çalışma modunda girişi tekrar besler. Bir sonraki saat kenarı, her flip-flopa da ha önce tuttuğu ikili değeri aktarır ve durum değişikliği gerçekleşmez.



ŞEKİL 6.7

Dört bitlik evrensel kaydırma kaydedici

Tablo 6.3

Şekil 6.7 için Kayıt Fonksiyon Tablosu

Mod Kontrolü		
s_1	s_0	Kayıt İşlemi
0	0	Değişiklik yok
0	1	Sağa kaydır
1	0	Sola kaydır
1	1	Paralel yükle

When $s_1s_0=01$, çoklayıcının girişlerinin terminal 1'inin D girişlerine bir yolu vardır. Bu, flip-flop'lara seri girişin aktarılmasıyla sağlanan bir sağa kaydırma işlemi gerçekleştirir.

A_3 . When $s_0=10$, diğer seri girişin

flip-flop A_0 'ya girmesiyle bir sola kaydırma işlemi gerçekleşir. Finally, when $s_1s_0=11$, paralel giriş hatlarındaki ikili bilgi,

bir sonraki saat kenarında kaydediciye aynı anda aktarılır. Verilerin

enters MSB_{in} için sağa kaydırma işlemi ve LSB_{in} için sola kaydırma işlemi yaptığını unutmayın.

$Clear_b$ tüm flip-flop'ları temizleyen aktif-düşük bir sinyaldir.

Kaydırma kayıtları, genellikle birbirinden uzakta bulunan dijital sistemlerle arayüz sağlamak için kullanılır. Örneğin, iki nokta arasında bir n bit miktarını iletmek gerektiğini varsayalım. Mesafe uzunsa, n bitini paralel olarak iletmek için n hat kullanmak maliyetli olacaktır. Bilgiyi seri olarak, bir seferde bir bit iletmek için tek bir hat kullanmak daha ekonomiktir. Verici, n bit veriyi paralel olarak bir kaydırma kaydedicisine alır ve ardından veriyi ortak hat boyunca seri olarak iletir. Alıcı, verileri seri olarak bir kaydırma kaydedicisine alır. Tüm n bit alındığında, bunlar kaydedicinin çıkışlarından paralel olarak alınabilir. Böylece, verici verilerin paralel-serial dönüşümünü gerçekleştirirken, alıcı serial-paralel dönüşüm yapar.

6.3 DALGALANMA SAYICILARI

Uygulanan giriş pulsları ile belirli bir durum dizisinden geçen bir kayıt, *sayıcı* olarak adlandırılır. Giriş pulsları saat pulsları olabilir veya dış bir kaynaktan gelebilir ve sabit bir zaman aralığında veya rastgele gerçekleşebilir. Durum dizisi, ikili sayı dizisini veya başka bir durum dizisini takip edebilir. İkili sayı dizisini takip eden bir sayıcı, *ikili sayıcı* olarak adlandırılır. Bir

n -bit ikili sayıcı, n adet flip-flop içerir ve 0'dan $2^n - 1$ 'e kadar ikili olarak sayabilir.

Sayıcılar iki kategoriye ayrılır: dalgalanma sayıcıları ve senkron sayıcılar.

Bir dalgalanma sayıcısında, bir flip-flop çıkış geçişi diğer flip-flop'ları tetiklemek için bir kaynak görevi görür. Diğer bir deyişle, bazı veya tüm flip-flop'ların C girişi, ortak saat pulsları ile değil, diğer flip-flop çıkışlarında meydana gelen geçişle tetiklenir.

Bir senkron sayıcıda, tüm flip-flop'ların C girişleri ortak saati alır.

Senkron sayıcılar bir sonraki iki bölümde sunulmaktadır. Burada, ikili ve BCD dalgalanma sayıcılarını tanıtıyor ve çalışma prensiplerini açıklıyoruz.

İkili Dalgalı Sayıcı

A binary ripple counter consists of a series connection of complementing flip-flops, with the output of each flip-flop connected to the C input of the next higher order flip-flop. En düşük anlamlı biti tutan flip-flop, gelen sayım darbelerini alır. Bir tamamlayıcı flip-flop, JK flip-flop'undan J ve K girişleri birleştirilerek veya T flip-flop'undan elde edilebilir. Üçüncü bir olasılık, D flip-flop'unu kullanmak ve tamamlayıcı çıkışı D girişi ne bağlamaktır. Bu şekilde, D girişi her zaman mevcut durumun tamamlayıcısıdır ve bir sonraki saat darbesi flip-flop'un tamamlanmasına neden olacaktır. İki 4-bit binary ripple counter'ın mantık diyagramı Şekil 6.8'de gösterilmektedir. Sayıcı, (a) kısmında T tipi ve (b) kısmında D tipi tamamlayıcı flip-flop'larla inşa edilmiştir.

The output of each flip-flop is connected to the C input of the next flip-flop in sequence.

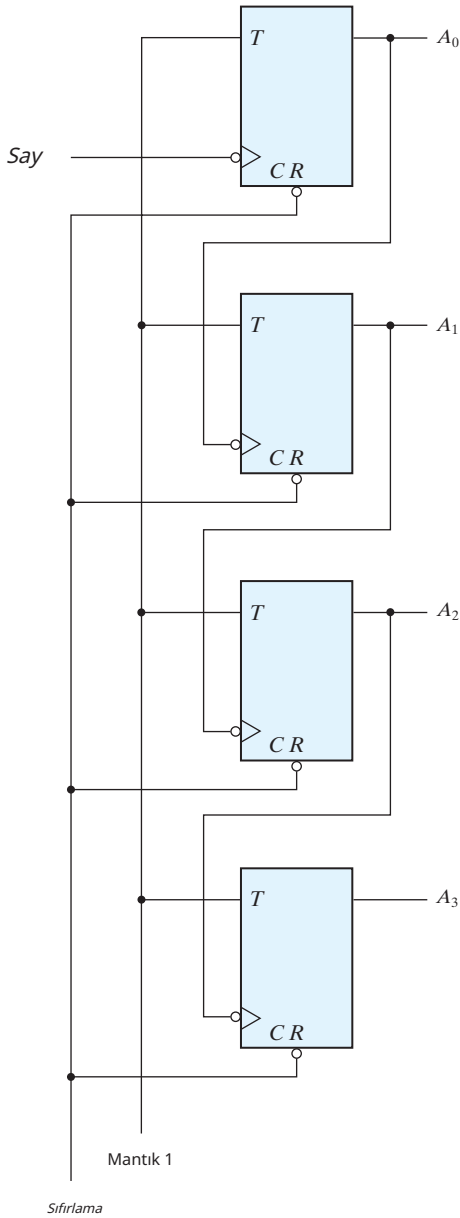
En az anlamlı biti tutan flip-flop, gelen sayım darbelerini alır. Tüm flip-flop'ların girişleri (a) daimi bir mantık 1'e bağlıdır, bu da her flip-flop'un C girişiindeki sinyali negatif bir geçişten geçtiğinde tamamlayıcı hale gelmesini sağlar. C 'nin yaındaki dinamik gösterge sembolünün önündeki baloncuk, flip-flop'ların girişin negatif kenar geçişine yanıt verdiğini gösterir. Negatif geçiş, C 'nin bağlı olduğu önceki flip-flop'un çıkışı 1'den 0'a geçtiğinde gerçekleşir.

Dört bitli ikili dalgalı sayıcının çalışmasını anlamak için, Tablo 6.4'te listelenen ilk dokuz ikili sayıya bakın. Sayım, ikili 0 ile başlar ve her sayım darbesi girişi ile 1 artar. 15 sayımından sonra, sayaç 0'a geri döner ve sayımı tekrarlar. En az anlamlı bit, A_0 , her sayım darbesi girişi ile tamamlanır. A_0 her 1'den 0'a geçtiğinde, A_1 'i tamamlar. A_1 her 1'den 0'a geçtiğinde, A_2 'yi tamamlar. A_2 her 1'den 0'a geçtiğinde, A_3 'ü tamamlar ve bu şekilde dalgalı sayıcının diğer daha yüksek dereceli bitleri için devam eder. Örneğin, 0011 sayımından 0100 sayımına geçişi düşünün. A_0 , sayım darbesi ile tamamlanır. A_0 1'den 0'a geçtiği için, A_1 'i tetikler ve tamamlar. Sonuç olarak, A_1 1'den 0'a geçer, bu da A_2 'yi tamamlar ve onu 0'dan 1'e değiştirir. A_2 , A_3 'ü tetiklemez, çünkü A_2 pozitif bir geçiş üretir ve flip-flop yalnızca negatif geçişlere yanıt verir.

Bu nedenle, 0011'den 0100'e kadar olan sayım, bitleri birer birer değiştirerek elde edilir, bu nedenle

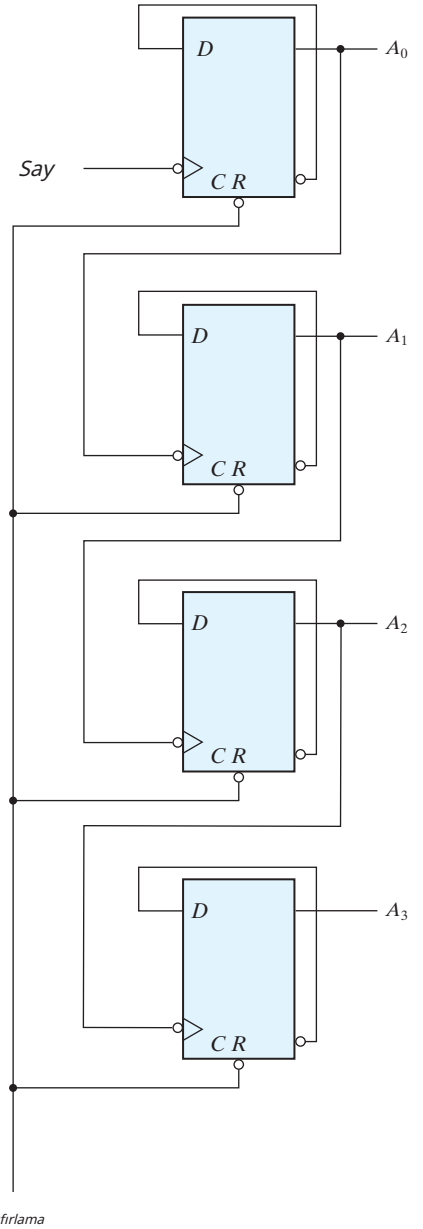
Tablo 6.4
İkili Sayım Dizisi

A_3	A_2	A_1	A_0
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0



Sıfırlama

(a) T flip-flop'lar ile



Sıfırlama

(b) D flip-flop'lar ile

ŞEKİL 6.8
Dört bitli ikili dalgalı sayaç

0011'den 0010'a, ardından 0000'a ve nihayet 0100'a geçer. Flip-flop'lar sırayla birer birer değişir ve sinyal, bir aşamadan diğerine dalgalı bir şekilde sayaçta y ayılır.

Ters sayım yapan bir ikili sayaç, *ikili geri sayım sayacı* olarak adlandırılır. Bir geri sayım sayacında, ikili sayı her giriş sayım darbesi ile 1 azaltılır. Dört bitli bir geri sayım sayacının sayımı ikili 15'ten başlar ve ikilisayılar 14, 13, 12, . . . , 0'a kadar devam eder ve ardından tekrar 15'e döner. Bir ikiligeri sayım sayacı nın sayım dizisi, en az anlamlı bitin her sayım darbesi ile tamamlandığını gösterir. Dizideki diğer herhangi bir bit, önceki en az anlamlı bit 0'dan 1'e geçtiğinde tama mlanır. Bu nedenle, bir ikili geri sayım sayacının diyagramı, tüm flip-flop'ların saat sinyalinin pozitif kenarında tetiklendiği sürece Şekil 6.8'deki ikili dalgalı sayaca be nzer. (C girişlerindeki baloncuk bulunmamalıdır.) Negatif kenar tetiklemeli flip-flo p'lar kullanılıyorsa, her flip-flop'un C girişi, önceki flip-flop'un tamamlanmış çıkışın a bağlanmalıdır. Böylece, gerçek çıkış 0'dan 1'e geçtiğinde, tamamlanmış çıkış 1'd en 0'a geçer ve gerekli olduğu gibi bir sonraki flip-flop'u tamamlar.

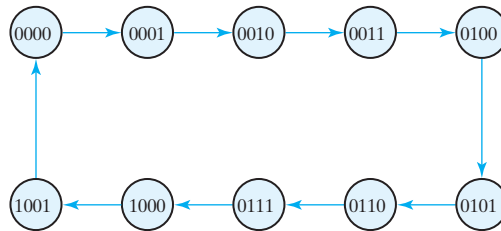
BCD Ripple Sayıcı

Onluk bir sayıcı, 10 durumdan oluşan bir diziyi takip eder ve 9 sayımından sonra 0'a döner. Böyle bir sayıcının her ondalık basamağı temsil etmek için en az dört flip-flop'a sahip olması gere kir, çünkü onluk bir basamak, en az dört bit ile temsil edilen bir ikili kod ile gösterilir. Bir ondalık s ayıcının durum dizisi, bir ondalık basamağı temsil etmek için kullanılan ikili kod tarafından belirle nir. BCD kullanıldığında, durum dizisi Şekil 6.9'daki durum diyagramında gösterildiği gibidir. Bir ondalık sayıcı, 1001'den (ondalık basamak 9'un kodu) sonraki durumun 0000 o lduğu (ondalık basamak 0'ın kodu) bir ikili sayıcıya benzer.

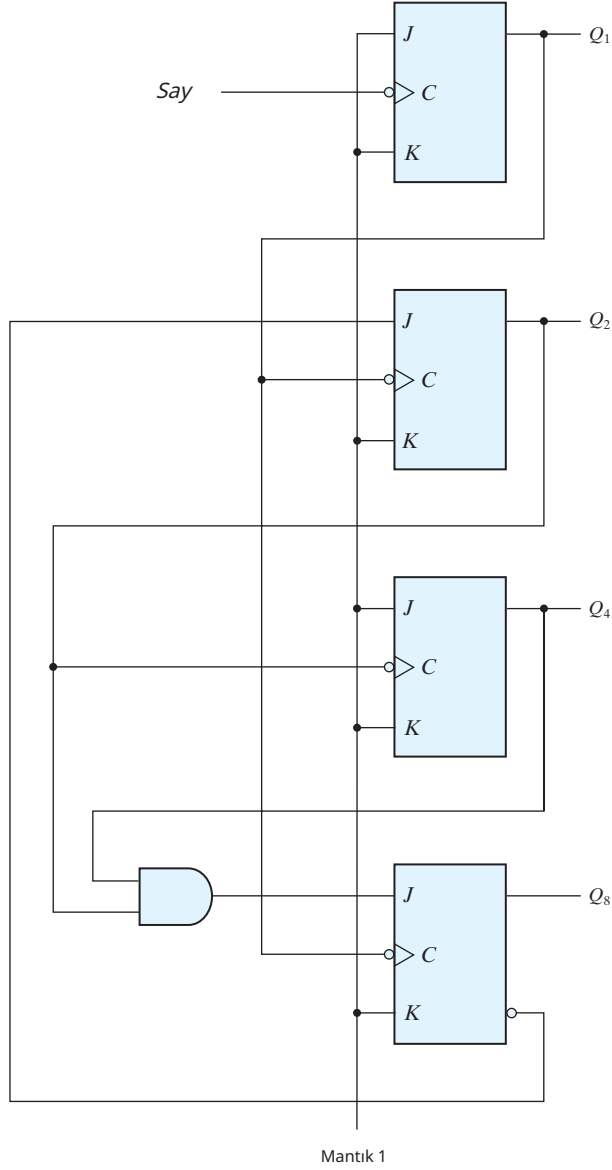
JK flip-flop'ları kullanan bir BCD ripple sayıcının mantık diyagramı Şekil 6.10'da gösterilmiştir. Dört çıkış, BCD kodundaki karşılık gelen bitin ikili ağırlığına eşit bir sayısal alt si mge ile birlikte *Q* harfi ile belirtilmiştir. Çıktının

Q_1 hem Q_2 hem de Q_8 için *C* girişlerine uygulandığını ve Q_2 çıkışının Q_4 için *C* girişine uygulandığını unutmayın. *J* ve *K* girişleri ya sürekli bir 1 sinyaline ya da diğer flip-flop'ların çıkışlarına bağlanmıştır.

Bir dalga sayacı, asenkron ardışık bir devredir. Flip-flop geçişini etkileyen sin yaller, 1'den 0'a geçiş şekline bağlıdır. Sayacın çalışması

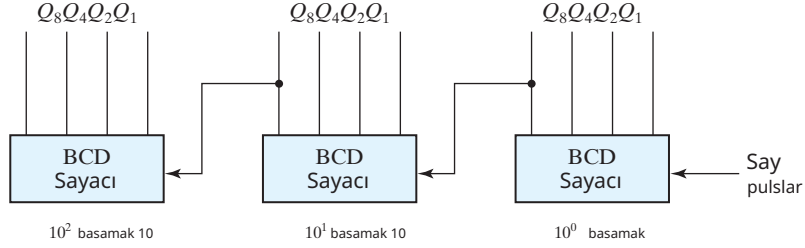


ŞEKİL 6.9
Ondalık BCD sayıcı durum diyagramı



ŞEKİL 6.10
BCD dalgalı sayıcı

flip-flop geçişleri için bir dizi koşul ile açıklanabilir. Bu koşullar, mantık diyagramından ve bir *JK flip-flop çalışır. C girişi 1'den 0'a geçtiğinde, flip-flop, J = 1 ise ayarlanır, K = 1 ise sıfırlanır, J=K= 1 ise tamamlanır, J=K= 0 ise değişmeden kalır.*



ŞEKİL 6.11
Üç ondalık BCD sayıcı blok diyagramı

Bu koşulların bir BCD dalgalı sayıcı tarafından gereken diziye sağladığını doğrulamak için, fl ip-flop geçişlerinin gerçekten Fig. 6.9 'da belirtilen durumlar dizisini takip ettiğini doğrulam ak gerekir. Q_1 her saat pulsünden sonra durum değiştirir. Q_2 her seferinde Q_1 1'de n 0'a geçtiğinde tamamlanır, eğer $Q_8=0$ ise. Q_8 1 olduğunda, Q_2 0'da kalır. Q_4 he r seferinde Q_2 1'den 0'a geçtiğinde tamamlanır. Q_8 Q_2 veya Q_4 0 olduğu sürece 0'da kalır. Her iki Q_2 ve Q_4 1 olduğunda, Q_8 Q_1 1'den 0'a geçtiğinde tamamlanır. Q_8 bir sonraki geçişte Q_1 sıfırlanır.

Fig. 6.10 'daki BCD sayıcı, *onluk* sayıcıdır, çünkü 0'dan 9'a kadar sayar. 0'd an 99'a kadar ondalık saymak için iki onluk sayıcıya ihtiyacımız var. 0'dan 999'a ka dar saymak için üç onluk sayıcıya ihtiyacımız var. Birden fazla onluk sayıcı, her onlu k için bir BCD sayıcıyı kaskad olarak bağlayarak oluşturulabilir. Üç onluk sayıcı Fi g. 6.11 'de gösterilmiştir. İkinci ve üçüncü onluğa girişler, önceki onluğun Q_8 çıkışından gelir. Bir onlukta Q_8 1'den 0'a geçtiğinde, kendi onluğu 9'dan 0'a g eçerken bir sonraki daha yüksek onluğun sayımını tetikler.

6.4 SENKRON SAYACILAR

Senkron sayıcılar, saat darbelerinin tüm flip-flop'ların girişlerine uygulandığı rip ple sayıcılardan farklıdır. Ortak bir saat, tüm flip-flop'ları aynı anda tetikler, bir r ipple sayıcıda olduğu gibi ardışık olarak birer birer değil. Bir flip-flop'un tamaml anıp tamamlanmayacağı kararı, saat kenarında T veya J ve K gibi veri girişlerini n değerlerine göre belirlenir. Eğer $T = 0$ veya $J = K = 0$ ise, flip-flop durumu de ği şmez. Eğer $T = 1$ veya $J = K = 1$ ise, flip-flop tamamlanır.

Senkron sayıcıların tasarım prosedürü Bölüm 5.8'de sunulmuştur ve üç bitli bir ikili sayıcının tasarımı Şekil 5.31 ile birlikte gerçekleştirilmiştir. Bu bölümde, bazı tipik senkron sayıcıları sunuyor ve çalışma prensiplerini açıklıyoruz.

İkili Sayıcı

Senkron bir ikili sayıcının tasarımı o kadar basittir ki, ardışık bir mantık tasarım sürecinden geçmeye gerek yoktur. Senkron bir ikili sayıcıda, en az anlamlı konu mdaki flip-flop her darbe ile tamamlanır. Başka bir konumdaki bir flip-flop ise...

konum, alt anlamlı pozisyonlardaki tüm bitler 1'e eşit olduğunda tamamlanır.

Örneğin, dört bitli bir sayacın mevcut durumu $A_3A_2A_1A_0 = 0011$ ise, bir sonraki sayım 0100'dür. A_0 her zaman tamamlanır. A_1 mevcut durumun $A_0 = 1$ olması nedeniyle tamamlanır. A_2 mevcut durumun $A_1A_0 = 11$ olması nedeniyle tamamlanır. Ancak, A_3 tamamlanmaz, *çünkü mevcut durum*

AZA1A0 = 011 olup, bu tüm 1'ler koşulunu sağlamaz.

Senkranize ikili sayıcılar düzenli bir desene sahiptir ve tamamlayıcı flip-floplar ve kapılar ile inşa edilebilir. Düzenli desen, Şekil 6.12'de gösterilen dört bitli sayacın üzerinden görülebilir. Tüm flip-flopların C girişleri ortak bir saate bağlıdır. Sayaç, *Count_enable* ile etkinleştirilir. Eğer etkinleştirme girişi 0 ise, tüm J ve K girişleri 0'a eşit olur ve saat sayacın durumunu değiştirmez. İlk aşama, A_0 , sayıcı etkinleştirildiğinde J ve K 1'e eşittir. Diğer J ve K girişleri, tüm önceki en az anlamlı aşamalar 1'e eşit ve sayım etkinleştirildiğinde 1'e eşittir. VE kapılarının zinciri, her aşamadaki J ve K girişleri için gerekli mantığı üretir. Sayıcı, her aşamada bir ek flip-flop ve tüm önceki flip-flop çıkışları 1 olduğunda 1 çıkışı veren bir VE kapısı ile birlikte herhangi bir sayıda aşamaya genişletilebilir.

Flip-flop'ların saat sinyalinin pozitif kenarında tetiklendiğini unutmayın. Saatin polaritesi burada önemli değildir, ancak dalgalı sayıcıda önemlidir. Senkron sayıcı, pozitif veya negatif saat kenarı ile tetiklenebilir. İkili sayıcıdaki tamamlayıcı flip-flop'lar JK tipi, T tipi veya D tipi XOR kapıları ile olabilir. Üç tipin eşdeğeri Şekil 5.13'te gösterilmektedir.

Artı-Eksi İkili Sayıcı

Senkron bir geri sayım ikili sayıcı, ikili durumları ters sırayla, 1111'den 0000'a ve tekrar 1111'e geri dönerek geçer. Geri sayım sayıcısını alışıldık şekilde tasarlamak mümkün değildir, ancak sonuç, aşağı doğru ikili sayımın incelenmesiyle tahmin edilebilir. En az anlamlı pozisyonundaki bit, her darbe ile tamamlanır. *Diğer herhangi bir pozisyonundaki bir bit, tüm daha düşük anlamlı bitler 0'a eşit olduğunda tamamlanır.*

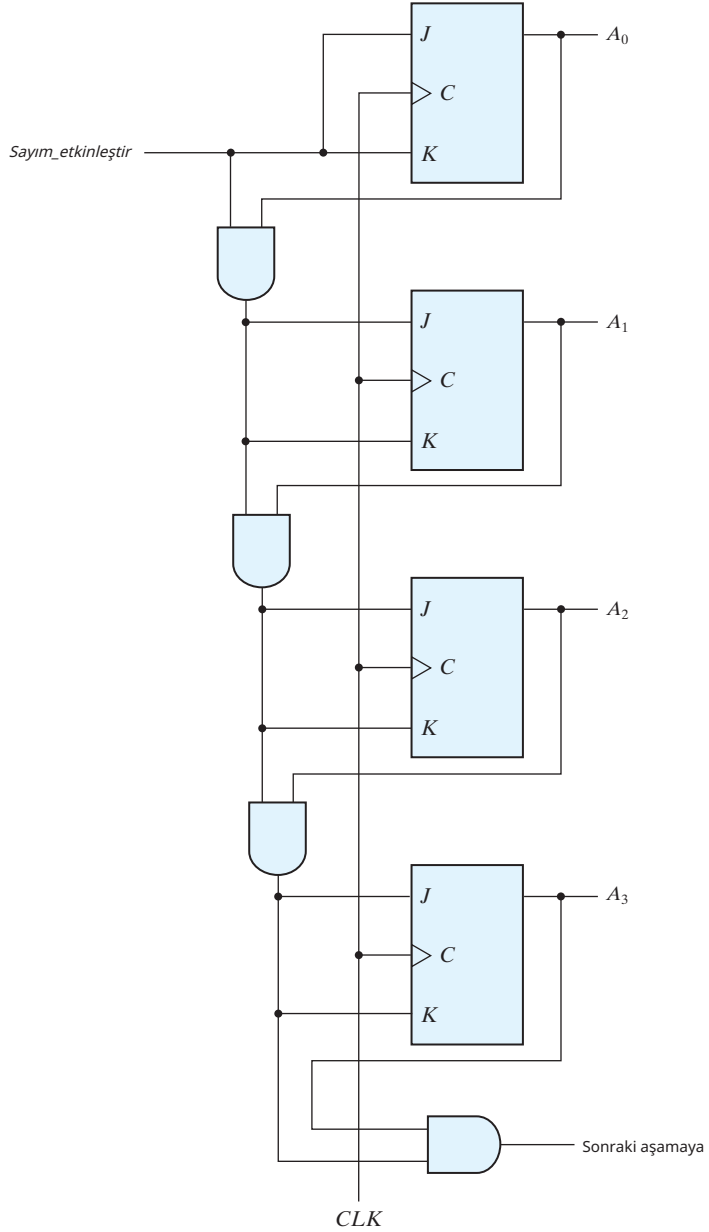
Örneğin, mevcut 0100 durumundan sonraki durum 0011'dir. En az anlamlı bit her zaman tamamlanır. İkinci anlamlı bit, birinci

bitin 0 olduğu için tamamlanır. Üçüncü anlamlı bit, ilk iki bit 0'a eşit olduğu için tamamlanır.

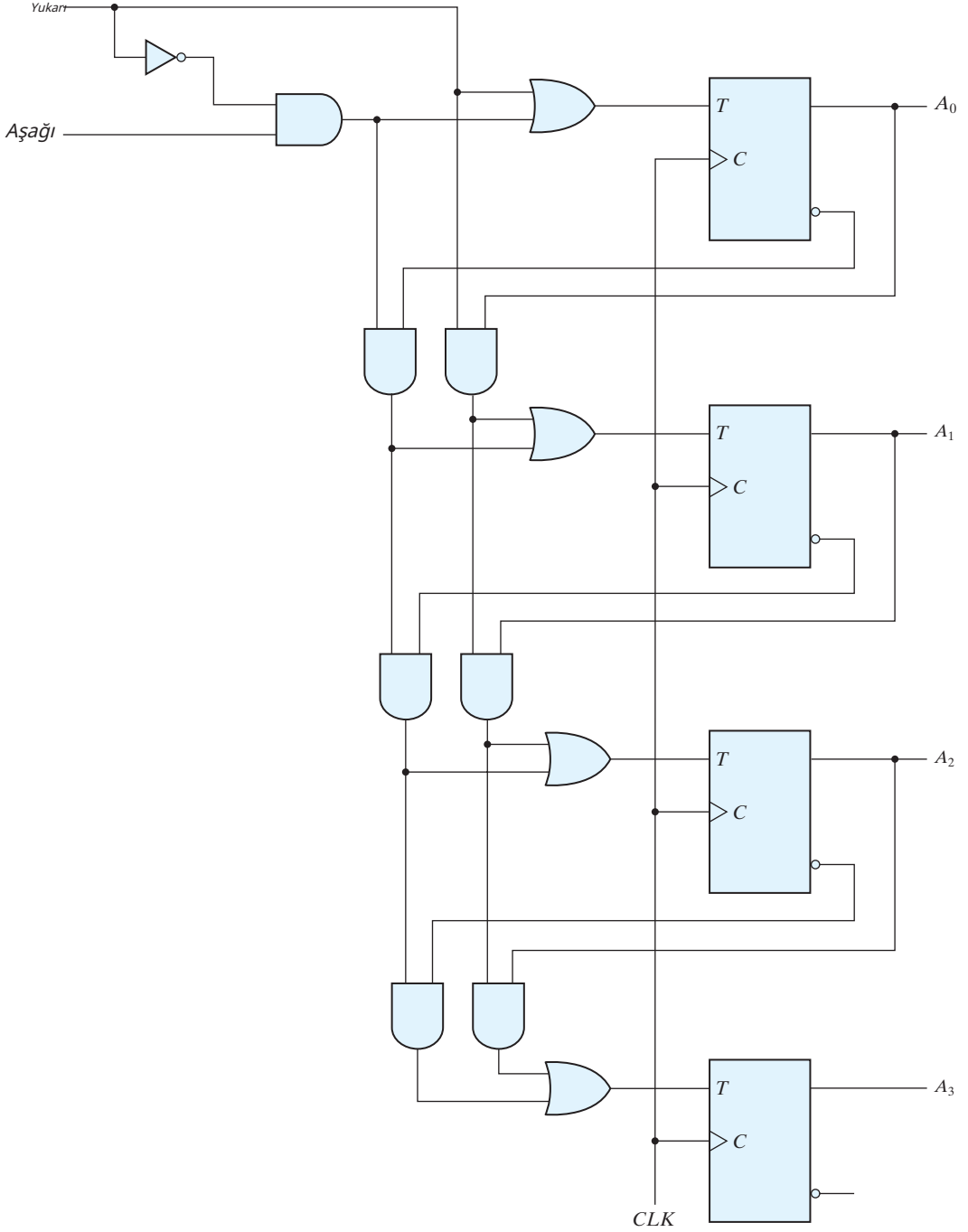
Ancak dördüncü bit değişmez, çünkü tüm daha düşük anlamlı bitler 0'a eşit değildir.

Bir geri sayım ikili sayıcı, Şekil 6.12'de gösterildiği gibi inşa edilebilir, ancak AND kapılarının girişleri, önceki flip-flopların normal çıkışlarından değil, tamamlanmış çıkışlardan gelmelidir. İki işlem, yukarı veya aşağı sayabilen bir sayıcı oluşturmak için tek bir devrede birleştirilebilir. Şekil 6.13'te, T flip-flopları kullanan bir yukarı-aşağı ikili sayıcının devresi gösterilmektedir. Bir yukarı kontrol girişi ve bir aşağı kontrol girişi vardır. Yukarı girişi 1 olduğunda, devre yukarı sayar, çünkü T girişleri, flip-flopların önceki normal çıkışlarının değerlerinden sinyaller alır. Aşağı girişi 1 ve yukarı girişi 0 olduğunda, devre aşağı sayar, çünkü önceki flip-flopların tamamlanmış çıkışları T girişlerine uygulanır.

Yukarı ve aşağı girişleri her ikisi de 0 olduğunda, devre durum değiştirmez ve kalır.



ŞEKİL 6.12
Dört bitli senkron ikili sayıcı



ŞEKİL 6.13
Dört bitli yukarı-aşağı ikili sayıcı

a. Yukarı ve aşağı girişleri her ikisi de 1 olduğunda, devre yukarı sayar. Bu koşul seti, belirli bir anda yalnızca bir işlemin gerçekleştirilmesini sağlar. Yukarı girişinin, aşağı girişine göre önceliği olduğunu unutmayın.

BCD Sayıcı

A BCD sayacı, ikili kodlu ondalık sistemde 0000'dan 1001'e ve geri 0000'a kadar sayar. 9 sayımından sonra 0'a dönüş nedeniyle, bir BCD sayacı düz bir ikili sayımın aksine düzenli bir örneğe sahip değildir. Bir BCD senkron sayacının devresini elde etmek için, ardışık bir devre tasarım prosedüründen geçmek gereklidir.

BCD sayacının durum tablosu Tablo 6.5'te listelenmiştir. Giriş koşulları için T flip-flop'lar mevcut ve sonraki durum koşullarından elde edilir. Ayrıca tabloda, mevcut durum 1001 olduğunda 1'e eşit olan bir çıkış y gösterilmektedir. Bu şekilde, y bir sonraki daha yüksek onluğun sayımını etkinleştirebilirken, aynı darbe mevcut onluğa 1001'den 0000'a değiştirir.

Flip-flop giriş denklemleri haritalar aracılığıyla basitleştirilebilir. Minterm 10'dan 15'e kadar kullanılan durumlar, önemsenmeyen terimler olarak alınır. Basitleştirilmiş fonksiyonlar şunlardır:

$$T_{Q1} = 1$$

$$T_{Q2} = Q_8'Q_1$$

$$T_{Q4} = Q_2Q_1$$

$$T_{Q8} = Q_8Q_1 + Q_4Q_2Q_1$$

$$y = Q_8Q_1$$

Dört T flip-flop, beş AND kapısı ve bir OR kapısıyla devre kolayca çizilebilir. Senkron BCD sayıcılar, herhangi bir uzunluktaki ondalık sayılar için bir sayıcı oluşturmak üzere zincirlenebilir. Zincirleme, Şekil 6.11'deki gibi yapılır, ancak çıkış y , bir sonraki daha yüksek anlamlı onluğun sayım girişine bağlanmalıdır.

Tablo 6.5

BCD Sayıcı için Durum Tablosu

Mevcut Durum				Sonraki Durum				Çıktı	Flip-Flop Girişleri			
Q_8	Q_4	Q_2	Q_1	Q_8	Q_4	Q_2	Q_1	y	TQ_8	TQ_4	TQ_2	TQ_1
0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	1	0	0	1	0	0	0	0	1	1
0	0	1	0	0	0	1	1	0	0	0	0	1
0	0	1	1	0	1	0	0	0	0	1	1	1
0	1	0	0	0	1	0	1	0	0	0	0	1
0	1	0	1	0	1	1	0	0	0	0	1	1
0	1	1	0	0	1	1	1	0	0	0	0	1
0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	0	0	1	0	0	0	0	1
1	0	0	1	0	0	0	0	1	1	0	0	1

Paralel Yüklemeli İkili Sayıcı

Dijital sistemlerde kullanılan sayıcılar, genellikle sayım işlemi öncesinde sayıcıya bir başlangıç ikili sayıyı aktarmak için paralel yükleme yeteneğine ihtiyaç duyar.

Şekil 6.14, paralel yükleme yeteneğine sahip ve sayıcı olarak çalışabilen dört bitlik bir kaydı n üst düzey blok diyagramı sembolünü ve mantık diyagramını göstermektedir. Giriş yük kontrolü 1'e eşit olduğunda, sayım işlemini devre dışı bırakır ve dört veri girişinden verilerin dört flip-flop'a aktarılmasına neden olur. Her iki kontrol girişi 0 olduğunda, saat darbeleri kaydın durumunu değiştirmez.

Taşıma çıkışı, sayım girişi etkinleştirildiğinde tüm flip-flop'lar 1'e eşit olduğunda 1 olur. Bu, bir sonraki anlamlı biti tutan flip-flop'un tamamlanması için gereken koşuldur. Taşıma çıkışı, sayıcıyı dört bitten daha fazlasına genişletmek için faydalıdır. Taşıma, tüm dört flip-flop'un çıkışlarından doğrudan üretildiğinde sayıcının hızı artar, çünkü taşıma bitini üretmek için gereken gecikme azalır. 1111 durumundan 0000 durumuna geçerken yalnızca bir kapı gecikmesi meydana gelirken, Şekil 6.12'de gösterilen AND kapısı zincirinde dört kapı gecikmesi meydana gelir. Benzer şekilde, her flip-flop, tüm önceki flip-flop çıkışlarını doğrudan alan bir AND kapısıyla ilişkilidir; AND kapıları bir zincir halinde bağlanmak yerine.

Sayacın çalışması, Tablo 6.6'da özetlenmiştir. Dört kontrol girişi—*Clear*, *CLK*, *Load*, ve *Count*—bir sonraki durumu belirler. *Clear* girişi asenkron olup, 0'a eşit olduğunda, saat darbeleri veya diğer girişlerin varlığına bakılmaksızın sayacı sıfırlanmasına neden olur. Bu ilişki, diğer girişler için önemsiz durumları simgeleyen X girişleriyle tabloda gösterilmektedir. *Clear* girişi, diğer tüm işlemler için 1 durumunda olmalıdır. *Load* ve *Count* girişleri her ikisi de 0 olduğunda, saat darbeleri uygulanırsa bile çıkışlar değişmez. 1 değerine sahip bir *Load* girişi, *CLK*'nin pozitif kenarında girişlerden *I0*-*I3* kayıtlarına aktarım yapılmasına neden olur. Giriş verileri, *Count* girişinin değerine bakılmaksızın kayıtlarına yüklenir, çünkü Sayım girişi, Yük girişi etkinleştirildiğinde engellenir. Yük girişi, sayacın çalışmasını kontrol etmek için 0 olmalıdır.

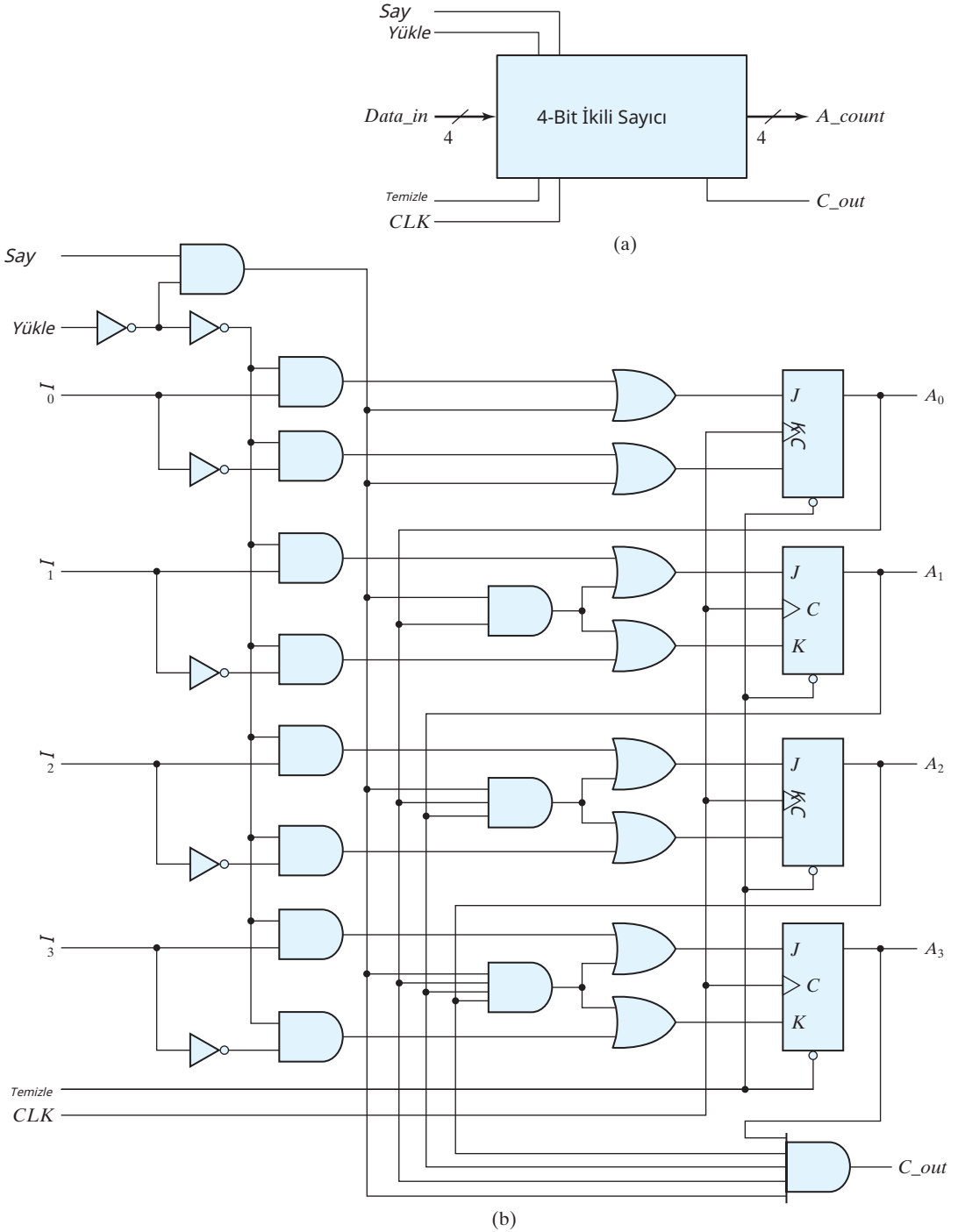
Paralel yükü olan bir sayaç, istenen herhangi bir sayım dizisini oluşturmak için kullanılabilir. Şekil 6.15, paralel yükü olan bir sayacın BCD sayımını oluşturmak için kullanıldığı iki yolu göstermektedir. Her durumda, sayımın *CLK* girişi üzerinden etkinleştirilmesi için *Sayım* kontrolü 1 olarak ayarlanır. Ayrıca, *Yük* kontrolünün sayımı engellediğini ve temizleme işleminin diğer kontrol girişlerinden bağımsız olduğunu hatırlayın.

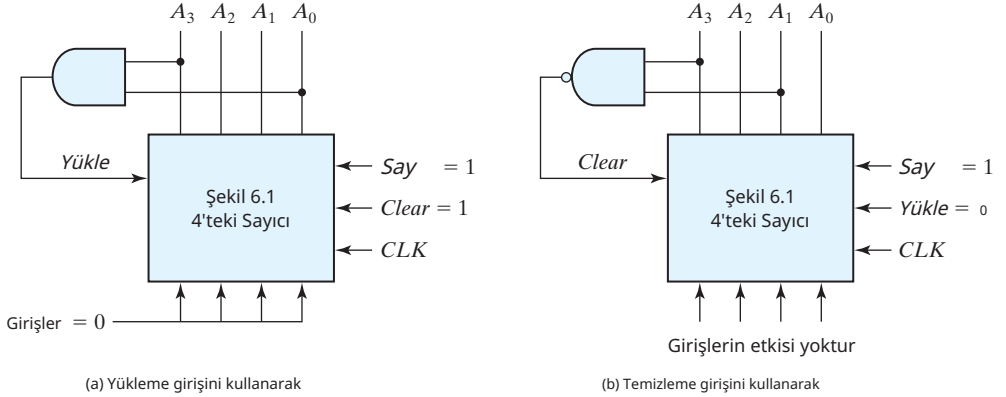
Şekil 6.15 (a) içindeki AND kapısı, 1001 durumunun oluşumunu tespit eder. Sayaç başlangıçta 0'a temizlenir ve ardından *Temizle* ve *Sayım* girişleri 1 olarak ayarlanır, böylece sayaç her zaman aktiftir. AND kapısının çıkışı 0 olduğu sürece, her pozitif kenar saat

Tablo 6.6

Şekil 6.14 için Sayaç Fonksiyon Tablosu

Temizle	CLK	Yük	Sayım	Fonksiyonu
0	X	X	X	0'a Temizle
1	↑	1	X	Girişleri Yükle
1	↑	0	1	Sonraki ikili durumu Say
1	↑	0	0	Değişiklik yok





ŞEKİL 6.15

Paralel yükleme ile bir BCD sayacı elde etmenin iki yolu

sayacı 1 artırır. Çıktı 1001 sayısına ulaştığında, hem A_0 hem de A_3 1 olur, AND kapısının çıktısını 1 yapar. Bu durum *Yükle* girişini etkinleştirir; bu nedenle, bir sonraki saat kenarında kayıt saymaz, ancak dört girişinden y yüklenir. Tüm dört giriş mantık 0'a bağlı olduğundan, 1001 sayısından sonra kayda tüm 0'lar değeri yüklenir. Böylece, devre 0000'dan 1001'e ve tekrar 0000'a geçer, bu da BCD sayıcıda gereklidir.

Şekil 6.15 (b)'de, NAND kapısı 1010 sayısını algılar, ancak bu sayı oluşur oluşmaz, kayıt temizlenir. 1010 sayısının herhangi bir anlamlı süre boyunca kalma şansı yoktur, çünkü kayıt hemen 0'a döner. 1010'dan 1011'e ve hemen ardından 0000'a geçerken çıktıda A_0 anlık bir dalgalanma meydana gelir. Bu dalgalanma istenmeyen bir durum olabilir ve bu nedenle, bu yapılandırma önerilmez. Eğer sayıcının senkron temizleme girişi varsa, 1001 sayısı oluştuğunda saatin ardından sayıcıyı temizlemek mümkündür.

6.5 DİĞER SAYICILAR

Sayıclar, istenen herhangi bir durum dizisini oluşturacak şekilde tasarlanabilir. Bir bölme- N sayacı (modulo- N sayacı olarak da bilinir), tekrar eden bir N durumu dizisinden geçen bir sayıcıdır. Dizi, ikili sayım izleyebilir veya başka herhangi bir rastgele dizi olabilir. Sayıclar, dijital bir sistemdeki işlem dizisini kontrol etmek için zamanlama sinyalleri üretmekte kullanılır. Sayıclar ayrıca kaydırma kayıtları aracılığıyla da inşa edilebilir. Bu bölümde, ikili olmayan sayıcılara birkaç örnek sunuyoruz.

Kullanılmayan Durumlarla Sayıcı

n flip-flop içeren bir devre, 2 ikili duruma sahiptir. Bir sıralı

devrenin, bu maksimum olası durum sayısından daha azını kullandığı durumlar vardır. Kullanılmayan durumlar

Bölüm 6.5 Diğer Sayıcılar 279 sıralı devreyi tanımlarken durum tablosunda listelenmemiştir. Giriş denklemlerini basitleştirirken, kullanılmayan durumlar don't-care koşulları olarak ele alınabilir veya belirli bir sonraki duruma atanabilir. Devre tasarlandığında ve inşa edildiğinde, çalışması sırasında dış müdahale devrenin kullanılmayan durumlardan birine girmesine neden olabilir. Bu durumda, devrenin normal çalışmaya devam edebilmesi için sonunda geçerli durumlardan birine girmesini sağlamak önemlidir. Aksi takdirde, eğer sıralı devre kullanılmayan durumlar arasında dolaşırsa, onu planlanan durum geçişleri dizisine geri getirmek için hiçbir yol olmayacaktır. Kullanılmayan durumlar don't-care koşulları olarak ele alındığında, devre tasarlandıktan sonra kullanılmayan durumların etkisini belirlemek için araştırma yapılmalıdır. Kullanılmayan bir durumdan sonraki durum, devre tasarlandıktan sonra devrenin analizi ile belirlenebilir.

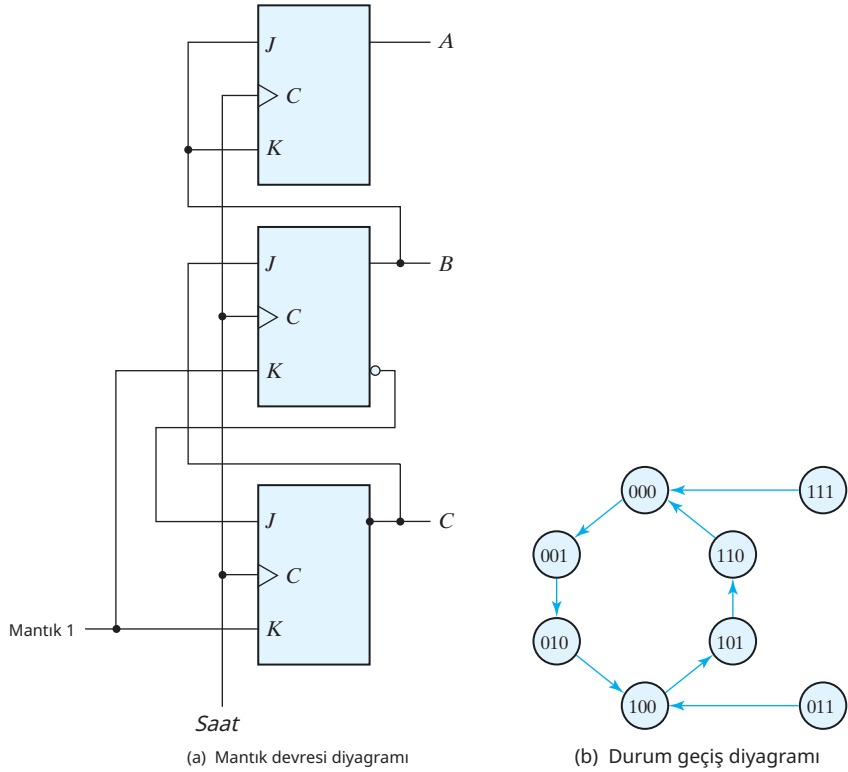
Örnek olarak, Tablo 6.7'de belirtilen sayıcıyı düşünün. Sayım, altı durumun tekrarlanan bir dizisine sahiptir; flip-flop'lar B ve C ikili sayım 00, 01, 10'u tekrar ederken, flip-flop A her üç sayımda bir 0 ve 1 arasında geçiş yapmaktadır. Sayıcının sayım dizisi düz bir ikili değildir ve 011 ile 111 durumları sayımda yer almaz. JK flip-flop'larının seçimi, tabloda listelenen flip-flop giriş koşullarını sonuçlandırır. Girişler K_B ve K_C sütunlarında yalnızca 1'ler ve X'ler içerdiğinden, bu girişler her zaman 1'e eşittir. Diğer flip-flop giriş denklemleri, mintermler 3 ve 7'yi dikkate alarak basitleştirilebilir. Basitleştirilmiş denklemler şunlardır:

$$\begin{aligned} J_A &= B & K_A &= B \\ J_B &= C & K_B &= 1 \\ J_C &= B' & K_C &= 1 \end{aligned}$$

Sayıcının mantık diyagramı Şekil 6.16 (a)'de gösterilmektedir. İki kullanılmayan durum olduğundan, devreyi analiz ederek etkilerini belirliyoruz. Eğer devre bir hataya sebebiyet veriyorsa 011 durumunda olursa, bir saat darbesi uygulandıktan sonra devre 100 durumuna geçer. Bu işlem, mantık diyagramının incelenmesiyle belirlenebilir; $B=1$ olduğunda, bir sonraki saat kenarı A 'yı tamamlar ve C 'yi 0'a sıfırlar, ve ne zaman $C=1$, bir sonraki saat kenarı B 'yi tamamlar. Benzer şekilde, mevcut durum 111'den bir sonraki durumu 000 olarak değerlendirebiliriz.

Tablo 6.7
Sayıcının Durum Tablosu

Mevcut Durum			Sonraki Durum			Flip-Flop Girişleri					
A	B	C	A	B	C	J_A	K_A	J_B	K_B	J_C	K_C
0	0	0	0	0	1	0	X	0	X	1	X
0	0	1	0	1	0	0	X	1	X	X	1
0	1	0	1	0	0	1	X	X	1	0	X
1	0	0	1	0	1	X	0	0	X	1	X
1	0	1	1	1	0	X	0	1	X	X	1
1	1	0	0	0	0	X	1	X	1	0	X



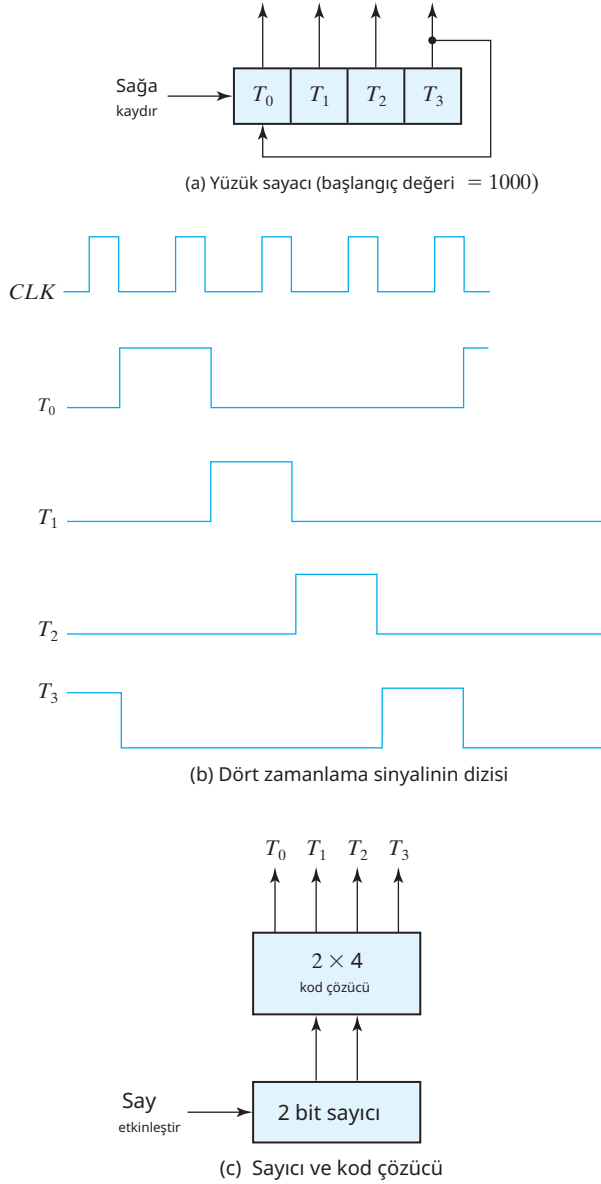
ŞEKİL 6.16

Kullanılmayan durumlarla sayaç

Kullanılmayan durumların etkisini içeren durum diyagramı Şekil 6.16 (b)'de gösterilmiştir. Eğer devre dışsal bir müdahale nedeniyle kullanılmayan durumlardan birine giderse, bir sonraki sayım darbesi onu geçerli durumlardan birine aktarır ve devre doğru bir şekilde saymaya devam eder. Böylece, sayıcı kendini düzeltir. Kendini düzelten bir sayıcıda, eğer sayıcı kullanılmayan durumlardan birindeyse, bir veya daha fazla saat darbesinden sonra normal sayım sırasına ulaşır. Alternatif bir tasarım, her kullanılmayan durumu belirli bir sonraki duruma yönlendirmek için ek mantık kullanabilir.

Halka Sayıcı

Zamansal sinyaller, dijital bir sistemdeki işlem sırasını kontrol eden, bir kaydırma kaydedici ve ya bir dekodere ile bir sayaç tarafından üretilir. Bir *halka sayıcı*, belirli bir zamanda yalnızca bir flip-flop'un ayarlandığı dairesel bir kaydırma kaydedicisidir; diğer tüm flip-flop'lar sıfırlanır. Tek bit, zamansal sinyal dizisini üretmek için bir flip-flop'tan diğerine kaydırılır. Şekil 6.17 (a), bir halka sayıcı olarak bağlanmış dört bitlik bir kaydırma kaydedicisini göstermektedir. Kaydedicinin başlangıç değeri 1000'dir ve Preset/Sıfırla flip-flop'larına ihtiyaç duyar. Tek bit



ŞEKİL 6.17

Zamanlama sinyallerinin üretilmesi

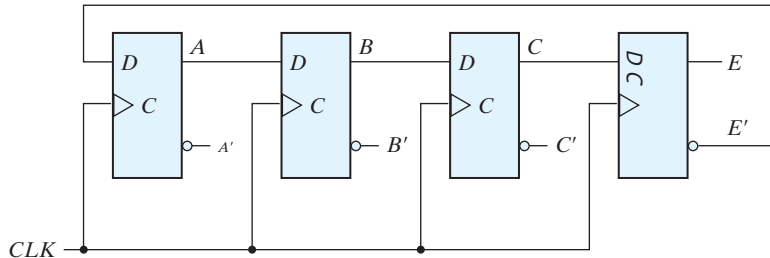
her saat darbesi ile sağa kaydırılır ve T_3 'den T_0 'a geri döner. Her flip-flop, her dört saat d öngüsünde bir kez 1 durumundadır ve Şekil 6.17 (b)'de gösterilen dört zamanlama sinyali linden birini üretir. Her çıkış, bir saat darbesinin negatif kenar geçişinden sonra 1 olur v e bir sonraki saat döngüsü boyunca 1 olarak kalır.

Alternatif bir tasarım için, zamanlama sinyalleri, dört farklı durumdan geçen iki bitlik bir sayıcı tarafından üretilebilir. Şekil 6.17 (c)'de gösterilen kod çözücü, sayıcının dört durumunu çözümler ve gerekli zamanlama sinyalleri dizisini üretir.

2 zamanlama sinyali üretmek için, ya 2 flip-flop içeren bir kaydırma kaydı ya da bir n -bit ikili sayıcı ile birlikte bir n -to- 2^n -hat çözücü. Örneğin, 16 zamanlama sinyali, bir halka sayıcı olarak bağlanmış 16-bit kaydırma kaydedici veya bir 4-bit ikili sayıcı ve bir 4-to-16-hat çözücü ile üretilebilir. İlk durumda, 16 flip-flop'a ihtiyacımız var. İkincisinde, dekoder için 4 flip-flop ve 16 dört girişli AND kapısına ihtiyacımız var. Ayrıca, zamanlama sinyallerini bir kaydırma kaydı ve bir dekoder kombinasyonu ile üretmek de mümkündür. Bu şekilde, flip-flop sayısı bir halka sayıcındaki sayıdan daha azdır ve de koder yalnızca iki girişli kapılar gerektirir. Bu kombinasyona *Johnson sayacı* denir.

Johnson Sayacı

Bir k bitlik halka sayacı, flip-floplar arasında tek bir biti döndürerek k ayırt edilebilir durum sağlar. Durum sayısı, kaydırma kaydı bir *switch-tail* halka sayacı olarak bağlandığında iki katına çıkarılabilir. Bir switch-tail halka sayacı, son flip-flopun tamamlanmış çıkışının ilk flip-flopun girişine bağlandığı dairesel bir kaydırma kaydıdır. Şekil 6.18 (a) böyle bir kaydırma kaydını göstermektedir. Dairesel bağlantı,



(a) Dört aşamalı switch-tail halka sayacı

Dizi numarası	Flip-flop çıkışları				Çıkış için gerekli A ND kapısı
	A	B	C	E	
1	0	1	1	0	$A'E'$
2	0	1	1	1	AB'
3	0	0	1	1	BC'
4	1	0	1	1	CE'
5	1	0	1	1	AE
6	1	0	0	1	$A'B$
7	1	0	0	0	$B'C$
8	1	0	0	0	$C'E$

(b) Sayım dizisi ve gerekli kod çözme

ŞEKİL 6.18

Bir Johnson sayacı inşası

en sağdaki flip-flop'un tamamlayıcı çıkışı, en soldaki flip-flop'un girişine bağlanır. Kayıt, her saat darbesiyle içeriğini bir kez sağa kaydırır ve aynı zamanda, E flip-flop'un tamamlayıcı değeri A flip-flop'a aktarılır.

Temizlenmiş bir durumdan başlayarak, anahtarlı kuyruk halkası sayıcı, Şekil 6.18 (b)'de listelenen sekiz durumdan geçer. Genel olarak, bir k bit anahtarlı kuyruk halkası sayıcı, 2^k durumda n geçecektir. Tüm 0'lardan başlayarak, her kaydırma işlemi soldan 1'leri ekler ve kayıt tamamen 1'lerle dolana kadar devam eder. Sonraki dizilerde, soldan 0'lar eklenir ve kayıt tekrar tamamen 0'larla dolana kadar devam eder.

Johnson sayıcı, 2^k zaman sinyalleri için çıkış sağlamak üzere 2^k kodlama kapısına sahip bir k bit anahtarlı kuyruk halkası sayıcıdır. Kodlama kapıları Şekil 6.18'de gösterilmemiştir, ancak tablonun son sütununda belirtilmiştir. Tabloda listelenen sekiz VE kapısı, devreye bağlandığında Johnson sayıcısının inşasını tamamlayacaktır. Her kapı belirli bir durum dizisi sırasında etkinleştirildiğinden, kapıların çıkışları peş peşe sekiz zaman sinyali üretir.

Bir k bit anahtarlı kuyruk halkası sayıcının 2^k zaman sinyali elde etmek için çözülmesi düzenli bir deseni takip eder. Tüm 0'lar durumu, iki uç flip-flop çıkışının tamamlayıcısını alarak çözülür. Tüm 1'ler durumu, iki uç flip-flop'un normal çıkışlarını alarak çözülür. Diğer tüm durumlar, dizideki bitişik 1, 0 veya 0, 1 deseninden çözülür.

Örneğin, 7. dizide flip-flop'lar B ve C arasında bitişik bir 0, 1 deseni vardır. Çözümüş çıkış, B 'nin tamamlayıcısını alarak ve C 'nin normal çıkışını alarak elde edilir, yani $B \oplus C$.

Şekil 6.18 (a)'deki devrenin bir dezavantajı, kullanılmayan bir durumda bulunursa, bir geçersiz durumdan diğerine geçmeye devam etmesi ve asla geçerli bir duruma ulaşamamasıdır. Bu zorluğu, devreyi bu istenmeyen durumu önleyecek şekilde değiştirmekle düzeltebiliriz. Bir düzeltme prosedürü, flip-flop B 'nin çıkışını flip-flop C 'nin D girişine giden bağlantısını kesmek ve bunun yerine flip-flop C 'nin girişini aşağıdaki fonksiyonla etkinleştirmektir.

$$D_C = (A + C)B$$

burada D_C , flip-flop C 'nin D girişi için flip-flop giriş denklemi.

Johnson sayaçları herhangi bir zamanlama dizisi sayısı için inşa edilebilir. Gerekli flip-flop sayısı, zamanlama sinyallerinin sayısının yarısıdır. Kod çözme kapılarının sayısı, zamanlama sinyalleri sayısına eşittir ve yalnızca iki girişli kapılar gereklidir.

6.6 KAYITLAR VE SAYAÇLAR İÇİN HDL

Kayıtlar ve sayaçlar, Verilog'da ya davranışsal ya da yapısal düzeyde tanımlanabilir. Davranışsal modelleme, kayıtların yalnızca işlev tablosu tarafından belirlenen işlemlerini, önceden düşünülmüş bir yapı olmadan tanımlar. Yapısal düzeyde bir tanım, devreyi kapılar, flip-flop'lar ve çoklayıcılar gibi bileşenlerin bir koleksiyonu açısından gösterir. Çeşitli bileşenler, çok seviyeli bir mantık diyagramının temsiline benzer bir tasarımın hiyerarşik tanımını oluşturmak için örneklenir. Bu bölümdeki örnekler, her iki tür tanımla da gösterecektir. Her ikisi de faydalıdır. Bir makine karmaşık olduğunda, hiyerarşik bir tanım, makinenin daha basit ve daha kolay tanımlanabilir birimlere fiziksel olarak bölünmesini sağlar.

Kaydırma Kaydı

Bölüm 6.2'de sunulan evrensel kaydırma kaydı, paralel yüklemeye sahip iki yönlü bir kaydırma kaydıdır. Kayıt ile gerçekleştirilen dört saatli işlem, Tablo 6.6'da belirt ilmiştir. Kayıt ayrıca asenkron olarak temizlenebilir. Şekil 6.7 (a)'da gösterilen dört bitlik evrensel kaydırma kaydının davranışsal tanımı için seçtiğimiz isim, isim *Shift_Register_4_beh*, üst düzey blok diyagram sembolünün iç detaylarının davranışsal m odelini belirtir ve bu modeli yapısal olandan ayırır. Davranışsal model HDL Örneği 6.1'd e sunulmuştur ve yapısal model HDL Örneği 6.2'de verilmiştir. Şekil 6.7 (a)'deki üst düze y blok diyagram sembolü, dört bitlik evrensel kaydırma kaydının iki seçim girişine (*s1,s0*), kaydırma kaydını kontrol etmek için iki seri girişe (*shift_left, shift_right*), iki seri veri yol u girişine (MSB_in ve LSB_in), dört bitlik bir paralel girişi (*I_par*) ve dört bitlik bir paralel çıkışı (*A_par*) sahip olduğunu göstermektedir. Vektörün elemanları

I_par[3:0] Şekil 6.7'deki bitlere $I_3, \dots, I_0$ karşılık gelir ve benzer şekilde *A_par[3:0]* için de geçerlidir.

Her zaman bloğu, kayıt ile gerçekleştirilebilecek beş işlemi tanımlar.

Clear girişi, aktif düşük bir sinyal ile kaydı asenkron olarak temizler. *Clear* , ka ydın saat sinyalinin pozitif kenarına yanıt vermesi için yüksek olmalıdır. Kaydın dört saatli işlemi, **case** ifadesindeki iki seçim girişinin değerlerine göre belirleni r. (*s1* ve *s0* iki bitlik bir vektör haline getirilir ve **case** ifadesinin ifade argüma nı olarak kullanılır.) Kaydırma işlemi, seri girişin ve kaydın üç bitinin birleştirilm esiyle belirtilir. Örneğin, ifade

$$A_par \leftarrow \{MSB_in, A_par[3:1]\}$$

sağ kaydırma işlemi için seri veri girişinin birleştirilmesini belirtir (*MSB_in*) bits *A_par[3:1]* çıkış veri yolu . Bir vektör içindeki bitlerin ardışık bir aralığına *parça a seçimi* denir. Birleştirmenin dört bitlik sonucu, saat darbesi işlemi tetiklendiği nde *A_par[3:0]* kaydına aktarılır. Bu akış, sağ kaydırma işlemi gerçekleştirir ve kaydı yeni bilgilerle günceller.

Kaydırma işlemi, *A_par[0]* içeriğini *A_par[1]* içeriği ile üzerine yazar.

Dikkat edin ki, devrenin işlevselliği, herhangi bir özel donanımdan bağımsız olarak tanı mlanmıştır. Bir sentez aracı, Şekil 6.7 (b) yapısında kaydırma kaydını uygulamak için ASI C hücrelerinin bir ağ listesini oluşturacaktır.

HDL Örneği 6.1 (Evrensel Kaydırma Kaydı - Davranışsal Model)

// 4 bitlik evrensel kaydırma kaydının davranışsal tanımı

// Şekil 6.7 ve Tablo 6.3

```

modül Shift_Register_4_beh (           // V2001, 2005           // V2001, 2005
    çıkış reg [3:0] A_par, // Çıkış kaydı
    input [3:0] I_par, // Paralel girdi
    input s1, s0, // Girişleri Seç
    MSB_in, LSB_in, // Seri girişler
    CLK, Clear_b // Saat ve Temizle
);

```

```

her zaman @ (posedge CLK, negedge Clear_b) // V2001, 2005
eğer (Clear_b == 0) A_par <= 4'b0000;

aksi takdirde
case ({s1, s0})
2'b00: A_par <= A_par; // Değişiklik yok
2'b01: A_par <= {MSB_in, A_par[3: 1]}; // Sağa kaydır
2'b10: A_par <= {A_par[2: 0], LSB_in}; // Sola kaydır
2'b11: A_par <= I_par; // Girişin paralel yüklemesi
endcase
endmodule

```

reg türündeki değişkenler, yeni bir değer atanana kadar değerlerini korur. Aşağıdaki alternatif case ifadesini düşünün:

```

case ({s1, s0})
// 2'b00: A_par <= A_par; // Değişiklik yok
2'b01: A_par <= {MSB_in, A_par [3: 1]}; // Sağa kaydır
2'b10: A_par <= {A_par [2: 0], LSB_in}; // Sola kaydır
2'b11: A_par <= I_par; // Girişin paralel yüklemesi
endcase

```

Durum ögesi 2 b00 olmadan, **case** ifadesi

{s1, s0} ve durum ögeleri arasında eşleşme bulamaz, bu nedenle kayıt A_par değişmeden kalır.

Evrensel kaydırma kaydının yapısal modeli,

Şekil 6.7 (b)'deki mantık diyagramına atıfta bulunarak tanımlanabilir. Diyagram, kaydın dört çoklayıcı ve dört D flip-flop içerdiğini göstermektedir. Bir mux ve flip-flop birlikte kaydırma kaydının bir aşaması olarak modellenir. Aşama, bir mux için bir modül ve bir D flip-flop için başka bir modülün örneklenmesi ve bağlantısı ile yapısal bir modeldir. Basitlik açısından, yapının en alt düzey modülleri çoklayıcı ve flip-flop'un davranışsal modelleridir. Aşamaların doğru bir şekilde bağlanmasına dikkat edilmelidir. Kaydın yapısal tanımı HDL Örneği 6.2'de gösterilmektedir. Üst düzey modül, girişleri ve çıkışları tanımlar ve ardından kaydın bir aşamasının dört kopyasını örnekler. Dört örnekleme, dört aşama arasındaki bağlantıları belirtir ve mantık diyagramında belirtilen kaydın ayarlı yapısını sağlar. Flip-flop'un davranışsal tanımı, tek bir kenar hassas döngüsel davranış (bir **always** bloğu) kullanır. Atama ifadeleri, nonblocking atama operatörünü (<=) kullanır, mux'un modeli tek bir düzey hassas davranış kullanır ve atamalar blocking atama operatörünü (=) kullanır.

HDL Örneği 6.2 (Evrensel Kaydırma Kaydedici-Yapısal Model)

// 4 bitlik evrensel kaydırma kaydedicisinin yapısal tanımı (bkz. Şekil 6.7)

```

modül Shift_Register_4_str ( // V2001, 2005
    çıkış [3: 0] A_par, // Paralel çıkış
    giriş [3: 0] I_par, // Paralel giriş

```

```

giriş      s1, s0,                                // Mod seçimi
giriş      MSB_in, LSB_in, CLK, Clear_b          // Seri girişler, saat, temizle
);

// mod kontrolü için veri yolu
assign    [1:0] select = {s1, s0};

// Dört aşamanın örneğini oluştur
stage ST0 (A_par[0], A_par[1], LSB_in, I_par[0], A_par[0], select, CLK, Clear_b);
stage ST1 (A_par[1], A_par[2], A_par[0], I_par[1], A_par[1], select, CLK, Clear_b);
stage ST2 (A_par[2], A_par[3], A_par[1], I_par[2], A_par[2], select, CLK, Clear_b);
stage ST3 (A_par[3], MSB_in, A_par[2], I_par[3], A_par[3], select, CLK, Clear_b);
endmodule

// Kaydırma kaydının bir aşaması
module stage (i0, i1, i2, i3, Q, select, CLK, Clr_b);
  input      i0,                                // döngü bit seçimi
            i1,                                // sağa kaydırma için sol komşudan veya seri girişten veri
            i2,                                // sağ komşudan veya seri girişten veriler için sola kaydırma
            i3;                                // paralel girişten veriler

  çıkış      Q;

  giriş [1:0] sec; // aşama modu kontrol veri yolu          // aşama modu kontrol veri yolu
  giriş      CLK, Clr_b; // Saat, flip-flop'lar için temizleme
  tel        mux_out;

// mux ve flip-flop'u oluştur
Mux_4_x_1 M0      (mux_out, i0, i1, i2, i3, select);
D_fl ip_fl op M1  (Q, mux_out, CLK, Clr_b);
endmodule

// 4x1 çoklayıcı          // davranışsal model
module Mux_4_x_1 (mux_out, i0, i1, i2, i3, select);
  output      mux_out;
  giriş      i0, i1, i2, i3;
  input [1:0] select;
  reg        mux_out;
  her zaman  @ (sec, i0, i1, i2, i3)
  durum (sec)
    2'b00:    mux_out = i0;
    2'b01:    mux_out = i1;
    2'b10:    mux_out = i2;
    2'b11:    mux_out = i3;
  enddurum
modülsonu

```



```
// D flip-flop'un davranışsal modeli
modül D_flip_flop (Q, D, CLK, Clr_b);
    çıkış          a;
    giriş          D, CLK, Clr;
    reg            a;

    her zaman @ ( posedge CLK, negedge Clr_b)
        eğer (!Clr_b) Q <= 1'b0; aksi takdirde Q <= D;
endmodule
```

Yukarıdaki örnekler, dijital bir devreyi modellemenin farklı stillerini göstermek için evrensel bir kaydırma kaydının iki tanımını sunmuştur. Bir simülasyon, modellerin aynı işlevselliğe sahip olduğunu doğrulamalıdır. Pratikte, bir tasarımcı yalnızca davranışsal modeli geliştirir, bu model daha sonra sentezlenir. Sentezlenen devrenin işlevi, derlendiği davranışsal tanım ile karşılaştırılabilir. Tasarımcının yapısal bir model geliştirmesi gereğini ortadan kaldırmak, tasarım sürecinin verimliliğinde büyük bir iyileşme sağlar.

Eşzamanlı Sayıcı

HDL Örneği 6.3, Şekil 6.14'ten paralel yükleme ile eşzamanlı sayıcının davranışsal modelini sunar: Binary_Counter_4_Par_Load. Count, Load, CLK ve Clear_b, sayıcının Tablo 6.6'da belirtilen işlevine göre çalışmasını belirleyen girişlerdir. Sayıcının dört veri girişi, dört veri çıkışı ve bir taşıma çıkışı vardır.

İç veri hatları (I3, I2, I1, I0), davranışsal modelde Data_in[3:0] olarak gruplandırılmıştır. Benzer şekilde, sayım bitlerini tutan kayıt (A3, A2, A1, A0) A_count[3:0] olarak adlandırılır. Bir devrenin HDL modelinde tanımlayıcıların, modelin belgelerindeki tanımlayıcılarla tam olarak eşleşmesi iyi bir uygulamadır. Ancak, devre düzeyindeki tanımlayıcılar bir el kitabında bulunanlar ise, genellikle kısa ve karmaşık oldukları için bu her zaman mümkün değildir ve HDL ile mevcut olan metni kullanmazlar. Şekil 6.14 (a) içindeki üst düzey blok diyagram sembolü, bir devre diyagramında kullanılan isimler ile HDL modelinde kullanılabilecek ifade isimleri arasında bir arayüz görevi görür. Taşıma çıkışı C_out, bir kombinasyonel devre tarafından üretilir ve bir assign ifadesi ile belirtilir. Sayım 15'e ulaştığında ve sayıcı sayım durumunda olduğunda C_out = 1 olur.

Bu nedenle, C_out = 1 eğer Count = 1, Load = 0, ve A = 1111; aksi takdirde C_out = 0. Her zaman blok, register'da gerçekleştirilecek işlemi,

Clear_b, LoadCount değerlerine bağlı olarak belirtir. Clear_b sinyalinin 0 (aktif-düşük sinyal) olması, A değerini 0'a sıfırlar.

Aksi takdirde, eğer Clear_b = 1, saat sinyalinin pozitif kenarıyla üç operasyondan biri tetiklenir. if, else if, ve else ifadeleri, Clear, Load, ve Count kontrol sinyalleri arasında bir öncelik belirler; bu, Tablo 6.6 'da belirtilmiştir. Clear_bLoad ve Count sinyallerini geçersiz kılar; Load Count sinyalinin geçersiz kılar. Bir sentez aracı, davranışsal modelden Şekil 6.14 (b) devresini üretecektir.

HDL Örneği 6.3 (Senkran Sayıcı)

```
// Dört bitli ikili sayaç paralel yükleme ile (V2001, 2005)
// Şekil 6.14 ve Tablo 6.6'ya bakınız
modül İkili_Sayacı_4_Par_Yükle (
    çıkış reg [3: 0] A_sayım, // Veri çıkışı
    çıkış C_çıkış,           // Taşıma çıkışı
    giriş [3: 0] Veri_girişi, // Veri girişi
    giriş Sayı,              // Aktif yüksek saymak için
    Yükle,                  // Aktif yüksek yüklemek için
    CLK,                    // Pozitif kenar hassas
    Clear_b                 // Aktif düşük
);
atan C_out = Sayı && (~Yükle) && (A_sayı == 4'b1111);
always @ ( pozitif kenar CLK, negatif kenar Clear_b)
    eğer (~Clear_b) A_sayı <= 4'b0000;
    else if (Load)           A_count <= Data_in;
    else if (Count)          A_count <= A_count + 1'b1;
    aksi takdirde            A_count <= A_count; // redundant statement
endmodule
```

Dalgalı Sayıcı

Dalgalı sayıcının yapısal tanımı HDL Örneği 6.4'te gösterilmiştir. İlk modül, ikinci modülde tanımlanan dört içsel tamamlayıcı flip-flop'u örnekler. *Comp_D_flip_flop* (*Q*, *CLK*, *Reset*) . İlk flip-flop'un saat (giriş *CLK*) sinyali, dış kontrol sinyali *Count* ile bağlantılıdır. (*Count* örnek *F0* port listesinde *CLK* ile değiştirilir.) İkinci flip-flop'un saat girişi, birincinin çıkışına bağlıdır. (*A0* örnek *F1* içinde *CLK* ile değiştirilir.) Benzer şekilde, diğer flip-flop'ların her birinin saati, bir önceki flip-flop'un çıkışına bağlıdır. Bu şekilde, flip-flop'lar bir dalgalı sayıcı oluşturmak için birbirine zincirlenmiştir; bu, Şekil 6.8 (b)'de gösterilmiştir.

İkinci modül, gecikmeli bir tamamlayıcı flip-flop'u tanımlar. Bir tamamlayıcı flip-flop'un devresi, tamamlayıcı çıkışı *D* girişine bağlanarak oluşturulur. Sayacı başlatabilmek için flip-flop'a bir sıfırlama girişi eklenmiştir; aksi takdirde simülatör, flip-flop'un çıkışına bilinmeyen bir değer (*x*) atayacak ve işe yaramaz sonuçlar üretecektir. Flip-flop'a, saatin uygulandığı andan flip-flop'un çıkışını tamamladığı ana kadar iki zaman birimi gecikme atanmıştır. Gecikme, $Q \leq \#2 Q$ ifadesiyle belirtilmiştir. Gecikme operatörünün, engellemeyen atama operatörünün sağında yer aldığını unutmayın. Bu tür bir gecikme, *intra-atama gecikmesi* olarak adlandırılır ve *Q*'nun tamamlanmış değerinin *Q*'ya atanmasını erteleyerek etkilere bulunur. Gecikmeyi modellemenin etkisi, simülasyon sonuçlarında belirgin olacaktır. Bu modelleme tarzı simülasyonda faydalı olabilir, ancak model sentezlenirken kaçınılmalıdır. Sentez sonuçları, araca erişilen ASIC hücre kütüphanesine bağlıdır, sentezlenecek model içinde ortaya çıkabilecek herhangi bir yayılma gecikmesine değil.

HDL Örneği 6.4 (Ripple Sayıcı)

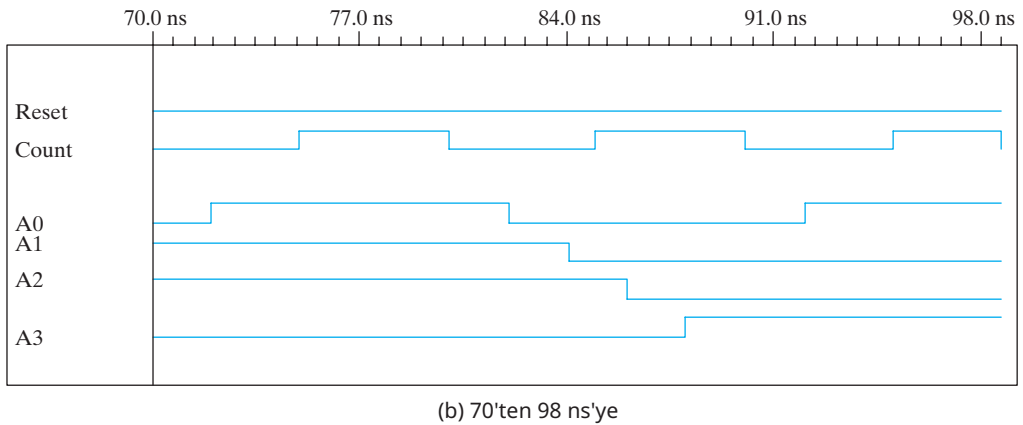
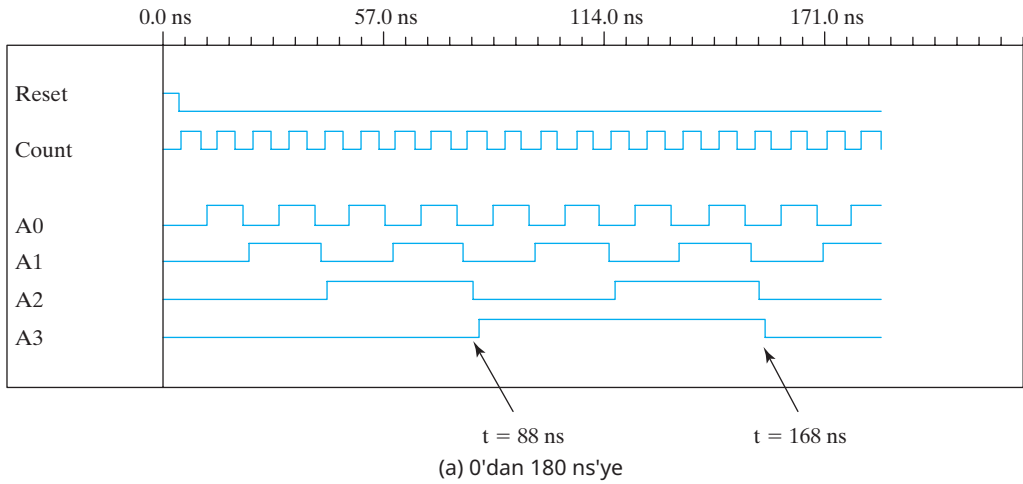
```
// Ripple sayıcı (Bkz. Şekil 6.8 (b))
time ölçeği 1ns / 100 ps
modül Ripple_Counter_4bit (A3, A2, A1, A0, Count, Reset);
    çıkış A3, A2, A1, A0;
    giriş Count, Reset;
// Tamamlayıcı flip-flop'u örnek
    Comp_D_flip_flop F0 (A0, Count, Reset);
    Comp_D_flip_flop F1 (A1, A0, Reset);
    Comp_D_flip_flop F2 (A2, A1, Reset);
    Comp_D_flip_flop F3 (A3, A2, Reset);
endmodule
// Gecikmeli flip-flop'u tamamlama
// D flip-flop'a giriş = Q'
module Comp_D_flip_flop (Q, CLK, Reset);
    output Q;
    input CLK, Reset;
    reg Q;
    always @ (negedge CLK, posedge Reset)
    if (Reset) Q <= 1'b0;

    else Q <= #2 ~Q; // atama içi gecikme // atama içi gecikme
endmodule
// Ripple sayıcını test etmek için uyarı
module t_Ripple_Counter_4bit;
    reg Count;
    reg Reset;
    tel A0, A1, A2, A3;
// Ripple sayıcını başlat
    Ripple_Counter_4bit M0 (A3, A2, A1, A0, Count, Reset);
    always
    #5 Count = ~Count;
    initial
    begin
    Count = 1'b0;
    Reset = 1'b1;
    #4 Reset = 1'b0;
    end

    initial #170 $finish;

endmodule
```

HDL Örneği 6.4'teki test tezgahı modülü, ripple sayıcının işlevselliğini simüle etmek ve doğrulamak için bir uyarı sağlar. **always** ifadesi, 10 zaman birimi döngüsüne sahip sürekli çalışan bir saat üretir. Flip-flop'lar, saatin negatif kenarında tetiklenir, bu da $t = 10, 20, 30$ ve sonrasında her 10 zaman biriminde gerçekleşir. Bu simülasyondan elde edilen dalga formları Şekil 6.19'da gösterilmektedir. Kontrol sinyali *Count* her 10 ns'de negatif olur. *A0* her *Count*'nin negatif kenarında tamamlanır, ancak 2 ns gecikmelidir. Her flip-flop, önceki flip-flop 1'den 0'a geçtiğinde tamamlanır. $t = 80$ ns'den sonra, tüm dört flip-flop tamamlanır çünkü sayaç 0111'den 1000'e geçer. Her çıkış 2 ns gecikmelidir ve bu nedenle, *A3*, $t = 88$ ns'de 0'dan 1'e ve 168 ns'de 1'den 0'a geçer. Gecikme sürelerinin sayacın son bitine nasıl biriktiğine dikkat edin, bu da çok yavaş bir sayı aç hareketine neden olur. Bu, sayacın pratik kullanımını sınırlar.



ŞEKİL 6.19
HDL Örneği 6.4'ün simülasyon çıktısı

PROBLEMLER

(* ile işaretlenmiş problemlerin cevapları kitabın sonunda yer almaktadır. Uygun olduğunda, bir mantık tasarımı ve ilgili HDL modelleme problemi çapraz referans verilmiştir.)

Not: Bir Verilog tanımının yazılması ve doğrulanması gereken her problem için, simülasyon sırasında hangi işlevsel özelliklerin test edileceğini ve bunların nasıl test edileceğini belirlemek için bir test planı yazılmalıdır. Örneğin, simüle edilen makine reset durumunda olmayan bir durumda iken reset sinyalinin etkinleştirilmesiyle anlık reset testi yapılabilir. Test planı, planı uygulayacak bir test tezgahının geliştirilmesine rehberlik etmelidir. Modeli test tezgahı kullanarak simüle edin ve davranışını doğru olduğunu doğrulayın. Sentez araçları ve bir ASIC hücre kütüphanesi veya bir alan programlanabilir kapı dizisi (FPGA) araç seti mevcutsa, Problemler 6.34–6.51 için geliştirilen Verilog tanımları sentez egzersizleri olarak atanabilir. Sentez araçları tarafından üretilen kapı düzeyindeki devre, simülasyon sonuçlarıyla ön sentez modelinin simülasyon sonuçlarıyla karşılaştırılmalıdır.

Bazı HDL problemlerinde, kullanılmayan durumlar ile ilgili bir sorunla başa çıkma ihtiyacı olabilir (bkz. Bölüm 4'teki HDL Örneği 4.8'den önceki varsayılan durum maddesinin tartışması).

2 giriшли NAND kapısını Şekil 6.1'deki kayıta ekleyin ve kapı çıkışını tüm flip-flopların C girişlerine bağlayın. NAND kapısının bir girişı saat jeneratöründen saat darbeleriyle

NAND kapısının bir girişı saat jeneratöründen saat darbelerini alırken, diğer girişı paralel yük kontrolü sağlar.

Değiştirilmiş kaydın çalışmasını açıklayın. Bu devrenin neden operasyonel sorunlar yaşayabileceğini açıklayın.

Şekil 6.2'deki kayda senkron temizleme girişı ekleyin. Değiştirilmiş kayıt, paralel yük yeteneğine ve senkron temizleme yeteneğine sahip olacaktır. Kayıt, saat pozitif geçişten geçtiğinde ve temizleme girişı 1'e eşit olduğunda senkron olarak temizlenir. (HDL—bkz. Problem 6.35(a), (b).)

Seri ve paralel transfer arasındaki fark nedir? Seri verileri paralel verilere ve paralel verileri seri verilere nasıl dönüştüreceğinizi açıklayın. Hangi tür kayıt gereklidir?

6.4 * Dört bitlik bir kaydın içeriği başlangıçta 0110'dır. Kayıt, seri girişı 1011100 olan altı kez sağa kaydırılır. Her kaydirmeden sonra kaydın içeriği nedir?

Şekil 6.7'de gösterilen dört bitlik evrensel kaydırma kaydedici, bir IC bileşen paketi içinde yer almaktadır. (HDL—bkz. Problem 6.52.)

(a) Tüm girişı ve çıkışları gösteren IC'nin blok diyagramını çizin. Güç kaynağı için iki pin ekleyin.

(b) Bu IC'lerden ikisini kullanarak sekiz bitlik evrensel kaydırma kaydedici oluşturacak bir blok diyagramı çizin.

D flip-flop'ları kullanarak paralel yüklemeli dört bitlik bir kaydırma kaydedici tasarlayın. İki kontrol girişı vardır: *shift* ve *load*. *shift* = 1 olduğunda, kaydedicinin içeriği bir pozisyona kaydırılır. Yeni veriler, *load* = 1 ve *shift* = 0 olduğunda kaydediciye aktarılır. Her iki kontrol girişı de 0 olduğunda, kaydedicinin içeriği değişmez. (HDL—bkz. Problem 6.35(c), (d).)

D flip-flop'ları ve mod seçim girişleri *s* 1 ve *s* 0 olan dört 4×1 çoklayıcı ile dört bitlik bir kaydedicinin mantık diyagramını çizin. Kaydedici, aşağıdaki fonksiyon tablosuna göre çalışır. (HDL—bkz. Problem 6.35(e), (f).)

s_1	s_0	Kayıt İşlemi
0	0	Değişiklik yok
1	0	Dört çıkışı tamamlayın
0	1	Saatiyle senkronize olarak kaydı 0'a temizle
1	1	Paralel verileri yükle

- 6.8** * Şekil 6.6'daki seri toplayıcı, iki dört bitlik kaydedici kullanır. Kayıt A ikili sayıyı 0 101 tutar ve kayıt B 0111'i tutar. Taşıma flip-flop'u başlangıçta 0'a sıfırlanmıştır. Her kaydırımdan sonra kayıt A ve taşıma flip-flop'undaki ikili değerleri listeleyin. (HDL—bkz. Problem 6.54).
- Seri bir toplayıcıyı ($A+B$) uygulamanın iki yolu Bölüm 6.2'de gösterilmiştir. Devrelerin seri çıkarıcılarına ($A-B$) dönüştürülmesi için değiştirilmesi gerekmektedir.
- (a) Şekil 6.5'teki devreyi kullanarak, $A+B$ 'nin 2'nin tamamlayıcısını gerçekleştirmek için gereken değişiklikleri gösterin. (HDL—Problem 6.35(h)'e bakın.)
- (b) * Şekil 6.6'daki devreyi kullanarak, Tablo 6.2'yi bir toplayıcıdan bir çıkarıcı devresine dönüştürmek için gereken değişiklikleri gösterin. (Problem 4.12'ye bakın). (HDL—Problem 6.35(i)'e bakın.)
- 6.10** Bir kaydırma kaydedici ve bir flip-flop ile seri bir 2'nin tamamlayıcısını tasarlayın. İkili sayı bir taraftan kaydırılır ve 2'nin tamamlayıcısı kaydırma kaydedicinin diğer tarafına kaydırılır. (HDL—Problem 6.35(j)'e bakın.)
- 6.11** Bir ikili dalgalı sayaç, saatin pozitif kenarında tetiklenen flip-flop'lar kullanır. Eğer
- (a) flip-flop'ların normal çıkışları saate bağlıdır ve
- (b) flip-flop'ların tamamlayıcı çıkışları saate mi bağlı?
- 6.12** Bir dört bitlik ikili dalgalı geri sayım sayacının mantık diyagramını çizin
- (a) saate pozitif kenarında tetiklenen flip-flop'lar ve
- (b) Saatin negatif kenarında tetiklenen flip-flop'lar.
- 6.13** Bir BCD dalgalı sayacın, 1010 sayımının gerçekleştirmesi tespit eden asenkron temizleme ile dört bitlik bir ikili dalgalı sayacı ve bir NAND kapısı kullanarak inşa edilebileceğini gösterin. (HDL—bkz. Problem 6.35(k).)
- 6.14** Sonraki sayıya ulaşmak için aşağıdaki sayımlardan sonra 10 bitlik bir ikili dalgalı sayacın kaç flip-flop'unun tamamlanması gerekecek?
- (a) *1001100111
- (b) 1111000111
- (c) 0000001111
- 6.15** * Bir flip-flop, saat kenarının olduğu andan çıkışın tamamlandığı ana kadar 3 ns gecikmeye sahiptir. Bu flip-flopları kullanan 10 bitlik ikili ripple sayacında maksimum gecikme nedir? Sayacın güvenilir bir şekilde çalışabileceği maksimum frekans nedir?
- 6.16** * Şekil 6.10'da gösterilen BCD ripple sayacı dört flip-flop ve 16 duruma sahiptir, bunlardan yalnızca 10'u kullanılmaktadır. Devreyi analiz edin ve diğer altı kullanılmayan durum için bir sonraki durumu belirleyin. Eğer bir gürültü sinyali devreyi kullanılmayan durumlardan birine gönderirse ne olur? (HDL—Problem 6.54'e bakın.)

6.17

* D flip-flop'lar ile dört bitlik ikili senkron sayıcı tasarlayın.

Şekil 6.13'teki yukarı-aşağı sayıcıda, hem yukarı hem de aşağı girişleri etkinleştirildiğin de hangi işlem gerçekleştirilir? Her iki giriş 1 olduğunda sayıcının durumunun değişim emesi için devreyi değiştirin. (HDL—bkz. Problem 6.35(I).)

T flip-flop'lar kullanan bir BCD sayıcı için flip-flop giriş denklemleri Bölüm 6.4'te verilmiştir.

(a) JK flip-flop'lar ve (b)* D flip-flop'lar kullanan bir BCD sayıcı için giriş denklemlerini eld e edin. Üç tasarımı karşılaştırarak hangisinin en verimli olduğunu belirleyin.

Şekil 6.14'teki paralel yüklemeli ikili sayıcıyı, tüm girişleri ve çıkışları gösteren bir blok diyagraf amında kapatın.

(a) Bu dört bloğun bağlantılarını göstererek paralel yüklemeli 16 bitlik bir sayıcı üreti n.

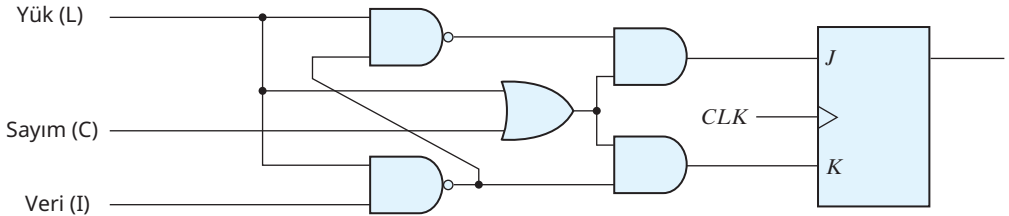
(b) 0'dan ikili 127'ye kadar sayan bir ikili sayıcı oluşturun.

6.21

* Şekil 6.14'teki sayıcının iki kontrol girişi vardır—Yükle (L) ve Say (C)—ve bir ve girişi, (I_i).

(a) İlk aşamadaki J ve K flip-flop giriş denklemlerini L, C, ve I_i cinsinden türetin.

(b) Bir eşdeğer devrenin birinci aşamasının mantık diyagramı Şekil P6.21'de gösterilmiştir. Bu devrenin (a) ile eşdeğer olduğunu doğrulayın.



ŞEKİL P6.21

6.22

Şekil 6.14 için, mod-10 sayıcı için üç alternatif verin (yani, sayım 12 farklı durumd an oluşan bir dizi boyunca gelişir).

(a) AND kapısı ve yük girişini kullanarak.

(b) Çıkış taşımasını kullanarak.

(c) NAND kapısı ve asenkron temizleme girişini kullanarak.

6.23

Tam olarak on iki saat döngüsü boyunca açık kalan bir çıkış sinyali sağlayan bir zamanlama devresi tasarla yın. Bir başlangıç sinyali çıkışı 1 durumuna gönderir ve on iki saat döngüsünden sonra sinyal 0 duruma na geri döner. (HDL—bkz. Problem 6.45.)

6.24

* Aşağıdaki ikili tekrar eden diziden geçen bir T flip-flop ile bir sayaç tasarlayı n: 0, 1, 3, 7, 6, 4. İkili durumlar 010 ve 101'in dikkate alınmadığı koşullar olarak kabul edildiğ inde, sayacın düzgün çalışmayabileceğini gösterin. Tasarımı düzeltmenin bir yolunu bulun. (HDL—bkz. Problem 6.55.)

6.25

Altı tekrar eden zamanlama sinyali T_0 ile T_5 üretmek gereklidir, bunlar Şekil 6.17 (c)'de gösterilenlere benzer. Devreyi tasarlayın (HDL—bkz. Problem 6.46.):

(a) yalnızca flip-flop'lar.

(b) bir sayaç ve bir kod çözücü.

6.26

* Dijital bir sistem, 80 MHz frekansında darbeler üreten bir saat üreticisine sahiptir. 50 ns döngü süresine sahip bir saat sağlayan bir devre tasarlayın.

JK flip-flop'lar kullanarak,

- Aşağıdaki tekrarlanan ikili diziyi içeren bir sayaç tasarlayın: 0, 1, 2, 3, 4, 5, 6. (HDL—Problem 6.50(a), 6.51'e bakın.)
- Sayacın mantık diyagramını çizin.

D flip-flop'ları kullanarak,

- *Aşağıdaki tekrar eden ikili diziyi içeren bir sayaç tasarlayın: 0, 1, 2, 4, 6. (HDL—bkz. Problem 6.50(b).)
- Sayacın mantık diyagramını çizin.
- Aşağıdaki tekrar eden ikili diziyi içeren bir sayaç tasarlayın: 0, 2, 4, 6, 8.
- Sayacın mantık diyagramını çizin.

Şekil 6.18 (a) 'deki anahtarlı kuyruk halkası sayacındaki sekiz kullanılmayan durumu listeleyin. Bu durumların her biri için son durumu belirleyin ve eğer sayaç geçersiz bir durumda bulunursa, geçerli bir duruma geri dönmediğini gösterin. Metinde önerildiği gibi devreyi değiştirin ve sayacın aynı durum dizisini ürettiğini ve devrenin kullanılmayan durumlardan herhangi birinden geçerli bir duruma ulaştığını gösterin.

Bir Johnson sayıcı ile n flip-flop'un 2^n durumdan oluşan bir dizi ürettiğini gösterin. Beş flip-flop ile üretilen 10 durumu ve her bir 10 AND kapısının

çıkışlarının Boolean terimlerini listeleyin.

Dört bitlik kayıt için HDL davranışsal ve yapısal tanımlarını yazın ve doğrulayın.

Şekil 6.1.

- Paralel yükleme ve asenkron temizleme ile dört bitlik bir kaydın HDL davranışsal tanımını yazın ve doğrulayın.
- Şekil 6.2'de gösterilen paralel yükleme ile dört bitlik kaydın HDL yapısal tanımını yazın ve doğrulayın. Flip-flop girişleri için 2×1 çoklayıcı kullanın. Bir asenkron temizleme girişi ekleyin.
- Her iki tanımlı da bir test tezgahı kullanarak doğrulayın.

Aşağıdaki uyarı programı, HDL Örneği 6.3'te tanımlanan paralel yüklemeli ikili sayacı simüle etmek için kullanılır. Sayacın çıktısını ve $t = 0$ 'dan $t = 155$ ns'ye kadar taşıma çıktısını göstere n dalga formlarını çizin.

// Örnek 6.3'ün ikili sayacını test etmek için uyarı

modül testcounter;

reg Count, Load, CLK, Clr;

reg [3: 0] IN;

wire CO;

wire [3: 0] A;

counter cnt (Count, Load, IN, CLK, Clr, A, CO);

always

#5 CLK = ~CLK;

initial

begin

Clr = 0;

CLK = 1;

Load = 0; Count = 1;


```

#5 Clr = 1;
#40 Yük = 1; Giriş = 4'b1001;
#10 Yük = 0;
#70 Sayım = 0;
#20 $finish ;
SON
endmodule

```

6.34 * Dört bitlik kaydırma kayıtcısının HDL davranış tanımını yazın ve doğrulayın (bkz. Şekil 6.3).

Yazın ve doğrulayın

- (a) Problem 6.2'de tanımlanan kayıt için yapısal HDL modeli
- (b) *Problem 6.2'de tanımlanan kayıt için davranışsal HDL modeli
- (c) Problem 6.6'da tanımlanan kayıt için yapısal HDL modeli
- (d) Problem 6.6'da tanımlanan kayıt için davranışsal HDL modeli
- (e) Problem 6.7'de tanımlanan kayıt için yapısal HDL modeli
- (f) Problem 6.7'de tanımlanan kayıt için bir davranışsal HDL modeli
- (g) Şekil 6.8 (b)'de tanımlanan ikili sayıcı için bir davranışsal HDL modeli
- (h) Problem 6.9(a)'da tanımlanan seri çıkarıcı için bir davranışsal HDL modeli
- (i) Problem 6.9(b)'da tanımlanan seri çıkarıcı için bir davranışsal HDL modeli
- (j) Problem 6.10'da tanımlanan seri 2'nin tamamlayıcısı için bir davranışsal HDL modeli
- (k) Problem 6.13'te tanımlanan BCD ripple sayacının davranışsal HDL modeli.
- (l) Problem 6.18'de tanımlanan yukarı-aşağı sayacının davranışsal HDL modeli.

Şekil 6.13, Tablo 6.5 ve Tablo 6.6 ile tanımlanan dört bitlik yukarı-aşağı sayacının HDL davranışsal ve yapısal tanımlarını yazın ve doğrulayın.

Problem 6.24'te tanımlanan sayacın davranışsal tanımını yazın ve doğrulayın.

- (a) *Bir **if ... else** ifadesi kullanarak
- (b) Bir **case** ifadesi kullanarak
- (c) Sonlu durum makinesi.

Dört bitlik bir yukarı-aşağı sayacın HDL davranış tanımını yazın ve doğrulayın

paralel yükleme için aşağıdaki kontrol girişlerini kullanarak:

- (a) * Sayaç, üç işlem için üç kontrol girişine sahiptir: *Yukarı*, *Aşağı*, ve *Yükle*. Öncelik sırası: *Yükle*, *Yukarı*, ve *Aşağı*.
- (b) Sayaç, dört işlemi belirtmek için iki seçim girişine sahiptir: *Yükle*, *Yukarı*, *Aşağı*, ve değişiklik yok.

Şekil 6.16'daki sayacın HDL davranışsal ve yapısal tanımlarını yazın ve doğrulayın.

Şekil 6.17 (a)'da gösterilen benzer sekiz bitlik bir halka sayacın HDL tanımını yazın ve doğrulayın.

Bir dört bitlik switch-tail ring (Johnson) sayacının HDL tanımını yazın ve doğrulayın (Şekil 6.18 a).

6.42 * *Binary_Counter_4_Par_Load* içindeki if ifadesinin son maddesiyle ilgili yorum, HDL Örneği 6.3'te ifadenin gereksiz olduğunu belirtmektedir. Bu ifadenin, tanımını uyguladığı davranışı değiştirmeden neden kaldırılabilirliğini açıklayın.

6.43

Şekil 6.4'te gösterilen şema, verilerin kaydırma register A'dan kaydırma register B'ye seri aktarımını kontrol etmek için saati ka

patmaktadır. Her bir hücrenin girişinde çoklayıcılar kullanarak, saat yolunu değiştirmeyen alternatif bir devre yapısal modelini geliştirin.

Tasarım hiyerarşisinin en üst seviyesi, kaydırma registerlarını örneklemektir. Kaydırma registerlarını tanımlayan modül, flip-flop ve mux örneklemelerine sahip olmalıdır. Mux ve flip-flop modüllerini davranışsal modellerle tanımlayın. Makinenin sıfırlanması gereğini dikkate aldığınızdan emin olun.

Devreyi simüle etmek ve verilerin aktarımını göstermek için bir test tezgahı geliştirin.

Şekil 6.5'te gösterilen seri toplayıcının tasarımını, D flip-flop'a kapatılmış saati kaldırarak ve saat sinyali doğrudan sağlayarak değiştirin. D flip-flop'u, kaydırma durdurulduğunda flip-flop'un içeriğini geri döndürmek için bir mux ile artırın ve kaydırma aktif olduğunda tam toplayıcının taşıma çıkışını sağlayın. Kaydırma registerları da bu özelliği içermelidir, kapatılmış saat kullanmak yerine. Tasarımın en üst seviyesi, kaydırma registerları, tam toplayıcı, D flip-flop ve mux için davranışsal modeller kullanarak modülleri örneklemelidir. Asenkron sıfırlama varsayalım. Devreyi simüle etmek ve verilerin aktarımını göstermek için bir test tezgahı geliştirin.

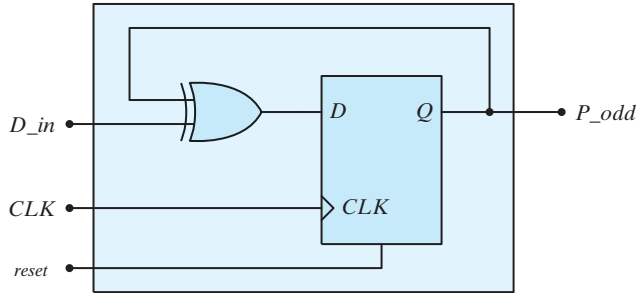
- 6.45** * Bir sonlu durum makinesinin davranışsal tanımını yazın ve doğrulayın, bu makine Problem 6.24'te tanımlanan sayacı uygulamak için.

Problem 6.25, zamanlama sinyalleri üretmek için bir devre uygulamasını belirtmektedir.

- (a) Sadece flip-floplar.
(b) Bir sayaç ve bir dekodör.

Alternatif olarak, çıkışının zamanlama sinyallerini üreten bir durum makinesinin davranışsal tanımını yazın (gerçek donanımı dikkate almadan) T_0 ile T_5 .

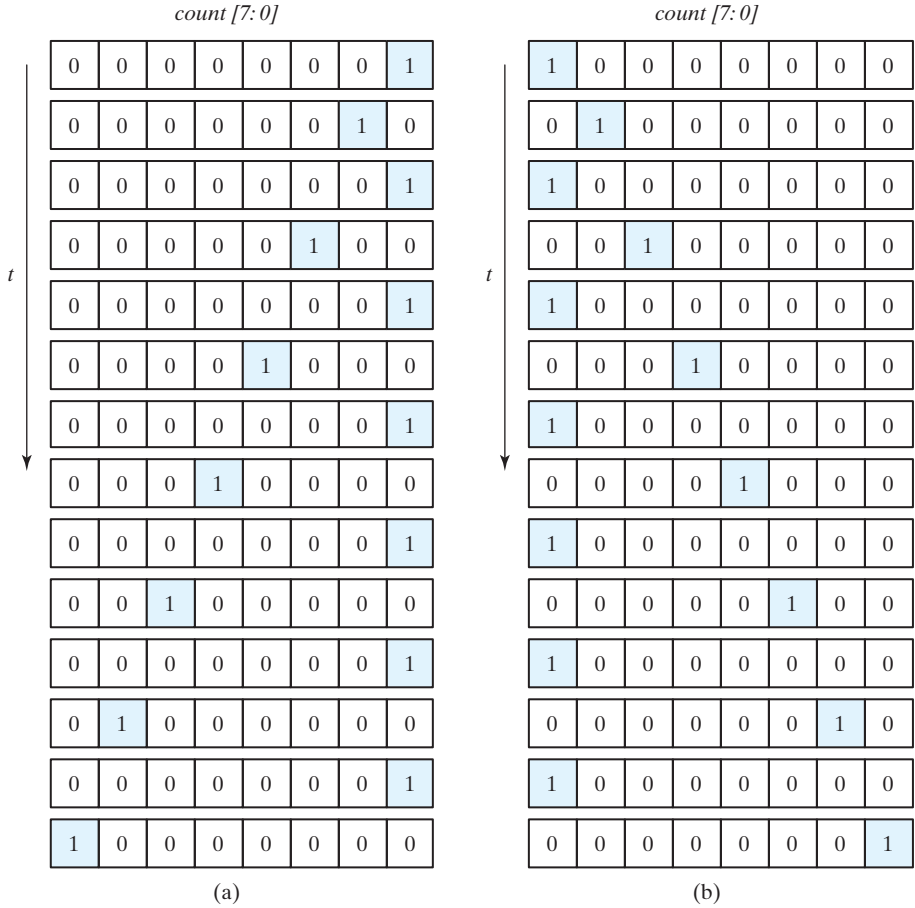
- 6.47** Şekil P6.47'de gösterilen devrenin davranışsal tanımını yazın ve devrenin çıkışının, girişin ardışık örneklerinde tek sayıda 1 varsa doğrulandığını kontrol edin.



ŞEKİL P6.47

Problem 6.47 için Devre

- 6.48** Şekil P6.48 (a)'da gösterilen sayacın davranışsal tanımını yazın ve doğrulayın; Şekil P6.48(b)'deki sayaç için tekrarlayın.
- 6.49** HDL Örneği 6.1'de tanımlanan evrensel kaydırma kaydının işlevselliğini doğrulamak için bir test planı yazın. Test planını kullanarak, HDL Örneği 6.1'de verilen modeli simüle edin.
- 6.50** Yazın ve tanımlanan sayacın davranış modelini doğrulayın
(a) Problem 6.27
(b) Problem 6.28
- 6.51** Bir durum makinesi gerektirmeden, bir kaydırma kaydı ve ek mantık kullanarak, Şekil 5.27'de tanımlanan dizilim dedektörüne alternatif bir model yazın ve doğrulayın. Uygulamaları karşılaştırın.
- 6.52** Şekil 6.7'deki evrensel kaydırma kaydının bir Verilog yapısal modelini yazın. Tüm çalışma modlarını doğrulayın.



ŞEKİL P6.48

Problem 6.48 için devre

Şekil 6.5'teki seri toplayıcının, kelimeler eklenme kaydına tekrar tekrar kaydırıldığında bir t oplayıcı olarak çalıştığını doğrulayın.

Şekil 6.6'daki seri toplayıcının yapısal modelini yazın ve doğrulayın.

Şekil 6.10'daki BCD dalgali sayıcı için bir yapısal model yazın ve doğrulayın.

Şekil 6.12'deki senkron ikili sayacın yapısal modelini yazın ve doğrulayın.

Şekil 6.13'teki yukarı-aşağı sayacının yapısal modelini yazın ve doğrulayın.

Tüm çalışma modlarını yazın ve doğrulayın

(a) Şekil 6.14'teki ikili sayıcıya ait yapısal model

(b) Şekil 6.14'teki ikili sayacın bir davranış modeli.

6.59 Yazın ve doğrulayın

(a) Şekil 6.18 (a) 'da anahtarlı kuyruk halkası sayacının yapısal modeli

(b) Şekil 6.18 (a) 'da anahtarlı kuyruk çan sayacının davranışsal modeli

REFERANSLAR

- Mano, M. M. ve C. R. Kime . 2007. *Mantık ve Bilgisayar Tasarım Temelleri*, 4. baskı. Upper Saddle River, NJ: Prentice Hall.
- Nelson V. P., H. T. Nagle, J. D. Irwin, ve B. D. Carroll . 1995. *Dijital Mantık Devresi Analizi ve Tasarımı*. Upper Saddle River, NJ: Prentice Hall.
- HAYES, J. P. 1993. *Dijital Mantık Tasarımına Giriş*. Reading, MA: Addison-Wesley.
- WAKERLY, J. F. 2000. *Dijital Tasarım: İlkeler ve Uygulamalar*, 3. baskı. Upper Saddle River, NJ: Prentice Hall.
- DIETMEYER, D. L. 1988. *Dijital Sistemlerin Mantık Tasarımı*, 3. baskı. Boston: Allyn Bacon.
- GAJSKI, D. D. 1997. *Dijital Tasarım İlkeleri*. Upper Saddle River, NJ: Prentice Hall.
- ROTH, C. H. 2009. *Mantık Tasarımının Temelleri*, 6. baskı. St. Paul: West.
- KATZ, R. H. 1994. *Çağdaş Mantık Tasarımı*. Upper Saddle River, NJ: Prentice Hall.
- CILETTI, M. D. 1999. *Verilog HDL ile Modelleme, Sentez ve Hızlı Prototipleme*. Upper Saddle River, NJ: Prentice Hall.
- B HASKER, J. 1999. *Verilog HDL'e Giriş*. Allentown, PA: Star Galaxy Press.
- Thomas, D. E. ve P. R. Moorby. 2002. *VeriLog Donanım Tanım Dili*, 5. baskı. Boston: Kluwer Akademik Yayıncılar.
- B HASKER, J. 1998. *Verilog HDL Synthesis*. Allentown, PA: Star Galaxy Press.
- PALNITKAR, S. 1996. *Verilog HDL: A Guide to Digital Design and Synthesis*. Mountain View, CA: SunSoft Press (A Prentice Hall Title).
- CILETTI, M. D. 2010. *Advanced Digital Design with the Verilog HDL*, 2e. Upper Saddle River, NJ: Prentice Hall.
- CILETTI, M. D. 2004. *Starter's Guide to Verilog 2001*. Upper Saddle River, NJ: Prentice Hall.

WEB SEARCH TOPICS

BCD sayacı
Johnson sayacı
Halka sayacı
Dizi dedektörü
Senkron sayacı
Anahtar kuyruk halka sayacı
Yükseltilmiş-aşağı sayacı

Bölüm 7

Hafıza ve Programlanabilir Mantık

7.1 GİRİŞ

Bir hafıza birimi, ikili bilgilerin depolanması için aktarıldığı ve işlenmesi gerektiğinde bilgilerin geri alındığı bir cihazdır. Veri işleme gerçekleştiğinde, hafızadan bilgi, işleme birimindeki seçilen kayıtlara aktarılır.

İşleme biriminde elde edilen ara ve nihai sonuçlar, hafızaya geri aktarılır. Bir giriş cihazından alınan ikili bilgi hafızada depolanır ve bir çıkış cihazına aktarılır. Bir hafıza birimi, büyük miktarda ikili bilgiyi depolayabilen hücrelerin bir koleksiyonudur.

Dijital sistemlerde kullanılan iki tür hafıza vardır: rastgele erişim hafızası (RAM) ve salt okunur hafıza (ROM). RAM, daha sonra kullanılmak üzere yeni bilgileri depolar. Hafızaya yeni bilgi yazma işlemi, hafızaya bilgi depolama süreci olarak adlandırılır. Depolanan bilgilerin hafızadan dışarı aktarılması işlemi ise hafızadan okuma işlemi olarak adlandırılır. RAM, hem yazma hem de okuma işlemlerini gerçekleştirebilir. ROM yalnızca okuma işlemini gerçekleştirebilir. Bu, uygun ikili bilgilerin zaten hafızada depolandığı ve herhangi bir zamanda geri alınabileceği veya okunabileceği anlamına gelir.

Ancak, bu bilgi yazılarak değiştirilemez.

ROM, *programlanabilir mantık cihazı* (PLD)dir. Bu tür bir cihazda depolanan ikili bilgi, bir şekilde belirtilir ve ardından donanımın içine yerleştirilir; bu işleme *programlama* denir. Burada "programlama" kelimesi, cihaza yerleştirilen bitleri belirleyen bir donanım prosedürünü ifade eder.

ROM, bir PLD örneğidir. Diğer benzer birimler programlanabilir mantık dizisi (PLA), programlanabilir dizi mantığı (PAL) ve alan-programlanabilir kapı dizisi (FPGA)dir. Bir PLD, elektronik yollarla bağlı iç mantık kapılarına sahip bir entegre devredir.

**ŞEKİL 7.1****Geleneksel ve dizi mantık diyagramları için VE kapısı**

Cihazın orijinal durumunda, tüm sigortalar sağlamdır. Cihazı programlamak, belirli bir mantık fonksiyonunun istenen konfigürasyonunu elde etmek için kaldırılması gereken yollar boyunca bu sigortaları patlatmayı içerir.

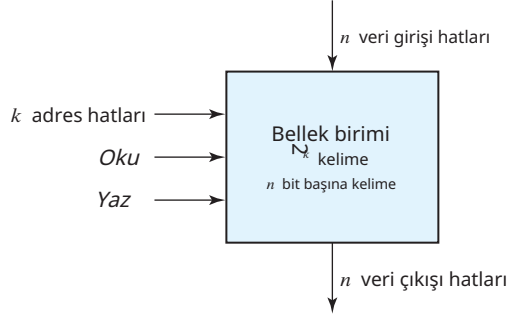
Bu bölümde, PLD'lerin konfigürasyonunu tanıtlıyoruz ve dijital sistemlerin tasarımında kullanımları için prosedürleri belirtiyoruz. Ayrıca, belirli bir mantık fonksiyonu (kombinasyonel veya sıralı) için gereken iç bağlantıyı sağlamak üzere cihazın içine bir bit akışı indirerek yapılandırılan CMOS FPGA'ları sunuyoruz.

Tipik bir PLD, yüzlerce ila milyonlarca kapıdan oluşabilir ve bunlar yüzlerce ila binlerce iç yol aracılığıyla birbirine bağlıdır. Böyle bir cihazın iç mantık diyagramını özlü bir şekilde göstermek için, dizi mantığına uygulanabilir özel bir kapı sembolü kullanmak gereklidir. Şekil 7.1, çoklu girişli bir VE kapısı için geleneksel ve dizi mantığı sembollerini göstermektedir. Kapıya birden fazla giriş hattı yerine, kapıya giren tek bir hat çizeriz. Giriş hatları bu tek hatla dik olarak çizilir ve iç sigortalar aracılığıyla kapıya bağlanır. Benzer bir şekilde, bir VE kapısı için dizi mantığını çizebiliriz. Kapıların girişleri için bu tür grafiksel temsil, dizi mantığı diyagramlarında bölüm boyunca kullanılacaktır.

7.2 RASTGELE ERİŞİM HAFIZASI

Bir bellek birimi, bilgi transferi için gerekli olan ilgili devrelerle birlikte depolama hücrelerinin bir koleksiyonudur. Belleğin mimarisi, bilginin iç konumlarından herhangi birinden seçici olarak geri alınabilmesini sağlar. İstenilen rastgele bir konuma bilgi transferinin ne kadar sürdüğü her zaman aynıdır—bu nedenle adı rastgele erişim hafızası (RAM) olarak kısaltılmıştır. Buna karşılık, manyetik bantta depolanan bilgilere erişim süresi, verilerin konumuna bağlıdır.

Bir bellek birimi, bit grupları olarak adlandırılan ikili bilgileri depolar. Bellekteki bir kelime, depolama alanından bir bütün olarak hareket eden bitlerin bir varlığıdır. Bir bellek kelimesi, 1'ler ve 0'lar grubudur ve bir sayı, bir talimat, bir veya daha fazla alfanümerik karakter veya başka herhangi bir ikili kodlanmış bilgiyi temsil edebilir. 8 bitlik bir grup, bir bayt olarak adlandırılır. Çoğu bilgisayar belleği, uzunluğu 8 bitin katları olan kelimeleri kullanır. Bu nedenle, 16 bitlik bir kelime iki bayt içerir ve 32 bitlik bir kelime dört bayttan oluşur. Bir bellek biriminin kapasitesi genellikle birimin depolayabileceği toplam bayt sayısı olarak belirtilir.



ŞEKİL 7.2

Bir bellek biriminin blok diyagramı

Bellek ile çevresi arasındaki iletişim, veri giriş ve çıkış hatları, adres seçim hatları ve transfer yönünü belirten kontrol hatları aracılığıyla sağlanır. Bir bellek biriminin blok diyagramı Şekil 7.2'de gösterilmiştir. n veri giriş hatları, bellekte depolanacak bilgiyi sağlar ve n veri çıkış hatları, bellekten çıkan bilgiyi temin eder. k adres hatları, mevcut birçok kelime arasından seçilen belirli kelimeyi belirtir. İki kontrol girişi, istenen transfer yönünü belirtir: Yazma girişi, ikili verilerin belleğe aktarılmasını sağlar ve Okuma girişi, ikili verilerin bellekten çıkarılmasını sağlar.

Bellek birimi, içerdiği kelime sayısı ve her kelimedeki bit sayısı ile tanımlanır. Adres s hatları belirli bir kelimeyi seçer. Bellekteki her kelimeye, 0'dan başlayarak $2^k - 1$ 'e kadar olan bir tanımlama numarası, yani bir adres atanır; burada k adres hatlarının sayısı ifade eder. Bellek içinde belirli bir kelimenin seçimi, k bitlik adresin adres hatlarına uygulanmasıyla gerçekleştirilir. Dahili bir kodlayıcı bu adresi kabul eder ve belirtilen kelimeyi seçmek için gereken yolları açar. Belleklerin boyutları büyük ölçüde değişiklik gösterir ve 10 bitlik bir adres gerektiren 1,024 kelimedenden, 32 adres bitine ihtiyaç duyan 232 kelimeye kadar uzanabilir. Bellekteki kelime (veya byte) sayısına genellikle K (kilo), M (mega) ve G (giga) harflerinden biri ile atıfta bulunmak adettendir. K , 2^{10} , M , 2^{20} , ve G , 2^{30} eşittir. Böylece, $64K = 2^{16}$, $2M = 2^{21}$, ve $4G = 2^{32}$.

Örneğin, her biri 16 bit olan 1K kelime kapasitesine sahip bir bellek birimini düşünelim. $1K = 1,024 = 2^{10}$ ve 16 bit iki byte olduğundan, belleğin $2,048 = 2K$ byte alabileceğini söyleyebiliriz. Şekil 7.3, bu belleğin ilk üç ve son üç kelimesinin olası içeriklerini göstermektedir. Her kelime, iki byte'a bölünebilen 16 bit içerir. Kelimeler, 0'dan 1,023'e kadar olan ondalık adresleri ile tanınır. Eşdeğer ikili adres 10 bitten oluşur. İlk adres on 0 ile belirtilirken; son adres on 1 ile belirtilir, çünkü 1,023 ikili olarak 111111111'e eşittir. Bellekteki bir kelime, ikili adresi ile seçilir. Bir kelime okunduğunda veya yazıldığında, bellek tüm 16 bit üzerinde tek bir birim olarak işlem yapar.

$1K \times 16$ bellek, Şekil 7.3'te adres için 10 bit ve her kelime için 16 bit içerir. Başka bir örnek olarak, $64K \times 10$ bellek, adres için 16 bit içerecektir (çünkü $64K = 2^{16}$) ve her kelime 10 bitten oluşacaktır. Gerekli adres bitlerinin sayısı

Hafıza adresi		Hafıza içeriği
İkili	Ondalık	
000000000	0	1011010101011101
000000001	1	1010101110001001
000000010	2	0000110101000110
	⋮	⋮
111111101	1021	1001110100010100
111111110	1022	0000110100011110
111111111	1023	1101111000100101

ŞEKİL 7.3
1024 × 16 bellek içeriği

bir bellek, bellekte saklanabilen toplam kelime sayısına bağlıdır ve her kelimedeki bit sayısından bağımsızdır. Adresteki bit sayısı, $2^k \geq m$ ilişkisine göre belirlenir, burada m toplam kelime sayısını ifade eder ve k bu ilişkiyi sağlamak için gereken adres bitlerinin sayısını ifade eder.

Yazma ve Okuma İşlemleri

RAM'in gerçekleştirebileceği iki işlem yazma ve okuma işlemleridir. Daha önce belirtildiği gibi, yazma sinyali bir transfer-giriş işlemi ve okuma sinyali bir transfer-çıkış işlemi belirtir. Bu kontrol sinyallerinden birini kabul ettiğinde, belleğin içindeki devreler istenen işlemi sağlar.

Yeni bir kelimenin belleğe aktarılması için atılması gereken adımlar şunlardır:

- İstenilen kelimenin ikili adresini adres hatlarına uygulayın.
- Bellekte saklanması gereken veri bitlerini veri giriş hatlarına uygulayın.
- Yazma girişini etkinleştirin.

Bellek birimi, giriş veri hatlarından bitleri alacak ve bunları adres hatlarıyla belirtilen kelimeye depolayacaktır.

Bellekten saklanan bir kelimeyi çıkarmak için atılması gereken adımlar şunlardır:

- İstenilen kelimenin ikili adresini adres hatlarına uygulayın.
- Okuma girişini etkinleştirin.

Tablo 7.1
Bellek Çipine Kontrol Girdileri

Bellek Etkinleştirme	Okuma/Yazma	Bellek İşlemi
0	X	Yok
1	0	Seçilen kelimeye yaz
1	1	Seçilen kelimedenden oku

Bellek birimi, adres tarafından seçilen kelimedenden bitleri alacak ve bunları çıkış v eri hatlarına uygulayacaktır. Seçilen kelimenin içeriği okuma işleminden sonra değişmez, yani kelime işlemi yıkıcı değildir.

Ticari bellek bileşenleri, entegre devre yongalarında bazen okuma ve yazma için iki kontrol girişini biraz farklı bir yapılandırmada sağlar. İki işlemi kontrol etmek için ayrı okuma ve yazma girişlerine sahip olmak yerine, çoğu entegre dev re iki başka kontrol girişi sağlar: Bir giriş birimi seçer ve diğeri işlemi belirler. B u kontrol girişlerinden kaynaklanan bellek işlemleri Tablo 7.1'de belirtilmiştir.

Bellek etkinleştirme (bazen çip seçimi olarak adlandırılır), büyük bir belleğin çoklu çip uygulamasında belirli bellek çipini etkinleştirmek için kullanılır. Bellek etkinleştir me pasif olduğunda, bellek çipi seçilmez ve hiçbir işlem gerçekleştirilmez. Bellek etki nleştirme girişi aktif olduğunda, okuma/yazma girişi gerçekleştirilecek işlemi belirler

HDL'deki Bellek Tanımı

Bellek, Verilog donanım tanım dili (HDL) kullanılarak bir dizi kayıt ile modellenir . İki boyutlu bir dizi kullanarak **reg** anahtar kelimesi ile tanımlanır. Dizideki ilk sayı, bir kelimedeki bit sayısını (kelime uzunluğu) belirtir ve ikincisi bellekteki k elime sayısını (bellek derinliği) verir. Örneğin, her biri 16 bit olan 1.024 kelimed en oluşan bir bellek şu şekilde tanımlanır:

```
reg [15: 0] memword [0: 1023];
```

Bu ifade, her biri 16 bit içeren 1.024 kaydın iki boyutlu bir dizisini tanımlar. *mem word* tanımındaki ikinci dizi aralığı, bellekteki toplam kelime sayısını belirtir ve belleğin adresine eşdeğerdir. Örneğin,

memword[512] adres 512'deki 16 bitlik bellek kelimesine atıfta bulunur.

Bir bellek biriminin çalışması HDL Örneği 7.1'de gösterilmektedir. Bellek, her biri dört bit olan 64 kelimeye sahiptir. İki kontrol girişi vardır: *Enable* ve *ReadWrite* . *DataIn* ve *DataOut* hatları her biri dört bit içerir. Giriş *Address* altı bit olmalıdır (çünkü $2^6=64$). Bellek, kayıtların iki boyutlu bir dizisi olarak tanımlanır ve Mem herhangi bir 64 kelimedenden birine erişmek için bir indeks ile referans alınabilecek bir tanımlayıcı olarak kullanılır. Bir bellek işlemi, *Enable* girişinin aktif olmasını gerektirir. *ReadWrite* girişi, işlemin türünü belirler. Eğer *ReadWrite* 1 ise, bellek, aşağıdaki ifade ile sembolize edilen bir okuma işlemi gerçekleştirir:

$\text{DataOut} \leftarrow \text{Mem} [\text{Adres}];$

Bu ifadenin yürütülmesi, Address ile belirtilen seçilen bellek kelimesinden dört bitin DataOut hatlarına aktarılmasına neden olur. Eğer $\text{ReadWrite} = 0$ ise, bellek bir yazma işlemi gerçekleştirir ve bu işlem ifadenin sembolü ile gösterilir.

$\text{Mem} [\text{Adres}] \leftarrow \text{DataIn};$

Bu ifadenin yürütülmesi, dört bitlik DataIn hatlarından Address ile seçilen bellek kelimesine bir aktarım yapılmasına neden olur.

$\text{Enable} = 0$ 'a eşit olduğunda, bellek devre dışı kalır ve çıkışların yüksek empedans durumunda olduğu varsayılır, bu durum z sembolü ile gösterilir.

Bu nedenle, bellek üç durumlu çıkışlara sahiptir.

HDL Örneği 7.1

```
// Belleğin okuma ve yazma işlemleri
// Bellek boyutu her biri dört bit olan 64 kelimedir.

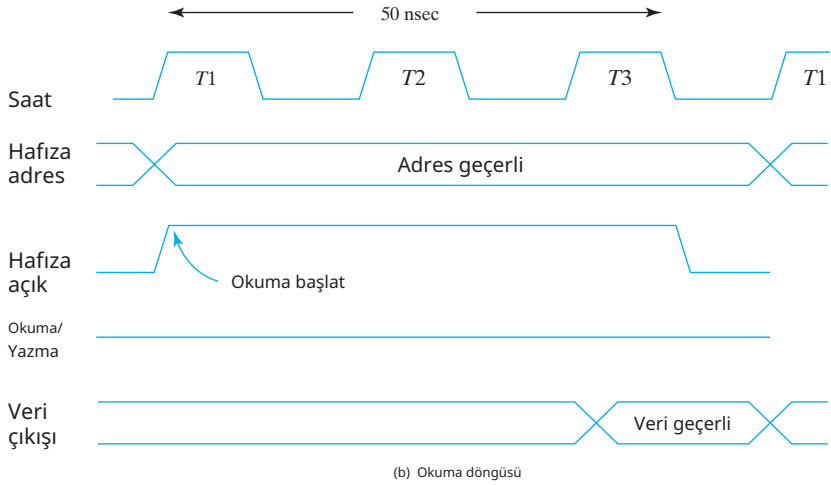
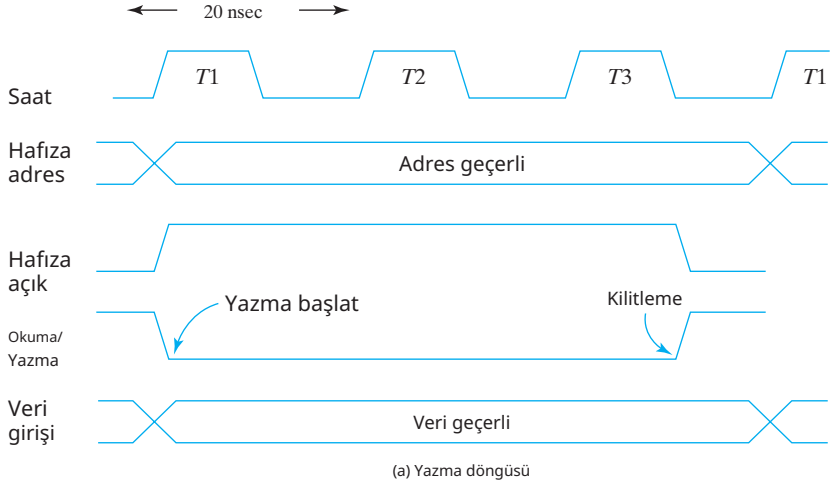
modül bellek (Enable, ReadWrite, Address, DataIn, DataOut);
girdi Enable, ReadWrite;
girdi [3:0] DataIn;
girdi [5:0] Address;
çıkış [3:0] DataOut;
kayıt [3:0] DataOut;
mem [31:0] Mem [0:63]; // 64 x 4 bellek
her zaman @ (Enable veya ReadWrite)
if (Enable)
    if (ReadWrite) DataOut = Mem [Address]; // Oku
    else Mem [Address] = DataIn; // Yaz
else DataOut = 4'bz; // Yüksek empedans durumu
endmodule
```

Zamanlama Dalga Biçimleri

Hafıza biriminin çalışması, merkezi işlem birimi (CPU) gibi bir dış cihaz tarafında kontrol edilir. CPU genellikle kendi saatiyle senkronize edilir. Ancak, hafıza içsel bir saat kullanmaz. Bunun yerine, okuma ve yazma işlemleri kontrol girişleriyle belirlenir. Hafızanın *erişim süresi*, bir kelimeyi seçmek ve okumak için gereken zamandır. Hafızanın *döngü süresi*, bir yazma işlemini tamamlamak için gereken zamandır. CPU, hafızanın okuma ve yazma işlemleriyle içsel saatli işlemleri senkronize edecek şekilde hafıza kontrol sinyallerini sağlamalıdır.

Bu, hafızanın erişim süresi ve döngü süresinin, sabit bir CPU saat döngüsü sayısına eşit bir zaman içinde olması gerektiği anlamına gelir.

Örneğin, bir CPU'nun 50 MHz saat frekansında çalıştığını varsayalım, bu da bir saat döngüsü için 20 ns'lik bir periyot verir. Ayrıca, CPU'nun erişim süresi ve döngü süresi 50 ns'yi aşmayan bir hafızayla iletişim kurduğunu varsayalım. Bu,



ŞEKİL 7.4

Hafıza döngüsü zamanlama dalga formları

yazma döngüsü, seçilen kelimenin depolanmasını 50 ns'lik bir aralık içinde sonlandırır ve okuma döngüsü, seçilen kelimenin çıkış verilerini 50 ns veya daha kısa bir sürede sağlar. (İki sayı her zaman aynı değildir.) CPU döngüsünün süresi 20 ns olduğunda n, her bellek isteği için en az iki buçuk, muhtemelen üç saat döngüsü ayırmak gerekecektir.

Şekil 7.4'te gösterilen bellek zamanlaması, 50 MHz saat hızına sahip bir CPU ve 50 ns maksimum döngü süresine sahip bir bellek içindir. (a) kısmındaki yazma döngüsü, üç 20 ns döngüsünü göstermektedir: T1,

T_2 , ve T_3 . Yazma işlemi için, CPU hafızaya adres ve giriş verilerini sağlamalıdır. Bu, T_1 'in başında yapılır. (Adres ve veri dalga formlarında kesişen iki hat, çoklu hatların değerinde olası bir değişikliği belirtir.) Hafıza etkinleştirme ve okuma/yazma sinyalleri, adres hatlarındaki sinyaller kararlı hale geldikten sonra etkinleştirilmelidir, böylece diğer hafıza kelimelerindeki verilerin yok edilmesi önlenir.

Hafıza etkinleştirme sinyali yüksek seviyeye geçer ve okuma/yazma sinyali düşük seviyeye geçerek bir yazma işlemini belirtir. İki kontrol sinyali en az 50 ns boyunca aktif kalmalıdır. Kontrol sinyalleri devre dışı bırakıldıktan sonra adres ve veri sinyalleri kısa bir süre kararlı kalmalıdır. Üçüncü saat döngüsünün tamamlanmasıyla, hafıza yazma işlemi tamamlanır ve CPU bir sonraki T_1 döngüsü ile hafızaya tekrar erişebilir.

Şekil 7.4 (b)'de gösterilen okuma döngüsü, hafıza için CPU tarafından sağlanan bir adrese sahiptir. Okuma işlemi için hafıza etkinleştirme ve okuma/yazma sinyalleri yüksek seviyede olmalıdır. Hafıza, adres tarafından seçilen kelimenin verisini, hafıza etkinleştirildiği andan itibaren 50 ns (veya daha kısa) içinde çıkış veri hatlarına yerleştirir. CPU, veriyi T_3 'ün negatif geçişi sırasında iç kayıtlarından birine aktarabilir. Bir sonraki T_1 döngüsü başka bir hafıza isteği için kullanılabilir.

Hafıza Türleri

Bir hafıza sisteminin erişim modu, kullanılan bileşenlerin türü tarafından belirlenir. Rastgele erişim hafızasında, kelime konumları uzayda ayrılmış olarak düşünülebilir; her kelime belirli bir konumu kaplar. Sıralı erişim hafızasında, bazı ortamlarda depolanan bilgiler hemen erişilebilir değildir, ancak yalnızca belirli zaman aralıklarında mevcuttur. Bir manyetik disk veya bant ünitesi bu türdendir. Her hafıza konumu sırayla okuma ve yazma başlıklarından geçer, ancak bilgi yalnızca istenen kelimeye ulaşıldığında okunur. Rastgele erişim hafızasında, erişim süresi kelimenin belirli konumuna bakılmaksızın her zaman aynıdır. Sıralı erişim hafızasında, bir kelimeye erişim süresi, kelimenin okuma başlığının konumuna göre konumuna bağlıdır; bu nedenle, erişim süresi değişkendir.

Entegre devre RAM üniteleri iki çalışma modunda mevcuttur: *statik* ve *dinamik*. Statik RAM (SRAM), esasen ikili bilgileri depolayan iç kilitlerden oluşur. Depolanan bilgi, üniteye güç uygulandığı sürece geçerliliğini korur. Dinamik RAM (DRAM), ikili bilgiyi, çip içinde MOS transistörleri tarafından sağlanan kapasitörler üzerindeki elektrik yükleri şeklinde depolar. Kapasitörlerdeki depolanan yük zamanla boşalmaya eğilimlidir ve kapasitörler, dinamik hafızayı *yenileyerek* periyodik olarak yeniden şarj edilmelidir. Yenileme, bozulmakta olan yükü geri yüklemek için her birkaç milisaniyede kelimeler arasında döngü yaparak gerçekleştirilir. DRAM, tek bir hafıza çipinde daha düşük güç tüketimi ve daha büyük depolama kapasitesi sunar. SRAM kullanımı daha kolaydır ve daha kısa okuma ve yazma döngülerine sahiptir.

Güç kapatıldığında saklanan bilgileri kaybeden bellek birimlerine *volatile* denir. CMOS entegre devre RAM'leri, hem statik hem de dinamik, bu kategoriye girer, çünkü ikili hücrelerin saklanan bilgiyi korumak için dış güç gereksinimi vardır. Buna karşılık, manyetik disk gibi *nonvolatile* bir bellek, güç kesildikten sonra saklanan bilgiyi korur.

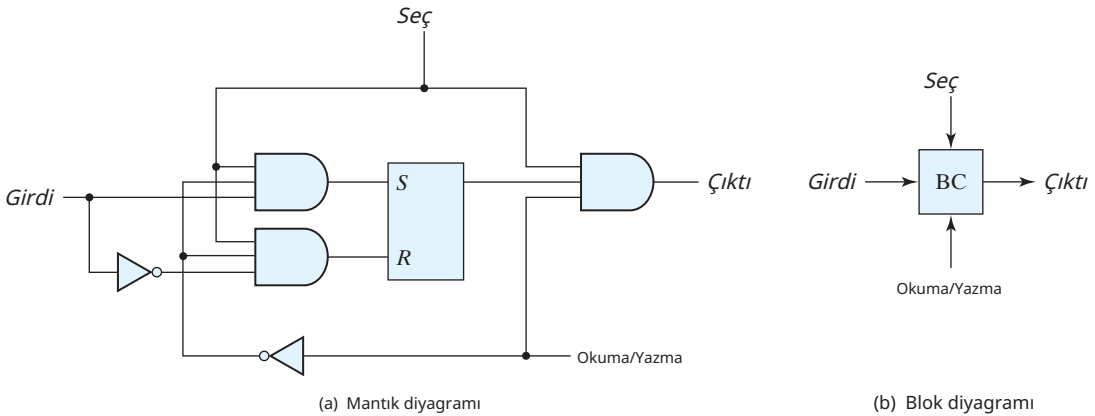
bellek, manyetik bileşenlerde saklanan verilerin, güç kapatıldıktan sonra korun an manyetizasyon yönü ile temsil edilmesi nedeniyle bilgiyi koruyabilir. ROM, b aşka bir nonvolatile bellek türüdür. Nonvolatile bir bellek, dijital bilgisayarların, bilgisayar açıldığında tekrar ihtiyaç duyulacak programları saklamasını sağlar. Değiştirilemeyen programlar ve veriler ROM'da saklanırken, diğer büyük programlar manyetik disklerde tutulur. Bu son programlar ihtiyaç duyulduğunda bilgisayar RAM'ine aktarılır. Güç kapatılmadan önce, bilgisayar RAM'inden ikili bilgiler diske aktarılır, böylece bilgiler korunur.

7.3 BELLEK KODLAMA

Bir bellek biriminde depolama bileşenlerine ihtiyaç duymanın yanı sıra, giriş adresi tarafından belirtilen bellek kelimesini seçmek için kod çözme devrelerine de ihtiyaç vardır. Bu bölümde, bir RAM'in iç yapısını sunuyoruz ve kod çözücünün çalışmasını gösteriyoruz. Tüm belleği tek bir diyagramda gösterebilmek için, burada sunulan bellek birimi 4 bitlik dört kelimeden oluşan 16 bitlik küçük bir kapasiteye sahiptir. Daha büyük belleklerde kullanılan daha verimli bir kod çözme şemasını göstermek için iki boyutlu çakışan kod çözme düzeninin bir örneği sunulmuştur. Ardından, DRAM entegre devrelerinde yaygın olarak kullanılan adres çoklama örneğini veriyoruz.

İç Yapı

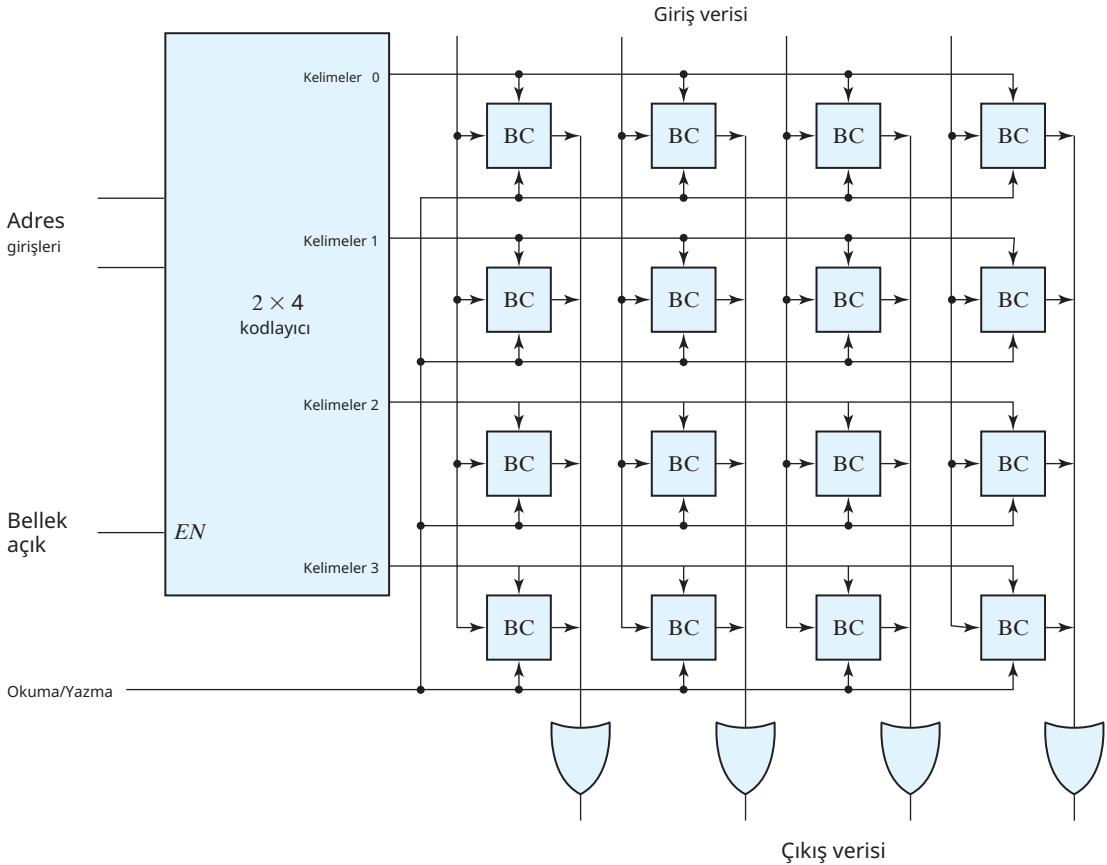
Bir RAM'in dahili yapısı, m kelime ve n bit içeren $m \times n$ binary depolama hücreleri ve bireysel kelimeleri seçmek için ilgili kod çözme devrelerinden oluşur. Binary depolama hücresi, bir bellek biriminin temel yapı taşıdır. Bir bit bilgi depolayan bir binary hücrenin eşdeğer mantığı Şekil 7.5'te gösterilmiştir. Hücrenin depolama kısmı, bir SR kilidi ile modellenmiştir ve bir D kilidi oluşturmak için ilgili kapılarla birleştirilmiştir. Aslında,



ŞEKİL 7.5
Hafıza hücresi

hücre, dört ila altı transistörden oluşan bir elektronik devredir. Yine de, mantık sembolleri açısından modellemek mümkündür ve pratikte de uygundur. İkili bir depolama hücresi, entegre devre çipinde mümkün olduğunca fazla hücre yerleştirebilmek için çok küçük olmalıdır. İkili hücre, içindeki latch'te bir bit depolar. Seçim girişi, hücreyi okuma veya yazma için etkinleştirir ve okuma/yazma girişi, hücre seçildiğinde hücrenin işlemini belirler. Okuma/yazma girişinde bir 1, latch'ten çıkış terminaline bir yol oluşturarak okuma işlemini sağlar. Okuma/yazma girişinde bir 0, giriş terminalinden latch'e bir yol oluşturarak yazma işlemini sağlar.

Bir küçük RAM'in mantıksal yapısı Şekil 7.6'da gösterilmiştir. Bu RAM, her biri dört bit olan dört kelimedenden oluşur ve toplamda 16 ikili hücreye sahiptir. BC ile etiketlenmiş küçük bloklar, Şekil 7.5 (b)'de belirtildiği gibi üç girişi ve bir çıkışı olan ikili hücreyi temsil eder. Dört kelimedenden oluşan bir bellek, iki adres hattına ihtiyaç duyar. İki adres girişi, dört kelimedenden birini seçmek için bir 2×4 kodlayıcıdan geçer. Kodlayıcı,



ŞEKİL 7.6
4 × 4 RAM'in Diyagramı

k etkinleştirme girişi. Bellek etkinleştirme 0 olduğunda, kod çözücünün tüm çıkışları 0'dır ve bellek kelimelerinin hiçbirisi seçilmez. Bellek seçimi 1 olduğunda, dört kelimedenden biri, iki adres hattındaki değere göre seçilir. Bir kelime seçildiğinde, okuma/yazma girişi işlemi belirlir. Okuma işlemi sırasında, seçilen kelimenin dört biti, çıkış terminallerine OR kapıları aracılığıyla geçer.

(OR kapılarının, Şekil 7.1'de belirlenen dizi mantığına göre çizildiğini unutmayın.) Yazma işlemi sırasında, giriş hatlarında mevcut olan veriler, seçilen kelimenin dört ikili hücrelerine aktarılır. Seçilmeyen ikili hücreler devre dışı bırakılır ve önceki ikili değerleri değişmeden kalır. Kod çözücüsüne giden bellek seçme girişi 0 olduğunda, hiçbir kelime seçilmez ve tüm hücrelerin içeriği, okuma/yazma girişinin değerine bakılmaksızın değişmeden kalır.

Ticari RAM'ler binlerce kelime kapasitesine sahip olabilir ve her kelime 1 ile 64 bit arasında değişebilir. Büyük kapasiteli bir belleğin mantıksal yapısı, burada gösterilen yapılandırmanın doğrudan bir uzantısı olacaktır. 2^k kelimeye sahip bir bellek, n bit başına kelime için k adres hattı gerektirir ve bu hatlar bir $k \times 2^k$ dekoderine gider. Dekoderin her bir çıkışı, okuma veya yazma için n bitlik bir kelimeyi seçer.

Kesin Kodlama

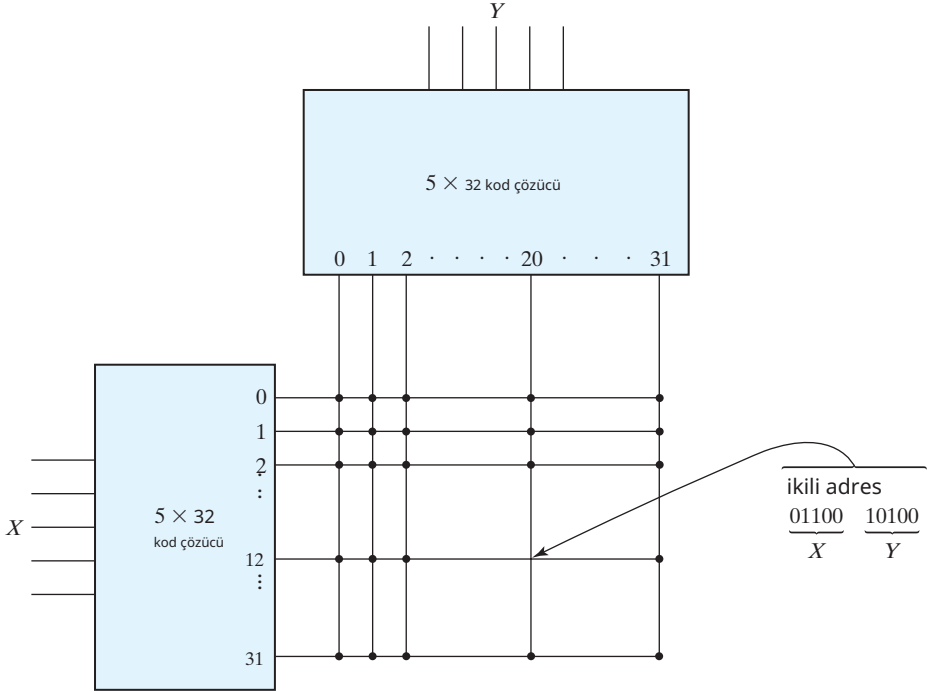
k girişi ve $2k$ çıkışı olan bir dekoder, her kapı için k girişi olan $2k$ AND kapısına ihtiyaç duyar. Toplam kapı sayısı ve her kapı başına giriş sayısı, iki boyutlu bir seçim şeması kullanılarak azaltılabilir. İki boyutlu kodlamadaki temel fikir, bellek hücrelerini mümkün olduğunca kareye yakın bir dizi halinde düzenlemektir. Bu yapılandırma da, bir $k/2$ girişli iki dekoder, bir k girişli dekoder yerine kullanılır. Bir dekoder satır seçim işlemini, diğeri ise iki boyutlu matris yapılandırmasında sütun seçim işlemini gerçekleştirir.

İki boyutlu seçim deseni, 1K-kelime belleği için Şekil 7.7'de gösterilmektedir. Tek bir $10 \times 1,024$ dekoder kullanmak yerine, iki 5×32 dekoder kullanıyoruz. Tek dekoder ile her biri 10 girişi olan 1.024 VE kapısına ihtiyacımız var. İki dekoder durumunda, her biri 5 girişi olan 64 VE kapısına ihtiyacımız var. Adresin en anlamlı beş biti girdi X 'e ve en az anlamlı beş biti girdi Y 'ye gider. Bellek dizisi içindeki her kelime, bir X hattı ve bir Y hattının kesişimi ile seçilir. Böylece, bellekteki her kelime, 32 satırdan 1'i ve 32 sütundan 1'i arasındaki kesişimle seçilir; toplamda 1.024 kelime vardır. Her kesişimin, herhangi bir bit sayısına sahip olabilecek bir kelimeyi temsil ettiğini unutmayın.

Örnek olarak, adresi 404 olan kelimeyi düşünelim. 404'ün 10 bitlik ikili karşılığı 01100 10100'dür. Bu, $X=01100$ (ikili 12) ve $Y=10100$ (ikili 20) yapar. Seçilen n bit kelime, X dekoder çıkış numarası 12 ve Y dekoder çıkış numarası 20'de yer alır. Kelimenin tüm bitleri okuma veya yazma için seçilir.

Adres Çöklama

Şekil 7.5'te modellenen SRAM bellek hücresi genellikle altı transistör içerir. Daha yüksek yoğunlukta bellekler inşa etmek için, bir hücredeki transistör sayısını azaltmak gereklidir. DRAM hücresi tek bir MOS transistörü ve bir kondansatör içerir. Depolanan yük



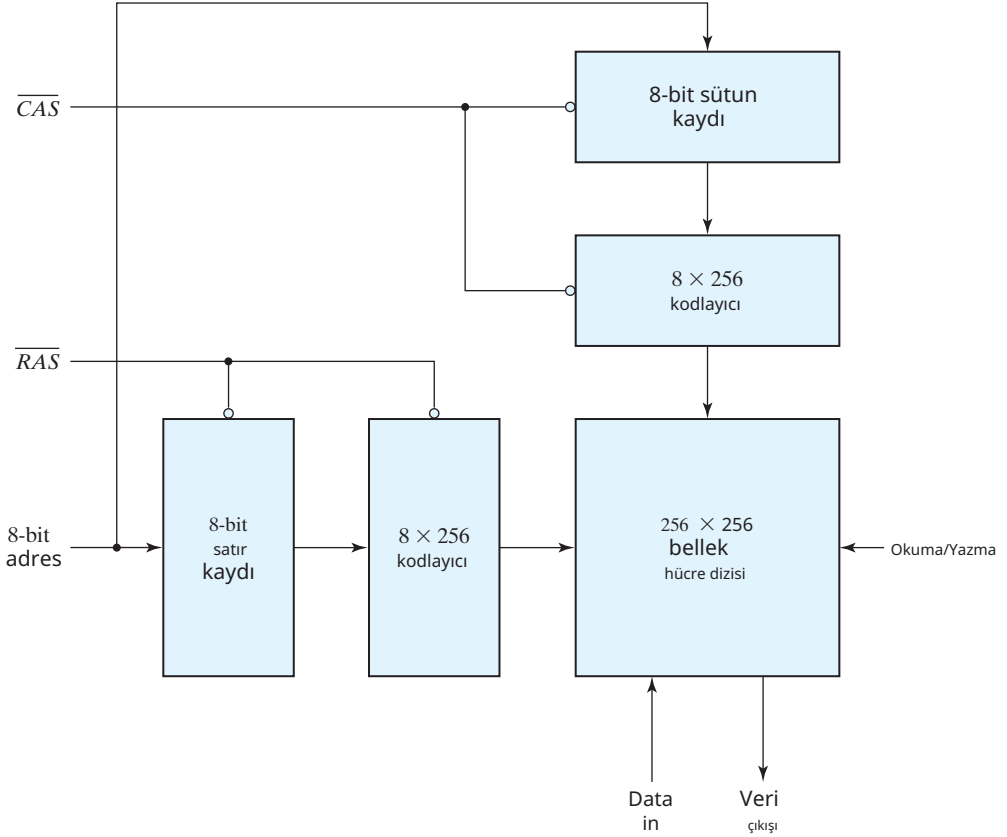
ŞEKİL 7.7

1K kelimelik bellek için iki boyutlu kod çözme yapısı

Kapasitör zamanla deşarj olur ve bellek hücreleri, belleği yenileyerek periyodik olarak yeniden şarj edilmelidir. Basit hücre yapıları nedeniyle, DRAM'ler genellikle SRAM'lerin dört katı yoğun luğa sahiptir. Bu, belirli bir boyuttaki çipe dört kat daha fazla bellek kapasitesinin yerleştirilme sine olanak tanır. DRAM depolamanın bit başına maliyeti, SRAM depolamanın maliyetinin üç il a dört katı daha düşüktür. DRAM hücrelerinin daha düşük güç gereksinimi nedeniyle ek maliy et tasarrufu sağlanır. Bu avantajlar, DRAM'ı kişisel dijital bilgisayarlardaki büyük bellekler için t ercih edilen teknoloji haline getirir. DRAM yongaları 64K'dan 256M bite kadar kapasitelerde m evcuttur. Çoğu DRAM, 1 bit kelime boyutuna sahiptir, bu nedenle daha büyük bir kelime boyu tu üretmek için birkaç yonganın birleştirilmesi gerekir.

Büyük kapasiteleri nedeniyle, DRAM'lerin adres çözümlemesi iki boyutlu bir dizi şeklinde düzenlenmiştir ve daha büyük bellekler genellikle birden fazla dizi ye sahiptir. IC paketindeki pin sayısını azaltmak için, tasarımcılar bir dizi adres g iriş pini ile adres bileşenlerini barındıran adres çoklama yöntemini kullanır. İki boyutlu bir dizide, adres iki kısımda farklı zamanlarda uygulanır; önce satır adr esi, sonra sütun adresi. Adresin her iki kısmı için aynı pin seti kullanıldığından, paketin boyutu önemli ölçüde azalır.

64K kelime belleğini adres çoklama fikrini açıklamak için kullanacağız. Şekil 7.8'de kod çözme yapılandırmasının bir diyagramı gösterilmektedir. Bellek,



ŞEKİL 7.8
64K DRAM için adres çoklama

256 satır ve 256 sütun halinde düzenlenmiş iki boyutlu bir hücre dizisi, toplamda $2^8 \times 2^8 = 2^{16} = 64K$ kelime. Tek bir veri girişi satırı, tek bir veri çıkışı satırı, ve bir okuma/yazma kontrolü ile birlikte, ayrıca sekiz bitlik bir adres girişi ve iki adres strobu bulunmaktadır, bunlar, sırasıyla satır ve sütun adreslerini kendi kayıtlarına aktarmak için dahil edilmiştir. Satır adres strobu (RAS), sekiz bitlik satır kaydını etkinleştirir ve sütun adres strobu (CAS), sekiz bitlik sütun kaydını etkinleştirir. Strobu sembolünün üstündeki çubuk, kayıtların sinyalin sıfır seviyesinde etkinleştirildiğini gösterir.

16 bitlik adres, RAS ve CAS kullanılarak DRAM'e iki adımda uygulanır. Başlangıçta, both stroblar 1 durumundadır. 8 bitlik satır adresi adres girişlerine uygulanır ve RAS 0'a değiştirilir. Bu, satır adresini satır adres kaydına yükler. RAS ayrıca satır kodlayıcısını etkinleştirir, böylece satır adresini çözebilir ve dizinin bir satırını seçebilir. Satır seçimi için yerleşme süresine eşdeğer bir süre geçtikten sonra, RAS tekrar 1 seviyesine döner. 8 bitlik sütun adresi daha sonra adres girişlerine uygulanır ve CAS 0 durumuna çekilir. Bu, sütun adresini sütun kaydına aktarır ve

kolon kodlayıcısını etkinleştirir. Şimdi adresin iki kısmı kendi register'larında, kodlayıcılar bunları satır ve kolon adresine karşılık gelen bir hücreyi seçmek için çözmüştür ve o hücre üzerinde bir okuma veya yazma işlemi gerçekleştirilebilir. CAS, başka bir bellek işlemi başlatmadan önce 1 seviyesine geri dönmelidir.

7.4 HATA TESPİTİ VE DÜZENLEME

Bir bellek biriminin veri yolunu etkileyen elektrik sinyallerinin dinamik fiziksel etkileşimi, ikili bilgilerin depolanması ve geri alınmasında ara sıra hatalara neden olabilir. Bir bellek biriminin güvenilirliği, hata tespit ve düzeltme kodları kullanılarak artırılabilir. En yaygın hata tespit yöntemi parite bitidir.

(Bkz. Bölüm 3.9.) Bir parite biti, bellek içinde veri kelimesi ile birlikte üretilir ve saklanır. Kelimenin paritesi, bellekten okunduktan sonra kontrol edilir. Eğer okunan bitlerin paritesi doğruysa, veri kelimesi kabul edilir. Eğer parite kontrolü bir tersine dönüşle sonuçlanırsa, bir hata tespit edilir, ancak düzeltilemez.

Hata düzeltme kodu, bellek içinde veri kelimesi ile birlikte saklanan birden fazla parite kontrol biti üretir. Her kontrol biti, veri kelimesindeki bir grup bitin paritesidir. Kelime bellekten geri okunduğunda, ilişkili parite bitleri de bellekten okunur ve okunan verilerden üretilen yeni bir kontrol biti seti ile karşılaştırılır. Kontrol bitleri doğruysa, hata meydana gelmemiştir. Kontrol bitleri saklanan parite ile eşleşmiyorsa, hatalı olan biti tanımlamak için kullanılabilecek benzersiz bir desen, yani bir *sendrom* üretir. Tek bir hata, bir bitin yazma veya okuma işlemi sırasında 1'den 0'a veya 0'dan 1'e değer değiştirmesiyle meydana gelir. Hatalı olan belirli bit tanımlanırsa, hata hatalı bitin tamamlanmasıyla düzeltilir.

Hamming Kodu

RAM'lerde kullanılan en yaygın hata düzeltme kodlarından biri R. W. tarafından tasarlandı. Hamming. Hamming kodunda, k parite bitleri n bitlik veri kelimesine eklenir ve yeni bir kelime $n+k$ bit oluşturur. Bit pozisyonları 1'den $n+k$ 'ye kadar sıralı olarak numaralandırılır. 2'nin kuvveti olarak numaralandırılan bu pozisyonlar parite bitleri için ayrılmıştır. Kalan bitler ise veri bitleridir. Kod, herhangi bir uzunluktaki kelimelerle kullanılabilir. Kodun genel özelliklerini vermeden önce, sekiz bitlik bir veri kelimesi ile işleyişini gösterelim.

Örneğin, 8 bitlik veri kelimesi 11000100'ü düşünelim. 8 bitlik kelimeye 4 parite bit ekliyoruz ve 12 biti aşağıdaki gibi düzenliyoruz:

Bit pozisyonu:	1	2	3	4	5	6	7	8	9	10	11	12
	P_1	P_2	1	P_4	1	0	0	P_8	0	1	0	0

4 parite biti, P_1, P_2, P_4 , ve P_8 , sırasıyla 1, 2, 4 ve 8. pozisyonlardadır. Veri kelimesinin 8 biti kalan pozisyonlardadır. Her parite biti aşağıdaki gibi hesaplanır:

$$P_1 = \text{XOR of bits (3, 5, 7, 9, 11)} = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$P_2 = \text{XOR of bits (3, 5, 7, 10, 11)} = 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0$$

$$P_4 = \text{XOR of bits (5, 6, 7, 12)} = 1 \oplus 0 \oplus 0 \oplus 0 = 1$$

$$P_8 = \text{XOR of bits (9, 10, 11, 12)} = 0 \oplus 1 \oplus 0 \oplus 0 = 1$$

Unutmayın ki, özel-VE işlemi tek işlevi yerine getirir: Değişkenlerdeki 1'lerin tek sayısı için 1'e ve çift sayısı için 0'a eşittir. Böylece, her parite biti, kontrol edilen pozisyonlardaki toplam 1 sayısının, parite biti dahil, her zaman çift olmasını sağlamak için ayarlanır.

8 bitlik veri kelimesi, 4 parite biti ile birlikte 12 bitlik bir bileşik kelime olarak bellekte saklanır. 4 P bitini uygun pozisyonlarına yerleştirerek, bellekte saklanan 12 bitlik bileşik kelimeyi elde ederiz:

	0	0	1	1	1	0	0	1	0	1	0	0
Bit pozisyonu:	1	2	3	4	5	6	7	8	9	10	11	12

12 bit bellekte okunduğunda, tekrar hata kontrolü yapılır. Parite, parite bitini de içeren aynı bit kombinasyonu üzerinde kontrol edilir. 4 kontrol biti aşağıdaki gibi değerlendirilir:

$$C_1 = \text{XOR of bits (1, 3, 5, 7, 9, 11)}$$

$$C_2 = \text{XOR of bits (2, 3, 6, 7, 10, 11)}$$

$$C_4 = \text{XOR of bits (4, 5, 6, 7, 12)}$$

$$C_8 = \text{XOR of bits (8, 9, 10, 11, 12)}$$

A 0 kontrol biti, kontrol edilen bitler üzerinde çift pariteyi belirtir ve 1, tek pariteyi belirtir.

Bitler çift parite ile saklandığından, sonuç, $C = C_8 C_4 C_2 C_1 = 0000$, hata olmadığını gösterir. Ancak, eğer $C \neq 0$, o zaman kontrol bitleri tarafından oluşturulan 4 bitlik ikili sayı hatalı bitin konumunu verir.

Örneğin, aşağıdaki üç durumu düşünün:

Bit pozisyonu:	1	2	3	4	5	6	7	8	9	10	11	12	
0		0	1	1	1	0	0	1	0	1	0	0	Hata yok
1		0	1	1	1	0	0	1	0	1	0	0	Bit 1'de hata
0		0	1	1	0	0	0	1	0	1	0	0	Bit 5'te hata

İlk durumda, 12 bitlik kelimede hata yoktur. İkinci durumda, bit pozisyonu numara 1'de bir hata vardır çünkü 0'dan 1'e değişmiştir. Üçüncü durum,

bit pozisyonu 5'te, 1'den 0'a bir değişiklik ile bir hata göstermektedir. İlgili bitlerin XOR'unu değerlendirerek, 4 kontrol bitini aşağıdaki gibi belirliyoruz:

	C_4	C_3	C_2	C_1
Hata yoksa:	0	0	0	0
Bit 1'de hata ile:	0	0	0	1
Bit 5'te hata ile:	0	1	0	1

Böylece, hata yoksa $C = 0000$; bit 1'de hata ile $C = 0001$; ve bit 5'te hata ile $C = 0101$. İkili sayı C 0000'a eşit olmadığında, bu, hatalı bitin pozisyonunu verir. Hata, ilgili bitin tamamlanmasıyla düzeltilebilir. Verilen kelimede veya parite bitlerinden birinde hata olabileceğini unutmayın.

Hamming kodu, herhangi bir uzunluktaki veri kelimeleri için kullanılabilir. Genel olarak, Hamming kodu k kontrol bitleri ve n veri bitlerinden oluşur, toplamda $n+k$ bit vardır. Sendrom değeri C k bit içerir ve 0 ile $2^k - 1$ arasında 2^k değer aralığına sahiptir. Bu değerlerden biri, genellikle sıfır, hata tespit edilmediğini belirtmek için kullanılır ve $2^k - 1$ değer bırakır, bu da hangi $n+k$ bitinin hatalı olduğunu gösterir. Bu $2^k - 1$ değerinin her biri, hatalı bir biti benzersiz bir şekilde tanımlamak için kullanılabilir. Bu nedenle, k aralığı $n+k$ ile eşit veya daha büyük olmalıdır, bu da aşağıdaki ilişkiyi verir.

$$2^k - 1 \geq n + k$$

k cinsinden n için çözüldüğünde, elde ederiz.

$$2^k - 1 - k \geq n$$

Bu ilişki, k kontrol bitleri ile birlikte kullanılabilecek veri bitlerinin sayısını belirlemek için bir formül verir. Örneğin, $k = 3$ olduğunda, kullanılabilecek veri bitlerinin sayısı $n \dots (2^3 - 1 - 3) = 4$ 'tür. $k = 4$ için, $2^4 - 1 - 4 = 11$ elde ederiz, bu da $n \dots 11$ verir. Veri kelimesi 11 bitten aza olabilir, ancak en az 5 bit olmalıdır; aksi takdirde, yalnızca 3 kontrol bitine ihtiyaç duyulacaktır. Bu, önceki örnekteki 8 veri biti için 4 kontrol bitinin kullanılmasını haklı çıkarır. Farklı k değerleri için n aralıkları Tablo 7.2'de listelenmiştir.

Parity üretimi ve kontrolü için bitlerin gruplandırılması, 0'dan $2^k - 1$ 'e kadar olan ikili sayıların listesinden belirlenebilir. En az anlamlı bit, ikili sayılarda 1, 3, 5, 7 gibi sayılarda 1'dir. İkinci anlamlı bit, ikili sayılarda 1'dir.

Tablo 7.2
Kontrol Bitleri İçin Veri Bitleri Aralığı

Kontrol Bitleri Sayısı, k	Veri Bitleri Aralığı, n
3	2–4
4	5–11
5	12–26
6	27–57
7	58–120

2, 3, 6, 7, vb. Bu sayıları, Hamming kodunda parite bitlerini oluşturma ve kontrol etme işlemlerinde kullanılan bit pozisyonlarıyla karşılaştırdığımızda, koddaki bit grupları ile ikili sayım dizisindeki 1-bitlerin pozisyonu arasındaki ilişkiyi not ediyoruz. Her bit grubu, 2'nin bir kuvveti olan bir sayı ile başlar: 1, 2, 4, 8, 16, vb. Bu sayılar aynı zamanda parite bitlerinin pozisyon numaralarıdır.

Tek Hata Düzeltme, Çift Hata Tespiti

Hamming kodu yalnızca tek bir hatayı tespit edebilir ve düzeltebilir. Kodlanmış kelimeye başka bir parite biti ekleyerek, Hamming kodu tek bir hatayı düzeltmek ve çift hataları tespit etmek için kullanılabilir. Bu ek parite bitini dahil edersek, önceki 12 bitlik kodlanmış kelime 00111010100 P_{13} haline gelir, burada P_{13} diğer 12 bitin özel-VE'sinden değerlendirilir. Bu, 0011100101001 (çift parite) olan 13 bitlik kelimeyi üretir. 13 bitlik kelime bellekten okunduğunda, kontrol bitleri değerlendirilir, ayrıca tüm 13 bit üzerindeki parite P de değerlendirilir. Eğer $P=0$ ise, parite doğrudur (çift parite), ancak eğer $P=1$ ise, o zaman 13 bit üzerindeki parite yanlıştır (tek parite). Aşağıdaki dört durum ortaya çıkabilir:

Eğer $C=0$ ve $P=0$ ise, hata oluşmadı.

... $\neq 0$ ve $P=1$, düzeltilebilecek tek bir hata meydana geldi.

İf $C \neq 0$ ve $P=0$, çift bir hata meydana geldiği tespit edildi, ancak bu hata düzeltildi.

Eğer $C=0$ ve $P=1$ ise, P_{13} bitinde bir hata oluştu.

Bu şema iki veya daha fazla hatayı tespit edebilir, ancak tüm bu

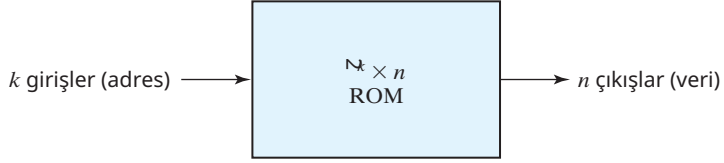
hataları tespit edeceği garanti edilmez.

Entegre devreler, tek hata düzeltme ve çift hata tespiti için parite bitlerini oluşturmak ve kontrol etmek amacıyla değiştirilmiş bir Hamming kodu kullanır. Değiştirilmiş Hamming kodu, XOR işlemini hesaplamak için kullanılan bit sayısını dengeleyen daha verimli bir parite yapılandırması kullanır. 8 bitlik veri kelimesi ve 5 bitlik kontrol kelimesi kullanan tipik bir entegre devre IC türü 74637'dir. 16 ve 32 bitlik veri kelimeleri için başka entegre devreler de mevcuttur. Bu devreler, yazma ve okuma işlemleri sırasında tek bir hatayı düzeltmek veya çift hataları tespit etmek için bir bellek birimi ile birlikte kullanılabilir.

7.5 SADECE OKUNUR BELLEK

Bir salt okunur bellek (ROM), esasen kalıcı ikili informasyonun depolandığı bir bellek cihazıdır. İkili bilgi, tasarımcı tarafından belirlenmeli ve birimi oluşturmak için gerekli bağlantı desenini oluşturacak şekilde birime yerleştirilmelidir. Desen belirlendikten sonra, güç kapatılıp açıldığında bile birim içinde kalır.

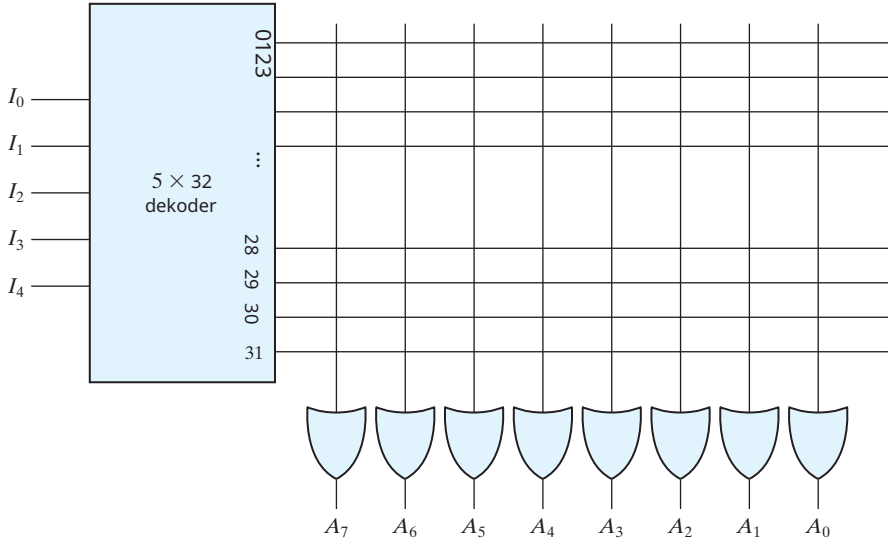
Bir ROM'un blok diyagramı, k girişi ve n çıkışı ile birlikte Şekil 7.9'da gösterilmiştir. Girişler, belleğin adresini sağlar ve çıkışlar, adres tarafından seçilen saklanan kelimelerin veri bitlerini verir. Bir ROM'daki kelime sayısı, 2^k kelimeyi belirtmek için n^k adres giriş hattının gerektiği gerçeğinden belirlenir. ROM'un veri girişi olmadığını unutmayın, çünkü yazma işlemi yoktur. Entegre



ŞEKİL 7.9
ROM blok diyagramı

devre ROM yongaları bir veya daha fazla etkinleştirme girişi içerir ve bazen büyük ROM dizileri nin inşasını kolaylaştırmak için üç durumlu çıkışlarla gelir.

Örneğin, bir 32×8 ROM'u düşünün. Birim, her biri 8 bit olan 32 kelimeden oluşur. Adres için 0'dan 31'e kadar olan ikili sayıları oluşturan beş giriş hattı vardır. Şekil 7.10, bu ROM'un iç mantık yapısını göstermektedir. Beş giriş, bir 5×32 kod çözücü aracılığıyla 32 farklı çıkışa ayrıştırılır. Kod çözücünün her çıkışı bir bellek adresini temsil eder. Kod çözücünün 32 çıkışı, sekiz OR kapısının her birine bağlanır. Diyagram, karmaşık devreler de kullanılan dizi mantık konvansiyonunu göstermektedir. (Bkz. Şekil 6.1.) Her OR kapısı, 32 girişi varmış gibi düşünülmelidir. Kod çözücünün her çıkışının, her OR kapısının girişlerinden birine bağlandığı görülmektedir. Her OR kapısının 32 giriş bağlantısı vardır ve 8 OR kapısı bulunduğundan, ROM toplamda $32 \times 8 = 256$ iç bağlantıya sahiptir. Genel olarak, bir $2^k \times n$ ROM, bir iç $k \times 2^k$ kod çözücü ve n OR kapısına sahip olacaktır. Her OR kapısının 2^k girişi vardır ve bunlar kod çözücünün her bir çıkışına bağlanmıştır.



ŞEKİL 7.10
 32×8 ROM'un iç mantığı

Şekil 7.10'daki 256 kesişim programlanabilir. İki hat arasındaki programlanabilir bağlantı, mantıksal olarak, iki hattın bağlı olduğu (kapalı) veya bağlı olmadığı (açık) bir anahtara dönüştürülebilen bir anahtara eşdeğerdir. İki hat arasında ki programlanabilir kesişim bazen *kesişim noktası* olarak adlandırılır. Kesişim anahtarlarını uygulamak için çeşitli fiziksel cihazlar kullanılır. En basit teknolojilerden biri, iki noktayı normalde bağlayan bir sigorta kullanır, ancak sigortaya yüksek voltajlı bir darbe uygulandığında açılır veya “patlar”.

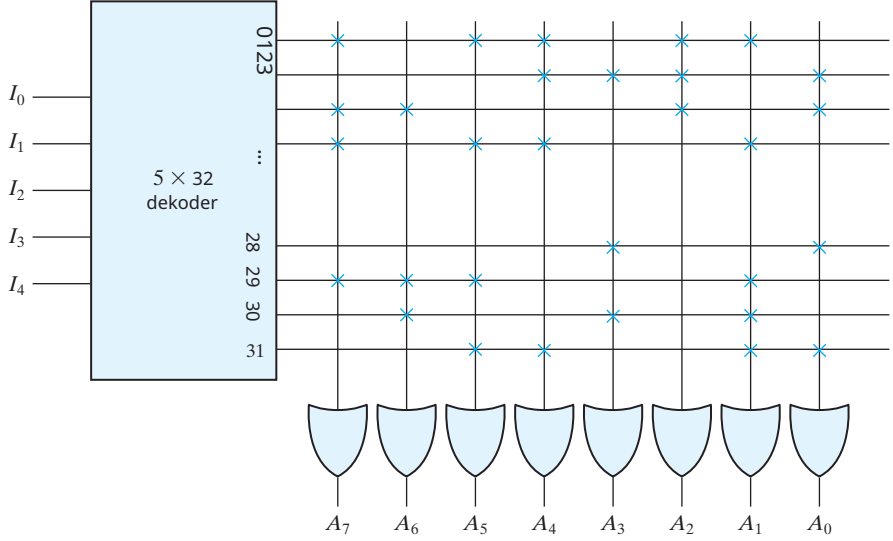
Bir ROM'un iç ikili depolaması, her adresteki kelime içeriğini gösteren bir doğruluk tablosu ile belirtilir. Örneğin, bir 32×8 ROM'un içeriği, Tablo 7.3'te gösterilen benzer bir doğruluk tablosu ile belirtilebilir. Doğruluk tablosu, altında tüm 32 adresin listelendiği beş girişi gösterir. Her adres, çıktı sütunlarında listelenen 8 bitlik bir kelime depolar. Tablo, ROM'daki yalnızca ilk dört ve son dört kelimeyi gösterir. Tam tablo, tüm 32 kelimenin listesini içermelidir.

ROM'u programlayan donanım prosedürü, verilen bir doğruluk tablosuna göre sigorta bağlantılarını patlatır. Örneğin, Tablo 7.3'te verilen doğruluk tablosuna göre ROM'un programlanması, Şekil 7.11'de gösterilen yapılandırmayı sonuçlandırır. Doğruluk tablosunda listelenen her 0, bir bağlantının yokluğunu belirtirken, listelenen her 1, bir bağlantı ile elde edilen bir yolu belirtir. Örneğin, tablo, adres 3'te kalıcı depolama için sekiz bitlik 10110010 kelimesini belirtir. Kelimedeki dört 0, kod çözücünün çıkış 3'ü ile çıkışlarıyla ilişkili OR kapılarının girişleri arasındaki sigorta bağlantılarını patlatarak programlanır. Kelimedeki dört 1, mantık diyagramlarında kalıcı bir bağlantı için kullanılan bir noktanın yerine geçici bir bağlantıyı belirtmek için bir * ile işaretlenmiştir. ROM'un girişi 00011 olduğunda, kod çözücünün tüm çıkışları 0'dır, çıkış 3 hariç, bu çıkış mantık 1'de bulunmak tadır. Kod çözücü çıkış 3'teki mantık 1'e eşdeğer sinyal, bağlantılar aracılığıyla A7, A5, A4 ve A1'in OR kapısı çıkışlarına yayılır. Diğer dört çıkış 0'da kalır. Sonuç olarak, depolanan 10110010 kelimesi sekiz veri çıkışına uygulanır.

Tablo 7.3

ROM Gerçeklik Tablosu (Kismi)

Girişler					Çıkışlar							
I_4	I_3	I_2	I_1	I_0	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	0	0	0	0	1	0	1	1	0	1	1	0
0	0	0	0	1	0	0	0	1	1	1	0	1
0	0	0	1	0	1	1	0	0	0	1	0	1
0	0	0	1	1	1	0	1	1	0	0	1	0
1	1	1	0	0	0	0	0	0	1	0	0	1
1	1	1	0	1	1	1	1	0	0	0	1	0
1	1	1	1	0	0	1	0	0	1	0	1	0
1	1	1	1	1	0	0	1	1	0	0	1	1



ŞEKİL 7.11

Tablo 7.3'e göre ROM'un programlanması

Kombinasyonel Devre Uygulaması

Bölüm 4.9'da, bir kod çözücünün 2^k girdi değişkenlerinin mintermlerini ürettiği gösterilmiştir. Boolean fonksiyonlarının mintermlerini toplamak için OR kapıları ekleyerek, istenen herhangi bir kombinasyonel devreyi üretebildik. ROM, esasen bir minterm üretici oluşturmak için hem kod çözücüyü hem de OR kapılarını tek bir cihaz içinde içeren bir cihazdır. Fonksiyona dahil olan mintermler için bağlantılar seçerek, ROM çıkışları kombinasyonel bir devredeki çıkış değişkenlerinin Boolean fonksiyonlarını temsil edecek şekilde programlanabilir.

ROM'un iç işleyişi iki şekilde yorumlanabilir. İlk yorum, sabit bir kelime deseni içeren bir bellek birimi olarakdır. İkinci yorum ise, kombinasyonel devreyi uygulayan bir birim olarakdır. Bu açıdan bakıldığında, her çıkış terminali, mintermlerin toplamı olarak ifade edilen bir Boolean fonksiyonunun çıktısı olarak ayrı ayrı değerlendirilir. Örneğin, Şekil 7.11'deki ROM, beş giriş değişkeninin her birinin bir fonksiyonu olan sekiz çıkışa sahip bir kombinasyonel devre olarak düşünülebilir. Çıkış A_7 mintermlerin toplamı olarak şu şekilde ifade edilebilir:

$$A_7(I_4, I_3, I_2, I_1, I_0) = \Sigma(0, 2, 3, \dots, 29)$$

(Üç nokta, şekil içinde belirtilmemiş olan 4 ile 27 arasındaki mintermleri temsil eder.) Şekildeki $\times$ ile işaretlenmiş bir bağlantı, toplam için bir minterm üretir. Diğer tüm kesişim noktaları bağlı değildir ve toplamda yer almaz.

Pratikte, bir kombinasyonel devre bir ROM aracılığıyla tasarlandığında, mantığı tasarlamak veya birimin içindeki kapı bağlantılarını göstermek gerekli değildir. Tasarımcının yapması gereken tek şey, belirli ROM'u IC numarasıyla belirtmek ve geçerli doğruluk tablosunu sağlamaktır. Doğruluk tablosu, ROM'u programlamak için gerekli tüm bilgileri verir. Doğruluk tablosuna eşlik edecek bir iç mantık diyagramına ihtiyaç yoktur.

ÖRNEK 7.1

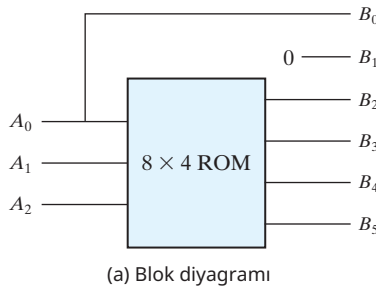
Bir ROM kullanarak kombinasyonel bir devre tasarlayın. Devre, üç bitlik bir sayıyı alır ve girdi sayısının karesine eşit bir ikili sayı üretir.

İlk adım, kombinasyonel devrenin doğruluk tablosunu çıkarmaktır. Çoğu durumda, bu yeterlidir. Diğer durumlarda, çıkış değişkenlerindeki belirli özellikleri kullanarak ROM için kısmi bir doğruluk tablosu kullanabiliriz. Tablo 7.4, kombinasyonel devre için doğruluk tablosudur. Tüm olası ikili sayıları karşılamak için üç girdi ve altı çıkış gereklidir. Çıkış B_0 'nın her zaman girdi A_0 'ya eşit olduğunu not ediyoruz, bu nedenle B_0 'ı bir ROM ile üretmeye gerek yoktur, çünkü bu bir girdi değişkenine eşittir. Ayrıca, çıkış B_1 her zaman 0'dır, bu nedenle bu çıkış bilinen bir sabittir. Aslında, ROM ile yalnızca dört çıkış üretmemiz gerekiyor; diğer ikisi kolayca elde edilir. Gerekli minimum ROM boyutu üç girdi ve dört çıkışa sahip olmalıdır. Üç girdi sekiz kelimeyi belirtir, bu nedenle ROM boyutu 8×4 olmalıdır. ROM uygulaması Şekil 7.12'de gösterilmiştir. Üç girdi, her biri dört bit olan sekiz kelimeyi belirtir. Şekil 7.12'deki doğruluk tablosu (b), ROM'un programlanması için gereken bilgileri belirtir. Şekil 7.12'deki blok diyagramı (a), kombinasyonel devrenin gerekli bağlantılarını göstermektedir. Şekil 7.12 (a), kombinasyonel devrenin gerekli bağlantılarını göstermektedir.

Tablo 7.4

Örnek 7.1'in Devresi için Doğruluk Tablosu

Girişler			Çıkışlar						Ondalık
A_2	A_1	A_0	B_5	B_4	B_3	B_2	B_1	B_0	
0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1	1
0	1	0	0	0	0	1	0	0	4
0	1	1	0	0	1	0	0	1	9
1	0	0	0	1	0	0	0	0	16
1	0	1	0	1	1	0	0	1	25
1	1	0	1	0	0	1	0	0	36
1	1	1	1	1	0	0	0	1	49



A_2	A_1	A_0	B_5	B_4	B_3	B_2
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	1
0	1	1	0	0	1	0
1	0	0	0	1	0	0
1	0	1	0	1	1	0
1	1	0	1	0	0	1
1	1	1	1	1	0	0

(b) ROM gerçeklik tablosu

ŞEKİL 7.12

Örnek 7.1'in ROM uygulaması

ROM Türleri

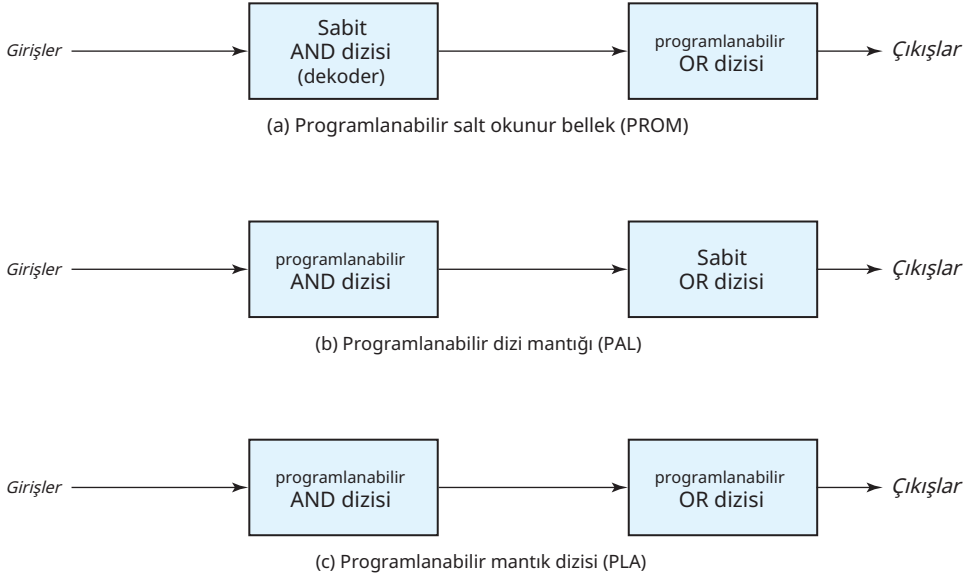
ROM'daki gerekli yollar dört farklı şekilde programlanabilir. İlk şekil *maske programlama* olarak adlandırılır ve yarı iletken şirketi tarafından birimin son üretim sürecinde gerçekleştirilir. Bir ROM'un üretilmesi için müşterinin ROM'un karşılamasını istediği doğru tabloyu doldurması gerekir. Doğru tablo, üretici tarafından sağlanan özel bir formda veya belirli bir formatta bir bilgisayar çıktı ortamında sunulabilir. Üretici, müşterinin doğru tablosuna göre 1'leri ve 0'ları üretmek için yollar için ilgili maskeyi yapar. Bu prosedür maliyetlidir çünkü satıcı, belirli bir ROM'un özel maskelemesi için müşteriden özel bir ücret talep eder. Bu nedenle, maske programlama yalnızca aynı ROM konfigürasyonundan büyük bir miktar sipariş edilecekse ekonomik olarak değerlidir.

Küçük miktarlar için, *pro- olarak adlandırılan ikinci bir ROM türünü kullanmak daha ekonomiktir. programlanabilir salt okunur bellek, veya PROM*. Sipariş verildiğinde, PROM birimleri tüm füzyonlar sağlamdır ve depolanan kelimelerin bitlerinde tüm 1'leri verir. PROM'daki füzyonlar, cihaza özel bir pin aracılığıyla yüksek voltajlı bir darbe uygulanarak patlatılır. Patlayan bir sigorta, ikili 0 durumunu tanımlar ve sağlam bir sigorta, ikili 1 durumunu verir. Bu prosedür, kullanıcının laboratuvar ortamında PROM'u programlamasına ve giriş adresleri ile saklanan kelimeler arasındaki istenen ilişkiyi elde etmesine olanak tanır. Prosedürü kolaylaştırmak için ticari olarak mevcut olan PROM programlayıcıları adı verilen özel aletler bulunmaktadır. Her durumda, ROM'ları programlama prosedürleri donanım prosedürleridir, programlama kelimesi kullanılabilebilir.

ROM'ları veya PROM'ları programlama donanım prosedürü geri dönüşümsüzdür ve bir kez programlandığında, sabit desen kalıcıdır ve değiştirilemez. Bir bit deseni belirlendikten sonra, bit deseni değiştirilmek isteniyorsa birim atılmalıdır. Üçüncü tür ROM, silinebilir PROM (EPROM) olup, daha önce programlanmış olsa bile başlangıç durumuna yeniden yapılandırılabilir. EPROM, belirli bir süre boyunca özel bir ultraviyole ışık altında tutulduğunda, programlanmış bağlantılar olarak hizmet eden iç yüzey kapıları boşaltan kısa dalga radyasyonu iletimi sağlar. Silindikten sonra, EPROM başlangıç durumuna döner ve yeni bir değer setine yeniden programlanabilir.

Dördüncü tür ROM, elektrikle silinebilir PROM (EEPROM veya E^2PROM) dir. Bu cihaz EPROM'a benzer, ancak daha önce programlanmış bağlantılar, ultraviyole ışık yerine elektrik sinyali ile silinebilir. Avantajı, cihazın soketinden çıkartılmadan silinebilmesidir.

Flash bellek cihazları EEPROM'lara benzer, ancak özel bir programlayıcıya ihtiyaç duymadan devre içinde cihazı seçici olarak programlamak ve silmek için ek yerleşik devreler içerir. Modern teknolojiye cep telefonları, dijital kameralar, set üstü kutular, dijital TV, telekomünikasyon, kalıcı veri depolama ve mikrodenetleyiciler gibi geniş bir uygulama alanına sahiptirler. Düşük güç tüketimleri, onları dizüstü ve notebook bilgisayarlar için cazip bir depolama ortamı haline getirir. Flash bellekler, örneğin 16 ile 64 K bayt boyutunda bellek blokları eşzamanlı olarak silinmesine olanak tanıyan ek devreler de içerir. EEPROM'lar gibi, flash bellekler de yorgunluğa maruz kalır ve genellikle yaklaşık 10^5 blok silme döngüsüne sahiptir.



ŞEKİL 7.13

Üç PLD'nin temel yapılandırması

Kombinasyonel PLD'ler

PROM, programlanabilir mantık cihazı (PLD) olan kombinasyonel bir entegre devredir; programlanabilir kapılar, bir AND dizisi ve bir OR dizisine bölünerek AND-OR çarpım toplamı uygulaması sağlar. AND-OR dizisindeki programlanabilir bağlantıların yerleştirilmesine göre üç ana kombinasyonel PLD türü vardır. Şekil 7.13, üç PLD'nin yapılandırmasını göstermektedir. PROM, bir dekode olarak inşa edilmiş sabit bir AND dizisine ve programlanabilir bir OR dizisine sahiptir. Programlanabilir OR kapıları, Boolean fonksiyonlarını çarpım toplamı biçiminde uygular. PAL, programlanabilir bir AND dizisi ve sabit bir OR dizisine sahiptir. AND kapıları, Boolean fonksiyonları için çarpım terimlerini sağlamak üzere programlanır ve her OR kapısında mantıksal olarak toplanır. En esnek PLD, hem AND hem de OR dizilerinin programlanabileceği PLA'dır. AND dizisindeki çarpım terimleri, gerekli çarpım-toplamı uygulamasını sağlamak için herhangi bir OR kapısı tarafından paylaşılabilir. PAL ve PLA isimleri, PLD'lerin geliştirilmesi sırasında farklı satıcılardan ortaya çıkmıştır. Bu bölümde PROM ile kombinasyonel devrelerin uygulanması gösterilmiştir. Kombinasyonel devrelerin PLA ve PAL ile tasarımı, sonraki iki bölümde sunulmaktadır.

7.6 PROGRAMLANABİLİR LOJİK DİZİ

PLA, PROM ile kavramsal olarak benzerdir, ancak PLA değişkenlerin tam dekodunu sağlamaz ve tüm mintermleri üretmez. Dekoder, herhangi bir ürün üretmek için programlanabilen AND kapıları dizisi ile değiştirilir.

giriş değişkenlerinin terimi. Ürün terimleri daha sonra gerekli Boolean fonksiyonları için ürünlerin toplamını sağlamak üzere OR kapılarına bağlanır.

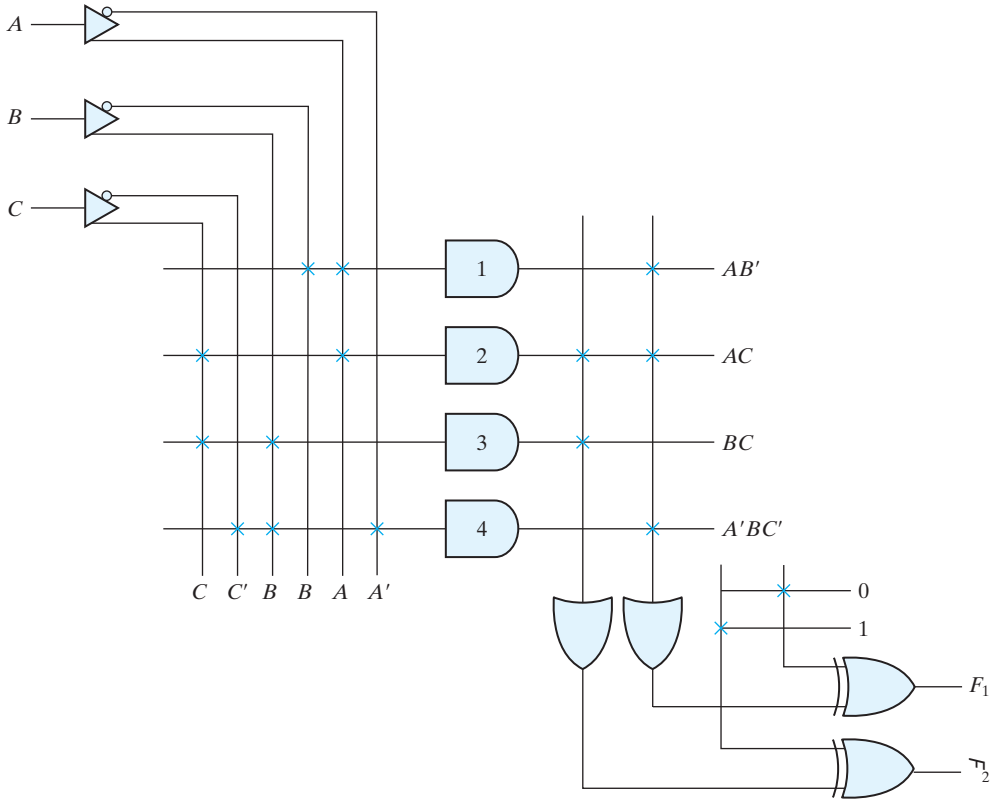
Üç girişi ve iki çıkışı olan bir PLA'nın iç mantığı Şekil 7.14'te gösterilmiştir.

Bu tür bir devre ticari olarak faydalı olacak kadar küçük değildir, ancak burada bir PLA'nın tipik mantık yapılandırmasını göstermek için sunulmuştur. Diyagram, karmaşık devreler için dizi mantık grafik sembollerini kullanmaktadır. Her giriş, diyagramda hem doğru hem de tamamlayıcı çıkışlara sahip bir bileşik grafik sembolü ile gösterilen bir tampon-invertör kombinasyonundan geçer.

Her giriş ve onun tamamlayıcısı, dikey ve yatay çizgiler arasındaki kesişimlerle gösterildiği gibi, her AND kapısının girişlerine bağlanır. AND kapılarının çıkışları, her OR kapısının girişlerine bağlanır. OR kapısının çıkışı bir XOR kapısına gider; burada diğer giriş, mantık 1 veya mantık 0'a eşit bir sinyal almak üzere programlanabilir. XOR girişi 1'e bağlandığında çıkış ters çevrilir (çünkü $x \oplus 1 = x'$). XOR girişi 0'a bağlandığında çıkış değişmez (çünkü $x \oplus 0 = x$). Şekil 7.14'teki PLA'da uygulanan belirli Boolean fonksiyonları şunlardır:

$$F_1 = AB' + AC + A'BC'$$

$$F_2 = (AC + BC)'$$



ŞEKİL 7.14

Üç girişi, dört ürün terimi ve iki çıkışı olan PLA

Her AND kapısında üretilen ürün terimleri, diyagramda kapının çıkışı boyunca listelenmiştir. Ürün terimi, kesişim noktaları bağlı ve bir $\times$ ile işaretlenmiş olan girişlerden belirlenir. Bir OR kapısının çıkışı, seçilen ürün terimlerinin mantıksal toplamını verir. Çıkış, gerçekleştirilen mantığa bağlı olarak tamamlanabilir veya gerçek formunda bırakılabilir.

Bir PLA'nın sigorta haritası tablo biçiminde belirtilebilir. Örneğin, Şekil 7.14'ün PL A'sını belirten programlama tablosu Tablo 7.5'te listelenmiştir. PLA programlama tablosu üç bölümden oluşur. İlk bölüm, ürün terimlerini sayısal olarak listeler. İkinci bölüm, girişler ile AND kapıları arasındaki gerekli yolları belirtir. Üçüncü bölüm, AND ve OR kapıları arasındaki yolları belirtir. Her çıkış değişkeni için, XOR kapısını programlamak üzere T (doğru için) veya C (tamamlayıcı için) olabilir. Solda listelenen ürün terimleri tablonun bir parçası değildir; yalnızca referans için dahil edilmiştir. Her ürün terimi için, girişler 1, 0 veya — (tire) ile işaretlenir. Ürün teriminde bir değişken, doğru olduğu formda görünüyorsa, karşılık gelen giriş değişkeni 1 ile işaretlenir. Tamamlanmış görünüyorsa, karşılık gelen giriş değişkeni 0 ile işaretlenir. Değişken ürün teriminde yoksa, bir tire ile işaretlenir.

Girişler ile AND kapıları arasındaki yollar, programlama tablosundaki "Girişler" sütun başlığı altında belirtilmiştir. Giriş sütununda 1, giriş değişkeninden AND kapısına bir bağlantıyı belirtir. Giriş sütununda 0, değişkenin tamamlayıcısından AND kapısının girişine bir bağlantıyı belirtir. Bir tire, hem giriş değişkeninde hem de onun tamamlayıcısında patlamış bir sigortayı belirtir. AND kapısının girişindeki açık bir terminalin 1 gibi davrandığı varsayılmaktadır.

AND ve OR kapıları arasındaki yollar, "Çıkışlar" sütun başlığı altında belirtilmiştir. Çıkış değişkenleri, fonksiyona dahil olan ürün terimleri için 1 ile işaretlenmiştir. Çıkış sütununda 1 olan her ürün terimi, AND kapısının çıkışından OR kapısının girişine bir yol gerektirir. Tire ile işaretlenenler, patlamış bir sigortayı belirtir. OR kapısının girişindeki açık bir terminalin 0 gibi davrandığı varsayılmaktadır. Son olarak, bir T (doğru) çıkışı, ilgili XOR kapısının diğer girişinin 0'a bağlanmasını gerektirir ve bir C (tamamlayıcı) 1'e bir bağlantıyı belirtir.

Tablo 7.5
PLA Programlama Tablosu

		Girişler			Çıkışlar	
		A	B	C	(T)	(C)
Ürün Terimi					F ₁	F ₂
AB'	1	1	0	—	1	—
AC	2	1	—	1	1	1
BC	3	—	1	1	—	1
A BC	4	0	1	0	1	—

Not: Tirelerin anlamları için metne bakın.

PLA'nın boyutu, giriş sayısı, ürün terimleri sayısı ve çıkış sayısı ile belirtilir. Tipik bir entegre devre PLA'nın 16 girişi, 48 ürün terimi ve sekiz çıkışı olabilir. n girişi, k ürün terimi ve m çıkışı için, PLA'nın iç mantığı n tampon-invertör kapıları, k VE kapıları, m VEYA kapıları ve m XOR kapıları içerir. Girişler ile VE dizisi arasında $2n \times k$ bağlantı vardır, $k \times m$ AND ve OR dizileri arasındaki bağlantılar ve m bağlantılarla ilişkili XOR kapıları.

Dijital bir sistemi bir PLA ile tasarlarırken, birim içindeki bağlantıları gösterme ihtiyacı yoktur; bu, Şekil 7.14'te olduğu gibi. Gerekli mantığı sağlamak için PLA'nın programlanabileceği bir PLA programlama tablosu yeterlidir. Bir ROM'da olduğu gibi, PLA maske programlanabilir veya alan programlanabilir olabilir. Maske programlamada, müşteri bir PLA programlama tablosunu üreticiye sunar. Bu tablo, satıcı tarafından müşteri tarafından belirtilen gerekli iç mantığına sahip özel bir PLA üretmek için kullanılır. Mevcut olan ikinci tür PLA, kullanıcı tarafından ticari bir donanım programlayıcı birimi aracılığıyla programlanabilen alan programlanabilir mantık dizisidir (FPLA).

Bir PLA ile kombinasyonel bir devre uygularken, AND kapılarının sonlu sayıda olduğunu göz önünde bulundurarak, farklı ürün terimlerinin sayısını azaltmak için dikkatli bir inceleme yapılmalıdır. Bu, her Boolean fonksiyonunu minimum terim sayısına sadeleştirerek yapılabilir. Bir terimdeki literal sayısı önemli değildir, çünkü tüm giriş değişkenleri zaten mevcuttur. Her fonksiyonun hem gerçek değeri hem de tamamlayıcı, daha az ürün terimi ile ifade edilebilecek olanı ve diğer fonksiyonlarla ortak ürün terimleri sağlayan görmek için sadeleştirilmelidir.

ÖRNEK 7.2

Aşağıdaki iki Boolean fonksiyonunu bir PLA ile uygulayın:

$$F_1(A, B, C) = \sum(0, 1, 2, 4)$$

$$F_2(A, B, C) = \sum(0, 5, 6, 7)$$

İki fonksiyon, Şekil 7.15'teki haritalarda sadeleştirilmiştir. Fonksiyonların hem gerçek değeri hem de tamamlayıcı, toplam-çarpım formuna sadeleştirilmiştir. Minimum ürün terimi sayısını veren kombinasyon şudur:

$$F_1 = (AB + AC + BC)$$

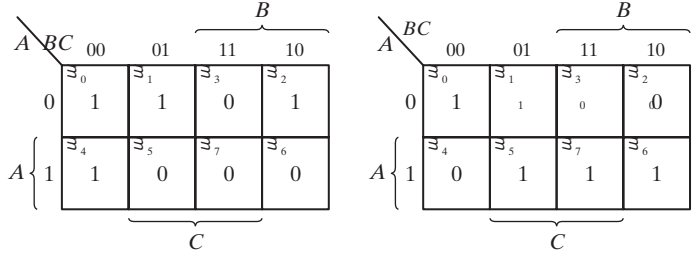
ve

$$F_2 = AB + AC + A \bar{B} C$$

Bu kombinasyon dört farklı ürün terimi verir: AB, AC, BC ve $A \bar{B} C$. Kombinasyon için PLA programlama tablosu Şekil üzerinde gösterilmiştir. Çıktı F_1 , tabloda üzerinde C işareti olmasına rağmen gerçek çıktıdır. Bunun nedeni, F_1 'in bir AND-OR devresi ile üretilmesi ve OR kapısının çıkışında mevcut olmasıdır. XOR kapısı, gerçek F_1 çıktısını üretmek için fonksiyonu tamamlar.

PLA programlama tablosu						
Ürün terimi		Girişler			Çıkışlar	
		A	B	C	(C)	(T)
					Σ_1	Σ_2
AB	1	1	1	-	11	1
AC	2	1	-	1	11	1
BC	3	-	1	1	1-	-
A B C'	4	0	0	0	-	1

ŞEKİL 7.15
Örnek 7.2'nin Çözümü



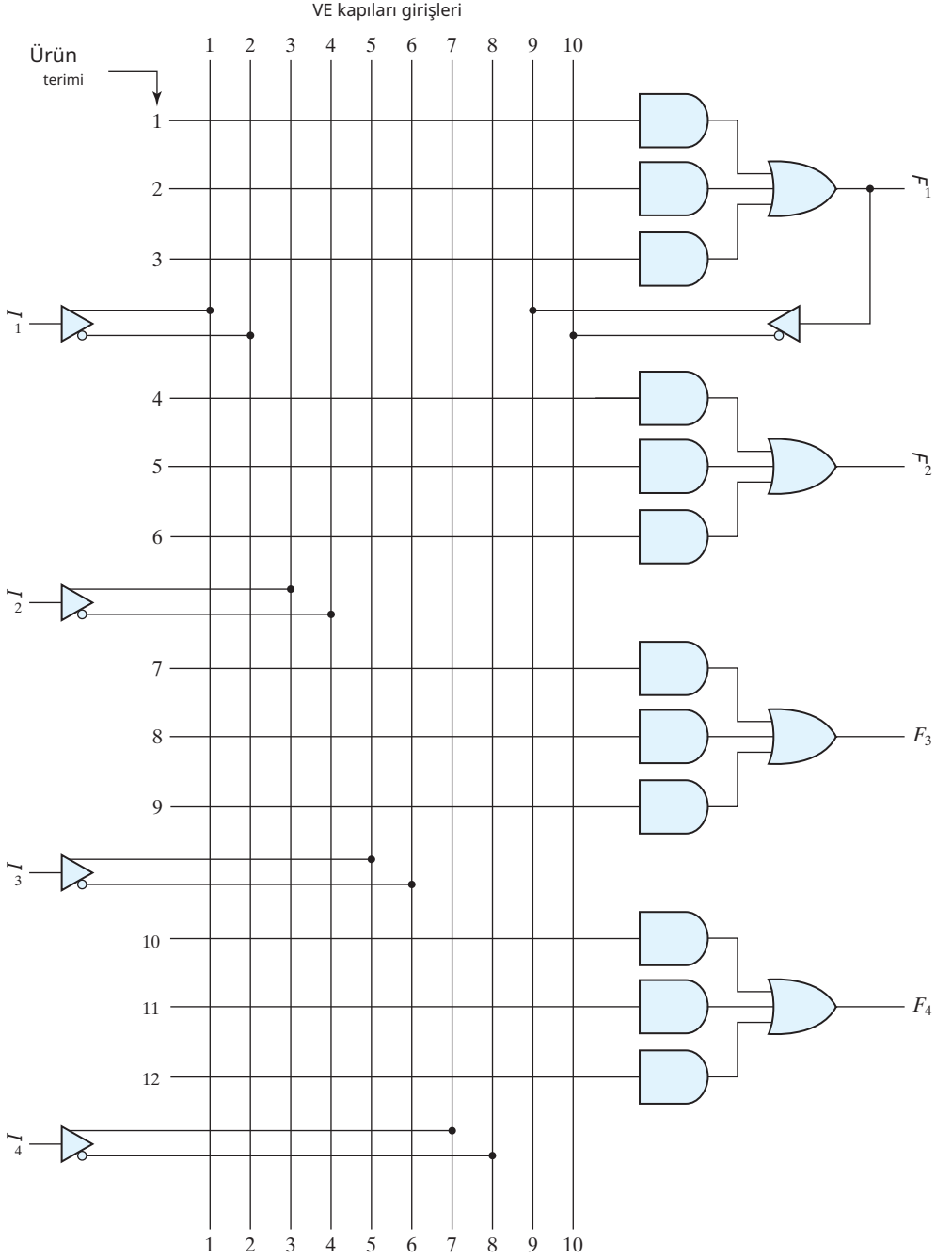
Örnek 7.2'de kullanılan kombinyonel devre, bir PLA ile uygulamak için çok basit tir. Sadece gösterim amacıyla sunulmuştur. Tipik bir PLA'nın çok sayıda girişi ve ürün terimi vardır. Bu kadar çok değişkenle Boolean fonksiyonlarının basitleştirilmesi, bilgisayar destekli basitleştirme prosedürleri aracılığıyla gerçekleştirilmelidir. Bilgisayar destekli tasarım (CAD) programı, her fonksiyonu ve tamamlayıcısını minimum terim sayısına indirger. Program daha sonra, tüm fonksiyonları doğru olduğu veya tamamlayıcı formda olduğu şekliyle kapsayan minimum sayıda ürün terimini seçer. PLA programlama tablosu daha sonra oluşturulur ve gerekli sigorta haritası elde edilir. Sigorta haritası, entegre devredeki iç sigortaları patlatma donanım prosedüründe n geçen bir FPLA programlayıcısına uygulanır.

7.7 PROGRAMLANABİLİR DİZİ MANTIK

PAL, sabit bir VEYA dizisi ve programlanabilir bir VE dizisi ile bir programlanabilir mantık cihazıdır. Sadece VE kapıları programlanabilir olduğundan, PAL programlaması PLA'dan daha kolaydır, ancak PLA kadar esnek değildir. Şekil 7.16, dört girişi ve dört çıkışı olan tipik bir PAL'ın mantık konfigürasyonunu göstermektedir. Her girişin bir tampon-invertör kapısı vardır ve her çıkış sabit bir VEYA kapısı tarafından üretilir. Birimde dört bölüm vardır, her biri üç genişliğinde bir VE-VEYA dizisinden oluşur; bu, her bölümde üç programlanabilir VE kapısı ve bir sabit VEYA kapısı olduğu anlamına gelir. Her VE kapısının, diyagramda her yatay çizgiyi kesen 10 dikey çizgi ile gösterilen 10 programlanabilir giriş bağlantısı vardır. Yatay çizgi, VE kapısının çoklu giriş konfigürasyonunu simgeler. Çıkışlardan biri bir tampon-invertör kapısına bağlanır ve ardından VE kapılarının iki girişine geri beslenir.

Ticari PAL cihazları, Şekil 7.16'da gösterilenden daha fazla kapı içerir. Tipik bir PAL entegre devresi sekiz girdi, sekiz çıkış ve her biri sekiz geniş AND-OR dizisinden oluşan sekiz bölüm içerebilir. Çıkış terminalleri bazen üç durumlu tamponlar veya inverterler tarafından sürdürülmektedir.

PAL ile tasarım yaparken, Boolean fonksiyonları her bir bölüme sığacak şekilde basitleştirilmelidir. PLA ile olan durumun aksine, bir ürün terimi iki veya daha fazla OR kapısı arasında paylaşılamaz. Bu nedenle, her fonksiyon kendi başına, dikkate alınmaksızın basitleştirilebilir.



ŞEKİL 7.16

Dört girişi, dört çıkışı ve üç genişliğinde AND-OR yapısına sahip PAL

ortak ürün terimlerine. Her bölümdeki ürün terimlerinin sayısı sabittir ve eğer fonsiyondaki terim sayısı çok büyükse, bir Boolean fonksiyonunu uygulamak için iki bölüm kullanmak gerekebilir.

Bir kombinasyon devresinin tasarımında bir PAL kullanmanın bir örneği olarak, aşağıdaki Boolean fonksiyonlarını göz önünde bulundurun, toplam-mintermler formunda verilmiştir:

$$w(A, B, C, D) = \sum(2, 12, 13)$$

$$x(A, B, C, D) = \sum(7, 8, 9, 10, 11, 12, 13, 14, 15)$$

$$y(A, B, C, D) = \sum(0, 2, 3, 4, 5, 6, 7, 8, 10, 11, 15)$$

$$z(A, B, C, D) = \sum(1, 2, 8, 12, 13)$$

Dört fonksiyonu minimum terim sayısına indirmek, aşağıdaki

Boolean fonksiyonlarını verir:

$$w = ABC + A'B'CD'$$

$$x = A + BCD$$

$$y = A'B + CD + B'D'$$

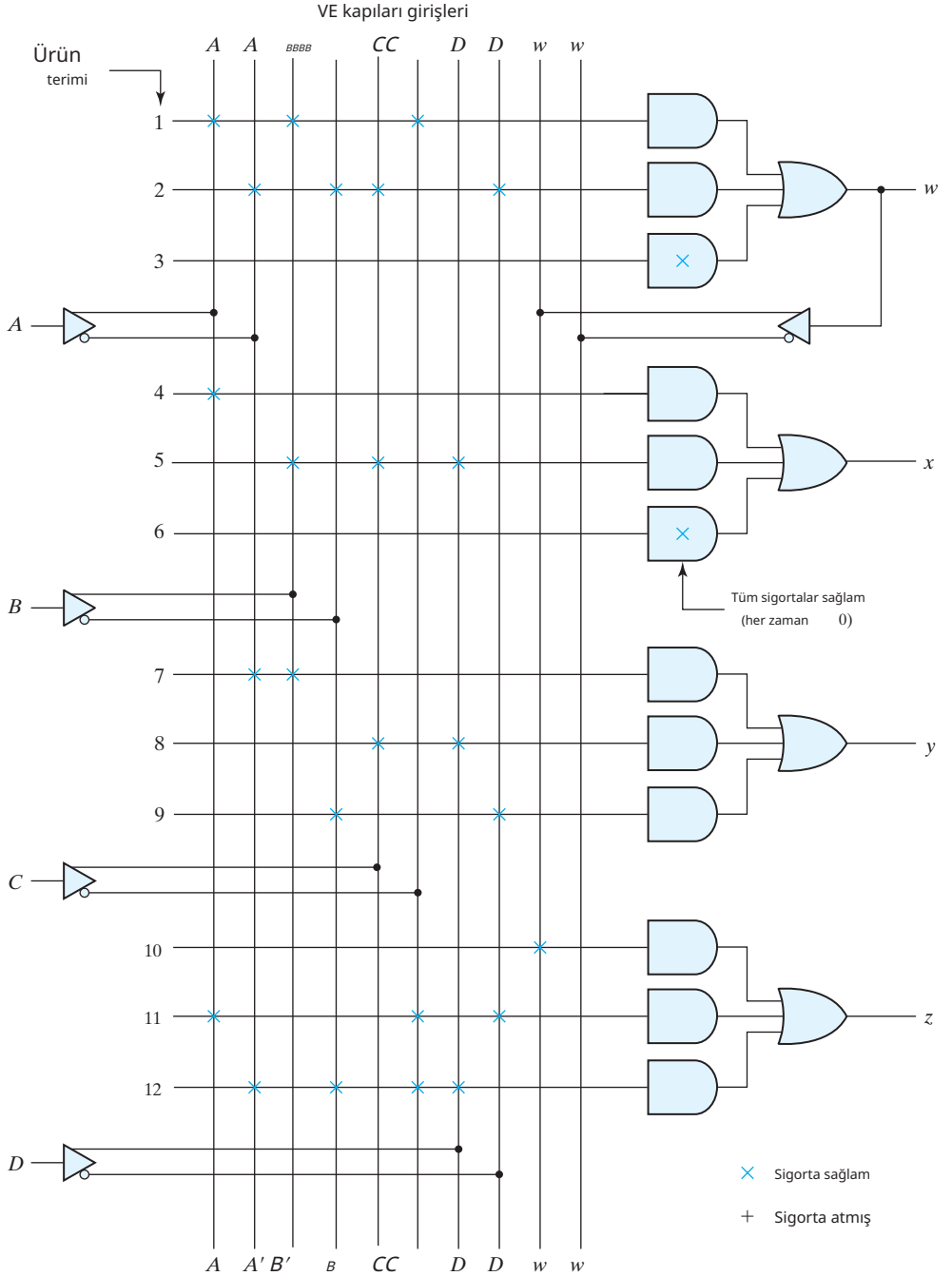
$$\begin{aligned} z &= ABC' + A'B'CD' + AC'D' + A'B'C'D \\ &= w + AC'D' + A'B'C'D \end{aligned}$$

z fonksiyonunun dört çarpan terimi olduğunu unutmayın. Bu terimlerin ikisinin mantıksal toplamı w ile eşittir. w kullanarak, z için terim sayısını dörtten üçe düşürmek mümkündür.

PAL programlama tablosu, PLA için kullanılanla benzerdir, ancak sadece AND kapılarının girişlerinin programlanması gerekir. Tablo 7.6 PAL'ı listeler.

Tablo 7.6
PAL Programlama Tablosu

Ürün Terimi	AND Girişleri					Çıkışlar
	A	B	C	D	w	
1	1	1	0	—	—	$w = ABC' + A'B'CD'$
2	0	0	1	0	—	
3	—	—	—	—	—	
4	1	—	—	—	—	$x = A + BCD$
5	—	1	1	1	—	
6	—	—	—	—	—	
7	0	1	—	—	—	$y = A'B + CD + B'D'$
8	—	—	1	1	—	
9	—	0	—	0	—	
10	—	—	—	—	1	$z = w + AC'D' + A'B'C'D$
11	1	—	0	0	—	
12	0	0	0	1	—	



ŞEKİL 7.17

PAL için sigorta haritası, Tablo 7.6'da belirtildiği gibi

lean fonksiyonu için programlama tablosu. Tablo, her birinde üç ürün terimi bulunan dört bölüme ayrılmıştır, Fig. 7.16'daki PAL ile uyumlu olacak şekilde. İlk iki bölüm, Boolean fonksiyonunu uygulamak için yalnızca iki ürün terimine ihtiyaç duyar. Son bölüm, çıkış z , dört ürün terimine ihtiyaç duyar. w , çıkışını kullanarak, fonksiyonu üç terime indirebiliriz.

Programlama tablosunda belirtildiği gibi PAL için sigorta haritası, Şekil 7.17'de gösterilmiştir. Tablodaki her 1 veya 0 için, diyagramdaki karşılık gelen kesişimi sağlam sigorta sembolü ile işaretleriz. Her tire için, diyagramı hem doğru hem de tamamlayıcı girişlerde atmış sigortalar ile işaretleriz. AND kapısı kullanılmıyorsa, tüm giriş sigortalarını sağlam bırakırız. Karşılık gelen giriş, her giriş değişkeninin hem doğru değerini hem de tamamlayıcısını aldığından, $AA=0$ ve AND kapısının çıkışı her zaman 0'dır.

Tüm PLD'lerde olduğu gibi, PAL'lerle tasarım, CAD teknikleri kullanılarak kolaylaştırılır. İç fuse'lerin atılması, özel elektronik aletlerin yardımıyla gerçekleştirilen bir donanım prosedürüdür.

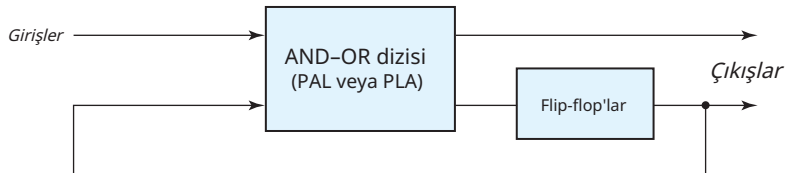
7.8 SIRALI PROGRAMLANABİLİR CİHAZLAR

Dijital sistemler, flip-flop'lar ve kapılarla tasarlanır. Kombinasyonel PLD yalnızca kapıardan oluştuğundan, tasarımda kullanıldıklarında harici flip-flop'ların dahil edilmesi gereklidir. Sıralı programlanabilir cihazlar hem kapıları hem de flip-flop'ları içerir. Bu şekilde, cihaz çeşitli sıralı devre işlevlerini yerine getirecek şekilde programlanabilir. Piyasada mevcut olan birkaç tür sıralı programlanabilir cihaz bulunmaktadır ve her cihazın her tür içinde satıcıya özgü varyantları vardır. Bu cihazların iç mantığı burada gösterilemeyecek kadar karmaşıktır. Bu nedenle, detaylı yapısına girmeden üç ana türü tanımlayacağız:

1. Sıralı (veya basit) programlanabilir mantık cihazı (SPLD)
2. Karmaşık programlanabilir mantık cihazı (CPLD)
3. Alan programlanabilir kapı dizisi (FPGA)

Sıralı PLD bazen karmaşık PLD'den ayırt etmek için basit PLD olarak adlandırılır. SPLD, entegre devre çipinin içinde AND-OR dizisine ek olarak flip-flop'ları içerir. Sonuç, Şekil 7.18'de gösterilen bir sıralı devredir.

Bir PAL veya PLA, bir kayıt oluşturmak için bağlı bir dizi flip-flop eklenerek değiştirilir. Devre çıkışları, OR kapılarından veya çıkışlardan alınabilir.



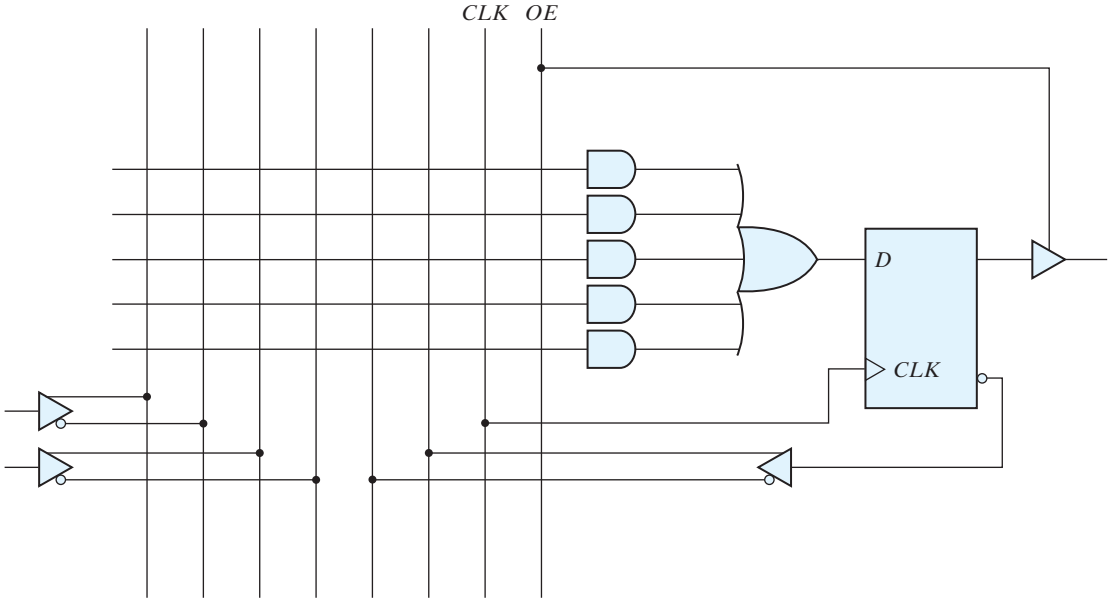
ŞEKİL 7.18
Sıralı programlanabilir mantık cihazı

flip-flop'lar. AND dizisi ile oluşturulan ürün terimlerine flip-flop çıkışlarını dahil etmek için ek programlanabilir bağlantılar mevcuttur. Flip-flop'lar D veya JK tipi olabilir.

Sıralı devre uygulamasını desteklemek için geliştirilen ilk programlanabilir cihaz, sah a programlanabilir mantık sıralayıcısıdır (FPLS). Tipik bir FPLS, birkaç çıkış flip-flop'ları s ürmekte olan bir PLA etrafında organize edilmiştir. Flip-flop'lar, JK veya D tipi olarak çalışacak şekilde programlanabilir olmaları açısından esnekler. FPLS, çok fazla programlanabilir bağlantıya sahip olduğu için ticari olarak başarılı olamamıştır. SPLD'de en çok k ullanılan yapılandırma, D flip-flop'ları ile birlikte kombinasyonel PAL'dır. Flip-flop'ları iç eren bir PAL, AND-OR dizisine ek olarak flip-flop'lar içerdiğini belirtmek için kayıtlı PAL olarak adlandırılır. Bir SPLD'nin her bölümü makro hücre olarak adlandırılır; bu, bir to plam-ürün kombinasyonel mantık fonksiyonu ve isteğe bağlı bir flip-flop içeren bir devr edir. AND-OR toplam-ürün fonksiyonu varsayacağız, ancak pratikte, bu, Bölüm 3.7'de s unulan iki seviyeli uygulamalardan herhangi biri olabilir.

Şekil 7.19, temel bir makro hücrenin mantığını göstermektedir. AND-OR dizisi, Şekil 7.16'daki kombinasyonel PAL ile aynıdır. Çıkış, ortak bir saat girişine bağlı bir kenar tetiklemeli D flip-flop tarafından sürülmemekte ve saat kenarında durum değiştirmektedir.

Flip-flop'un çıkışı, diyagramda OE olarak işaretlenmiş bir çıkış etkinleştirme sinyali tarafından kontrol edilen üç durumlu bir tampon (veya inverter) ile bağlantılıdır. Flip-flop'un çıkışı, ardışık devre için mevcut durum koşulunu sağlamak amacıyla programlanabilir AND kapılarından birinin girişine geri beslenmektedir. Tipik bir SPLD, bir IC paketinde 8 ila 10 makro hücreye sahiptir.



ŞEKİL 7.19

Temel makro hücre mantığı

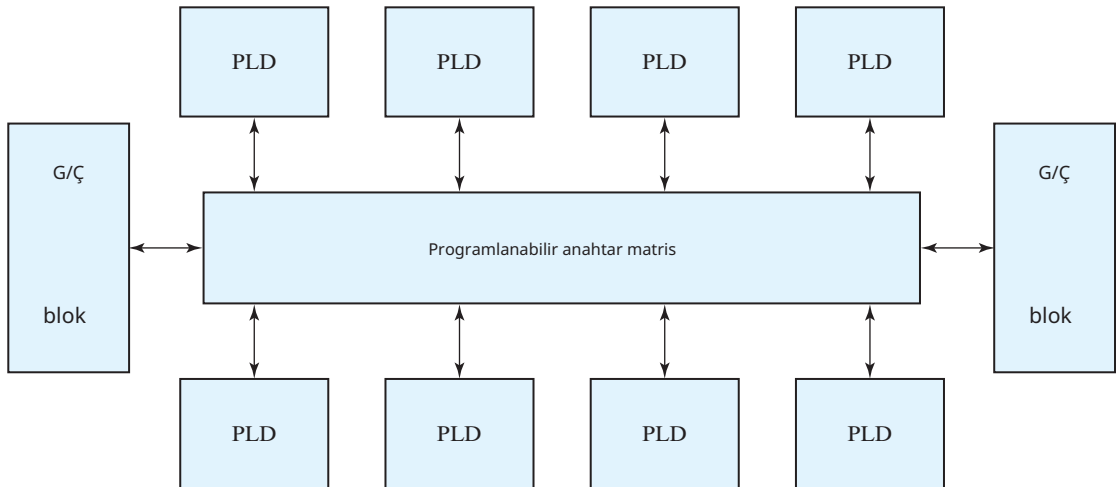
ketinde. Tüm flip-flop'lar ortak CLK girişine bağlıdır ve tüm üç durumlu tamponlar OE girişine göre kontrol edilmektedir.

AND dizisini programlamanın yanı sıra, bir makro hücre başka programlama özelliklerine de sahip olabilir. Tipik programlama seçenekleri, flip-flop'u kullanma veya atlama yeteneği, saat kenarı polaritesinin seçimi, kayıt için ön ayar ve temizleme seçimi ve bir çıkışın gerçek değeri veya tamamlayıcısının seçimini içerir. Gerçek/tamamlayıcı durumu programlamak için bir XOR kapısı kullanılır. Çoklayıcılar, seçim girişlerini programlayarak iki veya dört farklı yol arasında seçim yapar.

Dijital bir sistemin PLD'ler kullanılarak tasarımı genellikle tam spesifikasyonu üretmek için birkaç cihazın bağlantısını gerektirir. Bu tür bir uygulama için, tek bir entegre devrede bir araya getirilmiş bireysel PLD'lerin bir koleksiyonu olan karmaşık programlanabilir mantık cihazı (CPLD) kullanmak daha ekonomiktir. Programlanabilir bir bağlantı yapısı, PLD'lerin bireysel PLD'lerle yapılabileceği şekilde birbirine bağlanmasına olanak tanır.

Şekil 7.20, bir CPLD'nin genel yapılandırmasını göstermektedir. Cihaz, programlanabilir bir anahtar matris aracılığıyla birbirine bağlı birden fazla PLD'den oluşmaktadır. Giriş-çıkış (I/O) blokları, IC pinlerine bağlantıları sağlar. Her I/O pini, üç durumlu bir tampon tarafından sürülür ve giriş veya çıkış olarak çalışacak şekilde programlanabilir. Anahtar matris, I/O bloklarından giriş alır ve bunları bireysel makro hücrelere yönlendirir. Benzer şekilde, makro hücrelerden seçilen çıkışlar gerektiğinde çıkışlara gönderilir. Her PLD genellikle 8 ile 16 makro hücre içerir ve genellikle tamamen bağıdır. Eğer bir makro hücrede kullanılmayan ürün terimleri varsa, bunlar diğer yakın makro hücreler tarafından kullanılabilir. Bazı durumlarda makro hücre flip-flop'u, bir D, JK veya T flip-flop olarak çalışacak şekilde programlanır.

Farklı üreticiler, CPLD'lerin genel mimarisine farklı yaklaşımlar benimsemiştir. Farklılık gösterdikleri alanlar arasında bireysel PLD'ler (bazen



ŞEKİL 7.20
Genel CPLD yapılandırması

işlev blokları), makro hücre türü, G/Ç blokları ve programlanabilir bağlantı yapısı bulunmaktadır. Bir satıcıya özgü cihazı araştırmanın en iyi yolu, üreticinin literatürüne bakmaktır.

VLSI tasarımında kullanılan temel bileşen, bir silikon alanında üretilen ve binlerce kez tekrarlanan bir kapı deseninden oluşan kapı dizisidir. Tek bir IC chip içinde, kullanılan teknolojiye bağlı olarak, bin ile birkaç yüz bin kapı dizisi üretilmektedir. Kapı dizileri ile tasarım, müşterinin üreticiye istenen bağlantı desenini sağlamasını gerektirir. Üretim sürecinin ilk birkaç aşaması, nihai mantık işlevinden bağımsız ve ortaktır. Tasarımcı tarafından verilen spesifikasyonlara göre kapıları birbirine bağlamak için ek üretim adımları gereklidir.

Bir alan-programlanabilir kapı dizisi (FPGA), kullanıcının konumunda programlanabilen bir VLSI devresidir. Tipik bir FPGA, programlanabilir giriş ve çıkış blokları ile çevrili milyonlarca mantık bloğundan oluşan bir dizi içerir ve programlanabilir bağlantılar aracılığıyla birbirine bağlanır. Bu cihazlar grubunda geniş bir iç yapı çeşitliliği bulunmaktadır. Her tür cihazın performansı, mantık bloklarındaki devreye ve programlanmış bağlantılarının verimliliğine bağlıdır.

Tipik bir FPGA mantık bloğu, arama tabloları, çoklayıcılar, kapılar ve flip-flop'lerden oluşur. Bir arama tablosu, bir SRAM'de saklanan bir doğruluk tablosudur ve mantık bloğu için kombinasyonel devre işlevlerini sağlar. Bu işlevler, Bölüm 7.5'te açıklandığı gibi, ROM ile uygulandığı şekilde arama tablosundan gerçekleştirilir. Örneğin, 16×2 SRAM, dört girişi ve iki çıkışı olan bir kombinasyonel devrenin doğruluk tablosunu saklayabilir. Kombinasyonel mantık bölümü, flip-flop için giriş denklemlerini ve mantık bloğunun çıkışını yapılandırmak için bir dizi programlanabilir çoklayıcı ile birlikte kullanılır.

RAM'in yerine ROM kullanmanın avantajı, tablonun belirli bir şekilde belleğe yazılarak programlanabilmesidir. Dezavantajı ise belleğin volatile olması ve güç kesildiğinde arama tablosunun içeriğinin yeniden yüklenmesi gerekliliğidir. Program, ya bir ana bilgisayardan ya da bir yerleşik PROM'dan download edilebilir. Program, FPGA yeniden programlanana kadar veya güç kapatılana kadar SRAM'de kalır. Cihaz, her güç açıldığında yeniden programlanmalıdır. FPGA'yı yeniden programlama yeteneği, programda farklı mantık uygulamaları kullanarak çeşitli uygulamalara hizmet edebilir.

PLD, CPLD veya FPGA ile tasarım, sentez prosedürünü kolaylaştırmak için kapsamlı bilgisayar destekli tasarım (CAD) araçları gerektirir. Mevcut olan araçlar arasında şematik giriş paketleri ve hardware description languages (HDLs), örneğin ABEL, VHDL ve Verilog bulunmaktadır. Sentez araçları, HDL'de yazılmış yüksek seviyeli bir tasarım tanımına uyması için mantık bloklarını tahsis eden, yapılandıran ve birleştiren araçlardır. CMOS FPGA teknolojisinin bir örneği olarak, Xilinx FPGA'yı tartışacağız.¹

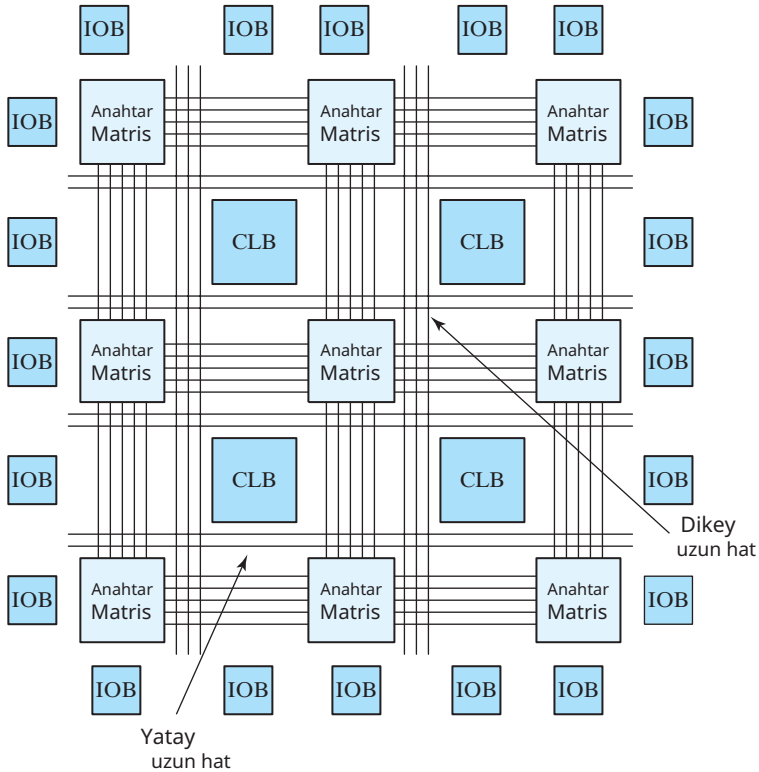
¹ Alternatif bir CMOS FPGA mimarisi için www.Altera.com adresine bakın.

Xilinx FPGA'ları

Xilinx, 1985 yılında XC2000 cihaz ailesi ile dünyanın ilk ticari FPGA'sını piyasaya sürdü .² XC3000 ve XC4000 aileleri kısa süre sonra geldi ve günümüzün Spartan™ ve Virtex™ cihaz aileleri için zemin hazırladı. Cihazların her evrimi, yoğunluk, performans, güç tüketimi, voltaj seviyeleri, pin sayıları ve işlevsellikte iyileştirmeler getirdi. Örneğin , Spartan cihaz ailesi başlangıçta maksimum 40K sistem kapısı sunuyordu, ancak günümüzün Spartan-6'sı 150,000 mantık hücresi ve 4.8Mb blok RAM sunmaktadır.

Temel Xilinx Mimarisi

Spartan ve önceki cihaz ailelerinin temel mimarisi, yapılandırılabilir mantık blokları (CLB'ler), çeşitli yerel ve küresel yönlendirme kaynakları, giriş-çıkış (I/O) blokları (IOB'ler), programlanabilir I/O tamponları ve SRAM tabanlı yapılandırma belleğinden oluşmaktadır, Şekil 7.21'de gösterildiği gibi.



ŞEKİL 7.21

Xilinx Spartan ve öncül cihazların temel mimarisi

²Xilinx ürünleri hakkında ayrıntılı ve güncel bilgi için www.Xilinx.com adresini ziyaret edin.

Yapılandırılabilir Mantık Bloğu (CLB)

Her CLB, bir programlanabilir arama tablosu, çoklayıcılar, kayıtlar ve kontrol sinyalleri için yollar içermektedir, bu da Şekil 7.22'de gösterilmiştir. Arama tablosunun iki fonksiyon üretici (F ve G), dört girişin herhangi bir keyfi fonksiyonunu üretebilir ve üçüncüsü (H), üç girişin herhangi bir Boole fonksiyonunu üretebilir. H-fonksiyon bloğu, girişlerini F ve G arama tablolarından veya dış girişlerden alabilir. Üç fonksiyon üretici, (1) üç bağımsız değişken setinin üç farklı fonksiyonunu (ikisi dört girişli ve biri üç girişli - bir fonksiyon CLB içinde kaydedilmelidir), (2) beş değişkenin keyfi bir fonksiyonunu, (3) dört değişkenin keyfi bir fonksiyonunu ve bazı altı değişkenin fonksiyonlarını, ve (4) dokuz değişkenin bazı fonksiyonlarını üretmek üzere programlanabilir.

Her CLB, ortak bir saat ile kenar tetiklemeli flip-flop olarak yapılandırılabilen iki depolama cihazına sahiptir veya XC4000X'te, flip-flop veya ortak bir saat ile şeffaf latch olarak yapılandırılabilir (her iki kenar için programlanmış ve ayrı olarak tersine çevrilebilir) ve bir etkinleştirme ile. Depolama elemanları, girişlerini fonksiyon üreticilerinden veya D girişinden alabilir. Diğer eleman, dış bir girişi alabilir. H1 girişi. Fonksiyon üreticileri, depolama elemanlarının çıkışlarından bağımsız olarak doğrudan ve bağımsız bir şekilde iki çıkışı (X ve Y) de sürebilir. Bu çıkışların hepsi, bağlantı ağına bağlanabilir. Depolama elemanları, güç açıldığında küresel bir set/sıfırlama ile sürülür; küresel set/sıfırlama, belirli bir depolama elemanı için yerel S/R kontrol programlamasıyla eşleşecek şekilde programlanmıştır.

Dağıtılmış RAM

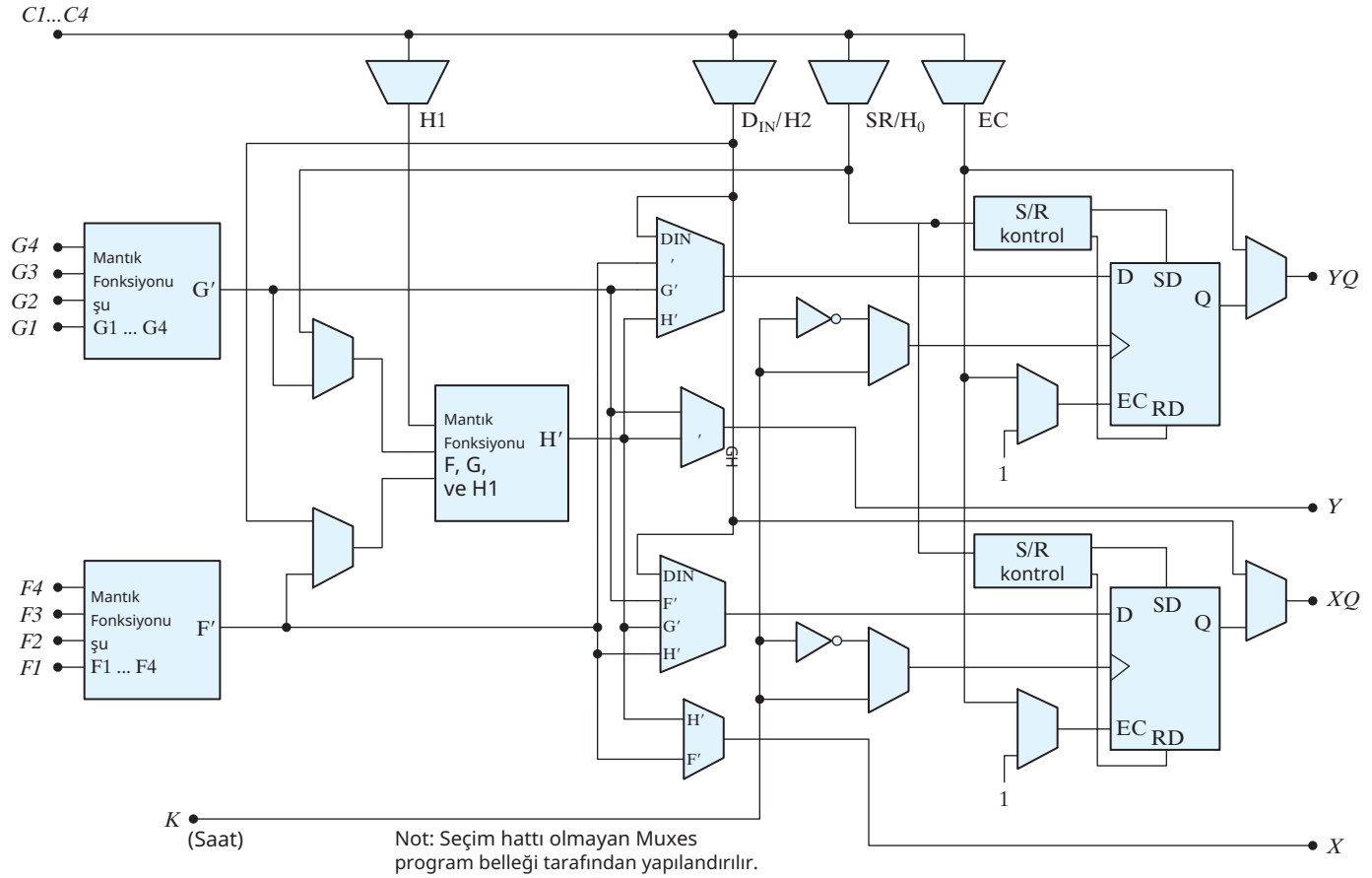
CLB içindeki üç fonksiyon üretici, ya 16×2 çift portlu RAM ya da 32×1 tek portlu RAM olarak kullanılabilir. XC4000 cihazlarının blok RAM'i yoktur, ancak CLB'lerin bir grubu bir bellek dizisi oluşturabilir. Spartan cihazları, dağıtılmış RAM'e ek olarak blok RAM'e sahiptir.

Bağlantı Kaynakları

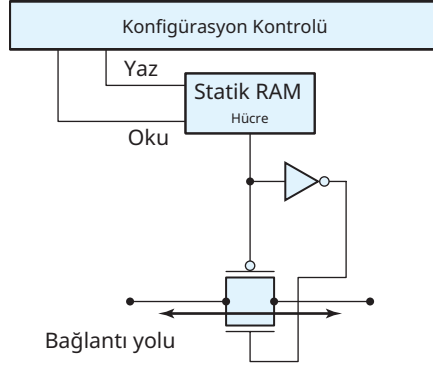
Bir dizi anahtar matrisleri, cihaz boyunca dallanma ve yönlendirme için genel amaçlı bağlantı sağlamak üzere CLB'lerin mimarisinin üzerine yerleştirilmiştir. Bağlantı, general amaçlı üç tür bağlantıya sahiptir: tek uzunluklu hatlar, çift uzunluklu hatlar ve uzun hatlar. Yatay ve dikey tek uzunluklu hatların bir ızgarası, her kutu içinde iki sinyal yolları arasında az sayıda bağlantı sağlayan bir dizi anahtar kutusunu bağlar, tam bir çapraz anahtar değil. Her CLB, CLB'nin üstünde veya altında en yakın yatay hatlara sinyalleri iletebilen bir çift üç durumlu tampon içerir.

Doğrudan (özel) bağlantı hatları, aynı sütun veya satırdaki bitişik dikey ve yatay CLB'ler arasında yönlendirme sağlar. Bunlar, metal üzerinden nispeten yüksek hızlı yerel bağlantılardır, ancak sinyal yollarını yapılandıran iletim kapıları aracılığıyla yönlendirme nedeniyle sertleştirilmiş metal bağlantılardan daha hızlı değildir. Doğrudan bağlantı hatları, anahtar matrislerini kullanmaz, böylece bir matris üzerinden giden yolların neden olduğu gecikmeyi ortadan kaldırır.³

³ Xilinx belgelerine bakarak CLB'ler arasında yerel bağlantılar kurmak için pin çıkış konvansiyonlarını inceleyin.



ŞEKİL 7.22
CLB mimarisi

**ŞEKİL 7.23**

Bir PIP iletim kapasını kontrol eden RAM hücresi

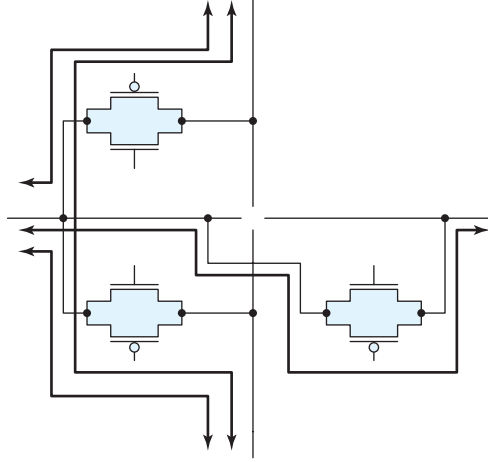
Çift uzunlukta hatlar, bir anahtarlama matrix'ine girmeden önce iki CLB'nin mesafesini kat eder, her diğer CLB'yi atlayarak. Bu hatlar, bir anahtarlama matrix'ini yolundan çıkararak, ara uzunluktaki bağlantıların daha verimli bir şekilde uygulanmasını sağlar ve böylece y olun gecikmesini azaltır.

Uzun hatlar, tüm diziyi dikey ve yatay olarak kapsar. Düşük kayma, yüksek fan-out kontrol sinyallerini sürer. Uzun dikey hatlar, hatları segmentlere ayıran ve dizinin yarısını kapsayan an iki bağımsız yönlendirme kanalı sağlayan programlanabilir bir ayırıcıya sahiptir, ancak a yını sütunda yer alır. Yönlendirme kaynakları, yönlendirme yazılımı tarafından otomatik olarak kullanılır. Saat dağıtımı için sekiz düşük kaymalı küresel tampon bulunmaktadır.

Uzun hatları süren sinyaller tamponlanır. Uzun hatlar, bitişik CLB'ler veya IOB'ler t arafından sürülebilir ve CLB'lere sunulan üç durumlu tamponlara bağlanabilir. Uzun hatlar, mimari içinde üç durumlu veri yolları sağlar ve kablolu-VE mantığını uygular. Her yatay uzun hat, üç durumlu bir tampon tarafından sürülmektedir ve eğer hat üzerinde herhangi bir s ürücü etkin değilse, hattı mantıksal 1'e çekmek için bir pull-up dirençle bağlantı kuracak şekilde programlanabilir.

Cihazın programlanabilir bağlantı kaynakları, CLB'leri ve IOB'leri ya doğrudan ya da anahtar kutuları aracılığıyla bağlar. Bu kaynaklar, iki katmanlı metal segmentlerden oluşan bir ızgara ve anahtar kutuları içindeki programlanabilir bağlantı noktaları (PIP'ler) içerir. Bir PIP, durumu (açık veya kapalı) programlanabilir bellek içindeki bir statik RAM hücresinin içeriği tarafından belirlenen bir CMOS iletim kapısıdır. Bağlantı, iletim kapısı açık olduğunda (yani, n-kanal transistörün kapısına 1 uygulandığında) ve p-kanal transistörün kapısına 0 uygulandığında kurulur. Böylece, cihaz, kontrol bellek hücresinin içeriğini değiştirerek yeniden programlanabilir.

Bir anahtar kutusundaki PIP tabanlı bağlantı mimarisi, Fig. 7.24'te gösterilmişti r; bu, bir PIP aracılığıyla olası sinyal yollarını göstermektedir. CMOS iletim kapılarının konfigürasyonu, yatay bir hat ile karşıt yatay hat arasındaki ve bağlantıdaki dikey hatlar arasındaki bağlantıyı belirler. Her anahtar matris PIP, tam bağlantıyı sağlamak için altı geçiş transistörü gerektirir.



ŞEKİL 7.24
Programlanabilir PIP için devre

G/Ç Bloğu (IOB)

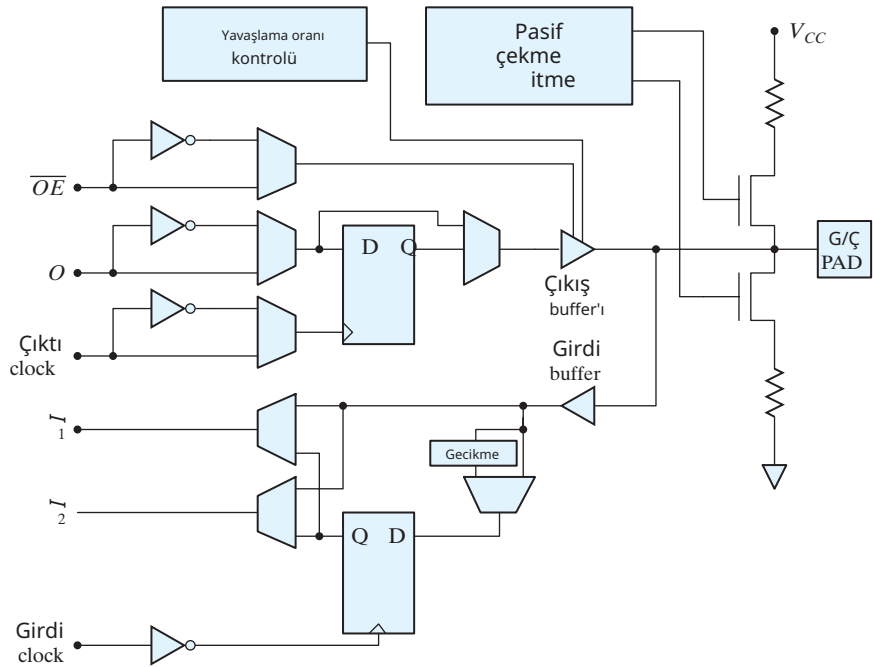
Her programlanabilir G/Ç pini, TTL ve CMOS sinyal seviyeleri ile uyumluluk için tamponlara sahip programlanabilir bir IOB'ye sahiptir. Şekil 7.25, programlanabilir bir IOB için basitleştirilmiş bir şemayı göstermektedir. Bir giriş, çıkış veya iki yönlü port olarak kullanılabilir. Giriş olarak yapılandırılan bir IOB, doğrudan, kilitlenmiş veya kayıtlı girdi alabilir. Çıkış yapılan dırmasında, IOB doğrudan veya kayıtlı çıkışa sahiptir. Bir IOB'nin çıkış tamponu, kayma ve geçiş kontrolüne sahiptir. IOB'nin giriş ve çıkış yoluna sunulan kayıtlar, ayrı, tersine çevrilebilir saatler tarafından sürülmektedir. Küresel bir set/sıfırlama vardır.

İçsel gecikme elemanları, bir saat sinyalinin bir IOB'ye ulaşmadan önce küresel bir tampon üzerinden geçerken indüklenen gecikmeyi telafi eder. Bu strateji, harici bir pin üzerindeki verilerdeki tutma koşulunu ortadan kaldırır. Bir IOB'nin üç durumlu çıkışı, çıkış tamponunu yüksek empedans durumuna getirir. Çıkış ve çıkış için etkinleştirme tersine çevrilebilir. Çıkış tamponunun geçiş hızı, kritik olmayan sinyallerin ana güç hattında geçişlerini en aza indirmek için kontrol edilebilir. IOB pini, gereksiz güç tüketimini ve gürültü önlemek için yukarı çekme veya aşağı çekme için programlanabilir.

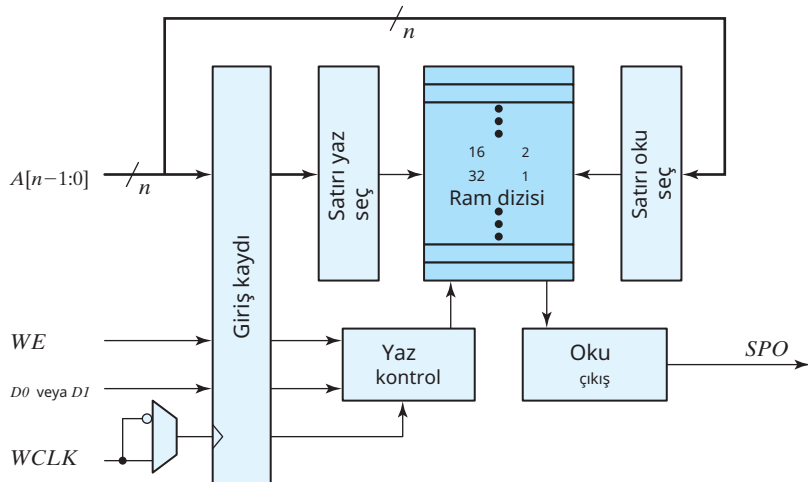
Aygıtlar, IEEE 1149.1 (JTAG) sınır tarama standardını desteklemek için gömülü mantığa sahiptir. Bir yonga üzerindeki test erişim noktası (TAP) denetleyicisi bulunmaktadır ve I/O hücresi bir kaydırma kaydı olarak yapılandırılabilir. Test sırasında, cihaz, bir PC kartındaki tüm pinlerin bağlı olduğunu ve düzgün çalıştığını doğrulamak için karttaki çiplerin tüm I/O pinlerinin seri bir zincirini oluşturarak kontrol edilebilir. Bir ana üç durumlu kontrol sinyali, tüm IOB'leri kart testi için yüksek empedans moduna alır.

Geliştirmeler

Spartan çipleri, gömülü yumuşak çekirdekleri barındırabilir ve yonga üzerindeki dağıtılmış, çift portlu, senkron RAM (SelectRAM), ilk giren ilk çıkar kaydı (FIFO) uygulamak için kullanılabilir.

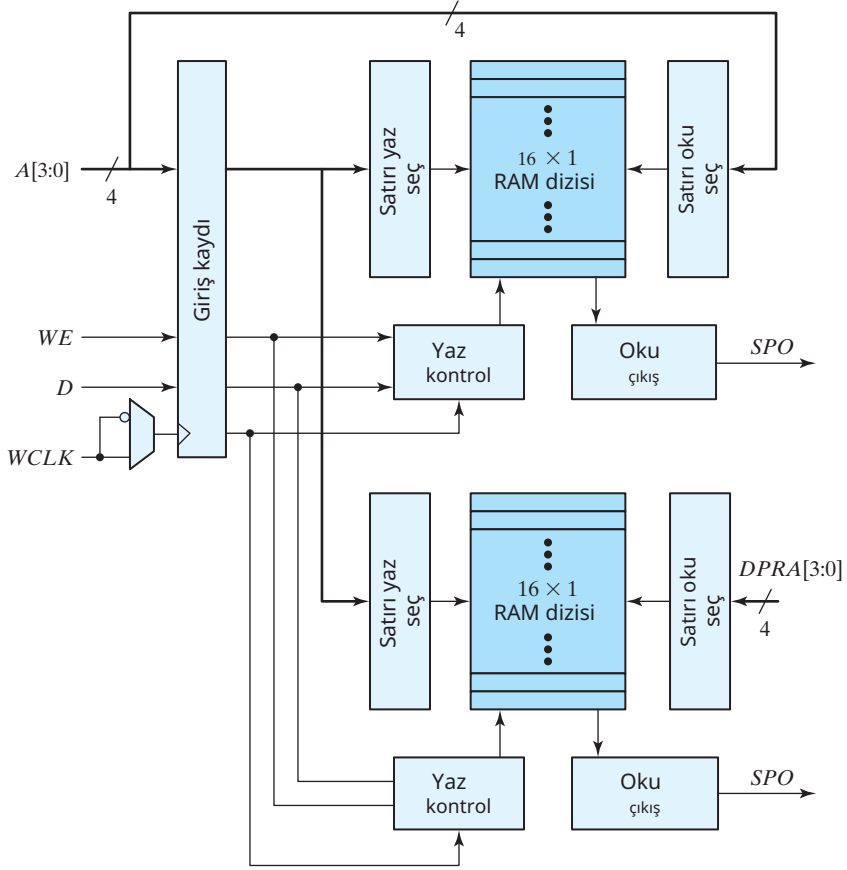


ŞEKİL 7.25
XC4000 serisi IOB



ŞEKİL 7.26
Bir arama tablosundan oluşturulan dağıtılmış RAM hücresi

dosyaları (FIFO'lar), kaydırma kayıtları ve geçici bellekler. Bloklar, herhangi bir genişlik ve derinlikte birleştirilebilir ve parçanın herhangi bir yerinde konumlandırılabilir, ancak kullanımları mantık içinde mevcut CLB'leri azaltır. Şekil 7.26, yonga üzerindeki RAM'in yapısını göstermektedir.



ŞEKİL 7.27
Spartan çift port RAM

eşzamanlı yazma ve asenkron okuma ile tek portlu RAM'i uygulamak için bir arama tablosu programlama. Her CLB, 16×2 veya 32×1 bellek olarak programlanabilir.

İkili portlu RAM'ler, Şekil 7.27'de gösterilen yapı ile bir Spartan cihazında taklit edilir, bir adet (ortak) yazma portu ve iki asenkron okuma portu vardır. Bir CLB, maksimum boyutu 16×1 olan bir bellek oluşturabilir.

Xilinx Spartan XL FPGA'lar

Spartan XL çipleri, Spartan çiplerinin daha ileri bir geliştirmesidir ve daha yüksek hız ve yoğunluk (40.000 sistem kapısı, bunların yaklaşık 6.000'i kullanılabilir) ve çip üzerinde dağıtılmış SelectRAM belleği sunar.⁴ Cihazların arama tabloları, 2^{2n} farklı işlevleri n girişi ile uygulayabilir.

⁴ Xilinx FPGA için maksimum mantık kapısı sayısı, yalnızca mantık işlevlerinden (bellek yok) oluşan bir tasarımda gerçekleştirilebilecek maksimum mantık kapısı sayısının bir katıdır. Mantık kapasitesi, aynı sayıda ve türde mantık işlevlerini uygulamak için gerekli olan iki girişli NAND kapılarının sayısı cinsinden ifade edilir (Xilinx Uygulama Notu).

Tablo 7.7
Xilinx Spartan XL Aygıt Ailesinin Özellikleri

Spartan XL	XCS05/XL	XCS10/XL	XCS20/XL	XCS30/XL	XCS40/XL
Sistem Kapıları ¹	2K–5K	3K–10K	7K–20K	10K–30K	13K–40K
Mantık Hücreleri ²		238 466	950	1,368	1,862
Maksimum Mantık Kapıları	3,000	5,000	10,000	13,000	20,000
Flip-Flops	360	616	1,120	1,536	2,016
Max RAM Bitleri	3,200	6,272	12,800	18,432	25,088
Max Avail I/O	77	112	160	192	224

¹ CLB'lerin %20–30'u RAM olarak.

² 1 Mantık hücresi = dört girişli arama tablosu + flip-flop.

XL serisi, düşük maliyet, düşük güç, düşük paketleme ve düşük test maliyetinin tasarımı kısıtlayan önemli faktörler olduğu uygulamalar için hedeflenmiştir. Spartan XL cihazları, ardışık arama tablolarının sayısına bağlı olarak, 80 MHz'e kadar sistem performansı sunar; bu tablolar, daha uzun yollar ekleyerek performansı azaltır. Tablo 7.7, Spartan XL ailesindeki cihazların önemli özelliklerini sunmaktadır.

Spartan XL ve önceki cihazların mimarisi, bir dizi CLB

taklasının, bir dizi anahtar matrisinin içinde karışık olarak yer aldığı ve IOB'lerin çevrelediği bir yapıya sahiptir. Bu cihazlar yalnızca dağıtılmış belleği destekler; bu da, mantık için kullanılabilecek CLB sayısını azaltır. Görevdeki bellek miktarının görece küçük olması, cihazları, dış bellek cihazlarıyla yapılan işlemlerin performans hedeflerini tehlikeye atmadığı uygulamalarla sınırlı hale getirir. Spartan II serisi ile birlikte, Xilinx, yeni bir mimari de yapılandırılabilir gömülü blok belleği ve dağıtılmış belleği desteklemiştir.

Xilinx Spartan II FPGA'lar

Hız (200 MHz I/O anahtarlama frekansı), yoğunluk (200,000 sistem kapısı kadar) ve çalışma voltajı (2.5 V) gibi iyileştirmelerin yanı sıra, Spartan II cihazlarını Spartan cihazlarından ayıran dört başka özellik vardır: (1) yongada blok bellek, (2) yenilikçi bir mimari, (3) birden fazla I/O standardını destekleme ve (4) gecikme kilitli döngüler (DLL'ler).⁵ Spartan II cihaz ailesi, 0.22/0.18- μ m CMOS teknolojisi ile üretilmiş olup, altı katman metal ile bağlantı sağlamakta ve önceki nesil cihazların dağıtılmış belleğine ek olarak yapılandırılabilir blok belleği içermektedir; blok bellek, mantık veya dağıtılmış bellek miktarını azaltmaz.

⁵ Spartan II cihazları düşük voltajlı diferansiyel sinyalizasyonu (LVDS) veya düşük voltajlı pozitif emiter bağlı mantık (LVPECL) I/O standartlarını desteklemez.

Büyük bir yongada bellek, sistem performansını artırarak dış bellek erişim ihtiyacı nı ortadan kaldırabilir veya azaltabilir.

Güvenilir saat dağıtımı, yüksek hızlı dijital devrelerin senkronize çalışması için anahtardır. Saat sinyali, bir devrenin farklı kısımlarına farklı zamanlarda ulaşırsa, cihaz doğru çalışmayabilir. Saat kayması, kayıtların ayar süresini uzatarak bir devrenin mevcut zaman bütçesini azaltır. Ayrıca, bir kaydırma kaydedicisindeki flip-flop'un etkili tutma zamanı marjını kısaltabilir ve kaydın yanlış kaydırılmasına neden olabilir. Yüksek saat frekanslarında (daha kısa saat periyotları), kaymanın etkisi daha belirgindir çünkü bu, saat döngü süresinin daha büyük bir kısmını temsil eder. Saat kaymasını en aza indirmek için FPGA'larda genellikle tamponlu saat ağaçları kullanılır. Xilinx, yüksek hızlı devrelerde saat senkronizasyonu veya yönetimi için tamamen dijital DLL'ler sağlar. DLL'ler, saat dağıtım gecikmesini ortadan kaldırır ve frekans çarpanları, frekans bölücüler ve saat aynaları sağlar.

Spartan II cihazları, bir video yakalama sisteminin yapıştırma mantığını ve bir ISDN modeminin yapıştırma mantığını uygulamak gibi uygulamalar için uygundur. Cihaz özellikleri Tablo 7.8'de özetlenmiştir ve Spartan serisindeki teknolojinin evrimi Tablo 7.9'daki verilerde aşırtır.

Tablo 7.8
Spartan II Cihaz Özellikleri

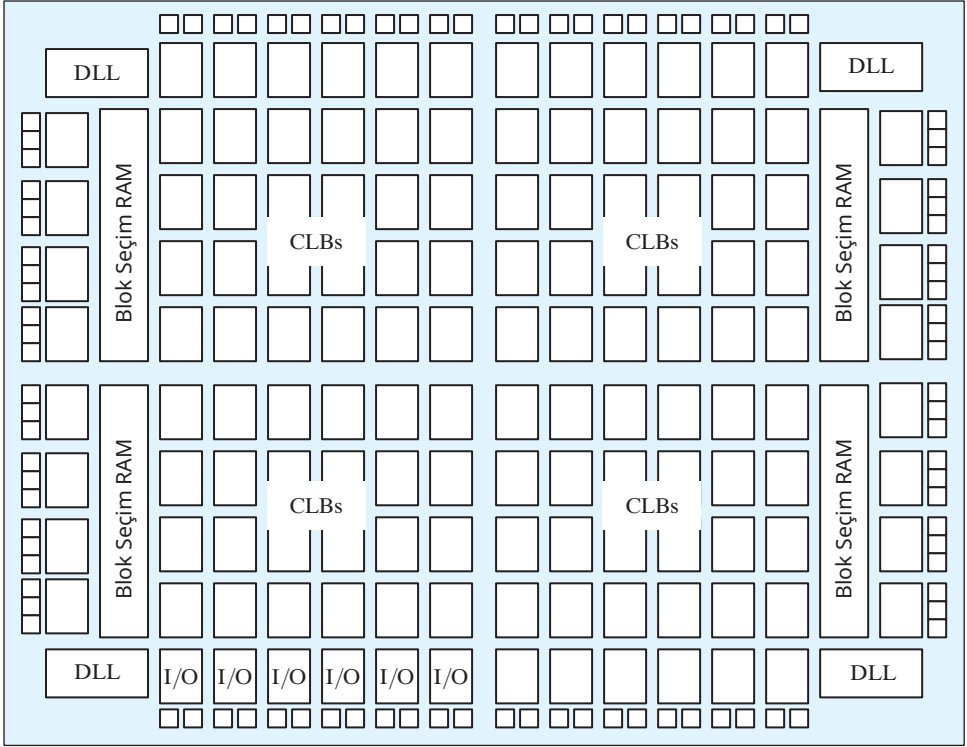
Spartan II FPGA'lar	XC2S15	XC2S30	XC2S50	XC2S100	XC2S150	XC2S200
Sistem Kapıları ¹	6K–15K	13K–30K	23K–50K	37K–100K	52K–150K	71K–200K
Mantık Hücreleri ²	432	972	1,728	2,700	3,888	5,292
Blok RAM Bitleri	16,384	24,576	32,768	40,960	49,152	57,344
Maks. Mevcut G/Ç	86	132	176	196	260	284

¹ CLB'lerin %20–30'u RAM olarak.

² 1 Mantık hücresi = dört girişli arama tablosu + flip-flop.

Tablo 7.9
Spartan Cihaz Ailelerinin Karşılaştırması

Parça	Spartan	Spartan XL	Spartan II
Mimarisi	XC4000 Tabanlı	XC4000 Tabanlı	Virtex Tabanlı
Max # Sistem Kapıları	5K–40K	5K–40K	15K–200K
Bellek	Dağıtılmış RAM	Dağıtılmış RAM	Blok + Dağıtılmış
G/Ç Performansı	80 MHz	100 MHz	200 MHz
G/Ç Standartları	4	4	16
Çekirdek Voltajı	5 V	3.3 V	2.5 V
DLL'ler	Hayır	Hayır	Evet



ŞEKİL 7.28
Spartan II mimarisi

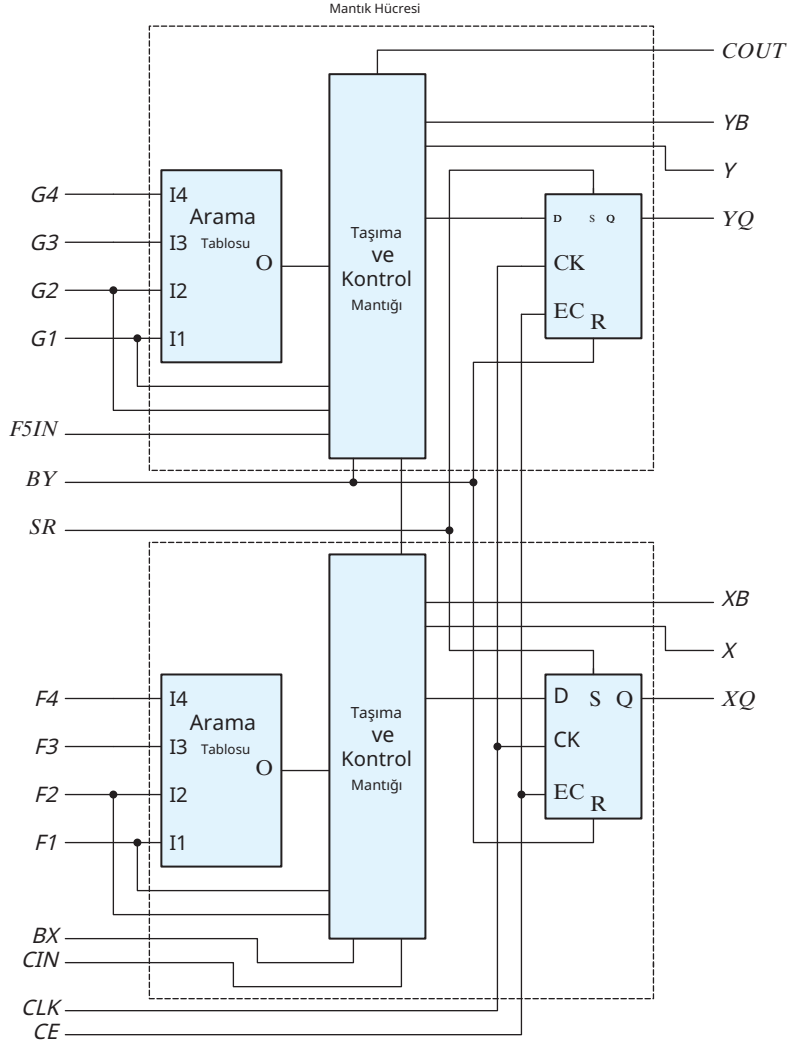
Spartan II cihazının üst düzey karolarla tasarlanmış mimarisi, Şekil 7.28'de gösterildiği gibi, Xilinx parçalarının yeni bir organizasyon yapısını işaret eder. CLB'lerin dört çeyreğinden her biri bir DLL ile desteklenir ve 4,096 bitlik bir RAM bloğu ile çevrelenmiştir, ayrıca çipin çevresi IOB'lerle kaplıdır.

Her CLB, bir çift dilim olarak düzenlenmiş dört mantık hücresi içerir. Şekil 7.29'da gösterilen her mantık hücresi, dört giriшли bir arama tablosu, taşıma ve kontrol için mantık ve bir D tipi flip-flop içerir. CLB, beş veya altı girişi işlevlerini yapılandırmak için ek mantık içerir.

Spartan II parça ailesi, bir çip içi blok RAM'ın esnekliğini ve kapasitesini sağlar; ayrıca, her arama tablosu 16×1 RAM (dağıtılmış) olarak yapılandırılabilir ve bir mantık hücresindeki arama tablosu çifti 16×2 bit RAM veya 32×1 bit RAM olarak yapılandırılabilir.

Spartan II ailesinin IOB'ları, çeşitli yüksek hızlı belleklerin referans, çıkış voltajı ve e sonlandırma voltajlarını desteklemek için bireysel olarak programlanabilir.

⁶ Parçalar, 14 bloğa kadar (56K bit) mevcuttur.

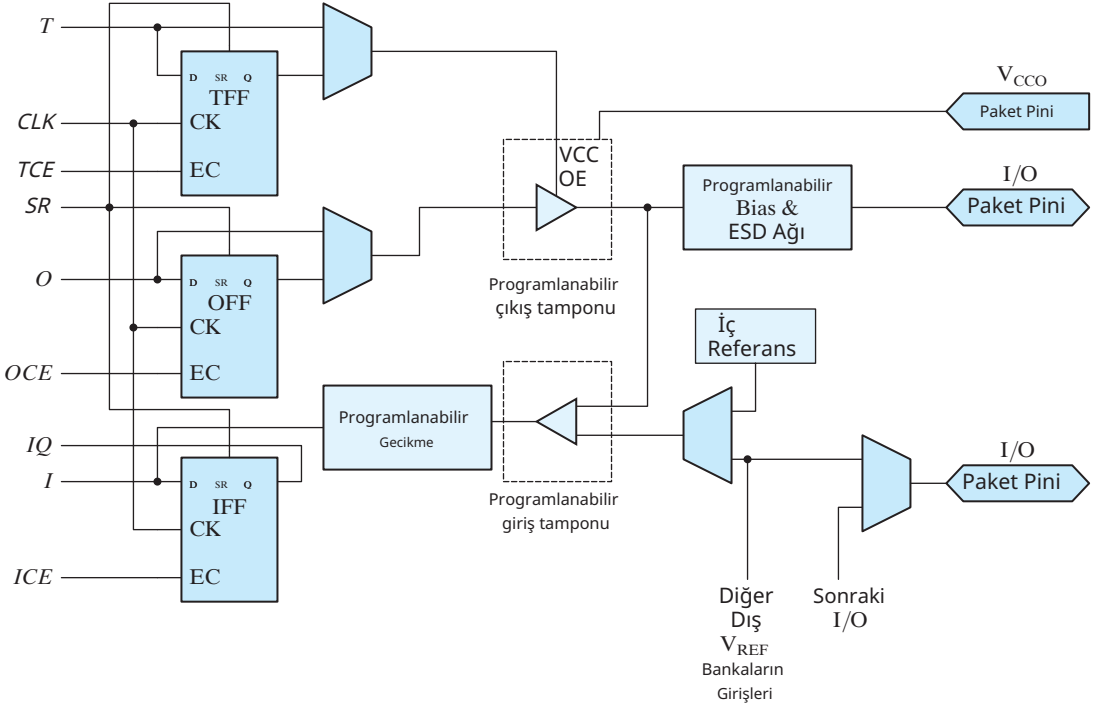


ŞEKİL 7.29
Spartan II CLB dilimi

ve veri yolu standartları. (Bkz. Şekil 7.30.) Her IOB, D-tip flip-flop veya seviye duyarlı latch olarak işlev görebilen üç kayda sahiptir.

Bir kayıt (TFF) programlanabilir çıkış tamponunu (senkronize olarak) kontrol eden sinyali kaydetmek için kullanılabilir. İkinci bir kayıt (OFF) iç mantıktan gelen bir sinyali kaydetmek için programlanabilir.

(Alternatif olarak, iç mantıktan gelen bir sinyal doğrudan çıkış tamponuna geçebilir.) Üçüncü cihaz, I/O pad'den gelen sinyali kaydedebilir. (Alternatif olarak, bu



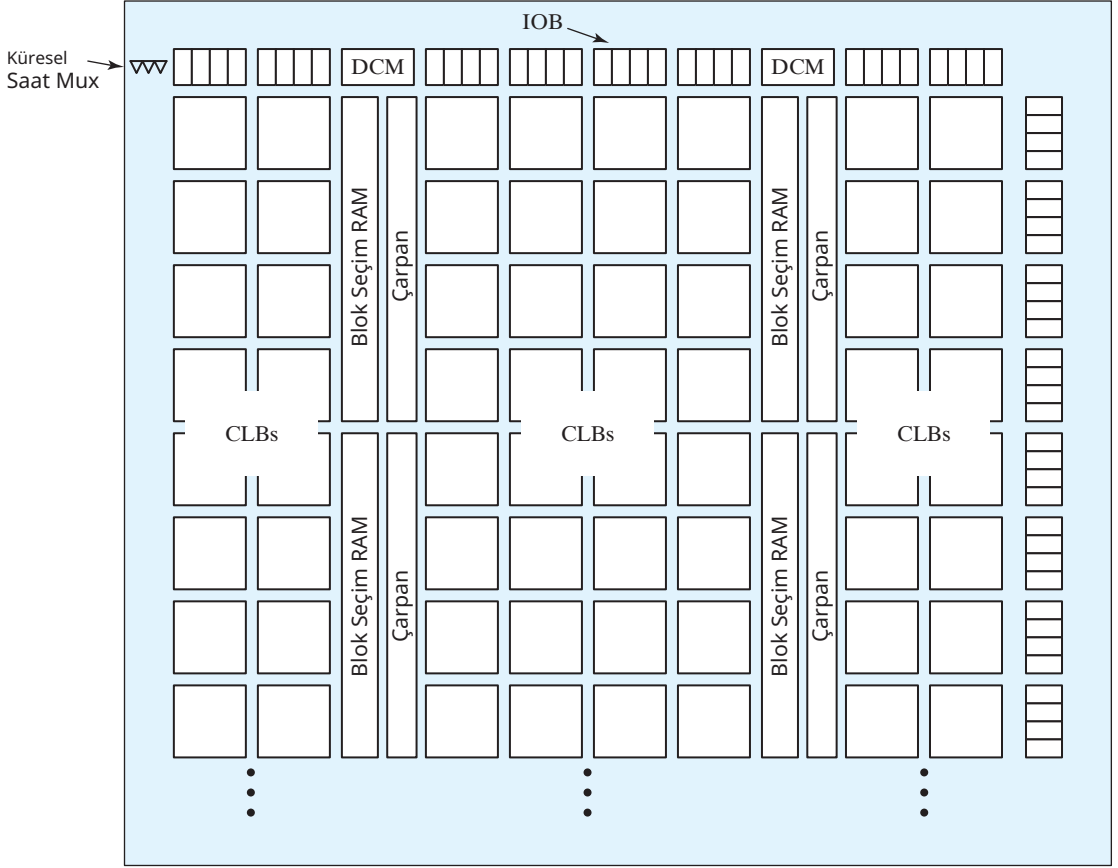
ŞEKİL 7.30
Spartan II IOB

sinyal doğrudan iç mantığa geçebilir.) Ortak bir saat her kaydı sürer, bununla birlikte her birinin bağımsız bir saat etkinleştirilmesi vardır. Giriş yolundaki programlanabilir gecikme elemanı, pad'den pad'e tutma süresini ortadan kaldırmak için kullanılabilir.

Xilinx Virtex FPGA'lar

Virtex cihaz serisi⁷ Xilinx teknolojisinin öncü noktasıdır. Bu cihaz ailesi, karmaşık sistem düzeyi ve sistem üzerinde çip tasarımlarının çözümünü etkileyen dört ana faktörü ele alır: (1) entegrasyon seviyesi, (2) gömülü bellek miktarı, (3) performans (zamanlama) ve (4) alt sistem arayüzleri. Aile, yüksek performanslı mantık, seri bağlantı, sinyal işleme ve gömülü işleme (örneğin, kablosuz iletişim) gerektiren uygulamalara yöneliktir. İşlem kuralları

⁷ Virtex, Virtex-II, II Platform, II-Pro/Pro X ve Virtex-5 Çoklu Platform FPGA.



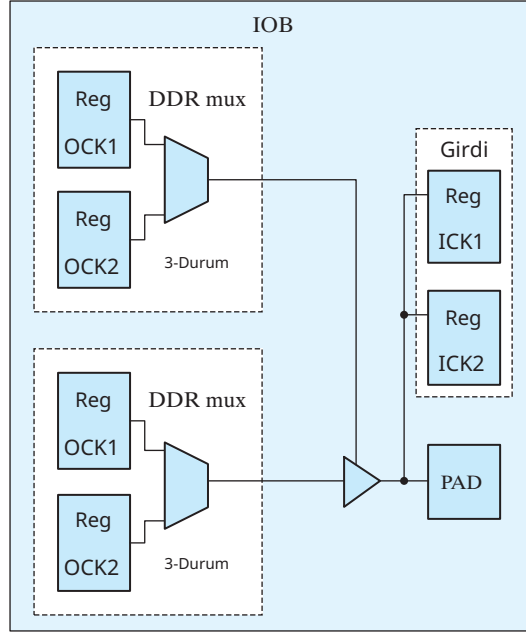
DCM: Saat Yöneticisi

ŞEKİL 7.31

Virtex II genel mimarisi

önde gelen Virtex parçaları 65 nm'de, 1 V çalışma voltajıyla durmaktadır. Kurallar 330,000'e kadar mantık hücresi ve saat etkinleştirmeli 200,000'den fazla dahili flip-flop ile birlikte, 10 Mb'den fazla blok RAM ve 550 MHz saat teknolojisini tek bir yonga içinde barındırmaktadır.

Virtex ailesi, LVDS ve LVPECL dahil olmak üzere 20 farklı I/O standardı için fiziksel (elektriksel) ve protokol desteği sunar, bireysel olarak programlanabilir pinlerle. 12'ye kadar dijital saat yöneticisi, birden fazla saat alanı ve yüksek frekanslı I/O gerektiren senkron uygulamalarda frekans sentezi ve faz kaydırma desteği sağlar. Virtex mimarisi Şekil 7.31'de gösterilmiştir ve IOB'si Şekil 7.32'de gösterilmektedir.



ŞEKİL 7.32
Virtex IOB bloğu

PROBLEMLER

* ile işaretlenmiş problemlerin cevapları kitabın sonunda yer almaktadır.

Aşağıdaki bellek birimleri, kelime sayısı ile kelime başına bit sayısının çarpımı ile belirtilmiştir. Her durumda kaç adres hattı ve girdi-çıkıttı veri hattı gereklidir?

(a) 1Kb	(b) 20	(c) 256	(d) 256K
---------	--------	---------	----------

7.2 * Problem 7.1'de listelenen belleklerde saklanan byte sayısını verin.

7.3 * Şekil 7.3'te gösterilen bellekteki 563 numaralı kelime, 1.212'nin ikilik karşılığını içerir. 10 bitlik adresi ve kelimenin 16 bitlik bellek içeriğini listeyin.

Yazma ve okuma işlemleri için bellek döngüsü zamanlama dalga formlarını gösterin. 150 MHz'lik bir CPU saati ve 20 ns'lik bir bellek döngü süresi varsayın.

Örnek 7.1'de tanımlanan ROM için bir test tezgahı yazın. Test programı, adres 5'te ikilik 7 ve adres 7'de ikilik 5 depolar. Ardından, depolanan içeriklerini doğrulamak için iki adres okunur.

Şekil 7.6'daki 4x4 RAM'i tüm giriş ve çıkışları gösteren bir blok diyagramında kaplayın. Üç durumlu çıkışlar varsayılarak, dört 4x4 RAM birimi kullanarak bir 8x8 bellek oluşturun.

7.7 * A $16K \times 4$ bellek, iç dekoderi ikiye ayırarak tesadüfi kodlama kullanır X -seçimi ve Y -seçimi.

- (a) Her bir dekoderin boyutu nedir ve adresi kodlamak için kaç AND kapısı gereklidir?
 (b) Giriş adresi 6,000'ün ikili karşılığı olduğunda etkinleştirilen X ve Y seçim hatlarını belirleyin.

7.8 * (a) 256K bayt bellek kapasitesi sağlamak için kaç adet $32K \times 8$ RAM çipi gereklidir?

- (b) 256K bayt erişmek için kaç adres hattı kullanılmalıdır? Bu hatların kaç tanesi tüm çiplerin adres girişlerine bağlıdır?
 (c) Çip seçme girişleri için kaç hat kodlanmalıdır? Dekoderin boyutunu belirtin.

Bir DRAM çipi iki boyutlu adres çoklama kullanır. 13 ortak adres pini vardır, satır adresi sütun adresinden bir bit daha fazladır. Belleğin kapasitesi nedir?

7.10 * Verilen 8 bitlik veri kelimesi 01011011 için, tek hataları düzelten ve çift hataları tespit eden Hamming kodu için 13 bitlik bileşik kelimeyi oluşturun.

7.11 * 11 bitlik veri kelimesi 11001001010 için 15 bitlik Hamming kod kelimesini elde edin.

7.12 * 8 bit veri ve 4 parite bit içeren 12 bitlik Hamming kod kelimesi bellikten okunur. Okunan 12 bitlik kelime aşağıdaki gibiyse, belleğe yazılan orijinal 8 bitlik veri kelimesi nedir:

- (a) 0000 11100000 (b) 101110000110
 (c) 101111110100

7.13 * Veri kelimesi ile birlikte tek hata düzeltme ve çift hata tespiti sağlamak için kaç parite kontrol biti eklenmelidir, veri kelimesi

- (a) 16 bit. (b) 32 bit.
 (c) 48 bit.

D_3, D_5, D_6 , ve D_7 için Hamming kodunu oluşturmak gereklidir, P_1, P_2 , ve P_4 ile birlikte üç parite biti.

(a) * 0010 veri kelimesi için 7 bitlik bileşik kod kelimesini değerlendirin.

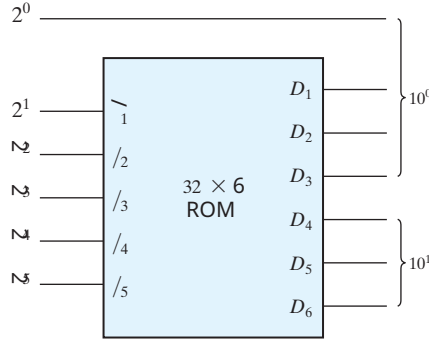
Üç kontrol bitini değerlendirin, C_4, C_2 , ve C_1 , hata olmadığını varsayarak.

- (c) Belleğe yazma sırasında D_5 bitinde bir hata olduğunu varsayın. Bitteki hatanın nasıl tespit edildiğini ve düzeltildiğini gösterin.
 (d) Kodu çift hata tespiti eklemek için parite bitini P_8 ekleyin. Hataların P_2 ve D_5 bitlerinde meydana geldiğini varsayın. Çift hatanın nasıl tespit edildiğini gösterin.

64×8 ROM yongası kullanarak, sekiz yonga ve bir dekoder ile 512×8 ROM oluşturun.

7.16 * $4,096 \times 8$ bitlik bir ROM yongası iki yonga seçme girişine sahiptir ve 5 V güç kaynağından çalışır. Entegre devre paketi için kaç pin gereklidir? Bir blok diyagramı çizin ve ROM'daki tüm giriş ve çıkış terminallerini etiketleyin.

32×6 ROM, Şekil P7.17'de gösterildiği gibi, 2 0 hattı ile birlikte altı bitlik bir ikili sayıyı karşılamak için iki basamaklı BCD sayısına dönüştürür. Örneğin, ikili 100001, BCD 011 00 11'e (ondalık 33) dönüşür. ROM için doğruluk tablosunu belirtin.



ŞEKİL P7.17

7.18

* Aşağıdaki kombinasyonel devre bileşenleri için doğruluk tablosunu barındıracak bir ROM'un boyutunu (kelime sayısı ve kelime başına bit sayısı) belirtin:

- (a) iki 4 bitlik ikili kelimeyi çarpan bir ikili çarpan,
- (b) bir 4 bit toplama-çıkarma,
- (c) ortak seçim ve etkinleştirme girişlerine sahip dört katlı iki-tek hat çoklayıcı, ve
- (d) bir etkinleştirme girişi olan BCD-yedi segment dekodere.

Aşağıda listelenen dört Boolean fonksiyonu için PLA programlama tablosunu oluşturun. Ürün terimlerinin sayısını en aza indirin.

$$A(x, y, z) = \Sigma(1, 3, 5, 6)$$

$$B(x, y, z) = \Sigma(0, 1, 6, 7)$$

$$C(x, y, z) = \Sigma(3, 5)$$

$$D(x, y, z) = \Sigma(1, 2, 4, 5, 7)$$

7.20 8×4 ROM'un Boolean fonksiyonlarını uygulayan doğruluk tablosunu oluşturun

$$A(x, y, z) = \Sigma(0, 3, 4, 6)$$

$$B(x, y, z) = \Sigma(0, 1, 4, 7)$$

$$C(x, y, z) = \Sigma(1, 5)$$

$$D(x, y, z) = \Sigma(0, 1, 3, 5, 7)$$

Şimdi ROM'u bir bellek olarak düşünelim. Adres 1 ve 4'teki bellek içeriğini belirtin.

7.21 Üç bitlik bir sayıyı kareleyen kombinasyonel devre için PLA programlama tablosunu çıkarın. Ürün terimlerinin sayısını en aza indirin. (Eşdeğer ROM uygulaması için bkz. Şekil 7.12.)

7.22 Dört bitlik bir sayıyı kareleyen kombinasyonel devre için ROM programlama tablosunu çıkarın. Ürün terimlerinin sayısını en aza indirin.

7.23 BCD-den-fazla-3-kodu dönüştürücüsü için Boolean

fonsksiyonlarının Fig. 4.3'te basitleştirildiği PLA programlama tablosunu listeleyin.

Problem 7.23'ü bir PAL kullanarak tekrarlayın.

7.25

* Aşağıda üç girişli, dört çıkışlı kombinasyonel devrenin doğruluk tablosu bulunmaktadır:

Girişler			Çıkışlar			
x	y	z	A	B	C	D
0	0	0	0	1	0	0
0	0	1	1	1	1	1
0	1	0	1	0	1	1
0	1	1	0	1	0	1
1	0	0	1	1	1	0
1	0	1	0	0	0	1
1	1	0	1	0	1	0
1	1	1	0	1	1	1

Devre için PAL programlama tablosunu oluşturun ve Fig. 7.17'de gösterilen benzer bir PAL diyagramında sigorta haritasını işaretleyin.

Fig. 7.19'daki kayıtlı makro hücrelerini kullanarak, iki girişi x ve y olan ve giriş denklemleri A ve B tanımlanan bir sıralı devre için sigorta haritasını gösterin.

$$D_A = x \oplus y \oplus A$$

7.27

Fig. 7.16'daki PAL diyagramını, Fig. 7.19'daki gibi OR kapıları ile çıkışlar arasında üç saatli D tipi flip-flop ekleyerek değiştirin. Diyagram, bir sıralı devrenin blok diyagramına uygun olmalıdır. Değişiklik, flip-flop çıkışlarının AND dizisine programlanabilir bağlantılarla bağlanması için üç ek tampon-invertör kapısı ve altı dikey hat gerektirecektir. Değiştirilmiş kayıtlı PAL diyagramını kullanarak, bir çıkış taşıyıcısına sahip üç bitli ikili sayacı uygulayacak sigorta haritasını gösterin.

Fonksiyonları uygulamak için bir PLA devresi çizin.

$$F_1 = A \oplus B \oplus AC + A \oplus BC$$

$$F_2 = (AC + AB + BC)$$

7.29

Problem 7.26'da tanımlanan PLA için programlama tablosunu geliştirin.

REFERANSLAR

1. Hamming, R. W. 1950. Hata Tespit ve Hata Düzeltme Kodları. *Bell Syst. Tech. J.* 29: 147–160.
2. KITSON, B. 1989. *Programlanabilir Dizi Mantığı El Kitabı*. Sunnyvale, CA: Advanced Micro Devices.
3. Lin, S. ve D. J. Costello, Jr. 2004. *Hata Kontrol Kodlaması*. 2. baskı. Englewood Cliffs, NJ: Prentice-Hall.
4. *Hafıza Bileşenleri El Kitabı*. 1986. Santa Clara, CA: Intel.
5. Nelson, V. P., H. T. Nagle, J. D. Irwin, ve B. D. Carroll. 1995. *Dijital Mantık Devresi Analizi ve Tasarımı*. Upper Saddle River, NJ: Prentice Hall.

350 Bölüm 7 Bellek ve Programlanabilir Mantık

- . *Programlanabilir Mantık Veri Kitabı*, 2. baskı. 1994. San Jose, CA: Xilinx, Inc.
7. T occi, R. J. ve N. S. W idmer . 2004. *Dijital Sistemler İlkeleri ve Uygulamaları*, 9. baskı. Upper Saddle River, NJ: Prentice Hall.
8. TRIMBERGER, S. M. 1994. *Alan Programlanabilir Kapı Dizisi Teknolojisi*. Boston: Kluwer Akademik Yayıncılar.
9. WAKERLY, J. F. 2006. *Dijital Tasarım: İlkeler ve Uygulamalar*, 4. baskı. Upper Saddle River, NJ: Prentice Hall.

WEB SEARCH TOPICS

TRISA
Kapı dizisi
Programlanabilir dizi mantığı
Programlanabilir mantık veri kitabı
RAM
*

Bölüm 8

Kayıt Transfer Seviyesinde Tasarım

8.1 GİRİŞ

Birçok dijital sistemin davranışı, girdilerinin geçmişine bağlıdır ve gelecek eylemlerini belirleyen koşullar, önceki eylemlerin sonuçlarına bağlıdır. Bu tür sistemlere “hafızaya” sahip olduğu söylenir. Bir dijital sistem, flip-flop'lar ve kapılarla oluşturulmuş bir ardışık mantık sistemidir. Ardışık devreler, Bölüm 5'te gösterildiği gibi durum tabloları aracılığıyla tanımlanabilir. Bir durum tablosu ile büyük bir dijital sistemi tanımlamak güçtür, çünkü durum sayısı muazzam olacaktır. Bu zorluğun üstesinden gelmek için, dijital sistemler modüler bir yaklaşım ile tasarlanır. Sistem, her biri belirli bir işlevi yerine getiren alt sistemlere bölünmüştür. Modüller, kayıtlar, kod çözücüler, çoklayıcılar, aritmetik elemanlar ve kontrol mantığı gibi dijital cihazlardan inşa edilir. Çeşitli modüller, bir dijital sistemi oluşturmak için datapath'ler ve kontrol sinyalleri ile birbirine bağlanır. Bu bölümde, büyük, karmaşık dijital sistemleri tanımlamak ve tasarlamak için bir tasarım metodolojisi tanıtacağız.

8.2 KAYIT TRANSFER SEVİYESİ NOTASYONU

Dijital sistemlerin modülleri, en iyi şekilde bir dizi kayıt ve bunlarda saklanan ikili bilgilere uygulanan işlemlerle tanımlanır. Kayıt işlemlerine örnekler *kaydırma*, *sayma*, *temizleme* ve *yükleme* gibi işlemlerdir. Kayıtlar, dijital sistemin temel bileşenleri olarak kabul edilir. Kayıtlarda saklanan veriler üzerinde gerçekleştirilen bilgi akışı ve işleme, *kayıt transfer işlemleri* olarak adlandırılır. Daha sonra, bir donanım tanım dili (HDL) ile dijital sistemin kayıt transfer işlemlerine karşılık gelen operatörlerin nasıl dahil edildiğini göreceğiz.

Bir dijital sistem, aşağıdaki üç bileşenle belirtildiğinde *kayıt transfer seviyesi* (RTL) olarak temsil edilir:

- Sistemdeki kayıtlar kümesi.
- Kayıtlarda saklanan verilere uygulanan işlemler.
- Sistemdeki işlemler dizisini denetleyen kontrol.

Bir kayıt, ikili bilgileri saklayan ve bir veya daha fazla temel işlemi gerçekleştirme yeteneğine sahip bağlı flip-flop grubudur. Bir kayıt, yeni bilgiler yükleyebilir veya bilgileri sağa veya sola kaydırabilir. Bir sayaç, bir sayıyı sabit bir değer (örneğin, 1) ile artıran bir kayıttır. Bir flip-flop, ayarlanabilen, temizlenebilen veya tamamlanabilen bir bitlik kayıttır. Aslında, herhangi bir ardışık devrenin flip-flop'ları ve ilgili kapıları, bu tanıma göre kayıtlar olarak adlandırılabilir.

Kayıtlar içinde saklanan bilgilere uygulanan işlemler, bir saat döngüsü sırasında bir veri kelimesinin bitleri üzerinde paralel olarak gerçekleştirilen temel işlemlerdir. İşlem tarafından üretilen veriler, işlemin gerçekleştirildiği öncesinde kayıta bulunan ikili bilgileri değiştirebilir. Alternatif olarak, sonuç başka bir kayda aktarılabilir (yani, bir kayıt üzerindeki bir işlem, içeriğini değiştirmeyebilir). Bölüm 6'da tanımlanan dijital devreler, temel işlemleri gerçekleştiren kayıtlardır. Paralel yükleme özelliğine sahip bir sayaç, bir artırma ve yükleme işlemlerini gerçekleştirebilir. İki yönlü bir kaydırma kaydı, içeriğini belirli bir yönde bir veya daha fazla bit kaydırarak sağa ve sola kaydırma işlemlerini gerçekleştirebilir.

Dijital bir sistemdeki işlemler, işlemleri belirli bir şekilde sıralayan sinyallerle kontrol edilir. Önceki işlemlerin sonuçlarına bağlı olan belirli koşullar, gelecekteki işlemlerin sırasını belirleyebilir. Bir dijital sistemin kontrol mantığının çıktıları, sistemin kayıtlarındaki çeşitli işlemleri başlatan ikili değişkenlerdir.

Bir kayıttan diğerine bilgi transferi, bir yer değiştirme operatörü aracılığıyla sembolik olarak belirtilir. İfade

$$R2 \leftarrow R1$$

"<original>", R1 kayıt içeriğinin R2 kaydına aktarımını belirtir—yani, R2 kaydının içeriğinin R1 kaydının içeriğiyle değiştirilmesidir. Örneğin, 01011010 değerini tutan sekiz bitlik R2 kaydının içeriği, 10100101 değerini tutan R1 kaydıyla değiştirilebilir. Tanım gereği, kaynak kaydı R1 içeriği aktarım sonrası değişmez. Sadece R1 kaydına kopyalanır. Ok, aktarımı ve yönünü simgeler; aktarım yapılan kayıttan, içeriği alacak kayda doğru işaret eder. Bir kontrol sinyali, işlemin ne zaman gerçekleştirileceğini belirler.

Bir dijital sistemdeki denetleyici, kayıt işlemlerini yöneten kontrol sinyallerinin çıktıları olan sonlu durum makinesidir (bkz. Bölüm 5). Senkron makineler de, işlemler sistem saatine göre senkronize edilir. Örneğin, kayıt

R2 Saatin pozitif kenarında içeriğinin değiştirilmesi için senkronize edilebilir.

Bir kayıt transfer işlemini belirten bir ifade, kaynak kaydın çıkışlarından hedefe gidilen bir veri yolunun (yani, bir dizi devre bağlantısının) mevcut olduğunu ima eder.

dın girişleri ve hedef kaydın paralel yükleme yeteneğine sahip olduğunu belirtir. Veriler, bir tel boyunca içeriklerini tekrar tekrar kaydırarak, bir bit bir seferde olmak üzere, kayıtlar arasında seri olarak da aktarılabilir. Normalde, bir kayıt transfer işleminin her saat döngüsünde değil, yalnızca önceden belirlenmiş bir koşul altında gerçekleşmesini isteriz.

Bir kayıt transfer işlemini yöneten koşullu bir ifade, aşağıdaki gibi bir

if-then ifadesi ile sembolize edilir:

Eğer ($TI = 1$) o zaman ($R2 \leftarrow R1$)

burada TI kontrol bölümünde üretilen bir kontrol sinyalidir. Saatin, kayıt transfer ifadelerinde bir değişken olarak dahil edilmediğini unutmayın. Tüm transferlerin bir saat kenarı geçişinde (yani, 0'dan 1'e veya 1'den 0'a geçişte) gerçekleştiği varsayılmalıdır. TI gibi bir kontrol koşulu saat geçişinden önce doğru hale gelse de, gerçek transfer saat geçişi gerçekleşene kadar gerçekleşmez. Transferler, saat sinyalinin eylemiyle başlatılır ve senkronize edilir, ancak çıkışların gerçek geçişi (fiziksel bir sistemde) kayıtların çıkışlarında anlık geçişler oluşturmaz. Yayılma gecikmeleri, kayıtların flip-flop'larını uygulayan transistörlerin fiziksel özelliklerine ve cihazları bağlayan tellerin özelliklerine bağlıdır. Ancak, fiziksel bir sistemde bir neden ile etkisi arasında her zaman, ne kadar küçük olursa olsun, bir gecikme vardır.

Bir virgöl, aynı anda (eşzamanlı) gerçekleştirilen iki veya daha fazla işlemi ayırmak için kullanılabilir. Aşağıdaki ifadeyi düşünün:

Eğer ($T3 = 1$) ise ($R2 \leftarrow R1, R1 \leftarrow R2$)

Bu ifade, iki kayıt arasındaki içeriği değiştiren bir işlemi belirtir; ayrıca, her iki kayıttaki işlem,

$T3 = 1$ olduğu sürece aynı saat kenarı tarafından tetiklenir. Bu eşzamanlı (eşzamanlı) işlem, ortak bir saat (senkronizasyon sinyali) tarafından kontrol edilen kenar tetiklemeli flip-flop'lara sahip kayıtlarla mümkündür. Kayıt transferlerine dair diğer örnekler şunlardır:

$R1 \leftarrow R1 + R2$	$R2$ 'nin içeriğini $R1$ 'e ekleyin ($R1, R1 + R2$ alır)
$R3 \leftarrow R3 + 1$	$R3$ 'ü 1 artırın (yukarı sayın)
$R4 \leftarrow \text{shr } R4$	$R4$ 'ü sağa kaydırın
$R5 \leftarrow 0$	Temizle $R5$ 0'a

Donanımda, toplama ikili paralel toplayıcı ile yapılır, artırma bir sayıcı ile yapılır ve kaydırma işlemi bir kaydırma kaydedici ile uygulanır. Dijital sistemlerde en sık karşılaşılan işlem türleri dört kategoriye ayrılabilir:

- Aktarım işlemleri, verileri bir kayıttan diğerine (yani, kopyalayarak) aktarır.
- Aritmetik işlemler, kayıtlar üzerindeki verilere aritmetik (örneğin, çarpma) uygular.
- Mantık işlemleri, kayıtlar üzerindeki sayısal olmayan verilere bit manipülasyonu (örneğin, mantıksal VEYA) uygular.
- Kaydırma işlemleri, verileri kayıtlar arasında kaydırır.

Taşıma işlemi, kaynak kayıttan hedef kayıta taşınan verinin bilgi içeriğini değiştirmez, kaynak ve hedef aynı olmadıkça.

Diğer üç işlem, transfer sırasında bilgi içeriğini değiştirir. Kayıt transfer notasyonu ve çeşitli kayıt transfer işlemlerini temsil etmek için kullanılan semboller standartlaştırılmamıştır. Bu metinde, iki tür notasyon kullanıyoruz. Bu bölümde tanıtılan notasyon, kayıt transfer seviyesindeki dijital sistemleri belirtmek ve açıklamak için gayri resmi olarak kullanılacaktır. Bir sonraki bölüm, standartlaştırılmış Verilog HDL'de kullanılan RTL sembollerini tanıtmaktadır.

8.3 HDL'DE KAYIT TRANSFER SEVİYESİ

Dijital sistemler, bir HDL aracılığıyla kayıt transfer seviyesinde tanımlanabilir. Verilog HDL'de, RTL işlemlerinin tanımları, kayıt işlemlerini ve donanım tarafından uygulanan kombinasyonel mantık fonksiyonlarını belirtmek için davranışsal ve veri akışı yapılarının bir kombinasyonunu kullanır. Kayıt transferleri, *kenar hassasiyetine sahip döngüsel bir davranış içinde prosedürel atama ifadeleri aracılığıyla belirtilir*.

Kombinasyonel devre fonksiyonları, sürekli atan ifadeler veya *seviye duyarlı döngüsel davranış içindeki prosedürel atama ifadeleri aracılığıyla RTL seviyesinde belirtilir*. Bir kayıt transferini belirtmek için kullanılan sembol ya bir eşittir işareti (=) ya da bir ok (<=); bir kombinasyonel devre fonksiyonunu belirtmek için kullanılan sembol ise bir eşittir işaretidir. Saat ile senkronizasyon, bir her zaman ifadesine, saat olayına duyarlılığı **posedge** veya **negedge** ile nitelendiren bir olay kontrol ifadesi ilişkilendirilerek temsil edilir. *Her zaman* anahtar kelimesi, ilişkili durum ifadeleri bloğunun simülasyon süresi boyunca tekrar tekrar yürütüleceğini belirtir. @ operatörü ve ifadeler bloğundan önceki event kontrol ifadesi, ifadelerin yürütülmesini saat olayına senkronize eder.

Aşağıdaki örnekler, Verilog'da bir kayıt transferi işlemini belirtmenin çeşitli yollarını göstermektedir:

```
(a) assign S = A + B; // Toplama işlemi için sürekli atama
// Toplama işlemi için sürekli atama

(b) always (A, B) // Seviye duyarlı döngüsel davranış
// Seviye duyarlı döngüsel davranış
S = A + B; // Toplama işlemi için kombinasyonel mantık
// Kenar duyarlı döngüsel davranış

(c) her zaman @ (neg kenar saat) // Kenar duyarlı döngüsel davranış
// Kenar duyarlı döngüsel davranış
begin
  RA = RA + RB; // Toplama için engelleyici prosedürel atama
  RD = RA; // Kayıt transfer işlemi
end

(d) her zaman @ (neg kenar saat) // Kenar duyarlı döngüsel davranış
// Kenar duyarlı döngüsel davranış
begin
  RA <= RA + RB; // Toplama için engellemeyen prosedürel atama
  RD <= RA; // Kayıt transfer işlemi
end
```

Sürekli atamalar (örneğin, **assign S = A + B;**) kombinasyonel mantık devrelerini temsil etmek ve tanımlamak için kullanılır. Simülasyonda, sağ taraftaki ifadenin değıştiğinde sürekli atama ifadesi yürütülür. Yürütmenin etkisi hemen gerçekleşir.

(Sol taraftaki değişken güncellenir.) Benzer şekilde, seviye duyarlı döngüsel davranış (örneğin, **always** @ (A, B)) simülasyon sırasında bir değişiklik algılandığında yürütülür. Atama işlemlerinin etkisi hemen gerçekleşir. Sürekli atama ifadesi (**assign** $S = A + B$) bir binary toplayıcıyı tanımlar ve girişleri A ve B ile çıkışı S vardır. Sürekli atama ifadesindeki hedef operand (bu durumda S) bir kayıt ve ri türü olamaz, ancak bir ağ türü olmalıdır, örneğin, **wire**. İkinci örnekteki seviye duyarlı döngüsel davranışta yapılan prosedürel atama, toplama için bir kombinasyonel devreyi tanımlamanın alternatif bir yolunu gösterir. Döngüsel davranış içinde, duyarlılık listesinin mekanizması, S çıktısının, A , veya B , veya her iki si değiştiğinde güncelleneceğini garanti eder.

İki tür prosedürel atama vardır: *bloklama* ve *bloklamayan*. İkisi semboller i ve işlemleri ile ayırt edilir. *Bloklama* atamaları, atama operatörü olarak eşitlik sembolünü (=) kullanırken, *bloklamayan* atamalar sol ok (<=) operatörünü kullanır. Bloklama atama ifadeleri, bir ardışık blokta listelenen sıraya göre sıralı olarak yürütülür; yürütüldüklerinde, bir sonraki ifade yürütülmeden önce bell eğin içeriği üzerinde anında bir etki yaparlar.

Bloklamayan atamalar *eşzamanlı* olarak yapılır. Bu özellik, atamayı sol tarafları na yapmadan önce, ifadelerin sağ tarafındaki her ifadenin değerlendirilmesi ile uygulanır. Sonuç olarak, *herhangi bir atamanın sonucu ile başka bir atamayı etkileyen bir ifadenin değerlendirilmesi arasında etkileşim yoktur*. Ayrıca, kenar duyarlı döngüsel davranışla ilişkili ifadeler, belirtilen kenar koşulu gerçekleşene kadar yürütülmez. Yukarıda verilen örnekte (c) kısmını düşünün. Bloklama prosedürel atama listesinde, ilk ifade toplamı ($RA + RB$) RA 'ya aktarır ve ikinci ifade RA 'nın yeni değerini RD 'ye aktarır.

Saati olayı sonrası RA değer, RA ve RB değerlerinin saati olayı öncesindeki toplamıdır. İşlemin tamamlanmasında, hem RA hem de RD aynı değere sahiptir. Engellemeyen prosedürel atama ((d) yukarıda), iki atamanın eşzamanlı olarak gerçekleştirilmesiyle RD RA değerini alır. Her iki örnekteki etkinlik, düşen kenar geçişi geçiren saat tarafından başlatılır.

Sistemdeki kayıtlar aynı anda (eşzamanlı) saatlenir. Her flip-flop'un D girişi, diğer flip-flop'ların girişlerinden bağımsız olarak, çıkışına atanacak değeri belirler. RTL tasarımında senkron işlemleri sağlamak ve bir HDL modelinin, modelde n sentezlenen devre ile eşleşmesini sağlamak için, kenar duyarlı döngüsel davranış içinde bir değere atanan tüm değişkenler için engellemeyen prosedürel atamaların kullanılması gereklidir (**always** saatlenmiş). Kenar duyarlı döngüsel davranışta görünen engellemeyen atama, senkron sıralı devrenin donanım davranışını doğru bir şekilde modellemektedir. Genel olarak, engelleyici atama operatörü (=) yalnızca birden fazla atama ifadesinin sıralı bir düzenini belirtmek gerektiğinde bir prosedürel atama ifadesinde kullanılır.

HDL Operatörleri

RTL tasarımında kullanılan Verilog HDL operatörleri ve sembolleri Tablo 8.1'de listelenmiştir. Aritmetik, mantıksal ve kaydırma operatörleri, kayıt transfer işlemlerini tanımlar.

Tablo 8.1

Verilog 2001 HDL Operatörleri

Operatör Türü	Sembol	Gerçekleştirilen İşlem
Aritmetik	+	toplama
	-	çıkarma
	*	çarpma
	/	bölüm
	%	modül
	**	üs alma
Bitwise veya Azaltma	~	inkar (tamamlayıcı)
	&	VE
		VEYA
	^	özel-VEYA (XOR)
Mantıksal	!	inkar
	&&	VE
		VEYA
Kaydırma	>>	mantıksal sağa kaydırma
	<<	mantıksal sola kaydırma
	>>>	aritmetik sağa kaydırma
	<<<	aritmetik sola kaydırma
	{ , }	birleştirme
İlişkisel	>	büyüktür
	<	küçüktür
	==	eşitlik
	!=	eşitsizlik
	===	durum eşitliği
	!==	durum eşitsizliği
	>=	büyüktür veya eşittir
	<=	küçüktür veya eşittir

Mantıksal ve ilişkisel operatörler kontrol koşullarını belirtir ve argümanları olarak Boolean ifadeleri alır.

Aritmetik operatörlerin operandları sayılardır. +, -, *, ve / operatörleri, sırasıyla bir çift operandın toplamını, farkını, çarpımını ve bölümünü oluşturur. Üslü operatör (**) 2001 yılında dile eklenmiştir ve bir taban ile üssü olan gerçek, tam sayıdan oluşan bir çift hassasiyetli kayan nokta değeri oluşturur.

veya imzalı değer. Negatif sayılar 2'nin tamamlayıcı biçiminde temsil edilir. Mod-ülüs operatörü, iki sayının bölünmesinden kalan değeri üretir. Örneğin,

14 % 3, 2 olarak değerlendirilir.

İkili kelimeler için iki tür ikili operatör vardır: bit düzeyinde ve azaltma. Bit düzeyindeki operatörler, iki vektör operandı üzerinde bit bit işlem yaparak bir vektör sonucu oluşturur. Bir operandın her bitini alır ve diğer operandın karşılık gelen biti ile işlem yapar. Negasyon (~) bir tekil operatördür; tek bir vektör operandının bitlerini tamamlayarak bir vektör sonucu oluşturur. Azaltma operatörleri de tekildir, tek bir operand üzerinde çalışır ve bir skalar (bir bit) sonucu üretir. Bir kelimenin bitleri üzerinde sağdan sola çiftler halinde işlem yapar ve bir bit sonucu verir. Örneğin, azaltma NOR (~|) operand 00101 ile 0, operand 00000 ile 1 sonucunu verir. İlk iki bit üzerinde NOR işleminin uygulanmasının sonucu üçüncü bit ile kullanılır ve bu şekilde devam eder. Negasyon, bir azaltma operatörü olarak kullanılmaz — bir vektör üzerindeki işlemi bir vektör üretir. İki skalar operand üzerinde çalışan bit düzeyindeki operatörler için doğru tablolar, ilgili Verilog ilkelinin (örneğin, ve ilkelinin ve & bit düzeyindeki operatörün aynı doğru tablosuna sahip olduğu) Bölüm 4.12'deki Tablo 4.9'da listelenenlerle aynıdır. İki skalar girişi olan bir AND kapısının çıktısı, iki bit üzerinde & operatörü ile işlem yaparak üretilen sonuçla aynıdır.

Mantıksal ve ilişkisel operatörler, Boolean ifadeleri oluşturmak için kullanılır ve değişkenler veya ifadeleri operand olarak alabilir. (Not: Bir değişken de bir ifadedir.) Temelde doğru veya yanlış koşulları belirlemek için kullanılırlar, mantıksal ve ilişkisel operatörler, ifade edilen koşul doğruysa 1, yanlışsa 0 değerini alır. Eğer koşul belirsizse, x değerini alırlar. Bir operand olan değişken, değişkenin değeri sıfıra eşitse 0, sıfıra eşit değilse 1 değerini alır. Örneğin, eğer $A = 1010$ ve $B = 0000$ ise, o zaman ifade A Boolean değeri 1'dir (söz konusu sayı 0'a eşit değildir) ve ifade B Boolean değeri 0'dır.

Bu değerlerle yapılan diğer işlemlerin sonuçları aşağıdaki gibidir:

$A \&\& B = 0$	// Mantıksal VE:	$(1010) \&\& (0000) = 0$
$A \& B = 0000$	// Bit düzeyinde VE:	$(1010) \& (1010) = (0000)$
$A B = 1$	// Mantıksal VEYA:	$(1010) (0000) = 1$
$A B = 1010$	// Bit düzeyinde VEYA:	$(1010) (0000) = (1010)$
$!A = 0$	// Mantıksal olumsuzlama	$!(1010) = !(1) = 0$
$\sim A = 0101$	// Bit düzeyinde olumsuzlama	$\sim(1010) = (0101)$
$!B = 1$	// Mantıksal olumsuzlama	$!(0000) = !(0) = 1$
$\sim B = 1111$	// Bit düzeyinde olumsuzlama	$\sim(0000) = 1111$
$(A > B) = 1$	// büyüktür	
$(A == B) = 0$	// kimlik (eşitlik)	

İlişkisel operatörler == ve != Verilog'un dört değerli mantık sisteminde bit düzeyinde eşitlik (kimlik) ve eşitsizlik test eder. Örneğin, eğer $A = 0xx0$ ve $B = 0xx0$ ise, test

$A = \sim B$ doğru olarak değerlendirilirken, test $A = \sim B$ x olarak değerlendirilecektir.

Verilog 2001, mantıksal ve aritmetik kaydırma operatörlerine sahiptir. Mantıksal kaydırma operatörleri, bir vektör operandını belirli bir bit sayısı kadar sağa veya sola kaydırır. Boşalan bit pozisyonları sıfırlarla doldurulur. Örneğin, eğer $R = 11010$ ise, o zaman ifade

```
R = R >> 1;
```

R 'yi sağa bir pozisyon kaydırır. Mantıksal sağa kaydırma işlemi sonucunda elde edilen R değeri $(11010) \gg 1, 01101$ 'dir. Buna karşılık, aritmetik sağa kaydırma operatörü, kelime sağa kaydırıldığında boşalan hücreyi (en anlamlı bit (MSB)) orijinal içeriğiyle doldurur. Aritmetik sola kaydırma operatörü, kelime sola kaydırıldığında boşalan hücreyi 0 ile doldurur. Aritmetik sağa kaydırma operatörü, bir sayının işaret uzantısının önemli olduğu durumlarda kullanılır. Eğer $R = 11010$ ise, o zaman ifade

```
R >>> 1;
```

sonucu üretir $R = 11101$; eğer $R = 01101$ ise, sonucu üretir $R = 00110$.

Mantıksal sola kaydırma ile aritmetik sola kaydırma operatörleri arasında hiçbir ayrım yoktur.

Birleştirme operatörü, birden fazla operatifi eklemek için bir mekanizma sağlar. Boş pozisyonlara aktarılan bitleri de içeren bir kaydırma belirtmek için kullanılabilir. Bu işlemin bir yönü, kaydırma kaydedici için HDL Örneği 6.1'de gösterilmiştir.

İfadeler soldan sağa değerlendirilir ve operatörleri soldan sağa (koşullu operatör hariç) öncelik sırasına göre ilişkilendirilir. Bu öncelik sırası Tablo 8.2'de gösterilmiştir. Örneğin, $A + B - C$ ifadesinde, B değeri A ile toplanır ve ardından C sonuçtan çıkarılır. $A + B/C$ ifadesinde, değeri BC ile bölünür, ardından sonuç A 'ya eklenir çünkü bölme operatörü ($/$) hesaplama önceliği toplama operatöründen (+) daha yüksektir. Önceliği belirlemek için parantez kullanın. Örneğin, $(A + B)C$ ifadesi, $A + B/C$.

Döngü İfadeleri

Verilog HDL, prosedürel ifadeleri tekrar tekrar yürüten dört tür döngüye sahiptir: *repeat*, *forever*, *while* ve *for*. Tüm döngü ifadeleri bir **initial** veya

always bloğu içinde yer almalıdır.

repeat döngüsü, ilişkili ifadeleri belirli bir sayıda yürütür. Aşağıda daha önce kullanılan bir örnek bulunmaktadır:

```
initial
begin
    clock = 1'b0;
    repeat (16)
        #5 clock = ~ clock;
end
```

Bu kod, saati 16 kez değiştirir ve 10 zaman birimi döngü süresine sahip sekiz saat döngüsü üretir.

Tablo 8.2
Verilog Operatör Önceliği

+ - ! ~ & ~& ~ ^ ~^ ^~ (ünary)	En yüksek öncelik
**	
* / %	
+ - (ikili)	
<< >> <<< >>>	
< < = > > =	
== != === !==	
& (binary)	
^ ^~ ~^ (binary)	
(binary)	
&&	
?: (conditional operator)	
{ } { } { }	
	En düşük öncelik

forever döngüsü, bir prosedürel ifadenin veya bir prosedürel ifadeler bloğunun koşulsuz, tekrarlı yürütülmesine neden olur. Örneğin, aşağıdaki döngü, 20 zaman birimi döngü süresine sahip sürekli bir saat üretir:

```
initial
begin
    clock = 1'b0;
    forever
        #10 clock = ~ clock;
end
```

while döngüsü, bir ifade doğru olduğu sürece bir ifadeyi veya bir dizi ifadeyi tekrar tekrar çalıştırır. Eğer ifade başlangıçta yanlışsa, ifade asla çalıştırılmaz. Aşağıdaki örnek, while döngüsünün kullanımını göstermektedir:

```
tam sayı count;
başlangıç
başla
    count = 0;
    while (count < 64)
        #5 count = count + 1;
bitir
```

count değeri 0'dan 63'e kadar artırılır. Her artış beş zaman birimi ile geciktirilir ve döngü 64 sayısında sona erer.

Döngü ifadeleriyle çalışırken, döngüyü dizinlemek için bazen tam sayı veri tipini kullanmak uygun olabilir. Tam sayılar, önceki örnekte olduğu gibi tam sayı anahtar kelimesi ile tanımlanır. Bir döngüyü dizinlemek için **reg** değişkeni kullanmak mümkün olsa da, bazen sayım amaçları için **reg** yerine bir tam sayı değişkeni tanımlamak daha uygun olabilir. Veri türü olarak tanımlanan değişkenler **reg** işaretli sayılar olarak saklanır. Veri türü olarak tanımlanan **integer** ise 2'nin tamamlayıcı formatında işaretli sayılar olarak saklanır. Bir tam sayının varsayılan genişliği en az 32 bittir.

for döngüsü, değişkenleri dizinlenmiş bir dizi ifade tarafından ima edilen işlemleri ifade etmenin kompakt bir yoludur. **for** döngüsü, iki noktalı virgül ile ayrılmış üç bölüm içerir:

- Bir başlangıç koşulu.
- Sonlandırma koşulunu kontrol eden bir ifade.
- Kontrol değişkenini değiştiren bir atama.

Aşağıda bir **for** döngüsünün örneği verilmiştir:

```
for (j = 0; j < 8; j = j + 1)
  begin
    // prosedürel ifadeler buraya gelir
  son
```

for döngüsü, prosedürel ifadelerin yürütülmesini sekiz kez tekrarlar. Kontrol değişkeni j, başlangıç koşulu j = 0'dır ve döngü, j = 8'den küçük olduğu sürece tekrarlanır. Döngü ifadesinin her yürütülmesinden sonra, j değeri 1 artırılır.

İki-üç-dört hat dekodерinin **for** döngüsü kullanılarak bir tanımlı HDL Örneği 8.1'de gösterilmiştir. Çıktı Y bir prosedürel ifade değerlendirildiğinden, **reg** türünde tanımlanmalıdır. Döngünün kontrol değişkeni **integer** k'dir. Döngü genişletildiğinde (açıldığında), aşağıdaki dört koşulu elde ederiz (IN ve Y ikili, Y için indeks ondalık olarak gösterilmektedir):

```
eğer GİRDİ = 00 ise Y(0) = 1; aksi takdirde Y(0) = 0;
eğer GİRDİ = 01 ise Y(1) = 1; aksi takdirde Y(1) = 0;
eğer GİRDİ = 10 ise Y(2) = 1; aksi takdirde Y(2) = 0;
eğer GİRDİ = 11 ise Y(3) = 1; aksi takdirde Y(3) = 0;
```

HDL Örneği 8.1 (Dekoder)

```
// döngü ifadesi kullanarak 2 x 4 dekodерin tanımlı
modül dekodер (GİRDİ, Y);
  girdi  [1: 0] GİRDİ;           // İki ikili girdi
  çıkış  [3: 0] Y;              // Dört ikili çıkış
0) reg [3:
  tam sayı                      k;           // Döngü için kontrol (indeks) değişkeni
```

Y;

```

her zaman @ (IN)
için (k = 0; k <= 3; k = k + 1)
eğer (IN == k) Y[k] = 1;
aksi takdirde Y[k] = 0;
endmodule

```

Mantık Sentezi

Mantık sentezi, bir bilgisayar tabanlı programın (yani, bir sentez aracı) bir mantık devresinin HDL modelini, kaynak kodu tarafından belirtilen işlemleri gerçekleştiren optimize edilmiş bir kapı netlistine dönüştürdüğü otomatik bir süreçtir. Sentezlenmiş tasarımı do nanımda uygulayan çeşitli hedef teknolojiler bulunmaktadır. Bir HDL tanımının etkili kul lanımı, tasarımcıların belirli sentez araçları için uygun olan satıcıya özgü bir stil benimse melerini gerektirir. Tasarımı uygulayan IC türü, bir uygulamaya özel entegre devre (ASIC), programlanabilir mantık cihazı (PLD) veya alan programlanabilir kapı dizisi (FPGA) ol abilir. Mantık sentezi, endüstride büyük devreleri verimli, doğru ve hızlı bir şekilde tasar lamak ve uygulamak için yaygın olarak kullanılmaktadır.

Mantık sentez araçları, HDL'nin kaynak kodunu yorumlar ve bunu optimize edilmiş kapı yapısına çevirir, Karnaugh haritalarını kullanarak manuel yöntemlerle yapılacak tüm işleri (doğru bir şekilde) gerçekleştirir. Mantık sentezi amacıyla yazılan dizaynlar genellikle kayıt transfer seviyesinde (RTL) olur.

Bu, RTL tanımında kullanılan HDL yapılarının, kapı seviyesinde bir tanıma doğrudan dönüştürülebilmesi nedeniyle. Aşağıdaki örnekler, bir mantık sentezleyicisinin bir HDL yapısını nasıl yorumlayabileceğini ve bunu bir kapı yapısına dönüştürebileceğini tartışmaktadır.

Sürekli atama (**assign**) ifadesi, kombinyonel devreleri tanımlamak için kullanılır. Bir HDL'de, bu bir mantık devresi için bir Boolean denklemini temsil eder. Atama ifadesinin sağ tarafında bir Boolean ifadesi ile sürekli bir atama, ifadenin uygulanmasını sağlayan gerekli kapı devresine sentezlenir.

Bir toplama operatörü (+) içeren bir ifade, tam toplayıcı devreleri kullanarak binary bir toplayıcı olarak yorumlanır. Bir çıkarma operatörü (–) içeren bir ifade, tam toplayıcılar ve özel-VE kapılarından oluşan bir kapı seviyesi çıkarıcıya dönüştürülür (Şekil 4.13). Koşullu bir operatör içeren bir ifade,

atan $Y = S ? In_1 : In_0;$

kontrol girişi S ve veri girişleri In_1

ve In_0 olan iki girişli bir çoklayıcıya dönüşür. Birden fazla koşullu operatör içeren bir ifade, daha büyük bir çoklayıcıyı belirtir.

Döngüsel bir davranış (**always** ...) olay kontrol ifadesinin seviye duyarlı mı yoksa ken ar duyarlı mı olduğuna bağlı olarak, bir kombinyonel veya sıralı devreyi ima edebilir. Bir sentez aracı, davranış içinde referans verilen her değişkene duyarlı olan bir seviye duyarlı döngüsel davranışı kombinyonel mantık olarak yorumlayacaktır (örneğin, bir atamanın sağ tarafında değişkenin görünmesiyle).

ifadeleri). Kombinasyonel mantığın bir tanımındaki olay kontrol ifadesi, herhangi bir sinyalin kenarına duyarlı olmayabilir. Örneğin,

```
her zaman @ (In_1 veya In_0 veya S) // Alternatif: (In_1, In_0, S)
eğer (S) Y = In_1;
aksi takdirde Y = In_0;
```

// Alternatif: (In_1, In_0, S)

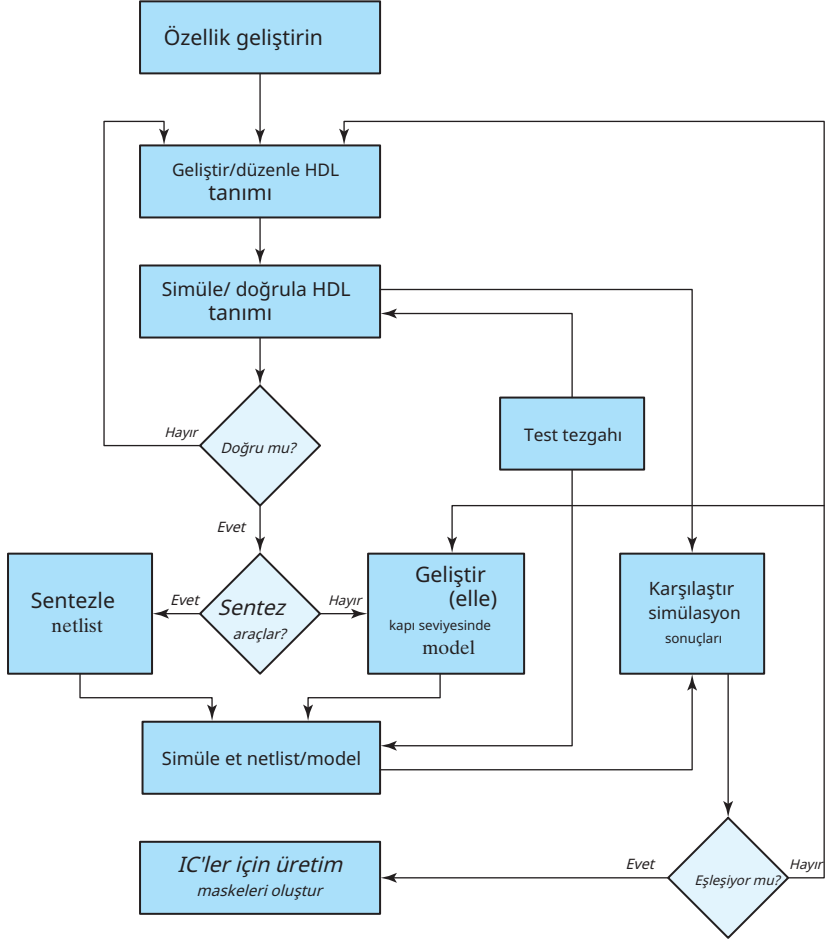
bir iki-tek hat multiplexere dönüştürülür. Alternatif olarak, **case** ifadesi büyük multiplexerlere işaret etmek için kullanılabilir. **case** ifadesi, x ve z mantık değerlerini, durum ifadesinde veya bir durum ögesinde göründüklerinde önemsiz (don't-cares) olarak ele alır.

Bir kenar-hassas döngüsel davranış (örneğin, **always @ (posedge clock)**) senkron (saatli) ardışık devreyi tanımlar. İlgili devrenin uygulanması, *D* flip-flopları ve olay kontrol ifadesiyle ilişkili ifadeler tarafından belirtilen senkron kayıt transfer işlemlerini gerçekleştiren kapılardan oluşur.

Bu tür devrelerin örnekleri kayıtlar ve sayaçlardır. Bir **case** ifadesi ile ardışık bir devre tanımı, *D* flip-flopları ve flip-floplara girişleri oluşturan kapılarla bir kontrol devresine dönüştürülür. Böylece, bir RTL tanımındaki her ifade sentezleyici tarafından yorumlanır ve karşılık gelen bir kapı ve flip-flop devresine atanır. Sentezlenebilir ardışık devreler için, olay kontrol ifadesi saatin (senkronizasyon sinyali) pozitif veya negatif kenarına hassas olmalıdır, ancak her ikisine de değil.

Endüstrinin dijital sistemleri tasarlamak için kullandığı sürecin basitleştirilmiş akış diyagramı Şekil 8.1'de gösterilmektedir. HDL tasarımının RTL tanımı simüle edilir ve doğru çalışıp çalışmadığı kontrol edilir. İşlevsel özellikleri, devrenin davranışı için belirtilenlerle eşleşmelidir. Test tezgahı, simülatöre uyarı sinyalleri sağlar. Simülasyonun sonucu tatmin edici değilse, HDL tanımı düzeltilir ve tekrar kontrol edilir. Simülasyonun çalışması geçerli bir tasarım gösterdiğinde, RTL tanımı mantık sentezleyici tarafından derlenmeye hazırdır. Tanımda bulunan tüm hatalar (sözdizimsel ve işlevsel) sentezden önce ortadan kaldırılmalıdır. Sentez aracı, model tarafından temsil edilen tasarımın kapı seviyesindeki tanımına eşdeğer bir netlist oluşturur. Model, spesifikasyonun işlevselliğini ifade edemezse, devre de bunu başaramayacaktır. Kapı seviyesi devresi, RTL tasarımını kontrol etmek için kullanılan aynı uyarı seti ile simüle edilir. Herhangi bir düzeltme gerekiyorsa, tatmin edici bir simülasyon elde edilene kadar süreç tekrarlanır. İki simülasyonun sonuçları karşılaştırılır ve eşleşip eşleşmediği kontrol edilir. Eğer eşleşmiyorlarsa, tasarımcı RTL tanımını değiştirerek tasarımdaki hataları düzeltmelidir. Ardından tanım, yeni bir kapı seviyesi tanımı oluşturmak için mantık sentezleyici tarafından tekrar derlenir. Tasarımcı, tüm simülasyon testlerinin sonuçlarından memnun kaldığında, devrenin tasarımı fiziksel uygulama için bir teknolojiye hazırdır. Pratikte, seçilen donanım teknolojisinde devrenin zamanlama spesifikasyonlarının karşılanıp karşılanmadığını doğrulamak için ek testler yapılacaktır. Bu konu, bu metnin kapsamı dışındadır.

Mantık sentezi tasarımcıya birkaç avantaj sağlar. Bir HDL tanımı yazmak ve kapı düzeyinde bir gerçekleştirme sentezlemek, devreyi şematik diyagramlardan manuel girişle geliştirmekten daha az zaman alır. Tanımın değiştirilmesinin kolaylığı, tasarım alternatiflerinin keşfini kolaylaştırır. Tasarımın geçerliliğini simülasyonla kontrol etmek, bir donanım prototipi üretmekten daha hızlı, daha kolay, daha az maliyetli ve daha az risklidir.



ŞEKİL 8.1

HDL tabanlı modelleme, doğrulama ve sentez için basitleştirilmiş bir akış diyagramı

Entegre devreyi üretmek için bir şematik ve veritabanı sentez araçları tarafından otomatik olarak oluşturulabilir. HDL modeli, farklı araçlar tarafından farklı teknolojiye (örneğin, ASIC hücreleri veya FPGA'lar) derlenebilir ve modelin oluşturulması için yapılan yatırımlara çoklu geri dönüş sağlar.

8.4 ALGORİTMİK DURUM MAKİNELERİ (ASM'ler)

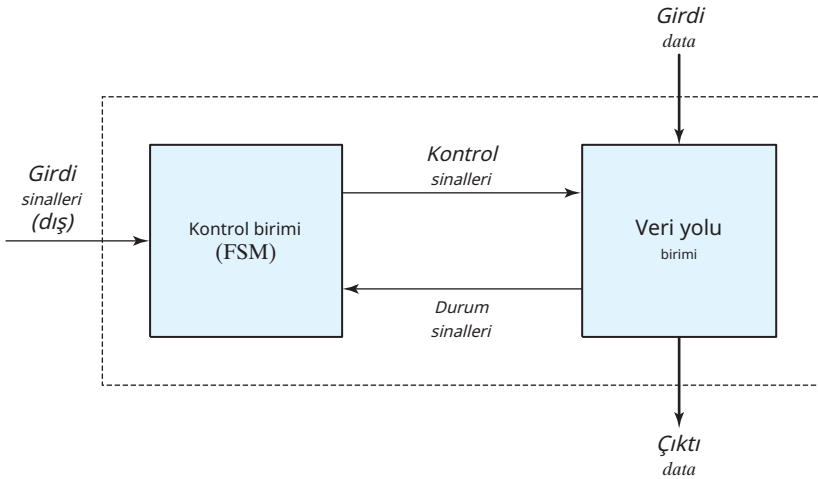
Dijital bir sistemde depolanan ikili bilgi, veri veya kontrol bilgisi olarak sınıflandırılabilir. Veriler, aritmetik, mantık, kaydırma ve diğer benzer veri işleme işlemleri gerçekleştirilerek işlenen bilgi (ikili kelimeler) ayrık unsurlardır.

işlemler. Bu işlemler, toplama devreleri, kod çözücüler, çoklayıcılar, sayaçlar ve kaydırma kayıtları gibi dijital donanım bileşenleri ile uygulanır. Kontrol bilgisi, makinenin veri bölümündeki çeşitli işlemleri koordine eden ve yürüten komut sinyalleri sağlar, böylece istenen veri işleme görevlerini yerine getirir.

Dijital bir sistemin mantığının tasarımı iki ayrı çabaya bölünebilir. Bir kısım, veri işleme işlemlerini gerçekleştiren dijital devrelerin tasarımı ile ilgilidir. Diğer kısım ise, verilerin çeşitli manipülasyonlarının hangi sırayla gerçekleştirileceğini belirleyen kontrol devrelerinin tasarımı ile ilgilidir.

Bir dijital sistemde kontrol mantığı ile veri işleme işlemleri arasındaki ilişki Şekil 8.2'de gösterilmektedir. Veri işleme yolu, genellikle *veri yolu birimi* olarak adlandırılır, sistemin gereksinimlerine göre kayıtlar içindeki verileri manipüle eder. Kontrol birimi, veri yolu birimine bir dizi komut gönderir. Veri yolu biriminden kontrol birimine giden bir iç geri bildirim yolunun, kontrol biriminin kontrol sinyalleri (kontrol biriminin çıktıları) dizisini belirlemek için dış (birincil) girişlerle birlikte kullandığı durum koşullarını sağladığını unutmayın. Bu geri bildirim ilişkisinin bir HDL ile nasıl modellenmesi gerektiğini anlamanın çok önemli olduğunu daha sonra göreceğiz.

Veri yolu birimindeki işlemlerin sıralanması için sinyalleri üreten kontrol mantığı, sonlu durum makinesi (FSM), yani senkron sıralı devredir. Sistemin kontrol komutları, birincil girişler, durum sinyalleri ve makinenin durumu fonksiyonları olarak FSM tarafından üretilir. Belirli bir durumda, kontrolörün çıktıları veri yolu biriminin girişleridir ve hangi işlemleri gerçekleştireceğini belirler. Durum koşullarına ve diğer dış girişlere bağlı olarak, FSM bir sonraki duruma geçerek diğer işlemleri başlatır. Kontrol mantığı olarak işlev gören dijital devreler, veri yolundaki işlemleri başlatmak için bir zaman dizisi sinyali sağlar ve ayrıca kontrol alt sisteminin bir sonraki durumunu belirler.



ŞEKİL 8.2
Kontrol ve veri yolu etkileşimi

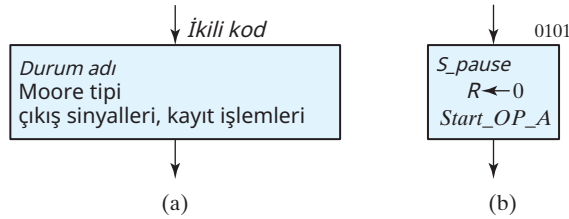
Bir dijital sistemin kontrol dizisi ve veri yolu görevleri, bir donanım algoritması aracılığıyla belirlenir. Bir algoritma, bir probleme çözüm elde etmenin nasıl yapılacağını belirten sonlu sayıda prosedürel adımdan oluşur. Bir donanım algoritması, belirli bir ekipmanla problemi çözmek için bir prosedürdür. Dijital tasarımın en zorlu ve yaratıcı kısmı, gerekli hedeflere ulaşmak için donanım algoritmalarının formüle edilmesidir. Amaç, algoritmaları entegre devre olarak silikona uygulamaktır.

Bir akış diyagramı, bir algoritmanın prosedürel adımlarının ve karar yollarının sırasını belirtmek için uygun bir yoldur. Bir donanım algoritması için bir akış diyagramı, sözlü talimatları, işlemlerin sırasını ve bunların yürütülmesi için gerekli koşulları listeyen bir bilgi diyagramına çevirir. Bir algoritmik durum makinesi (ASM) diyagramı, dijital donanımda yürütme için algoritmaları özel olarak tanımlamak üzere geliştirilmiş bir akış diyagramıdır. Bir durum makinesi, bir dijital sistemin temel yapısı olan ardışık devre için başka bir terimdir.

ASM Şeması

ASM şeması, geleneksel bir akış şemasına benzer, ancak biraz farklı bir şekilde yorumlanır. Geleneksel bir akış şeması, bir algoritmanın prosedürel adımlarını ve karar yollarını ardışık bir şekilde tanımlar, zaman ilişkisini dikkate almadan. ASM şeması, olayların sırasını, yani olayların zaman içindeki sıralamasını ve bir ardışık kontrolörün durumları ile bir durumdan diğerine geçerken meydana gelen olaylar arasındaki zamanlama ilişkisini tanımlar (yani, durum değişiklikleriyle senkronize olan olaylar). Şema, dijital sistemde kontrol sırasını ve veri yolu işlemlerini doğru bir şekilde belirtmek için dijital donanımın kısıtlamalarını dikkate alarak uyarlanmıştır.

ASM şeması, üç temel unsurdan oluşur: durum kutusu, karar kutusu ve koşul kutusu. Kutular, makine çalışırken durumların ardışık önceliğini ve evrimini gösteren yönlendirilmiş kenarlarla birbirine bağlanır. ASM şemasına bilgi eklemenin çeşitli yolları vardır. Birinde, kontrol sırasındaki bir durum, Şekil 8.3 (a)'da gösterildiği gibi bir durum kutusu ile belirtilir. Durum kutusunun şekli, içinde kontrolün belirtilen durumda bulunurken ürettiği kayıt işlemleri veya çıkış sinyallerinin adlarının yazılı olduğu bir dikdörtgendir. Duruma, kutunun sol üst köşesine yerleştirilen sembolik bir ad verilir. Duruma atanan ikili kod, sağ üst köşeye yerleştirilir. (Durum sembolü ve kodu yerleştirilebilir)



ŞEKİL 8.3

ASM şeması durum kutusu

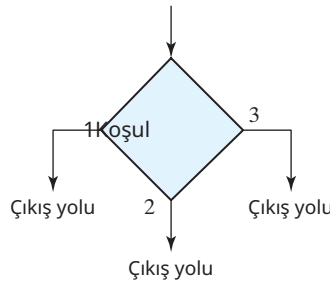
diğer yerlerde de.) Şekil 8.3 (b) bir durum kutusuna örnek vermektedir. Durum un sembolik adı S_{pause} olup, ona atanan ikili kod 0101'dir. Kutunun içinde yazılı olan kayıt işlemi $R \leftarrow 0$, kayıt R 'nin 0'a sıfırlanacağını belirtmektedir. Kutunun içindeki $Start_OP_A$ adı, örneğin, makine S_{pause} durumundayken etkin olan ve veri yolu biriminde belirli bir işlemi başlatan bir Moore tipi çıkış sinyalini göstermektedir.

Şekil 8.3 (b)'de gösterilen durum kutusu tarzı bazen ASM grafiklerinde kullanılmaktadır, ancak bu, kayıt işlemi $R \leftarrow 0$ 'ın ne zaman gerçekleştirileceği konusunda kafa karışıklığına yol açabilir. İşlem durum kutusunun içinde yazılı olsa da, aslında makinenin S_{pause} durumundan bir sonraki duruma geçiş yaptığı zaman gerçekleşir. Aslında, durum kutusunun içinde kayıt işlemini yazmak, kontrolörün makine durum değiştirdiğinde kayıt işleminin gerçekleşmesini sağlayacak bir sinyali etkinleştirmesi gerektiğini belirtmenin bir yoludur (her ne kadar kafa karıştırıcı olabilir).

Daha sonra dijital tasarıma daha uygun bir grafik ve notasyon tanıtacağız ve bu, bir durum makinesi tarafından kontrol edilen kayıt işlemleriyle ilgili herhangi bir belirsizliği ortadan kaldıracaktır.

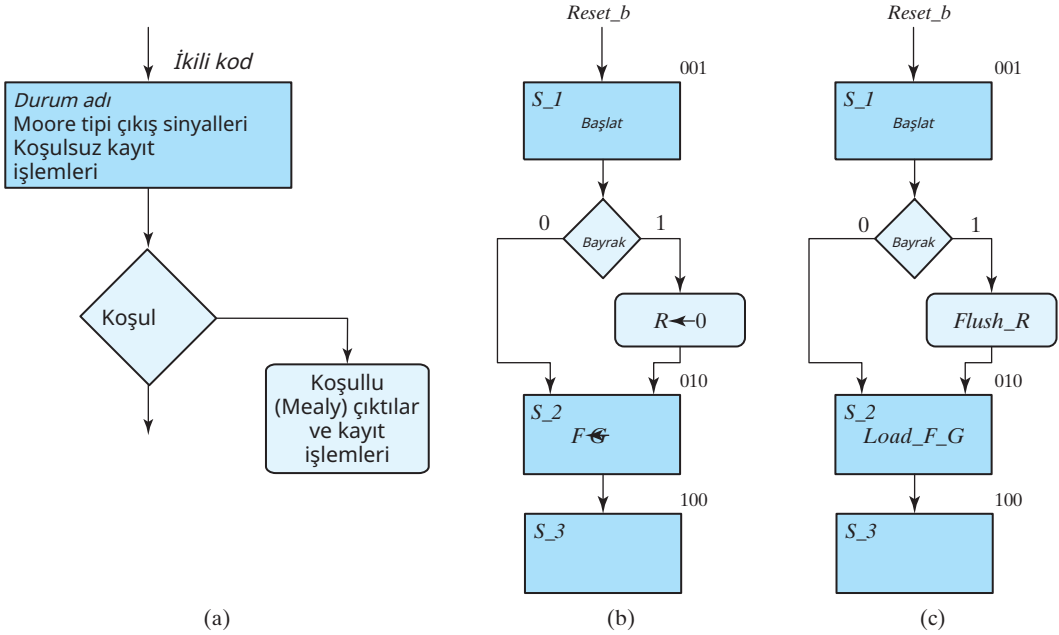
Bir ASM grafiğinin karar kutusu, bir girişin (yani, birincil veya harici bir giriş ya da durum veya iç sinyal) kontrol altındaki alt sistem üzerindeki etkisini tanımlar. Kutu elmas şeklindedir ve iki veya daha fazla çıkış yolu vardır; bu, Şekil 8.4'te gösterilmiştir. Test edilecek giriş koşulu kutunun içine yazılır. Koşulun değerlendirilmesine bağlı olarak bir çıkış yolu seçilir. İkili durumda, koşul doğruysa bir yol, yanlışsa başka bir yol seçilir. Bir giriş koşuluna ikili bir değer atandığında, ikili yol sırasıyla 1 ve 0 ile gösterilir.

Bir ASM grafiğinin durum ve karar kutuları, geleneksel akış şemalarında kullanılanlarla benzerdir. Üçüncü unsur, koşullu kutu, ASM grafiğine özgüdür. Koşullu kutunun şekli Şekil 8.5 (a)'da gösterilmiştir. Yuvarlak köşeleri, onu durum kutusundan ayırır. Koşullu kutuya giden giriş yolu, bir karar kutusunun çıkış yollarından birinden gelmelidir. Koşullu kutunun içinde listelenen çıktılar, belirli bir durumda Mealy tipi sinyaller olarak üretilir; koşullu kutuda listelenen kayıt işlemleri, durumdan geçişle ilişkilidir. Şekil 8.5 (b), bir koşullu kutu ile bir örneği göstermektedir. Kontrol, durum S_1 'deyken $Start$ çıkış sinyalini üretir ve $Flag$ girişinin durumunu kontrol eder. Eğer $Flag = 1$ ise, R 0'a sıfırlanır; aksi takdirde, R değişmeden kalır. Her iki durumda da, bir sonraki durum S_2 . Bir kayıt işlemi



ŞEKİL 8.4

ASM şeması karar kutusu



ŞEKİL 8.5

ASM şeması koşullu kutu ve örnekler

S₂ ile ilişkilidir. Yine belirtmek gerekir ki bu tür bir şema kafa karışıklığına neden olabilir, çünkü durum makinesi, S₁ durumunda iken belirtilen kayıt işlemi R ← 0'ı gerçekleştirmez veya S₂ durumunda iken FdG işlemini gerçekleştirmez. Notasyon altında, kontrolör S₁ durumundayken, kayıt işlemi R ← 0'ın veri yolu biriminde¹ gerçekleştirilmesi için bir Mealy tipi sinyal göndermesi gerektiğini belirtir, Flag = 0 koşuluna tabi olarak. Benzer şekilde, S₂ durumunda, kontrolör, veri yolu biriminde FdG işleminin gerçekleştirilmesine neden olan bir Moore tipi çıkış sinyali üretmelidir. Veri yolu birimindeki işlemler, durumun S₁'den S₂'ye ve S₂'den S₃'ye geçmesini sağlayan saat kenarına senkronize edilmiştir. Böylece, belirli bir durumdaki kontrol sinyali, bir sonraki saat geçişi gerçekleştiğinde veri yolundaki bir kaydın işlemini etkiler. İşlemin sonucu bir sonraki durumda belirgindir.

ASM grafiği Şekil 8.5 (b)'de veri yolunun ve kontrolcünün tanımlarını karıştırmaktadır. Sadece kontrolcü için bir ASM grafiği Şekil 8.5 (c)'de gösterilmektedir; burada kayıt işlemleri çıkarılmıştır. Yerine, veri yolu biriminin işlemlerini başlatmak için kontrol birimi tarafından üretilmesi gereken kontrol sinyalleri bulunmaktadır. Bu grafik, kontrolcüyü tanımlamak için faydalıdır, ancak veri yolu hakkında yeterli bilgi içermemektedir.

(Bu konuyu daha sonra ele alacağız.)

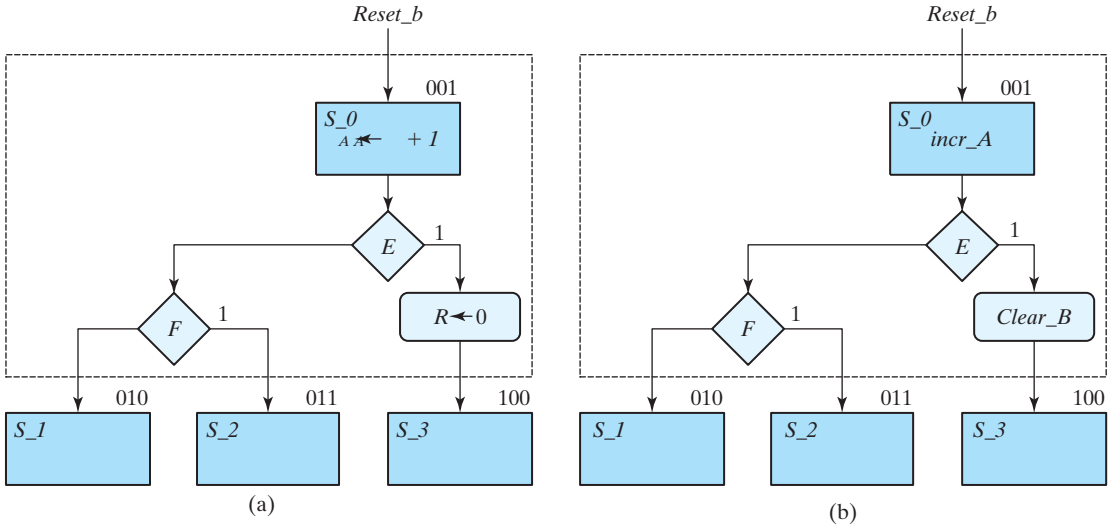
¹ Eğer yol bir durum kutusundan geldiyse, iddia edilen sinyaller yalnızca durum ve kutu içinde listelenmelidir.

ASM Bloğu

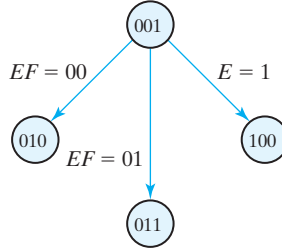
ASM bloğu, bir durum kutusu ve çıkış yoluna bağlı tüm karar ve koşul kutularında n oluşan bir yapıdır. Bir ASM bloğunun bir girişi ve karar kutularının yapısıyla temsil edilen herhangi sayıda çıkış yolu vardır. Bir ASM şeması, bir veya daha fazla bir birine bağlı bloktan oluşur. Bir ASM bloğunun örneği Şekil 8.6'da verilmiştir. Durum ile ilişkili S_0 iki karar kutusu ve bir koşul kutusudur. Diyagram, bloğu tüm yapı etrafında kesik çizgilerle ayırt eder, ancak bu genellikle yapılmaz, çünkü ASM şeması her bloğu yapısından benzersiz bir şekilde tanımlar. Karar veya koşul kutusu olmayan bir durum kutusu basit bir bloğu oluşturur.

ASM şemasındaki her blok, sistemin bir saat darbesi aralığındaki durumunu tanımlar (yani, saat sinyalinin iki ardışık aktif kenarı arasındaki aralık). Şekil 8.6 (a)'da ki durum ve koşul kutularındaki işlemler, denetleyicinin durumu S_0 durumunda n bir sonraki duruma geçerken ortak bir saat darbesi ile başlatılır. Aynı saat darbesi, sistem denetleyicisini bir sonraki durumlardan birine, S_1 , S_2 , veya S_3 , ikili değerler tarafından belirlenen E ve F . Kontrolör için ASM diyagramı yalnızca Şekil 8.6 (b)'de gösterilmiştir. Moore tipi sinyal $incr_A$ koşulsuz olarak makine S_0 durumundayken etkinleştirilir; Mealy tipi sinyal $Clear_R$ ise durum S_0 olduğunda ve E etkinleştirildiğinde koşullu olarak üretilir. Genel olarak, kontrolörün Moore tipi çıktıları koşulsuz olarak üretilir ve bir durum kutusu içinde gösterilir; Mealy tipi çıktılar koşullu olarak üretilir ve bir karar kutusunu terk eden kenarla ara bağlı koşullu kutularda gösterilir.

ASM diyagramı, bir durum geçiş diyagramına benzer. Her durum bloğu, bir ardışık devredeki bir duruma eşdeğerdir. Karar kutusu, ikili bilgiye eşdeğerdir.



ŞEKİL 8.6
ASM blokları



ŞEKİL 8.7

Şekil 8.6'nın ASM diyagramına eşdeğer durum diyagramı

bir durum diyagramında iki durumu bağlayan yönlendirilmiş hatlar boyunca yazılmıştır. Sonuç olarak, bazen grafiği bir durum diyagramına dönüştürmek ve ardından kontrol mantığını tasarlamak için ardışık devre prosedürlerini kullanmak uygun olur. Örnek olarak, Şekil 8.6'daki ASM grafiği, Şekil 8.7'de bir durum diyagramı olarak çizilmiştir (çıkışlar atlanmıştır). Durumlar, içlerinde ikili değerleri yazılı dairelerle sembolize edilmiştir. Yönlendirilmiş hatlar, bir sonraki durumu belirleyen koşulları gösterir. Veri yolu biriminde gerçekleştirilmesi gereken koşulsuz ve koşullu işlemler durum diyagramında belirtilmemiştir.

Sadeleştirmeler

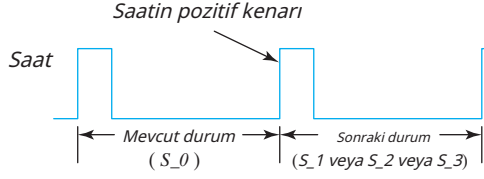
ASM şemasının ikili karar kutusu, yalnızca belirtilen karar değişkenine karşılık gelen kenarı etiketleyerek ve diğer kenarı etiketsiz bırakarak sadeleştirilebilir. Bir diğer sadeleştirme, sıfırlama koşulu sağlandığında meydana gelen durum geçişlerine karşılık gelen kenarların atlanmasıdır. Belirtilmeyen çıkış sinyalleri şemada gösterilmez; bir çıkış sinyalinin adı varsa, bu sinyalin belirtildiği anlamına gelir.

Zamanlama Dikkate Alınması

Dijital bir sistemdeki tüm kayıtlar ve flip-floplar için zamanlama, bir ana saat üretici tarafından kontrol edilir. Saat darbeleri yalnızca veri yolundaki kayıtlara değil, aynı zamanda kontrol birimini uygulayan durum makinesindeki tüm flip-floplara da uygulanır. Girişler de saate senkronize edilir, çünkü genellikle aynı saat sinyallerini kullanan başka bir devrenin çıkışları olarak üretilirler. Giriş sinyali, saatin bağımsız olarak rast gele bir zamanda değişirse, buna asenkron giriş denir. Asenkron girişler çeşitli sorunlara neden olabilir. Tasarımı sadeleştirmek için, tüm girişlerin saatle senkronize olduğunu ve bir kenar geçişine yanıt olarak durum değiştirdiğini varsayacağız.

Bir geleneksel akış diyagramı ile bir ASM diyagramı arasındaki en büyük fark, çeşitli işlemler arasındaki zaman ilişkisini yorumlamada yatmaktadır. Örneğin, Şekil 8.6 bir geleneksel akış diyagramı olsaydı, listelenen işlemler birbirini takip eden bir sıra olarak kabul edilecekti: Önce kayıt A artırılır ve ancak o zaman E değerlendirilir.

E = 1 ise, kayıt R temizlenir ve kontrol durumu S₃'e geçer. Aksi takdirde (E = 0 ise), bir sonraki adım F'yi değerlendirmek ve S₁ veya S₂ durumuna geçmektir. Buna karşılık, bir ASM diyagramı tüm bloğu birim olarak değerlendirir. Bloğun içinde belirtilen tüm kayıt işlemleri, sistemin S₀'dan bir sonraki duruma geçerken aynı saat darbesinin kenar geçişinde senkronize olarak gerçekleşmelidir.



ŞEKİL 8.8

Durumlar arasındaki geçiş

Bloğun içindeki tüm işlemler, sistem S_0'dan bir sonraki duruma geçerken aynı saat darbe sinin kenar geçişinde senkronize olarak gerçekleşmelidir. Bu olaylar dizisi, Şekil 8.8'de görse olarak sunulmuştur. Bu illüstrasyonda, tüm flip-flopların pozitif kenar tetiklenmesini var sayıyoruz. Bir etkin asenkron sıfırlama sinyali (reset_b), kontrol devresini S_0 durumuna geçirir. S_0 durumundayken, kontrol devreleri E ve F girişlerini kontrol eder ve uygun sinyalleri buna göre üretir. Eğer reset_b etkin değilse, aşağıdaki işlemler saat sinyalinin bir sonraki pozitif kenarında aynı anda gerçekleşir:

Kayıt *A* artırılır.

Eğer $E=1$ ise, kayıt *R* temizlenir.

Kontrol, Şekil 8.7'de belirtilen bir sonraki duruma aktarılır.

Veriyolu üzerindeki iki işlemin ve kontrol mantığındaki durum değişikliğinin aynı anda gerçekleştiğini unutmayın. Ayrıca, Şekil 8.6 (a)'daki ASM grafiğinin veriyolu biriminde gerçekleşmesi gereken kayıt işlemlerini gösterdiğini, ancak kontrol birimi tarafından oluşturulacak kontrol sinyalini belirtmediğini de unutmayın. Tersine, Şekil 8.6 (b)'deki grafik kontrol sinyallerini gösterirken, veriyolu işlemlerini göstermez. Şimdi, mantık tasarımcılarının ihtiyaç duyduğu netlik ve tam bilgiyi sağlamak için bir ASMD grafiği sunacağız.

ASMD Grafiği

Algoritmik durum makinesi ve veriyolu (ASMD) grafikleri, ASM grafiklerinin sunduğu bilgiyi netleştirmek ve belirli bir veriyolu birimi için bir kontrol birimi tasarlamak için etkili bir araç sağlamak amacıyla geliştirilmiştir. Bir ASMD grafiği, üç önemli şekilde bir ASM grafiğinden farklıdır: (1) Bir ASMD grafiği, bir durum kutusu içinde kayıt işlemlerini listelemez, (2) bir ASMD grafiğinin kenarları, kenarın gösterdiği durum geçişi ile eşzamanlı olan kayıt işlemleri ile not edilmiştir ve (3) bir ASMD grafiği, grafiğin kenarlarını not eden kayıt işlemlerini kontrol eden sinyalleri tanımlayan koşullu kutuları içerir. Böylece, *bir ASMD grafiği kayıt işlemlerini ilişkilendirir. durum geçişleri ile değil, durumlarla*; ayrıca, kayıt işlemlerini onları tetikleyen sinyallerle ilişkilendirir. Sonuç olarak, bir ASMD diyagramı, karmaşık bir dijital makinenin veri yolu ve kontrol birimlerine ayrılmasını temsil eder ve aralarındaki ilişkiyi açıkça gösterir. Kayıt işlemlerinin zamanlaması veya onları başlatan sinyaller hakkında karışıklık için yer yoktur.

Tasarımcılar, bir veri yolu biriminin kontrolörü için açıklamalı ve tamamen tanımlanmış bir ASM diyagramı oluşturan üç aşamalı bir süreçte bir ASMD diyagramı oluştururlar.

ASMD grafiği oluşturma adımları şunlardır:

Yalnızca denetleyicinin durumlarını ve durum geçişlerine neden olan giriş sinyallerini gösteren bir ASM grafiği oluşturun²

ASM grafiğini, veri yolu biriminin eşzamanlı kayıt işlemlerini belirtmek için ASM grafiğinin kenarlarını not ederek ASMD grafiğine dönüştürün (yani, bir durum geçişi ile eşzamanlı olan kayıt işlemleri) ve

ASMD grafiğini, denetleyici tarafından üretilen ve veri yolu biriminde belirtilen işlemlere neden olan kontrol sinyallerini tanımlayacak şekilde değiştirin.

Bu süreç tarafından üretilen ASMD grafiği, kontrolörün sonlu durum makinesini açık ve eksiksiz bir şekilde tanımlar, veri yolu biriminin kayıt işlemlerini belirler, veri yolunun kontrolöre durumunu bildiren sinyalleri tanımlar ve kayıt işlemlerini kontrol eden sinyallere bağlar.

Bir durum makinesinin önemli bir kullanımı, bir kontrolör ve bir veri yoluna bölünmüş bir ardışık makededeki kayıt işlemlerini kontrol etmektir. Bir ASMD grafiği, kontrolörün ASM grafiğini kontrol ettiği veri yoluna bağlar ve bu, tüm senkron dijital donanım tasarımını temsil eden evrensel bir model olarak hizmet eder. ASMD grafikleri, bir ardışık makinenin tasarımını, veri yolunun tasarımını kontrolörün tasarımından ayırarak net bir ilişkiyi korurken netleştirmeye yardımcı olur. Durum geçişleriyle eşzamanlı olarak gerçekleşen kayıt işlemleri, bu kayıtların kontrolörün bir parçası olmaması nedeniyle, grafiğin bir yolunda not edilir, durum kutularında veya yol üzerindeki koşul kutularında değil. Kontrolör tarafından üretilen çıktılar, veri yolunun kayıtlarını kontrol eden sinyallerdir ve ASMD grafiğinde not edilen kayıt işlemlerini tetikler.

8.5 TASARIM ÖRNEĞİ (ASMD ŞEMASI)

Şimdi ASMD şemasının ve kayıt transferi temsilinin kullanımını gösteren basit bir örnek sunacağız. Bir sistemin başlangıç spesifikasyonlarından başlayarak, dijital donanımın tasarlandığı uygun bir ASMD şemasının geliştirilmesine geçiyoruz.

Veri yolu birimi, iki JK flip-flop E ve F ile bir dört bitlik ikili sayıcı $A[3:0]$ 'dan oluşacaktır. A içindeki bireysel flip-flop'lar A_3 , A_2 , A_1 ve A_0 olarak adlandırılır ve A_3 , sayımın en anlamlı bitini tutar. *Start* sinyali, sayıcıyı A ve flip-flop'u F temizleyerek sistemin çalışmasını başlatır. Her bir sonraki saat darbesinde, sayıcı 1 artırılır ve işlemler durana kadar devam eder. Sayıcı bitleri A_2 ve A_3 işlemlerin sıranı belirler:

Eğer $A_2=0$ ise, E 0'a temizlenir ve sayım devam eder.

Eğer $A_2=1$ ise, E 1'e ayarlanır; ardından, eğer $A_3=0$ ise, sayım devam eder, ancak eğer $A_3=1$ ise, F bir sonraki saat darbesinde 1'e ayarlanır ve sistem saymayı durdurur.

² Genel olarak, kontrol biriminin girdileri, dış (birincil) girdiler ve

veri yolu biriminden kaynaklanan durum sinyalleridir.

O zaman, eğer $Başlat = 0$ ise, sistem başlangıç durumunda kalır, ancak eğer $Başlat = 1$ ise, işlem döngüsü tekrarlanır.

Sistemin mimarisinin blok diyagramı Şekil 8.9 (a)'da gösterilmiştir; burada (1) veri yolu biriminin kayıtları, (2) dış (birincil) giriş sinyalleri, (3) veri yolu biriminden kontrol birimine geri beslenen durum sinyalleri ve (4) kontrol birimi tarafından üretilen ve veri yolu birimine giren kontrol sinyalleri bulunmaktadır. Kontrol sinyallerinin adlarının, veri yolu biriminde yürütülmesini sağladıkları işlemleri açıkça belirttiğini unutmayın. Örneğin, clr_A_F kayıtları A ve F temizler. Sinyalin adı $reset_b$ (alternatif olarak, $reset_bar$) sıfırlama eyleminin aktif düşük olduğunu belirtir. Her bir birimin iç detayları gösterilmemiştir.

ASMD Grafiği

Sistem için bir ASMD diyagramı, asenkron sıfırlama eylemi için Şekil 8.9 (b)'de ve senkron sıfırlama eylemi için Şekil 8.9 (c)'de gösterilmiştir. Diyagram, kontrolörün durum geçişlerini ve bu geçişlerle ilişkili veri yolu işlemlerini göstermektedir. Diyagram, kontrolör tarafından üretilen kontrol sinyallerini tanımlamadığı için nihai formunda değildir. Engelleyici olmayan Verilog operatörü ($\leftarrow$) ok yerine gösterilmektedir ($\leftarrow$) kayıt aktarım işlemleri için çünkü nihayetinde bir Verilog

tanımı yazmak için ASMD diyagramını kullanacağız.

Sıfırlama eylemi senkron olduğunda, sıfırlama durumuna geçiş saatle senkronizedir. Bu geçiş, diyagramda S_idle için gösterilmektedir, ancak diğer tüm senkron sıfırlama yolları açıklık için hariç tutulmuştur. Sistem, S_idle durumunda kalır, Start belirtildiye kadar. Bu olduğunda (yani, $Start = 1$), durum S_I 'ye geçer. Bir sonraki saat kenarında, A_2 ve A_3 (öncelik sırasına göre çözümlenen) değerlerine bağlı olarak, durum S_I veya S_2 'ye geri döner. S_2 'den, S_idle 'ye koşulsuz olarak geçer, burada bir başka Start belirtmesini bekler.

Diyagramın kenarları, saat (yani, senkronize) kenarında meydana gelen durum geçişlerini temsil eder (örneğin, yükselen kenar) ve veri yolunda gerçekleşecek kayıt işlemleri ile not edilmiştir. $Start$ S_idle 'de belirtildiğinde, durum S_I 'ye geçecektir ve kayıtlar A ve F temizlenecektir. Şunu unutmayın ki, bir kayıt işlemi bir durum kutusunu terk eden kenar üzerinde not edilmişse, işlem koşulsuz olarak gerçekleşir ve bir Moore tipi sinyal tarafından kontrol edilir.

Örneğin, kayıt A her saat kenarında artırılırken

makinanın S_I durumunda olduğu sürede. Öte yandan, kayıt işlemi kayıt

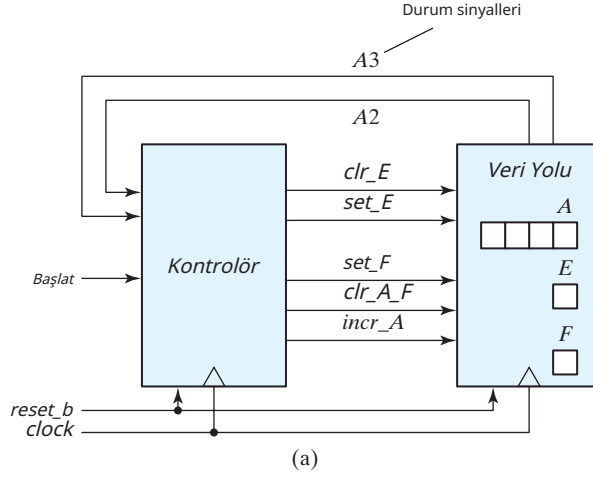
E , karar kutusundan çıkış kenarını A_2 için not eder. İşlemi kontrol eden sinyal, sistem S_I durumundayken ve A_2 değeri 1 olduğunda etkinleşen bir Mealy tipi sinyal olacaktır. Benzer şekilde, A ve F sinyallerini temizleyen kontrol sinyali koşullu olarak etkinleşir: Sistem S_idle durumundadır ve $Start$ etkinleşmiştir.

S_I durumunda sayacın artırıldığını göstermekle birlikte, not edilmiş yollar, diğer işlemlerin de aynı saat kenarıyla koşullu olarak gerçekleştiğini göstermektedir:

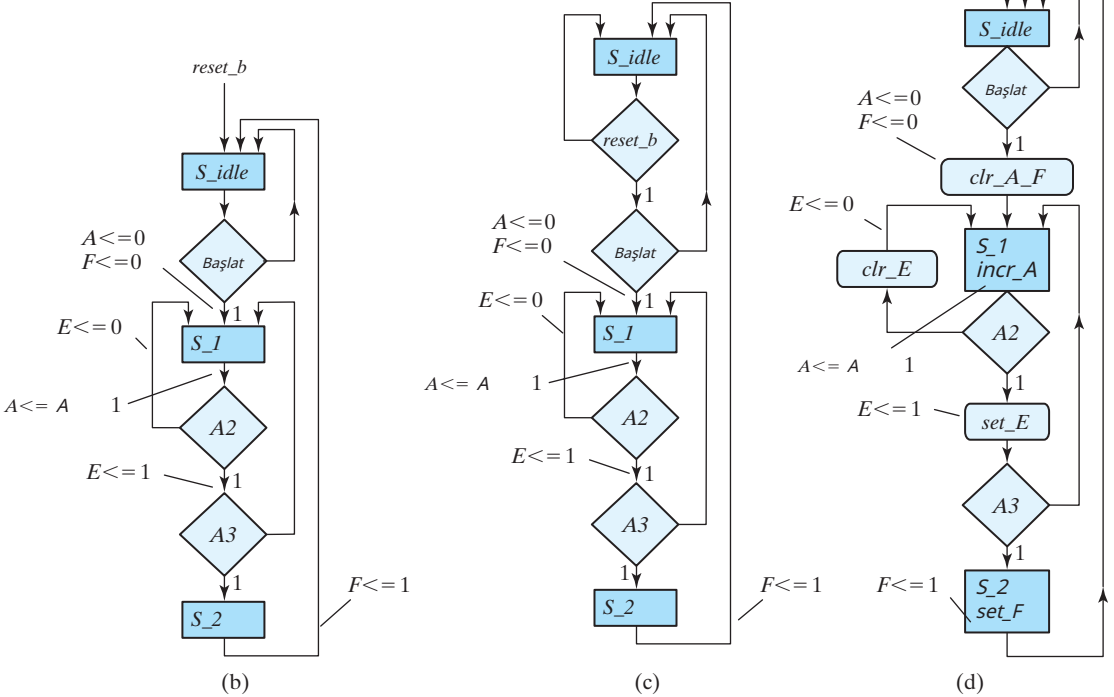
Ya E temizlenir ve kontrol S_I durumunda kalır ($A_2=0$) ya da

E ayarlanır ve kontrol S_I durumunda kalır ($A_2A_3=10$) ya da

E ayarlanır ve kontrol S_2 durumuna geçer ($A_2A_3=11$).



Not: A3, A[3]'ü; A2, A[2]'yi belirtir,
 $\leq$, engellemeyen atamayı belirtir
 reset_b, aktif-düşük sıfırlama koşulunu belirtir



ŞEKİL 8.9

(a) Tasarım örneği için blok diyagramı

(b) Kontrolör durum geçişleri için ASMD grafiği, asenkron sıfırlama

(c) Kontrolör durum geçişleri için ASMD grafiği, senkron sıfırlama

(d) Tamamen tanımlanmış bir kontrolör için ASMD grafiği, asenkron sıfırlama

Kontrol S_2 durumundayken, flip-flop'u ayarlamak için bir Moore tipi kontrol sinyali verilmelidir. F_1 'e ve durum, saatin bir sonraki aktif kenarında S_{idle} durumuna geri döner.

ASMD grafiğini oluşturmanın üçüncü ve son adımı, kontrolör tarafından üretilen sinyaller için koşullu kutular eklemek veya durum kutularına Moore tipi sinyaller eklemektir. Bu, Şekil 8.9 (d)'de gösterildiği gibidir. Sinyal clr_A_F , durum S_{idle} 'de *koşullu olarak üretilir*, $incr_A$ koşulsuz olarak S_1 'de *üretilir*, clr_E ve set_E koşullu olarak üretilir in S_1 , ve set_F koşulsuz olarak S_2 'de üretilir. ASM diyagramı üç duruma ve üç bloğa sahiptir. S_{idle} ile ilişkili blok, bir durum kutusu, bir karar bloğu ve bir koşullu kutudan oluşur. S_2 ile ilişkili blok yalnızca durum kutusundan oluşur. $clock$ ve $reset_b$ 'ye ek olarak, kontrol mantığında bir dış giriş, $Start$, ve dört durum girişi, A_2 ve A_3 .

Bu örnekte, bir tasarımın sözlü (metin) tanımının (şartnamesinin) nasıl bir ASMD grafiğine çevrildiğini gösterdik. Bu grafik, veri yolunun kontrolcüsünü tamamen tanımlar, kontrol sinyallerini ve bunlara bağlı kayıt işlemlerini belirtir. Bu tasarım örneği mutlaka pratik bir uygulamaya sahip değildir ve genel olarak, yorumlamaya bağlı olarak, kontrolcü için üç aşamalı tasarım süreciyle üretilen ASMD grafiği basitleştirilebilir ve farklı bir şekilde formüle edilebilir. Ancak, ASMD grafiği oluşturulduktan sonra, devre tasarım süreci oldukça basittir.

Pratikte, tasarımcılar ASMD grafiğini kullanarak kontrolcünün ve veri yolunun Verilog modellerini yazarlar ve ardından Verilog tanımından doğrudan bir devre sentezlerler. Öncelikli sistemi manuel olarak tasarlayacağız ve ardından HDL tanımını yazacağız, sentezi sentez araçlarına erişimi olanlar için isteğe bağlı bir adım olarak tutacağız.

Zamanlama Sırası

Bir ASMD grafiğindeki her blok, bir ortak saat darbesiyle başlatılacak işlemleri kontrol eden sinyalleri belirtir. Bloktaki durum ve koşul kutularında belirtilen kontrol sinyalleri, kontrolcünün belirtilen durumda olduğu sırada oluşur ve not edilmiş işlemler, durumun durumdan çıkış kenarı boyunca geçiş yaptığı zaman veri yolu biriminde gerçekleşir. Bir durumdan diğerine geçiş, kontrol mantığında gerçekleştirilir. İlgili zamanlama ilişkisini anlamak için, her saat kenarından sonra adım adım işlemler sırasını listeleyeceğiz, sinyalin $Start$ doğrulanmasıyla başlayarak sistemin sıfırlama (ilk) durumuna, S_{idle} geri dönene kadar.

Tablo 8.3, her saat darbesinden sonra sayacın ve iki flip-flopun ikili değerleri ni göstermektedir. Tablo ayrıca A_2 ve A_3 'ün durumunu ayrı olarak ve kontrolörün mevcut durumunu göstermektedir. Giriş sinyali $Start$, sayacı ve flip-flop F_1 'i temizlediği anda S_1 durumuyla başlıyoruz. Makinenin S_{idle} durumuna sıfırlama koşulundan girmeden önce çalıştığını varsayacağız.

Bu nedenle, E 'nin değeri 1 olarak varsayılmaktadır, çünkü E , makine S_2 'ye girdiğinde 1 olarak ayarlanır, ardından S_{idle} 'a geçmeden önce (tablonun altında gösterildiği gibi) ve çünkü E , S_{idle} 'dan S_1 'e geçiş sırasında değişmez. Sistem, sonraki 13 saat darbesi boyunca S_1 durumunda kalır. Her darbe sayacı artırır ve ya E 'yi temizler ya da ayarlar. A_2 'nin 1 olduğu zaman ile E 'nin 1'e ayarlandığı zaman arasındaki ilişkiye dikkat edin.

Tablo 8.3
Tasarım Örneği için İşlem Sırası

Sayıcı				Flip-Flop'lar		Koşullar	Durum
A_3	A_2	A_1	A_0	E	F		
0	0	0	0	1	0	$A_2=0, A_3=0$	S_I
0	0	0	1	0	0		
0	0	1	0	0	0		
0	0	1	1	0	0		
0	1	0	0	0	0	$A_2=1, A_3=0$	
0	1	0	1	1	0		
0	1	1	0	1	0		
0	1	1	1	1	0		
1	0	0	0	1	0	$A_2=0, A_3=1$	
1	0	0	1	0	0		
1	0	1	0	0	0		
1	0	1	1	0	0		
1	1	0	0	0	0	$A_2=1, A_3=1$	S_2
1	1	0	1	1	0		
1	1	0	1	1	1		
							S_idle

$A = (A_3 A_2 A_1 A_0)$ 0011 olduğunda, bir sonraki (4.) saat darbesi sayacı 0100'e artırır, ancak aynı saat kenarı A_2 'nin değerini 0 olarak görür, bu nedenle E temiz kalır. Bir sonraki (5.) darbe sayacı 0100'den 0101'e değiştirir ve A_2 , saat darbesi gelmeden önce 1'e eşit olduğu için E 1'e ayarlanır. Benzer şekilde, E , sayım 0111'den 1000'e geçerken değil, 1000'den 1001'e geçerken 0'a temizlenir, bu da ne zaman A_2 sayacı mevcut değerinde 0'dır.

Sayım 1100'e ulaştığında, hem A_2 hem de A_3 1'e eşit olur. Bir sonraki saat kenarı A değeri ni 1 artırır, E değerini 1 olarak ayarlar ve kontrolü S_2 durumuna aktarır. Kontrol, yalnızca bir saat periyodu boyunca S_2 'de kalır. S_2 durumundan çıkan yol ile ilişkili saat kenarı flip-flop F değeri ni 1 yapar ve kontrolü S_idle durumuna aktarır. Sistem, $Start$ 0'a eşit olduğu sürece başlangıç durumu S_idle 'da kalır.

Tablo 8.3 'ten yapılan bir gözlem, E üzerinde gerçekleştirilen işlemlerin bir saat darbesi kadar geciktiği izlenimini verebilir. Bu, bir ASMD şeması ile bir geleneksel akış şeması arasındaki farktır. Eğer Şekil 8.9 (d) geleneksel bir akış şeması olsaydı, A 'nın önce artırıldığını ve artırılan değer A_2 'nin durumunu kontrol etmek için kullanıldığını varsayardık. ASMD şemasında bir blok tarafından belirtilen dijital donanımda gerçekleştirilen işlemler, aynı saat döngüsü içinde gerçekleşir ve zaman içinde birbirini takip eden bir işlem sırası şeklinde değil, geleneksel bir akış şemasında yaygın olarak yorumlandığı gibi gerçekleşmez. Böylece, karar kutusunda dikkate alınacak A_2 değeri alınır.

mevcut durumdaki sayacın değerinden ve artırılmadan önce. Bu, E için karar kutusunun S_1 durumu ile aynı blokta yer alması nedeniyledir. Kontrol birimindeki dijital devreler, mevcut blokta belirtilen tüm işlemler için sinyalleri üretir bir sonraki saat darbesinin gelmesinden önce. Bir sonraki saat kenarı, register'lar ve flip-flop'lar dahil olmak üzere, kontrolör içindeki flip-flop'lar da dahil olmak üzere, mevcut çıkış sinyallerinin değerlerini kullanarak bir sonraki durumu belirler. Böylece, datapath birimindeki işlemleri kontrol eden sinyaller, operasyonların gerçekleştirildiği saat kenarından önceki saat döngüsünde (kontrol durumu) kontrolörde oluşur.

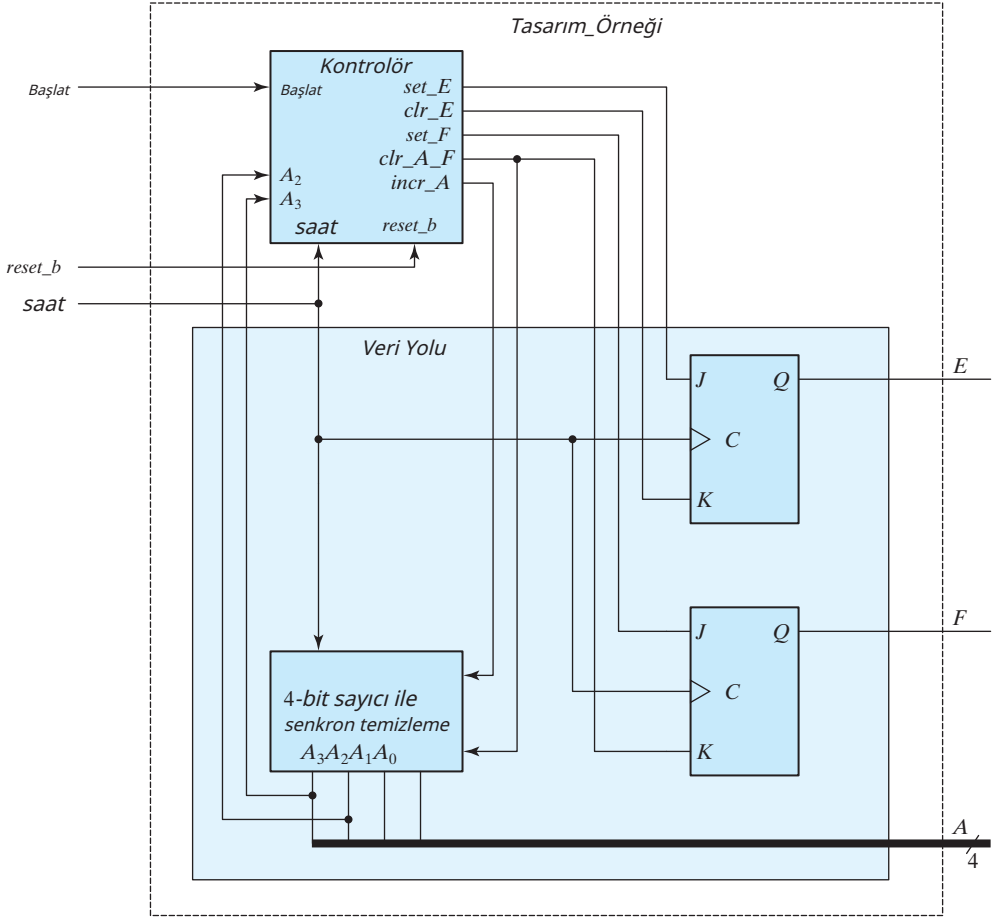
Kontrolcü ve Veri Yolu Donanım Tasarımı

ASMD grafiği, dijital sistemi tasarlamak için gereken tüm bilgileri sağlar — veri yolu ve kontrolör. Kontrolör ile veri yolu donanımı arasındaki gerçek sınır keyfi olabilir, ancak öncelikle, veri yolu biriminin yalnızca işlemleriyle ilişkili donanımı ve belki de kontrolör tarafından kullanılan durum sinyallerini oluşturmak için gereken mantığı içermesini savunuyoruz; ikincisi, kontrol biriminin veri yolu biriminin işlemlerini kontrol eden sinyalleri üretmek için gereken tüm mantığı içermesidir. Veri yolunun tasarımı için gereksinimler, ASMD grafiğinin durum ve koşul kutularındaki kontrol sinyalleriyle belirtilir ve veri yolu işlemlerini gösteren kenarların notasyonlarıyla tanımlanır. Kontrol mantığı, karar kutularından ve gerekli durum geçişlerinden belirlenir. Veri yolu ve kontrolörün donanım yapılandırması Şekil 8.10'da gösterilmektedir.

Kontrol biriminin giriş sinyallerinin dış (birincil) girişler (*Başlat*, *reset_b*, ve *saat*) ve veri yolundan gelen durum sinyalleri (A_2 ve A_3) olduğunu unutmayın. Durum sinyalleri, veri yolunun mevcut durumu hakkında bilgi sağlar. Bu bilgi, birincil girişler ve makinenin mevcut durumu hakkında bilgi ile birlikte, kontrolörün çıktısını ve bir sonraki durumun değerini oluşturmak için kullanılır. Kontrolörün çıktıları, veri yoluna girişlerdir ve saat geçiş yaptığında hangi işlemlerin gerçekleştirileceğini belirler. Ayrıca, kontrol biriminin durumu kontrol biriminin bir çıktısı değildir.

Kontrol alt sistemi, yalnızca girişleri ve çıkışları ile birlikte Şekil 8.10'da gösterilmiştir; isimler ASMD şemasındaki isimlerle eşleşmektedir. Kontrol cihazının detaylı tasarımı daha sonra ele alınacaktır. Veri yolu birimi, dört bitlik bir ikili sayıcı ve iki JK flip-flop'lar. Sayıcı, Şekil 6.12'de gösterilenle benzer, ancak senkron temizleme işlemi için ek iç kapılar gereklidir. Sayıcı, denetleyici durumu S_1 olduğunda her saat darbesiyle artırılır. Sadece kontrol durumu S_{idle} olduğunda ve *Start* 1'e eşit olduğunda temizlenir. *clr_A_F* sinyali için mantık, denetleyicide yer alacak ve her iki koşulun da mevcut olduğunu garanti etmek için bir VE kapısı gerektirecektir. Benzer şekilde, denetleyicinin *set_E* ve *clr_E* sinyallerini oluşturmak için VE kapıları kullanacağımızı öngörebiliriz. Denetleyicinin S_1 durumunda olup olmadığına ve

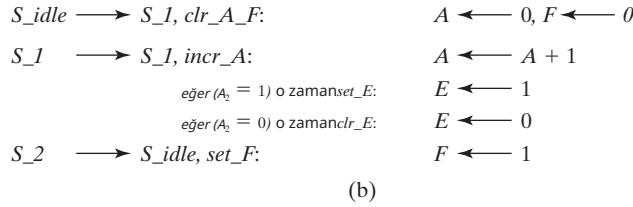
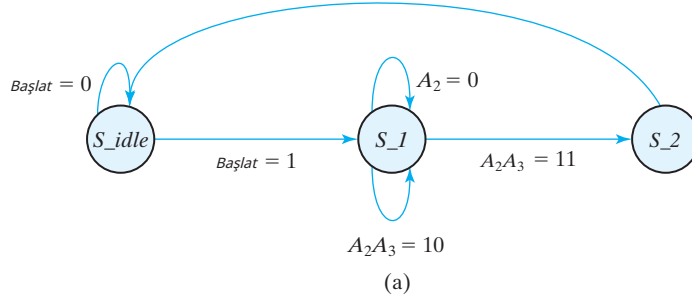
A_2 iddia edilir, *set_F* flip-flop F kontrol eder ve durum S_2 sırasında koşulsuz olarak iddia edilir. Not edin ki, kontrol birimindeki flip-floplar da dahil olmak üzere tüm flip-floplar ve kayıtlar, ortak bir saat kullanır.



ŞEKİL 8.10
Tasarım örneği için veri yolu ve denetleyici

Register Transfer Representation

Bir dijital sistem, sistemdeki kayıtları, gerçekleştirilen işlemleri ve kontrol dizisi ni belirterek kayıt transfer düzeyinde temsil edilir. Kayıt işlemleri ve kontrol bilgileri, bir ASMD grafiği ile belirtilebilir. Kontrol mantığını, veri yolunun kayıt işlemlerinden ayırmak uygundur. ASMD grafiği, bu ayrımı sağlar ve bir veri yolu için bir denetleyici tasarlamak için net bir adım dizisi sunar. Kontrol bilgileri ve kayıt transfer işlemleri ayrıca ayrı olarak temsil edilebilir, bu şekilde gösterilmiştir. Şekil 8.11. Durum diyagramı kontrol dizisini belirtir ve kayıt işlemleri, <original>FIGURE 8.10</original>da tanıtılan kayıt transfer notasyonu ile temsil edilir.

**ŞEKİL 8.11**

Tasarım örneğinin kayıt transfer seviyesi tanımı

Bölüm 8.2. Durum geçişi ve kaydı kontrol eden sinyal, operasyon ile gösterilmektedir. Bu temsil, sistemin Fig. 8.9 (d) ASMD şemasında tanımlanan temsili bir alternatiftir. Sadece ASMD şeması gerçekten gerekli, ancak kontrolör için durum diyagramı, manuel tasarımda yararlı olan bir alternatif temsildir. Durum diyagramı için bilgiler doğrudan ASMD şemasından alınmaktadır. Durum adları her durum kutusunda belirtilmiştir. Durum değişikliğine neden olan koşullar, ASMD şemasının elmas şeklindeki karar kutularının içinde belirtilmiştir ve durum diyagramını notlandırmak için kullanılmaktadır. Durumlar arasındaki yönlendirilmiş çizgiler ve her biriyle ilişkili koşul, ASMD şemasındaki aynı yolu izlemektedir. Üç durumun her biri için kayıt transfer işlemleri, durum adının ardından listelenmiştir. Bunlar, durum kutularından veya ASMD şemasının notlandırılmış kenarlarından alınmıştır.

Durum Tablosu

Durum diyagramı, kontrolörün ardışık devresinin tasarlanabileceği bir durum tablosuna dönüştürülebilir. Öncelikle, ASMD şemasındaki her duruma ikili değerler atayabiliriz. Kontrol ardışık devresinde n flip-flop için, ASMD şeması en fazla 2 durumu barındırabilir. 3 veya 4 duruma sahip bir şema, iki flip-flop ile bir ardışık devre gerektirir. 5'ten 8'e kadar durumlar için üç flip-flop gereklidir. Her flip-flop değer kombinasyonu, durumlardan birine karşılık gelen bir ikili sayıyı temsil eder.

Bir kontrolör için *durum tablosu* mevcut durumların ve girişlerin yanı sıra bunların karşılık gelen sonraki durumları ve çıkışlarının bir listesidir. Çoğu durumda, birçok önemsiz giriş koşulu vardır.

Tablo 8.4

Şekil 8.10 için Kontrolör Durum Tablosu

Mevcut-Durum Sembolü	Mevcut Durum		Girişler			Sonraki Durum		Çıkışlar				
	G_1	G_0	Başla	A_2	A_3	G_1	G_0	set_F	clr_F	set_F	clr_A_F	incr_A
S_idle	0	0	0	X	X	0	0	0	0	0	0	0
S_idle	0	0	1	X	X	0	1	0	0	0	1	0
S_1	0	1	X	0	X	0	1	0	1	0	0	1
S_1	0	1	X	1	0	0	1	1	0	0	0	1
S_1	0	1	X	1	1	1	1	1	0	0	0	1
S_2	1	1	X	X	X	0	0	0	0	1	0	0

bu nedenle, durum tablosunu bu koşulları dikkate alacak şekilde düzenlemek önerilir. Üç duruma aşağıdaki ikili değerleri atıyoruz: $S_idle=00$, $S_1=01$, ve $S_2=11$. İkili durum 10 kullanılmaz ve bir önemsiz koşul olarak değerlendirilecektir. Durum diyagramına karşılık gelen durum tablosu Tablo 8.4'te gösterilmiştir. İki flip-flop gereklidir ve bunlar G_1 ve G_0 olarak etiketlenmiştir. Üç girdi ve beş çıktı vardır. Girdiler, karar kutularındaki koşullardan alınmaktadır. Çıkışlar girdilere ve kontrolün mevcut durumuna bağlıdır. Tablo'da her olası durum geçişi için bir satır olduğunu unutmayın. Başlangıç durumu 00, *Start* değerine bağlı olarak 01 durumuna geçer veya 00'da kalır. Diğer iki girdi, bu durumda bir sonraki durumu belirlemedikleri için önemsiz X'ler ile işaretlenmiştir. Sistem, *Start*=1 olduğunda ikili durum 00'da iken, kontrol birimi gerekli kayıt işlemlerini başlatmak için *clr_A_F* etiketli bir çıkış sağlar. İkili durum 01'den geçiş, girdilere A_2 ve A_3 bağlıdır. Sistem, yalnızca $A_2A_3=11$ olduğunda ikili durum 11'e geçer; aksi takdirde, ikili durum 01'de kalır. Son olarak, ikili durum 11, girdi değişkenlerinden bağımsız olarak 00'a geçer.

Kontrol Mantiği

Bir durum tablosundan başlayarak ardışık bir devre tasarlama prosedürü, Bölüm 5'te sunulmuştur. Bu prosedür Tablo 8.4'e uygulandığında, giriş denklemlerini basitleştirmek için beş değişkenli haritalar kullanmamız gerekecektir. Bunun nedeni, tablonun mevcut durum ve giriş sütunları altında beş değişkenin listelenmiş olmasıdır. Giriş denklemlerini basitleştirmek için haritalar kullanmak yerine, bunları durum tablosundan doğrudan inceleme yoluyla elde edebiliriz. D flip-flop'lar ile kontrolörün ardışık devresini tasarlamak için, durum tablosundaki bir sonraki durum sütunlarını gözden geçirmek ve her flip-flop'u 1'e ayarlamak için gerekli tüm koşulları çıkarmak gereklidir. Tablo 8.4'ten, G_1 'in bir sonraki durum sütununda beşinci satırda tek bir 1 olduğunu not ediyoruz. Flip-flop G_1 'in D girişi

her iki giriş A_2 ve A_3 1'e eşit olduğunda mevcut durum S_1 sırasında 1'e eşit olacaktır. Bu durum, D flip-flop giriş denklemi ile ifade edilir

$$D_{G1} = S_1 A_2 A_3$$

Benzer şekilde, G_0 için bir sonraki durum sütununda dört 1 vardır ve bu flip-flop'u ayarlamak için koşulu

$$D_{G0} = \text{Başla } S_{idle} + S_1$$

Beş çıkış fonksiyonunu türetmek için, ikili durum 10'un kullanılmadığı gerçeğinden yararlanabiliriz, bu da clr_A_F denklemini basitleştirir ve aşağıdaki basitleştirilmiş çıkış denklemleri setini elde etmemizi sağlar:

$$set_E = S_1 A_2$$

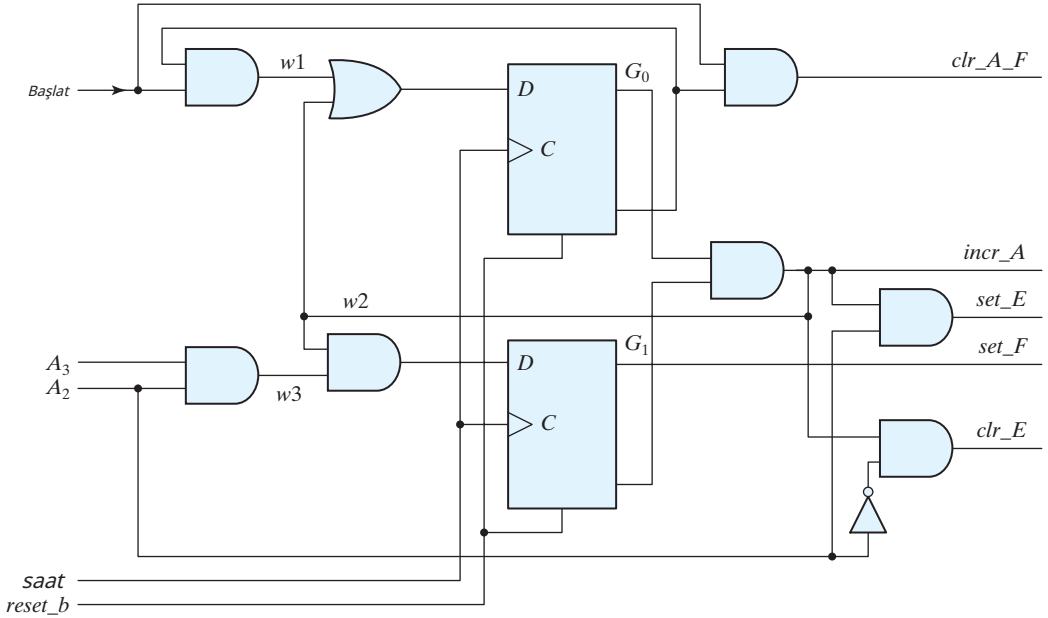
$$clr_E = S_1 A_2$$

$$set_F = S_2$$

$$clr_A_F = \text{Başla } S_{idle}$$

$$incr_A = S_1$$

Kontrolörün iç detaylarını gösteren mantık diyagramı Şekil 8.10'da çizilmiştir. Şekil 8.12'de gösterilmiştir. Çıktı denklemlerini Tablo 8.4'ten türetmiş olsak da, bunlar doğrudan Şekil 8.9 (d)'nin incelenmesiyle de elde edilebilir. Bu basit örnek,



ŞEKİL 8.12

Şekil 8.10 için kontrol biriminin mantık diyagramı

bir veri yolunun kontrolcüsünün manuel tasarımı, bir ASMD grafiğini başlangıç noktası olarak kullanarak. Sentez araçlarının bu adımları otomatik olarak gerçekleştirdiği gerçeği takdir edilmelidir.

8.6 HDL TASARIM ÖRNEĞİ AÇIKLAMASI

Önceki bölümlerde, kombinasyonel devrelerin, sıralı devrelerin ve çoklayıcılar, sayaçlar ve registerlar gibi çeşitli standart bileşenlerin HDL açıklamalarına örnekler verdik. Şimdi bu bileşenleri belirli bir tasarımın açıklamasına dahil etme aşamasındayız. Daha önce belirtildiği gibi, bir tasarım ya yapısal ya da davranışsal düzeyde tanımlanabilir. Davranışsal açıklamalar, ya register transfer düzeyinde ya da soyut algoritmik düzeyde sınıflandırılabilir. Sonuç olarak, şimdi üç tasarım düzeyini ele alıyoruz: yapısal açıklama, RTL açıklaması ve algoritma-tabanlı davranışsal açıklama.

Yapısal *açıklama* en düşük ve en ayrıntılı düzeydir. Dijital sistem, fiziksel bileşenler ve bunların bağlantıları açısından tanımlanır. Çeşitli bileşenler kapılar, flip-flop'lar ve çoklayıcılar ile sayaçlar gibi standart devreleri içerebilir. Tasarım, işlevsel birimlere hiyerarşik olarak ayrılır ve her birim bir HDL modülü ile tanımlanır. Üst düzey bir modül, tüm sistemi alt düzey modülleri örnekleyerek birleştirir. Bu açıklama tarzı, tasarımcının sistemin işlevselliğini anlamak için yeterli deneyime sahip olmasını ve ayrıca diğer işlevsel elemanları seçip bağlayarak bunu uygulayabilmesini gerektirir.

RTL açıklaması, dijital sistemi kayıtlar, gerçekleştirilen işlemler ve işlemleri sıralayan kontrol açısından tanımlar. Bu tür bir açıklama, tasarım sürecini basitleştirir çünkü tasarımın çeşitli işlemleri arasındaki ilişkiyi belirleyen prosedürel ifadelerden oluşur ve herhangi bir belirli yapıya atıfta bulunmaz. RTL açıklaması, kayıtlar arasında belirli bir donanım yapılandırmasını ima eder ve tasarımcının tasarımı otomatik olarak, manuel olarak değil, standart dijital bileşenlere sentezleyebileceği bir tasarım oluşturmaya olanak tanır.

Algoritma tabanlı davranışsal açıklama, en soyut seviyedir ve tasarımın işlevini, bir programlama diline benzer prosedürel, algoritmik bir biçimde tanımlar. Tasarımın donanımla nasıl uygulanacağına dair herhangi bir ayrıntı sağlamaz. Algoritma tabanlı davranışsal açıklama, tasarım fikirlerini doğrulamak ve ticaret dengelerini keşfetmek için karmaşık sistemleri simüle etmek için en uygun olandır. Bu seviyedeki açıklamalar, programlama dillerini anlayan teknik olmayan kullanıcılar için erişilebilir durumdur. Ancak bazı algoritmalar sentezlenebilir olmayabilir.

Artık RTL ve yapısal tanımları, önceki bölümdeki tasarım örneğini kullanarak açıklayacağız. Tasarım örneği, gelecekteki örnekler için bir kodlama stili modeli olarak hizmet edecek ve Verilog diline yapılan revizyonlarla desteklenen alternatif sözdizimi seçeneklerini kullanacaktır. (Algoritmik tabanlı bir tanım, Bölüm 8.9'da gösterilmektedir.)

RTL Tanımı

Şekil 8.10'daki blok diyagramı, tasarım örneğini tanımlamaktadır. Tasarım örneğinin HDL tanımı, bir Verilog modülünde tek bir RTL tanımı olarak yazılabilir veya

kontrolör ve veri yolu için ayrı modüllerin örneklemeleri olan üst düzey bir modül olarak yazılabilir. İlk seçenek, işlevsel birimler arasındaki sınırları basitçe göz ardı eder; ikinci seçenekteki modüller, Şekil 8.9 (a) ve Şekil 8.10'daki sınırları belirler. İkinci seçeneği savunuyoruz, çünkü genel olarak kontrolör ile veri yolu arasında daha net bir ayırım yapmaktadır. Bu seçim, belirli bir veri yolu için alternatif if kontrolörlerin kolayca değiştirilmesine de olanak tanır (örneğin, bir RTL modelini yapısal bir modelle değiştirmek). Tasarım örneğinin RTL tanımı, HDL Örneği 8.2'de gösterilmektedir. Tanım, kontrolör, veri yolu ve aralarındaki arayüzün (yani, kontrolörün çıktıları ve durum sinyalleri) tam tanımını içeren Şekil 8.9 (d) ASMD grafiğini takip etmektedir. Benzer şekilde, tanımımız üç modülden oluşmaktadır: *Design_Example_RTL*, *Controller_RTL* ve *Datapath_RTL*. Kontrolör ve veri yolu birimlerinin tanımları doğrudan Şekil 8.9 (d) 'den alınmıştır.

Design_Example_RTL modülün giriş ve çıkış portlarını tanımlar ve

Controller_RTL ile *Datapath_RTL* örneklendirir. Tanımın bu aşamasında,

A vektör olarak tanımlamayı hatırlamak önemlidir. Bunu yapmamak, *port mismatch* hatalarına yol açacaktır

tanımlar bir araya getirildiğinde. Durum sinyallerinin $A[2]$ ve $A[3]$,

bunlar dışında $A[0]$ ve $A[1]$, kontrolöre iletildiğine dikkat edin. Kontrolörün birincil (dış) girişleri

Start, *clock* (sistemi senkronize etmek için) ve *reset_b* dir. Aktif-düşük giriş sinyali *reset_b* kontrolörün durumunu *S_idle* olarak başlatmak için gereklidir.

O sinyal olmadan, kontrolör bilinen bir başlangıç durumuna yerleştirilemez.

Kontrolör, üç döngüsel (**always**) davranışla tanımlanır. Kenar-duyarlı davranış, bir reset koşulunun doğrulanıp doğrulanmadığına bağlı olarak, saat sinyalinin pozitif kenarında durumu günceller. İki seviye-duyarlı davranış, ASMD grafiği tarafından belirtilen kontrolörün bir sonraki durumu ve çıkışları için kombinasyonel mantığı tanımlar. Tanımın, tüm çıkışlara varsayılan atamalar içerdiğini unutmayın (örneğin, *set_E* = 0). Bu yaklaşım, **case** mantığının kodunu, yalnızca değişkenlerin açık atamalarını ifade ederek basitleştirmeyi sağlar (yani, değerler istisna ile atanır). Bu yaklaşım ayrıca, atama mantığı boyunca her yolun her değişkene bir değer atamasını garanti eder. Böylece, bir sentez aracı mantığı kombinasyonel olarak yorumlayacaktır; her mantık yolunda her değişkene bir değer atanmaması, mantığı uygulamak için şeffaf bir latch (hafıza) gereksinimini ima eder. Sentez araçları latch sağlayacak, bu da silikon alanını israf edecektir.

Kontrolcünün üç durumu sembolik isimlerle verilmiştir ve ikili değerlere kodlanmıştır. Kullanılan olası iki bitlik desenlerden yalnızca üçü kullanılmaktadır, bu nedenle bir sonraki durum mantığı için durum ifadesi, üç atanan koddan birinin tespit edilmediği olasılığını ele almak için bir varsayılan atama içermektedir. Alternatif olarak, donanımın bir sonraki duruma (*next_state* = 2 *bx*) keyfi bir atama yapmasına izin vermektir. Ayrıca, bir sonraki durum mantığının ilk durumu, her mantık iş parçacığında bir sonraki durumun atanmasını garanti etmek için *next_state* = *S_idle* atamasını yapmaktadır. Bu, mantık iş parçacıklarının her birinde bir sonraki duruma atama yapmayı unutmaya olasılığına karşı bir önlemdir; bu durumda tanım, bir sentez aracının şeffaf bir latch ile uygulayacağı bir bellek gereksinimini ima eder.

Datapath_RTL 'nin tanımı, *Controller_RTL* 'den her kontrol sinyalinin bir doğrulanmasını test ederek yazılmıştır. Kayıt transfer işlemleri aşağıda gösterilmektedir.

Bölüm 8.6 HDL Tasarım Örneği Açıklaması 383

the AS MD chart (Şekil 8.9 (d)). Dikkat edilmesi gereken, kayıt transfer işlemleri için (sembol $\leq$) engellemeyen atamaların kullanılmasıdır. Bu, kayıt işlemlerinin ve durum geçişlerinin eşzamanlı olmasını sağlar; bu özellik, kontrol durumu S_1 sırasında özellikle önemlidir. Bu durumda, A_1 artırılır ve $A_2 \leftarrow A_1/2$ değeri, bir sonraki saat döngüsünde kayıt E üzerinde gerçekleştirilecek işlemi belirlemek için kontrol edilir. Geçerli bir senkron tasarım gerçekleştirmek için, $A_2/2$ kontrol edilmeden önce A_1 artırılmamalıdır. Engelleyici atamalar kullanılıyorsa, E kontrol eden iki ifadeyi önce yerleştirmek ve A_1 ifadesini artıran sonuncu olarak yerleştirmek gerekirdi. Ancak, engellemeyen atamalar kullanarak, ifadelerin sıralanma düzeni hakkında endişelenmeden gerekli senkronizasyonu sağlıyoruz. $Datapath_RTL$ içindeki sayaç A senkronize bir şekilde temizlenir çünkü clr_A_F saate senkronizedir.

Kontrolörün ve veri yolunun döngüsel davranışları bir zincir tepkimesi içinde etkileşir: Saati aktif kenarında, durum ve veri yolu kayıtları güncellenir. Durumdaki, birincil giriş eki veya durum girişindeki bir değişiklik, denetleyicinin seviye duyarlı davranışlarının bir sonraki durumun ve çıkışların değerini güncellemesine neden olur. Güncellenmiş değerler, durum geçişini ve veri yolunun güncellemelerini belirlemek için saatin bir sonraki aktif kenarında kullanılır.

Tasarımın manuel yöntemi (1) veri yolu ile denetleyici arasındaki arayüzü gösteren bir blok diyagramı (Şekil 8.9 (a)), (2) sistem için bir ASMD grafiği (Şekil 8.9 (d)), (3) denetleyicinin flip-flop'larına girişler için mantık denklemleri ve (4) denetleyiciyi uygulayan bir devre (Şekil 8.12) geliştirmiştir. Buna karşılık, bir RTL modeli denetleyicinin durum geçişlerini ve veri yolunun işlemlerini, bunları uygulayan devreyi otomatik olarak sentezleme yönünde bir adım olarak tanımlar. Veri yolu ve denetleyici tanımları her iki durumda da doğrudan ASMD grafiğinden türetilmiştir.

HDL Örneği 8.2

```
// Tasarım örneğinin RTL tanımı (bkz. Şekil 8.11)
module Design_Example_RTL (A, E, F, Start, clock, reset_b);
    // Tasarımın üst düzey modülünün portlarını belirtin
    // Blok diyagramına bakın, Şekil 8.10
    çıkış [3: 0] A;
    çıkış E, F;
    girdi Başla, saat, reset_b;
    // Kontrolör ve veri yolu birimlerini oluşturun
    Controller_RTL M0 (set_E, clr_E, set_F, clr_A_F, incr_A, A[2], A[3], Başla, saat, reset_b);
    Datapath_RTL M1 (A, E, F, set_E, clr_E, set_F, clr_A_F, incr_A, saat);
endmodule

modül Controller_RTL (set_E, clr_E, set_F, clr_A_F, incr_A, A2, A3, Başla, saat, reset_b);
    çıkış reg set_E, clr_E, set_F, clr_A_F, incr_A;
    girdi Başla, A2, A3, saat, reset_b;
```

```

reg [1:0] durum, sonraki_durum;

parametre S_idle = 2'b00, S_1 = 2'b01, S_2 = 2'b11; // Durum kodları
her zaman @(yükselen saat, düşen reset_b) // Durum geçişleri (kenar duyarlı)
if (reset_b == 0) state <= S_idle;
else state <= next_state;

// ASMD grafiğinden (Şekil 8.9 d) doğrudan sonraki durum mantığını kodlayın
her zaman @ (durum, Başla, A2, A3) başla // Sonraki durum mantığı (seviye duyarlı)
sonraki_durum = S_idle;

durum (durum)
S_idle: eğer (Başla) sonraki_durum = S_1; aksi takdirde sonraki_durum = S_idle;
        eğer (A2 & A3) sonraki_durum = S_2; aksi takdirde sonraki_durum = S_1;
S_2: sonraki_durum = S_idle;
varsayılan : sonraki_durum = S_idle;
endcase
end

// ASMD diyagramından (Şekil 8.9 d) doğrudan kod çıkış mantığı
always @(durum, Başlat, A2) begin
set_E = 0; // varsayılan atanmış değerler varsayılan atamalar; istisna ile atanır
clr_E = 0;
set_F = 0;
clr_A_F = 0;
incr_A = 0;
case (durum)
S_idle: if (Başlat) clr_A_F = 1;
S_1: begin incr_A = 1; if (A2) set_E = 1; else clr_E = 1; end
S_2: set_F = 1;
endcase
end
endmodule

module Datapath_RTL (A, E, F, set_E, clr_E, set_F, clr_A_F, incr_A, clock);
output reg [31:0] A; // sayaç için kayıt // sayaç için kayıt
output reg E, F; // bayraklar
input set_E, clr_E, set_F, clr_A_F, incr_A, clock;
// ASMD şemasından doğrudan kod kayıt transfer işlemleri (Şekil 8.9 (d))
always@ (posedge clock) begin
if (set_E) E <= 1;
if (clr_E) E <= 0;
if (set_F) F <= 1;
if (clr_A_F) begin A <= 0; F <= 0; end
if (incr_A) A <= A + 1;
end
endmodule

```

Tasarım Açıklamasını Test Etme

Tasarım örneği için işlemler sırası önceki bölümde incelenmiştir. Tablo 8.3, kayıt A artırılırken E ve F değerlerini göstermektedir. HDL açıklamasının geçerliliğini doğrulamak için devreyi kontrol eden bir test geliştirmek öğreticidir. HDL Örneği 8.3'teki test tezgahı böyle bir modül sağlar. (Test tezgahlarının yazım prosedürü Bölüm 4.12'de açıklanmaktadır.) Test modülü, $Start$, $clock$ ve $reset_b$ için sinyaller üretir ve A , E ve F kayıtlarından elde edilen sonuçları kontrol eder.

Başlangıçta, $reset_b$ sinyali kontrolörü başlatmak için 0 olarak ayarlanır ve $Start$ ile $clock$ 0 olarak ayarlanır. Zaman $t = 5$ 'te, $reset_b$ sinyali 1 olarak ayarlanarak devre dışı bırakılır, $Start$ girişi 1 olarak ayarlanarak etkinleştirilir ve saat 16 döngü boyunca tekrarlanır. $\$monitor$ i fadesi, her 10 ns'de A , E ve F değerlerini gösterir. Simülasyonun çıktısı, simülasyon günlüğü altında örnekte listelenmiştir. Başlangıçta, zaman $t = 0$ 'da, kayıtların değerleri bilinmemektedir, bu nedenle x sembolü ile işaretlenmiştir. İlk pozitif saat geçişi, zaman = 10'da A ve F 'yi temizler, ancak E 'yi etkilemez, bu nedenle bu zamanda E bilinmemektedir. Tablo'nu geri kalanı Tablo 8.3 ile aynıdır. Zaman = 160'da $Start$ hala 1'e eşit olduğundan, tablodaki son giriş A ve F 'nin 0'a temizlendiğini ve E 'nin değişmediğini ve 1'de kaldığını göstermektedir. Bu, S_idle 'dan S_1 'ye ikinci geçiş sırasında gerçekleşir.

HDL Örneği 8.3

```
// Tasarım örneği için test tezgahı
timescale 1 ns / 1 ps
modül t_Tasarım_Örneği_RTL;
    reg        Başla, saat, reset_b;
tel [3: 0] A;
tel        E, F;
// Tasarım örneğini başlat
    Tasarım_Örneği_RTL M0 (A, E, F, Başla, saat, sıfırla_b);
// Uyarı dalga formlarını tanımla
başlangıç #500 $finish ; // Kronometre
    başlangıç
    başla
        sıfırla_b = 0;
        Başla = 0;
        saat = 0;
        #5 reset_b = 1; Başla = 1;
    tekrar (32)
    başla
        #5 saat = ~ saat;           // Saat üreteci
end
```

son

başlangıç

\$monitor ("A = %b E = %b F = %b zaman = %0d", A, E, F, **\$time**);

endmodule

Simülasyon kaydı:

A = xxxx E = x F = x zaman = 0

A = 0000 E = x F = 0 zaman = 10

A = 0001 E = 0 F = 0 zaman = 20

A = 0010 E = 0 F = 0 zaman = 30

A = 0011 E = 0 F = 0 zaman = 40

A = 0100 E = 0 F = 0 zaman = 50

A = 0101 E = 1 F = 0 zaman = 60

A = 0110 E = 1 F = 0 zaman = 70

A = 0111 E = 1 F = 0 zaman = 80

A = 1000 E = 1 F = 0 zaman = 90

A = 1001 E = 0 F = 0 zaman = 100

A = 1010 E = 0 F = 0 zaman = 110

A = 1011 E = 0 F = 0 zaman = 120

A = 1100 E = 0 F = 0 zaman = 130

A = 1101 E = 1 F = 0 zaman = 140

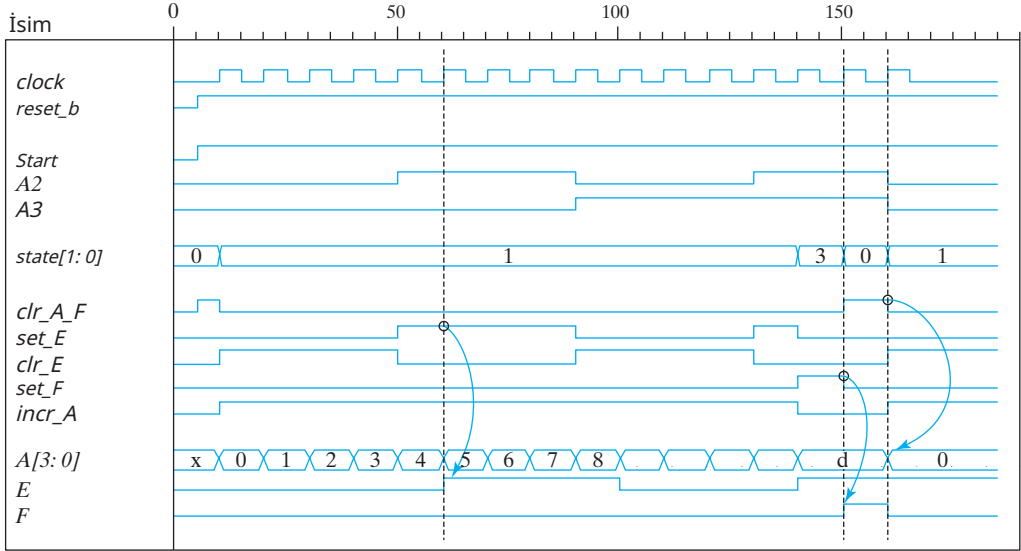
A = 1101 E = 1 F = 1 zaman = 150

A = 0000 E = 1 F = 0 zaman = 160

Tasarım_Örneği_RTL ile test tezgahının simülasyonu tarafından üretilen dalga formları Şekil 8.13'te gösterilmiştir. Sayısal değerler onaltılık formatta gösterilmektedir. Sonuçlar bir kontrol sinyali ile neden olduğu işlem arasındaki ilişkiye dikkat çekmek için not edilmiştir. Örneğin, denetleyici set_E sinyali bir çarpan döngüsü önce saat kenarında E 1'e ayarlandığında etkinleştirir. Benzer şekilde, set_F sinyali, çarpan döngüsü sırasında, F 1'e ayarlandığı kenardan önce etkinleştirilir. Ayrıca, clr_A_F sinyali, A ve F sıfırlanmadan önceki döngüde oluşur. Tasarım_Örneği_RTL için daha kapsamlı bir doğrulama, makinenin çalışırken rastgele bir sıfırlama ile geri kazanıldığını doğrulayacaktır (yani, makine çalışırken rastgele etkinleştirilen bir sıfırlama). Simülasyonun çıktısındaki sinyaller, (1) saat ve reset_b, (2) Başlat ve durum girişleri, (3) durum, (4) kontrol sinyalleri ve (5) veri yolu kayıtları olarak gruplar halinde listelenmiştir. Durumun her zaman görüntülenmesi şiddetle önerilir, çünkü bu bilgi, makinenin doğru çalıştığını doğrulamak ve çalışmadığında tanımı hata ayıklamak için gereklidir. Seçilen ikili durum kodu için, S_idle=002=0H, S_1=012=1H, ve S_2=112=3H.

Yapısal Tanım

Bir tasarımın RTL tanımı, dijital devrenin işlevsel davranışını belirleyen prosedürel ifadelerden oluşur. Bu tür bir tanım,



ŞEKİL 8.13

Simülasyon sonuçları için *Tasarım_Örneği_RTL*

HDL sentez araçları, tasarımın eşdeğer kapı seviyesindeki devresini elde etmenin mümkün olduğu araçlardır. Tasarımı işlevi yerine yapısı ile tanımlamak da mümkünüdür. Bir tasarımın yapısal tanımı, devre elemanlarını ve bunların bağlantılarını tanımlayan bileşenlerin örneklemelerinden oluşur. Bu bağlamda, yapısal bir tanım, devrenin şematik diyagramı veya blok diyagramı ile eşdeğerdir. Günümüz tasarım uygulamaları, RTL tanımlarına büyük ölçüde dayanmakta, ancak burada iki yaklaşıma karşılaştırmak için bir yapısal tanım sunacağız.

Kolaylık açısından, devre tekrar iki parçaya ayrılmıştır: kontrolör ve datapath. Şekil 8.10'daki blok diyagramı, bu

birimler arasındaki yüksek seviyeli bölünmeyi göstermektedir ve Şekil 8.12, kontrolörün ek yapısal detaylarını sağlamaktadır.

Veri yolunun yapısı Şekil 8.10'da belirgindir ve flip-flop'lar ile senkron temizleme özelliğine sahip dört bitlik sayıcıdan oluşmaktadır. Verilog tanımının en üst seviyesi, *Design_Example_RTL*, *Controller_RTL* ve *Datapath_RTL* sırasıyla *Design_Example_STR*, *Controller_STR*, ve *Datapath_STR*, olacaktır. *Controller_STR*

ve *Datapath_STR* tanımları yapısal olacaktır.

HDL Örneği 8.4, tasarım örneğinin yapısal tanımını sunmaktadır. Bu, (1) üst düzey modül, *Design_*

Example_STR, (2) denetleyici ve veri yolu modüllerini tanımlayan modüller, (3) flip-flop'ları ve sayaçları tanımlayan modüller ve (4) denetleyicinin mantığını uygulayan kapıları içeren iç içe geçmiş bir modül ve kapı hiyerarşisinden oluşmaktadır.

Basitlik açısından, sayaç ve flip-flop'lar RTL modelleri ile tanımlanmaktadır.

Üst düzey modül (bkz. Şekil 8.10), (1) denetleyici ve veri yolu modüllerini örnekleyerek, (2) birincil (dış) giriş sinyallerini beyan ederek,

(3) çıkış sinyallerini beyan ederek, (4) denetleyici tarafından üretilen ve veri yolu birimine bağlı kontrol sinyallerini beyan ederek ve (5) veri yolu birimi tarafından üretilen ve denetleyiciye bağlı durum sinyallerini beyan ederek tüm tasarımı kapsar. Port listesi, RTL tanımında kullanılan liste ile aynıdır. Çıkışlar, burada yalnızca veri yolu modülünün çıkışlarını üst düzey modülün çıkışlarına bağlamak için kullanıldıkları için **wire** türü olarak beyan edilmiştir; mantık değerleri veri yolu modülünde belirlenmektedir.

Kontrol modülü, Şekil 8.12'nin devresini tanımlar. İki flip-flop'un çıkışları *G1* ve *G0* **wire** veri tipi olarak tanımlanmıştır. *G1* ve *G0* **reg** veri tipi olarak tanımlanmaz çünkü bunlar bir örneklenmiş *D* flip-flop'un çıkışlarıdır. *DG1* ve *DG0* tanımsız tanımlayıcılardır, yani örtük hatlardır. Bir değişkenin adı, tanımlandığı modül veya prosedürel blok ile sınırlıdır. Ağlar, bir prosedürel blok içinde tanımlanamaz (örneğin, **begin** ... **end**). Hatırlanması gereken kural, bir değişkenin yalnızca bir prosedürel ifade tarafından atanmışsa (yani, döngüsel veya tek geçişli davranışta bir prosedürel blok içinde bir engelleyici veya engellemeyen atama ifadesi veya ardışık bir UDP'nin çıkışında) tanımlanmış bir kayıt türü (örneğin, **reg**) olması gerektirir. Örneklenmiş kapılar, devrenin kombinasyonel kısmını belirtir. İki flip-flop girişi denklemi ve üç çıkış denklemi vardır. Flip-flop'ların çıkışları *G1* ve *G0* ve giriş denklemleri *DG1* ve *DG0* çıkış *Q* ve giriş *D* ile örneklenmiş flip-flop'larda yer değiştirir. *D* flip-flop'u daha sonra bir sonraki modülde tanımlanır. Veri yolu biriminin yapısı, *JK* flip-flop'larına doğrudan girişlere sahiptir. HDL tanımındaki modüller ile Şekil 8.9, 8.10 ve 8.12'deki yapılar arasındaki ilişkiye dikkat edin.

HDL Örneği 8.4

// Tasarım örneğinin yapısal tanımı (Şekil 8.9(a), 8.12)

modül Tasarım_Örneği_STR

(çıkış [3: 0] A, // V 2001 port sözdizimi

çıkış E, F,

girdi Başla, saat, sıfırla_b

);

Kontrolcü_STR M0 (clr_A_F, set_E, clr_E, set_F, incr_A, Başla, A[2], A[3], saat, sıfırla_b);

Datapath_STR M1 (A, E, F, clr_A_F, set_E, clr_E, set_F, incr_A, clock);

endmodule

module Controller_STR

(output clr_A_F, set_E, clr_E, set_F, incr_A,

inputStart, A2, A3, clock, reset_b

);

wire G0, G1;

parameter S_idle = 2'b00, S_1 = 2'b01, S_2 = 2'b11;

wire w1, w2, w3;

```

    not (G0_b, G0);
    not (G1_b, G1);
    buf (incr_A, w2);
    buf (set_F, G1);
    not (A2_b, A2);
    or (D_G0, w1, w2);
    and (w1, Start, G0_b);
    and (clr_A_F, G0_b, Start);
    and (w2, G0, G1_b);
    and (set_E, w2, A2);
    and (clr_E, w2, A2_b);
    and (D_G1, w3, w2);
    and (w3, A2, A3);
    D_flip_flop_AR M0 (G0, D_G0, clock, reset_b);
    D_flip_flop_AR M1 (G1, D_G1, clock, reset_b);
endmodule

// veri yolu birimi

module Datapath_STR
( output [3: 0] A, A,
  çıkış E, F,
  input clr_A_F, set_E, clr_E, set_F, incr_A, clock
);
    JK_flip_flop_2 M0 (E, E_b, set_E, clr_E, clock);
    JK_flip_flop_2 M1 (F, F_b, set_F, clr_A_F, clock);
    Counter_4 M2 (A, incr_A, clr_A_F, clock);
endmodule

// Senkron temizleme ile sayaç

modül Sayıcı_4 ( çıkış reg [3: 0] A, girdi artır, temizle, saat);
    her zaman @ (pozitif kenar saat)
        if (clear) A <= 0; else if (incr) A <= A + 1;
endmodule

module D_flip_flop_AR (Q, D, CLK, RST);
    output Q;
    input D, CLK, RST;
    reg Q;

    always @ (posedge CLK, negedge RST)
        if (RST == 0) Q <= 1'b0;
        else Q <= D;
endmodule

// JK flip-flop'un tanımı

```

```

modül JK_flip_flop_2 (Q, Q_not, J, K, CLK);
    çıkış      Q, Q_not;
girdi        J, K, CLK;
    reg        Q;
atama        Q_not = ~Q;
her zaman    @ (posedge CLK)
    kasa      ({J, K})
        2'b00:    Q <= Q;
        2'b01:    Q <= 1'b0;
        2'b10:    Q <= 1'b1;
        2'b11:    Q <= ~Q;
    endcase
endmodule

module t_Design_Example_STR;
    reg Start, clock, reset_b;
    wire [3: 0] A;
    wire      E, F;

    // Tasarım örneğini oluştur

    Design_Example_STR M0 (A, E, F, Start, clock, reset_b);

    // Uyarı dalga formlarını tanımla

başlangıç #500 $bitir ; // Kronometre
başla
    reset_b = 0;
    Başla = 0;
    saat = 0;
    #5 reset_b = 1; Başla = 1;
    tekrar (32)
        başla
    #5 saat = ~ saat;          // Saat üreteci
    end
end

başlangıç
    $izle ("A = %b E = %b F = %b zaman = %0d", A, E, F, $zaman );
bitir

```

Yapısal tanım, RTL tanımını doğrulayan test tezgahı ile test edildi ve Şekil 8.1 3'te gösterilen sonuçları üretti. Gerekli tek değişiklik, örneğin instantiation'ının *Design_Example_RTL* yerine *Design_Example_STR* ile değiştirilmesidir. Simülasyon sonuçları *Design_Example_STR* ile *Design_Example_RTL* için eşleşti. Ancak, iki tanımın karşılaştırılması, RTL stilinin daha kolay olduğunu göstermektedir.

yazmak için ve sentez araçları otomatik olarak

register'ları, kombinasyonel mantığı ve bunların bağlantılarını sentezleyebilirse daha hızlı sonuçlar elde edilecektir.

8.7 SIRALI İKİLİ ÇARPAN

Bu bölüm ikinci bir tasarım örneğini tanıtmaktadır. İkili çarpma için bir donanım algoritma sı sunmakta, uygulanması için kayıt yapılandırmasını önermekte ve ardından bir ASMD grafiği kullanarak veri yolunu ve denetleyicisini nasıl tasarlayacağınızı göstermektedir.

İnceleyeceğimiz sistem, iki işaretli ikili sayıyı çarpır. Bölüm 4.7'de çarpma işlemini gerçekleştirmek için geliştirilen donanım, birçok toplayıcı ve AND kapısı içeren bir kombinasyon devresi çarpmanı ile sonuçlanmış ve entegre devre olarak büyük bir silikon alanı gerektirmiştir. Buna karşılık, bu bölümde daha verimli bir donanım algoritması, yalnızca bir toplayıcı ve bir kaydırma kaydı kullanan bir ardışık çarpın ile sonuçlanmaktadır. Donanım ve silikon alanındaki tasarruf, alan (donanım) - zaman alanında bir denge ile sağlanmaktadır. Paralel bir toplayıcı daha fazla donanım kullanır, ancak sonucunu bir saat döngüsünde oluşturur; ardışık bir toplayıcı daha az donanım kullanır, ancak sonucunu oluşturmak için birden fazla saat döngüsü alır.

İki ikili sayının çarpımı, kağıt ve kalemle ardışık (yani, sıralı) toplama ve kaydırma işlemleriyle yapılır. Süreç, sayısal bir örnekle en iyi şekilde gösterilmektedir. İki ikili sayı olan 10111 ve 10011'i çarpalım:

$$\begin{array}{r}
 23 \quad 10111 \text{ çarpın} \\
 19 \quad \underline{10011 \text{ çarpın}} \\
 \quad 10111 \\
 \quad 10111 \\
 \quad 00000 \\
 \quad 00000 \\
 \quad 10111 \\
 \hline
 437 \quad 110110101 \quad \text{ürün}
 \end{array}$$

Süreç, çarpınların kopyalarını ardışık olarak ekleyip kaydırmaktan oluşur. Çarpının ardışık bitleri incelenir, en az anlamlı bit önce. Eğer çarpın biti 1 ise, çarpın aşağı kopyalanır; aksi takdirde, 0'lar aşağı kopyalanır. Ardışık satırlarda kopyalanan sayılar, önceki sayıdan bir pozisyon sola kaydırılır. Son olarak, sayılar toplanır ve toplamaları çarpını oluşturur. İki ikili sayının çarpımından elde edilen çarpın, n bit her biri için en fazla $2n$ bit olabilir. Toplama ve kaydırma işlemlerinin algoritma tarafından gerçekleştirildiği açıktır.

Çarpma işlemi dijital donanım ile uygulandığında, süreci biraz değiştirmek uygundur. Öncelikle, ardışık bir makine sentezleme bağlamında, ikili çarpma için ekle ve kaydır algoritmasının tek bir saat döngüsünde veya birden fazla saat döngüsünde gerçekleştirilebileceğini not ediyoruz. Tek bir saat döngüsü süresinde çarpını oluşturma seçeneği, Bölüm 4.7'de tartışılan paralel çarpın devresini sentezleyecektir.

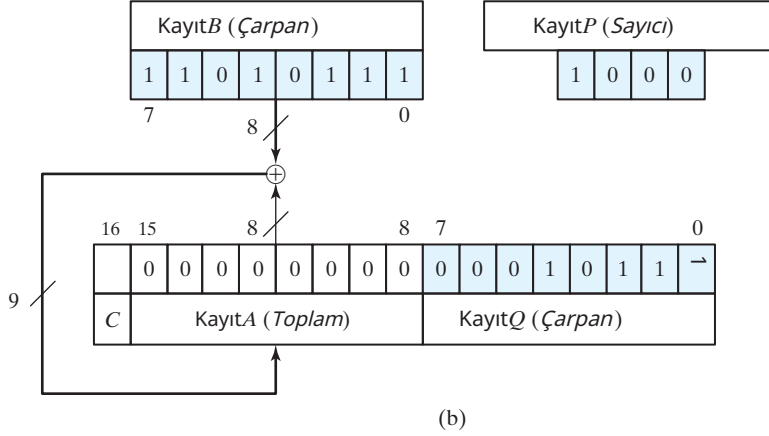
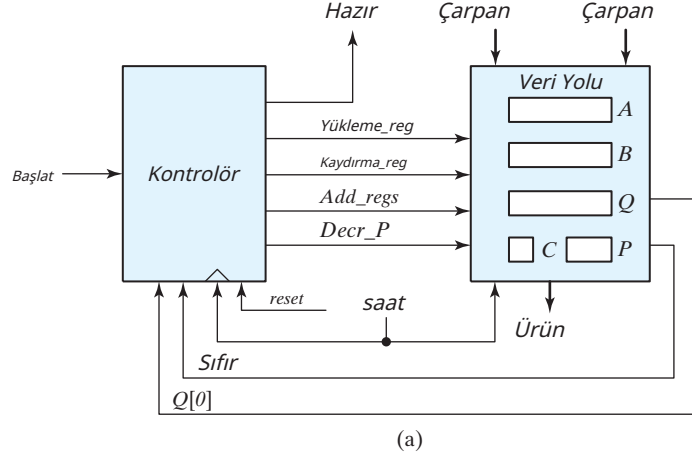
Çarpanı tek bir saat döngüsünde oluşturmak, çarpan ve kısmi çarpanı biriktiren bir RTL modeline eklenir. Çarpan, çarpan ve kısmi çarpanın değerleri kayıtların da saklanır ve içeriklerinin kaydırma ve toplama işlemleri bir durum makinesi kontrolü altında gerçekleştirilir. Çarpma çabasını birden fazla saat döngüsüne dağıtmanın birçok olasılığı arasında, yalnızca bir kısmi çarpanın oluşturulup bir saat döngüsünde biriktirildiği durumu dikkate alacağız. (Bir alternatif, bir saat döngüsünde iki kısmi çarpan oluşturmak ve biriktirmek için ek donanım kullanmak olacaktır, ancak bu daha fazla mantık kapısı ve ya daha hızlı devreler ya da daha yavaş bir saat gerektirecektir.) Çarpanda bulunan 1'ler kadar çok ikili sayıyı aynı anda saklayıp toplamak için dijital devreler sağlamak yerine, yalnızca iki ikili sayıyı toplamak ve kısmi çarpanları bir kayıta biriktirmek için gereken donanımı sağlamak daha az maliyetlidir. İkincisi, çarpanı sola kaydırmak yerine, oluşturulan kısmi çarpan sağa kaydırılır. Bu, kısmi çarpanı ve çarpanı gerekli görece konumlarda bırakır. Üçüncüsü, çarpanın karşılık gelen biti 0 olduğunda, kısmi çarpana tüm 0'ları eklemeye gerek yoktur, çünkü bunu yapmak sonuç değerini değiştirmeyecektir.

Kaydedici Yapılandırması

Sıralı ikili çarpan için bir blok diyagramı Şekil 8.14 (a)'da gösterilmiştir ve veri yolunun kayıt yapılandırması Şekil 8.14 (b)'de gösterilmiştir. Çarpan, kayıt B' 'de saklanır, çarpan kayıt Q' 'de saklanır ve kısmi çarpan kayıt A' 'de oluşturulur ve A ve Q' 'de saklanır. Paralel bir toplayıcı, kayıt B' 'nin içeriğini kayıt A' 'ya ekler. C flip-flop, toplama işleminden sonra taşıyıcıyı saklar. Sayıcı P başlangıçta çarpanın bit sayısına eşit bir ikili sayı tutacak şekilde ayarlanmıştır. Bu sayıcı, her kısmi çarpan oluşturulduktan sonra azaltılır. Sayıcının içeriği sıfıra ulaştığında, çarpan çift kayıt A ve Q' 'de oluşturulur ve işlem durur. Kontrol mantığı, $Start = 1$ olana kadar başlangıç durumunda kalır. Sistem daha sonra çarpma işlemini gerçekleştirir. A ve B' 'nin toplamı, kısmi çarpanın n en anlamlı bitlerini oluşturur ve bu A' 'ya aktarılır. Toplama işleminden çıkan taşıyıcı, ister 0 ister 1 olsun, C' 'ye aktarılır. Hem kısmi çarpan A' 'de hem de çarpan Q' 'de sağa kaydırılır. A' 'nin en az anlamlı biti, Q' 'nin en anlamlı konumuna kaydırılır, C' 'den gelen taşıyıcı A' 'nin en anlamlı konumuna kaydırılır ve 0 C' 'ye kaydırılır. Sağ kaydırma işlemi sonrasında, kısmi çarpanın bir biti Q' 'ye aktarılırken, Q' 'deki çarpan bitleri bir konum sağa kaydırılır. Bu şekilde, kayıt Q' 'nin en az anlamlı biti, $Q[0]$ ile belirtilen, bir sonraki incelenmesi gereken çarpan bitini tutar. Kontrol mantığı, bu giriş bitine dayanarak ekleyip eklemeyeceğine karar verir. Kontrol mantığı ayrıca, sayıcı P 'nin sıfır olup olmadığını kontrol eden bir devreden *Zero* sinyali alır.

$Q[0]$ ve

Zero kontrol birimi için durum girişleridir. Giriş sinyali *Start* harici bir kontrol girişidir. Kontrol mantığının çıktıları, veri yolu birimindeki gerekli işlemleri başlatır.



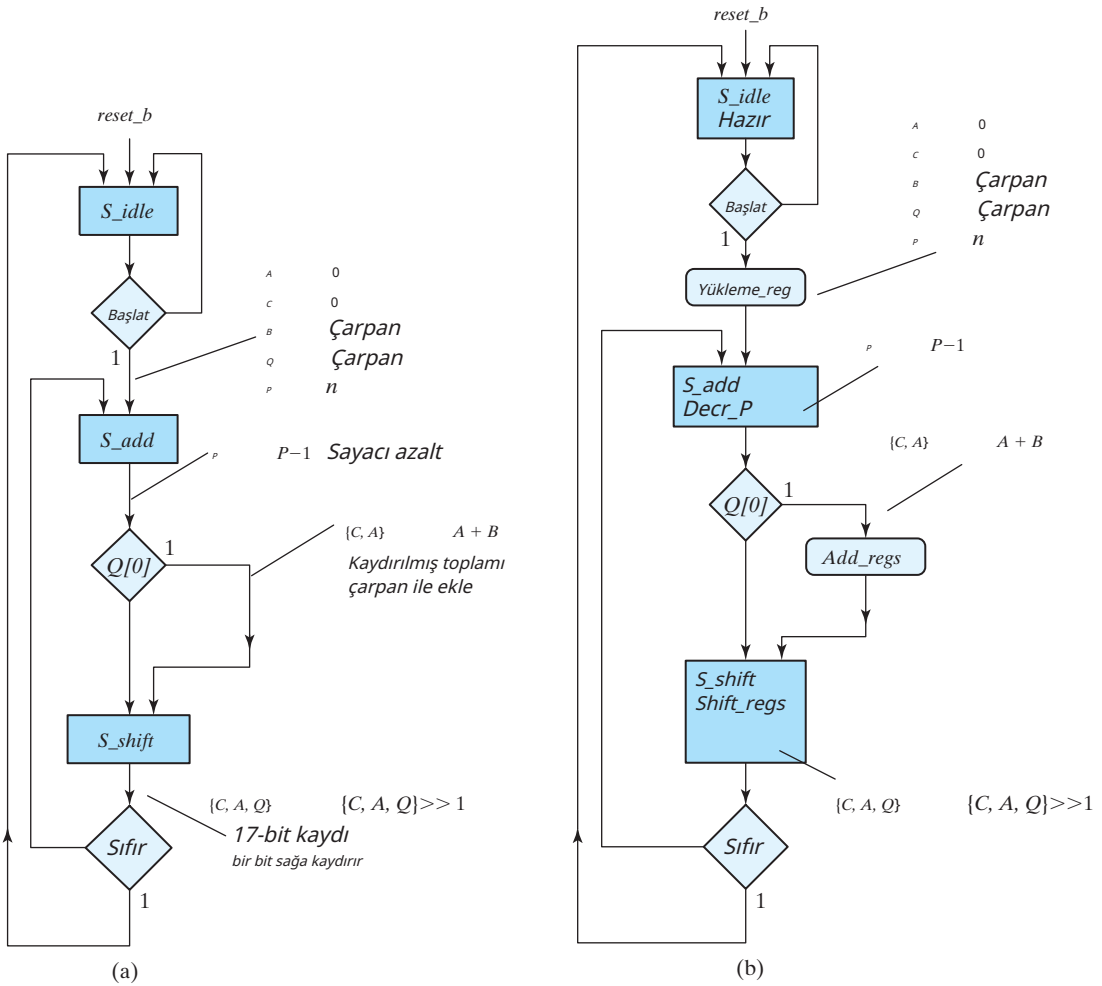
ŞEKİL 8.14

(a) İkili çarpanının blok diyagramı ve (b) veri yolu

Kontrolör ile veri yolu arasındaki arayüz, durum sinyalleri ve kontrolörün çıkış sinyallerinden oluşur. Kontrol sinyalleri, veri yolunun senkron kayıt işlemlerini yönetir. Sinyal *Load_regs* veri yolunun iç kayıtlarını yükler, *Shift_regs* kaydırma kaydını kaydırır, *Add_regs* çarpan ve kayıt A'nın toplamını oluşturur ve *Decr_P* sayacı azaltır. Kontrolör ayrıca, makinenin çarpmaya hazır olduğunu ana ortama bildirmek için *Ready* çıkışını oluşturur. Ürünü tutan kaydın içeriği yürütme sırasında değişir, bu nedenle içeriğinin geçerli olduğunu belirten bir sinyal alın olması faydalıdır. Tekrar belirtmek gerekir ki, kontrol durumunun kontrol birimi ile veri yolu arasındaki bir arayüz sinyali değildir. Sadece veri yolunu kontrol etmek için gereken sinyaller arayüze dahil edilmiştir. Durumu arayüze koymak, veri yolunda bir kodlayıcı gerektirir ve kontrol sinyallerinden daha geniş ve daha aktif bir veri yolu gerektirir. Bu iyi değil.

ASMD Şeması

İkili çarpma için ASMD şeması Şekil 8.15'te gösterilmiştir. Ara form Şekil 8.15 (a)'da kontrolörün ASM şemasını kayıt işlemleri ile not eder ve tamamlanmış şema Şekil 8.15 (b)'de kontrolörün Moore ve Mealy çıktısını tanımlar. Başlangıçta, çarpma işlemi B de ve çarpma Q 'dadır. Devre başlangıç durumunda ve $Start=0$ olduğunda, hiçbir işlem gerçekleşmez ve sistem S_idle durumunda $Ready$ 'nin onaylandığı şekilde kalır. Çarpma işlemi $Start=1$ olduğunda başlatılır. Ardından, (1) kontrol S_add durumu na geçer, (2) kayıt A ve taşıma flip-flop'u C 0'a sıfırlanır, (3) kayıtlar



ŞEKİL 8.15
İkili çarpma için ASMD şeması

B ve Q çarpın ve çarpını yükler, sırasıyla, ve (4) sekans sayacı P çarpında bulunan bit sayısına eşit olan bir ikili sayıya n ayarlanır. Durum $S_add, Q[0]$ içindeki çarpın biti kontrol edilir ve eğer 1'e eşitse, B içindeki çarpın, A içindeki kısmi ürüne eklenir. Toplamadan gelen taş C 'ye aktarılır. A içindeki kısmi ürün ve C değiştirilmez eğer $Q[0] = 0$. Sayaç $P - 1$ azaltılır, $Q[0]$ değerine bakılmaksızın, böylece $Decr_P$ durumu oluşur.

S_add kontrolcünün Moore çıktısı olarak. Her iki durumda da, bir sonraki durum S_shift . Kayıtlar C, A , ve Q birleştirilerek tek bir bileşik kayıt CAQ olarak gösterilir, ve içeriği bir kez sağa kaydırılarak yeni bir parçalı ürün elde edilir. Bu kaydırma işlemi, akış diyagramında Verilog mantıksal sağa kaydırma operatörü ile sembolize edilir, $>>$. Bu, kayıt

aktarım notasyonundaki aşağıdaki ifadeye eşdeğerdir:

$$\text{Shift right } CAQ, C \leftarrow 0$$

Bireysel kayıt sembolleri açısından, kaydırma işlemi aşağıdaki kayıt işlemleriyle tanımlanabilir:

$$\begin{aligned} A &\leftarrow \text{shr } A, A_{n-1} \leftarrow C \\ Q &\leftarrow \text{shr } Q, Q_{n-1} \leftarrow A_0 \\ C &\leftarrow 0 \end{aligned}$$

Hem kayıtlar A hem de Q sağa kaydırılır. A 'nın en soldaki biti, A_{n-1} ile belirtilebilir, C 'den gelen taşı alır. Q 'nın en soldaki biti, Q_{n-1} , A 'nın en sağdaki konumunda n A_0 'da bir biti alır ve C '0'a sıfırlanır. Özünde, bu, C 'deki seri girişi 0 olan bileşik kayıt CAQ için uzun bir kaydırmadır.

Her bir kısmi ürünün oluşumundan sonra sayaç P içindeki değer kontrol edilir. Eğer P içeriği sıfırdan farklıysa, durum biti $Zero$ 0'a eşitlenir ve yeni bir kısmi ürün oluşturmak için işlem tekrarlanır. İşlem, sayaç 0'a ulaştığında ve kontrolörün durum girişi $Zero - 1$ 'e eşit olduğunda durur. A içinde oluşturulan kısmi ürün, Q içine birer birer kaydırılır ve sonunda çarpını değiştirir. Nihai ürün A ve Q içinde mevcuttur; A en anlamlı bitleri, Q ise ürünün en az anlamlı bitlerini tutar.

Önceki sayısal örnek, çarpma işlemini netleştirmek için Tablo 8.5'te tekrarlanmıştır. Prosedür, ASMD şemasında belirtilen adımları takip eder. Tablo'da gösterilen veriler, simülasyon sonuçlarıyla karşılaştırılabilir.

Veri işleme alt sistemi için gereken kayıt türleri, ASMD şemasında listelenen kayıt işlemlerinden türetilir. Kayıt A , toplayıcıdan gelen toplamı kabul etmek için paralel yüklenebilen bir kaydırma kayıdır ve kaydı 0'a sıfırlamak için senkron temizleme yeteneğine sahip olmalıdır. Kayıt Q , bir kaydırma kayıdır. Sayıcı P , bir ikili aşağı sayıcıdır ve bir ikili sabit paralel yükleme imkanı vardır. C flip-flop'u, giriş taşıyıcısını kabul etmek ve senkron temizleme özelliğine sahip olacak şekilde tasarlanmalıdır. Kayıtlar B ve Q , çarpma işleminin başlamasından önce çarpın ve çarpını almak için paralel yükleme yeteneğine ihtiyaç duyar.

Tablo 8.5

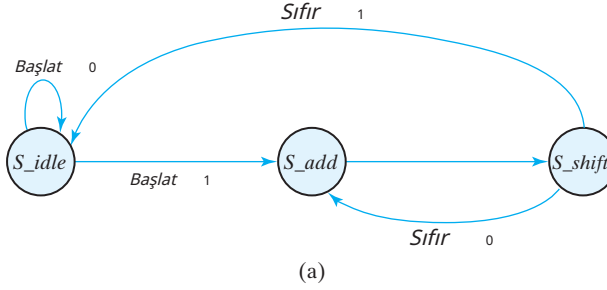
İkili Çarpan için Sayısal Örnek

Çarpan $B = 10111_2 = 17_H = 23_{10}$	Çarpan $Q = 10011_2 = 13_H = 19_{10}$			
	C	A	Q	P
Çarpan içinde Q	0	00000	10011	101
$Q=1$; add B		<u>10111</u>		
İlk kısmi ürün	0	10111	100	
Sağa kaydır CAQ	0	01011	11001	
$Q=1$; add B		<u>10111</u>		
İkinci kısmi ürün	1	00010	011	
Sağa kaydır CAQ	0	10001	01100	
$Q=0$; sağa kaydır CAQ	0	01000	10110	010
$Q=0$; sağa kaydır CAQ	0	00100	01011	001
$Q=1$; add B		<u>10111</u>		
Beşinci kısmi ürün	0	11011		
Sağa kaydır CAQ	0	01101	10101	000
Son ürün $AQ=0110110101_2 = 1b5_H$				

8.8 KONTROL LOJİĞİ

Dijital bir sistemin tasarımı iki parçaya ayrılabilir: veri yolu birimindeki kayıt transferlerinin tasarımı ve kontrol biriminin kontrol mantığının tasarımı. Kontrol mantığı, sonlu durum makinesidir; Mealy ve Moore tipi çıktıları, veri yolunun işlemlerini kontrol eder. Kontrol birimine girişler, birincil (harici) girişler ve veri yolundan kontrolöre geri beslenen iç durum sinyalleridir. Sistem tasarımı, ASMD şemasından türetilen bir RTL tanımından sentezlenebilir. Alternatif olarak, manuel bir tasarım, kontrolörün durumunu tutan flip-flop'lara girişleri yöneten mantığı türetmelidir. Kontrolörün durum diyagramını oluşturmak için gereken bilgi, zaten ASMD şemasında bulunmaktadır; çünkü durum kutularını belirten dikdörtgen bloklar, ardışık devrenin durumlarıdır. Karar kutularını belirten elmas şeklindeki bloklar, durum diyagramındaki bir sonraki durum geçişi için mantıksal koşulları ve koşullu çıktının doğrulamalarını belirler.

Örnek olarak, önceki bölümde geliştirilen ikili çarpan için kontrol durum diyagramı Şekil 8.16 (a)'da gösterilmiştir. Diyagram için bilgiler doğrudan Şekil 8.15'teki ASMD şemasından alınmıştır. Üç durum S_{idle} ile S_{shift} dikdörtgen durum kutularından alınmıştır. Girişler $Start$ ve $Zero$ elmas şeklindeki karar kutularından alınmıştır. Üç durumun her biri için kayıt transfer işlemleri Şekil 8.16 (b)'de listelenmiştir ve ASMD şemasındaki ilgili durum ve koşul kutularından alınmıştır. Durum geçişlerini belirlemek başlangıç odak noktasıdır, bu nedenle kontrolörün çıktıları gösterilmemiştir.



Durum Geçişi		Kayıt İşlemleri
Başlangıç	Son	
S_idle		Başlangıç durumu
S_idle	S_add	$A = 0, C = 0, P = dp_width$
S_add	S_shift	$P = P - 1$
S_shift		if ($Q[0]$) then ($A \leq A + B, C < C_{out}$)
		shift right $[CAQ], C < 0$

(b)

ŞEKİL 8.16**İkili çarpan için kontrol spesifikasyonları**

Kontrol mantığını uygularken iki adım gerçekleştirmeliyiz: (1) gerekli durumların sırasını belirlemek ve (2) kayıt işlemlerini kontrol etmek için sinyaller sağlamaktır. Durumların sırası ASMD şemasında veya durum diyagramında belirtilmiştir. Kayıtları n işlemlerini kontrol etmek için sinyaller, ASMD şemasında anot edilmiş kayıt transfer durumu ifadelerinde veya tablo formatında listelenmiştir. Çarpan için bu sinyaller şunlardır: *Load_regs* (veri yolu birimindeki kayıtları paralel yüklemek için), *Decr_P* (sayacı azaltmak için), *Add_regs* (çarpan ve kısmi çarpımı toplamak için) ve *Shift_regs* (kayıt CAQ kaydırmak için). Kontrol biriminin blok diyagramı Şekil 8.14 (a)'de gösterilmiştir. Kontrolöre girişler *Start*, *Q[0]* ve *Zero*'dur ve çıkışlar *Ready*, *Load_regs*, *Decr_P*, *Add_regs* ve *Shift_regs* olarak ASMD şemasında belirtilmiştir. *Q[0]*'nın yalnızca kontrolörün çıkışını etkilediğini, durum geçişlerini etkilemediğini not ediyoruz. Makine *S_add*'dan *S_shift*'a koşulsuz olarak geçiş yapar.

Tasarımda önemli bir adım, durumlara kodlanmış ikili değerlerin atanmasıdır. En basit atama, Tablo 8.6'da gösterildiği gibi ikili sayıların sıralamasıdır. Başka bir atama, bir sayıdan diğerine geçerken yalnızca bir bitin değiştiği Gray kodudur. Kontrol tasarımında sıkça kullanılan bir durum ataması *one-hot* atama sıdır. Bu atama, devredeki durum sayısı kadar bit ve flip-flop kullanır. Herhangi bir anda yalnızca bir bit 1'e eşittir (sıcak olan).

Tablo 8.6
Kontrol için Durum Ataması

Durum	İkili	Gray Kodu	One-Hot
S_{idle}	00	00	001
S_{add}	01	01	010
S_{shift}	10	11	100

diğer tümüleri 0 (hepsi soğuk) olarak tutulur. Bu tür bir atama, her bir

durum için bir flip-flop kullanır. Gerçekten de, one-hot kodlama, diğer kodlama türlerinden daha fazla flip-flop kullanır, ancak bu genellikle makinenin bir sonraki durumu ve çıktısı için daha basit bir çözme mantığına yol açar. Çözme mantığı, makineye durumlar eklendikçe daha karmaşık hale gelmediğinden, makinenin çalışabileceği hız, durumu çözmek için gereken zamanla sınırlı değildir.

Kontrolör bir ardışık devre olduğundan, Bölüm 5'te özetlenen ardışık mantık prosedürü ile elle tasarlanabilir. Ancak, çoğu durumda bu yöntem, tipik bir kontrol devre sinin sahip olabileceği çok sayıda durum ve girdi nedeniyle elle gerçekleştirilmesi zordur. Sonuç olarak, klasik ardışık mantık yönteminin varyasyonları olarak kabul edilecek kontrol mantığı tasarımı için özel yöntemler kullanmak gereklidir.

Şimdi iki böyle tasarım prosedürünü sunacağız. Biri bir sıra kaydedici ve kod çözücü kullanırken, diğeri her durum için bir flip-flop kullanır. Yöntem küçük bir devre için sunulacak, ancak daha büyük devrelere de uygulanabilir. Elbette, bu yöntemlerin gerekliliği, bir HDL tanımlarından devreyi otomatik olarak sentezleyen bir yazılım varsa ortadan kalkar.

Sıra Kaydedici ve Kod Çözücü

Sıra-kaydedici-ve-kod-çözücü (manuel) yöntemi, adından da anlaşılacağı gibi, kontrol durumları için n bir kaydedici ve her bir duruma karşılık gelen bir çıkış sağlamak için bir kod çözücü kullanır. (Eğer bir one-hot kodu kullanılıyorsa, kod çözücüyü ihtiyaç yoktur.) n flip-flop'a sahip bir kaydedici, en fazla 2^n duruma sahip olabilir ve n'e 2'ye kadar olan bir kod çözücü en fazla 2^n çıkışa sahiptir. n-bit sıra kaydedici esasen n flip-flop ile birlikte, durum geçişlerini etkileyen ilgili kapılarla bir devredir.

İkili çarpanın denetleyicisi için ASMD diyagramı ve durum diyagramı üç duruma ve iki girdiye sahiptir. (Q[0]'ı dikkate almaya gerek yoktur.) Sıra kaydedici ve kod çözücü ile tasarımı uygulamak için kaydedici için iki flip-flop ve iki-üç-dört hat kod çözücüne ihtiyacımız var. Kod çözücünün çıkışları, denetleyicinin Moore tipi çıkışlarını doğrudan oluşturacaktır. Mealy tipi çıkışlar, Moore çıkışları ve girdilerden oluşturulacaktır.

Kontrolörün sonlu durum makinesi için durum tablosu Tablo 8.7'de gösterilmiştir. Bu, Şekil 8.15 (b) veya Şekil 8.16 (a) 'daki ASMD şemasından doğrudan türetilmiştir. İki flip-flop'u G_1 ve G_0 olarak adlandırıyoruz ve ikili durumları S_{idle} , S_{add} ve S_{shift} 'e sırasıyla 00, 01 ve 10 olarak atıyoruz. Giriş sütunlarının, giriş değişkeninin bir sonraki durumu belirlemek için kullanılmadığı durumlarda don't-care (önemsiz) girişleri vardır.

Tablo 8.7

Kontrol Devresi için Durum Tablosu

Mevcut-Durum Sembol	Mevcut Durum		Girişler			Sonraki Eyalet						
	G_1	G_0	Başla	$Q[0]$	Sıfır	G_1	G_0	Hazır	Yükle_regs	Azalt_P	Add_regs	Shift_regs
S_idle	0	0	0	X	X	0	0	1	0	0	0	0
S_idle	0	0	1	X	X	0	1	1	1	0	0	0
S_add	0	1	X	0	X	1	0	0	0	1	0	0
S_add	0	1	X	1	X	1	0	0	0	1	1	0
S_shift	1	0	X	X	0	0	1	0	0	0	0	1
S_shift	1	0	X	X	1	0	0	0	0	0	0	1

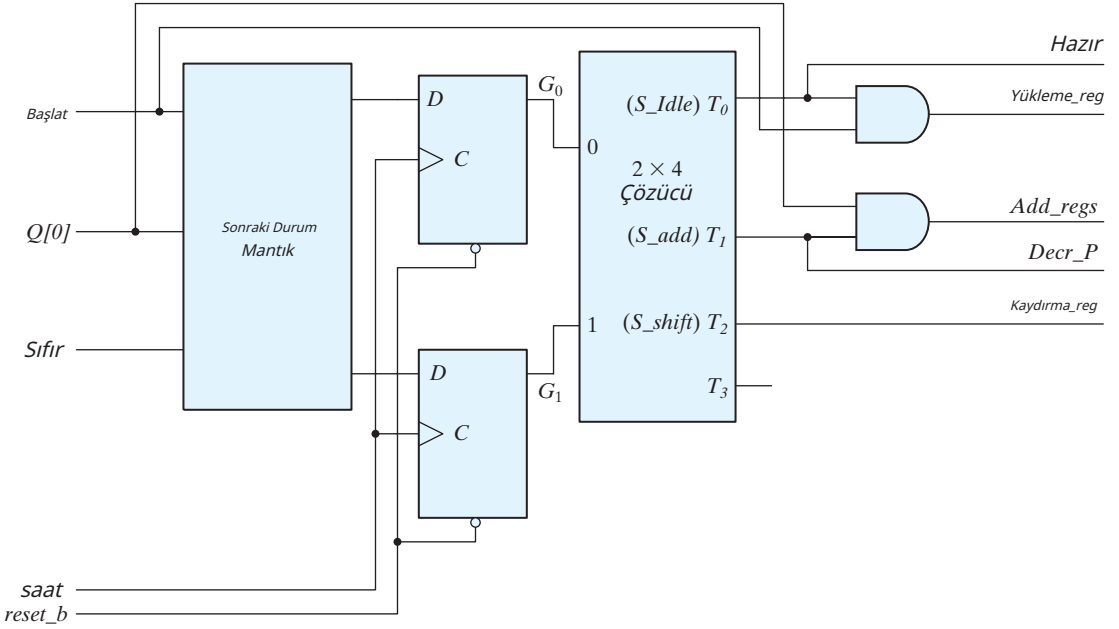
durum. Kontrol devresinin çıkışları ASMD şemasında verilen isimlerle belirtilmiştir. B elirli bir Moore tipi çıkış değişkeni, herhangi bir anda 1'e eşit olan, mevcut durumun eşdeğer ikili değerinden belirlenir. Bu çıkış değişkenleri Tablo 8.7'de gölgelenmiştir. Böylece, mevcut durum $G_1G_0=00$ olduğunda, çıkış *Ready* 1'e eşit olmalı, diğer çıkışlar ise 0'da kalmalıdır. Moore tipi çıkışlar yalnızca mevcut durumun bir fonksiyonu olduğundan, iki girişi olan bir dekoder devresi ile üretilmeleri mümkündür: G_1 ve G_0 ve dekoder çıkışlarının üçü T_0 ile T_2 kullanılarak, durum geri beslemesi için bağlantıları içermeyen Şekil 8.17 (a) 'de gösterildiği gibi.

Kontrolörün durum makinesi, durum tablosundan, Bölüm 5'te sunulan klasik prosedür aracılığıyla tasarlanabilir. Bu örneğin az sayıda durumu ve girişi vardır, bu nedenle Boolean fonksiyonlarını basitleştirmek için haritalar kullanabiliriz. Çoğu kontrol mantığı uygulamasında, durum ve giriş sayısı çok daha fazladır. Genel olarak, klasik yöntemin uygulanması, flip-flop'lar için basitleştirilmiş giriş denklemlerini elde etmek için aşırı miktarda çalışma gerektirir ve hata yapma olasılığı yüksektir. Tasarım, dekoder çıkışlarının tasarımda kullanılabilir olduğunu dikkate alırsak basitleştirilebilir. Flip-flop çıkışlarını mevcut durum koşulları olarak kullanmak yerine, mevcut durum koşullarını belirtmek için dekoderin çıkışlarını kullanıyoruz. Ayrıca, flip-flop denklemlerini basitleştirmek için haritalar kullanmak yerine, durum tablosunun incelenmesiyle doğrudan elde edebiliriz. Örneğin, durum tablosundaki bir sonraki durum koşullarından, G_1 mevcut durum S_add olduğunda 1'e eşit olduğunu ve mevcut durum S_idle veya S_shift olduğunda 0'a eşit olduğunu buluyoruz. Bu koşullar, aşağıdaki denklemle belirtilebilir

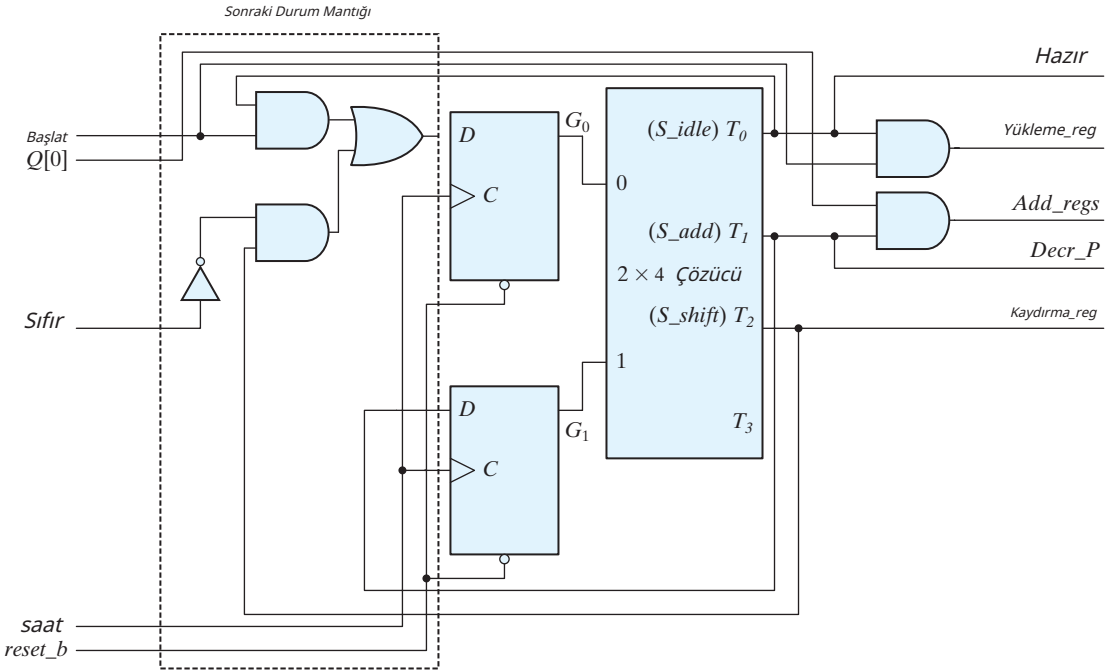
$$D_{G1} = T_1$$

burada D_{G1} flip-flop G_1 için D giriştir. Benzer şekilde, G_0 için D girişi

$$D_{G0} = T_0Start + T_2Zero'$$



(a)



(b)

ŞEKİL 8.17

İkili çarpan için bir sıra kaydedici ve dekoder kullanan kontrolün mantık diyagramı

Durum tablosundan gözlem yoluyla giriş denklemleri türetilirken, Boolean fonksiyonlarının en iyi şekilde basitleştirildiğinden emin olamayız. (Sentetik araçlar bu detayı otomatik olarak halleder.) Genel olarak, türetilen denklemlerin gerekli durum geçişlerini gerçekten ürettiğinden emin olmak için devreyi analiz etmek tavsiye edilir.

Kontrol devresinin mantık diyagramı Şekil 8.17 (b)'de çizilmiştir. İki flip-flop içeren bir kayıt cihazından oluşur: G_1 ve G_0 ve bir 2×4 kodlayıcı. Kodlayıcının çıkışları, bir sonraki durum mantığına girişleri ve kontrol çıkışlarını üretmek için kullanılır. Kontrol cihazının çıkışları, gerekli kayıt işlemlerini etkinleştirmek için veri yoluna bağlanmalıdır.

Tek Isı Tasarımı (Her Durum İçin Bir Flip-Flop)

Kontrol mantığı tasarımının bir diğer yöntemi, her durum için bir flip-flop ile sonuçlanan tek-ısı atamasıdır. Herhangi bir zamanda yalnızca bir flip-flop 1 içerir; diğerleri 0'a sıfırlanır. Tek 1, karar mantığı kontrolünde bir flip-floptan diğerine yayılır. Böyle bir yapılandırmada, her flip-flop, kontrol bitinin ona aktarıldığı zaman mevcut olan bir durumu temsil eder.

Bu yöntem, ardışık devre için maksimum sayıda flip-flop kullanır. Örneğin, 12 durumu olan bir ardışık devre, en az dört flip-flop gerektirir. Buna karşılık, her durum için bir flip-flop yöntemiyle devre, her durum için bir tane olmak üzere 12 flip-flop gerektirir. İlk bakışta, bu yöntemin sistem maliyetini artıracak gibi düşünülebilir, çünkü daha fazla flip-flop kullanılmaktadır. Ancak bu yöntem, görünmeyen bazı avantajlar sunar. Bir avantaj, mantığın ASMD grafiği veya durum diyagramı incelenerek tasarlanmasının sağladığı basitliktir. D türü flip-floplar kullanıldığında, durum veya uyarı tablolarına ihtiyaç yoktur. One-hot yöntemi, tasarım çabasını azaltır, operasyonel basitliği artırır ve toplam kapı sayısında bir azalma sağlayabilir, çünkü bir kodlayıcıya ihtiyaç yoktur.

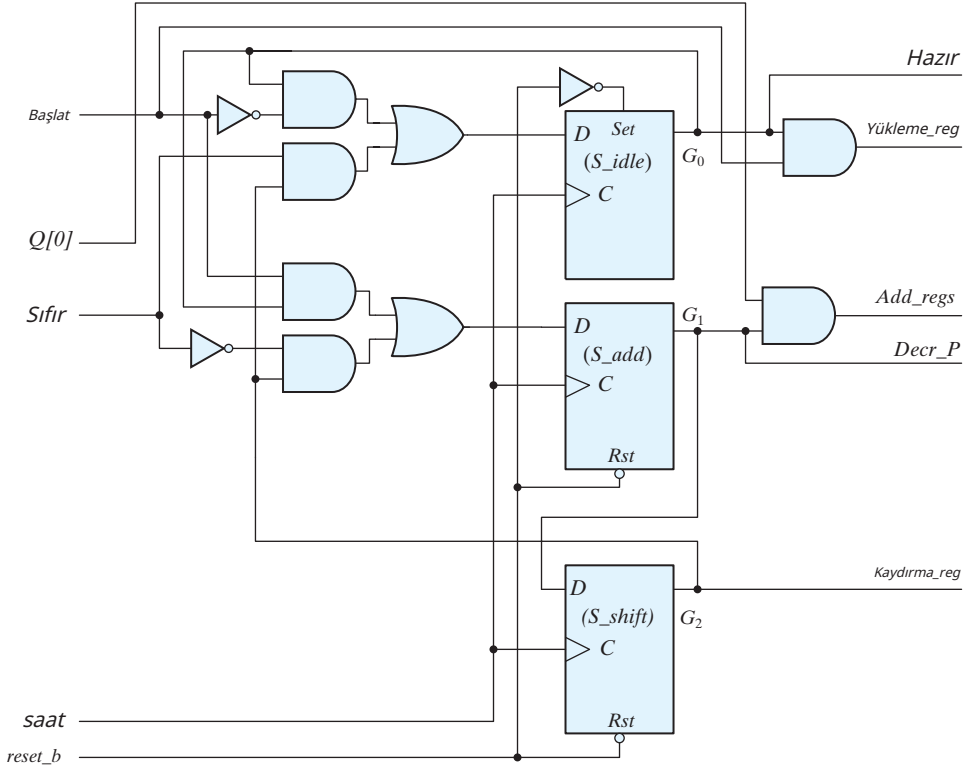
Bir one-hot durum ataması için tasarım prosedürü, Şekil 8.16 (a) durum diyagramı tarafından belirtilen kontrol devresini elde ederek gösterilecektir. Durum diyagramında üç durum bulunduğundan, üç D flip-flop seçiyoruz ve çıkışlarını G_0 , G_1 ve G_2 olarak etiketliyoruz. Her flip-flop 1'e ayarlamak için giriş denklemleri, mevcut durumdan ve duruma giden ilgili yönlendirilmiş hatlar boyunca giriş koşullarından belirlenir. Örneğin, G_0 flip-flopuna giriş olan D, makine G_0 durumundaysa ve Start ifadesi belirtilmemişse veya makine G_2 durumundaysa ve Zero ifadesi belirtilmişse 1'e ayarlanır. Bu koşullar, giriş denklemleriyle belirtilmiştir:

$$D_{G_0} = G_0 Start' + G_2 Zero$$

Aslında, bir flip-flop'u 1'e ayarlamak için gereken koşul, ilgili flip-flop durumuna giden yönlendirilmiş çizgilerde belirtilen koşul ile önceki flip-flop durumunun AND'lenmesiyle doğrudan durum diyagramından elde edilir. Bir duruma giden birden fazla yönlendirilmiş çizgi varsa, tüm koşullar OR'lenmelidir. Diğer üç flip-flop için bu prosedürü kullanarak, kalan giriş denklemlerini elde ederiz:

$$D_{G_1} = G_0 Start + G_2 Zero'$$

$$D_{G_2} = G_1$$



ŞEKİL 8.18

Tek sıcak durum kontrolörü için mantık diyagramı

Bir-hot kontrolörünün mantık diyagramı (her durum için bir flip-flop ile) Şekil 8.18'de gösterilmiştir. Devre, G0'dan G2'ye kadar etiketlenmiş üç D flip-flop'tan oluşur ve giriş denklemleriyle belirtilen ilgili kapılarla birlikte gelir. Başlangıçta, flip-flop G0 1'e ayarlanmalı ve diğer tüm flip-flop'lar 0'a sıfırlanmalıdır, böylece başlangıç durumunu temsil eden flip-flop etkinleştirilir. Bu, flip-flop G0 üzerinde asenkron bir ön ayar ve diğer flip-flop'lar için asenkron bir temizleme kullanılarak yapılabilir. Başladıktan sonra, her durum için bir flip-flop ile kontrolör, uygun şekilde bir durumdan diğerine geçecektir. Her saat kenarında yalnızca bir flip-flop 1'e ayarlanacak; diğerleri 0'a sıfırlanacak, çünkü D girişleri 0'a eşittir.

8.9 İKİLİ ÇARPANIN HDL AÇIKLAMASI

RTL tasarımının HDL açıklamasına dair ikinci bir örnek, Bölüm 8.7'de tasarlanan ikili çarpın için HDL Örneği 8.5'te verilmiştir. Basitlik açısından, tüm açıklama "düzleştirilmiş" ve tek bir modüle kapsüllenmiştir. Yorumlar, kontrolörü ve veri yolunu tanımlayacaktır. Açıklamanın ilk kısmı, belirtilen tüm girişleri ve çıkışları beyan eder.

aki blok diyagramında. Makine, simülasyon verileri ile Tablo 8.5'te listelenen sayısal örneğin çarpım sonucu arasında bir karşılaştırma sağlamak için beş bitlik bir veri yoluna parametreleştirilecektir. Aynı model, yalnızca parametrelerin değerini değiştirerek farklı boyutlarda bir veri yolu için kullanılabilir. Tanımın ikinci kısmı, kontrolördeki ve veri yolundaki tüm kayıtları, ayrıca durumların one-hot kodlamasını bildirir. Üçüncü kısım, birleştirilmiş kayıt için örtük kombinasyonel mantığı (sürekli atama ifadeleri) belirtir *CAQ*, *Sıfır* durum sinyali ve *Hazır* çıkış sinyali. *Zero* ve *Ready* için sürekli atamalar, Boolean ifadesinin **wire** birimlerine atanmasıyla gerçekleştirilir. Sonraki bölüm, kontrol birimini tanımlamakta, durum geçişlerini tanımlamak için tek kenar hassas döngüsel davranış ve sonraki durum ve çıkışlar için kombinasyonel mantığı tanımlamak için seviye hassas döngüsel davranış kullanmaktadır. Yine, varsayılan atamaların *next_state*, *Load_regs*, *Decr_P*, *Add_regs*, ve *Shift_regs* için yapıldığını unutmayın. Durum geçişleri ve çıkış mantığı, Şekil 8.15 (b) ASMD şemasından doğrudan yazılmıştır.

Veri yolu birimi, ayrı bir kenar hassas döngüsel davranış içinde kayıt işlemleri tanımlar.³ (Açıklık için, ayrı döngüsel davranışlar kullanılır; veri yolunun tanımını kontrolörün tanımıyla karıştırmıyoruz.) Her kontrol girişi dekode edilir ve ilişkili işlemleri belirtmek için kullanılır. Toplama ve çıkarma işlemleri, donanım da kombinasyonel mantıkla uygulanacaktır. *Load_regs* sinyali, sayacın ve diğer kayıtların başlangıç değerleriyle yüklenmesine neden olur, vb. Kontrolör ve veri yolu ayrı birimlere ayrıldığı için, kontrol sinyalleri veri yolunun davranışını tamamen belirler; kontrolörün durumu hakkında açık bilgiye ihtiyaç yoktur ve bu bilgi veri yolu birimine sağlanmaz.

Kontrolörün bir sonraki durum mantığı, kullanılmayan kodlardan herhangi birini *S_idle* 'a eşlemek için bir sentez aracını yönlendiren bir varsayılan durum ögesi içerir. Varsayılan durum ögesi ve **case** ifadesinden önceki varsayılan atamalar, makinenin kullanılmayan bir duruma girmesi durumunda geri dönmesini sağlar. Ayrıca, istemeden latch'lerin sentezlenmesini de engeller. (Unutmayın, bir sentez aracı, kombinasyonel mantık olarak tasarlanan bir şeyin aslında mantığın giriş-çıkış fonksiyonunu tamamen belirlemediği durumlarda latch'leri sentezler.)

HDL Örneği 8.5 (Sıralı Çarpın)

modül Sequential_Binary_Multiplier (Ürün, Hazır, Çarpın, Çarpın, Başlat, clock, reset_b);

// Varsayılan yapılandırma: beş bit veri yolu

parametre

çıkı [2*dp_width -1: 0]

çıkış

girdi [dp_width -1: 0]

input

dp_width = 5; // Veri yolunun genişliğine ayarla

Ürün;

Hazır;

Çarpın, Çarpın;

Başla, saat, reset_b;

³ Buradaki veri yolunun genişliği *dp-width* 'dir.

```

parameter          BC_size = 3;    // Bit sayacı boyutu
parametre          S_idle = 3'b001, // one-hot kodu

                                S_add = 3'b010,
                                S_shift = 3'b100;

reg                [2: 0]          durum, sonraki_durum;
reg                [dp_width-1: 0] A, B, Q;          // Veri yolu için boyutlandırılmış
reg                C;
reg                [BC_boyut-1: 0] P;
reg                Yükle_regs, Azalt_P, Ekle_regs, Kaydır_regs;

```

// Çeşitli kombinasyonel mantık

```

atama              Ürün = {A, Q};
wire              Sifir = (P == 0);          // sayaç sıfır
                                // alternatif
                                // Sifir = ~|P;
wire              Ready = (state == S_idle); // kontrolör durumu

```

// kontrol ünitesi

```

her zaman @ (pozitif kenar saat, negatif kenar reset_b)
    Eğer (~reset_b) durum <= S_idle; aksi takdirde durum <= next_state;

her zaman @ (durum, Başlat, Q[0], Sifir) başla
    next_state = S_idle;
    Load_regs = 0;
    Decr_P = 0;
    Add_regs = 0;
    Shift_regs = 0;
    durum (durum)
S_idle:      başla Eğer (Başlat) next_state = S_add; Load_regs = 1; bitir
S_add:      başla next_state = S_shift; Decr_P = 1; Eğer ( Q[0]) Add_regs = 1; bitir
S_shift:    başla Shift_regs = 1; Eğer (Sifir) next_state = S_idle;
                                aksi takdirde next_state = S_add; bitir
    default : sonraki_durum = S_idle;
endcase

```

bitiş

// veri yolu birimi

```

her zaman @ (pozitif kenar saat) başla
    Eğer (Load_regs) başla
        P <= dp_width;
        A <= 0;
        C <= 0;
        B <= Çarpan;
        Q <= Çarpan;
    son
    if (Ekle_regs) {C, A} <= A + B;

```

```

if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
if (Decr_P) P <= P - 1;
son
endmodule

```

Çarpanı Test Etme

HDL Örneği 8.6, çarpanı test etmek için bir test tezgahı göstermektedir. Girişler ve çıkışlar, Şekil 8.14 (a)'da gösterilen blok diyagramındaki ile aynıdır. Bir sistemin HDL tanımının, bir kaç giriş sinyali uygulandığında ürettiği çıktıya dayanarak doğru olduğunu sonuçlandırma k naif bir yaklaşımdır. Test etme ve doğrulama için daha stratejik bir yaklaşım, tasarımın veri yolu ve kontrol birimine bölünmesini kullanır. Bu bölme, kontrolörün ve veri yolunun ayrı doğrulamasını destekler. Veri yolunun her işlemi doğru bir şekilde gerçekleştirdiğini ve durum sinyallerini doğru bir şekilde ürettiğini doğrulamak için ayrı bir test tezgahı geliştirilebilir. Veri yolu birimi doğrulandıktan sonra, bir sonraki adım, her kontrol sinyalinin kontrol birimi tarafından doğru bir şekilde oluşturulduğunu doğrulamaktır. Kontrol biriminin, ASM D şemasında belirtilen tam işlevselliği sergilediğini doğrulamak için ayrı bir test tezgahı kullanılabilir (yani, doğru durum geçişlerini yapması ve dış girişler ile durum sinyallerine yanıt olarak çıktısını sağlaması).

Doğrulanmış bir kontrol birimi ve doğrulanmış bir veri yolu birimi birlikte, sistemin doğru çalışacağını garanti etmez. Tasarım sürecinin son adımı, doğrulanmış modellerin bir ana modül içinde entegre edilmesi ve genel makinenin işlevselliğinin doğrulanmasıdır. Kontrolör ile veri yolu arasındaki arayüz, portların doğru bir şekilde bağlandığını doğrulamak için incelenmelidir. Örneğin, sinyallerin listelenmiş sırasındaki bir uyumsuzluk derleyici tarafından tespit edilmeyebilir. Veri yolu birimi ve kontrol birimi doğrulandıktan sonra, üçüncü bir test tezgahı, sistemin belirtilen işlevselliğini doğrulamalıdır. Pratikte, bu işlevselliği tanımlayan kapsamlı bir test planı yazmayı gerektirir. Örneğin, test planı, ardışık çarpmanın *Ready* sinyalini *S_idle* durumunda doğrulaması gerektiğini belirtecektir. Bir test planı yazma egzersizi akademik değildir: Test planının kalitesi ve kapsamı, doğrulama çabasının değerini belirler. Test planı, test tezgahının geliştirilmesine rehberlik eder ve nihai tasarımın spesifikasyonuna uygun olma olasılığını artırır.

Bir HDL modelini test etmek ve doğrulamak genellikle makinenin giriş ve çıkışlarından daha fazla bilgiye erişim gerektirir. Kontrol biriminin durumu, kontrol sinyalleri, durum sinyalleri ve veri yolunun iç kayıtları hakkında bilgi, hata ayıklama için gerekli olabilir. Neyse ki, Verilog, tasarım hiyerarşisinin herhangi bir seviyesindeki herhangi bir değişkenin test tezgahına görünür olmasını sağlamak için tanımlayıcıları hiyerarşik olarak de-referanslama mekanizması sunar. Prosedürel ifadeler, makinenin hata ayıklama çabalarını desteklemek için gereken bilgileri gösterebilir. Simülatörler, tasarım hiyerarşisindeki herhangi bir değişkeni n dalga formlarını görüntülemek için bu mekanizmayı kullanır. Mekanizmayı kullanmak için, değişkeni hiyerarşik yol adıyla referans alırız. Örneğin, *P* kaydı içinde

veri yolu birimi çarpanın bir çıkış portu değildir, ancak şu şekilde referans alınabilir *M0.P*. Hiyerarşik yol adı, modül tanımlayıcıları veya blok adlarının ardışık dizisinden oluşur, noktalarla ayrılır ve değişkenin tasarım hiyerarşisindeki konumunu belirtir. Ayrıca, simülatörlerin genellikle bir tasarımın hiyerarşisinin tüm seviyelerini gösteren bir grafik kullanıcı arayüzüne sahip olduğunu belirtelim.

HDL Örneği 8.6'daki ilk test tezgahı, hesaplamaların sonucunu göstermek için sistem görevi **\$strobe** kullanır. Bu görev, Bölüm 4.12'de açıklanan **\$display** ve **\$monitor** görevlerin e benzer. **\$strobe** sistem görevi, belirli bir zaman adımındaki tüm atamaların yürütüldükten sonra verilerin yalnızca görüntülenmesini sağlamak için bir senkronizasyon mekanizması sağlar. Bu, zaman adımının bir saat kenarında başladığı ve birden fazla atamanın simülasyonun aynı zaman adımımda gerçekleşebileceği senkronize ardışık devrelerde çok kullanışlıdır. Sistem saat sinyalinin pozitif kenarına senkronize olduğunda, **\$strobe** ifadesinden sonra **always @ (posedge clock)** ifadesi, ekranın saat darbesinden sonraki sinyalin değerlerini göstermesini sağlar.

Test tezgahı modülü *t_Sequential_Binary_Multiplier* HDL Örneği 8.6'da modülü *Sequential_Binary_Multiplier* HDL Örneği 8.5'ten örneklenmiştir. Her iki modül simüle edilirken kaynak dosyalar olarak dahil edilmelidir. Bu simülasyonun sonucu, Tablo 8.5'teki numaralarla aynı olan bir simülasyon günlüğü gösterir. Kod, çarpan ve çarpan değerlerinin beş bitlik çarpımını kapsamlı bir şekilde test etmek için ikinci bir test tezgahı içerir. Simülasyon sonuçlarının bir örneği için dalga biçimleri Şekil 8.19'da gösterilmektedir. *Çarpan*, *Çarpan* ve *Ürün* sayısal değerleri ondalık ve on altılık formatlarda gösterilmektedir. Kontrol durumu, kontrol sinyalleri, durum sinyalleri ve kayıt işlemlerinin gösterilen dalga biçimlerini inceleyerek içgörü kazanılabilir. Çarpan ve test tezgahındaki geliştirmeler, bu bölümün sonunda yer alan problemlerle ele alınmaktadır. Bu örnekte, $19_{10} \times 23_{10} = 437_{10}$, ve $17_H + 0b_H = 02_H$ $C = 1$ içi geçerlidir. Taşıma bitinin gerekliliğine dikkat edin.

HDL Örneği 8.6

// İkili çarpan için test tezgahı

modül t_Sequential_Binary_Multiplier;

parametre

tel [2*dp_width -1: 0]

wire

reg [dp_width -1: 0]

reg

// Çarpanı başlat

Sequential_Binary_Multiplier M0 (Ürün, Hazır, Çarpan, Çarpan, Başla, saat, sıfırla_b);

// Uyarı dalga formlarını oluştur

ilk #200 \$finish ;

ilk

başla

Başla = 0;

dp_width = 5;

// Veri yolunun genişliğine ayarla

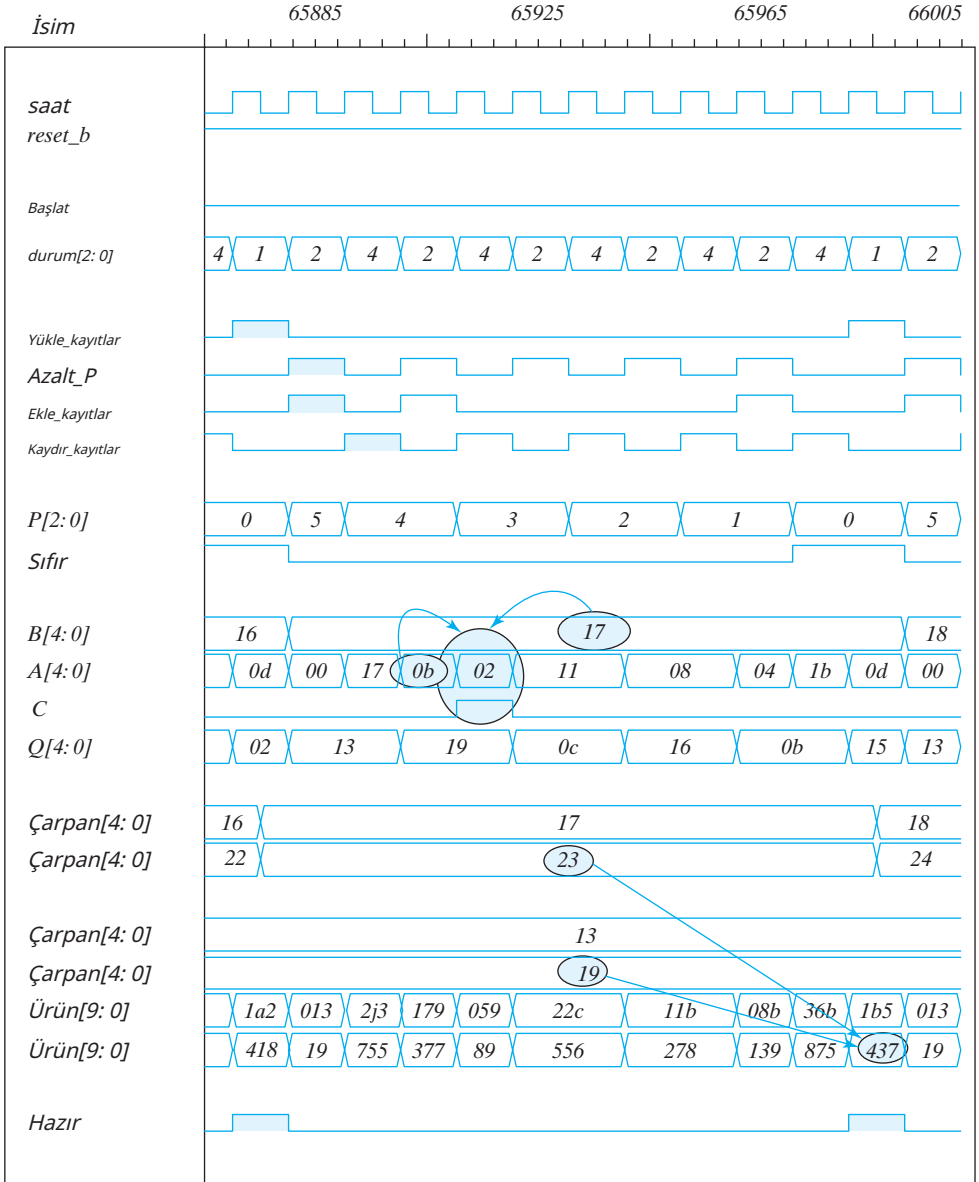
Ürün;

// Çarpandan çıkan

Hazır;

Çarpan, Çarpan; // Çarpana girişler

Başla, saat, reset_b;



ŞEKİL 8.19

Bir sıcak durum denetleyicisi için simülasyon dalga formları

408 Bölüm 8 Kayıt Transfer Seviyesinde Tasarım

```
sıfırla_b = 0;
#2 Başla = 1; reset_b = 1;
Çarpan = 5'b10111; Çarpıcı = 5'b10011;
#10 Başla = 0;
son
başlangıç
başla
saat = 0;
tekrar (26) #5 saat = ~saat;
son
// Sonuçları göster ve Tablo 8.5 ile karşılaştır
her zaman @ ( yükselen saat)
$strobe ("C=%b A=%b Q=%b P=%b zaman=%0d",M0.C,M0.A,M0.Q,M0.P, $zaman );
bitir
```

Simülasyon kaydı:

```
C=0 A=00000 Q=10011 P=101 zaman=5
C=0 A=10111 Q=10011 P=100 zaman=15
C=0 A=01011 Q=11001 P=100 zaman=25
C=1 A=00010 Q=11001 P=011 zaman=35
C=0 A=10001 Q=01100 P=011 zaman=45
C=0 A=10001 Q=01100 P=010 zaman=55
C=0 A=01000 Q=10110 P=010 zaman=65
C=0 A=01000 Q=10110 P=001 zaman=75
C=0 A=00100 Q=01011 P=001 zaman=85
C=0 A=11011 Q=01011 P=000 zaman=95
C=0 A=01101 Q=10101 P=000 zaman=105
C=0 A=01101 Q=10101 P=000 zaman=115
C=0 A=01101 Q=10101 P=000 zaman=125
```

/* Kapsamlı simülasyon için test tezgahı

modül t_Sıralı_Ikili_Çarpan;

```
parameter dp_width = 5; // Veri yolunun genişliği
tel [2 * dp_width -1: 0] Ürün;
tel Hazır;
reg [dp_width -1: 0] Çarpan, Çarpıcı;
reg Başla, saat, reset_b;
Sequential_Binary_Multiplier M0 (Ürün, Hazır, Çarpan, Çarpıcı, Başla, saat,
reset_b);
başlangıç #1030000 $finish ;
başlangıç başla saat = 0; #5 sonsuza dek #5 saat = ~saat; son
başlangıç fork
reset_b = 1;
#2 reset_b = 0;
#3 reset_b = 1;
join
```

```

initial begin #5 Başlangıç = 1; son
initial begin
#5 Çarpan = 0;
Çarpan = 0;
tekrar (32) #10 begin Çarpan = Çarpan + 1;
tekrar (32) @ (posedge M0.Ready) 5 Çarpan = Çarpan + 1;
son
end
endmodule
*/

```

Paralel Çarpan için Davranışsal Tanım

Yapısal modelleme, bir dijital makinenin işlevselliğini kapı düzeyindeki donanım birimlerinin birbiriyle bağlantısını belirleyerek dolaylı olarak tanımlar. Bu modelleme biçiminde, bir sentez aracı Boolean optimizasyonu gerçekleştirir ve bir devrenin HDL tanımını belirli bir teknolojiye, örneğin CMOS'a, ait kapı listesine çevirir. Bu seviyedeki donanım tasarımı genellikle zeka ve birikmiş deneyim gerektirir. En zahmetli ve detaylı modelleme biçimidir. Buna karşılık, davranışsal RTL modelleme işlevselliği soyut bir şekilde, HDL operatörleri açısından tanımlar. RTL modeli, kayıtların kapı düzeyindeki uygulanışını veya içeriklerini manipüle eden işlemleri kontrol eden mantığı belirtmez—bu görevler bir sentez aracı tarafından gerçekleştirilir. RTL modelleme, işlemleri açıkça saat döngülerine atayarak dolaylı olarak planlar. Davranışsal modellemenin en soyut biçimi yalnızca bir algoritmayı tanımlar, fiziksel bir uygulama, bir kaynak seti veya bunların kullanımı için bir program referansı olmaksızın. Böylece, algoritmik modelleme bir tasarımcının, donanım karmaşıklığına karşı işleme hızını değiş tokuş etme alanında (donanım) ve zaman alanında trade-off'ları keşfetmesine olanak tanır.

HDL Örneği 8.7, bir ikili çarpan için bir RTL modeli ve bir algoritmik model sunmaktadır. Her ikisi de seviye duyarlı döngüsel bir davranış kullanır. RTL modeli, bir çarpanın işlevselliğini tek bir ifadeyle ifade eder. Bir sentez aracı, çarpma operatörünü Bölüm 4.7'de gösterilen kapı düzeyindeki devre ile ilişkilendirecektir. Simülasyonda, çarpan veya çarpan değiştiğinde, ürün güncellenecektir. Ürünü oluşturmak için gereken süre, sentez aracının kullandığı standart hücre kütüphanesindeki kapıların yayılma gecikmelerine bağlı olacaktır. İkinci model, çarpanın algoritmik bir tanımıdır. Bir sentez aracı, algoritmanın döngüsünü açacak ve Bölüm 4.7'de gösterilen kapı düzeyindeki devreye ihtiyaç duyulduğunu çıkaracaktır.

Bir sentez aracının, ilişkili HDL modelinin simüle edilip doğru sonuçlar üretmesi ne rağmen, belirli bir algoritmik tanım sentezleyemeyeceğini unutmayın. Bir zorluk, bir algoritmanın ima ettiği işlem sırasının tek bir saat döngüsünde fiziksel olarak gerçekleştirilemeyecek olmasıdır. Bu durumda, işlemlerin birden fazla saat döngüsüne dağıtılması gerekecektir. RTL mantığını sentezlemek için bir araç, gerekli çaba dağıtımını otomatik olarak gerçekleştiremeyecektir, ancak bir araç ki

HDL Örneği 8.7

```

// Paralel bir çarpanın (n = 8) Davranışsal (RTL) tanımı
modül Çarp (Ürün, Çarpan, Çarpan);
    input [7: 0] Çarpan, Çarpan;
    output reg [15: 0] Ürün;
    her zaman @ (Çarpan, Çarpan)
        Ürün = Çarpan * Çarpan;
endmodule

modül Algoritmik_Ikili_Çarpan #( parametre dp_genişlik = 5 ) (
    output [2*dp_genişlik -1: 0] Ürün, input [dp_genişlik -1: 0] Çarpan, Çarpan);
    reg [dp_genişlik -1: 0] A, B, Q; // Veri yolu için boyutlandırılmış
    reg C; // Veri yolu için boyutlandırılmış
    integer k;
    Product = {C, A, Q};
    always @ (Multiplier, Multiplicand) begin
        Q = Multiplier;
        B = Multiplicand;
        C = 0;
        A = 0;
        for(k = 0; k <= dp_width -1; k = k + 1) begin
            if (Q[0]) {C, A} = A + B;
            {C, A, Q} = {C, A, Q} >> 1;
        end
    end
endmodule

modül t_Algoritmik_Ikili_Çarpan;
    parametre dp_width = 5; // Veri yolunun genişliği
    wire [2 * dp_width -1: 0] Ürün;
    reg [dp_width -1: 0] Çarpan, Çarpan;
    integer Exp_Değeri;
    Hata;
    Algorithmic_Binary_Multiplier M0 (Ürün, Çarpan, Çarpan);
    // Hata tespiti
    initial # 1030000 bitiş ;
    her zaman @ (Ürün) başla
        Exp_Value = Çarpan * Çarpan;
        // Exp_Value = Çarpan * Çarpan +1; // Hata engelleme için hata enjekte et
        Hata = Exp_Value ^ Ürün;
    son
    // 5 bit operandlar için çarpan ve çarpanı kapsamlı bir şekilde üret
    ilk başla
        #5 Çarpan = 0;
        Çarpan = 0;

```

```

tekrar (32) #10 başla Çarpan = Çarpan + 1;
tekrar (32) #5 Çarpan = Çarpan + 1;
son
end
modül sonu

```

algoritmaları sentezlemek için tasarlanmıştır ve başarılı olmalıdır. Aslında, bir davranışsal sentez aracı, çarpma işlemini gerçekleştirmek için kayıtları ve toplayıcıları tahsis etmelidir. Eğer tüm işlemler için yalnızca tek bir toplayıcı paylaşılacaksa, etkinlik birden fazla saat döngüsüne ve doğru sıraya dağıtılmalıdır; bu da nihayetinde, veri yolunun kontrolörü için açıkça tasarladığımız ardışık ikili çarpma yol açar. Davranışsal sentez araçları, farklı ve daha karmaşık bir modelleme tarzı gerektirir ve bu metnin kapsamı dışındadır.

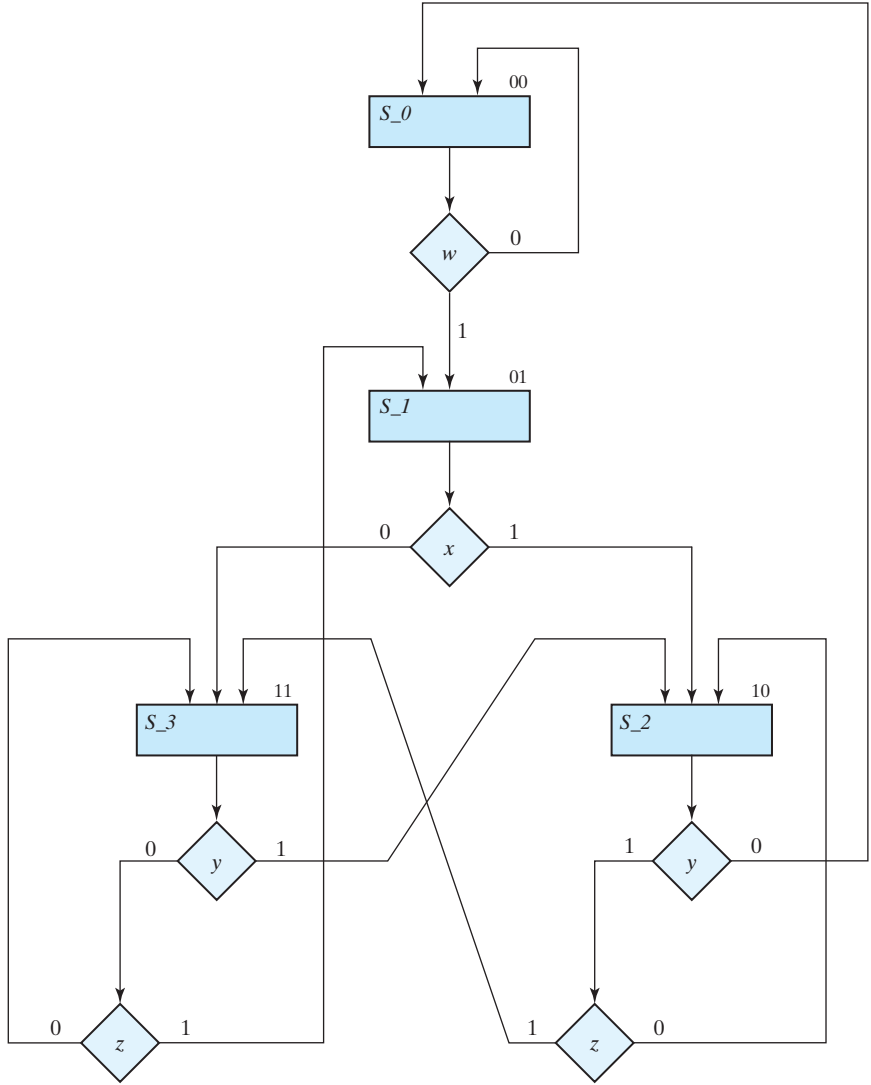
8.10 ÇOKLU KAYNAKLARLA TASARIM

Kontrolör tasarımı için kayıt ve kodlayıcı şeması üç bölümden oluşur: ikili durum değişimini tutan flip-flop'lar, kontrol çıktıları üreten kodlayıcı ve bir sonraki durum ve çıkış sinyallerini belirleyen kapılar. Bölüm 4.11'de, bir kombinasyon devresinin bir eysel kapılar yerine çoklayıcılar (multiplexer) ile uygulanabileceği gösterilmiştir. Kapıları çoklayıcılarla değiştirmek, üç seviyeli bileşenlerin düzenli bir desenini ortaya çıkarır. İlk seviye, kaydın bir sonraki durumunu belirleyen çoklayıcılardan oluşur. İkinci seviye, mevcut ikili durumu tutan bir kayıttan oluşur. Üçüncü seviye, her kontrol durumu için benzersiz bir çıkış hattını belirten bir kodlayıcıya sahiptir. Bu üç bileşen, birçok entegre devrede önceden tanımlanmış standart hücrelerdir.

Örneğin, dört durum ve dört kontrol girişi içeren Şekil 8.20'deki ASM grafiğini düşünelim. Sadece durum dizisini yöneten kontrol sinyalleriyle ilgileniyoruz.

Bu sinyaller, veri yolunun kayıt işlemlerinden bağımsızdır, bu nedenle grafiğin kenarları veri yolu kayıt işlemleri ile not edilmemiştir ve grafik kontrolörün çıkış sinyallerini tanımlamaz. Her durum için ikili atama, durum kutularının sağ üst köşesin de belirtilmiştir. Karar kutuları, dört kontrol girişinin bir fonksiyonu olarak durum geçişlerini belirtir: w , x , y , ve z . Şekil 8.21'de gösterilen üç seviyeli kontrol uygulaması, iki çoklayıcı, MUX1 ve MUX2; iki flip-flop ile bir kayıt, G_1 ve G_0 ; ve dört çıkışı olan bir kodlayıcı— d_0, d_1, d_2 , ve d_3 , sırasıyla S_0, S_1, S_2 , ve S_3 'ye karşılık gelir. Durum-kayıt flip-flop'larının çıkışları, kodlayıcı girişlerine ve ayrıca çoklayıcıların seçim girişlerine uygulanır.

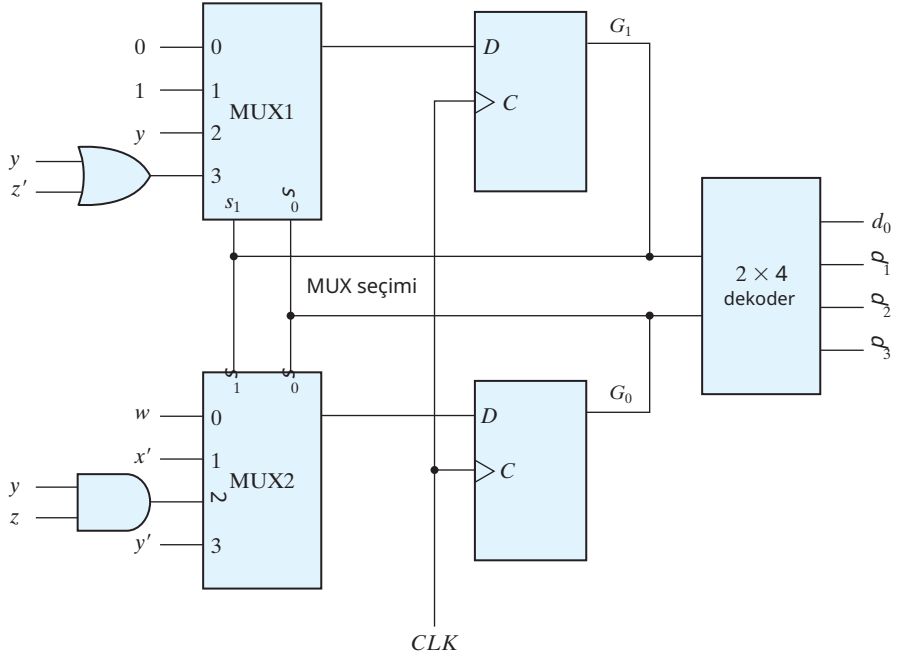
Bu şekilde, kayıt defterinin mevcut durumu, her çoklayıcıdan bir girişi seçmek için kullanılır. Çoklayıcıların çıkışları daha sonra D girişlerine uygulanır G_1 ve G_0 . Her çoklayıcının amacı, bir flip-flop'a, bir sonraki durum vektörünün o bitinin ikili değerine eşit bir girdi üretmektir. Çoklayıcıların girişleri



ŞEKİL 8.20

Dört kontrol girişi ile ASM grafiği örneği

ASM şemasında verilen karar kutuları ve durum geçişlerinden belirlenir. Örneğin, durum 0 0, giriş w değerine bağlı olarak 00'da kalır veya 01'e geçer. Her iki durumda da G_1 için bir sonraki durum 0 olduğundan, MUX1 giriş 0'a mantık 0'a eşit bir sinyal yerleştiririz. G_0 için bir sonraki durum 0 ise $w = 0$ ve 1 ise $w = 1$. G_0 için bir sonraki durum w 'ye eşit olduğundan, kontrol girişi w MUX2 giriş 0'a uygulanır. Bu, çoklayıcıların seçim girişleri mevcut durum 00'a eşit olduğunda, çoklayıcıların çıkışlarının bir sonraki saat darbesinde kayda aktarılan ikili değeri sağladığı anlamına gelir.



ŞEKİL 8.21
Multiplexer'lar ile kontrol uygulaması

Çoklayıcı girişlerinin değerlendirilmesini kolaylaştırmak için, ASM şemasındaki her olası durum geçişi için giriş koşullarını gösteren bir tablo hazırlıyoruz. Tablo 8.8, Şekil 8.2 0 için bu bilgiyi vermektedir. Mevcut durum 00 veya 01'den iki geçiş ve mevcut durum 1 0 veya 11'den üç geçiş vardır. Geçiş setleri, tablonun üzerinde yatay çizgilerle ayrılmıştır. Tabloda listelenen giriş koşulları, ASM şemasındaki karar kutularından elde edilmiştir. Örneğin, Şekil 8.20'den, mevcut durum 01'in, $x=1$ olduğunda bir sonraki durum 10'a veya $x=0$ olduğunda bir sonraki durum 11'e geçeceğini not ediyoruz. Tabloda, bu giriş koşullarını sırasıyla x ve x' olarak işaretliyoruz. Tablodaki "çoklayıcı girişleri" altındaki iki sütun, MUX1 ve MUX2'ye uygulanması gereken giriş değerlerini belirtmektedir. Her mevcut durum için çoklayıcı girişi, flip-flop'un bir sonraki durumu 1'e eşit olduğunda giriş koşullarından belirlenir. Böylece, mevcut durum 01'den sonra, G_1 her zaman 1'e eşit olur ve G_0 bir x 'in tamamlayıcısı na eşit olur. Bu nedenle, kayıt cihazının mevcut durumu 01 olduğunda MUX1'in girişi 1'e ve MUX2'nin girişi x 'e eşitlenir. Başka bir örnek olarak, mevcut durum 10'dan sonra, giriş koşulları yz veya $y'z$ olduğunda G_1 1'e eşit olmalıdır. Bu iki Boole terimi birleştirildiğinde ve basitleştirildiğinde, tabloda gösterildiği gibi tek bir ikili değişken y elde ederiz. G_0 bir sonraki durumu, giriş koşulları $yz=11$ olduğunda 1'e eşittir. Eğer bir mevcut durumdan sonra G_1 0'da kalıyorsa, çoklayıcı girişine 0 koyarız, mevcut durum 00'da gösterildiği gibi.

Tablo 8.8
Multiplexer Giriş Koşulları

Mevcut Durum		Sonraki Eyalet		Giriş Koşulu	Girişler	
G_1	G_0	G_1	G_0	s	MUX1	MUX2
0	0	0	0	w		
0	0	0	1	w	0	w
0	1	1	0	x		
0	1	1	1	x'	1	x'
1	0	0	0	y'		
1	0	1	0	yz'		
1	0	1	1	yz	$yz' + yz = y$	yz
1	1	0	1	$y'z$		
1	1	1	0	y		
1	1	1	1	$y'z'$	$y + y'z' = y + z'$	$y'z + y'z' = y'$

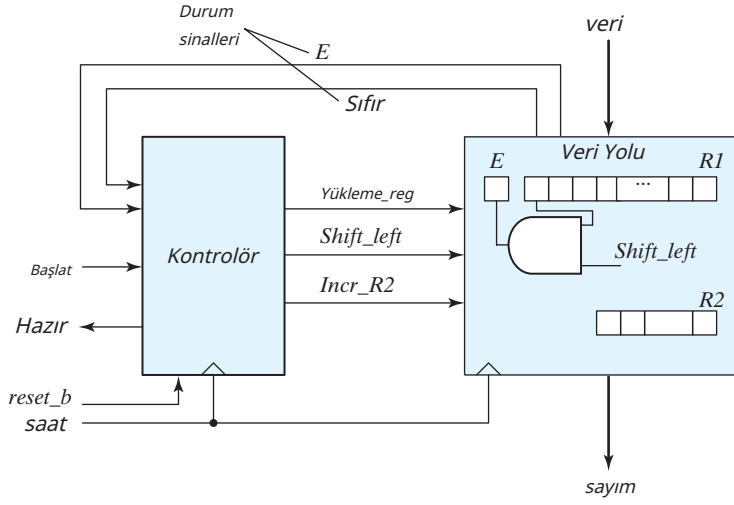
MUX1. Eğer G_1 'in bir sonraki durumu her zaman 1 ise, çoklayıcı girişine 1 yerleştiririz, bu MUX1 için mevcut durum 01'de gösterildiği gibidir. MUX1 ve MUX2 için diğer girişler benzer bir şekilde türetilir. Tablo'dan gelen çoklayıcı girişleri, Şekil 8.21'deki kontrol uygulamasında kullanılır. Bir flip-flop'un bir sonraki durumu iki veya daha fazla kontrol değişkeninin bir fonksiyonu ise, çoklayıcı girişinde bir veya daha fazla kapı gerekebilir. Aksi takdirde, çoklayıcı girişi kontrol değişkenine, kontrol değişkeninin tamamlayıcısına, 0'a veya 1'e eşittir.

Tasarım Örneği: Bir Kayıttaki Birlerin Sayısını Sayma

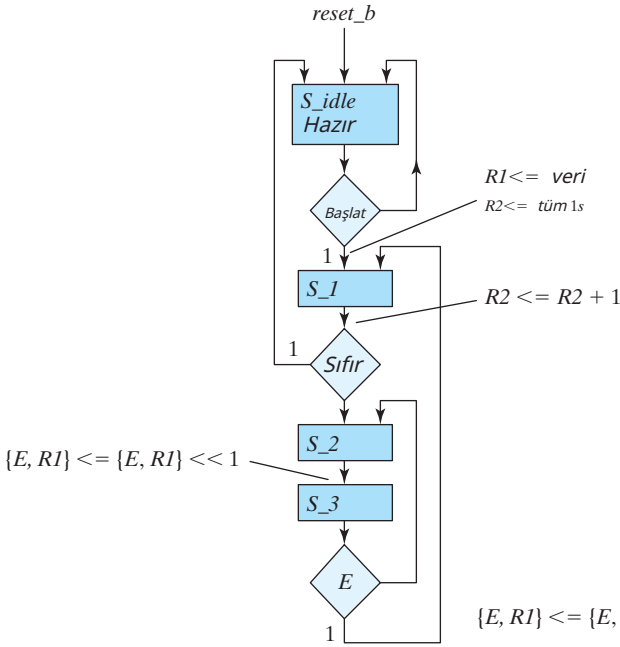
Bir kontrol biriminin mantığının çoklayıcı uygulamasını, bir veri kelimesindeki 1'lerin sayısını sayacak bir sistem tasarım örneği ile göstereceğiz. Örnek, ASM D grafiğinin formülasyonunu ve veri yolu alt sisteminin uygulanmasını da göserecektir.

Çeşitli alternatifler arasından, iki kayıt R1 ve R2 ile bir flip-flop E'den oluşan bir birler sayıcıyı ele alacağız. (Daha verimli bir uygulama, bölümün sonunda yer alan problemlerle ele alınmaktadır.) Sistem, R1 kaydına yüklenen sayıda 1'lerin sayısını sayar ve R2 kaydını bu sayıya ayarlar. Örneğin, R1'e yüklenen ikili sayı 10111001 ise, devre R1'deki beş 1'i sayar ve R2 kaydını ikili sayım 101'e ayarlar. Bu, her bir biti R1 kaydından birer birer flip-flop E'ye kaydırarak yapılır. E'deki değer kontrol tarafından kontrol edilir ve her seferinde 1'e eşit olduğunda, R2 kaydı 1 artırılır.

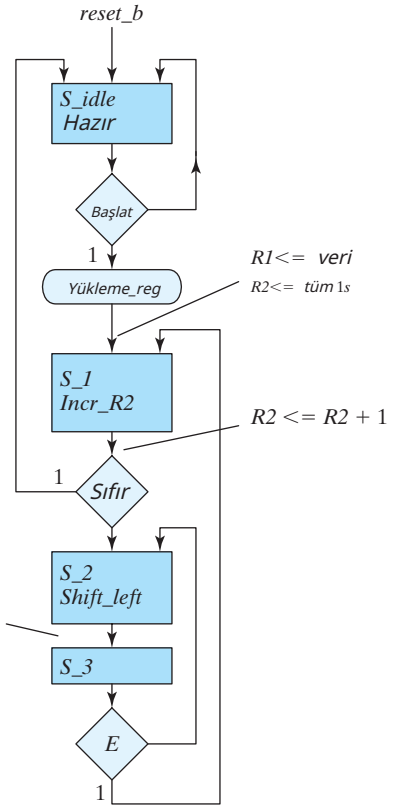
Veri yolu ve kontrolörün blok diyagramı Şekil 8.22 (a)'da gösterilmektedir. Veri yolu, R1, R2 ve E kayıtlarını içerir ve ayrıca R1'in en soldaki bitini E'ye kaydırmak için mantık içerir. Birim ayrıca R1'in 0 olup olmadığını tespit etmek için bir NOR kapısı gibi mantık içerir, ancak bu



(a)



(b)



(c)

ŞEKİL 8.22

Sayım-birleri devresi için blok diyagramı ve ASMD grafiği

Şekilde detaylar atlanmıştır). Dışsal giriş sinyali *Başla* makinenin çalışmasını başlatır; *Hazır* makinenin durumunu dış çevreye gösterir.

Kontrolör, veri yolundan *E* ve *Zero* durum giriş sinyallerine sahiptir. Bu sinyaller, bir kaydın en yüksek anlamlı bitini (MSB) tutan içeriğini ve veri kelimesinin 0 olduğu koşulunu belirtir. *E* flip-flop'un çıkışıdır. *Zero* ise, *R1* kaydının tüm 0'ları kontrol eden bir devrenin çıkışıdır. Devre, *R1* 0'a eşit olduğunda (yani, *R1* 1'lerden boş olduğunda) çıkış *Zero* = 1 üretir.

Bir ön ASMD grafiği, durum dizisini ve kayıt işlemlerini göstermektedir. Şekil 8.22 (b)'de ve tam ASMD grafiği Şekil 8.22 (c)'de gösterilmektedir. İddia etmek *Başla* denetleyici ile *S_idle* durumu *S_1*'ye aktarır, aynı anda reg-ister *R1* i kili veri kelimesi ile yüklenir ve *R2* hücreleri 1'ler ile doldurulur. Tüm 1'ler ile bir sayıyı artırmanın, bir sayaç kaydında tüm 0'lar ile bir sayı üretir.

Bu nedenle, *S_1* den *S_2* ye ilk geçiş *R2* 'yi temizleyecektir. Sonraki geçişlerde *R2* işlenmiş veri bitlerinin sayısını tutacaktır.

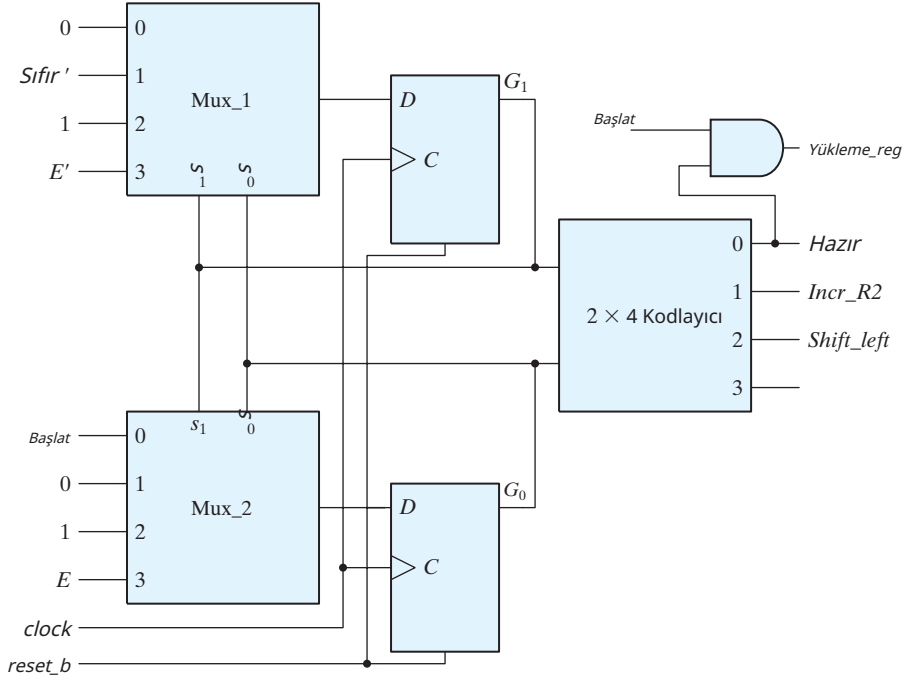
R1 'in içeriği, *Zero* tarafından belirtildiği gibi, *S_1* 'de de incelenecektir. Eğer *R1* boşsa, *Zero* = 1, ve durum *S_idle* 'e geri döner, burada *Ready* belirtilir. Durum *S_1* 'de, *Incr_R2* belirtilir veri yolu biriminin her saat darbesinde *R2* 'yi artırmasını sağlamak için. Eğer *R1* 1'lerden boş değilse, o zaman *Zero* = 0, bu da kayıta bazı 1'lerin saklandığını gösterir. *R1* 'deki sayı kaydırılır ve en soldaki bit *E* 'ye aktarılır. Bu, *E* 'ye bir 1 aktarılana kadar gerekli olduğu kadar yapılır. *E* 'de tespit edilen her 1 için, kayıt *R2* artırılır ve kayıt *R1* tekrar daha fazla 1 için kontrol edilir. Ana döngü, *R1* 'deki tüm 1'ler sayılana kadar tekrarlanır. *S_3* 'ün durum kutusunun kayıt işlemi yoktur, ancak ona bağlı blok *E* için karar kutusunu içerir.

Ayrıca, kaydırma kaydedici *R1* 'e seri girişin 0'a eşit olması gerektiğini unutmayın çünkü dışarıdaki 1'leri *R1* 'e kaydırmak istemiyoruz. Şekil 8.22 (a)'deki kayıt *R1* bir kaydırma kaydedicisidir.

Kaydedici *R2* paralel yükleme ile bir sayıcıdır. Kontrol için çoklayıcı giriş koşulları Tablo 8.9 'dan belirlenmiştir. Giriş koşulları, her olası ikili durum geçişi için ASMD grafiğinden elde edilir. Dört durum atanmıştır

Tablo 8.9
Tasarım Örneği için Çoklayıcı Giriş Koşulları

Mevcut Durum		Sonraki Eyalet		Giriş Koşulları	Çoklayıcı Girişleri	
<i>G₁</i>	<i>G₀</i>	<i>G₁</i>	<i>G₀</i>		MUX1	MUX2
0	0	0	0	<i>Başlat</i>		
0	0	0	1	<i>Başlat</i>	0	<i>Başlat</i>
0	1	0	0	<i>Sıfır</i>		
0	1	1	0	<i>Sıfır</i> '	<i>Sıfır</i>	0
1	0	1	1	<i>None</i>	1	1
1	1	1	0	<i>E</i> '		
1	1	0	1	<i>E</i>	<i>E</i> '	<i>E</i>



ŞEKİL 8.23
Birler sayacı devresi için kontrol uygulaması

ikili değerler 00 ile 11 arasında. Mevcut durum 00'dan geçiş, *Başlat*'a bağlıdır. Mevcut durum 01'den geçiş, *Sıfır*'a bağlıdır ve mevcut durum 11'den geçiş *E*'ye bağlıdır. Mevcut durum 10, koşulsuz olarak bir sonraki durum 11'e geçer. Tablo 1' deki MUX1 ve MUX2 altındaki değerler, sırasıyla G_1 ve G_0 için bir sonraki durumun Boolean giriş koşullarından belirlenir.

Tasarım örneğinin kontrol uygulaması Şekil 8.23'te gösterilmiştir. Bu, ilk seviye yedeki çoklayıcılarla üç seviyeli bir uygulamadır. Çoklayıcılara girişler Tablo 8.9'dan elde edilir. HDL Örneği 8.8'deki Verilog tanımı, kontrolör ve veri yolu için yapısal modelleri örnekler. Kodun listesi, yapılarını uygulayan daha düşük seviyeli modülleri içerir. Veri yolu biriminin kayıtları temizlemek için bir sıfırlama sinyali yoktur, ancak flip-flop, kaydırıcı kayıt ve sayıcı için aktif-düşük sıfırlama modelleri vardır. Bu, Verilog veri türü **supply1**'in bu portları, içindeki mantık değeri 1'e sabitlemek için kullanıldığını göstermektedir.

Datapath_STR. Ayrıca, test tezgahının, modül portlarını iç sinyallere erişim sağlamak için değiştirilmeden hata ayıklama ve doğrulama görevlerini kolaylaştırmak amacıyla denetleyicinin durumuna erişmek için hiyerarşik referanssız erişim kullandığını da unutmayın. Dikkat edilmesi gereken bir diğer ayrıntı, kaydırma kaydına seri girişin 0'a sabitlenmiş olmasıdır. Daha düşük seviyedeki modüller, basitlik için davranışsal olarak tanımlanmıştır.

HDL Örneği 8.8 (Birler Sayacı)

```
modül Count_Ones_STR_STR (count, Ready, data, Start, clock, reset_b);
// Mux – kodlayıcı kontrol mantığı uygulaması
// kontrolör yapısaldır
// veri yolu yapısaldır
```

```
parametre R1_boyutu = 8, R2_boyutu = 4;
çıkış [R2_boyutu -1: 0] sayım;
çıkış Hazır;
girdi [R1_boyutu -1: 0] veri;
girdi Başla, saat, reset_b;
tel Yükle_kayıtlar, Shift_sol, Artır_R2, Sıfır, E;
```

```
Kontrolör_YAP M0 (Hazır, Yükle_kayıtlar, Shift_sol, Artır_R2, Başla, E, Sıfır, saat, sıfırla_b);
Veri_yolu_YAP M1 (sayım, E, Sıfır, veri, Yükle_kayıtlar, Shift_sol, Artır_R2, saat);
```

```
endmodule
```

```
modül Controller_STR (Hazır, Yükle_kayıtlar, Sola kaydır, Artır_R2, Başlat, E, Sıfır, saat,
sıfırla_b);
```

```
çıktı Hazır;
output Load_regs, Shift_left, Incr_R2;
girdi Start;
input E, Zero;
girdi clock, reset_b;
supply0 GND;
supply1 PWR;
parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11; // İkili kod
tel Load_regs, Shift_left, Incr_R2;
tel G0, G0_b, D_in0, D_in1, G1, G1_b;
tel Zero_b = ~Zero;
wire E_b = ~E;
select = {G1, G0};
Decoder_out;
Ready = ~Decoder_out[0];
Incr_R2 = ~Decoder_out[1];
Shift_left = ~Decoder_out[2];
(Load_regs, Ready, Start);
mux_4x1_beh Mux_1 (D_in1, GND, Zero_b, PWR, E_b, select);
mux_4x1_beh Mux_0 (D_in0, Start, GND, PWR, E, select);
D_flip_flop_AR_b M1 (G1, G1_b, D_in1, clock, reset_b);
D_flip_flop_AR_b M0 (G0, G0_b, D_in0, clock, reset_b);
decoder_2x4_df M2 (Decoder_out, G1, G0, GND);
```

```
endmodule
```

```

modül Datapath_STR (sayı, E, Sıfır, veri, Load_regs, Shift_left, Incr_R2, saat);
parametre                                R1_boyut = 8, R2_boyut = 4;
çıkış [R2_boyut -1: 0] sayı;
çıkış                                E, Zero;
giriş [R1_boyut -1: 0] veri;
girdi                                Load_regs, Shift_left, Incr_R2, saat;
wire [R1_boyut -1: 0] R1;
wire Zero;
supply0 Gnd;
supply1 Pwr;
assign Zero = (R1 == 0); // örtük kombinasyon mantığı // örtük kombinasyon mantığı
Shift_Reg M1 (R1, data, Gnd, Shift_left, Load_regs, clock, Pwr); (R1, data, Gnd, Shift_left, Load_regs, clock, Pwr);
Counter M2 (count, Load_regs, Incr_R2, clock, Pwr); (count, Load_regs, Incr_R2, clock, Pwr);
D_flip_flop_AR M3 (E, w1, clock, Pwr); M3 (E, w1, saat, Pwr);
ve (w1, R1[R1_boyut - 1], Shift_left); (w1, R1[R1_boyut - 1], Shift_left);
endmodule

```

```

modül Shift_Reg (R1, veri, SI_0, Shift_left, Load_regs, saat, reset_b);

```

```

    parametre R1_boyut = 8;
çıkış [R1_boyut -1: 0] R1;
girdi [R1_boyut -1: 0] veri;
input SI_0, Shift_left, Load_regs;
girdi clock, reset_b;
reg [R1_boyut -1: 0] R1;
her zaman @ (pozitif kenar saat, negatif kenar reset_b)
    eğer (reset_b == 0) R1 <= 0;
aksi takdirde başla
    eğer (Load_regs) R1 <= veri; aksi takdirde
    eğer (Shift_left) R1 <= {R1[R1_boyut -2: 0], SI_0}; son

```

```

modül sonu

```

```

modül Sayıcı (R2, Load_regs, Incr_R2, saat, reset_b);

```

```

    parametre R2_boyut = 4;
çıkış [R2_boyutu -1: 0] R2;
girdi Load_regs, Incr_R2;
girdi clock, reset_b;
reg [R2_boyutu -1: 0] R2;
her zaman @ (posedge saat, negedge reset_b)
    eğer (reset_b == 0) R2 <= 0;
    aksi takdirde eğer (Load_regs) R2 <= {R2_boyut {1'b1}}; // 1 ile doldur // 1 ile doldur
    aksi takdirde eğer (Incr_R2 == 1) R2 <= R2 + 1;

```

```

endmodule

```

```

modül D_flip_flop_AR (Q, D, CLK, RST_b);

```

```

    çıkış Q;
girdi D, CLK, RST_b;

```

```

reg          Q;
her zaman   @ (posedge CLK, negedge RST_b)
    eğer (RST_b == 0) Q <= 1'b0;
    else Q <= D;
endmodule

modül   D_flip_flop_AR_b (Q, Q_b, D, CLK, RST_b);
    çıkış   Q, Q_b;
girdi      D, CLK, RST_b;
reg       Q;
atama     Q_b = ~Q;
her zaman @ (posedge CLK, negedge RST_b)
    if (RST_b == 0) Q <= 1'b0;
    else Q <= D;
endmodule

// Dört'e bir hat çoklayıcısının davranışsal tanımı
// Verilog 2005 port sözdizimi
module mux_4x1_beh
(çıkış kaydı      m_out,
girdi            in_0, in_1, in_2, in_3,
girdi [1: 0]      seçim
);
her zaman @ (in_0, in_1, in_2, in_3, select) // Verilog 2005 sözdizimi      // Verilog 2005 sözdizimi
    case (seçim)
2'b00:      m_out = in_0;
2'b01:      m_out = in_1;
2'b10:      m_out = in_2;
2'b11:      m_out = in_3;
    endcase
endmodule

// İki-üç hatlı kod çözücünün veri akışı tanımı
// Şekil 4.19'a bakın. Not: Şekil E sembolünü kullanıyor, ancak
// Verilog modeli işlevselliği net bir şekilde belirtmek için enable kullanıyor.
module decoder_2x4_df (D, A, B, enable);
    output      [0: 3]   D;
girdi         A, B;
input         enable;

atama         D[0] = !(A && !B && !enable),
                D[1] = !(A && B && !enable),
                D[2] = !(A && !B && !enable),
                D[3] = !(A && B && !enable);

endmodule

module t_Count_Ones;

```

```

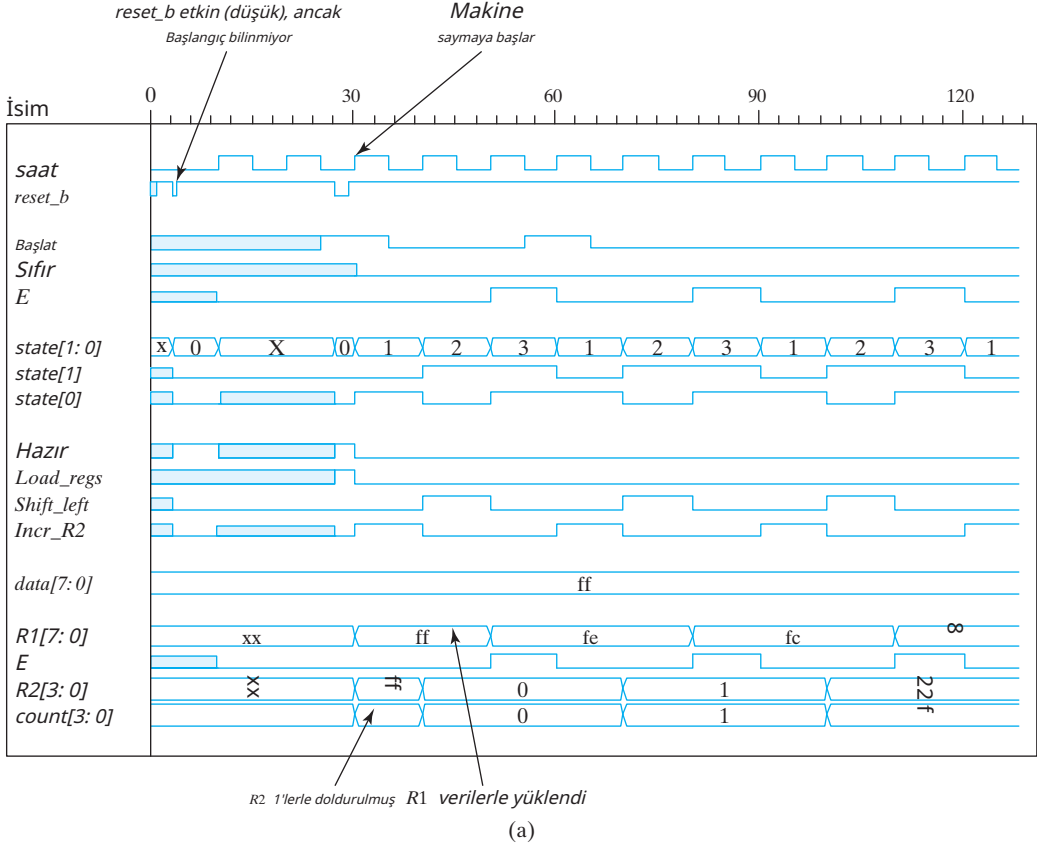
parameter R1_boyutu = 8, R2_boyutu = 4;
wire [R2_boyutu -1: 0] R2;
wire [R2_boyutu -1: 0] sayım;
wire Hazır;
reg [R1_boyut -1: 0] veri;
reg Başla, saat, reset_b;
wire [1: 0] eyalet; // Hata ayıklama için yalnızca kullanın
assign state = {M0.M0.G1, M0.M0.G0};
Count_Ones_STR_STR M0 (count, Ready, data, Start, clock, reset_b);
initial #650 $finish ;
initial begin clock = 0; #5 sonsuz #5 clock = ~clock; end
initial fork
    #1 reset_b = 1;
    #3 reset_b = 0;
    #4 reset_b = 1;
    #27 reset_b = 0;
    #29 reset_b = 1;
    #355 reset_b = 0;
    #365 reset_b = 1;
    #4 data = 8'Hff;
    #145 data = 8'haa;
    # 25 Start = 1;
    # 35 Start = 0;
    #55 Start = 1;
    #65 Start = 0;
    #395 Start = 1;
    #405 Start = 0;
join
modül sonu

```

Birler Sayacını Test Etme

HDL Örneği 8.8'deki test tezgahı, Şekil 8.24'teki simülasyon sonuçlarını üretmek için kullanıldı. Açıklama için notlar eklenmiştir. Şekil 8.24 (a)'da, *reset_b* düşük seviyeye geçiyor $t = 3$ 'te kontrolörü *S_idle*'ye yönlendirmek için, ancak *Start* henüz atanmış bir değere sahip değil. (Varsayılan x'dir.) Sonuç olarak, kontrolör bir sonraki saat döngüsünde bilinmeyen bir duruma giriyor (gölgelendirilmiş dalga formu) ve çıktıları bilinmiyor.⁴ *reset_b* tekrar (düşük) olarak $t = 27$ 'de etkinleştirildiğinde, durum *S_idle*'ye giriyor. Ardından, *Start* = 1 olduğunda, *reset_b* devre dışı bırakıldıktan sonraki ilk saat döngüsünde, (1) kontrolör *S_I*'ye giriyor, (2) *Load_regs R1*'yi *data*'nın değeri olan 8 Hff ile ayarlıyor ve (3) *R2* 1'lerle dolduruluyor. Bir sonraki saat döngüsünde, *R2*

⁴Unutmayın, bu simülasyon Verilog'un dört değerli mantık sistemindedir. Gerçek donanımda, değerler 0 veya 1 olacaktır. Girişler için bilinen bir uygulanan değer olmadan, sonraki durum ve çıktılar belirsiz olacaktır, sıfırlama sinyali uygulandıktan sonra bile.



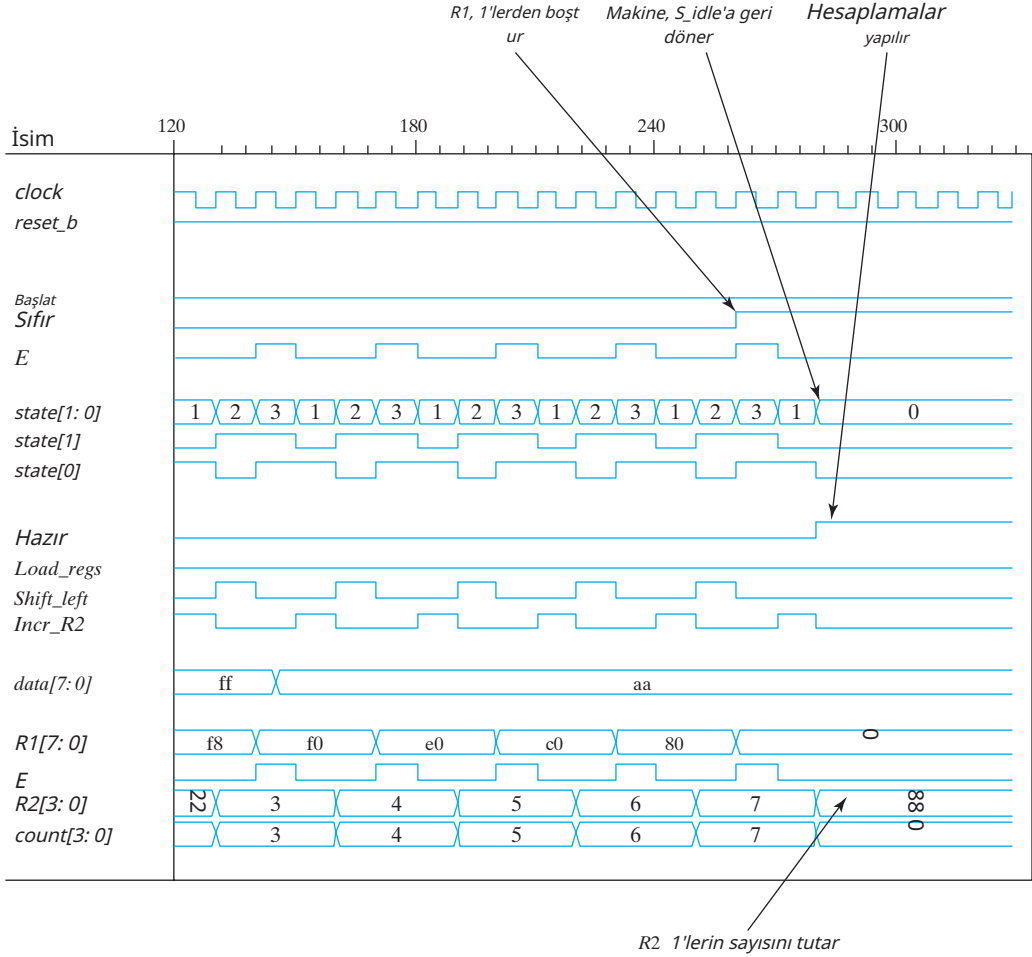
ŞEKİL 8.24

Birler sayacı devresi için simülasyon dalga biçimleri

0'dan başlamaktadır. *Shift_left* kontrolör *S_2* durumundayken etkinleştirilir ve *incr_R2* kontrolör *S_1* durumundayken etkinleştirilir. *incr_R2* etkinleştirildikten sonra *R2* bir sonraki döngüde artırılır. *S_3* durumunda hiçbir çıktı etkinleştirilmez. Sayma dizisi, *Zero* etkinleştirilene kadar Şekil 8.24 (b)'de devam eder ve *E* veri kelimesinin son 1'ini tutar. Bir sonraki saat *count* = 8 üretir ve *state* *S_idle* durumuna döner. (Ek testler bölümün sonunda yer alan problemlerle ele alınmaktadır.)

Bir devre, manuel olarak veya araçlarla sentezlendikten sonra, HDL davranışsal mod elinin ürettiği simülasyon sonuçlarının fiziksel devrenin kapı (standart hücreler) netli sti ile eşleştiğini doğrulamak gereklidir. Herhangi bir sorunu çözmek önemlidir.

Bölüm 8.11 Yarışsız Tasarım (Yazılım Yarış Koşulları) 423



(b)

Şekil 8.24 (Devam Ediyor)

uyumsuzluk, çünkü davranışsal modelin doğru olduğu varsayılmıştır. Bir simülasyonun sonuçları arasında çeşitli uyumsuz kaynaklar vardır, ancak HDL tabanlı tasarım metodolojisinde tipik olarak meydana gelen birini ele alacağız. Üç gerçek, potansiyel probleme katkıda bulunur: (1) bir veri yolu birimi ile bir kontrol birimi arasında, veri yolu biriminden geri beslenen durum sinyallerini içeren bir fiziksel geri besleme yolu vardır; (2) engellenmiş prosedürel atamalar hemen yürütülür ve davranışsal modeller 0 yayılma gecikmeleri ile simüle edilir, bu da girişleri değiştiğinde kombinasyonel mantığın çıkışlarında anlık değişiklikler yaratır (yani, girişlerdeki ve çıkışlardaki değişiklikler simülasyonun aynı zaman adımında planlanır); ve (3) bir simülatörün belirli bir zaman adımında aynı değişkene yapılan birden fazla engellenmiş atamayı yürütme sırası belirsizdir (yani, tahmin edilemez).

Şimdi, tüm atamaların engellenmiş atama operatörü ile yapıldığı bir HDL modelin e sahip ardışık bir makineyi düşünün. Bir saat darbesinde, veri yolundaki kayıt işlemi eri, denetleyicideki durum geçişleri, denetleyicinin bir sonraki durum ve çıkış mantığı nın güncellemeleri ve veri yolundaki durum sinyallerinin güncellemeleri, simülasyon un aynı zaman adımımda gerçekleşecek şekilde planlanmıştır. Hangisi önce gerçekleş ir? Saat darbesi meydana geldiğinde, denetleyicinin durumu kayıt işlemleri gerçekle ştirilmeden önce değişirse varsayalım. Durumdaki değişiklik, kontrol biriminin çıkışla rını değiştirebilir. Çıkışların yeni değerleri, veri yolu nihayetinde o aynı saat darbesin de atamalarını gerçekleştirdiğinde kullanılacaktır. Sonuç, veri yolu atamalarını kontr ol birimi durumunu ve çıkışlarını güncellemeden önce gerçekleştirmiş olsaydı elde e dilecek sonuçla aynı olmayabilir. Tersine, saat darbesi meydana geldiğinde, veri yolu birimi önce işlemlerini gerçekleştirir ve durum sinyallerini güncellerse varsayalım. Güncellenmiş durum sinyalleri, denetleyicinin bir sonraki durumunun değerinde bir d eğişikliğe neden olabilir ve bu da durumu güncellemek için kullanılacaktır. Sonuç, du rumun, veri yolundaki kenar duyarlı işlemler gerçekleştirilmeden önce güncellenmiş olsaydı elde edilecek sonuçtan farklı olabilir. Her iki durumda da, kayıt transfer işlem lerinin ve sistemin farklı temsillerindeki durum geçişlerinin zamanlaması uyuşmayab ilir. *Uyuşmazlığın tespit edilememesi, felaket sonuçlar doğurabilir.*

tasarımın kullanıcısı. Uyuşmazlığın kaynağını bulmak çok zaman alıcı ve maliy etli olabilir. Uyuşmazlığı önlemek için tasarımınızda katı bir disiplin izlemek da ha iyidir. Neyse ki, bu ikilem için bir çözüm var.

Bir tasarımcı, engellenmiş atamalarla kombinasyonel mantığı modelleme kuralını gö zlemleyerek ve durum geçişlerini ve kenar duyarlı kayıt işlemlerini engellenmemiş atam alarla modelleyerek az önce tanımlanan yazılım yarış koşullarını ortadan kaldırabilir. En gellenmemiş operatörler kullanıldığında, bu metindeki tüm örneklerde gösterildiği gibi, bir yazılım yarışı gerçekleşemez çünkü engellenmemiş operatörün örnekleme mekaniz ması, bir durum geçişi veya kenar duyarlı veri yolu işlemi ile veri yolu birimindeki kayıtl a rın bir sonraki durumu veya girişlerini oluşturan kombinasyonel mantık arasındaki geri bildirim yolunu keser. Bu mekanizma, simülatörlerin engellenmemiş atama ifadelerinin sağ tarafındaki ifadeleri herhangi bir engellenmiş atama yapılmadan önce değerlendir mesinden kaynaklanır. Böylece, engellenmemiş atamalar engellenmiş atamaların sonu çlarından etkilenemez. Bu, donanım gerçeğiyle örtüşmektedir. Her zaman kombinasyo nel mantığı modellemek için engelleyici operatörü kullanın ve kenar duyarlı kayıt işlemi eri ve durum geçişlerini modellemek için engellenmemiş operatörü kullanın.

Veri yolunun ve kontrolcünün fiziksel yapısının birlikte fiziksel (yani, donanı m) bir yarış durumu oluşturduğu görünebilir, çünkü durum sinyalleri kontrolc üye geri beslenir ve kontrolcünün çıktıları veri yoluna ileri beslenir. Ancak, zam an analizi, kontrolcünün çıktısındaki bir değişikliğin veri yolu mantığına ve ard ı ndan kontrolcünün giriş mantığına zamanında yayılmayacağını doğrulayabilir , böylece kontrolcünün çıktısı üzerinde bir etkisi olmaz. Durum, bir sonraki saa t darbesine kadar güncellenemez, durum sinyalleri bir sonraki durumun değeri ni güncellese bile. Flip-flop geri besleme yolunu keser.

25saat döngüleri arasında. Pratikte, zaman analizi devrenin belirtilen saat frekansında çalışacağını doğrular veya yayılma gecikmeleri sorunlu olan sinyal yollarını tanımlar. Unutmayın, tasarım doğru mantığı uygulamalı ve saatin belirli ettiği hızda çalışmalıdır.

8.12 LATCH-ÜCRETSİZ TASARIM (NEDEN SİLİKON İSRAFI?)

Sürekli atamalar, kombinasyonel mantığı örtük olarak modellemektedir. Geri besleme içermeyen sürekli bir atama, kombinasyonel mantığa sentezlenecek ve mantığın giriş-çıkış ilişkisi devrenin tüm girişlerine otomatik olarak duyarlı olacaktır. Simülasyon da, simülatör tüm sürekli atamaların sağ taraflarını izler, referans alınan değişkenlerden herhangi birinde bir değişiklik tespit eder ve etkilenen bir atama ifadesinin sol taraflarını günceller. Sürekli bir atamadan farklı olarak, döngüsel bir davranış, atama ifadeleri tarafından referans alınan tüm değişkenlere tamamen duyarlı olmayabilir. Eğer seviye duyarlı döngüsel bir davranış kombinasyonel mantığı tanımlamak için kullanılıyorsa, duyarlılık listesinin, davranıştaki bir atama ifadesinin sağ tarafında referans alınan her değişkeni içermesi esastır. Liste eksikse, davranış tarafından tanımlanan mantık, mantığın çıkışlarında latch'lerle sentezlenecektir. Bu uygulama silikon alanını israf eder ve davranışsal modelin simülasyonu ile sentezlenen devre arasında bir uyumsuzluk olabilir. Bu zorluklar, duyarlılık listesinin tam olmasını sağlayarak önlenir, ancak büyük devrelerde, seviye duyarlı döngüsel bir davranışın duyarlılık listesi ne her referans alınan değişkeni dahil etmemek kolaydır.

Sonuç olarak, Verilog 2001, yanlışlıkla latch'lerin sentezlenme riskini azaltmak için yeni bir operatör ekledi.

Verilog 2001'de, @ ve * token'ları @* veya @(*) olarak birleştirilebilir ve bir duyarlılık listesi olmadan, ilgili ifadenin yürütülmesinin, mantıktaki bir atama ifadesinin sağ tarafında referans verilen her değişkene duyarlı olduğunu belirtmek için kullanılır. Aslında, @* operatörü, mantığın yorumlanması gerektiğini belirtir.

HDL Örneği 8.9

Aşağıdaki seviye duyarlı döngüsel davranış, iki kanallı bir çoklayıcıyı sentezleyecektir:

```
module mux_2_V2001 ( output reg [31: 0] y, input [31: 0] a, b, input sel);
    always @*
        y = sel ? a: b;
modül sonu
```

Döngüsel davranış, *a*, *b* ve *sel*'den oluşan örtük bir duyarlılık listesine sahiptir.

ve seviye duyarlı kombinasyonel mantık olarak sentezlenmiştir; mantık, prosedürel atamalar t arafından referans verilen tüm değişkenlerden oluşan örtük bir duyarlılık listesine sahiptir. @* operatörünün kullanılması, latch'lerin yanlışlıkla sentezlenmesini önleyecektir.

8.13 DİĞER DİL ÖZELLİKLERİ

Bu metindeki örnekler, mantık tasarımı için giriř niteliğindeki bir ders için uygun olan V erilog HDL özelliklerini kullanmıştır. Verilog 2001, tasarımcılar için çok yararlı olan anca k burada ele alınmayan özellikler içermektedir. Bunlar arasında çok boyutlu diziler, değ i şken parça seçimleri, dizi bit ve parça seçimleri, işaretli reg, ağ ve port bildirimleri ile ye rel parametreler bulunmaktadır. Bu geliřtirmeler, Verilog 2001 ve Verilog 2005 kullana n daha ileri düzey metinlerde ele alınmaktadır.

PROBLEMLER

* ile işaretlenmiş problemlerin cevapları kitabın sonunda yer almaktadır.

8.1 * Aşağıdaki kayıt transfer notasyonu ile belirtilen işlemler için kelimel

erle açıklayın ve HDL ifadelerini yazın:

$R2 \leftarrow R2 + 1, R1 \leftarrow R$

$R3 \leftarrow R3 - 1$

(c) Eğer $(S_1 = 1)$ o zaman $(R0 \leftarrow R1)$ aksi takdirde eğer $(S_2 = 1)$ o zaman $(R0 \leftarrow R2)$

Aktif-düşük senkron sıfırlama ile bir mantık devresi iki kontrol giriři x ve y içerir. Eğer $x = 1$ is e ve $y = 0$ ise, kayıt R 1 artırılır ve kontrol ikinci duruma geçer. Eğer $x = 0$ ise ve $y = 1$ ise, kayıt R sıfıra temizlenir ve kontrol başlangıç durumundan üçüncü duruma geçer.

Aksi takdirde, kontrol başlangıç durumunda kalır. (1) kontrolör, veri yolu birimi (iç kayıtlarla birlikte) ve sinyalleri gösteren bir blok diyagramı çizin ve (2) başlangıç durumundan itibaren bir ASMD şemasının kısmını çizin.

ASMD grafiklerini aşağıdaki durum geçişleri için çizin:

(a) Eğer $x = 1$ ise, kontrol durumdan S_1 duruma S_2 'ye geçer; eğer $x = 0$ ise, bir koşullu işlem üretin:

$R \leftarrow R + 2$ ve S_1 'den S_2 'ye geçin.

(b) Eğer $x = 1$ ise, kontrol S_1 'den S_2 'ye ve ardından S_3 'e geçer; eğer $x = 0$ ise, kontrol S_1 'den S_3 'e geçer.

(c) Durumdan başlayın S_1 ; sonra eğer $xy = 11$ ise, S_2 'ye gidin; eğer $xy = 01$ ise S_3 'ye gidin; ve eğer $xy = 10$ ise, S_1 'ye gidin; aksi takdirde, S_3 'ye gidin.

Bir ASM bloğunda, üç kontrol değişkeninin x , y , ve z sekiz olası ikili değerini kontrol eden karar kutularından çıkan sekiz çıkış yolunu gösterin.

ASM ve ASMD grafiklerinin geleneksel akış şemasından nasıl farklı olduğunu açıklayın. Fig. 8.5 'i bir örnek olarak kullanarak, yorumlamadaki farkı gösterin. Bir ASM grafiğı ile bir AS MD grafiğı arasındaki farkı açıklayın. Kendi kelimelerinizle, bir ASMD grafiğinin kullanımını v e avantajlarını tartışın.

Bir odadaki insan sayısını sayan dijital bir sistem için bir blok diyagramı ve bir AS MD grafiğı oluşturun. İnsanların odaya girdiğı tek kapıda, ışık kesildiğinde x siny alini 1'den 0'a değiřtiren bir fotocell bulunmaktadır. Odayı terk ederler

Benzer bir fotocell ile ikinci bir kapı, ışık kesildiğinde sinyali y 1'den 0'a değiştirir. Veri yolu devresi, odada kaç kişinin olduğunu gösteren bir ekrana sahip yukarı-aşağı sayıcıdan oluşur.

- 8.7** * İki sekiz bitlik kayıt RA ve RB için bir blok diyagramı ve ASMD grafiği çizin. Devre, çıkarma işlemini gerçekleştirir.

$$RA \leftarrow RA - RB$$

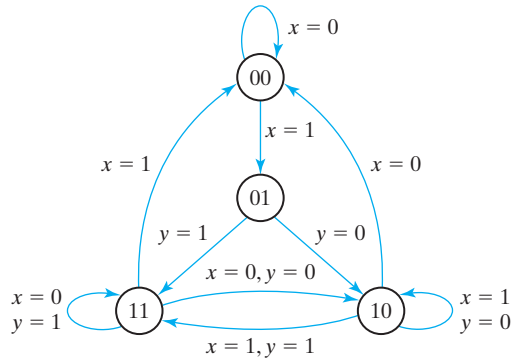
Çıkarma için 1.5. Bölümde açıklanan yöntemi kullanın ve cevap negatifse bir borç flip-flop'unu 1 olarak ayarlayın. Devrenin bir HDL modelini yazın ve doğrulayın.

- 8.8** * Üç 16 bitlik kayıt AR , BR , ve CR ile bir dijital devre tasarlayın ve aşağıdaki işlemleri gerçekleştirin:

- İki 16 bitlik işaretli sayıyı (2'nin tamamlayıcı temsiliinde) AR ve BR 'ye aktarın.
- AR içindeki sayı negatifse, AR içindeki sayıyı 2'ye bölün ve sonucu CR kaydına aktarın.
- AR içindeki sayı pozitif ama sıfır değilse, BR içindeki sayıyı 2 ile çarpın ve sonucu CR kaydına aktarın.
- AR içindeki sayı sıfırsa, CR kaydını 0'a temizleyin.
- Devre için bir davranışsal model yazın ve doğrulayın.

- 8.9** * Şekil 8.11 (a) tarafından verilen durum diyagramına sahip bir kontrolör tasarlayın. Her durum için bir flip-flop kullanın (tek sıcak atama). Kontrolörün RTL ve yapısal modellerini yazın, simüle edin, doğrulayın ve karşılaştırın.

- 8.10** Bir kontrol biriminin durum diyagramı Şekil P8.10'da gösterilmiştir. Dört durumu ve iki girişi x ve y vardır. Eşdeğer ASM şemasını çizin. Kontrolörün bir Verilog modelini yazın ve doğrulayın.



ŞEKİL P8.10

Problemler 8.10 ve 8.11 için kontrol durum diyagramı

- 8.11** * Kontrolörün durum diyagramının gösterildiği şekilde tasarlayın. D flip-flop'ları kullanın.

- 8.12** Dört bitlik senkron temizleme ile belirtilen sayacı tasarlayın. 8.10 . Asenkron temizleme için tekrarlayın.

8.13

Simüle et Tasarım_Örneği_STR (HDL Örneği 8.4'e bakın) ve davranışının RTL tanımıyla eşleştiğini doğrulayın. Durum bilgilerini elde etmek için $G0$ ve $G1$ durum için birleştirilmiş bir vektör olarak görüntüleyin.

8.14

Makine, kullanılmayan bir duruma girdiğinde, *Design_Example_RTL* (bkz. HDL Örnek 8.2) herhangi bir sonuç doğurur mu?

8.15

HDL Örnek 8.2'de Design_Example_RTL'yi simüle edin ve

çalışması sırasında beklenmeyen bir sıfırlama durumundan, yani "çalışan sıfırlama" veya "anlık sıfırlama" durumundan kurtulduğunu doğrulayın.

8.16

* İki ikili sayıyı tekrar ekleme yöntemiyle çarpan bir dijital devre için bir blok diyagramı ve bir ASMD grafiği geliştirin. Örneğin, 5×4 'ü çarpmak için, dijital sistem çarpanı dört kez ekleyerek çarpımı değerlendirir: $5 + 5 + 5 + 5 = 20$. Devreyi tasarlayın. Çarpan, BR kaydında, çarpan AR kaydında ve çarpım PR kaydında olsun. Bir toplayıcı devresi, BR içeriğini PR ile toplar. Bir sıfır tespit sinyali, AR değerinin 0 olup olmadığını gösterir. Devrenin bir Verilog davranışsal modelini yazın ve doğrulayın.

8.17

* İki n bit sayının çarpımının, uzunluğu $2n$ bitten az veya ona eşit

bir sonuç verdiğini kanıtlayın.

8.18

* Şekil 8.14'te, Q kaydı çarpanı tutar ve B kaydı çarpanı tutar.

Her sayının 16 bitten oluştuğunu varsayalım.

(a) Ürün için kaç bit beklenebilir ve nerede mevcuttur?

(b) P sayacında kaç bit vardır ve başlangıçta içine yüklenen ikili sayı nedir?

(c) P sayacındaki sıfırı kontrol eden devreyi tasarlayın.

8.19

11011 (çarpan) ve 10111 (çarpan) iki sayının çarpma işlemi sırasında C , A , Q ve P kayıtlarının içeriğini, Tablo 8.5'e benzer bir şekilde listeleyin.

8.20

* Bölüm 8.8'de tanımlanan ikili çarpan işlemi gerçekleştirme için geçen süreyi belirleyin. Q kaydının n bit olduğunu ve saat döngüsünün

8.21

Şekil 8.16'daki durum diyagramı tarafından belirtilen ikili çarpanın kontrol devresini, çoklayıcılar, bir kod çözücü ve bir kayıt kullanarak tasarlayın.

8.22

Şekil P8.22, ardışık ikili çarpan için alternatif bir ASMD grafiğini göstermektedir. Sistemin bir RTL modelini yazın ve doğrulayın. Bu tasarımı, Şekil 8.15 (b)'de tanımlanan ASMD grafiği ile karşılaştırın.

8.23

Şekil P8.23, ardışık ikili çarpan için alternatif bir ASMD grafiğini göstermektedir. Sistemin bir RTL modelini yazın ve doğrulayın. Bu tasarımı, Şekil 8.15 (b)'de tanımlanan ASMD grafiği ile karşılaştırın.

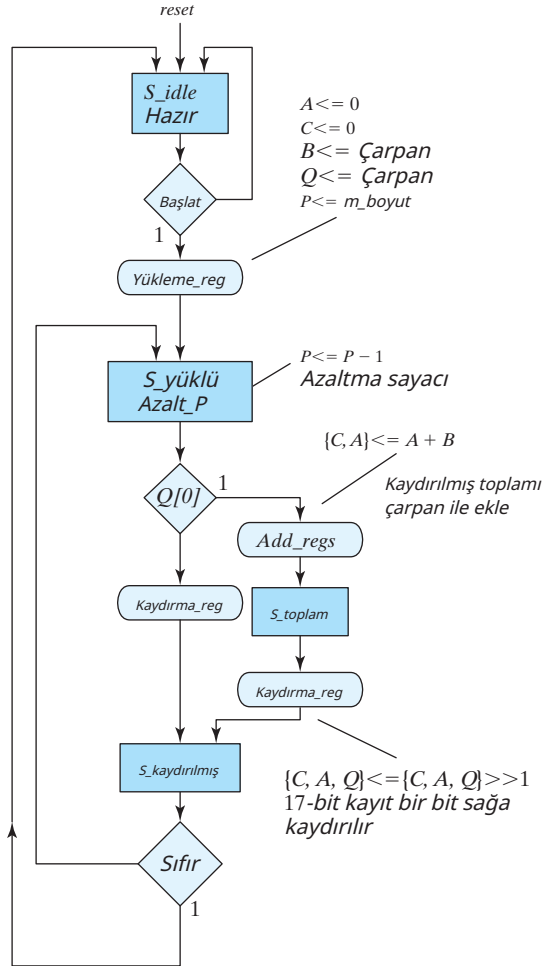
8.24

HDL Örneği 8.5'te verilen ardışık ikili çarpanın HDL tanımı, kontrolör ve veri yolunun tanımlarını tek bir Verilog modülünde kapsar. Kontrolör ve veri yolunu ayrı modüllerde kapsayan bir model yazın ve doğrulayın.

8.25

Şekil 8.15'teki ASMD grafiği ile tanımlanan ardışık ikili çarpan, çarpan veya kaydırılan çarpanın 0 olup olmadığını dikkate almaz. Bu nedenle, veriden bağımsız olarak sabit bir saat döngüsü sayısı için çalışır.

(a) Herhangi bir kelimenin sıfır olduğu bulunmaz çalışmayı sonlandıracak daha verimli bir çarpan için bir ASMD grafiği geliştirin.



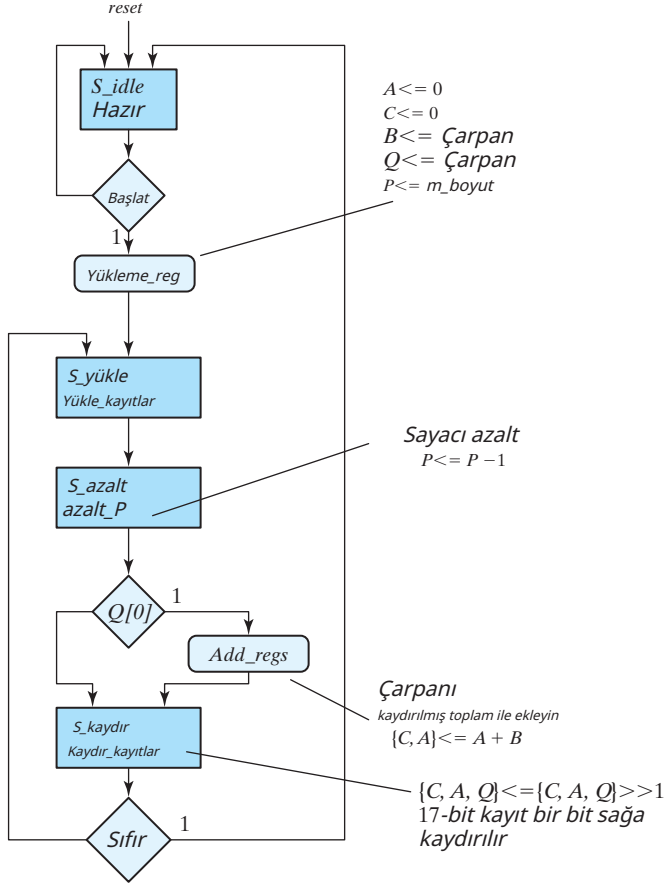
ŞEKİL P8.22

Problem 8.22 için ASMD grafiği

- (b) Devrenin HDL tanımını yazın. Kontrolör ve veri yolu ayrı Verilog modüllerinde kapsüllenmelidir.
- (c) Bir test planı ve test tezgahı yazın ve devreyi doğrulayın.

ASMD grafiğini, Şekil 8.15'te gösterilen ardışık ikili çarpan için, aynı saat döngüsünde ekleme ve kaydırma işlemleri yapacak şekilde değiştirin. Sistemin RTL tanımını yazın ve doğrulayın.

HDL Örneği 8.6'da verilen ikinci test tezgahı, çarpan ve çarpan için tüm olası değerler için bir ürün üretir. Her sonucun doğru olduğunu doğrulamak pratik olmayacağından, beklenen ürünü oluşturan bir ifade içerecek şekilde test tezgahını değiştirin.

**ŞEKİL P8.23**

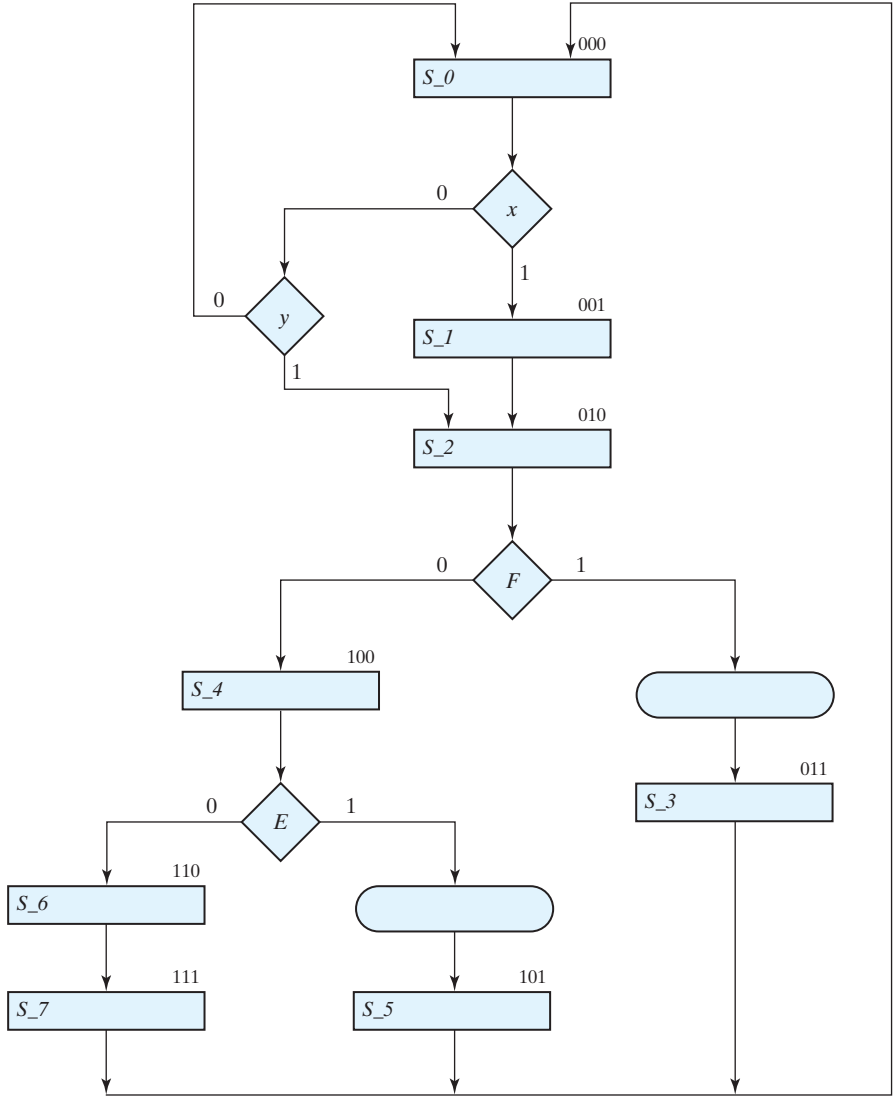
Problem 8.23 için ASMD şeması

RTL tanımı tarafından üretilen sonucu beklenen sonuçla karşılaştırmak için ek ifadeler yazın. Simülasyonunuz, karşılaştırmının sonucunu gösteren bir hata sinyali üretmelidir. Çarpanın yapısal modeli için de bunu tekrarlayın.

Bölüm 8.8'de tasarlanan çarpanın HDL yapısal tanımını yazın. Şekil 8.14 (a) blok diyagramını ve Şekil 8.18 kontrol devresini kullanın. Tasarımı simüle edin ve HDL Örneği 8.6'nın test tezgahını kullanarak işlevselliğini doğrulayın.

Bir sonlu durum makinesi için eksik bir ASMD grafiği, Şekil P8.29'da gösterilmiştir. Kayıt işlemleri belirtilmemiştir, çünkü yalnızca kontrol mantığını tasarlamakla ilgileniyoruz.

- Eşdeğer durum diyagramını çizin.
- Her durum için bir flip-flop ile kontrol birimini tasarlayın.
- Kontrol birimi için durum tablosunu listeleyin.



ŞEKİL P8.29
Problem 8.29 için ASMD grafiği

- (d) Üç D flip-flop, bir dekoder ve kapılarla kontrol birimini tasarlayın.
 - (e) Kontrol birimi için çoklayıcı giriş koşullarını gösteren bir tablo çıkarın.
 - (f) Üç çoklayıcı, üç flip-floplu bir kayıt ve bir
- 3 × 8 dekoder ile kontrol birimini tasarlayın.
- (g) (f) sonuçlarını kullanarak, kontrolörün yapısal modelini yazın ve doğrulayın.
 - (h) Kontrolörün RTL tanımını yazın ve doğrulayın.

8.30 * E değerinin her HDL bloğunda ne olduğunu varsayarak, $RA = 1$?

(a) $RA \leftarrow RA - 1$;

Eğer $(RA == 0) E = 1$;

aksi takdirde $E = 0$;

(b) $RA \leftarrow RA - 1$;

Eğer $(RA == 0) E \leftarrow 1$;

aksi takdirde $E \leftarrow 0$;

8.31 * Tablo 8.2 'de listelenen Verilog HDL operatörlerini kullanarak, $A = 4$ b0110

$B = 4$ b0010, ve $C = 4$ b0000 varsayarak aşağıdaki işlemlerin sonucunu değerlendirin:

$A * B$; $A + B$; $A - B$; $\sim C$; $A \& B$; $A | B$; $A \wedge B$; $\& A$; $\sim | C$; $A || B$; $A \&\& C$; $|A$; $A < B$; $A > B$;
 $A != B$;

8.32 Aşağıdaki always bloğunu dikkate alın:

```
always @ ( posedge CLK)
  if (S1) R1 <= R1 + R2;
  else if (S2) R1 <= R1 + 1;
  else R1 <= R1;
```

R1 için paralel yüklemeli dört bitlik bir sayaç kullanarak (Şekil 6.15'te olduğu gibi) ve dört bitlik bir toplayıcı, blok için bileşenlerin ve kontrol sinyallerinin bağlantılarını gösteren bir blok diyagramı çizin.

8.33 Çok seviyeli durum ifadesi genellikle bir mantık sentezleyici tarafından donanıma multiplexer'lara dönüştürülür. Aşağıdaki case bloğunu donanıma nasıl çevireceksiniz (her biri sekiz bitlik kayıtlar varsayalım)?

```
case (state)
  S0: R4 = R0;
  S1: R4 = R1;
  S2: R4 = R2;
  S3: R4 = R3;
```

endcase

8.34

Bir kayıttaki birlerin sayısını sayan bir devre tasarımı Bölüm 8.10'da gerçekleştirilmiştir. Devre için blok diyagramı Şekil 8.22 (a)'de gösterilmiştir, devre için tam bir ASMD grafiği Şekil 8.22 (c)'de görünmektedir ve veri yolu ile kontrolörün yapısal HDL modelleri HDL Örneği 8.8'de verilmiştir. ASMD grafiğinde belirtilen işlemler ve sinyal adlarını kullanarak,

(a) Datapath_BEH yazın, birlerin sayıcı veri yolu biriminin RTL tanımı.

Bir test planı yazın, test edilecek işlevselliği belirleyin ve planı uygulamak için bir test tezgahı yazın. Test planını yürütün ve veri yolu biriminin işlevselliğini doğrulayın, ayrıca test planını simülasyonda üretilen dalga formlarıyla ilişkilendiren notlu simülasyon sonuçları üretin.

(b) Kontrol biriminin RTL tanımını yazın *Controller_BEH*, bir tane sayıcı için.

Bir test planı yazın, test edilecek işlevselliği belirleyin ve planı uygulamak için bir test tezgahı yazın. Test planını yürütün ve kontrol biriminin işlevselliğini doğrulayın, ayrıca test planını simülasyonda üretilen dalga formlarıyla ilişkilendiren notlu simülasyon sonuçları üretin.

(c) Count_Ones_BEH_BEH yazın, Controller_BEH ve Datapath_BEH kapsayan ve entegre eden bir üst düzey modül. Bir test planı ve bir test tezgahı yazın ve tanımı doğrulayın. Test planını simülasyonda üretilen dalga formlarıyla ilişkilendiren notlu simülasyon sonuçları üretin.

- (d) Bir tane sıcak kontrolörün RTL tanımını yazın *Controller_BEH_1Hot*, Şekil 8.22 (c) A SMD grafiğini uygulayan. Test edilecek işlevselliği belirleyen bir test planı yazın ve planı uygulamak için bir test tezgahı yazın. Test planını yürütün ve test planını simülasyonda üretilen dalga formlarıyla ilişkilendiren notlu simülasyon sonuçları üretin.
- (e) Yaz *Count_Ones_BEH_1_Hot*, *Controller_BEH_1_Hot* ve *Datapath_BEH* modülünü kapsayan üst düzey bir modül. Bir test planı ve bir test tezgahı yazın ve deskripsiyonu doğrulayın. Test planını simülasyonda üretilen dalga formlarıyla ilişkilendiren notlu simülasyon sonuçları üretin.

Bir kayıttaki birlerin sayısını sayan bir devre için HDL tanımı ve test tezgahı HDL Örneği 8.8'de verilmiştir. Test tezgahını değiştirin ve devreyi simüle ederek sistemin aşağıdaki veri desenleri için doğru çalıştığını doğrulayın: 8 hff, 8 h0f, 8'hf0, 8 h00, 8 haa, 8 h0a, 8 ha0, 8 h55, 8 h05, 8 h50, 8 ha5, ve 8 h5a.

Bir kayıttaki birlerin sayısını sayan bir devrenin tasarımı Bölüm 8.10'da gerçekleştirilmiştir. Devre için blok diyagramı Şekil 8.22 (a)'da gösterilmiştir, bu devre için tam bir ASMD grafiği Şekil 8.22 (c)'de görünmektedir ve veri yolu ile kontrolörün yapısal HDL modelleri HDL Örneği 8.8'de verilmiştir. ASMD grafiğinde belirtilen işlemler ve sinyal adlarını kullanarak,

- (a) Her durum için bir flip-flop kullanarak kontrol mantığını tasarlayın (bir sıcak atama). Dört flip-flop için giriş denklemlerini listeleyin.
- (b) Yazın *Controller_Gates_1_Hot*, devrenin kapı seviyesinde HDL yapısal tanımı, (a) kısmında tasarlanan kontrolü ve

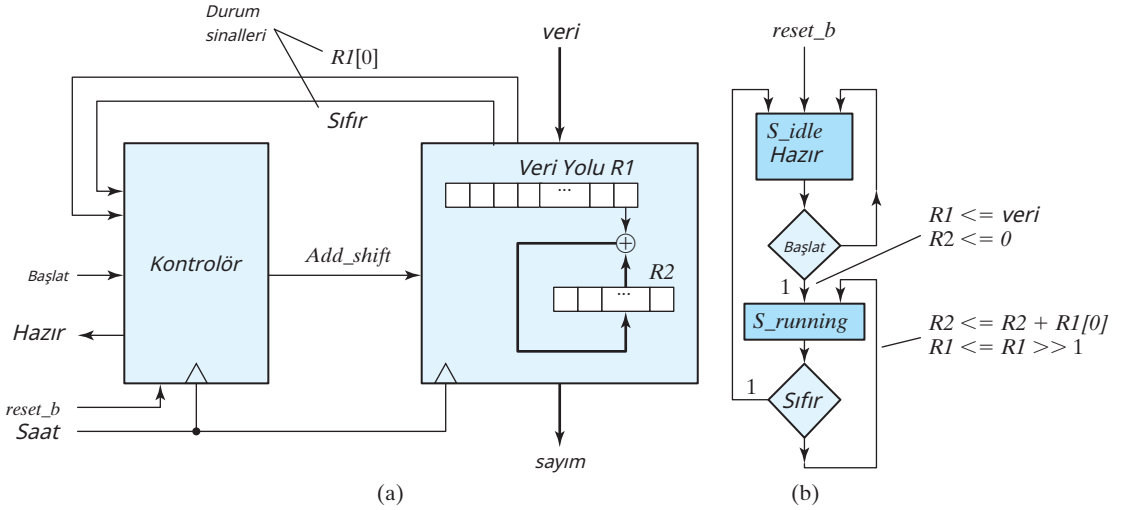
Şekil 8.22 (a)'da gösterilen sinyalleri kullanarak.

- (c) Bir test planı ve bir test tezgahı yazın, ardından kontrolörü doğrulayın.
- (d) Yazın *Count_Ones_Gates_1_Hot_STR*, *Controller_Gates_1_Hot* ve *Datapath_STR* örneklerini kapsayan ve entegre eden bir üst düzey modül. Tanımı doğrulamak için bir test planı ve bir test tezgahı yazın. Test planını simülasyonda üretilen dalga formlarıyla ilişkilendiren notlu simülasyon sonuçları üretin.

8.37 HDL Örneği 8.8'de sunulan devre ile karşılaştırıldığında, bir veri kelimesindeki birlerin sayısını sayan daha verimli bir devre, blok diyagramı ve Şekil P8.37'de kısmen tamamlanmış ASMD grafiği ile tanımlanmaktadır. Bu devre, aynı saat döngüsünde toplama ve kaydırma işlemlerini gerçekleştirir ve her saat döngüsünde veri kaydının LSB'sini sayacı kaydına ekler.

- (a) ASMD grafiğini tamamlayın.
- (b) ASMD grafiğini kullanarak devrenin RTL tanımını yazın. Üst düzey bir Verilog modülü, *Count_of_ones_2_Beh* veri yolu ve kontrol birimleri için ayrı modüller örnekleyecektir.
- (c) Kontrol mantığını tasarlayın, her durum için bir flip-flop kullanarak (bir hot assignment). Flip-floplar için giriş denklemlerini listeleyin.
- (d) Devrenin HDL yapısal tanımını yazın, (c) bölümünde tasarlanan kontrolörü ve Şekil P8.37 (a) blok diyagramını kullanarak.
- (e) Devreyi test etmek için bir test tezgahı yazın. RTL ve yapısal programlarda açıklanan işlemi doğrulamak için devreyi simüle edin.

8.38 İmzalı büyüklük temsiliinde iki imzalı ikili sayının toplanması, sıradan aritmetik kurallarını takip eder: Eğer iki sayı aynı işarete sahipse (her ikisi de pozitif veya her ikisi de negatif), iki büyüklük toplanır ve toplam ortak işarete sahiptir; eğer iki sayı zıt işaretlere sahipse, daha küçük büyüklük daha büyük olandan çıkarılır ve

**ŞEKİL P8.37**

(a) Birler sayacı için alternatif devre

(b) Problem 8.37 için ASMD Şeması

sonuç daha büyük büyüklüğün işaretine sahiptir. İmzalı büyüklük temsilinde iki 8-bit imzalı sayıyı toplamak için bir HDL davranışsal tanım yazın ve doğrulayın. Sayının en soldaki biti işareti tutar ve diğer yedi bit büyüklüğü tutar.

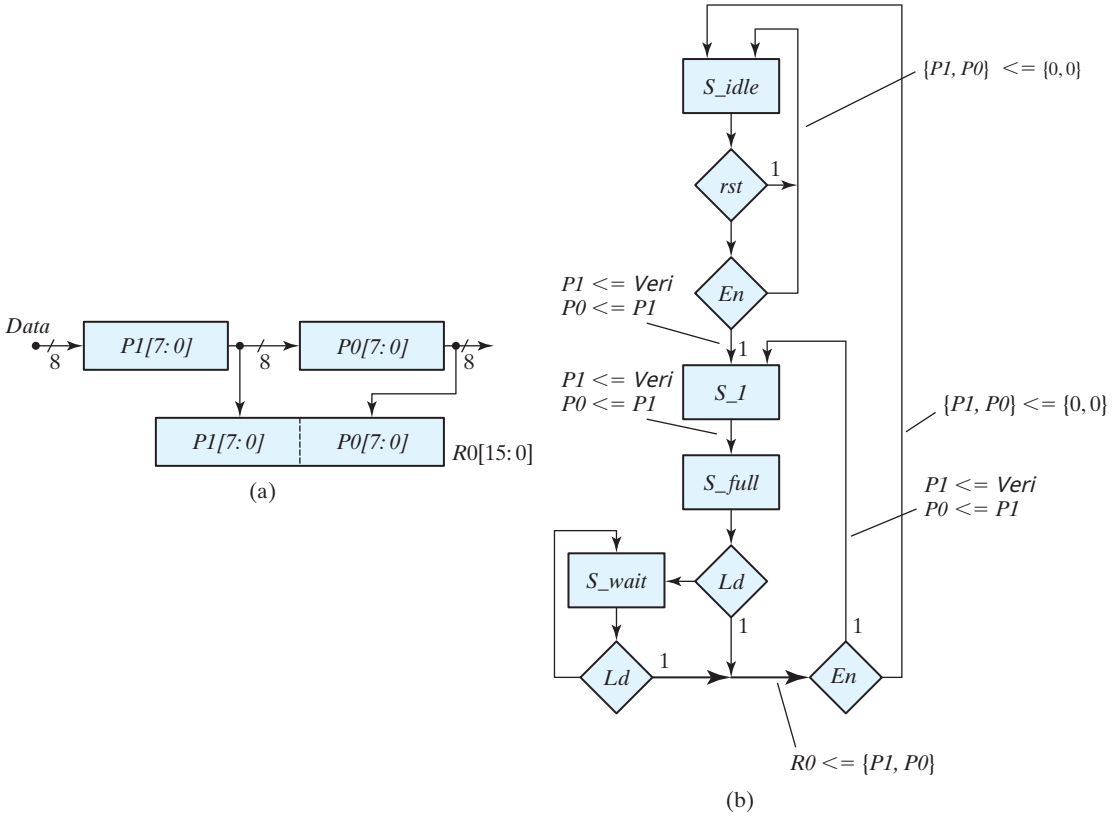
8.39

* Problem 8.16'da tasarlanan devre için,

- Devrenin yapısal HDL tanımını yazın ve doğrulayın. Veri yolu ve kontrolör ayrı birimlerde tanımlanmalıdır.
- Devreye ait bir RTL tanımını yazın ve doğrulayın. Veri yolu ve kontrolör ayrı birimlerde tanımlanacaktır.

Şekil 8.14 (a)'da verilen ardışık çarpan blok diyagramını ve Şekil 8.15 (b)'deki ASMD şemasını, 32 bit kelimeleri çarpan, ancak 8 bit (bayt genişliğinde) harici veri yolları ile tanımlayacak şekilde değiştirin. Makine, (ilk) sıfırlama durumunda *Ready* sinyalini verecektir.

Start sinyali verildiğinde, makine, ardışık saat döngülerinde tek bir 8 bit veri yolundan veri baytlarını alacak (önce çarpan baytları, ardından çarpan baytları, en az anlamlı bayt önce) ve verileri veri yolu kayıtlarına depolayacaktır. *Got_Data* transfer tamamlandı ığında bir saat döngüsü boyunca verilecektir. *Run* sinyali verildiğinde, ürün ardışık olarak oluşturulacaktır. *Done_Product* çarpma tamamlandığında bir saat döngüsü boyunca verilecektir. *Send_Data* sinyali verildiğinde, ürünün her baytı, en az anlamlı bayttan başlayarak, bir saat döngüsü boyunca 8 bit çıkış yoluna yerleştirilecektir. Ürün iletildikten sonra makine başlangıç durumuna dönecektir. Ürün oluşturulurken veri göndermeye veya almaya çalışmamak gibi güvenlik önlemlerini göz önünde bulundurun. Ayrıca, gereksiz çarpmanın önüne geçebilecek diğer özellikleri de düşünün. Örneğin, kaydırılan çarpanın 1'lerden boşsa çarpmaya devam etmeyin.



ŞEKİL P8.41

İki aşamalı boru kaydı: Veri yolu birimi ve ASMD şeması

Blok diyagramı ve kısmen tamamlanmış ASMD grafiği, Şekil P8.41'de, paralel giriş ve çıkışa sahip 2:1 azaltıcı olarak işlev gören iki aşamalı bir boru hattının davranışını tanımlamaktadır. Azaltıcılar, dijital sinyal işleyicilerinde, verileri yüksek saat hızına sahip bir veri yolundan daha düşük saat hızına sahip bir veri yoluna taşımak için kullanılır; bu süreçte verileri paralel formattan seri formata dönüştürür. Gösterilen veri yolunda, veri borusuna, borunun içeriğinin bir tutma kaydına boşaltılması veya bir işlemci tarafından tüketilmesi gereken hızın iki katı hızda tam kelimeler aktarılabilir. Tutma kaydının içeriği $R0$ serisi olarak dışarı kaydırılabilir, böylece veri akışının genel paralel-seri dönüşümü gerçekleştirilir. ASMD grafiği, makinenin S_idle durumuna senkronize sıfırlama yaptığını gösterir; burada rst iptal edilene ve En etkinleştirilene kadar bekler. rst eylemi altında S_idle durumuna diğer durumlardan geçecek senkron geçişlerin gösterilmediğini unutmayın. En etkinleştirildiğinde, makine S_idle durumundan S_I durumuna geçer; bu, borunun MSByte'ını yükleyen eşzamanlı kayıt işlemleri ile birlikte gerçekleşir.

$Veri$ ve $P1$ içeriğini LSByte'a ($P0$) taşıyın. Bir sonraki saat döngüsünde, durum S_full , ve şimdi boru dolu. Eğer Ld bir sonraki saat döngüsünde etkinleştirilirse, makine S_I durumuna geçerken boruyu bir tutma kaydına $R0$ döker. Eğer Ld etkinleştirilmezse, makine

girer S_{wait} ve Ld doğrulandığı zamana kadar orada kalır, bu noktada boruyu boşaltır ve S_I veya S_{idle} durumuna geri döner, bu da En doğrulandığına bağlıdır. R_0 'daki veri hızı, bir dış veri yolundan birime sağlanan veri hızının yarısıdır.

(a) Tam ASMD grafiğini geliştirin.

(a) bölümünde geliştirilen ASMD grafiğini kullanarak, veri yolunun bir HDL modelini yazın ve doğrulayın.

(c) Kontrol biriminin bir Verilog davranışsal modelini yazın ve doğrulayın.

(d) Veri yolunu ve kontrolörü üst düzey bir modüle kapsülleyin ve entegre

sistemi doğrulayın.

Şekil 8.22'de tanımlanan birler sayma devresinin ortadan kaldırılması gereken bir gecikmesi vardır. Bu, E durum sinyalinin, R_1 'in MSB'sinin kaydırıldığı bir flip-flopun çıktısı olarak oluşmasından kaynaklanır. Gecikmeyi ortadan kaldıran bir tasarım geliştirin.

KAYNAKLAR

ARNOLD, M. G. 1999. *Verilog Dijital Bilgisayar Tasarımı*. Upper Saddle River, NJ: Prentice Hall.

B HASKER, J. 1997. *Verilog HDL Primer*. Allentown, PA: Star Galaxy Press.

B HASKER, J. 1998. *Verilog HDL Synthesis*. Allentown, PA: Star Galaxy Press.

CILETTI, M. D. 2003. *Modeling, Synthesis, and Rapid Prototyping with Verilog HDL*. Upper Saddle River, NJ: Prentice Hall.

CILETTI, M. D. 2010. *Advanced Digital Design with the Verilog HDL*. Upper Saddle River, NJ: Prentice Hall.

CLARE, C. R. 1971. *Designing Logic Systems Using State Machines*. New York: McGraw-Hill.

HAYES, J. P. 1993. *Introduction to Digital Logic Design*. Reading, MA: Addison-Wesley.

IEEE Standard Hardware Description Language Based on the Verilog Hardware Description Language (IEEE Std 1364-2005). 2005. New York: Institute of Electrical and Electronics Engineers.

MANO, M. M. 1993. *Bilgisayar Sistemi Mimarisi*, 3. baskı. Upper Saddle River, NJ: Prentice Hall.

Mano, M. M., ve C. R. Kime. 2005. *Mantık ve Bilgisayar Tasarım Temelleri*, 3. baskı. Upper Saddle River, NJ: Prentice Hall.

PALNITKAR, S. 2007. *Verilog HDL: Dijital Tasarım ve Sentez için Bir Rehber*. Mountain View, CA: SunSoft Press (bir Prentice Hall Yayını).

SMITH, D. J. 1996. *HDL Çip Tasarımı*. Madison, AL: Doone Yayınları.

Thomas, D. E., ve P. R. Moorby. 2002. *Verilog Donanım Tanım Dili*, 5. baskı. Boston: Kluwer Akademik Yayıncılar.

Winkler, D., ve F. Prosser. 1987. *Dijital Tasarım Sanatı*, 2. baskı. Englewood Cliffs, NJ: Prentice-Hall.

WEB ARAMA KONULARI

Algoritmik durum makinesi
Algoritmik durum makinesi şeması
Asenkron devre
Decimator
Dijital kontrol birimi
Dijital veri yolu birimi
Mealy makinesi
Moore makinesi
Yarış durumu

Bölüm 9

Standart IC'ler ve FPGA'lar ile Laboratuvar Deneyleri

9.1 DENEKLERE GİRİŞ

Bu bölüm, dijital devreler ve mantık tasarımı üzerine 17 laboratuvar deneyini sunmaktadır. Deneyler, bu kitabı kullanan öğrencilere uygulamalı deneyim kazandırmaktadır. Dijital devreler, laboratuvar ortamında kolayca monte edilebilen breadboard'lara yerleştirilmiş standart entegre devreler (IC'ler) kullanılarak inşa edilebilir. Deneyler, kitapta sunulan materyale göre sıralanmıştır. Son bölüm, deneylerde sunulan dijital devrelerin işlevselliğini simüle etmek ve doğrulamak için Verilog HDL kullanma önerileri ile birlikte bir dizi ekten oluşmaktadır. Eğer bir FPGA prototipleme kartı mevcutsa, deneyler standart IC'lere alternatif olarak bir FPGA'da uygulanabilir.

Deneylerin gerçekleştirilmesi için uygun bir mantık breadboard'un aşağıdaki ekipmanlara sahip olması gerekir:

- Işık yayan diyot (LED) gösterge lambaları.
- Mantık-1 ve mantık-0 sinyalleri sağlamak için anahtarlar.
- Tekil darbeler üretmek için butonlu pulsler ve debounce devreleri.
- En az iki frekansa sahip bir saat darbe üretici: dijital sinyallerde yavaş değişiklikleri gözlemlemek için yaklaşık 1 darbe/saniye düşük frekans ve bir osiloskopta dalga formlarını gözlemlemek için daha yüksek frekans.
- 5 V'luk bir güç kaynağı.
- IC'leri monte etmek için soket şeritleri.
- Bağlantı kabloları ve kablo kesmek için bir çift kablo soyucu.

Gerekli ekipmanları içeren dijital mantık eğitimleri birkaç üreticiden temin edilebilir. Bir dijital mantık eğitmeni LED lambalar, anahtarlar, pulsler,

r güç kaynağı ve IC soket şeritleri içerir. Bazı deneyler ek anahtarlar, lambalar veya I C soket şeritleri gerektirebilir. Daha fazla lehimsiz soket ve tak-çıkartma anahtarları ile lambalar içeren genişletilmiş breadboard'lar gerekebilir.

Gerekli ek ekipman, çift iz osiloskopu (Deney 1, 2, 8 ve 15 için), hata ayıklama için kullanılacak bir mantık probu ve bir dizi IC'dir. Deneyler için gereken IC'ler TTL veya CMOS serisi 7400'dür.

Deneylerde kullanılacak entegre devreler, küçük ölçekli entegrasyon (SSI) veya orta ölçekli entegrasyon (MSI) devreleri olarak sınıflandırılabilir. SSI devreleri bireysel kapılar veya flip-flop'lar içerirken, MSI devreleri belirli dijital işlevleri yerine getirir. Deneyler için gerekli olan sekiz SSI kapı entegre devresi—iki girişli NAND, NOR, AND, OR ve XOR kapıları, inverterler ve üç girişli ile dört girişli NAND kapıları—Şekil 9.1'de gösterilmiştir. Kapıların pin atamaları diyagramda belirtilmiştir. Pinler 1'den 14'e kadar numaralandırılmıştır. 14 numaralı pin V_{CC} olarak işaretlenmiştir ve 7 numaralı pin GND (toprak) olarak işaretlenmiştir. Bunlar, devrenin düzgün çalışması için 5 V'luk bir güç kaynağına bağlanması gereken besleme terminalleridir. Her entegre devre, tanımlama numarası ile tanınır; örneğin, iki girişli NAND kapıları, numarası 7400 olan entegre devre içinde bulunur.

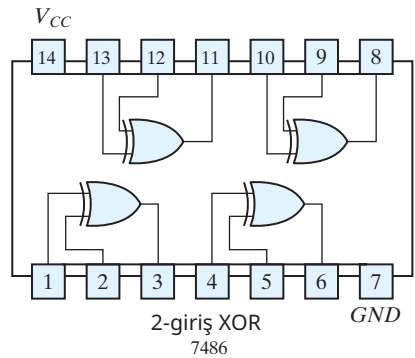
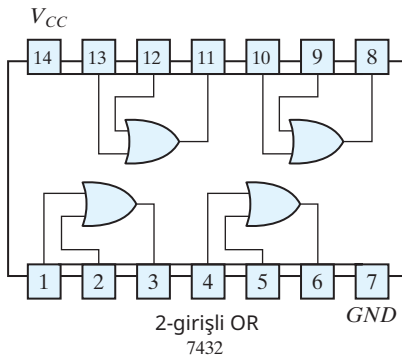
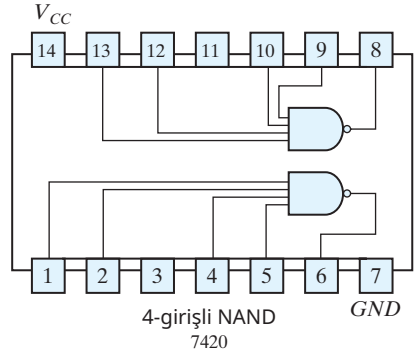
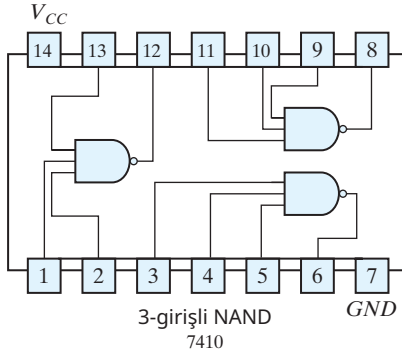
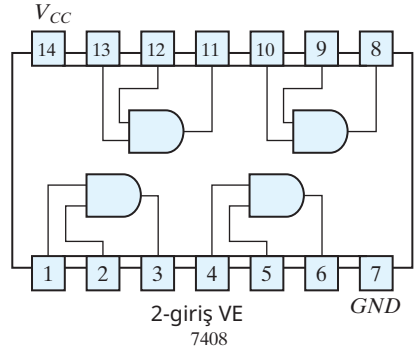
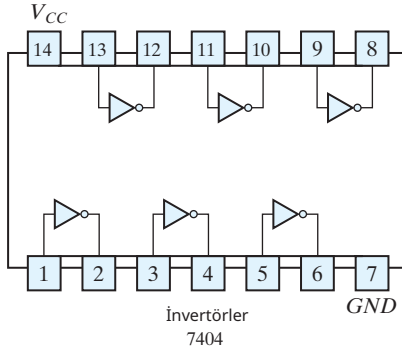
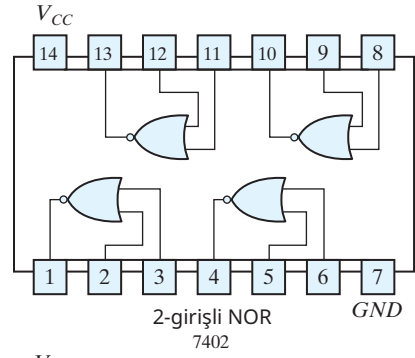
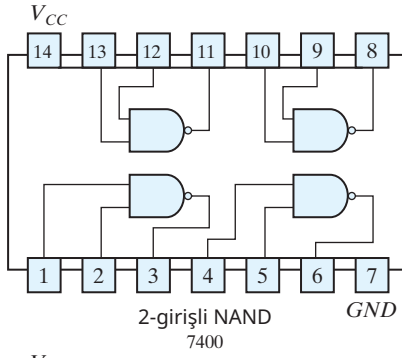
MSI devrelerinin ayrıntılı tanımları, üreticiler tarafından yayımlanan veri kitaplarında bulunabilir. Ticari bir MSI devresi ile deneyim kazanmanın en iyi yolu, entegre devrelerin iç, dış ve elektriksel özellikleri hakkında tam bilgi sağlayan bir veri kitabındaki tanımını incelemektir. Çeşitli yarı iletken şirketleri, 7400 serisi için veri kitapları yayımlar. Deneyler için gerekli olan MSI devreleri, ilk kez kullandıklarında tanıtılır ve açıklanır. Devrenin çalışması, önceki bölümlerdeki benzer devrelere atıfta bulunarak açıklanır. Bu bölümde verilen MSI devreleri hakkındaki bilgiler, deneylerin yeterince yapılması için yeterli olmalıdır. Yine de, bir veri kitabına başvurmak her zaman tercih edilir, çünkü devrelerin daha ayrıntılı tanımını sağlar.

Şimdi burada benimsenen MSI devrelerinin sunum yöntemini göstereceğiz. Örnek olarak, 7493 tipi ripple sayıcı IC'yi tanıtıyoruz. Bu IC, Deney 1'de ve sonraki deneylerde kombinasyonel devrelerin çalışmasını doğrulamak için bir dizi ikili sayı üretmek amacıyla kullanılmaktadır.

Bir veri kitabında bulunan 7493 IC hakkında bilgiler Şekil 9.2(a) ve (b)'de gösterilmektedir. Kısım (a), iç mantık devresinin diyagramını ve dış pinlere bağlantısını göstermektedir. Tüm girişler ve çıkışlar sembolik harflerle verilmiş ve pin numaralarına atanmıştır. Kısım (b), IC'nin fiziksel düzenini ve 14 pininin sinyal adlarına atanmasını göstermektedir. Bazı pinler devre tarafından kullanılmamaktadır ve NC (bağlantı yok) olarak işaretlenmiştir.

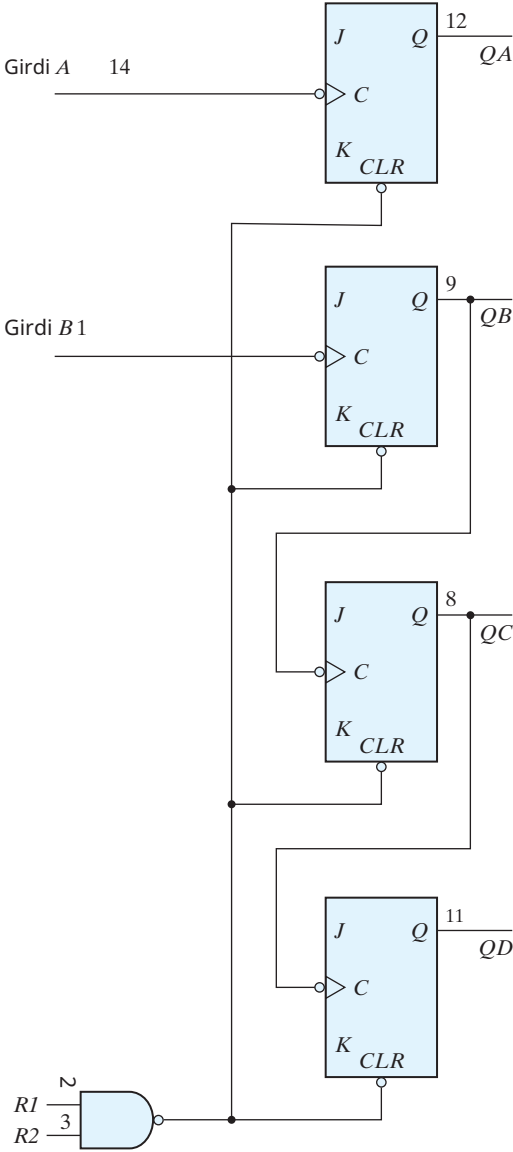
IC bir sokete yerleştirilir ve kablolar soket terminalleri aracılığıyla çeşitli pinlere bağlanır. Bu bölümde şematik diyagramlar çizerken, IC'yi blok diyagramı şeklin de göstereceğiz, Şekil 9.2 (c) gibi. IC numarası (burada, 7493) bloğun içine yazılmıştır. Tüm giriş terminalleri bloğun sol tarafında ve tüm çıkış terminalleri sağ tarafında yer almaktadır. Sinyallerin harf sembolleri, örneğin A,

R1, ve QA, blok içinde yazılmıştır ve ilgili pin numaraları, 14, 2 ve 12 gibi, dış hatlar boyunca yazılmıştır. V_{CC} , ve GND pin 5 ve 10'a bağlı güç terminalleridir. Blok boyutu, tüm girişleri karşılamak için değişebilir.

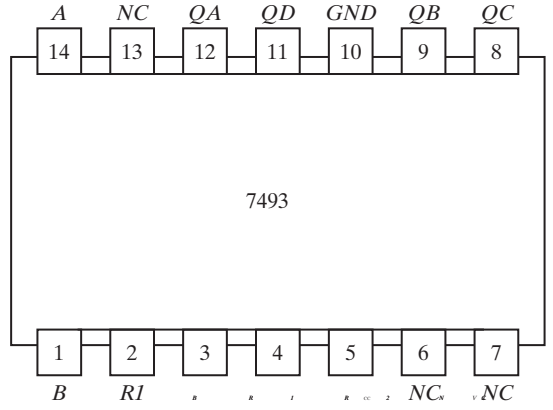


ŞEKİL 9.1

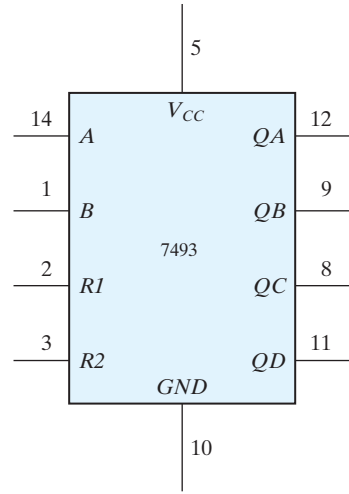
Tanımlama numaraları ve pin atamaları ile IC paketlerindeki dijital kapılar



(a) İç devre şeması



(b) Fiziksel düzen (NC: bağlantı yok)



(c) Şematik diyagram

ŞEKİL 9.2**İC tipi 7493 dalgalı sayıcı**

ve çıkış terminalleri. Girişler veya çıkışlar bazen bloğun üstünde veya basında yerleştirilebilir.

Devrenin çalışması, Şekil 6.8(a)'da gösterilen dalgalı sayıcıya benzer ve her flip-flop için asenkron bir temizleme vardır. Giriş $R1$ veya $R2$ veya her ikisi de mantık 0'a eşit olduğunda

(toprak), tüm asenkron temizlemeler 1'e eşittir ve devre dışıdır. Dört flip-flop'u 0'a te mizlemek için, NAND kapısının çıkışı 0'a eşit olmalıdır. Bu, hem girişlerin $R1$ hem de $R2$ mantık 1'de (yaklaşık 5 V) olmasıyla sağlanır. J ve K girişlerinin bağlantısının ol madığını unutmayın. Dış bağlantısı olmayan bir giriş terminalinin, mantık 1'e eşdeğ e r bir sinyal üretme etkisi olduğu TTL devrelerinin bir özelliğidir. Ayrıca, çıkış QA giri ş B ile dahili olarak bağlı dır.

7493 IC, giriş B ve flip-flop'lar QB , QC , ve QD kullanarak üç bitli bir sayaç ol ar ak çalışabilir. Çıkış QA giriş B ile bağlandığında dört bitli bir sayaç olarak çalışab ilir. Bu nedenle, devreyi dört bitli bir sayaç olarak çalıştırmak için pin 12 ile pin 1 a rasında bir dış bağlantı olması gerekmektedir. Sıfırlama girişleri, pin 2 ve 3'te sıras ıyla $R1$ ve $R2$ topraklanmalıdır. Pin 5 ve 10, 5 V güç kaynağına bağlanmalıdır. Gir iş darbeleri, pin 14'teki giriş A üzerine uygulanmalı ve sayacın dört flip-flop çıkışı, pin 12, 9, 8 ve 11'de sırasıyla QA , QB , QC , ve QD alınmalıdır; burada QA en az anlamlı bit olmaktadır.

Şekil 9.2 (c), bu bölümde tüm MSI devrelerinin grafiksel olarak nasıl semboli ze edileceğini göstermektedir. Her IC için, bu şekle benzer bir blok diyagramı verilecektir. IC blok diyagramındaki giriş ve çıkışlar için harf sembolleri, veri kit abında kullanılan sembollere göre olacaktır.

Tablo 9.1
Deneyler için Gerekli Entegre Devreler

IC Numarası	Açıklama	Grafik Sembolü	
		9. Bölümde	10. Bölümde
Çeşitli kapılar		Şekil 9.1	Şekil 10.1
7447	BCD-yedi-segment kod çözücü	Şekil 9.8	—
7474	Çift D tipi flip-floplar	Şekil 9.13	Şekil 10.9(b)
7476	Çift JK tipi flip-floplar	Şekil 9.12	Şekil 10.9(a)
7483	Dört bitlik ikili toplayıcı	Şekil 9.10	Şekil 10.2
7493	Dört bitlik dalgali sayaç	Şekil 9.2	Şekil 10.13
74151	8×1 çoklayıcı	Şekil 9.9	Şekil 10.7(a)
74155	3×8 çözücü	Şekil 9.7	Şekil 10.6
74157	Dörtlü 2×1 çoklayıcılar	Şekil 9.17	Şekil 10.7(b)
74161	Dört bitli senkron sayıcı	Şekil 9.15	Şekil 10.14
74189	16×4 rastgele erişim belleği	Şekil 9.18	Şekil 10.15
74194	İki yönlü kaydırma kaydedici	Şekil 9.19	Şekil 10.12
74195	Dört bitli kaydırma kaydedici	Şekil 9.16	Şekil 10.11
7730	Yedi segmentli LED ekran	Şekil 9.8	—
72555	Timer (aynı 555)	Şekil 9.21	—

mlerden mantık diyagramlarına atıfta bulunularak açıklanacaktır. Devrenin işle yişi, bir doğru tablo veya bir fonksiyon tablosu aracılığıyla belirtilecektir.

Diğer olası grafik semboller, Bölüm 10'da sunulmuştur. Bunlar, Elektrik ve Elektro nik Mühendisleri Enstitüsü tarafından onaylanan standart grafik sembollerdir ve IEE E Standard 91-1984'te verilmiştir. SSI kapıları için standart grafik semboller dikdörtg en şekillere sahiptir, Şekil 10.1'de gösterildiği gibi. 7493 IC'sinin standart grafik semb olü Şekil 10.13'te gösterilmiştir. Bu sembol, Şekil 9.2 (c)'de gösterilen sembolün yerin e kullanılabilir. Deneyleri gerçekleştirmek için gereken diğer IC'lerin standart grafik s embollerini Bölüm 10'da sunulmuştur. Standart semboller tercih edilirse, mantık devr elerinin şematik diyagramlarını çizmek için kullanılabilirler.

Tablo 9.1, bu deneyler için gereken IC'leri, bu bölümde sunulan şekil numara larıyla birlikte listelemektedir. Ayrıca, tablo, eşdeğer standart grafik sembollerini n çizildiği Bölüm 10'daki şekil numaralarını da listelemektedir.

Sonraki 18 bölüm, dijital entegre devreler kullanmayı gerektiren 18 donanım deneyini s unmaktadır. Bölüm 9.20, bir Verilog HDL derleyicisi ve simülatörü gerektiren HDL simülasy on deneylerini özetlemektedir.

9.2 DENEY 1: İKİLİ VE ONDALIK SAYILAR

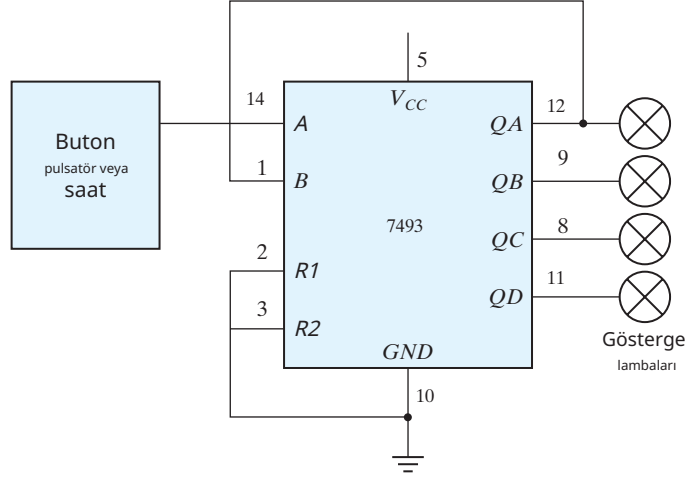
Bu deney, ikili sayıların sayım dizisini ve ikili kodlu ondalık (BCD) temsiliğini göste rmektedir. Laboratuvar ortamında kullanılan breadboard'a bir giriş niteliği taşıy ve öğrenciyi katot ışın oscilloskobu ile tanıştırmak. Deney sırasında faydalı olabile cek metin referans materyali, ikili sayılarla ilgili Bölüm 1.2 ve BCD sayılarıyla ilgi li Bölüm 1.7'de bulunabilir.

İkili Sayım

IC tipi 7493, Şekil 9.2'de gösterildiği gibi dört flip-flop içerir. Bu flip-floplar, ikili vey a BCD olarak saymak için bağlanabilir. Dış terminalleri bağlayarak IC'yi dört bitlik bir ikili sayaç olarak çalışacak şekilde bağlayın, Şekil 9.3'te gösterildiği gibi. Bu, pin 12'den (çıkış QA) pin 1'e (giriş B) bir tel bağlayarak yapılır. Pin 14'teki giriş A, tek darbeler sağlayan bir pulsere bağlanır. İki sıfırlama girişi, R1 ve R2, toprağa bağla nır. Dört çıkış, dört gösterge lambasına gider; sayacın en düşük bitinin QA'dan en sağdaki gösterge lambasına bağlı olduğunu unutmayın. IC'ye 5 V ve toprak sağla mayı unutmayın. Tüm bağlantılar, güç kaynağı kapalıyken yapılmalıdır.

Gücü açın ve dört gösterge lambasını gözlemleyin. Çıkıştaki dört bitlik sayı, b utonlu puls üreticisinde üretilen her puls için 1 artırılır. Sayım, ikili 15'e kadar gi der ve sonra 0'a geri döner. Sayıcının 14 numaralı pinindeki girişi puls üreticisin den ayırın ve saniyede yaklaşık 1 puls üreten bir saat üreticisine bağlayın. Bu, ot omatik bir ikili sayım sağlayacaktır.

İkili sayıcının, kombinasyon devrelerini test etmek için giriş ikili sinyalleri sağlamak ama cıyla sonraki deneylerde kullanılacağını unutmayın.



ŞEKİL 9.3
İkili sayaç

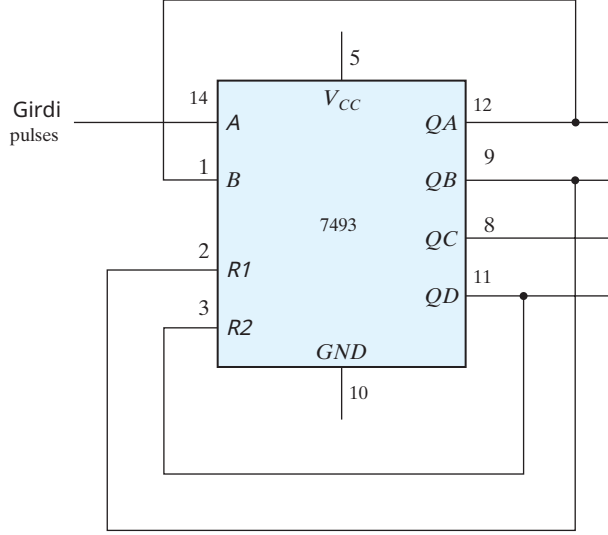
Osiloskop Ekranı

Saatin frekansını 10 kHz veya daha yüksek bir değere artırın ve çıkışını bir osiloskopa bağlayın. Osiloskopta saat çıkışını gözlemleyin ve dalga formunu çizin. İki kanallı bir osiloskop kullanarak, QA çıkışını bir kanala ve saat çıkışını ikinci kanala bağlayın. Saat darbesi 1'den 0'a negatif geçiş yaptığında QA çıkışının her seferinde tamamlandığını not edin. Ayrıca, ilk flip-flopun çıkışındaki saat frekansının giriş saat frekansının yarısı olduğunu da not edin. Her flip-flop, gelen frekansı 2 ile böler. Dört bitlik sayıcı, gelen frekansı QD çıkışında 16 ile böler. Saatin sayıcının dört çıkışıyla olan ilişkisini gösteren bir zaman diyagramı elde edin. En az 16 saat döngüsü içerdiğinden emin olun. İki kanallı bir osiloskolla ilerlemenin yolu şu şekildedir: Öncelikle, saat darbelerini ve QA 'yı gözlemleyin ve zaman dalga formlarını kaydedin. Ardından, QA ile birlikte QB 'nin dalga formlarını gözlemleyip kaydedin, ardından QB 'nin dalga formlarını gözlemleyin.

QC ve ardından QC ile QD . Sonucun, en az 16 saat döngüsü içeren tek bir bileşik diyagramda saat ile dört çıkış arasındaki ilişkiyi gösteren bir diyagram olması gerekir.

BCD Sayımı

BCD temsili, 0'dan 9'a kadar olan kodlanmış ondalık rakamları temsil etmek için 000 0 ile 1001 arasındaki ikili sayıları kullanır. IC tipi 7493, Şekil 9.4'te gösterilen dış bağlantılar yapılarak bir BCD sayacı olarak çalıştırılabilir. Çıkışlar QB ve QD iki sıfırlama girişine, $R1$ ve $R2$ bağlanır. Hem $R1$ hem de $R2$ 1'e eşit olduğunda, sayacın içindeki dört hücre, giriş darbesine bakılmaksızın 0'a sıfırlanır. Sayaç 0'dan başlar ve her giriş darbesi onu 1 artırır, 1001 sayısına ulaşana kadar devam eder. Sonraki darbe çıkışı 1010'a değiştirir, böylece QB ve QD 1'e eşit olur. Bu geçici çıkış



ŞEKİL 9.4
BCD sayacı

sürdürülen, çünkü dört hücre hemen 0'a temizlenir, bu da çıktının 0000'a gitmesi ne neden olur. Böylece, 1001 sayımından sonraki darbe çıktığı 0000'a değiştirir, B CD sayımı üretir.

IC'yi BCD sayıcı olarak çalışacak şekilde bağlayın. Girişi bir pulsere ve dört çıkışı gö sterge lambalarına bağlayın. Sayımın 0000'dan 1001'e gittiğini doğrulayın.

Girişi pulseden ayırın ve bir saat jeneratörüne bağlayın. Saat dalga formun u ve osiloskopta dört çıkışı gözlemleyin. Saat ile dört çıkış arasındaki ilişkiyi gö steren doğru bir zaman diyagramı elde edin. Osiloskop ekranında ve bileşik z aman diyagramında en az 10 saat döngüsü içermeyi unutmayın.

Çıkış Deseni

BCD sayıcıya gelen sayım darbeleri sürekli olduğunda, sayaç 0000'dan 1001'e ve tekrar 0000'a kadar olan diziyi tekrar eder. Bu, dört çıkıştaki her bitin her 1 0 darbeden bir tekrar eden sabit bir 1'ler ve 0'lar deseni ürettiği anlamına geli r. Bu desenler, 0000'dan 1001'e kadar olan ikili sayıların bir listesinden tahmin edilebilir. Liste, en az anlamlı bit olan çıkış QA'nın, alternatif 1'ler ve 0'lar dese nini ürettiğini gösterecektir. En anlamlı bit olan çıkış QD, sekiz 0'dan sonra iki 1'den oluşan bir desen üretir. Diğer iki çıkışın desenini elde edin ve ardından osiloskopta dört deseni kontrol edin. Bu, bir kanalda saat darbelerini ve diğer kanalda bir çıkış dalga formunu gösteren çift izli bir osiloskop ile yapılır. İlgili ç ıkış için 1'ler ve 0'lar desenini, darbelerin 1'den 0'a değiştiği dikey pozisyonlar daki çıkış seviyelerini gözlemleyerek elde edilir.

Diğer Sayımlar

IC türü 7493, 0'dan çeşitli son sayılara kadar saymak için bağlanabilir. Bu, bir veya iki çıkışı sıfırlama girişlerine bağlayarak yapılır, $R1$ ve $R2$. Böylece, eğer $R1$ Şekil 9.4'te QA 'ya bağlanırsa, QB yerine, sonuçta elde edilen sayı 0000'dan 1000'e kadar olacaktır, bu da 1001'de $n-1$ eksiktir ($QD=1$ ve $QA=1$).

$R1$ ve $R2$ son sayıyı nasıl etkilediğini bilginizi kullanarak, 7493

IC'yi 0000'dan aşağıdaki son sayılara kadar saymak için bağlayın:

- (a) 0101
- (b) 0111
- (c) 1011

Her devreyi bağlayın ve puls kaynağından uygulanan darbeleri kullanarak sayım dizisini doğrulayın ve gösterge lambalarındaki çıkış sayısını gözlemleyin. Başlangıç sayısı, son sayının değerinden büyükse, çıkış 0'a temizlenene kadar giriş darbelerini uygulamaya devam edin.

9.3 DENEY 2: DİJİTAL LOJİK KAPILAR

Bu deneyde, çeşitli entegre devre kapılarının mantık davranışını inceleyeceksiniz:

7400 dörtlü iki girişli NAND kapıları

7402 dörtlü iki girişli NOR kapıları

7404 altı girişli inverterler

7408 dörtlü iki girişli AND kapıları

7432 dörtlü iki girişli OR kapıları

7486 dörtlü iki girişli XOR kapıları

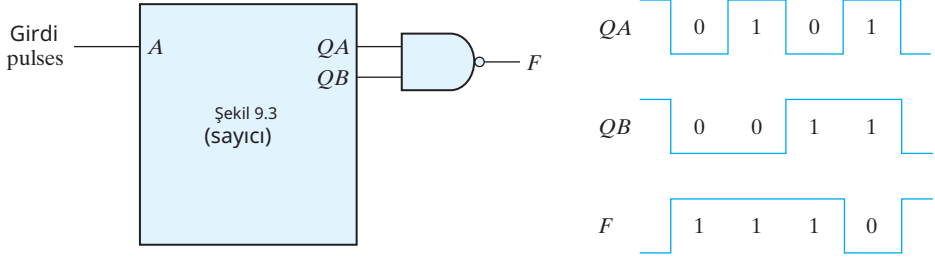
Farklı kapılara pin atamaları Şekil 9.1'de gösterilmiştir. "Dörtlü" ifadesi, pakette dört kap bulunduğunu belirtir. Dijital mantık kapıları ve özellikleri Bölüm 2.8'de tartışılmaktadır. Bir NAND uygulaması Bölüm 3.7'de ele alınmaktadır.

Doğruluk Tabloları

Listelenen her entegreden bir kap kullanarak kapının doğruluk tablosunu elde edin. Doğruluk tablosu, kapının girişlerini anahtarlara ve çıkışını bir gösterge lambasına bağlayarak elde edilir. Sonuçlarınızı Şekil 2.5'te listelenen doğruluk tablolarıyla karşılaştırın.

Dalgalar

Listelenen her kap için, kapının giriş-çıkış dalga formunu elde edin. Dalga formları osiloskopta gözlemlenmelidir. Kapiya giriş sağlamak için bir ikili sayacın (Şekil 9.3) iki düşük dereceli çıkışını kullanın. Örnek olarak, NAND kapısı için devre ve dalga formları Şekil 9.5'te gösterilmektedir. Osiloskop ekranı bu dalga formunu tekrar edecektir, ancak yalnızca tekrarsız kısmı kaydetmelisiniz.



ŞEKİL 9.5

NAND kapısı için dalga formları

İletim Gecikmesi

7404 entegresinin içindeki altı inverteri ardışık olarak bağlayın. Çıkış, girişle aynı olacaktır, tek fark, sinyalin tüm altı inverterden geçmesi için geçen süre kadar gecikmiş olacaktır. İlk inverterin girişine saat darbeleri uygulayın. Osiloskopu kullanarak, altıncı inverterin çıkışına kadar olan gecikmeyi, darbenin yükselişi sırasında ve tekrar düşüşü sırasında belirleyin. Bu, giriş saat darbelerini kanallardan birine ve altıncı inverterin çıkışını ikinci kanala uygulayarak çift izleme osiloskobu ile yapılır. Zaman tabanı düğmesini en düşük zaman-bölüm ayarına getirin. İki darbenin yükselme veya düşme süresi ekranda görünmelidir. Toplam gecikmeyi 6'ya bölerek her inverter için ortalama iletim gecikmesini elde edin.

Evrensel NAND Kapısı

Tek bir 7400 entegresi kullanarak, aşağıdaki devreyi bağlayın:

- bir inverter,
- iki girişli AND,
- iki girişli OR,
- iki girişli NOR,
- iki girişli XOR. (Bkz. Şekil 3.32.)

Her durumda, devrenizi doğrulamak için doğruluk tablosunu kontrol edin.

NAND Devresi

Tek bir 7400 entegresi kullanarak, Boolean

fonksiyonunu gerçekleştiren NAND kapıları ile bir devre oluşturun.

$$F = AB + CD$$

- Devre şemasını çizin.
- Dört girişin bir fonksiyonu olarak F için doğruluk tablosunu elde edin.
- Devreyi bağlayın ve doğru tabloyu doğrulayın.

1'lerin ve 0'ların desenlerini kaydedin F girişleri A, B, C ve D ikili 0'dan

binary 15'e geçerken.

Şekil 9.3'te gösterilen ikili sayıcının dört çıkışını NAND devresinin dört girişine bağlayın. Sayıcıdan gelen giriş saat darbelerini bir dual-trace osiloskopun bir kanalına ve çıkış F 'yi diğer kanala bağlayın. Her saat darbesinden sonra F 'nin 1'ler ve 0'lar desenini gözlemleyin ve kaydedin, ardından bunu 4. adımda kaydedilen desene karşılaştırın.

9.4 DENEY 3: BOOLEAN FONKSİYONLARININ SİMLİFİKASYONU

Bu deney, bir Boolean fonksiyonu ile ilgili mantık diyagramı arasındaki ilişkiyi göstermektedir. Boolean fonksiyonları, Bölüm 3'te tartışıldığı gibi harita yöntemi kullanılarak basitleştirilmiştir. Mantık diyagramları, Bölüm 3.7'de açıklandığı gibi NAND kapıları ile çizilecektir.

Kullanılacak mantık diyagramları için kapı IC'leri, aşağıdaki NAND kapılarının içeren Şekil 9.1'den olmalıdır:

7400 iki girişli NAND

7404 inverter (bir girişli NAND)

7410 üç girişli NAND

7420 dört girişli NAND

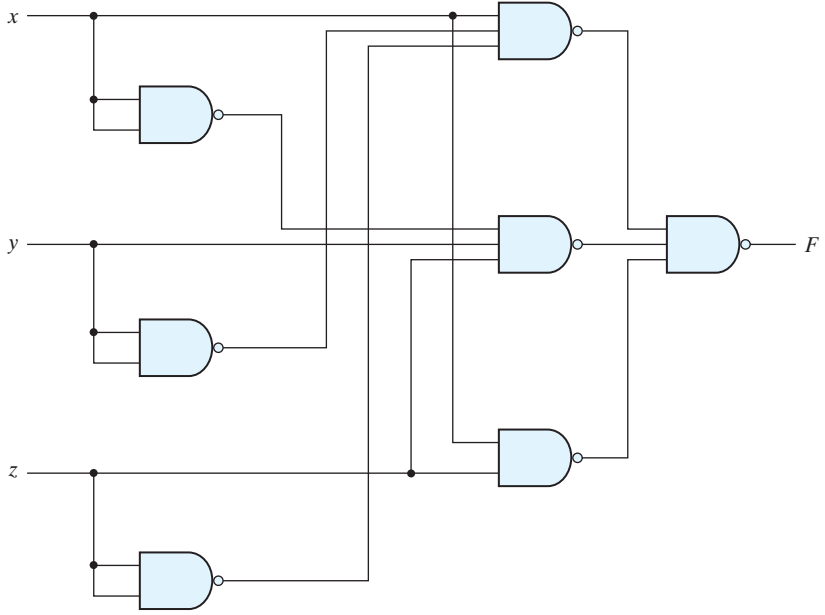
Bir NAND kapısının girişi kullanılmıyorsa, açık bırakılmamalı, bunun yerine kullanılan başka bir girişe bağlanmalıdır. Örneğin, devre bir inverter gerektiriyorsa ve 7400 IC'sinde ekstra bir iki girişli kapı varsa, o zaman kapının her iki girişi birleştirilerek bir inverter için tek bir giriş oluşturulmalıdır.

Mantık Diyagramı

Deneyin bu kısmı, verilen bir mantık diyagramı ile başlar ve kapı sayısını ve muhtemelen IC sayısını azaltmak için basitleştirme prosedürlerini uygulamaya devam ederiz. Şekil 9.6'da gösterilen mantık diyagramı iki IC gerektirir - bir 7400 ve bir 7410. Girişler için inverterlerin x, y ve z 7400 IC'sindeki kalan üç kapıdan elde edildiğini unutmayın. Eğer inverterler bir 7404 IC'sinden alınsaydı, devre üç IC gerektirecekti. Ayrıca, SSI devreleri çizerken, kapıların MSI devrelerinde olduğu gibi bloklar içinde kapatıldığını da unutmayın.

Kapılardaki tüm giriş ve çıkışlara pin numaraları atayın ve devreyi x, y ve z girişlerinin üç anahtara gittiği ve çıkışın F bir gösterge lambasına bağlandığı şekilde bağlayın. Devreyi doğruluğu tablosunu elde ederek test edin.

Devrenin Boolean fonksiyonunu elde edin ve harita yöntemini kullanarak basitleştirin. Orijinal devreyi kesmeden basitleştirilmiş devreyi inşa edin. Her iki devreyi de aynı girişleri uygulayarak test edin ve ayrı çıkışları gözlemleyin. Sekiz olası giriş kombinasyonunun her biri için iki devrenin aynı çıkışlara sahip olduğunu gösterin. Bu, basitleştirilmiş devrenin tam olarak orijinal devre gibi davrandığını kanıtlayacaktır.



ŞEKİL 9.6
Deney 3 için mantık diyagramı

Boolean Fonksiyonları

İki Boolean fonksiyonunu toplam-minterm formunda düşünün:

$$F_1(A, B, C, D) = (0, 1, 4, 5, 8, 9, 10, 12, 13)$$

$$F_2(A, B, C, D) = (3, 5, 7, 8, 10, 11, 13, 15)$$

Bu fonksiyonları haritalar aracılığıyla basitleştirin. Dört girişi, A , B , C ve D , ve iki çıkışı, F_1 ve F_2 olan bir bileşik mantık diyagramı elde edin. İki fonksiyonu birlikte, minimum sayıda NAND entegre devre (IC) kullanarak uygulayın. İlgili terim her iki fonksiyon için gerekli ise aynı kapıyı çoğaltmayın. Mümkünse, mevcut IC'lerdeki ekstra kapıları inverterler için kullanın. Devreyi bağlayın ve çalışmasını kontrol edin. Devreden elde edilen F_1 ve F_2 için doğruluk tablosu, listelenen minterm'lerle uyumlu olmalıdır.

Tamamlayıcı

Aşağıdaki Boolean fonksiyonunu bir haritada çizin:

$$F = A'D + BD + B'C + AB'D$$

Haritadaki 1'leri birleştirerek F için ürünlerin toplamı formunda basitleştirilmiş fonksiyonu elde edin. Ardından haritadaki 0'ları birleştirerek F için de basitleştirilmiş fonksiyonu elde edin, yine ürünlerin toplamı formunda. Hem F hem de F' 'yi NAND kapıları ile uygulayın ve iki devreyi aynı giriş anahtarlarına bağlayın, ancak ayrı çıkış gösterge lambalarına bağlayın. Her devrenin laboratuvar ortamında doğruluk tablosunu elde edin ve bunların birbirinin tamamlayıcısı olduğunu gösterin.

9.5 DENEY 4: BİRLEŞİK DEVRELER

Bu deneyde, dört birleşik mantık devresi tasarlayacak, inşa edecek ve test edeceksiniz. İlk iki devre NAND kapıları ile, üçüncü XOR kapıları ile ve dördüncü bir kodlayıcı ve NAND kapıları ile inşa edilecektir. Parite üretici ile ilgili referans Bölüm 3.9'da bulunabilir. Kodlayıcı ile uygulama Bölüm 4.9'da tartışılmaktadır.

Tasarım Örneği

Dört girişi— A , B , C ve D —ve bir çıkışı, F olan bir birleşik devre tasarlayın. $F = A = 1$ olduğunda 1'e eşit olmalıdır, eğer $B=0$ ise, ya da $B=1$ olduğunda, eğer C veya D 1'e eşit değilse. Aksi takdirde, çıkış 0'a eşit olmalıdır.

- Devreye ait gerçeklik tablosunu elde edin.
- Çıkış fonksiyonunu basitleştirin.
- Minimum sayıda IC kullanarak NAND kapıları ile devrenin mantık diyagramını çizin.
- Devreyi inşa edin ve verilen koşulları doğrulayarak düzgün çalışıp çalışmadığını test edin.

Çoğunluk Mantığı

Çoğunluk mantığı, girişlerin çoğunluğu 1 olduğunda çıkışı 1 olan bir dijital devredir. Aksi takdirde çıkış 0'dır. Minimum sayıda IC kullanarak NAND kapıları ile üç girişli bir çoğunluk devresi tasarlayın ve test edin.

Parity Üretici

Dört mesaj bitinden bir çift parite biti üreten bir devre tasarlayın, inşa edin ve test edin. XOR kapıları kullanın. Bir XOR kapısı daha ekleyerek devreyi genişletin, böylece bir tek parite biti de üretsinsin.

Decodör Uygulaması

Bir kombinasyon devresinin üç girişi vardır— x , y ve z —ve üç çıkışı vardır— F_1 , F_2 , ve F_3 . Devre için basitleştirilmiş Boolean fonksiyonları

$$F_1 = xz + x'y'z'$$

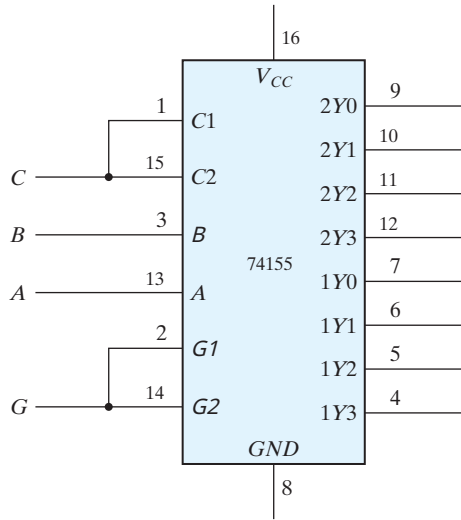
$$F_2 = x'y + xy'z'$$

$$F_3 = xy + x'y'z$$

74155 kodlayıcı entegresi ve harici

NAND kapıları kullanarak kombinasyonel devreyi uygulayın ve test edin.

Kodlayıcının blok diyagramı ve doğruluk tablosu Şekil 9.7'de gösterilmiştir. 74155, çift 2×4 kodlayıcı veya tek 3×8 kodlayıcı olarak bağlanabilir. 3×8 kodlayıcı istendiğinde, girişler $C1$ ve $C2$, ayrıca girişler $G1$ ve $G2$, blok diyagramında gösterildiği gibi bir araya bağlanmalıdır. Devrenin işlevi, Şekil 4.18'de gösterilenle benzerdir. G etkinleştirme girişidir ve doğru çalışma için 0'a eşit olmalıdır. Sekiz çıkış, veri kitabında verilen sembollerle etiketlenmiştir. 74155, NAND kapıları kullanır; bu nedenle, seçilen çıkış 0'a giderken tüm diğer çıkışlar 1'de kalır. Kodlayıcı ile uygulama, Şekil 4.21'de gösterildiği gibidir; ancak 74155 kullanıldığında OR kapıları dış NAND kapıları ile değiştirilmelidir.



Gerçek tablo

Girdiler				Çıktılar															
<i>a</i>	<i>c</i>	<i>b</i>	<i>A</i>	<i>0</i>	<i>2</i>	<i>1Y</i>	<i>2</i>	<i>2Y</i>	<i>2</i>	<i>3Y</i>	<i>1</i>	<i>0Y</i>	<i>1</i>	<i>1Y</i>	<i>1</i>	<i>2Y</i>	<i>1</i>	<i>Y3</i>	
100000000	X00001111	X00110011	X01010101	101111111	110111111	110111111	111011111	111011111	111101111	111110111	111111011	111111101	111111110	111111111	111111111	111111111	111111111	111111111	

ŞEKİL 9.7

74155 tipi IC, 3×8 kodlayıcı olarak bağlanmıştır

9.6 DENEY 5: KOD DÖNÜŞTÜRÜCÜLER

Bir ikili koddan diğerine dönüşüm dijital sistemlerde yaygındır. Bu deneyde, üç kombinasyonel devre dönüştürücüsü tasarlayıp inşa edeceksiniz. Kod dönüşümü Bölüm 4.4'te tartışılmaktadır.

Gray Kodu ile İkili

Dört girişi ve dört çıkışı olan bir kombinasyonel devre tasarlayın, bu devre dört bitlik Gray kodu sayısını (Tablo 1.6) eşdeğer dört bitlik ikili sayıya dönüştürsün. Devreyi özel-VE kapıları ile gerçekleştirin. (Bu, bir 7486 IC ile yapılabilir.) Devreyi dört anahtara ve dört gösterge lambasına bağlayın ve doğru çalışmayı kontrol edin.

9'un Tamamlayıcısı

BCD'de bir ondalık basamağı temsil eden dört giriş hattı ve giriş basamağının 9'un tamamlayıcısını üreten dört çıkış hattı olan bir kombinasyonel devre tasarlayın. Giriş BCD sayısının a bir hata tespit eden beşinci bir çıkış sağlayın. Bu çıkış, dört giriş BCD kodunun kullanılmayan kombinasyonlarından birine sahip olduğunda mantık 1'e eşit olmalıdır. Şekil 9.1'de listelenen kapılardan herhangi birini kullanın, ancak kullanılan toplam IC sayısını en aza indirin.

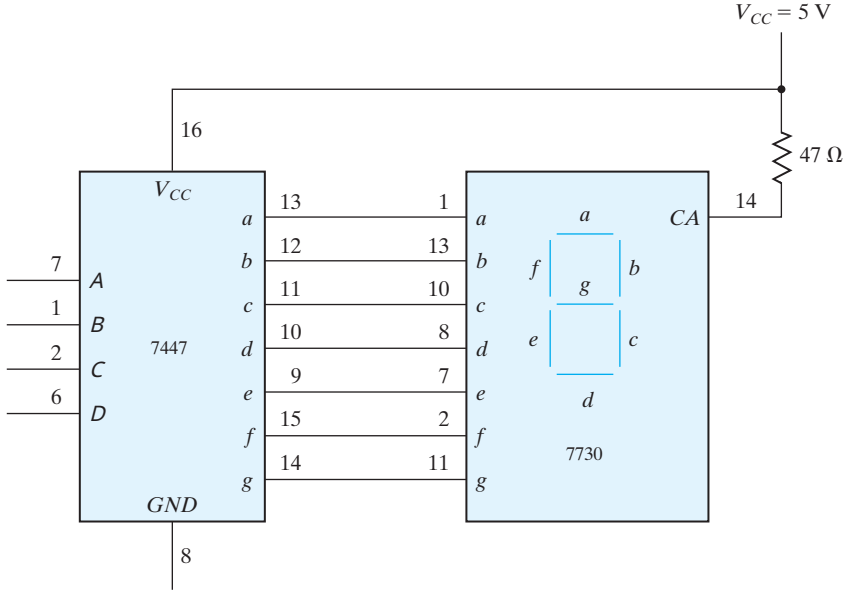
Yedi Segmentli Gösterge

Yedi segmentli bir gösterge, 0'dan 9'a kadar olan ondalık rakamlardan herhangi birini göstermek için kullanılır. Genellikle, ondalık rakam BCD formatında mevcuttur. Bir BCD'den yedi segmentli kodlayıcı, BCD formatında bir ondalık rakam alır ve karşılık gelen yedi segment kodunu üretir; bu, Problem 4.9'da resimsel olarak gösterilmiştir.

Şekil 9.8, kodlayıcı ile gösterge arasındaki gerekli bağlantıları göstermektedir. 7447 entegresi, BCD rakamı için dört girişi olan bir BCD'den yedi segmentli kodlayıcı/sürücüdür. Giriş *D* en anlamlı ve giriş *A* en az anlamlıdır. Dört bitlik BCD rakamı, *a* ile *g* arasındaki çıkışlarla yedi segmentli bir koda dönüştürülür. 7447'nin çıkışları, 7730 (veya eşdeğeri) yedi segmentli göstergeye uygulanır. Bu entegre, paketin üstünde yedi ışık yayan diyot (LED) segmentini içerir. 14 numaralı pin üzerindeki giriş, tüm LED'ler için ortak anot ($\overline{CA}$)'dır. Seçilen LED segmentlerine uygun akımı sağlamak için V_{CC} 'ye 47 Ω 'lık bir direnç gereklidir. Diğer eşdeğer yedi segmentli gösterge entegreleri, ek anot terminalleri içerebilir ve farklı direnç değerleri gerektirebilir.

Şekil 9.8'de gösterilen devreyi oluşturun. Dört anahtardan dört bitlik BCD rakamlarını uygulayın ve 0'dan 9'a kadar olan ondalık gösterimi gözlemleyin. 1010 ile 1111 arasındaki girişlerin BCD'de bir anlamı yoktur. Dekodere bağlı olarak, bu değerler ya boş bir ekran ya da anlamsız bir desenin görüntülenmesine neden olabilir. Kullanılmayan altı giriş kombinasyonunun çıkış desenlerini gözlemleyin ve kaydedin.

Bölüm 9.7 Deney 6: Çoklayıcılar ile Tasarım 453



ŞEKİL 9.8

BCD-yedi segment kod çözücü (7447) ve yedi segment gösterge (7730)

9.7 DENEY 6: ÇOKLAYICILAR İLE TASARIM

Bu deneyde, bir kombinasyonel devre tasarlayacak ve bunu Bölüm 4.11'de açıklandığı gibi çoklayıcılar ile uygulayacaksınız. Kullanılacak çoklayıcı, Şekil 9.9'da gösterilen IC tipi 74151'dir. 74151'in iç yapısı, Şekil 4.25'te gösterilen diyagrama benzer, tek farkı dört yerine sekiz girişe sahip olmasıdır. Sekiz giriş, D0'dan D7'ye kadar adlandırılmıştır. Üç seçim hattı—C, B ve A—çoklayıcıya uygulanacak belirli girişi seçer. Bir strobe kontrol S, bir etkinleştirme sinyali olarak işlev görür. Fonksiyon tablosu, çıkış Y'nin seçim hatlarına bağlı olarak değerini belirtir. Çıkış W, Y'nin tamamlayıcısıdır. Doğru çalışma için, strobe girişi S'nin topraklanması gerekir.

Tasarı Spesifikasyonları

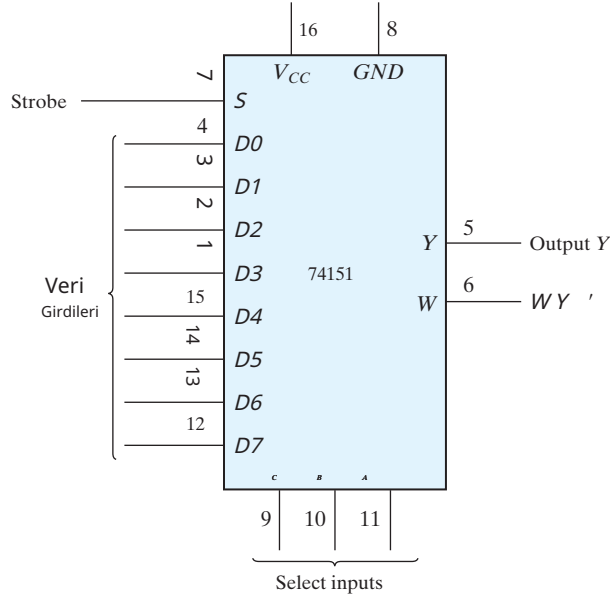
Küçük bir şirketin 10 hisse senedi vardır ve her hisse senedi sahibine bir oy hakkı verir hisse senedi sahipleri toplantısında. 10 hisse senedi dört kişi tarafından aşağıdaki gibi sahiplenilmiştir:

Bay W: 1 hisse

Bay X: 2 hisse

Bay Y: 3 hisse

Bayan Z: 4 hisse



Fonksiyon tablosu

Strobe S	Seç C B A			Çıktı Y
1	X	X	X	0
0	0	0	0	D0
0	0	0	1	D1
0	0	1	0	D2
0	0	1	1	D3
0	1	0	0	D4
0	1	0	1	D5
0	1	1	0	D6
0	1	1	1	D7

ŞEKİL 9.9

IC tipi 74151 38×1 çoklayıcı

Bu kişilerin her birinin, hisse senetleri için evet oyu verirken kapatacağı ve hayır oyu verirken açacağı bir anahtarı vardır.

Her bir önlem için evet oyu veren toplam hisse senedi sayısını gösteren bir devre tasarlamak gereklidir. Gerekli sayıyı göstermek için Şekil 9.8'de gösterildiği gibi yedi segmentli bir gösterg ve bir kodlayıcı kullanın. Eğer tüm hisseler bir önlem için hayır oyu verirse, gösterge boş olmalıdır. (7447'ye ikili giriş 15 verildiğinde tüm yedi segment boş kalır.) Eğer bir önlem için 10 hisse evet oyu verirse, gösterge 0 göstermelidir. Aksi takdirde, gösterge evet oyu veren hisse sayısına eşit bir ondalık sayı göstermektedir. Hisse sahiplerinin anahtarlarından gelen girişleri 7447 için BCD rakamına dönüştüren kombinasyonel devreyi tasarlamak için dört 74151 çoklayıcı kullanın. Mantık 1 için 5 V kullanmayın. Girişi topraklanmış bir inverterin çıkışını kullanın.

9.8 DENEY 7: TOPLAYICILAR VE ÇIKARICILAR

Bu deneyde, çeşitli toplayıcı ve çıkarıcı devreler inşa edecek ve test edeceksiniz. Çıkarıcı devre, iki sayının göreceli büyüklüklerini karşılaştırmak için kullanılır. Toplayıcılar Bölüm 4.3'te tartışılmaktadır. 2'nin tamamlayıcı ile çıkarma, Bölüm 1.6'da açıklanmaktadır. Dört bitli paralel toplayıcı-çıkarıcı Şekil 4.13'te gösterilmiştir ve iki sayının karşılaştırması Bölüm 4.8'de açıklanmaktadır.

Yarım Toplayıcı

Bir XOR kapısı ve iki NAND kapısı kullanarak bir yarım toplayıcı devresi tasarlayın, inşa edin ve test edin.

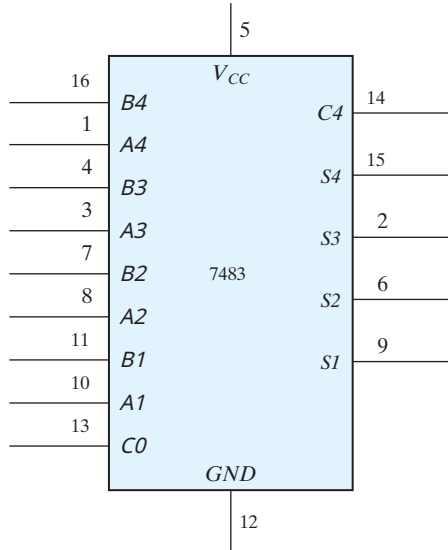
Tam Toplayıcı

İki entegre devre (IC), 7486 ve 7400 kullanarak bir tam toplayıcı devresi tasarlayın, inşa edin ve test edin.

Paralel Toplayıcı

IC türü 7483, dört bitli ikili paralel toplayıcıdır. Pin ataması Şekil 9.10'da gösterilmiştir. İki dört bitli giriş ikili sayılar $A1$ ile $A4$ ve $B1$ ile $B4$ arasındadır. Dört bitli sum, $S1$ ile $S4$ arasından elde edilir. $C0$ giriş taşıma ve $C4$ çıkış taşımasıdır.

Güç kaynağı ve toprak terminallerini bağlayarak 7483 dört bitlik ikili toplayıcıyı test edin. Ardından dört A girişi, 1001 gibi sabit bir ikili sayıya bağlayın ve B girişlerini ve giriş taşımasını beş anahtara bağlayın. Beş çıkış,



ŞEKİL 9.10

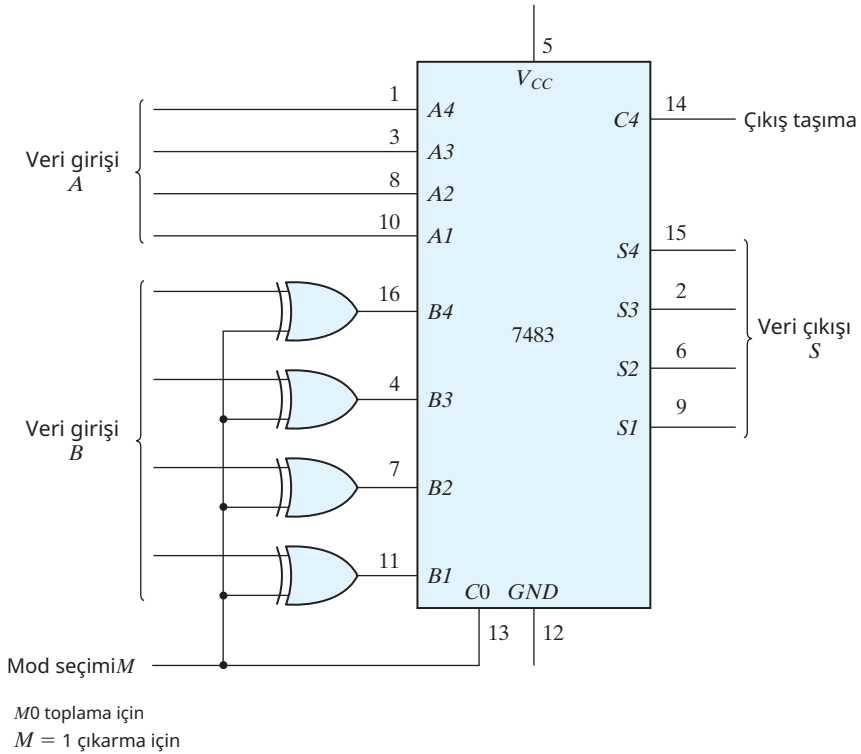
IC tipi 7483 dört bitli ikili toplayıcı

gösterge lambaları. Birkaç ikili sayının toplamını yapın ve çıkış toplamı ile çıkış taşıma anın doğru değerleri verdiğini kontrol edin. Giriş taşıması 1'e eşit olduğunda, çıkış toplamına 1 eklediğini gösterin.

Toplayıcı-Çıkarıcı

İki ikili sayı, çıkarılacak sayının 2'nin tamamlayıcısını alarak ve bunu çıkarandan ekleyerek çıkarılabilir. 2'nin tamamlayıcısı, 1'in tamamlayıcısını alıp 1 ekleyerek elde edilebilir. $A - B$ işlemini gerçekleştirmek için, B 'nin dört bitini tamamlıyoruz, bunları A 'nın dört bitine ekliyoruz ve giriş taşıması ile 1 ekliyoruz. Bu, Şekil 9.11'de gösterildiği gibi yapılır. Dört XOR kapısı, mod seçimi $M = 1$ olduğunda B 'nin bitlerini tamamlar (çünkü $x \oplus 1 = \bar{x}$ ve $M = 0$ olduğunda B 'nin bitlerini değiştirmeden bırakır (çünkü $x \oplus 0 = x$). Böylece, mod seçimi M 1'e eşit olduğunda, giriş taşıması $C0$ 1'e eşittir ve toplam çıkışı A ile B 'nin 2'nin tamamlayıcısının toplamıdır. M 0'a eşit olduğunda, giriş taşıması 0'a eşittir ve toplam $A+B$ üretir.

Toplayıcı-çıkarıcı devresini bağlayın ve düzgün çalışıp çalışmadığını test edin. Dört A girişi sabit ikili sayı 1001'e ve B girişlerini anahtarlara bağlayın. Uygulayın



ŞEKİL 9.11
Dört bitli toplayıcı-çıkarıcı

aşağıdaki işlemleri gerçekleştirin ve çıkış toplamı ile çıkış

taşıma değerlerini kaydedin C_4 :

$$\begin{array}{rcl} 9 + 5 & 9 - 5 \\ 9 + 9 & 9 - 9 \\ 9 + 15 & 9 - 15 \end{array}$$

Toplama sırasında, çıkış taşımasının 15'i aştığında 1'e eşit olduğunu gösterin. $A \geq B$ olduğunda, çıkarma işleminin doğru sonucu verdiğini gösterin, $A - B$ ve çıkış taşıması C_4 1'e eşittir, ancak $A < B$ olduğunda, çıkarma işlemi $B - A$ 'nın 2'nin tamamlayıcısını verir ve çıkış taşıması 0'a eşittir.

Büyüklik Karşılaştırıcı

İki sayının karşılaştırılması, bir sayının diğerinden büyük, eşit veya küçük olup olmadığını belirleyen bir işlemdir. İki sayı, A ve B , önce $A - B$ çıkarılarak karşılaştırılabilir; bu, Şekil 9.11'de gösterildiği gibidir. Eğer S çıktısı sıfıra eşitse, o zaman $A = B$. C_4 'ten gelen çıkış taşı, sayıların göreceli büyüklüklerini belirler: $C_4 = 1$ olduğunda, $A \geq B$; $C_4 = 0$ olduğunda, $A < B$; ve $C_4 = 1$ ve $S \neq 0$, $A > B$.

Şekil 9.11'deki çıkarıcı devresine karşılaştırma mantığını sağlamak için ekleme yapmak gereklidir. Bu, beş girişi olan bir kombinasyon devresi ile yapılır— S üzerinden S_4 ve C_4 —ve üç çıktı, x , y , ve z ile belirtilmiştir, böylece

$$\begin{array}{lll} x = 1 & \text{if } A = B & (S = 0000) \\ y = 1 & \text{if } A < B & (C_4 = 0) \\ z = 1 & \text{if } A > B & (C_4 = 1 \text{ and } S \neq 0000) \end{array}$$

7404 ve 7408 entegreleri ile kombinasyonel devre uygulanabilir.

Karşılaştırıcı devresini oluşturun ve çalışmasını test edin. Her bir çıkışı kontrol etmek için A ve B için en az iki sayı seti kullanın: x , y ve z .

9.9 DENEY 8: FLIP-FLOPLAR

Bu deneyde, çeşitli

latch'ler ve flip-flop'ların yapısını, testini ve çalışma prensibini inceleyeceksiniz. Latch'lerin ve flip-flop'ların iç yapısı Bölüm 5.3 ve 5.4'te bulunabilir.

SR Kilit

İki çapraz bağlı NAND kapısından oluşan bir SR kilidi inşa edin. İki girişi anahtarlara ve iki çıkışı gösterge lambalarına bağlayın. İki anahtarı mantık 1'e ayarlayın, ardından her bir anahtarı ayrı ayrı mantık 0 konumuna ve tekrar 1'e döndürün. Devrenin fonksiyon tablosunu elde edin.

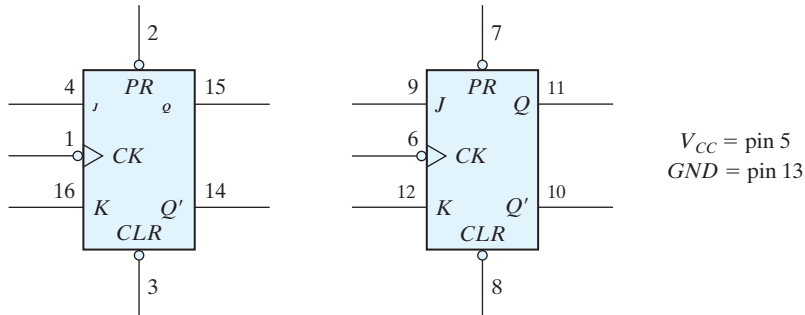
D Kilit

D kilidi dört NAND kapısı (sadece bir 7400 IC) ile inşa edin ve fonksiyon tablosunu doğrulayın.

Master-Slave Flip-Flop

İki D kilidi ve bir inverter kullanarak bir master-slave D flip-flop bağlayın. D girişi bir anahtara ve saat girişini bir pulsere bağlayın. Master kilidinin çıkışı nı bir gösterge lambasına ve slave kilidinin çıkışını başka bir gösterge lambasına bağlayın. Giriş değerini çıkışın tamamlayıcı değeri olarak ayarlayın. Pulsere basın ve ardından tek bir darbe üretmek için bırakın. Darbenin pozitif olduğunda master'ın değiştiğini ve darbenin negatif olduğunda slave'in değişimi takip ettiğini gözlemleyin. İki gösterge lambasını gözlemlerken butona birkaç kez tekrar basın. Girişten master'a ve master'dan slave'e transfer sırasını açıklayın.

Saati pulserden ayırın ve bir saat jeneratörüne bağlayın. Flip-flop'un tamamı ayıcı çıkışını D girişine bağlayın. Bu, flip-flop'un her saat darbesiyle tamamlanmasını sağlar. İkili iz osiloskopu kullanarak, saat ve ana ile köle çıkışlarının dalga formlarını gözlemleyin. Ana ve köle çıkışları arasındaki gecikmenin saat dönüsünün pozitif yarısına eşit olduğunu doğrulayın. Saat dalga formu ile ana ve köle çıkışları arasındaki ilişkiyi gösteren bir zaman diyagramı elde edin.



Fonksiyon tablosu

Girdiler					Çıkışlar	
Preset	Clear	Saat	J	K	Q	Q'
0	1	XXX	XXX	XXX	1	0
0	0	XXX	XXX	XXX	1	1
1	1		0011	0	Değişiklik yok	
1	1			1		
1	1			0	0	1
1	1		1	1	Değiştir	

ŞEKİL 9.12

IC türü 7476 çift JK master-slave flip-flop'lar

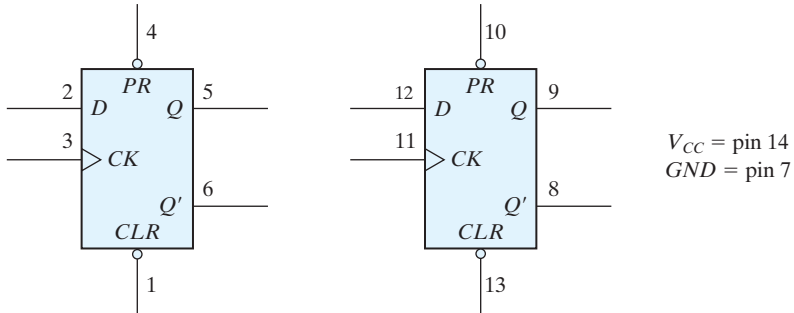
Kenar Tetiklemeli Flip-Flop

Altı NAND kapısı kullanarak bir D-tipi pozitif kenar tetiklemeli flip-flop oluşturun. Saat girişini bir pulsere, D girişini bir anahtara ve çıkışı Q'yu bir gösterge lambasına bağlayın. D değerini Q'nun tamamlayıcısı olarak ayarlayın. Flip-flop çıkışının yalnızca saat darbesinin pozitif geçişine yanıt olarak değiştiğini gösterin. Saat girişi mantıksal 1 olduğunda, saat negatif geçişten geçtiğinde veya saat girişi mantıksal 0 olduğunda çıkışın değişmediğini doğrulayın. D girişini her zaman Q çıkışının tamamlayıcısına karşılık gelecek şekilde değiştirmeye devam edin.

Girişin pulsarından ayırın ve saat jeneratörüne bağlayın. Tamamlayıcı çıkışı Q D girişine bağlayın. Bu, çıkışın saat darbesinin her pozitif geçişi ile tamamlanmasını sağlar. İki kanallı bir osiloskop kullanarak, giriş saati ile çıkış Q arasındaki zamanlama ilişkisini gözlemleyin ve kaydedin. Çıkışın pozitif kenar geçişine yanıt olarak değiştiğini gösterin.

IC Flip-Flops

IC tipi 7476, ön ayar ve temizleme ile iki JK ana-köle flip-flop'tan oluşur. Her flip-flop için pin ataması Şekil 9.12'de gösterilmiştir. Fonksiyon tablosu devrenin çalışmasını belirtir. Tablo üzerindeki ilk üç giriş, asenkron işlemi belirtir.



Fonksiyon tablosu

Girdiler				Çıkışlar	
Preset	Clear	Saat	D	Q	Q'
0	1	X	X	1	0
1	0	X	X	0	1
0	0	X	X	1	1
1	1	↑	0	0	1
1	1	↑	1	1	0
1	1	0	X	Değişiklik yok	

ŞEKİL 9.13

IC tipi 7474 çift D pozitif kenar tetiklemeli flip-flop'lar

ön ayar ve temizleme girişleri. Bu girişler, bir NAND SR kilidi gibi davranır ve saat ten bağımsızdır veya J ve K girişleri. (X'ler, önemsiz durumları gösterir.) Fonksiyon tablosundaki son dört girdi, hem ön ayar hem de temizleme girişleri mantık 1 'de tutulduğunda saatin çalışmasını belirtir. Saat değeri tek bir darbe olarak gösterilir. Darbenin pozitif geçişi ana flip-flop'u değiştirir ve negatif geçiş, köle flip-flop'u ve devrenin çıkışını da değiştirir. $J=K=0$ olduğunda, çıkış değişmez. Flip-flop, $J=K=1$ olduğunda geçiş yapar veya tamamlanır. Bir 7476 flip-flop'un çalışmasını araştırın ve fonksiyon tablosunu doğrulayın.

IC tipi 7474, ön ayar ve temizleme ile birlikte iki adet D pozitif kenar tetiklemeli flip-flop'tan oluşur. Pin ataması Şekil 9.13'te gösterilmiştir. Fonksiyon tablosu, ön ayar ve temizleme işlemlerini ve saatin çalışmasını belirtir. Saat, pozitif kenar tetiklemeli flip-flop olduğunu belirtmek için yukarı ok ile gösterilmiştir. Flip-flop'lardan birinin çalışmasını inceleyin ve fonksiyon tablosunu doğrulayın.

9.10 DENEY 9: SIRALI DEVRELER

Bu deneyde, üç senkron sıralı devre tasarlayacak, inşa edecek ve test edeceksiniz. IC tipi 7476 (Şekil 9.12) veya 7474 (Şekil 9.13) kullanın. Toplam IC sayısını en aza indirecek herhangi bir kapı türünü seçin. Senkron sıralı devrelerin tasarımı Bölüm 5.7'de ele alınmaktadır.

Enable ile Yukarı-Aşağı Sayıcı

Yukarı veya aşağı sayan iki bitlik bir sayıcı tasarlayın, inşa edin ve test edin. Enable girişi E, sayıcının açık mı kapalı mı olduğunu belirler. Eğer $E = 0$ ise, sayıcı devre dışı kalır ve flip-flop'lara saat darbeleri uygulanmasına rağmen mevcut sayımda kalır. Eğer $E = 1$ ise, sayıcı etkinleştirilir ve ikinci bir giriş, x, sayım yönünü belirler. Eğer $x = 1$ ise, devre 00, 01, 10, 11 sırasıyla yukarı doğru sayar ve sayım tekrar eder. Eğer $x = 0$ ise, devre 11, 10, 01, 00 sırasıyla aşağı doğru sayar ve sayım tekrar eder. Saati devre dışı bırakmak için E'yi kullanmayın. Sıralı devreyi E ve x girişleri ile tasarlayın.

Durum Diyagramı

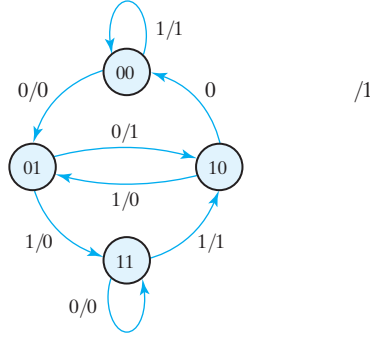
Durum diyagramı Şekil 9.14'te gösterilen bir ardışık devre tasarlayın, inşa edin ve test edin.

İki flip-flop'u A ve B olarak, girişi x ve çıkışı y olarak belirleyin.

En az anlamlı flip-flop'un B çıkışını x girişine bağlayın ve saat darbelerinin uygulanmasıyla meydana gelecek durum ve çıkış dizisini tahmin edin. Devreyi test ederek durum geçişini ve çıkışı doğrulayın.

Sayıcı Tasarımı

Aşağıdaki ikili durum dizisinden geçen bir sayıcı tasarlayın, inşa edin ve test edin: 0, 1, 2, 3, 6, 7, 10, 11, 12, 13, 14, 15 ve tekrar 0'a dönerek tekrarlayın. İkili durumlar 4, 5, 8 ve 9'un kullanılmadığını unutmayın. Sayıcının kendiliğinden başlaması gerekir; yani, devre dört geçersiz durumdan herhangi birinden başlarsa, sayım darbeleri devreyi geçerli durumlardan birine aktarmalıdır, böylece sayım doğru bir şekilde devam etmelidir.



ŞEKİL 9.14
Deney 9 için durum diyagramı

Gerekli sayım dizisi için devrenin çalışmasını kontrol edin. Sayıcının kendiliğinden başladığını doğrulayın. Bu, devreyi her kullanılmayan duruma ön ayar ve temizleme girişleri aracılığıyla başlatarak ve ardından sayıcının geçerli durumlardan birine ulaşmış ulaşmadığını görmek için darbeler uygulayarak yapılır.

9.11 DENEY 10: SAYICILAR

Bu deneyde, çeşitli dalgali ve senkron sayıcı devreleri inşa edecek ve test edeceksiniz. Dalgali sayıcılar Bölüm 6.3'te, senkron sayıcılar ise Bölüm 6.4'te ele alınmaktadır.

Dalgali Sayıcı

İki 7476 entegresi kullanarak dört bitlik bir ikili dalgali sayıcı inşa edin (Şekil 9.12). Tüm senkron temizleme ve ön ayar girişlerini mantık 1'e bağlayın. Sayım darbe girişini bir darbe üretene bağlayın ve sayıcının doğru çalıştığını kontrol edin.

Sayıcıyı yukarı yerine aşağı sayacak şekilde değiştirin. Her giriş darbesinin sayıcıyı 1 azaltıp azaltmadığını kontrol edin.

Senkron Sayıcı

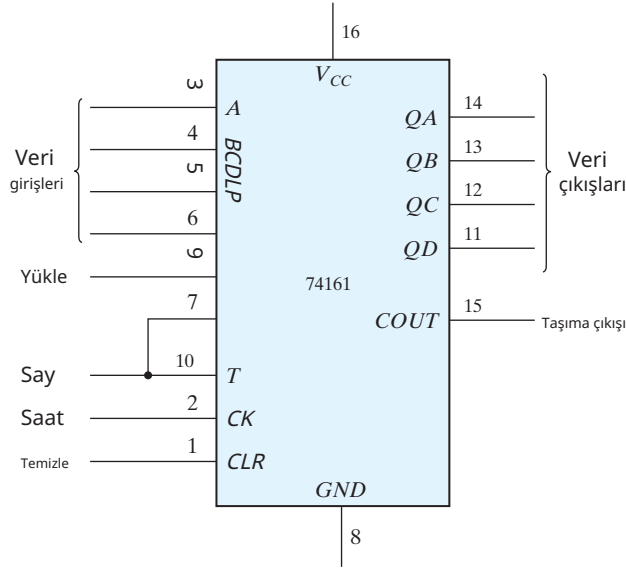
Senkron dört bitlik bir ikili sayıcı inşa edin ve çalışmasını kontrol edin. İki 7476 entegresi ve bir 7408 entegresi kullanın.

Onluk Sayıcı

0000'dan 1001'e kadar sayan bir senkron BCD sayıcı tasarlayın. İki 7476 entegresi ve bir 7408 entegresi kullanın. Sayıcının doğru sıralama için test edin. Sayıcının kendiliğinden başlatılıp başlatılmadığını belirleyin. Bu, sayıcıyı ön ayar ve temizleme girişleri aracılığıyla kullanılmayan altı duruma başlatarak yapılır. Darbelerin uygulanması, sayıcı kendiliğinden başlatılıyorsa sayıcıyı geçerli durumlardan birine aktaracaktır.

Paralel Yüklemeli İkili Sayıcı

74161 tipi IC, paralel yükleme ve asenkron sıfırlama ile dört bitli senkron ikili sayıcıdır. İç mantığı, Şekil 6.14'te gösterilen devreye benzer. Giriş ve çıkışlara pin atamaları Şekil 9.15'te gösterilmiştir. Yük sinyali etkinleştirildiğinde, dört veri girişi dört iç flip-flop'a, QA ile QD arasında aktarılır; QD en anlamlı bittir. P ve T adında iki sayım etkinleştirme girişi vardır. Sayıcının çalışması için her ikisinin de 1 olması gerekir. Fonksiyon tablosu, Tablo 6.6'ya benzer, bir istisna ile: 74161'de yük girişi 0 olduğunda etkinleştirilir. Giriş verilerini yüklemek için, sıfırlama girişi 1'e ve yük girişi 0'a eşit olmalıdır. İki sayım girişi, önem semeyen koşullara sahiptir ve 1 veya 0'a eşit olabilir. İç flip-flop'lar, saat darbesinin pozitif geçişinde tetiklenir. Devre, yük girişi 1'e eşit olduğunda ve her iki sayım girişi P ve T 1'e eşit olduğunda bir sayıcı olarak çalışır. Eğer P veya T 0'a gide



Fonksiyon tablosu

Clear	Saat	Yükle	Say	Fonksiyon
0	X	X	X	Çıktıları 0'a temizle
1	↑	0	1	Giriş verilerini yükle
1	↑	1	1	Sonraki ikili değere say
1	↑	1	0	Çıktıda değişiklik yok

ŞEKİL 9.15

Paralel yüklemeli IC türü 74161 ikili sayıcı

mez. Taşıma çıkışı, dört veri çıkışının tamamı 1 olduğunda 1'e eşittir. 74161 IC'nin işlev tablosuna göre çalışmasını doğrulamak için bir deney yapın.

74161 IC'nin, iki girişli NAND kapısıyla birlikte, 0000'dan 1001'e kadar sayan senkron BCD sayıcı olarak nasıl çalıştırılabileceğini gösterin. Temizleme girişini kullanmayın. 1001 sayısını tespit etmek için NAND kapısını kullanın, bu da sayıcıya tüm 0'ların yüklenmesine neden olur.

9.12 DENEY 11: KAYDIRMA KAYITLARI

Bu deneyde, kaydırma kayıtlarının çalışmasını araştıracaksınız. Kullanılacak entegre devre, paralel yüklemeli 74195 kaydırma kayıtdır. Kaydırma kayıtları Bölüm 6.2'de açıklanmaktadır.

IC Kaydırıcı Kayıt

IC tipi 74195, paralel yükleme ve asenkron temizleme özelliklerine sahip dört bitlik bir kaydırma kayıt cihazıdır.

Giriş ve çıkışlara yapılan pin atamaları Şekil 9.16'da gösterilmiştir. Tek kontrol hattı $SH > LD$ (kaydır/yükle) kaydın senkronize çalışmasını belirler.

$SH/LD = 0$, kontrol girişi yükleme modundadır ve dört veri girişi dört iç flip-flop'a aktarılır, QA üzerinden QD . $SH > LD = 1$ olduğunda, kontrol girişi kaydırma modundadır ve kayıttaki bilgi QA üzerinden QD 'ye doğru sağa kaydırılır. Kaydırma sırasında QA 'ya seri girişi, J ve K 'dan belirlenir.

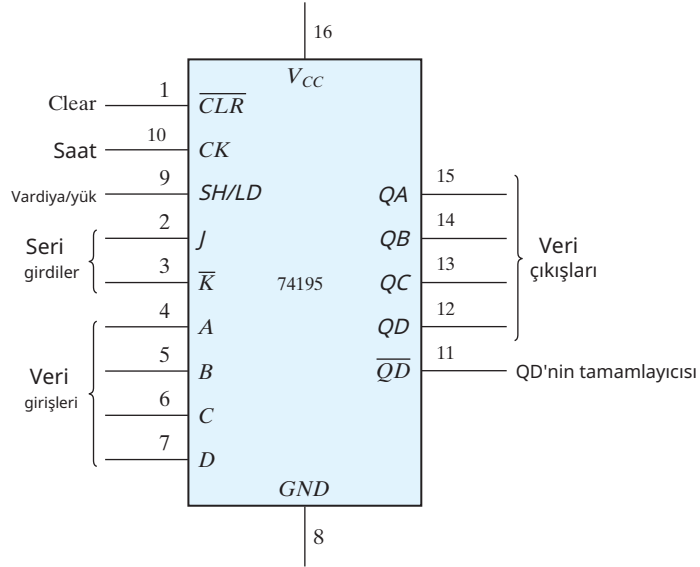
inputs. İki girişi, J ve K 'nin tamamlayıcısı gibi davranır JK flip-flop. Her iki J ve K 0'a eşit olduğunda, flip-flop QA kaydırmadan sonra 0'a temizlenir. Her iki girişi 1'e eşit olduğunda, QA kaydırmadan sonra 1'e ayarlanır. J ve K girişleri için diğer iki koşul, flip-flop QA 'nın kaydırmadan sonraki çıkışında bir tamamlayıcı veya değişiklik sağlamaz.

74195 için fonksiyon tablosu, kaydın çalışma modunu gösterir. Temizleme girişi 0'a gittiğinde, dört flip-flop asenkron olarak 0'a temizlenir; yani, bir saat sinyaline ihtiyaç duymadan. Senkron işlemler, saat sinyalinin pozitif geçişinden etkilenir. Giriş verilerini yüklemek için, SH/LD 0'a eşit olmalı ve pozitif bir saat darbesi geçişi gerçekleşmelidir. Sağa kaydırmak için, SH/LD 1'e eşit olmalıdır. J ve K girişleri, seri girişi oluşturmak için birleştirilmelidir.

74195 entegresinin çalışmasını doğrulayan bir deney gerçekleştirin. İşlev tablosunda listelenen tüm işlemleri yerine getirdiğini gösterin. İşlev tablonuzda $JK = 01$ ve 10 için iki koşulu da dahil edin.

Halka Sayıcı

Halka sayıcı, seri çıkıştan gelen sinyalin QD 'nin seri girişine girdiği dairesel bir kaydırma kayıt cihazıdır. Seri girişi oluşturmak için J ve K girişlerini birleştirin. Halka sayıcıyı 1000 başlangıç değerine ayarlamak için yükleme koşulunu kullanın. Tek bitin kaydırma koşuluyla döndürülmesini sağlayın ve her saat darbesinden sonra kaydın durumunu kontrol edin.



Fonksiyon tablosu

Clear	Kayırdırma/ yükleme	Saat	J	$\overline{K}$	Seri girdi	Fonksiyon
0	XXXXX					Asenkron temizleme
1	X	0	$\otimes$	XX		Çıktıda değişiklik yok
1	0	$\uparrow$		x	x	Yükleme girdi verileri
1	1	$\uparrow$	0	0	0	QA'dan QD'ye, QA'ya doğru kaydırma = 0
1	1	$\uparrow$	1	1	1	QA'dan QD'ye, QA'ya doğru kaydırma = 1

ŞEKİL 9.16

Paralel yüklemeli IC türü 74195 kaydırma kaydedici

Bir anahtar kuyruk halka sayıcı, seri giriş için QD'nin tamamlayıcı çıkışını kullanır. Anahtar kuyruk halka sayıcıyı 0000 olarak ön ayarlayın ve kaydırmadan kaynaklanacak durum dizisini tahmin edin. Her kaydırmadan sonra durum dizisini gözlemleyerek tahmininizi doğrulayın.

Geri Besleme Kaydırma Kaydı

Geri besleme kaydırma kaydı, seri girişi seçilen kayıt çıkışlarının bazı fonksiyonlarına bağlı olan bir kaydırma kaydıdır. Seri girişi olan bir geri besleme kaydırma kaydını bağlayın.

rın özel-VE'si QC ve QD. Kaydedicinin durumlarının dizisini tahmin edin, 1000 durumundan başlayarak. Her saat darbesinden sonra durum dizisini gözlemleyerek tahmininizi doğrulayın.

İki Yönlü Kaydırma Kaydı

74195 entegresi yalnızca QA yönünde QD tarafına kaydırabilir. Kaydı, yükleme modunu kullanarak sola kaydırma işlemi (QD yönünde QA) elde etmek için iki yönlü bir kaydırma kaydına dönüştürmek mümkündür. Bu, her flip-flop'un çıkışını solundaki flip-flop'un girişine bağlayarak ve

SH/LD girişinin yükleme modunu sola kaydırma kontrolü olarak kullanarak gerçekleştirilir. Giriş D sola kaydırma işlemi için seri giriş haline gelir.

74195'i iki yönlü bir kaydırma kaydı olarak bağlayın (paralel yük olmadan). Sağ kaydırma için seri girişi bir anahtara bağlayın. Sola kaydırmayı, seri çıkış QA ile seri giriş D bağlayarak bir halka sayacı olarak oluşturun. Kaydı temizleyin ve ardından tek bir 1'i seri giriş anahtarından kaydırarak işlemi kontrol edin. Üç kez daha sağa kaydırın ve seri giriş anahtarından 0'lar ekleyin. Ardından, sola kaydırma (yükleme) kontrolü ile sola döndürün. Tek 1 görünür kalmalıdırken kaydırma işlemi yapılmalıdır.

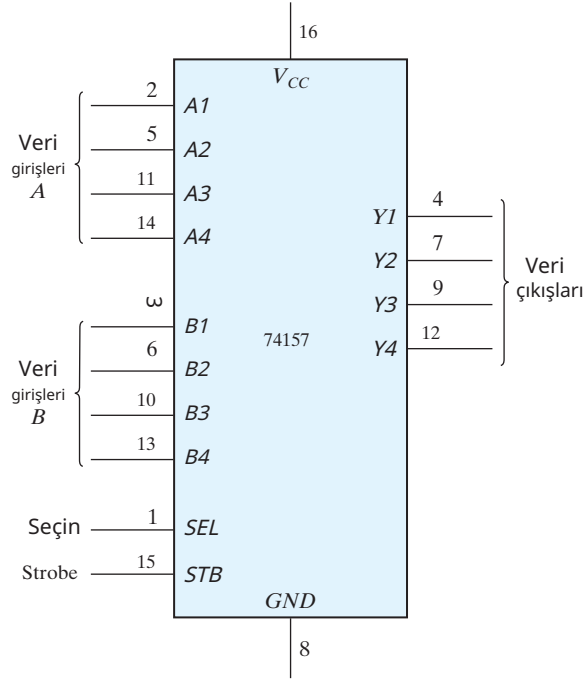
Paralel Yüklemeli İki Yönlü Kaydırma Kaydı

74195 entegresi, bir çoklayıcı devresi ile birlikte kullanılarak paralel yüklemeli iki yönlü bir kaydırma kaydına dönüştürülebilir. Bu amaçla 74157 entegresini kullanacağız. 74157, iç mantığı Şekil 4.26'da gösterilen dört adet ikiye bir çoklayıcıdır. 74157'nin giriş ve çıkış pin atamaları Şekil 9.17'de gösterilmiştir. Enable girişi 74157'de bir strobe olarak adlandırıldığını unutmayın.

74195 kaydını ve 74157 çoklayıcısını kullanarak paralel yüklemeli bir iki yönlü kaydırma kaydı oluşturun. Devre aşağıdaki işlemleri gerçekleştirebilmelidir:

- Asenkron temizleme
- Sağa kaydırma
- Sola kaydırma
- Paralel yükleme
- Senkron temizleme

Temizleme, saat ve SH/LD girişlerinin ve 74157'nin strobe ve seçim girişlerinin bir fonksiyonu olarak beş işlem için bir tablo çıkarın. Devreyi bağlayın ve fonksiyon tablonuzu doğrulayın. Kayıt için başlangıç değeri sağlamak amacıyla paralel yükleme koşulunu kullanın ve ikili bilgiyi kaybetmemek için her iki kaydırmanın seri çıkışlarını seri girişlerine bağlayın.



Fonksiyon tablosu

Strobe	Seç	Veri çıkışları Y
1	X	Hepsi 0
0	0	Veri girişlerini seçin A
0	1	Veri girişlerini seçin B

ŞEKİL 9.17

IC türü 74157 dörtlü 2x1 çoklayıcılar

9.13 DENEY 12: SERİ EKLEME

Bu deneyde, bir seri toplayıcı-çıkarıcı devresi inşa edecek ve test edeceksiniz. İki ikili sayının seri eklenmesi, Bölüm 6.2'de açıklandığı gibi, kaydırma kayıtları ve tam toplayıcı kullanılarak yapılabilir.

Seri Toplayıcı

Şekil 6.6'daki diyagramdan başlayarak, aşağıdaki entegre devreleri kullanarak dört bitlik bir seri toplayıcı tasarlayın ve inşa edin: 74195 (iki adet), 7408, 7486 ve 7476. Kayıt B için dört anahardan paralel veri kabul etme imkanı sağlayın ve seri girişini toprağa bağlayarak ekleme sırasında 0'ların kayıt B içine kaydırılmasını sağlayın. Temizlemek için bir anahtar sağlayın

kayıtlar ve flip-flop. Kayıt

B paralel verileri kabul edip etmeyeceğini veya toplama sırasında kaydırılıp kaydırılmayacağını belirtmek için başka bir anahtara ihtiyaç olacaktır.

Toplayıcıyı Test Etme

Seri toplayıcınızı test etmek için, ikili toplama işlemini gerçekleştirin $5+6+15=26$. Bu, öncelikle kayıtları ve taşıma flip-flop'unu temizleyerek yapılır. İkili değeri 0101'i

kayıt *B*'ye paralel olarak yükleyin. *B*'yi *A*'ya seri olarak eklemek için dört darbe uygulayın ve *A*'daki sonucun 0101 olduğunu kontrol edin. (7476 için saat darbelerinin Şekil 9.12'de gösterildiği gibi olması gerektiğini unutmayın.) Paralel olarak 0110'u *B*'ye yükleyin ve bunu *A*'ya seri olarak ekleyin. *A*'nın doğru toplamı aldığını kontrol edin. Paralel olarak 1111'i yükleyin *B* ve ekle *A*. *A* içindeki değerin 1010 olduğunu ve taşıma flip-flop'unun ayarlandığını kontrol et.

Kayıtları ve flip-flop'u temizleyin ve seri toplama cihazınızın düzgün çalıştığını doğrulamak için birkaç farklı sayı deneyin.

Seri Toplayıcı-Çıkarıcı

Eğer 6.2. bölümdeki seri çıkarıcı tasarımı için kullanılan prosedürü takip edersek (bu, $A - B$ çıkarır), çıkış farkının çıkış toplamıyla aynı olduğunu göreceğiz, ancak borç flip-flop'unun *J* ve *K* girişinin *QD* (74195'te mevcut) tamamlayıcısına ihtiyaç duyduğunu göreceğiz. 7486'daki diğer iki XOR kapısını kullanarak, seri toplayıcıyı mod kontrolü *M* olan bir seri toplayıcı-çıkarıcıya dönüştürün. $M=0$ olduğunda, devre $A+B$ toplar. $M=1$, devre $A-B$ çıkarır ve flip-flop borcu taşır, taşıma yerine.

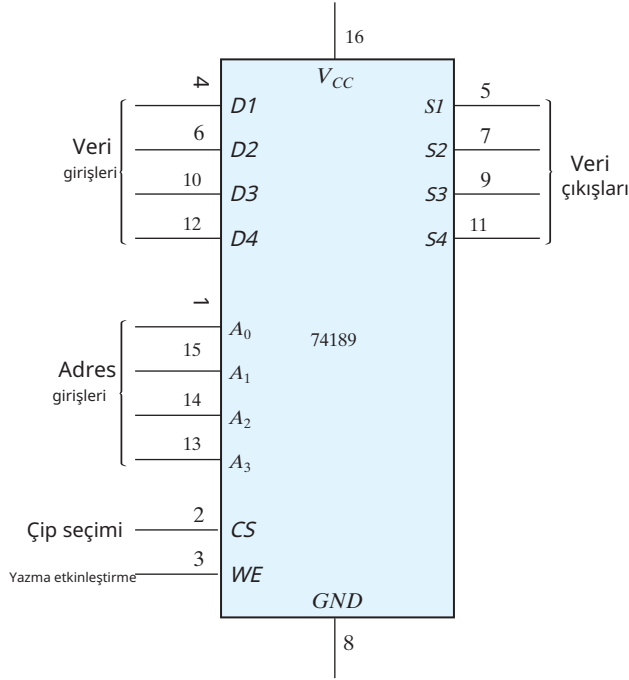
Devreye ait toplama kısmını, değişikliğin işlemi değiştirmedikten emin olmak için önerilen işlemleri tekrarlayarak test edin. Seri çıkarıcı kısmını, çıkarma işlemi ni gerçekleştirerek test edin: $15 - 4 - 5 - 13 = -7$. İkili 15, önce 0'a temizlenerek ve *B*'den 15 eklenerek *A* kaydına aktarılabilir. Çıkarma işlemi sırasında ara sonuçları kontrol edin. Not edin ki, -7 , flip-flop'ta 1 borç ile 7'nin 2'nin tamamlayıcı olarak *g* örnecektir.

9.14 DENEY 13: BELLEK ÜNİTESİ

Bu deneyde, rastgele erişim belleği (RAM) biriminin davranışını ve depolama kapasitesini araştıracağız. RAM, yalnızca okunabilir bellek (ROM) simüle etmek için kullanılacaktır. ROM simülatörü daha sonra, Bölüm 7.5'te açıklandığı gibi, kombinyasyonel devreleri uygulamak için kullanılacaktır. Bellek ünitesi, Bölüm 7.2 ve 7.3'te tartışılmaktadır.

IC RAM

IC tipi 74189, 16×4 RAM'dır. İç mantık, Şekil 7.6'da gösterilen 4×4 RAM devresine benzerdir. Giriş ve çıkışlara pin atamaları Şekil 9.18'de gösterilmiştir. Dört adres girişi, bellekteki 16 kelimeden 1'ini seçer. Adresin en az anlamlı biti *A* ve en anlamlı biti *A*₃'dür. Çip seçme (*CS*) girişi, belleği etkinleştirmek için 0'a eşit olmalıdır. Eğer *CS* 1'e eşitse, bellek devre dışıdır ve dört çıkış da yüksek empedans durumundadır. Yazma etkinleştirme (*WE*) girişi, işlev tablosunda belirtildiği gibi, işlem türünü belirler. Yazma işlemi, $WE=0$ olduğunda gerçekleştirilir. Bu



Fonksiyon tablosu

CS	WE	İşlem	Veri çıktıları
001	01X	Yaz Oku Devre dışı bırak	Yüksek empedans Seçilen kelimenin tamamlayıcısı Yüksek empedans

ŞEKİL 9.18

IC türü 74189 16×4 RAM

işlem, ikili sayının veri girişlerinden seçilen kelimeye aktarılmasıdır. Okuma işlemi, $WE=1$ olduğunda gerçekleştirilir. Bu işlem, seçilen kelimede saklanan tamamlayıcı değeri çıkış veri hatlarına aktarır. Bellek, bellek genişlemesini kolaylaştırmak için üç durumlu çıkışlara sahiptir.

RAM'in Test Edilmesi

74189'un çıktıları tamamlayıcı değerler ürettiğinden, çıktıları normal değerlerine değiştirmek için dört invertör eklememiz gerekiyor. Aşağıdaki bağlantıları yaptıktan sonra RAM test edilebilir: Adres girişlerini 7493 IC'sini kullanarak bir ikili sayıcıya bağlayın (Şekil 9.3'te gösterilmiştir). Dört veri girişini anahtarlarla bağlayın ve

inverterine veri çıkışları. Adres için dört gösterge lambası ve inverterlerin çıkışları için dört tane daha sağlayın. Girişi CS toprağa bağlayın ve WE bir anahtar (veya negatif darbe sağlayan bir puls) ile bağlayın. Belleğe birkaç kelime depolayın ve ardından yazma ve okuma işlemlerinin düzgün çalıştığını doğrulamak için bunları okuyun. WE anahtarını kullanırken dikkatli olmalısınız. Belleğe yazmak istemediğiniz sürece her zaman WE girişini okuma modunda bırakın. Yazmanın doğru yolu, önce sayacı ve dört anahtardaki girişleri ayarlamaktır. Ardından, kelimeyi belleğe depolayın, WE anahtarını yazma konumuna çevirin ve okuma konumuna geri döndürün. WE yazma modundayken adresi veya girişleri değiştirmemeye dikkat edin.

ROM Simülatörü

ROM simülatörü, yalnızca okuma modunda çalışan bir RAM'den elde edilir. 1'ler ve 0'ların deseni, simüle edilen RAM'e birim geçici olarak yazma moduna alınarak girilir. Simülasyon, birimi okuma moduna alarak ve adres hatlarını ROM'a giriş olarak gerçekleştirilir. ROM, ardından herhangi bir kombinasyonel devreyi uygulamak için kullanılabilir.

Tablo 1.6'da tanımlandığı gibi, dört bitlik bir ikili sayıyı eşdeğer Gray koduna dönüştüren ROM simülatörünü kullanarak bir kombinasyonel devre uygulayın. Bu, aşağıdaki gibi yapılır: Kod dönüştürücünün doğruluk tablosunu elde edin. Doğruluk tablosunu, adres girişlerini ikili değere ve veri girişlerini karşılık gelen Gray kodu değerine ayarlayarak 74189 belleğine depolayın. Tabloya ait 16 giriş bellek yazıldıktan sonra, ROM simülatörü, WE hattını kalıcı olarak mantık 1'e bağlayarak ayarlanır. Kod dönüştürücüyü, adres hatlarına girişleri uygulayarak ve veri çıkış hatlarında doğru çıktıları doğrulayarak kontrol edin.

Bellek Genişletme

Bellek birimini iki 74189 entegresi kullanarak 32×4 RAM'e genişletin. İki entegre arasında seçim yapmak için CS girişlerini kullanın. Veri çıkışlarının üç durumlu olduğunu unutmayın, bu nedenle iki entegre arasında mantık VE işlemi elde etmek için terminal çiftlerini birleştirebilirsiniz. Devrenizi, üç bitlik bir sayıyı iki bitlik bir sayıya ekleyerek dört bitlik bir toplam üreten bir ROM simülatörü olarak kullanarak test edin. Örneğin, ROM'un girişi 10110 ise, çıkışın hesaplanması $101 + 10 = 0111$ 'dir. (Girişin ilk üç biti 5'i, son iki biti 2'yi temsil eder ve çıkış toplamı ikili 7'dir.) Dört bitlik adres sağlamak için sayıcıyı ve adresin beşinci biti için bir anahtar kullanın.

9.15 DENEY 14: LAMBA HENTBOLU

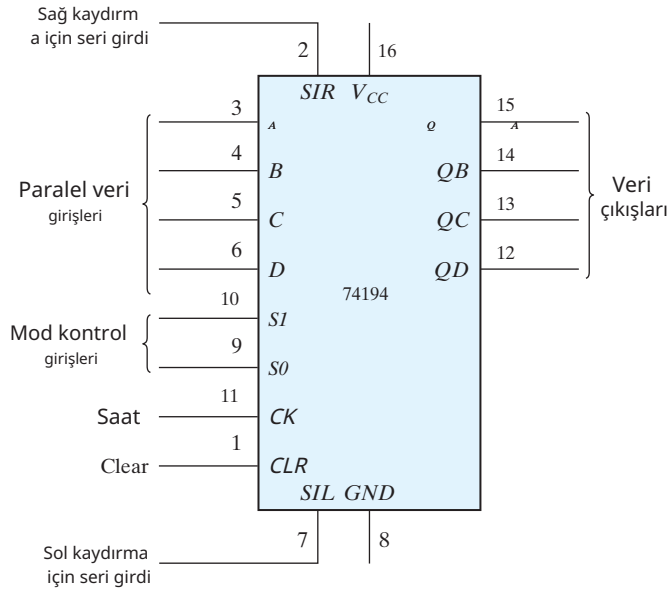
Bu deneyde, hareket eden topu simüle etmek için tek bir ışık kullanarak bir elektronik hentbol oyunu inşa edeceksiniz. Deney, paralel yükleme ile iki yönlü bir kaydırma kaydının uygulanmasını göstermektedir. Ayrıca, flip-flop'ların asenkron girişlerinin çalışmasını da gösterir. Öncelikle deney için gerekli olan bir entegre devreyi tanıtacağız ve ardından simüle edilmiş lamba hentbol oyununun mantık diyagramını sunacağız.

IC Tipi 74194

Bu, paralel yükleme ile dört bitli iki yönlü bir kaydırma kaydırıcıdır. İç mantık, Şekil 6.7'de gösterilenle benzerdir. Giriş ve çıkışlara pin atamaları Şekil 9.19'da gösterilmiştir. İki mod kontrol girişi, işlev tablosunda belirtilen işlem türünü belirler.

Mantık Diyagramı

Elektronik lambda hentbol oyununun mantık diyagramı Şekil 9.20'de gösterilmiştir. İki 74194 entegre devresi, bir çift D flip-flop 7474 entegre devresi ve üç kapı entegre devresi: 7400, 7404 ve 7408'den oluşmaktadır. Top, soldan sağa kaydırılan hareket eden bir ışıkla simüle edilmektedir.

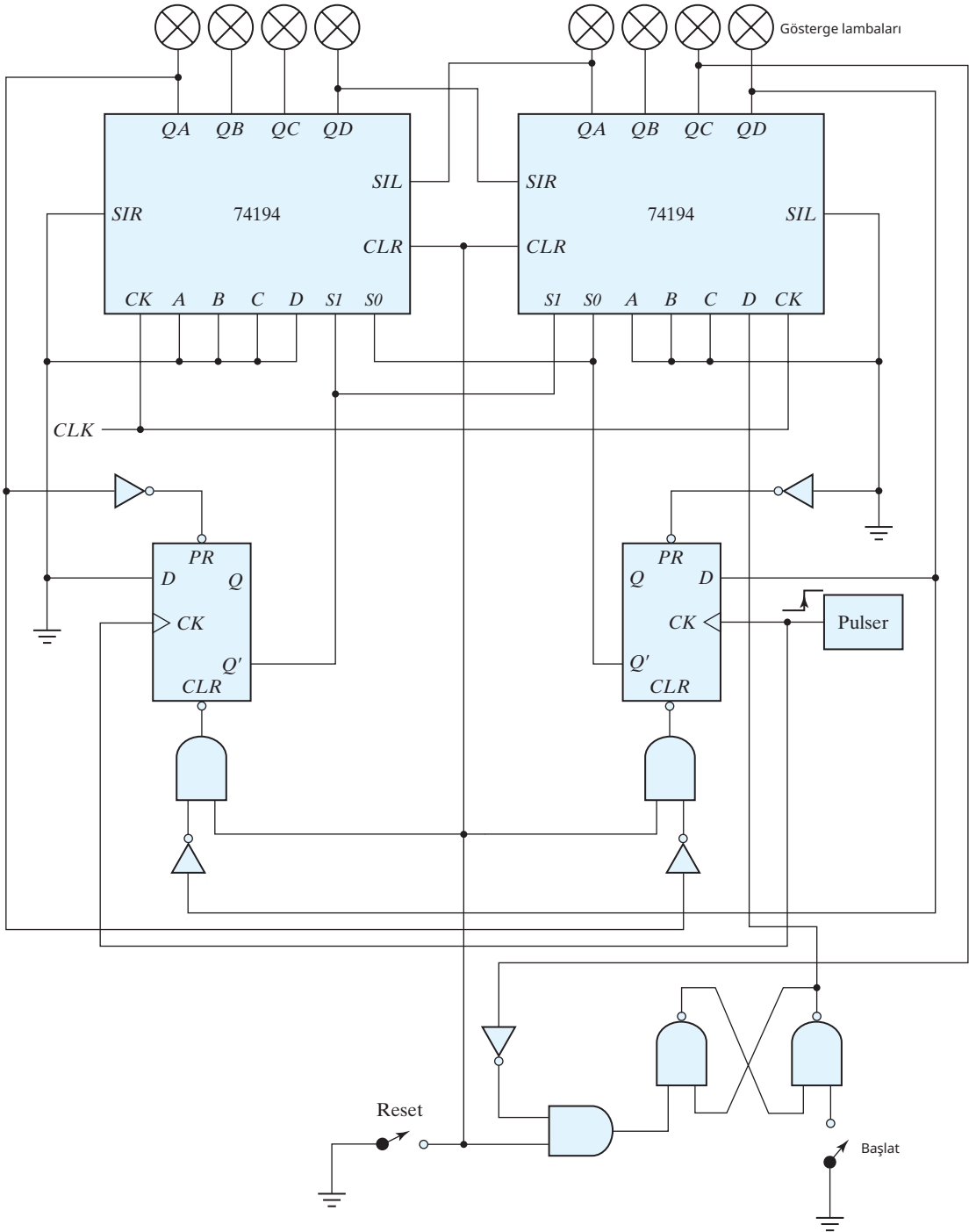


Fonksiyon tablosu

Temizle	Saat	Mod		Fonksiyon
		S1	S0	
0	x	x	x	Çıktıları 0'a temizle
1	↑	0	0	Çıktıda değişiklik yok
1	↑	0	1	Sağa kaydırma yönü QA ile QD.SIR ile QA
1	↑	1	0	Sola kaydırma yönü QD ile QA.SIL ile QD
1	↑	1	1	Paralel yükleme giriş verileri

ŞEKİL 9.19

IC türü 74194 iki yönlü kaydırma kaydedici paralel yükleme ile



ŞEKİL 9.20
Lamba el topu mantık diyagramı

iki yönlü kaydırma kaydı. Işığın hareket ettiği hız, saatin frekansı tarafından belirlenir. Devre önce sıfırlama anahtarı ile başlatılır. Başlatma anahtarı, topu (bir g österge lambası) en sağa yerleştirerek oyunu başlatır. Oyuncu, topun sola hareket etmeye başlaması için puls butonuna basmalıdır. Tek ışık, en soldaki konuma (duvar) ulaşana kadar sola kayar; bu noktada top, hareket eden ışığın kaydırma yönünü tersine çevirerek oyuncuya geri döner. Işık tekrar en sağdaki konuma olduğunda, oyuncu kaydırma yönünü tersine çevirmek için puls butonuna tekrar basmalıdır. Oyuncu puls butonuna çok erken veya çok geç basarsa, top kaybolur ve ışık söner. Oyun, başlatma anahtarını açıp kapatarak yeniden başlatılabilir. Oyun sırasında başlatma anahtarı açık (mantık 1) olmalıdır.

Circuit Analizi

Döngüyü bağlamadan önce, devre şemasını analiz edin ve devrenin nasıl çalıştığını anlamadığınızdan emin olun. Özellikle, aşağıdaki soruları yanıtlamaya çalışın:

Reset anahtarının işlevi nedir?

Başlatma anahtarı topraklandığında en sağdaki ışık nasıl yanar? Oyun başlama dan önce başlatma anahtarını mantık-1 konumuna yerleştirmenin neden gerekli olduğunu açıklayın.

Top hareket ettirildiğinde iki mod kontrol girişi, S1 ve S0 ne olur?

Top sola hareket ederken pulsere basılırsa mod kontrol girişlerine ve topa ne olur? Top sağa hareket ediyorsa, ancak henüz en sağdaki konuma ulaşmadıysa ne olur?

Top en sağdaki konuma döndüğünde, ancak pulsere henüz basılmadıysa, pulsere basıldığında mod kontrol girişlerinin durumu nedir? Basılmazsa ne olur?

Oyunu Oynamak

Şekil 9.20'deki devreyi bağlayın. Oyunu oynayarak devrenin düzgün çalışıp çalışmadığını test edin. Pulsere pozitif kenar geçişi sağlaması gerektiğini ve hem reset hem de başlatma anahtarlarının oyunda açık (yani mantık-1 durumunda) olması gerektiğini unutmayın. Düşük bir saat hızıyla başlayın ve saat frekansını artırarak hentbol oyununu daha zorlu hale getirin.

Kayıpların Sayısını Hesaplama

Bir devre tasarlayın, bu devre oyuncunun oyunu oynarken kaybettiği sayıyı tutar. Sayımı 0'dan 9'a kadar göstermek için BCD'den yedi segmentli bir dekode ve yedi segmentli bir ekran kullanın, Fig. 9.8'de olduğu gibi. Sayım, 7493'ü bir dalgalı ondalık sayaç olarak veya 74161 ve bir NAND kapısını senkron ondalık sayaç olarak kullanarak yapılır. Devre sıfırlandığında ekran 0 göstermelidir. Top kaybolduğunda ve ışık kapandığında, ekran 1 artmalıdır. Oyun sırasında ışık yanmaya devam ederse, ekrandaki sayı değişmemelidir. Nihai tasarım bir

uanlama devresi, oyuncu ışık kaybolduğunda her seferinde otomatik olarak on dalık gösterimi artırılır.

Lamp Ping-Pong™

Şekil 9.20'deki devreyi, bir lamba Ping-Pong oyunu elde edecek şekilde değiştirin. Bu oyuna iki oyuncu katılabilir ve her oyuncunun kendi pulsörü vardır. Sağ pulsöre sahip oyuncu, top en sağ konumdayken topu geri gönderir ve sol pulsöre sahip oyuncu, top en sol konumdayken topu geri gönderir. Ping-Pong oyunu için gereken tek değişiklik, ikinci bir pulsör ve birkaç telin değiştirilmesidir.

İkinci bir başlatma devresi ile oyun, iki oyuncudan biri (yani, biri servis yapar) tarafından başlatılabilir. Bu ekleme isteğe bağlıdır.

9.16 DENEY 15: SAAT DARBE JENERATÖRÜ

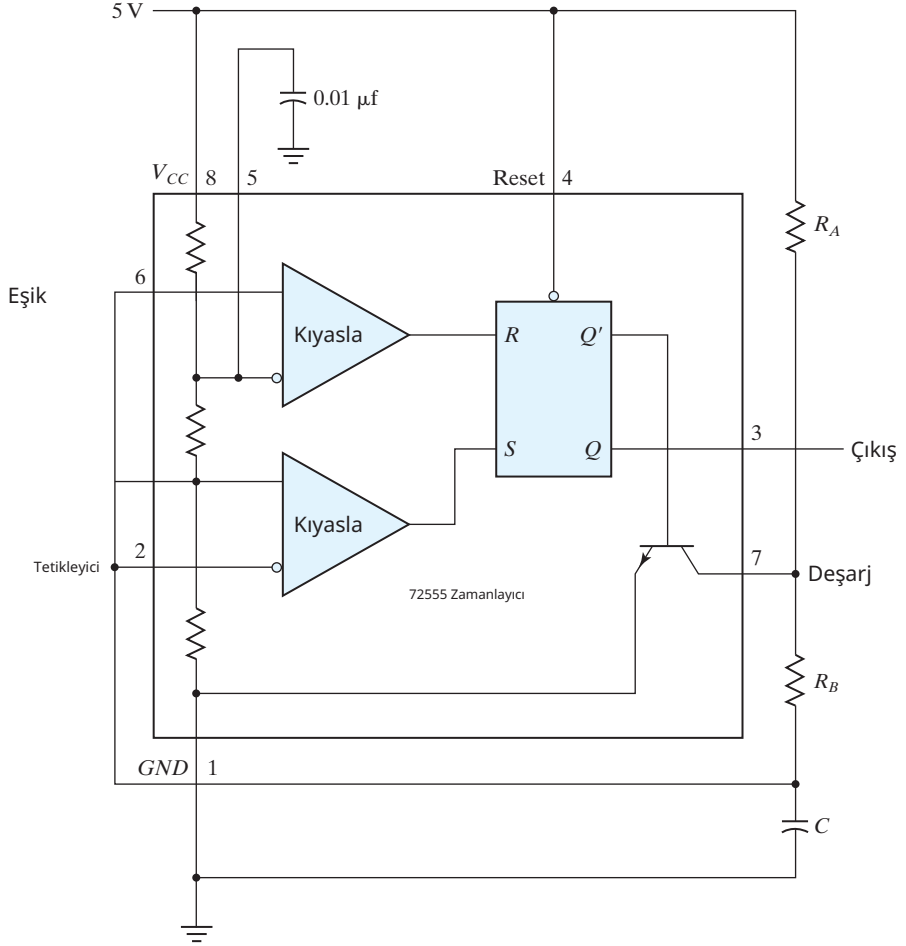
Bu deneyde, bir IC zamanlayıcı birimi kullanacak ve belirli bir frekansta saat darbeleri üretmek için bağlayacaksınız. Devre, iki harici direnç ve iki harici kapasitörün bağlantısını gerektirir. Dalga formlarını gözlemlemek ve darbelerin frekansını ölçmek için katot ışın oscilloskobu kullanılır.

IC Zamanlayıcı

IC tipi 72555 (veya 555), iç mantığı Şekil 9.21'de gösterilen hassas bir zamanlayıcı devresidir. (Dirençler, R_A ve R_B , ve iki kapasitör IC'nin bir parçası değildir.) Devre, iki voltaj karşılaştırıcı, bir flip-flop ve bir iç transistörden oluşur. Üç iç dirençten yere kadavolan voltaj bölümü, karşılaştırıcıların sabit girişlerine $V_{CC} = 5\text{ V}$ üzerinden ve V_{CC} (sırasıyla 3.3 V ve 1.7 V) üretir. Pin 6'daki eşik girişi 3.3 V'yi aştığında, üst karşılaştırıcı flip-flop'u sıfırlar ve çıkış yaklaşık 0 V'ye düşer. Pin 2'deki tetikleme girişi 1.7 V'nin altına düştüğünde, alt karşılaştırıcı flip-flop'u ayarlar ve çıkış yaklaşık 5 V'ye yükselir. Çıkış düşük olduğunda, Q yüksektir ve transistörün baz-emiter bağlantısı ileri yönde polarizedir. Çıkış yüksek olduğunda, Q düşüktür ve transistör kesilir. (Bkz. Bölüm 10.3.) Zamanlayıcı devresi, harici bir RC devresi tarafından kontrol edilen doğru zaman gecikmeleri üretebilir. Bu deneyde, IC zamanlayıcı kararsız modda çalıştırılacak ve saat darbeleri üretecektir.

Döngü İşlemi

Şekil 9.21, devrenin astable çalışması için dış bağlantıları göstermektedir. Kapasitör C dirençler üzerinden R_A ve R_B kesildiğinde akım geçer ve boşalır. R_B transistör ileri yönde polarize olduğunda ve iletim sağladığında. Kapasitör üzerindeki şarj voltajı C 3.3 V'ye ulaştığında, pin 6'daki eşik girişi flip-flop'un sıfırlanmasına neden olur ve transistör açılır. Boşaltma voltajı 1.7 V'ye ulaştığında, pin 2'deki tetikleme girişi flip-flop'un ayarlanmasına neden olur ve transistör kapanır. Böylece, çıkış sürekli olarak değişir.



ŞEKİL 9.21

IC türü 72555 zamanlayıcı, saat darbe üretici olarak bağlanmıştır

flip-flop'un çıkışında iki voltaj seviyesi arasında. Çıkış, şarj süresine eşit bir süre boyunca yüksek kalır. Bu süre, aşağıdaki denklemden belirlenir

$$t_H = 0.693(R_A + R_B)C$$

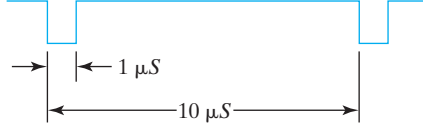
Çıkış, deşarj süresine eşit bir süre boyunca düşük kalır. Bu süre, eşitlikten belirlenir

$$t_L = 0.693R_B C$$

Saat Darbe Üretici

Kondansatör ile başlayarak $C = 0.001 \mu F$ değerine sahip, saat

darbeleri üretmek için R_A ve R_B değerlerini hesaplayın, Şekil 9.22'de gösterildiği gibi. Darbe genişliği düşük seviyede $1 \mu s$ 'dir ve tekrar eder.



ŞEKİL 9.22

Saat üretici için çıkış dalga formu

100 kHz frekans hızı (her 10 μ s). Devreyi bağlayın ve çıkışı

oscilloscope'da kontrol edin.

Kondansatör üzerindeki çıkışı C, gözlemleyin ve tetikleme ile eşik değerleri arasında olduğunu doğrulamak için iki seviyesini kaydedin.

Transistörün 7. pinindeki kolektördeki dalga biçimini gözlemleyin ve tüm ilgili bilgileri kaydedin. Devrenin işleyişini analiz ederek dalga biçimini açıklayın.

Değişken bir direnç (potansiyometre) R_A ile seri bağlayarak değişken frekanslı bir darbe üretici oluşturun. Düşük seviye süresi 1 μ s'de kalır. Frekans 20 ile 100 kHz arasında olmalıdır.

7404 inverter ile düşük seviyeli darbeleri yüksek seviyeli darbeler haline getirin. Bu, değişik frekans aralığına sahip 1 μ s'lik pozitif darbeler üretecektir.

9.17 DENEY 16: PARALEL TOPLAYICI VE TOPLAYICI

Bu deneyde, toplamı bir kaydediciye yüklenebilen dört bitlik bir paralel toplayıcı inşa edeceksiniz. Toplanacak sayılar bir RAM'de saklanacaktır. Bellekten bir dizi ikili sayı seçilecek ve bunların toplamı kaydediciye biriktirilecektir.

Blok Diyagramı

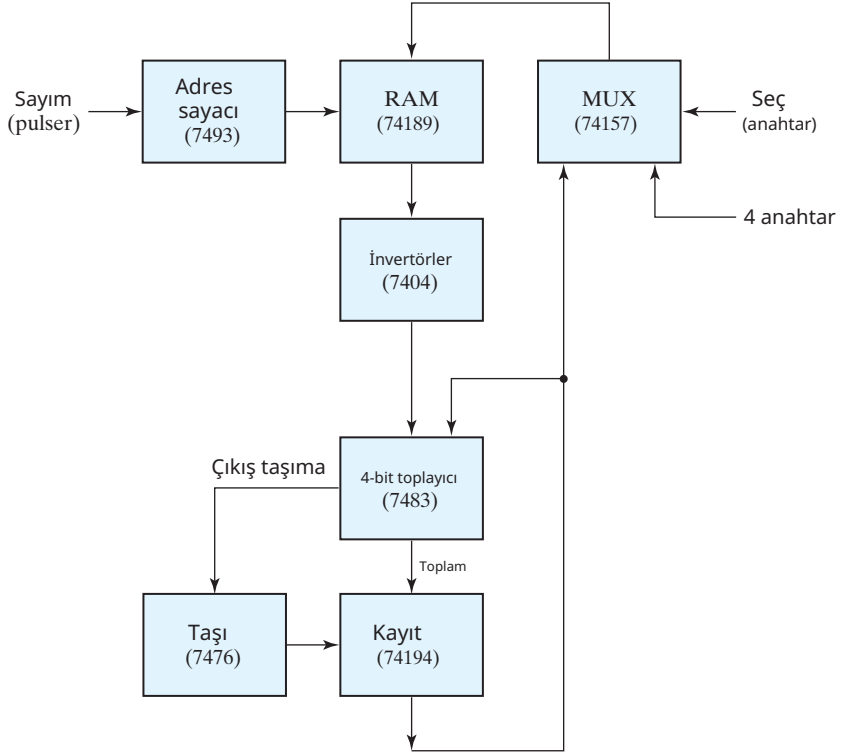
Devreyi inşa etmek için 9.14 bölümündeki bellek deneyinden RAM devresini, dört bitlik bir paralel toplayıcıyı, paralel yüklemeli dört bitlik bir kaydediciyi, bir taşıma flip-flopunu ve bir çoklayıcıyı kullanın. Blok diyagramı ve kullanılacak entegre devreler Şekil 9.23'te gösterilmiştir.

RAM'e dört anahtardan veya kaydedicinin çıkışlarındaki dört bitlik veriden bilgi yazılabilir. Seçim, bir çoklayıcı aracılığıyla yapılır. RAM'deki veriler, kaydedicinin içeriğine eklenebilir ve toplam kaydediciye geri aktarılabilir.

Kayıt Kontrolü

74194 kaydedicisini ve 7476 taşıma flip-flopunu kontrol etmek için aşağıdaki gibi anahtarlar sağlayın:

- (a) YÜKLE koşulu, bir saat darbesi uygulandığında toplamı kaydediciye ve çıkışı taşımasını flip-flopa aktarır.



ŞEKİL 9.23

Deney 16 için paralel toplayıcının blok diyagramı

- (b) SHIFT durumu, saat darbesinin uygulanmasıyla birlikte, taşıma flip-flop'un dan alınan taşıma ile kaydın en sağdan sola kaydırılmasını sağlar. Taşıma flip-flop'undaki değer kaydırma sırasında değişmemelidir.
- (c) NO-CHANGE durumu, saat darbeleri uygulansa bile kaydın ve flip-flop'un içeriğini değiştirmeden bırakır.

Taşıma Devresi

Önceki spesifikasyonlara uymak için, toplayıcıdan çıkan taşıma ile J ve K giri şleri arasında bir devre sağlamak gereklidir, böylece çıkış taşıması yalnızca YÜK LE (LOAD) durumu etkinleştirildiğinde ve flip-flop'un saat girişine bir darbe uygulandığında flip-flop'a aktarılır (0 veya 1'e eşit olup olmadığına bakılmaksızın). YÜKLE (LOAD) durumu devre dışı bırakıldığında veya SHIFT durumu etkinleştirildiğinde taşıma flip-flop'u değişmemelidir.

Ayrıntılı Devre

Ayrıntılı bir diyagram çizin ve tüm IC'ler arasındaki bağlantıları gösterin. Devreyi bağlayın, ve kayıt ve taşıma flip-flop'un çıkışları ile RAM'in adres ve çıkış verileri için gösterge lambaları sağlayın.

Devreyi Kontrol Etme

RAM'de 0110, 1110, 1101, 0101 ve 0011 sayılarını depolayın ve ardından bunları kayıt içine birer birer ekleyin. Temizlenmiş bir kayıt ve flip-flop ile başlayın. Aşağıdaki toplamda her eklemekten sonra kayıt ve taşıma çıkışındaki değerleri tahmin edin ve yaptığınız sonuçları doğrulayın:

$$0110 + 1110 + 1101 + 0101 + 0011$$

Döngü İşlemi

Kayıt ve taşıma flip-flop'u sıfıra temizleyin ve aşağıda belirtilen adreslerde RAM'de aşağıdaki dört bitlik sayıları depolayın:

Adres	İçerik
0	0110
3	1110
6	1101
9	0101
12	0011

Şimdi aşağıdaki dört işlemi gerçekleştirin:

1. LOAD

koşulunu kullanarak adres 0'ın içeriğini kaydediciye ekleyin.

Kaydediciden RAM'de adres 1'e toplamı depolayın.

3. SHIFT koşuluyla kaydedicinin içeriğini sağa kaydırın ve taşıyın.

4. Kaydedicinin kaydırılmış içeriğini RAM'de adres 2'ye depolayın.

RAM'deki ilk üç konumun içeriğinin aşağıdaki gibi olduğunu kontrol edin:

Adres	İçerikler
0	0110
1	0110
2	0011

RAM'de depolanan diğer dört ikili sayı için yukarıdaki dört işlemi tekrarlayın. Adım 2'de kaydediciden toplamı depolamak için adres 4, 7, 10 ve 13'ü kullanın.

Adres 5, 8, 11 ve 14'ü, adım 4'teki kaydırılmış değeri depolamak için kullanın. Adresler 0'dan 14'e kadar olan RAM içeriğinin ne olacağını tahmin edin ve sonuçlarınızı doğrulamak için kontrol edin.

9.18 DENEY 17: İKİLİ ÇARPAN

Bu deneyde, 2 adet dört bitlik işaretli sayıyı çarparak sekiz bitlik bir ürün üretmek için bir devre tasarlayıp inşa edeceksiniz. İki ikili sayıyı çarpma algoritması Bölüm 8.7'de sunulmuştur. Bu deneyde uygulanan algoritma, yalnızca dört bitlik bir veri yolunu ele alması ve bir bit sayacını azaltmak yerine artırması bakımından Şekil 8.14 ve 8.15'te tanımlanan algoritmadan farklıdır.

Blok Diyagramı

ASMD grafiği ve önerilen entegre devrelerle birlikte ikili çarpmanın blok diyagramı Şekil 9.24 (a) ve (b)'de gösterilmiştir. Çarpıcı, B , bir kayıt yerine dört anahtardan elde edilmektedir. Çarpıcı, Q , başka bir dört anahtardan elde edilmektedir. Ürün, sekiz gösterge lambası ile görüntülenir. Sayıcı P 0 olarak başlatılır ve ardından her kısmi ürün oluşturulduktan sonra artırılır. Sayıcı dört sayısına ulaştığında, çıkış $Done$ 1 olur ve çarpma işlemi sona erer.

Kayıtların Kontrolü

Şekil 9.24 (a) için ikili çarpıcısındaki ASMD grafiği, üç kayıt ve veri yolu biriminin taşıma flip-flop'unun $Load_regs$ sinyalleri ile kontrol edildiğini göstermektedir. $Incr_P$, Add_regs , ve $Shift_regs$. Kontrol biriminin dış girişi sinyalleri $clock$, $reset_b$ (aktif-düşük) ve $Start$; kontrol birimine bir diğer girişi ise iç status sinyali, $Done$, veri yolu birimi tarafından oluşturulur ve sayacın dört sayısına ulaştığını belirtir, bu da çarpıcının bit sayısına karşılık gelir.

$Load_regs$ ürün kaydını (A) ve taşıma flip-flop'unu (C) temizler, çarpıcı B kaydına yükler, çarpıcı Q kaydına yükler ve bit sayacını temizler. $Incr_P$ bit sayacını kısmi ürünün birikimi ile eşzamanlı olarak artırır. Add_regs çarpıcı A 'ya ekler, eğer kaydırılan çarpıcının en az anlamlı biti ($Q[0]$) 1 ise. Flip-flop C , eklemeyi kaydeden bir taşıma için yer tutar.

Birleştirilmiş kayıt CAQ içeriğinin bir bit sağa kaydırılmasının sonucunu saklayarak güncellenir. $Shift_regs$ CAQ bir bit sağa kaydırır, bu da flip-flop C 'yi temizler.

Kontrol birimi için durum diyagramı Şekil 9.24 (c)'de gösterilmiştir. Veri yolu biriminin kayıt işlemlerini veya bunları kontrol eden çıkış sinyallerini göstermediğine dikkat edin. Bu bilgi Şekil 9.24 (d)'de açıktır. $Incr_P$ ve $Shift_regs$ sırasıyla S_add ve S_shift durumlarında koşulsuz olarak üretilmektedir. $Load_regs$

durumun içinde olduğu süre boyunca *Başla* koşullu olarak belirtilirse üretilmiştir $S_idle;Add_regs$ koşullu olarak S_add içinde $Q[0] = 1$ olduğunda etkinleştirilir.

Çarpma Örneği

Devreyi bağlamadan önce, çarpınların çalışmasını anladığınızdan emin olun. Bunu yapmak için, Tablo 8.5'e benzer bir tablo oluşturun, ancak çarpın için $B=1111$ ve çarpın için $Q=1011$ kullanın. Tabloyun sol tarafında listelenen her yorumla birlikte durumu belirtin.

Veri Yolu Tasarımı

Çarpının veri yolu kısmının detaylı bir diyagramını çizin, tüm IC pin bağlantılarını gösterin. Dört kontrol sinyalini anahtarlarla oluşturun ve bunları çeşitli kayıtlar için gerekli kontrol işlemlerini sağlamak üzere kullanın. Devreyi bağlayın ve her bir bileşenin düzgün çalıştığını kontrol edin. Kontrol sinyalleri 0 iken, çarpın anahtarlarını 1111 ve çarpın anahtarlarını 1011 olarak ayarlayın. Kontrol sinyallerini, Şekil 9.24 (c) durum diyagramında belirtildiği gibi, kontrol anahtarları aracılığıyla manuel olarak etkinleştirin. Her kontrol durumunda tek bir darbe uygulayın ve A ve Q kayıtlarının çıktıları ile C ve P'deki değerleri gözlemleyin. Bu çıktıları sayısal örneğinizdeki sayılarla karşılaştırarak devrenin düzgün çalıştığını doğrulayın. IC tipi 74161'in master-slave flip-flop'lara sahip olduğunu unutmayın. Manuel olarak çalıştırmak için, tek saat darbesinin negatif bir darbe olması gerekmektedir.

Kontrol Tasarımı

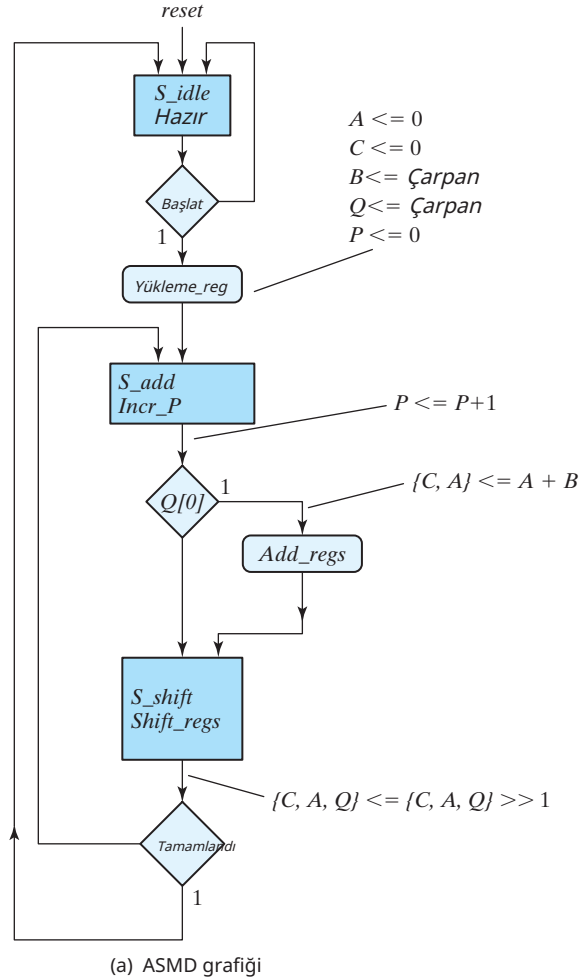
Durum diyagramında belirtilen kontrol devresini tasarlayın. Bölüm 8.8'de tartışılan herhangi bir kontrol uygulama yöntemini kullanabilirsiniz.

IC sayısını en aza indiren yöntemi seçin. Kontrol devresinin veri yolu birimine bağlanmadan önce çalışmasını doğrulayın.

Çarpın Kontrolü

Kontrol devresinin çıkışlarını veri yolu birimine bağlayın ve 1111 ile 1011 çarpma adımlarını tekrarlayarak toplam devre çalışmasını doğrulayın. Tekil saat darbeleri artık kontrol durumlarını da sıralamalıdır. (Manuel anahtarları çıkarın.) Başlangıç sinyali (*Başla*) kontrol durumunun S_idle olduğu sırada açık olan bir anahtar ile üretilir.

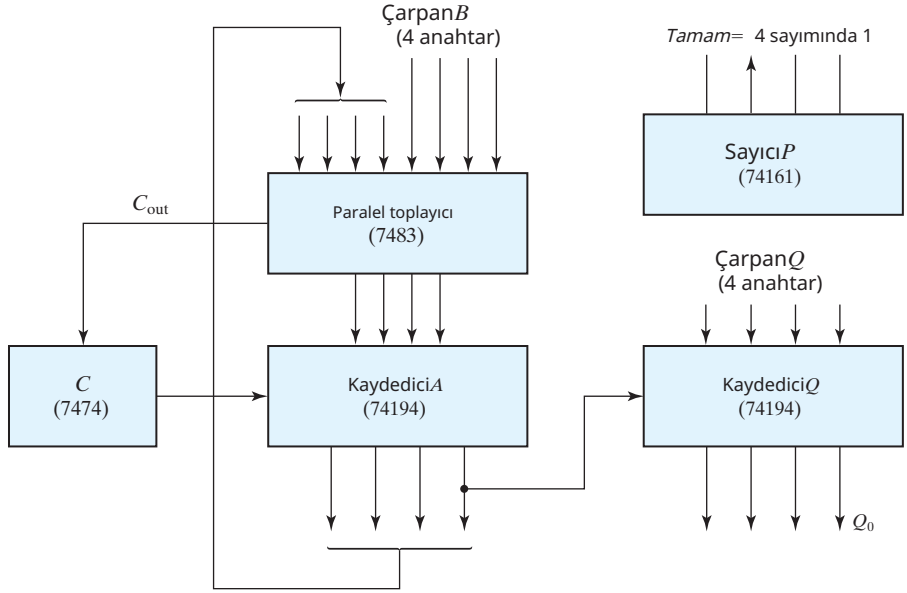
Başlangıç sinyalini (*Başla*) bir darbe üretici veya başka bir kısa darbe ile üretin ve çarpını bir saat üreticiden sürekli saat darbeleri ile çalıştırın. Darbe üreticine basmak *Başla* çarpma işlemi başlatmalıdır ve tamamlandığında, çarpım A ve Q kayıtlarında görülmelidir. Çarpmanın, *Başla* sinyali etkin olduğu sürece tekrarlanacağını unutmayın. *Başla* sinyalinin 0'a döndüğünden emin olun. Ardından anahtarları iki başka dört bitlik sayıya ayarlayın ve tekrar *Başla* butonuna basın. Yeni çarpım çıkışlarda görünmelidir. Devrenin çalışmasını doğrulamak için birkaç sayının çarpımını tekrarlayın.

**ŞEKİL 9.24**

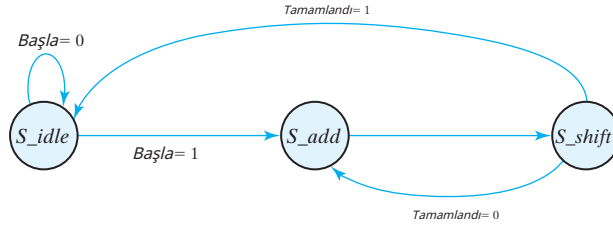
ASMD diyagramı, veri yolu blok diyagramı, kontrol durumu diyagramı ve ikili çarpan devresinin kayıt çalışmaları

9.19 VERILOG HDL SIMULASYON DENEMELERİ VE FPGA'larla HIZLI PROTOTİPLENDİRME

Alan programlanabilir kapı dizileri (FPGAs), endüstri tarafından, sistem karmaşık olduğunda, p azara çıkış süresi kısa olduğunda, bir FPGA'nın performansı (örneğin, hız) kabul edilebilir oldu ğunda ve potansiyel satış hacmi standart hücre tabanlı ASIC'e yatırım yapmayı gerektirmediği nde mantığı uygulamak için kullanılır. Devreler, bir FPGA'ya hızlı bir şekilde prototiplelenebilir.



(b) Veri yolu blok programı



(c) Kontrol durum diyagramı

Durum Geçişi	Kayıt İşlemleri	Kontrol sinyali
<u>Başlangıç</u> <u>Son</u>		
<i>S_idle</i>	Başlangıç durumu sıfırlama işlemi ile ulaşıldı	
<i>S_idle</i> <i>S_add</i>	$A \leq 0, C \leq 0, P \leq 0$	<i>Yükle_kayıtlar</i>
<i>S_add</i> <i>S_shift</i>	$P \leq P + 1$	<i>Artır_P</i>
	if ($Q[0]$) then ($A \leq A + B, C \leq C_{out}$)	<i>Ekle_kayıtlar</i>
<i>S_shift</i>	shift right [CAQ], $C \leq 0$	<i>Kaydır_kayıtlar</i>

(d) Kayıt işlemleri

HDL. HDL modeli doğrulandıktan sonra, tanım sentezlenir ve FPGA'ya haritalanır. FPGA satıcıları, bir devrenin HDL tanımını optimize edilmiş kapı seviyesinde bir tanıma sentezlemek ve elde edilen netlist'i FPGA'larının kaynaklarına haritalamak (uygulamak) için yazılım araçları sağlar. Bu süreç, bir devreyi bir breadboard üzerinde oluşturmanın gerektirdiği ayrıntılı IC montajını önler ve süreç, bir HDL tanımını düzenlemenin, bir breadboard'u yeniden kablolamaktan daha kolay ve hızlı olması nedeniyle, başarısızlık riski açısından önemli ölçüde daha az risk içerir.

Bu bölümde özetlenen donanım deneylerinin çoğu, Verilog donanım tanım dili (HDL) kullanarak karşılık gelen bir yazılım prosedürü ile desteklenebilir. Bu eklemeler için bir Verilog derleyicisi ve simülatörü gereklidir. Ek deneylerin iki katmanlı bir katılımı vardır. İlk olarak, uygulamalı laboratuvar deneylerinde belirtilen devreler Verilog ve bir simülatör kullanılarak tanımlanabilir, simüle edilebilir ve doğrulanabilir. İkincisi, uygun bir FPGA prototipleme kartı mevcutsa (örneğin, www.digilentinc.com adresine bakın), donanım deneyleri Verilog tanımlarını sentezleyerek ve devreleri bir FPGA'da uygulayarak gerçekleştirilebilir.

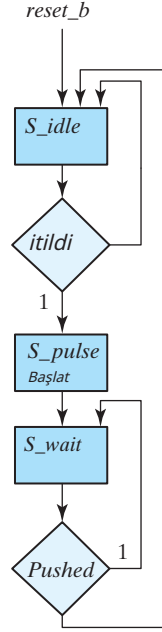
Bireysel (yapısal) donanım birimlerinin (örneğin, 4 bitlik bir sayaç) kimliğinin uygun olduğunda korunması, bunları ayrı Verilog modüllerine kapsülleyerek sağlanabilir; bu modüllerin iç detayları davranışsal olarak veya davranışsal ve yapısal modellerin bir karışımıyla tanımlanır.

Bir devreyi FPGA ile prototiplemek, iç kaynakları (örneğin, bir Xilinx FPGA'sının CLB'leri) ve FPGA'nın bağlantılarını yapılandırmak için indirilebilecek bir bit akışı üretmek amacıyla bir Verilog tanımını sentezlemeyi gerektirir. Üç detay dikkat gerektirir: (1) Prototipleme kartının pinleri FPGA'nın pinlerine bağlıdır ve sentezlenen devrenin donanım uygulaması, giriş ve çıkış sinyallerinin prototipleme kartının pinleri ile ilişkilendirilmesini gerektirir (bu ilişkilendirme, FPGA'nın tedarikçisi tarafından sağlanan sentez aracı kullanılarak yapılır (bu tür araçlar ücretsiz olarak mevcuttur)), (2) FPGA prototipleme kartları bir saat üreticisine sahiptir, ancak bazı durumlarda, deney için uygun bir frekansa sahip bir iç saat elde etmek amacıyla bir saat bölücü (Verilog'da) uygulamak gerekli olabilir ve (3) FPGA tabanlı bir devreye girişler, prototipleme kartında bulunan anahtarlar ve butonlar kullanılarak yapılabilir, ancak bir sayacın veya bir durum makinesinin aktivitesini kontrol etmek ve gözlemlemek için yazılımda bir pulsör devresi uygulamak gerekli olabilir (bkz. Deney 1'e ek).

Deney 1'e (Bölüm 9.2) Ek

Deney 1'de belirtilen sayaçların işlevselliği Verilog'da tanımlanabilir ve FPGA'da uygulanmak üzere sentezlenebilir. Şekil 9.3'te gösterilen devrenin, standart entegre devrelerle inşa edilmiş bir devrede sayının artmasını sağlamak için bir butonlu pulsör veya bir saat kullandığını unutmayın. FPGA'nın prototipleme kartındaki bir anahtar ile çalışacak bir yazılım pulsör devresi geliştirilebilir, böylece sayaçların çalışması görsel inceleme ile doğrulanabilir.

Yazılım pulsörünün, dış girişin (*Pushed*) mekanik bir anahtardan veya butonda alınacağı Şekil 9.25'te gösterilen ASM diyagramı vardır. Bu devre, bir saat döngüsü boyunca Start'ı belirler ve ardından anahtarın açılmasını (veya butonun) bekler.



ŞEKİL 9.25

Deney 1'in FPGA uygulaması için Pulser devresi

serbest bırakılacak) her anahtar veya buton eyleminin yalnızca bir *Başlat* darbesi üreteceğini sağlamak için. Sayıcı veya bir durum makinesi, anahtar kapandığında sıfırlama durumunda (*S_idle*) ise, darbe sayıcının veya durum makinesinin açıklamasını başlatacaktır. *Başlat* yeniden etkinleştirilmeden önce anahtarın açılması (veya butonun bırakılması) gerekecektir. Yazılım da rbe üretici kullanmak, sayımın her değerinin gözlemlenmesini sağlayacaktır.

Gerekirse, *Pushed* ile basit bir senkronizatör devresi kullanılabilir.

Deney 2'ye Ek (Bölüm 9.3)

Çeşitli mantık kapıları ve bunların yayılma gecikmeleri donanım deneyinde tanıtıldı. Bölüm 3.10'da, kapı gecikmeleri ile basit bir devre incelendi. Laboratuvar Verilog pro gramına giriş olarak, HDL Örneği 3.3'te tanımlanan devreyi derleyin ve ardından Fig. 3.38'de gösterilen dalga formlarını doğrulamak için simülatörü çalıştırın.

Aşağıdaki gecikmeleri Şekil 3.32(a)'da gösterilen özel-VE devresine atayın: 10 ns bir tersleyici için, 20 ns bir VE kapısı için ve 30 ns bir VEYA kapısı için. Devrenin girişi şu şekilde gider: $xy=00b$ $xy=01$.

- Her kapının çıkışındaki sinyalleri belirleyin $t=0$ ile $t=50$ ns.
- Gecikmeleri de içeren devrenin HDL tanımını yazın.

- (c) Bir uyarı modülü yazın (HDL Örneği 3.3'e benzer) ve devreyi simüle edin
(a) bölümündeki cevabı doğrulamak için.
- (d) Devreyi bir FPGA ile uygulayın ve çalışmasını test edin.

Deney 4'e Ek (Bölüm 9.5)

Kombinasyonel bir devrenin çalışması, çıktıyı kontrol ederek ve bunu devrenin doğruluk tablosu ile karşılaştırarak doğrulanır. HDL Örneği 4.10 (Bölüm 4.12), bir kombinasyonel devrenin doğruluk tablosunu elde etme prosedürünü simüle ederek göstermektedir.

- (a) Bu prosedürü tanımak için HDL'yi derleyin ve simüle edin
Örnek 4.10'u ve çıkış doğruluk tablosunu kontrol edin.
- (b) Deney 4'te, bir çoğunluk mantık devresi tasarladınız. Çoğunluk mantık devresinin HDL kapı düzeyindeki tanımını yazın ve doğruluk tablosunu görüntülemek için bir uyarıcı ekleyin. Devreyi derleyin ve simüle edin, çıkış tepkisini kontrol edin.
- (c) Bir FPGA'da çoğunluk mantık devresi birimlerini uygulayın ve çalışmasını test edin.

Deney 5'e (Bölüm 9.6) ek

Bu deney kod dönüşümü ile ilgilidir. Bölüm 4.4'te bir BCD'den fazladan 3'e dönüştürücü tasarlandı. Tasarımın sonucunu kullanarak bir HDL simülatörü ile kontrol edin.

- (a) Şekil 4.4'te gösterilen devrenin HDL kapı düzeyindeki tanımını yazın.
- (b) Şekil 4.3'te listelenen Boolean ifadelerini kullanarak bir veri akışı tanımı yazın.
- (c) Bir BCD'den fazladan 3'e dönüştürücünün HDL davranışsal tanımını yazın.
- (d) Doğruluk tablosunu doğrulamak için BCD'den fazladan 3'e dönüştürücü devresini simüle etmek ve test etmek için bir test tezgahı yazın. Üç devreyi de kontrol edin.
- (e) Davranışsal tanımı bir FPGA ile uygulayın ve devrenin çalışmasını test edin.

Deney 7'ye (Bölüm 9.8) Ek

Bu deneyde dört bitlik bir toplama-çıkarma devresi geliştirilmiştir. Bir toplama-çıkarma devresi ayrıca Bölüm 4.5'te geliştirilmiştir.

- (a) 7483 dört bitlik toplayıcı için HDL davranışsal tanımını yazın.
- (b) Adder-çıkarcı devresinin davranışsal tanımını yazın. Şekil 9.11.
- (c) Dört bitlik adder-çıkarcının HDL hiyerarşik tanımını yazın Şekil 4.13 (V d ahi). Bu, HDL Örneği 4.2'de (Bölüm 4.12) tanımlanan dört bitlik adder'ın değiştirilmiş bir versiyonunu örnekleyerek yapılabilir.
- (d) (c) kısmındaki devreleri simüle etmek ve test etmek için bir HDL test tezgahı yazın. Taşma (overflow) oluşturan değerleri kontrol edin ve doğrulayın $V=1$.
- (e) (c) kısmındaki devreyi bir FPGA ile uygulayın ve çalışmasını test edin.

Deney 8'e (Bölüm 9.9) Ek

Kenar tetiklemeli D flip-flop 7474, Şekil 9.13'te gösterilmiştir. Flip-flop'un asenkron preset ve clear girişleri vardır.

- 7474 D flip-flop'un HDL davranışsal tanımını yazın, yalnızca Q çıkışını kullanarak. (Preset = 0 olduğunda, Q 1'e gider ve Preset = 1 ve Clear = 0 olduğunda, Q 0'a gider. Bu nedenle, Preset Clear'dan önceliklidir.)
- 7474 D flip-flop'un HDL davranışsal tanımını yazın, her iki çıkışı kullanarak. İkinci çıkışı Q_not olarak etiketleyin ve bunun her zaman Q 'nın tamamlayıcı olmadığını not edin. (Preset=Clear= 0 olduğunda, hem Q hem de Q_not 1'e gider.)

Deney 9'a (Bölüm 9.10) Ek

Bu donanım deneyinde, size verilen durum diyagramına göre bir ardışık devre tasarlayıp test etmeniz istenmektedir. Durum diyagramı Şekil 9.14'te verilmiştir. Bu, HDL Örneği 5.5'te (Bölüm 5.6) tanımlanan bir modele benzer bir Mealy model ardışık devresidir.

- Şekil 9.14'ün durum diyagramının HDL tanımını yazın.
- Tasarımdan elde edilen ardışık devrenin HDL yapısal tanımını yazın. (Bu, Bölüm 5.6'daki HDL Örneği 5.7'ye benzer.)
- Şekil 9.24 (c) (Bölüm 9.18) bir kontrol durum diyagramını göstermektedir. Durum diyagramının HDL tanımını yazın, one-hot ikili atama kullanarak (Bölüm 5.7'deki Tablo 5.9'a bakın) ve dört çıkış— T_0, T_1, T_2 , ve T_3
- Veri yolu biriminin davranışsal modelini yazın ve birbirine bağlı kontrol birimi ile veri yolu biriminin doğru çalıştığını doğrulayın.
- Entegre devreyi bir FPGA ile uygulayın ve çalışmasını test edin.

Deney 10'a (Bölüm 9.11) Ek

Paralel yüklemeli senkron sayıcı IC türü 74161, Şekil 9.15'te gösterilmektedir. Bu devre, HDL Örneği 6.3'te (Bölüm 6.6) tanımlanan devreye benzer, iki istisna ile: Yük girişi 0 olduğunda etkinleştirilir ve sayımı kontrol eden iki giriş (P ve T) vardır. 74161 IC'nin HDL tanımını yazın. Sayıcıyı bir FPGA ile uygulayın ve çalışmasını test edin.

Deney 11'e (Bölüm 9.12) Ek

Bu deneyde, 74195 ve 74157 entegre devre türlerini kullanarak paralel yüklemeli iki yönlü bir kaydırma kaydedici tasarlanmıştır.

- 74195 kaydırma kaydedicisinin HDL tanımını yazın. Girişlerin J ve K seri girişi oluşturmak için birleştirildiğini varsayın.
- 74157 çoklayıcısının HDL tanımını yazın.

486 Bölüm 9 Laboratuvar Deneyleri

- (c) Bu deneyde tasarlanan dört bitli iki yönlü kaydırma kaydedicisinin HDL tanımını elde edin. (1) İki entegre devreyi örnekleyerek yapısal tanım yazın ve bunların bağlantılarını belirtin, (2) bu tasarım deneyinde elde edilen fonksiyon tablosunu kullanarak devrenin davranışsal tanımını yazın.
- (d) Devreyi bir FPGA ile uygulayın ve çalışmasını test edin.

Deney 13'e (Bölüm 9.14) Ek

Bu deney, rastgele erişim belleğinin (RAM) çalışmasını araştırmaktadır. Belleğin HDL'de nasıl tanımlandığı, Bölüm 7.2'de HDL Örneği 7.1 ile birlikte açıklanmaktadır.

- (a) Şekil 9.18'de gösterilen IC türü 74189 RAM'in HDL tanımını yazın.
- (b) Belleğin çalışmasını test etmek için, adres 0'a ikili 3 ve adres 14'e ikili 1 depolayan bir uyarı programı yazın. Ardından, sayıları doğru bir şekilde depolandığını kontrol etmek için iki adresten depolanan sayıları okuyun.
- (c) RAM'i bir FPGA ile uygulayın ve çalışmasını test edin.

Deney 14'e (Bölüm 9.15) ek

- (a) Şekil 9.19'da gösterilen paralel yükleme ile 74194 iki yönlü kaydırma kaydedicisinin HDL davranışsal tanımını yazın.
- (b) Kaydırma kaydedicisini bir FPGA ile uygulayın ve çalışmasını test edin.

Deney 16'ya (Bölüm 9.17) ek

Birikim kaydedici ve bellek birimi ile birlikte bir paralel toplayıcı, Şekil 9.23'teki blok diyagramında gösterilmiştir. Blok diyagramı tarafından belirtilen devrenin yapısal tanımını yazın. Bu devrenin HDL yapısal tanımı, çeşitli bileşenlerin örneklenmesiyle elde edilebilir. Bir tasarımın yapısal tanımına dair bir örnek, Bölüm 8.6'daki HDL Örneği 8.4'te bulunabilir. Öncelikle, her bileşenin davranışsal tanımını yazmak gereklidir. 7493 yerine 74161 sayacını kullanın ve 7476 JK flip-flop yerine 7474 D flip-flop'u yerleştirin. Çeşitli bileşenlerin blok diyagramı, Tablo 9.1'deki listeden bulunabilir. Her model için bir test tezgahı yazın ve ardından tüm tasarımı doğrulamak için bir test tezgahı yazın. Devreyi bir FPGA ile uygulayın ve çalışmasını test edin.

Deney 17'ye (Bölüm 9.18) Ek

Dört bitli ikili çarpanın blok diyagramı Şekil 9.24'te gösterilmiştir. Çarpan iki şekilde tanımlanabilir: (1) Şekil'in (b) kısmında listelenen kayıt transfer seviyesi ifadeleri ni kullanarak veya (2) Şekil'in (a) kısmında gösterilen blok diyagramını kullanarak

nın kayıt transfer seviyesi (RTL) formatında tanımı HDL Örneği 8.5'te (Bölüm 8.7) gerçekleştirilmiştir.

- (a) Blok diyagramında belirtilen entegre devre bileşenlerini kullanarak ikili çarpanın HDL yapısal tanımını yazın. Yapısal tanım, her bileşenin modül tanımını kullanarak ve ardından tüm bileşenleri birbirleriyle nasıl bağlantılı olduklarını göstermek için örnekleyerek elde edilir. (Bir örnek için Bölüm 8.5'e bakın.) Bileşenlerin HDL tanımları, önceki deneylerin çözümlerinden temin edilebilir. 7483, Deney 7(a) ile, 7474 Deney 8(a) ile, 74161 Deney 10 ile ve 74194 Deney 14 ile tanımlanmıştır. Kontrol tanımı, Deney 9(c) çözümünden temin edilebilir. Çarpanı doğrulamaya çalışmadan önce her yapısal birimi doğruladığınızdan emin olun.
- (b) FPGA ile ikili çarpanı uygulayın. Deney 1'de tanımlanan pulsör kullanın.

Bölüm 10

Standart Grafik Sembolleri

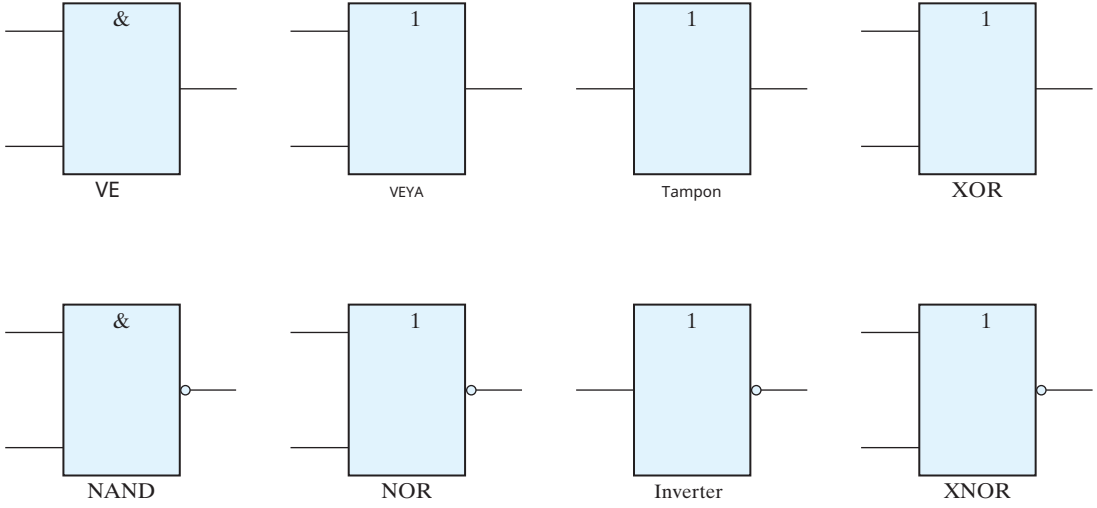
10.1 DİKDÖRTGEN ŞEKİLDE SİMBOLELER

Kapılar, kod çözücüler, çoklayıcılar ve kayıtlar gibi dijital bileşenler, entegre devreler olarak ticari olarak mevcuttur ve SSI veya MSI devreleri olarak sınıflandırılır. Kul lanıcının her işlevi, ona atanan benzersiz grafik sembolden tanıyabilmesi için bu ve diğer bileşenler için standart grafik semboller geliştirilmiştir. ANSI/IEEE Std. 91-1984 olarak bilinen bu standart, sanayi, hükümet ve meslek kuruluşları tarafından onaylanmış olup uluslararası standartlarla tutarlıdır.

Standart, her bir mantık işlevini temsil etmek için dikdörtgen şekilli bir çerçeve kullanır. Çerçevenin içinde, birimin gerçekleştirdiği mantıksal işlemi belirten genel bir nitelik sembolü vardır. Örneğin, bir çoklayıcı için genel nitelik sembolü MUX'tur. Çerçevenin boyutu keyfidir ve ya kare ya da dikdörtgen bir şekil olabilir, keyfi bir uzunluk-genişlik oranına sahiptir. Giriş hatları solda, çıkış hatları sağda yer alır. Eğer sinyal akışının yönü tersine dönerse, bu oklarla belirtilmelidir.

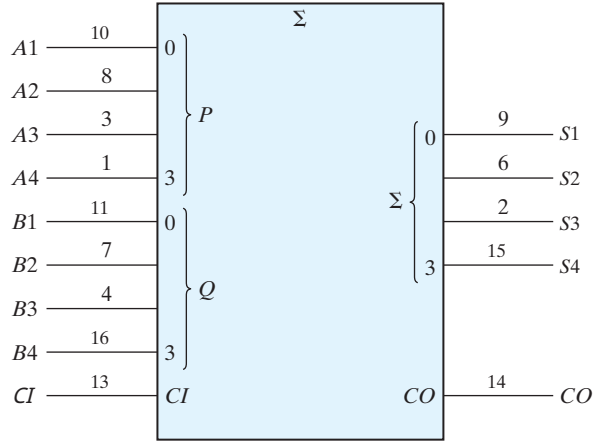
SSI kapıları için dikdörtgen şekilli grafik semboller Şekil 10.1'de gösterilmiştir. AND kapısı için nitelik sembolü ampersand (&)dir. OR kapısının nitelik sembolü, en az bir girişin aktif olması gerektiğini belirten "büyüktür veya eşittir 1" ifadesini gösterir, bu da çıkışın aktif olması için en az bir girişin aktif olması gerektiği anlamına gelir. Buffer kapısı için sembol 1'dir, bu da yalnızca bir girişin mevcut olduğunu gösterir. Özel-OR sembolü, çıkışın aktif olması için yalnızca bir girişin aktif olması gerektiğini belirtir. Çıkışta mantıksal inkâr küçük dairenin eklenmesi, kapıları tamamlayıcı değerlerine dönüştürür. Kapılar için dikdörtgen şekilli sembollerin önerilmesine rağmen, standart ayrıca Şekil 2.5'te gösterilen kapılar için ayrırt edici şekilli sembolleri de tanımaktadır.

Bir MSI standart grafik sembolü örneği, Şekil 10.2'de gösterilen dört bitli paralel toplayıcıdır. Toplayıcı için nitelik sembolü Yunan harfidir. Tercih edilen



ŞEKİL 10.1

Kapılar için dikdörtgen şekilli grafik semboller



ŞEKİL 10.2

Dört bitli paralel toplayıcı için standart grafik sembolü, IC türü 7483

aritmetik operandlar için harfler P ve Q 'dir. İki giriş türündeki bit gruplama sembolleri ve toplam çıkış, bitlerin 2 üssü ağırlıklarının ondalık karşılıklarıdır. Böylece, 3 olarak etiketlenen giriş, $2^3 = 8$ değerine karşılık gelir. Giriş taşıması CI ile ve çıkış taşıması CO ile gösterilir. Kontur ile temsil edilen dijital bileşen aynı zamanda ti cari bir entegre devre olduğunda, her giriş ve çıkış boyunca IC pin numarasını yazmak gelenekselidir. Böylece, IC tipi 7483, ileri taşıma ile dört bitli bir toplayıcıdır. 16 pinli bir pakette yer almaktadır. Pin numaraları

Dokuz girdi ve beş çıkış için olanlar Şekil 10.2'de gösterilmiştir. Diğer iki pin ise güç kaynağı içindir.

Diğer bileşenlerin grafik sembollerini tanıtmadan önce, bazı terimleri gözden geçirmek gereklidir. Bölüm 2.8'de belirtildiği gibi, pozitif mantık sistemi iki sinyal seviyesinden daha pozitif olanı (H ile belirtilen) mantık 1 olarak ve daha negatif sinyal seviyesini (L ile belirtilen) mantık 0 olarak tanımlar. Negatif mantık, ters atamayı varsayar.

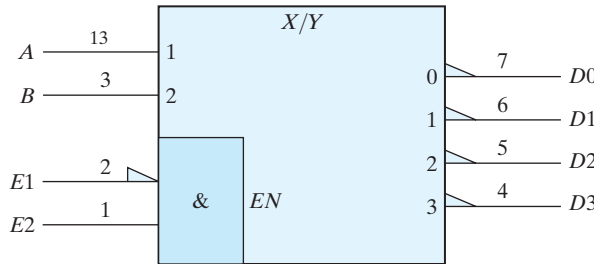
Üçüncü bir alternatif, sinyallerin tamamen H ve L değerleri açısından değerlendirildiği karışık mantık konvansiyonunu kullanmaktır. Devrede herhangi bir noktada, kullanıcı mantık polaritesini H veya L sinyaline mantık 1 atayarak tanımlamaya izin verilir. Karışık mantık notasyonu, herhangi bir giriş veya çıkış terminalinde negatif mantık polaritesini belirtmek için küçük bir dik üçgen grafik sembolü kullanır. (Bkz. Şekil 2.10(f).)

Entegre devre üreticileri, entegre devrelerin çalışmasını

H ve L sinyalleri. Bir giriş veya çıkış pozitif mantık açısından değerlendirildiğinde, *aktif yüksek* olarak tanımlanır. Negatif mantık açısından değerlendirildiğinde ise *aktif düşük* olarak tanımlanır.

Aktif-düşük girişler veya çıkışlar, küçük üçgen polarite göstergesi sembolünün varlığı ile tanınır. Pozitif mantık, tüm sistem boyunca yalnızca kullanıldığında, küçük üçgen polarite sembolü, olumsuzluğu belirten küçük daire ile eşdeğerdir. Bu kitapta, pozitif mantığı varsaydık ve mantık diyagramlarını çizerken küçük daireyi kullandık. Bir giriş veya çıkış hattı küçük daireyi içermediğinde, mantık 1 olduğunda aktif olarak tanımlıyoruz. Küçük daire sembolünü içeren bir giriş veya çıkış, mantık 0 durumunda aktif olarak kabul edilir. Ancak, standart diyagramları temsil eden tüm çizimlerde aktif-düşük atamayı göstermek için küçük üçgen polarite sembolünü kullanacağız. Bu, polarite sembolünün genellikle kullanıldığı entegre devre veri kitapları ile uyumlu olacaktır. Şekil 10.1'deki alt dört kapının, çıkış hatlarında küçük daire yerine küçük bir üçgen ile çizilebileceğini unutmayın.

Bir MSI devresi için başka bir grafik sembolü Şekil 10.3'te gösterilmektedir. Bu, IC türü 74155'in yarısını temsil eden 2'den 4'e hat çözücüsüdür. Girişler solda ve çıkışlar sağdadır. Tanımlayıcı sembol X/Y devrenin X kodundan Y koduna dönüşüğünü gösterir. Veri girişleri A ve B sırasıyla 2^0 ve 2^1 ile eşdeğer olan ikili ağırlıklar 1 ve 2 olarak atanmıştır. Çıkışlar, sırasıyla D_0 ile D_3 arasında 0'dan 3'e kadar numaralarla atanmıştır. Çözücüde bir aktif-düşük giriş E_1 ve bir aktif-yüksek giriş E_2 bulunmaktadır. Bu iki giriş, dahili bir VE kapısından geçer.



ŞEKİL 10.3

2-4 hat çözümleyici için standart grafik sembolü (IC türü 74155'in yarısı)

dekoderi etkinleştirmek için kapı. AND kapısının çıkışı EN (etkinleştir) olarak etiketlenmiştir ve E_1 düşük seviyede ve E_2 yüksek seviyede olduğunda etkinleştirilir.

10.2 NİTELİKLİ SEMBOLLER

IEEE standart grafik sembolleri, taslak ile birlikte kullanılacak nitelikli sembollerin bir listesini sağlar. Bir nitelikli sembol, elemanın genel mantık özelliklerini veya bir giriş veya çıkışın fiziksel özelliklerini belirtmek için temel taslağa eklenir. Tablo 10.1, standartta belirtilen bazı genel nitelikli sembolleri listelemektedir. Genel bir nitelikli sembol, diyagramda temsil edilen cihazın gerçekleştirdiği temel işlevi tanımlar. Dikdörtgen şekilli taslağın üst merkez pozisyonuna yerleştirilir. Kapılar, dekodörler ve toplayıcı için genel nitelikli semboller önceki diyagramlarda gösterilmiştir. Diğer semboller kendiliğinden açıklayıcıdır ve ilgili dijital elemanları temsil eden diyagramlarda daha sonra kullanılacaktır.

Girişler ve çıkışlarla ilişkili bazı nitelik sembolleri Şekil 10.4'te gösterilmiştir. Girişlerle ilişkili semboller, *sembol* etiketli sütunun sol tarafına yerleştirilmiştir. Çıkışlarla ilişkili semboller ise sütunun sağ tarafına yerleştirilmiştir. Aktif-düşük giriş veya çıkış sembolü, polarite göstergesidir. Belirtildiği gibi

Tablo 10.1
Genel Nitelik Sembolleri

Sembol	Açıklama
&	VE kapısı veya fonksiyonu
≥ 1	VEYA kapısı veya fonksiyonu
1	Buffer kapısı veya inverter
—	Özel-VE kapısı veya fonksiyonu
2k	Çift fonksiyon veya çift parite element
—	Tek fonksiyon veya tek parite elementi
X/Y	Kodlayıcı, dekodlayıcı veya kod dönüştürücü
MUX	Multiplexer
DMUX	Demultiplexer
Σ	Toplayıcı
Π	Çarpan
COMP	Büyüklik karşılaştırıcı
ALU	Aritmetik mantık birimi
SRG	Kaydırma kaydedici
CTR	Sayıcı
RCTR	Dalgalı sayıcı
ROM	Salt okunur bellek
RAM	Rastgele erişim belleği

Sembol	Açıklama
	Aktif-düşük giriş veya çıkış
	Mantık inkarı girişi veya çıkışı
	Dinamik gösterge girişi
	Üç durumlu çıkış
	Açık kollektör çıkışı
	Özel amplifikasyonlu çıkış
	Etkinleştirme girişi
	Bir depolama elemanına veri girişi
	Flip-flop girişleri
	Sağa kaydır
	Shift left
	Countup
	Countdown
	Register içeriği ikili 15'e eşittir

ŞEKİL 10.4
Giriş ve çıkışlarla ilişkili nitelik sembolleri

den, pozitif mantık varsayıldığında mantıksal inkâr ile eşdeğerdir. Dinamik giriş, flip-flop devrelerinde saat girişi ile ilişkilidir. Giriş, düşük seviyeden yüksek seviyeye geçişte aktiftir. Üç durumlu çıkış, mantıksal anlamı olmayan üçüncü bir yüksek empedans durumuna sahiptir. Devre etkinleştirildiğinde, çıkış normal 0 veya 1 mantık durumundadır, ancak devre devre dışı bırakıldığında, üç durumlu çıkış yüksek empedans durumundadır. Bu durum, açık devreye eşdeğerdir.

Açık kolektör çıkışının yüksek empedans koşulunu gösteren bir durumu vardır. Doğru mantık seviyesini üretmek için bazen harici bir direnç gereklidir. Elmas şekilli sembol, üstte (yüksek tip için) veya altta (düşük tip için) bir çubuk içerebilir. Yüksek veya düşük tip, çıkış yüksek empedans durumunda olmadığında mantık seviyesini belirtir. Örneğin, TTL tipi entegre devreler, açık kolektör çıkışları olarak adlandırılan özel çıkışlara sahiptir. Bu çıkışlar, altında bir çubuk bulunan elmas şekilli bir sembol ile tanınır. Bu, çıkışın ya yüksek empedans durumunda ya da düşük seviye durumunda olabileceğini gösterir. Bir dağıtım fonksiyonunun parçası olarak kullanıldığında, iki veya daha fazla açık kolektör NAND kapısı, ortak bir dirençle bağlandığında pozitif mantık AND fonksiyonu veya negatif mantık OR fonksiyonu gerçekleştirir.

Özel amplifikasyon ile elde edilen çıkış, özel sürüş yetenekleri sağlayan kapılarda kullanılır. Bu tür kapılar, saat sürücüler veya veri yolu odaklı vericiler gibi bileşenlerde kullanılır. *EN* sembolü, bir etkinleştirme girişini belirtir. Aktif olduğunda, tüm çıkışları etkinleştirme etkisine sahiptir. *EN* ile işaretlenmiş giriş pasif olduğunda, tüm çıkışlar devre dışı bırakılır. Flip-flop girişleri için semboller, alışıldık anlamı taşır. *D* girişi ayrıca bellek girişi gibi diğer depolama elemanlarıyla ilişkilidir.

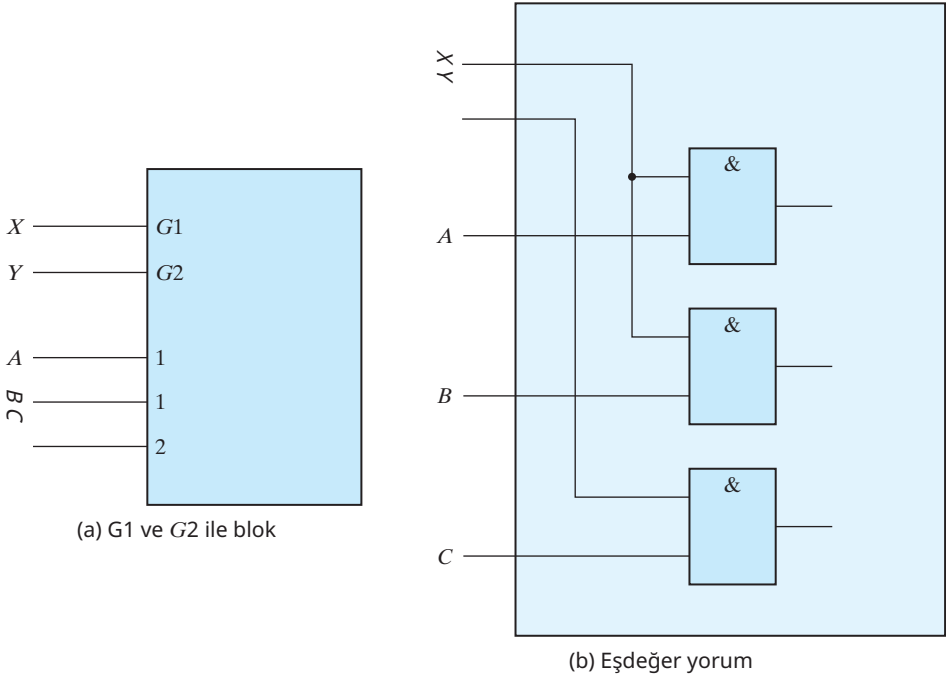
Sağa kaydırma ve sola kaydırma sembolleri, sırasıyla sağa veya sola işaret eden oklar ile gösterilir. Sayım yukarı ve sayım aşağı sayaçları için semboller, sırasıyla artı ve eksi sembolleridir. $CT = 15$ ile belirtilen bir çıkış, kayıt içeriği 15 ikili sayısına ulaştığında aktif olacaktır. Ana hat içinde gösterilen standart dışı bilgiler, köşeli parantezler içinde [böyle] yer alır.

10.3 BAĞIMLILIK NOTASYONU

Standart mantık sembollerinin en önemli yönü bağımlılık notasyonudur. Bağımlılık notasyonu, farklı girişler veya çıkışlar arasındaki ilişkiyi belirtmek için, aslında tüm elemanları ve aralarındaki bağlantıları göstermeden bir yol sağlar. Öncelikle AND bağımlılığı örneği ile bağımlılık notasyonunu göstereceğiz ve ardından bu notasyonla ilişkili diğer tüm sembolleri tanımlayacağız.

AND bağımlılığı, *G* harfi ile bir sayı ile temsil edilir. Bir diyagramda *G* ile ilişkilendirilmiş numarayla etiketlenmiş herhangi bir girdi veya çıktı, onunla AND'lanmış olarak kabul edilir. Örneğin, diyagramda bir girdi *G* 1 etiketiyle etiketlenmişse ve başka bir girdi 1 numarısıyla etiketlenmişse, o zaman *G* 1 ve 1 ile etiketlenmiş iki girdi dahi li olarak AND'lanmış olarak kabul edilir.

AND bağımlılığının bir örneği Şekil 10.5'te gösterilmektedir. (a) kısmında, iki AND bağımlılığı etiketi olan *G* 1 ve *G* 2 ile bir grafik sembolünün bir kısmını görüyoruz. 1 numarısıyla etiketlenmiş iki girdi ve 2 numarısıyla etiketlenmiş bir girdi bulunmaktadır. Eşdeğer



ŞEKİL 10.5
G (AND) bağımlılığı örneği

yorum, şeklin (b) kısmında gösterilmektedir. $G1$ ile ilişkili X girişi, A ve B ile AND'lenmiş olarak kabul edilir; bunlar 1 ile etiketlenmiştir. Benzer şekilde, Y girişi, $G2$ ile 2 arasındaki bağımlılığa uymak için C girişi ile AND'lenmiştir.

Standart, 10 diğer bağımlılığı tanımlar. Her bağımlılık bir harf sembolü ile gösterilir (EN hariç). Harf, giriş veya çıkışta görünür ve ardından bir sayı gelir. O bağımlılıktan etkilenen her giriş veya çıkış, aynı sayı ile etiketlenir. 11 bağımlılık ve bunların karşılık gelen harf tasnifi aşağıdaki gibidir:

- G VE (kapı) ilişkisini belirtir
- V VEYA ilişkisini belirtir
- N inkar (özel-VE) ilişkisini belirtir
- EN bir etkinleştirme eylemini belirtir
- C bir kontrol bağımlılığını tanımlar
- s Ayar eylemini belirtir
- R Sıfırlama eylemini belirtir
- M Mod bağımlılığını tanımlar
- A Adres bağımlılığını tanımlar

- z İç bağlantıyı gösterir
- x Kontrol edilen iletimi gösterir

V ve N bağımlılıkları, G ile benzer şekilde, Boolean VE ilişkisini belirtmek için kullanılır. EN bağımlılığı, EN nitelik sembolüne benzer, ancak ardından bir sayı gelir (örneğin, $EN\ 2$).

Yalnızca o numarayla işaretlenmiş çıktılar, ile ilişkili giriş devre dışı bırakıldığında devre dışı bırakılır. EN aktiftir.

Kontrol bağımlılığı C bir sıralı elemanda bir saat girişini tanımlamak için kullanılır yani hangi girişin onun tarafından kontrol edildiğini belirtir. Set S ve reset R bağımlılıkları bir SR flip-flop'un iç mantık durumlarını belirtmek için kullanılır. C , S , ve R bağımlılıkları Flip-flop devresi ile birlikte Bölüm 10.5'te açıklanmaktadır. Mod M bağımlılığı, birimin çalışma modunu seçen girişleri tanımlamak için kullanılır. Mod bağımlılığı, kayıtlar ve sayıcılarla birlikte Bölüm 10.6'da sunulmaktadır. Adres A bağımlılığı, bir bellek girişini tanımlamak için kullanılır. Bu, bellek birimi ile birlikte Bölüm 10.8'de tanıtılmaktadır.

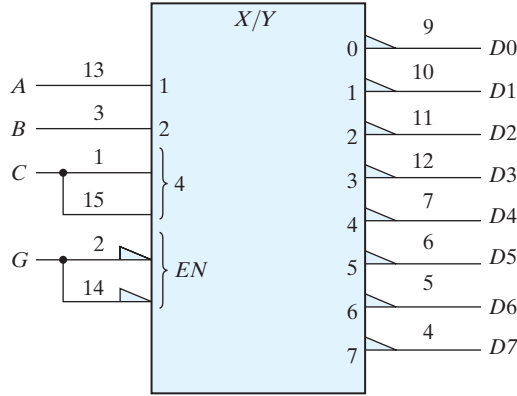
Z bağımlılığı, birim içindeki bağlantıları belirtmek için kullanılır. Bu, girişler, çıkışlar, iç girişler ve iç çıkışlar arasında herhangi bir kombinasyonda iç mantık bağlantılarının varlığını ifade eder. X bağımlılığı, bir CMOS iletim kapısındaki kontrol edilen iletim yolunu belirtmek için kullanılır.

10.4 KOMBİNASYONEL ELEMANLAR İÇİN SİMGELEME

Bu bölümdeki örnekler ve bu bölümün geri kalanı, çeşitli dijital bileşenleri grafik sembollerle temsil etmede standart kullanımı göstermektedir. Örnekler, girişler ve çıkışlardaki pin numaraları ile birlikte gerçek ticari entegre devreleri göstermektedir. Bu bölümde sunulan IC'lerin çoğu, Bölüm 9'da özetlenen önerilen deneylerle birlikte yer almaktadır.

Toplayıcı ve çözücü için grafik sembolleri Bölüm 10.2'de gösterilmiştir. IC tipi 74155, Şekil 10.6'da gösterildiği gibi 3×8 çözücü olarak bağlanabilir. (Bu çözücünün doğruluk tablosu Şekil 9.7'de gösterilmiştir.) IC'de iki C ve iki G girişi bulunmaktadır. Her çift, diyagramda gösterildiği gibi bir araya bağlanmalıdır. Etkinleştirme girişi düşük seviyede aktif durumdadır. Çıkışlar tamamen aktif düşük durumdadır. Girişler, sırasıyla 2^0 , 2^1 ve 2^2 ile eşdeğer olan ikili ağırlıklar 1, 2 ve 4 olarak atanmıştır. Çıkışlar 0'dan 7'ye kadar numaralarla atanmıştır. Girişlerin ağırlıklarının toplamı, aktif olan çıkışı belirler. Böylece, ağırlıkları 1 ve 4 olan iki giriş hattı etkinleştirildiğinde, toplam ağırlık $1 + 4 = 5$ olur ve çıkış 5 etkinleştirilir. Elbette, herhangi bir çıkışın aktif olması için EN girişi etkinleştirilmelidir.

Çözücü, *kodlayıcı* olarak adlandırılan daha genel bir bileşenin özel bir durumudur. Bir kodlayıcı, bir dizi girdi üzerinde bir ikili kod alıp, bir dizi çıkışta farklı bir ikili kod üreten bir cihazdır. Nitelik sembolü kullanmak yerine X/Y , kodlayıcı kod adıyla belirtilebilir. Örneğin, Şekil 10.6'daki 3-8 hat çözücüsü, 3 bitlik bir ikili sayıyı 0'dan 7'ye kadar 8 sekizli değere dönüştürdüğü için BIN/OCT adıyla sembolize edilebilir.



ŞEKİL 10.6

IC tipi 74155, 3×8 kodlayıcı olarak bağlanmıştır

Çoklayıcının grafik sembolünü göstermeden önce, AND bağımlılığının bir varyasyonunu göstermek gereklidir. AND bağımlılığı bazen G^0 gibi bir kısayol notasyonu ile temsil edilir. Bu sembol, 0'dan 7'ye kadar sekiz AND bağımlılığı sembolünü temsil eder:

$$G_0, G_1, G_2, G_3, G_4, G_5, G_6, G_7$$

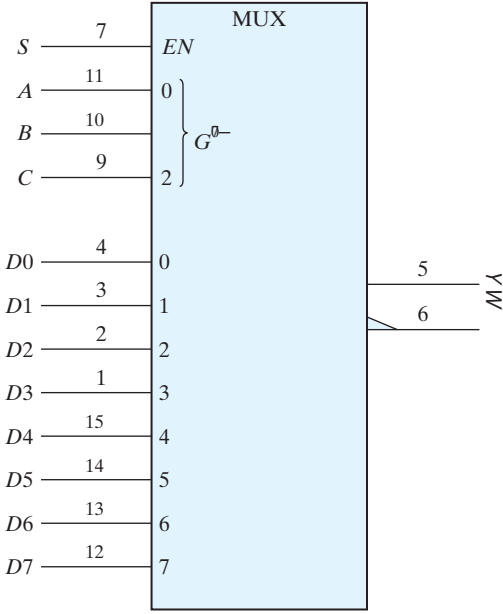
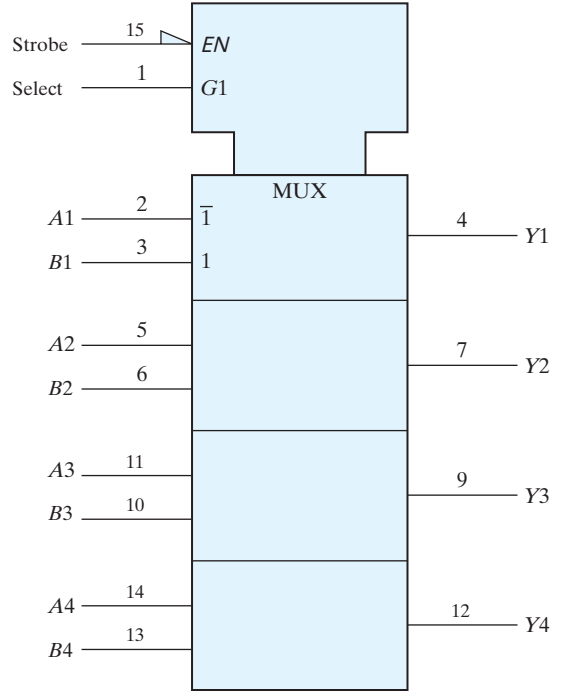
Herhangi bir anda, sekiz AND kapısından yalnızca biri aktif olabilir. Aktif AND kapısı, G sembolü ile ilişkili girişlerden belirlenir. Bu girişler, 2'nin kuvvetlerine eşit ağırlıklarla işaretlenmiştir. Az önce listelenen sekiz AND kapısı için, ağırlıklar sırasıyla 0, 1 ve 2'dir ve bunlar 2^0 , 2^1 ve 2^2 sayılarıyla ilişkilidir.

Herhangi bir anda aktif olan AND kapısı, aktif girişlerin ağırlıklarının toplamından belirlenir. Dolayısıyla, eğer girişler 0 ve 2 aktifse, o zaman aktif olan AND kapısının numarası $2^0 + 2^2 = 5$ 'tir. Bu, G_5 'in aktif olmasını sağlar ve diğer yedi AND kapısını inaktif hale getirir.

8×1 çoklayıcısı için standart grafik sembolü Şekil 10.7 (a)'da gösterilmiştir. Nitelikli sembol MUX, cihazı bir çoklayıcı olarak tanımlar. Blok içindeki semboller standart notasyonun bir parçasıdır, ancak dışarıda işaretlenmiş semboller kullanıcı tanımlı sembollerdir. 741551 IC'nin fonksiyon tablosu Şekil 9.9'da bulunabilir. AND bağımlılığı,

G^0 ile işaretlenmiştir ve parantez içinde yer alan girişlerle ilişkilidir.

Bu girdilerin ağırlıkları 0, 1 ve 2'dir. Aslında bunlar, seçim girdileri olarak adlandırdığımız şeylerdir. Sekiz veri girişi 0'dan 7'ye kadar numaralarla işaretlenmiştir. G sembolü ile ilişkili aktif girdilerin net ağırlığı, aktif olan veri girişindeki numarayı belirtir. Örneğin, eğer seçim girdileri $CBA = 110$ ise, o zaman G ile ilişkili 1 ve 2 numaralı girdiler aktiftir. Bu, $2^2 + 2^1 = 6$ için AND bağımlılığına sayısal bir değer verir, bu da G_6 'yı aktif hale getirir. G_6 , veri girişi numarası 6 ile AND'landığı için, bu girişi aktif hale getirir. Böylece, çıktı, etkinleştirme girişi aktif olduğu sürece veri girişi D_6 ile eşit olacaktır.

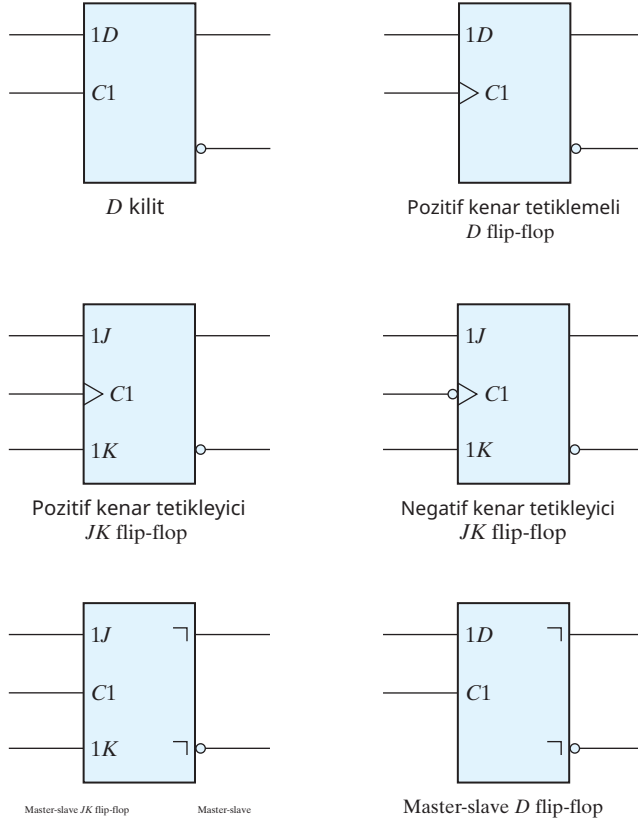
(a) IC tipi 74151 8×1 MUX(b) IC türü 74157 dördlü 2×1 MUX**ŞEKİL 10.7**

Çoklayıcılar için grafik semboller

Şekil 10.7(b), işlev tablosu Şekil 9.17'de listelenen dördlü 2×1 çoklayıcı IC türü 74157'yi temsil etmektedir. Enable ve seçim girişleri, dört çoklayıcının hepsi için ortaktır. Bu, diyagramın üst kısmındaki girintili kutu ile standart notasyonda gösterilmektedir; bu, ortak kontrol bloğu'nu temsil eder. Ortak kontrol bloğuna girişler, diyagramın tüm alt bölümlerini kontrol eder. Ortak enable girişi $\overline{EN}$ düşük seviyede aktif olduğunda etkindir. AND bağımlılığı, G 1, her çoklayıcı bölümünde hangi girişin aktif olduğunu belirler. $G = 0$ olduğunda, 1 ile işaretlenmiş A girişleri aktiftir. $G = 1$ olduğunda, B 1 ile işaretlenmiş girişler aktiftir. Aktif girişler, $\overline{EN}$ aktif olduğunda karşılık gelen çıkışlara uygulanır. Giriş sembollerinin 1 ve 1'in yalnızca üst bölümde işaretlendiğini ve her bölümde tekrarlanmaktan kaçınıldığını unutmayın.

10.5 TERSİNE DÖNÜŞÜCÜLER İÇİN SİMGE

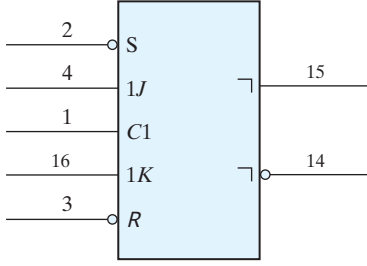
Farklı türdeki tersine dönüştürücülerin standart grafik sembolleri Şekil 10.8'de gösterilmektedir. Bir tersine dönüştürücü, solda girişler ve sağda çıkışlar olan dikdörtgen şeklinde bir blok ile temsil edilir. Bir çıkış, tersine dönüştürücünün normal durumunu belirtir ve

**ŞEKİL 10.8**

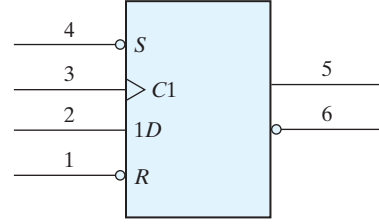
Flip-flop'lar için standart grafik semboller

küçük daireli bir olumsuzluk sembolü (veya polarite göstergesi) ile diğer çıktı, tamamlayıcı çıktıyı belirtir. Grafik semboller, üç tür flip-flop arasında ayrım yapar: the *D* latch, whose internal construction is shown in Fig. 6.5; the master–slave flip-flop, shown in Fig. 6.9; and the edge-triggered flip-flop, introduced in Fig. 6.12. The graphic symbol for the *D* latch or *D* flip-flop has inputs *D* and *C* indicated inside the block. The graphic symbol for the *JK* flip-flop has inputs *J*, *K*, and *C* inside. The notation *C* 1, *D*, 1 *J*, ve 1 *K* kontrol bağımlılığının örnekleridir. Girdi *C* 1, girdi 1 *D* üzerinde kontrol sağlar. *D* flip-flop ve 1 *J* ile 1 *K* girişleri bir *JK* flip-flop'ta.

D latch'in 1 *D* ve *C* 1 girişleri dışında başka sembolleri yoktur. Kenar tetiklemeli flip-flop, kontrol bağımlılığı *C* 1'in önünde ok başı şeklinde bir sembole sahiptir ve dinamik bir girişi belirtir. Dinamik gösterge sembolü, flip-flop'un giriş saat darbelerinin pozitif kenar geçişine yanıt verdiğini belirtir. Dinamik gösterge ile birlikte blok dışında bulunan küçük bir daire, flip-flop'u tetiklemek için negatif kenar geçişini belirtir. Master-slave, bir darbe tetiklemeli flip-flop olarak kabul edilir ve



(a) Yarım 7476 JK flip-flop



(b) Yarım 7474 D flip-flop

ŞEKİL 10.9

Doğrudan ayar ve sıfırlama ile IC flip-flop'lar

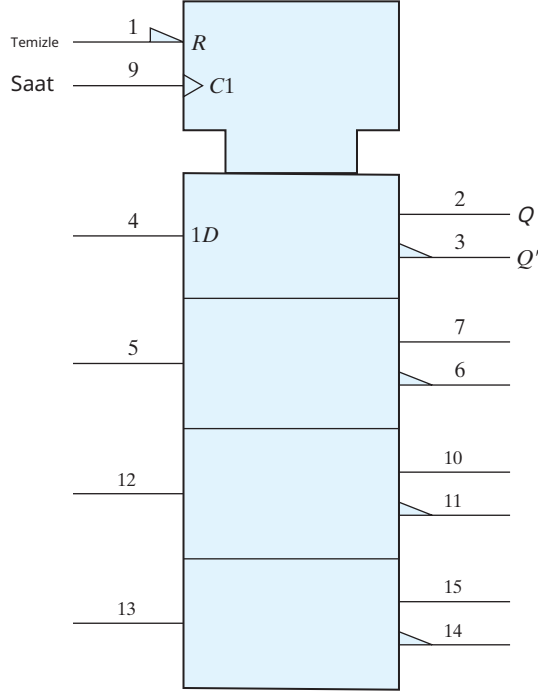
Çıkışların önünde ters çevrilmiş L sembolü ile böyle gösterilir. Bu, çıkış sinyalinin darbenin düşen kenarında değiştiğini göstermek içindir. Master-slave flip-flop'un dinamik gösterge olmadan çizildiğini unutmayın.

Entegre devre paketlerinde mevcut flip-flop'lar, flip-flop'u asenkron olarak ayarlamak ve sıfırlamak için özel girişler sağlar. Bu girişler genellikle doğrudan ayar ve doğrudan sıfırlama olarak adlandırılır. Saat sinyaline ihtiyaç duymadan sinyalin negatif seviyesinde çıkışı etkilerler. Doğrudan ayar ve sıfırlama ile bir master-slave JK flip-flop'un grafik sembolü Şekil 10.9 (a)'da gösterilmiştir. Notasyonlar C 1, $1J$ ve $1K$ kontrol bağımlılığını temsil eder, bu da C 1'deki saat girişinin $1J$ ve $1K$ girişlerini kontrol ettiğini gösterir. S ve R harflerin önünde 1 yoktur ve bu nedenle C 1'deki saat tarafından kontrol edilmezler. S ve R girişlerinin, mantık-0 seviyesinde aktif olduğunu belirtmek için giriş hatları boyunca küçük bir dairesi vardır. 7476 flip-flop'un fonksiyon tablosu Şekil 9.12'de gösterilmiştir.

Pozitif kenar tetiklemeli D flip-flop'un doğrudan ayar ve sıfırlama ile ilgili grafik sembolü Şekil 10.9 (b)'de gösterilmiştir. Saatin pozitif kenar geçişi, giriş C 1'i kontrol eder ve giriş $1D$ 'yi etkiler. S ve R girişleri saatten bağımsızdır. Bu, işlev tablosu Şekil 9.13'te listelenen IC tipi 7474'tür.

10.6 KAYITLAR İÇİN SİMGELER

Bir kaydedici için standart grafik sembol, ortak saat girişi olan bir grup flip-flop için kullanılan sembolle eşdeğerdir. Şekil 10.10, ortak saat ve temizleme girişlerine sahip dört D flip-flop'tan oluşan IC tipi 74175'in standart grafik sembolünü göstermektedir. Saat girişi C 1 ve temizleme girişi R ortak kontrol bloğunda görünmektedir. Ortak kontrol bloğuna giden girişler, diyagramın alt kesimlerinde bulunan her bir elemana bağlanmıştır. Notasyon C 1, tüm $1D$ girişler. Böylece, her flip-flop ortak saat girişine göre tetiklenir. Dinamik input sembolü ile ilişkili C 1, flip-flopların giriş saatinin pozitif kenarında tetiklendiğini gösterir. Ortak R girişi, girişi düşük seviyede olduğunda tüm flip-flopları sıfırlar. $1D$ sembolü yalnızca üst bölümde bir kez yerleştirilmiştir.



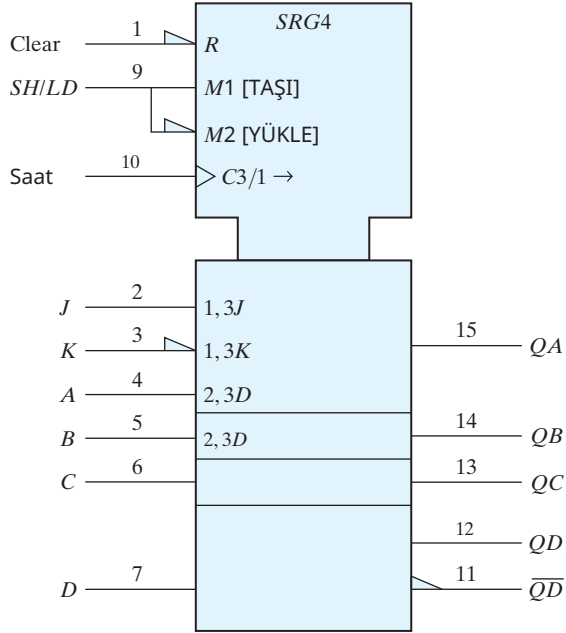
ŞEKİL 10.10

IC türü 74175 dörtlü flip-flop için grafik sembolü

her bölümde tekrarlamak. Bu diyagramdaki flip-flop'ların tamamlayıcı çıkışları, inkâr sembolü yerine polarite sembolü ile işaretlenmiştir.

Paralel yüklemeli bir kaydırma kaydının standart grafik sembolü Şekil 10.11'de gösterilmiştir. Bu, işlev tablosu Şekil 9.16'da bulunan IC türü 74195'tir. Kaydırma kaydı için nitelik sembolü *SRG* ve ardından aşamaların sayısını belirten bir sayı gelir. Böylece, *SRG 4*, dört bitlik bir kaydırma kaydını belirtir. Ortak kontrol bloğu, kaydırma ve yükleme işlemleri için sırasıyla *M 1* ve *M 2* olmak üzere iki mod bağımlılığına sahiptir. IC'nin, *SH LD* (kaydırma/yükleme) olarak etiketlenmiş tek bir girişi olduğunu ve bunun iki moda göstermek için iki hat olarak ayrıldığını unutmayın. *M 1*, *SH LD* girişi yüksek olduğunda aktiftir ve *M 2*, *SH LD* girişi düşük olduğunda aktiftir. *M 2*, giriş hattındaki polarite göstergesi ile aktif düşük olarak tanınır. Bu semboldeki kuralı not edin: Pin 9'da aslında tek bir giriş olduğunu kabul etmeliyiz, ancak iki moda atamak için iki parçaya ayrılmıştır, *M 1* ve *M 2*. Kontrol bağımlılığı *C 3*, saat girişi içindir. *C 3* girişiindeki dinamik sembol, flip-flop'ların saat sinyalinin pozitif kenarında tetiklendiğini gösterir. *C 3*'ü takip eden /1 → sembolü, kayıt modü *M 1* aktif olduğunda sağa ve ya aşağı doğru kaydırıldığını belirtir.

Ortak kontrol bloğunun altındaki dört bölüm, dört flip-flop'u temsil eder. Flip-flop *QA* üç girişi vardır: İkisi seri (kaydırma) işlemi ile ilişkilidir ve biri de paralel (yükleme) işlemi ile *QD* arasına neden olur. Paralel girişin yalnızca birinci ve ikinci bölümlerde etiketlendiğini unutmayın.



ŞEKİL 10.11

Paralel yüklemeli bir kaydırma kaydedicisinin grafik sembolü, IC tipi 74195

Seri giriş etiketi 1, 3 J flip-flop QA girişinin aktif olduğunu gösterir, bu da $M1$ (kaydırma) aktif olduğunda ve $C3$ pozitif bir saat geçişi yaptığında geçerlidir. Diğer seri giriş etiketi 1, 3 K giriş hattında, bir JK flip-flop'taki giriş K 'nın tamamlayıcısına karşılık gelen bir polarite sembolü taşır. QA girişinin üçüncü girişi ve diğer flip-flop'ların girişleri paralel giriş verileri içindir. Her giriş, 2, 3 D etiketi ile gösterilir. 2, $M2$ (yükleme) içindir ve 3, saat $C3$ içindir. Pin numarası 9'daki giriş düşük seviyedeysse, $M1$ aktiftir ve $C3$ 'teki saat pozitif geçişi, dört girişten A ile D arasındaki paralel aktarımı dört flip-flop'a, QA ile QD arasına neden olur. Paralel girişin yalnızca birinci ve ikinci bölümlerde etiketlendiğini unutmayın.

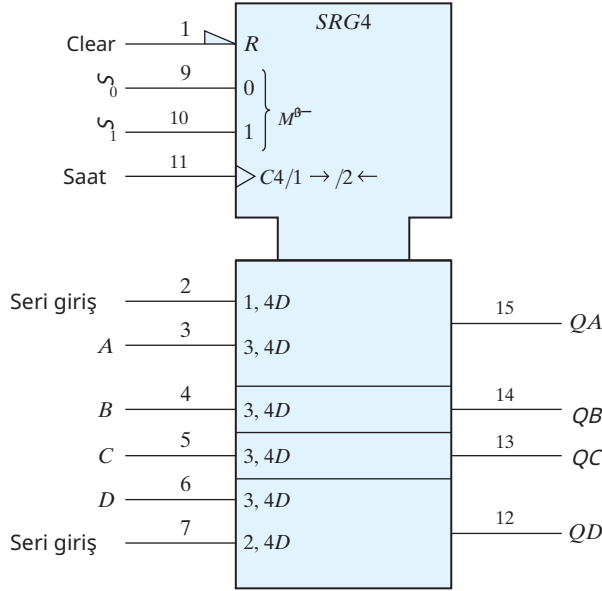
Aşağıdaki diğer iki bölümde olduğu varsayılmaktadır.

Şekil 10.12, paralel yüklemeli iki yönlü kaydırma kaydedicisinin grafik sembolünü göstermektedir, IC tipi 74194. Bu IC için işlev tablosu Şekil 9.19'da listelenmiştir. Ortak kontrol bloğu, tüm flip-flop'ları asenkron olarak 0'a sıfırlamak için bir R girişi göstermektedir. Mod seçimi iki girişi vardır ve mod bağımlılığı M , 0 ile 3 arasında ikili değerler alabilir.

Bu, $M03$ sembolü ile gösterilmektedir, bu da $M0$, $M1$, $M2$, $M3$ 'ü temsil eder ve çoklayıcılardaki G bağımlılığı için notasyona benzerdir. Saatle ilişkili sembol

$$C4/1 \rightarrow /2 \leftarrow$$

$C4$, saat için kontrol bağımlılığıdır. $/1$ S sembolü, kaydedicinin mod $M1$ ($S1S0 = 10$) olduğunda sağa (bu durumda aşağıya) kaydırıldığını belirtir. $/2$ d sembolü



ŞEKİL 10.12

Paralel yüklemeli iki yönlü kaydırma kaydedici için grafik sembol, IC türü 74194

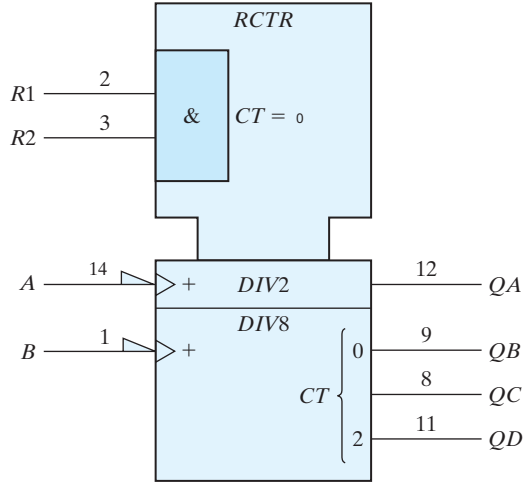
modun $M \cdot 2S(S_0 = 10)$ olduğunda, kaydın sola (bu durumda yukarı) kaydırıldığını gösterir.

Sayfa 90 derece saat yönünün tersine döndüğünde sağ ve sol yönler elde edilir.

Aşağıdaki ortak kontrol bloğunun altındaki bölümler dört flip-flop'u temsil eder. İlk flip-flop, sağa kaydırma için $1, 4D \pmod{M-1}$, saat $C-4$, girdi D ile gösterilen bir seri girişi vardır. Son flip-flop, sola kaydırma için $2, 4D \pmod{M-2}$, saat $C-4$, girdi D ile gösterilen bir seri girişi vardır. Tüm dört flip-flop'un $3, 4D \pmod{M-3}$, saat $C-4$, girdi D ile gösterilen bir paralel girişi vardır. Böylece, $M-3 \cdot S_1 \cdot S_0 = 11$ paralel yükleme içindir. Kalan mod $M-0 \cdot S_1 \cdot S_0 = 00$ girdi etiketlerinde yer almadığı için çıkışlar üzerinde etkisi yoktur.

10.7 SAYACIN SİMGELEME

İkili dalgalı sayacın standart grafik sembolü Şekil 10.13'te gösterilmiştir. Dalgalı sayacın nitelikli sembolü $RCTR$ 'dir. $DIV 2$, tek flip-flop QA 'dan elde edilen bölücü-2 devresini temsil eder. $DIV 8$ tanımı, diğer üç flip-flop'tan elde edilen bölücü-8 sayacı içindir. Diyagram, iç devre şeması Şekil 9.2'de gösterilen IC tipi 7493'ü temsil eder. Ortak kontrol bloğunda, $R-1$ ve $R-2$ girişlerine sahip bir iç AND kapısı bulunmaktadır. Bu girişlerin her ikisi de 1'e eşit olduğunda, sayacın içi riği sıfıra gider. Bu, ile gösterilmektedir.



ŞEKİL 10.13

Ripple sayıcı için grafik sembol, IC tipi 7493

sembol $CT=0$. Sayım girişi tüm flip-flop'ların saat girişlerine gitmediğinden, $CT=0$ etiketi yoktur ve bunun yerine, sayım artırma işlemini belirtmek için $+$ sembolü kullanılır. $+$ sembolünün yanındaki dinamik sembol ve giriş

hat boyunca yer alan polarite sembolü, sayımın giriş sinyalinin negatif kenar geçişi ile etkilendiğini gösterir. Çıkıştaki 0'dan 2'ye kadar olan bit gruplaması, 2'nin kuvvetlerine karşılık gelen ağırlık değerlerini temsil eder. Böylece, $0, 2^0 = 1$ değerini ve $2, 2^2 = 4$ değerini temsil eder.

Dört bitli sayıcı için standart grafik sembolü, paralel yükleme ile IC tipi 74161, Şekil 10.14'te gösterilmiştir. Senkron sayıcı için nitelikli sembol CTR ve ardından $DIV 16$ (16 ile böl) sembolü gelir; bu, sayıcının döngü uzunluğunu verir. Pin 9'da tek bir yük girişi vardır ve bu iki moda, $M 1$ ve $M 2$ 'ye ayrılır. $M 1$, pin 9'daki yük girişi düşük olduğunda aktiftir ve $M 2$, pin 9'daki yük girişi yüksek olduğunda aktiftir.

$M 1$, giriş hattındaki polarite göstergesine göre aktif düşük olarak tanınır. Sayım-enable girişleri G bağımlılıklarını kullanır. $G 3$, T girişine ve $G 4$, sayım enable'in P girişine ilişkilidir. Saatle ilişkili etiket

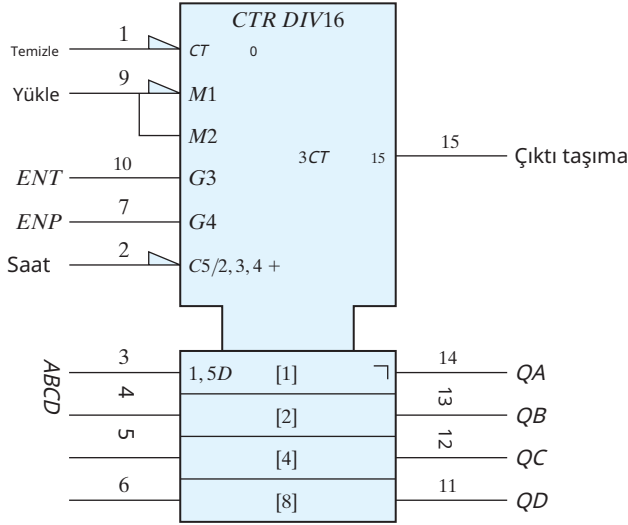
$$C5/2, 3, 4 +$$

Bu, devrenin $M 2, G 3$ ve $G 4$ aktif olduğunda (yük = 1, $ENT=1$ ve $ENP=1$) ve saat $C 5$ pozitif bir geçiş yaptığında yukarı saydığı anlamına gelir.

Bu durum, Şekil 9.15'te listelenen 74161'in fonksiyon tablosunda belirtilmiştir. Paralel girişlerin etiketi 1, 5 D , D girişlerinin $M 1$ aktif olduğunda (yük = 0) ve saat pozitif bir geçiş yaptığında aktif olduğunu belirtir. Çıkış taşıma, etiketle belirtilmiştir

$$3CT = 15$$

Bu, çıkış taşımanın aktif olduğu (1'e eşit) anlamına gelir, eğer $G 3$ aktiftir ($ENT=1$) ve sayacın içeriği (CT) 15'tir (ikili 1111). Çıkışların



ŞEKİL 10.14

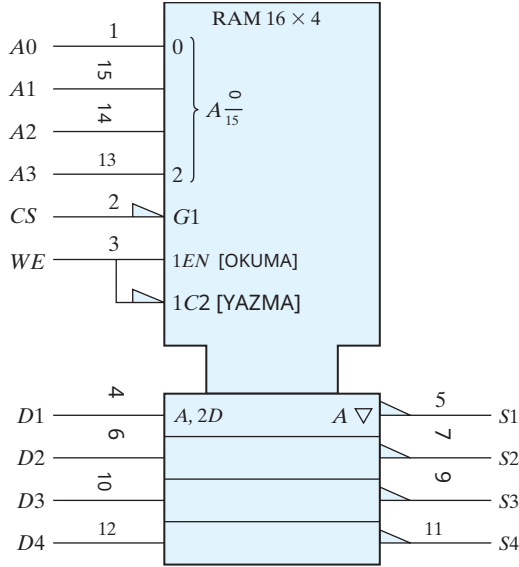
Paralel Yüklemeli 4-Bit İkili Sayıcı için Grafik Sembol, IC Tipi 74161

ters bir L sembolüne sahiptir, bu da tüm flip-flop'ların master-slave tipinde olduğunu gösterir. C 5 girişindeki polarite sembolü, giriş saat sinyali için ters bir darbe belirtir. Bu, ana cihazın saat darbesinin negatif geçişinde tetiklendiği ve kölenin pozitif geçişte durum değiştirdiği anlamına gelir. Böylece, çıkış saat darbesinin pozitif geçişinde değişir. IC tipi 74LS161 (düşük güç Schottky versiyonu) pozitif kenar tetiklemeli flip-flop'lara sahiptir.

10.8 RAM İÇİN SİMGE

Rastgele erişim belleği (RAM) 74189 için standart grafik sembolü Şekil 10.15'te gösterilmiştir. RAM nitelik sembolünü takip eden 16×4 sayıları, kelime sayısını ve kelime başına bit sayısını belirtir. Ortak kontrol bloğu dört adres hattı ve iki kontrol girişi ile gösterilmiştir. Kelimenin her bir biti, bir giriş ve çıkış veri hattı ile ayrı bir bölümde gösterilmiştir. Adres bağımlılığı A , belleğin adres girişlerini tanımlamak için kullanılır. Adresten etkilenen veri girişleri ve çıkışları A harfi ile etiketlenmiştir. 0'dan 3'e kadar olan bit grubu, $A0$ 'dan $A15$ 'e kadar olan ikili adresi sağlar. Ters üçgen, üç durumlu çıkışları belirtir. Polarite sembolü, çıkışların terisini belirtir.

Belleğin çalışması bağımlılık notasyonu ile belirtilmiştir. RAM grafik sembolü dört bağımlılık kullanır: A (adres), G (VE), EN (etkinleştir), ve C (kontrol). G 1, 1 EN ve 1 C 2 ile VE'li olarak değerlendirilmelidir çünkü G 1'in harfinin ardından 1 vardır ve diğer ikisi etiketlerinde 1 taşımaktadır. EN bağımlılığı kullanılmaktadır.



ŞEKİL 10.15
16×4 RAM için grafik sembolü, IC türü 74189

veri çıkışını kontrol eden bir etkin giriş tanımlamak için. Bağımlılık $C2$, $2D$ etiketinin belirttiği gibi girişleri kontrol eder. Bu nedenle, bir yazma işlemi için $G1$ ve $1C2$ bağımlılığı ($CS=0$), $C2$ ve $2D$ bağımlılığı ($WE=0$), ve A bağımlılığı, dört adres girişindeki ikili adresi belirtir. Bir okuma işlemi için, $G1$ ve $1EN$ bağımlılıkları ($CS=0$, $WE=1$) ve çıktılar için A bağımlılığına sahibiz. Bu bağımlılıkların yorumlanması, 74189 RAM'in fonksiyon tablosunda listelenen bellek işlemini sonuçlandırır (bkz. Web Arama Konuları).

PROBLEMLER

10.1

Şekil 9.1, pin ataması ile çeşitli küçük ölçekli entegrasyon devrelerini göstermektedir. Bu bilgiyi kullanarak, 7400, 7404 ve 7486 entegre devreleri için dikkdörtgen şeklindeki grafik sembollerini çizin.

10.2

Aşağıdakileri kendi kelimelerinizle tanımlayın:

- (a) Pozitif ve negatif mantık.
- (b) Aktif yüksek ve aktif düşük.
- (c) Polarite göstergesi.
- (d) Dinamik gösterge.
- (e) Bağımlılık notasyonu.

10.3

Üç Boolean bağımlılığını içeren bir grafik sembolü örneği gösterin— G , V , and N . Eşdeğer yorumu çizin.

10.4

BCD'den ondalık kod çözücüsünün grafik sembolünü çizin. Bu, 4 girişi ve 10 çıkışı olan bir kod çözücüye benzer.

10.5

Üç etkinleştirme girişi olan bir ikili-sekizli çözücünün grafik sembolünü çizin, E_1, E_2 , and E_3 . Devre, $E_1=1$, $E_2=0$ ve $E_3=0$ olduğunda etkinleştirilir (pozitif mantık varsayımıyla).

10.6

Ortak seçim girişlerine sahip çift 4-e-1-hat çoklayıcıların grafik sembolünü çizin ve her çoklayıcı için ayrı bir etkinleştirme girişi ekleyin.

10.7

Aşağıdaki flip-floplar için grafik sembolünü çizin:

- (a) Negatif kenar tetiklemeli D flip-flop. (b) Master-slave JK flip-flop.
(c) Pozitif kenar tetiklemeli T flip-flop.

10.8

Standart grafik sembollerle kullanıldığında ortak kontrol bloğunun işlevini açıklayın.

10.9

Yükleme girişi için M 1 ve saat için C 2 etiketini kullanarak paralel yüklemeli dört bitlik bir kaydedicinin grafik sembolünü çizin.

10.10

Şekil 10.12'deki standart grafik diyagramında kullanılan tüm sembolleri açıklayın.

10.11

Yukarı-aşağı senkron ikili sayıcı için grafik sembolünü çizin, mod girişi (yukarı veya aşağı) ve sayım etkinleştirme girişi ile G bağımlılığı. Yukarı sayım ve aşağı sayım için çıkış taşımalarını gösterin.

10.12

256×1 RAM'in grafik sembolünü çizin. Üç durumlu çıkışlar için sembolü dahil edin.

REFERANSLAR

1.

IEEE Mantık Fonksiyonları için Grafik Sembolleri (ANSI/IEEE Std. 91-1984). 1984. New York: Elektrik ve Elektronik Mühendisleri Enstitüsü.

2.

Kampel, I. 1985. *Yeni Mantık Sembollerine Pratik Bir Giriş*. Boston: Butterworth.

3.

MANN, F. A. 1984. *Yeni Mantık Sembollerinin Açıklaması*. Dallas: Texas Instruments.

4.

TTL Veri Kitabı, Cilt 1. 1985. Dallas: Texas Instruments.

WEB ARAMA KONULARI

İki yönlü kaydırma kaydedici

Üç durumlu inverter

Üç durumlu tampon

Evrensel kaydırma kaydedici

7483 toplayıcı

74151 çoklayıcı

74155 çözücü

74157 çoklayıcı

7476 flip-flop

7474 flip-flop

74161 flip-flop

74194 kaydırma kaydedici

74175 dörtlü flip-flop

74195 kaydırma kaydedici

7494 sayaç

74161 sayaç

74LS161 flip-flop

74189 RAM

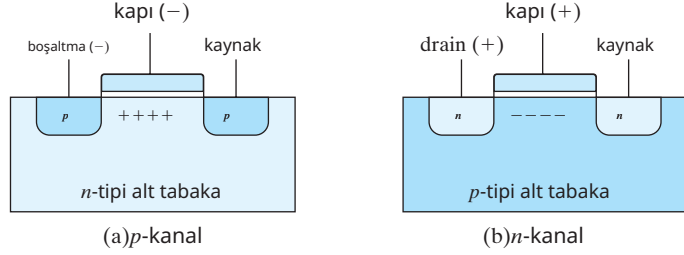
BCD'den ondalık çözücü

Rastgele erişim belleği

Yarı İletkenler ve CMOS Entegre Devreler

Yarı iletkenler, saf bir silikon kristalinin ince bir dilimini, silikonun kristal yapısına göre n ispeten kolayca uyum sağlayan küçük bir katkı maddesi ile doping yaparak oluşturulur. Katkı maddeleri, üç değerlik elektronu veya beş değerlik elektronu olup olmamasına göre farklılaştırılır. Bir silikon kristal yapısı, her silikon atomunun dört değerlik elektronunu en yakın dört komşusuyla paylaşacak şekilde düzenlenmiştir ve böylece değerlik yapısını tamamlar. Beş değerlik elektronu olan bir katkı maddesi atomu, n-tip katkı maddesi olarak adlandırılır, kristalin fiziksel yapısına uyum sağlar, ancak beşinci elektronları bağlı yapıdaki ana atomları tarafından yalnızca gevşek bir şekilde tutulur. Sonuç olarak, uygulanan bir elektrik alanı, bu tür elektronların akım olarak akmasına neden olabilir. Öte yandan, yalnızca üç değerlik elektronu olan bir katkı maddesi atomu, p-tip katkı maddesi, boş bir değerlik alanına sahiptir. Uygulanan elektrik alanının etkisi altında, bağlı yapıdaki komşu bir silikon atomundan bir elektron, ev sahibi atomundan sıçrayarak boş bir katkı maddesi alanını doldurabilir ve ev sahibi atomunda bir boşluk bırakır. Bu göç, elektronların boşluktan boşluğa sıçraması olarak görselleştirilir ve bir akım oluşturur.

Elektrik akımı, negatif yük taşıyıcıları olan elektronların hareketinden kaynaklanır. Ancak, akım, gelenek gereği, akış yönünün tersine ölçülür—Benjamin Franklin dönemlerinden beri. (Akımı, negatif yük taşıyan bir elektronun ters yönünde hareket eden eşdeğer pozitif yük olarak düşünün). Delikler akım yönünde hareket eder, ancak elektronların temel fiziksel hareketi ters yöndedir. Termal titreşim, her iki tür yük taşıyıcısının da bir yarı iletken içinde bulunmasına neden olur. Eğer çoğunluk taşıyıcısı bir delikse, cihazın p-tip cihaz olduğu söylenir; eğer çoğunluk taşıyıcısı bir elektron ise, cihazın n-tip cihaz olduğu söylenir. Bipolar transistörler her iki tür taşıyıcıya da dayanır. Metal-oksit silikon yarı iletkenleri, ya bir elektron ya da bir delik olan bir çoğunluk taşıyıcısına dayanır, ancak her ikisine birden dayanmaz. Doping türü ve göreceli miktarı, bir yarı iletken malzemenin türünü belirler.

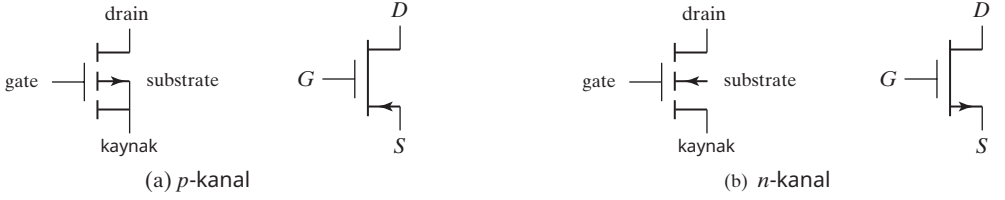


ŞEKİL A.1
MOS transistörünün temel yapısı

Bir metal-oksit yarı iletken (MOS) transistörünün temel yapısı Şekil A.1'de gösterilmiştir. *p*-kanal MOS transistörü, *n*-tip silikon malzemenin hafif doplanmış bir alt tabakasından oluşur. İki bölge, *p*-tip safsızlıklarla yoğun bir şekilde doplanarak *kaynak* ve *dren* oluşturmak için bir difüzyon süreci ile işlenir. Kaynak terminali, dış bir devreye yük taşıyıcıları sağlar; dren terminali, devreden yük taşıyıcılarını çıkarır. İki *p*-tip bölge arasındaki alan, *kanal* olarak hizmet eder. En basit formunda, kapı, kanaldan silikon dioksit ile yalıtılmış bir dielektrik ile ayrılmış bir metal plattır. Kapı terminalinde (alt tabana göre) negatif bir voltaj, kanalda indüklenmiş bir elektrik alanı oluşturur ve bu alan, alt tabandan *p*-tipi taşıyıcıları (delikler) çeker. Negatif voltajın büyüklüğü arttıkça, kapının altındaki bölgede daha fazla pozitif taşıyıcı birikir, iletkenlik artar ve kaynak ile drenaj arasında bir voltaj farkı sağlandığı sürece akım akabilir.

MOS yapılarının dört temel türü vardır. Kanal, çoğunluk taşıyıcılarının delikler mi yoksa elektronlar mı olduğuna bağlı olarak *p*-tipi veya *n*-tipi olabilir. Çalışma modu, sıfır kapı voltajındaki kanal bölgesinin durumuna bağlı olarak artırma veya boşaltma olabilir. Kanal başlangıçta *p*-tipi safsızlık ile hafifçe doplanmışsa (bu durumda 'difüze kanal' olarak adlandırılır), sıfır kapı voltajında iletken bir kanal vardır ve cihazın boşaltma modunda çalıştığı söylenir. Bu modda, kanal bir uygulanan kapı alanı tarafından boşaltılmadığı sürece akım akar. Kapının altındaki bölge başlangıçta yüklenmemişse, akım akmadan önce kapı alanı tarafından bir kanal indüklenmelidir. Böylece, kanal akımı kapı voltajı ile artırılır ve bu tür bir cihazın artırma modunda çalıştığı söylenir.

Kaynak, çoğunluk taşıyıcılarının cihaza girdiği terminaldir. Eğer çoğunluk taşıyıcısı bir delik ise (*p*-tipi kanal), kaynak terminali devreye akım sağlar; eğer çoğunluk taşıyıcısı bir elektron ise (*n*-tipi kanal), kaynak devreden akım çeker. Boşaltma, çoğunluk taşıyıcılarının cihazdan çıktığı terminaldir. Bir *p*-kanal MOS'ta, kaynak terminali alt tabakaya bağlıdır ve boşaltma terminaline negatif bir voltaj uygulanır. Kapı voltajı bir eşik voltajı V_T (yaklaşık -2 V) üzerinde olduğunda, kanalda akım akmaz ve boşaltma-kaynağı yolu açık devre gibidir. Kapı voltajı V_T 'ni yeterince negatif bir değerin altına düşüğünde, bir kanal oluşur ve *p*-tip taşıyıcılar kaynaktan boşaltmaya akar. *p*-tip taşıyıcılar pozitifdir ve kaynaktan boşaltmaya doğru pozitif bir akım akışına karşılık gelir.



ŞEKİL A.2

MOS transistörleri için semboller

n -kanal MOS'ta, kaynak terminali alt tabakaya bağlanır ve pozitif bir voltaj dren terminaline uygulanır. Kapı voltajı eşik voltajının altında olduğunda V_T (yaklaşık 2 V), kanalda akım akmaz. Kapı voltajı, kanalı oluşturmak için yeterince pozitif olduğunda V_T üzerinde, n -tip taşıyıcılar kaynaktan drene akar. n -tip taşıyıcılar negatiftir ve dren ile kaynak arasında pozitif akım akışına karşılık gelir. Eşik voltajı, kullanılan belirli işleme bağlı olarak 1 ile 4 V arasında değişebilir.

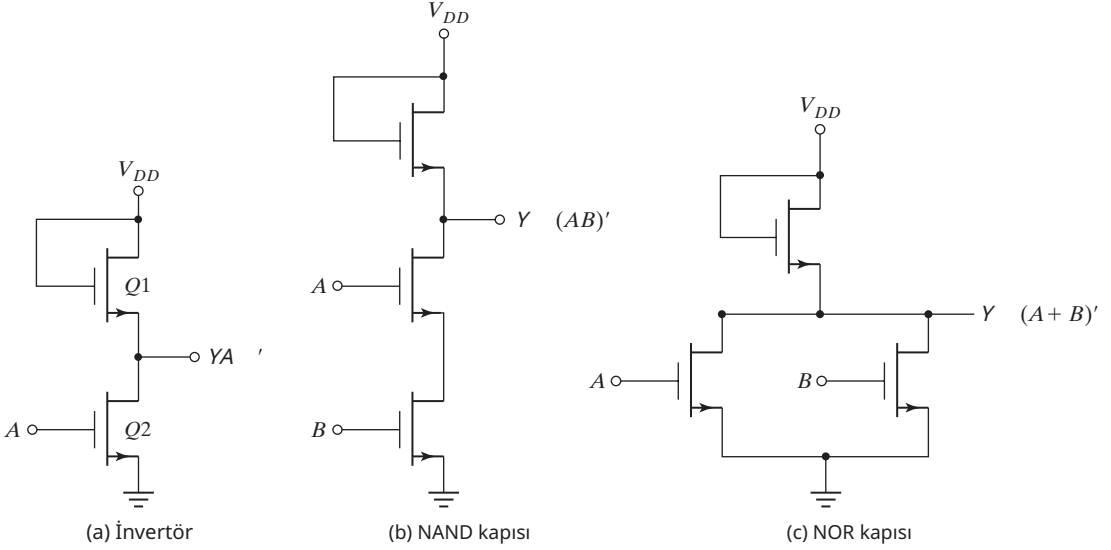
MOS transistörleri için grafik semboller Şekil A.2'de gösterilmiştir. Geliştirme tipi için sembol, kaynak ve dren arasında kesik hat bağlantısına sahip olanıdır. Bu sembolde, alt tabaka tanımlanabilir ve kaynağa bağlı olarak gösterilir. Alternatif bir sembol, alt tabakayı atlar ve bunun yerine kaynak terminalinde pozitif akım akışının yönünü göstermek için bir ok yerleştirilir (kaynak ile dren arasında p-kanal MOS'ta ve dren ile kaynak arasında n-kanal MOS'ta).

Kaynak ve drenin simetrik yapısı nedeniyle, MOS transistörü iki yönlü bir cihaz olarak çalıştırılabilir. Normalde taşıyıcıların kaynaktan drene akacak şekilde çalıştırılırsa da, taşıyıcıların dren ile kaynağa akmasına izin vermenin uygun olduğu durumlar vardır.

MOS cihazının bir avantajı, yalnızca bir transistör olarak değil, aynı zamanda bir direnç olarak da kullanılabilmesidir. Bir direnç, MOS'un kapı terminalini iletim için kalıcı olarak önyargılayarak elde edilir. Kaynak-drenaj voltajının kanal akımına oranı, direnç değerini belirler. Farklı direnç değerleri, MOS cihazının kanal uzunluğu ve genişliği sabitlenerek üretim sırasında oluşturulabilir.

MOS cihazlarını kullanan üç mantık devresi Şekil A.3'te gösterilmiştir. Bir n-kanal MOS için, besleme voltajı VDD pozitifdir (yaklaşık 5 V), böylece drenajdan kaynağa pozitif akım akışına izin verir. İki voltaj seviyesi, eşik voltajı V_T 'nin bir fonksiyonudur. Düşük seviye, sıfırdan V_T 'ye kadar herhangi bir değerdir ve yüksek seviye V_T 'den VDD'ye kadar değişir. n-kanal kapıları genellikle pozitif mantık kullanır. p-kanal MOS devreleri, kaynaktan drene pozitif akım akışına izin vermek için VDD için negatif bir voltaj kullanır. İki voltaj seviyesi, negatif eşik voltajı V_T 'nin üzerinde ve altında her ikisi de negatiftir. p-kanal kapıları genellikle negatif mantık kullanır.

Şekil A.3 (a)'de gösterilen inverter devresi iki MOS cihazı kullanmaktadır. Q_1 yük direnç olarak görev yaparken, Q_2 aktif cihaz olarak görev yapar. Yük-direnci MOS'un kapısı V_{DD} 'ye bağlıdır,

**ŞEKİL A.3***n-kanal MOS mantık devreleri*

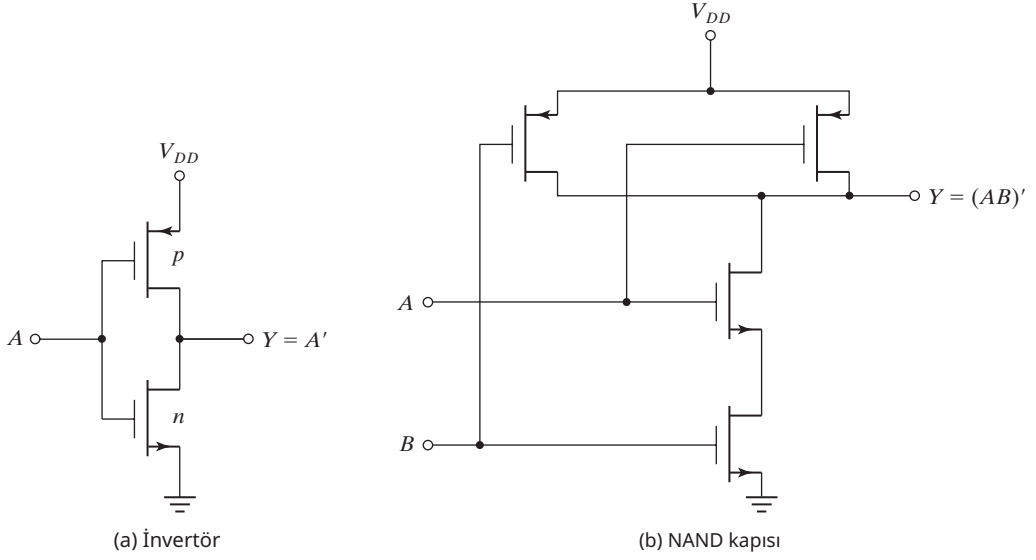
bu nedenle iletim durumunu korumaktadır. Giriş voltajı düşük olduğunda (altında V_T), Q_2 kapalıdır. Q_1 her zaman açık olduğundan, çıkış voltajı yaklaşık V_{DD} 'dir. Giriş voltajı yüksek olduğunda (üstünde V_T), Q_2 açılır. Akım, V_{DD} 'den yük direnci Q_1 üzerinde Q_2 'ye akar. İki MOS cihazının geometrisi, Q_2 'nin iletimdeyken direncinin, çıkışı Y yi V_T 'nin altında tutmak için Q_1 'nin direncinden çok daha az olması gerektiği şekilde olmalıdır.

Şekil A.3 (b)'de gösterilen NAND kapısı, transistörleri seri olarak kullanır. Girişler A ve B her iki transistörün de iletimde olması ve çıkışın düşük olması için yüksek olmalıdır. Eğer herhangi bir giriş düşükse, ilgili transistör kapalıdır ve çıkış yüksektir. Yine, iki aktif MOS cihazı tarafından oluşturulan seri direncin, yük direnci MOS'un direncinden çok daha az olması gerekir. Şekil A.3 (c)'de gösterilen NOR kapısı, transistörleri paralel olarak kullanır. Herhangi bir giriş yüksekse, ilgili transistör iletir ve çıkış düşüktür. Tüm girişler düşükse, tüm aktif transistörler kapalıdır ve çıkış yüksektir.

A.1 TAMAMLAYICI MOS

Tamamlayıcı MOS (CMOS) devreleri, hem n-kanal hem de p-kanal cihazların aynı alt tabakada üretilmesi gerçeğinden yararlanır. CMOS devreleri, mantık fonksiyonlarını oluşturmak için birbirine bağlı her iki tür MOS cihazını içerir. Temel devre, bir p-kanal transistör ve bir n-kanal transistörden oluşan invertterdir; bu, Şekil A.4 (a)'da gösterilmiştir. P-kanal cihazının kaynak terminali V_{DD} 'dedir ve n-kanal cihazının kaynak terminali toprak seviyesindedir.

V_{DD}



ŞEKİL A.4
CMOS mantık devreleri

giriş, $+3$ ile $+18$ V arasında herhangi bir yerde olabilir. İki voltaj seviyesi, düşük seviye için 0 V ve yüksek seviye için V_{DD} (tipik olarak, 5 V) dir.

Invertörün çalışmasını anlamak için, önceki bölümdeki MOS transistörünün davranışını gözden geçirmeliyiz:

n -kanal MOS, kapı kaynak voltajı pozitif olduğunda iletim yapar.

p -kanal MOS, kapı kaynak voltajı negatif olduğunda iletim yapar.

Her iki tür cihaz, kapı kaynak voltajı sıfır olduğunda kapalıdır.

Şimdi invertörün çalışmasını düşünelim. Giriş düşük olduğunda, her iki kapı da z sıfır potansiyelindedir. Giriş, p -kanal cihazının kaynağına göre $-V_{DD}$ ve n -kanal cihazının kaynağına göre 0 V'dir. Sonuç olarak, p -kanal cihazı açılır ve n -kanal cihazı kapalıdır. Bu koşullar altında, V_{DD} çıkışa düşük empedanslı bir yo l ve çıkıştan toprağa çok yüksek empedanslı bir yol vardır. Bu nedenle, çıkış volt aji normal yük koşullarında yüksek seviye V_{DD} seviyesine yaklaşır. Giriş yükse k olduğunda, her iki kapı da V_{DD} seviyesindedir ve durum tersine döner: p -ka nal cihazı kapalıdır ve n -kanal cihazı açıktır. Sonuç olarak, çıkış 0 V düşük seviye sine yaklaşır.

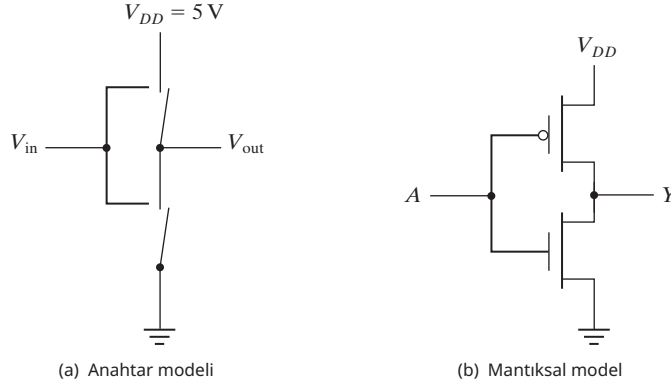
İki diğer CMOS temel kapısı Şekil A.4'te gösterilmiştir. İki girişli NAND kapısı, Şe kil A.4 (b)'te gösterildiği gibi, paralel olarak iki p -tipi birim ve seri olarak iki n -tipi bi rimden oluşur. Tüm girişler yüksek olduğunda, her iki p -kanal transistörü kapanır ve her iki n -kanal transistörü açılır. Çıkış, toprağa düşük empedansla bağlıdır ve d üşük bir durum üretir. Herhangi bir giriş düşükse, ilgili n -kanal transistörü kapanı r ve ilgili p -kanal transistörü açılır. Çıkış, V_{DD} 'ye bağlanır ve yüksek duruma geçer. Çok girişli NAND kapıları, Şekil A.4 (b)'te gösterilen düzenlemede, eşit sayıda p -tip i ve n -tipi transistörlerin paralel ve seri olarak yerleştirilmesiyle oluşturulabilir.

İki girişli NOR kapısı, Şekil A.4 (c)'te gösterildiği gibi, paralel olarak iki n -tipi birim v e seri olarak iki p -tipi birimden oluşur. Tüm girişler düşük olduğunda, her iki p -kanal birimi açıktır ve her iki n -kanal birimi kapalıdır. Çıkış, V_{DD} 'ye bağlanır ve yüksek duru ma geçer. Herhangi bir giriş yüksekse, ilgili p -kanal transistörü kapanır ve ilgili n -kan al transistörü açılır, çıkışı toprağa bağlayarak düşük seviyeli bir çıkış oluşturur.

MOS transistörleri, ya iletim yapan ya da açık olan elektronik anahtarlar olarak düşünülebilir. Örneğin, CMOS inverter, Şekil A.5 (a)'da gösterildiği gibi iki anahtar dan oluştuğu hayal edilebilir. Girişe düşük bir voltaj uygulamak, üst anahtarın (p) kapanmasına neden olur ve çıkışa yüksek voltaj sağlar. Girişe yüksek bir voltaj uy gulamak, alt anahtarın (n) kapanmasına neden olur ve çıkışı toprak ile bağlar.

Böylece, çıkış V_{out} , girişin V_{in} tamamlayıcısıdır. Ticari uygulamalar genellikle MO S transistörleri için anahtarların mantıksal davranışını vurgulamak amacıyla baş ka grafik semboller kullanır. Akım akış yönünü gösteren oklar atlanmıştır.

Bunun yerine, p -kanal transistörünün kapı girişi, düşük voltaj ile etkinleştirildiğini göste rmek için kapı terminalinde bir tersine çevirme balonu ile çizilmiştir. İnverter devresi, b u sembollerle Şekil A.5 (b)'de yeniden çizilmiştir. Girişteki mantık 0, üst transistörün ileti m yapmasına neden olur ve çıkışı mantık 1 yapar. Girişteki mantık 1, alt transistörü etki nleştirir ve çıkışı mantık 0 yapar.



ŞEKİL A.5
CMOS inverter

CMOS Özellikleri

Bir CMOS mantık devresi statik bir durumda olduğunda, güç tüketimi çok düşüktür. Bunun nedeni, devre durumu değişmediğinde, güç kaynağı ile toprak arasındaki yolda her zaman en az bir transistörün kapalı olmasıdır. Sonuç olarak, tipik bir CMOS kapısının statik güç tüketimi yaklaşık 0.01 mW civarındadır. Ancak, devre 1 MHz hızında durum değiştirirken, güç tüketimi yaklaşık 1 mW'ye çıkar ve 10 MHz'de yaklaşık 5 mW olur.

CMOS mantığı genellikle 3 ile 18 V arasında bir voltaj aralığında tek bir güç kaynağı ile çalışacak şekilde belirtilir ve tipik bir V_{DD} değeri 5 V'dir. CMOS'un daha büyük bir güç kaynağı voltajında çalıştırılması, yayılma gecikme süresini azaltır ve gürültü marjını artırır, ancak güç tüketimi artırır. $V_{DD}=5$ V ile yayılma gecikme süresi, kullanılan CMOS türüne bağlı olarak 5 ile 20 ns arasında değişir. Gürültü marjı genellikle güç kaynağı voltajının yaklaşık %40'ıdır. CMOS kapılarının fan-out'u 1 MHz frekansta çalıştırıldığında yaklaşık 30'dur. Kapıların çalışma frekansı arttıkça fan-out azalır.

CMOS dijital mantık ailesinin birkaç serisi vardır. 74C serisi, aynı numaraya sahip TTL cihazlarıyla pin ve işlev uyumludur. Örneğin, CMOS IC türü 74C04, TTL türü 7404 ile aynı pin konfigürasyonuna sahip altı inverter içerir. Yüksek hızlı CMOS 74HC serisi, 74C serisinin bir geliştirmesidir ve anahtarlama hızında on kat artış sağlar. 74HCT serisi, TTL IC'lerle elektriksel olarak uyumludur. Bu, bu serideki devrelerin, ek arayüz devrelerine ihtiyaç duymadan TTL IC'lerin giriş ve çıkışlarına bağlanabileceği anlamına gelir. CMOS'un daha yeni versiyonları, yüksek hızlı 74VHC serisi ve TTL uyumlu versiyonu 74VHCT'dir.

CMOS üretim süreci, TTL'den daha basittir ve daha yüksek paket yoğunluğu sağlar. Bu nedenle, belirli bir silikon alanına daha fazla devre yerleştirilebilir ve fonksiyon başına maliyet düşer. Bu özellik, CMOS devrelerinin düşük güç tüketimi, iyi gürültü bağışıklığı ve makul yayılma gecikmesi ile birlikte, CMOS'u dijital mantık ailesi olarak en popüler standart haline getirir.

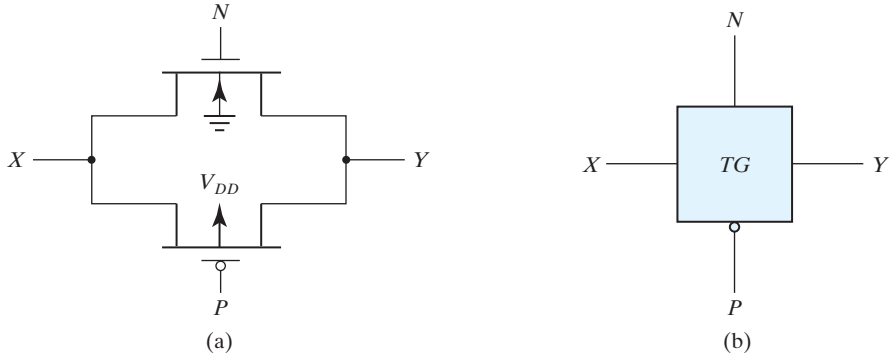
A.2 CMOS İLETİM KAPISI DEVRELERİ

Diğer dijital mantık ailelerinde bulunmayan özel bir CMOS devresi, *iletim kapısı*. İletim kapısı esasen bir giriş mantık seviyesi ile kontrol edilen elektronik bir anahtardır. CMOS teknolojisi ile üretildiğinde çeşitli dijital bileşenlerin inşasını basitleştirmek için kullanılır.

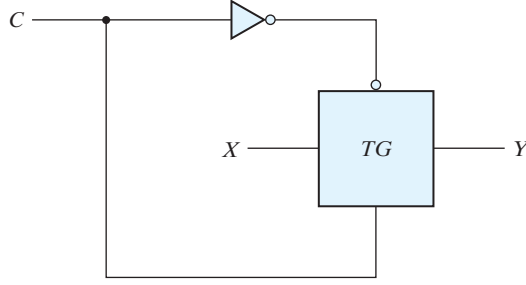
Şekil A.6(a), iletim kapısının temel devresini göstermektedir. Bir CMOS inverter, bir p -kanal transistörün bir n -kanal transistörle seri olarak bağlanmasından oluşurken, bir iletim kapısı bir n -kanal ve bir p -kanal MOS transistörünün paralel olarak bağlanmasıyla oluşur.

n -kanal altlığı toprakla bağlantılıdır ve p -kanal altlığı V_{DD} ile bağlantılıdır. N kapısı V_{DD} seviyesindeyken ve P kapısı toprak seviyesindeyken, her iki transistör de iletim yapar ve giriş X ile çıkış Y arasında kapalı bir yol vardır. N kapısı toprak seviyesindeyken ve P kapısı V_{DD} seviyesindeyken, her iki transistör de kapalıdır ve X ile Y arasında açık bir devre vardır. Şekil A.4(b), iletim kapısının blok diyagramını göstermektedir. P -kanal kapısının ucu, olumsuzlama sembolü ile işaretlenmiştir. Şekil A.4(c), anahtarın pozitif mantık ataması açısından davranışını göstermektedir; burada V_{DD} mantık 1'e ve toprak mantık 0'a eşdeğerdir.

İletim kapısı genellikle bir inverter ile bağlantılıdır, bu Şekil A.7'de gösterilmiştir. Bu tür bir düzenlemeye *iki yönlü anahtar* denir. Kontrol girişi C doğrudan n -kanal kapısına ve tersine p -kanal kapısına bağlanır. $C=1$ olduğunda,



ŞEKİL A.6
İletim kapısı (TG)



ŞEKİL A.7

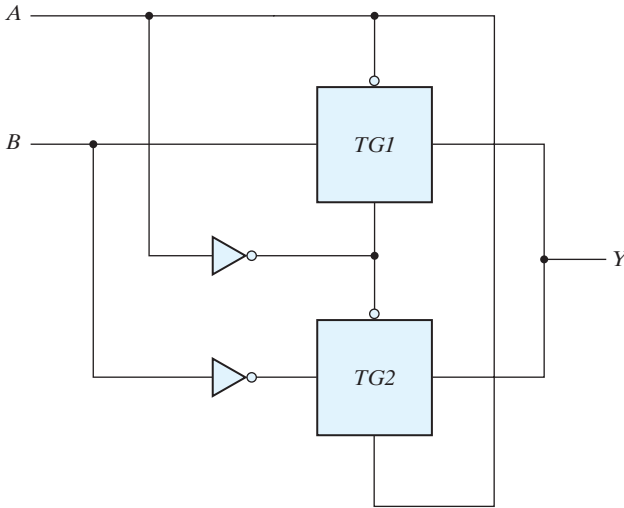
İki yönlü anahtar

anahtar kapalıdır, X ve Y arasında bir yol oluşturur. $C=0$ olduğunda, anahtar açıktır, X ve Y arasındaki yolu keser.

Çeşitli devreler, iletim kapısını kullanarak inşa edilebilir. CMOS ailesindeki bir bileşen olarak faydasını göstermek için, üç örnek sunacağız.

Özel-VE kapısı, iki iletim kapısı ve iki inverter ile inşa edilebilir, bu şekilde gösterilmiştir. Şekil A.8. Giriş A , iletim kapılarındaki yolları kontrol eder ve giriş B çıkış Y ile kapılar aracılığıyla bağlantılıdır. Giriş A 0'a eşit olduğunda, iletim kapısı $TG1$ kapalıdır ve çıkış Y giriş B ile eşittir. Giriş A 1'e eşit olduğunda, $TG2$ kapalıdır ve çıkış Y giriş B 'nin tamamlayıcısına eşittir. Bu, Şekil A.8'de gösterildiği gibi özel-VE doğruluk tablosunu oluşturur.

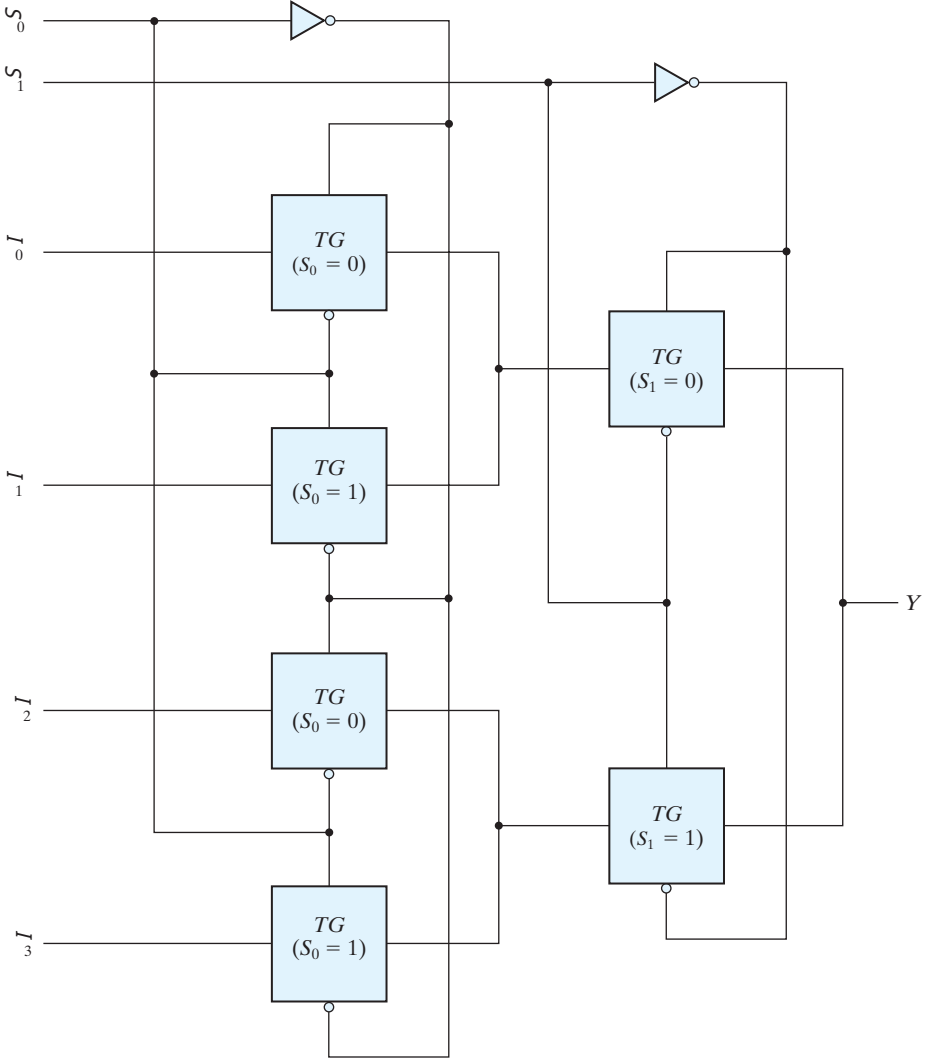
İletim kapıları ile inşa edilebilecek bir diğer devre çoklayıcıdır. Dört'e bir hat çoklayıcısı, iletim kapıları ile uygulanmıştır ve Şekil A.9'da gösterilmiştir. TG devresi, yatay girişi ile



A	B	$TG1$	$TG2$	Y
0	0	kapalı	açık	0
0	1	kapalı	açık	1
1	0	açık	kapalı	0
1	1	açık	kapalı	1

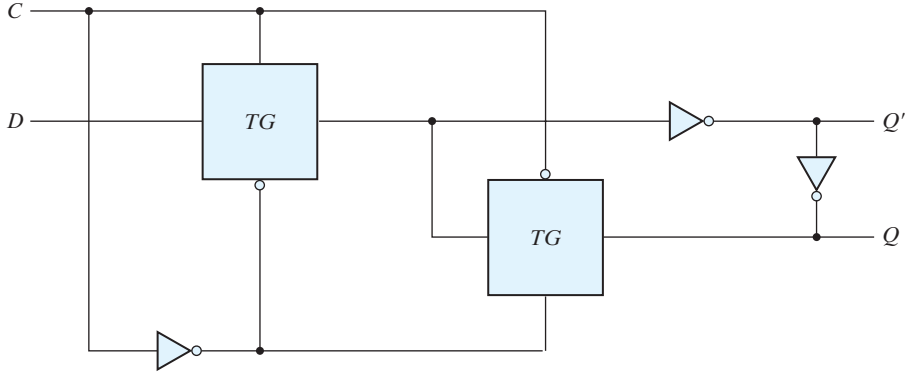
ŞEKİL A.8

İletim kapıları ile oluşturulmuş Özel-VE



ŞEKİL A.9
İletim kapıları ile çoklayıcı

çıkış hatları, iki dikey kontrol girişinin, dairesel olmayan terminalde 1 ve dairesel terminalde 0 değerine sahip olduğu durumlarda. Kontrol girişlerinde zıt polarite olduğunda, yol kesilir ve devre açık bir anahtar gibi davranır. İki seçim girişi, S_1 ve S_0 , iletim yolunu TG devrelerinde kontrol eder. Her kutunun içinde, iletim kapısı anahtarının kapalı olması için gereken koşul belirtilmiştir. Böylece, eğer $S_0=0$ ve $S_1=0$ ise, giriş I_0 ile çıkış Y arasında, $S_0=0$ ve $S_1=0$ ile işaretlenmiş iki TG üzerinden kapalı bir yol vardır. Diğer üç giriş, diğer TG devrelerinden biri tarafından çıkıştan ayrılmıştır.



ŞEKİL A.10

Kapalı D geçiş kapıları ile kilit

Seviye duyarlı D flip-flop, genellikle kapalı D latch olarak adlandırılır, iletim kapıları ile inşa edilebilir, Şekil A.10'da gösterildiği gibi. C girişi iki iletim kapısını TG kontrol eder. $C=1$ olduğunda, D girişine bağlı TG kapalı bir yola sahiptir ve çıkış Q ile bağlı olan açık bir yola sahiptir. Bu yapı, giriş D üzerindeki iki invertere kadar çıkış Q için eşdeğer bir devre üretir. Böylece, çıkış, C aktif kaldığı sürece veri girişini takip eder. C 0'a geçtiğinde, ilk TG giriş D devreden ayırır ve ikinci TG çıkıştaki iki invertör arasında kapalı bir yol oluşturur. Böylece, C 1'den 0'a geçtiği anda giriş D da bulunan değer, Q çıkışında konur.

Bir master-slave D flip-flop, Şekil A.10'da gösterilen türde iki devre ile inşa edilebilir. İlk devre master, ikinci devre ise slave'dir. Böylece, bir master-slave D flip-flop, dört iletim kapısı ve altı invertör ile inşa edilebilir.

A.3 ANAHTAR DÜZEYİ MODELLEME HDL İLE

CMOS, entegre devrelerle birlikte kullanılan baskın dijital mantık ailesidir. Tanım gereği, CMOS, bir NMOS ve bir PMOS transistörün tamamlayıcı bağlantısıdır. MOS transistörleri, ya iletim yapan ya da açık olan elektronik anahtarlar olarak düşünülebilir. MOS anahtarları arasındaki bağlantıları belirleyerek, tasarımcı bir dijital devreyi CMOS ile inşa edebilir. Bu tür bir tanım, Verilog HDL'de anahtar-düzeyi modelleme olarak adlandırılır.

İki tür MOS anahtarı, Verilog HDL'de **nmos** ve **pmos** anahtar kelimeleri ile belirtilir. Transistörün üç terminalini belirterek örneklenir, Şekil A.2'de gösterildiği gibi:

nmos (drain, source, gate);

pmos (drain, source, gate);

Anahtarlar, ilkel olarak kabul edilir, bu nedenle bir örnek adı kullanımı isteğe bağlıdır.

MOS devreleri tasarlandığında, bir güç kaynağına (V_{DD}) ve toprağa bağlanmalar belirtilmelidir. Güç ve toprak, **supply1** ve **supply0** anahtar kelimeleri ile tanımlanır. Örneğin, aşağıdaki ifadelerle belirtilir:

supply1 PWR;
supply0 GRD;

supply1 türündeki kaynaklar, VDD ile eşdeğerdir ve mantık 1 değerine sahiptir. supply0 türündeki kaynaklar, toprak bağlantısı ile eşdeğerdir ve mantık 0 değerine sahiptir.

Şekil A.4 (a)'deki CMOS inverterinin tanımı HDL Örneği A.1'de gösterilmiştir. Giriş, çıkış ve iki güç kaynağı önce tanımlanır. Modül, bir PMOS ve bir NMOS transistörünü örnekler. Çıkış Y her iki transistörün drenaj terminallerinde ortaktır. Giriş de her iki transistörün kapı terminallerinde ortaktır. PMOS transistörünün kaynak terminali PWR'ye ve NMOS transistörünün kaynak terminali GRD'ye bağlıdır.

HDL Örneği A.1

```
// Şekil A.4(a) CMOS inverter
modül inverter (Y, A);
  input A;
  output Y;
  power1 PWR;
  power0 GRD;

  pmos (Y, PWR, A); // (Drenaj, kaynak, kapı)      // (Drenaj, kaynak, kapı)
  nmos (Y, GRD, A); // (Drenaj, kaynak, kapı)      // (Drenaj, kaynak, kapı)
endmodül
```

İkinci modül, HDL Örneği A.2'de belirtilmiştir ve Şekil A.4 (b)'deki iki girişli CMOS NAND devresini tanımlar. Paralel bağlı iki PMOS transistörü vardır, kaynak terminalleri PWR'ye bağlıdır. Ayrıca, ortak bir terminale W1 bağlı iki NMOS transistörü seri olarak bağlanmıştır. İlk NMOS'un drenajı çıkışa bağlıdır ve ikinci NMOS'un kaynak terminali GRD'ye bağlıdır.

HDL Örneği A.2

```
// Şekil A.4(b) CMOS iki girişli NAND
modül NAND2 (Y, A, B);
  input A, B;
  output Y;
  supply1 PWR;
  supply0 GRD;

  w1 W1; // iki nmos arasındaki terminal
  pmos (Y, PWR, A); // Vdd'ye bağlı kaynak      // Vdd'ye bağlı kaynak
  pmos (Y, PWR, B); // paralel bağlantı          // paralel bağlantı
  nmos (W1, A, B); // seri bağlantı
```

```

nmos (Y, W1, A); // seri bağlantı
nmos (W1, GRD, B); // kaynak toprağa bağlı
endmodül

```

İletim Kapısı

İletim kapısı, Verilog HDL'de **cmos** anahtar kelimesi ile örneklenir. Bir out, bir giriş ve iki kontrol sinyali vardır, Şekil A.6'da gösterildiği gibi. Buna **cmos** açık anahtar denir. İlgili kod aşağıdaki gibidir:

```

cmos (çıkış, giriş, ncontrol, pcontrol); // genel açıklama

cmos (Y, X, N, P); // Şekil A.6 (b) için iletim kapısı

```

Normalde, ncontrol ve pcontrol birbirinin tamamlayıcısıdır. **cmos** anahtarı güç kaynaklarına ihtiyaç duymaz, çünkü V_{DD} ve toprak MOS transistörlerin alt tabakalarının a bağlıdır. İletim kapıları, CMOS devreleri ile çoklayıcılar ve flip-flop'lar inşa etmek için faydalıdır.

HDL Örneği A.3, **cmos** anahtarları ile bir devreyi tanımlar. Şekil A.8'deki özel-VE devresi iki iletim kapısı ve iki inverter içerir. İki inverter, bir CMOS inverterini tanımlayan modül içinde örneklenmiştir. İki **cmos** anahtarı, dildeki temel bileşenler oldukları için bir örnek adı olmadan örneklenmiştir. Devrenin çalışmasını test etmek için bir test modülü eklenmiştir. İki girişin tüm olası kombinasyonlarını uygulayarak, simülatörün sonucu özel-VE devresinin çalışmasını doğrular. Simülasyonun çıktısı aşağıdaki gibidir:

```

A = 0  B = 0  Y = 0
A = 0  B = 1  Y = 1
A = 1  B = 0  Y = 1
A = 1  B = 1  Y = 0

```

HDL Örneği A.3

```
//CMOS_XOR, CMOS anahtarları ile, Şekil A.8
```

```

modül CMOS_XOR (A, B, Y);
  input A, B;
  output Y;
  tel A_b, B_b;
  // inverter'ı oluştur
  inverter v1 (A_b, A);
  inverter v2 (B_b, B);
  // cmos anahtarını oluştur
  cmos (Y, B_b, A, A_b);
endmodule

```

```

// (çıkış, giriş, ncontrol, pcontrol)

```

520 Ek Yarı İletkenler ve CMOS Entegre Devreler

```
// CMOS inverter Şekil A.4(a)
modül inverter (Y, A);
    input A;
    output Y;
    güç1 PWR;
    güç0 GND;

    pmos (Y, PWR, A); //(Drenaj, kaynak, kapı)
    nmos (Y, GND, A); //(Drenaj, kaynak, kapı)
endmodül

// CMOS_XOR'u test etmek için uyarı
modül test_CMOS_XOR;
    reg A,B;
    tel Y;
    // CMOS_XOR'u örnekle
    CMOS_XOR X1 (A, B, Y);
    // Gerçeklik tablosunu uygula
    ilk
        başla
            A = 1'b0; B = 1'b0;
            #5 A = 1'b0; B = 1'b1;
            #5 A = 1'b1; B = 1'b0;
            #5 A = 1'b1; B = 1'b1;
        bitir
    // Sonuçları göster
    ilk
        $monitor ("A=%b B=%b Y=%b", A, B, Y);
endmodule
```

WEB ARAMA KONULARI

İletken
Yarı İletken
İzolatör
Malzemelerin elektriksel özellikleri
Değerlik elektronu
Diyot
Transistör
CMOS süreci
CMOS mantık kapısı
CMOS inverter

521

522 Seçilen Problemlere Cevaplar

1.29 Steve Jobs

1.31 $62+32=94$ yazdırma karakterleri

1.32 sağdan 6. bit

1.33 (a) 897 (b) 564 (c) 871 (d) 2,199

BÖLÜM 2

2.2 (a) . (b) . (c) . (d) .

2.3 (a) . (b) $z(x+y)$ (c) $x \cdot y$ (d) $x(w+y)$ (e) .

2.4 (a) $AB+C$ (b) $x + y+z$ (c) . (d) $A'(B+C'A)$

2.9 (a) $xy+xy'$

2.11 $F(x, y, z) = \Sigma(1, 4, 5, 6, 7)$

2.12 (a) 10100000 00011101 (d) 01001110

2.14 (b) $(x + y) + (x+y) + (y+z)'$

2.15 $T_1 = A'(BC)$

$T_2 = A+BC = T_1'$

2.17 (a) $\Sigma(3, 5, 6, 7) = \Pi(0, 1, 2, 4)$

2.18 (c) $F = yz + y(w+x)$

2.19 $\Sigma(1, 3, 5, 7, 9, 11, 13, 15) = \Pi(0, 2, 4, 6, 8, 10, 12, 14)$

2.22 (a) $AB+BC = (A+C)B$ $x + y+z$

BÖLÜM 3

3.1 (a) $xy + xz$ (b) $xy + z$ (c) $x + yz$ (d) $x \cdot y + x'z + yz$

3.2 (a) $x \cdot y + xz$ (b) $y + xz$

3.3 (a) $xy+xz$ (b) $x + yz$ (c) $z + xy$

3.4 (a) . (b) $BCD+ABD$ (c) $ABD+ABC+CD$

(a) $wx+w'x'y$

3.5 (a) $xz + wyz + wxy$ (d) $BD+BD + AB$ $BD+BD + AD$

3.6 (a) $BD + ABD + ABC$ (b) $xy + x'z + wx'y$

3.7 (a) $x \cdot y + z$ (c) $AC+BD + A'BD+B'C$ (veya CD)

3.8 (a) $F(x, y, z) = \Sigma(3, 5, 6, 7)$ (b) $F(A, B, C, D) = \Sigma(1, 3, 5, 9, 12, 13, 14)$

3.9 (a) Önemli: xz ve xz ; Önemsiz: wx ve wz

(a) $F = BD + AC + ABD + (CD \text{ veya } BC)$

3.10 (c) $F = BC + AC + ABD$

Temel: BC, AC

Temel Olmayan: $AB, A B, B CD, A C D$

3.11 (a) $F = A'B'D' + AD'E + B'C'D'$

3.12 (b) $F = (A + D')(B' + D')$

3.13 (a) $F = xy + z' = (x + z')(y + z')$

3.15 (b) $F = B'D' + CD' + ABC'D = \Sigma(0, 2, 6, 8, 10, 13, 14)$

3.17 $F' = AC' + BC' + BD$

3.19 (a) $F = (w + z')(x' + z')(w' + x' + y')$

3.30 $F = (A \oplus B)(C \oplus D)$

3.35 HDL tanımı Eşlik Eden Web Sitesi'nde mevcuttur.

Satır 1: Tireye izin verilmez, alt çizgi kullanın: Örnek_3.

Satırı noktalı virgöl (;) ile sonlandırın.

Satır 2: girdiler girdi (sonunda s yok).

Son virgölü (,) noktalı virgöl (;) ile değiştirin. Çıktı beyan edilmiştir ancak port listesinde görünmemektedir ve eğer

girdi listesinde yer alması amaçlanıyorsa, bir virgöl ile takip edilmelidir. Eğer Çıktı çıktı kelimesinin yanlış yazıymısa ve çıktı portlarını beyan etmek içinse, C noktalı virgöl (;) ile takip edilmelidir. F noktalı virgöl (;) ile takip edilmelidir.

Satır 3: B girdi (Satır 2) ve çıktı (Satır 3) olarak beyan edilemez. Satırı noktalı virgöl (;) ile sonlandırın.

Satır 4: A modül için bir girdi ise, ilkelin çıktısı olamaz.

Satır 5: NOT kapısı için çok fazla giriş (sadece ikiye izin verilir).

Satır 6: OR küçük harfle yazılmalıdır: "or" olarak değiştirin.

Satır 7: **endmodule** yanlış yazılmıştır. Noktalı virgölü kaldırın (endmodule'dan sonra noktalı virgöl yok).

BÖLÜM 4

4.1 (a) $F_1 = A + B'C + BD' + B'D$
 $F_2 = A'B + D$

4.2 $F = ABC + A D$
 $G = ABC + A D$

4.3 (b) 1024 satır ve 14 sütun

4.4 (a) $F = x y + x z$

4.6 $F = xy + xz + yz$

4.7 (a) $w = A \quad x = A \oplus B \quad y = x \oplus C \quad z = y \oplus D$

4.8 $w = AB + AC \quad D$

524 Seçilen Problemlere Cevaplar

4.10 Girdiler: A, B, C, D; Çıktılar: w, x, y, z

$$z = D$$

$$y = C \oplus D$$

$$x = B \oplus (C + D)$$

$$w = A \oplus (B + C + D)$$

4.12 (b) $Diff = x \oplus y \oplus B_{in}$

$$B_{out} = x \oplus x \oplus B_{in} + y B_{in}$$

4.13

	Toplam	C	V
(a)	1101	0	1
(b)	0001	1	1
(c)	0100	1	0
(d)	1011	0	1
(e)	1111	0	0

$$(a) \quad 1101 \quad 0 \quad 1$$

$$(b) \quad 0001 \quad 1 \quad 1$$

$$(c) \quad 0100 \quad 1 \quad 0$$

$$(d) \quad 1011 \quad 0 \quad 1$$

$$(e) \quad 1111 \quad 0 \quad 0$$

4.14 30 ns

4.18 $w = A' B' C'$

$$x = B \oplus C$$

$$y = C$$

$$z = D'$$

4.22 $w = AB + ACD$

$$x = B' C' + B' D' + BCD$$

$$y = C' D + CD'$$

$$z = D'$$

4.28 (a) $F_1 = \Sigma(0, 5, 7)$

$$F_2 = \Sigma(2, 3, 4)$$

$$F_3 = \Sigma(1, 6, 7)$$

4.29 $x = D'_0 D'_1$

$$y = D'_0 D_1 + D'_0 D'_2$$

4.34 $F(A, B, C, D) = \Sigma(1, 6, 7, 9, 10, 11, 12)$

4.35 (a) When $AB = 00, F = D$

$$\text{When } AB = 01, F = (C + D)'$$

$$\text{When } AB = 10, F = CD$$

$$\text{When } AB = 11, F = 1$$

4.39 HDL tanımı Companion Web Sitesi'nde mevcuttur.

4.42 (c) HDL tanımı Companion Web Sitesi'nde mevcuttur.

- 4.50 HDL tanımı Companion Web Sitesi'nde mevcuttur.
- 4.56 `assign eşleřtir = (A == B);` // Varsayılan `reg [3: 0] A, B;`
- 4.57 HDL tanımı Eřlik Eden Web Sitesinde mevcuttur.

BÖLÜM 5

- 5.4 (b) $PQ + NQ$
- 5.7 $S = x \oplus y \oplus Q$
 $Q(t + 1) = xy + xQ + yQ$
- 5.8 00, 01, 10 tekrar eden bir dizisi olan bir saya
- 5.9 (a) $A(t + 1) = xA' + AB$
 $B(t + 1) = xB' + A'B$
- 5.10 (c) $A(t + 1) = xB + x'A + yA + y'A'B'$
 $B(t + 1) = xA'B' + x'A'B + yA'B'$
- 5.11 Mevcut durum: 00 00 01 00 01 11 00 01 11 10 00 01 11 10 10
 Girdi: 0 1 0 1 1 0 1 1 1 0 1 1 1 1 0
 ıktı: 0 0 1 0 0 1 0 0 0 1 0 0 0 0 1
 Sonraki durum: 00 01 00 01 11 00 01 11 10 00 01 11 10 10 00
- 5.12
- | Mevcut durum | Sonraki durum
0 1 | ıktı
0 1 |
|--------------|----------------------|--------------|
| <i>a</i> | <i>f b</i> | 0 0 |
| <i>b</i> | <i>d a</i> | 0 0 |
| <i>d</i> | <i>g a</i> | 1 0 |
| <i>f</i> | <i>f b</i> | 1 1 |
| <i>g</i> | <i>g d</i> | 0 1 |
- 5.13 (a) Durum: *a f b c e d g h g g h a*
 Girdi: 0 1 1 1 0 0 1 0 0 1 1
 ıktı: 0 1 0 0 0 1 1 1 0 1 0
- (b) Durum: *a f b a b d g d g g d a*
 Girdi: 0 1 1 1 0 0 1 0 0 1 1
 ıktı: 0 1 0 0 0 1 1 1 0 1 0
- 5.15 $D_Q = Q'J + QK'$

5.16 $D_A = Ax' + Bx$

$D_B = A'x + Bx'$

5.18 $J_A = K_A = (BF + B'F')E$

$J_B = K_B = E$

5.19 (a) $D_A = A'B'x_{in}$

$D_B = A + C'x_{in}' + BCx_{in}$

$D_C = Cx_{in}' + Ax_{in} + A'B'x_{in}'$

$y_{out} = A'x_{in}$

5.23 (a) $RegA = 125, RegB = 125$

$RegA = 125, RegB = 30$

5.26 (a)

$Q(t + 1) = JQ' + K'Q$

When $Q = 0, Q(t + 1) = J$

When $Q = 1, Q(t + 1) = K'$

module JK_Behavior (**output reg** Q, **input** J, K, CLK);

always @ (posedge CLK)

if (Q == 0) Q <= J;

else Q <= ~K;

endmodül

5.31 HDL tanımı Eşlik Eden Web Sitesi'nde mevcuttur.

Not: İfadeler, eşzamanlı atamaların etkisini üretecek bir sırayla yazılmalıdır.

BÖLÜM 6

6.4 0110; 0011; 0001; 1000; 1100; 1110; 0111; 1011

6.8 $A = 0010, 0001, 1000, 1100$. Taşı=1, 1, 1, 0

6.9 (b) $J_Q = x'y; K_Q = (x' + y)'$

6.14 (a) 4

6.15 30 ns; 33.3 MHz

6.16 $1010 \rightarrow 1011 \rightarrow 0100$

$1100 \rightarrow 1101 \rightarrow 0100$

$1110 \rightarrow 1111 \rightarrow 0000$

6.17 $D_{A0} = A_0 \oplus E$

$$D_{A1} = A_1 \oplus (A_0 E)$$

$$D_{A2} = A_2 \oplus (A_1 A_0 E)$$

$$D_{A3} = A_3 \oplus (A_2 A_1 A_0 E)$$

6.19 (b) $D_{Q1} = Q_1'$

$$D_{Q2} = Q_2 Q_1' + Q_8' Q_2' Q_1$$

$$D_{Q4} = Q_4 Q_1' + Q_4 Q_2' + Q_4' Q_2' Q_1$$

$$D_{Q8} = Q_8 Q_1' + Q_4 Q_2 Q_1$$

6.21 $J_{A0} = LI_0 + L \ C$

$$K_{A0} = LI_0' + L \ C$$

6.24 $T_A = A \oplus B$

$$T_B = B \oplus C$$

$$T_C = AC + A'C' \quad (\text{not self-starting})$$

$$= AC + A'B'C' \quad (\text{self-starting})$$

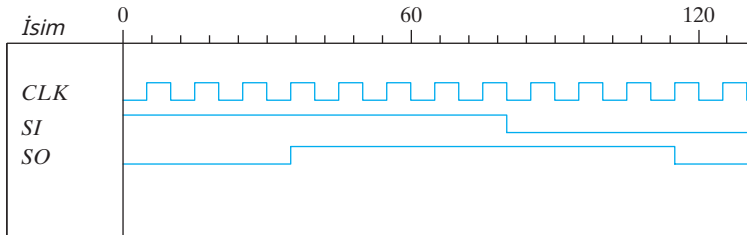
6.26 Saati üretici 12.5 ns'lik bir periyoda sahiptir. Dört darbe saymak için 2 bitlik bir saya kullanın.

6.28 $D_A = A \oplus B$

$$D_B = AB' + C$$

$$D_C = A'B'C'$$

6.34 HDL tanımı Companion Web Sitesi'nde mevcuttur. Problem 6.34 için simülasyon sonuçları şu şekildedir:



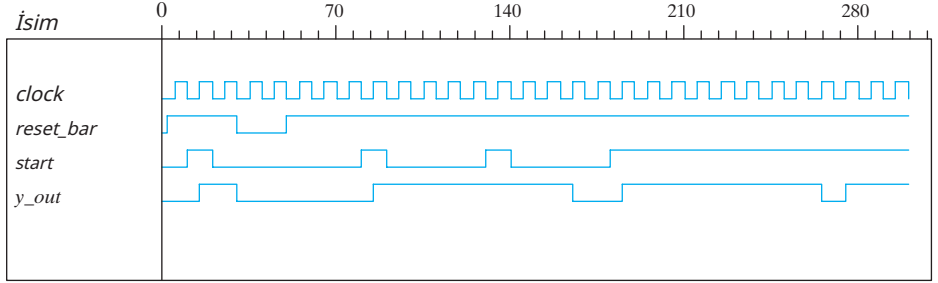
6.35 (b) HDL tanımı Eşlik Eden Web Sitesi'nde mevcuttur.

6.37 HDL tanımı Eşlik Eden Web Sitesi'nde mevcuttur.

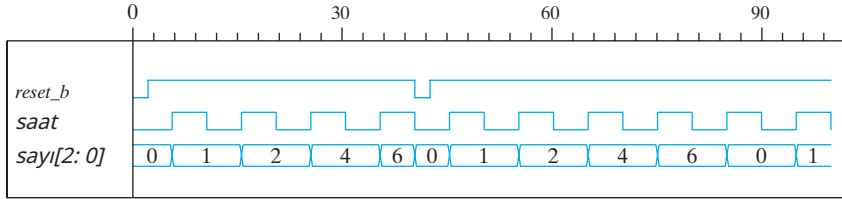
6.38 (a) HDL tanımı Eşlik Eden Web Sitesi'nde mevcuttur.

528 Seçilen Problemlere Cevaplar

- 6.42** Çünkü A bir kayıt değişkenidir, atanan değeri unutuluncaya kadar saklar. Bu nedenle, ifade $A \leq A$ ifadesinin atlandığı durumla aynı etkidir.
- 6.45** HDL tanımı Companion Web Sitesi'nde mevcuttur. Problem 6.45 için simülasyon sonuçları şu şekildedir:



- 6.50** (b) HDL tanımı Companion Web Sitesi'nde mevcuttur. Problem 6.50 için simülasyon sonuçları şu şekildedir:



BÖLÜM 7

- 7.2** (a) 2^{13} (b) 2 (c) 2 (d) 21
- 7.3** Adres: 1 0001 1011=011B (hex)
Veri: 100 1011 1100=4BC (hex)

7.7 (a) 7×128 kodlayıcı, 256 VE kapıları (b) $x = 46; y = 112$

7.8 (a) 8 ip (b) 185 (c) 3×8 decoder

7.10 0001 1011 1011 1

7.11 101 110 011 001 010

7.12 (a) 0101 1010; (b) 1100110; (c) 1110100

7.13 (a) , (b) , (c) ,

7.14 (a) 0101010

7.16 24 pin

7.20 Ürün terimleri: $yz, xz, x'y, xy, x'yz$

7.25 $A = yz' + xz' + x'y'z$

$B = x'y' + yz + y'z'$

$C = A + xyz$

$D = z + x'y$

BÖLÜM 8

8.1 (a) Transfer ve artış aynı anda, yani aynı saat kenarında gerçekleşir. Transferden sonra, R2 saat

kenarından önce R1 içindeki içerikleri tutar ve R2 bir önceki değerini 1 artırarak tutar.

(b) R3 içeriğini bir azalt.

(c) Eğer ($S_1 = 1$), R1 içeriğini R0'ya aktar. Eğer ($S_1 = 0$ ve $S_2 = 1$), R2 içeriğini R0'ya aktar.

8.7 RTL notasyonu:

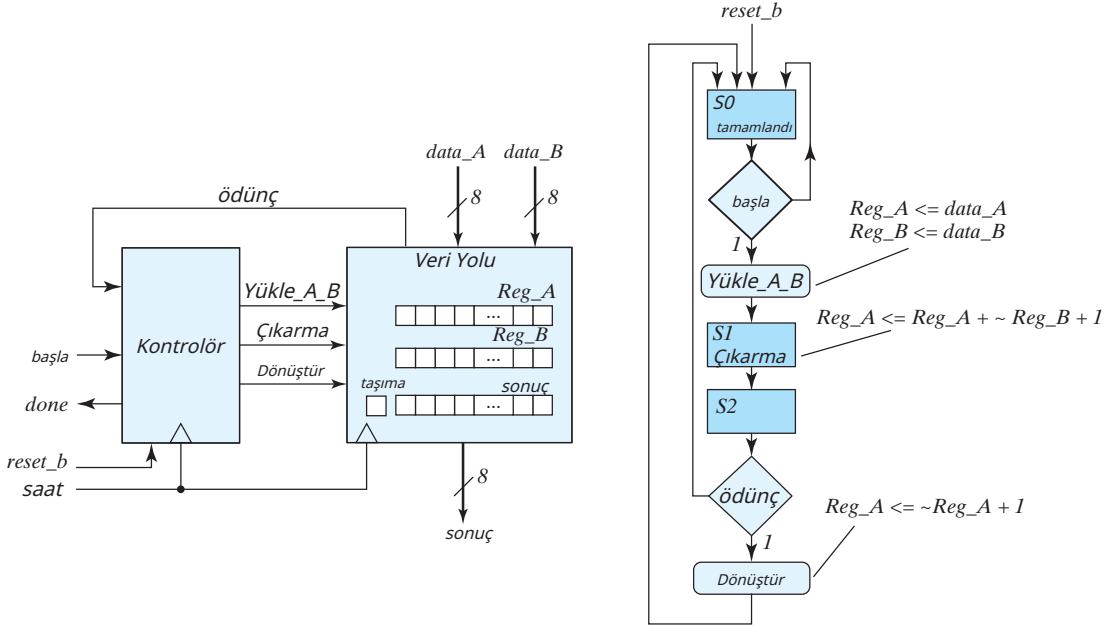
S0: Başlangıç durumu: eğer (start = 1) ise ($RA \leftarrow \text{data_A}, RB \leftarrow \text{data_B}, S1$ 'e git).

S1: {Taşı, RA} $\leftarrow RA + (2 \cdot RB$ 'nin tamamlayanı), S2'ye git.

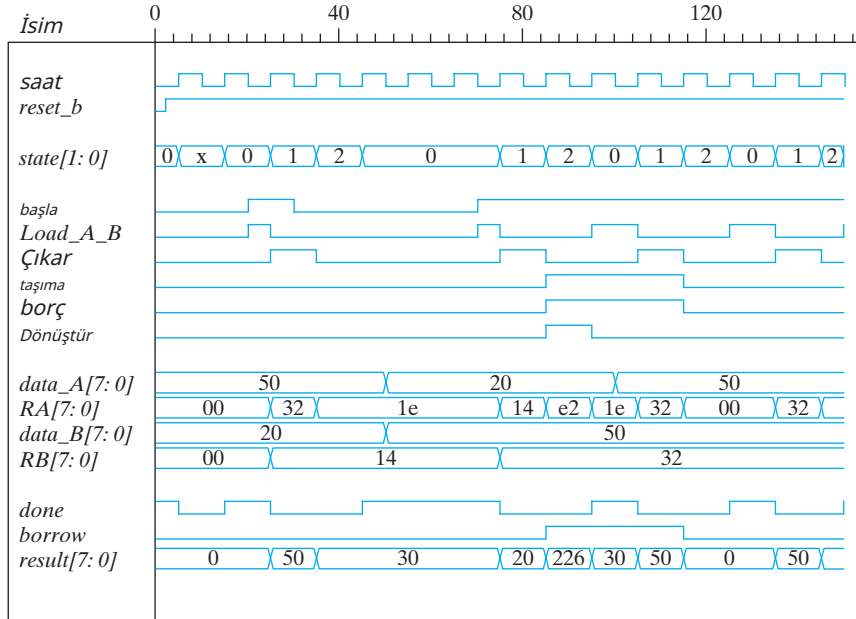
S2: Eğer (borrow = 0) S0'a git. Eğer (borrow = 1) o zaman RA d(2>s tamamlayıcı of RA), S0'a git.

530 Seçilen Problemlere Cevaplar

Blok diyagramı ve ASMD grafiği:



HDL tanımı Companion Web Sitesi'nde mevcuttur. Simülasyon sonuçları için Problemler 8.7 aşağıdadır:



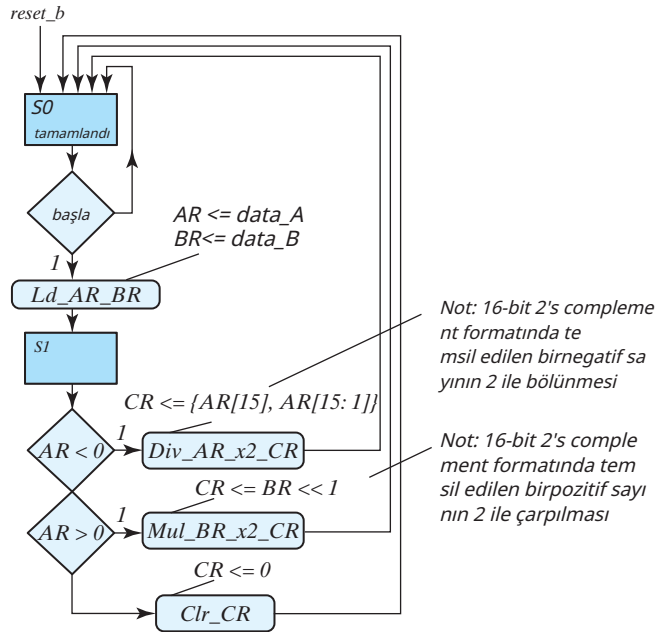
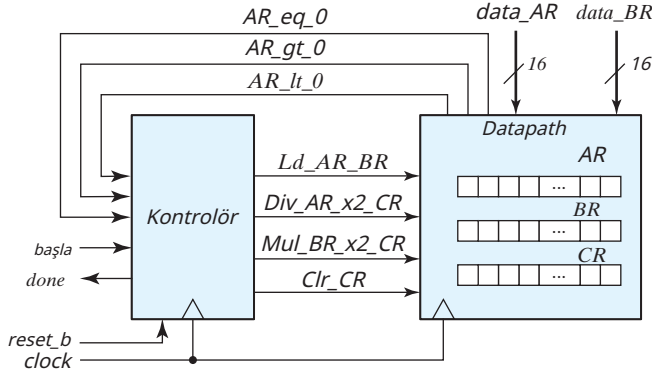
8.8 RTL notasyonu:

S0: eğer (başla=1) $AR \leftarrow$ girdi verisi, $BR \leftarrow$ girdi verisi, *S1*'ye git.

S1: eğer $(AR[15]) = 1$ (işaret biti negatif) ise $CR \leftarrow AR$ (sağa kaydırılmış, işaret uzantısı).

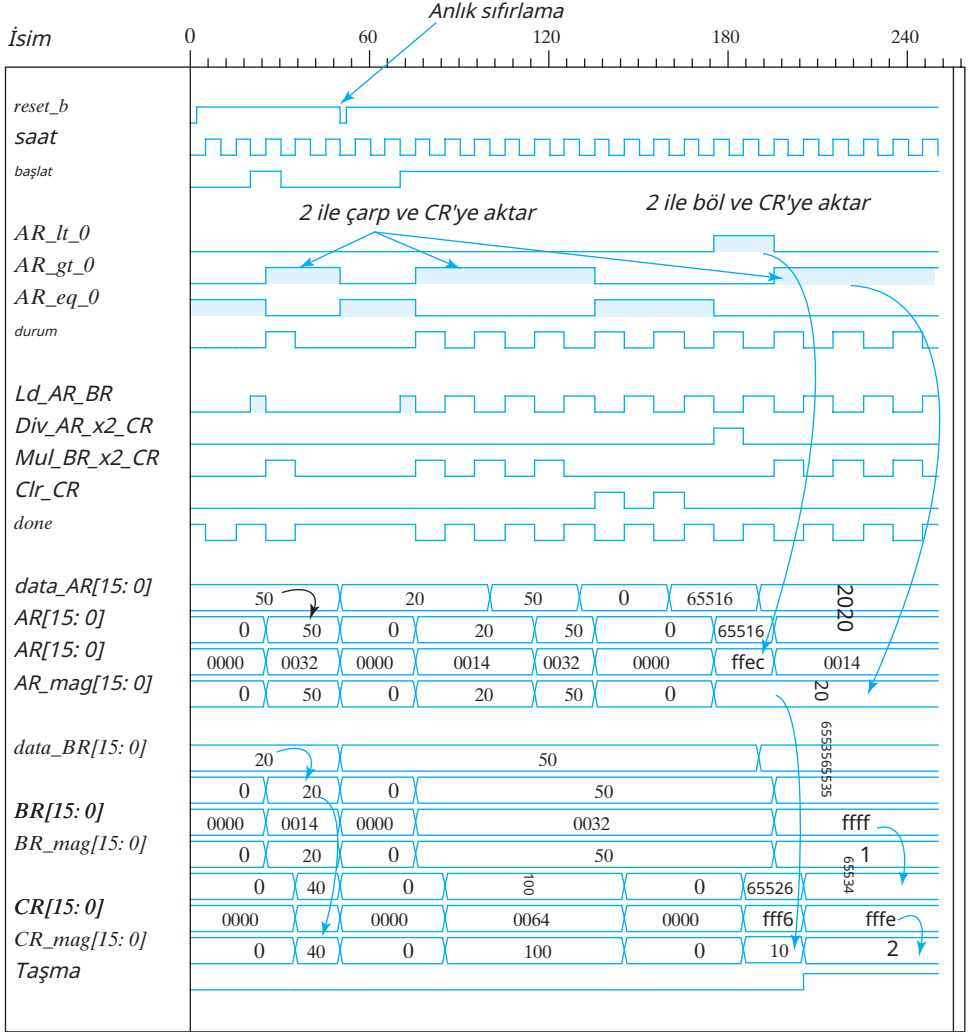
aksi takdirde eğer (pozitif sıfırdan farklı) ise $(Taşma \leftarrow BR[15] \oplus [14])$, $CR \leftarrow BR$ (sola kaydırılmış)

)
aksi takdirde eğer $(AR = 0)$ ise $(CR \leftarrow 0)$.



HDL tanımları Eşlik Eden Web Sitesi'nde mevcuttur. Problem 8.8 için simülasyon sonuçları aşağıdadır:

532 Seçilen Problemlere Cevaplar



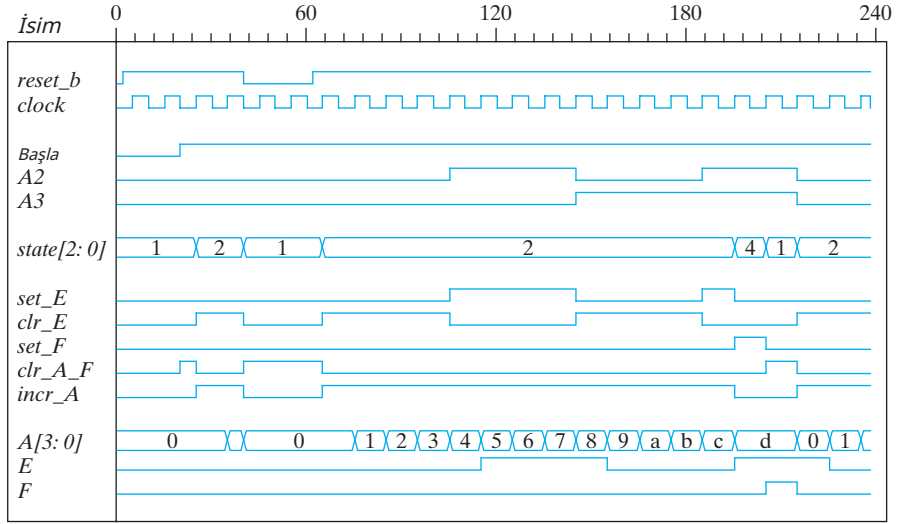
8.9 Tasarım denklemleri:

$$D_{S_idle} = S_2 + S_idle \text{ Start'}$$

$$D_{S_1} = S_idle \text{ Start} + S_1(A2 \ A3)'$$

$$D_{S_2} = A2 \ A3 \ S_1$$

HDL tanımı Companion Web Sitesi'nde mevcuttur. Problem 8.9 için simülasyon sonuçları şu şekildedir:



8.11 $D_A = A \cdot B \cdot A \cdot x$

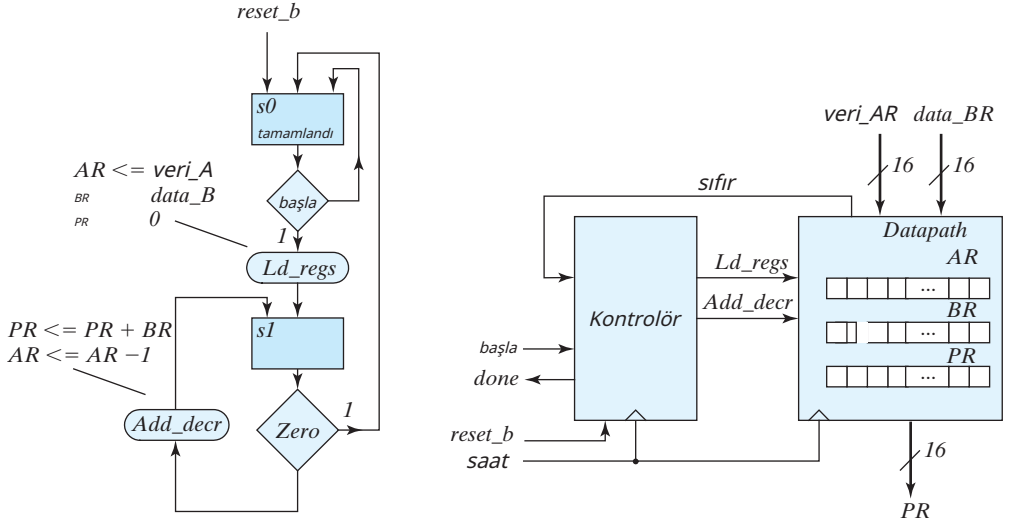
$D_B = A \cdot B \cdot A \cdot B \cdot x \cdot y$

8.16 RTL notasyonu:

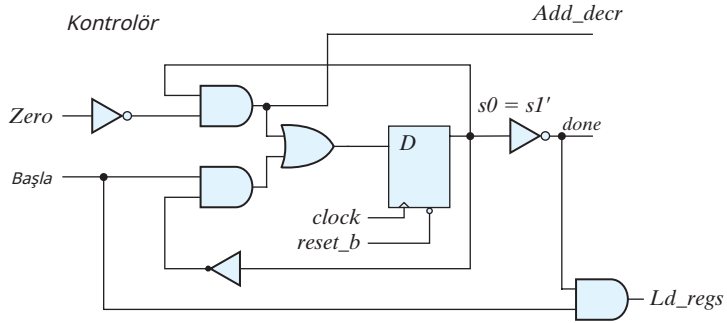
s0: (ilk durum) Eğer start = 0 s0 durumuna geri dön, Eğer (start = 1) o zaman BR d çarpan, AR d çarpan, PR d 0, s1'e git.

s1: (Sıfır için AR kontrol et) Sıfır= 1 eğer AR= 0, eğer (Sıfır= 1) o zaman s0 durumuna geri dön (done) Eğer (Sıfır = 0) o zaman s1 durumuna git, PR d PR + BR, AR d AR - 1.

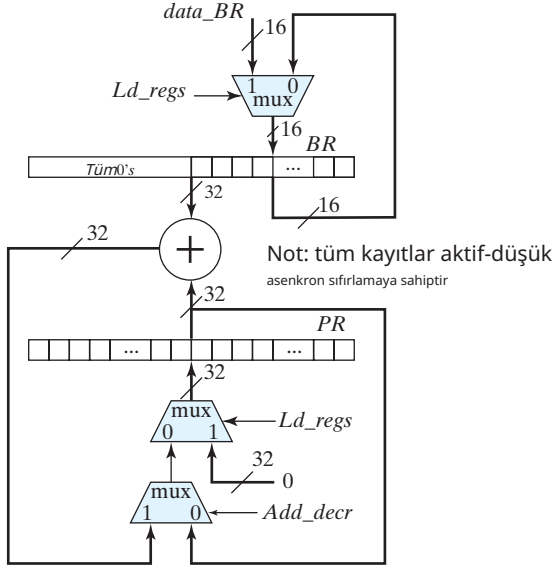
Veri yolunun iç mimarisi, ürünü (PR) tutmak için çift genişlikte bir kayıt, çarp anı (AR) tutmak için bir kayıt, çarpanı (BR) tutmak için bir kayıt, çift genişlikte bir paralel toplayıcı ve tek genişlikte bir paralel toplayıcıdan oluşmaktadır. Tek genişlikteki toplayıcı, çarpma birimini azaltma işlemini gerçekleştirmek için kullanılır. Çarpma tamamen 1'lerden oluşan bir kelime eklemek, çarpma an 1'in 2'nin tamamlayıcısını çıkarmayı sağlar. Aşağıdaki Şekil 8.16 (a), devrenin ASMD diyagramını, blok diyagramını ve kontrol cihazını göstermektedir. Şekil 8.16 (b), veri yolunun iç mimarisini göstermektedir. Şekil 8.16 (c), devrenin simülasyon sonuçlarını göstermektedir.



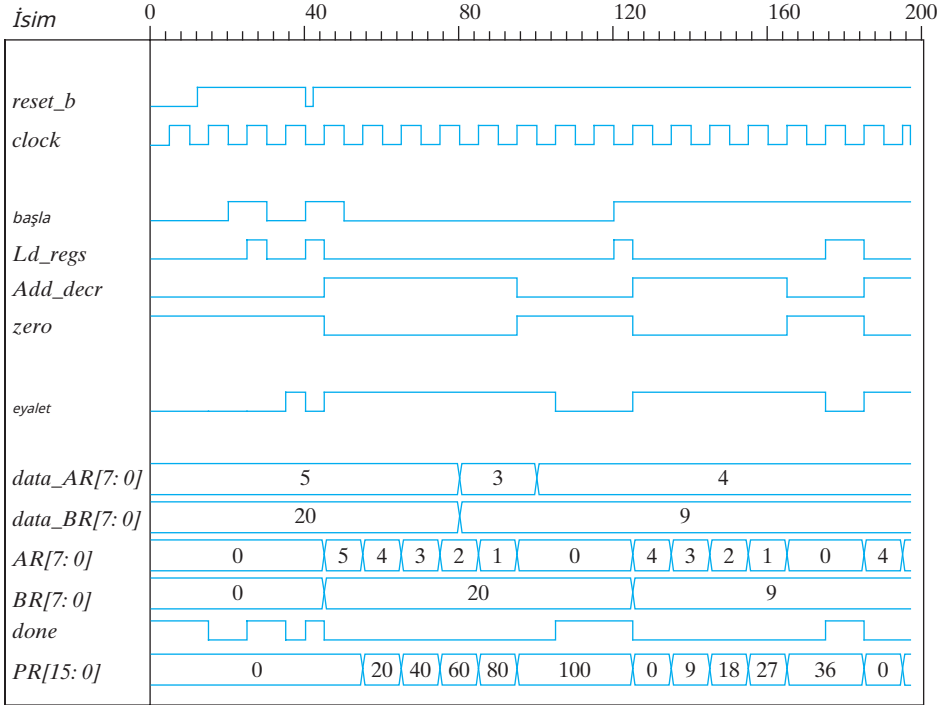
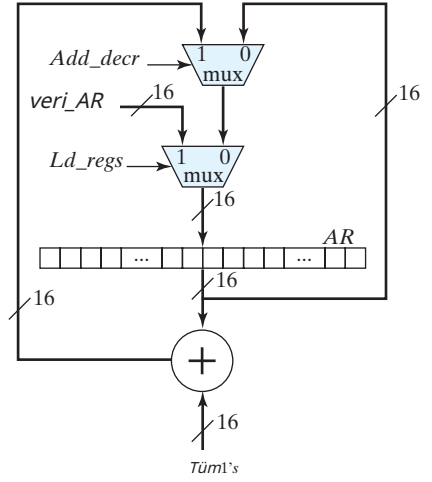
Not: Çıktı olarak OR kapısının formunu oluşturun, girdi olarak AR kaydının bitlerini kullanın.



(a) ASMD grafiği, blok diyagramı ve kontrolör



(b) Veri Yolu



(c) Simülasyon sonuçları

536 Seçilen Problemlere Cevaplar

8.17 $(2^n - 1)(2^n - 1) < (2^{2n} - 1)$ için $n \geq 1$

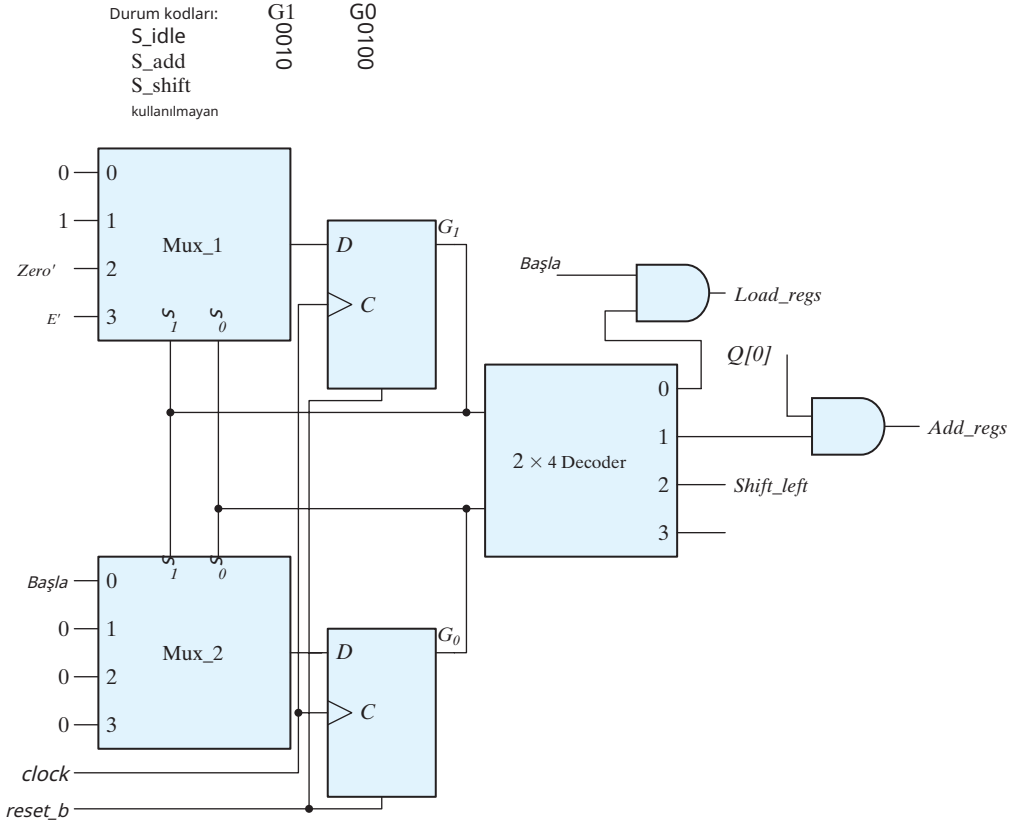
8.18 (a) Maksimum ürün boyutu 32 bit, A ve Q kayıtlarında mevcuttur.

bu P sayacın başlangıçta 16'yı (ikili 10000) yüklemek için 5 bit olması gerekir.

bu Z (sıfır) tespiti, 5 girişli NOR kapısıyla üretilir.

8.20 $2(n+1)t$

8.21 8.21



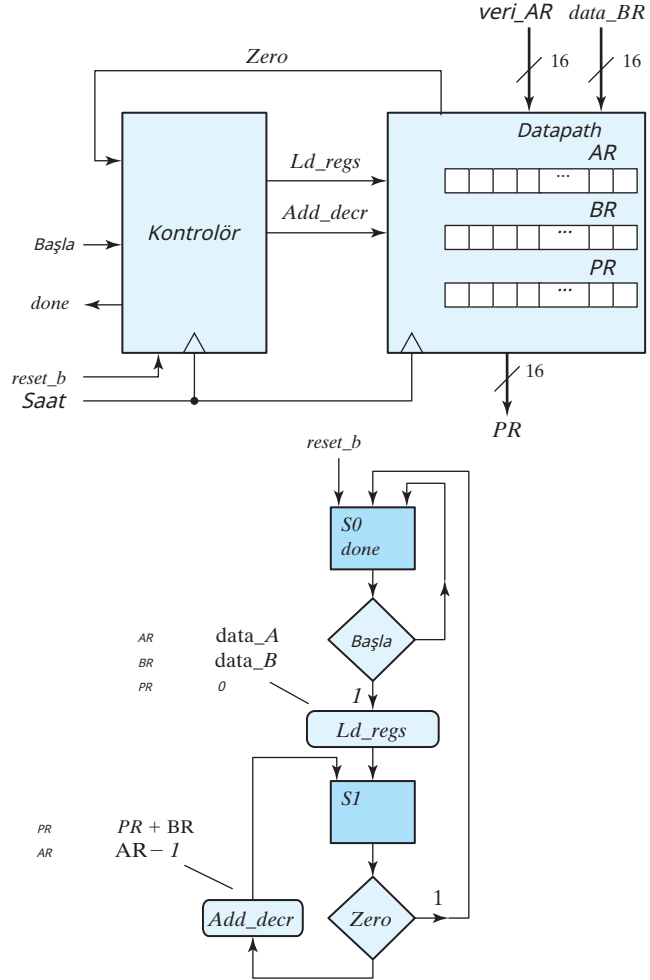
8.30 (a) $E=1$ (b) $E=0$

8.31 $A=0110, B=0010, C=0000$.

$$\begin{aligned}
 A * B &= 1100 & A \mid B &= 0110 & A \&\& C &= 0 \\
 A + B &= 1000 & A \wedge B &= 0100 & \mid A &= 1 \\
 A - B &= 0100 & \&A &= 0 & A < B &= 0 \\
 \sim C &= 1111 & \sim \mid C &= 1 & A > B &= 1 \\
 A \& B &= 0010 & A \parallel B &= 1 & A \neq B &= 1
 \end{aligned}$$

8.39

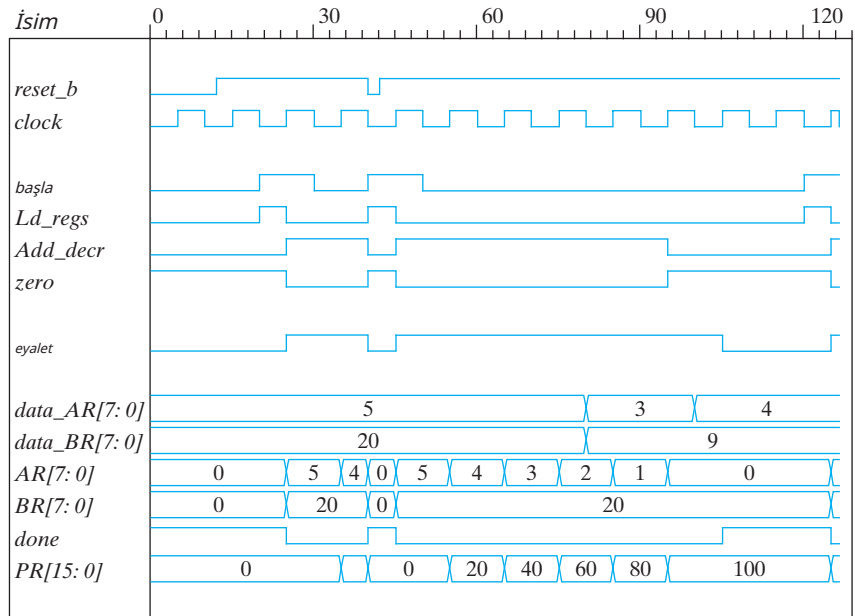
Blok diyagramı ve ASMD şeması:



HDL tanımı Companion Web Sitesi'nde mevcuttur. Simölasyon sonuçları

Problem 8.39 için ařağıdadır:

538 Seçilen Problemlere Cevaplar



İndeks

A

ABEL, 332
Absorpsiyon teoremi, 45
Soyut davranış modeli, 109
Toplayıcılar ve çıkarıcılar (deney)
 toplayıcı-çıkarıcı (dört bit), 456 – 457
 tam toplayıcı, 455
 yarım toplayıcı, 455
büyüklük karşılaştırıcı, 457
 paralel toplayıcı, 455 – 456
Toplayıcı kimlik, 40
Boolean
 fonksiyonu, 48 – 49
Algoritmik durum makinesi ve
 datapath (ASMD) grafik,
 370 – 371
kontrolör ve datapath donanım
 tasarımı, 376
kontrol mantığı, 379 – 381, 396, 398
tasarım örnekleri, 371 – 381
kayıt transferi temsili,
 377 – 378
durum tablosu, 378 – 379
zamanlama dizisi, 374 – 376
Algoritmik durum makineleri (ASM'ler),
 363 – 371
algoritmik durum makinesi ve
 datapath (ASMD) grafik,
 370 – 371
tasarım örnekleri, 371 – 381
ikili kod ataması, 365 – 366
blok, 368 – 369

grafik, 365 – 368
koşullu kutu ve örnekler, 367
 kontrol mantığı, 364
 kontrol birimi, 364
 veri yolu birimi, 364
 ASM şemasının karar kutusu, 366
 Mealy tipi sinyaller, 366 – 368
 basitleştirmeler, 369
durum ve karar kutuları, 366
 durum kutusunun stili, 365 – 366
 zamanlama dikkate alındığında, 369 – 370
her zaman blok, 358
her zaman ifade, 164, 176, 217, 219, 228,
 290, 354 – 355, 382
Amerikan Standart Kodu için
 Bilgi Değişimi
 (ASCII), 24 – 26
Analogdan dijitale çevirici, 2
Bir ifade ile AND'lenmiş, 53
AND kapısı, 30, 32 – 33, 42, 46 – 47, 50,
 57 – 58, 60, 65, 90, 113, 321, 323
maxtermlerin AND'lenmesi, 55
AND-invert grafik sembolü, 92
AND-invert sembolü, 90 – 91
AND-NOR diyagramları, 98 – 99
AND-OR diyagramları, 90, 98 – 99
AND-OR-INVERT fonksiyonu,
 97 – 98
Uygulamaya özel entegre devre
 (ASIC), 68
Aritmetik toplama, 39
Aritmetik işlemler, 5

ASCII NAK (negatif onay)
 kontrol karakteri, 27
atama ifadesi, 115, 164, 171, 228,
 354 – 355, 361
Birleşim yasası, 39
 cebirsel kanıtları, 45
Asenkron sıralı devre, 191

B

Silme (BS) kontrolü, 26
Taban- r sistemi, 4, 10
8'li sistem, 4
BCD toplayıcı, 144 – 146
BCD kodu, 22 – 23
BCD dalgalı sayıcı, 269 – 271
BCD senkron sayıcı, 275
begin anahtar kelime, 115, 177, 217
Davranışsal modelleme, 174 – 176
İki yönlü kaydırma kaydedici, 264, 352
İki taraflı anahtar, 514 – 515
İkili toplayıcı-çıkarıcı,
 kombinasyonel devreler, 133 – 144
 ikili toplayıcı, 136 – 138
 ikili çıkarıcı, 141 – 142
 taşıma yayılması, 138 – 141
 tam toplayıcı, 135 – 136
 yarım toplayıcı, 134
 taşma, 143 – 144
İkili ve ondalık sayılar
 (deney)
BCD sayımı, 444 – 445
ikili sayım, 443

İkili ve ondalık sayılar (devam.)

sayımlar, 446
 osiloskop, 444
 çıkış deseni, 445
 İkili hücre, 27
 İkili kodlu ondalık (BCD), 130 – 131
 eklemeler, 20 – 21
 kod, 22 – 23
 İkili kodlar, 2, 18 – 27
 8, 4, -2, -1 kodu, 22 – 23
 ondalık sayıların eklenmesi, 21
 ASCII karakter kodu, 24 – 26
 BCD kodu, 20 – 23
 2421 kodu, 22 – 23
 hata tespit kodu, 26 – 27
 fazladan-3 kodu, 22 – 23
 Gray kodu, 23 – 24
 ondalık sayıların çıkarılması, 21
 İkili rakam, *Bak* Bit
 İkili bilgi işleme, 29 – 30 dijital ma
 ntık devrelerinin ikili bilgi işle
 me, 30 İkili mantık:

tanımı, 30 – 31

mantık kapıları, 31 – 33

İkili çarpan, 146 – 148 İk
 ili çarpan, HDL tanımı, 402 – 411

paralel çarpanın davranışsal tan
 ımı, 409 – 411

datapath birimi, 403
 çarpanı test etme, 405 – 409

İkili çarpan (deney),
 478 – 480

blok diyagramı, 478
 çarpanı kontrol etme, 479
 kayıtların kontrolü, 478 – 479
 datapath tasarımı, 479
 kontrol tasarımı, 479
 çarpma örneği, 479

İkili sayılar, 3 – 6, 9 – 10
 aritmetik işlemler, 5 – 6
 tamamlayanı, 10 – 11

iki sayının toplamı, 6

İkili operatör:

*, 39

+, 39

•, 40

tanım, 38

İkili dalgalı sayaç, 267 – 269

İkili sinyaller, 3, 32

İkili depolama, 27 – 30

İkili senkron sayaç, 271 – 272

paralel yük ile, 276 – 278

yukarı-aşağı, 272 – 275

Bipolar transistörler, 507

Bit, 2, 5

Engelleme atamaları, 219 – 220, 355

Blok ifadesi, 115

Boolean cebiri, 30, 47, 126

kapı tipi devrelerde uygulama, 42

aksiyomatik tanımı, 40 – 43

temel tanımlar, 38 – 40

temel teoremler, 43 – 45

kanonik formlar, 51 – 58

arasında dönüşüm, 55 – 56

ikilik, 43

maxterimler, 51 – 52

VEleme, 55

tanım, 55

çarpımı, 54 – 55

miniterimler, 51 – 52

tanım, 55

toplamı, 52 – 53

operatör prosedürü, 45 – 46

standart formlar, 56 – 58

iki değerli, 41 – 43

Boolean ifadeleri, HDL için, 115 – 116

Boolean fonksiyonu, 126

cebirsal manipülasyon, 48 – 49

tamamlayanı, 49 – 50

tanım, 46

kapılarla uygulama, 48

çok seviyeli NAND devresi, 93 – 95

NAND kapıları ile, 90 – 91

NOR uygulaması, 95 – 97

16 olası fonksiyon, 58 – 60

toplamların çarpımı biçimi, 84 – 88

çarpanların toplamı biçimi, 84 – 88

doğru tablo içinde, 46

iki seviyeli uygulama, 91 – 93

Boolean fonksiyonu basitleştirme

(deney)

Boolean fonksiyonları toplam-

miniterimler biçiminde, 449

tamamlayanı, 449

kapı entegre devreleri, 448

mantık diyagramı, 448

Baloncuk, 60

Tampon devresi, 60

Yerleşik sistem fonksiyonları, 178

Bayt, 5, 26

c

Satır sonu (CR) kontrolü, 26

Zincirleme NAND kapıları, 63

durum ifadesi, 176, 382

durum öğeleri, 175

case ifade, 175, 362, 403

casex yapı, 176

casex ifade, 362

casez yapı, 176

Merkezi işlem birimi, 2

Flip-flop için karakteristik tablolar, 201 – 202

Çip, 66

Temizleme işlemi, 351

Saatli ardışık devreler, 191

Saat üretici, 191

Saat darbe üretici (deney),

474 – 475

devre çalışması, 473 – 474

IC zamanlayıcı, 473

Saat darbeleri, 191

Kapalı yapı, 42

2421 kodu, 22 – 23

Kod dönüştürücüler (deney)

Gri koddan eşdeğer ikiliye, 452

dokuzun tamamlayıcısı, 452

yedi segmentli gösterim, 452 – 453

İkili sayı sisteminin katsayıları, 4

Kombinasyonel devreler:

analiz prosedürü, 126 – 129

ikili toplayıcı-çıkarıcı, 133 – 144

ikili toplayıcı, 136 – 138

ikili çıkarıcı, 141 – 142

taşıma yayılması, 138 – 141

tam toplayıcı, 135 – 136

yarım toplayıcı, 134

taşma, 143 – 144

ikili çarpan, 146 – 148

blok diyagramı, 125 – 126

ondalık toplayıcı, 144 – 146

çözücüler, 150 – 155

kombinasyonel mantık

uygulaması, 154 – 155

çıktı Boole fonksiyonlarını üretme,

127 – 128

tasarım prosedürü, 129 – 133

kod dönüştürme örneği, 130 – 133

kodlayıcılar, 155 – 157

öncelik, 156 – 157

geri besleme yolu, 127

donanım tanım dili

(HDL) hakkında, 164 – 181

davranışsal modelleme, 174 – 176

data akışı modelleme, 171 – 174

test tezgahı örneği, 176 – 181

gate düzeyinde modelleme, 164 – 169

üç durumlu kapılar, 169 – 170

büyüklik karşılaştırıcı, 148 – 150

multiplexer, 158 – 164

dijital sistemlerin tasarımında kullanılır, 126

Kombinasyonel devreler (deney)

dekoder uygulaması, 450 – 451

tasarım örneği, 450

çoğunluk mantığı, 45

0 parite üretici, 450

Kombinasyonel programlanabilir mantık

cihazı (PLD), 321

Virgül, 179

Değişmeli yasa, 39, 42

Tamamlayıcı metal-oksit ya
rı iletken (CMOS), 67 Ta
mamlayıcı MOS (CMOS) devreleri, 510
– 513

iki yönlü anahtar, 514 – 515

özellikler, 513

CMOS üretim süreci, 513

CMOS mantık devresi, 513

iletim kapıları ile özel-VE'nin inş
ası, 515

74C serisi, 513

dört-bir-hat çoklayıcı, 515

IC türü 74C04, 513

yayılma gecikme süresi, 513

statik güç tüketimi, 513

iletim kapısı, 514 – 517

Tamamlayıcılar, 10 – 14, 44, 55, 87

azalmış taban, 10 – 11

taban, 11 – 12

çıkarma ile, 12 – 14 Bil

gisayar destekli tasarım (CAD) sistemleri, 67
– 68, 118

VLSI devrelerinin bilgisayar destekli tasarımı,
67 – 68

Uzlaşma teoremi, 49

Kontrol karakterleri, 2

5 Bir tasarım için kayıt ve kod çö
zücü şeması, 411 Kontrol man

tığı, 396 – 402 ASMD

grafikleri, 379 – 381, 3

96, 398

blok diyagramı, 393

D flip-fl op, 401

Gri kod, 397 – 398

giriş kod *Başla* ve *Sıfır*

kararlar, 396

bir flip-fl op per durumu, 401 – 402

bir sıcak atama,

397, 401 – 402

sıra-kayıt-ve-dekodör

(el ile) yöntemi, 398 – 401

durum ataması, 398

uygulama adımları, 397

Sayaclar:

tanımlandı, 255

HDL için:

dalgalanma, 288 – 290

senkron, 287 – 288

Johnson, 282 – 283

halka, 280 – 282

dalgalanma:

BCD, 269 – 271

binary, 267 – 269

simge, 502 – 504

senkron:

BCD, 275

binary, 271 – 272

paralel yük ile binary sayacı,

276 – 278

yukarı-aşağı binary, 272 – 275

kullanılmayan durumlarla, 278 – 280

Sayaclar (deney)

paralel yük ile binary sayacı,

462 – 463

ondalık sayıcı, 461

dalgali sayıcı, 461

senkron dört bitli ikili

sayacı, 461

Sayım işlemi, 351

Kesim noktası, 317

D

Veri akışı modelleme, kombinasyonel

mantık, 171 – 174

Veri yolu birimi, 364

Ondalık toplayıcı, kombinasyonel

devreler, 144 – 146

Ondalık eşdeğer, ikili

sayı, 4

Ondalık sayı sistemi, 4

Modül beyanı, 112

Kodlayıcılar, 150 – 155

kombinasyonel mantık uygulaması,

154 – 155

varsayılan anahtar kelime, 176

Degeneratif formlar, kapılar, 98 – 99

Gecikme kontrol operatörü, 218

DeMorgan teoremi, 45, 49 – 50, 55, 62,

84, 91 – 92

Bağımlılık notasyonu, 493 – 495

Tükenme modu, 508

Tasarım girişi, 109

Kombinasyonel devrelerin tasarımı,

129 – 133

D flip-fl op, 198 – 200, 255, 263

analiz, 210

karakteristik tablo, 202

kombinasyonel PAL'de, 330

kontrol mantığında, 401

kenar tetiklemeli için gra

fık sembol, 200 tut

ma süresi, 199

ana-köle, 517

pozitif kenar tetiklemeli, 203

kurulum süresi, 199

Difüzyon kanalı, 508

Dijital çağ, 1

Dijital entegre devreler, 66 – 67

fan-in, 67

fan-out, 67

gürültü marjı, 67

güç dağılımı, 67

yayılma gecikmesi, 67

Dijital mantık devreleri:

ikili bilgi işlemi, 30

simgeler için, 32

Dijital mantık ailesi, 66 – 67

Dijital mantık kapıları, 60 – 65

çoklu girişlerin uzantısı, 62 – 63

pozitif ve negatif mantık, 63 – 65

Dijital mantık kapıları (deney)

NAND devresi, 447 – 448

yayılma gecikmesi, 447

doğruluk tablosu, 446

evrensel NAND kapısı, 447

dalg formları, 446 – 447

Dijital sistemler, 1 – 3

bilgi akış yetenekleri, 30

Dijital çok yönlü disk (DVD), 3

Azaltılmış taban tamamlayıcı, 10 – 11

Sdisplay görev, 178 – 179, 181

Dağıtım yasası, 39, 42, 54, 57

D latch, 195 – 196, 457

Belgelendirme dili, 109

Önemsiz durumlar, 88

Önemsiz mintermler, 88 – 90

Doping maddeleri, 507

Drenaj terminali, 508

İkili ilkesi, 43

İkili teoremi, 44

E

Kenar hassas döngüsel davranış, 354

Kenar tetiklemeli *D* flip-fl op, 330

Sekiz bitlik alfasayısal karakter

kodu, 28

Sekiz bitlik kod, 27

8, 4, –2, –1 kodu, 22 – 23

Elektriksel olarak silinebilir PROM, 320

Elektronik tasarım otomasyonu (EDA), 68

else ifadesi, 222

Emiter bağlı mantık (ECL), 67

Kodlayıcılar, 155 – 157

öncelik, 156 – 157

Son-dönüş taşıma, 13

end anahtar kelimesi, 115, 177, 217

endprimitive, 117

endtable, 117

Geliştirme modu, 508

Silinebilir PROM, 320

Hata tespit ve

hata düzeltme kodları:

Hamming, 312 – 315

tek hata düzeltme ve çift

hata tespiti, 315

ETX (metin sonu), 26

Olay kontrol ifadesi, 175

Olay kontrol operatörü, 218

Aşırı-3 kodu, 22 – 23, 130

Özel-NOR fonksiyonu, 103

- F**
Fan-in, 67
Fan-out, 67
Hatasız devre, 110
Hata simülasyonu, 110
Alan, 39
Alan-programlanabilir kapı dizisi
(FPGA), 68 , 299 , 329 – 330 , 438 ,
480 – 482 *Ayrıca bakınız* Xilinx FPGA
Dosya ayırıcı (FS) kontrolü, 26
\$finish ifadesi, 178
\$finish sistemi, 115
Sonlu durum makinesi (FSM), 364
Beş değişkenli K-haritası, 84
Flash bellek cihazları, 320
Flip-flop, tanımlandı, 192
Flip-flop devreleri, 259
ASMD, 371
karakteristik tablo, 201 – 202
Clear_b girişi, 256
temiz veya doğrudan sıfırlama, 203
saat yanıtı, 197
D flip-fl op, 198 – 200 , 255 , 263
analiz, 210
karakteristik tablo, 202
kombinasyonel PAL'de, 330
kenar tetiklemeli için grafik sembol
, 200
tutma süresi, 199
master-slave, 517
pozitif kenar tetiklemeli, 203
kurulum süresi, 199
doğrudan girişler, 203
giriş denklemi, 209 – 210
JK flip-fl op, 200 – 201 , 263
analiz, 210 – 213
karakteristik denklem, 203
karakteristik tablo, 202
master-slave, 198 , 517
pozitif kenar tetiklemeli, 199
sinyal geçişi, 197
semboller, 497 – 499
T (toggle) *fl ip-fl* op, 200 – 201
analiz, 213 – 214
özellik denklemi, 203
özellik tablosu, 202
Flip-fl op giriş denklemleri, 209 – 210
Flip-fl ops (deney)
D latch, 457
IC türü *fl ip-fl* op, 459 – 460
master-slave *D flip-fl* op, 458
pozitif kenar tetiklemeli *fl ip-fl* op, 459
SR latch, 457
sonsuz döngü, 359
fork ... join blok, 226
for döngü, 360
Dört bitlik veri saklama kaydı, 257
Dört bitlik kayıt, 256
Dört bitlik evrensel kaydırma kaydı, 265
Dört basamaklı ikili eşdeğer, 9
Dört'e bir çoklayıcı, 163
Dört değişkenli Boolean fonksiyonları, harita
minimizasyonu, 80 – 84
Dört değişkenli K-haritası, 80 – 84
Franklin, Benjamin, 507
Tam toplayıcı (FA) devresi, 261 – 262
Fonksiyonel hatalar, 109
Fonksiyonel doğrulama, 181
Fonksiyon blokları, 332
G
Kapı gecikmeleri, 113 – 115
Kapı örnekleme, 112
Kapı seviyesi minimizasyonu, 73
VE-VEYA-TERS
uygulaması, 99 – 100
önemsemeyen koşullar, 88 – 90
özel-VEYA (XOR) fonksiyonu,
103 – 108
tek fonksiyon, 104 – 106
parite üretimi ve kontrolü,
106 – 108
donanım tanım dili
(HDL), 108 – 118
Boolean ifadeleri, 115 – 116
kapı gecikmeleri, 113 – 115
kullanıcı tanımlı primitifler (UDP'ler),
116 – 118
harita yöntemi:
beş değişkenli K-haritası, 84
dört değişkenli K-haritası, 80 – 84
bir fonksiyonun asal etkileyenleri,
82 – 84
üç değişkenli K-haritası, 75 – 76
iki değişkenli K-haritası, 74 – 75
NAND devreleri, 90 – 91
degeneres olmayan formlar, 98 – 99
VE-VE DEĞİŞTİR
uygulama, 100
toplamların çarpımı basitleştirme,
84 – 88 , 90
tablo özeti ve örnek,
100 – 102
birden fazla girişi olan kapılar, 33
Kapı voltajı, 508
genel amaçlı dijital bilgisayar, 2
Giga (G) bayt, 5
grafik kullanıcı arayüzleri (GUI'ler), 1
grafik semboller, 32
Gray kodu, 23 – 24 , 397 – 398
Gray kodunu eşdeğer
binary'e, 452
H
yarım toplayıcı, 167
Hamming kodu, 312 – 315
Taşınabilir cihazlar, 190
Donanım tanım dili (HDL),
68 , 108 – 118
algoritmik tabanlı davranış
sal anım, 381
ikili çarpan, 402 – 411
Boolean ifadeleri, 115 – 116
devre gösterimi, 111
kombinasyonel devreler, 164 – 181
davranışsal modelleme, 174 – 176
veri akışı modelleme, 171 – 174
test tezgahı örneği, 176 – 181
uç durumlu kapılar, 169 – 170
tasarım örneği tanımı,
381 – 391
kapı gecikmeleri, 113 – 115
dalgalı sayıcı için, 288 – 290
RTL tanımı, 381 – 385
yapısal tanım, 381 , 386 – 391
anahtar seviyesi modelleme, 517 – 520
senkron sayıcı için, 287 – 288
tasarım tanımının test edilmesi, 385 – 386
iletim kapısı, 519 – 520
kullanıcı-tanımlı primitifler (UDPs),
116 – 118
Donanım sinyal üreteçleri, 115
HDL tabanlı tasarım metodolojisi, 3
Heuristikler, 30
Onaltılık (taban-16) sayı sistemi,
4 – 5 , 8 – 10
Yüksek empedans durumu, 162 – 163
Delikler, 507
Yatay tabulasyon (HT) kontrolü, 26
Huntington postülatları, 42
I
7493 IC, 439 , 442 – 443
IC türü 74194, 470
IC türü flip-flop, 459 – 460
Kimlik elemanı, 39
if-else ifadesi, 174
if ifadesi, 222
if-then ifadesi, 353
İkincil kombinasyonel mantık, 116
Tam olarak tanımlanmamış fonksiyonlar, 88
ilk blok, 177 , 179 , 358
ilk ifadesi, 115 , 177 , 217 – 219
giriş beyanı, 117
3-girişli NAND kapısı, 63
3-girişli NOR kapısı, 63
Kapılar için giriş-çıkış sinyalleri, 33
Giriş-çıkış birimleri, 2
Modülün örnekleme, 112

tam sayı k , 360

tam sayı anahtar kelime, 176

Entegre devreler:

VLSI devrelerinin bilgisayar destekli ta
sarımı, 67 – 68

diijital entegre devreler, 66 – 67

fan-in, 67

fan-out, 67

gürültü marjı, 67

güç dağılımı, 67

propagasyon gecikmesi, 67

entegrasyon seviyeleri, 66

Entegre devreler (IC'ler), 438 – 439

deneyler için gerekli, 442

İnternet, 2

Bir elemanın tersine, 39

İnvertör devresi, 509

İnvertör kapısı, 66

Invert-OR grafik sembolü, 93

iPod Touch™, 1

J

J/K flip-fl op, 200 – 201 , 263 , 371

analizi, 210 – 213

karakteristik denklem, 203

karakteristik tablo, 202

K

Karnaugh haritası, 73

Kilo (K) bayt, 5

K-haritası, *Bak* Karnaugh haritası

L

Laboratuvar deneyleri:

toplama ve çıkarma (deney 7)

toplama-çıkarma (dört bit),

456 – 457

tam toplayıcı, 455

yarım toplayıcı, 455

büyüklik karşılaştırıcı, 457

paralel toplayıcı, 455 – 456

ikili ve ondalık sayılar

(deney 1)

BCD sayımı, 444 – 445

ikili sayım, 443

sayımlar, 446

osiloskop, 444

çıkış deseni, 445

ikili çarpma (deney 17),

478 – 480

blok diyagramı, 478

çarpanı kontrol etme, 479

kayıtların kontrolü, 478 – 479

datapath tasarımı, 479

kontrol tasarımı, 479

çarpma örneği, 479

Boolean fonksiyonu basitleştirme (deney 3)

Boolean fonksiyonları

minterm toplamı formunda, 449

tamamlayıcı, 449

kapı entegre devreleri, 448

mantık diyagramı, 448

saat darbe üretici

(deney 15), 474 – 475

devre çalışması, 473 – 474

entegre devre zamanlayıcı, 473

kod dönüştürücüler (deney 5)

Gri koddan eşdeğer ikiliye, 452

dokuzun tamamlayıcısı, 452

yedi segmentli gösterim, 452 – 453

kombinasyonel devreler (deney 4)

kodlayıcı uygulaması, 450 – 451

tasarım örneği, 450

çoğunluk mantığı, 450

parite üretici, 450

sayaçlar (deney 10)

paralel yüklemeli ikili sayaç,

462 – 463

ondalık sayaç, 461

dalgalı sayaç, 461

senkron dört bitli ikili

sayaç, 461

sayısal mantık kapıları (deney 2)

NAND devresi, 447 – 448

yayılma gecikmesi, 447

doğruluk tablosu, 446

evrensel NAND kapısı, 447

dalga formları, 446 – 447

flip-flop'lar (deney 8)

D latch, 457

IC tipi flip-flop, 459 – 460

master-slave D flip-fl op, 458

pozitif kenar tetiklemeli

flip-flop, 459

SR latch, 457

lamba hentbolu (deney 14)

devre analizi, 472

kayıpların sayısını sayma, 472 – 473

IC tipi 74194, 470

lamp Ping-Pong oyunu, 473

mantık diyagramı, 470 – 472

oyunu oynama, 472

bellek birimi (deney 13)

IC RAM, 467 – 468

bellek genişletme, 469

ROM simülatörü, 469

RAM testi, 468 – 469

çoklayıcı tasarımı (deney 6)

tasarım spesifikasyonları, 453 – 454

paralel toplayıcı ve akümülatör (deney 16)

blok diyagramı, 475

taşıma devresi, 476

devreyi kontrol etme, 477

devre çalışması, 477 – 478

kayıt kontrolü, 475 – 476

detaylı devre, 477

sequential circuits (deney 9)

sayaç tasarımı, 460 – 461

durum diyagramı, 460

enable ile yukarı-aşağı sayıcı, 460

seri toplama (deney 12)

seri toplayıcı, 466 – 467

seri toplayıcı-çıkarcı, 467

toplayıcının test edilmesi, 467

kaydırma kayıtları (deney 11)

iki yönlü kaydırma kaydı, 465

iki yönlü kaydırma kaydı ile

yerel yükleme (IC türü 74157),

465 – 466

geri besleme kaydırma kaydı, 464 – 465

IC kaydırma kaydı, 463

halka sayıcı, 463 – 466

4 Verilog HDL simülasyon deneyleri

ve FPGA'lar ile hızlı prototi
pleme:

deney 1, 482 – 483

deney 2, 483 – 484

deney 4, 484

deney 5, 484

deney 7, 484

deney 8, 485

deney 9, 485

deney 10, 485

deney 11, 485 – 486

deney 13, 486

deney 14, 486

deney 16, 486

deney 17, 486 – 487

Lamba hentbolu (deney)

devre analizi, 472

kayıpların sayısını sayma, 472 – 473

IC tipi 74194, 470

lamba Ping-Pong oyunu, 473

mantık diyagramı, 470 – 472

oyunu oynama, 472

Lamba Ping-Pong oyunu, 473

Büyük ölçekli entegrasyon (LSI)

cihazları, 66

Kilitler, 193 – 196 , 220 – 223

D kilidi, 195 – 196 , 457

NAND kilidi, 194

NOR kilidi, 194

SR latch, 193 – 195 , 457

Latch-free tasarım, 425 – 426

Seviye duyarlı döngüsel davranış, 354

Yükleme işlemi, 351

Mantık devre diyagramı, 46 – 47

Mantık devreleri, 3

Dijital entegre devrelerin mantık aileleri, 67

Mantık kapıları, 31 – 33

Mantık simülörleri, 125

Mantık sentezi, 109, 361 – 363

M

Makro hücreler, 330 – 331

Büyüklük karşılaştırıcı, 148 – 150

Harita minimizasyon yöntemi:

beş değişkenli K-haritası, 84

dört değişkenli K-haritası, 80 – 84

bir fonksiyonun asal etkileyenleri, 82 – 84

üç değişkenli K-haritası, 75 – 76

iki değişkenli K-haritası, 74 – 75

Maske programlama, 320

Usta-köle flip-flop, 198

D flip-fl op, 458, 517

Matematiksel sistem, birinin postülatları, 39

Maxterm, 51 – 52

VEleme, 55

tanım, 55

ürünün, 54 – 55

Sonlu durum makinesinin Mealy modeli, 214 – 217

Mealy_Sıfır_Algılayıcı, 226 – 227

Orta ölçekli entegrasyon (MSI)

devreler, 66, 126, 439

Bellek yongaları, 66

Bellek kod çözme:

çakışan, 309 – 312

iç yapı, 307 – 309

Bellek kayıtları, 29

Bellek birimi, 2, 29

Bellek birimi (deney)

IC RAM, 467 – 468

hafıza genişletme, 469

ROM simülör, 469

RAM testi, 468 – 469

Metal-oksit yarı iletken (MOS), 67

Metal-oksit silikon yarı iletkenleri, 507

temel yapı, 508

türleri, 508

Miniterm, 51 – 52

tanım, 55

önemsemeyen, 88 – 90

ve asal etkileyenler, 83

toplamı, 52 – 53

Minterm, 51

Modül, 111

modül ... endmodül anahtar kelime çifti, 116, 169

\$monitor ifadesi, 178, 180

\$monitor sistem görevi, 179

Moore sonlu durum makinesi modeli, 214 – 217

Moore tipi sıfır dedektörü ardışık devresi, 228

En anlamlı bit (MSB), 358

Çoklu-IC MSI tasarımı, 126

Çoklayıcı tasarımı (deney), 453 – 454

Multiplexerler, 158 – 164

tasarımı ile, 411 – 422

bir sayacın test edilmesi, 421 – 422

N

İsim ilişkilendirme mekanizması, 178

NAND devreleri, 90 – 91, 447 – 448

NAND kapısı, 58, 60, 63, 66, 90 – 93, 439, 510

NAND kilidi, 194

NAND-NAND diyagramları, 98 – 99

N bit, 27

N-kanal MOS, 509 – 510

Negatif mantık VE kapısı, 65

Negatif mantık polaritesi, 64

nege anahtar kelime, 219, 222, 354

Netlist, 109

Dokuzun tamamlayıcısı, 452

nmos anahtar kelime, 517

Gürültü marjı, 67

Engellemeyen atamalar, 219 – 220, 355

Kapıların degenerate olmayan formları, 98 – 99

NOR kapısı, 60, 63, 66, 90, 510

NOR kilidi, 194

NOR-NOR diyagramları, 98 – 99

NOT kapısı, 30, 32, 42, 58, 113

N-tip dopant, 507

Sayı tabanı dönüşümleri, 6 – 8

O

Sekizli sayı sistemi, 4, 8 – 10

Tek işlev, 62

Bir sıcak atama, 397, 401 – 402

Açık Verilog Uluslararası (OVI), 110

VE-VEYA diyagramları, 98 – 99

VE-VEYA-TERS işlevi, 98

VEYA ilexx', 54

OR kapısı, 30, 32 – 33, 42, 46 – 47, 50, 57 – 58, 60, 65, 90, 113, 316, 323

OR-NAND diyagramları, 98 – 99

çıkış beyanı, 117

P

Paralel toplayıcı ve akümülatör (deney)

blok diyagramı, 475

taşıma devresi, 476

devreyi kontrol etme, 477

devre çalışması, 477 – 478

kayıt kontrolü, 475 – 476

detaylı devre, 477

Paralel yük kontrolü, 264

parametre beyanı, 224

Parite biti, 26

Parite hatası, 26 – 27

P-channel MOS, 509

pmos anahtar kelime, 517

Polarite göstergesi, 65

Port listesi, 112

posedge anahtar kelime, 219 – 222, 354

Pozitif kenar tetiklemeli flip-flop, 459

Pozitif tam sayılar, 14

Pozitif mantık VE kapısı, 65

Pozitif mantık polaritesi, 64

Matematiksel bir sistemin postülatları, 39

Boolean cebirinin postülatları,

43 – 44

Güç dağılımı, 67

Önceden tanımlanmış ilkel, 112

Bir fonksiyonun asal etkileyenleri, 82 – 84

primitive ... endprimitive

anahtar kelime çifti, 116

İlkel kapılar, 165

primitive anahtar kelime, 117

İşlemci kayıtları, 29

Maksimum terimlerin çarpımı biçimi, 87

Toplamların çarpımı, 57

Toplamların çarpımı biçimi, Boolean

fonksiyonu, 84 – 88, 90

Program, 1

Programlanabilir dizi mantığı (PAL),

299, 321

buffer-inverter kapısı, 325

ticari, 325

sigorta haritası, 328 –

329 programlama tablosu

u, 327 Programlanabilir mantık di

zisi (PLA) Boolean fonksiyonları, 3

22

özel yapım, 324

sigorta haritası, 323

iç mantık, 322

programlama tablosu, 323

boyutu, 324

Programlanabilir mantık cihazı (PLD), 66,

68, 299

Programlanabilir salt okunur bellek

(PROM), 320

İletim gecikmesi, 67, 110, 447

P-tip cihaz, 507 – 508

Q

Kalite sembolleri, 491 – 493

R

Yarışsız tasarım, 422 – 425

Radix tamamlayıcı, 11 – 12

R -izin verilen rakamlar, 5

Rastgele erişim belleği (RAM),

299 – 307

HDL'deki bellek tanımı,

303 – 304

sembol, 504 – 505

zamanlama dalga formları, 304 – 306

bellek türleri, 306 – 307

yazma ve okuma işlemleri, 302 – 303

Salt okunur bellek (ROM), 299,

315 – 321

blok diyagramı, 316

kombinasyonel devre

uygulaması, 318

32×8 örneği, 316

donanım prosedürü, 317

girişler ve çıkışlar, 316

iç ikili depolama, 317

doğruluk tablosu, 317

türler, 320

Kayıt ayırıcı (RS) kontrolü, 26

Dörtgen şekilli semboller, 488 – 491

Kayıt (lar), 27

tanımlandı, 255

fazla-3 kodu, 27

dört bit, 256

HDL için, 284 – 287

yükleme veya güncelleme, 257

paralel yüklemeye ile, 257

kaydırma, 258 – 266

seri toplama, 261 – 263

bilginin seri aktarımı,

259 – 261

evrensel, 263 – 266

sembol, 499 – 502

bilgilerin aktarımı, 28 – 30

Kayıt aktarım seviyesi (RTL), 3

algoritmik durum makineleri (ASM'ler),

363 – 371

blok, 368 – 36

9 grafik, 365 – 368 , 37

0 – 371 kontrol mantığı ile ve

ri işleme işlemleri arasınd

aki ilişki, 364

basitleştirmeler, 369

zamanlama dikkate alındığında, 369 – 370

kombinasyonel devre fonksiyonları, 354

kontrol mantığı, 396 – 402

HDL'de, 354 – 363

modelleme, doğrulama

ve sentez için akış diyagramı, 353

mantık sentezi, 361 – 363

döngü ifadeleri, 358 – 361

operatörler, 355 – 358

prosedürel atamalar, 355

HDL tanımları:

ikili devrelerin, 402 – 411

kombinasyonel devrelerin,

381 – 391

latch-free tasarım, 425 – 426

çoklayıcılarla, 411 – 422

notasyon, 351 – 354

prosedürel atamalar, 355

yayıma gecikmeleri, 353

yarışma içermeyen tasarım, 422 – 425

ardışık ikili çarpan,

391 – 396

işlem türü, 353

Verilog HDL için, 426

reg anahtar kelime, 168 , 175 , 177 , 179 ,

220 – 221 , 360

tekrar döngü, 358

Ripple_carry_4_bit_adder, 169

Ripple sayıcı:

BCD, 269 – 271

ikili, 267 – 269

HDL için, 288 – 290

s

Şematik yakalama, 68

Şematik girişi, 68

Yarı iletkenler, 507

Hassasiyet listesi, 175

Sıralı ikili çarpan:

ASMD grafiği, 394 – 396

denetleyici ile

datapath arasındaki arayüz, 393

ikili çarpan için sayısal örne

k, 396

kayıt yapılandırması, 392 – 393

veri işleme alt sistemi için

gerekli kayıtlar, 395 Sır

alı devreler (deney) sayaç tasarı

mı, 460 – 461 durum diya

gramı, 460

enable ile yukarı-aşağı sayıcı, 460

Sıralı programlanabilir cihazlar,

329 – 346

AND-OR ürün toplamı

fonksiyonu, 330

karmaşık programlanabilir mantık cihazı

(CPLD), 329 , 331

yapılandırma, 331

alan programlanabilir kapı dizisi

(FPGA), 329 – 330 , 332

giriş-çıkış (I/O) blokları, 330

kayıtlı, 330

sıralı (veya basit) programlanabilir

mantık cihazı (SPLD), 329

Seri toplama (deney)

seri toplayıcı, 466 – 467

seri toplayıcı-çıkartıcı, 467

toplayıcının test edilmesi, 467

Elemanlar kümesi, 38

Doğal sayılar kümesi, 39

Operatörler kümesi, 38

Gerçek sayılar kümesi, 39

Shift-left kontrolü, 264

Shift işlemi, 351

Shift kayıtları (deney)

iki yönlü kaydırma kaydı, 465

iki yönlü kaydırma kaydı ile

yatay yüklemeye (IC tipi 74157),

465 – 466

geri besleme kaydırma kaydı, 464 – 465

IC kaydırma kaydı, 463

halka sayıcı, 463 – 464

Shift-right kontrolü, 264

Sinyaller, 2

atama, 64

İmzalı ikili sayılar, 14 – 18

aritmetik toplama, 16 – 17

aritmetik çıkarma, 17 – 18

imzalı tamamlayıcı sistem, 15

imzalı büyüklük konvansiyonu, 15

imzalı tamamlayıcı sistem, 15 , 21

imzalı büyüklük konvansiyonu, 15

imzalı 10'luk tamamlayıcı sistem, 21

silikon kristal yapısı, 507

Basit_Döngü, 112 – 113

Basit_Döngü_gecikme, 114

Tek geçiş davranışı, 217

Küçük ölçekli entegrasyon (SSI) devreleri,

439

Küçük ölçekli entegrasyon (SSI) cihazları, 66

Yazılım programları, 68

Kaynak terminali, 508

Spartan™ , 333 , 339 – 344

SR kilitli, 193 – 195 , 457

Standart hücreler, 126

Boolean cebirinin standart biçimi,

56 – 58

Standart ürün, 51

Standart toplamalar, 51

Durum tablosu, 378 – 379

STX (metin başlangıcı), 26

Ürünlerin toplamı, 56 , 62 , 88 , 91

Toplam terimler, 57

sağlama1 ve sağlama0 anahtar kelimesi, 518

Anahtarlama cebiri, 43

Switch seviyesinde modelleme, 517 – 520

Semboller, 61 , 171

!, 171

%, 178

&, 171

Semboller (*devamı*)

&, 171
 Σ , 53
 * /, 111
 +, 171
 / *, 111
 = =, 171
 @, 174–175, 354, 425–426
 $\wedge$, 171
 $\downarrow$, 171
 " | ", 174
 ~, 171
 ?:, 171
 (&), (/), and (~), 115
 $\oplus$, 58
 aktif-düşük giriş veya çıkış, 492
 toplayıcı (Σ), 491
 VE kapısı veya fonksiyonu (&), 491
 aritmetik mantık birimi (ALU), 491
 aritmetik operatörler (+, -, *, /), 356
 tampon kapısı veya tersleyici, 491
 coder, decoder, veya kod dönüştürücü (X/Y), 491
 kombinasyonel elemanlar için, 495 – 497
 kayıt içeriği ikili
 15, 492
 geri sayım, 492
 sayaç (CTR), 491
 sayaçlar için, 502 – 504
 artış, 492
 bir depolama elemanına veri girişi, 492
 demultiplexer (DMUX), 491
 dijital mantık devreleri için, 32
 dinamik gösterge girişi, 492
 etkinleştirme girişi, 492
 çift fonksiyon veya çift parite elemanı (2k), 491
 özel-VE kapısı veya fonksiyonu (= 1), 491
 üs alma operatörü (**), 356
 flip-flop girişleri, 492
 flip-floplar için, 497 – 499
 mantıksal inkar girişi veya çıkışı, 492
 büyüklük karşılaştırıcı (COMP), 491
 MOS transistörü için, 509
 multiplexer (MUX), 491
 çarpan (Π), 491
 tek fonksiyon veya tek parite elemanı (2k+1), 491
 açık-kollektör çıkışı, 492
 VEYA kapısı veya fonksiyonu (≥ 1), 491
 özel amplifikasyon ile çıkış, 492
 (Π), 55
 RAM için, 504 – 505
 rastgele erişim belleği (RAM), 491
 salt okunur bellek (ROM), 491

registerlar için, 499 – 502
 ripple sayıcı (RCTR), 491
 noktalı virgöl (;), 112, 174
 sola kaydır, 492
 kaydırma kaydedici (SRG), 491
 sağa kaydır, 492
 eşik çizgiler (//), 111
 üç durumlu çıkış, 492
 Verilog HDL operatörleri, 356
Senkron sayıcı:
 BCD, 275
 ikili, 271 – 272
 paralel yükleme ile, 276 – 278
 yukarı-aşağı, 272 – 275
 HDL için, 287 – 288
Senkron ardışık devre, 191
Senkron ardışık mantık:
 tıklamalı ardışık devreler, analizi
 bunun, 204 – 217
 tasarımı, 236 – 245
D flip-floplar, analizi, 210
 flip-flop giriş denklemleri, 209 – 210
JK flip-flops, analizi, 210 – 213
 Mealy ve Moore sonlu durum makineleri modelleri, 214 – 217
 durum diyagramı, 207 – 209
 durum denklemleri, 205 – 206
 durum tablosu, 206 – 207
 yapısal tanım, 228 – 230
T flip-flops, analizi, 213 – 214
 tasarım prosedürü:
 uyarım tablosu, 239 – 241
 üç bitli ikili sayıcı mantık diyagramı, 245
 üç bitli ikili sayıcı için haritalar, 245
 kullanarak *D* flip-flop ops, 238 – 239
 kullanarak *JK* flip-flop ops, 241 – 243
 kullanarak *T* flip-flop ops, 243 – 245
HDL modelleri:
 davranışsal modelleme, 217 – 220
 flip-floplar ve latch'ler, 220 – 223
 durum diyagramı, 223 – 227
 sıralı devreler, 190 – 192
 durum ataması, 235 – 236
 durum azaltma, 231 – 235
 depolama elemanları:
 flip-floplar, 196 – 204
 latch'ler, 193 – 196
Sistem ilkeleri, 116
T
 tablo, 117
 Tera (T) bayt, 5
 Test tezgahı, 109
T flip-floplar, analizi, 213 – 214

Boolean cebirinin teoremleri, 43 – 45
 kanıtlar, 44 – 45
Termal hareket, etkisi
 yarı iletken, 507
 Üç girişli özel-VE kapısı, 64
 Üç girişli NAND kapısı, 91
 Üç durumlu tampon kapısı, 162
 Üç durumlu tamponlar, 163
 Üç durumlu kapılar, 162 – 164, 169 – 170
 Üç değişkenli K-haritası, 75 – 76
\$time, 178
 zaman ölçeği derleyici, 113
 Zaman diyagramları, 32
 Zaman doğrulaması, 110, 181
 Transfer fonksiyonu, 60
Bilgi transferi, arasında
 register'lar, 28 – 30
 Transistörler, 2
 Transistör-transistör mantığı (TTL), 67
 Tetikleyici, 196
 trianhtar kelime, 170
 Doğruluk tablosu, 31, 46, 52 – 53, 86, 109, 129
 ve Boolean cebiri, 45
 iki ikili değişkenin 16 fonksiyonu için, 58
 ROM, 317
T_Simple_Circuit_prop_delay, 114
T (toggle) flip-flop ops, 200 – 201
 analysis, 213 – 214
 karakteristik denklem, 203
 karakteristik tablo, 202
 İki seviyeli kapı yapısı, 57
 İki seviyeli uygulama, 56 – 57
 Boolean fonksiyonu, 91 – 93
 İki girişli çoklayıcı, 163, 174
 İki değerli Boolean cebiri, 41 – 43
 tanım, 41
 ikili işlem kuralları, 41 – 42
 İki değişkenli K-haritası, 74 – 75
U
 Tek yönlü kaydırma kaydedici, 264
 Evrensel kapı, 90
 Evrensel NAND kapısı, 447
 Evrensel kaydırma kaydedici, 263 – 266
 Kullanıcı tanımlı ilkel (UDP'ler), 116 – 118
V
 Vektörler, 166
 Doğrulama, 181
 Verilog 2001, 426
 Verilog 2005, 426
 Verilog HDL, 68, 115, 118, 332, 354, 438

akış diyagramı, 363
 mantıksal ve ilişkisel operatörler, 357
 ikili kelimeler için mantık operatörleri, 357
 döngü ifadeleri, 358 – 361
 operatör önceliği, 359
 operatörler, 355 – 358
 kayıt transfer işlemi, 354
 anahtar düzey modelleme, 517 – 520
 Verilog modülü, 112
 Verilog ifadeleri, 115Veril
 og sistem görevleri, 178 – 181Ç
 ok büyük ölçekli entegrasyon (VLSI) de
 vreleri, 66 – 67, 126
 kapı dizisi, 332
 VHDL, 332

Virtex™, 333, 344 – 346
 Voltajla çalışan mantık devreleri, 31

W

while döngüsü, 359
 Wired-AND kapısı, 97
 Wired mantık, 97
wire anahtar kelimesi, 112, 170, 179
\$write, 178

X

XC2000, 333
 XC3000, 333
 XC4000, 333

Xilinx FPGA:
 temel mimari, 333
 yapılandırılabilir mantık bloğu
 (CLB), 334
 dağıtılmış RAM, 334
 geliştirmeler, 337 – 339
 bağlantı hatları,
 334 – 336
 G/Ç bloğu (IOB), 337
 seri, 333
 Spartan II, 340 – 344
 Spartan XL çipleri, 339 – 340
 Virtex, 344 – 346
 XOR kapısı, 323
 XOR işlemi, 315